

Fundamentals of Data Engineering

Bản dịch song ngữ Anh–Việt · Plan and Build Robust Data Systems

Joe Reis & Matt Housley (nguyên tác)

2026

Mục lục

Preface — Lời nói đầu	13
What This Book Isn't — Cuốn sách này KHÔNG phải là gì	13
What This Book Is About — Cuốn sách này nói về điều gì	14
Who Should Read This Book — Ai nên đọc cuốn sách này	15
Prerequisites — Kiến thức cần có trước	15
What You'll Learn and How It Will Improve Your Abilities — Bạn sẽ học được gì và nó sẽ cải thiện năng lực của bạn ra sao	16
The Book Outline — Dàn ý của cuốn sách	17
Conventions Used in This Book — Các quy ước được dùng trong cuốn sách này	18
TIP — MẸO	19
NOTE — GHI CHÚ	19
WARNING — CẢNH BÁO	19
How to Contact Us — Cách liên hệ với chúng tôi	19
Acknowledgments — Lời cảm ơn	20
 Part I. Foundation and Building Blocks — Phần I. Nền tảng và các khối xây dựng	 23
Chapter 1. Data Engineering Described — Chương 1. Mô tả về Kỹ thuật Dữ liệu	24
What Is Data Engineering? — Kỹ thuật Dữ liệu là gì?	24
Data Engineering Defined — Định nghĩa Kỹ thuật Dữ liệu	26
The Data Engineering Lifecycle — Vòng đời Kỹ thuật Dữ liệu	26
Evolution of the Data Engineer — Sự tiến hóa của Kỹ sư Dữ liệu	28
The early days: 1980 to 2000, from data warehousing to the web — Những ngày đầu: 1980 đến 2000, từ kho dữ liệu đến web	28
The early 2000s: The birth of contemporary data engineering — Đầu thập niên 2000: Sự ra đời của kỹ thuật dữ liệu đương đại	29
The 2000s and 2010s: Big data engineering — Thập niên 2000 và 2010: Kỹ thuật big data	31
The 2020s: Engineering for the data lifecycle — Thập niên 2020: Kỹ thuật cho vòng đời dữ liệu	33
Data Engineering and Data Science — Kỹ thuật Dữ liệu và Khoa học Dữ liệu	35
Data Engineering Skills and Activities — Kỹ năng và Hoạt động của Kỹ thuật Dữ liệu	37
Data Maturity and the Data Engineer — Độ trưởng thành Dữ liệu và Kỹ sư Dữ liệu	38
Stage 1: Starting with data — Giai đoạn 1: Bắt đầu với dữ liệu	39
Stage 2: Scaling with data — Giai đoạn 2: Mở rộng quy mô với dữ liệu	41
Stage 3: Leading with data — Giai đoạn 3: Dẫn dắt bằng dữ liệu	42
The Background and Skills of a Data Engineer — Nền tảng và Kỹ năng của một Kỹ sư Dữ liệu	43
Business Responsibilities — Trách nhiệm Nghiệp vụ	45
Technical Responsibilities — Trách nhiệm Kỹ thuật	46
THE UNREASONABLE EFFECTIVENESS OF SQL — HIỆU QUẢ PHI LÝ CỦA SQL	49

KEEPING PACE IN A FAST-MOVING FIELD — GIỮ NHỊP TRONG MỘT LĨNH VỰC CHUYỂN ĐỘNG NHANH	50
The Continuum of Data Engineering Roles, from A to B — Dải liên tục các Vai trò Kỹ thuật Dữ liệu, từ A đến B	50
Data Engineers Inside an Organization — Kỹ sư Dữ liệu bên trong một Tổ chức	52
Internal-Facing Versus External-Facing Data Engineers — Kỹ sư Dữ liệu Hướng Nội bộ so với Hướng Bên ngoài	52
Data Engineers and Other Technical Roles — Kỹ sư Dữ liệu và các Vai trò Kỹ thuật Khác	54
Upstream stakeholders — Các bên liên quan thượng nguồn	55
Downstream stakeholders — Các bên liên quan hạ nguồn	57
Data Engineers and Business Leadership — Kỹ sư Dữ liệu và Ban Lãnh đạo Doanh nghiệp	60
Data in the C-suite — Dữ liệu trong Ban C-suite	60
Data engineers and project managers — Kỹ sư dữ liệu và các quản lý dự án	62
Data engineers and product managers — Kỹ sư dữ liệu và các quản lý sản phẩm	63
Data engineers and other management roles — Kỹ sư dữ liệu và các vai trò quản lý khác	63
NOTE — LƯU Ý	64
Conclusion — Kết luận	64
Additional Resources	65
Chapter 2. The Data Engineering Lifecycle — Chương 2. Vòng đời kỹ thuật dữ liệu	67
What Is the Data Engineering Lifecycle? — Vòng đời kỹ thuật dữ liệu là gì?	67
The Data Lifecycle Versus the Data Engineering Lifecycle — Vòng đời dữ liệu so với vòng đời kỹ thuật dữ liệu	69
Generation: Source Systems — Sinh dữ liệu: Hệ thống nguồn	70
Evaluating source systems: Key engineering considerations — Đánh giá hệ thống nguồn: Những cân nhắc kỹ thuật then chốt	72
Storage — Lưu trữ	74
Evaluating storage systems: Key engineering considerations — Đánh giá hệ thống lưu trữ: Những cân nhắc kỹ thuật then chốt	75
Understanding data access frequency — Hiểu về tần suất truy cập dữ liệu	77
Selecting a storage system — Chọn một hệ thống lưu trữ	77
Ingestion — Thu nạp	78
Key engineering considerations for the ingestion phase — Những cân nhắc kỹ thuật then chốt cho giai đoạn thu nạp	78
Batch versus streaming — Theo lô so với theo luồng	79
Key considerations for batch versus stream ingestion — Những cân nhắc then chốt cho thu nạp theo lô so với theo luồng	80
Push versus pull — Đẩy so với kéo	81
Transformation — Biến đổi	82
Key considerations for the transformation phase — Những cân nhắc then chốt cho giai đoạn biến đổi	83
Serving Data — Phục vụ dữ liệu	85
Analytics — Phân tích	85
MULTITENANCY — ĐA THUÊ BAO	87
Machine learning — Học máy	88
Reverse ETL — Reverse ETL	89
Major Undercurrents Across the Data Engineering Lifecycle — Các dòng chảy ngầm chính xuyên suốt vòng đời kỹ thuật dữ liệu	91
Security — An ninh	92
Data Management — Quản lý dữ liệu	93

Data governance — Quản trị dữ liệu	95
MASTER DATA MANAGEMENT — QUẢN LÝ DỮ LIỆU CHỦ	101
Data modeling and design — Mô hình hóa và thiết kế dữ liệu	102
Data lineage — Truy vết nguồn gốc dữ liệu	103
Data integration and interoperability — Tích hợp và khả năng tương tác dữ liệu	104
Data lifecycle management — Quản lý vòng đời dữ liệu	104
Ethics and privacy — Đạo đức và quyền riêng tư	105
Orchestration — Điều phối	106
DataOps — DataOps	108
Automation — Tự động hóa	110
Observability and monitoring — Khả năng quan sát và giám sát	111
Incident response — Ứng phó sự cố	112
DataOps summary — Tóm tắt DataOps	113
Data Architecture — Kiến trúc dữ liệu	114
Software Engineering — Kỹ thuật phần mềm	114
Core data processing code — Mã xử lý dữ liệu cốt lõi	115
Development of open source frameworks — Phát triển các framework mã nguồn mở	115
Streaming — Theo luồng	116
Infrastructure as code — Hạ tầng dưới dạng mã	116
Pipelines as code — Đường ống dưới dạng mã	117
General-purpose problem solving — Giải quyết vấn đề đa dụng	117
Conclusion — Kết luận	118
Additional Resources	119

Chapter 3. Designing Good Data Architecture — Chương 3. Thiết kế kiến trúc dữ liệu tốt 121

What Is Data Architecture? — Kiến trúc dữ liệu là gì?	121
Enterprise Architecture, Defined — Định nghĩa kiến trúc doanh nghiệp	122
TOGAF's definition — Định nghĩa của TOGAF	122
Gartner's definition — Định nghĩa của Gartner	123
EABOK's definition — Định nghĩa của EABOK	123
Our definition — Định nghĩa của chúng tôi	124
Data Architecture Defined — Định nghĩa kiến trúc dữ liệu	126
TOGAF's definition — Định nghĩa của TOGAF	126
DAMA's definition — Định nghĩa của DAMA	127
Our definition — Định nghĩa của chúng tôi	127
“Good” Data Architecture — Kiến trúc dữ liệu “tốt”	128
Principles of Good Data Architecture — Các nguyên tắc của kiến trúc dữ liệu tốt	129
Principle 1: Choose Common Components Wisely — Nguyên tắc 1: Lựa chọn các thành phần chung một cách khôn ngoan	131
Principle 2: Plan for Failure — Nguyên tắc 2: Dự liệu cho thất bại	132
Principle 3: Architect for Scalability — Nguyên tắc 3: Kiến trúc cho khả năng mở rộng quy mô	133
Principle 4: Architecture Is Leadership — Nguyên tắc 4: Kiến trúc là sự lãnh đạo	134
Principle 5: Always Be Architecting — Nguyên tắc 5: Luôn luôn kiến trúc	135
Principle 6: Build Loosely Coupled Systems — Nguyên tắc 6: Xây dựng các hệ thống ghép lỏng	135
Principle 7: Make Reversible Decisions — Nguyên tắc 7: Đưa ra những quyết định có thể đảo ngược	138
Principle 8: Prioritize Security — Nguyên tắc 8: Ưu tiên an ninh	138
Hardened-perimeter and zero-trust security models — Mô hình an ninh chu vi kiên cố và zero-trust	138
The shared responsibility model — Mô hình trách nhiệm chia sẻ	139

Data engineers as security engineers — Kỹ sư dữ liệu trong vai trò kỹ sư an ninh	140
Principle 9: Embrace FinOps — Nguyên tắc 9: Đón nhận FinOps	140
Major Architecture Concepts — Các khái niệm kiến trúc chính	143
Domains and Services — Miền và dịch vụ	143
Distributed Systems, Scalability, and Designing for Failure — Hệ phân tán, khả năng mở rộng quy mô, và thiết kế cho thất bại	144
TIP — MẸO	145
Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices — Ghép chặt so với ghép lỏng: Tầng, monolith, và microservices	147
Architecture tiers — Các tầng kiến trúc	147
Monoliths — Monolith	151
Microservices — Microservices	152
Considerations for data architecture — Những điều cần cân nhắc cho kiến trúc dữ liệu	153
User Access: Single Versus Multitenant — Quyền truy cập người dùng: Đơn thuê bao so với đa thuê bao	154
Event-Driven Architecture — Kiến trúc hướng sự kiện	155
Brownfield Versus Greenfield Projects — Dự án brownfield so với greenfield	157
Brownfield projects — Dự án brownfield	157
Greenfield projects — Dự án greenfield	158
Examples and Types of Data Architecture — Các ví dụ và loại kiến trúc dữ liệu	159
Data Warehouse — Kho dữ liệu	159
The cloud data warehouse — Kho dữ liệu đám mây	162
Data marts — Data mart	163
Data Lake — Hồ dữ liệu	164
Convergence, Next-Generation Data Lakes, and the Data Platform — Sự hội tụ, các hồ dữ liệu thế hệ mới, và nền tảng dữ liệu	165
Modern Data Stack — Ngăn xếp dữ liệu hiện đại	166
Lambda Architecture — Kiến trúc Lambda	168
Kappa Architecture — Kiến trúc Kappa	169
The Dataflow Model and Unified Batch and Streaming — Mô hình Dataflow và sự hợp nhất theo lô và theo luồng	169
Architecture for IoT — Kiến trúc cho IoT	170
Devices — Thiết bị	171
Interfacing with devices — Giao tiếp với các thiết bị	172
Scratching the surface of the IoT — Mới chạm tới bề mặt của IoT	174
Data Mesh — Data mesh	174
Other Data Architecture Examples — Các ví dụ kiến trúc dữ liệu khác	176
Who's Involved with Designing a Data Architecture? — Ai tham gia vào việc thiết kế một kiến trúc dữ liệu?	177
Conclusion — Kết luận	178
Additional Resources	178

Chapter 4. Choosing Technologies Across the Data Engineering Lifecycle — Chương 4. Lựa chọn công nghệ xuyên suốt vòng đời kỹ thuật dữ liệu **182**

Team Size and Capabilities — Quy mô và năng lực của nhóm	184
Speed to Market — Tốc độ ra thị trường	185
Interoperability — Khả năng tương tác	185
Cost Optimization and Business Value — Tối ưu chi phí và giá trị kinh doanh	186
Total Cost of Ownership — Tổng chi phí sở hữu	187
Total Opportunity Cost of Ownership — Tổng chi phí cơ hội của sở hữu	188

FinOps — FinOps	189
Today Versus the Future: Immutable Versus Transitory Technologies — Hôm nay so với tương lai: công nghệ bất biến so với công nghệ tạm thời	190
Our Advice — Lời khuyên của chúng tôi	192
Location — Vị trí	193
On Premises — On Premises	193
Cloud — Cloud	194
A BRIEF DETOUR ON CLOUD ECONOMICS — MỘT KHÚC LẠC ĐỀ NGẮN VỀ KINH TẾ HỌC CLOUD	196
Cloud Services and Credit Default Swaps — Dịch vụ cloud và hợp đồng hoán đổi rủi ro tín dụng	196
Cloud $\neq$ On Premises — Cloud $\neq$ On Premises	198
Data Gravity — Lực hấp dẫn dữ liệu	199
Hybrid Cloud — Hybrid Cloud	199
Multicloud — Multicloud	201
Decentralized: Blockchain and the Edge — Phi tập trung: Blockchain và biên (Edge)	202
Our Advice — Lời khuyên của chúng tôi	202
Choose technologies for the present, but look toward the future — Chọn công nghệ cho hiện tại, nhưng hướng nhìn về tương lai	203
Cloud Repatriation Arguments — Các lập luận về hồi hương khỏi cloud	204
You are not Dropbox, nor are you Cloudflare — Bạn không phải Dropbox, cũng không phải Cloudflare	204
Build Versus Buy — Tự xây dựng so với mua	206
Open Source Software — Phần mềm mã nguồn mở	208
Community-managed OSS — OSS được cộng đồng quản lý	208
Commercial OSS — OSS thương mại	210
Proprietary Walled Gardens — Các khu vườn có tường bao độc quyền	213
Independent offerings — Các giải pháp độc lập	213
Cloud platform proprietary service offerings — Các giải pháp dịch vụ độc quyền của nền tảng cloud	215
Our Advice — Lời khuyên của chúng tôi	215
Monolith Versus Modular — Monolith so với mô-đun	216
Monolith — Monolith	217
Modularity — Mô-đun hóa	218
The Distributed Monolith Pattern — Mô thức Monolith phân tán	220
Our Advice — Lời khuyên của chúng tôi	221
Serverless Versus Servers — Serverless so với Servers	222
Serverless — Serverless	222
Containers — Container	223
WARNING — CẢNH BÁO	224
When Infrastructure Makes Sense — Khi nào hạ tầng là hợp lý	224
Our Advice — Lời khuyên của chúng tôi	226
Optimization, Performance, and the Benchmark Wars — Tối ưu hóa, hiệu năng và các cuộc chiến benchmark	227
Big Data...for the 1990s — Dữ liệu lớn... cho thập niên 1990	229
Nonsensical Cost Comparisons — Các so sánh chi phí vô nghĩa	229
Asymmetric Optimization — Tối ưu hóa bất đối xứng	230
Caveat Emptor — Người mua hãy coi chừng	230
Undercurrents and Their Impacts on Choosing Technologies — Các dòng chảy ngầm và tác động của chúng đối với việc chọn công nghệ	230

Data Management — Quản lý dữ liệu	231
DataOps — DataOps	231
Data Architecture — Kiến trúc dữ liệu	232
Orchestration Example: Airflow — Ví dụ về điều phối: Airflow	232
Software Engineering — Kỹ thuật phần mềm	234
Conclusion — Kết luận	234

Part II. The Data Engineering Lifecycle in Depth — Phần II. Vòng đời kỹ thuật dữ liệu chuyên sâu 236

Chapter 5. Data Generation in Source Systems — Chương 5. Sinh dữ liệu trong các hệ thống nguồn 237

Sources of Data: How Is Data Created? — Các nguồn dữ liệu: Dữ liệu được tạo ra như thế nào?238	
Source Systems: Main Ideas — Các hệ thống nguồn: Những ý chính	239
Files and Unstructured Data — Tập và dữ liệu phi cấu trúc	239
APIs — API	240
Application Databases (OLTP systems) — Database ứng dụng (hệ thống OLTP)	240
ACID — ACID	241
Atomic transactions — Giao dịch nguyên tử	242
OLTP and analytics — OLTP và phân tích	242
Online Analytical Processing System — Hệ thống xử lý phân tích trực tuyến	243
Change Data Capture — Thu thập thay đổi dữ liệu	244
Logs — Log	244
Log encoding — Mã hóa log	245
Log resolution — Độ phân giải của log	246
Log latency: Batch or real time — Độ trễ của log: Theo lô hay thời gian thực	246
Database Logs — Log của database	246
CRUD — CRUD	247
Insert-Only — Chỉ-thêm (Insert-Only)	248
Messages and Streams — Tin nhắn và luồng	249
Types of Time — Các loại thời gian	251
Source System Practical Details — Chi tiết thực tế của hệ thống nguồn	252
Databases — Database	252
Major considerations for understanding database technologies — Các cân nhắc chính để hiểu các công nghệ database	252
Relational databases — Database quan hệ	254
Nonrelational databases: NoSQL — Database phi quan hệ: NoSQL	256
APIs — API	264
REST — REST	264
GraphQL — GraphQL	265
Webhooks — Webhook	266
RPC and gRPC — RPC và gRPC	266
Data Sharing — Chia sẻ dữ liệu	267
Third-Party Data Sources — Các nguồn dữ liệu bên thứ ba	268
Message Queues and Event-Streaming Platforms — Các hàng đợi tin nhắn và nền tảng luồng sự kiện	269
Message queues — Hàng đợi tin nhắn	269
Event-streaming platforms — Nền tảng luồng sự kiện	271
Whom You'll Work With — Bạn sẽ làm việc với ai	274
Undercurrents and Their Impact on Source Systems — Các dòng chảy ngầm và tác động của chúng lên các hệ thống nguồn	276

Security — An ninh	276
Data Management — Quản lý dữ liệu	277
DataOps — DataOps	278
Data Architecture — Kiến trúc dữ liệu	280
Orchestration — Điều phối	281
Software Engineering — Kỹ thuật phần mềm	282
Conclusion — Kết luận	283
Additional Resources	284
Chapter 6. Storage — Chương 6. Lưu trữ	285
Raw Ingredients of Data Storage — Nguyên liệu thô của lưu trữ dữ liệu	288
Magnetic Disk Drive — Ổ đĩa từ	289
Solid-State Drive — Ổ đĩa thể rắn	292
Random Access Memory — Bộ nhớ truy cập ngẫu nhiên	293
Networking and CPU — Mạng và CPU	295
Serialization — Tuần tự hóa	295
Compression — Nén	297
Caching — Caching	297
Data Storage Systems — Các hệ thống lưu trữ dữ liệu	298
Single Machine Versus Distributed Storage — Lưu trữ trên một máy đơn so với lưu trữ phân tán	299
Eventual Versus Strong Consistency — Nhất quán cuối cùng so với nhất quán mạnh	299
File Storage — Lưu trữ tệp	301
Local disk storage — Lưu trữ đĩa cục bộ	303
Network-attached storage — Lưu trữ gắn mạng	303
Cloud file system services — Các dịch vụ hệ thống tệp đám mây	304
Block Storage — Lưu trữ khối	304
Block storage applications — Các ứng dụng của lưu trữ khối	305
RAID — RAID	305
Storage area network — Mạng lưu trữ	306
Cloud virtualized block storage — Lưu trữ khối ảo hóa đám mây	306
Local instance volumes — Các khối instance cục bộ	307
Object Storage — Lưu trữ đối tượng	310
Object stores for data engineering applications — Kho đối tượng cho các ứng dụng kỹ thuật dữ liệu	312
Object lookup — Tra cứu đối tượng	313
Object consistency and versioning — Nhất quán và phiên bản hóa đối tượng	314
Storage classes and tiers — Các lớp và tầng lưu trữ	317
Object store-backed filesystems — Hệ thống tệp dựa trên kho đối tượng	318
Cache and Memory-Based Storage Systems — Các hệ thống lưu trữ dựa trên cache và bộ nhớ	318
Example: Memcached and lightweight object caching — Ví dụ: Memcached và caching đối tượng nhẹ	318
Example: Redis, memory caching with optional persistence — Ví dụ: Redis, caching bộ nhớ với khả năng bền vững tùy chọn	319
The Hadoop Distributed File System — Hệ thống tệp phân tán Hadoop	319
Hadoop is dead. Long live Hadoop! — Hadoop đã chết. Hadoop vạn tuế!	320
Streaming Storage — Lưu trữ luồng	320
Indexes, Partitioning, and Clustering — Chỉ mục, phân vùng và gom cụm	321
The evolution from rows to columns — Sự tiến hóa từ hàng sang cột	322
From indexes to partitions and clustering — Từ chỉ mục đến phân vùng và gom cụm	323

Example: Snowflake micro-partitioning — Ví dụ: Vi phân vùng của Snowflake	323
Data Engineering Storage Abstractions — Các trừu tượng lưu trữ cho kỹ thuật dữ liệu . . .	324
The Data Warehouse — Kho dữ liệu	325
The Data Lake — Hồ dữ liệu	326
The Data Lakehouse — Data Lakehouse	326
Data Platforms — Các nền tảng dữ liệu	327
Stream-to-Batch Storage Architecture — Kiến trúc lưu trữ luồng-sang-lô	328
Big Ideas and Trends in Storage — Các ý tưởng lớn và xu hướng trong lưu trữ	328
Data Catalog — Danh mục dữ liệu	329
Catalog application integration — Tích hợp ứng dụng với danh mục	329
Automated scanning — Quét tự động	329
Data portal and social layer — Cổng dữ liệu và tầng xã hội	330
Data catalog use cases — Các tình huống sử dụng của danh mục dữ liệu	330
Data Sharing — Chia sẻ dữ liệu	330
Schema — Lược đồ	331
Separation of Compute from Storage — Tách biệt tính toán khỏi lưu trữ	332
Colocation of compute and storage — Đặt cùng chỗ tính toán và lưu trữ	332
Separation of compute and storage — Tách biệt tính toán và lưu trữ	333
Hybrid separation and colocation — Tách biệt và đặt cùng chỗ theo kiểu lai	334
Zero copy cloning — Sao chép không tốn bộ nhớ	336
Data Storage Lifecycle and Data Retention — Vòng đời lưu trữ dữ liệu và lưu giữ dữ liệu . .	337
Hot, warm, and cold data — Dữ liệu nóng, ấm và lạnh	337
Data retention — Lưu giữ dữ liệu	340
Single-Tenant Versus Multitenant Storage — Lưu trữ đơn thuê bao so với đa thuê bao . . .	341
Whom You'll Work With — Những người bạn sẽ làm việc cùng	343
Undercurrents — Các dòng chảy ngầm	344
Security — An ninh	344
Data Management — Quản lý dữ liệu	345
Data catalogs and metadata management — Danh mục dữ liệu và quản lý siêu dữ liệu . . .	345
Data versioning in object storage — Phiên bản hóa dữ liệu trong lưu trữ đối tượng	345
Privacy — Quyền riêng tư	345
DataOps — DataOps	346
Systems monitoring — Giám sát hệ thống	346
Observing and monitoring data — Quan sát và giám sát dữ liệu	346
Data Architecture — Kiến trúc dữ liệu	346
Orchestration — Điều phối	347
Software Engineering — Kỹ thuật phần mềm	347
Conclusion — Kết luận	348
Additional Resources	348
Chapter 7. Ingestion — Chương 7. Thu nạp (Ingestion)	349
What Is Data Ingestion? — Thu nạp dữ liệu là gì?	350
DATA PIPELINES DEFINED — ĐỊNH NGHĨA ĐƯỜNG ỐNG DỮ LIỆU	350
Key Engineering Considerations for the Ingestion Phase — Các cân nhắc kỹ thuật chính cho giai đoạn thu nạp	351
Bounded Versus Unbounded — Có biên so với Không biên	353
Frequency — Tần suất	354
Synchronous Versus Asynchronous Ingestion — Thu nạp Đồng bộ so với Bất đồng bộ	356
Serialization and Deserialization — Tuần tự hóa và Giải tuần tự hóa	357
Throughput and Scalability — Thông lượng và Khả năng mở rộng	358

Reliability and Durability — Độ tin cậy và Độ bền	359
Payload — Tải trọng	360
Kind — Loại	360
Shape — Hình dạng	360
Size — Kích thước	361
Schema and data types — Lược đồ và kiểu dữ liệu	361
Metadata — Siêu dữ liệu	363
Push Versus Pull Versus Poll Patterns — Các mẫu hình Đẩy so với Kéo so với Thăm dò	364
Batch Ingestion Considerations — Các cân nhắc khi Thu nạp theo Lô	365
Snapshot or Differential Extraction — Trích xuất Ảnh chụp hoặc Chênh lệch	367
File-Based Export and Ingestion — Xuất và Thu nạp dựa trên Tập	367
ETL Versus ELT — ETL so với ELT	368
Extract — Trích xuất	368
Load — Nạp	368
Inserts, Updates, and Batch Size — Chèn, Cập nhật và Kích thước Lô	368
Data Migration — Di trú Dữ liệu	369
Message and Stream Ingestion Considerations — Các cân nhắc khi Thu nạp Tin nhắn và Stream	370
Schema Evolution — Tiến hóa Lược đồ	370
Late-Arriving Data — Dữ liệu Đến muộn	371
Ordering and Multiple Delivery — Thứ tự và Giao nhiều lần	371
Replay — Phát lại	371
Time to Live — Thời gian tồn tại	372
Message Size — Kích thước Tin nhắn	372
Error Handling and Dead-Letter Queues — Xử lý Lỗi và Hàng đợi Thư chết	372
Consumer Pull and Push — Kéo và Đẩy ở Bên tiêu thụ	373
Location — Vị trí	374
Ways to Ingest Data — Các cách Thu nạp Dữ liệu	374
Direct Database Connection — Kết nối Database Trực tiếp	374
Change Data Capture — Thu thập Thay đổi Dữ liệu	376
Batch-oriented CDC — CDC định hướng theo lô	376
Continuous CDC — CDC liên tục	377
CDC and database replication — CDC và sao chép database	378
CDC considerations — Các cân nhắc về CDC	378
APIs — APIs	379
Message Queues and Event-Streaming Platforms — Các Hàng đợi Tin nhắn và Nền tảng Event-Streaming	380
Managed Data Connectors — Các Connector Dữ liệu Được quản lý	382
Moving Data with Object Storage — Di chuyển Dữ liệu với Kho lưu trữ Đối tượng	383
EDI — EDI	383
Databases and File Export — Database và Xuất Tập	384
Practical Issues with Common File Formats — Các Vấn đề Thực tiễn với Các Định dạng Tập Phổ biến	384
Shell — Shell	385
SSH — SSH	386
SFTP and SCP — SFTP và SCP	386
Webhooks — Webhook	387
Web Interface — Giao diện Web	388
Web Scraping — Web Scraping	388
Transfer Appliances for Data Migration — Các Thiết bị Truyền tải cho Di trú Dữ liệu	389
Data Sharing — Chia sẻ Dữ liệu	390

Whom You'll Work With — Những người bạn sẽ làm việc cùng	390
Upstream Stakeholders — Các Bên liên quan Thượng nguồn	391
Downstream Stakeholders — Các Bên liên quan Hạ nguồn	391
Undercurrents — Các dòng chảy ngầm	392
Security — An ninh	393
Data Management — Quản lý Dữ liệu	393
Schema changes — Các thay đổi lược đồ	393
Data ethics, privacy, and compliance — Đạo đức dữ liệu, quyền riêng tư, và tuân thủ	394
DataOps — DataOps	396
Data-quality tests — Các bài kiểm thử chất lượng dữ liệu	397
Orchestration — Điều phối	398
Software Engineering — Kỹ thuật Phần mềm	399
Conclusion — Kết luận	400
Additional Resources	400

Chapter 8. Queries, Modeling, and Transformation — Chương 8. Truy vấn, Mô hình hóa và Biến đổi

401

Queries — Truy vấn	403
Data Modeling — Mô hình hóa Dữ liệu	421
Transformations — Biến đổi	446
Whom You'll Work With — Những Người Bạn Sẽ Làm Việc Cùng	471
Undercurrents — Các Dòng chảy Ngầm	473
Conclusion — Kết luận	477
Additional Resources	478

Chapter 9. Serving Data for Analytics, Machine Learning, and Reverse ETL — Chương 9. Phục vụ dữ liệu cho phân tích, học máy và reverse ETL

480

General Considerations for Serving Data — Những điều cần cân nhắc chung khi phục vụ dữ liệu	481
Trust — Niềm tin	482
What's the Use Case, and Who's the User? — Tình huống sử dụng là gì, và ai là người dùng?	483
Data Products — Sản phẩm dữ liệu	485
Self-Service or Not? — Tự phục vụ hay không?	486
Data Definitions and Logic — Định nghĩa và logic dữ liệu	487
Data Mesh — Data mesh	489
Analytics — Phân tích	489
Business Analytics — Phân tích nghiệp vụ	490
Operational Analytics — Phân tích vận hành	492
BUSINESS AND OPERATIONAL ANALYTICS — PHÂN TÍCH NGHIỆP VỤ VÀ PHÂN TÍCH VẬN HÀNH	494
Embedded Analytics — Phân tích nhúng	495
Machine Learning — Học máy	496
What a Data Engineer Should Know About ML — Những điều kỹ sư dữ liệu nên biết về ML	497
Ways to Serve Data for Analytics and ML — Các cách phục vụ dữ liệu cho phân tích và ML	500
File Exchange — Trao đổi tệp	500
Databases — Cơ sở dữ liệu	501
Streaming Systems — Hệ thống theo luồng	503
Query Federation — Liên kết truy vấn	504
Data Sharing — Chia sẻ dữ liệu	505
Semantic and Metrics Layers — Tầng ngữ nghĩa và tầng chỉ số	506
Serving Data in Notebooks — Phục vụ dữ liệu trong notebook	507

CREDENTIAL HANDLING — XỬ LÝ THÔNG TIN XÁC THỰC	509
Reverse ETL — Reverse ETL	510
NOTE — GHI CHÚ	512
Ways to Serve Data with Reverse ETL — Các cách phục vụ dữ liệu với reverse ETL	512
Whom You'll Work With — Bạn sẽ làm việc với ai	513
Undercurrents — Các dòng chảy ngầm	514
Security — An ninh	514
Data Management — Quản lý dữ liệu	516
DataOps — DataOps	517
Data Architecture — Kiến trúc dữ liệu	518
Orchestration — Điều phối	518
Software Engineering — Kỹ thuật phần mềm	519
Conclusion — Kết luận	520
Additional Resources	521

Part III. Security, Privacy, and the Future of Data Engineering — Phần III. An ninh, Quyền riêng tư và Tương lai của Kỹ thuật Dữ liệu 523

Chapter 10. Security and Privacy — Chương 10. An ninh và Quyền riêng tư 524

People — Con người	525
The Power of Negative Thinking — Sức mạnh của Tư duy Tiêu cực	526
Always Be Paranoid — Luôn Luôn Hoài Nghi	526
Processes — Quy trình	527
Security Theater Versus Security Habit — An ninh Hình thức so với Thói quen An ninh	527
Active Security — An ninh Chủ động	528
The Principle of Least Privilege — Nguyên tắc Đặc quyền Tối thiểu	528
Shared Responsibility in the Cloud — Trách nhiệm Chia sẻ trên Cloud	529
Always Back Up Your Data — Luôn Sao lưu Dữ liệu của Bạn	529
An Example Security Policy — Một Ví dụ về Chính sách An ninh	530
EXAMPLE SECURITY POLICY Protect Your Credentials — CHÍNH SÁCH AN NINH MẪU Bảo vệ Thông tin Xác thực của Bạn	530
Protect Your Devices — Bảo vệ Thiết bị của Bạn	531
Software Update Policy — Chính sách Cập nhật Phần mềm	531
Technology — Công nghệ	532
Patch and Update Systems — Vá lỗi và Cập nhật Hệ thống	532
Encryption — Mã hóa	533
Encryption at rest — Mã hóa khi lưu trữ	533
Encryption over the wire — Mã hóa trên đường truyền	533
Logging, Monitoring, and Alerting — Ghi nhật ký, Giám sát và Cảnh báo	534
Network Access — Truy cập Mạng	535
Security for Low-Level Data Engineering — An ninh cho Kỹ thuật Dữ liệu Cấp thấp	536
Internal security research — Nghiên cứu an ninh nội bộ	537
Conclusion — Kết luận	537
Additional Resources	538

Chapter 11. The Future of Data Engineering — Chương 11. Tương lai của Kỹ thuật Dữ liệu 539

The Data Engineering Lifecycle Isn't Going Away — Vòng đời Kỹ thuật Dữ liệu sẽ không Biến mất	540
The Decline of Complexity and the Rise of Easy-to-Use Data Tools — Sự suy giảm của Độ phức tạp và Sự trỗi dậy của các Công cụ Dữ liệu Dễ sử dụng	541

The Cloud-Scale Data OS and Improved Interoperability — Hệ điều hành Dữ liệu Quy mô Đám mây và Khả năng Tương tác Cải thiện	542
“Enterprisey” Data Engineering — Kỹ thuật Dữ liệu kiểu “Doanh nghiệp”	545
Titles and Responsibilities Will Morph... — Chức danh và Trách nhiệm sẽ Biến hóa...	546
Moving Beyond the Modern Data Stack, Toward the Live Data Stack — Vượt ra ngoài Modern Data Stack, Hướng tới Live Data Stack	547
The Live Data Stack — Live Data Stack	548
Streaming Pipelines and Real-Time Analytical Databases — Đường ống Streaming và Database Phân tích Thời gian thực	549
The Fusion of Data with Applications — Sự hợp nhất của Dữ liệu với Ứng dụng	551
The Tight Feedback Between Applications and ML — Vòng Phản hồi Chặt chẽ giữa Ứng dụng và ML	552
Dark Matter Data and the Rise of...Spreadsheets?! — Dữ liệu Vật chất Tối và Sự trỗi dậy của... Bảng tính?!	552
Conclusion — Kết luận	553

Appendix A. Serialization and Compression Technical Details — Phụ lục A. Chi tiết kỹ thuật về Tuần tự hóa và Nén	556
Serialization Formats — Các định dạng Tuần tự hóa	556
Row-Based Serialization — Tuần tự hóa theo Hàng	557
CSV: The nonstandard standard — CSV: Chuẩn không chuẩn	557
XML — XML	557
JSON and JSONL — JSON và JSONL	558
Avro — Avro	558
Columnar Serialization — Tuần tự hóa theo Cột	558
Parquet — Parquet	559
ORC — ORC	560
Apache Arrow or in-memory serialization — Apache Arrow hay tuần tự hóa trong bộ nhớ	560
Hybrid Serialization — Tuần tự hóa Lai	561
Hudi — Hudi	562
Iceberg — Iceberg	562
Database Storage Engines — Các Engine Lưu trữ Cơ sở dữ liệu	562
Compression: gzip, bzip2, Snappy, etc. — Nén: gzip, bzip2, Snappy, v.v.	563

Appendix B. Cloud Networking — Phụ lục B. Mạng trên đám mây (Cloud Networking)	565
Cloud Network Topology — Cấu trúc liên kết mạng trên đám mây (Cloud Network Topology)	565
Data Egress Charges — Phí xuất dữ liệu (Data Egress Charges)	565
Availability Zones — Availability Zone	566
Regions — Region	567
GCP-Specific Networking and Multiregional Redundancy — Mạng đặc thù của GCP và dự phòng đa region	568
Direct Network Connections to the Clouds — Kết nối mạng trực tiếp tới các đám mây	569
CDNs — CDN	569
The Future of Data Egress Fees — Tương lai của phí xuất dữ liệu	569

Preface — Lời nói đầu

How did this book come about? The origin is deeply rooted in our journey from data science into **data engineering**. We often jokingly refer to ourselves as recovering data scientists. We both had the experience of being assigned to data science projects, then struggling to execute these projects due to a lack of proper foundations. Our journey into data engineering began when we undertook data engineering tasks to build foundations and infrastructure. With the rise of data science, companies splashed out lavishly on data science talent, hoping to reap rich rewards. Very often, data scientists struggled with basic problems that their background and training did not address—data collection, data cleansing, data access, data **transformation**, and data infrastructure. These are problems that data engineering aims to solve.

Cuốn sách này ra đời như thế nào? Nguồn gốc của nó bắt rễ sâu xa từ hành trình của chúng tôi đi từ khoa học dữ liệu (data science) sang kỹ thuật dữ liệu (data engineering). Chúng tôi thường nửa đùa nửa thật tự gọi mình là những nhà khoa học dữ liệu đang cai nghiện. Cả hai chúng tôi đều từng trải qua việc được giao các dự án khoa học dữ liệu, rồi phải vật lộn để triển khai những dự án đó vì thiếu nền tảng phù hợp. Hành trình bước vào kỹ thuật dữ liệu của chúng tôi bắt đầu khi chúng tôi đảm nhận các tác vụ kỹ thuật dữ liệu để xây dựng nền tảng và hạ tầng. Khi khoa học dữ liệu trở dậy, các công ty vung tiền hào phóng cho nhân tài khoa học dữ liệu, hy vọng gặt hái những phần thưởng dồi dào. Rất thường xuyên, các nhà khoa học dữ liệu lại chật vật với những vấn đề cơ bản mà nền tảng và quá trình đào tạo của họ không hề đề cập đến—thu thập dữ liệu, làm sạch dữ liệu, truy cập dữ liệu, biến đổi (transformation) dữ liệu và hạ tầng dữ liệu. Đó chính là những vấn đề mà kỹ thuật dữ liệu hướng tới giải quyết.

What This Book Isn't — Cuốn sách này KHÔNG phải là gì

Before we cover what this book is about and what you'll get out of it, let's quickly cover what this book isn't. This book isn't about data engineering using a particular tool, technology, or platform. While many excellent books approach data engineering technologies from this perspective, these books have a short shelf life. Instead, we try to focus on the fundamental concepts behind data engineering.

Trước khi nói về việc cuốn sách này viết về điều gì và bạn sẽ thu được gì từ nó, hãy nhanh chóng điểm qua việc cuốn sách này không phải là gì. Cuốn sách này không nói về kỹ thuật dữ liệu thông qua một công cụ, công nghệ hay nền tảng cụ thể nào. Dù có nhiều cuốn sách xuất sắc tiếp cận các công nghệ kỹ thuật dữ liệu theo góc nhìn này, những cuốn sách đó có tuổi thọ ngắn. Thay vào đó, chúng tôi cố gắng tập trung vào các khái niệm nền tảng đứng sau kỹ thuật dữ liệu.

What This Book Is About — Cuốn sách này nói về điều gì

This book aims to fill a gap in current data engineering content and materials. While there's no shortage of technical resources that address specific data engineering tools and technologies, people struggle to understand how to assemble these components into a coherent whole that applies in the real world. This book connects the dots of the end-to-end data lifecycle. It shows you how to stitch together various technologies to serve the needs of **downstream** data consumers such as analysts, data scientists, and **machine learning** engineers. This book works as a complement to O'Reilly books that cover the details of particular technologies, platforms and programming languages. The big idea of this book is the **data engineering lifecycle** : data **generation**, **storage**, **ingestion**, transformation, and **serving**. Since the dawn of data, we've seen the rise and fall of innumerable specific technologies and vendor products, but the data engineering life cycle stages have remained essentially unchanged. With this framework, the reader will come away with a sound understanding for applying technologies to real-world business problems. Our goal here is to map out principles that reach across two axes. First, we wish to distill data engineering into principles that can encompass any relevant technology . Second, we wish to present principles that will stand the test of time . We hope that these ideas reflect lessons learned across the data technology upheaval of the last twenty years and that our mental framework will remain useful for a decade or more into the future. One thing to note: we unapologetically take a cloud-first approach. We view the cloud as a fundamentally transformative development that will endure for decades; most on-premises data systems and workloads will eventually move to cloud hosting. We assume that infrastructure and systems are ephemeral and scalable , and that data engineers will lean toward deploying managed services in the cloud. That said, most concepts in this book will translate to non-cloud environments.

Cuốn sách này hướng tới việc lấp đầy một khoảng trống trong nội dung và tài liệu về kỹ thuật dữ liệu hiện nay. Dù không hề thiếu các tài nguyên kỹ thuật bàn về những công cụ và công nghệ kỹ thuật dữ liệu cụ thể, người ta lại chật vật trong việc hiểu cách lắp ráp những thành phần này thành một tổng thể mạch lạc có thể áp dụng vào thế giới thực. Cuốn sách này nối các điểm của vòng đời dữ liệu đầu-cuối. Nó chỉ cho bạn cách ghép nối các công nghệ khác nhau để phục vụ nhu cầu của những bên tiêu thụ dữ liệu hạ nguồn (downstream) như các nhà phân tích, nhà khoa học dữ liệu và kỹ sư học máy. Cuốn sách này đóng vai trò hỗ trợ cho các cuốn sách của O'Reilly vốn đi sâu vào chi tiết của những công nghệ, nền tảng và ngôn ngữ lập trình cụ thể. Ý tưởng lớn của cuốn sách này là vòng đời kỹ thuật dữ liệu (data engineering lifecycle): sinh dữ liệu (generation), lưu trữ, thu nạp (ingestion), biến đổi và phục vụ (serving). Kể từ buổi bình minh của dữ liệu, chúng ta đã chứng kiến sự trỗi dậy và suy tàn của vô số công nghệ cụ thể và sản phẩm của các nhà cung cấp, nhưng các giai đoạn của vòng đời kỹ thuật dữ liệu về cơ bản vẫn không thay đổi. Với khung tư duy này, người đọc sẽ ra về với một sự hiểu biết vững chắc để áp dụng công nghệ vào các bài toán kinh doanh trong thế giới thực. Mục tiêu của chúng tôi ở đây là vạch ra những nguyên tắc trải dài trên hai trục. Thứ nhất, chúng tôi muốn chưng cất kỹ thuật dữ liệu thành những nguyên tắc có thể bao trùm bất kỳ công nghệ liên quan nào. Thứ hai, chúng tôi muốn trình bày những nguyên tắc đứng vững được trước thử thách của thời gian. Chúng tôi hy vọng những ý tưởng này phản ánh các bài học rút ra từ cơn biến động công nghệ dữ liệu suốt hai mươi năm qua và rằng khung tư duy của chúng tôi sẽ còn hữu ích trong một thập kỷ hoặc hơn nữa của tương lai. Có một điều cần lưu ý: chúng tôi áp dụng cách tiếp cận ưu tiên đám mây (cloud-first) một cách không e ngại. Chúng tôi xem đám mây là một bước phát triển có tính chuyển hóa căn bản sẽ trường tồn trong nhiều thập kỷ; hầu hết các hệ thống và khối lượng công việc dữ liệu tại chỗ (on-premises) rồi sẽ chuyển sang lưu trữ trên đám mây. Chúng tôi giả định rằng hạ tầng và hệ thống là phù du và có khả năng mở rộng, và rằng các kỹ sư dữ liệu sẽ thiên về việc triển khai các dịch vụ

được quản lý (managed service) trên đám mây. Dù vậy, hầu hết các khái niệm trong cuốn sách này đều có thể chuyển dịch sang môi trường không phải đám mây.

Who Should Read This Book — Ai nên đọc cuốn sách này

Our primary intended audience for this book consists of technical practitioners, mid- to senior-level software engineers, data scientists, or analysts interested in moving into data engineering; or data engineers working in the guts of specific technologies, but wanting to develop a more comprehensive perspective. Our secondary target audience consists of data stakeholders who work adjacent to technical practitioners—e.g., a data team lead with a technical background overseeing a team of data engineers, or a director of data warehousing wanting to migrate from on-premises technology to a cloud-based solution. Ideally, you're curious and want to learn—why else would you be reading this book? You stay current with data technologies and trends by reading books and articles on data warehousing/data lakes, **batch** and **streaming** systems, **orchestration**, modeling, management, analysis, developments in cloud technologies, etc. This book will help you weave what you've read into a complete picture of data engineering across technologies and paradigms.

Đối tượng độc giả chính mà chúng tôi nhắm tới gồm các nhà thực hành kỹ thuật, các kỹ sư phần mềm cấp trung đến cấp cao, các nhà khoa học dữ liệu hoặc nhà phân tích quan tâm đến việc chuyển sang kỹ thuật dữ liệu; hoặc những kỹ sư dữ liệu đang làm việc trong lòng những công nghệ cụ thể nhưng muốn phát triển một góc nhìn toàn diện hơn. Đối tượng độc giả thứ cấp của chúng tôi gồm các bên liên quan dữ liệu (stakeholder) làm việc kề cận với các nhà thực hành kỹ thuật—chẳng hạn một trưởng nhóm dữ liệu có nền tảng kỹ thuật đang giám sát một đội ngũ kỹ sư dữ liệu, hoặc một giám đốc về kho dữ liệu (data warehouse) muốn di trú từ công nghệ tại chỗ sang giải pháp dựa trên đám mây. Lý tưởng nhất là bạn tò mò và muốn học hỏi—chứ còn vì sao khác mà bạn lại đọc cuốn sách này? Bạn cập nhật các công nghệ và xu hướng dữ liệu bằng cách đọc sách và bài viết về kho dữ liệu/hồ dữ liệu (data lake), hệ thống theo lô (batch) và theo luồng (streaming), điều phối (orchestration), mô hình hóa, quản lý, phân tích, các phát triển trong công nghệ đám mây, v.v. Cuốn sách này sẽ giúp bạn đan kết những gì đã đọc thành một bức tranh hoàn chỉnh về kỹ thuật dữ liệu xuyên suốt các công nghệ và mô thức.

Prerequisites — Kiến thức cần có trước

We assume a good deal of familiarity with the types of data systems found in a corporate setting. In addition, we assume that readers have some familiarity with SQL and Python (or some other programming language), and experience with cloud services. Numerous resources are available for aspiring data engineers to practice Python and SQL. Free online resources abound (blog posts, tutorial sites, YouTube videos), and many new Python books are published every year. The cloud provides unprecedented opportunities to get hands-on experience with data tools. We suggest that aspiring data engineers set up accounts with cloud services such as AWS, Azure, Google Cloud Platform, Snowflake, Databricks, etc. Note that many of these platforms have free tier options, but readers should keep a close eye on costs, and work with small quantities of data and single node clusters as they study. Developing familiarity with corporate data systems outside of a corporate environment remains difficult and this creates certain barriers for aspiring data engineers who have yet to land their first data job. This book can help. We suggest that data novices read for high level ideas, and then look at materials in the additional resources section at the end of each chapter. On a second read through, note any unfamiliar terms and technologies. You can utilize Google, Wikipedia, blog posts, YouTube videos, and vendor sites to become familiar with new terms and fill

gaps in your understanding.

Chúng tôi giả định bạn đã khá quen thuộc với các loại hệ thống dữ liệu thường gặp trong môi trường doanh nghiệp. Ngoài ra, chúng tôi giả định người đọc có chút quen thuộc với SQL và Python (hoặc một ngôn ngữ lập trình khác), cùng kinh nghiệm với các dịch vụ đám mây. Có vô số tài nguyên dành cho những người mong muốn trở thành kỹ sư dữ liệu để luyện tập Python và SQL. Tài nguyên trực tuyến miễn phí có rất nhiều (bài blog, các trang hướng dẫn, video YouTube), và mỗi năm lại có nhiều sách Python mới được xuất bản. Đám mây mang lại những cơ hội chưa từng có để có được trải nghiệm thực hành với các công cụ dữ liệu. Chúng tôi gợi ý những ai khao khát trở thành kỹ sư dữ liệu hãy lập tài khoản với các dịch vụ đám mây như AWS, Azure, Google Cloud Platform, Snowflake, Databricks, v.v. Lưu ý rằng nhiều nền tảng trong số này có các lựa chọn bậc miễn phí (free tier), nhưng người đọc nên theo dõi sát chi phí, và làm việc với lượng dữ liệu nhỏ cùng các cụm (cluster) đơn nút trong khi học. Việc phát triển sự quen thuộc với các hệ thống dữ liệu doanh nghiệp bên ngoài môi trường doanh nghiệp vẫn còn khó khăn, và điều này tạo ra một số rào cản nhất định cho những ai khao khát trở thành kỹ sư dữ liệu mà chưa kiếm được công việc dữ liệu đầu tiên. Cuốn sách này có thể giúp ích. Chúng tôi gợi ý những người mới về dữ liệu hãy đọc để nắm các ý tưởng ở mức cao, rồi xem các tài liệu trong phần tài nguyên bổ sung ở cuối mỗi chương. Trong lần đọc thứ hai, hãy ghi chú lại bất kỳ thuật ngữ và công nghệ nào còn xa lạ. Bạn có thể tận dụng Google, Wikipedia, các bài blog, video YouTube và các trang của nhà cung cấp để làm quen với những thuật ngữ mới và lấp đầy những lỗ hổng trong hiểu biết của mình.

What You'll Learn and How It Will Improve Your Abilities — Bạn sẽ học được gì và nó sẽ cải thiện năng lực của bạn ra sao

This book aims to help you build a solid foundation for solving real world data engineering problems. By the end of this book you will understand:

Cuốn sách này hướng tới việc giúp bạn xây dựng một nền tảng vững chắc để giải quyết các bài toán kỹ thuật dữ liệu trong thế giới thực. Đến cuối cuốn sách này bạn sẽ hiểu:

How data engineering impacts your current role (data scientist, software engineer, or data team lead).

Kỹ thuật dữ liệu tác động ra sao tới vai trò hiện tại của bạn (nhà khoa học dữ liệu, kỹ sư phần mềm, hoặc trưởng nhóm dữ liệu).

How to cut through the marketing hype and choose the right technologies, **data architecture**, and processes.

Làm thế nào để xuyên thấu lớp tung hô tiếp thị (marketing hype) và chọn đúng công nghệ, kiến trúc dữ liệu và quy trình.

How to use the data engineering lifecycle to design and build a robust architecture.

Làm thế nào để dùng vòng đời kỹ thuật dữ liệu nhằm thiết kế và xây dựng một kiến trúc vững chắc.

Best practices for each stage of the data lifecycle.

Các thực hành tốt nhất cho từng giai đoạn của vòng đời dữ liệu.

And you will be able to:

Và bạn sẽ có thể:

Incorporate data engineering principles in your current role (data scientist, analyst, software engineer, data team lead, etc.)

Đưa các nguyên tắc kỹ thuật dữ liệu vào vai trò hiện tại của mình (nhà khoa học dữ liệu, nhà phân tích, kỹ sư phần mềm, trưởng nhóm dữ liệu, v.v.)

Stitch together a variety of cloud technologies to serve the needs of downstream data consumers.

Ghép nối nhiều công nghệ đám mây khác nhau để phục vụ nhu cầu của các bên tiêu thụ dữ liệu hạ nguồn.

Assess data engineering problems with an end-to-end framework of best practices

Đánh giá các bài toán kỹ thuật dữ liệu bằng một khung đầu-cuối gồm các thực hành tốt nhất.

Incorporate **data governance** and security across the data engineering lifecycle.

Tích hợp quản trị dữ liệu (data governance) và bảo mật xuyên suốt vòng đời kỹ thuật dữ liệu.

The Book Outline — Dàn ý của cuốn sách

This book is composed of four parts:

Cuốn sách này gồm bốn phần:

Part I, “Foundation and Building Blocks”

Phần I, “Nền tảng và các Khối xây dựng”

Part II, “The Data Engineering Lifecycle in Depth”

Phần II, “Vòng đời Kỹ thuật Dữ liệu một cách Chuyên sâu”

Part III, “Security, **Privacy**, and the Future of Data Engineering”

Phần III, “Bảo mật, Quyền riêng tư, và Tương lai của Kỹ thuật Dữ liệu”

Appendices A and B : cloud networking, **serialization** and **compression**

Phụ lục A và B: mạng đám mây, tuần tự hóa (serialization) và nén (compression)

In Part I , we begin by defining data engineering in Chapter 1 , then map out the data engineering lifecycle in Chapter 2 . In Chapter 3 , we discuss good architecture . In Chapter 4 , we introduce a framework for choosing the right technology—while we frequently see technology and architecture conflated, these are in fact very different topics. Part II builds on Chapter 2 to cover the data engineering lifecycle in depth; each lifecycle stage—data generation, storage, ingestion, transformation and serving—is covered in its own chapter. Part II is arguably the heart of the book, and the other chapters exist to support the core ideas covered here. Part III covers additional topics. In Chapter 10 , we discuss security and privacy . While security has always been an important part of the data engineering profession, it has only become more critical with the rise of for profit hacking and state sponsored cyber attacks. And what can we say of privacy? The era of corporate privacy nihilism is over—no company wants to see its name appear in the headline of an article on sloppy privacy practices. Reckless handling of personal data can also have significant legal ramifications with the advent of GDPR, CCPA and other regulations. In short, security and privacy must be top priorities in any data engineering work. In the course of working in data engineering,

doing research for this book and interviewing numerous experts, we thought a good deal about where the field is going in the near and long term. Chapter 11 outlines our highly speculative ideas on the future of data engineering. By its nature, the future is a slippery thing. Time will tell if some of our ideas are correct. We would love to hear from our readers on how their visions of the future agree with or differ from our own. In the appendix, we cover a handful of technical topics that are extremely relevant to the day to day practice of data engineering, but didn't fit into the main body of the text. Specifically, cloud networking is a critical topic as data engineering shifts into the cloud, and engineers need to understand serialization and compression both to work directly with data files, and to assess performance considerations in data systems.

Trong Phần I, chúng tôi bắt đầu bằng việc định nghĩa kỹ thuật dữ liệu ở Chương 1, rồi vạch ra vòng đời kỹ thuật dữ liệu ở Chương 2. Ở Chương 3, chúng tôi bàn về một kiến trúc tốt. Ở Chương 4, chúng tôi giới thiệu một khung để chọn đúng công nghệ—dù chúng tôi thường thấy người ta gộp lẫn công nghệ và kiến trúc, thực ra đây là hai chủ đề rất khác nhau. Phần II xây dựng dựa trên Chương 2 để bàn về vòng đời kỹ thuật dữ liệu một cách chuyên sâu; mỗi giai đoạn của vòng đời—sinh dữ liệu, lưu trữ, thu nạp, biến đổi và phục vụ—được trình bày trong một chương riêng. Có thể nói Phần II là trái tim của cuốn sách, và các chương khác tồn tại để hỗ trợ những ý tưởng cốt lõi được đề cập ở đây. Phần III bàn về các chủ đề bổ sung. Ở Chương 10, chúng tôi bàn về bảo mật và quyền riêng tư. Dù bảo mật xưa nay luôn là một phần quan trọng của nghề kỹ thuật dữ liệu, nó chỉ càng trở nên trọng yếu hơn khi nạn tấn công vì lợi nhuận và các cuộc tấn công mạng do nhà nước bảo trợ trở nên phổ biến. Còn quyền riêng tư thì sao? Thời đại chủ nghĩa hư vô về quyền riêng tư của doanh nghiệp đã chấm dứt—không công ty nào muốn thấy tên mình xuất hiện trên tựa đề một bài báo về những thực hành quyền riêng tư cầu thả. Việc xử lý dữ liệu cá nhân một cách liêm chính cũng có thể kéo theo những hệ lụy pháp lý đáng kể với sự ra đời của GDPR, CCPA và các quy định khác. Tóm lại, bảo mật và quyền riêng tư phải là những ưu tiên hàng đầu trong mọi công việc kỹ thuật dữ liệu. Trong quá trình làm việc trong lĩnh vực kỹ thuật dữ liệu, nghiên cứu cho cuốn sách này và phỏng vấn vô số chuyên gia, chúng tôi đã suy nghĩ rất nhiều về việc lĩnh vực này sẽ đi về đâu trong ngắn hạn và dài hạn. Chương 11 phác họa những ý tưởng hết sức mang tính suy đoán của chúng tôi về tương lai của kỹ thuật dữ liệu. Tự bản chất của nó, tương lai là một thứ trơn tuột. Thời gian sẽ trả lời liệu một số ý tưởng của chúng tôi có đúng hay không. Chúng tôi rất muốn nghe từ độc giả xem tầm nhìn của họ về tương lai đồng thuận hay khác biệt với chúng tôi ra sao. Trong phần phụ lục, chúng tôi đề cập một số ít chủ đề kỹ thuật cực kỳ liên quan đến thực hành hằng ngày của kỹ thuật dữ liệu, nhưng lại không vừa với phần thân chính của văn bản. Cụ thể, mạng đám mây là một chủ đề trọng yếu khi kỹ thuật dữ liệu dịch chuyển vào đám mây, và các kỹ sư cần hiểu tuần tự hóa và nén cả để làm việc trực tiếp với các tệp dữ liệu, lẫn để đánh giá các yếu tố hiệu năng trong hệ thống dữ liệu.

Conventions Used in This Book — Các quy ước được dùng trong cuốn sách này

The following typographical conventions are used in this book: *Italic* Indicates new terms, URLs, email addresses, filenames, and file extensions.

Các quy ước về kiểu chữ sau đây được dùng trong cuốn sách này: *Chữ nghiêng* (Italic) chỉ các thuật ngữ mới, URL, địa chỉ email, tên tệp và phần mở rộng tệp.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable

or function names, databases, data types, environment variables, statements, and keywords.

Dùng cho các đoạn liệt kê chương trình (program listing), cũng như bên trong các đoạn văn để chỉ các phần tử chương trình như tên biến hoặc tên hàm, cơ sở dữ liệu, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Cho thấy các lệnh hoặc văn bản khác mà người dùng nên gõ nguyên văn theo đúng như vậy.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

Cho thấy văn bản nên được thay bằng các giá trị do người dùng cung cấp hoặc bằng các giá trị được xác định theo ngữ cảnh.

TIP — MẸO

This element signifies a tip or suggestion.

Phần tử này biểu thị một mẹo hoặc gợi ý.

NOTE — GHI CHÚ

This element signifies a general note.

Phần tử này biểu thị một ghi chú chung.

WARNING — CẢNH BÁO

This element indicates a warning or caution.

Phần tử này biểu thị một cảnh báo hoặc lưu ý thận trọng.

How to Contact Us — Cách liên hệ với chúng tôi

Please address comments and questions concerning this book to the publisher:

Vui lòng gửi các nhận xét và câu hỏi liên quan đến cuốn sách này tới nhà xuất bản:

O'Reilly Media, Inc.

O'Reilly Media, Inc.

1005 Gravenstein Highway North

1005 Gravenstein Highway North

Sebastopol, CA 95472

Sebastopol, CA 95472

800-998-9938 (in the United States or Canada)

800-998-9938 (tại Hoa Kỳ hoặc Canada)

707-829-0515 (international or local)

707-829-0515 (quốc tế hoặc địa phương)

707-829-0104 (fax)

707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/fundamentals-of-data>.

Chúng tôi có một trang web cho cuốn sách này, nơi chúng tôi liệt kê các đính chính, ví dụ và mọi thông tin bổ sung khác. Bạn có thể truy cập trang này tại <https://oreil.ly/fundamentals-of-data>.

Email bookquestions@oreilly.com to comment or ask technical questions about this book. For news and information about our books and courses, visit <https://oreilly.com>. Find us on LinkedIn: <https://linkedin.com/company/oreilly-media> Follow us on Twitter: <https://twitter.com/oreillymedia> Watch us on YouTube: <https://www.youtube.com/oreillymedia>

Gửi email tới bookquestions@oreilly.com để nhận xét hoặc đặt câu hỏi kỹ thuật về cuốn sách này. Để biết tin tức và thông tin về sách và khóa học của chúng tôi, hãy truy cập <https://oreilly.com>. Tìm chúng tôi trên LinkedIn: <https://linkedin.com/company/oreilly-media> Theo dõi chúng tôi trên Twitter: <https://twitter.com/oreillymedia> Xem chúng tôi trên YouTube: <https://www.youtube.com/oreillymedia>

Acknowledgments — Lời cảm ơn

When we started writing this book, we were warned by many people that we faced a hard task. A book like this has a lot of moving parts, and due to its comprehensive view of the field of data engineering, it required a ton of research, interviews, discussions, and deep thinking. We won't claim to have captured every nuance of data engineering, but we hope that the results resonate with you. Numerous individuals contributed to our efforts, and we're grateful for the support we received from many experts. First, thanks to our amazing crew of technical reviewers. They slogged through many readings, and gave invaluable (and often ruthlessly blunt) feedback. This book would be a fraction of itself without their efforts. In no particular order, we give endless thanks to Bill **Inmon**, Andy Petrella, Matt Sharp, Tod Hanseman, Chris Tabb, Danny Lebzyon, Martin Kleppman, Scott Lorimor, Nick Schrock, Lisa Steckman, and Alex Woolford. Second, we've had a unique opportunity to talk with the leading experts in the field of data on our live shows, podcasts, meetups, and endless private calls. Their ideas helped shape our book. There are too many people to name individually, but we'd like to give shoutouts to Bill Inmon, Jordan Tigani, Zhamak Dehghani, Shruti Bhat, Eric Tschetter, Benn Stancil, Kevin Hu, Michael Rogove, Ryan Wright, Egor Gryaznov, Chad Sanderson, Julie Price, Matt Turck, Monica Rogati, Mars Lan, Pardhu Gunnam, Brian Suk, Barr Moses, Lior Gavish, Bruno Aziza, Gian Merlino, DeVaris Brown, Todd Beauchene, Tudor Girba, Scott Taylor, Ori Rafael, Lee Edwards, Bryan Offutt, Ollie Hughes, Gilbert Eijkelenboom, Chris Bergh, Fabiana Clemente, Andreas Kretz, Ori Reshef, Nick Singh, Mark Balkenende, Kenten Danas, Brian Olsen, Lior Gavish, Rhaghu Murthy, Greg Coquillo, David Aponte, Demetrios Brinkmann, Sarah Catanzaro, Michel Tricot, Levi Davis, Ted Walker, Carlos Kemeny, Josh Benamram, Chanin Nantasenamat, George Firican, Jordan Goldmeir, Minhaaj Rehman, Luigi Patrino, Vin Vashista, Danny Ma, Jesse Anderson, Alessya Visnjic, Vishal Singh, Dave Langer, Roy Hasson, Todd Odess, Che

Sharma, Scott Breitenother, Ben Taylor, Thom Ives, John Thompson, Brent Dykes, Josh Tobin, Mark Kosiba, Tyler Pugliese, Douwe Maan, Martin Traverso, Curtis Kowalski, Bob Davis, Koo Ping Shung, Ed Chenard, Matt Sciorma, Tyler Folkman, Jeff Baird, Tejas Manohar, Paul Singman, Kevin Stumpf, Willem Pineaar, and Michael Del Balso from Tecton, Emma Dahl, Harpreet Sahota, Ken Jee, Scott Taylor, Kate Strachnyi, Kristen Kehrer, Taylor Miller, Abe Gong, Ben Castleton, Ben Rogojan, David Mertz, Emmanuel Raj, Andrew Jones, Avery Smith, Brock Cooper, Jeff Larson, Jon King, Holden Ackerman, Miriah Peterson, Felipe Hoffa, David Gonzalez, Richard Wellman, Susan Walsh, Ravit Jain, Lauren Balik, Mikiko Bazeley, Mark Freeman, Mike Wimmer, Alexey Shchedrin, Mary Clair Thompson, Julie Burroughs, Jason Pedley, Freddy Drennan, Jake Carter, Jason Pedley, Kelly and Matt Phillipps, Brian Campbell, Faris Chebib, Dylan Gregerson, Ken Myers, and many others. If you're not mentioned specifically, don't take it personally. You know who you are. Let us know and we'll get you on the next edition. We'd also like to thank the Ternary Data team, our students, and the countless people around the world who've supported us. It's a great reminder the world is a very small place. Working with the O'Reilly crew was amazing! Special thanks to Jess Haberman for having confidence in us during the book proposal process, our amazing and extremely patient development editors Nicole Taché and Michele Cronin for invaluable editing, feedback and support. Thank you also to the superb production crew at O'Reilly (Greg and crew).

Khi chúng tôi bắt đầu viết cuốn sách này, nhiều người đã cảnh báo rằng chúng tôi đang đối mặt với một nhiệm vụ khó khăn. Một cuốn sách như thế này có rất nhiều bộ phận chuyển động, và do nó mang góc nhìn toàn diện về lĩnh vực kỹ thuật dữ liệu, nó đòi hỏi một khối lượng nghiên cứu, phỏng vấn, thảo luận và suy ngẫm sâu sắc khổng lồ. Chúng tôi sẽ không dám tuyên bố đã nắm bắt được mọi sắc thái của kỹ thuật dữ liệu, nhưng chúng tôi hy vọng kết quả sẽ cộng hưởng với bạn. Vô số cá nhân đã đóng góp cho nỗ lực của chúng tôi, và chúng tôi biết ơn sự hỗ trợ nhận được từ nhiều chuyên gia. Trước hết, xin cảm ơn đội ngũ phản biện kỹ thuật tuyệt vời của chúng tôi. Họ đã lê bước qua biết bao lần đọc, và đưa ra những phản hồi vô giá (và thường là thẳng thừng đến tàn nhẫn). Cuốn sách này sẽ chỉ còn lại một phần nhỏ của chính nó nếu không có nỗ lực của họ. Không theo thứ tự nào cụ thể, chúng tôi gửi lời cảm ơn vô tận tới Bill Inmon, Andy Petrella, Matt Sharp, Tod Hanseman, Chris Tabb, Danny Lebzyon, Martin Kleppman, Scott Lorimer, Nick Schrock, Lisa Steckman, và Alex Woolford. Thứ hai, chúng tôi đã có cơ hội độc nhất để trò chuyện với những chuyên gia hàng đầu trong lĩnh vực dữ liệu qua các chương trình phát trực tiếp, podcast, buổi gặp gỡ (meetup) và vô số cuộc gọi riêng. Ý tưởng của họ đã giúp định hình nên cuốn sách của chúng tôi. Có quá nhiều người để gọi tên từng cá nhân, nhưng chúng tôi muốn dành lời tri ân tới Bill Inmon, Jordan Tigani, Zhamak Dehghani, Shruti Bhat, Eric Tschetter, Benn Stancil, Kevin Hu, Michael Rogove, Ryan Wright, Egor Gryaznov, Chad Sanderson, Julie Price, Matt Turck, Monica Rogati, Mars Lan, Pardhu Gunnam, Brian Suk, Barr Moses, Lior Gavish, Bruno Aziza, Gian Merlino, DeVaris Brown, Todd Beauchene, Tudor Girba, Scott Taylor, Ori Rafael, Lee Edwards, Bryan Offutt, Ollie Hughes, Gilbert Eijkelenboom, Chris Bergh, Fabiana Clemente, Andreas Kretz, Ori Reshef, Nick Singh, Mark Balkenende, Kenten Danas, Brian Olsen, Lior Gavish, Rhaghu Murthy, Greg Coquillo, David Aponte, Demetrios Brinkmann, Sarah Catanzaro, Michel Tricot, Levi Davis, Ted Walker, Carlos Kemeny, Josh Benamram, Chanin Nantasenamat, George Firican, Jordan Goldmeir, Minhaaj Rehman, Luigi Patruno, Vin Vashista, Danny Ma, Jesse Anderson, Alessya Visnjic, Vishal Singh, Dave Langer, Roy Hasson, Todd Odess, Che Sharma, Scott Breitenother, Ben Taylor, Thom Ives, John Thompson, Brent Dykes, Josh Tobin, Mark Kosiba, Tyler Pugliese, Douwe Maan, Martin Traverso, Curtis Kowalski, Bob Davis, Koo Ping Shung, Ed Chenard, Matt Sciorma, Tyler Folkman, Jeff Baird, Tejas Manohar, Paul Singman, Kevin Stumpf, Willem Pineaar, và Michael Del Balso từ Tecton, Emma Dahl, Harpreet Sahota, Ken Jee, Scott Taylor, Kate Strachnyi, Kristen Kehrer, Taylor Miller, Abe Gong, Ben Castleton, Ben Rogojan, David Mertz, Emmanuel Raj, Andrew Jones,

Avery Smith, Brock Cooper, Jeff Larson, Jon King, Holden Ackerman, Miriah Peterson, Felipe Hoffa, David Gonzalez, Richard Wellman, Susan Walsh, Ravit Jain, Lauren Balik, Mikiko Bazeley, Mark Freeman, Mike Wimmer, Alexey Shchedrin, Mary Clair Thompson, Julie Burroughs, Jason Pedley, Freddy Drennan, Jake Carter, Jason Pedley, Kelly và Matt Phillipps, Brian Campbell, Faris Chebib, Dylan Gregerson, Ken Myers, và nhiều người khác. Nếu bạn không được nhắc đến cụ thể, xin đừng nhận đó là chuyện cá nhân. Bạn biết bạn là ai. Hãy cho chúng tôi biết và chúng tôi sẽ đưa bạn vào lần xuất bản tiếp theo. Chúng tôi cũng muốn cảm ơn đội ngũ Ternary Data, các học viên của chúng tôi, và vô số người trên khắp thế giới đã ủng hộ chúng tôi. Đó là một lời nhắc tuyệt vời rằng thế giới này thật nhỏ bé. Làm việc với đội ngũ O'Reilly thật tuyệt vời! Xin cảm ơn đặc biệt Jess Haberman vì đã tin tưởng chúng tôi trong quá trình đề xuất cuốn sách, các biên tập viên phát triển tuyệt vời và cực kỳ kiên nhẫn của chúng tôi là Nicole Taché và Michele Cronin vì sự biên tập, phản hồi và hỗ trợ vô giá. Cũng xin cảm ơn đội ngũ sản xuất xuất sắc tại O'Reilly (Greg và đồng đội).

Joe would like to thank his family—Cassie, Milo, and Ethan—for letting him write a book. They had to endure a ton, and Joe promises to never write another book again ;) Matt would like to thank his friends and family for their enduring patience and support. He's still hopeful that Seneca will deign to give a five star review after a good deal of toil and missed family time around the holidays.

Joe muốn cảm ơn gia đình mình—Cassie, Milo và Ethan—vì đã để anh viết một cuốn sách. Họ đã phải chịu đựng rất nhiều, và Joe hứa sẽ không bao giờ viết một cuốn sách nào nữa ;) Matt muốn cảm ơn bạn bè và gia đình vì sự kiên nhẫn và ủng hộ bền bỉ của họ. Anh vẫn còn hy vọng rằng Seneca sẽ hạ cố cho một đánh giá năm sao sau bao công sức nhọc nhằn và những khoảng thời gian bên gia đình bị bỏ lỡ quanh dịp lễ.

Part I. Foundation and Building Blocks — Phần I. Nền tảng và các khối xây dựng

Chapter 1. Data Engineering

Described — Chương 1. Mô tả về Kỹ thuật Dữ liệu

If you work in data or software, you may have noticed **data engineering** emerging from the shadows and now sharing the stage with data science. Data engineering is one of the hottest fields in data and technology, and for a good reason. It builds the foundation for data science and **analytics** in production. This chapter explores what data engineering is, how the field was born and its evolution, the skills of data engineers, and with whom they work.

Nếu bạn làm việc trong lĩnh vực dữ liệu hoặc phần mềm, có lẽ bạn đã nhận thấy kỹ thuật dữ liệu (data engineering) đang bước ra khỏi bóng tối và giờ đây chia sẻ sân khấu cùng khoa học dữ liệu. Kỹ thuật dữ liệu là một trong những lĩnh vực nóng nhất trong dữ liệu và công nghệ, và điều đó có lý do chính đáng. Nó xây dựng nền tảng cho khoa học dữ liệu và phân tích (analytics) trong môi trường vận hành. Chương này khám phá kỹ thuật dữ liệu là gì, lĩnh vực này ra đời và tiến hóa ra sao, các kỹ năng của kỹ sư dữ liệu, và họ làm việc cùng những ai.

What Is Data Engineering? — Kỹ thuật Dữ liệu là gì?

Despite the current popularity of data engineering, there's a lot of confusion about what data engineering means and what data engineers do. Data engineering has existed in some form since companies started doing things with data—such as predictive analysis, descriptive analytics, and reports—and came into sharp focus alongside the rise of data science in the 2010s. For the purpose of this book, it's critical to define what data engineering and **data engineer** mean. First, let's look at the landscape of how data engineering is described and develop some terminology we can use throughout this book. Endless definitions of data engineering exist. In early 2022, a Google exact-match search for “what is data engineering?” returns over 91,000 unique results. Before we give our definition, here are a few examples of how some experts in the field define data engineering:

Bất chấp sự phổ biến hiện nay của kỹ thuật dữ liệu, vẫn còn rất nhiều nhầm lẫn về việc kỹ thuật dữ liệu nghĩa là gì và kỹ sư dữ liệu làm những gì. Kỹ thuật dữ liệu đã tồn tại dưới một hình thức nào đó kể từ khi các công ty bắt đầu làm việc với dữ liệu—chẳng hạn như phân tích dự đoán, phân tích mô tả, và báo cáo—và được chú ý mạnh mẽ song hành cùng sự trỗi dậy của khoa học dữ liệu trong thập niên 2010. Vì mục đích của cuốn sách này, việc định nghĩa kỹ thuật dữ liệu và kỹ sư dữ liệu nghĩa là gì là điều then chốt. Trước hết, hãy nhìn vào bức tranh tổng thể về cách kỹ thuật dữ liệu được mô tả và xây dựng một số thuật ngữ mà ta có thể dùng xuyên suốt cuốn sách này. Có vô số định nghĩa về kỹ thuật dữ liệu

tồn tại. Vào đầu năm 2022, một tìm kiếm khớp chính xác trên Google cho cụm “what is data engineering?” trả về hơn 91.000 kết quả độc nhất. Trước khi đưa ra định nghĩa của riêng mình, đây là một vài ví dụ về cách một số chuyên gia trong lĩnh vực định nghĩa kỹ thuật dữ liệu:

Data engineering is a set of operations aimed at creating interfaces and mechanisms for the flow and access of information. It takes dedicated specialists—data engineers—to maintain data so that it remains available and usable by others. In short, data engineers set up and operate the organization’s data infrastructure, preparing it for further analysis by data analysts and scientists. —From “Data Engineering and Its Main Concepts” by AlexSoft 1

Kỹ thuật dữ liệu là một tập hợp các thao tác nhằm tạo ra các giao diện và cơ chế cho dòng chảy và truy cập thông tin. Cần đến những chuyên gia chuyên trách—các kỹ sư dữ liệu—để duy trì dữ liệu sao cho nó luôn sẵn sàng và sử dụng được bởi người khác. Nói ngắn gọn, kỹ sư dữ liệu thiết lập và vận hành hạ tầng dữ liệu của tổ chức, chuẩn bị nó để các nhà phân tích dữ liệu và nhà khoa học dữ liệu phân tích sâu hơn. —Trích từ “Data Engineering and Its Main Concepts” của AlexSoft 1

The first type of data engineering is SQL-focused. The work and primary **storage** of the data is in relational databases. All of the data processing is done with SQL or a SQL-based language. Sometimes, this data processing is done with an **ETL** tool. The second type of data 2 engineering is Big Data-focused. The work and primary storage of the data is in Big Data technologies like Hadoop, Cassandra, and HBase. All of the data processing is done in Big Data frameworks like MapReduce, Spark, and Flink. While SQL is used, the primary processing is done with programming languages like Java, Scala, and Python. —Jesse Anderson 3

Loại kỹ thuật dữ liệu thứ nhất tập trung vào SQL. Công việc và nơi lưu trữ chính của dữ liệu nằm trong các cơ sở dữ liệu quan hệ. Toàn bộ việc xử lý dữ liệu được thực hiện bằng SQL hoặc một ngôn ngữ dựa trên SQL. Đôi khi, việc xử lý dữ liệu này được thực hiện bằng một công cụ ETL. Loại kỹ thuật dữ liệu thứ hai 2 tập trung vào Big Data. Công việc và nơi lưu trữ chính của dữ liệu nằm trong các công nghệ Big Data như Hadoop, Cassandra, và HBase. Toàn bộ việc xử lý dữ liệu được thực hiện trong các framework Big Data như MapReduce, Spark, và Flink. Mặc dù SQL có được sử dụng, việc xử lý chính được thực hiện bằng các ngôn ngữ lập trình như Java, Scala, và Python. —Jesse Anderson 3

In relation to previously existing roles, the data engineering field could be thought of as a superset of **business intelligence** and data warehousing that brings more elements from software engineering. This discipline also integrates specialization around the operation of so-called “big data” distributed systems, along with concepts around the extended Hadoop ecosystem, stream processing, and in computation at scale. —Maxime Beauchemin 4

Xét trong tương quan với các vai trò đã tồn tại trước đó, lĩnh vực kỹ thuật dữ liệu có thể được xem như một tập hợp cha của trí tuệ doanh nghiệp (BI) và kho dữ liệu (data warehousing), mang thêm nhiều yếu tố từ kỹ thuật phần mềm. Bộ môn này cũng tích hợp sự chuyên biệt hóa xoay quanh việc vận hành các hệ phân tán “big data” gọi là, cùng với các khái niệm quanh hệ sinh thái Hadoop mở rộng, xử lý luồng (stream processing), và tính toán ở quy mô lớn. —Maxime Beauchemin 4

Data engineering is all about the movement, manipulation, and management of data. —Lewis Gavin 5

Kỹ thuật dữ liệu xoay quanh việc di chuyển, thao tác, và quản lý dữ liệu. —Lewis Gavin 5

Wow! It’s entirely understandable if you’ve been confused about data engineering. That’s only a handful of definitions, and they contain an enormous range of opinions about the meaning of data

engineering .

Chà! Hoàn toàn dễ hiểu nếu bạn từng bối rối về kỹ thuật dữ liệu. Đó mới chỉ là một nhóm định nghĩa, và chúng chứa đựng một dải vô cùng rộng các quan điểm về ý nghĩa của kỹ thuật dữ liệu.

Data Engineering Defined — Định nghĩa Kỹ thuật Dữ liệu

When we unpack the common threads of how various people define data engineering, an obvious pattern emerges: a data engineer gets data, stores it, and prepares it for consumption by data scientists, analysts, and others. We define data engineering and data engineer as follows:

Khi ta bóc tách những sợi chỉ chung trong cách nhiều người khác nhau định nghĩa kỹ thuật dữ liệu, một khuôn mẫu hiển nhiên hiện ra: một kỹ sư dữ liệu lấy dữ liệu, lưu trữ nó, và chuẩn bị nó để được tiêu thụ bởi các nhà khoa học dữ liệu, nhà phân tích, và những người khác. Chúng tôi định nghĩa kỹ thuật dữ liệu và kỹ sư dữ liệu như sau:

Data engineering is the development, implementation, and maintenance of systems and processes that take in raw data and produce high-quality, consistent information that supports **downstream** use cases, such as analysis and **machine learning**. Data engineering is the intersection of security, data management, **DataOps**, **data architecture**, **orchestration**, and software engineering. A data engineer manages the **data engineering lifecycle**, beginning with getting data from source systems and ending with **serving** data for use cases, such as analysis or machine learning.

Kỹ thuật dữ liệu là việc phát triển, triển khai, và bảo trì các hệ thống và quy trình tiếp nhận dữ liệu thô và tạo ra thông tin chất lượng cao, nhất quán, hỗ trợ các trường hợp sử dụng ở hạ nguồn (downstream), chẳng hạn như phân tích và học máy (machine learning). Kỹ thuật dữ liệu là giao điểm của an ninh, quản lý dữ liệu, DataOps, kiến trúc dữ liệu, điều phối (orchestration), và kỹ thuật phần mềm. Một kỹ sư dữ liệu quản lý vòng đời kỹ thuật dữ liệu, bắt đầu từ việc lấy dữ liệu từ các hệ thống nguồn (source system) và kết thúc ở việc phục vụ (serving) dữ liệu cho các trường hợp sử dụng, chẳng hạn như phân tích hoặc học máy.

The Data Engineering Lifecycle — Vòng đời Kỹ thuật Dữ liệu

It is all too easy to fixate on technology and miss the bigger picture myopically. This book centers around a big idea called the data engineering lifecycle (Figure 1-1), which we believe gives data engineers the holistic context to view their role.

Thật quá dễ để chăm chăm vào công nghệ mà bỏ lỡ bức tranh lớn một cách thiển cận. Cuốn sách này xoay quanh một ý tưởng lớn gọi là vòng đời kỹ thuật dữ liệu (Hình 1-1), thứ mà chúng tôi tin rằng mang lại cho các kỹ sư dữ liệu bối cảnh toàn diện để nhìn nhận vai trò của họ.

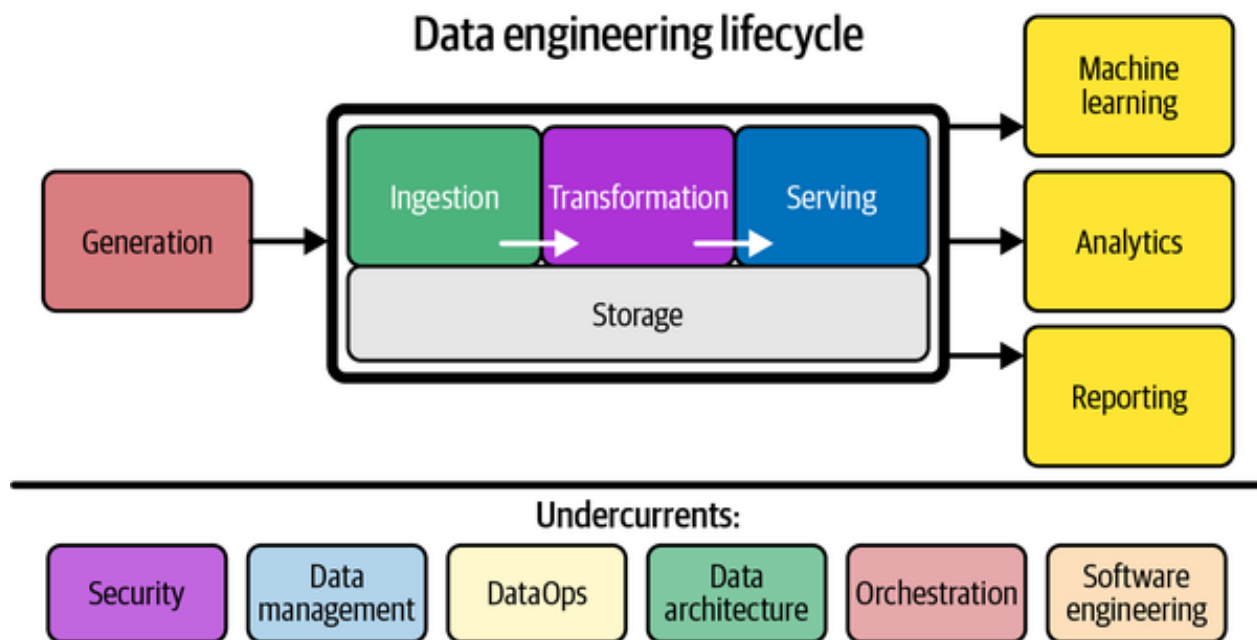


Figure 1-1. The data engineering lifecycle The data engineering lifecycle shifts the conversation away from technology and toward the data itself and the end goals that it must serve. The stages of the data engineering lifecycle are as follows:

Hình 1-1. Vòng đời kỹ thuật dữ liệu Vòng đời kỹ thuật dữ liệu chuyển dịch cuộc trò chuyện ra khỏi công nghệ và hướng về chính dữ liệu cùng các mục tiêu cuối mà nó phải phục vụ. Các giai đoạn của vòng đời kỹ thuật dữ liệu như sau:

Generation

Sinh dữ liệu (generation)

Storage

Lưu trữ

Ingestion

Thu nạp (ingestion)

Transformation

Biến đổi (transformation)

Serving

Phục vụ (serving)

The data engineering lifecycle also has a notion of **undercurrents**—critical ideas across the entire lifecycle. These include security, data management, DataOps, data architecture, orchestration, and software engineering. We cover the data engineering lifecycle and its undercurrents more extensively in Chapter 2 . Still, we introduce it here because it is essential to our definition of data engineering and the discussion that follows in this chapter. Now that you have a working definition of data engineering and an introduction to its lifecycle, let's take a step back and look at a bit of history.

Vòng đời kỹ thuật dữ liệu cũng có một khái niệm gọi là các dòng chảy ngầm (undercurrents)—những ý tưởng then chốt trải khắp toàn bộ vòng đời. Chúng bao

gồm an ninh, quản lý dữ liệu, DataOps, kiến trúc dữ liệu, điều phối, và kỹ thuật phần mềm. Chúng tôi trình bày vòng đời kỹ thuật dữ liệu và các dòng chảy ngầm của nó kỹ lưỡng hơn ở Chương 2. Tuy vậy, chúng tôi giới thiệu nó ở đây vì nó thiết yếu đối với định nghĩa kỹ thuật dữ liệu của chúng tôi và đối với phần thảo luận tiếp theo trong chương này. Giờ đây khi bạn đã có một định nghĩa khả dụng về kỹ thuật dữ liệu và một phần giới thiệu về vòng đời của nó, hãy lùi lại một bước và nhìn vào đôi chút lịch sử.

Evolution of the Data Engineer — Sự tiến hóa của Kỹ sư Dữ liệu

History doesn't repeat itself, but it rhymes. —A famous adage often attributed to Mark Twain
Understanding data engineering today and tomorrow requires a context of how the field evolved. This section is not a history lesson, but looking at the past is invaluable in understanding where we are today and where things are going. A common theme constantly reappears: what's old is new again.

Lịch sử không lặp lại chính nó, nhưng nó gieo vần. —Một câu châm ngôn nổi tiếng thường được gán cho Mark Twain Việc hiểu kỹ thuật dữ liệu của hôm nay và ngày mai đòi hỏi bối cảnh về cách lĩnh vực này đã tiến hóa. Phần này không phải một bài học lịch sử, nhưng nhìn vào quá khứ là vô giá để hiểu chúng ta đang ở đâu hôm nay và mọi thứ đang đi về đâu. Một chủ đề chung liên tục tái xuất hiện: cái cũ lại trở thành cái mới.

The early days: 1980 to 2000, from data warehousing to the web — Những ngày đầu: 1980 đến 2000, từ kho dữ liệu đến web

The birth of the data engineer arguably has its roots in data warehousing, dating as far back as the 1970s, with the business **data warehouse** taking shape in the 1980s and Bill **Inmon** officially coining the term data warehouse in 1990. After engineers at IBM developed the relational database and Structured Query Language (SQL), Oracle popularized the technology. As nascent data systems grew, businesses needed dedicated tools and data pipelines for reporting and business intelligence (BI). To help people correctly model their business logic in the data warehouse, Ralph **Kimball** and Inmon developed their respective eponymous data-modeling techniques and approaches, which are still widely used today. Data warehousing ushered in the first age of scalable analytics, with new **massively parallel processing** (MPP) databases that use multiple processors to crunch large amounts of data coming on the market and supporting unprecedented volumes of data. Roles such as BI engineer, ETL developer, and data warehouse engineer addressed the various needs of the data warehouse. Data warehouse and BI engineering were a precursor to today's data engineering and still play a central role in the discipline. The internet went mainstream around the mid-1990s, creating a whole new generation of web-first companies such as AOL, Yahoo, and Amazon. The dot-com boom spawned a ton of activity in web applications and the backend systems to support them—servers, databases, and storage. Much of the infrastructure was expensive, monolithic, and heavily licensed. The vendors selling these backend systems likely didn't foresee the sheer scale of the data that web applications would produce.

Sự ra đời của kỹ sư dữ liệu có thể nói là bắt nguồn từ kho dữ liệu (data warehousing), có niên đại xa tận thập niên 1970, với kho dữ liệu doanh nghiệp định hình trong thập niên 1980 và Bill Inmon chính thức đặt ra thuật ngữ kho dữ liệu (data warehouse) vào năm 1990. Sau khi các kỹ sư tại IBM phát triển cơ sở dữ liệu quan hệ và Ngôn ngữ Truy vấn Có cấu trúc (SQL), Oracle đã phổ biến hóa công nghệ này. Khi các hệ thống dữ liệu non trẻ lớn lên, các doanh nghiệp cần đến những công cụ chuyên dụng và các đường ống dữ liệu

(data pipeline) cho báo cáo và trí tuệ doanh nghiệp (BI). Để giúp người ta mô hình hóa đúng logic nghiệp vụ của họ trong kho dữ liệu, Ralph Kimball và Inmon đã phát triển các kỹ thuật và cách tiếp cận mô hình hóa dữ liệu mang tên chính họ, những thứ đến nay vẫn còn được dùng rộng rãi. Kho dữ liệu mở ra kỷ nguyên đầu tiên của phân tích có khả năng mở rộng, với các cơ sở dữ liệu xử lý song song quy mô lớn (MPP) mới sử dụng nhiều bộ xử lý để nghiên cứu khối lượng dữ liệu lớn ra mắt thị trường và hỗ trợ những khối lượng dữ liệu chưa từng có. Các vai trò như kỹ sư BI, lập trình viên ETL, và kỹ sư kho dữ liệu đáp ứng các nhu cầu đa dạng của kho dữ liệu. Kỹ thuật kho dữ liệu và BI là tiền thân của kỹ thuật dữ liệu ngày nay và vẫn đóng một vai trò trung tâm trong bộ môn này. Internet trở nên phổ biến đại trà vào khoảng giữa thập niên 1990, tạo ra cả một thế hệ mới các công ty ưu tiên web như AOL, Yahoo, và Amazon. Bùng nổ dot-com đã sản sinh vô số hoạt động trong các ứng dụng web và các hệ thống backend hỗ trợ chúng—máy chủ, cơ sở dữ liệu, và lưu trữ. Phần lớn hạ tầng khi đó đắt đỏ, dạng monolith, và bị cấp phép nặng nề. Các nhà cung cấp bán các hệ thống backend này có lẽ đã không lường trước được quy mô khổng lồ của dữ liệu mà các ứng dụng web sẽ tạo ra.

The early 2000s: The birth of contemporary data engineering — Đầu thập niên 2000: Sự ra đời của kỹ thuật dữ liệu đương đại

Fast-forward to the early 2000s, when the dot-com boom of the late '90s went bust, leaving behind a tiny cluster of survivors. Some of these companies, such as Yahoo, Google, and Amazon, would grow into powerhouse tech companies. Initially, these companies continued to rely on the traditional monolithic, relational databases and data warehouses of the 1990s, pushing these systems to the limit. As these systems buckled, updated approaches were needed to handle data growth. The new generation of the systems must be cost-effective, scalable, available, and reliable. Coinciding with the explosion of data, commodity hardware—such as servers, RAM, disks, and flash drives—also became cheap and ubiquitous. Several innovations allowed distributed computation and storage on massive computing clusters at a vast scale. These innovations started decentralizing and breaking apart traditionally monolithic services. The “big data” era had begun. The Oxford English Dictionary defines big data as “extremely large data sets that may be analyzed computationally to reveal patterns, trends, and associations, especially relating to human behavior and interactions.” Another famous and succinct description of big data is the three V’s of data: velocity, variety, and volume. In 2003, Google published a paper on the Google File System, and shortly after that, in 2004, a paper on MapReduce, an ultra-scalable data-processing paradigm. In truth, big data has earlier antecedents in MPP data warehouses and data management for experimental physics projects, but Google’s publications constituted a “big bang” for data technologies and the cultural roots of data engineering as we know it today. You’ll learn more about MPP systems and MapReduce in Chapters 3 and 8, respectively. The Google papers inspired engineers at Yahoo to develop and later open source Apache Hadoop in 2006. It’s hard to overstate the impact of Hadoop. Software engineers interested in large-scale data problems were drawn to the possibilities of this new open source technology ecosystem. As companies of all sizes and types saw their data grow into many terabytes and even petabytes, the era of the big data engineer was born. Around the same time, Amazon had to keep up with its own exploding data needs and created elastic computing environments (Amazon Elastic Compute Cloud, or EC2), infinitely scalable storage systems (Amazon Simple Storage Service, or S3), highly scalable NoSQL databases (Amazon DynamoDB), and many other core data building blocks. Amazon elected to offer these services for internal and external consumption through Amazon Web Services (AWS), becoming the first popular public cloud. AWS created an ultra-flexible pay-as-you-go resource marketplace by virtualizing and reselling vast pools of commodity hardware. Instead of purchasing hardware for a data center, developers could simply

rent compute and storage from AWS. As AWS became a highly profitable growth engine for Amazon, other public clouds would soon follow, such as Google Cloud, Microsoft Azure, and DigitalOcean. The public cloud is arguably one of the most significant innovations of the 21st century and spawned a revolution in the way software and data applications are developed and deployed. The early big data tools and public cloud laid the foundation for today's data ecosystem. The modern data landscape—and data engineering as we know it now—would not exist without these innovations.

Tua nhanh đến đầu thập niên 2000, khi bong bóng dot-com cuối thập niên '90 vỡ tan, để lại một cụm nhỏ những kẻ sống sót. Một số công ty trong đó, như Yahoo, Google, và Amazon, sẽ lớn lên thành những công ty công nghệ hùng mạnh. Ban đầu, các công ty này tiếp tục dựa vào những cơ sở dữ liệu quan hệ và kho dữ liệu dạng monolith truyền thống của thập niên 1990, đẩy các hệ thống này đến giới hạn. Khi các hệ thống này oằn xuống, cần đến những cách tiếp cận cập nhật để xử lý sự tăng trưởng dữ liệu. Thế hệ hệ thống mới phải tiết kiệm chi phí, có khả năng mở rộng, sẵn sàng, và đáng tin cậy. Trùng với sự bùng nổ của dữ liệu, phần cứng phổ thông—như máy chủ, RAM, đĩa, và ổ flash—cũng trở nên rẻ và phổ biến khắp nơi. Một số đổi mới đã cho phép tính toán và lưu trữ phân tán trên các cụm máy tính khổng lồ ở quy mô rộng lớn. Những đổi mới này bắt đầu phi tập trung hóa và phá vỡ các dịch vụ vốn dạng monolith truyền thống. Kỷ nguyên “big data” đã bắt đầu. Từ điển Tiếng Anh Oxford định nghĩa big data là “những tập dữ liệu cực kỳ lớn có thể được phân tích bằng tính toán để hé lộ các khuôn mẫu, xu hướng, và mối liên hệ, đặc biệt liên quan đến hành vi và tương tác của con người.” Một mô tả nổi tiếng và súc tích khác về big data là ba chữ V của dữ liệu: velocity (vận tốc), variety (đa dạng), và volume (khối lượng). Năm 2003, Google công bố một bài báo về Google File System, và ngay sau đó, vào năm 2004, một bài báo về MapReduce, một mô thức xử lý dữ liệu có khả năng mở rộng cực cao. Thực ra, big data có những tiền thân sớm hơn trong các kho dữ liệu MPP và quản lý dữ liệu cho các dự án vật lý thực nghiệm, nhưng các công bố của Google đã tạo nên một “vụ nổ lớn” cho các công nghệ dữ liệu và cội rễ văn hóa của kỹ thuật dữ liệu như chúng ta biết ngày nay. Bạn sẽ tìm hiểu thêm về các hệ thống MPP và MapReduce lần lượt ở Chương 3 và Chương 8. Các bài báo của Google đã truyền cảm hứng cho các kỹ sư tại Yahoo phát triển và sau đó mã nguồn mở hóa Apache Hadoop vào năm 2006. Khó có thể nói quá về tác động của 6 Hadoop. Các kỹ sư phần mềm quan tâm đến các bài toán dữ liệu quy mô lớn bị cuốn hút bởi những khả năng của hệ sinh thái công nghệ mã nguồn mở mới này. Khi các công ty thuộc mọi quy mô và loại hình chứng kiến dữ liệu của họ tăng lên đến nhiều terabyte và thậm chí petabyte, kỷ nguyên của kỹ sư big data đã ra đời. Cũng khoảng thời gian đó, Amazon phải bắt kịp nhu cầu dữ liệu đang bùng nổ của chính mình và đã tạo ra các môi trường tính toán co giãn (Amazon Elastic Compute Cloud, hay EC2), các hệ thống lưu trữ có khả năng mở rộng vô hạn (Amazon Simple Storage Service, hay S3), các cơ sở dữ liệu NoSQL có khả năng mở rộng cao (Amazon DynamoDB), và nhiều khối xây dựng dữ liệu cốt lõi khác. Amazon đã chọn 7 cung cấp các dịch vụ này cho việc tiêu thụ nội bộ và bên ngoài thông qua Amazon Web Services (AWS), trở thành đám mây công cộng phổ biến đầu tiên. AWS đã tạo ra một thị trường tài nguyên trả theo mức dùng cực kỳ linh hoạt bằng cách ảo hóa và bán lại những bể phần cứng phổ thông khổng lồ. Thay vì mua phần cứng cho một trung tâm dữ liệu, các nhà phát triển có thể đơn giản thuê tính toán và lưu trữ từ AWS. Khi AWS trở thành một cỗ máy tăng trưởng sinh lời cao cho Amazon, các đám mây công cộng khác sẽ sớm nối gót, như Google Cloud, Microsoft Azure, và DigitalOcean. Đám mây công cộng có thể nói là một trong những đổi mới quan trọng nhất của thế kỷ 21 và đã khơi mào một cuộc cách mạng trong cách các ứng dụng phần mềm và dữ liệu được phát triển và triển khai. Các công cụ big data thời kỳ đầu và đám mây công cộng đã đặt nền móng cho hệ sinh thái dữ liệu ngày nay. Bức tranh dữ liệu hiện đại—và kỹ thuật dữ liệu như chúng ta biết hiện giờ—sẽ không tồn tại nếu không có những đổi mới này.

The 2000s and 2010s: Big data engineering — Thập niên 2000 và 2010: Kỹ thuật big data

Open source big data tools in the Hadoop ecosystem rapidly matured and spread from Silicon Valley to tech-savvy companies worldwide. For the first time, any business had access to the same bleeding-edge data tools used by the top tech companies. Another revolution occurred with the transition from **batch** computing to **event streaming**, ushering in a new era of big “real-time” data. You’ll learn about batch and event streaming throughout this book. Engineers could choose the latest and greatest—Hadoop, Apache Pig, Apache Hive, Dremel, Apache HBase, Apache Storm, Apache Cassandra, Apache Spark, Presto, and numerous other new technologies that came on the scene. Traditional enterprise-oriented and GUI-based data tools suddenly felt outmoded, and code-first engineering was in vogue with the ascendance of MapReduce. We (the authors) were around during this time, and it felt like old dogmas died a sudden death upon the altar of big data. The explosion of data tools in the late 2000s and 2010s ushered in the big data engineer. To effectively use these tools and techniques—namely, the Hadoop ecosystem including Hadoop, YARN, Hadoop Distributed File System (HDFS), and MapReduce—big data engineers had to be proficient in software development and low-level infrastructure hacking, but with a shifted emphasis. Big data engineers typically maintained massive clusters of commodity hardware to deliver data at scale. While they might occasionally submit pull requests to Hadoop core code, they shifted their focus from core technology development to data delivery. Big data quickly became a victim of its own success. As a buzzword, big data gained popularity during the early 2000s through the mid-2010s. Big data captured the imagination of companies trying to make sense of the ever-growing volumes of data and the endless barrage of shameless marketing from companies selling big data tools and services. Because of the immense hype, it was common to see companies using big data tools for small data problems, sometimes standing up a Hadoop cluster to process just a few gigabytes. It seemed like everyone wanted in on the big data action. Dan Ariely tweeted, “Big data is like teenage sex: everyone talks about it, nobody really knows how to do it, everyone thinks everyone else is doing it, so everyone claims they are doing it.” Figure 1-2 shows a snapshot of Google Trends for the search term “big data” to get an idea of the rise and fall of big data.

Các công cụ big data mã nguồn mở trong hệ sinh thái Hadoop nhanh chóng trưởng thành và lan rộng từ Thung lũng Silicon đến các công ty am hiểu công nghệ trên khắp thế giới. Lần đầu tiên, bất kỳ doanh nghiệp nào cũng có quyền truy cập vào cùng những công cụ dữ liệu tiên tiến nhất mà các công ty công nghệ hàng đầu sử dụng. Một cuộc cách mạng khác xảy ra với sự chuyển đổi từ tính toán theo lô (batch) sang luồng sự kiện (event streaming), mở ra một kỷ nguyên mới của dữ liệu “thời gian thực” lớn. Bạn sẽ tìm hiểu về batch và event streaming xuyên suốt cuốn sách này. Các kỹ sư có thể chọn những thứ mới và tốt nhất—Hadoop, Apache Pig, Apache Hive, Dremel, Apache HBase, Apache Storm, Apache Cassandra, Apache Spark, Presto, và vô số công nghệ mới khác xuất hiện trên sân khấu. Các công cụ dữ liệu truyền thống hướng doanh nghiệp và dựa trên GUI bỗng cảm thấy lỗi thời, và kỹ thuật ưu tiên mã (code-first) trở nên thịnh hành cùng sự lên ngôi của MapReduce. Chúng tôi (các tác giả) đã có mặt trong giai đoạn này, và có cảm giác như những giáo điều cũ đã chết đột ngột trên bàn thờ của big data. Sự bùng nổ các công cụ dữ liệu cuối thập niên 2000 và thập niên 2010 đã khai sinh kỹ sư big data. Để sử dụng hiệu quả các công cụ và kỹ thuật này—cụ thể là hệ sinh thái Hadoop bao gồm Hadoop, YARN, Hadoop Distributed File System (HDFS), và MapReduce—các kỹ sư big data phải thành thạo phát triển phần mềm và máy mô hạ tầng cấp thấp, nhưng với trọng tâm đã dịch chuyển. Các kỹ sư big data thường duy trì những cụm khổng lồ gồm phần cứng phổ thông để cung cấp dữ liệu ở quy mô lớn. Mặc dù họ thỉnh thoảng có thể gửi pull request vào mã lõi Hadoop, họ đã chuyển trọng tâm từ phát triển công nghệ lõi sang phân phối dữ liệu. Big data nhanh chóng trở

thành nạn nhân của chính thành công của mình. Là một từ thời thượng, big data trở nên phổ biến trong giai đoạn đầu thập niên 2000 đến giữa thập niên 2010. Big data đã chiếm lĩnh trí tưởng tượng của các công ty đang cố gắng hiểu được những khối lượng dữ liệu không ngừng tăng lên và làn sóng tiếp thị trợ trên bất tận từ các công ty bán công cụ và dịch vụ big data. Vì sự thổi phồng cực lớn, ta thường thấy các công ty dùng công cụ big data cho những bài toán dữ liệu nhỏ, đôi khi dựng lên cả một cụm Hadoop chỉ để xử lý vài gigabyte. Đường như ai cũng muốn nhúng tay vào trào lưu big data. Dan Ariely đã tweet, “Big data giống như chuyện chần gối tuổi teen: ai cũng nói về nó, không ai thực sự biết làm thế nào, ai cũng nghĩ mọi người khác đang làm, nên ai cũng tuyên bố mình đang làm.” Hình 1-2 cho thấy một ảnh chụp Google Trends của từ khóa tìm kiếm “big data” để hình dung sự lên và xuống của big data.

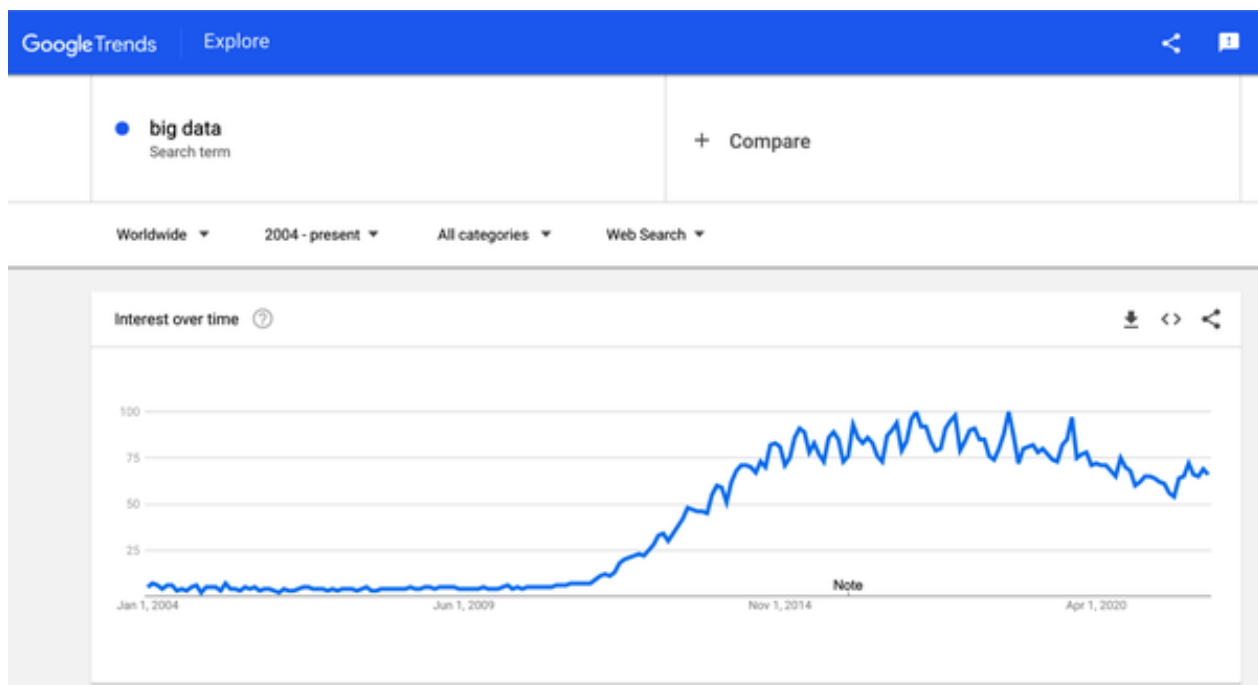


Figure 1-2. Google Trends for “big data” (March 2022) Despite the term’s popularity, big data has lost steam. What happened? One word: simplification. Despite the power and sophistication of open source big data tools, managing them was a lot of work and required constant attention. Often, companies employed entire teams of big data engineers, costing millions of dollars a year, to babysit these platforms. Big data engineers often spent excessive time maintaining complicated tooling and arguably not as much time delivering the business’s insights and value. Open source developers, clouds, and third parties started looking for ways to abstract, simplify, and make big data available without the high administrative overhead and cost of managing their clusters, and installing, configuring, and upgrading their open source code. The term big data is essentially a relic to describe a particular time and approach to handling large amounts of data. Today, data is moving faster than ever and growing ever larger, but big data processing has become so accessible that it no longer merits a separate term; every company aims to solve its data problems, regardless of actual data size. Big data engineers are now simply data engineers .

Hình 1-2. Google Trends cho “big data” (tháng 3 năm 2022) Bất chấp sự phổ biến của thuật ngữ này, big data đã mất đà. Chuyện gì đã xảy ra? Một từ thôi: đơn giản hóa. Bất chấp sức mạnh và sự tinh vi của các công cụ big data mã nguồn mở, việc quản lý chúng là rất nhiều công sức và đòi hỏi chú ý liên tục. Thường thì các công ty thuê cả những đội ngũ kỹ sư big data, tốn hàng triệu đô la mỗi năm, để trông coi các nền tảng này. Các kỹ sư

big data thường dành quá nhiều thời gian bảo trì các bộ công cụ phức tạp và có thể nói là không dành nhiều thời gian như vậy cho việc cung cấp những hiểu biết và giá trị cho doanh nghiệp. Các nhà phát triển mã nguồn mở, các đám mây, và các bên thứ ba bắt đầu tìm cách trừu tượng hóa, đơn giản hóa, và làm cho big data khả dụng mà không có chi phí quản trị và vận hành cao của việc quản lý các cụm của họ, cũng như cài đặt, cấu hình, và nâng cấp mã nguồn mở của họ. Thuật ngữ big data về cơ bản là một di tích để mô tả một thời điểm và cách tiếp cận cụ thể trong việc xử lý lượng dữ liệu lớn. Ngày nay, dữ liệu di chuyển nhanh hơn bao giờ hết và ngày càng lớn hơn, nhưng xử lý big data đã trở nên dễ tiếp cận đến mức không còn xứng đáng có một thuật ngữ riêng nữa; mọi công ty đều hướng đến giải quyết các bài toán dữ liệu của mình, bất kể kích thước dữ liệu thực tế. Các kỹ sư big data giờ đây đơn giản chỉ là các kỹ sư dữ liệu.

The 2020s: Engineering for the data lifecycle — Thập niên 2020: Kỹ thuật cho vòng đời dữ liệu

At the time of this writing, the data engineering role is evolving rapidly. We expect this evolution to continue at a rapid clip for the foreseeable future. Whereas data engineers historically tended to the low-level details of monolithic frameworks such as Hadoop, Spark, or Informatica, the trend is moving toward decentralized, modularized, managed, and highly abstracted tools. Indeed, data tools have proliferated at an astonishing rate (see Figure 1-3). Popular trends in the early 2020s include the **modern data stack** , representing a collection of off-the-shelf open source and third-party products assembled to make analysts' lives easier. At the same time, data sources and data formats are growing both in variety and size. Data engineering is increasingly a discipline of interoperation, and connecting various technologies like LEGO bricks, to serve ultimate business goals.

Tại thời điểm viết cuốn sách này, vai trò kỹ thuật dữ liệu đang tiến hóa nhanh chóng. Chúng tôi kỳ vọng sự tiến hóa này sẽ tiếp tục với nhịp độ nhanh trong tương lai gần. Trong khi trước đây các kỹ sư dữ liệu thường chăm lo các chi tiết cấp thấp của những framework monolith như Hadoop, Spark, hay Informatica, xu hướng đang dịch chuyển về phía các công cụ phi tập trung, mô-đun hóa, được quản lý, và trừu tượng hóa cao. Quả thực, các công cụ dữ liệu đã nở rộ với tốc độ đáng kinh ngạc (xem Hình 1-3). Các xu hướng phổ biến vào đầu thập niên 2020 bao gồm ngăn xếp dữ liệu hiện đại (modern data stack), đại diện cho một tập hợp các sản phẩm mã nguồn mở và bên thứ ba có sẵn được lắp ghép để làm cho cuộc sống của các nhà phân tích dễ dàng hơn. Đồng thời, các nguồn dữ liệu và định dạng dữ liệu đang tăng lên cả về sự đa dạng lẫn kích thước. Kỹ thuật dữ liệu ngày càng là một bộ môn của khả năng liên thông, và kết nối các công nghệ khác nhau như những viên gạch LEGO, để phục vụ các mục tiêu nghiệp vụ tối hậu.

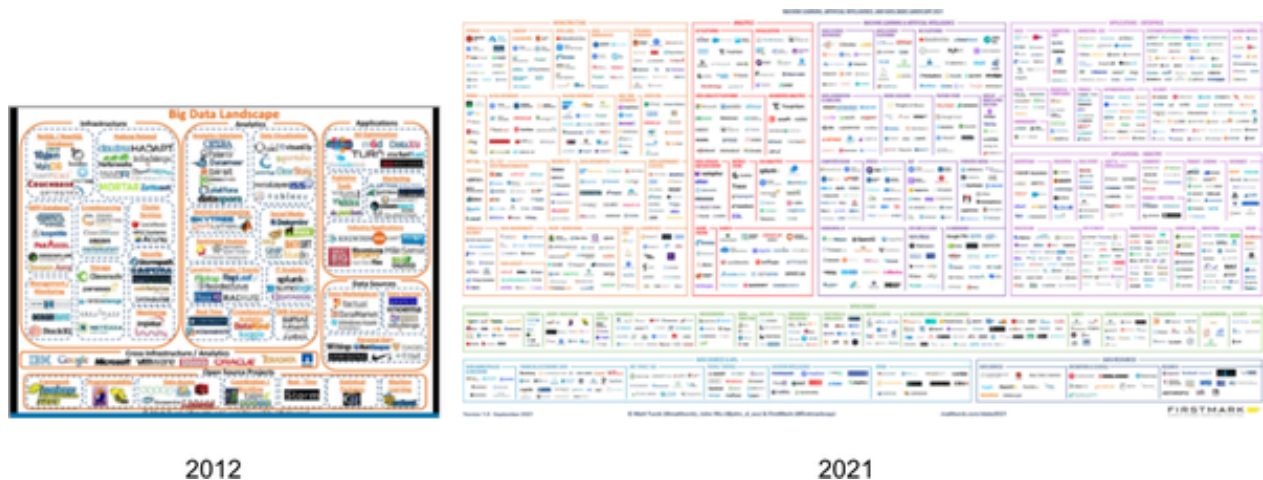


Figure 1-3. Matt Turck's Data Landscape in 2012 versus 2021 The data engineer we discuss in this book can be described more precisely as a data lifecycle engineer. With greater abstraction and simplification, a data lifecycle engineer is no longer encumbered by the gory details of yesterday's big data frameworks. While data engineers maintain skills in low-level data programming and use these as required, they increasingly find their role focused on things higher in the value chain: security, data management, DataOps, data architecture, orchestration, and general **data lifecycle management**. 8

Hình 1-3. Bức tranh Dữ liệu của Matt Turck năm 2012 so với 2021 Kỹ sư dữ liệu mà chúng tôi bàn đến trong cuốn sách này có thể được mô tả chính xác hơn là một kỹ sư vòng đời dữ liệu. Với sự trừu tượng hóa và đơn giản hóa lớn hơn, một kỹ sư vòng đời dữ liệu không còn bị vướng bận bởi những chi tiết rườm rà của các framework big data ngày xưa. Trong khi các kỹ sư dữ liệu vẫn duy trì kỹ năng lập trình dữ liệu cấp thấp và dùng chúng khi cần, họ ngày càng thấy vai trò của mình tập trung vào những thứ cao hơn trong chuỗi giá trị: an ninh, quản lý dữ liệu, DataOps, kiến trúc dữ liệu, điều phối, và quản lý vòng đời dữ liệu nói chung. 8

As tools and workflows simplify, we've seen a noticeable shift in the attitudes of data engineers. Instead of focusing on who has the “biggest data,” open source projects and services are increasingly concerned with managing and governing data, making it easier to use and discover, and improving its quality. Data engineers are now conversant in acronyms such as CCPA and GDPR ; as they engineer pipelines, they concern themselves 9 with **privacy**, anonymization, data garbage collection, and compliance with regulations. What's old is new again. While “enterprisey” stuff like data management (including **data quality** and governance) was common for large enterprises in the pre-big-data era, it wasn't widely adopted in smaller companies. Now that many of the challenging problems of yesterday's data systems are solved, neatly productized, and packaged, technologists and entrepreneurs have shifted focus back to the “enterprisey” stuff, but with an emphasis on decentralization and agility, that contrasts with the traditional enterprise command-and-control approach. We view the present as a golden age of data lifecycle management. Data engineers managing the data engineering lifecycle have better tools and techniques than ever before. We discuss the data engineering lifecycle and its undercurrents in greater detail in the next chapter.

Khi các công cụ và quy trình làm việc đơn giản hóa, chúng tôi đã thấy một sự dịch chuyển rõ rệt trong thái độ của các kỹ sư dữ liệu. Thay vì tập trung vào việc ai có “dữ liệu lớn nhất”, các dự án và dịch vụ mã nguồn mở ngày càng quan tâm đến việc quản lý và quản trị dữ liệu, làm cho nó dễ sử dụng và khám phá hơn, và cải thiện chất lượng của nó. Các kỹ sư dữ

liệu giờ đây thông thạo các từ viết tắt như CCPA và GDPR; khi họ thiết kế các đường ống, họ tự bận tâm 9 với quyền riêng tư, ẩn danh hóa, thu gom rác dữ liệu, và tuân thủ các quy định. Cái cũ lại trở thành cái mới. Trong khi những thứ “kiểu doanh nghiệp” như quản lý dữ liệu (bao gồm chất lượng và quản trị dữ liệu) là phổ biến đối với các doanh nghiệp lớn trong kỷ nguyên tiền-big-data, chúng lại không được áp dụng rộng rãi trong các công ty nhỏ hơn. Giờ đây khi nhiều bài toán hóc búa của các hệ thống dữ liệu ngày xưa đã được giải quyết, được sản phẩm hóa và đóng gói gọn gàng, các nhà công nghệ và doanh nhân đã chuyển trọng tâm trở lại những thứ “kiểu doanh nghiệp”, nhưng với sự nhấn mạnh vào phi tập trung hóa và linh hoạt, tương phản với cách tiếp cận chỉ-huy-và-kiểm-soát kiểu doanh nghiệp truyền thống. Chúng tôi xem hiện tại là một thời kỳ hoàng kim của quản lý vòng đời dữ liệu. Các kỹ sư dữ liệu quản lý vòng đời kỹ thuật dữ liệu có những công cụ và kỹ thuật tốt hơn bao giờ hết. Chúng tôi thảo luận vòng đời kỹ thuật dữ liệu và các dòng chảy ngầm của nó chi tiết hơn trong chương tiếp theo.

Data Engineering and Data Science — Kỹ thuật Dữ liệu và Khoa học Dữ liệu

Where does data engineering fit in with data science? There’s some debate, with some arguing data engineering is a subdiscipline of data science. We believe data engineering is separate from data science and analytics. They complement each other, but they are distinctly different. Data engineering sits **upstream** from data science (Figure 1-4), meaning data engineers provide the inputs used by data scientists (downstream from data engineering), who convert these inputs into something useful.

Kỹ thuật dữ liệu phù hợp ở đâu trong tương quan với khoa học dữ liệu? Có một số tranh luận, với một số người cho rằng kỹ thuật dữ liệu là một bộ môn con của khoa học dữ liệu. Chúng tôi tin rằng kỹ thuật dữ liệu tách biệt khỏi khoa học dữ liệu và phân tích. Chúng hỗ trợ cho nhau, nhưng chúng khác biệt rõ rệt. Kỹ thuật dữ liệu nằm ở thượng nguồn (upstream) so với khoa học dữ liệu (Hình 1-4), nghĩa là các kỹ sư dữ liệu cung cấp các đầu vào được các nhà khoa học dữ liệu sử dụng (ở hạ nguồn so với kỹ thuật dữ liệu), những người chuyển đổi các đầu vào này thành thứ gì đó hữu ích.

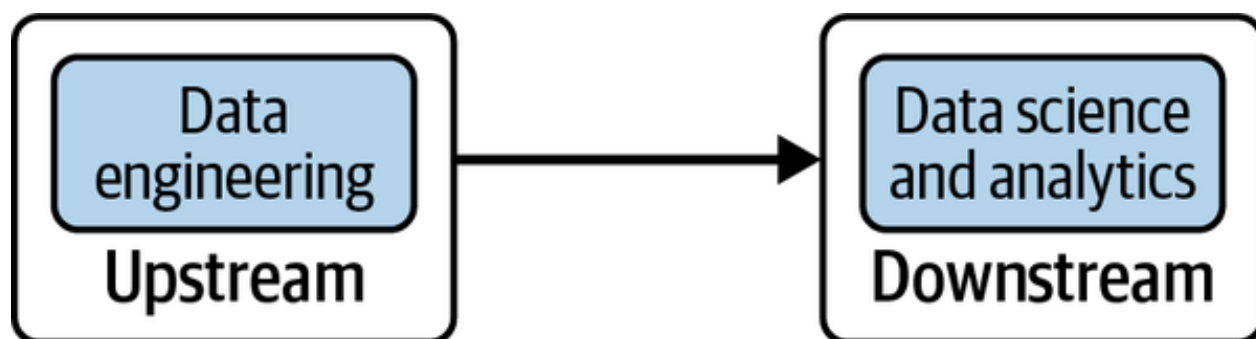


Figure 1-4. Data engineering sits upstream from data science Consider the Data Science Hierarchy of Needs (Figure 1-5). In 2017, Monica Rogati published this hierarchy in an article that showed where AI and machine learning (ML) sat in proximity to more “mundane” areas such as data movement/storage, collection, and infrastructure.

Hình 1-4. Kỹ thuật dữ liệu nằm ở thượng nguồn so với khoa học dữ liệu Hãy xét đến Tháp Nhu cầu Khoa học Dữ liệu (Hình 1-5). Năm 2017, Monica Rogati đã công bố tháp nhu cầu này trong một bài viết cho thấy AI và học máy (ML) nằm ở vị trí gần kề với các lĩnh vực

“tầm thường” hơn như di chuyển/lưu trữ dữ liệu, thu thập, và hạ tầng.

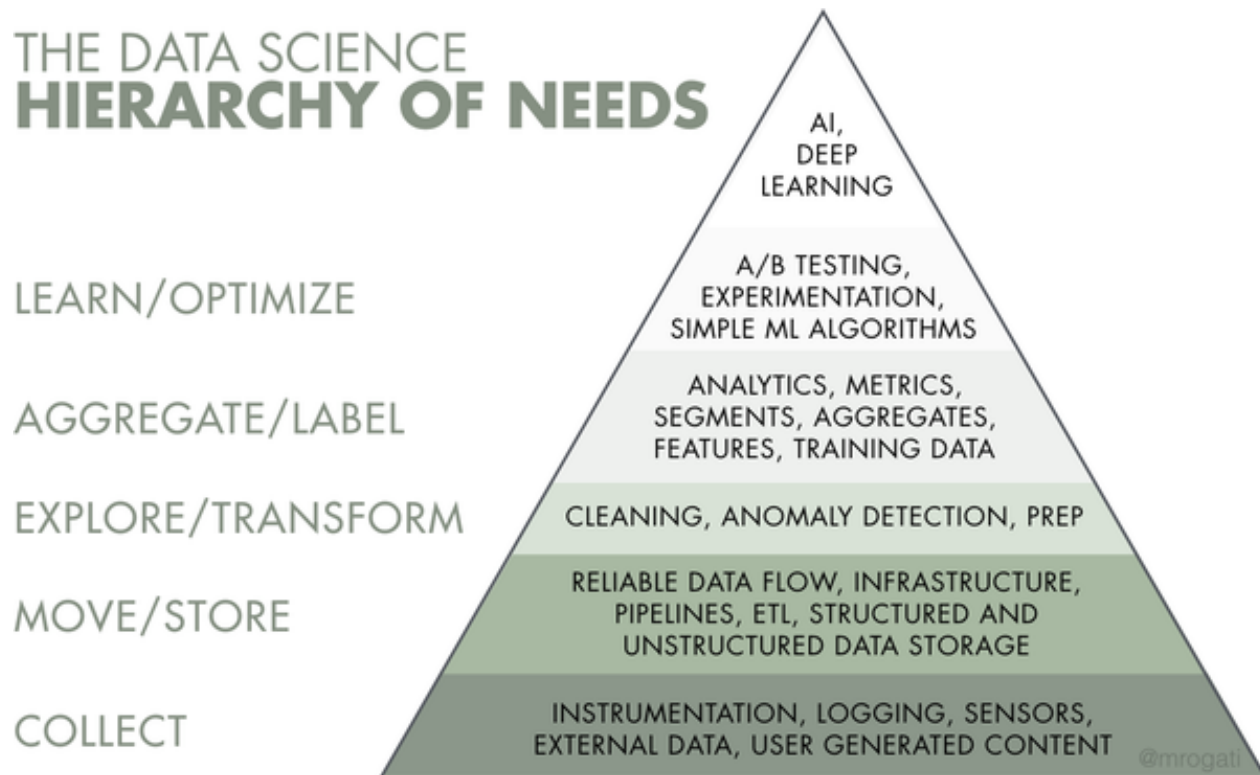


Figure 1-5. The Data Science Hierarchy of Needs Although many data scientists are eager to build and tune ML models, the reality is an estimated 70% to 80% of their time is spent toiling in the bottom three parts of the hierarchy—gathering data, cleaning data, processing data—and only a tiny slice of their time on analysis and ML. Rogati argues that companies need to build a solid data foundation (the bottom three levels of the hierarchy) before tackling areas such as AI and ML. Data scientists aren’t typically trained to engineer production-grade data systems, and they end up doing this work haphazardly because they lack the support and resources of a data engineer. In an ideal world, data scientists should spend more than 90% of their time focused on the top layers of the pyramid: analytics, experimentation, and ML. When data engineers focus on these bottom parts of the hierarchy, they build a solid foundation for data scientists to succeed. With data science driving advanced analytics and ML, data engineering straddles the divide between getting data and getting value from data (see Figure 1-6). We believe data engineering is of equal importance and visibility to data science, with data engineers playing a vital role in making data science successful in production.

Hình 1-5. Tháp Nhu cầu Khoa học Dữ liệu Mặc dù nhiều nhà khoa học dữ liệu háo hức xây dựng và tinh chỉnh các mô hình ML, thực tế là ước tính 70% đến 80% thời gian của họ được dành để vật lộn ở ba phần dưới cùng của tháp—thu thập dữ liệu, làm sạch dữ liệu, xử lý dữ liệu—và chỉ một lát nhỏ thời gian của họ dành cho phân tích và ML. Rogati lập luận rằng các công ty cần xây dựng một nền tảng dữ liệu vững chắc (ba tầng dưới cùng của tháp) trước khi giải quyết các lĩnh vực như AI và ML. Các nhà khoa học dữ liệu thường không được đào tạo để thiết kế các hệ thống dữ liệu cấp độ vận hành, và họ rất cuộc làm công việc này một cách chấp vá vì thiếu sự hỗ trợ và nguồn lực của một kỹ sư dữ liệu. Trong một thế giới lý tưởng, các nhà khoa học dữ liệu nên dành hơn 90% thời gian của họ tập trung vào các tầng trên cùng của kim tự tháp: phân tích, thử nghiệm, và ML. Khi các kỹ sư dữ liệu tập trung vào những phần dưới cùng này của tháp, họ xây dựng một nền tảng vững chắc để các nhà khoa học dữ liệu thành công. Với khoa học dữ liệu thúc đẩy

phân tích nâng cao và ML, kỹ thuật dữ liệu bắc qua ranh giới giữa việc lấy dữ liệu và việc thu được giá trị từ dữ liệu (xem Hình 1-6). Chúng tôi tin rằng kỹ thuật dữ liệu có tầm quan trọng và độ hiện diện ngang bằng với khoa học dữ liệu, với các kỹ sư dữ liệu đóng một vai trò sống còn trong việc làm cho khoa học dữ liệu thành công trong môi trường vận hành.



Figure 1-6. A data engineer gets data and provides value from the data

Hình 1-6. Một kỹ sư dữ liệu lấy dữ liệu và cung cấp giá trị từ dữ liệu

Data Engineering Skills and Activities — Kỹ năng và Hoạt động của Kỹ thuật Dữ liệu

The skill set of a data engineer encompasses the “undercurrents” of data engineering: security, data management, DataOps, data architecture, and software engineering. This skill set requires an understanding of how to evaluate data tools and how they fit together across the data engineering lifecycle. It’s also critical to know how data is produced in source systems and how analysts and data scientists will consume and create value after processing and curating data. Finally, a data engineer juggles a lot of complex moving parts and must constantly optimize along the axes of cost, agility, **scalability**, simplicity, reuse, and interoperability (Figure 1-7). We cover these topics in more detail in upcoming chapters.

Tập kỹ năng của một kỹ sư dữ liệu bao trùm các “dòng chảy ngầm” của kỹ thuật dữ liệu: an ninh, quản lý dữ liệu, DataOps, kiến trúc dữ liệu, và kỹ thuật phần mềm. Tập kỹ năng này đòi hỏi sự hiểu biết về cách đánh giá các công cụ dữ liệu và cách chúng khớp với nhau xuyên suốt vòng đời kỹ thuật dữ liệu. Cũng rất quan trọng là biết dữ liệu được tạo ra như thế nào trong các hệ thống nguồn và các nhà phân tích cùng các nhà khoa học dữ liệu sẽ tiêu thụ và tạo ra giá trị như thế nào sau khi xử lý và biên tập dữ liệu. Cuối cùng, một kỹ sư dữ liệu phải tung hứng rất nhiều bộ phận chuyển động phức tạp và phải liên tục tối ưu dọc theo các trục chi phí, sự linh hoạt, khả năng mở rộng, sự đơn giản, khả năng tái sử dụng, và khả năng liên thông (Hình 1-7). Chúng tôi trình bày các chủ đề này chi tiết hơn ở các chương sắp tới.



Figure 1-7. The balancing act of data engineering As we discussed, in the recent past, a data engineer was expected to know and understand how to use a small handful of powerful and monolithic technologies (Hadoop, Spark, Teradata, Hive, and many others) to create a data solution. Utilizing these technologies often requires a sophisticated understanding of software engineering, networking, distributed computing, storage, or other low-level details. Their work would be devoted to cluster administration and maintenance, managing overhead, and writing pipeline and transformation jobs, among other tasks. Nowadays, the data-tooling landscape is dramatically less complicated to manage and deploy. Modern data tools considerably abstract and simplify workflows. As a result, data engineers are now focused on balancing the simplest and most cost-effective, best-of-breed services that deliver value to the business. The data engineer is also expected to create agile data architectures that evolve as new trends emerge. What are some things a data engineer does

not do? A data engineer typically does not directly build ML models, create reports or dashboards, perform data analysis, build key performance indicators (KPIs), or develop software applications. A data engineer should have a good functioning understanding of these areas to serve stakeholders best.

Hình 1-7. Hành động cân bằng của kỹ thuật dữ liệu Như chúng ta đã thảo luận, trong quá khứ gần đây, một kỹ sư dữ liệu được kỳ vọng phải biết và hiểu cách sử dụng một nhóm nhỏ các công nghệ mạnh mẽ và dạng monolith (Hadoop, Spark, Teradata, Hive, và nhiều thứ khác) để tạo ra một giải pháp dữ liệu. Việc sử dụng các công nghệ này thường đòi hỏi sự hiểu biết tinh vi về kỹ thuật phần mềm, mạng, tính toán phân tán, lưu trữ, hoặc các chi tiết cấp thấp khác. Công việc của họ sẽ dành cho quản trị và bảo trì cụm, quản lý chi phí phụ trợ, và viết các tác vụ đường ống và biến đổi, cùng nhiều nhiệm vụ khác. Ngày nay, bức tranh công cụ dữ liệu ít phức tạp hơn nhiều để quản lý và triển khai. Các công cụ dữ liệu hiện đại trừu tượng hóa và đơn giản hóa đáng kể các quy trình làm việc. Kết quả là, các kỹ sư dữ liệu giờ đây tập trung vào việc cân bằng những dịch vụ đơn giản nhất và tiết kiệm chi phí nhất, tốt nhất trong loại của chúng, mang lại giá trị cho doanh nghiệp. Kỹ sư dữ liệu cũng được kỳ vọng tạo ra các kiến trúc dữ liệu linh hoạt tiến hóa khi các xu hướng mới xuất hiện. Một kỹ sư dữ liệu không làm những việc gì? Một kỹ sư dữ liệu thường không trực tiếp xây dựng các mô hình ML, tạo báo cáo hay bảng điều khiển, thực hiện phân tích dữ liệu, xây dựng các chỉ số hiệu suất chính (KPI), hay phát triển các ứng dụng phần mềm. Một kỹ sư dữ liệu nên có sự hiểu biết vận hành tốt về các lĩnh vực này để phục vụ các bên liên quan một cách tốt nhất.

Data Maturity and the Data Engineer — Độ trưởng thành Dữ liệu và Kỹ sư Dữ liệu

The level of data engineering complexity within a company depends a great deal on the company's data maturity. This significantly impacts a data engineer's day-to-day job responsibilities and career progression. What is data maturity, exactly? Data maturity is the progression toward higher data utilization, capabilities, and integration across the organization, but data maturity does not simply depend on the age or revenue of a company. An early-stage startup can have greater data maturity than a 100-year-old company with annual revenues in the billions. What matters is the way data is leveraged as a competitive advantage. Data maturity models have many versions, such as Data Management Maturity (DMM) and others, and it's hard to pick one that is both simple and useful for data engineering. So, we'll create our own simplified data maturity model. Our data maturity model (Figure 1-8) has three stages: starting with data, **scaling** with data, and leading with data. Let's look at each of these stages and at what a data engineer typically does at each stage.

Mức độ phức tạp của kỹ thuật dữ liệu trong một công ty phụ thuộc rất nhiều vào độ trưởng thành dữ liệu của công ty đó. Điều này tác động đáng kể đến các trách nhiệm công việc hằng ngày và lộ trình sự nghiệp của một kỹ sư dữ liệu. Độ trưởng thành dữ liệu chính xác là gì? Độ trưởng thành dữ liệu là sự tiến triển hướng tới mức độ sử dụng, năng lực, và tích hợp dữ liệu cao hơn trong toàn tổ chức, nhưng độ trưởng thành dữ liệu không đơn thuần phụ thuộc vào tuổi đời hay doanh thu của một công ty. Một startup giai đoạn đầu có thể có độ trưởng thành dữ liệu cao hơn một công ty trăm tuổi với doanh thu hàng năm hàng tỷ đô la. Điều quan trọng là cách dữ liệu được tận dụng như một lợi thế cạnh tranh. Các mô hình độ trưởng thành dữ liệu có nhiều phiên bản, như Data Management Maturity (DMM) và những mô hình khác, và thật khó để chọn một mô hình vừa đơn giản vừa hữu ích cho kỹ thuật dữ liệu. Vì vậy, chúng tôi sẽ tạo ra mô hình độ trưởng thành dữ liệu đơn giản hóa của riêng mình. Mô hình độ trưởng thành dữ liệu của chúng tôi (Hình 1-8) có ba giai đoạn:

bắt đầu với dữ liệu, mở rộng quy mô với dữ liệu, và dẫn dắt bằng dữ liệu. Hãy xem xét từng giai đoạn này và xem một kỹ sư dữ liệu thường làm gì ở mỗi giai đoạn.

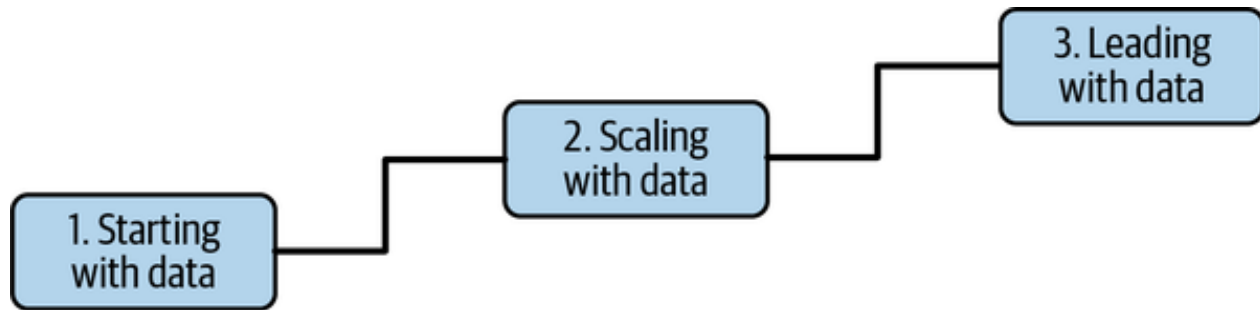


Figure 1-8. Our simplified data maturity model for a company

Hình 1-8. Mô hình độ trưởng thành dữ liệu đơn giản hóa của chúng tôi cho một công ty

Stage 1: Starting with data — Giai đoạn 1: Bắt đầu với dữ liệu

A company getting started with data is, by definition, in the very early stages of its data maturity. The company may have fuzzy, **loosely** defined goals or no goals. Data architecture and infrastructure are in the very early stages of planning and development. Adoption and utilization are likely low or nonexistent. The data team is small, often with a headcount in the single digits. At this stage, a data engineer is usually a generalist and will typically play several other roles, such as data scientist or software engineer. A data engineer's goal is to move fast, get traction, and add value. The practicalities of getting value from data are typically poorly understood, but the desire exists. Reports or analyses lack formal structure, and most requests for data are ad hoc. While it's tempting to jump headfirst into ML at this stage, we don't recommend it. We've seen countless data teams get stuck and fall short when they try to jump to ML without building a solid data foundation. That's not to say you can't get wins from ML at this stage—it is rare but possible. Without a solid data foundation, you likely won't have the data to train reliable ML models nor the means to deploy these models to production in a scalable and repeatable way. We half-jokingly call ourselves “recovering data scientists”, mainly from personal experience with being involved in premature data science projects without adequate data maturity or data engineering support. A data engineer should focus on the following in organizations getting started with data:

Một công ty mới bắt đầu với dữ liệu, theo định nghĩa, đang ở những giai đoạn rất sớm của độ trưởng thành dữ liệu. Công ty có thể có những mục tiêu mơ hồ, được định nghĩa lỏng lẻo hoặc không có mục tiêu nào. Kiến trúc và hạ tầng dữ liệu đang ở những giai đoạn rất sớm của việc lập kế hoạch và phát triển. Mức độ áp dụng và sử dụng có thể thấp hoặc không tồn tại. Đội ngũ dữ liệu nhỏ, thường với quân số ở hàng đơn vị. Ở giai đoạn này, một kỹ sư dữ liệu thường là người đa năng và sẽ thường đảm nhận nhiều vai trò khác, chẳng hạn như nhà khoa học dữ liệu hoặc kỹ sư phần mềm. Mục tiêu của một kỹ sư dữ liệu là di chuyển nhanh, tạo được đà, và tạo thêm giá trị. Các khía cạnh thực tiễn của việc thu được giá trị từ dữ liệu thường được hiểu kém, nhưng khao khát thì có. Báo cáo hay phân tích thiếu cấu trúc chính thức, và hầu hết các yêu cầu về dữ liệu đều là tùy hứng (ad hoc). Mặc dù thật hấp dẫn để lao đầu vào ML ở giai đoạn này, chúng tôi không khuyến nghị điều đó. Chúng tôi đã thấy vô số đội ngũ dữ liệu mắc kẹt và thất bại khi họ cố nhảy vào ML mà không xây dựng một nền tảng dữ liệu vững chắc. Điều đó không có nghĩa là bạn không thể gặt hái thắng lợi từ ML ở giai đoạn này—điều đó hiếm nhưng có thể. Không có một nền tảng dữ liệu vững chắc, bạn có lẽ sẽ không có dữ liệu để huấn luyện các mô hình ML

đáng tin cậy, cũng không có phương tiện để triển khai các mô hình này vào môi trường vận hành theo cách có khả năng mở rộng và lặp lại được. Chúng tôi nửa đùa nửa thật tự gọi mình là “những nhà khoa học dữ liệu đang phục hồi”, chủ yếu từ kinh nghiệm cá nhân về việc tham gia vào các dự án khoa học dữ liệu non yếu mà không có độ trưởng thành dữ liệu hay sự hỗ trợ kỹ thuật dữ liệu đầy đủ. Một kỹ sư dữ liệu nên tập trung vào những điều sau trong các tổ chức mới bắt đầu với dữ liệu:

Get buy-in from key stakeholders, including executive management. Ideally, the data engineer should have a sponsor for critical initiatives to design and build a data architecture to support the company's goals.

Tranh thủ sự ủng hộ từ các bên liên quan chủ chốt, bao gồm ban điều hành cấp cao. Lý tưởng nhất, kỹ sư dữ liệu nên có một nhà bảo trợ cho các sáng kiến then chốt để thiết kế và xây dựng một kiến trúc dữ liệu hỗ trợ các mục tiêu của công ty.

Define the right data architecture (usually solo, since a data architect likely isn't available). This means determining business goals and the competitive advantage you're aiming to achieve with your data initiative. Work toward a data architecture that supports these goals. See Chapter 3 for our advice on “good” data architecture.

Định nghĩa kiến trúc dữ liệu đúng đắn (thường là một mình, vì một kiến trúc sư dữ liệu có lẽ không sẵn có). Điều này có nghĩa là xác định các mục tiêu nghiệp vụ và lợi thế cạnh tranh mà bạn nhắm tới đạt được với sáng kiến dữ liệu của mình. Hãy làm việc hướng tới một kiến trúc dữ liệu hỗ trợ các mục tiêu này. Xem Chương 3 để biết lời khuyên của chúng tôi về kiến trúc dữ liệu “tốt”.

Identify and audit data that will support key initiatives and operate within the data architecture you designed.

Xác định và kiểm toán dữ liệu sẽ hỗ trợ các sáng kiến chủ chốt và vận hành trong khuôn khổ kiến trúc dữ liệu mà bạn đã thiết kế.

Build a solid data foundation for future data analysts and data scientists to generate reports and models that provide competitive value. In the meantime, you may also have to generate these reports and models until this team is hired.

Xây dựng một nền tảng dữ liệu vững chắc để các nhà phân tích dữ liệu và nhà khoa học dữ liệu tương lai tạo ra các báo cáo và mô hình mang lại giá trị cạnh tranh. Trong thời gian chờ đợi, bạn cũng có thể phải tự tạo ra các báo cáo và mô hình này cho đến khi đội ngũ này được tuyển dụng.

This is a delicate stage with lots of pitfalls. Here are some tips for this stage:

Đây là một giai đoạn tế nhị với nhiều cạm bẫy. Đây là một số mẹo cho giai đoạn này:

Organizational willpower may wane if a lot of visible successes don't occur with data. Getting quick wins will establish the importance of data within the organization. Just keep in mind that quick wins will likely create technical debt. Have a plan to reduce this debt, as it will otherwise add friction for future delivery.

Ý chí của tổ chức có thể suy giảm nếu không có nhiều thành công hữu hình xảy ra với dữ liệu. Việc giành được những thắng lợi nhanh sẽ thiết lập tầm quan trọng của dữ liệu trong tổ chức. Chỉ cần lưu ý rằng những thắng lợi nhanh có khả năng sẽ tạo ra nợ kỹ thuật. Hãy có một kế hoạch để giảm khoản nợ này, vì nếu không nó sẽ tạo thêm ma sát cho việc phân phối trong tương lai.

Get out and talk to people, and avoid working in silos. We often see the data team working in a bubble, not communicating with people outside their departments and getting perspectives and feedback from business stakeholders. The danger is you'll spend a lot of time working on things of little use to people.

Hãy ra ngoài và nói chuyện với mọi người, và tránh làm việc trong các ốc đảo. Chúng tôi thường thấy đội ngũ dữ liệu làm việc trong một cái bong bóng, không giao tiếp với người ngoài bộ phận của họ và không thu được góc nhìn cũng như phản hồi từ các bên liên quan nghiệp vụ. Mối nguy là bạn sẽ dành rất nhiều thời gian làm những thứ ít hữu ích với người khác.

Avoid undifferentiated heavy lifting. Don't box yourself in with unnecessary technical complexity. Use off-the-shelf, turnkey solutions wherever possible.

Tránh việc nặng nhọc không tạo khác biệt. Đừng tự nhốt mình vào sự phức tạp kỹ thuật không cần thiết. Hãy dùng các giải pháp có sẵn, dùng-được-ngay bất cứ khi nào có thể.

Build custom solutions and code only where this creates a competitive advantage.

Chỉ xây dựng các giải pháp và mã tùy chỉnh ở những nơi điều này tạo ra một lợi thế cạnh tranh.

Stage 2: Scaling with data — Giai đoạn 2: Mở rộng quy mô với dữ liệu

At this point, a company has moved away from ad hoc data requests and has formal data practices. Now the challenge is creating scalable data architectures and planning for a future where the company is genuinely data-driven. Data engineering roles move from generalists to specialists, with people focusing on particular aspects of the data engineering lifecycle. In organizations that are in stage 2 of data maturity, a data engineer's goals are to do the following:

Tại thời điểm này, một công ty đã rời xa các yêu cầu dữ liệu tùy hứng và có các thực hành dữ liệu chính thức. Giờ đây thách thức là tạo ra các kiến trúc dữ liệu có khả năng mở rộng và lập kế hoạch cho một tương lai mà công ty thực sự được dẫn dắt bởi dữ liệu. Các vai trò kỹ thuật dữ liệu chuyển từ người đa năng sang chuyên gia, với những người tập trung vào các khía cạnh cụ thể của vòng đời kỹ thuật dữ liệu. Trong các tổ chức đang ở giai đoạn 2 của độ trưởng thành dữ liệu, các mục tiêu của một kỹ sư dữ liệu là làm những điều sau:

Establish formal data practices

Thiết lập các thực hành dữ liệu chính thức

Create scalable and robust data architectures

Tạo ra các kiến trúc dữ liệu có khả năng mở rộng và bền bỉ

Adopt DevOps and DataOps practices

Áp dụng các thực hành DevOps và DataOps

Build systems that support ML

Xây dựng các hệ thống hỗ trợ ML

Continue to avoid undifferentiated heavy lifting and customize only when a competitive advantage results

Tiếp tục tránh việc nặng nhọc không tạo khác biệt và chỉ tùy chỉnh khi điều đó mang lại một lợi thế cạnh tranh

We return to each of these goals later in the book. Issues to watch out for include the following:

Chúng tôi sẽ quay lại từng mục tiêu này sau trong cuốn sách. Các vấn đề cần để mắt bao gồm những điều sau:

As we grow more sophisticated with data, there's a temptation to adopt bleeding-edge technologies based on social proof from Silicon Valley companies. This is rarely a good use of your time and energy. Any technology decisions should be driven by the value they'll deliver to your customers.

Khi chúng ta trở nên tinh vi hơn với dữ liệu, có một cám dỗ là áp dụng các công nghệ tiên tiến nhất dựa trên bằng chứng xã hội từ các công ty Thung lũng Silicon. Đây hiếm khi là cách dùng thời gian và năng lượng của bạn một cách tốt đẹp. Bất kỳ quyết định công nghệ nào cũng nên được dẫn dắt bởi giá trị mà chúng sẽ mang lại cho khách hàng của bạn.

The main bottleneck for scaling is not cluster nodes, storage, or technology but the data engineering team. Focus on solutions that are simple to deploy and manage to expand your team's **throughput**.

Nút thắt cổ chai chính cho việc mở rộng quy mô không phải là các nút cụm, lưu trữ, hay công nghệ mà là chính đội ngũ kỹ thuật dữ liệu. Hãy tập trung vào các giải pháp đơn giản để triển khai và quản lý nhằm mở rộng thông lượng của đội ngũ bạn.

You'll be tempted to frame yourself as a technologist, a data genius who can deliver magical products. Shift your focus instead to pragmatic leadership and begin transitioning to the next maturity stage; communicate with other teams about the practical utility of data. Teach the organization how to consume and leverage data.

Bạn sẽ bị cám dỗ để định hình bản thân như một nhà công nghệ, một thiên tài dữ liệu có thể tạo ra những sản phẩm kỳ diệu. Thay vào đó, hãy chuyển trọng tâm sang sự lãnh đạo thực dụng và bắt đầu chuyển tiếp sang giai đoạn trưởng thành tiếp theo; hãy giao tiếp với các đội ngũ khác về tính hữu dụng thực tế của dữ liệu. Hãy dạy cho tổ chức cách tiêu thụ và tận dụng dữ liệu.

Stage 3: Leading with data — Giai đoạn 3: Dẫn dắt bằng dữ liệu

At this stage, the company is data-driven. The automated pipelines and systems created by data engineers allow people within the company to do self-service analytics and ML. Introducing new data sources is seamless, and tangible value is derived. Data engineers implement proper controls and practices to ensure that data is always available to the people and systems. Data engineering roles continue to specialize more deeply than in stage 2. In organizations in stage 3 of data maturity, a data engineer will continue building on prior stages, plus they will do the following:

Ở giai đoạn này, công ty được dẫn dắt bởi dữ liệu. Các đường ống và hệ thống tự động hóa được tạo ra bởi các kỹ sư dữ liệu cho phép mọi người trong công ty thực hiện phân tích tự phục vụ và ML. Việc đưa vào các nguồn dữ liệu mới diễn ra liền mạch, và giá trị hữu hình được thu về. Các kỹ sư dữ liệu triển khai các biện pháp kiểm soát và thực hành phù hợp để đảm bảo dữ liệu luôn sẵn sàng cho con người và hệ thống. Các vai trò kỹ thuật dữ liệu tiếp tục chuyên biệt hóa sâu hơn so với giai đoạn 2. Trong các tổ chức ở giai đoạn 3 của độ trưởng thành dữ liệu, một kỹ sư dữ liệu sẽ tiếp tục xây dựng dựa trên các giai đoạn trước, cộng thêm họ sẽ làm những điều sau:

Create automation for the seamless introduction and usage of new data

Tạo ra sự tự động hóa cho việc đưa vào và sử dụng dữ liệu mới một cách liền mạch

Focus on building custom tools and systems that leverage data as a competitive advantage

Tập trung vào việc xây dựng các công cụ và hệ thống tùy chỉnh tận dụng dữ liệu như một lợi thế cạnh tranh

Focus on the “enterprisey” aspects of data, such as data management (including **data governance** and quality) and DataOps

Tập trung vào các khía cạnh “kiểu doanh nghiệp” của dữ liệu, chẳng hạn như quản lý dữ liệu (bao gồm quản trị và chất lượng dữ liệu) và DataOps

Deploy tools that expose and disseminate data throughout the organization, including data catalogs, **data lineage** tools, and **metadata** management systems

Triển khai các công cụ phơi bày và lan tỏa dữ liệu khắp tổ chức, bao gồm danh mục dữ liệu (data catalog), công cụ truy vết nguồn gốc dữ liệu (data lineage), và các hệ thống quản lý siêu dữ liệu (metadata)

Collaborate efficiently with software engineers, ML engineers, analysts, and others

Cộng tác hiệu quả với các kỹ sư phần mềm, kỹ sư ML, nhà phân tích, và những người khác

Create a community and environment where people can collaborate and speak openly, no matter their role or position

Tạo ra một cộng đồng và môi trường nơi mọi người có thể cộng tác và nói chuyện cởi mở, bất kể vai trò hay vị trí của họ

Issues to watch out for include the following:

Các vấn đề cần để mắt bao gồm những điều sau:

At this stage, complacency is a significant danger. Once organizations reach stage 3, they must constantly focus on maintenance and improvement or risk falling back to a lower stage.

Ở giai đoạn này, sự tự mãn là một mối nguy đáng kể. Một khi các tổ chức đạt đến giai đoạn 3, họ phải liên tục tập trung vào bảo trì và cải tiến nếu không sẽ có nguy cơ tụt lại về một giai đoạn thấp hơn.

Technology distractions are a more significant danger here than in the other stages. There’s a temptation to pursue expensive hobby projects that don’t deliver value to the business. Utilize custom-built technology only where it provides a competitive advantage.

Sự xao nhãng bởi công nghệ là một mối nguy đáng kể hơn ở đây so với các giai đoạn khác. Có một cám dỗ theo đuổi những dự án sở thích tốn kém mà không mang lại giá trị cho doanh nghiệp. Hãy tận dụng công nghệ tự xây dựng chỉ ở những nơi nó mang lại một lợi thế cạnh tranh.

The Background and Skills of a Data Engineer — Nền tảng và Kỹ năng của một Kỹ sư Dữ liệu

Data engineering is a fast-growing field, and a lot of questions remain about how to become a data engineer. Because data engineering is a relatively new discipline, little formal training is available to enter the field. Universities don’t have a standard data engineering path. Although a handful of data engineering boot camps and online tutorials cover random topics, a common curriculum

for the subject doesn't yet exist. People entering data engineering arrive with varying backgrounds in education, career, and skill set. Everyone entering the field should expect to invest a significant amount of time in self-study. Reading this book is a good starting point; one of the primary goals of this book is to give you a foundation for the knowledge and skills we think are necessary to succeed as a data engineer. If you're pivoting your career into data engineering, we've found that the transition is easiest when moving from an adjacent field, such as software engineering, ETL development, database administration, data science, or data analysis. These disciplines tend to be "data aware" and provide good context for data roles in an organization. They also equip folks with the relevant technical skills and context to solve data engineering problems. Despite the lack of a formalized path, a requisite body of knowledge exists that we believe a data engineer should know to be successful. By definition, a data engineer must understand both data and technology. With respect to data, this entails knowing about various best practices around data management. On the technology end, a data engineer must be aware of various options for tools, their interplay, and their trade-offs. This requires a good understanding of software engineering, DataOps, and data architecture. Zooming out, a data engineer must also understand the requirements of data consumers (data analysts and data scientists) and the broader implications of data across the organization. Data engineering is a holistic practice; the best data engineers view their responsibilities through business and technical lenses.

Kỹ thuật dữ liệu là một lĩnh vực tăng trưởng nhanh, và còn rất nhiều câu hỏi về việc làm thế nào để trở thành một kỹ sư dữ liệu. Vì kỹ thuật dữ liệu là một bộ môn tương đối mới, có rất ít chương trình đào tạo chính thức để bước vào lĩnh vực này. Các trường đại học không có một lộ trình kỹ thuật dữ liệu chuẩn mực. Mặc dù có một nhóm các khóa cấp tốc (boot camp) về kỹ thuật dữ liệu và các hướng dẫn trực tuyến bao quát những chủ đề ngẫu nhiên, một chương trình giảng dạy chung cho chủ đề này vẫn chưa tồn tại. Những người bước vào kỹ thuật dữ liệu đến với những nền tảng khác nhau về giáo dục, sự nghiệp, và tập kỹ năng. Mọi người bước vào lĩnh vực này nên kỳ vọng phải đầu tư một lượng thời gian đáng kể vào việc tự học. Đọc cuốn sách này là một điểm khởi đầu tốt; một trong những mục tiêu chính của cuốn sách này là mang lại cho bạn một nền tảng về kiến thức và kỹ năng mà chúng tôi cho là cần thiết để thành công với tư cách một kỹ sư dữ liệu. Nếu bạn đang chuyển hướng sự nghiệp sang kỹ thuật dữ liệu, chúng tôi nhận thấy quá trình chuyển đổi dễ dàng nhất khi chuyển từ một lĩnh vực lân cận, chẳng hạn như kỹ thuật phần mềm, phát triển ETL, quản trị cơ sở dữ liệu, khoa học dữ liệu, hay phân tích dữ liệu. Các bộ môn này có xu hướng "nhận thức về dữ liệu" và cung cấp bối cảnh tốt cho các vai trò dữ liệu trong một tổ chức. Chúng cũng trang bị cho người ta các kỹ năng kỹ thuật và bối cảnh phù hợp để giải quyết các bài toán kỹ thuật dữ liệu. Bất chấp việc thiếu một lộ trình được chính thức hóa, có một khối kiến thức bắt buộc mà chúng tôi tin rằng một kỹ sư dữ liệu nên biết để thành công. Theo định nghĩa, một kỹ sư dữ liệu phải hiểu cả dữ liệu lẫn công nghệ. Về phía dữ liệu, điều này đòi hỏi biết về các thực hành tốt nhất khác nhau quanh quản lý dữ liệu. Về phía công nghệ, một kỹ sư dữ liệu phải nhận thức được các lựa chọn công cụ khác nhau, sự tương tác giữa chúng, và các đánh đổi của chúng. Điều này đòi hỏi một sự hiểu biết tốt về kỹ thuật phần mềm, DataOps, và kiến trúc dữ liệu. Phóng to ra, một kỹ sư dữ liệu cũng phải hiểu các yêu cầu của những bên tiêu thụ dữ liệu (các nhà phân tích dữ liệu và nhà khoa học dữ liệu) và những hệ quả rộng hơn của dữ liệu trên khắp tổ chức. Kỹ thuật dữ liệu là một thực hành toàn diện; các kỹ sư dữ liệu giỏi nhất nhìn nhận trách nhiệm của họ qua cả lăng kính nghiệp vụ lẫn kỹ thuật.

Business Responsibilities — Trách nhiệm Nghiệp vụ

The macro responsibilities we list in this section aren't exclusive to data engineers, but are crucial for anyone working in a data or technology field. Because a simple Google search will yield tons of resources to learn about these areas, we will simply list them for brevity: Know how to communicate with nontechnical and technical people.

Các trách nhiệm vĩ mô mà chúng tôi liệt kê trong phần này không chỉ dành riêng cho các kỹ sư dữ liệu, mà rất quan trọng đối với bất kỳ ai làm việc trong lĩnh vực dữ liệu hoặc công nghệ. Vì một tìm kiếm Google đơn giản sẽ cho ra vô số tài nguyên để tìm hiểu về các lĩnh vực này, chúng tôi sẽ chỉ liệt kê chúng cho ngắn gọn: Biết cách giao tiếp với người không chuyên về kỹ thuật và người chuyên về kỹ thuật.

Communication is key, and you need to be able to establish rapport and trust with people across the organization. We suggest paying close attention to organizational hierarchies, who reports to whom, how people interact, and which silos exist. These observations will be invaluable to your success.

Giao tiếp là then chốt, và bạn cần có khả năng thiết lập sự đồng điệu và niềm tin với mọi người trên khắp tổ chức. Chúng tôi gợi ý chú ý kỹ đến các hệ thống cấp bậc của tổ chức, ai báo cáo cho ai, mọi người tương tác như thế nào, và những ốc đảo nào tồn tại. Những quan sát này sẽ vô giá đối với thành công của bạn.

Understand how to scope and gather business and product requirements.

Hiểu cách phạm vi hóa và thu thập các yêu cầu nghiệp vụ và sản phẩm.

You need to know what to build and ensure that your stakeholders agree with your assessment. In addition, develop a sense of how data and technology decisions impact the business.

Bạn cần biết phải xây dựng cái gì và đảm bảo rằng các bên liên quan của bạn đồng ý với đánh giá của bạn. Ngoài ra, hãy phát triển một cảm nhận về việc các quyết định dữ liệu và công nghệ tác động đến doanh nghiệp như thế nào.

Understand the cultural foundations of Agile, DevOps, and DataOps.

Hiểu các nền tảng văn hóa của Agile, DevOps, và DataOps.

Many technologists mistakenly believe these practices are solved through technology. We feel this is dangerously wrong. Agile, DevOps, and DataOps are fundamentally cultural, requiring buy-in across the organization.

Nhiều nhà công nghệ lầm tưởng rằng các thực hành này được giải quyết thông qua công nghệ. Chúng tôi cảm thấy điều này sai một cách nguy hiểm. Agile, DevOps, và DataOps về cơ bản là thuộc về văn hóa, đòi hỏi sự ủng hộ trên khắp tổ chức.

Control costs.

Kiểm soát chi phí.

You'll be successful when you can keep costs low while providing outsized value. Know how to optimize for time to value, the **total cost of ownership**, and **opportunity cost**. Learn to monitor costs to avoid surprises.

Bạn sẽ thành công khi có thể giữ chi phí thấp trong khi mang lại giá trị vượt trội. Hãy biết cách tối ưu cho thời gian-để-có-giá-trị, tổng chi phí sở hữu (TCO), và chi phí cơ hội. Hãy học cách giám sát chi phí để tránh những bất ngờ.

Learn continuously.

Học hỏi liên tục.

The data field feels like it's changing at light speed. People who succeed in it are great at picking up new things while sharpening their fundamental knowledge. They're also good at filtering, determining which new developments are most relevant to their work, which are still immature, and which are just fads. Stay abreast of the field and learn how to learn.

Lĩnh vực dữ liệu có cảm giác như đang thay đổi với tốc độ ánh sáng. Những người thành công trong nó rất giỏi tiếp thu những thứ mới trong khi mài giũa kiến thức nền tảng của mình. Họ cũng giỏi sàng lọc, xác định những phát triển mới nào liên quan nhất đến công việc của họ, những cái nào vẫn còn non, và những cái nào chỉ là trào lưu nhất thời. Hãy nắm bắt kịp lĩnh vực và học cách học.

A successful data engineer always zooms out to understand the big picture and how to achieve outsized value for the business. Communication is vital, both for technical and nontechnical people. We often see data teams succeed based on their communication with other stakeholders; success or failure is rarely a technology issue. Knowing how to navigate an organization, scope and gather requirements, control costs, and continuously learn will set you apart from the data engineers who rely solely on their technical abilities to carry their career.

Một kỹ sư dữ liệu thành công luôn phóng to ra để hiểu bức tranh lớn và cách đạt được giá trị vượt trội cho doanh nghiệp. Giao tiếp là tối quan trọng, với cả người chuyên kỹ thuật lẫn người không chuyên kỹ thuật. Chúng tôi thường thấy các đội ngũ dữ liệu thành công dựa trên việc giao tiếp của họ với các bên liên quan khác; thành công hay thất bại hiếm khi là một vấn đề công nghệ. Việc biết cách điều hướng trong một tổ chức, phạm vi hóa và thu thập các yêu cầu, kiểm soát chi phí, và học hỏi liên tục sẽ giúp bạn nổi bật so với những kỹ sư dữ liệu chỉ dựa vào năng lực kỹ thuật của họ để duy dắt sự nghiệp.

Technical Responsibilities — Trách nhiệm Kỹ thuật

You must understand how to build architectures that optimize performance and cost at a high level, using prepackaged or homegrown components.

Bạn phải hiểu cách xây dựng các kiến trúc tối ưu hiệu năng và chi phí ở mức cao, sử dụng các thành phần đóng gói sẵn hoặc tự phát triển.

Ultimately, architectures and constituent technologies are building blocks to serve the data engineering lifecycle. Recall the stages of the data engineering lifecycle:

Rốt cuộc, các kiến trúc và các công nghệ cấu thành là những khối xây dựng để phục vụ vòng đời kỹ thuật dữ liệu. Hãy nhớ lại các giai đoạn của vòng đời kỹ thuật dữ liệu:

Generation

Sinh dữ liệu

Storage

Lưu trữ

Ingestion

Thu nạp

Transformation

Biến đổi

Serving

Phục vụ

The undercurrents of the data engineering lifecycle are the following:

Các dòng chảy ngầm của vòng đời kỹ thuật dữ liệu là những thứ sau:

Security

An ninh

Data management

Quản lý dữ liệu

DataOps

DataOps

Data architecture

Kiến trúc dữ liệu

Software engineering

Kỹ thuật phần mềm

Zooming in a bit, we discuss some of the tactical data and technology skills you'll need as a data engineer in this section; we discuss these in more detail in subsequent chapters. People often ask, should a data engineer know how to code? Short answer: yes. A data engineer should have production-grade software engineering chops. We note that the nature of software development projects undertaken by data engineers has changed fundamentally in the last few years. Fully managed services now replace a great deal of low-level programming effort previously expected of engineers, who now use managed open source, and simple plug-and-play software-as-a-service (SaaS) offerings. For example, data engineers now focus on high-level abstractions or writing pipelines as code within an orchestration framework.

Phóng to vào một chút, chúng tôi thảo luận một số kỹ năng dữ liệu và công nghệ mang tính chiến thuật mà bạn sẽ cần với tư cách một kỹ sư dữ liệu trong phần này; chúng tôi thảo luận chúng chi tiết hơn ở các chương sau. Người ta thường hỏi, một kỹ sư dữ liệu có nên biết cách viết mã không? Câu trả lời ngắn gọn: có. Một kỹ sư dữ liệu nên có bản lĩnh kỹ thuật phần mềm cấp độ vận hành. Chúng tôi lưu ý rằng bản chất của các dự án phát triển phần mềm mà các kỹ sư dữ liệu đảm nhận đã thay đổi về cơ bản trong vài năm qua. Các dịch vụ được quản lý hoàn toàn giờ đây thay thế một lượng lớn nỗ lực lập trình cấp thấp vốn trước đây được kỳ vọng từ các kỹ sư, những người giờ đây sử dụng mã nguồn mở được quản lý, và các sản phẩm phần mềm-như-một-dịch-vụ (SaaS) cảm-và-chạy đơn giản. Ví dụ, các kỹ sư dữ liệu giờ đây tập trung vào các trừu tượng cấp cao hoặc viết các đường ống dưới dạng mã trong một framework điều phối.

Even in a more abstract world, software engineering best practices provide a competitive advantage, and data engineers who can dive into the deep architectural details of a codebase give their companies an edge when specific technical needs arise. In short, a data engineer who can't write production-grade code will be severely hindered, and we don't see this changing anytime soon. Data engineers remain software engineers, in addition to their many other roles. What languages should a data engineer know? We divide data engineering programming languages into primary and secondary categories. At the time of this writing, the primary languages of data engineering are SQL, Python, a Java Virtual Machine (JVM) language (usually Java or Scala), and bash: SQL

Ngay cả trong một thế giới trù tượng hơn, các thực hành tốt nhất của kỹ thuật phần mềm vẫn mang lại một lợi thế cạnh tranh, và những kỹ sư dữ liệu có thể lặn sâu vào các chi tiết kiến trúc sâu của một cơ sở mã sẽ mang lại cho công ty của họ một lợi thế khi các nhu cầu kỹ thuật cụ thể phát sinh. Nói ngắn gọn, một kỹ sư dữ liệu không thể viết mã cấp độ vận hành sẽ bị cản trở nghiêm trọng, và chúng tôi không thấy điều này thay đổi trong tương lai gần. Các kỹ sư dữ liệu vẫn là kỹ sư phần mềm, bên cạnh nhiều vai trò khác của họ. Một kỹ sư dữ liệu nên biết những ngôn ngữ nào? Chúng tôi chia các ngôn ngữ lập trình kỹ thuật dữ liệu thành hai nhóm chính và phụ. Tại thời điểm viết cuốn sách này, các ngôn ngữ chính của kỹ thuật dữ liệu là SQL, Python, một ngôn ngữ Máy ảo Java (JVM) (thường là Java hoặc Scala), và bash: SQL

The most common interface for databases and data lakes. After briefly being sidelined by the need to write custom MapReduce code for big data processing, SQL (in various forms) has reemerged as the lingua franca of data.

Giao diện phổ biến nhất cho các cơ sở dữ liệu và hồ dữ liệu (data lake). Sau khi bị gạt sang một bên trong chốc lát bởi nhu cầu viết mã MapReduce tùy chỉnh cho xử lý big data, SQL (dưới nhiều dạng khác nhau) đã tái xuất hiện như là ngôn ngữ chung (lingua franca) của dữ liệu.

Python

Python

The bridge language between data engineering and data science. A growing number of data engineering tools are written in Python or have Python APIs. It's known as “the second-best language at everything.” Python underlies popular data tools such as pandas, NumPy, Airflow, sci-kit learn, TensorFlow, PyTorch, and PySpark. Python is the glue between underlying components and is frequently a first-class API language for interfacing with a framework.

Ngôn ngữ cầu nối giữa kỹ thuật dữ liệu và khoa học dữ liệu. Một số lượng ngày càng tăng các công cụ kỹ thuật dữ liệu được viết bằng Python hoặc có các API Python. Nó được biết đến là “ngôn ngữ tốt thứ hai ở mọi thứ”. Python làm nền tảng cho các công cụ dữ liệu phổ biến như pandas, NumPy, Airflow, sci-kit learn, TensorFlow, PyTorch, và PySpark. Python là chất keo giữa các thành phần bên dưới và thường là một ngôn ngữ API hạng nhất để giao tiếp với một framework.

JVM languages such as Java and Scala

Các ngôn ngữ JVM như Java và Scala

Prevalent for Apache open source projects such as Spark, Hive, and Druid. The JVM is generally more performant than Python and may provide access to lower-level features than a Python API (for example, this is the case for Apache Spark and Beam). Understanding Java or Scala will be beneficial if you're using a popular open source data framework.

Phổ biến đối với các dự án mã nguồn mở Apache như Spark, Hive, và Druid. JVM nói chung có hiệu năng cao hơn Python và có thể cung cấp quyền truy cập vào các tính năng cấp thấp hơn so với một API Python (ví dụ, đây là trường hợp của Apache Spark và Beam). Việc hiểu Java hoặc Scala sẽ có lợi nếu bạn đang dùng một framework dữ liệu mã nguồn mở phổ biến.

bash

bash

The command-line interface for Linux operating systems. Knowing bash commands and being comfortable using CLIs will significantly improve your productivity and workflow when you need to script or perform OS operations. Even today, data engineers frequently use command-line tools like awk or sed to process files in a **data pipeline** or call bash commands from orchestration frameworks. If you're using Windows, feel free to substitute PowerShell for bash.

Giao diện dòng lệnh cho các hệ điều hành Linux. Việc biết các lệnh bash và thoải mái sử dụng các CLI sẽ cải thiện đáng kể năng suất và quy trình làm việc của bạn khi bạn cần viết script hoặc thực hiện các thao tác hệ điều hành. Ngay cả ngày nay, các kỹ sư dữ liệu thường dùng các công cụ dòng lệnh như awk hoặc sed để xử lý các tệp trong một đường ống dữ liệu hoặc gọi các lệnh bash từ các framework điều phối. Nếu bạn đang dùng Windows, cứ thoải mái thay bash bằng PowerShell.

THE UNREASONABLE EFFECTIVENESS OF SQL — HIỆU QUẢ PHI LÝ CỦA SQL

The advent of MapReduce and the big data era relegated SQL to passé status. Since then, various developments have dramatically enhanced the utility of SQL in the data engineering lifecycle. Spark SQL, Google BigQuery, Snowflake, Hive, and many other data tools can process massive amounts of data by using declarative, set-theoretic SQL semantics. SQL is also supported by many streaming frameworks, such as Apache Flink, Beam, and Kafka. We believe that competent data engineers should be highly proficient in SQL. Are we saying that SQL is a be-all and end-all language? Not at all. SQL is a powerful tool that can quickly solve complex analytics and data transformation problems. Given that time is a primary constraint for data engineering team throughput, engineers should embrace tools that combine simplicity and high productivity. Data engineers also do well to develop expertise in composing SQL with other operations, either within frameworks such as Spark and Flink or by using orchestration to combine multiple tools. Data engineers should also learn modern SQL semantics for dealing with JavaScript Object Notation (JSON) parsing and nested data and consider leveraging a SQL management framework such as dbt (Data Build Tool). A proficient data engineer also recognizes when SQL is not the right tool for the job and can choose and code in a suitable alternative. A SQL expert could likely write a query to stem and tokenize raw text in a natural language processing (NLP) pipeline but would also recognize that coding in native Spark is a far superior alternative to this masochistic exercise.

Sự ra đời của MapReduce và kỷ nguyên big data đã đẩy SQL xuống vị thế lỗi thời. Kể từ đó, nhiều phát triển khác nhau đã nâng cao đáng kể tính hữu dụng của SQL trong vòng đời kỹ thuật dữ liệu. Spark SQL, Google BigQuery, Snowflake, Hive, và nhiều công cụ dữ liệu khác có thể xử lý những lượng dữ liệu khổng lồ bằng cách sử dụng ngữ nghĩa SQL khai báo, theo lý thuyết tập hợp. SQL cũng được hỗ trợ bởi nhiều framework theo luồng, chẳng hạn như Apache Flink, Beam, và Kafka. Chúng tôi tin rằng các kỹ sư dữ liệu có năng lực nên rất thành thạo SQL. Có phải chúng tôi đang nói rằng SQL là một ngôn ngữ vạn năng tốt cùng? Không hề. SQL là một công cụ mạnh mẽ có thể nhanh chóng giải quyết các bài toán phân tích và biến đổi dữ liệu phức tạp. Vì thời gian là một ràng buộc chính đối với thông lượng của đội ngũ kỹ thuật dữ liệu, các kỹ sư nên đón nhận các công cụ kết hợp sự đơn giản và năng suất cao. Các kỹ sư dữ liệu cũng nên phát triển chuyên môn trong việc kết hợp SQL với các thao tác khác, hoặc trong các framework như Spark và Flink hoặc bằng cách dùng điều phối để kết hợp nhiều công cụ. Các kỹ sư dữ liệu cũng nên học ngữ nghĩa SQL hiện đại để xử lý việc phân tích cú pháp JavaScript Object Notation (JSON) và dữ liệu lồng nhau và cân nhắc tận dụng một framework quản lý SQL như dbt (Data Build Tool). Một kỹ sư dữ liệu thành thạo cũng nhận ra khi nào SQL không phải là công cụ phù

hợp cho công việc và có thể chọn và viết mã trong một giải pháp thay thế phù hợp. Một chuyên gia SQL có lẽ có thể viết một truy vấn để cắt gốc từ và tách token văn bản thô trong một đường ống xử lý ngôn ngữ tự nhiên (NLP) nhưng cũng sẽ nhận ra rằng viết mã trong Spark gốc là một giải pháp thay thế vượt trội hơn nhiều so với bài tập tự hành xác này.

Data engineers may also need to develop proficiency in secondary programming languages, including R, JavaScript, Go, Rust, C/C++, C#, and Julia. Developing in these languages is often necessary when popular across the company or used with domain-specific data tools. For instance, JavaScript has proven popular as a language for user-defined functions in cloud data warehouses. At the same time, C# and PowerShell are essential in companies that leverage Azure and the Microsoft ecosystem.

Các kỹ sư dữ liệu cũng có thể cần phát triển sự thành thạo trong các ngôn ngữ lập trình phụ, bao gồm R, JavaScript, Go, Rust, C/C++, C#, và Julia. Việc phát triển trong các ngôn ngữ này thường cần thiết khi chúng phổ biến trong toàn công ty hoặc được dùng với các công cụ dữ liệu đặc thù lĩnh vực. Chẳng hạn, JavaScript đã chứng tỏ phổ biến như một ngôn ngữ cho các hàm do người dùng định nghĩa trong các kho dữ liệu trên đám mây. Đồng thời, C# và PowerShell là thiết yếu trong các công ty tận dụng Azure và hệ sinh thái Microsoft.

KEEPING PACE IN A FAST-MOVING FIELD — GIỮ NHỊP TRONG MỘT LĨNH VỰC CHUYỂN ĐỘNG NHANH

Once a new technology rolls over you, if you're not part of the steamroller, you're part of the road. —Stewart Brand How do you keep your skills sharp in a rapidly changing field like data engineering? Should you focus on the latest tools or deep dive into fundamentals? Here's our advice: focus on the fundamentals to understand what's not going to change; pay attention to ongoing developments to know where the field is going. New paradigms and practices are introduced all the time, and it's incumbent on you to stay current. Strive to understand how new technologies will be helpful in the lifecycle.

Một khi một công nghệ mới cán qua bạn, nếu bạn không phải là một phần của chiếc xe lu, thì bạn là một phần của con đường. —Stewart Brand Làm thế nào để bạn giữ cho kỹ năng của mình sắc bén trong một lĩnh vực thay đổi nhanh chóng như kỹ thuật dữ liệu? Bạn nên tập trung vào các công cụ mới nhất hay lặn sâu vào các nền tảng cơ bản? Đây là lời khuyên của chúng tôi: tập trung vào các nền tảng cơ bản để hiểu cái gì sẽ không thay đổi; chú ý đến các phát triển đang diễn ra để biết lĩnh vực đang đi về đâu. Các mô thức và thực hành mới được giới thiệu mọi lúc, và bạn có trách nhiệm phải cập nhật. Hãy nỗ lực hiểu các công nghệ mới sẽ hữu ích như thế nào trong vòng đời.

The Continuum of Data Engineering Roles, from A to B — Dải liên tục các Vai trò Kỹ thuật Dữ liệu, từ A đến B

Although job descriptions paint a data engineer as a “unicorn” who must possess every data skill imaginable, data engineers don't all do the same type of work or have the same skill set. Data maturity is a helpful guide to understanding the types of data challenges a company will face as it grows its data capability. It's beneficial to look at some critical distinctions in the kinds of work data engineers do. Though these distinctions are simplistic, they clarify what data scientists and data engineers do and avoid lumping either role into the unicorn bucket. In data science, there's the notion of type A and type B data scientists. 10 Type A data scientists —where A stands for analysis —focus on

understanding and deriving insight from data. Type B data scientists — where B stands for building —share similar backgrounds as type A data scientists and possess strong programming skills. The type B data scientist builds systems that make data science work in production. Borrowing from this data scientist continuum, we'll create a similar distinction for two types of data engineers: Type A data engineers

Mặc dù các bản mô tả công việc vẽ ra một kỹ sư dữ liệu như một “kỳ lân” phải sở hữu mọi kỹ năng dữ liệu có thể tưởng tượng được, các kỹ sư dữ liệu không phải tất cả đều làm cùng một loại công việc hay có cùng một tập kỹ năng. Độ trưởng thành dữ liệu là một chỉ dẫn hữu ích để hiểu các loại thách thức dữ liệu mà một công ty sẽ đối mặt khi nó phát triển năng lực dữ liệu của mình. Sẽ có ích khi nhìn vào một số khác biệt then chốt trong các loại công việc mà các kỹ sư dữ liệu làm. Mặc dù những khác biệt này đơn giản hóa, chúng làm rõ những gì các nhà khoa học dữ liệu và kỹ sư dữ liệu làm và tránh gộp một trong hai vai trò vào cái rọ kỳ lân. Trong khoa học dữ liệu, có khái niệm về các nhà khoa học dữ liệu loại A và loại B. 10 Các nhà khoa học dữ liệu loại A—nơi A là viết tắt của analysis (phân tích)—tập trung vào việc hiểu và rút ra hiểu biết từ dữ liệu. Các nhà khoa học dữ liệu loại B—nơi B là viết tắt của building (xây dựng)—có nền tảng tương tự như các nhà khoa học dữ liệu loại A và sở hữu kỹ năng lập trình mạnh mẽ. Nhà khoa học dữ liệu loại B xây dựng các hệ thống làm cho khoa học dữ liệu hoạt động trong môi trường vận hành. Vay mượn từ dải liên tục nhà khoa học dữ liệu này, chúng tôi sẽ tạo ra một sự phân biệt tương tự cho hai loại kỹ sư dữ liệu: Các kỹ sư dữ liệu loại A

A stands for abstraction . In this case, the data engineer avoids undifferentiated heavy lifting, keeping data architecture as abstract and straightforward as possible and not reinventing the wheel. Type A data engineers manage the data engineering lifecycle mainly by using entirely off-the-shelf products, managed services, and tools. Type A data engineers work at companies across industries and at all levels of data maturity.

A là viết tắt của abstraction (trừu tượng hóa). Trong trường hợp này, kỹ sư dữ liệu tránh việc nặng nhọc không tạo khác biệt, giữ cho kiến trúc dữ liệu trừu tượng và đơn giản hết mức có thể và không phát minh lại cái bánh xe. Các kỹ sư dữ liệu loại A quản lý vòng đời kỹ thuật dữ liệu chủ yếu bằng cách dùng các sản phẩm có sẵn hoàn toàn, các dịch vụ được quản lý, và các công cụ. Các kỹ sư dữ liệu loại A làm việc tại các công ty thuộc mọi ngành và ở mọi mức độ trưởng thành dữ liệu.

Type B data engineers

Các kỹ sư dữ liệu loại B

B stands for build . Type B data engineers build data tools and systems that scale and leverage a company's core competency and competitive advantage. In the data maturity range, a type B data engineer is more commonly found at companies in stage 2 and 3 (scaling and leading with data), or when an initial data use case is so unique and mission-critical that custom data tools are required to get started.

B là viết tắt của build (xây dựng). Các kỹ sư dữ liệu loại B xây dựng các công cụ và hệ thống dữ liệu mở rộng quy mô và tận dụng năng lực cốt lõi cùng lợi thế cạnh tranh của một công ty. Trong dải độ trưởng thành dữ liệu, một kỹ sư dữ liệu loại B thường gặp hơn ở các công ty trong giai đoạn 2 và 3 (mở rộng quy mô và dẫn dắt bằng dữ liệu), hoặc khi một trường hợp sử dụng dữ liệu ban đầu độc đáo và trọng yếu đến mức cần đến các công cụ dữ liệu tùy chỉnh để bắt đầu.

Type A and type B data engineers may work in the same company and may even be the same person! More commonly, a type A data engineer is first hired to set the foundation, with type B data engineer

skill sets either learned or hired as the need arises within a company.

Các kỹ sư dữ liệu loại A và loại B có thể làm việc trong cùng một công ty và thậm chí có thể là cùng một người! Phổ biến hơn, một kỹ sư dữ liệu loại A được tuyển trước để đặt nền móng, với các tập kỹ năng kỹ sư dữ liệu loại B hoặc được học hỏi hoặc được tuyển dụng khi nhu cầu phát sinh trong một công ty.

Data Engineers Inside an Organization — Kỹ sư Dữ liệu bên trong một Tổ chức

Data engineers don't work in a vacuum. Depending on what they're working on, they will interact with technical and nontechnical people and face different directions (internal and external). Let's explore what data engineers do inside an organization and with whom they interact.

Các kỹ sư dữ liệu không làm việc trong chân không. Tùy vào những gì họ đang làm, họ sẽ tương tác với những người chuyên kỹ thuật và không chuyên kỹ thuật và hướng về những phía khác nhau (nội bộ và bên ngoài). Hãy khám phá những gì các kỹ sư dữ liệu làm bên trong một tổ chức và họ tương tác với những ai.

Internal-Facing Versus External-Facing Data Engineers — Kỹ sư Dữ liệu Hướng Nội bộ so với Hướng Bên ngoài

A data engineer serves several end users and faces many internal and external directions (Figure 1-9). Since not all data engineering workloads and responsibilities are the same, it's essential to understand whom the data engineer serves. Depending on the end-use cases, a data engineer's primary responsibilities are external facing, internal facing, or a blend of the two.

Một kỹ sư dữ liệu phục vụ một số người dùng cuối và hướng về nhiều phía nội bộ và bên ngoài (Hình 1-9). Vì không phải tất cả các khối lượng công việc và trách nhiệm kỹ thuật dữ liệu đều giống nhau, điều thiết yếu là hiểu kỹ sư dữ liệu phục vụ ai. Tùy vào các trường hợp sử dụng cuối, các trách nhiệm chính của một kỹ sư dữ liệu là hướng bên ngoài, hướng nội bộ, hoặc một sự pha trộn của cả hai.

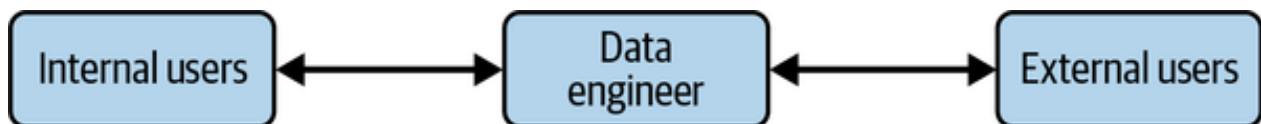


Figure 1-9. The directions a data engineer faces An external-facing data engineer typically aligns with the users of external-facing applications, such as social media apps, Internet of Things (IoT) devices, and ecommerce platforms. This data engineer architects, builds, and manages the systems that collect, store, and process transactional and event data from these applications. The systems built by these data engineers have a feedback loop from the application to the data pipeline, and then back to the application (Figure 1-10).

Hình 1-9. Các phía mà một kỹ sư dữ liệu hướng về Một kỹ sư dữ liệu hướng bên ngoài thường gắn với những người dùng của các ứng dụng hướng bên ngoài, chẳng hạn như các ứng dụng mạng xã hội, các thiết bị Internet vạn vật (IoT), và các nền tảng thương mại điện tử. Kỹ sư dữ liệu này thiết kế kiến trúc, xây dựng, và quản lý các hệ thống thu thập, lưu trữ, và xử lý dữ liệu giao dịch và dữ liệu sự kiện từ các ứng dụng này. Các hệ thống được

xây dựng bởi các kỹ sư dữ liệu này có một vòng phản hồi từ ứng dụng đến đường ống dữ liệu, rồi quay trở lại ứng dụng (Hình 1-10).

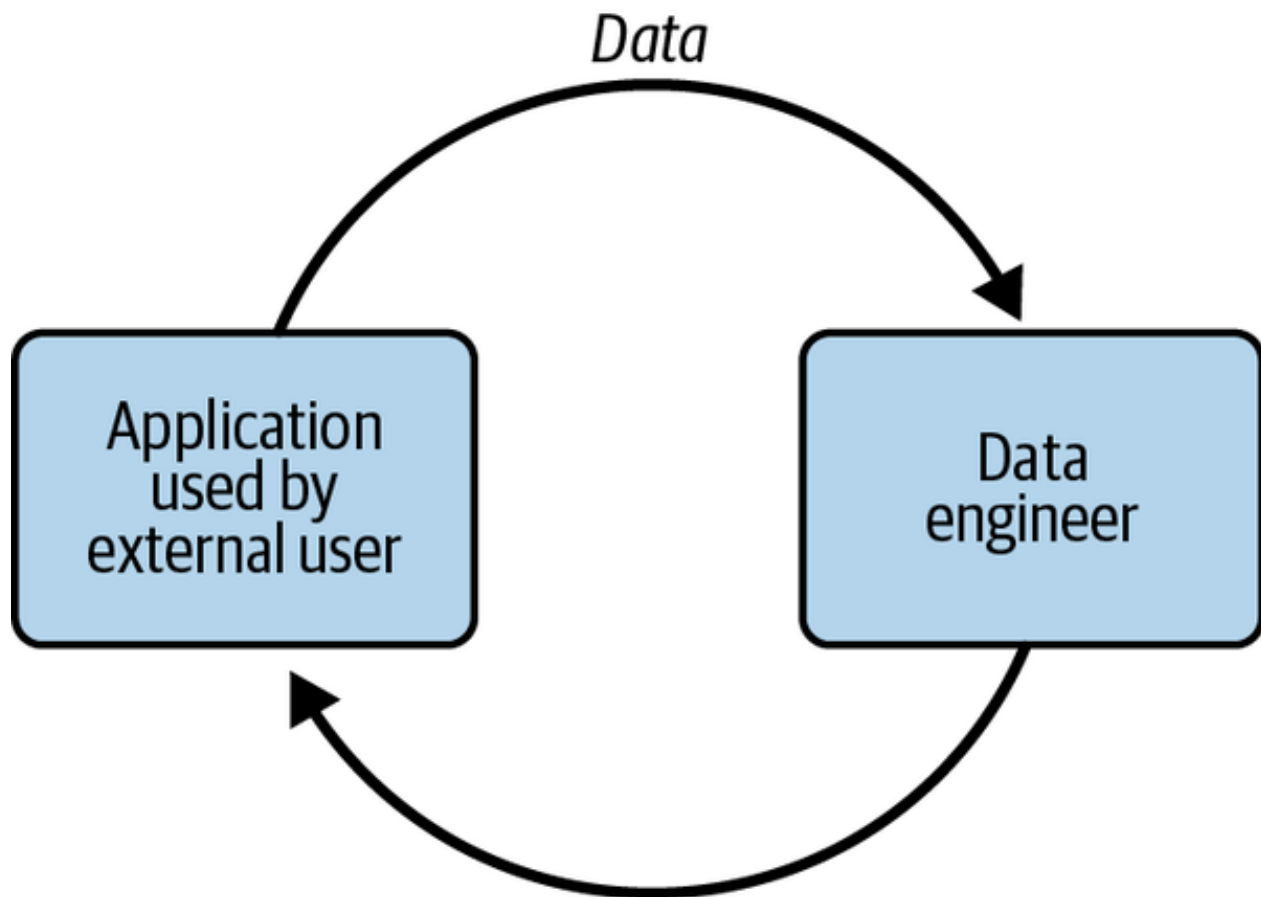


Figure 1-10. External-facing data engineer systems External-facing data engineering comes with a unique set of problems. External-facing query engines often handle much larger concurrency loads than internal-facing systems. Engineers also need to consider putting tight limits on queries that users can run to limit the infrastructure impact of any single user. In addition, security is a much more complex and sensitive problem for external queries, especially if the data being queried is **multitenant** (data from many customers and housed in a single table). An internal-facing data engineer typically focuses on activities crucial to the needs of the business and internal stakeholders (Figure 1-11). Examples include creating and maintaining data pipelines and data warehouses for BI dashboards, reports, business processes, data science, and ML models.

Hình 1-10. Các hệ thống của kỹ sư dữ liệu hướng bên ngoài Kỹ thuật dữ liệu hướng bên ngoài đi kèm với một tập các bài toán độc đáo. Các công cụ truy vấn hướng bên ngoài thường xử lý các tải đồng thời lớn hơn nhiều so với các hệ thống hướng nội bộ. Các kỹ sư cũng cần cân nhắc đặt các giới hạn chặt chẽ lên các truy vấn mà người dùng có thể chạy nhằm hạn chế tác động đến hạ tầng của bất kỳ một người dùng đơn lẻ nào. Ngoài ra, an ninh là một bài toán phức tạp và nhạy cảm hơn nhiều đối với các truy vấn bên ngoài, đặc biệt nếu dữ liệu được truy vấn là đa thuê bao (multitenant) (dữ liệu từ nhiều khách hàng và lưu chung trong một bảng đơn). Một kỹ sư dữ liệu hướng nội bộ thường tập trung vào các hoạt động trọng yếu đối với các nhu cầu của doanh nghiệp và các bên liên quan nội bộ (Hình 1-11). Các ví dụ bao gồm tạo và bảo trì các đường ống dữ liệu và kho dữ liệu cho các bảng điều khiển BI, báo cáo, quy trình nghiệp vụ, khoa học dữ liệu, và các mô hình ML.

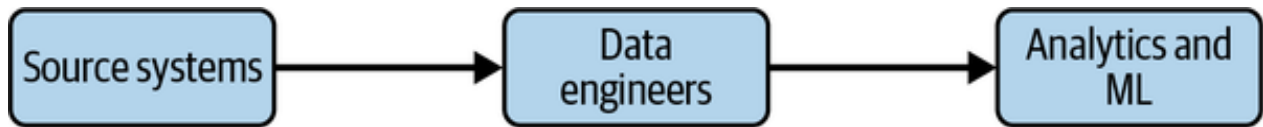


Figure 1-11. Internal-facing data engineer External-facing and internal-facing responsibilities are often blended. In practice, internal-facing data is usually a prerequisite to external-facing data. The data engineer has two sets of users with very different requirements for query concurrency, security, and more.

Hình 1-11. Kỹ sư dữ liệu hướng nội bộ Các trách nhiệm hướng bên ngoài và hướng nội bộ thường được pha trộn. Trong thực tế, dữ liệu hướng nội bộ thường là một điều kiện tiên quyết cho dữ liệu hướng bên ngoài. Kỹ sư dữ liệu có hai tập người dùng với các yêu cầu rất khác nhau về tính đồng thời của truy vấn, an ninh, và hơn thế.

Data Engineers and Other Technical Roles — Kỹ sư Dữ liệu và các Vai trò Kỹ thuật Khác

In practice, the data engineering lifecycle cuts across many domains of responsibility. Data engineers sit at the nexus of various roles, directly or through managers, interacting with many organizational units. Let's look at whom a data engineer may impact. In this section, we'll discuss technical roles connected to data engineering (Figure 1-12).

Trong thực tế, vòng đời kỹ thuật dữ liệu cắt ngang qua nhiều lĩnh vực trách nhiệm. Các kỹ sư dữ liệu nằm ở giao điểm của nhiều vai trò khác nhau, trực tiếp hoặc thông qua các quản lý, tương tác với nhiều đơn vị tổ chức. Hãy xem một kỹ sư dữ liệu có thể tác động đến những ai. Trong phần này, chúng tôi sẽ thảo luận các vai trò kỹ thuật kết nối với kỹ thuật dữ liệu (Hình 1-12).

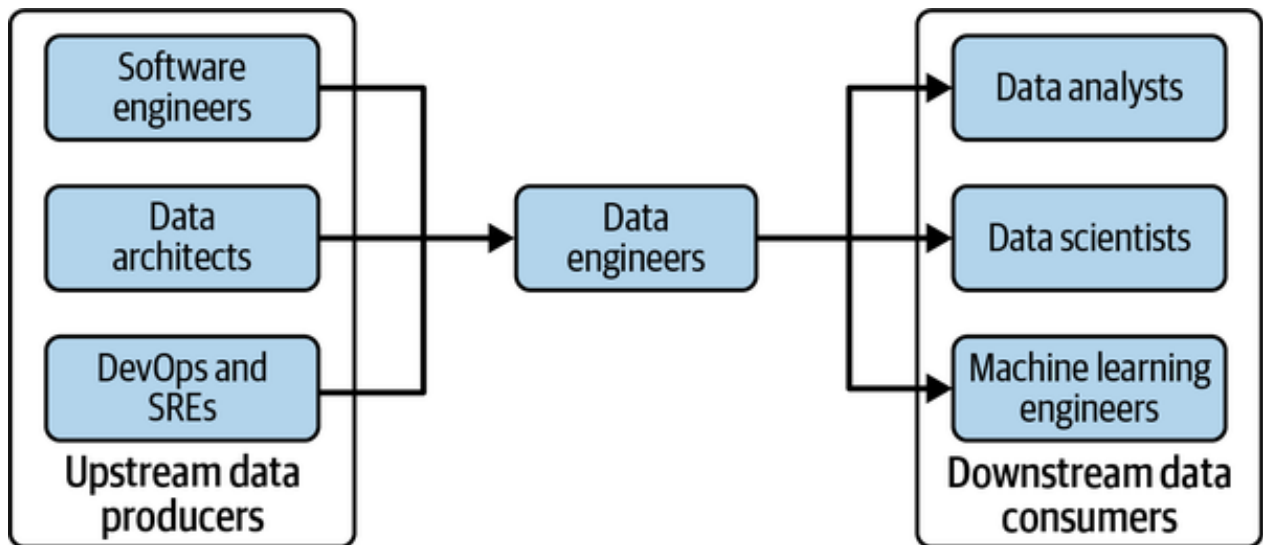


Figure 1-12. Key technical stakeholders of data engineering The data engineer is a hub between data producers , such as software engineers, data architects, and DevOps or site-reliability engineers (SREs), and data consumers , such as data analysts, data scientists, and ML engineers. In addition, data engineers will interact with those in operational roles, such as DevOps engineers. Given the pace at which new data roles come into vogue (analytics and ML engineers come to mind), this is by no means an exhaustive list.

Hình 1-12. Các bên liên quan kỹ thuật chính của kỹ thuật dữ liệu Kỹ sư dữ liệu là một trung tâm giữa các bên sản xuất dữ liệu, chẳng hạn như các kỹ sư phần mềm, kiến trúc sư dữ liệu, và các kỹ sư DevOps hoặc kỹ sư độ tin cậy hệ thống (SRE), và các bên tiêu thụ dữ liệu, chẳng hạn như các nhà phân tích dữ liệu, nhà khoa học dữ liệu, và kỹ sư ML. Ngoài ra, các kỹ sư dữ liệu sẽ tương tác với những người trong các vai trò vận hành, chẳng hạn như các kỹ sư DevOps. Với tốc độ các vai trò dữ liệu mới trở nên thịnh hành (các kỹ sư analytics và ML xuất hiện trong tâm trí), đây hoàn toàn không phải là một danh sách đầy đủ.

Upstream stakeholders — Các bên liên quan thượng nguồn

To be successful as a data engineer, you need to understand the data architecture you're using or designing and the source systems producing the data you'll need. Next, we discuss a few familiar upstream stakeholders: data architects, software engineers, and DevOps engineers.

Để thành công với tư cách một kỹ sư dữ liệu, bạn cần hiểu kiến trúc dữ liệu mà bạn đang dùng hoặc thiết kế và các hệ thống nguồn sản xuất ra dữ liệu mà bạn sẽ cần. Tiếp theo, chúng tôi thảo luận một vài bên liên quan thượng nguồn quen thuộc: các kiến trúc sư dữ liệu, kỹ sư phần mềm, và kỹ sư DevOps.

Data architects Data architects function at a level of abstraction one step removed from data engineers. Data architects design the blueprint for organizational data management, mapping out processes and overall data architecture and systems. They also serve as a bridge between an organization's technical and nontechnical sides. Successful data architects generally have “battle scars” from extensive engineering experience, allowing them to guide and assist engineers while successfully communicating engineering challenges to nontechnical business stakeholders. Data architects implement policies for managing data across silos and business units, steer global strategies such as data management and data governance, and guide significant initiatives. Data architects often play a central role in cloud migrations and **greenfield** cloud design. The advent of the cloud has shifted the boundary between data architecture and data engineering. Cloud data architectures are much more fluid than on-premises systems, so architecture decisions that traditionally involved extensive study, long lead times, purchase contracts, and hardware installation are now often made during the implementation process, just one step in a larger strategy. Nevertheless, data architects will remain influential visionaries in enterprises, working hand in hand with data engineers to determine the big picture of architecture practices and data strategies. Depending on the company's data maturity and size, a data engineer may overlap with or assume the responsibilities of a data architect. Therefore, a data engineer should have a good understanding of architecture best practices and approaches. Note that we have placed data architects in the upstream stakeholders section. Data architects often help design application data layers that are source systems for data engineers. Architects may also interact with data engineers at various other stages of the data engineering lifecycle. We cover “good” data architecture in Chapter 3.

Kiến trúc sư dữ liệu Các kiến trúc sư dữ liệu vận hành ở một mức trừu tượng cách kỹ sư dữ liệu một bước. Các kiến trúc sư dữ liệu thiết kế bản thiết kế cho việc quản lý dữ liệu của tổ chức, vạch ra các quy trình cùng kiến trúc dữ liệu và hệ thống tổng thể. Họ cũng đóng vai trò như một cầu nối giữa phía kỹ thuật và phía không chuyên kỹ thuật của một tổ chức. Các kiến trúc sư dữ liệu thành công nói chung có những “vết sẹo chiến trận” từ kinh nghiệm kỹ thuật dày dặn, cho phép họ dẫn dắt và hỗ trợ các kỹ sư trong khi truyền đạt thành công các thách thức kỹ thuật cho các bên liên quan nghiệp vụ không chuyên kỹ thuật. Các kiến trúc sư dữ liệu triển khai các chính sách để quản lý dữ liệu xuyên qua các ốc đảo và đơn vị nghiệp vụ, lèo lái các chiến lược toàn cục như quản lý dữ liệu và quản trị

dữ liệu, và dẫn dắt các sáng kiến quan trọng. Các kiến trúc sư dữ liệu thường đóng một vai trò trung tâm trong các cuộc di trú lên đám mây và thiết kế đám mây greenfield. Sự ra đời của đám mây đã dịch chuyển ranh giới giữa kiến trúc dữ liệu và kỹ thuật dữ liệu. Các kiến trúc dữ liệu trên đám mây linh động hơn nhiều so với các hệ thống tại chỗ (on-premises), nên các quyết định kiến trúc mà theo truyền thống liên quan đến nghiên cứu sâu rộng, thời gian chờ dài, các hợp đồng mua sắm, và cài đặt phần cứng giờ đây thường được đưa ra trong quá trình triển khai, chỉ là một bước trong một chiến lược lớn hơn. Tuy nhiên, các kiến trúc sư dữ liệu sẽ vẫn là những nhà tầm nhìn có ảnh hưởng trong các doanh nghiệp, làm việc sát cánh với các kỹ sư dữ liệu để xác định bức tranh lớn của các thực hành kiến trúc và các chiến lược dữ liệu. Tùy vào độ trưởng thành dữ liệu và quy mô của công ty, một kỹ sư dữ liệu có thể chồng lấn với hoặc đảm nhận các trách nhiệm của một kiến trúc sư dữ liệu. Do đó, một kỹ sư dữ liệu nên có một sự hiểu biết tốt về các thực hành tốt nhất và cách tiếp cận kiến trúc. Lưu ý rằng chúng tôi đã đặt các kiến trúc sư dữ liệu vào phần các bên liên quan thượng nguồn. Các kiến trúc sư dữ liệu thường giúp thiết kế các tầng dữ liệu ứng dụng vốn là các hệ thống nguồn cho các kỹ sư dữ liệu. Các kiến trúc sư cũng có thể tương tác với các kỹ sư dữ liệu ở nhiều giai đoạn khác của vòng đời kỹ thuật dữ liệu. Chúng tôi trình bày kiến trúc dữ liệu “tốt” ở Chương 3.

Software engineers Software engineers build the software and systems that run a business; they are largely responsible for generating the internal data that data engineers will consume and process. The systems built by software engineers typically generate application event data and logs, which are significant assets in their own right. This internal data contrasts with external data pulled from SaaS platforms or partner businesses. In well-run technical organizations, software engineers and data engineers coordinate from the inception of a new project to design application data for consumption by analytics and ML applications. A data engineer should work together with software engineers to understand the applications that generate data, the volume, frequency, and format of the generated data, and anything else that will impact the data engineering lifecycle, such as **data security** and **regulatory compliance**. For example, this might mean setting upstream expectations on what the data software engineers need to do their jobs. Data engineers must work closely with the software engineers.

Kỹ sư phần mềm Các kỹ sư phần mềm xây dựng phần mềm và hệ thống vận hành một doanh nghiệp; họ phần lớn chịu trách nhiệm tạo ra dữ liệu nội bộ mà các kỹ sư dữ liệu sẽ tiêu thụ và xử lý. Các hệ thống được xây dựng bởi các kỹ sư phần mềm thường tạo ra dữ liệu sự kiện ứng dụng và nhật ký (log), những thứ vốn là những tài sản quan trọng theo cách riêng của chúng. Dữ liệu nội bộ này tương phản với dữ liệu bên ngoài được kéo về từ các nền tảng SaaS hoặc các doanh nghiệp đối tác. Trong các tổ chức kỹ thuật được vận hành tốt, các kỹ sư phần mềm và kỹ sư dữ liệu phối hợp ngay từ lúc khởi đầu một dự án mới để thiết kế dữ liệu ứng dụng cho việc tiêu thụ bởi các ứng dụng phân tích và ML. Một kỹ sư dữ liệu nên làm việc cùng các kỹ sư phần mềm để hiểu các ứng dụng tạo ra dữ liệu, khối lượng, tần suất, và định dạng của dữ liệu được tạo ra, và bất cứ điều gì khác sẽ tác động đến vòng đời kỹ thuật dữ liệu, chẳng hạn như an ninh dữ liệu và tuân thủ quy định. Ví dụ, điều này có thể có nghĩa là đặt ra các kỳ vọng thượng nguồn về những gì các kỹ sư phần mềm dữ liệu cần để làm công việc của họ. Các kỹ sư dữ liệu phải làm việc sát cánh với các kỹ sư phần mềm.

DevOps engineers and site-reliability engineers DevOps and SREs often produce data through operational monitoring. We classify them as upstream of data engineers, but they may also be downstream, consuming data through dashboards or interacting with data engineers directly in coordinating operations of data systems.

Kỹ sư DevOps và kỹ sư độ tin cậy hệ thống Các kỹ sư DevOps và SRE thường tạo ra dữ liệu

thông qua việc giám sát vận hành. Chúng tôi phân loại họ là thượng nguồn so với các kỹ sư dữ liệu, nhưng họ cũng có thể là hạ nguồn, tiêu thụ dữ liệu thông qua các bảng điều khiển hoặc tương tác trực tiếp với các kỹ sư dữ liệu trong việc phối hợp vận hành các hệ thống dữ liệu.

Downstream stakeholders — Các bên liên quan hạ nguồn

The modern data engineering profession exists to serve downstream data consumers and use cases. This section discusses how data engineers interact with various downstream roles. We'll also introduce a few service models, including centralized data engineering teams and cross-functional teams.

Nghề kỹ thuật dữ liệu hiện đại tồn tại để phục vụ các bên tiêu thụ dữ liệu và các trường hợp sử dụng ở hạ nguồn. Phần này thảo luận cách các kỹ sư dữ liệu tương tác với nhiều vai trò hạ nguồn khác nhau. Chúng tôi cũng sẽ giới thiệu một vài mô hình dịch vụ, bao gồm các đội ngũ kỹ thuật dữ liệu tập trung và các đội ngũ liên chức năng.

Data scientists Data scientists build forward-looking models to make predictions and recommendations. These models are then evaluated on live data to provide value in various ways. For example, model scoring might determine automated actions in response to real-time conditions, recommend products to customers based on the browsing history in their current session, or make live economic predictions used by traders. According to common industry folklore, data scientists spend 70% to 80% of their time collecting, cleaning, and preparing data. In our experience, 12 these numbers often reflect immature data science and data engineering practices. In particular, many popular data science frameworks can become bottlenecks if they are not scaled up appropriately. Data scientists who work exclusively on a single workstation force themselves to downsample data, making data preparation significantly more complicated and potentially compromising the quality of the models they produce. Furthermore, locally developed code and environments are often difficult to deploy in production, and a lack of automation significantly hampers data science workflows. If data engineers do their job and collaborate successfully, data scientists shouldn't spend their time collecting, cleaning, and preparing data after initial exploratory work. Data engineers should automate this work as much as possible. The need for production-ready data science is a significant driver behind the emergence of the data engineering profession. Data engineers should help data scientists to enable a path to production. In fact, we (the authors) moved from data science to data engineering after recognizing this fundamental need. Data engineers work to provide the data automation and scale that make data science more efficient.

Nhà khoa học dữ liệu Các nhà khoa học dữ liệu xây dựng các mô hình hướng tương lai để đưa ra các dự đoán và khuyến nghị. Các mô hình này sau đó được đánh giá trên dữ liệu trực tiếp để cung cấp giá trị theo nhiều cách khác nhau. Ví dụ, việc chấm điểm mô hình có thể xác định các hành động tự động hóa để đáp lại các điều kiện thời gian thực, gợi ý sản phẩm cho khách hàng dựa trên lịch sử duyệt web trong phiên hiện tại của họ, hoặc đưa ra các dự đoán kinh tế trực tiếp được các nhà giao dịch sử dụng. Theo truyền thuyết phổ biến trong ngành, các nhà khoa học dữ liệu dành 70% đến 80% thời gian của họ để thu thập, làm sạch, và chuẩn bị dữ liệu. Theo kinh nghiệm của chúng tôi, 12 những con số này thường phản ánh các thực hành khoa học dữ liệu và kỹ thuật dữ liệu non nớt. Cụ thể, nhiều framework khoa học dữ liệu phổ biến có thể trở thành các nút thắt cổ chai nếu chúng không được mở rộng quy mô một cách phù hợp. Các nhà khoa học dữ liệu làm việc độc nhất trên một máy trạm đơn lẻ tự ép mình phải lấy mẫu giảm (downsample) dữ liệu, khiến cho việc chuẩn bị dữ liệu phức tạp hơn đáng kể và có khả năng làm tổn hại chất lượng của các mô hình mà họ tạo ra. Hơn nữa, mã và môi trường được phát triển

cục bộ thường khó triển khai trong môi trường vận hành, và việc thiếu tự động hóa cản trở đáng kể các quy trình làm việc khoa học dữ liệu. Nếu các kỹ sư dữ liệu làm tốt công việc của họ và cộng tác thành công, các nhà khoa học dữ liệu không nên dành thời gian của họ để thu thập, làm sạch, và chuẩn bị dữ liệu sau công việc khám phá ban đầu. Các kỹ sư dữ liệu nên tự động hóa công việc này hết mức có thể. Nhu cầu về khoa học dữ liệu sẵn-sàng-cho-vận-hành là một động lực đáng kể đằng sau sự ra đời của nghề kỹ thuật dữ liệu. Các kỹ sư dữ liệu nên giúp các nhà khoa học dữ liệu mở ra một con đường tới môi trường vận hành. Thực tế, chúng tôi (các tác giả) đã chuyển từ khoa học dữ liệu sang kỹ thuật dữ liệu sau khi nhận ra nhu cầu cơ bản này. Các kỹ sư dữ liệu làm việc để cung cấp sự tự động hóa dữ liệu và quy mô làm cho khoa học dữ liệu hiệu quả hơn.

Data analysts Data analysts (or business analysts) seek to understand business performance and trends. Whereas data scientists are forward-looking, a data analyst typically focuses on the past or present. Data analysts usually run SQL queries in a data warehouse or a **data lake**. They may also utilize spreadsheets for computation and analysis and various BI tools such as Microsoft Power BI, Looker, or Tableau. Data analysts are domain experts in the data they work with frequently and become intimately familiar with data definitions, characteristics, and quality problems. A data analyst's typical downstream customers are business users, management, and executives. Data engineers work with data analysts to build pipelines for new data sources required by the business. Data analysts' subject-matter expertise is invaluable in improving data quality, and they frequently collaborate with data engineers in this capacity.

Nhà phân tích dữ liệu Các nhà phân tích dữ liệu (hay nhà phân tích nghiệp vụ) tìm cách hiểu hiệu suất và xu hướng của doanh nghiệp. Trong khi các nhà khoa học dữ liệu hướng về tương lai, một nhà phân tích dữ liệu thường tập trung vào quá khứ hoặc hiện tại. Các nhà phân tích dữ liệu thường chạy các truy vấn SQL trong một kho dữ liệu hoặc một hồ dữ liệu. Họ cũng có thể tận dụng các bảng tính cho việc tính toán và phân tích và nhiều công cụ BI khác nhau như Microsoft Power BI, Looker, hoặc Tableau. Các nhà phân tích dữ liệu là các chuyên gia lĩnh vực về dữ liệu mà họ làm việc thường xuyên và trở nên am hiểu sâu sắc về các định nghĩa, đặc tính, và các vấn đề chất lượng của dữ liệu. Các khách hàng hạ nguồn điển hình của một nhà phân tích dữ liệu là người dùng nghiệp vụ, ban quản lý, và các lãnh đạo điều hành. Các kỹ sư dữ liệu làm việc với các nhà phân tích dữ liệu để xây dựng các đường ống cho các nguồn dữ liệu mới mà doanh nghiệp yêu cầu. Chuyên môn về lĩnh vực của các nhà phân tích dữ liệu là vô giá trong việc cải thiện chất lượng dữ liệu, và họ thường cộng tác với các kỹ sư dữ liệu với năng lực này.

Machine learning engineers and AI researchers Machine learning engineers (ML engineers) overlap with data engineers and data scientists. ML engineers develop advanced ML techniques, train models, and design and maintain the infrastructure running ML processes in a scaled production environment. ML engineers often have advanced working knowledge of ML and deep learning techniques, and frameworks such as PyTorch or TensorFlow. ML engineers also understand the hardware, services, and systems required to run these frameworks, both for model training and model deployment at a production scale. It's common for ML flows to run in a cloud environment where ML engineers can spin up and scale infrastructure resources on demand or rely on managed services. As we've mentioned, the boundaries between ML engineering, data engineering, and data science are blurry. Data engineers may have some DevOps responsibilities over ML systems, and data scientists may work closely with ML engineering in designing advanced ML processes. The world of ML engineering is snowballing and parallels a lot of the same developments occurring in data engineering. Whereas several years ago, the attention of ML was focused on how to build models, ML engineering now increasingly emphasizes incorporating best practices of machine learning operations (**MLOps**) and other mature practices previously adopted in software engineering and DevOps.

Kỹ sư học máy và nhà nghiên cứu AI Các kỹ sư học máy (kỹ sư ML) chồng lấn với các kỹ sư dữ liệu và nhà khoa học dữ liệu. Các kỹ sư ML phát triển các kỹ thuật ML nâng cao, huấn luyện các mô hình, và thiết kế cùng bảo trì hạ tầng chạy các tiến trình ML trong một môi trường vận hành có quy mô. Các kỹ sư ML thường có kiến thức làm việc nâng cao về các kỹ thuật ML và học sâu, và các framework như PyTorch hoặc TensorFlow. Các kỹ sư ML cũng hiểu phần cứng, dịch vụ, và hệ thống cần thiết để chạy các framework này, cho cả việc huấn luyện mô hình và triển khai mô hình ở quy mô vận hành. Việc các luồng ML chạy trong một môi trường đám mây nơi các kỹ sư ML có thể khởi tạo và mở rộng tài nguyên hạ tầng theo nhu cầu hoặc dựa vào các dịch vụ được quản lý là điều phổ biến. Như chúng tôi đã đề cập, các ranh giới giữa kỹ thuật ML, kỹ thuật dữ liệu, và khoa học dữ liệu là mờ nhạt. Các kỹ sư dữ liệu có thể có một số trách nhiệm DevOps đối với các hệ thống ML, và các nhà khoa học dữ liệu có thể làm việc sát cánh với kỹ thuật ML trong việc thiết kế các tiến trình ML nâng cao. Thế giới của kỹ thuật ML đang phình to nhanh chóng và song hành với rất nhiều phát triển tương tự đang diễn ra trong kỹ thuật dữ liệu. Trong khi vài năm trước, sự chú ý của ML tập trung vào việc làm thế nào để xây dựng các mô hình, kỹ thuật ML giờ đây ngày càng nhấn mạnh việc kết hợp các thực hành tốt nhất của vận hành học máy (MLOps) và các thực hành trưởng thành khác trước đây đã được áp dụng trong kỹ thuật phần mềm và DevOps.

AI researchers work on new, advanced ML techniques. AI researchers may work inside large technology companies, specialized intellectual property startups (OpenAI, DeepMind), or academic institutions. Some practitioners are dedicated to part-time research in conjunction with ML engineering responsibilities inside a company. Those working inside specialized ML labs are often 100% dedicated to research. Research problems may target immediate practical applications or more abstract demonstrations of AI. AlphaGo and GPT-3/GPT-4 are great examples of ML research projects. We've provided some references in "Additional Resources". AI researchers in well-funded organizations are highly specialized and operate with supporting teams of engineers to facilitate their work. ML engineers in academia usually have fewer resources but rely on teams of graduate students, postdocs, and university staff to provide engineering support. ML engineers who are partially dedicated to research often rely on the same support teams for research and production.

Các nhà nghiên cứu AI làm việc trên các kỹ thuật ML mới, nâng cao. Các nhà nghiên cứu AI có thể làm việc bên trong các công ty công nghệ lớn, các startup chuyên về sở hữu trí tuệ (OpenAI, DeepMind), hoặc các tổ chức học thuật. Một số người hành nghề chuyên tâm nghiên cứu bán thời gian kết hợp với các trách nhiệm kỹ thuật ML bên trong một công ty. Những người làm việc bên trong các phòng thí nghiệm ML chuyên biệt thường dành 100% cho nghiên cứu. Các bài toán nghiên cứu có thể nhắm tới các ứng dụng thực tiễn tức thời hoặc các minh chứng trừu tượng hơn về AI. AlphaGo và GPT-3/GPT-4 là những ví dụ tuyệt vời về các dự án nghiên cứu ML. Chúng tôi đã cung cấp một số tài liệu tham khảo trong "Additional Resources". Các nhà nghiên cứu AI trong các tổ chức được tài trợ tốt là cực kỳ chuyên biệt và vận hành với các đội ngũ kỹ sư hỗ trợ để tạo điều kiện cho công việc của họ. Các kỹ sư ML trong giới học thuật thường có ít nguồn lực hơn nhưng dựa vào các đội ngũ nghiên cứu sinh, nghiên cứu viên sau tiến sĩ, và nhân viên đại học để cung cấp hỗ trợ kỹ thuật. Các kỹ sư ML chuyên tâm một phần cho nghiên cứu thường dựa vào cùng các đội ngũ hỗ trợ cho cả nghiên cứu và vận hành.

Data Engineers and Business Leadership — Kỹ sư Dữ liệu và Ban Lãnh đạo Doanh nghiệp

We've discussed technical roles with which a data engineer interacts. But data engineers also operate more broadly as organizational connectors, often in a nontechnical capacity. Businesses have come to rely increasingly on data as a core part of many products or a product in itself. Data engineers now participate in strategic planning and lead key initiatives that extend beyond the boundaries of IT. Data engineers often support data architects by acting as the glue between the business and data science/analytics.

Chúng tôi đã thảo luận các vai trò kỹ thuật mà một kỹ sư dữ liệu tương tác. Nhưng các kỹ sư dữ liệu cũng vận hành rộng hơn như những người kết nối tổ chức, thường với năng lực không chuyên kỹ thuật. Các doanh nghiệp đã ngày càng dựa vào dữ liệu như một phần cốt lõi của nhiều sản phẩm hoặc như một sản phẩm tự thân. Các kỹ sư dữ liệu giờ đây tham gia vào việc lập kế hoạch chiến lược và dẫn dắt các sáng kiến chủ chốt vượt ra ngoài ranh giới của IT. Các kỹ sư dữ liệu thường hỗ trợ các kiến trúc sư dữ liệu bằng cách đóng vai trò như chất keo giữa nghiệp vụ và khoa học dữ liệu/phân tích.

Data in the C-suite — Dữ liệu trong Ban C-suite

C-level executives are increasingly involved in data and analytics, as these are recognized as significant assets for modern businesses. CEOs now concern themselves with initiatives that were once the exclusive province of IT, such as cloud migrations or deployment of a new customer **data platform**.

Các lãnh đạo điều hành cấp C ngày càng tham gia vào dữ liệu và phân tích, vì những thứ này được công nhận là các tài sản quan trọng đối với các doanh nghiệp hiện đại. Các CEO giờ đây tự bận tâm với các sáng kiến mà từng là lĩnh vực độc quyền của IT, chẳng hạn như các cuộc di trú lên đám mây hoặc việc triển khai một nền tảng dữ liệu khách hàng mới.

Chief executive officer Chief executive officers (CEOs) at nontech companies generally don't concern themselves with the nitty-gritty of data frameworks and software. Instead, they define a vision in collaboration with technical C-suite roles and company data leadership. Data engineers provide a window into what's possible with data. Data engineers and their managers maintain a map of what data is available to the organization—both internally and from third parties—in what time frame. They are also tasked to study primary data architectural changes in collaboration with other engineering roles. For example, data engineers are often heavily involved in cloud migrations, migrations to new data systems, or deployment of streaming technologies.

Giám đốc điều hành Các giám đốc điều hành (CEO) tại các công ty không chuyên công nghệ nói chung không tự bận tâm với những chi tiết tỉ mỉ của các framework dữ liệu và phần mềm. Thay vào đó, họ định nghĩa một tầm nhìn trong sự cộng tác với các vai trò C-suite kỹ thuật và ban lãnh đạo dữ liệu của công ty. Các kỹ sư dữ liệu cung cấp một ô cửa sổ nhìn vào những gì có thể làm được với dữ liệu. Các kỹ sư dữ liệu và các quản lý của họ duy trì một bản đồ về dữ liệu nào sẵn có cho tổ chức—cả nội bộ lẫn từ các bên thứ ba—trong khung thời gian nào. Họ cũng được giao nhiệm vụ nghiên cứu các thay đổi kiến trúc dữ liệu quan trọng trong sự cộng tác với các vai trò kỹ thuật khác. Ví dụ, các kỹ sư dữ liệu thường tham gia sâu vào các cuộc di trú lên đám mây, di trú sang các hệ thống dữ liệu mới, hoặc triển khai các công nghệ theo luồng.

Chief information officer A chief information officer (CIO) is the senior C-suite executive responsible

for information technology within an organization; it is an internal-facing role. A CIO must possess deep knowledge of information technology and business processes—either alone is insufficient. CIOs direct the information technology organization, setting ongoing policies while also defining and executing significant initiatives under the direction of the CEO. CIOs often collaborate with data engineering leadership in organizations with a well-developed data culture. If an organization is not very high in its data maturity, a CIO will typically help shape its data culture. CIOs will work with engineers and architects to map out major initiatives and make strategic decisions on adopting major architectural elements, such as enterprise resource planning (ERP) and customer relationship management (CRM) systems, cloud migrations, data systems, and internal-facing IT.

Giám đốc thông tin Một giám đốc thông tin (CIO) là lãnh đạo điều hành C-suite cấp cao chịu trách nhiệm về công nghệ thông tin bên trong một tổ chức; đó là một vai trò hướng nội bộ. Một CIO phải sở hữu kiến thức sâu về công nghệ thông tin và các quy trình nghiệp vụ—một mình thứ nào thôi là không đủ. Các CIO chỉ đạo tổ chức công nghệ thông tin, đặt ra các chính sách đang diễn ra trong khi cũng định nghĩa và thực thi các sáng kiến quan trọng dưới sự chỉ đạo của CEO. Các CIO thường cộng tác với ban lãnh đạo kỹ thuật dữ liệu trong các tổ chức có một văn hóa dữ liệu phát triển tốt. Nếu một tổ chức không cao lắm về độ trưởng thành dữ liệu, một CIO thường sẽ giúp định hình văn hóa dữ liệu của nó. Các CIO sẽ làm việc với các kỹ sư và kiến trúc sư để vạch ra các sáng kiến lớn và đưa ra các quyết định chiến lược về việc áp dụng các yếu tố kiến trúc lớn, chẳng hạn như các hệ thống hoạch định nguồn lực doanh nghiệp (ERP) và quản lý quan hệ khách hàng (CRM), các cuộc di trú lên đám mây, các hệ thống dữ liệu, và IT hướng nội bộ.

Chief technology officer A chief technology officer (CTO) is similar to a CIO but faces outward. A CTO owns the key technological strategy and architectures for external-facing applications, such as mobile, web apps, and IoT—all critical data sources for data engineers. The CTO is likely a skilled technologist and has a good sense of software engineering fundamentals and system architecture. In some organizations without a CIO, the CTO or sometimes the chief operating officer (COO) plays the role of CIO. Data engineers often report directly or indirectly through a CTO.

Giám đốc công nghệ Một giám đốc công nghệ (CTO) tương tự như một CIO nhưng hướng ra ngoài. Một CTO sở hữu chiến lược công nghệ và các kiến trúc then chốt cho các ứng dụng hướng bên ngoài, chẳng hạn như di động, ứng dụng web, và IoT—tất cả đều là các nguồn dữ liệu trọng yếu cho các kỹ sư dữ liệu. CTO có lẽ là một nhà công nghệ lành nghề và có một cảm nhận tốt về các nền tảng cơ bản của kỹ thuật phần mềm và kiến trúc hệ thống. Trong một số tổ chức không có CIO, CTO hoặc đôi khi giám đốc vận hành (COO) đóng vai trò của CIO. Các kỹ sư dữ liệu thường báo cáo trực tiếp hoặc gián tiếp thông qua một CTO.

Chief data officer The chief data officer (CDO) was created in 2002 at Capital One to recognize the growing importance of data as a business asset. The CDO is responsible for a company's data assets and strategy. CDOs are focused on data's business utility but should have a strong technical grounding. CDOs oversee data products, strategy, initiatives, and core functions such as **master data management** and privacy. Occasionally, CDOs manage business analytics and data engineering.

Giám đốc dữ liệu Chức danh giám đốc dữ liệu (CDO) được tạo ra vào năm 2002 tại Capital One để công nhận tầm quan trọng đang tăng lên của dữ liệu như một tài sản nghiệp vụ. CDO chịu trách nhiệm về các tài sản dữ liệu và chiến lược dữ liệu của một công ty. Các CDO tập trung vào tính hữu dụng nghiệp vụ của dữ liệu nhưng nên có một nền tảng kỹ thuật vững chắc. Các CDO giám sát các sản phẩm dữ liệu, chiến lược, sáng kiến, và các chức năng cốt lõi như quản lý dữ liệu chủ (MDM) và quyền riêng tư. Thông thường, các CDO quản lý phân tích nghiệp vụ và kỹ thuật dữ liệu.

Chief analytics officer The chief analytics officer (CAO) is a variant of the CDO role. Where both

roles exist, the CDO focuses on the technology and organization required to deliver data. The CAO is responsible for analytics, strategy, and decision making for the business. A CAO may oversee data science and ML, though this largely depends on whether the company has a CDO or CTO role.

Giám đốc phân tích Giám đốc phân tích (CAO) là một biến thể của vai trò CDO. Nơi cả hai vai trò cùng tồn tại, CDO tập trung vào công nghệ và tổ chức cần thiết để cung cấp dữ liệu. CAO chịu trách nhiệm về phân tích, chiến lược, và việc ra quyết định cho doanh nghiệp. Một CAO có thể giám sát khoa học dữ liệu và ML, mặc dù điều này phần lớn phụ thuộc vào việc công ty có một vai trò CDO hay CTO hay không.

Chief algorithms officer A chief algorithms officer (CAO-2) is a recent innovation in the C-suite, a highly technical role focused specifically on data science and ML. CAO-2s typically have experience as individual contributors and team leads in data science or ML projects. Frequently, they have a background in ML research and a related advanced degree. CAO-2s are expected to be conversant in current ML research and have deep technical knowledge of their company's ML initiatives. In addition to creating business initiatives, they provide technical leadership, set research and development agendas, and build research teams.

Giám đốc thuật toán Một giám đốc thuật toán (CAO-2) là một đổi mới gần đây trong ban C-suite, một vai trò có tính kỹ thuật cao tập trung cụ thể vào khoa học dữ liệu và ML. Các CAO-2 thường có kinh nghiệm với tư cách là cá nhân đóng góp và trưởng nhóm trong các dự án khoa học dữ liệu hoặc ML. Thường thì họ có một nền tảng về nghiên cứu ML và một bằng cấp nâng cao liên quan. Các CAO-2 được kỳ vọng thông thạo nghiên cứu ML hiện thời và có kiến thức kỹ thuật sâu về các sáng kiến ML của công ty họ. Bên cạnh việc tạo ra các sáng kiến nghiệp vụ, họ cung cấp sự lãnh đạo kỹ thuật, đặt ra các chương trình nghị sự nghiên cứu và phát triển, và xây dựng các đội ngũ nghiên cứu.

Data engineers and project managers — Kỹ sư dữ liệu và các quản lý dự án

Data engineers often work on significant initiatives, potentially spanning many years. As we write this book, many data engineers are working on cloud migrations, migrating pipelines and warehouses to the next generation of data tools. Other data engineers are starting greenfield projects, assembling new data architectures from scratch by selecting from an astonishing number of best-of-breed architecture and tooling options. These large initiatives often benefit from project management (in contrast to product management, discussed next). Whereas data engineers function in an infrastructure and service delivery capacity, project managers direct traffic and serve as gatekeepers. Most project managers operate according to some variation of Agile and Scrum, with Waterfall still appearing occasionally. Business never sleeps, and business stakeholders often have a significant backlog of things they want to address and new initiatives they want to launch. Project managers must filter a long list of requests and prioritize critical deliverables to keep projects on track and better serve the company. Data engineers interact with project managers, often planning sprints for projects and ensuing standups related to the sprint. Feedback goes both ways, with data engineers informing project managers and other stakeholders about progress and blockers, and project managers balancing the cadence of technology teams against the ever-changing needs of the business.

Các kỹ sư dữ liệu thường làm việc trên các sáng kiến quan trọng, có khả năng kéo dài nhiều năm. Khi chúng tôi viết cuốn sách này, nhiều kỹ sư dữ liệu đang làm việc trên các cuộc di trú lên đám mây, di trú các đường ống và kho dữ liệu sang thế hệ công cụ dữ liệu tiếp theo. Các kỹ sư dữ liệu khác đang khởi động các dự án greenfield, lắp ráp các kiến

trúc dữ liệu mới từ đầu bằng cách lựa chọn từ một số lượng đáng kinh ngạc các lựa chọn kiến trúc và công cụ tốt nhất trong loại. Các sáng kiến lớn này thường hưởng lợi từ quản lý dự án (tương phản với quản lý sản phẩm, được thảo luận tiếp theo). Trong khi các kỹ sư dữ liệu vận hành với năng lực cung cấp hạ tầng và dịch vụ, các quản lý dự án điều hướng luồng công việc và đóng vai trò người gác cổng. Hầu hết các quản lý dự án vận hành theo một biến thể nào đó của Agile và Scrum, với Waterfall vẫn xuất hiện thỉnh thoảng. Doanh nghiệp không bao giờ ngủ, và các bên liên quan nghiệp vụ thường có một tồn đọng đáng kể những thứ họ muốn giải quyết và các sáng kiến mới họ muốn khởi động. Các quản lý dự án phải sàng lọc một danh sách dài các yêu cầu và ưu tiên hóa các sản phẩm bàn giao trọng yếu để giữ cho các dự án đi đúng hướng và phục vụ công ty tốt hơn. Các kỹ sư dữ liệu tương tác với các quản lý dự án, thường lập kế hoạch các sprint cho các dự án và các buổi standup tiếp theo liên quan đến sprint. Phản hồi đi theo cả hai chiều, với các kỹ sư dữ liệu thông báo cho các quản lý dự án và các bên liên quan khác về tiến độ và các điểm nghẽn, và các quản lý dự án cân bằng nhịp độ của các đội ngũ công nghệ với các nhu cầu luôn thay đổi của doanh nghiệp.

Data engineers and product managers — Kỹ sư dữ liệu và các quản lý sản phẩm

Product managers oversee product development, often owning product lines. In the context of data engineers, these products are called data products . Data products are either built from the ground up or are incremental improvements upon existing products. Data engineers interact more frequently with product managers as the corporate world has adopted a data-centric focus . Like project managers, product managers balance the activity of technology teams against the needs of the customer and business.

Các quản lý sản phẩm giám sát việc phát triển sản phẩm, thường sở hữu các dòng sản phẩm. Trong bối cảnh các kỹ sư dữ liệu, các sản phẩm này được gọi là các sản phẩm dữ liệu (data product). Các sản phẩm dữ liệu hoặc được xây dựng từ con số không hoặc là các cải tiến gia tăng dựa trên các sản phẩm hiện có. Các kỹ sư dữ liệu tương tác thường xuyên hơn với các quản lý sản phẩm khi giới doanh nghiệp đã áp dụng một trọng tâm lấy-dữ-liệu-làm-trung-tâm. Giống như các quản lý dự án, các quản lý sản phẩm cân bằng hoạt động của các đội ngũ công nghệ với các nhu cầu của khách hàng và doanh nghiệp.

Data engineers and other management roles — Kỹ sư dữ liệu và các vai trò quản lý khác

Data engineers interact with various managers beyond project and product managers. However, these interactions usually follow either the services or cross-functional models. Data engineers either serve a variety of incoming requests as a centralized team or work as a resource assigned to a particular manager, project, or product. For more information on data teams and how to structure them, we recommend John Thompson's Building Analytics Teams (Packt) and Jesse Anderson's Data Teams (Apress). Both books provide strong frameworks and perspectives on the roles of executives with data, who to hire, and how to construct the most effective data team for your company.

Các kỹ sư dữ liệu tương tác với nhiều quản lý khác nhau vượt ra ngoài các quản lý dự án và sản phẩm. Tuy nhiên, các tương tác này thường tuân theo hoặc mô hình dịch vụ hoặc mô hình liên chức năng. Các kỹ sư dữ liệu hoặc phục vụ nhiều yêu cầu đến với tư cách

một đội ngũ tập trung hoặc làm việc như một nguồn lực được phân bổ cho một quản lý, dự án, hoặc sản phẩm cụ thể. Để biết thêm thông tin về các đội ngũ dữ liệu và cách cấu trúc chúng, chúng tôi khuyến nghị cuốn *Building Analytics Teams* (Packt) của John Thompson và cuốn *Data Teams* (Apress) của Jesse Anderson. Cả hai cuốn sách đều cung cấp các khung và góc nhìn mạnh mẽ về các vai trò của các lãnh đạo điều hành với dữ liệu, tuyển ai, và cách xây dựng đội ngũ dữ liệu hiệu quả nhất cho công ty của bạn.

NOTE — LƯU Ý

Companies don't hire engineers simply to hack on code in isolation. To be worthy of their title, engineers should develop a deep understanding of the problems they're tasked with solving, the technology tools at their disposal, and the people they work with and serve.

Các công ty không tuyển các kỹ sư đơn thuần để vọc mã trong sự cô lập. Để xứng đáng với chức danh của mình, các kỹ sư nên phát triển một sự hiểu biết sâu sắc về các bài toán mà họ được giao giải quyết, các công cụ công nghệ trong tầm tay họ, và những người mà họ làm việc cùng và phục vụ.

Conclusion — Kết luận

This chapter provided you with a brief overview of the data engineering landscape, including the following:

Chương này đã cung cấp cho bạn một cái nhìn tổng quan ngắn gọn về bức tranh kỹ thuật dữ liệu, bao gồm những điều sau:

Defining data engineering and describing what data engineers do

Định nghĩa kỹ thuật dữ liệu và mô tả những gì các kỹ sư dữ liệu làm

Describing the types of data maturity in a company

Mô tả các loại độ trưởng thành dữ liệu trong một công ty

Type A and type B data engineers

Các kỹ sư dữ liệu loại A và loại B

Whom data engineers work with

Các kỹ sư dữ liệu làm việc cùng những ai

We hope that this first chapter has whetted your appetite, whether you are a software development practitioner, data scientist, ML engineer, business stakeholder, entrepreneur, or venture capitalist. Of course, a great deal still remains to elucidate in subsequent chapters. Chapter 2 covers the data engineering lifecycle, followed by architecture in Chapter 3. The following chapters get into the nitty-gritty of technology decisions for each part of the lifecycle. The entire data field is in flux, and as much as possible, each chapter focuses on the immutables —perspectives that will be valid for many years amid relentless change.

Chúng tôi hy vọng rằng chương đầu tiên này đã khơi gợi sự thèm khát của bạn, bất kể bạn là một người hành nghề phát triển phần mềm, nhà khoa học dữ liệu, kỹ sư ML, bên liên quan nghiệp vụ, doanh nhân, hay nhà đầu tư mạo hiểm. Tất nhiên, còn rất nhiều điều vẫn cần được làm sáng tỏ trong các chương tiếp theo. Chương 2 trình bày vòng đời kỹ thuật dữ liệu, tiếp theo là kiến trúc ở Chương 3. Các chương sau đi vào những chi tiết tỉ mỉ của

các quyết định công nghệ cho mỗi phần của vòng đời. Toàn bộ lĩnh vực dữ liệu đang trong trạng thái biến động, và hết mức có thể, mỗi chương tập trung vào những điều bất biến (immutable)—những góc nhìn sẽ còn hiệu lực trong nhiều năm giữa sự thay đổi không ngừng.

Additional Resources

“On Complexity in Big Data” by Jesse Anderson (O’Reilly)

“Which Profession Is More Complex to Become, a Data Engineer or a Data Scientist?” thread on Quora

“The Future of Data Engineering Is the Convergence of Disciplines” by Liam Hausmann

The Information Management Body of Knowledge website

“Doing Data Science at Twitter” by Robert Chang

“A Short History of Big Data” by Mark van Rijmenam

“Data Engineering: A Quick and Simple Definition” by James Furbush (O’Reilly)

“Big Data Will Be Dead in Five Years” by Lewis Gavin

“The AI Hierarchy of Needs” by Monica Rogati

Chapter 1 of What Is Data Engineering? by Lewis Gavin (O’Reilly)

“The Three Levels of Data Analysis: A Framework for Assessing Data Organization Maturity” by Emilie Schario

“Data as a Product vs. Data as a Service” by Justin Gage

“The Downfall of the Data Engineer” by Maxime Beauchemin

“The Rise of the Data Engineer” by Maxime Beauchemin

“Skills of the Data Architect” by Bob Lambert “What Is a Data Architect? IT’s Data Framework Visionary” by Thor Olavsrud

“OpenAI’s New Language Generator GPT-3 Is Shockingly Good—and Completely Mindless” by Will Douglas Heaven

The AlphaGo research web page

“How CEOs Can Lead a Data-Driven Culture” by Thomas H. Davenport and Nitin Mittal

“Why CEOs Must Lead Big Data Initiatives” by John Weathington

“How Creating a Data-Driven Culture Can Drive Success” by Frederik Bussler

Building Analytics Teams by John K. Thompson (Packt)

Data Teams by Jesse Anderson (Apress)

“Information Management Body of Knowledge” Wikipedia page

“Information management” Wikipedia page

1 “Data Engineering and Its Main Concepts,” AlexSoft, last updated August 26, 2021, <https://oreil.ly/e94py>

. 2 ETL stands for extract, transform, load, a common pattern we cover in the book. 3 Jesse Anderson,

“The Two Types of Data Engineering,” June 27, 2018, <https://oreil.ly/dxDt6> . 4 Maxime Beauchemin,

“The Rise of the Data Engineer,” January 20, 2017, <https://oreil.ly/kNDmd> . 5 Lewis Gavin, What Is

Data Engineering? (Sebastapol, CA: O'Reilly, 2020), <https://oreil.ly/ELxLi> . 6 Cade Metz, “How Yahoo Spawned Hadoop, the Future of Big Data,” *Wired* , October 18, 2011, <https://oreil.ly/iaD9G> . 7 Ron Miller, “How AWS Came to Be,” *TechCrunch* , July 2, 2016, <https://oreil.ly/VJehv> . 8 DataOps is an abbreviation for data operations . We cover this topic in Chapter 2 . For more information, read the DataOps Manifesto .

9 These acronyms stand for California Consumer Privacy Act and General Data Protection Regulation , respectively. 10 Robert Chang, “Doing Data Science at Twitter,” *Medium* , June 20, 2015, <https://oreil.ly/xqjAx> . 11 Paramita (Guha) Ghosh, “Data Architect vs. Data Engineer,” *Dataversity*, November 12, 2021, <https://oreil.ly/TlyZY> . 12 A variety of references exist for this notion. Although this cliché is widely known, a healthy debate has arisen around its validity in different practical settings. For more details, see Leigh Dodds, “Do Data Scientists Spend 80% of Their Time Cleaning Data? Turns Out, No?” *Lost Boy* blog, January 31, 2020, <https://oreil.ly/szFww> ; and Alex Woodie, “Data Prep Still Dominates Data Scientists’ Time, Survey Finds,” *Datanami* , July 6, 2020, <https://oreil.ly/jDVWF> .

Chapter 2. The Data Engineering Lifecycle — Chương 2. Vòng đời kỹ thuật dữ liệu

The major goal of this book is to encourage you to move beyond viewing **data engineering** as a specific collection of data technologies. The data landscape is undergoing an explosion of new data technologies and practices, with ever-increasing levels of abstraction and ease of use. Because of increased technical abstraction, data engineers will increasingly become data lifecycle engineers, thinking and operating in terms of the principles of **data lifecycle management**. In this chapter, you'll learn about the **data engineering lifecycle**, which is the central theme of this book. The data engineering lifecycle is our framework describing “cradle to grave” data engineering. You will also learn about the **undercurrents** of the data engineering lifecycle, which are key foundations that support all data engineering efforts.

Mục tiêu lớn của cuốn sách này là khuyến khích bạn vượt ra khỏi quan niệm xem kỹ thuật dữ liệu (data engineering) chỉ là một tập hợp cụ thể các công nghệ dữ liệu. Bối cảnh dữ liệu đang trải qua một sự bùng nổ các công nghệ và thực hành dữ liệu mới, với mức độ trừu tượng và sự dễ dùng ngày càng tăng. Vì mức độ trừu tượng kỹ thuật tăng lên, các kỹ sư dữ liệu sẽ ngày càng trở thành những kỹ sư vòng đời dữ liệu, suy nghĩ và vận hành theo các nguyên tắc quản lý vòng đời dữ liệu. Trong chương này, bạn sẽ tìm hiểu về vòng đời kỹ thuật dữ liệu, vốn là chủ đề trung tâm của cuốn sách. Vòng đời kỹ thuật dữ liệu là khung mô tả kỹ thuật dữ liệu “từ lúc sinh ra đến lúc kết thúc”. Bạn cũng sẽ tìm hiểu về các dòng chảy ngầm (undercurrents) của vòng đời kỹ thuật dữ liệu, đó là những nền tảng cốt lõi hỗ trợ mọi nỗ lực kỹ thuật dữ liệu.

What Is the Data Engineering Lifecycle? — Vòng đời kỹ thuật dữ liệu là gì?

The data engineering lifecycle comprises stages that turn raw data ingredients into a useful end product, ready for consumption by analysts, data scientists, ML engineers, and others. This chapter introduces the major stages of the data engineering lifecycle, focusing on each stage's core concepts and saving details for later chapters. We divide the data engineering lifecycle into the following five stages (Figure 2-1, top):

Vòng đời kỹ thuật dữ liệu gồm các giai đoạn biến những nguyên liệu dữ liệu thô thành một sản phẩm cuối hữu ích, sẵn sàng cho các nhà phân tích, nhà khoa học dữ liệu, kỹ sư ML và những người khác sử dụng. Chương này giới thiệu các giai đoạn chính của vòng đời kỹ

thuật dữ liệu, tập trung vào các khái niệm cốt lõi của mỗi giai đoạn và để dành chi tiết cho các chương sau. Chúng tôi chia vòng đời kỹ thuật dữ liệu thành năm giai đoạn sau (Hình 2-1, phía trên):

Generation

Sinh dữ liệu (generation)

Storage

Lưu trữ

Ingestion Transformation

Thu nạp (ingestion) Biến đổi (transformation)

Serving data

Phục vụ dữ liệu

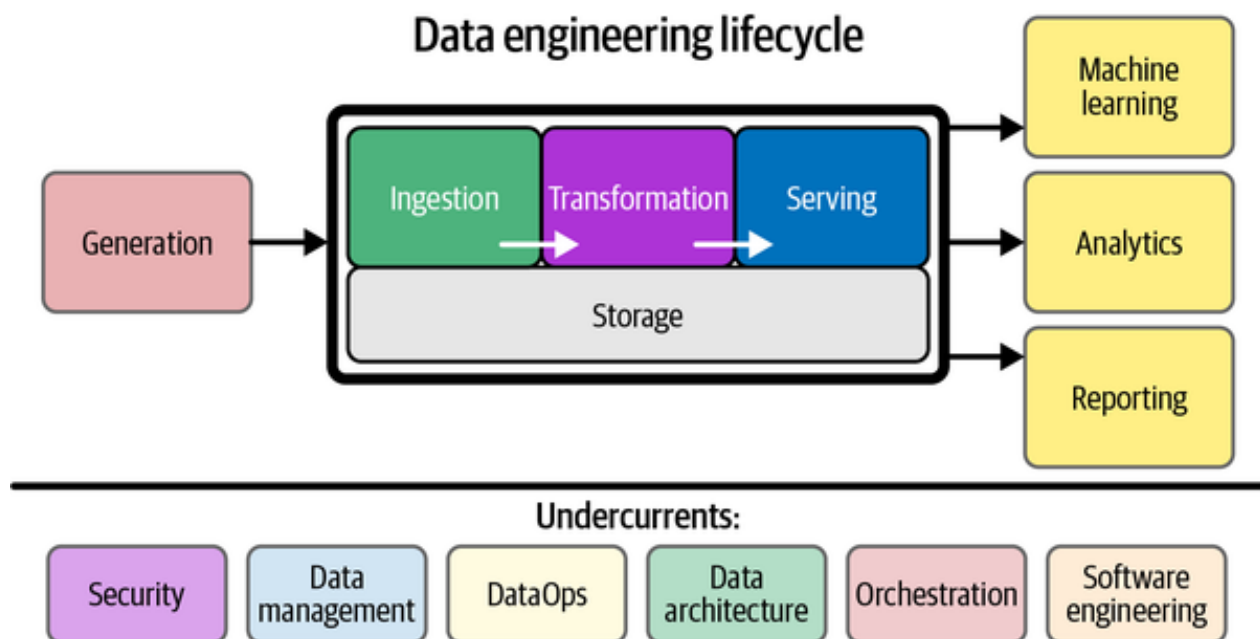


Figure 2-1. Components and undercurrents of the data engineering lifecycle We begin the data engineering lifecycle by getting data from source systems and storing it. Next, we transform the data and then proceed to our central goal, serving data to analysts, data scientists, ML engineers, and others. In reality, storage occurs throughout the lifecycle as data flows from beginning to end—hence, the diagram shows the storage “stage” as a foundation that underpins other stages. In general, the middle stages—storage, ingestion, transformation—can get a bit jumbled. And that’s OK. Although we split out the distinct parts of the data engineering lifecycle, it’s not always a neat, continuous flow. Various stages of the lifecycle may repeat themselves, occur out of order, overlap, or weave together in interesting and unexpected ways. Acting as a bedrock are undercurrents (Figure 2-1 , bottom) that cut across multiple stages of the data engineering lifecycle: security, data management, **DataOps**, **data architecture**, **orchestration**, and software engineering. No part of the data engineering lifecycle can adequately function without these undercurrents.

Hình 2-1. Các thành phần và dòng chảy ngầm của vòng đời kỹ thuật dữ liệu. Chúng ta bắt đầu vòng đời kỹ thuật dữ liệu bằng việc lấy dữ liệu từ các hệ thống nguồn (source system) và lưu trữ nó. Tiếp theo, chúng ta biến đổi dữ liệu rồi tiến tới mục tiêu trung tâm là phục

vụ (serving) dữ liệu cho các nhà phân tích, nhà khoa học dữ liệu, kỹ sư ML và những người khác. Trong thực tế, lưu trữ diễn ra xuyên suốt vòng đời khi dữ liệu chảy từ đầu đến cuối — vì vậy sơ đồ thể hiện “giai đoạn” lưu trữ như một nền tảng làm bệ đỡ cho các giai đoạn khác. Nhìn chung, các giai đoạn ở giữa — lưu trữ, thu nạp, biến đổi — có thể hơi lẫn lộn với nhau. Và điều đó không sao cả. Mặc dù chúng tôi tách riêng các phần khác biệt của vòng đời kỹ thuật dữ liệu, nó không phải lúc nào cũng là một dòng chảy gọn gàng, liên tục. Các giai đoạn khác nhau của vòng đời có thể lặp lại, diễn ra không theo thứ tự, chồng lấn, hoặc đan xen với nhau theo những cách thú vị và bất ngờ. Đóng vai trò nền móng là các dòng chảy ngầm (Hình 2-1, phía dưới) cắt ngang nhiều giai đoạn của vòng đời kỹ thuật dữ liệu: an ninh, quản lý dữ liệu, DataOps, kiến trúc dữ liệu, điều phối (orchestration) và kỹ thuật phần mềm. Không phần nào của vòng đời kỹ thuật dữ liệu có thể vận hành đầy đủ nếu thiếu các dòng chảy ngầm này.

The Data Lifecycle Versus the Data Engineering Lifecycle — Vòng đời dữ liệu so với vòng đời kỹ thuật dữ liệu

You may be wondering about the difference between the overall data lifecycle and the data engineering lifecycle. There’s a subtle distinction between the two. The data engineering lifecycle is a subset of the whole data lifecycle (Figure 2-2). Whereas the full data lifecycle encompasses data across its entire lifespan, the data engineering lifecycle focuses on the stages a **data engineer** controls.

Bạn có thể đang băn khoăn về sự khác biệt giữa vòng đời dữ liệu tổng thể và vòng đời kỹ thuật dữ liệu. Có một sự phân biệt tinh tế giữa hai khái niệm này. Vòng đời kỹ thuật dữ liệu là một tập con của toàn bộ vòng đời dữ liệu (Hình 2-2). Trong khi vòng đời dữ liệu đầy đủ bao trùm dữ liệu trong toàn bộ thời gian tồn tại của nó, vòng đời kỹ thuật dữ liệu tập trung vào các giai đoạn mà một kỹ sư dữ liệu kiểm soát.

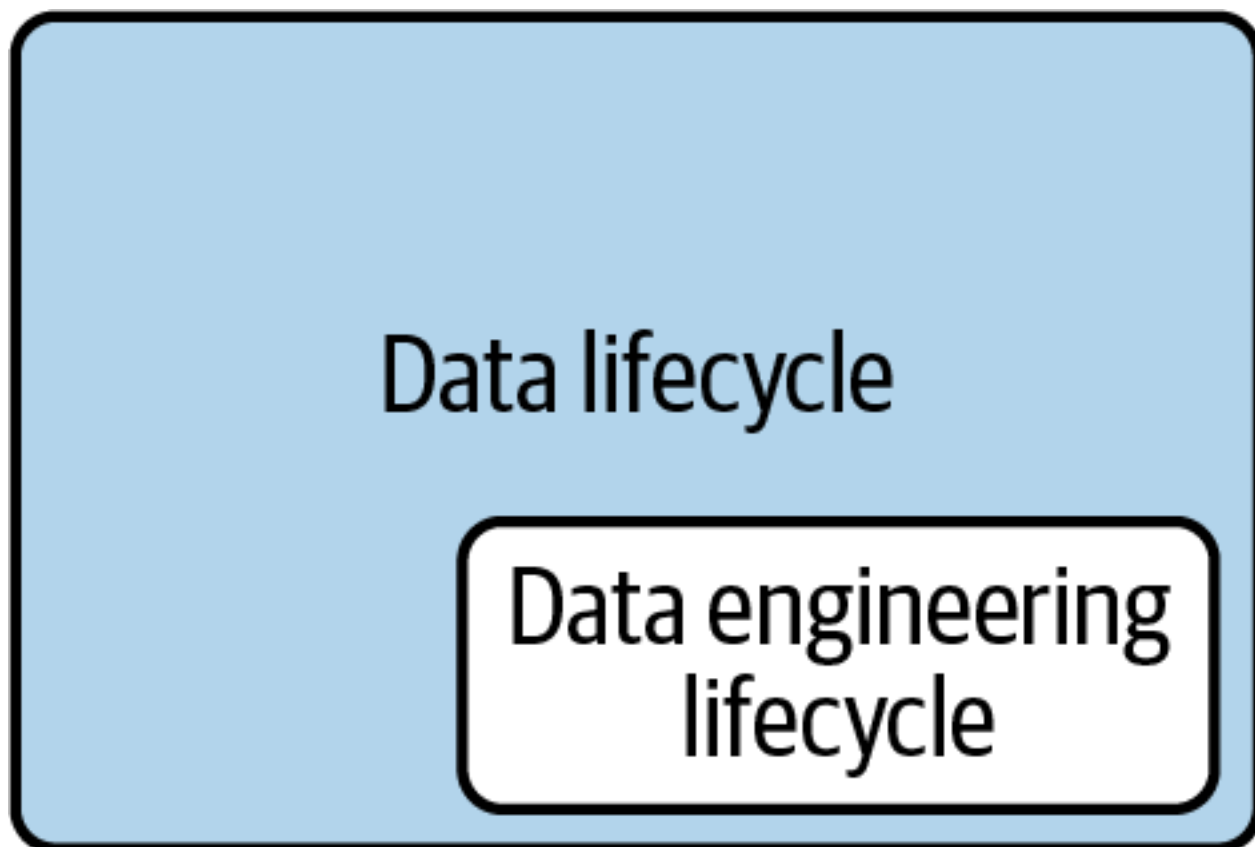


Figure 2-2. The data engineering lifecycle is a subset of the full data lifecycle

Hình 2-2. Vòng đời kỹ thuật dữ liệu là một tập con của vòng đời dữ liệu đầy đủ

Generation: Source Systems — Sinh dữ liệu: Hệ thống nguồn

A **source system** is the origin of the data used in the data engineering lifecycle. For example, a source system could be an IoT device, an application **message queue**, or a transactional database. A data engineer consumes data from a source system, but doesn't typically own or control the source system itself. The data engineer needs to have a working understanding of the way source systems work, the way they generate data, the frequency and velocity of the data, and the variety of data they generate.

Một hệ thống nguồn là nơi khởi nguồn của dữ liệu được dùng trong vòng đời kỹ thuật dữ liệu. Ví dụ, một hệ thống nguồn có thể là một thiết bị IoT, một hàng đợi tin nhắn (message queue) của ứng dụng, hoặc một cơ sở dữ liệu giao dịch. Một kỹ sư dữ liệu tiêu thụ dữ liệu từ một hệ thống nguồn, nhưng thường không sở hữu hay kiểm soát chính hệ thống nguồn đó. Kỹ sư dữ liệu cần có hiểu biết thực tiễn về cách các hệ thống nguồn hoạt động, cách chúng sinh ra dữ liệu, tần suất và vận tốc của dữ liệu, cũng như sự đa dạng của dữ liệu mà chúng sinh ra.

Engineers also need to keep an open line of communication with source system owners on changes that could break pipelines and **analytics**. Application code might change the structure of data in a field, or the application team might even choose to migrate the backend to an entirely new database technology. A major challenge in data engineering is the dizzying array of data source systems engineers must work with and understand. As an illustration, let's look at two common source systems, one very traditional (an application database) and the other a more recent example

(IoT swarms). Figure 2-3 illustrates a traditional source system with several application servers supported by a database. This source system pattern became popular in the 1980s with the explosive success of relational database management systems (RDBMSs). The application + database pattern remains popular today with various modern evolutions of software development practices. For example, applications often consist of many small service/database pairs with **microservices** rather than a single **monolith**.

Các kỹ sư cũng cần giữ một kênh liên lạc cởi mở với những người sở hữu hệ thống nguồn về các thay đổi có thể làm hỏng đường ống (pipeline) và hoạt động phân tích. Mã ứng dụng có thể thay đổi cấu trúc dữ liệu trong một trường, hoặc nhóm ứng dụng thậm chí có thể chọn di chuyển backend sang một công nghệ cơ sở dữ liệu hoàn toàn mới. Một thách thức lớn trong kỹ thuật dữ liệu là vô số hệ thống nguồn dữ liệu đến chóng mặt mà các kỹ sư phải làm việc cùng và phải hiểu. Để minh họa, hãy xem xét hai hệ thống nguồn phổ biến, một rất truyền thống (cơ sở dữ liệu ứng dụng) và một là ví dụ gần đây hơn (bầy thiết bị IoT). Hình 2-3 minh họa một hệ thống nguồn truyền thống với vài máy chủ ứng dụng được hỗ trợ bởi một cơ sở dữ liệu. Mô hình hệ thống nguồn này trở nên phổ biến vào thập niên 1980 với thành công bùng nổ của các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS). Mô hình ứng dụng + cơ sở dữ liệu vẫn còn phổ biến đến nay với nhiều biến thể tiến hóa hiện đại của thực hành phát triển phần mềm. Ví dụ, các ứng dụng thường gồm nhiều cặp dịch vụ/cơ sở dữ liệu nhỏ theo kiểu microservices thay vì một monolith duy nhất.

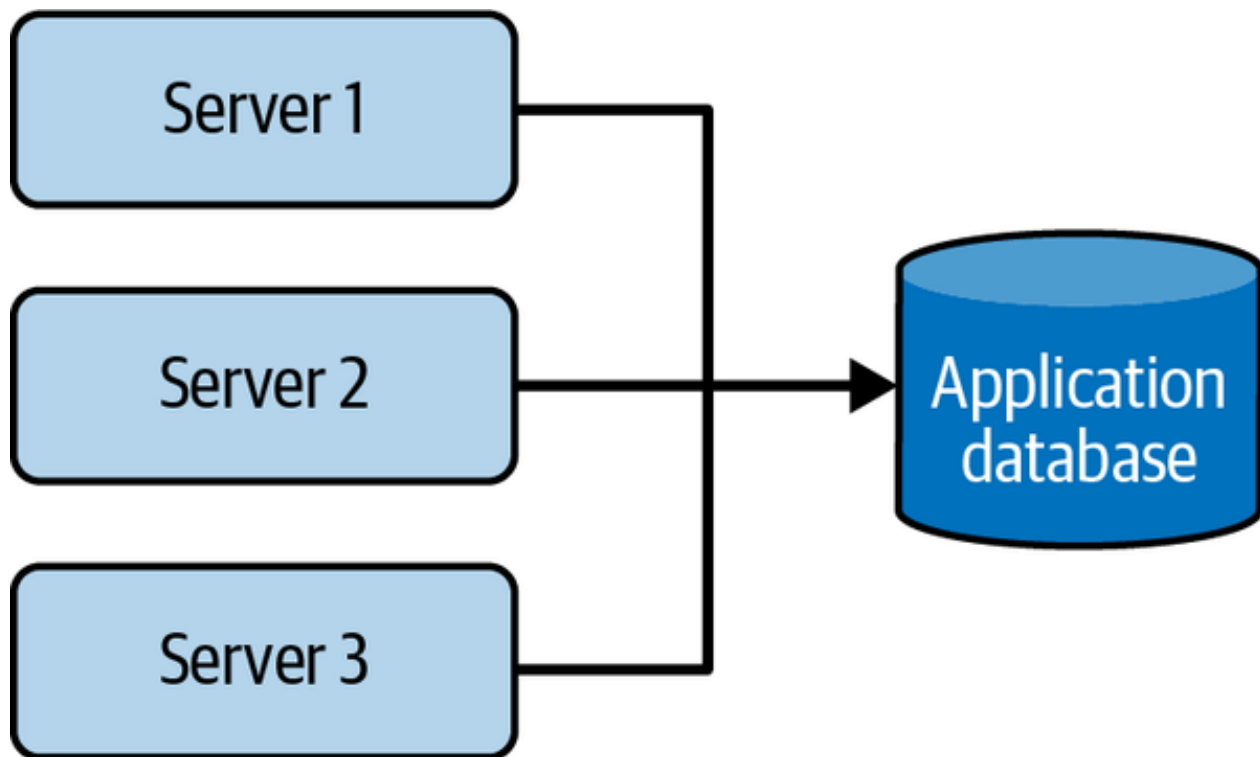


Figure 2-3. Source system example: an application database Let's look at another example of a source system. Figure 2-4 illustrates an IoT swarm: a fleet of devices (circles) sends data messages (rectangles) to a central collection system. This IoT source system is increasingly common as IoT devices such as sensors, smart devices, and much more increase in the wild.

Hình 2-3. Ví dụ về hệ thống nguồn: một cơ sở dữ liệu ứng dụng. Hãy xem một ví dụ khác về hệ thống nguồn. Hình 2-4 minh họa một bầy IoT: một đội các thiết bị (hình tròn) gửi các thông điệp dữ liệu (hình chữ nhật) đến một hệ thống thu thập trung tâm. Hệ thống

nguồn IoT này ngày càng phổ biến khi các thiết bị IoT như cảm biến, thiết bị thông minh và nhiều thứ khác gia tăng trong thực tế.

IoT swarm

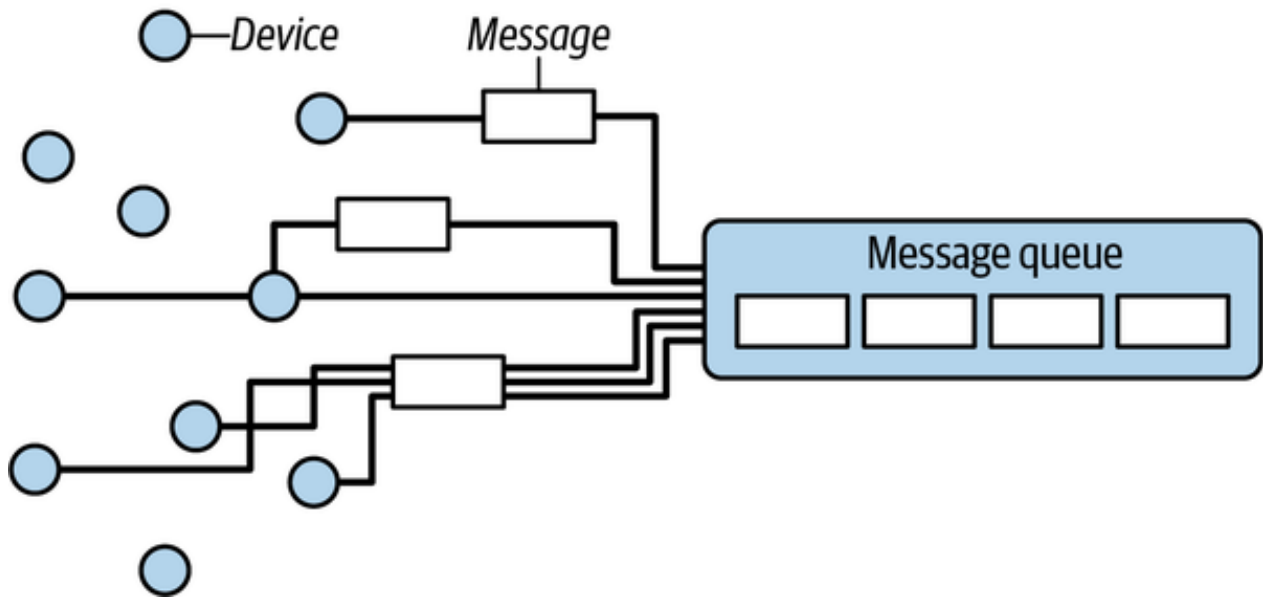


Figure 2-4. Source system example: an IoT swarm and messaging queue

Hình 2-4. Ví dụ về hệ thống nguồn: một bầy IoT và hàng đợi tin nhắn

Evaluating source systems: Key engineering considerations — Đánh giá hệ thống nguồn: Những cân nhắc kỹ thuật then chốt

There are many things to consider when assessing source systems, including how the system handles ingestion, state, and data generation. The following is a starting set of evaluation questions of source systems that data engineers must consider:

Có nhiều điều cần cân nhắc khi đánh giá các hệ thống nguồn, bao gồm cách hệ thống xử lý việc thu nạp, trạng thái và việc sinh dữ liệu. Sau đây là một tập câu hỏi đánh giá khởi đầu về các hệ thống nguồn mà kỹ sư dữ liệu phải cân nhắc:

What are the essential characteristics of the data source? Is it an application? A swarm of IoT devices?

Những đặc điểm cốt yếu của nguồn dữ liệu là gì? Nó là một ứng dụng? Một bầy thiết bị IoT?

How is data persisted in the source system? Is data persisted long term, or is it temporary and quickly deleted?

Dữ liệu được lưu giữ trong hệ thống nguồn như thế nào? Dữ liệu được lưu giữ lâu dài, hay chỉ tạm thời và nhanh chóng bị xóa?

At what rate is data generated? How many events per second? How many gigabytes per hour?

Dữ liệu được sinh ra với tốc độ nào? Bao nhiêu sự kiện mỗi giây? Bao nhiêu gigabyte mỗi giờ?

What level of consistency can data engineers expect from the output data? If you're running data-quality checks against the output data, how often do data inconsistencies occur—nulls where they aren't expected, lousy formatting, etc.?

Kỹ sư dữ liệu có thể kỳ vọng mức độ nhất quán nào ở dữ liệu đầu ra? Nếu bạn chạy các kiểm tra chất lượng dữ liệu trên dữ liệu đầu ra, các bất nhất dữ liệu xảy ra với tần suất nào — các giá trị null ở nơi không mong đợi, định dạng tệ, v.v.?

How often do errors occur?

Lỗi xảy ra với tần suất nào?

Will the data contain duplicates?

Dữ liệu có chứa bản trùng lặp không?

Will some data values arrive late, possibly much later than other messages produced simultaneously?

Liệu một số giá trị dữ liệu sẽ đến muộn, có thể muộn hơn nhiều so với các thông điệp khác được sinh ra đồng thời?

What is the **schema** of the ingested data? Will data engineers need to join across several tables or even several systems to get a complete picture of the data?

Lược đồ (schema) của dữ liệu thu nạp là gì? Kỹ sư dữ liệu có cần kết (join) qua nhiều bảng hoặc thậm chí nhiều hệ thống để có được bức tranh đầy đủ về dữ liệu không?

If schema changes (say, a new column is added), how is this dealt with and communicated to **downstream** stakeholders?

Nếu lược đồ thay đổi (ví dụ, thêm một cột mới), điều này được xử lý và truyền đạt đến các bên liên quan hạ nguồn (downstream) như thế nào?

How frequently should data be pulled from the source system?

Dữ liệu nên được kéo từ hệ thống nguồn với tần suất nào?

For stateful systems (e.g., a database tracking customer account information), is data provided as periodic snapshots or update events from **change data capture** (CDC)? What's the logic for how changes are performed, and how are these tracked in the source database?

Đối với các hệ thống có trạng thái (ví dụ, một cơ sở dữ liệu theo dõi thông tin tài khoản khách hàng), dữ liệu được cung cấp dưới dạng các ảnh chụp (snapshot) định kỳ hay các sự kiện cập nhật từ thu thập thay đổi dữ liệu (CDC)? Logic cho cách thực hiện các thay đổi là gì, và chúng được theo dõi trong cơ sở dữ liệu nguồn như thế nào?

Who/what is the data provider that will transmit the data for downstream consumption?

Ai/cái gì là nhà cung cấp dữ liệu sẽ truyền dữ liệu để tiêu thụ ở hạ nguồn?

Will reading from a data source impact its performance?

Việc đọc từ một nguồn dữ liệu có ảnh hưởng đến hiệu năng của nó không?

Does the source system have **upstream** data dependencies? What are the characteristics of these upstream systems?

Hệ thống nguồn có các phụ thuộc dữ liệu thượng nguồn (upstream) không? Đặc điểm của các hệ thống thượng nguồn này là gì?

Are data-quality checks in place to check for late or missing data?

Có sẵn các kiểm tra chất lượng dữ liệu để phát hiện dữ liệu đến muộn hoặc thiếu không?

Sources produce data consumed by downstream systems, including human-generated spreadsheets, IoT sensors, and web and mobile applications. Each source has its unique volume and cadence of data generation. A data engineer should know how the source generates data, including relevant quirks or nuances. Data engineers also need to understand the limits of the source systems they interact with. For example, will analytical queries against a source application database cause resource contention and performance issues? One of the most challenging nuances of source data is the schema. The schema defines the hierarchical organization of data. Logically, we can think of data at the level of a whole source system, drilling down into individual tables, all the way to the structure of respective fields. The schema of data shipped from source systems is handled in various ways. Two popular options are schemaless and fixed schema. Schemaless doesn't mean the absence of schema. Rather, it means that the application defines the schema as data is written, whether to a messaging queue, a flat file, a blob, or a document database such as MongoDB. A more traditional model built on relational database storage uses a fixed schema enforced in the database, to which application writes must conform. Either of these models presents challenges for data engineers. Schemas change over time; in fact, **schema evolution** is encouraged in the Agile approach to software development. A key part of the data engineer's job is taking raw data input in the source system schema and transforming this into valuable output for analytics. This job becomes more challenging as the source schema evolves. We dive into source systems in greater detail in Chapter 5 ; we also cover schemas and **data modeling** in Chapters 6 and 8 , respectively.

Các nguồn sinh ra dữ liệu được tiêu thụ bởi các hệ thống hạ nguồn, bao gồm bảng tính do con người tạo, cảm biến IoT, và ứng dụng web và di động. Mỗi nguồn có khối lượng và nhịp độ sinh dữ liệu riêng. Một kỹ sư dữ liệu nên biết cách nguồn sinh dữ liệu, bao gồm các đặc thù hoặc sắc thái liên quan. Kỹ sư dữ liệu cũng cần hiểu các giới hạn của những hệ thống nguồn mà họ tương tác. Ví dụ, các truy vấn phân tích chạy trên một cơ sở dữ liệu ứng dụng nguồn có gây tranh chấp tài nguyên và vấn đề hiệu năng không? Một trong những sắc thái thách thức nhất của dữ liệu nguồn là lược đồ. Lược đồ định nghĩa tổ chức phân cấp của dữ liệu. Về mặt logic, ta có thể nghĩ về dữ liệu ở cấp độ một hệ thống nguồn tổng thể, đào sâu vào từng bảng, và xuống tới cấu trúc của từng trường tương ứng. Lược đồ của dữ liệu được chuyển từ các hệ thống nguồn được xử lý theo nhiều cách. Hai lựa chọn phổ biến là không lược đồ (schemaless) và lược đồ cố định (fixed schema). Schemaless không có nghĩa là vắng mặt lược đồ. Thay vào đó, nó có nghĩa là ứng dụng định nghĩa lược đồ khi dữ liệu được ghi, dù là vào một hàng đợi tin nhắn, một tệp phẳng, một blob, hay một cơ sở dữ liệu tài liệu như MongoDB. Một mô hình truyền thống hơn dựa trên lưu trữ cơ sở dữ liệu quan hệ sử dụng một lược đồ cố định được áp đặt trong cơ sở dữ liệu, mà các thao tác ghi của ứng dụng phải tuân theo. Cả hai mô hình này đều đặt ra thách thức cho kỹ sư dữ liệu. Lược đồ thay đổi theo thời gian; thực ra, tiến hóa lược đồ (schema evolution) được khuyến khích trong cách tiếp cận Agile đối với phát triển phần mềm. Một phần then chốt trong công việc của kỹ sư dữ liệu là lấy đầu vào dữ liệu thô theo lược đồ hệ thống nguồn và biến đổi nó thành đầu ra giá trị cho phân tích. Công việc này trở nên khó khăn hơn khi lược đồ nguồn tiến hóa. Chúng ta đi sâu vào hệ thống nguồn chi tiết hơn trong Chương 5; chúng ta cũng đề cập đến lược đồ và mô hình hóa dữ liệu lần lượt trong Chương 6 và 8.

Storage — Lưu trữ

After ingesting data, you need a place to store it. Choosing a storage solution is key to success in the rest of the data lifecycle, and it's also one of the most complicated stages of the data lifecycle for a variety of reasons. First, data architectures in the cloud often leverage several storage solutions.

Second, few data storage solutions function purely as storage, with many supporting complex transformation queries; even **object storage** solutions may support powerful query capabilities—e.g., Amazon S3 Select. Third, while storage is a stage of the data engineering lifecycle, it frequently touches on other stages, such as ingestion, transformation, and serving. Storage runs across the entire data engineering lifecycle, often occurring in multiple places in a **data pipeline**, with storage systems crossing over with source systems, ingestion, transformation, and serving. In many ways, the way data is stored impacts how it is used in all of the stages of the data engineering lifecycle. For example, cloud data warehouses can store data, process data in pipelines, and serve it to analysts. **Streaming** frameworks such as Apache Kafka and Pulsar can function simultaneously as ingestion, storage, and query systems for messages, with object storage being a standard layer for data transmission.

Sau khi thu nạp dữ liệu, bạn cần một nơi để lưu trữ nó. Chọn giải pháp lưu trữ là yếu tố then chốt cho thành công trong phần còn lại của vòng đời dữ liệu, và nó cũng là một trong những giai đoạn phức tạp nhất của vòng đời dữ liệu vì nhiều lý do. Thứ nhất, các kiến trúc dữ liệu trên đám mây thường tận dụng nhiều giải pháp lưu trữ. Thứ hai, ít giải pháp lưu trữ dữ liệu hoạt động thuần túy như nơi lưu trữ, mà nhiều giải pháp hỗ trợ các truy vấn biến đổi phức tạp; ngay cả các giải pháp lưu trữ đối tượng (object storage) cũng có thể hỗ trợ khả năng truy vấn mạnh mẽ — ví dụ, Amazon S3 Select. Thứ ba, dù lưu trữ là một giai đoạn của vòng đời kỹ thuật dữ liệu, nó thường xuyên chạm đến các giai đoạn khác, như thu nạp, biến đổi và phục vụ. Lưu trữ chạy xuyên suốt toàn bộ vòng đời kỹ thuật dữ liệu, thường xuất hiện ở nhiều nơi trong một đường ống dữ liệu (data pipeline), với các hệ thống lưu trữ giao thoa với hệ thống nguồn, thu nạp, biến đổi và phục vụ. Theo nhiều cách, cách dữ liệu được lưu trữ ảnh hưởng đến cách nó được dùng trong tất cả các giai đoạn của vòng đời kỹ thuật dữ liệu. Ví dụ, các kho dữ liệu đám mây (cloud data warehouse) có thể lưu trữ dữ liệu, xử lý dữ liệu trong các đường ống, và phục vụ nó cho các nhà phân tích. Các framework theo luồng như Apache Kafka và Pulsar có thể đồng thời hoạt động như hệ thống thu nạp, lưu trữ và truy vấn cho các thông điệp, với lưu trữ đối tượng là một tăng tiêu chuẩn cho việc truyền tải dữ liệu.

Evaluating storage systems: Key engineering considerations — Đánh giá hệ thống lưu trữ: Những cân nhắc kỹ thuật then chốt

Here are a few key engineering questions to ask when choosing a storage system for a **data warehouse**, **data lakehouse**, database, or object storage:

Sau đây là một vài câu hỏi kỹ thuật then chốt cần đặt ra khi chọn một hệ thống lưu trữ cho kho dữ liệu, data lakehouse, cơ sở dữ liệu, hoặc lưu trữ đối tượng:

Is this storage solution compatible with the architecture's required write and read speeds?

Giải pháp lưu trữ này có tương thích với tốc độ ghi và đọc mà kiến trúc yêu cầu không?

Will storage create a bottleneck for downstream processes?

Lưu trữ có tạo ra điểm nghẽn cho các tiến trình hạ nguồn không?

Do you understand how this storage technology works? Are you utilizing the storage system optimally or committing unnatural acts? For instance, are you applying a high rate of random access updates in an object storage system? (This is an antipattern with significant performance overhead.)

Bạn có hiểu công nghệ lưu trữ này hoạt động như thế nào không? Bạn có đang sử dụng hệ thống lưu trữ một cách tối ưu hay đang làm những điều trái khoáy? Chẳng hạn, bạn có

đang áp dụng tốc độ cao các cập nhật truy cập ngẫu nhiên trong một hệ thống lưu trữ đối tượng không? (Đây là một phản mẫu với chi phí hiệu năng đáng kể.)

Will this storage system handle anticipated future scale? You should consider all capacity limits on the storage system: total available storage, read operation rate, write volume, etc.

Hệ thống lưu trữ này có xử lý được quy mô tương lai dự kiến không? Bạn nên cân nhắc mọi giới hạn dung lượng của hệ thống lưu trữ: tổng dung lượng khả dụng, tốc độ thao tác đọc, khối lượng ghi, v.v.

Will downstream users and processes be able to retrieve data in the required service-level agreement (SLA)?

Người dùng và tiến trình hạ nguồn có thể truy xuất dữ liệu trong thỏa thuận mức dịch vụ (SLA) yêu cầu không?

Are you capturing **metadata** about schema evolution, data flows, **data lineage**, and so forth? Metadata has a significant impact on the utility of data. Metadata represents an investment in the future, dramatically enhancing discoverability and institutional knowledge to streamline future projects and architecture changes.

Bạn có đang ghi nhận siêu dữ liệu (metadata) về tiến hóa lược đồ, luồng dữ liệu, truy vết nguồn gốc dữ liệu (data lineage), v.v. không? Siêu dữ liệu có ảnh hưởng đáng kể đến tính hữu dụng của dữ liệu. Siêu dữ liệu đại diện cho một khoản đầu tư vào tương lai, nâng cao đáng kể khả năng khám phá và tri thức tổ chức để hợp lý hóa các dự án và thay đổi kiến trúc trong tương lai.

Is this a pure storage solution (object storage), or does it support complex query patterns (i.e., a cloud data warehouse)?

Đây là giải pháp lưu trữ thuần túy (lưu trữ đối tượng), hay nó hỗ trợ các mẫu truy vấn phức tạp (tức là một kho dữ liệu đám mây)?

Is the storage system schema-agnostic (object storage)? Flexible schema (Cassandra)? Enforced schema (a cloud data warehouse)?

Hệ thống lưu trữ có bất khả tri lược đồ (schema-agnostic) (lưu trữ đối tượng) không? Lược đồ linh hoạt (Cassandra)? Lược đồ được áp đặt (một kho dữ liệu đám mây)?

How are you tracking master data, golden records **data quality**, and data lineage for **data governance**? (We have more to say on these in “Data Management” .)

Bạn theo dõi dữ liệu chủ, chất lượng dữ liệu bản ghi vàng (golden record), và truy vết nguồn gốc dữ liệu cho quản trị dữ liệu (data governance) như thế nào? (Chúng ta nói thêm về những điều này trong mục “Quản lý dữ liệu”.)

How are you handling **regulatory compliance** and data sovereignty? For example, can you store your data in certain geographical locations but not others?

Bạn xử lý tuân thủ quy định và chủ quyền dữ liệu như thế nào? Ví dụ, bạn có thể lưu trữ dữ liệu của mình ở một số vị trí địa lý nhất định nhưng không phải ở những nơi khác không?

Understanding data access frequency — Hiểu về tần suất truy cập dữ liệu

Not all data is accessed in the same way. Retrieval patterns will greatly vary based on the data being stored and queried. This brings up the notion of the “temperatures” of data. Data access frequency will determine the temperature of your data. Data that is most frequently accessed is called **hot** data. Hot data is commonly retrieved many times per day, perhaps even several times per second, in systems that serve user requests. This data should be stored for fast retrieval, where “fast” is relative to the use case. Lukewarm data might be accessed every so often—say, every week or month. Cold data is seldom queried and is appropriate for storing in an archival system. Cold data is often retained for compliance purposes or in case of a catastrophic failure in another system. In the “old days,” cold data would be stored on tapes and shipped to remote archival facilities. In cloud environments, vendors offer specialized storage tiers with very cheap monthly storage costs but high prices for data retrieval.

Không phải mọi dữ liệu đều được truy cập theo cùng một cách. Các mẫu truy xuất sẽ thay đổi rất nhiều tùy theo dữ liệu được lưu trữ và truy vấn. Điều này dẫn đến khái niệm về “nhiệt độ” của dữ liệu. Tần suất truy cập dữ liệu sẽ quyết định nhiệt độ của dữ liệu. Dữ liệu được truy cập thường xuyên nhất được gọi là dữ liệu nóng (hot data). Dữ liệu nóng thường được truy xuất nhiều lần mỗi ngày, thậm chí có thể vài lần mỗi giây, trong các hệ thống phục vụ yêu cầu người dùng. Dữ liệu này nên được lưu trữ để truy xuất nhanh, trong đó “nhanh” là tương đối tùy theo trường hợp sử dụng. Dữ liệu ấm có thể được truy cập thi thoảng — chẳng hạn, mỗi tuần hoặc mỗi tháng. Dữ liệu lạnh hiếm khi được truy vấn và phù hợp để lưu trữ trong một hệ thống lưu trữ dài hạn. Dữ liệu lạnh thường được giữ lại cho mục đích tuân thủ hoặc phòng trường hợp một hệ thống khác gặp sự cố thảm họa. Trong “ngày xưa”, dữ liệu lạnh sẽ được lưu trên băng từ và chuyển đến các cơ sở lưu trữ ở xa. Trong môi trường đám mây, các nhà cung cấp đưa ra các tầng lưu trữ chuyên dụng với chi phí lưu trữ hàng tháng rất rẻ nhưng giá truy xuất dữ liệu cao.

Selecting a storage system — Chọn một hệ thống lưu trữ

What type of storage solution should you use? This depends on your use cases, data volumes, frequency of ingestion, format, and size of the data being ingested—essentially, the key considerations listed in the preceding bulleted questions. There is no one-size-fits-all universal storage recommendation. Every storage technology has its trade-offs. Countless varieties of storage technologies exist, and it’s easy to be overwhelmed when deciding the best option for your data architecture. Chapter 6 covers storage best practices and approaches in greater detail, as well as the crossover between storage and other lifecycle stages.

Bạn nên dùng loại giải pháp lưu trữ nào? Điều này phụ thuộc vào các trường hợp sử dụng, khối lượng dữ liệu, tần suất thu nạp, định dạng, và kích thước của dữ liệu được thu nạp — về cơ bản là các cân nhắc then chốt được liệt kê trong các câu hỏi gạch đầu dòng phía trên. Không có một khuyến nghị lưu trữ phổ quát phù hợp cho tất cả. Mọi công nghệ lưu trữ đều có sự đánh đổi của nó. Vô số chủng loại công nghệ lưu trữ tồn tại, và rất dễ bị choáng ngợp khi quyết định lựa chọn tốt nhất cho kiến trúc dữ liệu của bạn. Chương 6 đề cập chi tiết hơn về các thực hành tốt nhất và cách tiếp cận lưu trữ, cũng như sự giao thoa giữa lưu trữ và các giai đoạn vòng đời khác.

Ingestion — Thu nạp

After you understand the data source and the characteristics of the source system you're using, you need to gather the data. The second stage of the data engineering lifecycle is data ingestion from source systems. In our experience, source systems and ingestion represent the most significant bottlenecks of the data engineering lifecycle. The source systems are normally outside your direct control and might randomly become unresponsive or provide data of poor quality. Or, your data ingestion service might mysteriously stop working for many reasons. As a result, data flow stops or delivers insufficient data for storage, processing, and serving. Unreliable source and ingestion systems have a ripple effect across the data engineering lifecycle. But you're in good shape, assuming you've answered the big questions about source systems.

Sau khi bạn hiểu nguồn dữ liệu và các đặc điểm của hệ thống nguồn bạn đang dùng, bạn cần thu thập dữ liệu. Giai đoạn thứ hai của vòng đời kỹ thuật dữ liệu là thu nạp dữ liệu từ các hệ thống nguồn. Theo kinh nghiệm của chúng tôi, các hệ thống nguồn và việc thu nạp đại diện cho những điểm nghẽn đáng kể nhất của vòng đời kỹ thuật dữ liệu. Các hệ thống nguồn thường nằm ngoài tầm kiểm soát trực tiếp của bạn và có thể ngẫu nhiên trở nên không phản hồi hoặc cung cấp dữ liệu chất lượng kém. Hoặc, dịch vụ thu nạp dữ liệu của bạn có thể đột nhiên ngừng hoạt động vì nhiều lý do. Kết quả là, luồng dữ liệu dừng lại hoặc cung cấp không đủ dữ liệu cho lưu trữ, xử lý và phục vụ. Các hệ thống nguồn và thu nạp không đáng tin cậy gây ra hiệu ứng lan tỏa khắp vòng đời kỹ thuật dữ liệu. Nhưng bạn đang ở vị thế tốt, giả sử bạn đã trả lời được những câu hỏi lớn về các hệ thống nguồn.

Key engineering considerations for the ingestion phase — Những cân nhắc kỹ thuật then chốt cho giai đoạn thu nạp

When preparing to architect or build a system, here are some primary questions about the ingestion stage:

Khi chuẩn bị thiết kế kiến trúc hoặc xây dựng một hệ thống, đây là một số câu hỏi chính về giai đoạn thu nạp:

What are the use cases for the data I'm ingesting? Can I reuse this data rather than create multiple versions of the same dataset?

Các trường hợp sử dụng cho dữ liệu tôi đang thu nạp là gì? Tôi có thể tái sử dụng dữ liệu này thay vì tạo ra nhiều phiên bản của cùng một tập dữ liệu không?

Are the systems generating and ingesting this data reliably, and is the data available when I need it?

Các hệ thống sinh ra và thu nạp dữ liệu này có đáng tin cậy không, và dữ liệu có sẵn khi tôi cần không?

What is the data destination after ingestion?

Đích đến của dữ liệu sau khi thu nạp là gì?

How frequently will I need to access the data?

Tôi sẽ cần truy cập dữ liệu thường xuyên đến mức nào?

In what volume will the data typically arrive?

Dữ liệu thường đến với khối lượng bao nhiêu?

What format is the data in? Can my downstream storage and transformation systems handle this format?

Dữ liệu ở định dạng nào? Các hệ thống lưu trữ và biến đổi hạ nguồn của tôi có xử lý được định dạng này không?

Is the source data in good shape for immediate downstream use? If so, for how long, and what may cause it to be unusable?

Dữ liệu nguồn có ở trạng thái tốt để dùng ngay ở hạ nguồn không? Nếu vậy, trong bao lâu, và điều gì có thể khiến nó không dùng được?

If the data is from a streaming source, does it need to be transformed before reaching its destination? Would an in-flight transformation be appropriate, where the data is transformed within the stream itself?

Nếu dữ liệu đến từ một nguồn theo luồng, nó có cần được biến đổi trước khi đến đích không? Một biến đổi trong lúc truyền (in-flight) có phù hợp không, nơi dữ liệu được biến đổi ngay trong chính luồng?

These are just a sample of the factors you'll need to think about with ingestion, and we cover those questions and more in Chapter 7 . Before we leave, let's briefly turn our attention to two major data ingestion concepts: **batch** versus streaming and push versus pull.

Đây chỉ là một mẫu các yếu tố bạn sẽ cần nghĩ đến với việc thu nạp, và chúng tôi đề cập những câu hỏi đó và nhiều hơn nữa trong Chương 7. Trước khi rời đi, hãy ngắn gọn chuyển sự chú ý sang hai khái niệm thu nạp dữ liệu lớn: theo lô so với theo luồng (batch versus streaming) và đẩy so với kéo (push versus pull).

Batch versus streaming — Theo lô so với theo luồng

Virtually all data we deal with is inherently streaming . Data is nearly always produced and updated continually at its source. Batch ingestion is simply a specialized and convenient way of processing this stream in large chunks—for example, handling a full day's worth of data in a single batch. Streaming ingestion allows us to provide data to downstream systems— whether other applications, databases, or analytics systems—in a continuous, real-time fashion. Here, real-time (or near real-time) means that the data is available to a downstream system a short time after it is produced (e.g., less than one second later). The **latency** required to qualify as real-time varies by domain and requirements. Batch data is ingested either on a predetermined time interval or as data reaches a preset size threshold. Batch ingestion is a one-way door: once data is broken into batches, the latency for downstream consumers is inherently constrained. Because of limitations of legacy systems, batch was for a long time the default way to ingest data. Batch processing remains an extremely popular way to ingest data for downstream consumption, particularly in analytics and ML. However, the separation of storage and compute in many systems and the ubiquity of **event**-streaming and processing platforms make the continuous processing of data streams much more accessible and increasingly popular. The choice largely depends on the use case and expectations for data timeliness.

Thực tế gần như tất cả dữ liệu chúng ta xử lý vốn dĩ là theo luồng (streaming). Dữ liệu hầu như luôn được sinh ra và cập nhật liên tục tại nguồn của nó. Thu nạp theo lô (batch ingestion) đơn giản là một cách chuyên biệt và tiện lợi để xử lý luồng này thành các khối lớn — ví dụ, xử lý dữ liệu trọn một ngày trong một lô duy nhất. Thu nạp theo luồng (streaming ingestion) cho phép chúng ta cung cấp dữ liệu cho các hệ thống hạ nguồn — dù là các ứng dụng khác, cơ sở dữ liệu, hay hệ thống phân tích — theo cách liên tục, thời

gian thực. Ở đây, thời gian thực (hay gần thời gian thực) nghĩa là dữ liệu có sẵn cho một hệ thống hạ nguồn một khoảng thời gian ngắn sau khi nó được sinh ra (ví dụ, dưới một giây sau). Độ trễ cần thiết để được coi là thời gian thực thay đổi tùy theo lĩnh vực và yêu cầu. Dữ liệu theo lô được thu nạp hoặc theo một khoảng thời gian định trước hoặc khi dữ liệu đạt một ngưỡng kích thước đặt sẵn. Thu nạp theo lô là một cánh cửa một chiều: một khi dữ liệu được chia thành các lô, độ trễ cho các bên tiêu thụ hạ nguồn vốn dĩ bị ràng buộc. Do hạn chế của các hệ thống cũ, theo lô trong một thời gian dài là cách mặc định để thu nạp dữ liệu. Xử lý theo lô vẫn là một cách cực kỳ phổ biến để thu nạp dữ liệu cho tiêu thụ hạ nguồn, đặc biệt trong phân tích và ML. Tuy nhiên, sự tách biệt giữa lưu trữ và tính toán trong nhiều hệ thống cùng sự phổ biến của các nền tảng phát luồng và xử lý sự kiện khiến việc xử lý liên tục các luồng dữ liệu trở nên dễ tiếp cận hơn nhiều và ngày càng phổ biến. Lựa chọn phần lớn phụ thuộc vào trường hợp sử dụng và kỳ vọng về tính kịp thời của dữ liệu.

Key considerations for batch versus stream ingestion — Những cân nhắc then chốt cho thu nạp theo lô so với theo luồng

Should you go streaming-first? Despite the attractiveness of a streaming-first approach, there are many trade-offs to understand and think about. The following are some questions to ask yourself when determining whether streaming ingestion is an appropriate choice over batch ingestion:

Bạn có nên ưu tiên theo luồng (streaming-first) không? Bất chấp sức hấp dẫn của cách tiếp cận streaming-first, có nhiều đánh đổi cần hiểu và suy nghĩ. Sau đây là một số câu hỏi cần tự hỏi khi xác định liệu thu nạp theo luồng có phải là lựa chọn phù hợp hơn thu nạp theo lô hay không:

If I ingest the data in real time, can downstream storage systems handle the rate of data flow?

Nếu tôi thu nạp dữ liệu theo thời gian thực, các hệ thống lưu trữ hạ nguồn có xử lý được tốc độ luồng dữ liệu không?

Do I need millisecond real-time data ingestion? Or would a micro-batch approach work, accumulating and ingesting data, say, every minute?

Tôi có cần thu nạp dữ liệu thời gian thực ở mức mili giây không? Hay một cách tiếp cận vi lô (micro-batch) sẽ ổn, tích lũy và thu nạp dữ liệu, chẳng hạn, mỗi phút?

What are my use cases for streaming ingestion? What specific benefits do I realize by implementing streaming? If I get data in real time, what actions can I take on that data that would be an improvement upon batch?

Các trường hợp sử dụng của tôi cho thu nạp theo luồng là gì? Tôi nhận được những lợi ích cụ thể nào khi triển khai theo luồng? Nếu tôi nhận dữ liệu theo thời gian thực, tôi có thể thực hiện những hành động nào trên dữ liệu đó mà cải thiện so với theo lô?

Will my streaming-first approach cost more in terms of time, money, maintenance, downtime, and **opportunity cost** than simply doing batch?

Cách tiếp cận streaming-first của tôi có tốn kém hơn về thời gian, tiền bạc, bảo trì, thời gian ngừng hoạt động và chi phí cơ hội so với việc chỉ làm theo lô không?

Are my streaming pipeline and system reliable and redundant if infrastructure fails?

Đường ống và hệ thống theo luồng của tôi có đáng tin cậy và dư thừa nếu hạ tầng gặp sự cố không?

What tools are most appropriate for the use case? Should I use a **managed service** (Amazon Kinesis, Google Cloud Pub/Sub, Google Cloud Dataflow) or stand up my own instances of Kafka, Flink, Spark, Pulsar, etc.? If I do the latter, who will manage it? What are the costs and trade-offs?

Công cụ nào phù hợp nhất cho trường hợp sử dụng? Tôi nên dùng một dịch vụ được quản lý (managed service) (Amazon Kinesis, Google Cloud Pub/Sub, Google Cloud Dataflow) hay tự dựng các phiên bản Kafka, Flink, Spark, Pulsar, v.v. của riêng mình? Nếu làm theo cách sau, ai sẽ quản lý nó? Chi phí và đánh đổi là gì?

If I'm deploying an ML model, what benefits do I have with online predictions and possibly continuous training?

Nếu tôi triển khai một mô hình ML, tôi có những lợi ích gì với dự đoán trực tuyến và có thể là huấn luyện liên tục?

Am I getting data from a live production instance? If so, what's the impact of my ingestion process on this source system?

Tôi có đang lấy dữ liệu từ một phiên bản sản xuất đang chạy (live production) không? Nếu vậy, tác động của tiến trình thu nạp của tôi lên hệ thống nguồn này là gì?

As you can see, streaming-first might seem like a good idea, but it's not always straightforward; extra costs and complexities inherently occur. Many great ingestion frameworks do handle both batch and micro-batch ingestion styles. We think batch is an excellent approach for many common use cases, such as model training and weekly reporting. Adopt true real-time streaming only after identifying a business use case that justifies the trade-offs against using batch.

Như bạn thấy, streaming-first có thể trông như một ý tưởng hay, nhưng nó không phải lúc nào cũng đơn giản; chi phí và độ phức tạp phụ trội vốn dĩ sẽ xuất hiện. Nhiều framework thu nạp tuyệt vời có xử lý cả phong cách thu nạp theo lô và vi lô. Chúng tôi cho rằng theo lô là một cách tiếp cận tuyệt vời cho nhiều trường hợp sử dụng phổ biến, chẳng hạn như huấn luyện mô hình và báo cáo hằng tuần. Hãy chỉ áp dụng theo luồng thời gian thực thực thụ sau khi đã xác định được một trường hợp sử dụng kinh doanh biện minh cho các đánh đổi so với việc dùng theo lô.

Push versus pull — Đẩy so với kéo

In the push model of data ingestion, a source system writes data out to a target, whether a database, object store, or filesystem. In the pull model, data is retrieved from the source system. The line between the push and pull paradigms can be quite blurry; data is often pushed and pulled as it works its way through the various stages of a data pipeline. Consider, for example, the extract, transform, load (**ETL**) process, commonly used in batch-oriented ingestion workflows. ETL's extract (**E**) part clarifies that we're dealing with a pull ingestion model. In traditional ETL, the ingestion system queries a current source table snapshot on a fixed schedule. You'll learn more about ETL and extract, load, transform (**ELT**) throughout this book. In another example, consider continuous CDC, which is achieved in a few ways. One common method triggers a message every time a row is changed in the source database. This message is pushed to a queue, where the ingestion system picks it up. Another common CDC method uses binary logs, which record every commit to the database. The database pushes to its logs. The ingestion system reads the logs but doesn't directly interact with the database otherwise. This adds little to no additional load to the source database. Some versions of batch CDC use the pull pattern. For example, in timestamp-based CDC, an ingestion system queries the source database and pulls the rows that have changed since the previous update. With streaming ingestion, data bypasses a backend database and is pushed directly to an endpoint, typically with data buffered

by an event-streaming platform. This pattern is useful with fleets of IoT sensors emitting sensor data. Rather than relying on a database to maintain the current state, we simply think of each recorded reading as an event. This pattern is also growing in popularity in software applications as it simplifies real-time processing, allows app developers to tailor their messages for downstream analytics, and greatly simplifies the lives of data engineers. We discuss ingestion best practices and techniques in depth in Chapter 7. Next, let's turn to the transformation stage of the data engineering lifecycle.

Trong mô hình đẩy (push) của thu nạp dữ liệu, một hệ thống nguồn ghi dữ liệu ra một đích, dù là một cơ sở dữ liệu, kho đối tượng, hay hệ thống tệp. Trong mô hình kéo (pull), dữ liệu được truy xuất từ hệ thống nguồn. Ranh giới giữa các mô thức đẩy và kéo có thể khá mờ; dữ liệu thường được đẩy và kéo khi nó di chuyển qua các giai đoạn khác nhau của một đường ống dữ liệu. Hãy xem xét, ví dụ, tiến trình trích xuất, biến đổi, nạp (ETL), thường được dùng trong các quy trình thu nạp theo hướng lô. Phần trích xuất (E) của ETL làm rõ rằng chúng ta đang xử lý một mô hình thu nạp kéo. Trong ETL truyền thống, hệ thống thu nạp truy vấn một ảnh chụp bảng nguồn hiện tại theo một lịch cố định. Bạn sẽ tìm hiểu thêm về ETL và trích xuất, nạp, biến đổi (ELT) xuyên suốt cuốn sách này. Trong một ví dụ khác, hãy xem xét CDC liên tục, vốn được thực hiện theo một vài cách. Một phương pháp phổ biến kích hoạt một thông điệp mỗi khi một hàng bị thay đổi trong cơ sở dữ liệu nguồn. Thông điệp này được đẩy đến một hàng đợi, nơi hệ thống thu nạp lấy nó. Một phương pháp CDC phổ biến khác sử dụng nhật ký nhị phân (binary log), vốn ghi lại mọi commit vào cơ sở dữ liệu. Cơ sở dữ liệu đẩy vào các nhật ký của nó. Hệ thống thu nạp đọc các nhật ký nhưng không tương tác trực tiếp với cơ sở dữ liệu theo cách khác. Điều này thêm rất ít hoặc không thêm tải bổ sung cho cơ sở dữ liệu nguồn. Một số phiên bản của CDC theo lô sử dụng mẫu kéo. Ví dụ, trong CDC dựa trên dấu thời gian, một hệ thống thu nạp truy vấn cơ sở dữ liệu nguồn và kéo các hàng đã thay đổi kể từ lần cập nhật trước. Với thu nạp theo luồng, dữ liệu bỏ qua một cơ sở dữ liệu backend và được đẩy trực tiếp đến một điểm cuối, thường với dữ liệu được đệm bởi một nền tảng phát luồng sự kiện. Mẫu này hữu ích với các đội cảm biến IoT phát ra dữ liệu cảm biến. Thay vì dựa vào một cơ sở dữ liệu để duy trì trạng thái hiện tại, chúng ta đơn giản nghĩ về mỗi giá trị đọc được ghi lại như một sự kiện. Mẫu này cũng đang ngày càng phổ biến trong các ứng dụng phần mềm vì nó đơn giản hóa xử lý thời gian thực, cho phép nhà phát triển ứng dụng điều chỉnh thông điệp của họ cho phân tích hạ nguồn, và đơn giản hóa rất nhiều cuộc sống của các kỹ sư dữ liệu. Chúng ta thảo luận sâu về các thực hành và kỹ thuật thu nạp tốt nhất trong Chương 7. Tiếp theo, hãy chuyển sang giai đoạn biến đổi của vòng đời kỹ thuật dữ liệu.

Transformation — Biến đổi

After you've ingested and stored data, you need to do something with it. The next stage of the data engineering lifecycle is transformation, meaning data needs to be changed from its original form into something useful for downstream use cases. Without proper transformations, data will sit inert, and not be in a useful form for reports, analysis, or ML. Typically, the transformation stage is where data begins to create value for downstream user consumption.

Sau khi bạn đã thu nạp và lưu trữ dữ liệu, bạn cần làm điều gì đó với nó. Giai đoạn tiếp theo của vòng đời kỹ thuật dữ liệu là biến đổi (transformation), nghĩa là dữ liệu cần được thay đổi từ dạng nguyên gốc thành thứ gì đó hữu ích cho các trường hợp sử dụng hạ nguồn. Nếu không có các biến đổi đúng đắn, dữ liệu sẽ nằm trơ ì, và không ở dạng hữu ích cho báo cáo, phân tích, hay ML. Thông thường, giai đoạn biến đổi là nơi dữ liệu bắt đầu tạo ra giá trị cho tiêu thụ của người dùng hạ nguồn.

Immediately after ingestion, basic transformations map data into correct types (changing ingested

string data into numeric and date types, for example), putting records into standard formats, and removing bad ones. Later stages of transformation may transform the data schema and apply **normalization**. Downstream, we can apply large-scale aggregation for reporting or featurize data for ML processes.

Ngay sau khi thu nạp, các biến đổi cơ bản ánh xạ dữ liệu vào các kiểu đúng (ví dụ, đổi dữ liệu chuỗi đã thu nạp thành kiểu số và kiểu ngày), đưa các bản ghi vào định dạng chuẩn, và loại bỏ các bản ghi xấu. Các giai đoạn biến đổi về sau có thể biến đổi lược đồ dữ liệu và áp dụng chuẩn hóa. Ở hạ nguồn, chúng ta có thể áp dụng tổng hợp quy mô lớn cho báo cáo hoặc tạo đặc trưng (featurize) dữ liệu cho các tiến trình ML.

Key considerations for the transformation phase — Những cân nhắc then chốt cho giai đoạn biến đổi

When considering data transformations within the data engineering lifecycle, it helps to consider the following:

Khi cân nhắc các biến đổi dữ liệu trong vòng đời kỹ thuật dữ liệu, sẽ hữu ích khi xem xét những điều sau:

What's the cost and return on investment (ROI) of the transformation? What is the associated business value?

Chi phí và lợi tức đầu tư (ROI) của biến đổi là gì? Giá trị kinh doanh liên quan là gì?

Is the transformation as simple and self-isolated as possible?

Biến đổi có đơn giản và tự cô lập nhất có thể không?

What business rules do the transformations support?

Các biến đổi hỗ trợ những quy tắc kinh doanh nào?

Am I minimizing data movement between the transformation and the storage system during transformation?

Tôi có đang giảm thiểu việc di chuyển dữ liệu giữa biến đổi và hệ thống lưu trữ trong quá trình biến đổi không?

You can transform data in batch or while streaming in flight. As mentioned in “Ingestion”, virtually all data starts life as a continuous stream; batch is just a specialized way of processing a data stream. Batch transformations are overwhelmingly popular, but given the growing popularity of stream-processing solutions and the general increase in the amount of streaming data, we expect the popularity of streaming transformations to continue growing, perhaps entirely replacing batch processing in certain domains soon. Logically, we treat transformation as a standalone area of the data engineering lifecycle, but the realities of the lifecycle can be much more complicated in practice. Transformation is often entangled in other phases of the lifecycle. Typically, data is transformed in source systems or in flight during ingestion. For example, a source system may add an event timestamp to a record before forwarding it to an ingestion process. Or a record within a streaming pipeline may be “enriched” with additional fields and calculations before it's sent to a data warehouse. Transformations are ubiquitous in various parts of the lifecycle. Data preparation, data wrangling, and cleaning—these transformative tasks add value for end consumers of data. Business logic is a major driver of data transformation, often in data modeling. Data translates business logic into reusable elements (e.g., a sale means “somebody bought 12 picture frames from me for \$30 each, or \$360 in total”). In this case, somebody bought 12 picture frames for \$30 each.

Data modeling is critical for obtaining a clear and current picture of business processes. A simple view of raw retail transactions might not be useful without adding the logic of accounting rules so that the CFO has a clear picture of financial health. Ensure a standard approach for implementing business logic across your transformations. Data featurization for ML is another data transformation process. Featurization intends to extract and enhance data features useful for training ML models. Featurization can be a dark art, combining domain expertise (to identify which features might be important for prediction) with extensive experience in data science. For this book, the main point is that once data scientists determine how to featurize data, featurization processes can be automated by data engineers in the transformation stage of a data pipeline. Transformation is a profound subject, and we cannot do it justice in this brief introduction. Chapter 8 delves into queries, data modeling, and various transformation practices and nuances.

Bạn có thể biến đổi dữ liệu theo lô hoặc trong lúc đang theo luồng (in flight). Như đã đề cập trong “Thu nạp”, thực tế gần như tất cả dữ liệu khởi đầu cuộc đời như một luồng liên tục; theo lô chỉ là một cách chuyên biệt để xử lý một luồng dữ liệu. Các biến đổi theo lô áp đảo về độ phổ biến, nhưng với độ phổ biến ngày càng tăng của các giải pháp xử lý luồng và sự gia tăng chung về lượng dữ liệu theo luồng, chúng tôi kỳ vọng độ phổ biến của các biến đổi theo luồng sẽ tiếp tục tăng, có lẽ thay thế hoàn toàn xử lý theo lô trong một số lĩnh vực sớm thôi. Về mặt logic, chúng ta xem biến đổi như một khu vực độc lập của vòng đời kỹ thuật dữ liệu, nhưng thực tế của vòng đời có thể phức tạp hơn nhiều trong thực hành. Biến đổi thường vướng víu vào các giai đoạn khác của vòng đời. Thông thường, dữ liệu được biến đổi trong các hệ thống nguồn hoặc trong lúc truyền trong quá trình thu nạp. Ví dụ, một hệ thống nguồn có thể thêm một dấu thời gian sự kiện (event timestamp) vào một bản ghi trước khi chuyển tiếp nó đến một tiến trình thu nạp. Hoặc một bản ghi trong một đường ống theo luồng có thể được “làm giàu” với các trường và phép tính bổ sung trước khi nó được gửi đến một kho dữ liệu. Các biến đổi hiện diện khắp nơi trong nhiều phần của vòng đời. Chuẩn bị dữ liệu, vật lộn với dữ liệu (data wrangling), và làm sạch — những tác vụ mang tính biến đổi này thêm giá trị cho người tiêu thụ cuối của dữ liệu. Logic kinh doanh là một động lực lớn của biến đổi dữ liệu, thường nằm trong mô hình hóa dữ liệu. Dữ liệu chuyển logic kinh doanh thành các phần tử có thể tái sử dụng (ví dụ, một lần bán có nghĩa là “ai đó đã mua 12 khung ảnh từ tôi với giá 30 đô-la mỗi cái, hay tổng cộng 360 đô-la”). Trong trường hợp này, ai đó đã mua 12 khung ảnh với giá 30 đô-la mỗi cái. Mô hình hóa dữ liệu là yếu tố then chốt để có được bức tranh rõ ràng và cập nhật về các tiến trình kinh doanh. Một góc nhìn đơn giản về các giao dịch bán lẻ thô có thể không hữu ích nếu không thêm logic của các quy tắc kế toán để giám đốc tài chính (CFO) có một bức tranh rõ ràng về sức khỏe tài chính. Hãy đảm bảo một cách tiếp cận chuẩn để triển khai logic kinh doanh xuyên suốt các biến đổi của bạn. Tạo đặc trưng dữ liệu cho ML là một tiến trình biến đổi dữ liệu khác. Tạo đặc trưng nhằm trích xuất và nâng cao các đặc trưng dữ liệu hữu ích cho việc huấn luyện các mô hình ML. Tạo đặc trưng có thể là một nghệ thuật bí ẩn, kết hợp chuyên môn lĩnh vực (để xác định những đặc trưng nào có thể quan trọng cho dự đoán) với kinh nghiệm sâu rộng về khoa học dữ liệu. Đối với cuốn sách này, điểm chính là một khi các nhà khoa học dữ liệu xác định cách tạo đặc trưng cho dữ liệu, các tiến trình tạo đặc trưng có thể được tự động hóa bởi các kỹ sư dữ liệu trong giai đoạn biến đổi của một đường ống dữ liệu. Biến đổi là một chủ đề sâu sắc, và chúng tôi không thể làm nó công bằng trong phần giới thiệu ngắn gọn này. Chương 8 đi sâu vào truy vấn, mô hình hóa dữ liệu, và nhiều thực hành cùng sắc thái biến đổi.

Serving Data — Phục vụ dữ liệu

You’ve reached the last stage of the data engineering lifecycle. Now that the data has been ingested, stored, and transformed into coherent and useful structures, it’s time to get value from your data. “Getting value” from data means different things to different users. Data has value when it’s used for practical purposes. Data that is not consumed or queried is simply inert. Data vanity projects are a major risk for companies. Many companies pursued vanity projects in the big data era, gathering massive datasets in data lakes that were never consumed in any useful way. The cloud era is triggering a new wave of vanity projects built on the latest data warehouses, object storage systems, and streaming technologies. Data projects must be intentional across the lifecycle. What is the ultimate business purpose of the data so carefully collected, cleaned, and stored? Data serving is perhaps the most exciting part of the data engineering lifecycle. This is where the magic happens. This is where ML engineers can apply the most advanced techniques. Let’s look at some of the popular uses of data: analytics, ML, and **reverse ETL**.

Bạn đã đến giai đoạn cuối cùng của vòng đời kỹ thuật dữ liệu. Giờ đây dữ liệu đã được thu nạp, lưu trữ và biến đổi thành các cấu trúc mạch lạc và hữu ích, đã đến lúc thu được giá trị từ dữ liệu của bạn. “Thu được giá trị” từ dữ liệu có nghĩa khác nhau với những người dùng khác nhau. Dữ liệu có giá trị khi nó được dùng cho các mục đích thực tiễn. Dữ liệu không được tiêu thụ hay truy vấn đơn giản là troi. Các dự án dữ liệu phù phiếm là một rủi ro lớn cho các công ty. Nhiều công ty theo đuổi các dự án phù phiếm trong kỷ nguyên big data, gom góp các tập dữ liệu khổng lồ trong các hồ dữ liệu (data lake) mà chưa bao giờ được tiêu thụ theo bất kỳ cách hữu ích nào. Kỷ nguyên đám mây đang khơi mào một làn sóng dự án phù phiếm mới được xây dựng trên các kho dữ liệu, hệ thống lưu trữ đối tượng và công nghệ theo luồng mới nhất. Các dự án dữ liệu phải có chủ đích xuyên suốt vòng đời. Mục đích kinh doanh tối hậu của dữ liệu được thu thập, làm sạch và lưu trữ cẩn thận đến vậy là gì? Phục vụ dữ liệu có lẽ là phần thú vị nhất của vòng đời kỹ thuật dữ liệu. Đây là nơi phép màu xảy ra. Đây là nơi các kỹ sư ML có thể áp dụng những kỹ thuật tiên tiến nhất. Hãy xem một số ứng dụng phổ biến của dữ liệu: phân tích (analytics), ML, và reverse ETL.

Analytics — Phân tích

Analytics is the core of most data endeavors. Once your data is stored and transformed, you’re ready to generate reports or dashboards, and do ad hoc analysis on the data. Whereas the bulk of analytics used to encompass BI, it now includes other facets such as operational analytics and customer-facing analytics (Figure 2-5). Let’s briefly touch on these variations of analytics.

Phân tích (analytics) là cốt lõi của hầu hết các nỗ lực dữ liệu. Một khi dữ liệu của bạn được lưu trữ và biến đổi, bạn đã sẵn sàng tạo báo cáo hoặc bảng điều khiển (dashboard), và làm phân tích đột xuất (ad hoc) trên dữ liệu. Trong khi phần lớn phân tích trước đây bao gồm trí tuệ doanh nghiệp (BI), giờ đây nó bao gồm các khía cạnh khác như phân tích vận hành và phân tích hướng đến khách hàng (Hình 2-5). Hãy ngắn gọn điểm qua các biến thể này của phân tích.

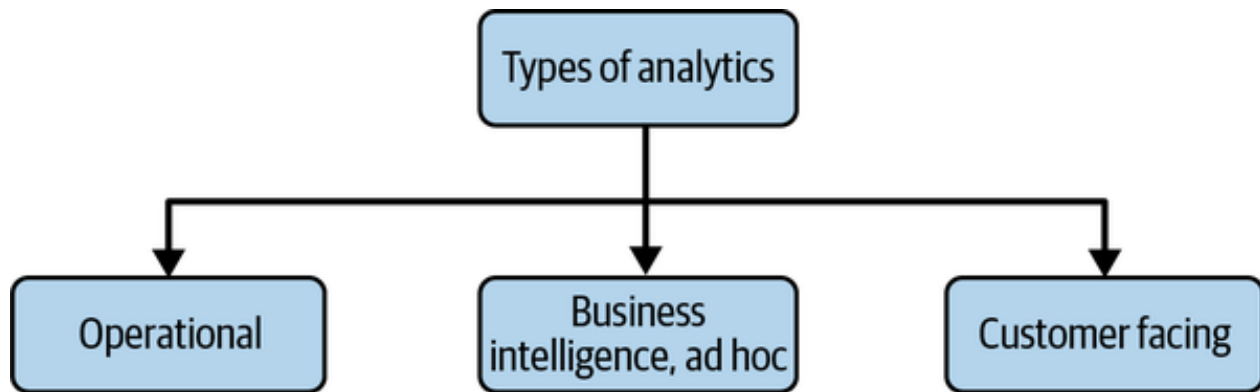


Figure 2-5. Types of analytics **Business intelligence** BI marshals collected data to describe a business's past and current state. BI requires using business logic to process raw data. Note that data serving for analytics is yet another area where the stages of the data engineering lifecycle can get tangled. As we mentioned earlier, business logic is often applied to data in the transformation stage of the data engineering lifecycle, but a logic-on-read approach has become increasingly popular. Data is stored in a clean but fairly raw form, with minimal postprocessing business logic. A BI system maintains a repository of business logic and definitions. This business logic is used to query the data warehouse so that reports and dashboards align with business definitions. As a company grows its data maturity, it will move from ad hoc data analysis to self-service analytics, allowing democratized data access to business users without needing IT to intervene. The capability to do self-service analytics assumes that data is good enough that people across the organization can simply access it themselves, slice and dice it however they choose, and get immediate insights. Although self-service analytics is simple in theory, it's tough to pull off in practice. The main reason is that poor data quality, organizational silos, and a lack of adequate data skills get in the way of allowing widespread use of analytics.

Hình 2-5. Các loại phân tích. Trí tuệ doanh nghiệp BI huy động dữ liệu được thu thập để mô tả trạng thái quá khứ và hiện tại của một doanh nghiệp. BI đòi hỏi dùng logic kinh doanh để xử lý dữ liệu thô. Lưu ý rằng phục vụ dữ liệu cho phân tích là một lĩnh vực nữa nơi các giai đoạn của vòng đời kỹ thuật dữ liệu có thể bị rối vào nhau. Như đã đề cập trước đó, logic kinh doanh thường được áp dụng lên dữ liệu trong giai đoạn biến đổi của vòng đời kỹ thuật dữ liệu, nhưng một cách tiếp cận logic-khi-đọc (logic-on-read) đã ngày càng phổ biến. Dữ liệu được lưu ở dạng sạch nhưng khá thô, với logic kinh doanh hậu xử lý tối thiểu. Một hệ thống BI duy trì một kho lưu trữ logic kinh doanh và các định nghĩa. Logic kinh doanh này được dùng để truy vấn kho dữ liệu sao cho các báo cáo và bảng điều khiển khớp với các định nghĩa kinh doanh. Khi một công ty phát triển độ chín dữ liệu của mình, nó sẽ chuyển từ phân tích dữ liệu đột xuất sang phân tích tự phục vụ (self-service analytics), cho phép truy cập dữ liệu dân chủ hóa đến người dùng kinh doanh mà không cần IT can thiệp. Khả năng làm phân tích tự phục vụ giả định rằng dữ liệu đủ tốt để mọi người trong toàn tổ chức có thể đơn giản tự truy cập, cắt lát và phân tích theo bất kỳ cách nào họ chọn, và nhận được hiểu biết tức thì. Mặc dù phân tích tự phục vụ đơn giản về lý thuyết, nó rất khó thực hiện trong thực tế. Lý do chính là chất lượng dữ liệu kém, các silo tổ chức, và sự thiếu hụt kỹ năng dữ liệu phù hợp cản trở việc cho phép sử dụng phân tích rộng rãi.

Operational analytics Operational analytics focuses on the fine-grained details of operations, promoting actions that a user of the reports can act upon immediately. Operational analytics could be a live view of inventory or real-time dashboarding of website health. In this case, data is consumed in real time, either directly from a source system or from a streaming data pipeline. The

types of insights in operational analytics differ from traditional BI since operational analytics is focused on the present and doesn't necessarily concern historical trends.

Phân tích vận hành Phân tích vận hành tập trung vào các chi tiết hạt mịn của hoạt động vận hành, thúc đẩy các hành động mà người dùng báo cáo có thể hành động ngay lập tức. Phân tích vận hành có thể là một góc nhìn trực tiếp về hàng tồn kho hoặc bảng điều khiển thời gian thực về sức khỏe website. Trong trường hợp này, dữ liệu được tiêu thụ theo thời gian thực, hoặc trực tiếp từ một hệ thống nguồn hoặc từ một đường ống dữ liệu theo luồng. Các loại hiểu biết trong phân tích vận hành khác với BI truyền thống vì phân tích vận hành tập trung vào hiện tại và không nhất thiết liên quan đến các xu hướng lịch sử.

Embedded analytics You may wonder why we've broken out embedded analytics (customer-facing analytics) separately from BI. In practice, analytics provided to customers on a SaaS platform come with a separate set of requirements and complications. Internal BI faces a limited audience and generally presents a limited number of unified views. Access controls are critical but not particularly complicated. Access is managed using a handful of roles and access tiers.

Phân tích hướng đến khách hàng Bạn có thể tự hỏi tại sao chúng tôi tách riêng phân tích hướng đến khách hàng (phân tích hướng đến khách hàng) khỏi BI. Trong thực tế, phân tích được cung cấp cho khách hàng trên một nền tảng SaaS đi kèm với một tập hợp yêu cầu và phức tạp riêng. BI nội bộ đối mặt với một đối tượng giới hạn và thường trình bày một số lượng giới hạn các góc nhìn hợp nhất. Kiểm soát truy cập là then chốt nhưng không đặc biệt phức tạp. Truy cập được quản lý bằng một số ít vai trò và tầng truy cập.

With customer-facing analytics, the request rate for reports, and the corresponding burden on analytics systems, go up dramatically; **access control** is significantly more complicated and critical. Businesses may be serving separate analytics and data to thousands or more customers. Each customer must see their data and only their data. An internal data-access error at a company would likely lead to a procedural review. A data leak between customers would be considered a massive breach of trust, leading to media attention and a significant loss of customers. Minimize your blast radius related to data leaks and security vulnerabilities. Apply tenant- or data-level security within your storage, and anywhere there's a possibility of data leakage.

Với phân tích hướng đến khách hàng, tốc độ yêu cầu báo cáo, và gánh nặng tương ứng lên các hệ thống phân tích, tăng lên đột ngột; kiểm soát truy cập phức tạp và then chốt hơn đáng kể. Các doanh nghiệp có thể đang phục vụ phân tích và dữ liệu riêng biệt cho hàng nghìn khách hàng hoặc hơn. Mỗi khách hàng phải thấy dữ liệu của họ và chỉ dữ liệu của họ. Một lỗi truy cập dữ liệu nội bộ tại một công ty có thể dẫn đến một cuộc rà soát quy trình. Một rò rỉ dữ liệu giữa các khách hàng sẽ bị coi là một sự vi phạm lòng tin khổng lồ, dẫn đến sự chú ý của truyền thông và mất mát khách hàng đáng kể. Hãy giảm thiểu bán kính ảnh hưởng (blast radius) liên quan đến rò rỉ dữ liệu và lỗ hổng bảo mật. Áp dụng bảo mật ở cấp thuê bao (tenant) hoặc cấp dữ liệu trong lưu trữ của bạn, và ở bất cứ nơi nào có khả năng rò rỉ dữ liệu.

MULTITENANCY — ĐA THUÊ BAO

Many current storage and analytics systems support multitenancy in various ways. Data engineers may choose to house data for many customers in common tables to allow a unified view for internal analytics and ML. This data is presented externally to individual customers through logical views with appropriately defined controls and filters. It is incumbent on data engineers to understand the minutiae of multitenancy in the systems they deploy to ensure absolute **data security** and isolation.

Nhiều hệ thống lưu trữ và phân tích hiện nay hỗ trợ đa thuê bao (multitenant) theo nhiều cách. Các kỹ sư dữ liệu có thể chọn chứa dữ liệu cho nhiều khách hàng trong các bảng chung để cho phép một góc nhìn hợp nhất cho phân tích nội bộ và ML. Dữ liệu này được trình bày ra bên ngoài cho từng khách hàng riêng lẻ thông qua các góc nhìn logic với các kiểm soát và bộ lọc được định nghĩa phù hợp. Trách nhiệm của các kỹ sư dữ liệu là hiểu các chi tiết nhỏ nhặt của đa thuê bao trong các hệ thống họ triển khai để đảm bảo an ninh và cô lập dữ liệu tuyệt đối.

Machine learning — Học máy

The emergence and success of ML is one of the most exciting technology revolutions. Once organizations reach a high level of data maturity, they can begin to identify problems amenable to ML and start organizing a practice around it. The responsibilities of data engineers overlap significantly in analytics and ML, and the boundaries between data engineering, ML engineering, and analytics engineering can be fuzzy. For example, a data engineer may need to support Spark clusters that facilitate analytics pipelines and ML model training. They may also need to provide a system that orchestrates tasks across teams and support metadata and cataloging systems that track data history and lineage. Setting these domains of responsibility and the relevant reporting structures is a critical organizational decision. The **feature store** is a recently developed tool that combines data engineering and ML engineering. Feature stores are designed to reduce the operational burden for ML engineers by maintaining feature history and versions, supporting feature sharing among teams, and providing basic operational and orchestration capabilities, such as backfilling. In practice, data engineers are part of the core support team for feature stores to support ML engineering. Should a data engineer be familiar with ML? It certainly helps. Regardless of the operational boundary between data engineering, ML engineering, business analytics, and so forth, data engineers should maintain operational knowledge about their teams. A good data engineer is conversant in the fundamental ML techniques and related data-processing requirements, the use cases for models within their company, and the responsibilities of the organization's various analytics teams. This helps maintain efficient communication and facilitate collaboration. Ideally, data engineers will build tools in partnership with other teams that neither team can make independently. This book cannot possibly cover ML in depth. A growing ecosystem of books, videos, articles, and communities is available if you're interested in learning more; we include a few suggestions in "Additional Resources". The following are some considerations for the serving data phase specific to ML:

Sự xuất hiện và thành công của học máy (machine learning) là một trong những cuộc cách mạng công nghệ thú vị nhất. Một khi các tổ chức đạt đến mức độ chín dữ liệu cao, họ có thể bắt đầu xác định những vấn đề phù hợp với ML và bắt đầu tổ chức một thực hành xoay quanh nó. Trách nhiệm của các kỹ sư dữ liệu chồng lấn đáng kể trong phân tích và ML, và ranh giới giữa kỹ thuật dữ liệu, kỹ thuật ML, và kỹ thuật phân tích có thể mờ nhạt. Ví dụ, một kỹ sư dữ liệu có thể cần hỗ trợ các cụm Spark hỗ trợ cho các đường ống phân tích và huấn luyện mô hình ML. Họ cũng có thể cần cung cấp một hệ thống điều phối các tác vụ qua các nhóm và hỗ trợ các hệ thống siêu dữ liệu và lập danh mục theo dõi lịch sử và nguồn gốc dữ liệu. Thiết lập các phạm vi trách nhiệm này và các cấu trúc báo cáo liên quan là một quyết định tổ chức then chốt. Feature store là một công cụ được phát triển gần đây kết hợp kỹ thuật dữ liệu và kỹ thuật ML. Feature store được thiết kế để giảm gánh nặng vận hành cho các kỹ sư ML bằng cách duy trì lịch sử và phiên bản đặc trưng, hỗ trợ chia sẻ đặc trưng giữa các nhóm, và cung cấp các khả năng vận hành và điều phối cơ bản, chẳng hạn như nạp lại (backfilling). Trong thực tế, các kỹ sư dữ liệu là một phần của nhóm hỗ trợ cốt lõi cho feature store để hỗ trợ kỹ thuật ML. Một kỹ sư dữ liệu có nên am hiểu

ML không? Điều đó chắc chắn có ích. Bất kể ranh giới vận hành giữa kỹ thuật dữ liệu, kỹ thuật ML, phân tích kinh doanh, v.v., các kỹ sư dữ liệu nên duy trì kiến thức vận hành về các nhóm của họ. Một kỹ sư dữ liệu giỏi thông thạo các kỹ thuật ML nền tảng và các yêu cầu xử lý dữ liệu liên quan, các trường hợp sử dụng cho các mô hình trong công ty của họ, và trách nhiệm của các nhóm phân tích khác nhau trong tổ chức. Điều này giúp duy trì giao tiếp hiệu quả và tạo điều kiện cho cộng tác. Lý tưởng nhất, các kỹ sư dữ liệu sẽ xây dựng công cụ trong sự hợp tác với các nhóm khác mà không nhóm nào có thể tự làm một mình. Cuốn sách này không thể nào đề cập ML một cách sâu sắc. Một hệ sinh thái đang phát triển gồm sách, video, bài viết và cộng đồng có sẵn nếu bạn quan tâm tìm hiểu thêm; chúng tôi đưa ra một vài gợi ý trong “Tài nguyên bổ sung”. Sau đây là một số câu nhắc cho giai đoạn phục vụ dữ liệu cụ thể cho ML:

Is the data of sufficient quality to perform reliable feature engineering? Quality requirements and assessments are developed in close collaboration with teams consuming the data.

Dữ liệu có đủ chất lượng để thực hiện kỹ thuật đặc trưng (feature engineering) đáng tin cậy không? Các yêu cầu và đánh giá chất lượng được phát triển trong sự hợp tác chặt chẽ với các nhóm tiêu thụ dữ liệu.

Is the data discoverable? Can data scientists and ML engineers easily find valuable data?

Dữ liệu có thể khám phá được không? Các nhà khoa học dữ liệu và kỹ sư ML có thể dễ dàng tìm thấy dữ liệu giá trị không?

Where are the technical and organizational boundaries between data engineering and ML engineering? This organizational question has significant architectural implications.

Ranh giới kỹ thuật và tổ chức giữa kỹ thuật dữ liệu và kỹ thuật ML nằm ở đâu? Câu hỏi tổ chức này có hệ quả kiến trúc đáng kể.

Does the dataset properly represent ground truth? Is it unfairly biased?

Tập dữ liệu có đại diện đúng cho sự thật nền (ground truth) không? Nó có thiên lệch một cách bất công không?

While ML is exciting, our experience is that companies often prematurely dive into it. Before investing a ton of resources into ML, take the time to build a solid data foundation. This means setting up the best systems and architecture across the data engineering and ML lifecycle. It's generally best to develop competence in analytics before moving to ML. Many companies have dashed their ML dreams because they undertook initiatives without appropriate foundations.

Dù ML thú vị, kinh nghiệm của chúng tôi là các công ty thường lao vào nó quá sớm. Trước khi đầu tư cả tấn tài nguyên vào ML, hãy dành thời gian xây dựng một nền tảng dữ liệu vững chắc. Điều này nghĩa là thiết lập các hệ thống và kiến trúc tốt nhất xuyên suốt vòng đời kỹ thuật dữ liệu và ML. Nhìn chung tốt nhất là phát triển năng lực trong phân tích trước khi chuyển sang ML. Nhiều công ty đã làm tiêu tan giấc mơ ML của họ vì họ tiến hành các sáng kiến mà không có nền tảng phù hợp.

Reverse ETL — Reverse ETL

Reverse ETL has long been a practical reality in data, viewed as an antipattern that we didn't like to talk about or dignify with a name. Reverse ETL takes processed data from the output side of the data engineering lifecycle and feeds it back into source systems, as shown in Figure 2-6 . In reality, this flow is beneficial and often necessary; reverse ETL allows us to take analytics, scored models, etc., and feed these back into production systems or SaaS platforms.

Reverse ETL từ lâu đã là một thực tế thực tiễn trong lĩnh vực dữ liệu, được xem như một phản mẫu mà chúng ta không thích nói đến hay làm cho nó có vẻ đáng được đặt tên. Reverse ETL lấy dữ liệu đã xử lý từ phía đầu ra của vòng đời kỹ thuật dữ liệu và đưa nó ngược trở lại các hệ thống nguồn, như thể hiện trong Hình 2-6. Trong thực tế, luồng này có lợi và thường cần thiết; reverse ETL cho phép chúng ta lấy phân tích, các mô hình đã chấm điểm, v.v., và đưa chúng ngược trở lại các hệ thống sản xuất hoặc nền tảng SaaS.

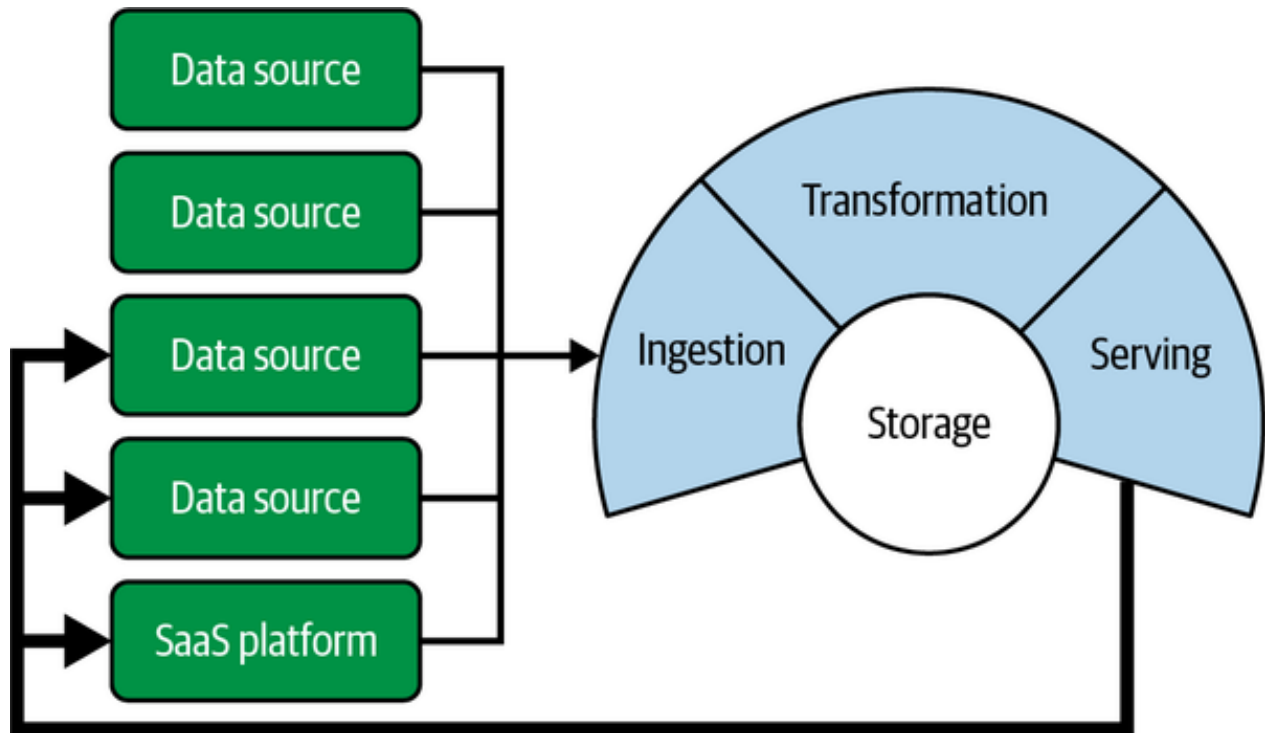


Figure 2-6. Reverse ETL Marketing analysts might calculate bids in Microsoft Excel by using the data in their data warehouse, and then upload these bids to Google Ads. This process was often entirely manual and primitive. As we’ve written this book, several vendors have embraced the concept of reverse ETL and built products around it, such as Hightouch and Census. Reverse ETL remains nascent as a field, but we suspect that it is here to stay. Reverse ETL has become especially important as businesses rely increasingly on SaaS and external platforms. For example, companies may want to push specific metrics from their data warehouse to a customer **data platform** or CRM system. Advertising platforms are another everyday use case, as in the Google Ads example. Expect to see more activity in reverse ETL, with an overlap in both data engineering and ML engineering. The jury is out on whether the term reverse ETL will stick. And the practice may evolve. Some engineers claim that we can eliminate reverse ETL by handling data transformations in an event stream and sending those events back to source systems as needed. Realizing widespread adoption of this pattern across businesses is another matter. The gist is that transformed data will need to be returned to source systems in some manner, ideally with the correct lineage and business process associated with the source system.

Hình 2-6. Reverse ETL. Các nhà phân tích tiếp thị có thể tính toán các mức giá thầu trong Microsoft Excel bằng cách dùng dữ liệu trong kho dữ liệu của họ, rồi tải các mức giá thầu này lên Google Ads. Tiến trình này thường hoàn toàn thủ công và sơ khai. Khi chúng tôi viết cuốn sách này, một số nhà cung cấp đã đón nhận khái niệm reverse ETL và xây dựng các sản phẩm xoay quanh nó, chẳng hạn như Hightouch và Census. Reverse ETL vẫn còn non trẻ như một lĩnh vực, nhưng chúng tôi ngờ rằng nó sẽ tồn tại lâu dài. Reverse ETL

đã trở nên đặc biệt quan trọng khi các doanh nghiệp ngày càng dựa vào SaaS và các nền tảng bên ngoài. Ví dụ, các công ty có thể muốn đẩy các chỉ số cụ thể từ kho dữ liệu của họ đến một nền tảng dữ liệu khách hàng hoặc hệ thống CRM. Các nền tảng quảng cáo là một trường hợp sử dụng thường nhật khác, như trong ví dụ Google Ads. Hãy kỳ vọng thấy nhiều hoạt động hơn trong reverse ETL, với sự chồng lấn trong cả kỹ thuật dữ liệu và kỹ thuật ML. Vẫn chưa rõ liệu thuật ngữ reverse ETL có trụ lại hay không. Và thực hành này có thể tiến hóa. Một số kỹ sư cho rằng chúng ta có thể loại bỏ reverse ETL bằng cách xử lý các biến đổi dữ liệu trong một luồng sự kiện và gửi các sự kiện đó ngược trở lại các hệ thống nguồn khi cần. Hiện thực hóa việc áp dụng rộng rãi mẫu này xuyên suốt các doanh nghiệp lại là một vấn đề khác. Ý chính là dữ liệu đã biến đổi sẽ cần được trả về các hệ thống nguồn theo một cách nào đó, lý tưởng là với truy vết nguồn gốc đúng và tiến trình kinh doanh liên quan đến hệ thống nguồn.

Major Undercurrents Across the Data Engineering Lifecycle — Các dòng chảy ngầm chính xuyên suốt vòng đời kỹ thuật dữ liệu

Data engineering is rapidly maturing. Whereas prior cycles of data engineering simply focused on the technology layer, the continued abstraction and simplification of tools and practices have shifted this focus. Data engineering now encompasses far more than tools and technology. The field is now moving up the value chain, incorporating traditional enterprise practices such as data management and cost optimization, and newer practices like DataOps. We've termed these practices undercurrents —security, data management, DataOps, data architecture, orchestration, and software engineering—that support every aspect of the data engineering lifecycle (Figure 2-7). In this section, we give a brief overview of these undercurrents and their major components, which you'll see in more detail throughout the book.

Kỹ thuật dữ liệu đang trưởng thành nhanh chóng. Trong khi các chu kỳ kỹ thuật dữ liệu trước đây đơn giản tập trung vào tầng công nghệ, sự trừu tượng hóa và đơn giản hóa liên tục của các công cụ và thực hành đã dịch chuyển trọng tâm này. Kỹ thuật dữ liệu giờ đây bao trùm nhiều hơn nhiều so với công cụ và công nghệ. Lĩnh vực này giờ đây đang dịch chuyển lên cao trong chuỗi giá trị, kết hợp các thực hành doanh nghiệp truyền thống như quản lý dữ liệu và tối ưu chi phí, cùng các thực hành mới hơn như DataOps. Chúng tôi đã gọi các thực hành này là các dòng chảy ngầm — an ninh, quản lý dữ liệu, DataOps, kiến trúc dữ liệu, điều phối và kỹ thuật phần mềm — vốn hỗ trợ mọi khía cạnh của vòng đời kỹ thuật dữ liệu (Hình 2-7). Trong mục này, chúng tôi đưa ra một tổng quan ngắn gọn về các dòng chảy ngầm này và các thành phần chính của chúng, mà bạn sẽ thấy chi tiết hơn xuyên suốt cuốn sách.

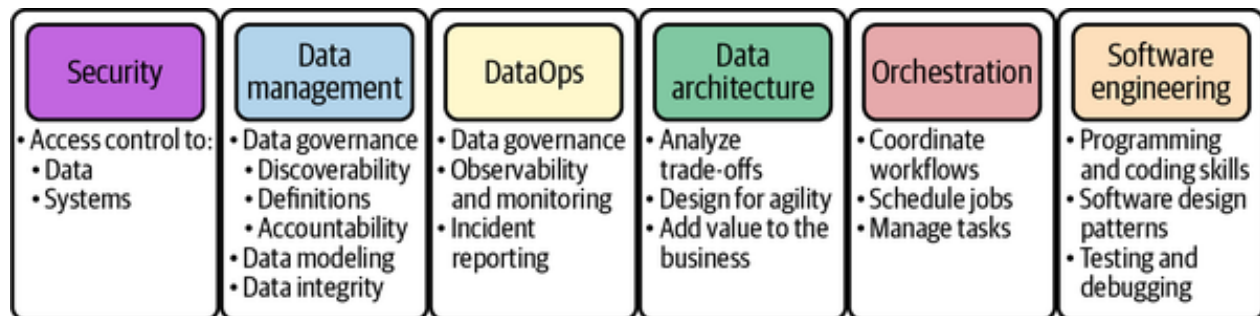


Figure 2-7. The major undercurrents of data engineering

Security — An ninh

Security must be top of mind for data engineers, and those who ignore it do so at their peril. That's why security is the first undercurrent. Data engineers must understand both data and access security, exercising the **principle of least privilege**. The principle of least privilege means giving a user or system access to only the essential data and resources to perform an intended function. A common antipattern we see with data engineers with little security experience is to give admin access to all users. This is a catastrophe waiting to happen! Give users only the access they need to do their jobs today, nothing more. Don't operate from a root shell when you're just looking for visible files with standard user access. When querying tables with a lesser role, don't use the superuser role in a database. Imposing the principle of least privilege on ourselves can prevent accidental damage and keep you in a security-first mindset. People and organizational structure are always the biggest security vulnerabilities in any company. When we hear about major security breaches in the media, it often turns out that someone in the company ignored basic precautions, fell victim to a phishing attack, or otherwise acted irresponsibly. The first line of defense for data security is to create a culture of security that permeates the organization. All individuals who have access to data must understand their responsibility in protecting the company's sensitive data and its customers. Data security is also about timing—providing data access to exactly the people and systems that need to access it and only for the duration necessary to perform their work. Data should be protected from unwanted visibility, both in flight and at rest, by using **encryption**, tokenization, data masking, obfuscation, and simple, robust access controls. Data engineers must be competent security administrators, as security falls in their domain. A data engineer should understand security best practices for the cloud and on prem. Knowledge of user and identity access management (IAM) roles, policies, groups, network security, password policies, and encryption are good places to start. Throughout the book, we highlight areas where security should be top of mind in the data engineering lifecycle. You can also gain more detailed insights into security in Chapter 10.

An ninh phải là điều luôn ở trong tâm trí các kỹ sư dữ liệu, và những ai phớt lờ nó sẽ tự chuốc lấy nguy hiểm. Đó là lý do an ninh là dòng chảy ngầm đầu tiên. Các kỹ sư dữ liệu phải hiểu cả an ninh dữ liệu và an ninh truy cập, thực thi nguyên tắc đặc quyền tối thiểu. Nguyên tắc đặc quyền tối thiểu nghĩa là cấp cho một người dùng hoặc hệ thống quyền truy cập chỉ đến dữ liệu và tài nguyên cốt yếu để thực hiện một chức năng dự định. Một phản mẫu phổ biến chúng tôi thấy ở các kỹ sư dữ liệu ít kinh nghiệm về an ninh là cấp quyền quản trị (admin) cho mọi người dùng. Đây là một thảm họa chờ chờ xảy ra! Hãy chỉ cấp cho người dùng quyền truy cập họ cần để làm công việc của họ hôm nay, không hơn. Đừng vận hành từ một root shell khi bạn chỉ đang tìm các tệp hiển thị với quyền truy cập người dùng tiêu chuẩn. Khi truy vấn các bảng với một vai trò thấp hơn, đừng dùng vai trò superuser trong một cơ sở dữ liệu. Áp đặt nguyên tắc đặc quyền tối thiểu lên chính mình có thể ngăn ngừa thiệt hại do vô ý và giữ bạn trong tư duy ưu tiên an ninh. Con người và cơ cấu tổ chức luôn là những lỗ hổng an ninh lớn nhất ở bất kỳ công ty nào. Khi chúng ta nghe về những vụ vi phạm an ninh lớn trên truyền thông, thường hóa ra là ai đó trong công ty đã phớt lờ các biện pháp phòng ngừa cơ bản, trở thành nạn nhân của một cuộc tấn công lừa đảo (phishing), hoặc hành xử vô trách nhiệm theo cách nào đó. Tuyến phòng thủ đầu tiên cho an ninh dữ liệu là tạo ra một văn hóa an ninh thấm nhuần khắp tổ chức. Tất cả các cá nhân có quyền truy cập dữ liệu phải hiểu trách nhiệm của họ trong việc bảo vệ dữ liệu nhạy cảm của công ty và của khách hàng. An ninh dữ liệu cũng là về thời điểm — cung cấp quyền truy cập dữ liệu đúng cho những người và hệ thống cần truy cập nó và chỉ trong khoảng thời gian cần thiết để thực hiện công việc của họ. Dữ liệu nên

được bảo vệ khỏi việc bị nhìn thấy ngoài ý muốn, cả trong lúc truyền và lúc nghỉ (at rest), bằng cách dùng mã hóa (encryption), token hóa, che mặt nạ dữ liệu (data masking), làm rối (obfuscation), và các kiểm soát truy cập đơn giản, vững chắc. Các kỹ sư dữ liệu phải là những quản trị viên an ninh có năng lực, vì an ninh nằm trong phạm vi của họ. Một kỹ sư dữ liệu nên hiểu các thực hành an ninh tốt nhất cho đám mây và tại chỗ (on prem). Kiến thức về các vai trò quản lý truy cập người dùng và danh tính (IAM), chính sách, nhóm, an ninh mạng, chính sách mật khẩu, và mã hóa là những điểm khởi đầu tốt. Xuyên suốt cuốn sách, chúng tôi làm nổi bật các lĩnh vực nơi an ninh nên luôn ở trong tâm trí trong vòng đời kỹ thuật dữ liệu. Bạn cũng có thể có được những hiểu biết chi tiết hơn về an ninh trong Chương 10.

Data Management — Quản lý dữ liệu

You probably think that data management sounds very...corporate. “Old school” data management practices make their way into data and ML engineering. What’s old is new again. Data management has been around for decades but didn’t get a lot of traction in data engineering until recently. Data tools are becoming simpler, and there is less complexity for data engineers to manage. As a result, the data engineer moves up the value chain toward the next rung of best practices. Data best practices once reserved for huge companies—data governance, **master data management**, data-quality management, metadata management—are now filtering down to companies of all sizes and maturity levels. As we like to say, data engineering is becoming “enterprisey.” This is ultimately a great thing! The Data Management Association International (DAMA) Data Management Body of Knowledge (DMBOK), which we consider to be the definitive book for enterprise data management, offers this definition:

Bạn có lẽ nghĩ rằng quản lý dữ liệu nghe rất... mang tính tập đoàn. Các thực hành quản lý dữ liệu “trường phái cũ” đang len lỏi vào kỹ thuật dữ liệu và kỹ thuật ML. Cái cũ lại trở thành mới. Quản lý dữ liệu đã có từ nhiều thập kỷ nhưng không nhận được nhiều sự chú ý trong kỹ thuật dữ liệu cho đến gần đây. Các công cụ dữ liệu đang trở nên đơn giản hơn, và có ít độ phức tạp hơn để kỹ sư dữ liệu phải quản lý. Kết quả là, kỹ sư dữ liệu dịch chuyển lên cao trong chuỗi giá trị, hướng tới nấc thang tiếp theo của các thực hành tốt nhất. Các thực hành dữ liệu tốt nhất từng dành riêng cho các công ty khổng lồ — quản trị dữ liệu, quản lý dữ liệu chủ, quản lý chất lượng dữ liệu, quản lý siêu dữ liệu — giờ đây đang lan tỏa xuống các công ty thuộc mọi quy mô và mức độ chín. Như chúng tôi thích nói, kỹ thuật dữ liệu đang trở nên “mang tính doanh nghiệp”. Suy cho cùng đây là một điều tuyệt vời! Cuốn Data Management Body of Knowledge (DMBOK) của Data Management Association International (DAMA), mà chúng tôi coi là cuốn sách định nghĩa về quản lý dữ liệu doanh nghiệp, đưa ra định nghĩa này:

Data management is the development, execution, and supervision of plans, policies, programs, and practices that deliver, control, protect, and enhance the value of data and information assets throughout their lifecycle. That’s a bit lengthy, so let’s look at how it ties to data engineering. Data engineers manage the data lifecycle, and data management encompasses the set of best practices that data engineers will use to accomplish this task, both technically and strategically. Without a framework for managing data, data engineers are simply technicians operating in a vacuum. Data engineers need a broader perspective of data’s utility across the organization, from the source systems to the C-suite, and everywhere in between. Why is data management important? Data management demonstrates that data is vital to daily operations, just as businesses view financial resources, finished goods, or real estate as assets. Data management practices form a cohesive framework that everyone can adopt to ensure that the organization gets value from data and

handles it appropriately.

Quản lý dữ liệu là việc phát triển, thực thi và giám sát các kế hoạch, chính sách, chương trình và thực hành nhằm cung cấp, kiểm soát, bảo vệ và nâng cao giá trị của các tài sản dữ liệu và thông tin xuyên suốt vòng đời của chúng. Cái này hơi dài, nên hãy xem nó gắn với kỹ thuật dữ liệu như thế nào. Các kỹ sư dữ liệu quản lý vòng đời dữ liệu, và quản lý dữ liệu bao trùm tập hợp các thực hành tốt nhất mà các kỹ sư dữ liệu sẽ dùng để hoàn thành nhiệm vụ này, cả về mặt kỹ thuật lẫn chiến lược. Nếu không có một khung để quản lý dữ liệu, các kỹ sư dữ liệu chỉ đơn giản là những kỹ thuật viên vận hành trong chân không. Các kỹ sư dữ liệu cần một góc nhìn rộng hơn về tính hữu dụng của dữ liệu xuyên suốt tổ chức, từ các hệ thống nguồn đến ban lãnh đạo cấp cao (C-suite), và mọi nơi ở giữa. Tại sao quản lý dữ liệu lại quan trọng? Quản lý dữ liệu chứng minh rằng dữ liệu là yếu tố sống còn cho hoạt động hằng ngày, hệt như cách doanh nghiệp xem nguồn lực tài chính, thành phẩm, hay bất động sản là các tài sản. Các thực hành quản lý dữ liệu tạo nên một khung gắn kết mà mọi người có thể áp dụng để đảm bảo tổ chức thu được giá trị từ dữ liệu và xử lý nó một cách thích đáng.

Data management has quite a few facets, including the following:

Quản lý dữ liệu có khá nhiều khía cạnh, bao gồm những điều sau:

Data governance, including discoverability and accountability

Quản trị dữ liệu, bao gồm khả năng khám phá và trách nhiệm giải trình

Data modeling and design

Mô hình hóa và thiết kế dữ liệu

Data lineage

Truy vết nguồn gốc dữ liệu

Storage and operations

Lưu trữ và vận hành

Data integration and interoperability

Tích hợp và khả năng tương tác dữ liệu

Data lifecycle management

Quản lý vòng đời dữ liệu

Data systems for advanced analytics and ML

Các hệ thống dữ liệu cho phân tích nâng cao và ML

Ethics and **privacy**

Đạo đức và quyền riêng tư

While this book is in no way an exhaustive resource on data management, let's briefly cover some salient points from each area as they relate to data engineering.

Mặc dù cuốn sách này không hề là một nguồn tài liệu đầy đủ về quản lý dữ liệu, hãy ngắn gọn điểm qua một số điểm nổi bật từ mỗi lĩnh vực khi chúng liên quan đến kỹ thuật dữ liệu.

Data governance — Quản trị dữ liệu

According to Data Governance: The Definitive Guide, “Data governance is, first and foremost, a data management function to ensure the quality, integrity, security, and usability of the data collected by an organization.” 1

Theo cuốn Data Governance: The Definitive Guide, “Quản trị dữ liệu, trước hết và trên hết, là một chức năng quản lý dữ liệu nhằm đảm bảo chất lượng, tính toàn vẹn, an ninh và khả năng sử dụng của dữ liệu được một tổ chức thu thập.” 1

We can expand on that definition and say that data governance engages people, processes, and technologies to maximize data value across an organization while protecting data with appropriate security controls. Effective data governance is developed with intention and supported by the organization. When data governance is accidental and haphazard, the side effects can range from untrusted data to security breaches and everything in between. Being intentional about data governance will maximize the organization’s data capabilities and the value generated from data. It will also (hopefully) keep a company out of the headlines for questionable or downright reckless data practices.

Chúng ta có thể mở rộng định nghĩa đó và nói rằng quản trị dữ liệu (data governance) huy động con người, tiến trình và công nghệ để tối đa hóa giá trị dữ liệu xuyên suốt một tổ chức trong khi bảo vệ dữ liệu bằng các kiểm soát an ninh phù hợp. Quản trị dữ liệu hiệu quả được phát triển có chủ đích và được tổ chức hậu thuẫn. Khi quản trị dữ liệu diễn ra tình cờ và lộn xộn, các tác dụng phụ có thể trải từ dữ liệu không đáng tin đến các vụ vi phạm an ninh và mọi thứ ở giữa. Có chủ đích về quản trị dữ liệu sẽ tối đa hóa các năng lực dữ liệu của tổ chức và giá trị tạo ra từ dữ liệu. Nó cũng sẽ (hy vọng vậy) giữ cho một công ty không lên mặt báo vì các thực hành dữ liệu đáng ngờ hoặc hoàn toàn liều lĩnh.

Think of the typical example of data governance being done poorly. A business analyst gets a request for a report but doesn’t know what data to use to answer the question. They may spend hours digging through dozens of tables in a transactional database, wildly guessing at which fields might be useful. The analyst compiles a “directionally correct” report but isn’t entirely sure that the report’s underlying data is accurate or sound. The recipient of the report also questions the validity of the data. The integrity of the analyst—and of all data in the company’s systems—is called into question. The company is confused about its performance, making business planning impossible. Data governance is a foundation for data-driven business practices and a mission-critical part of the data engineering lifecycle. When data governance is practiced well, people, processes, and technologies align to treat data as a key business driver; if data issues occur, they are promptly handled. The core categories of data governance are discoverability, security, and accountability. Within these core categories are subcategories, such as data 2 quality, metadata, and privacy. Let’s look at each core category in turn.

Hãy nghĩ về ví dụ điển hình của việc quản trị dữ liệu được làm dở. Một nhà phân tích kinh doanh nhận một yêu cầu về một báo cáo nhưng không biết dùng dữ liệu nào để trả lời câu hỏi. Họ có thể dành hàng giờ đào bới qua hàng tá bảng trong một cơ sở dữ liệu giao dịch, đoán mò xem trường nào có thể hữu ích. Nhà phân tích biên soạn một báo cáo “đúng hướng” nhưng không hoàn toàn chắc rằng dữ liệu nền tảng của báo cáo là chính xác hay đúng đắn. Người nhận báo cáo cũng nghi ngờ tính hợp lệ của dữ liệu. Sự chính trực của nhà phân tích — và của toàn bộ dữ liệu trong các hệ thống của công ty — bị đặt dấu hỏi. Công ty bối rối về hiệu quả hoạt động của mình, khiến việc lập kế hoạch kinh doanh trở nên bất khả thi. Quản trị dữ liệu là nền tảng cho các thực hành kinh doanh dựa trên dữ liệu và là một phần tối quan trọng của vòng đời kỹ thuật dữ liệu. Khi quản trị dữ liệu được thực hành tốt, con người, tiến trình và công nghệ liên kết để đối xử với dữ liệu như một

động lực kinh doanh then chốt; nếu các vấn đề dữ liệu xảy ra, chúng được xử lý kịp thời. Các hạng mục cốt lõi của quản trị dữ liệu là khả năng khám phá, an ninh, và trách nhiệm giải trình. Trong các hạng mục cốt lõi này là các hạng mục con, chẳng hạn như chất lượng dữ liệu, siêu dữ liệu, và quyền riêng tư. Hãy lần lượt xem từng hạng mục cốt lõi.

Discoverability In a data-driven company, data must be available and discoverable. End users should have quick and reliable access to the data they need to do their jobs. They should know where the data comes from, how it relates to other data, and what the data means. Some key areas of data discoverability include metadata management and master data management. Let's briefly describe these areas.

Khả năng khám phá Trong một công ty dựa trên dữ liệu, dữ liệu phải sẵn có và có thể khám phá được. Người dùng cuối nên có quyền truy cập nhanh chóng và đáng tin cậy đến dữ liệu họ cần để làm công việc của mình. Họ nên biết dữ liệu đến từ đâu, nó liên hệ với dữ liệu khác như thế nào, và dữ liệu có nghĩa là gì. Một số lĩnh vực then chốt của khả năng khám phá dữ liệu bao gồm quản lý siêu dữ liệu và quản lý dữ liệu chủ. Hãy ngắn gọn mô tả các lĩnh vực này.

Metadata Metadata is “data about data,” and it underpins every section of the data engineering lifecycle. Metadata is exactly the data needed to make data discoverable and governable. We divide metadata into two major categories: autogenerated and human generated. Modern data engineering revolves around automation, but metadata collection is often manual and error prone. Technology can assist with this process, removing much of the error-prone work of manual metadata collection. We're seeing a proliferation of data catalogs, data-lineage tracking systems, and metadata management tools. Tools can crawl databases to look for relationships and monitor data pipelines to track where data comes from and where it goes. A low-fidelity manual approach uses an internally led effort where various stakeholders crowdsource metadata collection within the organization. These data management tools are covered in depth throughout the book, as they undercut much of the data engineering lifecycle. Metadata becomes a byproduct of data and data processes. However, key challenges remain. In particular, interoperability and standards are still lacking. Metadata tools are only as good as their connectors to data systems and their ability to share metadata. In addition, automated metadata tools should not entirely take humans out of the loop. Data has a social element; each organization accumulates social capital and knowledge around processes, datasets, and pipelines. Human-oriented metadata systems focus on the social aspect of metadata. This is something that Airbnb has emphasized in its various blog posts on data tools, particularly its original Dataportal concept. Such tools should provide a 3 place to disclose data owners, data consumers, and domain experts. Documentation and internal wiki tools provide a key foundation for metadata management, but these tools should also integrate with automated data cataloging. For example, data-scanning tools can generate wiki pages with links to relevant data objects. Once metadata systems and processes exist, data engineers can consume metadata in useful ways. Metadata becomes a foundation for designing pipelines and managing data throughout the lifecycle. DMBOK identifies four main categories of metadata that are useful to data engineers:

Siêu dữ liệu Siêu dữ liệu là “dữ liệu về dữ liệu”, và nó làm nền cho mọi mục của vòng đời kỹ thuật dữ liệu. Siêu dữ liệu chính xác là dữ liệu cần thiết để làm cho dữ liệu có thể khám phá được và có thể quản trị được. Chúng ta chia siêu dữ liệu thành hai hạng mục chính: tự sinh và do con người tạo. Kỹ thuật dữ liệu hiện đại xoay quanh tự động hóa, nhưng thu thập siêu dữ liệu thường thủ công và dễ sai sót. Công nghệ có thể trợ giúp tiến trình này, loại bỏ phần lớn công việc dễ sai sót của việc thu thập siêu dữ liệu thủ công. Chúng ta đang thấy một sự sinh sôi của các danh mục dữ liệu (data catalog), các hệ thống theo dõi truy vết nguồn gốc dữ liệu, và các công cụ quản lý siêu dữ liệu. Các công cụ có thể quét (crawl) các cơ sở dữ liệu để tìm các mối quan hệ và giám sát các đường ống dữ liệu để theo dõi dữ

liệu đến từ đâu và đi về đâu. Một cách tiếp cận thủ công độ trung thực thấp dùng một nỗ lực do nội bộ dẫn dắt nơi các bên liên quan khác nhau huy động cộng đồng (crowdsourcing) thu thập siêu dữ liệu trong tổ chức. Các công cụ quản lý dữ liệu này được đề cập sâu xuyên suốt cuốn sách, vì chúng len lỏi vào phần lớn vòng đời kỹ thuật dữ liệu. Siêu dữ liệu trở thành một sản phẩm phụ của dữ liệu và các tiến trình dữ liệu. Tuy nhiên, các thách thức then chốt vẫn còn. Đặc biệt, khả năng tương tác và các tiêu chuẩn vẫn còn thiếu. Các công cụ siêu dữ liệu chỉ tốt bằng các bộ kết nối của chúng đến các hệ thống dữ liệu và khả năng chia sẻ siêu dữ liệu của chúng. Ngoài ra, các công cụ siêu dữ liệu tự động không nên hoàn toàn loại con người ra khỏi vòng lặp. Dữ liệu có một yếu tố xã hội; mỗi tổ chức tích lũy vốn xã hội và tri thức xoay quanh các tiến trình, tập dữ liệu, và đường ống. Các hệ thống siêu dữ liệu hướng đến con người tập trung vào khía cạnh xã hội của siêu dữ liệu. Đây là điều mà Airbnb đã nhấn mạnh trong các bài đăng blog khác nhau về các công cụ dữ liệu, đặc biệt là khái niệm Dataportal ban đầu của họ. Các công cụ như vậy nên cung cấp một nơi để công bố các chủ sở hữu dữ liệu, người tiêu thụ dữ liệu, và các chuyên gia lĩnh vực. Các công cụ tài liệu và wiki nội bộ cung cấp một nền tảng then chốt cho quản lý siêu dữ liệu, nhưng các công cụ này cũng nên tích hợp với việc lập danh mục dữ liệu tự động. Ví dụ, các công cụ quét dữ liệu có thể sinh ra các trang wiki với liên kết đến các đối tượng dữ liệu liên quan. Một khi các hệ thống và tiến trình siêu dữ liệu tồn tại, các kỹ sư dữ liệu có thể tiêu thụ siêu dữ liệu theo những cách hữu ích. Siêu dữ liệu trở thành một nền tảng cho việc thiết kế các đường ống và quản lý dữ liệu xuyên suốt vòng đời. DMBOK xác định bốn hạng mục siêu dữ liệu chính hữu ích cho các kỹ sư dữ liệu:

Business metadata Technical metadata

Siêu dữ liệu kinh doanh Siêu dữ liệu kỹ thuật

Operational metadata

Siêu dữ liệu vận hành

Reference metadata

Siêu dữ liệu tham chiếu

Let's briefly describe each category of metadata. Business metadata relates to the way data is used in the business, including business and data definitions, data rules and logic, how and where data is used, and the data owner(s). A data engineer uses business metadata to answer nontechnical questions about who, what, where, and how. For example, a data engineer may be tasked with creating a data pipeline for customer sales analysis. But what is a customer? Is it someone who's purchased in the last 90 days? Or someone who's purchased at any time the business has been open? A data engineer would use the correct data to refer to business metadata (data dictionary or **data catalog**) to look up how a "customer" is defined. Business metadata provides a data engineer with the right context and definitions to properly use data. Technical metadata describes the data created and used by systems across the data engineering lifecycle. It includes the data model and schema, data lineage, field mappings, and pipeline workflows. A data engineer uses technical metadata to create, connect, and monitor various systems across the data engineering lifecycle. Here are some common types of technical metadata that a data engineer will use:

Hãy ngắn gọn mô tả từng hạng mục siêu dữ liệu. Siêu dữ liệu kinh doanh liên quan đến cách dữ liệu được dùng trong doanh nghiệp, bao gồm các định nghĩa kinh doanh và dữ liệu, các quy tắc và logic dữ liệu, cách và nơi dữ liệu được dùng, và (các) chủ sở hữu dữ liệu. Một kỹ sư dữ liệu dùng siêu dữ liệu kinh doanh để trả lời các câu hỏi phi kỹ thuật về ai, cái gì, ở đâu, và như thế nào. Ví dụ, một kỹ sư dữ liệu có thể được giao nhiệm vụ tạo một đường ống dữ liệu cho phân tích doanh số khách hàng. Nhưng khách hàng là gì? Là người

đã mua trong 90 ngày qua? Hay người đã mua vào bất kỳ thời điểm nào doanh nghiệp đã mở cửa? Một kỹ sư dữ liệu sẽ dùng dữ liệu đúng để tra cứu siêu dữ liệu kinh doanh (từ điển dữ liệu hoặc danh mục dữ liệu) để tra cách “khách hàng” được định nghĩa. Siêu dữ liệu kinh doanh cung cấp cho kỹ sư dữ liệu ngữ cảnh và các định nghĩa đúng để dùng dữ liệu một cách thích đáng. Siêu dữ liệu kỹ thuật mô tả dữ liệu được tạo ra và sử dụng bởi các hệ thống xuyên suốt vòng đời kỹ thuật dữ liệu. Nó bao gồm mô hình dữ liệu và lược đồ, truy vết nguồn gốc dữ liệu, ánh xạ trường, và các quy trình đường ống. Một kỹ sư dữ liệu dùng siêu dữ liệu kỹ thuật để tạo, kết nối, và giám sát các hệ thống khác nhau xuyên suốt vòng đời kỹ thuật dữ liệu. Đây là một số loại siêu dữ liệu kỹ thuật phổ biến mà một kỹ sư dữ liệu sẽ dùng:

Pipeline metadata (often produced in orchestration systems)

Siêu dữ liệu đường ống (thường được tạo ra trong các hệ thống điều phối)

Data lineage

Truy vết nguồn gốc dữ liệu

Schema

Lược đồ

Orchestration is a central hub that coordinates workflow across various systems. Pipeline metadata captured in orchestration systems provides details of the workflow schedule, system and data dependencies, configurations, connection details, and much more. Data-lineage metadata tracks the origin and changes to data, and its dependencies, over time. As data flows through the data engineering lifecycle, it evolves through transformations and combinations with other data. Data lineage provides an audit trail of data’s evolution as it moves through various systems and workflows. Schema metadata describes the structure of data stored in a system such as a database, a data warehouse, a **data lake**, or a filesystem; it is one of the key differentiators across different storage systems. Object stores, for example, don’t manage schema metadata; instead, this must be managed in a metastore . On the other hand, cloud data warehouses manage schema metadata internally. These are just a few examples of technical metadata that a data engineer should know about. This is not a complete list, and we cover additional aspects of technical metadata throughout the book. Operational metadata describes the operational results of various systems and includes statistics about processes, job IDs, application runtime logs, data used in a process, and error logs. A data engineer uses operational metadata to determine whether a process succeeded or failed and the data involved in the process. Orchestration systems can provide a limited picture of operational metadata, but the latter still tends to be scattered across many systems. A need for better-quality operational metadata, and better metadata management, is a major motivation for next-generation orchestration and metadata management systems. Reference metadata is data used to classify other data. This is also referred to as lookup data . Standard examples of reference data are internal codes, geographic codes, units of measurement, and internal calendar standards. Note that much of reference data is fully managed internally, but items such as geographic codes might come from standard external references.

Điều phối là một trung tâm điều phối quy trình làm việc qua các hệ thống khác nhau. Siêu dữ liệu đường ống được nắm bắt trong các hệ thống điều phối cung cấp chi tiết về lịch quy trình làm việc, các phụ thuộc hệ thống và dữ liệu, các cấu hình, chi tiết kết nối, và nhiều hơn nữa. Siêu dữ liệu truy vết nguồn gốc dữ liệu theo dõi nguồn gốc và các thay đổi của dữ liệu, cùng các phụ thuộc của nó, theo thời gian. Khi dữ liệu chảy qua vòng đời kỹ thuật dữ liệu, nó tiến hóa qua các biến đổi và kết hợp với dữ liệu khác. Truy vết nguồn gốc dữ liệu cung cấp một dấu vết kiểm toán về sự tiến hóa của dữ liệu khi nó di chuyển qua các

hệ thống và quy trình khác nhau. Siêu dữ liệu lược đồ mô tả cấu trúc của dữ liệu được lưu trữ trong một hệ thống như một cơ sở dữ liệu, một kho dữ liệu, một hồ dữ liệu, hay một hệ thống tệp; nó là một trong những yếu tố khác biệt then chốt giữa các hệ thống lưu trữ khác nhau. Các kho đối tượng, ví dụ, không quản lý siêu dữ liệu lược đồ; thay vào đó, cái này phải được quản lý trong một metastore. Mặt khác, các kho dữ liệu đám mây quản lý siêu dữ liệu lược đồ nội bộ. Đây chỉ là một vài ví dụ về siêu dữ liệu kỹ thuật mà một kỹ sư dữ liệu nên biết. Đây không phải là một danh sách đầy đủ, và chúng tôi đề cập các khía cạnh bổ sung của siêu dữ liệu kỹ thuật xuyên suốt cuốn sách. Siêu dữ liệu vận hành mô tả các kết quả vận hành của các hệ thống khác nhau và bao gồm các thống kê về các tiến trình, ID công việc, nhật ký thời gian chạy ứng dụng, dữ liệu được dùng trong một tiến trình, và nhật ký lỗi. Một kỹ sư dữ liệu dùng siêu dữ liệu vận hành để xác định liệu một tiến trình đã thành công hay thất bại và dữ liệu liên quan đến tiến trình. Các hệ thống điều phối có thể cung cấp một bức tranh giới hạn về siêu dữ liệu vận hành, nhưng cái sau vẫn có xu hướng nằm rải rác qua nhiều hệ thống. Nhu cầu về siêu dữ liệu vận hành chất lượng tốt hơn, và quản lý siêu dữ liệu tốt hơn, là một động lực lớn cho các hệ thống điều phối và quản lý siêu dữ liệu thế hệ tiếp theo. Siêu dữ liệu tham chiếu là dữ liệu được dùng để phân loại dữ liệu khác. Cái này cũng được gọi là dữ liệu tra cứu (lookup data). Các ví dụ tiêu chuẩn của dữ liệu tham chiếu là mã nội bộ, mã địa lý, đơn vị đo lường, và các tiêu chuẩn lịch nội bộ. Lưu ý rằng phần lớn dữ liệu tham chiếu được quản lý hoàn toàn nội bộ, nhưng các mục như mã địa lý có thể đến từ các tham chiếu bên ngoài tiêu chuẩn.

Reference data is essentially a standard for interpreting other data, so if it changes, this change happens slowly over time.

Dữ liệu tham chiếu về cơ bản là một tiêu chuẩn để diễn giải dữ liệu khác, nên nếu nó thay đổi, thay đổi này diễn ra chậm theo thời gian.

Data accountability Data accountability means assigning an individual to govern a portion of data. The responsible person then coordinates the governance activities of other stakeholders. Managing data quality is tough if no one is accountable for the data in question. Note that people accountable for data need not be data engineers. The accountable person might be a software engineer or product manager, or serve in another role. In addition, the responsible person generally doesn't have all the resources necessary to maintain data quality. Instead, they coordinate with all people who touch the data, including data engineers. Data accountability can happen at various levels; accountability can happen at the level of a table or a log stream but could be as fine-grained as a single field entity that occurs across many tables. An individual may be accountable for managing a customer ID across many systems. For enterprise data management, a data domain is the set of all possible values that can occur for a given field type, such as in this ID example. This may seem excessively bureaucratic and meticulous, but it can significantly affect data quality.

Trách nhiệm giải trình dữ liệu Trách nhiệm giải trình dữ liệu nghĩa là giao một cá nhân quản trị một phần dữ liệu. Người chịu trách nhiệm sau đó điều phối các hoạt động quản trị của các bên liên quan khác. Quản lý chất lượng dữ liệu rất khó nếu không ai chịu trách nhiệm về dữ liệu đang xét. Lưu ý rằng những người chịu trách nhiệm về dữ liệu không nhất thiết phải là các kỹ sư dữ liệu. Người chịu trách nhiệm có thể là một kỹ sư phần mềm hoặc quản lý sản phẩm, hoặc phục vụ trong một vai trò khác. Ngoài ra, người chịu trách nhiệm nhìn chung không có tất cả các tài nguyên cần thiết để duy trì chất lượng dữ liệu. Thay vào đó, họ điều phối với tất cả những người chạm vào dữ liệu, bao gồm các kỹ sư dữ liệu. Trách nhiệm giải trình dữ liệu có thể diễn ra ở nhiều cấp; trách nhiệm có thể diễn ra ở cấp một bảng hoặc một luồng nhật ký nhưng có thể hạt mịn đến mức một thực thể trường đơn lẻ xuất hiện qua nhiều bảng. Một cá nhân có thể chịu trách nhiệm quản lý một ID khách hàng qua nhiều hệ thống. Đối với quản lý dữ liệu doanh nghiệp, một miền dữ

liệu (data domain) là tập hợp tất cả các giá trị có thể xuất hiện cho một loại trường nhất định, chẳng hạn như trong ví dụ ID này. Điều này có vẻ quá quan liêu và tỉ mỉ, nhưng nó có thể ảnh hưởng đáng kể đến chất lượng dữ liệu.

Data quality

Chất lượng dữ liệu

Can I trust this data? —Everyone in the business Data quality is the optimization of data toward the desired state and orbits the question, “What do you get compared with what you expect?” Data should conform to the expectations in the business metadata. Does the data match the definition agreed upon by the business? A data engineer ensures data quality across the entire data engineering lifecycle. This involves performing data-quality tests, and ensuring data conformance to schema expectations, data completeness, and precision.

Tôi có thể tin cậy dữ liệu này không? — Mọi người trong doanh nghiệp. Chất lượng dữ liệu là việc tối ưu hóa dữ liệu hướng tới trạng thái mong muốn và xoay quanh câu hỏi, “Bạn nhận được gì so với những gì bạn mong đợi?” Dữ liệu nên tuân theo các kỳ vọng trong siêu dữ liệu kinh doanh. Dữ liệu có khớp với định nghĩa được doanh nghiệp thống nhất không? Một kỹ sư dữ liệu đảm bảo chất lượng dữ liệu xuyên suốt toàn bộ vòng đời kỹ thuật dữ liệu. Điều này bao gồm thực hiện các kiểm tra chất lượng dữ liệu, và đảm bảo dữ liệu tuân thủ các kỳ vọng về lược đồ, tính đầy đủ của dữ liệu, và độ chính xác.

According to Data Governance: The Definitive Guide , data quality is defined by three main characteristics: 4

Theo cuốn Data Governance: The Definitive Guide, chất lượng dữ liệu được định nghĩa bởi ba đặc tính chính: 4

Accuracy

Tính chính xác (Accuracy)

Is the collected data factually correct? Are there duplicate values? Are the numeric values accurate?

Dữ liệu được thu thập có đúng về mặt sự kiện không? Có các giá trị trùng lặp không? Các giá trị số có chính xác không?

Completeness

Tính đầy đủ (Completeness)

Are the records complete? Do all required fields contain valid values?

Các bản ghi có đầy đủ không? Tất cả các trường bắt buộc có chứa các giá trị hợp lệ không?

Timeliness

Tính kịp thời (Timeliness)

Are records available in a timely fashion?

Các bản ghi có sẵn một cách kịp thời không?

Each of these characteristics is quite nuanced. For example, how do we think about bots and web scrapers when dealing with web event data? If we intend to analyze the customer journey, we must have a process that lets us separate humans from machine-generated traffic. Any bot-generated events misclassified as human present data accuracy issues, and vice versa. A variety of interesting problems arise concerning completeness and timeliness. In the Google paper introducing the Dataflow model, the authors give the example of an offline video platform that displays ads. The

5 platform downloads video and ads while a connection is present, allows the user to watch these while offline, and then uploads ad view data once a connection is present again. This data may arrive late, well after the ads are watched. How does the platform handle billing for the ads? Fundamentally, this problem can't be solved by purely technical means. Rather, engineers will need to determine their standards for late-arriving data and enforce these uniformly, possibly with the help of various technology tools. Data quality sits across the boundary of human and technology problems. Data engineers need robust processes to collect actionable human feedback on data quality and use technology tools to detect quality issues preemptively before downstream users ever see them. We cover these collection processes in the appropriate chapters throughout this book.

Mỗi đặc tính trong số này khá tinh tế. Ví dụ, chúng ta nghĩ thế nào về các bot và trình thu thập web (web scraper) khi xử lý dữ liệu sự kiện web? Nếu chúng ta dự định phân tích hành trình khách hàng, chúng ta phải có một tiến trình cho phép tách con người khỏi lưu lượng do máy sinh ra. Bất kỳ sự kiện do bot sinh ra mà bị phân loại nhầm thành con người đều đặt ra các vấn đề về tính chính xác của dữ liệu, và ngược lại. Một loạt các vấn đề thú vị nảy sinh liên quan đến tính đầy đủ và tính kịp thời. Trong bài báo của Google giới thiệu mô hình Dataflow, các tác giả đưa ra ví dụ về một nền tảng video ngoại tuyến hiển thị quảng cáo. Nền tảng tải xuống video và quảng cáo khi có kết nối, cho phép người dùng xem chúng trong khi ngoại tuyến, rồi tải lên dữ liệu lượt xem quảng cáo một khi có kết nối trở lại. Dữ liệu này có thể đến muộn, lâu sau khi các quảng cáo đã được xem. Nền tảng xử lý việc tính phí cho các quảng cáo như thế nào? Về cơ bản, vấn đề này không thể được giải quyết bằng các phương tiện thuần kỹ thuật. Thay vào đó, các kỹ sư sẽ cần xác định các tiêu chuẩn của họ cho dữ liệu đến muộn và thực thi chúng một cách đồng nhất, có thể với sự trợ giúp của nhiều công cụ công nghệ khác nhau. Chất lượng dữ liệu nằm trên ranh giới của các vấn đề con người và công nghệ. Các kỹ sư dữ liệu cần các tiến trình vững chắc để thu thập phản hồi khả thi từ con người về chất lượng dữ liệu và dùng các công cụ công nghệ để phát hiện các vấn đề chất lượng một cách phủ đầu trước khi người dùng hạ nguồn từng thấy chúng. Chúng tôi đề cập các tiến trình thu thập này trong các chương phù hợp xuyên suốt cuốn sách.

MASTER DATA MANAGEMENT — QUẢN LÝ DỮ LIỆU CHỦ

Master data is data about business entities such as employees, customers, products, and locations. As organizations grow larger and more complex through organic growth and acquisitions, and collaborate with other businesses, maintaining a consistent picture of entities and identities becomes more and more challenging. Master data management (MDM) is the practice of building consistent entity definitions known as golden records. Golden records harmonize entity data across an organization and with its partners. MDM is a business operations process facilitated by building and deploying technology tools. For example, an MDM team might determine a standard format for addresses, and then work with data engineers to build an API to return consistent addresses and a system that uses address data to match customer records across company divisions. MDM reaches across the full data cycle into operational databases. It may fall directly under the purview of data engineering, but is often the assigned responsibility of a dedicated team that works across the organization. Even if they don't own MDM, data engineers must always be aware of it, as they will collaborate on MDM initiatives.

Dữ liệu chủ là dữ liệu về các thực thể kinh doanh như nhân viên, khách hàng, sản phẩm, và địa điểm. Khi các tổ chức phát triển lớn hơn và phức tạp hơn thông qua tăng trưởng hữu cơ và mua lại, và cộng tác với các doanh nghiệp khác, việc duy trì một bức tranh

nhất quán về các thực thể và danh tính trở nên ngày càng thách thức. Quản lý dữ liệu chủ (MDM) là thực hành xây dựng các định nghĩa thực thể nhất quán được gọi là bản ghi vàng (golden record). Các bản ghi vàng hài hòa dữ liệu thực thể xuyên suốt một tổ chức và với các đối tác của nó. MDM là một tiến trình vận hành kinh doanh được hỗ trợ bằng việc xây dựng và triển khai các công cụ công nghệ. Ví dụ, một nhóm MDM có thể xác định một định dạng chuẩn cho địa chỉ, rồi làm việc với các kỹ sư dữ liệu để xây dựng một API trả về các địa chỉ nhất quán và một hệ thống dùng dữ liệu địa chỉ để khớp các bản ghi khách hàng qua các bộ phận của công ty. MDM vượt qua toàn bộ chu kỳ dữ liệu vào trong các cơ sở dữ liệu vận hành. Nó có thể nằm trực tiếp trong phạm vi của kỹ thuật dữ liệu, nhưng thường là trách nhiệm được giao cho một nhóm chuyên trách làm việc xuyên suốt tổ chức. Ngay cả khi họ không sở hữu MDM, các kỹ sư dữ liệu phải luôn ý thức về nó, vì họ sẽ cộng tác trong các sáng kiến MDM.

Data modeling and design — Mô hình hóa và thiết kế dữ liệu

To derive business insights from data, through business analytics and data science, the data must be in a usable form. The process for converting data into a usable form is known as data modeling and design . Whereas we traditionally think of data modeling as a problem for database administrators (DBAs) and ETL developers, data modeling can happen almost anywhere in an organization. Firmware engineers develop the data format of a record for an IoT device, or web application developers design the JSON response to an API call or a MySQL table schema—these are all instances of data modeling and design. Data modeling has become more challenging because of the variety of new data sources and use cases. For instance, strict normalization doesn't work well with event data. Fortunately, a new generation of data tools increases the flexibility of data models, while retaining logical separations of measures, dimensions, attributes, and hierarchies. Cloud data warehouses support the ingestion of enormous quantities of denormalized and semistructured data, while still supporting common data modeling patterns, such as **Kimball**, **Inmon**, and **data vault**. Data processing frameworks such as Spark can ingest a whole spectrum of data, from flat structured relational records to raw unstructured text. We discuss these data modeling and transformation patterns in greater detail in Chapter 8 . With the wide variety of data that engineers must cope with, there is a temptation to throw up our hands and give up on data modeling. This is a terrible idea with harrowing consequences, made evident when people murmur of the write once, read never (WORN) access pattern or refer to a data swamp . Data engineers need to understand modeling best practices as well as develop the flexibility to apply the appropriate level and type of modeling to the data source and use case.

Để rút ra các hiểu biết kinh doanh từ dữ liệu, thông qua phân tích kinh doanh và khoa học dữ liệu, dữ liệu phải ở một dạng có thể dùng được. Tiến trình chuyển đổi dữ liệu thành một dạng có thể dùng được được gọi là mô hình hóa và thiết kế dữ liệu (data modeling and design). Trong khi theo truyền thống chúng ta nghĩ về mô hình hóa dữ liệu như một vấn đề cho các quản trị viên cơ sở dữ liệu (DBA) và các nhà phát triển ETL, mô hình hóa dữ liệu có thể diễn ra gần như bất cứ đâu trong một tổ chức. Các kỹ sư firmware phát triển định dạng dữ liệu của một bản ghi cho một thiết bị IoT, hoặc các nhà phát triển ứng dụng web thiết kế phản hồi JSON cho một lời gọi API hay một lược đồ bảng MySQL — đây đều là các trường hợp của mô hình hóa và thiết kế dữ liệu. Mô hình hóa dữ liệu đã trở nên thách thức hơn vì sự đa dạng của các nguồn dữ liệu và trường hợp sử dụng mới. Chẳng hạn, chuẩn hóa nghiêm ngặt không hoạt động tốt với dữ liệu sự kiện. May mắn thay, một thế hệ công cụ dữ liệu mới làm tăng tính linh hoạt của các mô hình dữ liệu, trong khi vẫn giữ các tách biệt logic của các thước đo (measure), chiều (dimension), thuộc tính, và phân cấp. Các kho dữ liệu đám mây hỗ trợ việc thu nạp một lượng khổng lồ dữ liệu phi chuẩn

hóa và bán cấu trúc, trong khi vẫn hỗ trợ các mẫu mô hình hóa dữ liệu phổ biến, chẳng hạn như Kimball, Inmon, và Data Vault. Các framework xử lý dữ liệu như Spark có thể thu nạp cả một phổ dữ liệu, từ các bản ghi quan hệ có cấu trúc phẳng đến văn bản phi cấu trúc thô. Chúng ta thảo luận các mẫu mô hình hóa và biến đổi dữ liệu này chi tiết hơn trong Chương 8. Với sự đa dạng rộng lớn của dữ liệu mà các kỹ sư phải đối phó, có một cảm dỗ là giờ tay đầu hàng và từ bỏ mô hình hóa dữ liệu. Đây là một ý tưởng tệ hại với những hậu quả kinh hoàng, trở nên rõ ràng khi người ta thì thầm về mẫu truy cập ghi một lần, không bao giờ đọc (WORN) hoặc nhắc đến một đầm lầy dữ liệu (data swamp). Các kỹ sư dữ liệu cần hiểu các thực hành mô hình hóa tốt nhất cũng như phát triển sự linh hoạt để áp dụng mức độ và loại mô hình hóa phù hợp cho nguồn dữ liệu và trường hợp sử dụng.

Data lineage — Truy vết nguồn gốc dữ liệu

As data moves through its lifecycle, how do you know what system affected the data or what the data is composed of as it gets passed around and transformed? Data lineage describes the recording of an audit trail of data through its lifecycle, tracking both the systems that process the data and the upstream data it depends on. Data lineage helps with error tracking, accountability, and debugging of data and the systems that process it. It has the obvious benefit of giving an audit trail for the data lifecycle and helps with compliance. For example, if a user would like their data deleted from your systems, having lineage for that data lets you know where that data is stored and its dependencies.

Khi dữ liệu di chuyển qua vòng đời của nó, làm sao bạn biết hệ thống nào đã tác động lên dữ liệu hoặc dữ liệu được cấu thành từ gì khi nó được chuyển quanh và biến đổi? Truy vết nguồn gốc dữ liệu (data lineage) mô tả việc ghi lại một dấu vết kiểm toán của dữ liệu qua vòng đời của nó, theo dõi cả các hệ thống xử lý dữ liệu và dữ liệu thượng nguồn mà nó phụ thuộc vào. Truy vết nguồn gốc dữ liệu giúp ích cho việc theo dõi lỗi, trách nhiệm giải trình, và gỡ lỗi dữ liệu cùng các hệ thống xử lý nó. Nó có lợi ích hiển nhiên là cung cấp một dấu vết kiểm toán cho vòng đời dữ liệu và giúp ích cho việc tuân thủ. Ví dụ, nếu một người dùng muốn dữ liệu của họ bị xóa khỏi các hệ thống của bạn, việc có truy vết nguồn gốc cho dữ liệu đó cho bạn biết dữ liệu đó được lưu ở đâu và các phụ thuộc của nó.

Data lineage has been around for a long time in larger companies with strict compliance standards. However, it's now being more widely adopted in smaller companies as data management becomes mainstream. We also note that Andy Petrella's concept of **Data Observability** Driven Development (DODD) is closely related to data lineage. DODD observes data all along its lineage. This process is applied during development, testing, and finally production to deliver quality and conformity to expectations.

Truy vết nguồn gốc dữ liệu đã có từ lâu trong các công ty lớn hơn với các tiêu chuẩn tuân thủ nghiêm ngặt. Tuy nhiên, nó giờ đây đang được áp dụng rộng rãi hơn trong các công ty nhỏ hơn khi quản lý dữ liệu trở nên phổ thông. Chúng tôi cũng lưu ý rằng khái niệm Phát triển hướng khả năng quan sát dữ liệu (DODD) của Andy Petrella có liên hệ chặt chẽ với truy vết nguồn gốc dữ liệu. DODD quan sát dữ liệu dọc theo toàn bộ nguồn gốc của nó. Tiến trình này được áp dụng trong quá trình phát triển, kiểm thử, và cuối cùng là sản xuất để cung cấp chất lượng và sự phù hợp với các kỳ vọng.

Data integration and interoperability — Tích hợp và khả năng tương tác dữ liệu

Data integration and interoperability is the process of integrating data across tools and processes. As we move away from a single-stack approach to analytics and toward a heterogeneous cloud environment in which various tools process data on demand, integration and interoperability occupy an ever-widening swath of the data engineer's job. Increasingly, integration happens through general-purpose APIs rather than custom database connections. For example, a data pipeline might pull data from the Salesforce API, store it to Amazon S3, call the Snowflake API to load it into a table, call the API again to run a query, and then export the results to S3 where Spark can consume them. All of this activity can be managed with relatively simple Python code that talks to data systems rather than handling data directly. While the complexity of interacting with data systems has decreased, the number of systems and the complexity of pipelines has dramatically increased. Engineers starting from scratch quickly outgrow the capabilities of bespoke scripting and stumble into the need for orchestration. Orchestration is one of our undercurrents, and we discuss it in detail in “Orchestration” .

Tích hợp và khả năng tương tác dữ liệu là tiến trình tích hợp dữ liệu qua các công cụ và tiến trình. Khi chúng ta rời xa một cách tiếp cận stack đơn lẻ cho phân tích và hướng tới một môi trường đám mây không đồng nhất nơi nhiều công cụ xử lý dữ liệu theo yêu cầu, tích hợp và khả năng tương tác chiếm một dải ngày càng rộng trong công việc của kỹ sư dữ liệu. Ngày càng nhiều, tích hợp diễn ra thông qua các API đa dụng thay vì các kết nối cơ sở dữ liệu tùy chỉnh. Ví dụ, một đường ống dữ liệu có thể kéo dữ liệu từ Salesforce API, lưu nó vào Amazon S3, gọi Snowflake API để nạp nó vào một bảng, gọi API lần nữa để chạy một truy vấn, rồi xuất kết quả ra S3 nơi Spark có thể tiêu thụ chúng. Tất cả hoạt động này có thể được quản lý bằng mã Python tương đối đơn giản giao tiếp với các hệ thống dữ liệu thay vì xử lý dữ liệu trực tiếp. Trong khi độ phức tạp của việc tương tác với các hệ thống dữ liệu đã giảm, số lượng hệ thống và độ phức tạp của các đường ống đã tăng đột ngột. Các kỹ sư bắt đầu từ con số không nhanh chóng vượt qua các khả năng của việc viết script đặt riêng (bespoke scripting) và vấp phải nhu cầu điều phối. Điều phối là một trong các dòng chảy ngầm của chúng tôi, và chúng tôi thảo luận nó chi tiết trong “Điều phối”.

Data lifecycle management — Quản lý vòng đời dữ liệu

The advent of data lakes encouraged organizations to ignore data archival and destruction. Why discard data when you can simply add more storage ad infinitum? Two changes have encouraged engineers to pay more attention to what happens at the end of the data engineering lifecycle.

Sự ra đời của các hồ dữ liệu đã khuyến khích các tổ chức phớt lờ việc lưu trữ dài hạn và tiêu hủy dữ liệu. Tại sao phải vứt bỏ dữ liệu khi bạn có thể đơn giản thêm nhiều lưu trữ vô hạn? Hai thay đổi đã khuyến khích các kỹ sư chú ý hơn đến những gì xảy ra ở cuối vòng đời kỹ thuật dữ liệu.

First, data is increasingly stored in the cloud. This means we have pay-as-you-go storage costs instead of large up-front capital expenditures for an on-premises data lake. When every byte shows up on a monthly AWS statement, CFOs see opportunities for savings. Cloud environments make data archival a relatively straightforward process. Major cloud vendors offer archival-specific object storage classes that allow long-term **data retention** at an extremely low cost, assuming very infrequent access (it should be noted that data retrieval isn't so cheap, but that's for another conversation). These storage classes also support extra policy controls to prevent accidental or

deliberate deletion of critical archives. Second, privacy and data retention laws such as the GDPR and the CCPA require data engineers to actively manage data destruction to respect users’ “right to be forgotten.” Data engineers must know what consumer data they retain and must have procedures to destroy data in response to requests and compliance requirements. Data destruction is straightforward in a cloud data warehouse. SQL semantics allow deletion of rows conforming to a where clause. Data destruction was more challenging in data lakes, where write-once, read-many was the default storage pattern. Tools such as Hive ACID and Delta Lake allow easy management of deletion transactions at scale. New generations of metadata management, data lineage, and cataloging tools will also streamline the end of the data engineering lifecycle.

Thứ nhất, dữ liệu ngày càng được lưu trên đám mây. Điều này nghĩa là chúng ta có chi phí lưu trữ trả-theo-mức-dùng (pay-as-you-go) thay vì các khoản chi vốn lớn trả trước cho một hồ dữ liệu tại chỗ. Khi mỗi byte hiện lên trên bảng kê AWS hằng tháng, các giám đốc tài chính (CFO) thấy các cơ hội tiết kiệm. Các môi trường đám mây làm cho việc lưu trữ dài hạn dữ liệu thành một tiến trình tương đối đơn giản. Các nhà cung cấp đám mây lớn cung cấp các lớp lưu trữ đối tượng chuyên dụng cho lưu trữ dài hạn cho phép giữ dữ liệu lâu dài với chi phí cực kỳ thấp, giả định truy cập rất ít (cần lưu ý rằng truy xuất dữ liệu không rẻ đến vậy, nhưng đó là một cuộc trò chuyện khác). Các lớp lưu trữ này cũng hỗ trợ thêm các kiểm soát chính sách để ngăn việc xóa vô tình hay cố ý các kho lưu trữ then chốt. Thứ hai, các luật về quyền riêng tư và lưu giữ dữ liệu như GDPR và CCPA yêu cầu các kỹ sư dữ liệu chủ động quản lý việc tiêu hủy dữ liệu để tôn trọng “quyền được lãng quên” của người dùng. Các kỹ sư dữ liệu phải biết họ giữ lại dữ liệu người tiêu dùng nào và phải có các quy trình để tiêu hủy dữ liệu nhằm đáp ứng các yêu cầu và đòi hỏi tuân thủ. Tiêu hủy dữ liệu là đơn giản trong một kho dữ liệu đám mây. Ngữ nghĩa SQL cho phép xóa các hàng tuân theo một mệnh đề where. Tiêu hủy dữ liệu thách thức hơn trong các hồ dữ liệu, nơi ghi-một-lần, đọc-nhiều-lần là mẫu lưu trữ mặc định. Các công cụ như Hive ACID và Delta Lake cho phép quản lý dễ dàng các giao dịch xóa ở quy mô lớn. Các thể hệ mới của các công cụ quản lý siêu dữ liệu, truy vết nguồn gốc dữ liệu, và lập danh mục cũng sẽ hợp lý hóa phần cuối của vòng đời kỹ thuật dữ liệu.

Ethics and privacy — Đạo đức và quyền riêng tư

Ethical behavior is doing the right thing when no one else is watching. —Aldo Leopold The last several years of data breaches, misinformation, and mishandling of data make one thing clear: data impacts people. Data used to live in the Wild West, freely collected and traded like baseball cards. Those days are long gone. Whereas data’s ethical and privacy implications were once considered nice to have, like security, they’re now central to the general data lifecycle. Data engineers need to do the right thing when no one else is watching, because everyone will be watching someday. We hope that more organizations will encourage a culture of good data ethics and privacy. How do ethics and privacy impact the data engineering lifecycle? Data engineers need to ensure that datasets mask personally identifiable information (PII) and other sensitive information; bias can be identified and tracked in datasets as they are transformed. Regulatory requirements and compliance penalties are only growing. Ensure that your data assets are compliant with a growing number of data regulations, such as GDPR and CCPA. Please take this seriously. We offer tips throughout the book to ensure that you’re baking ethics and privacy into the data engineering lifecycle.

Hành vi đạo đức là làm điều đúng khi không có ai khác đang theo dõi. — Aldo Leopold. Vài năm gần đây của các vụ vi phạm dữ liệu, thông tin sai lệch, và việc xử lý dữ liệu sai trái làm rõ một điều: dữ liệu tác động đến con người. Dữ liệu từng sống trong miền Viễn Tây hoang dã, được thu thập và trao đổi tự do như những tấm thẻ bóng chày. Những ngày

đó đã qua từ lâu. Trong khi các hệ quả đạo đức và quyền riêng tư của dữ liệu từng được coi là điều có thì tốt, giống như an ninh, chúng giờ đây là trung tâm của vòng đời dữ liệu nói chung. Các kỹ sư dữ liệu cần làm điều đúng khi không có ai khác đang theo dõi, vì một ngày nào đó mọi người sẽ đều theo dõi. Chúng tôi hy vọng nhiều tổ chức hơn sẽ khuyến khích một văn hóa đạo đức dữ liệu và quyền riêng tư tốt. Đạo đức và quyền riêng tư tác động đến vòng đời kỹ thuật dữ liệu như thế nào? Các kỹ sư dữ liệu cần đảm bảo rằng các tập dữ liệu che mặt nạ thông tin định danh cá nhân (PII) và thông tin nhạy cảm khác; sự thiên lệch có thể được xác định và theo dõi trong các tập dữ liệu khi chúng được biến đổi. Các yêu cầu pháp lý và hình phạt tuân thủ chỉ đang gia tăng. Hãy đảm bảo rằng các tài sản dữ liệu của bạn tuân thủ một số lượng ngày càng tăng các quy định dữ liệu, chẳng hạn như GDPR và CCPA. Vui lòng coi trọng điều này. Chúng tôi đưa ra các mẹo xuyên suốt cuốn sách để đảm bảo rằng bạn đang đưa (baking) đạo đức và quyền riêng tư vào trong vòng đời kỹ thuật dữ liệu.

Orchestration — Điều phối

We think that orchestration matters because we view it as really the center of gravity of both the data platform as well as the data lifecycle, the software development lifecycle as it comes to data. —Nick Schrock, founder of Elementl

Orchestration is not only a central DataOps process, but also a critical part of the engineering and deployment flow for data jobs. So, what is orchestration? Orchestration is the process of coordinating many jobs to run as quickly and efficiently as possible on a scheduled cadence. For instance, people often refer to orchestration tools like Apache Airflow as schedulers. This isn't quite accurate. A pure scheduler, such as cron, is aware only of time; an orchestration engine builds in metadata on job dependencies, generally in the form of a directed acyclic graph (**DAG**). The DAG can be run once or scheduled to run at a fixed interval of daily, weekly, every hour, every five minutes, etc. As we discuss orchestration throughout this book, we assume that an orchestration system stays online with high availability. This allows the orchestration system to sense and monitor constantly without human intervention and run new jobs anytime they are deployed. An orchestration system monitors jobs that it manages and kicks off new tasks as internal DAG dependencies are completed. It can also monitor external systems and tools to watch for data to arrive and criteria to be met. When certain conditions go out of bounds, the system also sets error conditions and sends alerts through email or other channels. You might set an expected completion time of 10 a.m. for overnight daily data pipelines. If jobs are not done by this time, alerts go out to data engineers and consumers. Orchestration systems also build job history capabilities, visualization, and alerting. Advanced orchestration engines can backfill new DAGs or individual tasks as they are added to a DAG. They also support dependencies over a time range. For example, a monthly reporting job might check that an ETL job has been completed for the full month before starting. Orchestration has long been a key capability for data processing but was not often top of mind nor accessible to anyone except the largest companies. Enterprises used various tools to manage job flows, but these were expensive, out of reach of small startups, and generally not extensible. Apache Oozie was extremely popular in the 2010s, but it was designed to work within a Hadoop cluster and was difficult to use in a more heterogeneous environment. Facebook developed Dataswarm for internal use in the late 2000s; this inspired popular tools such as Airflow, introduced by Airbnb in 2014. Airflow was open source from its inception, and was widely adopted. It was written in Python, making it highly extensible to almost any use case imaginable. While many other interesting open source orchestration projects exist, such as Luigi and Conductor, Airflow is arguably the mindshare leader for the time being. Airflow arrived just as data processing was becoming more abstract and accessible, and engineers were increasingly interested in coordinating complex flows across multiple processors and storage systems, especially in cloud environments. At this writing, several nascent

open source projects aim to mimic the best elements of Airflow's core design while improving on it in key areas. Some of the most interesting examples are Prefect and Dagster, which aim to improve the portability and testability of DAGs to allow engineers to move from local development to production more easily. Argo is an orchestration engine built around Kubernetes primitives; Metaflow is an open source project out of Netflix that aims to improve data science orchestration. We must point out that orchestration is strictly a batch concept. The streaming alternative to orchestrated task DAGs is the streaming DAG. Streaming DAGs remain challenging to build and maintain, but next-generation streaming platforms such as Pulsar aim to dramatically reduce the engineering and operational burden. We talk more about these developments Chapter 8 .

Chúng tôi cho rằng điều phối quan trọng vì chúng tôi xem nó thực sự là trọng tâm của cả nền tảng dữ liệu lẫn vòng đời dữ liệu, vòng đời phát triển phần mềm khi nó liên quan đến dữ liệu. — Nick Schrock, người sáng lập Elementl. Điều phối không chỉ là một tiến trình DataOps trung tâm, mà còn là một phần then chốt của luồng kỹ thuật và triển khai cho các công việc dữ liệu. Vậy, điều phối là gì? Điều phối là tiến trình điều phối nhiều công việc để chạy nhanh nhất và hiệu quả nhất có thể theo một nhịp được lập lịch. Chẳng hạn, người ta thường gọi các công cụ điều phối như Apache Airflow là các bộ lập lịch (scheduler). Điều này không hoàn toàn chính xác. Một bộ lập lịch thuần túy, như cron, chỉ ý thức về thời gian; một công cụ điều phối tích hợp siêu dữ liệu về các phụ thuộc công việc, thường ở dạng một đồ thị có hướng phi chu trình (DAG). DAG có thể được chạy một lần hoặc được lập lịch chạy theo một khoảng cố định hằng ngày, hằng tuần, mỗi giờ, mỗi năm phút, v.v. Khi chúng tôi thảo luận điều phối xuyên suốt cuốn sách này, chúng tôi giả định rằng một hệ thống điều phối luôn trực tuyến với tính sẵn sàng cao. Điều này cho phép hệ thống điều phối cảm nhận và giám sát liên tục mà không cần con người can thiệp và chạy các công việc mới bất cứ khi nào chúng được triển khai. Một hệ thống điều phối giám sát các công việc mà nó quản lý và khởi động các tác vụ mới khi các phụ thuộc DAG nội bộ được hoàn thành. Nó cũng có thể giám sát các hệ thống và công cụ bên ngoài để theo dõi dữ liệu đến và các tiêu chí được thỏa mãn. Khi một số điều kiện vượt ngoài ngưỡng, hệ thống cũng đặt các điều kiện lỗi và gửi cảnh báo qua email hoặc các kênh khác. Bạn có thể đặt một thời điểm hoàn thành dự kiến là 10 giờ sáng cho các đường ống dữ liệu chạy qua đêm hằng ngày. Nếu các công việc không xong vào thời điểm này, các cảnh báo được gửi đến các kỹ sư dữ liệu và người tiêu thụ. Các hệ thống điều phối cũng xây dựng các khả năng lịch sử công việc, trực quan hóa, và cảnh báo. Các công cụ điều phối nâng cao có thể nạp lại (backfill) các DAG mới hoặc các tác vụ riêng lẻ khi chúng được thêm vào một DAG. Chúng cũng hỗ trợ các phụ thuộc trên một khoảng thời gian. Ví dụ, một công việc báo cáo hằng tháng có thể kiểm tra rằng một công việc ETL đã được hoàn thành cho trọn tháng trước khi bắt đầu. Điều phối từ lâu đã là một khả năng then chốt cho xử lý dữ liệu nhưng không thường ở trong tâm trí cũng không dễ tiếp cận với bất cứ ai ngoại trừ các công ty lớn nhất. Các doanh nghiệp đã dùng nhiều công cụ khác nhau để quản lý các luồng công việc, nhưng những công cụ này đắt đỏ, ngoài tầm với của các startup nhỏ, và nhìn chung không có khả năng mở rộng. Apache Oozie cực kỳ phổ biến trong thập niên 2010, nhưng nó được thiết kế để hoạt động trong một cụm Hadoop và khó dùng trong một môi trường không đồng nhất hơn. Facebook đã phát triển Dataswarm cho dùng nội bộ vào cuối thập niên 2000; điều này đã truyền cảm hứng cho các công cụ phổ biến như Airflow, được Airbnb giới thiệu vào năm 2014. Airflow là mã nguồn mở ngay từ đầu, và được áp dụng rộng rãi. Nó được viết bằng Python, khiến nó có khả năng mở rộng cao đến gần như bất kỳ trường hợp sử dụng nào có thể hình dung. Trong khi nhiều dự án điều phối mã nguồn mở thú vị khác tồn tại, chẳng hạn như Luigi và Conductor, Airflow được cho là dẫn đầu về mức độ phổ biến vào thời điểm hiện tại. Airflow ra đời ngay khi xử lý dữ liệu đang trở nên trừu tượng và dễ tiếp cận hơn, và các kỹ sư ngày càng quan tâm đến việc điều phối các luồng phức tạp qua nhiều bộ xử lý và hệ thống lưu trữ, đặc biệt trong các môi trường đám mây. Tại

thời điểm viết cuốn sách này, một số dự án mã nguồn mở non trẻ nhằm mô phỏng các yếu tố tốt nhất trong thiết kế cốt lõi của Airflow trong khi cải thiện nó ở các lĩnh vực then chốt. Một số ví dụ thú vị nhất là Prefect và Dagster, nhằm cải thiện tính khả chuyển và khả năng kiểm thử của các DAG để cho phép các kỹ sư chuyển từ phát triển cục bộ sang sản xuất dễ dàng hơn. Argo là một công cụ điều phối được xây dựng xoay quanh các nguyên thủy Kubernetes; Metaflow là một dự án mã nguồn mở ra đời từ Netflix nhằm cải thiện điều phối khoa học dữ liệu. Chúng tôi phải chỉ ra rằng điều phối là một khái niệm hoàn toàn thuộc về theo lô. Phương án thay thế theo luồng cho các DAG tác vụ được điều phối là DAG theo luồng. Các DAG theo luồng vẫn còn thách thức để xây dựng và duy trì, nhưng các nền tảng theo luồng thế hệ tiếp theo như Pulsar nhằm giảm đáng kể gánh nặng kỹ thuật và vận hành. Chúng ta nói thêm về các phát triển này trong Chương 8.

DataOps — DataOps

DataOps maps the best practices of Agile methodology, DevOps, and statistical process control (SPC) to data. Whereas DevOps aims to improve the release and quality of software products, DataOps does the same thing for data products. Data products differ from software products because of the way data is used. A software product provides specific functionality and technical features for end users. By contrast, a data product is built around sound business logic and metrics, whose users make decisions or build models that perform automated actions. A data engineer must understand both the technical aspects of building software products, and the business logic, quality, and metrics that will create excellent data products. Like DevOps, DataOps borrows much from lean manufacturing and supply chain management, mixing people, processes, and technology to reduce time to value. As Data Kitchen (experts in DataOps) describes it: 6 DataOps is a collection of technical practices, workflows, cultural norms, and architectural patterns that enable:

DataOps ánh xạ các thực hành tốt nhất của phương pháp Agile, DevOps, và kiểm soát quy trình bằng thống kê (SPC) vào dữ liệu. Trong khi DevOps nhằm cải thiện việc phát hành và chất lượng của các sản phẩm phần mềm, DataOps làm điều tương tự cho các sản phẩm dữ liệu. Các sản phẩm dữ liệu khác với các sản phẩm phần mềm vì cách dữ liệu được dùng. Một sản phẩm phần mềm cung cấp chức năng cụ thể và các tính năng kỹ thuật cho người dùng cuối. Ngược lại, một sản phẩm dữ liệu được xây dựng xoay quanh logic kinh doanh và các chỉ số đúng đắn, mà người dùng của nó đưa ra quyết định hoặc xây dựng các mô hình thực hiện các hành động tự động. Một kỹ sư dữ liệu phải hiểu cả các khía cạnh kỹ thuật của việc xây dựng các sản phẩm phần mềm, và logic kinh doanh, chất lượng, cùng các chỉ số sẽ tạo nên các sản phẩm dữ liệu xuất sắc. Giống như DevOps, DataOps vay mượn nhiều từ sản xuất tinh gọn (lean manufacturing) và quản lý chuỗi cung ứng, kết hợp con người, tiến trình và công nghệ để giảm thời gian tạo ra giá trị. Như Data Kitchen (các chuyên gia về DataOps) mô tả nó: DataOps là một tập hợp các thực hành kỹ thuật, quy trình làm việc, chuẩn mực văn hóa, và các mẫu kiến trúc cho phép:

Rapid innovation and experimentation delivering new insights to customers with increasing velocity

Đổi mới và thử nghiệm nhanh chóng cung cấp các hiểu biết mới cho khách hàng với vận tốc ngày càng tăng

Extremely high data quality and very low error rates

Chất lượng dữ liệu cực kỳ cao và tỷ lệ lỗi rất thấp

Collaboration across complex arrays of people, technology, and environments

Cộng tác qua các mảng phức tạp gồm con người, công nghệ, và môi trường

Clear measurement, monitoring, and transparency of results

Đo lường, giám sát, và minh bạch rõ ràng về kết quả

Lean practices (such as lead time reduction and minimizing defects) and the resulting improvements to quality and productivity are things we are glad to see gaining momentum both in software and data operations. First and foremost, DataOps is a set of cultural habits; the data engineering team needs to adopt a cycle of communicating and collaborating with the business, breaking down silos, continuously learning from successes and mistakes, and rapid iteration. Only when these cultural habits are set in place can the team get the best results from technology and tools. Depending on a company's data maturity, a data engineer has some options to build DataOps into the fabric of the overall data engineering lifecycle. If the company has no preexisting data infrastructure or practices, DataOps is very much a **greenfield** opportunity that can be baked in from day one. With an existing project or infrastructure that lacks DataOps, a data engineer can begin adding DataOps into workflows. We suggest first starting with observability and monitoring to get a window into the performance of a system, then adding in automation and incident response. A data engineer may work alongside an existing DataOps team to improve the data engineering lifecycle in a data-mature company. In all cases, a data engineer must be aware of the philosophy and technical aspects of DataOps. DataOps has three core technical elements: automation, monitoring and observability, and incident response (Figure 2-8). Let's look at each of these pieces and how they relate to the data engineering lifecycle.

Các thực hành tinh gọn (chẳng hạn như giảm thời gian dẫn (lead time) và giảm thiểu khiếm khuyết) và những cải thiện về chất lượng và năng suất từ đó là những điều chúng tôi vui mừng thấy đang lấy đà cả trong vận hành phần mềm và dữ liệu. Trước hết và trên hết, DataOps là một tập hợp các thói quen văn hóa; nhóm kỹ thuật dữ liệu cần áp dụng một chu kỳ giao tiếp và cộng tác với doanh nghiệp, phá bỏ các silo, liên tục học hỏi từ thành công và sai lầm, và lặp lại nhanh chóng. Chỉ khi các thói quen văn hóa này được thiết lập đúng chỗ thì nhóm mới có thể có được kết quả tốt nhất từ công nghệ và công cụ. Tùy vào độ chín dữ liệu của một công ty, một kỹ sư dữ liệu có một số lựa chọn để xây dựng DataOps vào trong cấu trúc của vòng đời kỹ thuật dữ liệu tổng thể. Nếu công ty không có hạ tầng hoặc thực hành dữ liệu sẵn có, DataOps rất nhiều là một cơ hội greenfield có thể được đưa vào ngay từ ngày đầu tiên. Với một dự án hoặc hạ tầng hiện có thiếu DataOps, một kỹ sư dữ liệu có thể bắt đầu thêm DataOps vào các quy trình làm việc. Chúng tôi gợi ý trước tiên bắt đầu với khả năng quan sát và giám sát để có một cửa sổ nhìn vào hiệu năng của một hệ thống, rồi thêm vào tự động hóa và ứng phó sự cố. Một kỹ sư dữ liệu có thể làm việc cùng một nhóm DataOps hiện có để cải thiện vòng đời kỹ thuật dữ liệu trong một công ty chín về dữ liệu. Trong mọi trường hợp, một kỹ sư dữ liệu phải ý thức về triết lý và các khía cạnh kỹ thuật của DataOps. DataOps có ba yếu tố kỹ thuật cốt lõi: tự động hóa, giám sát và khả năng quan sát, và ứng phó sự cố (Hình 2-8). Hãy xem từng mảnh ghép này và cách chúng liên quan đến vòng đời kỹ thuật dữ liệu.

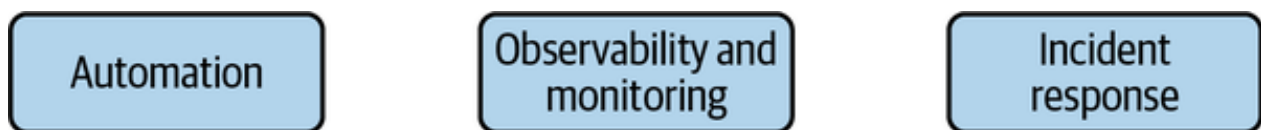


Figure 2-8. The three pillars of DataOps

Hình 2-8. Ba trụ cột của DataOps

Automation — Tự động hóa

Automation enables reliability and consistency in the DataOps process and allows data engineers to quickly deploy new product features, and improvements to existing workflows. DataOps automation has a similar framework and workflow to DevOps, consisting of change management (environment, code, and data version control), continuous integration/continuous deployment (CI/CD), and configuration as code. Like DevOps, DataOps practices monitor and maintain the reliability of technology and systems (data pipelines, orchestration, etc.), with the added dimension of checking for data quality, data/model drift, metadata integrity, and more. Let's briefly discuss the evolution of DataOps automation within a hypothetical organization. An organization with a low level of DataOps maturity often attempts to schedule multiple stages of data transformation processes using cron jobs. This works well for a while. As data pipelines become more complicated, several things are likely to happen. If the cron jobs are hosted on a cloud instance, the instance may have an operational problem, causing the jobs to stop running unexpectedly. As the spacing between jobs becomes tighter, a job will eventually run long, causing a subsequent job to fail or produce stale data. Engineers may not be aware of job failures until they hear from analysts that their reports are out-of-date. As the organization's data maturity grows, data engineers will typically adopt an orchestration framework, perhaps Airflow or Dagster. Data engineers are aware that Airflow presents an operational burden, but the benefits of orchestration eventually outweigh the complexity. Engineers will gradually migrate their cron jobs to Airflow jobs. Now, dependencies are checked before jobs run. More transformation jobs can be packed into a given time because each job can start as soon as upstream data is ready rather than at a fixed, predetermined time.

Tự động hóa cho phép độ tin cậy và tính nhất quán trong tiến trình DataOps và cho phép các kỹ sư dữ liệu nhanh chóng triển khai các tính năng sản phẩm mới, và các cải thiện cho các quy trình làm việc hiện có. Tự động hóa DataOps có một khung và quy trình làm việc tương tự DevOps, gồm quản lý thay đổi (kiểm soát phiên bản môi trường, mã, và dữ liệu), tích hợp liên tục/triển khai liên tục (CI/CD), và cấu hình dưới dạng mã. Giống như DevOps, các thực hành DataOps giám sát và duy trì độ tin cậy của công nghệ và các hệ thống (đường ống dữ liệu, điều phối, v.v.), với chiều bổ sung là kiểm tra chất lượng dữ liệu, độ trôi (drift) dữ liệu/mô hình, tính toàn vẹn siêu dữ liệu, và nhiều hơn nữa. Hãy ngắn gọn thảo luận sự tiến hóa của tự động hóa DataOps trong một tổ chức giả định. Một tổ chức với mức độ chín DataOps thấp thường cố gắng lập lịch nhiều giai đoạn của các tiến trình biến đổi dữ liệu bằng các công việc cron (cron job). Điều này hoạt động tốt một thời gian. Khi các đường ống dữ liệu trở nên phức tạp hơn, một số điều có khả năng xảy ra. Nếu các công việc cron được lưu trữ trên một phiên bản đám mây, phiên bản đó có thể gặp một vấn đề vận hành, khiến các công việc ngừng chạy bất ngờ. Khi khoảng cách giữa các công việc trở nên sát sao hơn, một công việc cuối cùng sẽ chạy lâu, khiến một công việc kế tiếp thất bại hoặc tạo ra dữ liệu cũ. Các kỹ sư có thể không ý thức về các thất bại công việc cho đến khi họ nghe từ các nhà phân tích rằng các báo cáo của họ đã lỗi thời. Khi độ chín dữ liệu của tổ chức tăng lên, các kỹ sư dữ liệu thường sẽ áp dụng một framework điều phối, có lẽ là Airflow hoặc Dagster. Các kỹ sư dữ liệu ý thức rằng Airflow đặt ra một gánh nặng vận hành, nhưng các lợi ích của điều phối cuối cùng lớn hơn độ phức tạp. Các kỹ sư sẽ dần di chuyển các công việc cron của họ sang các công việc Airflow. Giờ đây, các phụ thuộc được kiểm tra trước khi các công việc chạy. Nhiều công việc biến đổi hơn có thể được nhồi vào một khoảng thời gian nhất định vì mỗi công việc có thể bắt đầu ngay khi dữ liệu thượng nguồn sẵn sàng thay vì vào một thời điểm cố định, định trước.

The data engineering team still has room for operational improvements. A data scientist eventually deploys a broken DAG, bringing down the Airflow web server and leaving the data team operationally blind. After enough such headaches, the data engineering team members realize

that they need to stop allowing manual DAG deployments. In their next phase of operational maturity, they adopt automated DAG deployment. DAGs are tested before deployment, and monitoring processes ensure that the new DAGs start running properly. In addition, data engineers block the deployment of new Python dependencies until installation is validated. After automation is adopted, the data team is much happier and experiences far fewer headaches. One of the tenets of the DataOps Manifesto is “Embrace change.” This does not mean change for the sake of change, but goal-oriented change. At each stage of our automation journey, opportunities exist for operational improvement. Even at the high level of maturity that we’ve described here, further room for improvement remains. Engineers might embrace a next-generation orchestration framework that builds in better metadata capabilities. Or they might try to develop a framework that builds DAGs automatically based on data-lineage specifications. The main point is that engineers constantly seek to implement improvements in automation that will reduce their workload and increase the value that they deliver to the business.

Nhóm kỹ thuật dữ liệu vẫn còn dư địa cho các cải thiện vận hành. Một nhà khoa học dữ liệu cuối cùng triển khai một DAG hỏng, làm sập máy chủ web Airflow và khiến nhóm dữ liệu mù về mặt vận hành. Sau đủ những cơn đau đầu như vậy, các thành viên nhóm kỹ thuật dữ liệu nhận ra rằng họ cần ngừng cho phép triển khai DAG thủ công. Trong giai đoạn chín vận hành tiếp theo của họ, họ áp dụng triển khai DAG tự động. Các DAG được kiểm thử trước khi triển khai, và các tiến trình giám sát đảm bảo rằng các DAG mới bắt đầu chạy đúng. Ngoài ra, các kỹ sư dữ liệu chặn việc triển khai các phụ thuộc Python mới cho đến khi việc cài đặt được xác thực. Sau khi tự động hóa được áp dụng, nhóm dữ liệu vui vẻ hơn nhiều và trải qua ít cơn đau đầu hơn hẳn. Một trong các nguyên lý của DataOps Manifesto là “Đón nhận thay đổi”. Điều này không có nghĩa là thay đổi vì mục đích thay đổi, mà là thay đổi hướng đến mục tiêu. Tại mỗi giai đoạn của hành trình tự động hóa của chúng ta, các cơ hội tồn tại cho cải thiện vận hành. Ngay cả ở mức độ chín cao mà chúng tôi đã mô tả ở đây, dư địa cải thiện vẫn còn. Các kỹ sư có thể đón nhận một framework điều phối thể hệ tiếp theo tích hợp các khả năng siêu dữ liệu tốt hơn. Hoặc họ có thể thử phát triển một framework xây dựng các DAG một cách tự động dựa trên các đặc tả truy vết nguồn gốc dữ liệu. Điểm chính là các kỹ sư liên tục tìm cách triển khai các cải thiện trong tự động hóa nhằm giảm khối lượng công việc của họ và tăng giá trị mà họ cung cấp cho doanh nghiệp.

Observability and monitoring — Khả năng quan sát và giám sát

As we tell our clients, “Data is a silent killer.” We’ve seen countless examples of bad data lingering in reports for months or years. Executives may make key decisions from this bad data, discovering the error only much later. The outcomes are usually bad and sometimes catastrophic for the business. Initiatives are undermined and destroyed, years of work wasted. In some of the worst cases, bad data may lead companies to financial ruin. Another horror story occurs when the systems that create the data for reports randomly stop working, resulting in reports being delayed by several days. The data team doesn’t know until they’re asked by stakeholders why reports are late or producing stale information. Eventually, various stakeholders lose trust in the capabilities of the core data team and start their own splinter teams. The result is many different unstable systems, inconsistent reports, and silos. If you’re not observing and monitoring your data and the systems that produce the data, you’re inevitably going to experience your own data horror story. Observability, monitoring, logging, alerting, and tracing are all critical to getting ahead of any problems along the data engineering lifecycle. We recommend you incorporate SPC to understand whether events being monitored are out of line and which incidents are worth responding to. Petrella’s DODD method mentioned previously in this chapter provides an excellent framework for thinking about data observability.

DODD is much like test-driven development (TDD) in software engineering:

Như chúng tôi nói với các khách hàng của mình, “Dữ liệu là một kẻ giết người thầm lặng.” Chúng tôi đã thấy vô số ví dụ về dữ liệu xấu lẫn lộn trong các báo cáo hàng tháng hoặc hàng năm. Các lãnh đạo có thể đưa ra các quyết định then chốt từ dữ liệu xấu này, chỉ phát hiện ra lỗi muộn hơn nhiều. Các kết quả thường tệ và đôi khi thảm họa cho doanh nghiệp. Các sáng kiến bị phá hoại và hủy hoại, hàng năm trời công sức lãng phí. Trong một số trường hợp tệ nhất, dữ liệu xấu có thể dẫn các công ty đến chỗ phá sản tài chính. Một câu chuyện kinh hoàng khác xảy ra khi các hệ thống tạo ra dữ liệu cho báo cáo ngẫu nhiên ngừng hoạt động, dẫn đến các báo cáo bị trễ vài ngày. Nhóm dữ liệu không biết cho đến khi họ được các bên liên quan hỏi tại sao các báo cáo bị trễ hoặc tạo ra thông tin cũ. Cuối cùng, các bên liên quan khác nhau mất lòng tin vào các năng lực của nhóm dữ liệu cốt lõi và bắt đầu các nhóm ly khai của riêng họ. Kết quả là nhiều hệ thống bất ổn khác nhau, các báo cáo bất nhất, và các silo. Nếu bạn không quan sát và giám sát dữ liệu của bạn và các hệ thống tạo ra dữ liệu, bạn chắc chắn sẽ trải qua câu chuyện dữ liệu kinh hoàng của riêng mình. Khả năng quan sát, giám sát, ghi nhật ký, cảnh báo, và truy vết (tracing) đều then chốt để đi trước bất kỳ vấn đề nào dọc theo vòng đời kỹ thuật dữ liệu. Chúng tôi khuyên bạn kết hợp SPC để hiểu liệu các sự kiện đang được giám sát có nằm ngoài giới hạn không và sự cố nào đáng để ứng phó. Phương pháp DODD của Petrella đã đề cập trước đó trong chương này cung cấp một khung tuyệt vời để suy nghĩ về khả năng quan sát dữ liệu. DODD rất giống phát triển hướng kiểm thử (TDD) trong kỹ thuật phần mềm:

The purpose of DODD is to give everyone involved in the data chain visibility into the data and data applications so that everyone involved in the data value chain has the ability to identify changes to the data or data applications at every step—from ingestion to transformation to analysis—to help troubleshoot or prevent data issues. DODD focuses on making data observability a first-class consideration in the data engineering lifecycle. 7

Mục đích của DODD là cho tất cả mọi người tham gia vào chuỗi dữ liệu khả năng nhìn thấy dữ liệu và các ứng dụng dữ liệu để mọi người tham gia vào chuỗi giá trị dữ liệu có khả năng xác định các thay đổi đối với dữ liệu hoặc các ứng dụng dữ liệu ở mọi bước — từ thu nạp đến biến đổi đến phân tích — nhằm giúp khắc phục sự cố hoặc ngăn ngừa các vấn đề dữ liệu. DODD tập trung vào việc biến khả năng quan sát dữ liệu thành một cân nhắc hàng nhất trong vòng đời kỹ thuật dữ liệu. 7

We cover many aspects of monitoring and observability throughout the data engineering lifecycle in later chapters.

Chúng tôi đề cập nhiều khía cạnh của giám sát và khả năng quan sát xuyên suốt vòng đời kỹ thuật dữ liệu trong các chương sau.

Incident response — Ứng phó sự cố

A high-functioning data team using DataOps will be able to ship new data products quickly. But mistakes will inevitably happen. A system may have downtime, a new data model may break downstream reports, an ML model may become stale and provide bad predictions—countless problems can interrupt the data engineering lifecycle. Incident response is about using the automation and observability capabilities mentioned previously to rapidly identify root causes of an incident and resolve it as reliably and quickly as possible. Incident response isn’t just about technology and tools, though these are beneficial; it’s also about open and blameless communication, both on the data engineering team and across the organization. As Werner Vogels is famous for saying, “Everything breaks all the time.” Data engineers must be prepared for a disaster and ready

to respond as swiftly and efficiently as possible. Data engineers should proactively find issues before the business reports them. Failure happens, and when the stakeholders or end users see problems, they will present them. They will be unhappy to do so. The feeling is different when they go to raise those issues to a team and see that they are actively being worked on to resolve already. Which team's state would you trust more as an end user? Trust takes a long time to build and can be lost in minutes. Incident response is as much about retroactively responding to incidents as proactively addressing them before they happen.

Một nhóm dữ liệu vận hành tốt sử dụng DataOps sẽ có khả năng giao các sản phẩm dữ liệu mới một cách nhanh chóng. Nhưng các sai lầm chắc chắn sẽ xảy ra. Một hệ thống có thể có thời gian ngừng hoạt động, một mô hình dữ liệu mới có thể làm hỏng các báo cáo hạ nguồn, một mô hình ML có thể trở nên cũ và đưa ra các dự đoán xấu — vô số vấn đề có thể làm gián đoạn vòng đời kỹ thuật dữ liệu. Ứng phó sự cố là về việc dùng các khả năng tự động hóa và khả năng quan sát đã đề cập trước đó để nhanh chóng xác định các nguyên nhân gốc rễ của một sự cố và giải quyết nó một cách đáng tin cậy và nhanh nhất có thể. Ứng phó sự cố không chỉ là về công nghệ và công cụ, dù những thứ này có lợi; nó còn là về giao tiếp cởi mở và không đổ lỗi, cả trong nhóm kỹ thuật dữ liệu và xuyên suốt tổ chức. Như Werner Vogels nổi tiếng với câu nói, “Mọi thứ đều hỏng mọi lúc.” Các kỹ sư dữ liệu phải sẵn sàng cho một thảm họa và sẵn sàng ứng phó nhanh và hiệu quả nhất có thể. Các kỹ sư dữ liệu nên chủ động tìm ra các vấn đề trước khi doanh nghiệp báo cáo chúng. Thất bại xảy ra, và khi các bên liên quan hoặc người dùng cuối thấy các vấn đề, họ sẽ trình bày chúng. Họ sẽ không vui khi làm vậy. Cảm giác sẽ khác khi họ đi nêu các vấn đề đó với một nhóm và thấy rằng chúng đang được tích cực giải quyết. Là một người dùng cuối, bạn sẽ tin cậy trạng thái của nhóm nào hơn? Lòng tin mất một thời gian dài để xây dựng và có thể mất đi trong vài phút. Ứng phó sự cố vừa là về việc ứng phó hồi cứu với các sự cố vừa là về việc chủ động giải quyết chúng trước khi chúng xảy ra.

DataOps summary — Tóm tắt DataOps

At this point, DataOps is still a work in progress. Practitioners have done a good job of adapting DevOps principles to the data domain and mapping out an initial vision through the DataOps Manifesto and other resources. Data engineers would do well to make DataOps practices a high priority in all of their work. The up-front effort will see a significant long-term payoff through faster delivery of products, better reliability and accuracy of data, and greater overall value for the business. The state of operations in data engineering is still quite immature compared with software engineering. Many data engineering tools, especially legacy monoliths, are not automation-first. A recent movement has arisen to adopt automation best practices across the data engineering lifecycle. Tools like Airflow have paved the way for a new generation of automation and data management tools. The general practices we describe for DataOps are aspirational, and we suggest companies try to adopt them to the fullest extent possible, given the tools and knowledge available today.

Tại thời điểm này, DataOps vẫn còn là một công trình đang tiến hành. Các thực hành viên đã làm tốt việc thích nghi các nguyên tắc DevOps cho miền dữ liệu và phác thảo một tầm nhìn ban đầu thông qua DataOps Manifesto và các tài nguyên khác. Các kỹ sư dữ liệu sẽ làm tốt khi đặt các thực hành DataOps thành ưu tiên cao trong tất cả công việc của họ. nỗ lực trả trước sẽ thấy một sự đền đáp đáng kể dài hạn thông qua việc giao sản phẩm nhanh hơn, độ tin cậy và độ chính xác của dữ liệu tốt hơn, và giá trị tổng thể lớn hơn cho doanh nghiệp. Tình trạng vận hành trong kỹ thuật dữ liệu vẫn còn khá non nớt so với kỹ thuật phần mềm. Nhiều công cụ kỹ thuật dữ liệu, đặc biệt là các monolith cũ kỹ, không ưu tiên tự động hóa. Một phong trào gần đây đã nổi lên nhằm áp dụng các thực hành tự động hóa

tốt nhất xuyên suốt vòng đời kỹ thuật dữ liệu. Các công cụ như Airflow đã mở đường cho một thể hệ công cụ tự động hóa và quản lý dữ liệu mới. Các thực hành chung mà chúng tôi mô tả cho DataOps là khát vọng, và chúng tôi gợi ý các công ty cố gắng áp dụng chúng ở mức trọn vẹn nhất có thể, với các công cụ và kiến thức có sẵn ngày nay.

Data Architecture — Kiến trúc dữ liệu

A data architecture reflects the current and future state of data systems that support an organization's long-term data needs and strategy. Because an organization's data requirements will likely change rapidly, and new tools and practices seem to arrive on a near-daily basis, data engineers must understand good data architecture. Chapter 3 covers data architecture in depth, but we want to highlight here that data architecture is an undercurrent of the data engineering lifecycle. A data engineer should first understand the needs of the business and gather requirements for new use cases. Next, a data engineer needs to translate those requirements to design new ways to capture and serve data, balanced for cost and operational simplicity. This means knowing the trade-offs with design patterns, technologies, and tools in source systems, ingestion, storage, transformation, and serving data. This doesn't imply that a data engineer is a data architect, as these are typically two separate roles. If a data engineer works alongside a data architect, the data engineer should be able to deliver on the data architect's designs and provide architectural feedback.

Một kiến trúc dữ liệu phản ánh trạng thái hiện tại và tương lai của các hệ thống dữ liệu hỗ trợ nhu cầu và chiến lược dữ liệu dài hạn của một tổ chức. Vì các yêu cầu dữ liệu của một tổ chức có khả năng thay đổi nhanh chóng, và các công cụ và thực hành mới dường như đến gần như hằng ngày, các kỹ sư dữ liệu phải hiểu kiến trúc dữ liệu tốt. Chương 3 đề cập kiến trúc dữ liệu chuyên sâu, nhưng chúng tôi muốn làm nổi bật ở đây rằng kiến trúc dữ liệu là một dòng chảy ngầm của vòng đời kỹ thuật dữ liệu. Một kỹ sư dữ liệu trước tiên nên hiểu các nhu cầu của doanh nghiệp và thu thập các yêu cầu cho các trường hợp sử dụng mới. Tiếp theo, một kỹ sư dữ liệu cần chuyển dịch các yêu cầu đó để thiết kế các cách mới để nắm bắt và phục vụ dữ liệu, cân bằng cho chi phí và sự đơn giản vận hành. Điều này nghĩa là biết các đánh đổi với các mẫu thiết kế, công nghệ, và công cụ trong các hệ thống nguồn, thu nạp, lưu trữ, biến đổi, và phục vụ dữ liệu. Điều này không ngụ ý rằng một kỹ sư dữ liệu là một kiến trúc sư dữ liệu, vì đây thường là hai vai trò tách biệt. Nếu một kỹ sư dữ liệu làm việc cùng một kiến trúc sư dữ liệu, kỹ sư dữ liệu nên có khả năng hiện thực hóa các thiết kế của kiến trúc sư dữ liệu và cung cấp phản hồi kiến trúc.

Software Engineering — Kỹ thuật phần mềm

Software engineering has always been a central skill for data engineers. In the early days of contemporary data engineering (2000–2010), data engineers worked on low-level frameworks and wrote MapReduce jobs in C, C++, and Java. At the peak of the big data era (the mid-2010s), engineers started using frameworks that abstracted away these low-level details. This abstraction continues today. Cloud data warehouses support powerful transformations using SQL semantics; tools like Spark have become more user-friendly, transitioning away from low-level coding details and toward easy-to-use dataframes. Despite this abstraction, software engineering is still critical to data engineering. We want to briefly discuss a few common areas of software engineering that apply to the data engineering lifecycle.

Kỹ thuật phần mềm luôn là một kỹ năng trung tâm cho các kỹ sư dữ liệu. Trong những ngày đầu của kỹ thuật dữ liệu đương đại (2000–2010), các kỹ sư dữ liệu làm việc trên các

framework cấp thấp và viết các công việc MapReduce bằng C, C++, và Java. Tại đỉnh điểm của kỷ nguyên big data (giữa thập niên 2010), các kỹ sư bắt đầu dùng các framework trừu tượng hóa đi các chi tiết cấp thấp này. Sự trừu tượng hóa này tiếp tục đến ngày nay. Các kho dữ liệu đám mây hỗ trợ các biến đổi mạnh mẽ bằng ngữ nghĩa SQL; các công cụ như Spark đã trở nên thân thiện với người dùng hơn, chuyển dịch khỏi các chi tiết viết mã cấp thấp và hướng tới các dataframe để dùng. Bất chấp sự trừu tượng hóa này, kỹ thuật phần mềm vẫn then chốt với kỹ thuật dữ liệu. Chúng tôi muốn ngăn gọn thảo luận một vài lĩnh vực phổ biến của kỹ thuật phần mềm áp dụng cho vòng đời kỹ thuật dữ liệu.

Core data processing code — Mã xử lý dữ liệu cốt lõi

Though it has become more abstract and easier to manage, core data processing code still needs to be written, and it appears throughout the data engineering lifecycle. Whether in ingestion, transformation, or data serving, data engineers need to be highly proficient and productive in frameworks and languages such as Spark, SQL, or Beam; we reject the notion that SQL is not code. It's also imperative that a data engineer understand proper code-testing methodologies, such as unit, regression, integration, end-to-end, and smoke.

Mặc dù nó đã trở nên trừu tượng hơn và dễ quản lý hơn, mã xử lý dữ liệu cốt lõi vẫn cần được viết, và nó xuất hiện xuyên suốt vòng đời kỹ thuật dữ liệu. Dù trong thu nạp, biến đổi, hay phục vụ dữ liệu, các kỹ sư dữ liệu cần thành thạo và năng suất cao trong các framework và ngôn ngữ như Spark, SQL, hoặc Beam; chúng tôi bác bỏ quan niệm rằng SQL không phải là mã. Cũng bắt buộc rằng một kỹ sư dữ liệu hiểu các phương pháp kiểm thử mã đúng đắn, chẳng hạn như kiểm thử đơn vị, hồi quy, tích hợp, đầu-cuối, và smoke.

Development of open source frameworks — Phát triển các framework mã nguồn mở

Many data engineers are heavily involved in developing open source frameworks. They adopt these frameworks to solve specific problems in the data engineering lifecycle, and then continue developing the framework code to improve the tools for their use cases and contribute back to the community. In the big data era, we saw a Cambrian explosion of data-processing frameworks inside the Hadoop ecosystem. These tools primarily focused on transforming and serving parts of the data engineering lifecycle. Data engineering tool speciation has not ceased or slowed down, but the emphasis has shifted up the ladder of abstraction, away from direct data processing. This new generation of open source tools assists engineers in managing, enhancing, connecting, optimizing, and monitoring data. For example, Airflow dominated the orchestration space from 2015 until the early 2020s. Now, a new batch of open source competitors (including Prefect, Dagster, and Metaflow) has sprung up to fix perceived limitations of Airflow, providing better metadata handling, portability, and dependency management. Where the future of orchestration goes is anyone's guess. Before data engineers begin engineering new internal tools, they would do well to survey the landscape of publicly available tools. Keep an eye on the **total cost of ownership** (TCO) and opportunity cost associated with implementing a tool. There is a good chance that an open source project already exists to address the problem they're looking to solve, and they would do well to collaborate rather than reinventing the wheel.

Nhiều kỹ sư dữ liệu tham gia sâu vào việc phát triển các framework mã nguồn mở. Họ áp dụng các framework này để giải quyết các vấn đề cụ thể trong vòng đời kỹ thuật dữ liệu, rồi tiếp tục phát triển mã framework để cải thiện các công cụ cho các trường hợp sử dụng

của họ và đóng góp lại cho cộng đồng. Trong kỷ nguyên big data, chúng ta đã thấy một sự bùng nổ kiểu Cambri của các framework xử lý dữ liệu bên trong hệ sinh thái Hadoop. Các công cụ này chủ yếu tập trung vào các phần biến đổi và phục vụ của vòng đời kỹ thuật dữ liệu. Sự phân hóa loài (speciation) của công cụ kỹ thuật dữ liệu đã không dừng lại hay chậm lại, nhưng trọng tâm đã dịch chuyển lên cao trên nấc thang trừu tượng, rời xa khỏi xử lý dữ liệu trực tiếp. Thế hệ công cụ mã nguồn mở mới này trợ giúp các kỹ sư trong việc quản lý, nâng cao, kết nối, tối ưu, và giám sát dữ liệu. Ví dụ, Airflow thống trị không gian điều phối từ 2015 đến đầu thập niên 2020. Giờ đây, một loạt các đối thủ mã nguồn mở mới (bao gồm Prefect, Dagster, và Metaflow) đã nảy sinh để khắc phục các hạn chế được nhận thấy của Airflow, cung cấp xử lý siêu dữ liệu, tính khả chuyển, và quản lý phụ thuộc tốt hơn. Tương lai của điều phối đi về đâu thì ai cũng chỉ có thể đoán. Trước khi các kỹ sư dữ liệu bắt đầu xây dựng các công cụ nội bộ mới, họ sẽ làm tốt khi khảo sát bối cảnh các công cụ có sẵn công khai. Hãy để mắt đến tổng chi phí sở hữu (TCO) và chi phí cơ hội liên quan đến việc triển khai một công cụ. Có khả năng cao rằng một dự án mã nguồn mở đã tồn tại để giải quyết vấn đề họ đang tìm cách giải quyết, và họ sẽ làm tốt khi cộng tác thay vì phát minh lại bánh xe.

Streaming — Theo luồng

Streaming data processing is inherently more complicated than batch, and the tools and paradigms are arguably less mature. As streaming data becomes more pervasive in every stage of the data engineering lifecycle, data engineers face interesting software engineering problems. For instance, data processing tasks such as joins that we take for granted in the batch processing world often become more complicated in real time, requiring more complex software engineering. Engineers must also write code to apply a variety of windowing methods. Windowing allows real-time systems to calculate valuable metrics such as trailing statistics. Engineers have many frameworks to choose from, including various function platforms (OpenFaaS, AWS **Lambda**, Google Cloud Functions) for handling individual events or dedicated stream processors (Spark, Beam, Flink or Pulsar) for analyzing streams to support reporting and real-time actions.

Xử lý dữ liệu theo luồng vốn dĩ phức tạp hơn theo lô, và các công cụ cùng mô thức được cho là kém chín hơn. Khi dữ liệu theo luồng trở nên phổ biến hơn trong mọi giai đoạn của vòng đời kỹ thuật dữ liệu, các kỹ sư dữ liệu đối mặt với các vấn đề kỹ thuật phần mềm thú vị. Chẳng hạn, các tác vụ xử lý dữ liệu như kết (join) mà chúng ta coi là hiển nhiên trong thế giới xử lý theo lô thường trở nên phức tạp hơn trong thời gian thực, đòi hỏi kỹ thuật phần mềm phức tạp hơn. Các kỹ sư cũng phải viết mã để áp dụng nhiều phương pháp cửa sổ (windowing) khác nhau. Cửa sổ cho phép các hệ thống thời gian thực tính toán các chỉ số giá trị như các thống kê trễ (trailing statistics). Các kỹ sư có nhiều framework để chọn, bao gồm nhiều nền tảng hàm (OpenFaaS, AWS Lambda, Google Cloud Functions) để xử lý các sự kiện riêng lẻ hoặc các bộ xử lý luồng chuyên dụng (Spark, Beam, Flink hoặc Pulsar) để phân tích các luồng nhằm hỗ trợ báo cáo và các hành động thời gian thực.

Infrastructure as code — Hạ tầng dưới dạng mã

Infrastructure as code (IaC) applies software engineering practices to the configuration and management of infrastructure. The infrastructure management burden of the big data era has decreased as companies have migrated to managed big data systems (such as Databricks and Amazon EMR) and cloud data warehouses. When data engineers have to manage their infrastructure in a cloud environment, they increasingly do this through IaC frameworks rather than manually

spinning up instances and installing software. Several general-purpose and cloud-platform-specific frameworks allow automated infrastructure deployment based on a set of specifications. Many of these frameworks can manage cloud services as well as infrastructure. There is also a notion of IaC with containers and Kubernetes, using tools like Helm.

Hạ tầng dưới dạng mã (IaC) áp dụng các thực hành kỹ thuật phần mềm vào việc cấu hình và quản lý hạ tầng. Gánh nặng quản lý hạ tầng của kỷ nguyên big data đã giảm khi các công ty đã di chuyển sang các hệ thống big data được quản lý (chẳng hạn như Databricks và Amazon EMR) và các kho dữ liệu đám mây. Khi các kỹ sư dữ liệu phải quản lý hạ tầng của họ trong một môi trường đám mây, họ ngày càng làm điều này thông qua các framework IaC thay vì khởi tạo các phiên bản và cài đặt phần mềm một cách thủ công. Một số framework đa dụng và đặc thù theo nền tảng đám mây cho phép triển khai hạ tầng tự động dựa trên một tập các đặc tả. Nhiều framework trong số này có thể quản lý các dịch vụ đám mây cũng như hạ tầng. Cũng có một khái niệm IaC với các container và Kubernetes, dùng các công cụ như Helm.

These practices are a vital part of DevOps, allowing version control and repeatability of deployments. Naturally, these capabilities are precious throughout the data engineering lifecycle, especially as we adopt DataOps practices.

Các thực hành này là một phần thiết yếu của DevOps, cho phép kiểm soát phiên bản và khả năng lặp lại của các lần triển khai. Lẽ tự nhiên, các khả năng này quý giá xuyên suốt vòng đời kỹ thuật dữ liệu, đặc biệt khi chúng ta áp dụng các thực hành DataOps.

Pipelines as code — Đường ống dưới dạng mã

Pipelines as code is the core concept of present-day orchestration systems, which touch every stage of the data engineering lifecycle. Data engineers use code (typically Python) to declare data tasks and dependencies among them. The orchestration engine interprets these instructions to run steps using available resources.

Đường ống dưới dạng mã là khái niệm cốt lõi của các hệ thống điều phối ngày nay, vốn chạm đến mọi giai đoạn của vòng đời kỹ thuật dữ liệu. Các kỹ sư dữ liệu dùng mã (thường là Python) để khai báo các tác vụ dữ liệu và các phụ thuộc giữa chúng. Công cụ điều phối diễn giải các chỉ thị này để chạy các bước bằng các tài nguyên có sẵn.

General-purpose problem solving — Giải quyết vấn đề đa dụng

In practice, regardless of which high-level tools they adopt, data engineers will run into corner cases throughout the data engineering lifecycle that require them to solve problems outside the boundaries of their chosen tools and to write custom code. When using frameworks like Fivetran, Airbyte, or Singer, data engineers will encounter data sources without existing connectors and need to write something custom. They should be proficient in software engineering to understand APIs, pull and transform data, handle exceptions, and so forth.

Trong thực tế, bất kể họ áp dụng những công cụ cấp cao nào, các kỹ sư dữ liệu sẽ gặp phải các trường hợp góc ngách xuyên suốt vòng đời kỹ thuật dữ liệu đòi hỏi họ giải quyết các vấn đề ngoài ranh giới của các công cụ họ đã chọn và viết mã tùy chỉnh. Khi dùng các framework như Fivetran, Airbyte, hoặc Singer, các kỹ sư dữ liệu sẽ gặp các nguồn dữ liệu không có bộ kết nối sẵn có và cần viết một thứ gì đó tùy chỉnh. Họ nên thành thạo kỹ thuật phần mềm để hiểu các API, kéo và biến đổi dữ liệu, xử lý các ngoại lệ, v.v.

Conclusion — Kết luận

Most discussions we've seen in the past about data engineering involve technologies but miss the bigger picture of data lifecycle management. As technologies become more abstract and do more heavy lifting, a data engineer has the opportunity to think and act on a higher level. The data engineering lifecycle, supported by its undercurrents, is an extremely useful mental model for organizing the work of data engineering. We break the data engineering lifecycle into the following stages:

Hầu hết các thảo luận chúng tôi đã thấy trong quá khứ về kỹ thuật dữ liệu liên quan đến các công nghệ nhưng bỏ lỡ bức tranh lớn hơn về quản lý vòng đời dữ liệu. Khi các công nghệ trở nên trừu tượng hơn và làm nhiều việc nặng nhọc hơn, một kỹ sư dữ liệu có cơ hội suy nghĩ và hành động ở một cấp độ cao hơn. Vòng đời kỹ thuật dữ liệu, được hỗ trợ bởi các dòng chảy ngầm của nó, là một mô hình tư duy cực kỳ hữu ích để tổ chức công việc của kỹ thuật dữ liệu. Chúng tôi chia vòng đời kỹ thuật dữ liệu thành các giai đoạn sau:

Generation Storage

Sinh dữ liệu Lưu trữ

Ingestion

Thu nạp

Transformation

Biến đổi

Serving data

Phục vụ dữ liệu

Several themes cut across the data engineering lifecycle as well. These are the undercurrents of the data engineering lifecycle. At a high level, the undercurrents are as follows:

Một số chủ đề cũng cắt ngang vòng đời kỹ thuật dữ liệu. Đây là các dòng chảy ngầm của vòng đời kỹ thuật dữ liệu. Ở cấp độ cao, các dòng chảy ngầm như sau:

Security

An ninh

Data management

Quản lý dữ liệu

DataOps

DataOps

Data architecture

Kiến trúc dữ liệu

Orchestration

Điều phối

Software engineering

Kỹ thuật phần mềm

A data engineer has several top-level goals across the data lifecycle: produce optimum ROI and reduce costs (financial and opportunity), reduce risk (security, data quality), and maximize data value and utility. The next two chapters discuss how these elements impact good architecture design, along with choosing the right technologies. If you feel comfortable with these two topics, feel free to skip ahead to Part II, where we cover each of the stages of the data engineering lifecycle.

Một kỹ sư dữ liệu có một số mục tiêu cấp cao nhất xuyên suốt vòng đời dữ liệu: tạo ra ROI tối ưu và giảm chi phí (tài chính và cơ hội), giảm rủi ro (an ninh, chất lượng dữ liệu), và tối đa hóa giá trị và tính hữu dụng của dữ liệu. Hai chương tiếp theo thảo luận cách các yếu tố này tác động đến thiết kế kiến trúc tốt, cùng với việc chọn các công nghệ đúng. Nếu bạn thấy thoải mái với hai chủ đề này, cứ tự nhiên nhảy thẳng đến Phần II, nơi chúng tôi đề cập từng giai đoạn của vòng đời kỹ thuật dữ liệu.

Additional Resources

Managing the data engineering lifecycle:

“Staying Ahead of Debt” by Etai Mizrahi

Data transformation and processing:

“Data transformation” Wikipedia page

“Data processing” Wikipedia page

“A Comparison of Data Processing Frameworks” by Ludovic Santos

Undercurrents:

“The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing” by Tyler Akidau et al.

DataOps:

“Getting Started with DevOps Automation” by Jared Murrell

“Incident Management in the Age of DevOps” Atlassian web page

“The Seven Stages of Effective Incident Response” Atlassian web page

“Is DevOps Related to DataOps?” by Carol Jang and Jove Kuang

Data management:

DAMA International website

Data management and metadata:

“What Is Metadata” by Michelle Knight

Airbnb Data Portal:

“Democratizing Data at Airbnb” by Chris Williams et al.

“Five Steps to Begin Collecting the Value of Your Data” Lean-Data web page

Orchestration:

“An Introduction to Dagster: The Orchestrator for the Full Data Lifecycle” video by Nick Schrock
1 Evren Eryurek et al., Data Governance: The Definitive Guide (Sebastopol, CA: O’Reilly, 2021),
1, <https://oreil.ly/LFT4d> . 2 Eryurek, Data Governance , 5. 3 Chris Williams et al., “Democratizing

Data at Airbnb,” The Airbnb Tech Blog , May 12, 2017, <https://oreil.ly/dM332> . 4 Eryurek, Data Governance , 113. 5 Tyler Akidau et al., “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing,” Proceedings of the VLDB Endowment 8 (2015): 1792–1803, <https://oreil.ly/Z6XYy> . 6 “What Is DataOps,” DataKitchen FAQ page, accessed May 5, 2022, <https://oreil.ly/Ns06w> . 7 Andy Petrella, “Data Observability Driven Development: The Perfect Analogy for Beginners,” Kensu, accessed May 5, 2022, <https://oreil.ly/MxvSX> .

Chapter 3. Designing Good Data Architecture — Chương 3. Thiết kế kiến trúc dữ liệu tốt

Good **data architecture** provides seamless capabilities across every step of the data lifecycle and undercurrent. We'll begin by defining data architecture and then discuss components and considerations. We'll then touch on specific **batch** patterns (data warehouses, data lakes), **streaming** patterns, and patterns that unify batch and streaming. Throughout, we'll emphasize leveraging the capabilities of the cloud to deliver **scalability**, availability, and reliability.

Một kiến trúc dữ liệu (data architecture) tốt cung cấp các năng lực liền mạch xuyên suốt mọi bước của vòng đời dữ liệu và các dòng chảy ngầm (undercurrents). Chúng ta sẽ bắt đầu bằng việc định nghĩa kiến trúc dữ liệu rồi thảo luận về các thành phần và những điều cần cân nhắc. Tiếp đó, chúng ta sẽ đề cập tới các mẫu hình xử lý theo lô cụ thể (kho dữ liệu, hồ dữ liệu), các mẫu hình theo luồng, và các mẫu hình hợp nhất theo lô (batch) và theo luồng (streaming). Xuyên suốt, chúng ta sẽ nhấn mạnh việc tận dụng các năng lực của đám mây để mang lại khả năng mở rộng quy mô, tính sẵn sàng và độ tin cậy.

What Is Data Architecture? — Kiến trúc dữ liệu là gì?

Successful **data engineering** is built upon rock-solid data architecture. This chapter aims to review a few popular architecture approaches and frameworks, and then craft our opinionated definition of what makes “good” data architecture. Indeed, we won't make everyone happy. Still, we will lay out a pragmatic, domain-specific, working definition for data architecture that we think will work for companies of vastly different scales, business processes, and needs. What is data architecture? When you stop to unpack it, the topic becomes a bit murky; researching data architecture yields many inconsistent and often outdated definitions. It's a lot like when we defined data engineering in Chapter 1—there's no consensus. In a field that is constantly changing, this is to be expected. So what do we mean by data architecture for the purposes of this book? Before defining the term, it's essential to understand the context in which it sits. Let's briefly cover **enterprise architecture**, which will frame our definition of data architecture.

Kỹ thuật dữ liệu (data engineering) thành công được xây dựng trên một kiến trúc dữ liệu vững như bàn thạch. Chương này nhằm điểm qua một vài cách tiếp cận và khung kiến trúc phổ biến, rồi đưa ra định nghĩa mang quan điểm riêng của chúng tôi về điều gì làm nên kiến trúc dữ liệu “tốt”. Quả thực, chúng tôi sẽ không thể làm hài lòng tất cả mọi người. Dù vậy, chúng tôi sẽ trình bày một định nghĩa thực dụng, đặc thù theo lĩnh vực và có tính ứng dụng cho kiến trúc dữ liệu mà chúng tôi tin là sẽ phù hợp với các công ty có quy mô,

quy trình kinh doanh và nhu cầu hết sức khác nhau. Kiến trúc dữ liệu là gì? Khi bạn dừng lại để mổ xẻ nó, chủ đề này trở nên hơi mơ hồ; việc tra cứu kiến trúc dữ liệu cho ra nhiều định nghĩa thiếu nhất quán và thường đã lỗi thời. Điều này khá giống lúc chúng ta định nghĩa data engineering ở Chương 1 — không hề có sự đồng thuận. Trong một lĩnh vực thay đổi không ngừng, điều này là dễ hiểu. Vậy chúng tôi muốn nói gì khi dùng cụm từ kiến trúc dữ liệu trong phạm vi cuốn sách này? Trước khi định nghĩa thuật ngữ này, điều cốt yếu là phải hiểu bối cảnh mà nó nằm trong đó. Hãy điểm qua nhanh kiến trúc doanh nghiệp (enterprise architecture), vốn sẽ định hình định nghĩa của chúng ta về kiến trúc dữ liệu.

Enterprise Architecture, Defined — Định nghĩa kiến trúc doanh nghiệp

Enterprise architecture has many subsets, including business, technical, application, and data (Figure 3-1). As such, many frameworks and resources are devoted to enterprise architecture. In truth, architecture is a surprisingly controversial topic.

Kiến trúc doanh nghiệp có nhiều tập con, bao gồm kiến trúc nghiệp vụ, kỹ thuật, ứng dụng và dữ liệu (Hình 3-1). Vì vậy, có rất nhiều khung và tài nguyên dành riêng cho kiến trúc doanh nghiệp. Thật ra, kiến trúc là một chủ đề gây tranh cãi đến mức đáng ngạc nhiên.

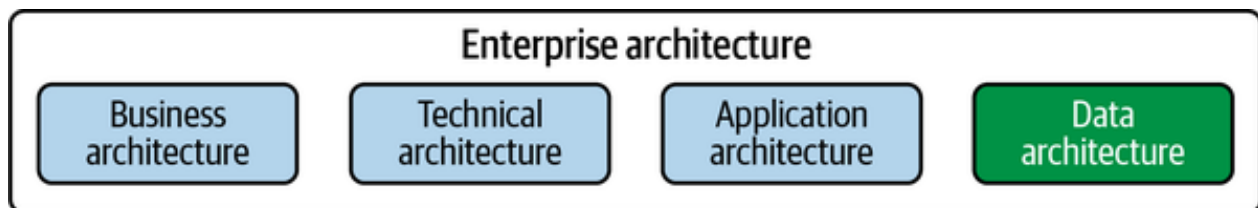


Figure 3-1. Data architecture is a subset of enterprise architecture The term enterprise gets mixed reactions. It brings to mind sterile corporate offices, command-and-control/waterfall planning, stagnant business cultures, and empty catchphrases. Even so, we can learn some things here. Before we define and describe enterprise architecture , let’s unpack this term. Let’s look at how enterprise architecture is defined by some significant thought leaders: TOGAF, Gartner, and EABOK.

Hình 3-1. Kiến trúc dữ liệu là một tập con của kiến trúc doanh nghiệp. Thuật ngữ doanh nghiệp (enterprise) gợi lên những phản ứng trái chiều. Nó khiến người ta liên tưởng đến những văn phòng công ty vô hồn, lối hoạch định kiểu chỉ huy-kiểm soát/thác nước, những văn hóa kinh doanh trì trệ và những khẩu hiệu rỗng tuếch. Dù vậy, chúng ta vẫn có thể học được vài điều ở đây. Trước khi định nghĩa và mô tả kiến trúc doanh nghiệp, hãy mổ xẻ thuật ngữ này. Hãy xem cách một số nhà tư tưởng có ảnh hưởng định nghĩa kiến trúc doanh nghiệp: TOGAF, Gartner và EABOK.

TOGAF’s definition — Định nghĩa của TOGAF

TOGAF is The Open Group Architecture Framework , a standard of The Open Group. It’s touted as the most widely used architecture framework today. Here’s the TOGAF definition :

TOGAF là The Open Group Architecture Framework, một chuẩn của The Open Group. Nó được ca ngợi là khung kiến trúc được sử dụng rộng rãi nhất hiện nay. Đây là định nghĩa của TOGAF:

The term “enterprise” in the context of “enterprise architecture” can denote an entire enterprise—encompassing all of its information and technology services, processes, and infrastructure—or a specific domain within the enterprise. In both cases, the architecture crosses multiple systems, and multiple functional groups within the enterprise.

Thuật ngữ “enterprise” trong bối cảnh “kiến trúc doanh nghiệp” có thể biểu thị toàn bộ một doanh nghiệp — bao trùm tất cả các dịch vụ thông tin và công nghệ, quy trình và hạ tầng của nó — hoặc một miền cụ thể bên trong doanh nghiệp. Trong cả hai trường hợp, kiến trúc đó trải dài qua nhiều hệ thống và nhiều nhóm chức năng bên trong doanh nghiệp.

Gartner’s definition — Định nghĩa của Gartner

Gartner is a global research and advisory company that produces research articles and reports on trends related to enterprises. Among other things, it is responsible for the (in)famous Gartner Hype Cycle. Gartner’s definition is as follows:

Gartner là một công ty nghiên cứu và tư vấn toàn cầu chuyên sản xuất các bài viết và báo cáo nghiên cứu về các xu hướng liên quan đến doanh nghiệp. Bên cạnh nhiều thứ khác, họ chịu trách nhiệm về Gartner Hype Cycle (in)famous. Định nghĩa của Gartner như sau:

Enterprise architecture (EA) is a discipline for proactively and holistically leading enterprise responses to disruptive forces by identifying and analyzing the execution of change toward desired business vision and outcomes. EA delivers value by presenting business and IT leaders with signature-ready recommendations for adjusting policies and projects to achieve targeted business outcomes that capitalize on relevant business disruptions.

Kiến trúc doanh nghiệp (EA) là một bộ môn nhằm chủ động và toàn diện dẫn dắt các phản ứng của doanh nghiệp trước những lực lượng gây xáo trộn, bằng cách nhận diện và phân tích việc thực thi sự thay đổi hướng tới tầm nhìn và kết quả kinh doanh mong muốn. EA mang lại giá trị bằng cách trình cho các lãnh đạo kinh doanh và CNTT những khuyến nghị sẵn sàng để ký duyệt nhằm điều chỉnh các chính sách và dự án để đạt được những kết quả kinh doanh mục tiêu, tận dụng được các xáo trộn kinh doanh liên quan.

EABOK’s definition — Định nghĩa của EABOK

EABOK is the Enterprise Architecture Book of Knowledge , an enterprise architecture reference produced by the MITRE Corporation. EABOK was released as an incomplete draft in 2004 and has not been updated since. Though seemingly obsolete, EABOK is still frequently referenced in descriptions of enterprise architecture; we found many of its ideas helpful while writing this book. Here’s the EABOK definition :

EABOK là Enterprise Architecture Book of Knowledge, một tài liệu tham khảo về kiến trúc doanh nghiệp do MITRE Corporation soạn thảo. EABOK được phát hành dưới dạng bản nháp chưa hoàn thiện vào năm 2004 và chưa được cập nhật kể từ đó. Dù có vẻ đã lỗi thời, EABOK vẫn thường xuyên được trích dẫn trong các mô tả về kiến trúc doanh nghiệp; chúng tôi thấy nhiều ý tưởng của nó hữu ích khi viết cuốn sách này. Đây là định nghĩa của EABOK:

Enterprise Architecture (EA) is an organizational model; an abstract representation of an Enterprise that aligns strategy, operations, and technology to create a roadmap for success.

Kiến trúc doanh nghiệp (EA) là một mô hình tổ chức; một biểu diễn trừu tượng của một Doanh nghiệp, gắn kết chiến lược, vận hành và công nghệ để tạo ra một lộ trình đi đến thành công.

Our definition — Định nghĩa của chúng tôi

We extract a few common threads in these definitions of enterprise architecture: change, alignment, organization, opportunities, problem-solving, and migration. Here is our definition of enterprise architecture, one that we feel is more relevant to today's fast-moving data landscape:

Chúng tôi rút ra một vài sợi chỉ chung trong các định nghĩa về kiến trúc doanh nghiệp này: thay đổi, gắn kết, tổ chức, cơ hội, giải quyết vấn đề và di trú. Đây là định nghĩa của chúng tôi về kiến trúc doanh nghiệp, một định nghĩa mà chúng tôi cảm thấy phù hợp hơn với bối cảnh dữ liệu chuyển động nhanh ngày nay:

Enterprise architecture is the design of systems to support change in the enterprise, achieved by flexible and reversible decisions reached through careful evaluation of trade-offs. Here, we touch on some key areas we'll return to throughout the book: flexible and reversible decisions, change management, and evaluation of trade-offs. We discuss each theme at length in this section and then make the definition more concrete in the latter part of the chapter by giving various examples of data architecture.

Kiến trúc doanh nghiệp là việc thiết kế các hệ thống nhằm hỗ trợ sự thay đổi trong doanh nghiệp, đạt được thông qua những quyết định linh hoạt và có thể đảo ngược được đưa ra nhờ đánh giá cẩn trọng các đánh đổi. Ở đây, chúng tôi đề cập đến một số lĩnh vực then chốt mà chúng ta sẽ quay lại xuyên suốt cuốn sách: những quyết định linh hoạt và có thể đảo ngược, quản lý thay đổi, và đánh giá các đánh đổi. Chúng tôi thảo luận từng chủ đề một cách kỹ lưỡng trong mục này, rồi làm cho định nghĩa cụ thể hơn ở phần sau của chương bằng cách đưa ra nhiều ví dụ về kiến trúc dữ liệu.

Flexible and reversible decisions are essential for two reasons. First, the world is constantly changing, and predicting the future is impossible. Reversible decisions allow you to adjust course as the world changes and you gather new information. Second, there is a natural tendency toward enterprise ossification as organizations grow. As organizations grow, they become increasingly risk averse and cumbersome. Adopting a culture of reversible decisions helps overcome this tendency by reducing the risk attached to a decision. Jeff Bezos is credited with the idea of one-way and two-way doors. A 1 one-way door is a decision that is almost impossible to reverse. For example, Amazon could have decided to sell AWS or shut it down. It would be nearly impossible for Amazon to rebuild a public cloud with the same market position after such an action. On the other hand, a two-way door is an easily reversible decision: you walk through and proceed if you like what you see in the room or step back through the door if you don't. Amazon might decide to require the use of DynamoDB for a new **microservices** database. If this policy doesn't work, Amazon has the option of reversing it and refactoring some services to use other databases. Since the stakes attached to each reversible decision (two-way door) are low, organizations can make more decisions, iterating, improving, and collecting data rapidly. Change management is closely related to reversible decisions and is a central theme of enterprise architecture frameworks. Even with an emphasis on reversible decisions, enterprises often need to undertake large initiatives. These are ideally broken into smaller changes, each one a reversible decision in itself. Returning to Amazon, we note a five-year gap (2007 to 2012) from the publication of a paper on the DynamoDB concept to Werner Vogels's announcement of the DynamoDB service on AWS. Behind the scenes, teams took numerous small actions to make DynamoDB a concrete reality for AWS customers. Managing such small actions is at the heart of

change management.

Những quyết định linh hoạt và có thể đảo ngược là thiết yếu vì hai lý do. Thứ nhất, thế giới thay đổi không ngừng, và dự đoán tương lai là điều bất khả thi. Những quyết định có thể đảo ngược cho phép bạn điều chỉnh hướng đi khi thế giới thay đổi và bạn thu thập thông tin mới. Thứ hai, có một xu hướng tự nhiên dẫn tới sự xơ cứng của doanh nghiệp khi các tổ chức lớn lên. Khi tổ chức phát triển, chúng ngày càng ngại rủi ro và công kênh. Việc áp dụng một văn hóa ra quyết định có thể đảo ngược giúp khắc phục xu hướng này bằng cách giảm rủi ro gắn liền với mỗi quyết định. Jeff Bezos được ghi nhận với ý tưởng về những cánh cửa một chiều và hai chiều. Một cánh cửa một chiều là một quyết định gần như không thể đảo ngược. Ví dụ, Amazon đáng lẽ đã có thể quyết định bán AWS hoặc đóng cửa nó. Sẽ gần như bất khả thi để Amazon tái dựng một đám mây công cộng với cùng vị thế thị trường sau một hành động như vậy. Ngược lại, một cánh cửa hai chiều là một quyết định dễ đảo ngược: bạn bước qua và tiếp tục nếu thích những gì thấy trong phòng, hoặc bước ngược lại qua cửa nếu không thích. Amazon có thể quyết định yêu cầu sử dụng DynamoDB cho một cơ sở dữ liệu microservices mới. Nếu chính sách này không hiệu quả, Amazon có lựa chọn đảo ngược nó và tái cấu trúc một số dịch vụ để dùng cơ sở dữ liệu khác. Vì mức rủi ro gắn với mỗi quyết định có thể đảo ngược (cửa hai chiều) là thấp, các tổ chức có thể ra nhiều quyết định hơn, lặp đi lặp lại, cải tiến và thu thập dữ liệu một cách nhanh chóng. Quản lý thay đổi liên hệ mật thiết với các quyết định có thể đảo ngược và là một chủ đề trung tâm của các khung kiến trúc doanh nghiệp. Ngay cả khi nhấn mạnh vào các quyết định có thể đảo ngược, các doanh nghiệp vẫn thường cần thực hiện những sáng kiến lớn. Những sáng kiến này lý tưởng nhất là được chia nhỏ thành những thay đổi nhỏ hơn, mỗi thay đổi tự thân là một quyết định có thể đảo ngược. Quay lại với Amazon, ta thấy có một khoảng cách năm năm (2007 đến 2012) từ lúc công bố bài báo về ý tưởng DynamoDB đến khi Werner Vogels thông báo về dịch vụ DynamoDB trên AWS. Hậu trường, các nhóm đã thực hiện vô số hành động nhỏ để biến DynamoDB thành hiện thực cụ thể cho khách hàng AWS. Quản lý những hành động nhỏ như vậy chính là cốt lõi của quản lý thay đổi.

Architects are not simply mapping out IT processes and vaguely looking toward a distant, utopian future; they actively solve business problems and create new opportunities. Technical solutions exist not for their own sake but in support of business goals. Architects identify problems in the current state (poor **data quality**, scalability limits, money-losing lines of business), define desired future states (agile data-quality improvement, scalable cloud data solutions, improved business processes), and realize initiatives through execution of small, concrete steps.

Các kiến trúc sư không chỉ đơn thuần phác họa các quy trình CNTT và mơ hồ hướng tới một tương lai không tưởng xa xôi; họ chủ động giải quyết các vấn đề kinh doanh và tạo ra những cơ hội mới. Các giải pháp kỹ thuật tồn tại không phải vì bản thân chúng mà để hỗ trợ các mục tiêu kinh doanh. Các kiến trúc sư nhận diện những vấn đề ở trạng thái hiện tại (chất lượng dữ liệu kém, giới hạn khả năng mở rộng, các mảng kinh doanh thua lỗ), xác định những trạng thái tương lai mong muốn (cải thiện chất lượng dữ liệu một cách linh hoạt, các giải pháp dữ liệu đám mây có khả năng mở rộng, cải tiến các quy trình kinh doanh), và hiện thực hóa các sáng kiến thông qua việc thực thi những bước nhỏ, cụ thể.

Technical solutions exist not for their own sake but in support of business goals. We found significant inspiration in *Fundamentals of Software Architecture* by Mark Richards and Neal Ford (O'Reilly). They emphasize that trade-offs are inevitable and ubiquitous in the engineering space. Sometimes the relatively fluid nature of software and data leads us to believe that we are freed from the constraints that engineers face in the hard, cold physical world. Indeed, this is partially true; patching a software bug is much easier than redesigning and replacing an airplane wing. However,

digital systems are ultimately constrained by physical limits such as **latency**, reliability, density, and energy consumption. Engineers also confront various nonphysical limits, such as characteristics of programming languages and frameworks, and practical constraints in managing complexity, budgets, etc. Magical thinking culminates in poor engineering. Data engineers must account for trade-offs at every step to design an optimal system while minimizing high-interest technical debt. Let's reiterate one central point in our enterprise architecture definition: enterprise architecture balances flexibility and trade-offs. This isn't always an easy balance, and architects must constantly assess and reevaluate with the recognition that the world is dynamic. Given the pace of change that enterprises are faced with, organizations—and their architecture—cannot afford to stand still.

Các giải pháp kỹ thuật tồn tại không phải vì bản thân chúng mà để hỗ trợ các mục tiêu kinh doanh. Chúng tôi tìm thấy nguồn cảm hứng đáng kể trong cuốn *Fundamentals of Software Architecture* của Mark Richards và Neal Ford (O'Reilly). Họ nhấn mạnh rằng các đánh đổi là không thể tránh khỏi và hiện diện ở khắp nơi trong không gian kỹ thuật. Đôi khi bản chất tương đối linh động của phần mềm và dữ liệu khiến chúng ta tin rằng mình được giải phóng khỏi những ràng buộc mà các kỹ sư phải đối mặt trong thế giới vật lý cứng nhắc, lạnh lùng. Quả thực, điều này đúng một phần; và một lỗi phần mềm dễ hơn nhiều so với thiết kế lại và thay thế một cánh máy bay. Tuy nhiên, các hệ thống số rất cuộc vẫn bị ràng buộc bởi các giới hạn vật lý như độ trễ (latency), độ tin cậy, mật độ và mức tiêu thụ năng lượng. Các kỹ sư cũng phải đối mặt với nhiều giới hạn phi vật lý khác, chẳng hạn các đặc tính của ngôn ngữ lập trình và framework, cùng những ràng buộc thực tế trong việc quản lý độ phức tạp, ngân sách, v.v. Lối tư duy thần kỳ kết cục là kỹ thuật tồi. Kỹ sư dữ liệu phải tính đến các đánh đổi ở mọi bước để thiết kế một hệ thống tối ưu trong khi giảm thiểu nợ kỹ thuật lẫn suất cao. Hãy nhắc lại một điểm trung tâm trong định nghĩa kiến trúc doanh nghiệp của chúng tôi: kiến trúc doanh nghiệp cân bằng giữa tính linh hoạt và các đánh đổi. Đây không phải lúc nào cũng là một sự cân bằng dễ dàng, và các kiến trúc sư phải liên tục đánh giá và đánh giá lại với nhận thức rằng thế giới luôn động. Trước tốc độ thay đổi mà các doanh nghiệp phải đối mặt, các tổ chức — và kiến trúc của họ — không thể đứng yên.

Data Architecture Defined — Định nghĩa kiến trúc dữ liệu

Now that you understand enterprise architecture, let's dive into data architecture by establishing a working definition that will set the stage for the rest of the book. Data architecture is a subset of enterprise architecture, inheriting its properties: processes, strategy, change management, and technology. Here are a couple of definitions of data architecture that influence our definition.

Giờ khi bạn đã hiểu kiến trúc doanh nghiệp, hãy đi sâu vào kiến trúc dữ liệu bằng cách thiết lập một định nghĩa có tính ứng dụng làm nền cho phần còn lại của cuốn sách. Kiến trúc dữ liệu là một tập con của kiến trúc doanh nghiệp, kế thừa các thuộc tính của nó: quy trình, chiến lược, quản lý thay đổi và công nghệ. Sau đây là một vài định nghĩa về kiến trúc dữ liệu có ảnh hưởng tới định nghĩa của chúng tôi.

TOGAF's definition — Định nghĩa của TOGAF

TOGAF defines data architecture as follows:

TOGAF định nghĩa kiến trúc dữ liệu như sau:

A description of the structure and interaction of the enterprise's major types and sources of data, logical data assets, physical data assets, and data management resources.

Một mô tả về cấu trúc và sự tương tác giữa các loại và nguồn dữ liệu chính của doanh nghiệp, các tài sản dữ liệu logic, các tài sản dữ liệu vật lý, và các tài nguyên quản lý dữ liệu.

DAMA's definition — Định nghĩa của DAMA

The DAMA DMBOK defines data architecture as follows:

DAMA DMBOK định nghĩa kiến trúc dữ liệu như sau:

Identifying the data needs of the enterprise (regardless of structure) and designing and maintaining the master blueprints to meet those needs. Using master blueprints to guide data integration, control data assets, and align data investments with business strategy.

Nhận diện các nhu cầu dữ liệu của doanh nghiệp (bất kể cấu trúc) và thiết kế, duy trì các bản thiết kế tổng thể để đáp ứng những nhu cầu đó. Sử dụng các bản thiết kế tổng thể để dẫn dắt việc tích hợp dữ liệu, kiểm soát các tài sản dữ liệu, và gắn kết các khoản đầu tư dữ liệu với chiến lược kinh doanh.

Our definition — Định nghĩa của chúng tôi

Considering the preceding two definitions and our experience, we have crafted our definition of data architecture :

Xét đến hai định nghĩa trên cùng kinh nghiệm của mình, chúng tôi đã xây dựng định nghĩa của chúng tôi về kiến trúc dữ liệu:

Data architecture is the design of systems to support the evolving data needs of an enterprise, achieved by flexible and reversible decisions reached through a careful evaluation of trade-offs. How does data architecture fit into data engineering? Just as the **data engineering lifecycle** is a subset of the data lifecycle, data engineering architecture is a subset of general data architecture. Data engineering architecture is the systems and frameworks that make up the key sections of the data engineering lifecycle. We'll use data architecture interchangeably with data engineering architecture throughout this book.

Kiến trúc dữ liệu là việc thiết kế các hệ thống nhằm hỗ trợ những nhu cầu dữ liệu đang tiến hóa của một doanh nghiệp, đạt được thông qua những quyết định linh hoạt và có thể đảo ngược được đưa ra nhờ đánh giá cẩn trọng các đánh đổi. Kiến trúc dữ liệu khớp vào data engineering như thế nào? Cũng như vòng đời kỹ thuật dữ liệu là một tập con của vòng đời dữ liệu, kiến trúc kỹ thuật dữ liệu là một tập con của kiến trúc dữ liệu nói chung. Kiến trúc kỹ thuật dữ liệu là các hệ thống và framework tạo nên những phần then chốt của vòng đời kỹ thuật dữ liệu. Xuyên suốt cuốn sách này, chúng tôi sẽ dùng kiến trúc dữ liệu thay thế cho kiến trúc kỹ thuật dữ liệu.

Other aspects of data architecture that you should be aware of are operational and technical (Figure 3-2). Operational architecture encompasses the functional requirements of what needs to happen related to people, processes, and technology. For example, what business processes does the data serve? How does the organization manage data quality? What is the latency requirement from when the data is produced to when it becomes available to query? Technical architecture outlines how data is ingested, stored, transformed, and served along the data engineering lifecycle. For instance, how will you move 10 TB of data every hour from a source database to your **data lake**? In short,

operational architecture describes what needs to be done, and technical architecture details how it will happen.

Những khía cạnh khác của kiến trúc dữ liệu mà bạn nên biết là kiến trúc vận hành và kiến trúc kỹ thuật (Hình 3-2). Kiến trúc vận hành bao trùm các yêu cầu chức năng về những gì cần xảy ra liên quan đến con người, quy trình và công nghệ. Ví dụ, dữ liệu phục vụ những quy trình kinh doanh nào? Tổ chức quản lý chất lượng dữ liệu ra sao? Yêu cầu độ trễ từ lúc dữ liệu được tạo ra đến khi nó sẵn sàng để truy vấn là bao nhiêu? Kiến trúc kỹ thuật phác họa cách dữ liệu được thu nạp (ingestion), lưu trữ, biến đổi (transformation) và phục vụ dọc theo vòng đời kỹ thuật dữ liệu. Chẳng hạn, bạn sẽ chuyển 10 TB dữ liệu mỗi giờ từ một cơ sở dữ liệu nguồn sang hồ dữ liệu (data lake) của mình như thế nào? Tóm lại, kiến trúc vận hành mô tả cái gì cần được làm, còn kiến trúc kỹ thuật trình bày chi tiết nó sẽ xảy ra như thế nào.



Figure 3-2. Operational and technical data architecture Now that we have a working definition of data architecture, let's cover the elements of “good” data architecture.

Hình 3-2. Kiến trúc dữ liệu vận hành và kỹ thuật. Giờ khi chúng ta đã có một định nghĩa có tính ứng dụng về kiến trúc dữ liệu, hãy điểm qua các yếu tố của kiến trúc dữ liệu “tốt”.

“Good” Data Architecture — Kiến trúc dữ liệu “tốt”

Never shoot for the best architecture, but rather the least worst architecture. —Neal Ford, Mark Richards According to Grady Booch , “Architecture represents the significant design decisions that shape a system, where significant is measured by cost of change.” Data architects aim to make significant decisions that will lead to good architecture at a basic level.

Đừng bao giờ nhắm tới kiến trúc tốt nhất, mà hãy nhắm tới kiến trúc ít tệ nhất. —Neal Ford, Mark Richards. Theo Grady Booch, “Kiến trúc đại diện cho những quyết định thiết kế quan trọng định hình một hệ thống, trong đó độ quan trọng được đo bằng chi phí thay đổi.” Các kiến trúc sư dữ liệu hướng tới việc đưa ra những quyết định quan trọng dẫn đến kiến trúc tốt ở mức cơ bản.

What do we mean by “good” data architecture? To paraphrase an old cliché, you know good when you see it. Good data architecture serves business requirements with a common, widely reusable set of building blocks while maintaining flexibility and making appropriate trade-offs. Bad architecture is authoritarian and tries to cram a bunch of one-size-fits-all decisions into a big ball of mud . Agility is the foundation for good data architecture; it acknowledges that the world is fluid. Good data architecture is flexible and easily maintainable . It evolves in response to changes within the business and new technologies and practices that may unlock even more value in the future. Businesses and their use cases for data are always evolving. The world is dynamic, and the pace of change in the data space is accelerating. Last year’s data architecture that served you well might not be sufficient for today, let alone next year. Bad data architecture is **tightly coupled**, rigid, overly centralized,

or uses the wrong tools for the job, hampering development and change management. Ideally, by designing architecture with reversibility in mind, changes will be less costly. The **undercurrents** of the data engineering lifecycle form the foundation of good data architecture for companies at any stage of data maturity. Again, these undercurrents are security, data management, **DataOps**, data architecture, **orchestration**, and software engineering. Good data architecture is a living, breathing thing. It's never finished. In fact, per our definition, change and evolution are central to the meaning and purpose of data architecture. Let's now look at the principles of good data architecture.

Chúng tôi muốn nói gì khi nói kiến trúc dữ liệu “tốt”? Diễn giải lại một câu sáo cũ, bạn nhận ra cái tốt khi bạn thấy nó. Kiến trúc dữ liệu tốt phục vụ các yêu cầu kinh doanh bằng một tập các khối xây dựng chung, có thể tái sử dụng rộng rãi, đồng thời duy trì tính linh hoạt và thực hiện những đánh đổi phù hợp. Kiến trúc tồi mang tính áp đặt và cố nhồi nhét một mớ quyết định kiểu một-cỡ-vừa-cho-tất-cả vào một cục bùn lớn (big ball of mud). Tính nhanh nhạy (agility) là nền tảng cho kiến trúc dữ liệu tốt; nó thừa nhận rằng thế giới luôn linh động. Kiến trúc dữ liệu tốt thì linh hoạt và dễ bảo trì. Nó tiến hóa để đáp ứng những thay đổi bên trong doanh nghiệp cùng các công nghệ và thực hành mới có thể mở khóa thêm nhiều giá trị trong tương lai. Các doanh nghiệp và những ca sử dụng dữ liệu của họ luôn tiến hóa. Thế giới luôn động, và tốc độ thay đổi trong không gian dữ liệu đang tăng tốc. Kiến trúc dữ liệu của năm ngoái từng phục vụ bạn tốt có thể không đủ cho hôm nay, chứ chưa nói đến năm sau. Kiến trúc dữ liệu tồi thì ghép chặt, cứng nhắc, tập trung hóa quá mức, hoặc dùng sai công cụ cho công việc, cản trở phát triển và quản lý thay đổi. Lý tưởng nhất, bằng cách thiết kế kiến trúc với tính đảo ngược trong tâm trí, các thay đổi sẽ ít tốn kém hơn. Các dòng chảy ngầm của vòng đời kỹ thuật dữ liệu tạo nên nền tảng cho kiến trúc dữ liệu tốt ở các công ty tại bất kỳ giai đoạn trưởng thành dữ liệu nào. Một lần nữa, các dòng chảy ngầm này là an ninh, quản lý dữ liệu, DataOps, kiến trúc dữ liệu, điều phối (orchestration) và kỹ thuật phần mềm. Kiến trúc dữ liệu tốt là một thực thể sống động, hít thở. Nó không bao giờ hoàn thiện. Thực tế, theo định nghĩa của chúng tôi, sự thay đổi và tiến hóa là trung tâm của ý nghĩa và mục đích của kiến trúc dữ liệu. Giờ hãy xem xét các nguyên tắc của kiến trúc dữ liệu tốt.

Principles of Good Data Architecture — Các nguyên tắc của kiến trúc dữ liệu tốt

This section takes a 10,000-foot view of good architecture by focusing on principles—key ideas useful in evaluating major architectural decisions and practices. We borrow inspiration for our architecture principles from several sources, especially the AWS Well-Architected Framework and Google Cloud's Five Principles for Cloud-Native Architecture. The AWS Well-Architected Framework consists of six pillars:

Mục này nhìn kiến trúc tốt từ góc độ tổng quát bằng cách tập trung vào các nguyên tắc — những ý tưởng then chốt hữu ích trong việc đánh giá các quyết định và thực hành kiến trúc quan trọng. Chúng tôi mượn cảm hứng cho các nguyên tắc kiến trúc của mình từ nhiều nguồn, đặc biệt là AWS Well-Architected Framework và Five Principles for Cloud-Native Architecture của Google Cloud. AWS Well-Architected Framework gồm sáu trụ cột:

Operational excellence

Sự xuất sắc trong vận hành (Operational excellence)

Security

An ninh (Security)

Reliability

Độ tin cậy (Reliability)

Performance efficiency

Hiệu suất hiệu quả (Performance efficiency)

Cost optimization

Tối ưu chi phí (Cost optimization)

Sustainability

Tính bền vững (Sustainability)

Google Cloud's Five Principles for Cloud-Native Architecture are as follows:

Five Principles for Cloud-Native Architecture của Google Cloud như sau:

Design for automation.

Thiết kế hướng tự động hóa.

Be smart with state.

Thông minh trong xử lý trạng thái.

Favor managed services.

Ưu tiên các dịch vụ được quản lý (managed service).

Practice defense in depth.

Thực hành phòng thủ theo chiều sâu.

Always be architecting.

Luôn luôn kiến trúc (Always be architecting).

We advise you to carefully study both frameworks, identify valuable ideas, and determine points of disagreement. We'd like to expand or elaborate on these pillars with these principles of data engineering architecture:

Chúng tôi khuyên bạn nghiên cứu kỹ cả hai khung, nhận diện những ý tưởng giá trị, và xác định những điểm bất đồng. Chúng tôi muốn mở rộng hoặc bổ sung cho các trụ cột này bằng những nguyên tắc kiến trúc kỹ thuật dữ liệu sau:

1. Choose common components wisely.
2. Plan for failure.
3. Architect for scalability.
4. Architecture is leadership.
5. Always be architecting.
 1. Lựa chọn các thành phần chung một cách khôn ngoan.
 2. Dự liệu cho thất bại.
 3. Kiến trúc cho khả năng mở rộng quy mô.
 4. Kiến trúc là sự lãnh đạo.
 5. Luôn luôn kiến trúc.
6. Build **loosely** coupled systems.
7. Make reversible decisions.
8. Prioritize security.

9. Embrace **FinOps**.

6. Xây dựng các hệ thống ghép lỏng.
7. Đưa ra những quyết định có thể đảo ngược.
8. Ưu tiên an ninh.
9. Đón nhận FinOps.

Principle 1: Choose Common Components Wisely — Nguyên tắc 1: Lựa chọn các thành phần chung một cách khôn ngoan

One of the primary jobs of a **data engineer** is to choose common components and practices that can be used widely across an organization. When architects choose well and lead effectively, common components become a fabric facilitating team collaboration and breaking down silos. Common components enable agility within and across teams in conjunction with shared knowledge and skills. Common components can be anything that has broad applicability within an organization. Common components include **object storage**, version-control systems, observability, monitoring and orchestration systems, and processing engines. Common components should be accessible to everyone with an appropriate use case, and teams are encouraged to rely on common components already in use rather than reinventing the wheel. Common components must support robust permissions and security to enable sharing of assets among teams while preventing unauthorized access. Cloud platforms are an ideal place to adopt common components. For example, compute and **storage** separation in cloud data systems allows users to access a shared storage layer (most commonly object storage) using specialized tools to access and query the data needed for specific use cases. Choosing common components is a balancing act. On the one hand, you need to focus on needs across the data engineering lifecycle and teams, utilize common components that will be useful for individual projects, and simultaneously facilitate interoperation and collaboration. On the other hand, architects should avoid decisions that will hamper the productivity of engineers working on domain-specific problems by forcing them into one-size-fits-all technology solutions. Chapter 4 provides additional details.

Một trong những công việc chính của kỹ sư dữ liệu là lựa chọn các thành phần và thực hành chung có thể được dùng rộng rãi khắp một tổ chức. Khi các kiến trúc sư chọn lựa tốt và dẫn dắt hiệu quả, các thành phần chung trở thành một tấm vải kết nối, tạo điều kiện cho sự cộng tác giữa các nhóm và phá bỏ các silo. Các thành phần chung tạo ra tính nhanh nhạy trong nội bộ và giữa các nhóm, kết hợp với kiến thức và kỹ năng được chia sẻ. Thành phần chung có thể là bất cứ thứ gì có khả năng áp dụng rộng rãi trong một tổ chức. Các thành phần chung bao gồm lưu trữ đối tượng (object storage), hệ thống quản lý phiên bản, hệ thống khả năng quan sát (observability), giám sát và điều phối, cùng các động cơ xử lý. Các thành phần chung nên có thể truy cập được với mọi người có ca sử dụng phù hợp, và các nhóm được khuyến khích dựa vào những thành phần chung đang được dùng thay vì phát minh lại bánh xe. Các thành phần chung phải hỗ trợ quyền hạn và an ninh chắc chắn để cho phép chia sẻ tài sản giữa các nhóm trong khi ngăn chặn truy cập trái phép. Các nền tảng đám mây là nơi lý tưởng để áp dụng các thành phần chung. Ví dụ, việc tách rời tính toán và lưu trữ trong các hệ thống dữ liệu đám mây cho phép người dùng truy cập một tầng lưu trữ dùng chung (thường nhất là lưu trữ đối tượng) bằng các công cụ chuyên biệt để truy cập và truy vấn dữ liệu cần thiết cho những ca sử dụng cụ thể. Lựa chọn các thành phần chung là một việc đòi hỏi cân bằng. Một mặt, bạn cần tập trung vào các nhu cầu xuyên suốt vòng đời kỹ thuật dữ liệu và các nhóm, tận dụng những thành phần chung sẽ hữu ích cho từng dự án riêng lẻ, đồng thời tạo điều kiện cho khả năng liên thông và cộng tác. Mặt khác, các kiến trúc sư nên tránh những quyết định cản trở năng suất của

các kỹ sư đang giải quyết những vấn đề đặc thù theo miền bằng cách ép họ vào các giải pháp công nghệ một-cỡ-vừa-cho-tất-cả. Chương 4 cung cấp thêm chi tiết.

Principle 2: Plan for Failure — Nguyên tắc 2: Dự liệu cho thất bại

Everything fails, all the time. —Werner Vogels Modern hardware is highly robust and durable. Even so, any hardware component will fail, given enough time. To build highly robust data systems, you must consider failures in your designs. Here are a few key terms for evaluating failure scenarios; we describe these in greater detail in this chapter and throughout the book: Availability

Mọi thứ đều hỏng, mọi lúc. —Werner Vogels. Phần cứng hiện đại có độ bền và độ tin cậy rất cao. Dù vậy, bất kỳ linh kiện phần cứng nào cũng sẽ hỏng nếu có đủ thời gian. Để xây dựng các hệ thống dữ liệu cực kỳ bền vững, bạn phải cân nhắc đến các sự cố trong thiết kế của mình. Sau đây là một vài thuật ngữ then chốt để đánh giá các kịch bản thất bại; chúng tôi mô tả chúng chi tiết hơn trong chương này và xuyên suốt cuốn sách: Tính sẵn sàng (Availability)

The percentage of time an IT service or component is in an operable state.

Tỷ lệ phần trăm thời gian một dịch vụ hoặc thành phần CNTT ở trạng thái có thể vận hành.

Reliability

Độ tin cậy (Reliability)

The system's probability of meeting defined standards in performing its intended function during a specified interval.

Xác suất hệ thống đáp ứng các tiêu chuẩn đã định trong việc thực hiện chức năng dự kiến của nó trong một khoảng thời gian xác định.

Recovery time objective

Mục tiêu thời gian phục hồi (Recovery time objective)

The maximum acceptable time for a service or system outage. The recovery time objective (RTO) is generally set by determining the business impact of an outage. An RTO of one day might be fine for an internal reporting system. A website outage of just five minutes could have a significant adverse business impact on an online retailer.

Khoảng thời gian gián đoạn dịch vụ hoặc hệ thống tối đa chấp nhận được. Mục tiêu thời gian phục hồi (RTO) thường được thiết lập bằng cách xác định tác động kinh doanh của một sự cố gián đoạn. Một RTO là một ngày có thể chấp nhận được với một hệ thống báo cáo nội bộ. Việc một website ngừng hoạt động chỉ năm phút có thể gây ra tác động kinh doanh bất lợi đáng kể với một nhà bán lẻ trực tuyến.

Recovery point objective

Mục tiêu điểm phục hồi (Recovery point objective)

The acceptable state after recovery. In data systems, data is often lost during an outage. In this setting, the recovery point objective (RPO) refers to the maximum acceptable data loss.

Trạng thái chấp nhận được sau khi phục hồi. Trong các hệ thống dữ liệu, dữ liệu thường bị mất trong một sự cố gián đoạn. Trong bối cảnh này, mục tiêu điểm phục hồi (RPO) ám chỉ mức mất mát dữ liệu tối đa chấp nhận được.

Engineers need to consider acceptable reliability, availability, RTO, and RPO in designing for failure. These will guide their architecture decisions as they assess possible failure scenarios.

Các kỹ sư cần cân nhắc mức độ tin cậy, tính sẵn sàng, RTO và RPO chấp nhận được khi thiết kế để ứng phó với thất bại. Những điều này sẽ dẫn dắt các quyết định kiến trúc của họ khi họ đánh giá các kịch bản thất bại khả dĩ.

Principle 3: Architect for Scalability — Nguyên tắc 3: Kiến trúc cho khả năng mở rộng quy mô

Scalability in data systems encompasses two main capabilities. First, scalable systems can scale up to handle significant quantities of data. We might need to spin up a large cluster to train a model on a petabyte of customer data or scale out a streaming **ingestion** system to handle a transient load spike. Our ability to scale up allows us to handle extreme loads temporarily. Second, scalable systems can scale down. Once the load spike ebbs, we should automatically remove capacity to cut costs. (This is related to principle 9.) An elastic system can scale dynamically in response to load, ideally in an automated fashion. Some scalable systems can also scale to zero: they shut down completely when not in use. Once the large model-training job completes, we can delete the cluster. Many **serverless** systems (e.g., serverless functions and serverless online analytical processing, or **OLAP**, databases) can automatically scale to zero. Note that deploying inappropriate **scaling** strategies can result in overcomplicated systems and high costs. A straightforward relational database with one failover node may be appropriate for an application instead of a complex cluster arrangement. Measure your current load, approximate load spikes, and estimate load over the next several years to determine if your database architecture is appropriate. If your startup grows much faster than anticipated, this growth should also lead to more available resources to rearchitect for scalability.

Khả năng mở rộng quy mô (scalability) trong các hệ thống dữ liệu bao gồm hai năng lực chính. Thứ nhất, các hệ thống có khả năng mở rộng có thể mở rộng lên (scale up) để xử lý những lượng dữ liệu khổng lồ. Chúng ta có thể cần khởi tạo một cụm lớn để huấn luyện mô hình trên một petabyte dữ liệu khách hàng, hoặc mở rộng ra (scale out) một hệ thống thu nạp theo luồng để xử lý một đợt tải đột biến nhất thời. Khả năng mở rộng lên cho phép chúng ta xử lý các tải cực lớn một cách tạm thời. Thứ hai, các hệ thống có khả năng mở rộng có thể thu hẹp xuống (scale down). Khi đợt tải đột biến lắng xuống, chúng ta nên tự động loại bỏ năng lực để cắt giảm chi phí. (Điều này liên quan đến nguyên tắc 9.) Một hệ thống đàn hồi (elastic) có thể mở rộng quy mô một cách động để đáp ứng tải, lý tưởng là theo cách tự động. Một số hệ thống có khả năng mở rộng cũng có thể mở rộng về không (scale to zero): chúng tắt hoàn toàn khi không được sử dụng. Khi công việc huấn luyện mô hình lớn hoàn tất, chúng ta có thể xóa cụm đó đi. Nhiều hệ thống serverless (ví dụ các hàm serverless và các cơ sở dữ liệu xử lý phân tích trực tuyến serverless, hay OLAP) có thể tự động mở rộng về không. Lưu ý rằng triển khai các chiến lược mở rộng không phù hợp có thể dẫn đến các hệ thống phức tạp quá mức và chi phí cao. Một cơ sở dữ liệu quan hệ đơn giản với một nút dự phòng có thể phù hợp cho một ứng dụng thay vì một sắp xếp cụm phức tạp. Hãy đo lường tải hiện tại của bạn, ước lượng các đợt tải đột biến, và ước tính tải trong vài năm tới để xác định xem kiến trúc cơ sở dữ liệu của bạn có phù hợp không. Nếu startup của bạn tăng trưởng nhanh hơn nhiều so với dự đoán, sự tăng trưởng này cũng sẽ dẫn đến nhiều tài nguyên sẵn có hơn để tái kiến trúc nhằm mở rộng quy mô.

Principle 4: Architecture Is Leadership — Nguyên tắc 4: Kiến trúc là sự lãnh đạo

Data architects are responsible for technology decisions and architecture descriptions and disseminating these choices through effective leadership and training. Data architects should be highly technically competent but delegate most individual contributor work to others. Strong leadership skills combined with high technical competence are rare and extremely valuable. The best data architects take this duality seriously. Note that leadership does not imply a command-and-control approach to technology. It was not uncommon in the past for architects to choose one proprietary database technology and force every team to house their data there. We oppose this approach because it can significantly hinder current data projects. Cloud environments allow architects to balance common component choices with flexibility that enables innovation within projects. Returning to the notion of technical leadership, Martin Fowler describes a specific archetype of an ideal software architect, well embodied in his colleague Dave Rice:

Các kiến trúc sư dữ liệu chịu trách nhiệm về các quyết định công nghệ và các mô tả kiến trúc, và truyền bá những lựa chọn này thông qua sự lãnh đạo và đào tạo hiệu quả. Kiến trúc sư dữ liệu nên có năng lực kỹ thuật rất cao nhưng ủy thác phần lớn công việc của một cá nhân đóng góp cho người khác. Kỹ năng lãnh đạo mạnh kết hợp với năng lực kỹ thuật cao là hiếm và cực kỳ giá trị. Những kiến trúc sư dữ liệu giỏi nhất xem trọng tính song hành này. Lưu ý rằng lãnh đạo không hàm ý một cách tiếp cận chỉ huy-kiểm soát đối với công nghệ. Trong quá khứ, không hiếm việc kiến trúc sư chọn một công nghệ cơ sở dữ liệu độc quyền và ép mọi nhóm phải lưu dữ liệu của họ ở đó. Chúng tôi phản đối cách tiếp cận này vì nó có thể cản trở đáng kể các dự án dữ liệu hiện tại. Các môi trường đám mây cho phép kiến trúc sư cân bằng giữa các lựa chọn thành phần chung với sự linh hoạt giúp đổi mới trong từng dự án. Quay lại với khái niệm lãnh đạo kỹ thuật, Martin Fowler mô tả một nguyên mẫu cụ thể của một kiến trúc sư phần mềm lý tưởng, được hiện thân rõ nét ở đồng nghiệp của ông, Dave Rice:

In many ways, the most important activity of Architectus Oryzus is to mentor the development team, to raise their level so they can take on more complex issues. Improving the development team's ability gives an architect much greater leverage than being the sole decision-maker and thus running the risk of being an architectural bottleneck. 2

Theo nhiều cách, hoạt động quan trọng nhất của Architectus Oryzus là cố vấn cho nhóm phát triển, nâng tầm họ để họ có thể đảm nhận những vấn đề phức tạp hơn. Cải thiện năng lực của nhóm phát triển mang lại cho một kiến trúc sư đòn bẩy lớn hơn nhiều so với việc là người ra quyết định duy nhất, và do đó chấp nhận rủi ro trở thành một nút thắt cổ chai kiến trúc.

An ideal data architect manifests similar characteristics. They possess the technical skills of a data engineer but no longer practice data engineering day to day; they mentor current data engineers, make careful technology choices in consultation with their organization, and disseminate expertise through training and leadership. They train engineers in best practices and bring the company's engineering resources together to pursue common goals in both technology and business. As a data engineer, you should practice architecture leadership and seek mentorship from architects. Eventually, you may well occupy the architect role yourself.

Một kiến trúc sư dữ liệu lý tưởng thể hiện những đặc điểm tương tự. Họ sở hữu các kỹ năng kỹ thuật của một kỹ sư dữ liệu nhưng không còn thực hành data engineering hằng ngày; họ cố vấn cho các kỹ sư dữ liệu hiện tại, đưa ra những lựa chọn công nghệ cẩn trọng trong sự tham vấn với tổ chức của mình, và truyền bá chuyên môn thông qua đào tạo và

lãnh đạo. Họ đào tạo các kỹ sư về thực hành tốt nhất và quy tụ các nguồn lực kỹ thuật của công ty để theo đuổi những mục tiêu chung cả về công nghệ lẫn kinh doanh. Là một kỹ sư dữ liệu, bạn nên thực hành lãnh đạo kiến trúc và tìm kiếm sự cố vấn từ các kiến trúc sư. Cuối cùng, rất có thể chính bạn sẽ đảm nhận vai trò kiến trúc sư.

Principle 5: Always Be Architecting — Nguyên tắc 5: Luôn luôn kiến trúc

We borrow this principle directly from Google Cloud’s Five Principles for Cloud-Native Architecture. Data architects don’t serve in their role simply to maintain the existing state; instead, they constantly design new and exciting things in response to changes in business and technology. Per the EABOK , an architect’s job is to develop deep knowledge of the baseline architecture (current state), develop a target architecture , and map out a sequencing plan to determine priorities and the order of architecture changes. We add that modern architecture should not be command-and-control or waterfall but collaborative and agile. The data architect maintains a target architecture and sequencing plans that change over time. The target architecture becomes a moving target, adjusted in response to business and technology changes internally and worldwide. The sequencing plan determines immediate priorities for delivery.

Chúng tôi mượn nguyên tắc này trực tiếp từ Five Principles for Cloud-Native Architecture của Google Cloud. Các kiến trúc sư dữ liệu không đảm nhận vai trò của họ chỉ để duy trì trạng thái hiện hữu; thay vào đó, họ liên tục thiết kế những điều mới mẻ và thú vị để đáp ứng những thay đổi trong kinh doanh và công nghệ. Theo EABOK, công việc của một kiến trúc sư là phát triển hiểu biết sâu sắc về kiến trúc nền (trạng thái hiện tại), phát triển một kiến trúc đích, và vạch ra một kế hoạch trình tự để xác định các ưu tiên cùng thứ tự của những thay đổi kiến trúc. Chúng tôi bổ sung rằng kiến trúc hiện đại không nên theo kiểu chỉ huy-kiểm soát hay thác nước mà nên mang tính cộng tác và nhanh nhạy. Kiến trúc sư dữ liệu duy trì một kiến trúc đích và các kế hoạch trình tự thay đổi theo thời gian. Kiến trúc đích trở thành một mục tiêu di động, được điều chỉnh để đáp ứng những thay đổi về kinh doanh và công nghệ ở cả nội bộ lẫn trên toàn cầu. Kế hoạch trình tự xác định các ưu tiên tức thời cho việc bàn giao.

Principle 6: Build Loosely Coupled Systems — Nguyên tắc 6: Xây dựng các hệ thống ghép lỏng

When the architecture of the system is designed to enable teams to test, deploy, and change systems without dependencies on other teams, teams require little communication to get work done. In other words, both the architecture and the teams are loosely coupled. —Google DevOps tech Architecture Guide In 2002, Bezos wrote an email to Amazon employees that became known as the Bezos API Mandate: 3

Khi kiến trúc của hệ thống được thiết kế để cho phép các nhóm kiểm thử, triển khai và thay đổi hệ thống mà không phụ thuộc vào các nhóm khác, các nhóm cần rất ít giao tiếp để hoàn thành công việc. Nói cách khác, cả kiến trúc lẫn các nhóm đều ghép lỏng. —Google DevOps tech Architecture Guide. Năm 2002, Bezos viết một email cho nhân viên Amazon, sau này được biết đến với tên Bezos API Mandate:

1. All teams will henceforth expose their data and functionality through service interfaces.
2. Teams must communicate with each other through these interfaces.

3. There will be no other form of interprocess communication allowed: no direct linking, no direct reads of another team's data store, no shared-memory model, no back-doors whatsoever. The only communication allowed is via service interface calls over the network.
4. It doesn't matter what technology they use. HTTP, Corba, Pubsub, custom protocols—doesn't matter.
 1. Từ nay trở đi, tất cả các nhóm sẽ phơi bày dữ liệu và chức năng của họ thông qua các giao diện dịch vụ.
 2. Các nhóm phải giao tiếp với nhau thông qua những giao diện này.
 3. Sẽ không có hình thức giao tiếp liên tiến trình nào khác được phép: không liên kết trực tiếp, không đọc trực tiếp kho dữ liệu của nhóm khác, không mô hình bộ nhớ chia sẻ, không có cửa hậu nào dưới bất kỳ hình thức nào. Hình thức giao tiếp duy nhất được phép là thông qua các lời gọi giao diện dịch vụ qua mạng.
 4. Họ dùng công nghệ gì không quan trọng. HTTP, Corba, Pubsub, các giao thức tùy chỉnh — đều không quan trọng.
5. All service interfaces, without exception, must be designed from the ground up to be externalizable. That is to say, the team must plan and design to be able to expose the interface to developers in the outside world. No exceptions.
 5. Tất cả các giao diện dịch vụ, không có ngoại lệ, phải được thiết kế từ đầu để có thể phơi bày ra bên ngoài. Tức là, nhóm phải lên kế hoạch và thiết kế sao cho có thể phơi bày giao diện cho các lập trình viên ở thế giới bên ngoài. Không có ngoại lệ.

The advent of Bezos's API Mandate is widely viewed as a watershed moment for Amazon. Putting data and services behind APIs enabled the loose coupling and eventually resulted in AWS as we know it now. Google's pursuit of loose coupling allowed it to grow its systems to an extraordinary scale. For software architecture, a loosely coupled system has the following properties:

Sự ra đời của API Mandate của Bezos được xem rộng rãi là một khoảnh khắc bước ngoặt của Amazon. Việc đặt dữ liệu và dịch vụ đằng sau các API đã tạo điều kiện cho sự ghép lỏng và cuối cùng cho ra đời AWS như chúng ta biết hiện nay. Việc Google theo đuổi sự ghép lỏng đã cho phép họ phát triển các hệ thống của mình lên một quy mô phi thường. Với kiến trúc phần mềm, một hệ thống ghép lỏng có những thuộc tính sau:

1. Systems are broken into many small components.
2. These systems interface with other services through abstraction layers, such as a messaging bus or an API. These abstraction layers hide and protect internal details of the service, such as a database backend or internal classes and method calls.
3. As a consequence of property 2, internal changes to a system component don't require changes in other parts. Details of code updates are hidden behind stable APIs. Each piece can evolve and improve separately.
4. As a consequence of property 3, there is no waterfall, global release cycle for the whole system. Instead, each component is updated separately as changes and improvements are made.
 1. Các hệ thống được chia thành nhiều thành phần nhỏ.
 2. Những hệ thống này giao tiếp với các dịch vụ khác thông qua các tầng trừu tượng, chẳng hạn một bus tin nhắn hoặc một API. Các tầng trừu tượng này che giấu và bảo vệ những chi tiết nội bộ của dịch vụ, chẳng hạn một backend cơ sở dữ liệu hoặc các lớp nội bộ và lời gọi phương thức.
 3. Là hệ quả của thuộc tính 2, những thay đổi nội bộ trong một thành phần hệ thống không đòi hỏi thay đổi ở các phần khác. Chi tiết của các cập nhật mã được ẩn sau các API ổn định. Mỗi mảnh có thể tiến hóa và cải thiện riêng rẽ.

4. Là hệ quả của thuộc tính 3, không có chu kỳ phát hành kiểu thác nước, toàn cục cho toàn hệ thống. Thay vào đó, mỗi thành phần được cập nhật riêng rẽ khi có thay đổi và cải tiến.

Notice that we are talking about technical systems . We need to think bigger. Let's translate these technical characteristics into organizational characteristics:

Lưu ý rằng chúng ta đang nói về các hệ thống kỹ thuật. Chúng ta cần nghĩ lớn hơn. Hãy chuyển những đặc tính kỹ thuật này thành các đặc tính tổ chức:

1. Many small teams engineer a large, complex system. Each team is tasked with engineering, maintaining, and improving some system components.
 1. Nhiều nhóm nhỏ cùng kỹ thuật hóa một hệ thống lớn, phức tạp. Mỗi nhóm được giao nhiệm vụ kỹ thuật hóa, duy trì và cải thiện một số thành phần hệ thống.
2. These teams **publish** the abstract details of their components to other teams via API definitions, message schemas, etc. Teams need not concern themselves with other teams' components; they simply use the published API or message specifications to call these components. They iterate their part to improve their performance and capabilities over time. They might also publish new capabilities as they are added or request new stuff from other teams. Again, the latter happens without teams needing to worry about the internal technical details of the requested features. Teams work together through loosely coupled communication .
3. As a consequence of characteristic 2, each team can rapidly evolve and improve its component independently of the work of other teams.
4. Specifically, characteristic 3 implies that teams can release updates to their components with minimal downtime. Teams release continuously during regular working hours to make code changes and test them.
 2. Các nhóm này công bố những chi tiết trừu tượng của thành phần của họ cho các nhóm khác thông qua các định nghĩa API, lược đồ (schema) tin nhắn, v.v. Các nhóm không cần bận tâm đến thành phần của các nhóm khác; họ chỉ đơn giản dùng API hoặc đặc tả tin nhắn đã công bố để gọi những thành phần này. Họ lặp đi lặp lại trên phần của mình để cải thiện hiệu suất và năng lực theo thời gian. Họ cũng có thể công bố các năng lực mới khi chúng được thêm vào, hoặc yêu cầu những thứ mới từ các nhóm khác. Một lần nữa, điều sau xảy ra mà các nhóm không cần lo lắng về các chi tiết kỹ thuật nội bộ của những tính năng được yêu cầu. Các nhóm làm việc cùng nhau thông qua giao tiếp ghép lỏng.
 3. Là hệ quả của đặc tính 2, mỗi nhóm có thể nhanh chóng tiến hóa và cải thiện thành phần của mình độc lập với công việc của các nhóm khác.
 4. Cụ thể, đặc tính 3 hàm ý rằng các nhóm có thể phát hành cập nhật cho thành phần của họ với thời gian gián đoạn tối thiểu. Các nhóm phát hành liên tục trong giờ làm việc bình thường để thực hiện các thay đổi mã và kiểm thử chúng.

Loose coupling of both technology and human systems will allow your data engineering teams to more efficiently collaborate with one another and with other parts of the company. This principle also directly facilitates principle 7.

Sự ghép lỏng của cả hệ thống công nghệ lẫn hệ thống con người sẽ cho phép các nhóm kỹ thuật dữ liệu của bạn cộng tác hiệu quả hơn với nhau cũng như với các bộ phận khác của công ty. Nguyên tắc này cũng trực tiếp tạo điều kiện cho nguyên tắc 7.

Principle 7: Make Reversible Decisions — Nguyên tắc 7: Đưa ra những quyết định có thể đảo ngược

The data landscape is changing rapidly. Today's **hot** technology or stack is tomorrow's afterthought. Popular opinion shifts quickly. You should aim for reversible decisions, as these tend to simplify your architecture and keep it agile. As Fowler wrote, "One of an architect's most important tasks is to remove architecture by finding ways to eliminate irreversibility in software designs." What was true when Fowler wrote this in 2003 is just as accurate 4 today. As we said previously, Bezos refers to reversible decisions as "two-way doors." As he says, "If you walk through and don't like what you see on the other side, you can't get back to before. We can call these Type 1 decisions. But most decisions aren't like that—they are changeable, reversible— they're two-way doors." Aim for two-way doors whenever possible. Given the pace of change—and the decoupling/modularization of technologies across your data architecture—always strive to pick the best-of-breed solutions that work for today. Also, be prepared to upgrade or adopt better practices as the landscape evolves.

Bối cảnh dữ liệu đang thay đổi nhanh chóng. Công nghệ hay stack đang hot hôm nay là thứ bị xao nhãng của ngày mai. Quan điểm phổ biến chuyển dịch nhanh chóng. Bạn nên nhắm tới những quyết định có thể đảo ngược, vì chúng có xu hướng đơn giản hóa kiến trúc của bạn và giữ nó nhanh nhẹn. Như Fowler đã viết, "Một trong những nhiệm vụ quan trọng nhất của kiến trúc sư là loại bỏ kiến trúc bằng cách tìm ra những cách triệt tiêu tính bất khả đảo ngược trong các thiết kế phần mềm." Điều đúng khi Fowler viết câu này năm 2003 thì hôm nay vẫn chính xác như vậy. Như chúng tôi đã nói trước đó, Bezos gọi các quyết định có thể đảo ngược là "những cánh cửa hai chiều". Như ông nói, "Nếu bạn bước qua và không thích những gì thấy ở phía bên kia, bạn không thể quay lại trạng thái trước. Ta có thể gọi đây là các quyết định Loại 1. Nhưng phần lớn quyết định không như vậy — chúng có thể thay đổi, có thể đảo ngược — chúng là những cánh cửa hai chiều." Hãy nhắm tới những cánh cửa hai chiều bất cứ khi nào có thể. Trước tốc độ thay đổi — và sự tách rời/mô-đun hóa của các công nghệ trong toàn bộ kiến trúc dữ liệu của bạn — hãy luôn cố gắng chọn những giải pháp tốt nhất trong loại của chúng phù hợp cho hôm nay. Đồng thời, hãy sẵn sàng nâng cấp hoặc áp dụng những thực hành tốt hơn khi bối cảnh tiến hóa.

Principle 8: Prioritize Security — Nguyên tắc 8: Ưu tiên an ninh

Every data engineer must assume responsibility for the security of the systems they build and maintain. We focus now on two main ideas: zero-trust security and the shared responsibility security model. These align closely to a cloud-native architecture.

Mọi kỹ sư dữ liệu đều phải đảm nhận trách nhiệm về an ninh của các hệ thống mà họ xây dựng và duy trì. Giờ chúng ta tập trung vào hai ý chính: an ninh zero-trust và mô hình an ninh trách nhiệm chia sẻ. Hai điều này gắn kết chặt chẽ với một kiến trúc cloud-native.

Hardened-perimeter and zero-trust security models — Mô hình an ninh chu vi kiên cố và zero-trust

To define zero-trust security , it's helpful to start by understanding the traditional hard-perimeter security model and its limitations, as detailed in Google Cloud's Five Principles:

Để định nghĩa an ninh zero-trust, sẽ hữu ích nếu bắt đầu bằng việc hiểu mô hình an ninh chu vi cứng truyền thống cùng những giới hạn của nó, như được trình bày chi tiết trong

Five Principles của Google Cloud:

Traditional architectures place a lot of faith in perimeter security, crudely a hardened network perimeter with “trusted things” inside and “untrusted things” outside. Unfortunately, this approach has always been vulnerable to insider attacks, as well as external threats such as spear phishing. The 1996 film Mission Impossible presents a perfect example of the hard-perimeter security model and its limitations. In the movie, the CIA hosts highly sensitive data on a storage system inside a room with extremely tight physical security. Ethan Hunt infiltrates CIA headquarters and exploits a human target to gain physical access to the storage system. Once inside the secure room, he can exfiltrate data with relative ease. For at least a decade, alarming media reports have made us aware of the growing menace of security breaches that exploit human targets inside hardened organizational security perimeters. Even as employees work on highly secure corporate networks, they remain connected to the outside world through email and mobile devices. External threats effectively become internal threats. In a cloud-native environment, the notion of a hardened perimeter erodes further. All assets are connected to the outside world to some degree. While virtual private cloud (VPC) networks can be defined with no external connectivity, the API control plane that engineers use to define these networks still faces the internet.

Các kiến trúc truyền thống đặt rất nhiều niềm tin vào an ninh chu vi, nói nôm na là một chu vi mạng được kiên cố hóa với “những thứ đáng tin cậy” bên trong và “những thứ không đáng tin cậy” bên ngoài. Đáng tiếc, cách tiếp cận này luôn dễ bị tấn công từ bên trong, cũng như các mối đe dọa bên ngoài như spear phishing. Bộ phim Mission Impossible năm 1996 trình bày một ví dụ hoàn hảo về mô hình an ninh chu vi cứng và những giới hạn của nó. Trong phim, CIA lưu trữ dữ liệu cực kỳ nhạy cảm trên một hệ thống lưu trữ bên trong một căn phòng có an ninh vật lý cực kỳ chặt chẽ. Ethan Hunt xâm nhập vào trụ sở CIA và lợi dụng một mục tiêu con người để có được quyền truy cập vật lý vào hệ thống lưu trữ. Một khi đã ở bên trong căn phòng an ninh, anh ta có thể tuồn dữ liệu ra ngoài tương đối dễ dàng. Trong ít nhất một thập kỷ, những bản tin đáng báo động trên truyền thông đã khiến chúng ta nhận thức được mối đe dọa ngày càng lớn của các vụ xâm phạm an ninh lợi dụng các mục tiêu con người bên trong những chu vi an ninh tổ chức được kiên cố hóa. Ngay cả khi nhân viên làm việc trên những mạng doanh nghiệp cực kỳ an toàn, họ vẫn kết nối với thế giới bên ngoài qua email và thiết bị di động. Các mối đe dọa bên ngoài hiệu quả trở thành mối đe dọa bên trong. Trong một môi trường cloud-native, khái niệm về một chu vi kiên cố càng xói mòn hơn nữa. Tất cả tài sản đều kết nối với thế giới bên ngoài ở một mức độ nào đó. Dù các mạng đám mây riêng ảo (VPC) có thể được định nghĩa không có kết nối bên ngoài, mặt phẳng điều khiển API mà các kỹ sư dùng để định nghĩa những mạng này vẫn hướng ra internet.

The shared responsibility model — Mô hình trách nhiệm chia sẻ

Amazon emphasizes the **shared responsibility model**, which divides security into the security of the cloud and security in the cloud. AWS is responsible for the security of the cloud:

Amazon nhấn mạnh mô hình trách nhiệm chia sẻ (shared responsibility model), vốn chia an ninh thành an ninh của đám mây và an ninh trong đám mây. AWS chịu trách nhiệm về an ninh của đám mây:

AWS is responsible for protecting the infrastructure that runs AWS services in the AWS Cloud. AWS also provides you with services that you can use securely. AWS users are responsible for security in the cloud:

AWS chịu trách nhiệm bảo vệ hạ tầng vận hành các dịch vụ AWS trong AWS Cloud. AWS

cũng cung cấp cho bạn các dịch vụ mà bạn có thể dùng một cách an toàn. Người dùng AWS chịu trách nhiệm về an ninh trong đám mây:

Your responsibility is determined by the AWS service that you use. You are also responsible for other factors including the sensitivity of your data, your organization's requirements, and applicable laws and regulations. In general, all cloud providers operate on this shared responsibility model. They secure their services according to published specifications. Still, it is ultimately the user's responsibility to design a security model for their applications and data and leverage cloud capabilities to realize this model.

Trách nhiệm của bạn được quyết định bởi dịch vụ AWS mà bạn dùng. Bạn cũng chịu trách nhiệm về các yếu tố khác bao gồm độ nhạy cảm của dữ liệu của bạn, các yêu cầu của tổ chức bạn, và các luật cùng quy định áp dụng. Nhìn chung, tất cả các nhà cung cấp đám mây đều vận hành theo mô hình trách nhiệm chia sẻ này. Họ bảo mật các dịch vụ của mình theo các đặc tả đã công bố. Tuy vậy, rốt cuộc trách nhiệm của người dùng là thiết kế một mô hình an ninh cho các ứng dụng và dữ liệu của họ và tận dụng các năng lực đám mây để hiện thực hóa mô hình này.

Data engineers as security engineers — Kỹ sư dữ liệu trong vai trò kỹ sư an ninh

In the corporate world today, a command-and-control approach to security is quite common, wherein security and networking teams manage perimeters and general security practices. The cloud pushes this responsibility out to engineers who are not explicitly in security roles. Because of this responsibility, in conjunction with more general erosion of the hard security perimeter, all data engineers should consider themselves security engineers. Failure to assume these new implicit responsibilities can lead to dire consequences. Numerous data breaches have resulted from the simple error of configuring Amazon S3 buckets with public access. Those who handle data must assume that they are ultimately responsible for securing it.

Trong thế giới doanh nghiệp ngày nay, một cách tiếp cận chỉ huy-kiểm soát đối với an ninh khá phổ biến, theo đó các nhóm an ninh và mạng quản lý các chu vi cùng những thực hành an ninh chung. Đám mây đẩy trách nhiệm này ra ngoài tới các kỹ sư không nằm rõ ràng trong các vai trò an ninh. Vì trách nhiệm này, cùng với sự xói mòn nói chung của chu vi an ninh cứng, tất cả kỹ sư dữ liệu nên tự xem mình là kỹ sư an ninh. Không đảm nhận những trách nhiệm ngầm mới này có thể dẫn đến những hậu quả thảm khốc. Vô số vụ rò rỉ dữ liệu đã bắt nguồn từ lỗi đơn giản là cấu hình các bucket Amazon S3 với quyền truy cập công khai. Những ai xử lý dữ liệu phải giả định rằng rốt cuộc họ chịu trách nhiệm bảo mật nó.

Principle 9: Embrace FinOps — Nguyên tắc 9: Đón nhận FinOps

Let's start by considering a couple of definitions of FinOps. First, the FinOps Foundation offers this:

Hãy bắt đầu bằng cách xem xét một vài định nghĩa về FinOps. Đầu tiên, FinOps Foundation đưa ra định nghĩa này:

FinOps is an evolving cloud financial management discipline and cultural practice that enables organizations to get maximum business value by helping engineering, finance, technology, and business teams to collaborate on data-driven spending decisions. In addition, J.R. Sormont and Mike Fuller provide the following definition in Cloud FinOps (O'Reilly):

FinOps là một bộ môn quản lý tài chính đám mây đang tiến hóa và một thực hành văn hóa, cho phép các tổ chức đạt được giá trị kinh doanh tối đa bằng cách giúp các nhóm kỹ thuật, tài chính, công nghệ và kinh doanh cộng tác trong những quyết định chi tiêu dựa trên dữ liệu. Ngoài ra, J.R. Sorment và Mike Fuller đưa ra định nghĩa sau trong cuốn Cloud FinOps (O'Reilly):

The term “FinOps” typically refers to the emerging professional movement that advocates a collaborative working relationship between DevOps and Finance, resulting in an iterative, data-driven management of infrastructure spending (i.e., lowering the unit economics of cloud) while simultaneously increasing the cost efficiency and, ultimately, the profitability of the cloud environment. The cost structure of data has evolved dramatically during the cloud era. In an on-premises setting, data systems are generally acquired with a capital expenditure (described more in Chapter 4) for a new system every few years in an on-premises setting. Responsible parties have to balance their budget against desired compute and storage capacity. Overbuying entails wasted money, while underbuying means hampering future data projects and driving significant personnel time to control system load and data size; underbuying may require faster technology refresh cycles, with associated extra costs.

Thuật ngữ “FinOps” thường ám chỉ phong trào chuyên môn đang nổi lên, ủng hộ một mối quan hệ làm việc cộng tác giữa DevOps và Tài chính, dẫn đến việc quản lý chi tiêu hạ tầng theo cách lặp lại, dựa trên dữ liệu (tức hạ thấp kinh tế đơn vị của đám mây), đồng thời gia tăng hiệu quả chi phí và rốt cuộc là lợi nhuận của môi trường đám mây. Cấu trúc chi phí của dữ liệu đã tiến hóa mạnh mẽ trong kỷ nguyên đám mây. Trong bối cảnh on-premises, các hệ thống dữ liệu thường được mua bằng một khoản chi đầu tư (được mô tả thêm trong Chương 4) cho một hệ thống mới vài năm một lần trong bối cảnh on-premises. Các bên chịu trách nhiệm phải cân bằng ngân sách của họ với năng lực tính toán và lưu trữ mong muốn. Mua thừa kéo theo lãng phí tiền bạc, trong khi mua thiếu nghĩa là cản trở các dự án dữ liệu tương lai và khiến tốn nhiều thời gian nhân sự để kiểm soát tải hệ thống và kích thước dữ liệu; mua thiếu có thể đòi hỏi các chu kỳ làm mới công nghệ nhanh hơn, kèm theo các chi phí phụ liên quan.

In the cloud era, most data systems are pay-as-you-go and readily scalable. Systems can run on a cost per query model, cost per processing capacity model, or another variant of a pay-as-you-go model. This approach can be far more efficient than the capital expenditure approach. It is now possible to scale up for high performance, and then scale down to save money. However, the pay-as-you-go approach makes spending far more dynamic. The new challenge for data leaders is to manage budgets, priorities, and efficiency. Cloud tooling necessitates a set of processes for managing spending and resources. In the past, data engineers thought in terms of performance engineering—maximizing the performance for data processes on a fixed set of resources and buying adequate resources for future needs. With FinOps, engineers need to learn to think about the cost structures of cloud systems. For example, what is the appropriate mix of AWS spot instances when running a distributed cluster? What is the most appropriate approach for running a sizable daily job in terms of cost-effectiveness and performance? When should the company switch from a pay-per-query model to reserved capacity? FinOps evolves the operational monitoring model to monitor spending on an ongoing basis. Rather than simply monitor requests and CPU utilization for a web server, FinOps might monitor the ongoing cost of serverless functions handling traffic, as well as spikes in spending trigger alerts. Just as systems are designed to fail gracefully in excessive traffic, companies may consider adopting hard limits for spending, with graceful failure modes in response to spending spikes. Ops teams should also think in terms of cost attacks. Just as a distributed denial-of-service (DDoS) attack can block access to a web server, many companies have discovered to their chagrin that excessive downloads from S3 buckets can drive spending through the roof and threaten a small startup with bankruptcy. When sharing data publicly, data teams can address these issues by

setting requester-pays policies, or simply monitoring for excessive data access spending and quickly removing access if spending begins to rise to unacceptable levels.

Trong kỷ nguyên đám mây, hầu hết các hệ thống dữ liệu là trả theo mức dùng (pay-as-you-go) và dễ dàng mở rộng quy mô. Các hệ thống có thể chạy theo mô hình chi phí trên mỗi truy vấn, mô hình chi phí trên mỗi năng lực xử lý, hoặc một biến thể khác của mô hình trả theo mức dùng. Cách tiếp cận này có thể hiệu quả hơn nhiều so với cách tiếp cận chi đầu tư. Giờ đây ta có thể mở rộng lên để có hiệu năng cao, rồi thu hẹp xuống để tiết kiệm tiền. Tuy nhiên, cách tiếp cận trả theo mức dùng khiến việc chi tiêu trở nên động hơn nhiều. Thách thức mới cho các lãnh đạo dữ liệu là quản lý ngân sách, ưu tiên và hiệu quả. Công cụ đám mây đòi hỏi một tập các quy trình để quản lý chi tiêu và tài nguyên. Trong quá khứ, kỹ sư dữ liệu nghĩ theo hướng kỹ thuật hiệu năng — tối đa hóa hiệu năng cho các tiến trình dữ liệu trên một tập tài nguyên cố định và mua đủ tài nguyên cho nhu cầu tương lai. Với FinOps, các kỹ sư cần học cách nghĩ về cấu trúc chi phí của các hệ thống đám mây. Ví dụ, tỷ lệ kết hợp các spot instance của AWS phù hợp khi chạy một cụm phân tán là bao nhiêu? Cách tiếp cận phù hợp nhất để chạy một công việc hằng ngày có quy mô đáng kể xét về hiệu quả chi phí và hiệu năng là gì? Khi nào công ty nên chuyển từ mô hình trả-trên-mỗi-truy-vấn sang năng lực dự trữ (reserved capacity)? FinOps tiến hóa mô hình giám sát vận hành để giám sát chi tiêu trên cơ sở liên tục. Thay vì chỉ giám sát các yêu cầu và mức sử dụng CPU cho một máy chủ web, FinOps có thể giám sát chi phí liên tục của các hàm serverless xử lý lưu lượng, cũng như các đợt tăng đột biến trong chi tiêu kích hoạt cảnh báo. Cũng như các hệ thống được thiết kế để hồng một cách duyên dáng khi lưu lượng quá lớn, các công ty có thể cân nhắc áp dụng các giới hạn cứng cho chi tiêu, với các chế độ hồng duyên dáng để đáp ứng các đợt tăng đột biến chi tiêu. Các nhóm vận hành cũng nên nghĩ theo hướng các cuộc tấn công chi phí. Cũng như một cuộc tấn công từ chối dịch vụ phân tán (DDoS) có thể chặn truy cập vào một máy chủ web, nhiều công ty đã ngậm ngùi phát hiện ra rằng việc tải xuống quá mức từ các bucket S3 có thể đẩy chi tiêu vọt lên trời và đe dọa một startup nhỏ với nguy cơ phá sản. Khi chia sẻ dữ liệu công khai, các nhóm dữ liệu có thể giải quyết những vấn đề này bằng cách thiết lập các chính sách requester-pays, hoặc đơn giản giám sát chi tiêu truy cập dữ liệu quá mức và nhanh chóng gỡ bỏ quyền truy cập nếu chi tiêu bắt đầu tăng đến những mức không chấp nhận được.

As of this writing, FinOps is a recently formalized practice. The FinOps Foundation was started only in 2019. However, we highly recommend you start thinking about FinOps early, before you encounter high cloud bills. Start your journey with the FinOps Foundation and O'Reilly's Cloud FinOps . We also suggest that data engineers involve themselves in the community process of creating FinOps practices for data engineering— in such a new practice area, a good deal of territory is yet to be mapped out. Now that you have a high-level understanding of good data architecture principles, let's dive a bit deeper into the major concepts you'll need to design and build good data architecture.

Vào thời điểm viết cuốn sách này, FinOps là một thực hành mới được chính thức hóa gần đây. FinOps Foundation chỉ mới được thành lập năm 2019. Tuy nhiên, chúng tôi rất khuyến nghị bạn bắt đầu nghĩ về FinOps sớm, trước khi bạn gặp những hóa đơn đám mây cao ngất. Hãy bắt đầu hành trình của bạn với FinOps Foundation và cuốn Cloud FinOps của O'Reilly. Chúng tôi cũng đề xuất kỹ sư dữ liệu tham gia vào quá trình cộng đồng tạo ra các thực hành FinOps cho data engineering — trong một lĩnh vực thực hành mới mẻ như vậy, còn rất nhiều vùng đất chưa được khai phá. Giờ khi bạn đã có hiểu biết tổng quan về các nguyên tắc của kiến trúc dữ liệu tốt, hãy đi sâu hơn một chút vào những khái niệm chính mà bạn sẽ cần để thiết kế và xây dựng kiến trúc dữ liệu tốt.

Major Architecture Concepts — Các khái niệm kiến trúc chính

If you follow the current trends in data, it seems like new types of data tools and architectures are arriving on the scene every week. Amidst this flurry of activity, we must not lose sight of the main goal of all of these architectures: to take data and transform it into something useful for **downstream** consumption.

Nếu bạn theo dõi các xu hướng hiện thời trong lĩnh vực dữ liệu, có vẻ như các loại công cụ và kiến trúc dữ liệu mới xuất hiện trên sân khấu mỗi tuần. Giữa cơn lốc hoạt động này, chúng ta không được đánh mất tầm nhìn về mục tiêu chính của tất cả các kiến trúc này: lấy dữ liệu và biến đổi nó thành thứ gì đó hữu ích cho việc tiêu thụ ở hạ nguồn.

Domains and Services — Miền và dịch vụ

A sphere of knowledge, influence, or activity. The subject area to which the user applies a program is the domain of the software. —Eric Evans Before diving into the components of the architecture, let's briefly cover two terms you'll see come up very often: domain and services. A domain is the real-world subject area for which you're architecting. A service is a set of functionality whose goal is to accomplish a task. For example, you might have a sales order-processing service whose task is to process orders as they are created. The sales order-processing service's only job is to process orders; it doesn't provide other functionality, such as inventory management or updating user profiles.

Một phạm vi tri thức, ảnh hưởng, hoặc hoạt động. Lĩnh vực chủ đề mà người dùng áp dụng một chương trình chính là miền của phần mềm. —Eric Evans. Trước khi đi sâu vào các thành phần của kiến trúc, hãy điểm qua nhanh hai thuật ngữ mà bạn sẽ thấy xuất hiện rất thường xuyên: miền (domain) và dịch vụ (service). Một miền là lĩnh vực chủ đề thực tế mà bạn đang kiến trúc cho nó. Một dịch vụ là một tập chức năng có mục tiêu hoàn thành một tác vụ. Ví dụ, bạn có thể có một dịch vụ xử lý đơn hàng bán hàng với tác vụ là xử lý các đơn hàng khi chúng được tạo ra. Công việc duy nhất của dịch vụ xử lý đơn hàng bán hàng là xử lý các đơn hàng; nó không cung cấp chức năng khác, chẳng hạn quản lý kho hoặc cập nhật hồ sơ người dùng.

A domain can contain multiple services. For example, you might have a sales domain with three services: orders, invoicing, and products. Each service has particular tasks that support the sales domain. Other domains may also share services (Figure 3-3). In this case, the accounting domain is responsible for basic accounting functions: invoicing, payroll, and accounts receivable (AR). Notice the accounting domain shares the invoice service with the sales domain since a sale generates an invoice, and accounting must keep track of invoices to ensure that payment is received. Sales and accounting own their respective domains.

Một miền có thể chứa nhiều dịch vụ. Ví dụ, bạn có thể có một miền bán hàng với ba dịch vụ: đơn hàng, lập hóa đơn, và sản phẩm. Mỗi dịch vụ có những tác vụ cụ thể hỗ trợ miền bán hàng. Các miền khác cũng có thể chia sẻ các dịch vụ (Hình 3-3). Trong trường hợp này, miền kế toán chịu trách nhiệm về các chức năng kế toán cơ bản: lập hóa đơn, trả lương, và các khoản phải thu (AR). Lưu ý miền kế toán chia sẻ dịch vụ hóa đơn với miền bán hàng vì một giao dịch bán hàng sinh ra một hóa đơn, và kế toán phải theo dõi các hóa đơn để đảm bảo nhận được thanh toán. Bán hàng và kế toán sở hữu các miền tương ứng của mình.

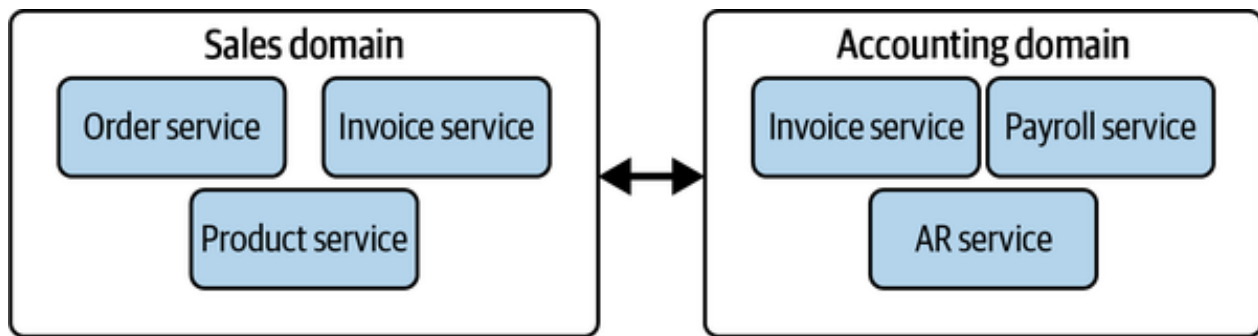


Figure 3-3. Two domains (sales and accounting) share a common service (invoices), and sales and accounting own their respective domains. When thinking about what constitutes a domain, focus on what the domain represents in the real world and work backward. In the preceding example, the sales domain should represent what happens with the sales function in your company. When architecting the sales domain, avoid cookie-cutter copying and pasting from what other companies do. Your company's sales function likely has unique aspects that require specific services to make it work the way your sales team expects. Identify what should go in the domain. When determining what the domain should encompass and what services to include, the best advice is to simply go and talk with users and stakeholders, listen to what they're saying, and build the services that will help them do their job. Avoid the classic trap of architecting in a vacuum.

Hình 3-3. Hai miền (bán hàng và kế toán) chia sẻ một dịch vụ chung (hóa đơn), và bán hàng cùng kế toán sở hữu các miền tương ứng của mình. Khi nghĩ về những gì cấu thành một miền, hãy tập trung vào điều mà miền đó đại diện trong thế giới thực và làm ngược lại từ đó. Trong ví dụ trên, miền bán hàng nên đại diện cho những gì xảy ra với chức năng bán hàng trong công ty của bạn. Khi kiến trúc miền bán hàng, hãy tránh sao chép-dán rập khuôn từ những gì các công ty khác làm. Chức năng bán hàng của công ty bạn nhiều khả năng có những khía cạnh độc đáo đòi hỏi những dịch vụ cụ thể để vận hành theo cách mà đội bán hàng của bạn mong đợi. Hãy nhận diện những gì nên nằm trong miền. Khi xác định miền nên bao trùm những gì và bao gồm những dịch vụ nào, lời khuyên tốt nhất là cứ đi nói chuyện với người dùng và các bên liên quan, lắng nghe những gì họ nói, và xây dựng những dịch vụ sẽ giúp họ làm việc của mình. Hãy tránh cái bẫy kinh điển là kiến trúc trong chân không.

Distributed Systems, Scalability, and Designing for Failure — Hệ phân tán, khả năng mở rộng quy mô, và thiết kế cho thất bại

The discussion in this section is related to our second and third principles of data engineering architecture discussed previously: plan for failure and architect for scalability. As data engineers, we're interested in four closely related characteristics of data systems (availability and reliability were mentioned previously, but we reiterate them here for completeness): Scalability

Phần thảo luận trong mục này liên quan đến nguyên tắc thứ hai và thứ ba của kiến trúc kỹ thuật dữ liệu đã bàn ở trên: dự liệu cho thất bại và kiến trúc cho khả năng mở rộng quy mô. Là kỹ sư dữ liệu, chúng ta quan tâm đến bốn đặc tính liên hệ mật thiết của các hệ thống dữ liệu (tính sẵn sàng và độ tin cậy đã được đề cập trước đó, nhưng chúng tôi nhắc lại ở đây cho đầy đủ): Khả năng mở rộng quy mô (Scalability)

Allows us to increase the capacity of a system to improve performance and handle the demand. For example, we might want to scale a system to handle a high rate of queries or process a huge data set.

Cho phép chúng ta tăng năng lực của một hệ thống để cải thiện hiệu năng và xử lý nhu cầu. Ví dụ, chúng ta có thể muốn mở rộng một hệ thống để xử lý một tốc độ truy vấn cao hoặc xử lý một tập dữ liệu khổng lồ.

Elasticity

Tính đàn hồi (Elasticity)

The ability of a scalable system to scale dynamically; a highly elastic system can automatically scale up and down based on the current workload. Scaling up is critical as demand increases, while scaling down saves money in a cloud environment. Modern systems sometimes scale to zero, meaning they can automatically shut down when idle.

Khả năng của một hệ thống mở rộng được để mở rộng quy mô một cách động; một hệ thống đàn hồi cao có thể tự động mở rộng lên và xuống dựa trên khối lượng công việc hiện tại. Mở rộng lên là then chốt khi nhu cầu tăng, trong khi thu hẹp xuống tiết kiệm tiền trong một môi trường đám mây. Các hệ thống hiện đại đôi khi mở rộng về không, nghĩa là chúng có thể tự động tắt khi rảnh rỗi.

Availability

Tính sẵn sàng (Availability)

The percentage of time an IT service or component is in an operable state.

Tỷ lệ phần trăm thời gian một dịch vụ hoặc thành phần CNTT ở trạng thái có thể vận hành.

Reliability

Độ tin cậy (Reliability)

The system's probability of meeting defined standards in performing its intended function during a specified interval.

Xác suất hệ thống đáp ứng các tiêu chuẩn đã định trong việc thực hiện chức năng dự kiến của nó trong một khoảng thời gian xác định.

TIP — MẸO

See PagerDuty's "Why Are Availability and Reliability Crucial?" web page for definitions and background on availability and reliability.

Xem trang web "Why Are Availability and Reliability Crucial?" của PagerDuty để biết các định nghĩa và bối cảnh về tính sẵn sàng và độ tin cậy.

How are these characteristics related? If a system fails to meet performance requirements during a specified interval, it may become unresponsive. Thus low reliability can lead to low availability. On the other hand, dynamic scaling helps ensure adequate performance without manual intervention from engineers—elasticity improves reliability. Scalability can be realized in a variety of ways. For your services and domains, does a single machine handle everything? A single machine can be scaled vertically; you can increase resources (CPU, disk, memory, I/O). But there are hard limits to possible resources on a single machine. Also, what happens if this machine dies? Given enough time, some components will eventually fail. What's your plan for backup and failover? Single machines generally can't offer high availability and reliability. We utilize a **distributed system** to realize higher overall scaling capacity and increased availability and reliability. **Horizontal** scaling allows you to add more machines to satisfy load and resource requirements (Figure 3-4). Common horizontally scaled systems have a leader node that acts as the main point of contact for the instantiation, progress, and

completion of workloads. When a workload is started, the leader node distributes tasks to the worker nodes within its system, completing the tasks and returning the results to the leader node. Typical modern distributed architectures also build in redundancy. Data is replicated so that if a machine dies, the other machines can pick up where the missing server left off; the cluster may add more machines to restore capacity.

Những đặc tính này liên hệ với nhau như thế nào? Nếu một hệ thống không đáp ứng được các yêu cầu hiệu năng trong một khoảng thời gian xác định, nó có thể trở nên không phản hồi. Do đó độ tin cậy thấp có thể dẫn đến tính sẵn sàng thấp. Mặt khác, mở rộng quy mô động giúp đảm bảo hiệu năng thỏa đáng mà không cần can thiệp thủ công từ các kỹ sư — tính đàn hồi cải thiện độ tin cậy. Khả năng mở rộng quy mô có thể được hiện thực hóa bằng nhiều cách. Với các dịch vụ và miền của bạn, một máy đơn có xử lý mọi thứ không? Một máy đơn có thể được mở rộng theo chiều dọc; bạn có thể tăng tài nguyên (CPU, đĩa, bộ nhớ, I/O). Nhưng có những giới hạn cứng về tài nguyên khả dĩ trên một máy đơn. Ngoài ra, điều gì xảy ra nếu máy này chết? Cho đủ thời gian, một số thành phần rất cuộc sẽ hỏng. Kế hoạch sao lưu và chuyển đổi dự phòng của bạn là gì? Các máy đơn nói chung không thể cung cấp tính sẵn sàng và độ tin cậy cao. Chúng ta tận dụng một hệ phân tán để hiện thực hóa năng lực mở rộng tổng thể cao hơn cùng tính sẵn sàng và độ tin cậy được nâng cao. Mở rộng theo chiều ngang cho phép bạn thêm nhiều máy hơn để thỏa mãn các yêu cầu về tải và tài nguyên (Hình 3-4). Các hệ thống mở rộng ngang phổ biến có một nút lãnh đạo (leader) đóng vai trò là điểm liên hệ chính cho việc khởi tạo, tiến trình và hoàn thành các khối lượng công việc. Khi một khối lượng công việc được khởi động, nút lãnh đạo phân phối các tác vụ tới các nút công nhân (worker) trong hệ thống của nó, hoàn thành các tác vụ và trả kết quả về cho nút lãnh đạo. Các kiến trúc phân tán hiện đại điển hình cũng tích hợp tính dư thừa. Dữ liệu được nhân bản (replicate) sao cho nếu một máy chết, các máy khác có thể tiếp tục từ chỗ mà máy chủ bị mất để lại; cụm có thể thêm nhiều máy hơn để khôi phục năng lực.

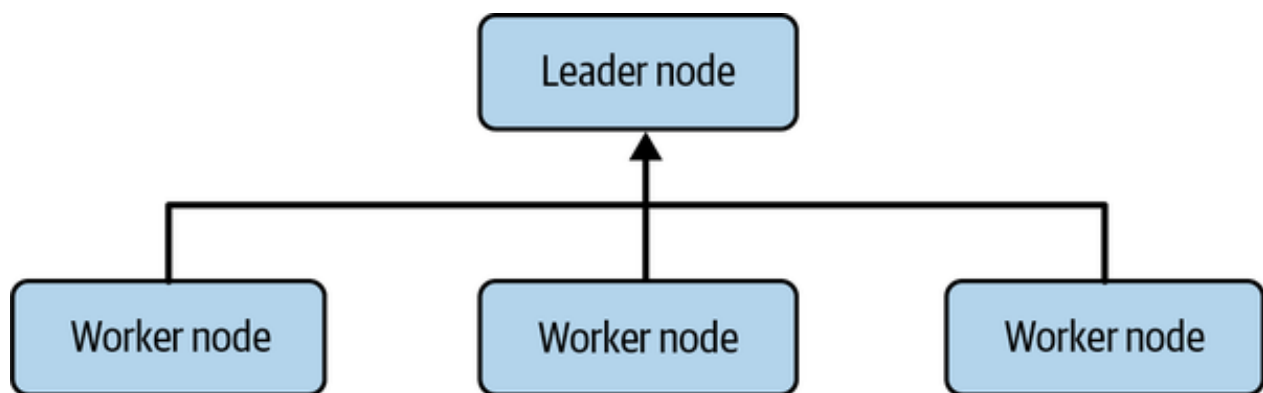


Figure 3-4. A simple horizontal distributed system utilizing a leader-follower architecture, with one leader node and three worker nodes. Distributed systems are widespread in the various data technologies you'll use across your architecture. Almost every cloud **data warehouse** object storage system you use has some notion of distribution under the hood. Management details of the distributed system are typically abstracted away, allowing you to focus on high-level architecture instead of low-level plumbing. However, we highly recommend that you learn more about distributed systems because these details can be extremely helpful in understanding and improving the performance of your pipelines; Martin Kleppmann's *Designing Data-Intensive Applications* (O'Reilly) is an excellent resource.

Hình 3-4. Một hệ phân tán theo chiều ngang đơn giản tận dụng kiến trúc leader-follower, với một nút lãnh đạo và ba nút công nhân. Hệ phân tán phổ biến rộng rãi trong các công

nghe dữ liệu khác nhau mà bạn sẽ dùng trong kiến trúc của mình. Hầu như mọi hệ thống lưu trữ đối tượng kho dữ liệu đám mây mà bạn dùng đều có một khái niệm phân tán nào đó dưới lớp vỏ. Các chi tiết quản lý của hệ phân tán thường được trừu tượng hóa đi, cho phép bạn tập trung vào kiến trúc cấp cao thay vì hệ thống ống nước cấp thấp. Tuy nhiên, chúng tôi rất khuyến nghị bạn tìm hiểu thêm về hệ phân tán vì những chi tiết này có thể cực kỳ hữu ích trong việc hiểu và cải thiện hiệu năng của các đường ống của bạn; cuốn *Designing Data-Intensive Applications* (O'Reilly) của Martin Kleppmann là một tài nguyên xuất sắc.

Tight Versus Loose Coupling: Tiers, Monoliths, and Microservices **— Ghép chặt so với ghép lỏng: Tầng, monolith, và microservices**

When designing a data architecture, you choose how much interdependence you want to include within your various domains, services, and resources. On one end of the spectrum, you can choose to have extremely centralized dependencies and workflows. Every part of a domain and service is vitally dependent upon every other domain and service. This pattern is known as tightly coupled. On the other end of the spectrum, you have decentralized domains and services that do not have strict dependence on each other, in a pattern known as loose coupling. In a loosely coupled scenario, it's easy for decentralized teams to build systems whose data may not be usable by their peers. Be sure to assign common standards, ownership, responsibility, and accountability to the teams owning their respective domains and services. Designing “good” data architecture relies on trade-offs between the tight and loose coupling of domains and services. It's worth noting that many of the ideas in this section originate in software development. We'll try to retain the context of these big ideas' original intent and spirit—keeping them agnostic of data—while later explaining some differences you should be aware of when applying these concepts to data specifically.

Khi thiết kế một kiến trúc dữ liệu, bạn lựa chọn mức độ phụ thuộc lẫn nhau mà bạn muốn đưa vào giữa các miền, dịch vụ và tài nguyên khác nhau. Ở một đầu của phổ, bạn có thể chọn có những phụ thuộc và luồng công việc cực kỳ tập trung. Mọi phần của một miền và dịch vụ phụ thuộc sống còn vào mọi miền và dịch vụ khác. Mẫu hình này được gọi là ghép chặt (tightly coupled). Ở đầu kia của phổ, bạn có những miền và dịch vụ phi tập trung không phụ thuộc chặt chẽ vào nhau, theo một mẫu hình gọi là ghép lỏng (loose coupling). Trong một kịch bản ghép lỏng, các nhóm phi tập trung dễ dàng xây dựng những hệ thống mà dữ liệu của chúng có thể không dùng được cho các đồng nghiệp. Hãy chắc chắn gán những chuẩn chung, quyền sở hữu, trách nhiệm và trách nhiệm giải trình cho các nhóm sở hữu các miền và dịch vụ tương ứng của họ. Thiết kế kiến trúc dữ liệu “tốt” dựa trên những đánh đổi giữa ghép chặt và ghép lỏng của các miền và dịch vụ. Đáng lưu ý là nhiều ý tưởng trong mục này bắt nguồn từ phát triển phần mềm. Chúng tôi sẽ cố giữ lại bối cảnh về ý định và tinh thần ban đầu của những ý tưởng lớn này — giữ chúng độc lập với dữ liệu — trong khi sau đó giải thích một số khác biệt mà bạn nên lưu ý khi áp dụng những khái niệm này cụ thể cho dữ liệu.

Architecture tiers — Các tầng kiến trúc

As you develop your architecture, it helps to be aware of architecture tiers. Your architecture has layers—data, application, business logic, presentation, and so forth—and you need to know how to decouple these layers. Because tight coupling of modalities presents obvious vulnerabilities, keep in mind how you structure the layers of your architecture to achieve maximum reliability and

flexibility. Let's look at single-tier and multitier architecture.

Khi phát triển kiến trúc của mình, sẽ hữu ích nếu bạn nhận thức được các tầng kiến trúc. Kiến trúc của bạn có các lớp — dữ liệu, ứng dụng, logic nghiệp vụ, trình bày, v.v. — và bạn cần biết cách tách rời những lớp này. Vì việc ghép chặt các phương thức tạo ra những điểm yếu rõ ràng, hãy ghi nhớ cách bạn cấu trúc các lớp của kiến trúc để đạt được độ tin cậy và tính linh hoạt tối đa. Hãy xem kiến trúc một tầng và đa tầng.

Single tier In a single-tier architecture, your database and application are tightly coupled, residing on a single server (Figure 3-5). This server could be your laptop or a single virtual machine (VM) in the cloud. The tightly coupled nature means if the server, the database, or the application fails, the entire architecture fails. While single-tier architectures are good for prototyping and development, they are not advised for production environments because of the obvious failure risks.

Một tầng. Trong một kiến trúc một tầng (single-tier), cơ sở dữ liệu và ứng dụng của bạn ghép chặt, nằm trên một máy chủ đơn (Hình 3-5). Máy chủ này có thể là laptop của bạn hoặc một máy ảo (VM) đơn trên đám mây. Bản chất ghép chặt nghĩa là nếu máy chủ, cơ sở dữ liệu, hoặc ứng dụng hỏng, toàn bộ kiến trúc hỏng theo. Dù các kiến trúc một tầng tốt cho việc tạo nguyên mẫu và phát triển, chúng không được khuyến dùng cho các môi trường sản xuất vì những rủi ro thất bại rõ ràng.

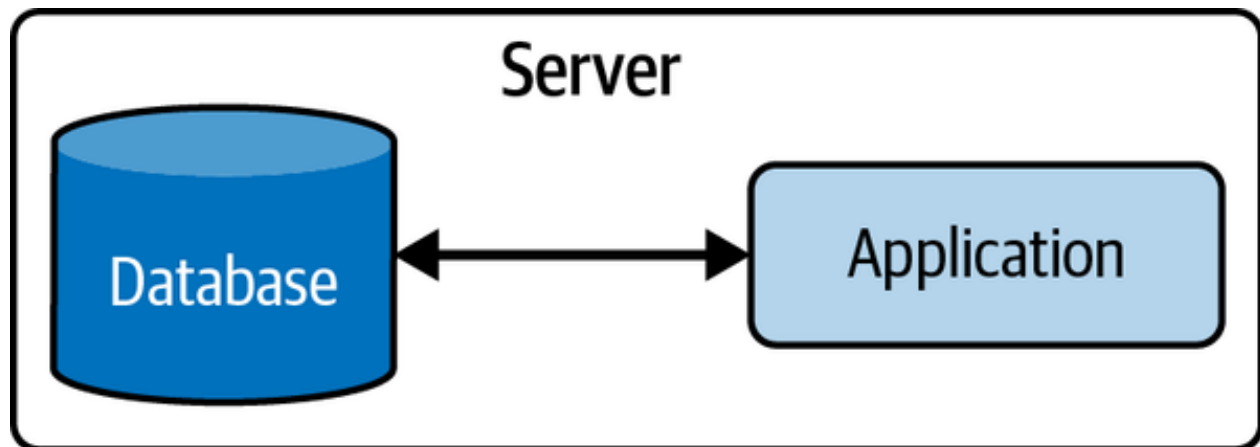


Figure 3-5. Single-tier architecture Even when single-tier architectures build in redundancy (for example, a failover replica), they present significant limitations in other ways. For instance, it is often impractical (and not advisable) to run **analytics** queries against production application databases. Doing so risks overwhelming the database and causing the application to become unavailable. A single-tier architecture is fine for testing systems on a local machine but is not advised for production uses.

Hình 3-5. Kiến trúc một tầng. Ngay cả khi các kiến trúc một tầng tích hợp tính dư thừa (ví dụ một bản sao chuyển đổi dự phòng), chúng vẫn bộc lộ những giới hạn đáng kể theo những cách khác. Chẳng hạn, thường là không thực tế (và không nên) chạy các truy vấn phân tích trên các cơ sở dữ liệu ứng dụng sản xuất. Làm vậy có nguy cơ làm quá tải cơ sở dữ liệu và khiến ứng dụng trở nên không khả dụng. Một kiến trúc một tầng thì ổn để kiểm thử các hệ thống trên một máy cục bộ nhưng không được khuyến dùng cho mục đích sản xuất.

Multitier The challenges of a tightly coupled single-tier architecture are solved by decoupling the data and application. A multitier (also known as n-tier) architecture is composed of separate layers: data, application, business logic, presentation, etc. These layers are bottom-up and hierarchical, meaning the lower layer isn't necessarily dependent on the upper layers; the upper layers depend

on the lower layers. The notion is to separate data from the application, and application from the presentation. A common multitier architecture is a three-tier architecture, a widely used client-server design. A three-tier architecture consists of data, application logic, and presentation tiers (Figure 3-6). Each tier is isolated from the other, allowing for separation of concerns. With a three-tier architecture, you're free to use whatever technologies you prefer within each tier without the need to be monolithically focused.

Đa tầng. Những thách thức của một kiến trúc một tầng ghép chặt được giải quyết bằng cách tách rời dữ liệu và ứng dụng. Một kiến trúc đa tầng (multitier, còn gọi là n-tier) gồm các lớp riêng biệt: dữ liệu, ứng dụng, logic nghiệp vụ, trình bày, v.v. Các lớp này đi từ dưới lên và phân cấp, nghĩa là lớp dưới không nhất thiết phụ thuộc vào các lớp trên; các lớp trên phụ thuộc vào các lớp dưới. Ý tưởng là tách dữ liệu khỏi ứng dụng, và ứng dụng khỏi trình bày. Một kiến trúc đa tầng phổ biến là kiến trúc ba tầng, một thiết kế client-server được dùng rộng rãi. Một kiến trúc ba tầng gồm các tầng dữ liệu, logic ứng dụng, và trình bày (Hình 3-6). Mỗi tầng được cô lập với các tầng khác, cho phép phân tách mối quan tâm. Với một kiến trúc ba tầng, bạn được tự do dùng bất kỳ công nghệ nào bạn ưa thích trong mỗi tầng mà không cần tập trung theo kiểu monolithic.

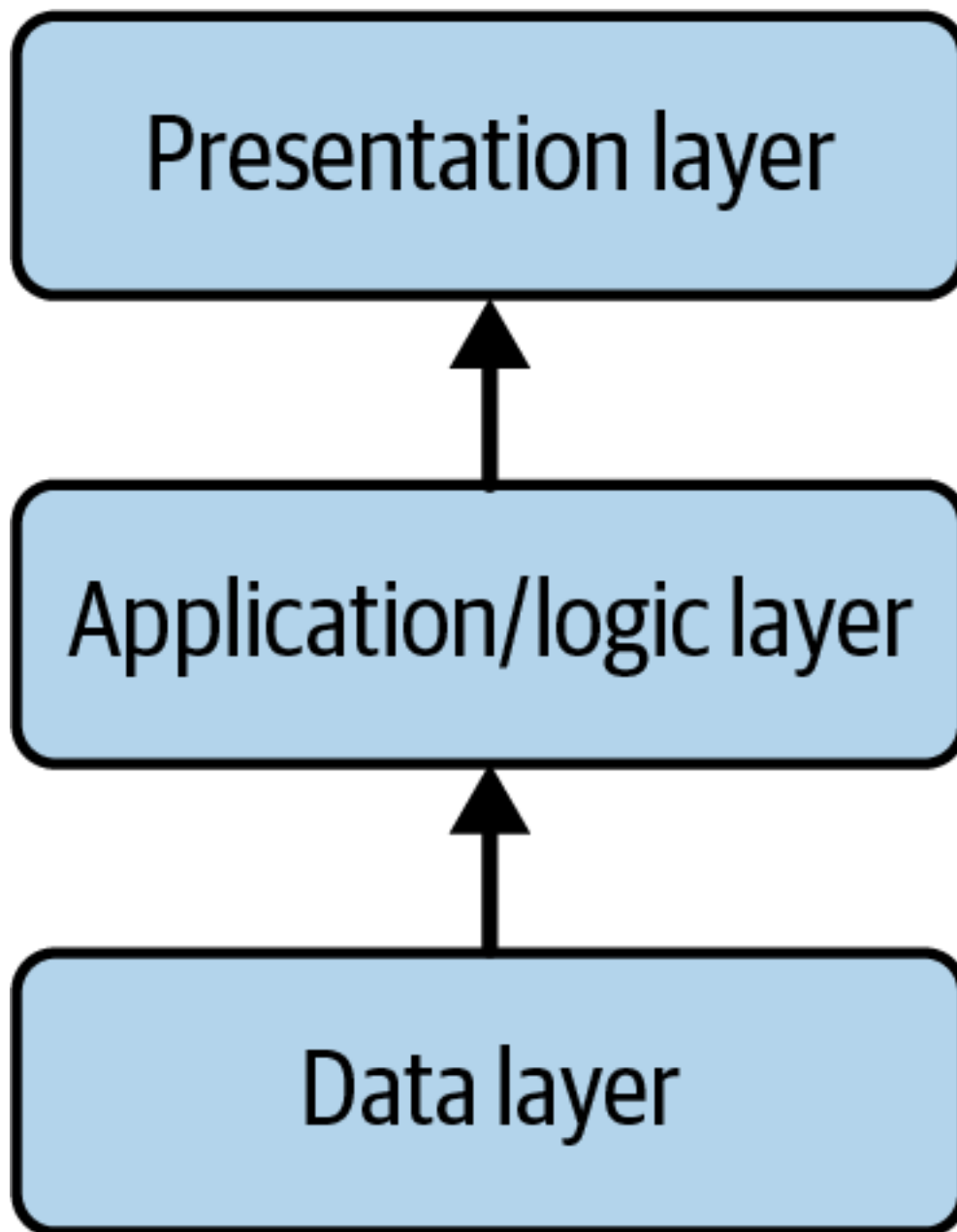


Figure 3-6. A three-tier architecture We've seen many single-tier architectures in production. Single-tier architectures offer simplicity but also severe limitations. Eventually, an organization or application outgrows this arrangement; it works well until it doesn't. For instance, in a single-tier architecture, the data and logic layers share and compete for resources (disk, CPU, and memory) in ways that are simply avoided in a multitier architecture. Resources are spread across various tiers. Data engineers should use tiers to evaluate their layered architecture and the way dependencies are handled. Again, start simple and bake in evolution to additional tiers as your architecture becomes more complex. In a multitier architecture, you need to consider separating your layers and the way resources are shared within layers when working with a distributed system. Distributed systems under the hood power many technologies you'll encounter across the data engineering lifecycle. First, think about whether you want resource contention with your nodes. If not, exercise a shared-nothing architecture : a single node handles each request, meaning other nodes do not share resources such as memory, disk, or CPU with this node or with each other. Data and resources are

isolated to the node. Alternatively, various nodes can handle multiple requests and share resources but at the risk of resource contention. Another consideration is whether nodes should share the same disk and memory accessible by all nodes. This is called a shared disk architecture and is common when you want shared resources if a random node failure occurs.

Hình 3-6. Một kiến trúc ba tầng. Chúng tôi đã thấy nhiều kiến trúc một tầng trong sản xuất. Các kiến trúc một tầng mang lại sự đơn giản nhưng cũng có những giới hạn nghiêm trọng. Cuối cùng, một tổ chức hoặc ứng dụng sẽ vượt quá sự sắp xếp này; nó vận hành tốt cho đến khi không còn tốt nữa. Chẳng hạn, trong một kiến trúc một tầng, các lớp dữ liệu và logic chia sẻ và cạnh tranh tài nguyên (đĩa, CPU, và bộ nhớ) theo những cách đơn giản là được tránh trong một kiến trúc đa tầng. Tài nguyên được trải đều khắp các tầng khác nhau. Kỹ sư dữ liệu nên dùng các tầng để đánh giá kiến trúc phân lớp của họ và cách các phụ thuộc được xử lý. Một lần nữa, hãy bắt đầu đơn giản và tích hợp sẵn khả năng tiến hóa lên thêm các tầng khi kiến trúc của bạn trở nên phức tạp hơn. Trong một kiến trúc đa tầng, bạn cần cân nhắc việc tách các lớp của mình và cách tài nguyên được chia sẻ trong các lớp khi làm việc với một hệ phân tán. Các hệ phân tán dưới lớp vỏ vận hành nhiều công nghệ mà bạn sẽ gặp xuyên suốt vòng đời kỹ thuật dữ liệu. Trước hết, hãy nghĩ xem bạn có muốn tranh chấp tài nguyên với các nút của mình không. Nếu không, hãy áp dụng một kiến trúc shared-nothing: một nút đơn xử lý mỗi yêu cầu, nghĩa là các nút khác không chia sẻ tài nguyên như bộ nhớ, đĩa, hay CPU với nút này hoặc với nhau. Dữ liệu và tài nguyên được cô lập về nút. Một cách khác, các nút khác nhau có thể xử lý nhiều yêu cầu và chia sẻ tài nguyên nhưng với rủi ro tranh chấp tài nguyên. Một điều cần cân nhắc khác là các nút có nên chia sẻ cùng đĩa và bộ nhớ mà tất cả các nút truy cập được hay không. Điều này được gọi là kiến trúc shared disk và phổ biến khi bạn muốn có tài nguyên chia sẻ nếu một sự cố hỏng nút ngẫu nhiên xảy ra.

Monoliths — Monolith

The general notion of a **monolith** includes as much as possible under one roof; in its most extreme version, a monolith consists of a single codebase running on a single machine that provides both the application logic and user interface. Coupling within monoliths can be viewed in two ways: technical coupling and domain coupling. Technical coupling refers to architectural tiers, while domain coupling refers to the way domains are coupled together. A monolith has varying degrees of coupling among technologies and domains. You could have an application with various layers decoupled in a multitier architecture but still share multiple domains. Or, you could have a single-tier architecture **serving** a single domain. The tight coupling of a monolith implies a lack of modularity of its components. Swapping out or upgrading components in a monolith is often an exercise in trading one pain for another. Because of the tightly coupled nature, reusing components across the architecture is difficult or impossible. When evaluating how to improve a monolithic architecture, it's often a game of whack-a-mole: one component is improved, often at the expense of unknown consequences with other areas of the monolith. Data teams will often ignore solving the growing complexity of their monolith, letting it devolve into a big ball of mud .

Khái niệm chung về một monolith bao gồm càng nhiều thứ càng tốt dưới một mái nhà; ở phiên bản cực đoan nhất, một monolith gồm một codebase đơn chạy trên một máy đơn cung cấp cả logic ứng dụng lẫn giao diện người dùng. Sự ghép nối trong các monolith có thể được nhìn theo hai cách: ghép kỹ thuật và ghép miền. Ghép kỹ thuật ám chỉ các tầng kiến trúc, còn ghép miền ám chỉ cách các miền được ghép với nhau. Một monolith có các mức độ ghép nối khác nhau giữa các công nghệ và miền. Bạn có thể có một ứng dụng với các lớp khác nhau được tách rời trong một kiến trúc đa tầng nhưng vẫn chia sẻ nhiều miền.

Hoặc, bạn có thể có một kiến trúc một tầng phục vụ một miền đơn. Bản chất ghép chặt của một monolith hàm ý sự thiếu vắng tính mô-đun của các thành phần của nó. Hoán đổi hoặc nâng cấp các thành phần trong một monolith thường là một bài tập đánh đổi nỗi đau này lấy nỗi đau khác. Vì bản chất ghép chặt, việc tái sử dụng các thành phần khắp kiến trúc là khó hoặc bất khả thi. Khi đánh giá cách cải thiện một kiến trúc monolithic, đó thường là một trò đập chuột chũi: một thành phần được cải thiện, thường với cái giá là những hệ quả không lường trước ở các vùng khác của monolith. Các nhóm dữ liệu thường sẽ phớt lờ việc giải quyết độ phức tạp ngày càng tăng của monolith của họ, để mặc nó thoái hóa thành một cục bùn lớn.

Chapter 4 provides a more extensive discussion comparing monoliths to distributed technologies. We also discuss the distributed monolith , a strange hybrid that emerges when engineers build distributed systems with excessive tight coupling.

Chương 4 cung cấp một thảo luận mở rộng hơn so sánh các monolith với các công nghệ phân tán. Chúng tôi cũng thảo luận về monolith phân tán (distributed monolith), một thể lai kỳ lạ nổi lên khi các kỹ sư xây dựng những hệ phân tán với mức ghép chặt quá mức.

Microservices — Microservices

Compared with the attributes of a monolith—interwoven services, centralization, and tight coupling among services—microservices are the polar opposite. Microservices architecture comprises separate, decentralized, and loosely coupled services. Each service has a specific function and is decoupled from other services operating within its domain. If one service temporarily goes down, it won't affect the ability of other services to continue functioning. A question that comes up often is how to convert your monolith into many microservices (Figure 3-7). This completely depends on how complex your monolith is and how much effort it will be to start extracting services out of it. It's entirely possible that your monolith cannot be broken apart, in which case, you'll want to start creating a new parallel architecture that has the services decoupled in a microservices-friendly manner. We don't suggest an entire refactor but instead break out services. The monolith didn't arrive overnight and is a technology issue as an organizational one. Be sure you get buy-in from stakeholders of the monolith if you plan to break it apart. If you'd like to learn more about breaking apart a monolith, we suggest reading the fantastic, pragmatic guide *Software Architecture: The Hard Parts* by Neal Ford et al. (O'Reilly).

So với các thuộc tính của một monolith — các dịch vụ đan xen, tập trung hóa, và ghép chặt giữa các dịch vụ — microservices là thái cực đối lập. Kiến trúc microservices bao gồm các dịch vụ riêng biệt, phi tập trung và ghép lỏng. Mỗi dịch vụ có một chức năng cụ thể và được tách rời khỏi các dịch vụ khác hoạt động trong miền của nó. Nếu một dịch vụ tạm thời ngừng hoạt động, nó sẽ không ảnh hưởng đến khả năng tiếp tục vận hành của các dịch vụ khác. Một câu hỏi thường xuyên xuất hiện là làm thế nào để chuyển đổi monolith của bạn thành nhiều microservices (Hình 3-7). Điều này hoàn toàn phụ thuộc vào độ phức tạp của monolith của bạn và lượng công sức cần bỏ ra để bắt đầu trích xuất các dịch vụ ra khỏi nó. Hoàn toàn có khả năng monolith của bạn không thể bị tách rời, trong trường hợp đó, bạn sẽ muốn bắt đầu tạo một kiến trúc song song mới có các dịch vụ được tách rời theo cách thân thiện với microservices. Chúng tôi không đề xuất một cuộc tái cấu trúc toàn diện mà thay vào đó tách dần các dịch vụ ra. Monolith không xuất hiện chỉ sau một đêm và là một vấn đề công nghệ cũng như tổ chức. Hãy chắc chắn bạn có được sự đồng thuận từ các bên liên quan của monolith nếu bạn dự định tách rời nó. Nếu bạn muốn tìm hiểu thêm về việc tách rời một monolith, chúng tôi đề xuất đọc cuốn hướng dẫn thực dụng tuyệt vời *Software Architecture: The Hard Parts* của Neal Ford và cộng sự (O'Reilly).

Considerations for data architecture — Những điều cần cân nhắc cho kiến trúc dữ liệu

As we mentioned at the start of this section, the concepts of tight versus loose coupling stem from software development, with some of these concepts dating back over 20 years. Though architectural practices in data are now adopting those from software development, it's still common to see very monolithic, tightly coupled data architectures. Some of this is due to the nature of existing data technologies and the way they integrate.

Như chúng tôi đã đề cập ở đầu mục này, các khái niệm ghép chặt so với ghép lỏng bắt nguồn từ phát triển phần mềm, với một số khái niệm trong số đó có từ hơn 20 năm trước. Mặc dù các thực hành kiến trúc trong dữ liệu giờ đang áp dụng những thực hành từ phát triển phần mềm, vẫn còn phổ biến việc thấy những kiến trúc dữ liệu rất monolithic, ghép chặt. Một phần của điều này là do bản chất của các công nghệ dữ liệu hiện hữu và cách chúng tích hợp.

For example, data pipelines might consume data from many sources ingested into a central data warehouse. The central data warehouse is inherently monolithic. A move toward a microservices equivalent with a data warehouse is to decouple the workflow with domain-specific data pipelines connecting to corresponding domain-specific data warehouses. For example, the sales **data pipeline** connects to the sales-specific data warehouse, and the inventory and product domains follow a similar pattern.

Ví dụ, các đường ống dữ liệu (data pipeline) có thể tiêu thụ dữ liệu từ nhiều nguồn được thu nạp vào một kho dữ liệu trung tâm. Kho dữ liệu trung tâm về bản chất là monolithic. Một bước tiến tới phiên bản tương đương kiểu microservices với một kho dữ liệu là tách rời luồng công việc bằng các đường ống dữ liệu đặc thù theo miền kết nối tới các kho dữ liệu đặc thù theo miền tương ứng. Ví dụ, đường ống dữ liệu bán hàng kết nối tới kho dữ liệu chuyên về bán hàng, và các miền kho hàng cùng sản phẩm theo một mẫu hình tương tự.

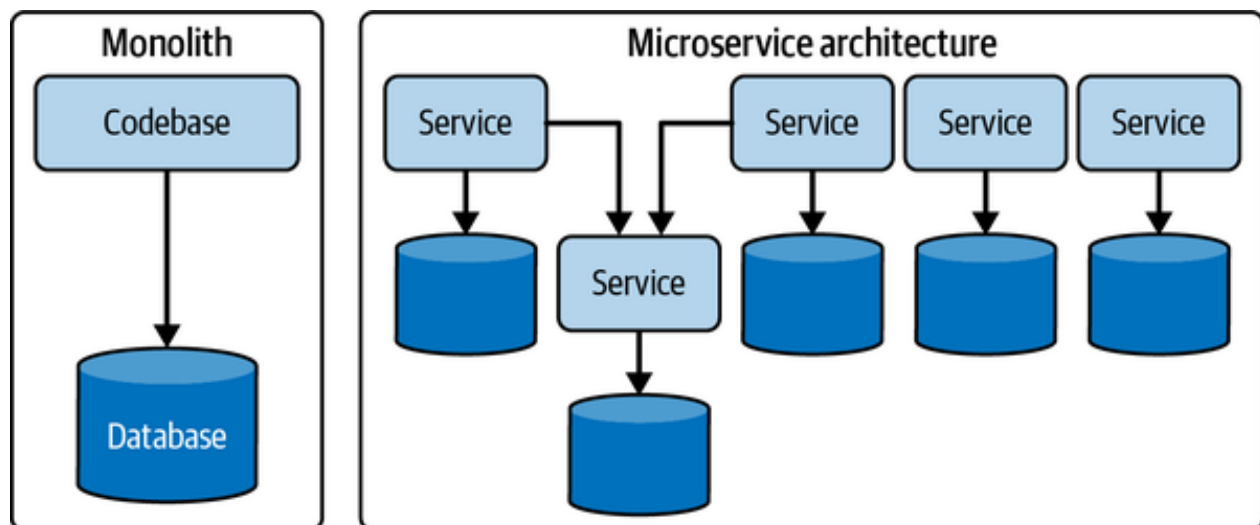


Figure 3-7. An extremely monolithic architecture runs all functionality inside a single codebase, potentially colocating a database on the same host server. Rather than dogmatically preach microservices over monoliths (among other arguments), we suggest you pragmatically use loose coupling as an ideal, while recognizing the state and limitations of the data technologies you're using within your data architecture. Incorporate reversible technology choices that allow for modularity.

and loose coupling whenever possible. As you can see in Figure 3-7, you separate the components of your architecture into different layers of concern in a vertical fashion. While a multitier architecture solves the technical challenges of decoupling shared resources, it does not address the complexity of sharing domains. Along the lines of single versus multitiered architecture, you should also consider how you separate the domains of your data architecture. For example, your analyst team might rely on data from sales and inventory. The sales and inventory domains are different and should be viewed as separate.

Hình 3-7. Một kiến trúc cực kỳ monolithic chạy toàn bộ chức năng bên trong một codebase đơn, có khả năng đặt một cơ sở dữ liệu trên cùng máy chủ host. Thay vì giáo điều rao giảng microservices hơn monolith (cùng nhiều lập luận khác), chúng tôi đề xuất bạn thực dụng dùng ghép lỏng như một lý tưởng, đồng thời nhận thức được trạng thái và những giới hạn của các công nghệ dữ liệu mà bạn đang dùng trong kiến trúc dữ liệu của mình. Hãy đưa vào những lựa chọn công nghệ có thể đảo ngược cho phép tính mô-đun và ghép lỏng bất cứ khi nào có thể. Như bạn thấy trong Hình 3-7, bạn tách các thành phần của kiến trúc thành các lớp quan tâm khác nhau theo chiều dọc. Trong khi một kiến trúc đa tầng giải quyết các thách thức kỹ thuật của việc tách rời các tài nguyên chia sẻ, nó không giải quyết độ phức tạp của việc chia sẻ các miền. Theo hướng kiến trúc một tầng so với đa tầng, bạn cũng nên cân nhắc cách bạn tách các miền của kiến trúc dữ liệu của mình. Ví dụ, đội phân tích của bạn có thể dựa vào dữ liệu từ bán hàng và kho hàng. Các miền bán hàng và kho hàng là khác nhau và nên được xem là riêng biệt.

One approach to this problem is centralization: a single team is responsible for gathering data from all domains and reconciling it for consumption across the organization. (This is a common approach in traditional data warehousing.) Another approach is the **data mesh**. With the data mesh, each software team is responsible for preparing its data for consumption across the rest of the organization. We'll say more about the data mesh later in this chapter. Our advice: monoliths aren't necessarily bad, and it might make sense to start with one under certain conditions. Sometimes you need to move fast, and it's much simpler to start with a monolith. Just be prepared to break it into smaller pieces eventually; don't get too comfortable.

Một cách tiếp cận cho vấn đề này là tập trung hóa: một đội đơn chịu trách nhiệm thu thập dữ liệu từ tất cả các miền và đối soát nó để tiêu thụ khắp tổ chức. (Đây là một cách tiếp cận phổ biến trong kho dữ liệu truyền thống.) Một cách tiếp cận khác là data mesh. Với data mesh, mỗi đội phần mềm chịu trách nhiệm chuẩn bị dữ liệu của mình để tiêu thụ khắp phần còn lại của tổ chức. Chúng tôi sẽ nói thêm về data mesh ở phần sau của chương này. Lời khuyên của chúng tôi: monolith không nhất thiết là xấu, và có thể hợp lý khi bắt đầu với một monolith trong một số điều kiện nhất định. Đôi khi bạn cần di chuyển nhanh, và bắt đầu với một monolith đơn giản hơn nhiều. Chỉ cần sẵn sàng tách nó thành những mảnh nhỏ hơn cuối cùng; đừng quá thoải mái.

User Access: Single Versus Multitenant — Quyền truy cập người dùng: Đơn thuê bao so với đa thuê bao

As a data engineer, you have to make decisions about sharing systems across multiple teams, organizations, and customers. In some sense, all cloud services are **multitenant**, although this multitenancy occurs at various grains. For example, a cloud compute instance is usually on a shared server, but the VM itself provides some degree of isolation. Object storage is a multitenant system, but cloud vendors guarantee security and isolation so long as customers configure their permissions correctly. Engineers frequently need to make decisions about multitenancy at a much smaller

scale. For example, do multiple departments in a large company share the same data warehouse? Does the organization share data for multiple large customers within the same table? We have two factors to consider in multitenancy: performance and security. With multiple large tenants within a cloud system, will the system support consistent performance for all tenants, or will there be a noisy neighbor problem? (That is, will high usage from one tenant degrade performance for other tenants?) Regarding security, data from different tenants must be properly isolated. When a company has multiple external customer tenants, these tenants should not be aware of one another, and engineers must prevent data leakage. Strategies for data isolation vary by system. For instance, it is often perfectly acceptable to use multitenant tables and isolate data through views. However, you must make certain that these views cannot leak data. Read vendor or project documentation to understand appropriate strategies and risks.

Là một kỹ sư dữ liệu, bạn phải đưa ra những quyết định về việc chia sẻ các hệ thống qua nhiều đội, tổ chức và khách hàng. Theo một nghĩa nào đó, tất cả các dịch vụ đám mây đều là đa thuê bao (multitenant), mặc dù tính đa thuê bao này diễn ra ở nhiều độ hạt khác nhau. Ví dụ, một thực thể tính toán đám mây thường nằm trên một máy chủ chia sẻ, nhưng bản thân VM cung cấp một mức cô lập nhất định. Lưu trữ đối tượng là một hệ thống đa thuê bao, nhưng các nhà cung cấp đám mây đảm bảo an ninh và cô lập miễn là khách hàng cấu hình quyền hạn của họ một cách đúng đắn. Các kỹ sư thường xuyên cần đưa ra những quyết định về tính đa thuê bao ở một quy mô nhỏ hơn nhiều. Ví dụ, nhiều phòng ban trong một công ty lớn có chia sẻ cùng một kho dữ liệu không? Tổ chức có chia sẻ dữ liệu cho nhiều khách hàng lớn trong cùng một bảng không? Chúng ta có hai yếu tố cần cân nhắc trong tính đa thuê bao: hiệu năng và an ninh. Với nhiều thuê bao lớn trong một hệ thống đám mây, liệu hệ thống có hỗ trợ hiệu năng nhất quán cho tất cả các thuê bao, hay sẽ có vấn đề noisy neighbor (hàng xóm ồn ào)? (Tức là, liệu mức sử dụng cao từ một thuê bao có làm suy giảm hiệu năng cho các thuê bao khác không?) Về an ninh, dữ liệu từ các thuê bao khác nhau phải được cô lập đúng cách. Khi một công ty có nhiều thuê bao khách hàng bên ngoài, những thuê bao này không nên biết về nhau, và các kỹ sư phải ngăn chặn rò rỉ dữ liệu. Các chiến lược cô lập dữ liệu khác nhau tùy theo hệ thống. Chẳng hạn, thường là hoàn toàn chấp nhận được khi dùng các bảng đa thuê bao và cô lập dữ liệu qua các view. Tuy nhiên, bạn phải chắc chắn rằng những view này không thể làm rò rỉ dữ liệu. Hãy đọc tài liệu của nhà cung cấp hoặc dự án để hiểu các chiến lược và rủi ro phù hợp.

Event-Driven Architecture — Kiến trúc hướng sự kiện

Your business is rarely static. Things often happen in your business, such as getting a new customer, a new order from a customer, or an order for a product or service. These are all examples of events that are broadly defined as something that happened, typically a change in the state of something. For example, a new order might be created by a customer, or a customer might later make an update to this order. An **event**-driven workflow (Figure 3-8) encompasses the ability to create, update, and asynchronously move events across various parts of the data engineering lifecycle. This workflow boils down to three main areas: event production, routing, and consumption. An event must be produced and routed to something that consumes it without tightly coupled dependencies among the producer, event router, and consumer.

Doanh nghiệp của bạn hiếm khi tĩnh. Mọi thứ thường xảy ra trong doanh nghiệp của bạn, chẳng hạn có một khách hàng mới, một đơn hàng mới từ một khách hàng, hoặc một đơn đặt cho một sản phẩm hoặc dịch vụ. Đây đều là những ví dụ về các sự kiện, được định nghĩa rộng là điều gì đó đã xảy ra, thường là một thay đổi trong trạng thái của thứ gì đó.

Ví dụ, một đơn hàng mới có thể được tạo bởi một khách hàng, hoặc một khách hàng có thể sau đó cập nhật đơn hàng này. Một luồng công việc hướng sự kiện (Hình 3-8) bao trùm khả năng tạo, cập nhật và di chuyển bất đồng bộ các sự kiện qua các phần khác nhau của vòng đời kỹ thuật dữ liệu. Luồng công việc này quy về ba mảng chính: sản xuất sự kiện, định tuyến, và tiêu thụ. Một sự kiện phải được sản xuất và định tuyến tới thứ gì đó tiêu thụ nó mà không có các phụ thuộc ghép chặt giữa bên sản xuất, bộ định tuyến sự kiện, và bên tiêu thụ.

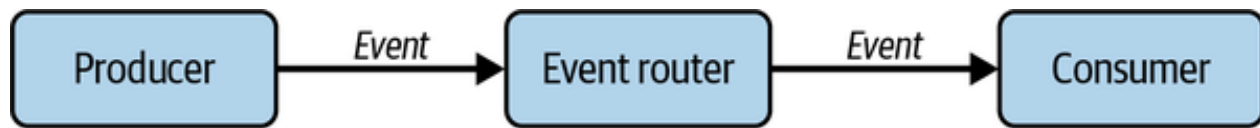


Figure 3-8. In an event-driven workflow, an event is produced, routed, and then consumed. An **event-driven architecture** (Figure 3-9) embraces the event-driven workflow and uses this to communicate across various services. The advantage of an event-driven architecture is that it distributes the state of an event across multiple services. This is helpful if a service goes offline, a node fails in a distributed system, or you'd like multiple consumers or services to access the same events. Anytime you have loosely coupled services, this is a candidate for event-driven architecture. Many of the examples we describe later in this chapter incorporate some form of event-driven architecture.

Hình 3-8. Trong một luồng công việc hướng sự kiện, một sự kiện được sản xuất, định tuyến, rồi được tiêu thụ. Một kiến trúc hướng sự kiện (Hình 3-9) đón nhận luồng công việc hướng sự kiện và dùng điều này để giao tiếp qua các dịch vụ khác nhau. Lợi thế của một kiến trúc hướng sự kiện là nó phân phối trạng thái của một sự kiện qua nhiều dịch vụ. Điều này hữu ích nếu một dịch vụ ngoại tuyến, một nút hỏng trong một hệ phân tán, hoặc bạn muốn nhiều bên tiêu thụ hoặc dịch vụ truy cập cùng các sự kiện. Bất cứ khi nào bạn có các dịch vụ ghép lỏng, đây là một ứng viên cho kiến trúc hướng sự kiện. Nhiều ví dụ mà chúng tôi mô tả ở phần sau của chương này tích hợp một dạng nào đó của kiến trúc hướng sự kiện.

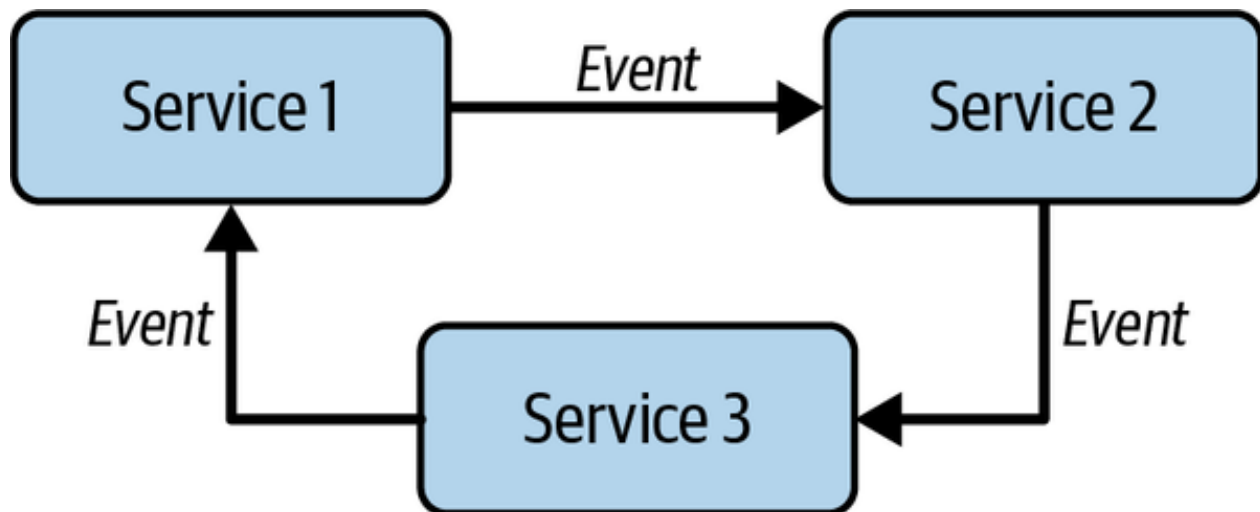


Figure 3-9. In an event-driven architecture, events are passed between loosely coupled services. You'll learn more about event-driven streaming and messaging systems in Chapter 5.

Hình 3-9. Trong một kiến trúc hướng sự kiện, các sự kiện được truyền giữa các dịch vụ ghép lỏng. Bạn sẽ tìm hiểu thêm về các hệ thống truyền theo luồng và tin nhắn hướng sự kiện trong Chương 5.

Brownfield Versus Greenfield Projects — Dự án brownfield so với greenfield

Before you design your data architecture project, you need to know whether you're starting with a clean slate or redesigning an existing architecture. Each type of project requires assessing trade-offs, albeit with different considerations and approaches. Projects roughly fall into two buckets: **brownfield** and **greenfield**.

Trước khi thiết kế dự án kiến trúc dữ liệu của mình, bạn cần biết liệu bạn đang bắt đầu với một tờ giấy trắng hay đang thiết kế lại một kiến trúc hiện hữu. Mỗi loại dự án đòi hỏi đánh giá các đánh đổi, mặc dù với những điều cần cân nhắc và cách tiếp cận khác nhau. Các dự án rơi đại khái vào hai nhóm: brownfield và greenfield.

Brownfield projects — Dự án brownfield

Brownfield projects often involve refactoring and reorganizing an existing architecture and are constrained by the choices of the present and past. Because a key part of architecture is change management, you must figure out a way around these limitations and design a path forward to achieve your new business and technical objectives. Brownfield projects require a thorough understanding of the legacy architecture and the interplay of various old and new technologies. All too often, it's easy to criticize a prior team's work and decisions, but it is far better to dig deep, ask questions, and understand why decisions were made. Empathy and context go a long way in helping you diagnose problems with the existing architecture, identify opportunities, and recognize pitfalls.

Các dự án brownfield thường liên quan đến việc tái cấu trúc và tái tổ chức một kiến trúc hiện hữu và bị ràng buộc bởi những lựa chọn của hiện tại và quá khứ. Vì một phần then chốt của kiến trúc là quản lý thay đổi, bạn phải tìm ra cách vòng quanh những giới hạn này và thiết kế một con đường tiến lên để đạt được các mục tiêu kinh doanh và kỹ thuật mới của mình. Các dự án brownfield đòi hỏi hiểu biết thấu đáo về kiến trúc cũ (legacy) và sự tương tác của nhiều công nghệ cũ và mới. Quá thường xuyên, người ta dễ dàng chỉ trích công việc và quyết định của một đội trước, nhưng tốt hơn nhiều là đào sâu, đặt câu hỏi, và hiểu vì sao các quyết định được đưa ra. Sự đồng cảm và bối cảnh đi một chặng đường dài trong việc giúp bạn chẩn đoán các vấn đề với kiến trúc hiện hữu, nhận diện các cơ hội, và nhận ra các cạm bẫy.

You'll need to introduce your new architecture and technologies and deprecate the old stuff at some point. Let's look at a couple of popular approaches. Many teams jump headfirst into an all-at-once or big-bang overhaul of the old architecture, often figuring out deprecation as they go. Though popular, we don't advise this approach because of the associated risks and lack of a plan. This path often leads to disaster, with many irreversible and costly decisions. Your job is to make reversible, high-ROI decisions. A popular alternative to a direct rewrite is the strangler pattern: new systems slowly and incrementally replace a legacy architecture's components. Eventually, the legacy architecture is completely replaced. 7 The attraction to the strangler pattern is its targeted and surgical approach of deprecating one piece of a system at a time. This allows for flexible and reversible decisions while assessing the impact of the deprecation on dependent systems. It's important to note that deprecation might be "ivory tower" advice and not practical or achievable. Eradicating legacy technology or architecture might be impossible if you're at a large organization. Someone, somewhere, is using these legacy components. As someone once said, "Legacy is a condescending way to describe something that makes money." If you can deprecate, understand there are numerous ways to deprecate your old architecture. It is critical to demonstrate value on

the new platform by gradually increasing its maturity to show evidence of success and then follow an exit plan to shut down old systems.

Bạn sẽ cần giới thiệu kiến trúc và công nghệ mới của mình và loại bỏ những thứ cũ ở một thời điểm nào đó. Hãy xem một vài cách tiếp cận phổ biến. Nhiều đội nhảy bổ vào một cuộc đại tu toàn bộ-cùng-lúc hoặc kiểu vụ nổ lớn (big-bang) đối với kiến trúc cũ, thường vừa làm vừa tính chuyện loại bỏ. Dù phổ biến, chúng tôi không khuyên dùng cách tiếp cận này vì những rủi ro liên quan và sự thiếu một kế hoạch. Con đường này thường dẫn đến thảm họa, với nhiều quyết định bất khả đảo ngược và tổn kém. Công việc của bạn là đưa ra những quyết định có thể đảo ngược, ROI cao. Một lựa chọn thay thế phổ biến cho việc viết lại trực tiếp là mẫu hình strangler: các hệ thống mới chậm rãi và từng bước thay thế các thành phần của một kiến trúc cũ. Cuối cùng, kiến trúc cũ được thay thế hoàn toàn. Điểm hấp dẫn của mẫu hình strangler là cách tiếp cận có mục tiêu và như phẫu thuật của nó là loại bỏ từng mảnh một của một hệ thống tại một thời điểm. Điều này cho phép những quyết định linh hoạt và có thể đảo ngược trong khi đánh giá tác động của việc loại bỏ lên các hệ thống phụ thuộc. Điều quan trọng cần lưu ý là việc loại bỏ có thể là lời khuyên kiểu “tháp ngà” và không thực tế hoặc bất khả thi. Việc xóa sổ công nghệ hoặc kiến trúc cũ có thể là bất khả thi nếu bạn đang ở một tổ chức lớn. Ai đó, ở đâu đó, đang dùng những thành phần cũ này. Như có người từng nói, “Legacy là một cách kể cả để mô tả thứ gì đó đang kiếm ra tiền.” Nếu bạn có thể loại bỏ, hãy hiểu rằng có vô số cách để loại bỏ kiến trúc cũ của bạn. Điều then chốt là chứng minh giá trị trên nền tảng mới bằng cách dần dần tăng độ trưởng thành của nó để cho thấy bằng chứng về thành công rồi theo một kế hoạch rút lui để tắt các hệ thống cũ.

Greenfield projects — Dự án greenfield

On the opposite end of the spectrum, a greenfield project allows you to pioneer a fresh start, unconstrained by the history or legacy of a prior architecture. Greenfield projects tend to be easier than brownfield projects, and many data architects and engineers find them more fun! You have the opportunity to try the newest and coolest tools and architectural patterns. What could be more exciting?

Ở đầu đối lập của phổ, một dự án greenfield cho phép bạn tiên phong một khởi đầu mới mẻ, không bị ràng buộc bởi lịch sử hay di sản của một kiến trúc trước đó. Các dự án greenfield có xu hướng dễ hơn các dự án brownfield, và nhiều kiến trúc sư và kỹ sư dữ liệu thấy chúng vui hơn! Bạn có cơ hội thử những công cụ và mẫu hình kiến trúc mới nhất và ngẫu nhiên nhất. Còn gì thú vị hơn?

You should watch out for some things before getting too carried away. We see teams get overly exuberant with shiny object syndrome. They feel compelled to reach for the latest and greatest technology fad without understanding how it will impact the value of the project. There’s also a temptation to do resume-driven development, stacking up impressive new technologies without prioritizing the project’s ultimate goals. Always prioritize requirements over building something cool. Whether you’re working on a brownfield or greenfield project, always focus on the tenets of “good” data architecture. Assess trade-offs, make flexible and reversible decisions, and strive for positive ROI. Now, we’ll look at examples and types of architectures—some established for decades (the data warehouse), some brand-new (the **data lakehouse**), and some that quickly came and went but still influence current architecture patterns (**Lambda** architecture).

Bạn nên cẩn thận với một số điều trước khi quá đà. Chúng tôi thấy các đội trở nên hào hứng thái quá với hội chứng vật-thể-lấp-lánh (shiny object syndrome). Họ cảm thấy bị

thời thúc phải với tới một công nghệ mới nhất và tuyệt nhất mà không hiểu nó sẽ tác động đến giá trị của dự án như thế nào. Cũng có một cám dỗ thực hiện phát triển kiểu lấy-thành-tích-cho-CV (resume-driven development), chất đồng những công nghệ mới ấn tượng mà không ưu tiên các mục tiêu rõ ràng của dự án. Hãy luôn ưu tiên các yêu cầu hơn là xây dựng thứ gì đó ngẫu nhiên. Dù bạn đang làm một dự án brownfield hay greenfield, hãy luôn tập trung vào các nguyên lý của kiến trúc dữ liệu “tốt”. Đánh giá các đánh đổi, đưa ra những quyết định linh hoạt và có thể đảo ngược, và phấn đấu cho ROI dương. Giờ chúng ta sẽ xem các ví dụ và loại kiến trúc — một số đã được thiết lập từ nhiều thập kỷ (kho dữ liệu), một số hoàn toàn mới (data lakehouse), và một số nhanh chóng đến rồi đi nhưng vẫn ảnh hưởng đến các mẫu hình kiến trúc hiện thời (kiến trúc Lambda).

Examples and Types of Data Architecture — Các ví dụ và loại kiến trúc dữ liệu

Because data architecture is an abstract discipline, it helps to reason by example. In this section, we outline prominent examples and types of data architecture that are popular today. Though this set of examples is by no means exhaustive, the intention is to expose you to some of the most common data architecture patterns and to get you thinking about the requisite flexibility and trade-off analysis needed when designing a good architecture for your use case.

Vì kiến trúc dữ liệu là một bộ môn trừu tượng, sẽ hữu ích nếu lập luận qua ví dụ. Trong mục này, chúng tôi phác họa các ví dụ và loại kiến trúc dữ liệu nổi bật đang phổ biến ngày nay. Mặc dù tập ví dụ này hoàn toàn không phải đầy đủ, ý định là phơi bày cho bạn một số mẫu hình kiến trúc dữ liệu phổ biến nhất và khiến bạn suy nghĩ về tính linh hoạt cần thiết cùng phân tích đánh đổi cần thiết khi thiết kế một kiến trúc tốt cho ca sử dụng của mình.

Data Warehouse — Kho dữ liệu

A data warehouse is a central data hub used for reporting and analysis. Data in a data warehouse is typically highly formatted and structured for analytics use cases. It's among the oldest and most well-established data architectures. In 1990, Bill **Inmon** originated the notion of the data warehouse, which he described as “a subject-oriented, integrated, nonvolatile, and time-variant collection of data in support of management's decisions.” Though 9 technical aspects of the data warehouse have evolved significantly, we feel this original definition still holds its weight today. In the past, data warehouses were widely used at enterprises with significant budgets (often in the millions of dollars) to acquire data systems and pay internal teams to provide ongoing support to maintain the data warehouse. This was expensive and labor-intensive. Since then, the scalable, pay-as-you-go model has made cloud data warehouses accessible even to tiny companies. Because a third-party provider manages the data warehouse infrastructure, companies can do a lot more with fewer people, even as the complexity of their data grows. It's worth noting two types of data warehouse architecture: organizational and technical. The organizational data warehouse architecture organizes data associated with certain business team structures and processes. The technical data warehouse architecture reflects the technical nature of the data warehouse, such as MPP. A company can have a data warehouse without an MPP system or run an MPP system that is not organized as a data warehouse. However, the technical and organizational architectures have existed in a virtuous cycle and are frequently identified with each other. The organizational data warehouse architecture has two main characteristics: Separates analytics processes (OLAP) from production databases (online

transaction processing) This separation is critical as businesses grow. Moving data into a separate physical system directs load away from production systems and improves analytics performance.

Một kho dữ liệu (data warehouse) là một trung tâm dữ liệu tập trung được dùng cho báo cáo và phân tích. Dữ liệu trong một kho dữ liệu thường được định dạng cao và cấu trúc hóa cho các ca sử dụng phân tích. Nó nằm trong số những kiến trúc dữ liệu cổ xưa và được thiết lập vững chắc nhất. Năm 1990, Bill Inmon khởi xướng khái niệm kho dữ liệu, mà ông mô tả là “một tập hợp dữ liệu định hướng theo chủ đề, tích hợp, không biến động, và biến thiên theo thời gian nhằm hỗ trợ các quyết định của ban quản lý.” Mặc dù các khía cạnh kỹ thuật của kho dữ liệu đã tiến hóa đáng kể, chúng tôi cảm thấy định nghĩa gốc này vẫn giữ được giá trị của nó ngày nay. Trong quá khứ, các kho dữ liệu được dùng rộng rãi tại các doanh nghiệp có ngân sách lớn (thường lên đến hàng triệu đô-la) để mua các hệ thống dữ liệu và trả cho các đội nội bộ cung cấp hỗ trợ liên tục để duy trì kho dữ liệu. Điều này tốn kém và thâm dụng lao động. Kể từ đó, mô hình trả-theo-mức-dùng, có khả năng mở rộng đã khiến các kho dữ liệu đám mây trở nên dễ tiếp cận ngay cả với những công ty rất nhỏ. Vì một nhà cung cấp bên thứ ba quản lý hạ tầng kho dữ liệu, các công ty có thể làm được nhiều hơn rất nhiều với ít người hơn, ngay cả khi độ phức tạp của dữ liệu của họ tăng lên. Đáng lưu ý là hai loại kiến trúc kho dữ liệu: tổ chức và kỹ thuật. Kiến trúc kho dữ liệu tổ chức tổ chức dữ liệu gắn với các cấu trúc và quy trình của một số đội kinh doanh nhất định. Kiến trúc kho dữ liệu kỹ thuật phản ánh bản chất kỹ thuật của kho dữ liệu, chẳng hạn MPP. Một công ty có thể có một kho dữ liệu mà không có một hệ thống MPP hoặc vận hành một hệ thống MPP không được tổ chức như một kho dữ liệu. Tuy nhiên, các kiến trúc kỹ thuật và tổ chức đã tồn tại trong một vòng tuần hoàn tích cực và thường được đồng nhất với nhau. Kiến trúc kho dữ liệu tổ chức có hai đặc điểm chính: Tách các tiến trình phân tích (OLAP) khỏi các cơ sở dữ liệu sản xuất (xử lý giao dịch trực tuyến). Sự tách biệt này là then chốt khi các doanh nghiệp phát triển. Việc chuyển dữ liệu vào một hệ thống vật lý riêng biệt hướng tải ra khỏi các hệ thống sản xuất và cải thiện hiệu năng phân tích.

Centralizes and organizes data Traditionally, a data warehouse pulls data from application systems by using **ETL**. The extract phase pulls data from source systems. The **transformation** phase cleans and standardizes data, organizing and imposing business logic in a highly modeled form. (Chapter 8 covers transformations and data models.) The load phase pushes data into the data warehouse target database system. Data is loaded into multiple data marts that serve the analytical needs for specific lines or business and departments. Figure 3-10 shows the general workflow. The data warehouse and ETL go hand in hand with specific business structures, including DBA and ETL developer teams that implement the direction of business leaders to ensure that data for reporting and analytics corresponds to business processes. An ETL system is not a prerequisite for a data warehouse, as you will learn when we discuss another load and transformation pattern, **ELT**.

Tập trung hóa và tổ chức dữ liệu. Theo truyền thống, một kho dữ liệu kéo dữ liệu từ các hệ thống ứng dụng bằng cách dùng ETL. Pha trích xuất (extract) kéo dữ liệu từ các hệ thống nguồn. Pha biến đổi (transformation) làm sạch và chuẩn hóa dữ liệu, tổ chức và áp đặt logic nghiệp vụ ở một dạng được mô hình hóa cao. (Chương 8 đề cập đến các biến đổi và mô hình dữ liệu.) Pha tải (load) đẩy dữ liệu vào hệ thống cơ sở dữ liệu đích của kho dữ liệu. Dữ liệu được tải vào nhiều data mart phục vụ các nhu cầu phân tích cho các mảng kinh doanh và phòng ban cụ thể. Hình 3-10 cho thấy luồng công việc chung. Kho dữ liệu và ETL đi đôi với nhau cùng các cấu trúc kinh doanh cụ thể, bao gồm các đội DBA và lập trình viên ETL hiện thực hóa định hướng của các lãnh đạo kinh doanh để đảm bảo rằng dữ liệu cho báo cáo và phân tích tương ứng với các quy trình kinh doanh. Một hệ thống ETL không phải là điều kiện tiên quyết cho một kho dữ liệu, như bạn sẽ học khi chúng ta thảo luận một mẫu hình tải và biến đổi khác, ELT.

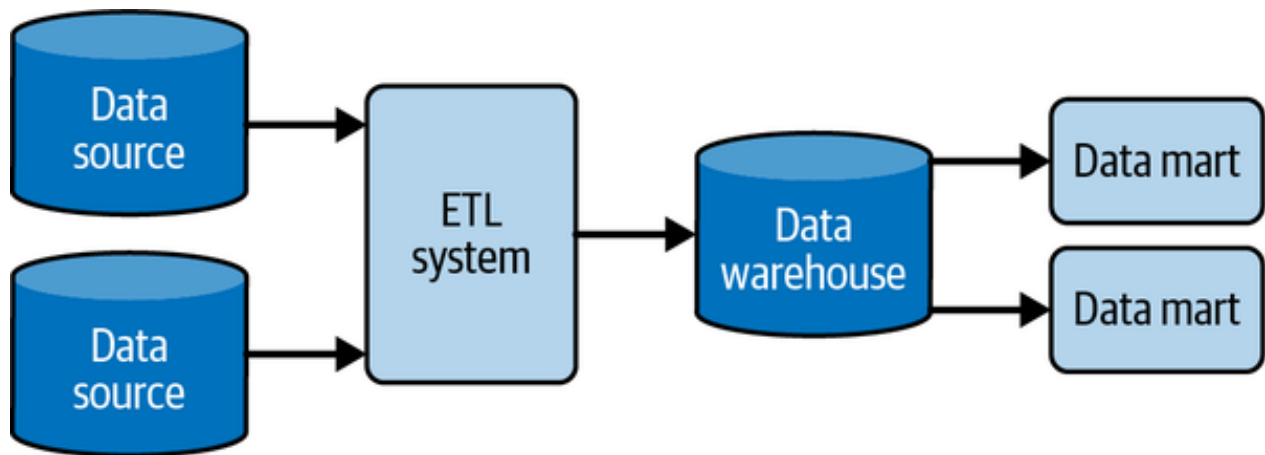


Figure 3-10. Basic data warehouse with ETL

Hình 3-10. Kho dữ liệu cơ bản với ETL

Regarding the technical data warehouse architecture, the first MPP systems in the late 1970s became popular in the 1980s. MPPs support essentially the same SQL semantics used in relational application databases. Still, they are optimized to scan massive amounts of data in parallel and thus allow high-performance aggregation and statistical calculations. In recent years, MPP systems have increasingly shifted from a row-based to a columnar architecture to facilitate even larger data and queries, especially in cloud data warehouses. MPPs are indispensable for running performant queries for large enterprises as data and reporting needs grow. One variation on ETL is ELT. With the ELT data warehouse architecture, data gets moved more or less directly from production systems into a staging area in the data warehouse. Staging in this setting indicates that the data is in a raw form. Rather than using an external system, transformations are handled directly in the data warehouse. The intention is to take advantage of the massive computational power of cloud data warehouses and data processing tools. Data is processed in batches, and transformed output is written into tables and views for analytics. Figure 3-11 shows the general process. ELT is also popular in a streaming arrangement, as events are streamed from a CDC process, stored in a staging area, and then subsequently transformed within the data warehouse.

Về kiến trúc kho dữ liệu kỹ thuật, các hệ thống MPP đầu tiên vào cuối thập niên 1970 trở nên phổ biến vào thập niên 1980. Các MPP hỗ trợ về cơ bản cùng ngữ nghĩa SQL được dùng trong các cơ sở dữ liệu ứng dụng quan hệ. Tuy nhiên, chúng được tối ưu để quét những lượng dữ liệu khổng lồ một cách song song và do đó cho phép tổng hợp và tính toán thống kê hiệu năng cao. Trong những năm gần đây, các hệ thống MPP đã ngày càng chuyển từ kiến trúc dựa trên hàng sang kiến trúc theo cột để tạo điều kiện cho dữ liệu và truy vấn còn lớn hơn nữa, đặc biệt trong các kho dữ liệu đám mây. Các MPP là không thể thiếu để chạy các truy vấn hiệu năng cho các doanh nghiệp lớn khi nhu cầu dữ liệu và báo cáo tăng lên. Một biến thể của ETL là ELT. Với kiến trúc kho dữ liệu ELT, dữ liệu được chuyển ít nhiều trực tiếp từ các hệ thống sản xuất vào một vùng tạm (staging area) trong kho dữ liệu. Tạm (staging) trong bối cảnh này cho thấy dữ liệu ở dạng thô. Thay vì dùng một hệ thống bên ngoài, các biến đổi được xử lý trực tiếp trong kho dữ liệu. Ý định là tận dụng sức mạnh tính toán khổng lồ của các kho dữ liệu đám mây và các công cụ xử lý dữ liệu. Dữ liệu được xử lý theo lô, và kết quả đã biến đổi được ghi vào các bảng và view để phân tích. Hình 3-11 cho thấy quy trình chung. ELT cũng phổ biến trong một sắp xếp theo luồng, khi các sự kiện được truyền theo luồng từ một tiến trình CDC, lưu trong một vùng tạm, rồi sau đó được biến đổi bên trong kho dữ liệu.

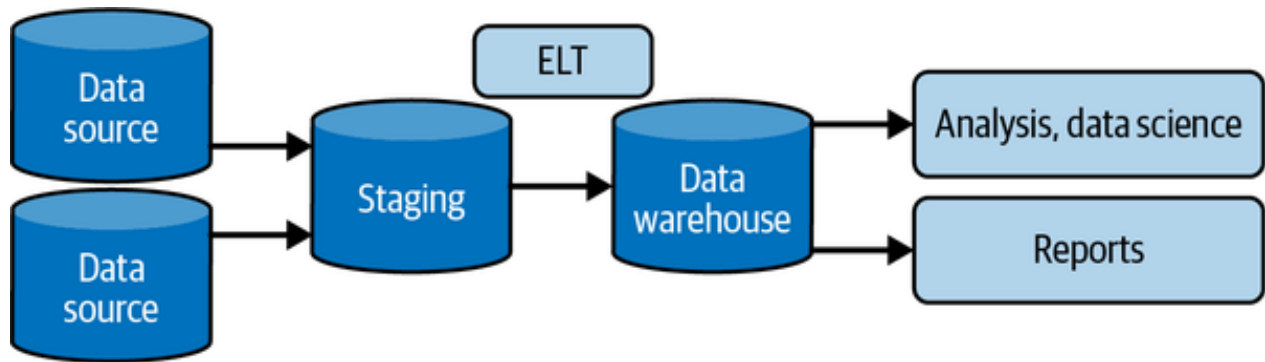


Figure 3-11. ELT—extract, load, and transform A second version of ELT was popularized during big data growth in the Hadoop ecosystem. This is transform-on-read ELT , which we discuss in “Data Lake” .

Hình 3-11. ELT — trích xuất, tải, và biến đổi. Một phiên bản thứ hai của ELT được phổ biến trong giai đoạn tăng trưởng big data trong hệ sinh thái Hadoop. Đây là ELT biến-đổi-khi-đọc (transform-on-read), mà chúng tôi thảo luận trong “Hồ dữ liệu”.

The cloud data warehouse — Kho dữ liệu đám mây

Cloud data warehouses represent a significant evolution of the on-premises data warehouse architecture and have thus led to significant changes to the organizational architecture. Amazon Redshift kicked off the cloud data warehouse revolution. Instead of needing to appropriately size an MPP system for the next several years and sign a multimillion-dollar contract to procure the system, companies had the option of spinning up a Redshift cluster on demand, scaling it up over time as data and analytics demand grew. They could even spin up new Redshift clusters on demand to serve specific workloads and quickly delete clusters when they were no longer needed.

Các kho dữ liệu đám mây đại diện cho một bước tiến hóa đáng kể của kiến trúc kho dữ liệu on-premises và do đó đã dẫn đến những thay đổi đáng kể với kiến trúc tổ chức. Amazon Redshift khởi động cuộc cách mạng kho dữ liệu đám mây. Thay vì cần định kích cỡ một hệ thống MPP một cách phù hợp cho vài năm tới và ký một hợp đồng nhiều triệu đô-la để mua hệ thống, các công ty có lựa chọn khởi tạo một cụm Redshift theo nhu cầu, mở rộng nó lên theo thời gian khi nhu cầu dữ liệu và phân tích tăng. Họ thậm chí có thể khởi tạo các cụm Redshift mới theo nhu cầu để phục vụ các khối lượng công việc cụ thể và nhanh chóng xóa các cụm khi không còn cần đến chúng nữa.

Google BigQuery, Snowflake, and other competitors popularized the idea of separating compute from storage. In this architecture, data is housed in object storage, allowing virtually limitless storage. This also gives users the option to spin up computing power on demand, providing ad hoc big data capabilities without the long-term cost of thousands of nodes. Cloud data warehouses expand the capabilities of MPP systems to cover many big data use cases that required a Hadoop cluster in the very recent past. They can readily process petabytes of data in a single query. They typically support data structures that allow the storage of tens of megabytes of raw text data per row or extremely rich and complex JSON documents. As cloud data warehouses (and data lakes) mature, the line between the data warehouse and the data lake will continue to blur. So significant is the impact of the new capabilities offered by cloud data warehouses that we might consider jettisoning the term data warehouse altogether. Instead, these services are evolving into a new **data platform** with much broader capabilities than those offered by a traditional MPP system.

Google BigQuery, Snowflake, và các đối thủ cạnh tranh khác đã phổ biến ý tưởng tách tính toán khỏi lưu trữ. Trong kiến trúc này, dữ liệu được lưu trong lưu trữ đối tượng, cho phép lưu trữ gần như vô hạn. Điều này cũng cho người dùng lựa chọn khởi tạo sức mạnh tính toán theo nhu cầu, cung cấp các năng lực big data tùy biến (ad hoc) mà không có chi phí dài hạn của hàng nghìn nút. Các kho dữ liệu đám mây mở rộng các năng lực của các hệ thống MPP để bao trùm nhiều ca sử dụng big data từng đòi hỏi một cụm Hadoop trong quá khứ rất gần đây. Chúng có thể dễ dàng xử lý petabyte dữ liệu trong một truy vấn đơn. Chúng thường hỗ trợ các cấu trúc dữ liệu cho phép lưu trữ hàng chục megabyte dữ liệu văn bản thô trên mỗi hàng hoặc các tài liệu JSON cực kỳ phong phú và phức tạp. Khi các kho dữ liệu đám mây (và các hồ dữ liệu) trưởng thành, ranh giới giữa kho dữ liệu và hồ dữ liệu sẽ tiếp tục mờ đi. Tác động của các năng lực mới do các kho dữ liệu đám mây mang lại lớn đến mức chúng ta có thể cân nhắc loại bỏ hẳn thuật ngữ kho dữ liệu. Thay vào đó, các dịch vụ này đang tiến hóa thành một nền tảng dữ liệu mới với các năng lực rộng hơn nhiều so với những gì một hệ thống MPP truyền thống cung cấp.

Data marts — Data mart

A **data mart** is a more refined subset of a warehouse designed to serve analytics and reporting, focused on a single suborganization, department, or line of business; every department has its own data mart, specific to its needs. This is in contrast to the full data warehouse that serves the broader organization or business. Data marts exist for two reasons. First, a data mart makes data more easily accessible to analysts and report developers. Second, data marts provide an additional stage of transformation beyond that provided by the initial ETL or ELT pipelines. This can significantly improve performance if reports or analytics queries require complex joins and aggregations of data, especially when the raw data is large. Transform processes can populate the data mart with joined and aggregated data to improve performance for live queries. Figure 3-12 shows the general workflow. We discuss data marts, and modeling data for data marts, in Chapter 8.

Một data mart là một tập con tinh chỉnh hơn của một kho dữ liệu được thiết kế để phục vụ phân tích và báo cáo, tập trung vào một tổ chức con, phòng ban, hoặc mảng kinh doanh đơn lẻ; mỗi phòng ban có data mart riêng của mình, đặc thù theo nhu cầu của nó. Điều này trái ngược với kho dữ liệu đầy đủ phục vụ tổ chức hoặc doanh nghiệp rộng hơn. Các data mart tồn tại vì hai lý do. Thứ nhất, một data mart làm cho dữ liệu dễ tiếp cận hơn đối với các nhà phân tích và lập trình viên báo cáo. Thứ hai, các data mart cung cấp một giai đoạn biến đổi bổ sung vượt ra ngoài giai đoạn được cung cấp bởi các đường ống ETL hoặc ELT ban đầu. Điều này có thể cải thiện đáng kể hiệu năng nếu các truy vấn báo cáo hoặc phân tích đòi hỏi các phép kết (join) và tổng hợp dữ liệu phức tạp, đặc biệt khi dữ liệu thô lớn. Các tiến trình biến đổi có thể nạp data mart với dữ liệu đã kết và đã tổng hợp để cải thiện hiệu năng cho các truy vấn trực tiếp. Hình 3-12 cho thấy luồng công việc chung. Chúng tôi thảo luận về các data mart, và mô hình hóa dữ liệu cho các data mart, trong Chương 8.

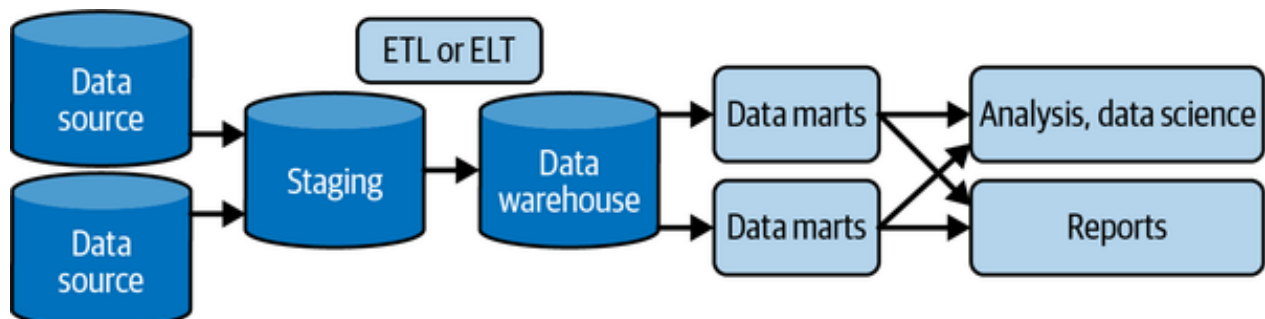


Figure 3-12. ETL or ELT plus data marts

Hình 3-12. ETL hoặc ELT cộng với các data mart

Data Lake — Hồ dữ liệu

Among the most popular architectures that appeared during the big data era is the data lake . Instead of imposing tight structural limitations on data, why not simply dump all of your data—structured and unstructured—into a central location? The data lake promised to be a democratizing force, liberating the business to drink from a fountain of limitless data. The first-generation data lake, “data lake 1.0,” made solid contributions but generally failed to deliver on its promise. Data lake 1.0 started with HDFS. As the cloud grew in popularity, these data lakes moved to cloud-based object storage, with extremely cheap storage costs and virtually limitless storage capacity. Instead of relying on a monolithic data warehouse where storage and compute are tightly coupled, the data lake allows an immense amount of data of any size and type to be stored. When this data needs to be queried or transformed, you have access to nearly unlimited computing power by spinning up a cluster on demand, and you can pick your favorite data-processing technology for the task at hand—MapReduce, Spark, Ray, Presto, Hive, etc. Despite the promise and hype, data lake 1.0 had serious shortcomings. The data lake became a dumping ground; terms such as data swamp , dark data , and WORN were coined as once-promising data projects failed. Data grew to unmanageable sizes, with little in the way of **schema** management, data cataloging, and discovery tools. In addition, the original data lake concept was essentially write-only, creating huge headaches with the arrival of regulations such as GDPR that required targeted deletion of user records.

Trong số những kiến trúc phổ biến nhất xuất hiện trong kỷ nguyên big data là hồ dữ liệu (data lake). Thay vì áp đặt những giới hạn cấu trúc chặt chẽ lên dữ liệu, sao không đơn giản đổ tất cả dữ liệu của bạn — có cấu trúc và phi cấu trúc — vào một vị trí trung tâm? Hồ dữ liệu hứa hẹn là một lực lượng dân chủ hóa, giải phóng doanh nghiệp để uống từ một suối nguồn dữ liệu vô tận. Hồ dữ liệu thế hệ đầu, “hồ dữ liệu 1.0”, đã có những đóng góp vững chắc nhưng nhìn chung không thực hiện được lời hứa của nó. Hồ dữ liệu 1.0 bắt đầu với HDFS. Khi đám mây ngày càng phổ biến, các hồ dữ liệu này chuyển sang lưu trữ đối tượng dựa trên đám mây, với chi phí lưu trữ cực kỳ rẻ và dung lượng lưu trữ gần như vô hạn. Thay vì dựa vào một kho dữ liệu monolithic nơi lưu trữ và tính toán ghép chặt, hồ dữ liệu cho phép lưu trữ một lượng dữ liệu khổng lồ ở bất kỳ kích thước và loại nào. Khi dữ liệu này cần được truy vấn hoặc biến đổi, bạn có quyền truy cập vào sức mạnh tính toán gần như không giới hạn bằng cách khởi tạo một cụm theo nhu cầu, và bạn có thể chọn công nghệ xử lý dữ liệu ưa thích của mình cho tác vụ trước mắt — MapReduce, Spark, Ray, Presto, Hive, v.v. Dù có lời hứa và sự thổi phồng, hồ dữ liệu 1.0 có những thiếu sót nghiêm trọng. Hồ dữ liệu trở thành một bãi rác; các thuật ngữ như đầm lầy dữ liệu (data swamp), dữ liệu tối (dark data), và WORN được đặt ra khi các dự án dữ liệu từng hứa hẹn thất bại. Dữ liệu phình to đến những kích thước không quản lý nổi, với rất ít công cụ quản lý lược đồ (schema), lập danh mục dữ liệu, và khám phá. Ngoài ra, khái niệm hồ dữ liệu nguyên thủy về cơ bản là chỉ ghi, tạo ra những cơn đau đầu khổng lồ với sự ra đời của các quy định như GDPR vốn đòi hỏi xóa có chủ đích các bản ghi người dùng.

Processing data was also challenging. Relatively banal data transformations such as joins were a huge headache to code as MapReduce jobs. Later frameworks such as Pig and Hive somewhat improved the situation for data processing but did little to address the basic problems of data management. Simple data manipulation language (DML) operations common in SQL— deleting or updating rows—were painful to implement, generally achieved by creating entirely new tables. While big data engineers radiated a particular disdain for their counterparts in data warehousing,

the latter could point out that data warehouses provided basic data management capabilities out of the box, and that SQL was an efficient tool for writing complex, performant queries and transformations. Data lake 1.0 also failed to deliver on another core promise of the big data movement. Open source software in the Apache ecosystem was touted as a means to avoid multimillion-dollar contracts for proprietary MPP systems. Cheap, off-the-shelf hardware would replace custom vendor solutions. In reality, big data costs ballooned as the complexities of managing Hadoop clusters forced companies to hire large teams of engineers at high salaries. Companies often chose to purchase licensed, customized versions of Hadoop from vendors to avoid the exposed wires and sharp edges of the raw Apache codebase and acquire a set of scaffolding tools to make Hadoop more user-friendly. Even companies that avoided managing Hadoop clusters using cloud storage had to spend big on talent to write MapReduce jobs. We should be careful not to understate the utility and power of first-generation data lakes. Many organizations found significant value in data lakes—especially huge, heavily data-focused Silicon Valley tech companies like Netflix and Facebook. These companies had the resources to build successful data practices and create their custom Hadoop-based tools and enhancements. But for many organizations, data lakes turned into an internal superfund site of waste, disappointment, and spiraling costs.

Xử lý dữ liệu cũng đầy thách thức. Những biến đổi dữ liệu tương đối tầm thường như các phép kết là một cơn đau đầu khổng lồ để viết mã thành các công việc MapReduce. Các framework sau này như Pig và Hive phần nào cải thiện tình hình cho xử lý dữ liệu nhưng làm rất ít để giải quyết các vấn đề cơ bản của quản lý dữ liệu. Các thao tác ngôn ngữ thao tác dữ liệu (DML) đơn giản phổ biến trong SQL — xóa hoặc cập nhật các hàng — là cực kỳ khó sở để hiện thực, thường được đạt được bằng cách tạo ra hoàn toàn các bảng mới. Trong khi các kỹ sư big data tỏ ra một sự khinh thường đặc biệt với các đồng nghiệp của họ trong kho dữ liệu, những người sau có thể chỉ ra rằng các kho dữ liệu cung cấp các năng lực quản lý dữ liệu cơ bản ngay từ đầu, và rằng SQL là một công cụ hiệu quả để viết các truy vấn và biến đổi phức tạp, hiệu năng. Hồ dữ liệu 1.0 cũng không thực hiện được một lời hứa cốt lõi khác của phong trào big data. Phần mềm mã nguồn mở trong hệ sinh thái Apache được ca ngợi là một phương tiện để tránh các hợp đồng nhiều triệu đô-la cho các hệ thống MPP độc quyền. Phần cứng rẻ, có sẵn sẽ thay thế các giải pháp tùy chỉnh của nhà cung cấp. Thực tế, chi phí big data phình to khi sự phức tạp của việc quản lý các cụm Hadoop buộc các công ty phải thuê những đội kỹ sư lớn với mức lương cao. Các công ty thường chọn mua các phiên bản Hadoop được cấp phép, tùy chỉnh từ các nhà cung cấp để tránh những dây điện lộ ra và các cạnh sắc của codebase Apache thô và để có một bộ công cụ giàn giáo nhằm làm cho Hadoop thân thiện với người dùng hơn. Ngay cả các công ty tránh được việc quản lý các cụm Hadoop bằng cách dùng lưu trữ đám mây cũng phải chi mạnh cho nhân tài để viết các công việc MapReduce. Chúng ta nên cẩn thận không đánh giá thấp tính hữu dụng và sức mạnh của các hồ dữ liệu thế hệ đầu. Nhiều tổ chức tìm thấy giá trị đáng kể trong các hồ dữ liệu — đặc biệt là các công ty công nghệ khổng lồ, tập trung nặng vào dữ liệu ở Thung lũng Silicon như Netflix và Facebook. Những công ty này có nguồn lực để xây dựng các thực hành dữ liệu thành công và tạo ra các công cụ và cải tiến dựa trên Hadoop tùy chỉnh của họ. Nhưng đối với nhiều tổ chức, các hồ dữ liệu biến thành một bãi rác độc hại nội bộ của lãng phí, thất vọng và chi phí leo thang.

Convergence, Next-Generation Data Lakes, and the Data Platform — Sự hội tụ, các hồ dữ liệu thế hệ mới, và nền tảng dữ liệu

In response to the limitations of first-generation data lakes, various players have sought to enhance the concept to fully realize its promise. For example, Databricks introduced the notion of a data

lakehouse . The lakehouse incorporates the controls, data management, and data structures found in a data warehouse while still housing data in object storage and supporting a variety of query and transformation engines. In particular, the data lakehouse supports atomicity, consistency, isolation, and durability (ACID) transactions, a big departure from the original data lake, where you simply pour in data and never update or delete it. The term data lakehouse suggests a convergence between data lakes and data warehouses. The technical architecture of cloud data warehouses has evolved to be very similar to a data lake architecture. Cloud data warehouses separate compute from storage, support petabyte-scale queries, store unstructured text and semistructured objects, and integrate with advanced processing technologies such as Spark or Beam. We believe that the trend of convergence will only continue. The data lake and the data warehouse will still exist as different architectures. In practice, their capabilities will converge so that few users will notice a boundary between them in their day-to-day work. We now see several vendors offering data platforms that combine data lake and data warehouse capabilities. From our perspective, AWS, Azure, Google Cloud , Snowflake , and Databricks are class leaders, each offering a constellation of tightly integrated tools for working with data, running the gamut from relational to completely unstructured. Instead of choosing between a data lake or data warehouse architecture, future data engineers will have the option to choose a converged data platform based on a variety of factors, including vendor, ecosystem, and relative openness.

Để đáp lại những giới hạn của các hồ dữ liệu thế hệ đầu, nhiều bên đã tìm cách nâng cao khái niệm này để hiện thực hóa đầy đủ lời hứa của nó. Ví dụ, Databricks giới thiệu khái niệm data lakehouse. Lakehouse tích hợp các kiểm soát, quản lý dữ liệu, và các cấu trúc dữ liệu được tìm thấy trong một kho dữ liệu trong khi vẫn lưu dữ liệu trong lưu trữ đối tượng và hỗ trợ nhiều loại động cơ truy vấn và biến đổi. Đặc biệt, data lakehouse hỗ trợ các giao dịch nguyên tử, nhất quán, cô lập và bền vững (ACID), một sự rời bỏ lớn so với hồ dữ liệu nguyên thủy, nơi bạn chỉ đơn giản đổ dữ liệu vào và không bao giờ cập nhật hay xóa nó. Thuật ngữ data lakehouse gợi ý một sự hội tụ giữa các hồ dữ liệu và các kho dữ liệu. Kiến trúc kỹ thuật của các kho dữ liệu đám mây đã tiến hóa để rất giống một kiến trúc hồ dữ liệu. Các kho dữ liệu đám mây tách tính toán khỏi lưu trữ, hỗ trợ các truy vấn quy mô petabyte, lưu trữ văn bản phi cấu trúc và các đối tượng bán cấu trúc, và tích hợp với các công nghệ xử lý tiên tiến như Spark hoặc Beam. Chúng tôi tin rằng xu hướng hội tụ sẽ chỉ tiếp tục. Hồ dữ liệu và kho dữ liệu vẫn sẽ tồn tại như những kiến trúc khác nhau. Trong thực tế, các năng lực của chúng sẽ hội tụ sao cho ít người dùng sẽ nhận thấy một ranh giới giữa chúng trong công việc hằng ngày của họ. Giờ chúng ta thấy một số nhà cung cấp cung cấp các nền tảng dữ liệu kết hợp các năng lực của hồ dữ liệu và kho dữ liệu. Theo quan điểm của chúng tôi, AWS, Azure, Google Cloud, Snowflake, và Databricks là những người dẫn đầu trong lớp của mình, mỗi bên cung cấp một chòm sao các công cụ tích hợp chặt chẽ để làm việc với dữ liệu, trải dài từ quan hệ đến hoàn toàn phi cấu trúc. Thay vì lựa chọn giữa một kiến trúc hồ dữ liệu hoặc kho dữ liệu, các kỹ sư dữ liệu tương lai sẽ có lựa chọn chọn một nền tảng dữ liệu hội tụ dựa trên nhiều yếu tố, bao gồm nhà cung cấp, hệ sinh thái, và mức độ mở tương đối.

Modern Data Stack — Ngăn xếp dữ liệu hiện đại

The **modern data stack** (Figure 3-13) is currently a trendy analytics architecture that highlights the type of abstraction we expect to see more widely used over the next several years. Whereas past data stacks relied on expensive, monolithic toolsets, the main objective of the modern data stack is to use cloud-based, plug-and-play, easy-to-use, off-the-shelf components to create a modular and cost-effective data architecture. These components include data pipelines, storage, transformation, data management/governance, monitoring, visualization, and exploration. The domain is still in flux, and

the specific tools are changing and evolving rapidly, but the core aim will remain the same: to reduce complexity and increase modularization. Note that the notion of a modern data stack integrates nicely with the converged data platform idea from the previous section.

Ngăn xếp dữ liệu hiện đại (modern data stack) (Hình 3-13) hiện là một kiến trúc phân tích đang thịnh hành làm nổi bật loại trừu tượng mà chúng tôi kỳ vọng sẽ thấy được dùng rộng rãi hơn trong vài năm tới. Trong khi các ngăn xếp dữ liệu trong quá khứ dựa vào những bộ công cụ đắt đỏ, monolithic, mục tiêu chính của modern data stack là dùng các thành phần dựa trên đám mây, cắm-là-chạy, dễ dùng, có sẵn để tạo ra một kiến trúc dữ liệu mô-đun và hiệu quả về chi phí. Các thành phần này bao gồm các đường ống dữ liệu, lưu trữ, biến đổi, quản lý/quản trị dữ liệu, giám sát, trực quan hóa, và khám phá. Lĩnh vực này vẫn đang biến động, và các công cụ cụ thể đang thay đổi và tiến hóa nhanh chóng, nhưng mục tiêu cốt lõi sẽ vẫn như cũ: giảm độ phức tạp và tăng mô-đun hóa. Lưu ý rằng khái niệm modern data stack tích hợp tốt với ý tưởng nền tảng dữ liệu hội tụ từ mục trước.

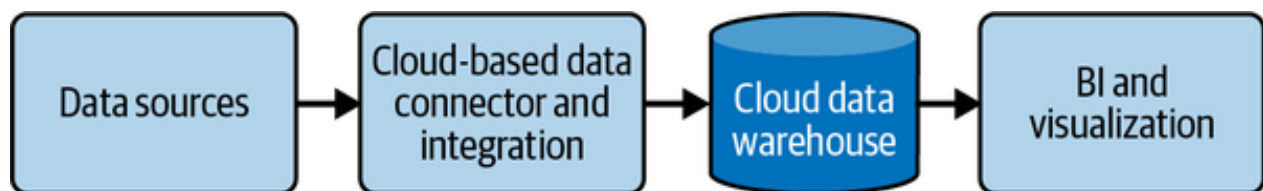


Figure 3-13. Basic components of the modern data stack Key outcomes of the modern data stack are self-service (analytics and pipelines), agile data management, and using open source tools or simple proprietary tools with clear pricing structures. Community is a central aspect of the modern data stack as well. Unlike products of the past that had releases and roadmaps largely hidden from users, projects and companies operating in the modern data stack space typically have strong user bases and active communities that participate in the development by using the product early, suggesting features, and submitting pull requests to improve the code. Regardless of where “modern” goes (we share our ideas in Chapter 11), we think the key concept of plug-and-play modularity with easy-to-understand pricing and implementation is the way of the future. Especially in analytics engineering, the modern data stack is and will continue to be the default choice of data architecture. Throughout the book, the architecture we reference contains pieces of the modern data stack, such as cloud-based and plug-and-play modular components.

Hình 3-13. Các thành phần cơ bản của modern data stack. Các kết quả then chốt của modern data stack là tự phục vụ (phân tích và đường ống), quản lý dữ liệu nhanh nhạy, và dùng các công cụ mã nguồn mở hoặc các công cụ độc quyền đơn giản với các cấu trúc giá rõ ràng. Cộng đồng cũng là một khía cạnh trung tâm của modern data stack. Không như các sản phẩm trong quá khứ có các bản phát hành và lộ trình phần lớn ẩn khỏi người dùng, các dự án và công ty hoạt động trong không gian modern data stack thường có nền tảng người dùng mạnh và các cộng đồng tích cực tham gia vào việc phát triển bằng cách dùng sản phẩm sớm, đề xuất các tính năng, và gửi các pull request để cải thiện mã. Bất kể “hiện đại” đi đến đâu (chúng tôi chia sẻ ý tưởng của mình trong Chương 11), chúng tôi nghĩ khái niệm then chốt về tính mô-đun cắm-là-chạy với giá và triển khai dễ hiểu là con đường của tương lai. Đặc biệt trong kỹ thuật phân tích, modern data stack đang và sẽ tiếp tục là lựa chọn mặc định của kiến trúc dữ liệu. Xuyên suốt cuốn sách, kiến trúc mà chúng tôi tham chiếu chứa các mảnh của modern data stack, chẳng hạn các thành phần mô-đun dựa trên đám mây và cắm-là-chạy.

Lambda Architecture — Kiến trúc Lambda

In the “old days” (the early to mid-2010s), the popularity of working with streaming data exploded with the emergence of Kafka as a highly scalable **message queue** and frameworks such as Apache Storm and Samza for streaming/real-time analytics. These technologies allowed companies to perform new types of analytics and modeling on large amounts of data, user aggregation and ranking, and product recommendations. Data engineers needed to figure out how to reconcile batch and streaming data into a single architecture. The Lambda architecture was one of the early popular responses to this problem. In a Lambda architecture (Figure 3-14), you have systems operating independently of each other—batch, streaming, and serving. The **source system** is ideally **immutable** and append-only, sending data to two destinations for processing: stream, and batch. In-stream processing intends to serve the data with the lowest possible latency in a “speed” layer, usually a NoSQL database. In the batch layer, data is processed and transformed in a system such as a data warehouse, creating precomputed and aggregated views of the data. The serving layer provides a combined view by aggregating query results from the two layers.

Trong “những ngày xưa” (đầu đến giữa thập niên 2010), sự phổ biến của việc làm việc với dữ liệu theo luồng bùng nổ với sự nổi lên của Kafka như một hàng đợi tin nhắn (message queue) có khả năng mở rộng cao và các framework như Apache Storm và Samza cho phân tích theo luồng/thời gian thực. Những công nghệ này cho phép các công ty thực hiện các loại phân tích và mô hình hóa mới trên những lượng dữ liệu lớn, tổng hợp và xếp hạng người dùng, và đề xuất sản phẩm. Các kỹ sư dữ liệu cần tìm ra cách đối soát dữ liệu theo lô và theo luồng vào một kiến trúc đơn. Kiến trúc Lambda là một trong những phản ứng phổ biến đầu tiên cho vấn đề này. Trong một kiến trúc Lambda (Hình 3-14), bạn có các hệ thống hoạt động độc lập với nhau — theo lô, theo luồng, và phục vụ. Hệ thống nguồn lý tưởng là bất biến và chỉ-thêm (append-only), gửi dữ liệu tới hai đích để xử lý: luồng và lô. Xử lý trong-luồng (in-stream) nhằm phục vụ dữ liệu với độ trễ thấp nhất có thể trong một lớp “tốc độ” (speed), thường là một cơ sở dữ liệu NoSQL. Trong lớp lô, dữ liệu được xử lý và biến đổi trong một hệ thống như một kho dữ liệu, tạo ra các view dữ liệu được tính toán trước và tổng hợp. Lớp phục vụ cung cấp một view kết hợp bằng cách tổng hợp các kết quả truy vấn từ hai lớp.

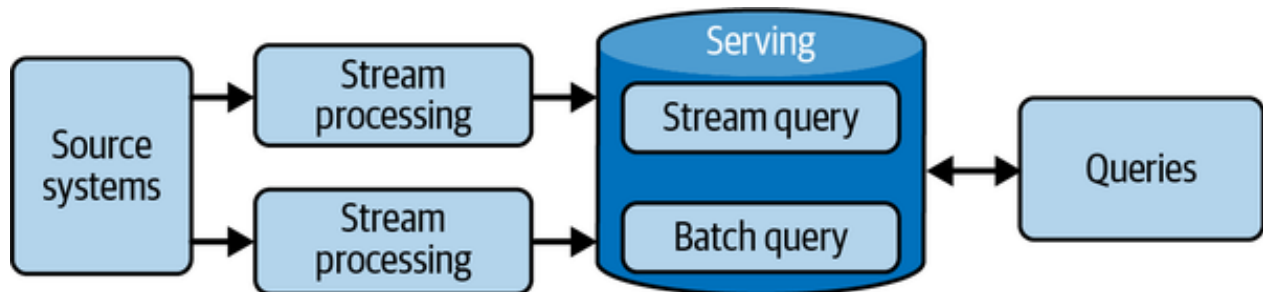


Figure 3-14. Lambda architecture Lambda architecture has its share of challenges and criticisms. Managing multiple systems with different codebases is as difficult as it sounds, creating error-prone systems with code and data that are extremely difficult to reconcile. We mention Lambda architecture because it still gets attention and is popular in search-engine results for data architecture. Lambda isn’t our first recommendation if you’re trying to combine streaming and batch data for analytics. Technology and practices have moved on. Next, let’s look at a reaction to Lambda architecture, the **Kappa architecture**.

Hình 3-14. Kiến trúc Lambda. Kiến trúc Lambda có phần thách thức và chỉ trích của nó. Quản lý nhiều hệ thống với các codebase khác nhau khó như nghe có vẻ vậy, tạo ra những hệ thống dễ lỗi với mã và dữ liệu cực kỳ khó đối soát. Chúng tôi đề cập đến kiến trúc

Lambda vì nó vẫn được chú ý và phổ biến trong các kết quả công cụ tìm kiếm về kiến trúc dữ liệu. Lambda không phải là khuyến nghị đầu tiên của chúng tôi nếu bạn đang cố kết hợp dữ liệu theo luồng và theo lô để phân tích. Công nghệ và thực hành đã tiến lên. Tiếp theo, hãy xem một phản ứng đối với kiến trúc Lambda, kiến trúc Kappa.

Kappa Architecture — Kiến trúc Kappa

As a response to the shortcomings of Lambda architecture, Jay Kreps proposed an alternative called Kappa architecture (Figure 3-15). The 10 central thesis is this: why not just use a stream-processing platform as the backbone for all data handling—ingestion, storage, and serving? This facilitates a true event-based architecture. Real-time and batch processing can be applied seamlessly to the same data by reading the live event stream directly and replaying large chunks of data for batch processing.

Như một phản ứng đối với những thiếu sót của kiến trúc Lambda, Jay Kreps đề xuất một lựa chọn thay thế gọi là kiến trúc Kappa (Hình 3-15). Luận điểm trung tâm là thế này: sao không chỉ dùng một nền tảng xử lý theo luồng làm xương sống cho toàn bộ việc xử lý dữ liệu — thu nạp, lưu trữ, và phục vụ? Điều này tạo điều kiện cho một kiến trúc thực sự dựa trên sự kiện. Xử lý thời gian thực và theo lô có thể được áp dụng liền mạch cho cùng dữ liệu bằng cách đọc trực tiếp luồng sự kiện trực tiếp và phát lại các khối lớn dữ liệu để xử lý theo lô.

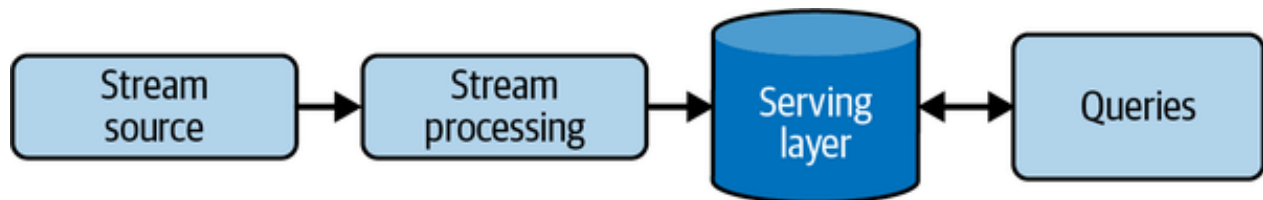


Figure 3-15. Kappa architecture Though the original Kappa architecture article came out in 2014, we haven't seen it widely adopted. There may be a couple of reasons for this. First, streaming itself is still a bit of a mystery for many companies. Second, Kappa architecture turns out to be complicated and expensive in practice. While some streaming systems can scale to huge data volumes, they are complex and expensive; batch storage and processing remain much more efficient and cost-effective for enormous historical datasets.

Hình 3-15. Kiến trúc Kappa. Mặc dù bài viết về kiến trúc Kappa nguyên thủy ra đời năm 2014, chúng tôi chưa thấy nó được áp dụng rộng rãi. Có thể có một vài lý do cho điều này. Thứ nhất, bản thân việc xử lý theo luồng vẫn còn là một điều bí ẩn đối với nhiều công ty. Thứ hai, kiến trúc Kappa hóa ra phức tạp và đắt đỏ trong thực tế. Trong khi một số hệ thống theo luồng có thể mở rộng tới những khối lượng dữ liệu khổng lồ, chúng phức tạp và đắt đỏ; lưu trữ và xử lý theo lô vẫn hiệu quả và tiết kiệm chi phí hơn nhiều cho những tập dữ liệu lịch sử khổng lồ.

The Dataflow Model and Unified Batch and Streaming — Mô hình Dataflow và sự hợp nhất theo lô và theo luồng

Both Lambda and Kappa sought to address limitations of the Hadoop ecosystem of the 2010s by trying to duct-tape together complicated tools that were likely not natural fits in the first place. The central challenge of unifying batch and streaming data remained, and Lambda and Kappa both provided inspiration and groundwork for continued progress in this pursuit. One of the central

problems of managing batch and stream processing is unifying multiple code paths. While the Kappa architecture relies on a unified queuing and storage layer, one still has to confront using different tools for collecting real-time statistics or running batch aggregation jobs. Today, engineers seek to solve this in several ways. Google made its mark by developing the Dataflow model and the Apache Beam framework that implements this model. The core idea in the Dataflow model is to view all data as events, as the aggregation is performed over various types of windows. Ongoing real-time event streams are **unbounded data**. Data batches are simply **bounded** event streams, and the boundaries provide a natural window. Engineers can choose from various windows for real-time aggregation, such as sliding or tumbling. Real-time and batch processing happens in the same system using nearly identical code. The philosophy of “batch as a special case of streaming” is now more pervasive. Various frameworks such as Flink and Spark have adopted a similar approach.

Cả Lambda và Kappa đều tìm cách giải quyết những giới hạn của hệ sinh thái Hadoop của thập niên 2010 bằng cách cố gắng dùng bằng dính dán lại với nhau những công cụ phức tạp mà nhiều khả năng vốn không phải là những thứ ăn khớp tự nhiên ngay từ đầu. Thách thức trung tâm của việc hợp nhất dữ liệu theo lô và theo luồng vẫn còn đó, và cả Lambda lẫn Kappa đều cung cấp cảm hứng và nền tảng cho sự tiến bộ tiếp tục trong việc theo đuổi này. Một trong những vấn đề trung tâm của việc quản lý xử lý theo lô và theo luồng là hợp nhất nhiều đường đi mã. Trong khi kiến trúc Kappa dựa vào một lớp xếp hàng và lưu trữ hợp nhất, người ta vẫn phải đối mặt với việc dùng các công cụ khác nhau để thu thập thống kê thời gian thực hoặc chạy các công việc tổng hợp theo lô. Ngày nay, các kỹ sư tìm cách giải quyết điều này bằng một số cách. Google ghi dấu ấn của mình bằng cách phát triển mô hình Dataflow và framework Apache Beam hiện thực mô hình này. Ý tưởng cốt lõi trong mô hình Dataflow là xem tất cả dữ liệu như các sự kiện, vì việc tổng hợp được thực hiện trên nhiều loại cửa sổ (window) khác nhau. Các luồng sự kiện thời gian thực đang diễn ra là dữ liệu không biên (unbounded). Các lô dữ liệu đơn giản là các luồng sự kiện có biên (bounded), và các biên cung cấp một cửa sổ tự nhiên. Các kỹ sư có thể chọn từ nhiều cửa sổ khác nhau cho việc tổng hợp thời gian thực, chẳng hạn trượt (sliding) hoặc lăn (tumbling). Xử lý thời gian thực và theo lô diễn ra trong cùng một hệ thống bằng cách dùng mã gần như giống hệt nhau. Triết lý “lô như một trường hợp đặc biệt của luồng” giờ đây lan tỏa rộng hơn. Nhiều framework khác nhau như Flink và Spark đã áp dụng một cách tiếp cận tương tự.

Architecture for IoT — Kiến trúc cho IoT

The Internet of Things (IoT) is the distributed collection of devices, aka things —computers, sensors, mobile devices, smart home devices, and anything else with an internet connection. Rather than generating data from direct human input (think data entry from a keyboard), IoT data is generated from devices that collect data periodically or continuously from the surrounding environment and transmit it to a destination. IoT devices are often low-powered and operate in low-resource/low bandwidth environments. While the concept of IoT devices dates back at least a few decades, the smartphone revolution created a massive IoT swarm virtually overnight. Since then, numerous new IoT categories have emerged, such as smart thermostats, car entertainment systems, smart TVs, and smart speakers. The IoT has evolved from a futurist fantasy to a massive data engineering domain. We expect IoT to become one of the dominant ways data is generated and consumed, and this section goes a bit deeper than the others you’ve read. Having a cursory understanding of IoT architecture will help you understand broader data architecture trends. Let’s briefly look at some IoT architecture concepts.

Internet vạn vật (IoT) là tập hợp phân tán các thiết bị, hay còn gọi là vạn vật (things) —

máy tính, cảm biến, thiết bị di động, thiết bị nhà thông minh, và bất cứ thứ gì khác có kết nối internet. Thay vì sinh dữ liệu từ đầu vào trực tiếp của con người (nghĩ đến việc nhập liệu từ bàn phím), dữ liệu IoT được sinh từ các thiết bị thu thập dữ liệu định kỳ hoặc liên tục từ môi trường xung quanh và truyền nó tới một đích. Các thiết bị IoT thường có công suất thấp và hoạt động trong các môi trường ít tài nguyên/băng thông thấp. Mặc dù khái niệm thiết bị IoT có từ ít nhất vài thập kỷ trước, cuộc cách mạng điện thoại thông minh đã tạo ra một bầy IoT khổng lồ gần như chỉ sau một đêm. Kể từ đó, nhiều hạng mục IoT mới đã nổi lên, chẳng hạn bộ điều nhiệt thông minh, hệ thống giải trí trên xe, TV thông minh, và loa thông minh. IoT đã tiến hóa từ một ảo tưởng của nhà tương lai học thành một lĩnh vực kỹ thuật dữ liệu khổng lồ. Chúng tôi kỳ vọng IoT sẽ trở thành một trong những cách thống trị để dữ liệu được sinh ra và tiêu thụ, và mục này đi sâu hơn một chút so với các mục khác mà bạn đã đọc. Có một hiểu biết sơ lược về kiến trúc IoT sẽ giúp bạn hiểu các xu hướng kiến trúc dữ liệu rộng hơn. Hãy điểm qua nhanh một số khái niệm kiến trúc IoT.

Devices — Thiết bị

Devices (also known as things) are the physical hardware connected to the internet, sensing the environment around them and collecting and transmitting data to a downstream destination. These devices might be used in consumer applications like a doorbell camera, smartwatch, or thermostat. The device might be an AI-powered camera that monitors an assembly line for defective components, a GPS tracker to record vehicle locations, or a Raspberry Pi programmed to download the latest tweets and brew your coffee. Any device capable of collecting data from its environment is an IoT device. Devices should be minimally capable of collecting and transmitting data. However, the device might also crunch data or run ML on the data it collects before sending it downstream—edge computing and edge **machine learning**, respectively. A data engineer doesn't necessarily need to know the inner details of IoT devices but should know what the device does, the data it collects, any edge computations or ML it runs before transmitting the data, and how often it sends data. It also helps to know the consequences of a device or internet outage, environmental or other external factors affecting data collection, and how these may impact the downstream collection of data from the device.

Các thiết bị (cũng được gọi là vạn vật) là phần cứng vật lý kết nối với internet, cảm nhận môi trường xung quanh chúng và thu thập, truyền dữ liệu tới một đích ở hạ nguồn. Những thiết bị này có thể được dùng trong các ứng dụng tiêu dùng như một camera chuông cửa, đồng hồ thông minh, hoặc bộ điều nhiệt. Thiết bị có thể là một camera được hỗ trợ AI giám sát một dây chuyền lắp ráp để tìm các linh kiện lỗi, một thiết bị theo dõi GPS để ghi lại vị trí xe, hoặc một Raspberry Pi được lập trình để tải xuống các tweet mới nhất và pha cà phê cho bạn. Bất kỳ thiết bị nào có khả năng thu thập dữ liệu từ môi trường của nó đều là một thiết bị IoT. Các thiết bị tối thiểu nên có khả năng thu thập và truyền dữ liệu. Tuy nhiên, thiết bị cũng có thể nghiền dữ liệu hoặc chạy ML trên dữ liệu nó thu thập trước khi gửi xuống hạ nguồn — lần lượt là điện toán biên (edge computing) và học máy biên (edge machine learning). Một kỹ sư dữ liệu không nhất thiết cần biết các chi tiết bên trong của các thiết bị IoT nhưng nên biết thiết bị làm gì, dữ liệu nó thu thập, bất kỳ tính toán biên hay ML nào nó chạy trước khi truyền dữ liệu, và tần suất nó gửi dữ liệu. Cũng hữu ích nếu biết các hậu quả của một sự cố thiết bị hoặc internet, các yếu tố môi trường hoặc bên ngoài khác ảnh hưởng đến việc thu thập dữ liệu, và những điều này có thể tác động đến việc thu thập dữ liệu ở hạ nguồn từ thiết bị như thế nào.

Interfacing with devices — Giao tiếp với các thiết bị

A device isn't beneficial unless you can get its data. This section covers some of the key components necessary to interface with IoT devices in the wild.

Một thiết bị không có lợi ích gì trừ khi bạn có thể lấy được dữ liệu của nó. Mục này đề cập đến một số thành phần then chốt cần thiết để giao tiếp với các thiết bị IoT ngoài thực địa.

IoT gateway An IoT gateway is a hub for connecting devices and securely routing devices to the appropriate destinations on the internet. While you can connect a device directly to the internet without an IoT gateway, the gateway allows devices to connect using extremely little power. It acts as a way station for **data retention** and manages an internet connection to the final data destination. New low-power WiFi standards are designed to make IoT gateways less critical in the future, but these are just rolling out now. Typically, a swarm of devices will utilize many IoT gateways, one at each physical location where devices are present (Figure 3-16).

Cổng IoT (IoT gateway). Một cổng IoT là một trung tâm để kết nối các thiết bị và định tuyến một cách an toàn các thiết bị tới các đích phù hợp trên internet. Trong khi bạn có thể kết nối một thiết bị trực tiếp với internet mà không cần một cổng IoT, cổng cho phép các thiết bị kết nối bằng cách dùng cực kỳ ít năng lượng. Nó hoạt động như một trạm trung chuyển cho việc lưu giữ dữ liệu và quản lý một kết nối internet tới đích dữ liệu cuối cùng. Các chuẩn WiFi công suất thấp mới được thiết kế để làm cho các cổng IoT bớt then chốt hơn trong tương lai, nhưng chúng chỉ mới đang được triển khai bây giờ. Thông thường, một bầy thiết bị sẽ tận dụng nhiều cổng IoT, mỗi cổng tại mỗi vị trí vật lý nơi có các thiết bị (Hình 3-16).

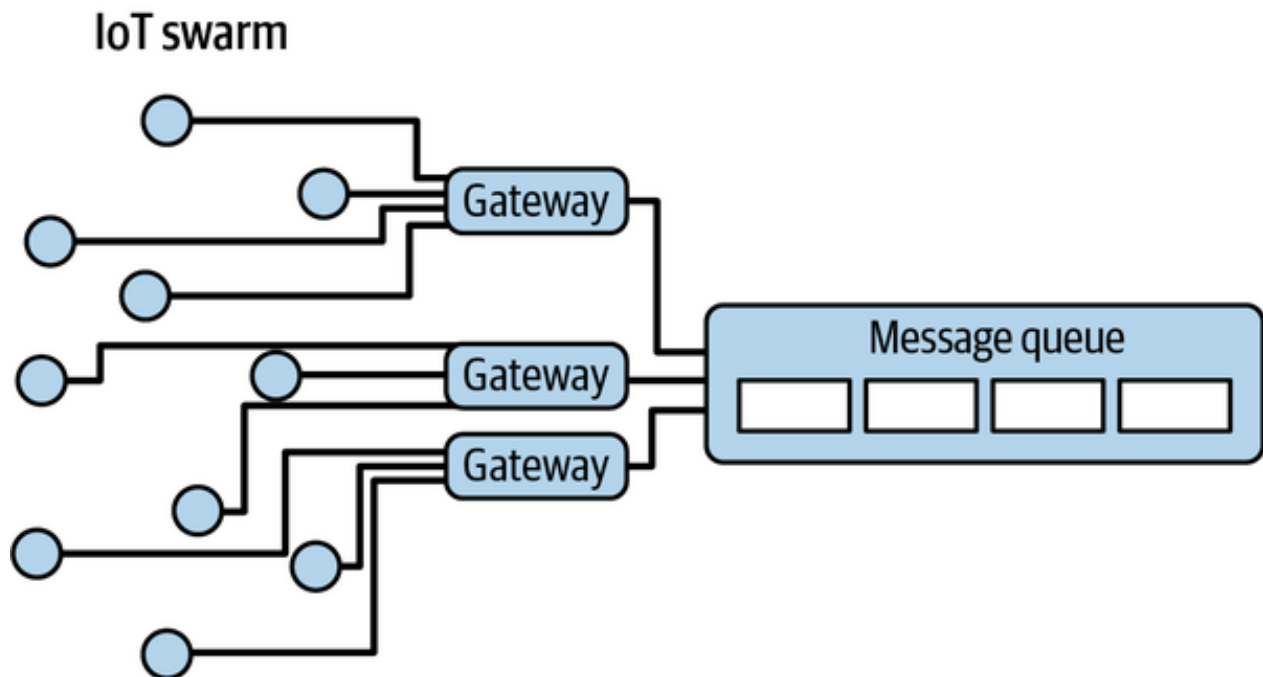


Figure 3-16. A device swarm (circles), IoT gateways, and message queue with messages (rectangles within the queue) Ingestion Ingestion begins with an IoT gateway, as discussed previously. From there, events and measurements can flow into an event ingestion architecture. Of course, other patterns are possible. For instance, the gateway may accumulate data and upload it in batches for later analytics processing. In remote physical environments, gateways may not have connectivity to a network much of the time. They may upload all data only when they are brought into the range of

a cellular or WiFi network. The point is that the diversity of IoT systems and environments presents complications—e.g., late-arriving data, data structure and schema disparities, data corruption, and connection disruption—that engineers must account for in their architectures and downstream analytics.

Hình 3-16. Một bày thiết bị (các vòng tròn), các cổng IoT, và hàng đợi tin nhắn với các tin nhắn (các hình chữ nhật bên trong hàng đợi). Thu nạp. Thu nạp bắt đầu với một cổng IoT, như đã thảo luận trước đó. Từ đó, các sự kiện và phép đo có thể chảy vào một kiến trúc thu nạp sự kiện. Tất nhiên, các mẫu hình khác cũng khả dĩ. Chẳng hạn, cổng có thể tích lũy dữ liệu và tải nó lên theo lô cho việc xử lý phân tích sau này. Trong các môi trường vật lý xa xôi, các cổng có thể không có kết nối tới một mạng phần lớn thời gian. Chúng có thể chỉ tải tất cả dữ liệu lên khi chúng được đưa vào trong phạm vi của một mạng di động hoặc WiFi. Điểm mấu chốt là sự đa dạng của các hệ thống và môi trường IoT đặt ra những phức tạp — ví dụ, dữ liệu đến muộn, sự chênh lệch về cấu trúc dữ liệu và lược đồ, hư hỏng dữ liệu, và gián đoạn kết nối — mà các kỹ sư phải tính đến trong các kiến trúc của họ và phân tích ở hạ nguồn.

Storage Storage requirements will depend a great deal on the latency requirement for the IoT devices in the system. For example, for remote sensors collecting scientific data for analysis at a later time, batch object storage may be perfectly acceptable. However, near real-time responses may be expected from a system backend that constantly analyzes data in a home monitoring and automation solution. In this case, a message queue or time-series database is more appropriate. We discuss storage systems in more detail in Chapter 6 .

Lưu trữ. Các yêu cầu lưu trữ sẽ phụ thuộc rất nhiều vào yêu cầu độ trễ cho các thiết bị IoT trong hệ thống. Ví dụ, đối với các cảm biến từ xa thu thập dữ liệu khoa học để phân tích vào một thời điểm sau, lưu trữ đối tượng theo lô có thể hoàn toàn chấp nhận được. Tuy nhiên, các phản hồi gần thời gian thực có thể được kỳ vọng từ một backend hệ thống liên tục phân tích dữ liệu trong một giải pháp giám sát và tự động hóa nhà. Trong trường hợp này, một hàng đợi tin nhắn hoặc cơ sở dữ liệu chuỗi thời gian (time-series database) thì phù hợp hơn. Chúng tôi thảo luận các hệ thống lưu trữ chi tiết hơn trong Chương 6.

Serving Serving patterns are incredibly diverse. In a batch scientific application, data might be analyzed using a cloud data warehouse and then served in a report. Data will be presented and served in numerous ways in a home-monitoring application. Data will be analyzed in the near time using a stream-processing engine or queries in a time-series database to look for critical events such as a fire, electrical outage, or break-in. Detection of an anomaly will trigger alerts to the homeowner, the fire department, or other entity. A batch analytics component also exists—for example, a monthly report on the state of the home.

Phục vụ. Các mẫu hình phục vụ cực kỳ đa dạng. Trong một ứng dụng khoa học theo lô, dữ liệu có thể được phân tích bằng một kho dữ liệu đám mây rồi được phục vụ trong một báo cáo. Dữ liệu sẽ được trình bày và phục vụ theo vô số cách trong một ứng dụng giám sát nhà. Dữ liệu sẽ được phân tích trong thời gian gần bằng một động cơ xử lý theo luồng hoặc các truy vấn trong một cơ sở dữ liệu chuỗi thời gian để tìm các sự kiện then chốt như một đám cháy, mất điện, hoặc đột nhập. Việc phát hiện một bất thường sẽ kích hoạt các cảnh báo tới chủ nhà, sở cứu hỏa, hoặc thực thể khác. Một thành phần phân tích theo lô cũng tồn tại — ví dụ, một báo cáo hằng tháng về tình trạng của ngôi nhà.

One significant serving pattern for IoT looks like **reverse ETL** (Figure 3- 17), although we tend not to use this term in the IoT context. Think of this scenario: data from sensors on manufacturing devices is collected and analyzed. The results of these measurements are processed to look for optimizations that will allow equipment to operate more efficiently. Data is sent back to reconfigure the devices

and optimize them.

Một mẫu hình phục vụ đáng kể cho IoT trông giống như reverse ETL (Hình 3-17), mặc dù chúng tôi có xu hướng không dùng thuật ngữ này trong bối cảnh IoT. Hãy nghĩ đến kịch bản này: dữ liệu từ các cảm biến trên các thiết bị sản xuất được thu thập và phân tích. Các kết quả của những phép đo này được xử lý để tìm các tối ưu hóa cho phép thiết bị vận hành hiệu quả hơn. Dữ liệu được gửi trở lại để tái cấu hình các thiết bị và tối ưu hóa chúng.

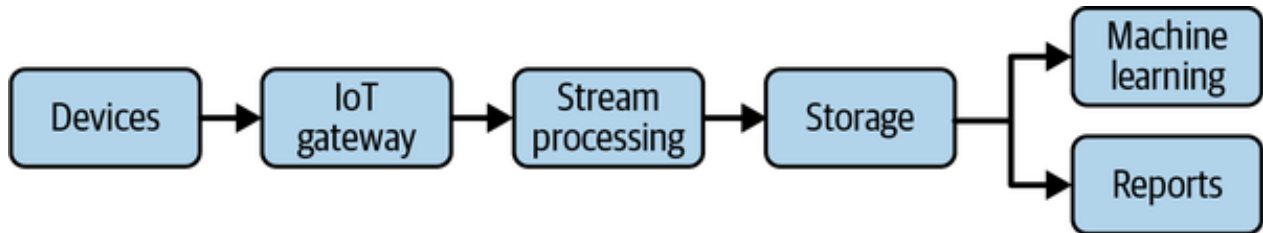


Figure 3-17. IoT serving pattern for downstream use cases

Hình 3-17. Mẫu hình phục vụ IoT cho các ca sử dụng ở hạ nguồn

Scratching the surface of the IoT — Mới chạm tới bề mặt của IoT

IoT scenarios are incredibly complex, and IoT architecture and systems are also less familiar to data engineers who may have spent their careers working with business data. We hope that this introduction will encourage interested data engineers to learn more about this fascinating and rapidly evolving specialization.

Các kịch bản IoT cực kỳ phức tạp, và kiến trúc cùng các hệ thống IoT cũng ít quen thuộc hơn với các kỹ sư dữ liệu có thể đã dành cả sự nghiệp của họ làm việc với dữ liệu kinh doanh. Chúng tôi hy vọng phần giới thiệu này sẽ khuyến khích các kỹ sư dữ liệu quan tâm tìm hiểu thêm về chuyên môn hấp dẫn và đang tiến hóa nhanh chóng này.

Data Mesh — Data mesh

The data mesh is a recent response to sprawling monolithic data platforms, such as centralized data lakes and data warehouses, and “the great divide of data,” wherein the landscape is divided between operational data and analytical data. The data mesh attempts to invert the challenges of centralized data architecture, taking the concepts of domain-driven design (commonly used in software architectures) and applying them to data architecture. Because the data mesh has captured much recent attention, you should be aware of it. A big part of the data mesh is decentralization, as Zhamak Dehghani noted in her groundbreaking article on the topic:

Data mesh là một phản ứng gần đây đối với các nền tảng dữ liệu monolithic lan tràn, chẳng hạn các hồ dữ liệu và kho dữ liệu tập trung, và “sự phân ly lớn của dữ liệu”, theo đó bối cảnh bị chia rẽ giữa dữ liệu vận hành và dữ liệu phân tích. Data mesh cố gắng đảo ngược những thách thức của kiến trúc dữ liệu tập trung, lấy các khái niệm của thiết kế hướng miền (domain-driven design, thường được dùng trong các kiến trúc phần mềm) và áp dụng chúng vào kiến trúc dữ liệu. Vì data mesh đã thu hút nhiều sự chú ý gần đây, bạn nên biết về nó. Một phần lớn của data mesh là phi tập trung hóa, như Zhamak Dehghani đã lưu ý trong bài viết đột phá của bà về chủ đề này:

In order to decentralize the monolithic data platform, we need to reverse how we think about data, its locality, and ownership. Instead of flowing the data from domains into a centrally owned data lake or platform, domains need to host and serve their domain datasets in an easily consumable way. 12

Để phi tập trung hóa nền tảng dữ liệu monolithic, chúng ta cần đảo ngược cách chúng ta nghĩ về dữ liệu, vị trí của nó, và quyền sở hữu. Thay vì cho dữ liệu chảy từ các miền vào một hồ hoặc nền tảng dữ liệu thuộc sở hữu tập trung, các miền cần host và phục vụ các tập dữ liệu miền của họ theo một cách dễ tiêu thụ.

Dehghani later identified four key components of the data mesh: 13

Dehghani sau đó nhận diện bốn thành phần then chốt của data mesh:

Domain-oriented decentralized data ownership and architecture

Quyền sở hữu dữ liệu và kiến trúc phi tập trung, định hướng theo miền

Data as a product

Dữ liệu như một sản phẩm (Data as a product)

Self-serve data infrastructure as a platform

Hạ tầng dữ liệu tự phục vụ như một nền tảng

Federated computational governance

Quản trị tính toán liên kết (Federated computational governance)

Figure 3-18 shows a simplified version of a data mesh architecture, with the three domains interoperating.

Hình 3-18 cho thấy một phiên bản đơn giản hóa của một kiến trúc data mesh, với ba miền liên thông với nhau.

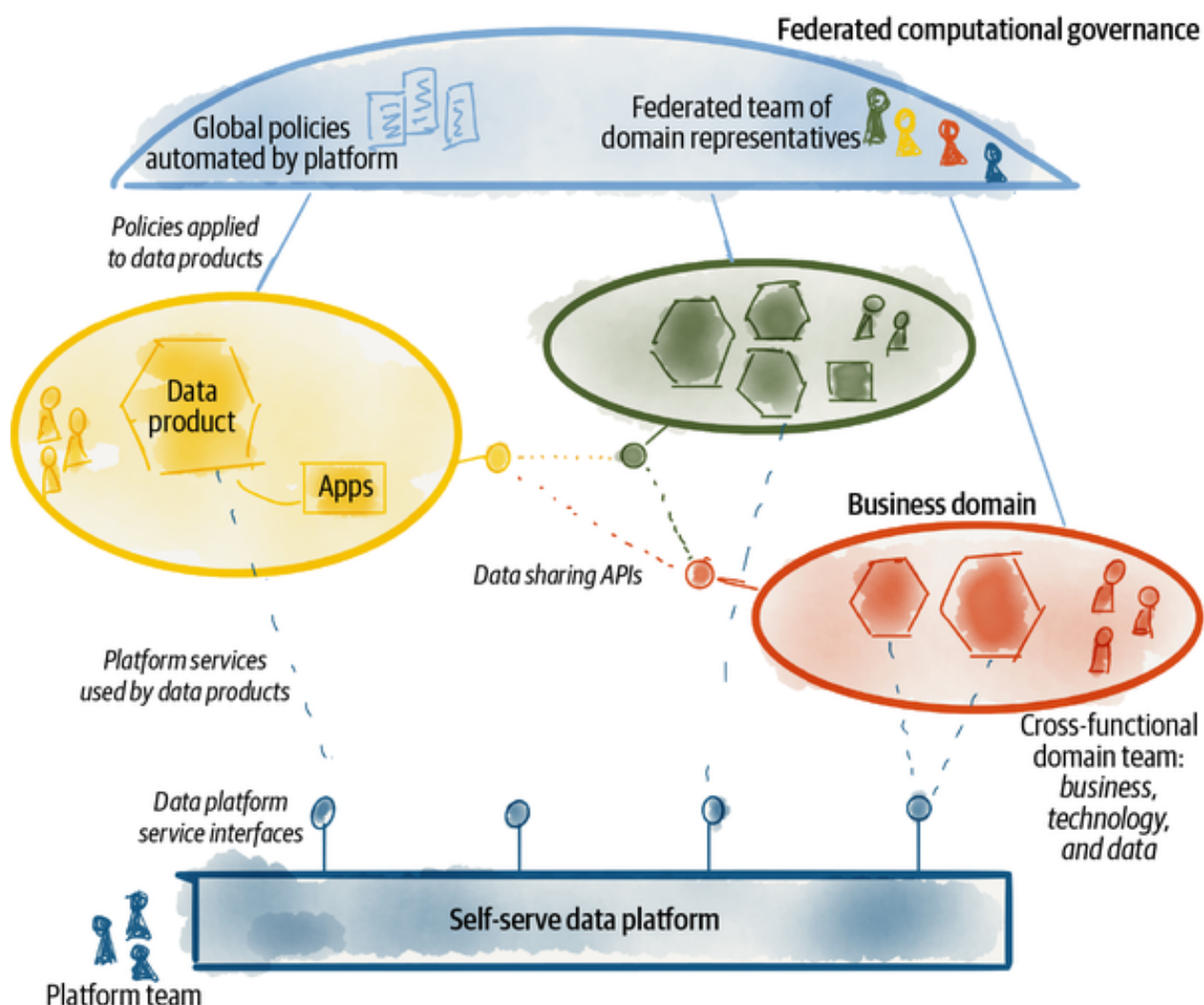


Figure 3-18. Simplified example of a data mesh architecture. Source: From Data Mesh, by Zhamak Dehghani. Copyright © 2022 Zhamak Dehghani. Published by O'Reilly Media, Inc. Used with permission. You can learn more about data mesh in Dehghani's book Data Mesh (O'Reilly).

Hình 3-18. Ví dụ đơn giản hóa về một kiến trúc data mesh. Nguồn: Từ Data Mesh, của Zhamak Dehghani. Bản quyền © 2022 Zhamak Dehghani. Xuất bản bởi O'Reilly Media, Inc. Dùng với sự cho phép. Bạn có thể tìm hiểu thêm về data mesh trong cuốn Data Mesh (O'Reilly) của Dehghani.

Other Data Architecture Examples — Các ví dụ kiến trúc dữ liệu khác

Data architectures have countless other variations, such as data fabric, data hub, scaled architecture, **metadata**-first architecture, event-driven architecture, live data stack (Chapter 11), and many more. And new architectures will continue to emerge as practices consolidate and mature, and tooling simplifies and improves. We've focused on a handful of the most critical data architecture patterns that are extremely well established, evolving rapidly, or both. As a data engineer, pay attention to how new architectures may help your organization. Stay abreast of new developments

by cultivating a high-level awareness of the data engineering ecosystem developments. Be open-minded and don't get emotionally attached to one approach. Once you've identified potential value, deepen your learning and make concrete decisions. When done right, minor tweaks—or major overhauls—in your data architecture can positively impact the business.

Các kiến trúc dữ liệu có vô số biến thể khác, chẳng hạn data fabric, data hub, kiến trúc mở rộng (scaled architecture), kiến trúc siêu-dữ-liệu-trước (metadata-first architecture), kiến trúc hướng sự kiện, live data stack (Chương 11), và còn nhiều nữa. Và các kiến trúc mới sẽ tiếp tục nổi lên khi các thực hành củng cố và trưởng thành, và công cụ đơn giản hóa và cải thiện. Chúng tôi đã tập trung vào một số ít các mẫu hình kiến trúc dữ liệu quan trọng nhất vốn được thiết lập cực kỳ vững chắc, đang tiến hóa nhanh chóng, hoặc cả hai. Là một kỹ sư dữ liệu, hãy chú ý đến cách các kiến trúc mới có thể giúp tổ chức của bạn. Hãy luôn cập nhật với các phát triển mới bằng cách nuôi dưỡng một nhận thức cấp cao về các phát triển của hệ sinh thái kỹ thuật dữ liệu. Hãy cởi mở và đừng gấn bó cảm xúc với một cách tiếp cận. Một khi bạn đã nhận diện được giá trị tiềm năng, hãy đào sâu việc học của mình và đưa ra những quyết định cụ thể. Khi được làm đúng, những điều chỉnh nhỏ — hoặc những cuộc đại tu lớn — trong kiến trúc dữ liệu của bạn có thể tác động tích cực đến doanh nghiệp.

Who's Involved with Designing a Data Architecture? — Ai tham gia vào việc thiết kế một kiến trúc dữ liệu?

Data architecture isn't designed in a vacuum. Bigger companies may still employ data architects, but those architects will need to be heavily in tune and current with the state of technology and data. Gone are the days of ivory tower data architecture. In the past, architecture was largely orthogonal to engineering. We expect this distinction will disappear as data engineering, and engineering in general, quickly evolves, becoming more agile, with less separation between engineering and architecture. Ideally, a data engineer will work alongside a dedicated data architect. However, if a company is small or low in its level of data maturity, a data engineer might do double duty as an architect. Because data architecture is an undercurrent of the data engineering lifecycle, a data engineer should understand “good” architecture and the various types of data architecture. When designing architecture, you'll work alongside business stakeholders to evaluate trade-offs. What are the trade-offs inherent in adopting a cloud data warehouse versus a data lake? What are the trade-offs of various cloud platforms? When might a unified batch/streaming framework (Beam, Flink) be an appropriate choice? Studying these choices in the abstract will prepare you to make concrete, valuable decisions.

Kiến trúc dữ liệu không được thiết kế trong chân không. Các công ty lớn hơn vẫn có thể tuyển dụng các kiến trúc sư dữ liệu, nhưng những kiến trúc sư đó sẽ cần hòa nhịp và cập nhật nhiều với tình trạng của công nghệ và dữ liệu. Đã qua rồi cái thời kiến trúc dữ liệu kiểu tháp ngà. Trong quá khứ, kiến trúc phần lớn trực giao với kỹ thuật. Chúng tôi kỳ vọng sự phân biệt này sẽ biến mất khi kỹ thuật dữ liệu, và kỹ thuật nói chung, nhanh chóng tiến hóa, trở nên nhanh nhạy hơn, với ít sự tách biệt hơn giữa kỹ thuật và kiến trúc. Lý tưởng nhất, một kỹ sư dữ liệu sẽ làm việc song song với một kiến trúc sư dữ liệu chuyên trách. Tuy nhiên, nếu một công ty nhỏ hoặc thấp về mức độ trưởng thành dữ liệu, một kỹ sư dữ liệu có thể đảm nhận hai vai trò vừa làm kỹ sư vừa làm kiến trúc sư. Vì kiến trúc dữ liệu là một dòng chảy ngầm của vòng đời kỹ thuật dữ liệu, một kỹ sư dữ liệu nên hiểu kiến trúc “tốt” và các loại kiến trúc dữ liệu khác nhau. Khi thiết kế kiến trúc, bạn sẽ làm việc song song với các bên liên quan kinh doanh để đánh giá các đánh đổi. Những đánh đổi vốn có trong việc áp dụng một kho dữ liệu đám mây so với một hồ dữ liệu là gì? Những đánh đổi

của các nền tảng đám mây khác nhau là gì? Khi nào một framework lô/luồng hợp nhất (Beam, Flink) có thể là một lựa chọn phù hợp? Nghiên cứu những lựa chọn này một cách trù tượng sẽ chuẩn bị cho bạn đưa ra những quyết định cụ thể, giá trị.

Conclusion — Kết luận

You’ve learned how data architecture fits into the data engineering lifecycle and what makes for “good” data architecture, and you’ve seen several examples of data architectures. Because architecture is such a key foundation for success, we encourage you to invest the time to study it deeply and understand the trade-offs inherent in any architecture. You will be prepared to map out architecture that corresponds to your organization’s unique requirements. Next up, let’s look at some approaches to choosing the right technologies to be used in data architecture and across the data engineering lifecycle.

Bạn đã học được cách kiến trúc dữ liệu khớp vào vòng đời kỹ thuật dữ liệu và điều gì làm nên kiến trúc dữ liệu “tốt”, và bạn đã thấy một vài ví dụ về các kiến trúc dữ liệu. Vì kiến trúc là một nền tảng then chốt cho sự thành công như vậy, chúng tôi khuyến khích bạn đầu tư thời gian để nghiên cứu nó sâu sắc và hiểu các đánh đổi vốn có trong bất kỳ kiến trúc nào. Bạn sẽ được chuẩn bị để vạch ra một kiến trúc tương ứng với các yêu cầu độ phức tạp của tổ chức bạn. Tiếp theo, hãy xem một số cách tiếp cận để chọn các công nghệ phù hợp được dùng trong kiến trúc dữ liệu và xuyên suốt vòng đời kỹ thuật dữ liệu.

Additional Resources

“Separating Utility from Value Add” by Ross Pettit

“Tactics vs. Strategy: SOA and the Tarpit of Irrelevancy” by Neal Ford

The Information Management Body of Knowledge website

“The Modern Data Stack: Past, Present, and Future” by Tristan Handy

“Potemkin Data Science” by Michael Correll

“A Comparison of Data Processing Frameworks” by Ludovik Santos

“Modern CI Is Too Complex and Misdirected” by Gregory Szorc

“Questioning the Lambda Architecture” by Jay Kreps

“End-to-End Serverless ETL Orchestration in AWS: A Guide” by Rittika Jindal

“A Brief Introduction to Two Data Processing Architectures—Lambda and Kappa for Big Data” by Iman Samizadeh

“How to Beat the Cap Theorem” by Nathan Marz “The Log: What Every Software Engineer Should Know About Real- Time Data’s Unifying Abstraction” by Jay Kreps

“Run Your Data Team Like a Product Team” by Emilie Schario and Taylor A. Murphy

“Data as a Product vs. Data as a Service” by Justin Gage

Martin Fowler articles:

“EagerReadDerivation”

“AnemicDomainModel”

“DomainDrivenDesign”

“Event Sourcing”

“ReportingDatabase”

“BoundedContext”

“Focusing on Events”

“Polyglot Persistence”

“UtilityVsStrategicDichotomy”

Data applications:

“Introducing Dagster: An Open Source Python Library for Building Data Applications” by Nick Schrock

“Down with Pipeline Debt: Introducing Great Expectations,” by the Great Expectations project

“Functional Data Engineering: A Modern Paradigm for Batch Data Processing” by Maxime Beauchemin

“What Is the Open Data Ecosystem and Why It’s Here to Stay” by Casber Wang “Staying Ahead of Data Debt” by Etai Mizrahi

“Choosing Open Wisely” by Benoit Dageville et al.

“The Ultimate Data Observability Checklist” by Molly Vorwerck

“Moving Beyond Batch vs. Streaming” by David Yaffe

“Disasters I’ve Seen in a Microservices World” by Joao Alves

Naming conventions:

“240 Tables and No Documentation?” by Alexey Makhotkin

“Data Warehouse Architecture: Overview” by Roelant Vos

“The Design and Implementation of Modern Column-Oriented Database Systems” by Daniel Abadi et al.

“Column-oriented DBMS” Wikipedia page

“The Data Dichotomy: Rethinking the Way We Treat Data and Services” by Ben Stopford

“The Curse of the Data Lake Monster” by Kiran Prakash and Lucy Chambers

“Software Infrastructure 2.0: A Wishlist” by Erik Bernhardsson

“Data Team Platform” by GitLab Data

“Falling Back in Love with Data Pipelines” by Sean Knapp

Build versus buy fallacy:

“What’s Wrong with MLOps?” by Laszlo Sragner

“Zachman Framework” Wikipedia page

“How TOGAF Defines Enterprise Architecture (EA)” by Avancier Limited

EABOK Draft , edited by Paula Hagan “What Is Data Architecture? A Framework for Managing Data” by Thor Olavsrud

“Google Cloud Architecture Framework” Google Cloud Architecture web page
 Microsoft’s “Azure Architecture Center”
 “Five Principles for Cloud-Native Architecture: What It Is and How to Master It” by Tom Grey
 “Choosing the Right Architecture for Global Data Distribution” Google Cloud Architecture web page
 “Principled Data Engineering, Part I: Architectural Overview” by Hussein Danish
 “A Personal Implementation of Modern Data Architecture: Getting Strava Data into Google Cloud Platform” by Matthew Reeve
 “The Cost of Cloud, a Trillion Dollar Paradox” by Sarah Wang and Martin Casado
 Data Architecture: A Primer for the Data Scientist by W.H. Inmon et al. (Academic Press)
 “Enterprise Architecture” Gartner Glossary definition
 EABOK website
 TOGAF framework website
 “Defining Architecture” ISO/IEC/IEEE 42010 web page
 “The Six Principles of Modern Data Architecture” by Joshua Klahr
 “The Rise of the Metadata Lake” by Prukalpa
 “The Top 5 Data Trends for CDOs to Watch Out for in 2021” by Prukalpa
 “Test Data Quality at Scale with Deequ” by Dustin Lange et al.
 “What the Heck Is Data Mesh” by Chris Riccomini
 “Reliable Microservices Data Exchange with the Outbox Pattern” by Gunnar Morling
 “Who Needs an Architect” by Martin Fowler
 “Enterprise Architecture’s Role in Building a Data-Driven Organization” by Ashutosh Gupta
 “How to Build a Data Architecture to Drive Innovation—Today and Tomorrow” by Antonio Castro et al.
 “Data Warehouse Architecture” tutorial at Javatpoint
 Snowflake’s “What Is Data Warehouse Architecture” web page
 “Three-Tier Architecture” by IBM Education
 “The Building Blocks of a Modern Data Platform” by Prukalpa
 “Data Architecture: Complex vs. Complicated” by Dave Wells
 “Data Fabric Defined” by James Serra
 “Data Fabric Architecture Is Key to Modernizing Data Management and Integration” by Ashutosh Gupta
 “Unified Analytics: Where Batch and Streaming Come Together; SQL and Beyond” Apache Flink Roadmap
 “What Is a Data Lakehouse?” by Ben Lorica et al.
 “Big Data Architectures” Azure documentation
 “Microsoft Azure IoT Reference Architecture” documentation

1 Jeff Haden, “Amazon Founder Jeff Bezos: This Is How Successful People Make Such Smart Decisions,” Inc. , December 3, 2018, <https://oreil.ly/QwIm0> . 2 Martin Fowler, “Who Needs an Architect?” IEEE Software , July/August 2003, <https://oreil.ly/wAMmZ> .

3 “The Bezos API Mandate: Amazon’s Manifesto for Externalization,” Nordic APIs, January 19, 2021, <https://oreil.ly/vIs8m> . 4 Fowler, “Who Needs an Architect?” 5 Ericka Chickowski, “Leaky Buckets: 10 Worst Amazon S3 Breaches,” Bitdefender Business Insights blog, Jan 24, 2018, <https://oreil.ly/pFEFO> . 6 “FinOps Foundation Soars to 300 Members and Introduces New Partner Tiers for Cloud Service Providers and Vendors,” Business Wire, June 17, 2019, <https://oreil.ly/XcwYO> . 7 Martin Fowler, “StranglerFigApplication,” June 29, 2004, <https://oreil.ly/PmqxB> . 8 Mike Loukides, “Resume Driven Development,” O’Reilly Radar , October 13, 2004, <https://oreil.ly/BUHa8> . 9 H.W. Inmon, Building the Data Warehouse (Hoboken: Wiley, 2005). 10 Jay Kreps, “Questioning the Lambda Architecture,” O’Reilly, July 2, 2014, <https://oreil.ly/wWR3n> . 11 Martin Fowler, “Data Mesh Principles and Logical Architecture,” MartinFowler.com, December 3, 2020, <https://oreil.ly/ezWE7> . 12 Zhamak Dehghani, “How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh,” MartinFowler.com, May 20, 2019, <https://oreil.ly/SqMe8> . 13 Zhamak Dehghani, “Data Mesh Principles,” MartinFowler.com , December 3, 2020, <https://oreil.ly/RymDi> .

Chapter 4. Choosing Technologies Across the Data Engineering Lifecycle — Chương 4. Lựa chọn công nghệ xuyên suốt vòng đời kỹ thuật dữ liệu

Data engineering nowadays suffers from an embarrassment of riches. We have no shortage of technologies to solve various types of data problems. Data technologies are available as turnkey offerings consumable in almost every way—open source, managed open source, proprietary software, proprietary service, and more. However, it’s easy to get caught up in chasing bleeding-edge technology while losing sight of the core purpose of data engineering: designing robust and reliable systems to carry data through the full lifecycle and serve it according to the needs of end users. Just as structural engineers carefully choose technologies and materials to realize an architect’s vision for a building, data engineers are tasked with making appropriate technology choices to shepherd data through the lifecycle to serve data applications and users. Chapter 3 discussed “good” **data architecture** and why it matters. We now explain how to choose the right technologies to serve this architecture. Data engineers must choose good technologies to make the best possible data product. We feel the criteria to choose a good data technology is simple: does it add value to a data product and the broader business? A lot of people confuse architecture and tools. Architecture is strategic ; tools are tactical . We sometimes hear, “Our data architecture are tools X , Y , and Z .” This is the wrong way to think about architecture. Architecture is the top-level design, roadmap, and blueprint of data systems that satisfy the strategic aims for the business. Architecture is the what , why , and when . Tools are used to make the architecture a reality; tools are the how . We often see teams going “off the rails” and choosing technologies before mapping out an architecture. The reasons vary: shiny object syndrome, resume-driven development, and a lack of expertise in architecture. In practice, this prioritization of technology often means they cobble together a kind of Dr. Suess fantasy machine rather than a true data architecture. We strongly advise against choosing technology before getting your architecture right. Architecture first, technology second. This chapter discusses our tactical plan for making technology choices once we have a strategic architecture blueprint. The following are some considerations for choosing data technologies across the **data engineering lifecycle**:

Kỹ thuật dữ liệu (data engineering) ngày nay đang chịu cảnh “giàu của đến mức bối rối”. Chúng ta không hề thiếu công nghệ để giải quyết đủ loại bài toán dữ liệu. Các công nghệ dữ liệu được cung cấp dưới dạng giải pháp “cắm là chạy”, có thể tiêu dùng theo gần như

mọi cách—mã nguồn mở, mã nguồn mở được quản lý, phần mềm độc quyền, dịch vụ độc quyền, và còn nhiều nữa. Tuy nhiên, ta rất dễ bị cuốn vào việc chạy theo công nghệ tối tân mà đánh mất mục đích cốt lõi của kỹ thuật dữ liệu: thiết kế những hệ thống mạnh mẽ và đáng tin cậy để đưa dữ liệu đi qua trọn vẹn vòng đời và phục vụ nó theo nhu cầu của người dùng cuối. Cũng như kỹ sư kết cấu phải cẩn trọng chọn công nghệ và vật liệu để hiện thực hóa tầm nhìn của kiến trúc sư cho một tòa nhà, kỹ sư dữ liệu (data engineer) có nhiệm vụ đưa ra những lựa chọn công nghệ phù hợp để dẫn dắt dữ liệu đi qua vòng đời nhằm phục vụ các ứng dụng và người dùng dữ liệu. Chương 3 đã bàn về kiến trúc dữ liệu (data architecture) “tốt” và tại sao nó quan trọng. Giờ đây chúng ta giải thích cách chọn đúng công nghệ để phục vụ kiến trúc ấy. Kỹ sư dữ liệu phải chọn công nghệ tốt để tạo ra sản phẩm dữ liệu tốt nhất có thể. Chúng tôi cho rằng tiêu chí để chọn một công nghệ dữ liệu tốt rất đơn giản: liệu nó có tăng thêm giá trị cho sản phẩm dữ liệu và cho doanh nghiệp rộng lớn hơn hay không? Nhiều người nhầm lẫn giữa kiến trúc và công cụ. Kiến trúc mang tính *chiến lược*; công cụ mang tính *chiến thuật*. Đôi khi chúng tôi nghe nói, “Kiến trúc dữ liệu của chúng tôi là các công cụ X, Y và Z.” Đây là cách nghĩ sai về kiến trúc. Kiến trúc là thiết kế ở tầng cao nhất, là lộ trình và bản thiết kế của các hệ thống dữ liệu nhằm thỏa mãn các mục tiêu chiến lược của doanh nghiệp. Kiến trúc là *cái gì, tại sao và khi nào*. Công cụ được dùng để biến kiến trúc thành hiện thực; công cụ là *bằng cách nào*. Chúng tôi thường thấy các nhóm “trật đường ray” và chọn công nghệ trước khi vạch ra kiến trúc. Lý do thì muôn hình vạn trạng: hội chứng thích đồ mới (shiny object syndrome), phát triển kiểu làm-đẹp-CV (resume-driven development), và sự thiếu chuyên môn về kiến trúc. Trên thực tế, việc ưu tiên công nghệ kiểu này thường có nghĩa là họ chấp vá nên một thứ máy móc hoang đường kiểu Dr. Seuss hơn là một kiến trúc dữ liệu đích thực. Chúng tôi mạnh mẽ khuyên không nên chọn công nghệ trước khi làm đúng kiến trúc. Kiến trúc trước, công nghệ sau. Chương này bàn về kế hoạch chiến thuật của chúng tôi cho việc đưa ra các lựa chọn công nghệ một khi đã có bản thiết kế kiến trúc chiến lược. Sau đây là một số điều cần cân nhắc khi chọn công nghệ dữ liệu xuyên suốt vòng đời kỹ thuật dữ liệu:

Team size and capabilities

Quy mô và năng lực của nhóm

Speed to market

Tốc độ ra thị trường

Interoperability

Khả năng tương tác (interoperability)

Cost optimization and business value

Tối ưu chi phí và giá trị kinh doanh

Today versus the future: **immutable** versus transitory technologies

Hôm nay so với tương lai: công nghệ bất biến (immutable) so với công nghệ tạm thời (transitory)

Location (cloud, on prem, hybrid cloud, multicloud)

Vị trí (cloud, on-premises, hybrid cloud, multicloud)

Build versus buy

Tự xây dựng so với mua (build versus buy)

Monolith versus modular

Monolith so với mô-đun hóa (modular)

Serverless versus servers

Serverless so với servers

Optimization, performance and the benchmark wars.

Tối ưu hóa, hiệu năng và các cuộc chiến benchmark.

The **undercurrents** of the data engineering lifecycle

Các dòng chảy ngầm (undercurrents) của vòng đời kỹ thuật dữ liệu

Team Size and Capabilities — Quy mô và năng lực của nhóm

The first thing you need to assess is your team's size and its capabilities with technology. Are you on a small team (perhaps a team of one) of people who are expected to wear many hats, or is the team large enough that people work in specialized roles? Will a handful of people be responsible for multiple stages of the data engineering lifecycle, or do people cover particular niches? Your team's size will influence the types of technologies you adopt. There is a continuum of simple to complex technologies, and a team's size roughly determines the amount of bandwidth your team can dedicate to complex solutions. We sometimes see small data teams read blog posts about a new cutting-edge technology at a giant tech company and then try to emulate these same extremely complex technologies and practices. We call this cargo-cult engineering, and it's generally a big mistake that consumes a lot of valuable time and money, often with little to nothing to show in return. Especially for small teams or teams with weaker technical chops, use as many managed and SaaS tools as possible, and dedicate your limited bandwidth to solving the complex problems that directly add value to the business. Take an inventory of your team's skills. Do people lean toward low-code tools, or do they favor code-first approaches? Are people strong in certain languages like Java, Python, or Go? Technologies are available to cater to every preference on the low-code to code-heavy spectrum. Again, we suggest sticking with technologies and workflows with which the team is familiar. We've seen data teams invest a lot of time in learning the shiny new data framework, only to never use it in production. Learning new technologies, languages, and tools is a considerable time investment, so make these investments wisely.

Điều đầu tiên bạn cần đánh giá là quy mô của nhóm và năng lực công nghệ của nó. Bạn ở trong một nhóm nhỏ (có lẽ chỉ một người) gồm những người được kỳ vọng phải kiêm nhiều vai, hay nhóm đủ lớn để mỗi người làm việc theo vai trò chuyên biệt? Liệu một dúm người sẽ chịu trách nhiệm cho nhiều giai đoạn của vòng đời kỹ thuật dữ liệu, hay mỗi người phụ trách một mảng hẹp riêng? Quy mô nhóm sẽ ảnh hưởng đến những loại công nghệ bạn áp dụng. Có một dải liên tục từ công nghệ đơn giản đến phức tạp, và quy mô nhóm đại khái quyết định lượng băng thông mà nhóm có thể dành cho các giải pháp phức tạp. Đôi khi chúng tôi thấy những nhóm dữ liệu nhỏ đọc các bài blog về một công nghệ tối tân mới mẻ ở một công ty công nghệ khổng lồ rồi cố bắt chước chính những công nghệ và thực hành cực kỳ phức tạp đó. Chúng tôi gọi đây là *kỹ thuật theo kiểu sùng bái hàng hóa* (cargo-cult engineering), và nó nhìn chung là một sai lầm lớn, tốn rất nhiều thời gian và tiền bạc quý giá, mà thường thì đổi lại được rất ít hoặc chẳng được gì. Đặc biệt với các nhóm nhỏ hoặc nhóm có nội lực kỹ thuật yếu hơn, hãy dùng càng nhiều công cụ được quản lý và SaaS càng tốt, và dành phần băng thông hữu hạn của bạn để giải quyết những bài toán phức tạp trực tiếp tăng thêm giá trị cho doanh nghiệp. Hãy kiểm kê các kỹ năng của nhóm. Mọi người thiên về công cụ ít-code (low-code), hay ưa cách tiếp cận code-first? Mọi người mạnh ở một số ngôn ngữ nhất định như Java, Python hay Go? Có sẵn

các công nghệ chiều theo mọi sở thích trên dải từ low-code đến nặng-code. Một lần nữa, chúng tôi gợi ý nên bám lấy những công nghệ và quy trình làm việc mà nhóm đã quen. Chúng tôi từng thấy các nhóm dữ liệu đầu tư rất nhiều thời gian học một framework dữ liệu mới bóng bẩy, rồi chẳng bao giờ dùng nó trong sản xuất. Học công nghệ, ngôn ngữ và công cụ mới là một khoản đầu tư thời gian đáng kể, vậy nên hãy đầu tư một cách khôn ngoan.

Speed to Market — Tốc độ ra thị trường

In technology, speed to market wins. This means choosing the right technologies that help you deliver features and data faster while maintaining high-quality standards and security. It also means working in a tight feedback loop of launching, learning, iterating, and making improvements. Perfect is the enemy of good. Some data teams will deliberate on technology choices for months or years without reaching any decisions. Slow decisions and output are the kiss of death to data teams. We've seen more than a few data teams dissolve for moving too slow and failing to deliver the value they were hired to produce. Deliver value early and often. As we've mentioned, use what works. Your team members will likely get better leverage with tools they already know. Avoid undifferentiated heavy lifting that engages your team in unnecessarily complex work that adds little to no value. Choose tools that help you move quickly, reliably, safely, and securely.

Trong công nghệ, tốc độ ra thị trường là yếu tố thắng cuộc. Điều này nghĩa là chọn đúng công nghệ giúp bạn giao tính năng và dữ liệu nhanh hơn trong khi vẫn duy trì tiêu chuẩn chất lượng cao và an ninh. Nó cũng nghĩa là làm việc trong một vòng phản hồi chặt chẽ gồm ra mắt, học hỏi, lặp lại, và cải tiến. Hoàn hảo là kẻ thù của tốt. Một số nhóm dữ liệu sẽ cân nhắc các lựa chọn công nghệ suốt nhiều tháng hoặc nhiều năm mà không đi đến quyết định nào. Quyết định và sản phẩm chậm chạp là nộ hôn thần chết đối với các nhóm dữ liệu. Chúng tôi đã thấy không ít nhóm dữ liệu tan rã vì hành động quá chậm và không giao được giá trị mà họ được thuê để tạo ra. Hãy giao giá trị sớm và thường xuyên. Như đã đề cập, hãy dùng cái gì hiệu quả. Các thành viên trong nhóm bạn nhiều khả năng phát huy đòn bẩy tốt hơn với những công cụ họ đã biết. Tránh những công việc nặng nhọc không tạo khác biệt (undifferentiated heavy lifting) khiến nhóm bạn sa vào những việc phức tạp một cách không cần thiết mà tạo ra ít hoặc không tạo ra giá trị. Hãy chọn những công cụ giúp bạn di chuyển nhanh, đáng tin cậy, an toàn và bảo mật.

Interoperability — Khả năng tương tác

Rarely will you use only one technology or system. When choosing a technology or system, you'll need to ensure that it interacts and operates with other technologies. Interoperability describes how various technologies or systems connect, exchange information, and interact. Let's say you're evaluating two technologies, A and B. How easily does technology A integrate with technology B when thinking about interoperability? This is often a spectrum of difficulty, ranging from seamless to time-intensive. Is seamless integration already baked into each product, making setup a breeze? Or do you need to do a lot of manual configuration to integrate these technologies? Often, vendors and open source projects will target specific platforms and systems to interoperate. Most data **ingestion** and visualization tools have built-in integrations with popular data warehouses and data lakes. Furthermore, popular data-ingestion tools will integrate with common APIs and services, such as CRMs, accounting software, and more. Sometimes standards are in place for interoperability. Almost all databases allow connections via Java Database Connectivity (JDBC) or Open Database Connectivity (ODBC), meaning that you can easily connect to a database by using these standards.

In other cases, interoperability occurs in the absence of standards. Representational state transfer (REST) is not truly a standard for APIs; every REST API has its quirks. In these cases, it's up to the vendor or open source software (OSS) project to ensure smooth integration with other technologies and systems. Always be aware of how simple it will be to connect your various technologies across the data engineering lifecycle. As mentioned in other chapters, we suggest designing for modularity and giving yourself the ability to easily swap out technologies as new practices and alternatives become available.

Hiếm khi bạn chỉ dùng một công nghệ hay một hệ thống. Khi chọn một công nghệ hay hệ thống, bạn sẽ cần bảo đảm rằng nó tương tác và vận hành được với các công nghệ khác. Khả năng tương tác (interoperability) mô tả cách các công nghệ hoặc hệ thống khác nhau kết nối, trao đổi thông tin và tương tác với nhau. Giả sử bạn đang đánh giá hai công nghệ, A và B. Khi nghĩ về khả năng tương tác, công nghệ A tích hợp với công nghệ B dễ dàng đến mức nào? Đây thường là một dải mức độ khó, trải từ liền mạch đến tốn nhiều thời gian. Liệu tích hợp liền mạch đã được tích hợp sẵn vào từng sản phẩm, khiến việc thiết lập trở nên dễ như bỡn? Hay bạn cần làm rất nhiều cấu hình thủ công để tích hợp các công nghệ này? Nhà cung cấp và các dự án mã nguồn mở thường nhắm tới những nền tảng và hệ thống cụ thể để tương tác. Hầu hết các công cụ thu nạp (ingestion) và trực quan hóa dữ liệu đều có tích hợp sẵn với các kho dữ liệu (data warehouse) và hồ dữ liệu (data lake) phổ biến. Hơn nữa, các công cụ thu nạp dữ liệu phổ biến sẽ tích hợp với những API và dịch vụ thông dụng, như các hệ CRM, phần mềm kế toán, và nhiều hơn nữa. Đôi khi đã có sẵn các tiêu chuẩn cho khả năng tương tác. Gần như mọi cơ sở dữ liệu (database) đều cho phép kết nối qua Java Database Connectivity (JDBC) hoặc Open Database Connectivity (ODBC), nghĩa là bạn có thể dễ dàng kết nối tới một cơ sở dữ liệu bằng các tiêu chuẩn này. Trong các trường hợp khác, khả năng tương tác diễn ra khi không có tiêu chuẩn. Representational state transfer (REST) thực chất không phải là một tiêu chuẩn cho API; mỗi REST API có những điểm kỳ quặc riêng. Trong những trường hợp này, việc đảm bảo tích hợp trơn tru với các công nghệ và hệ thống khác tùy thuộc vào nhà cung cấp hoặc dự án phần mềm mã nguồn mở (OSS). Hãy luôn lưu tâm đến mức độ đơn giản của việc kết nối các công nghệ khác nhau của bạn xuyên suốt vòng đời kỹ thuật dữ liệu. Như đã đề cập trong các chương khác, chúng tôi gợi ý thiết kế hướng mô-đun và trao cho bản thân khả năng dễ dàng thay thế công nghệ khi có các thực hành và lựa chọn thay thế mới xuất hiện.

Cost Optimization and Business Value — Tối ưu chi phí và giá trị kinh doanh

In a perfect world, you'd get to experiment with all the latest, coolest technologies without considering cost, time investment, or value added to the business. In reality, budgets and time are finite, and the cost is a major constraint for choosing the right data architectures and technologies. Your organization expects a positive ROI from your data projects, so you must understand the basic costs you can control. Technology is a major cost driver, so your technology choices and management strategies will significantly impact your budget. We look at costs through three main lenses: **total cost of ownership**, **opportunity cost**, and **FinOps**.

Trong một thế giới hoàn hảo, bạn sẽ được thử nghiệm tất cả những công nghệ mới nhất, ngẫu nhất mà không phải bận tâm về chi phí, thời gian đầu tư hay giá trị tăng thêm cho doanh nghiệp. Trên thực tế, ngân sách và thời gian là hữu hạn, và chi phí là một ràng buộc lớn cho việc chọn đúng kiến trúc và công nghệ dữ liệu. Tổ chức của bạn kỳ vọng một ROI dương từ các dự án dữ liệu của bạn, vậy nên bạn phải hiểu những chi phí cơ bản mà bạn có thể kiểm soát. Công nghệ là một tác nhân chi phí lớn, vậy nên các lựa chọn công

nghe và chiến lược quản lý của bạn sẽ tác động đáng kể đến ngân sách. Chúng tôi nhìn chi phí qua ba lăng kính chính: tổng chi phí sở hữu (total cost of ownership), chi phí cơ hội (opportunity cost), và FinOps.

Total Cost of Ownership — Tổng chi phí sở hữu

Total cost of ownership (TCO) is the total estimated cost of an initiative, including the direct and indirect costs of products and services utilized. Direct costs can be directly attributed to an initiative. Examples are the salaries of a team working on the initiative or the AWS bill for all services consumed. Indirect costs, also known as overhead, are independent of the initiative and must be paid regardless of where they're attributed. Apart from direct and indirect costs, how something is purchased impacts the way costs are accounted for. Expenses fall into two big groups: capital expenses and operational expenses. Capital expenses, also known as capex, require an up-front investment. Payment is required today. Before the cloud existed, companies would typically purchase hardware and software up front through large acquisition contracts. In addition, significant investments were required to host hardware in server rooms, data centers, and colocation facilities. These up-front investments—commonly hundreds of thousands to millions of dollars or more—would be treated as assets and slowly depreciate over time. From a budget perspective, capital was required to fund the entire purchase. This is capex, a significant capital outlay with a long-term plan to achieve a positive ROI on the effort and expense put forth. Operational expenses, also known as opex, are the opposite of capex in certain respects. Opex is gradual and spread out over time. Whereas capex is long-term focused, opex is short-term. Opex can be pay-as-you-go or similar and allows a lot of flexibility. Opex is closer to a direct cost, making it easier to attribute to a data project. Until recently, opex wasn't an option for large data projects. Data systems often required multimillion-dollar contracts. This has changed with the advent of the cloud, as **data platform** services allow engineers to pay on a consumption-based model. In general, opex allows for a far greater ability for engineering teams to choose their software and hardware. Cloud-based services let data engineers iterate quickly with various software and technology configurations, often inexpensively. Data engineers need to be pragmatic about flexibility. The data landscape is changing too quickly to invest in long-term hardware that inevitably goes stale, can't easily scale, and potentially hampers a **data engineer's** flexibility to try new things. Given the upside for flexibility and low initial costs, we urge data engineers to take an opex-first approach centered on the cloud and flexible, pay-as-you-go technologies.

Tổng chi phí sở hữu (total cost of ownership, TCO) là tổng chi phí ước tính của một sáng kiến, bao gồm cả chi phí trực tiếp lẫn gián tiếp của các sản phẩm và dịch vụ được sử dụng. *Chi phí trực tiếp* có thể được quy trực tiếp cho một sáng kiến. Ví dụ là lương của nhóm làm việc cho sáng kiến đó hoặc hóa đơn AWS cho tất cả dịch vụ đã tiêu dùng. *Chi phí gián tiếp*, còn gọi là *chi phí chung* (overhead), độc lập với sáng kiến và phải được trả bất kể chúng được quy về đâu. Ngoài chi phí trực tiếp và gián tiếp, cách mua một thứ gì đó cũng tác động đến cách hạch toán chi phí. Chi phí rơi vào hai nhóm lớn: chi phí vốn (capital expenses) và chi phí vận hành (operational expenses). *Chi phí vốn*, còn gọi là *capex*, đòi hỏi một khoản đầu tư trả trước. Phải thanh toán *ngay hôm nay*. Trước khi có cloud, các công ty thường mua phần cứng và phần mềm trả trước thông qua các hợp đồng mua sắm lớn. Thêm vào đó, cần những khoản đầu tư đáng kể để đặt phần cứng trong phòng máy chủ, trung tâm dữ liệu và các cơ sở colocation. Những khoản đầu tư trả trước này—thường lên tới hàng trăm nghìn đến hàng triệu đô la hoặc hơn—sẽ được coi là tài sản và khấu hao chậm rãi theo thời gian. Từ góc nhìn ngân sách, cần vốn để tài trợ cho toàn bộ thương vụ. Đó chính là capex, một khoản chi vốn đáng kể với một kế hoạch dài hạn để đạt ROI dương trên công sức và chi phí đã bỏ ra. *Chi phí vận hành*, còn gọi là *opex*, đối lập với capex ở một số

khía cạnh nhất định. Opex diễn ra dần dần và trải đều theo thời gian. Trong khi capex tập trung vào dài hạn, opex tập trung vào ngắn hạn. Opex có thể theo kiểu trả-theo-sử-dụng (pay-as-you-go) hoặc tương tự và cho phép rất nhiều linh hoạt. Opex gần với chi phí trực tiếp hơn, khiến nó dễ quy về một dự án dữ liệu hơn. Cho đến gần đây, opex không phải là một lựa chọn cho các dự án dữ liệu lớn. Các hệ thống dữ liệu thường đòi hỏi những hợp đồng nhiều triệu đô la. Điều này đã thay đổi với sự ra đời của cloud, khi các dịch vụ nền tảng dữ liệu cho phép kỹ sư thanh toán theo mô hình dựa trên mức tiêu dùng. Nhìn chung, opex cho phép các nhóm kỹ thuật có khả năng lớn hơn nhiều trong việc chọn phần mềm và phần cứng của mình. Các dịch vụ trên nền cloud cho phép kỹ sư dữ liệu lập nhanh với đủ loại cấu hình phần mềm và công nghệ, thường là với chi phí rẻ. Kỹ sư dữ liệu cần thực dụng về sự linh hoạt. Bối cảnh dữ liệu đang thay đổi quá nhanh đến mức không nên đầu tư vào phần cứng dài hạn—thứ chắc chắn sẽ lỗi thời, không dễ mở rộng quy mô, và có thể cản trở sự linh hoạt của kỹ sư dữ liệu trong việc thử những điều mới. Vì lợi thế về linh hoạt và chi phí ban đầu thấp, chúng tôi thúc giục các kỹ sư dữ liệu áp dụng cách tiếp cận opex-first xoay quanh cloud và các công nghệ linh hoạt, trả-theo-sử-dụng.

Total Opportunity Cost of Ownership — Tổng chi phí cơ hội của sở hữu

Any choice inherently excludes other possibilities. Total opportunity cost of ownership (TOCO) is the cost of lost opportunities that we incur in choosing a technology, an architecture, or a process. Note that ownership 1 in this setting doesn't require long-term purchases of hardware or licenses.

Bất kỳ lựa chọn nào về bản chất cũng loại trừ những khả năng khác. Tổng chi phí cơ hội của sở hữu (total opportunity cost of ownership, TOCO) là chi phí của những cơ hội bị mất mà ta phải gánh khi chọn một công nghệ, một kiến trúc, hay một quy trình. Lưu ý rằng *sở hữu 1* trong bối cảnh này không đòi hỏi việc mua phần cứng hay giấy phép dài hạn.

Even in a cloud environment, we effectively own a technology, a stack, or a pipeline once it becomes a core part of our production data processes and is difficult to move away from. Data engineers often fail to evaluate TOCO when undertaking a new project; in our opinion, this is a massive blind spot. If you choose data stack A, you've chosen the benefits of data stack A over all other options, effectively excluding data stacks B, C, and D. You're committed to data stack A and everything it entails—the team to support it, training, setup, and maintenance. What happens if data stack A was a poor choice? What happens when data stack A becomes obsolete? Can you still move to data stack B? How quickly and cheaply can you move to something newer and better? This is a critical question in the data space, where new technologies and products seem to appear at an ever-faster rate. Does the expertise you've built up on data stack A translate to the next wave? Or are you able to swap out components of data stack A and buy yourself some time and options? The first step to minimizing opportunity cost is evaluating it with eyes wide open. We've seen countless data teams get stuck with technologies that seemed good at the time and are either not flexible for future growth or simply obsolete. Inflexible data technologies are a lot like bear traps. They're easy to get into and extremely painful to escape.

Ngay cả trong một môi trường cloud, chúng ta về thực chất *sở hữu* một công nghệ, một ngăn xếp, hay một đường ống (pipeline) một khi nó trở thành một phần cốt lõi của các quy trình dữ liệu sản xuất của ta và khó để rời bỏ. Kỹ sư dữ liệu thường không đánh giá TOCO khi bắt tay vào một dự án mới; theo ý chúng tôi, đây là một điểm mù khổng lồ. Nếu bạn chọn ngăn xếp dữ liệu A, bạn đã chọn các lợi ích của ngăn xếp dữ liệu A hơn mọi lựa chọn khác, về thực chất là loại trừ các ngăn xếp dữ liệu B, C và D. Bạn đã cam kết với ngăn

xếp dữ liệu A và mọi thứ nó kéo theo—đội ngũ để hỗ trợ nó, đào tạo, thiết lập và bảo trì. Điều gì xảy ra nếu ngăn xếp dữ liệu A là một lựa chọn tồi? Điều gì xảy ra khi ngăn xếp dữ liệu A trở nên lỗi thời? Liệu bạn còn có thể chuyển sang ngăn xếp dữ liệu B không? Bạn có thể chuyển sang một thứ mới hơn và tốt hơn nhanh đến đâu và rẻ đến đâu? Đây là một câu hỏi then chốt trong lĩnh vực dữ liệu, nơi các công nghệ và sản phẩm mới dường như xuất hiện với tốc độ ngày càng nhanh. Liệu chuyên môn bạn đã tích lũy về ngăn xếp dữ liệu A có chuyển dịch được sang làn sóng kế tiếp không? Hay bạn có thể thay thế các thành phần của ngăn xếp dữ liệu A và mua cho mình chút thời gian cùng các lựa chọn? Bước đầu tiên để giảm thiểu chi phí cơ hội là đánh giá nó với đôi mắt mở to. Chúng tôi đã thấy vô số nhóm dữ liệu mắc kẹt với những công nghệ trông có vẻ tốt vào thời điểm đó nhưng hoặc không linh hoạt cho sự tăng trưởng trong tương lai hoặc đơn giản là lỗi thời. Các công nghệ dữ liệu thiếu linh hoạt giống hệt những cái bẫy gấu. Chúng dễ sa vào nhưng cực kỳ đau đớn để thoát ra.

FinOps — FinOps

We already touched on FinOps in “Principle 9: Embrace FinOps” . As we’ve discussed, typical cloud spending is inherently opex: companies pay for services to run critical data processes rather than making up-front purchases and clawing back value over time. The goal of FinOps is to fully operationalize financial accountability and business value by applying the DevOps-like practices of monitoring and dynamically adjusting systems. In this chapter, we want to emphasize one thing about FinOps that is well embodied in this quote:

Chúng ta đã chạm tới FinOps trong “Nguyên tắc 9: Đón nhận FinOps”. Như đã bàn, chỉ tiêu cloud điển hình về bản chất là opex: các công ty trả tiền cho dịch vụ để chạy các quy trình dữ liệu trọng yếu thay vì mua trả trước rồi thu hồi giá trị theo thời gian. Mục tiêu của FinOps là vận hành hóa hoàn toàn trách nhiệm tài chính và giá trị kinh doanh bằng cách áp dụng các thực hành kiểu DevOps là giám sát và điều chỉnh hệ thống một cách động. Trong chương này, chúng tôi muốn nhấn mạnh một điều về FinOps được thể hiện rõ trong câu trích dẫn sau:

If it seems that FinOps is about saving money, then think again. FinOps is about making money. Cloud spend can drive more revenue, signal customer base growth, enable more product and feature release velocity, or even help shut down a data center. 2

Nếu có vẻ như FinOps là chuyện tiết kiệm tiền, thì hãy nghĩ lại. FinOps là chuyện kiếm tiền. Chỉ tiêu cloud có thể thúc đẩy nhiều doanh thu hơn, báo hiệu sự tăng trưởng tệp khách hàng, cho phép tăng tốc độ phát hành sản phẩm và tính năng, hoặc thậm chí giúp đóng cửa một trung tâm dữ liệu. 2

In our setting of data engineering, the ability to iterate quickly and scale dynamically is invaluable for creating business value. This is one the major motivations for shifting data workloads to the cloud.

Trong bối cảnh kỹ thuật dữ liệu của chúng tôi, khả năng lặp nhanh và mở rộng quy mô một cách động là vô giá để tạo ra giá trị kinh doanh. Đây là một trong những động lực chính của việc chuyển các tải công việc (workload) dữ liệu lên cloud.

Today Versus the Future: Immutable Versus Transitory Technologies — Hôm nay so với tương lai: công nghệ bất biến so với công nghệ tạm thời

In an exciting domain like data engineering, it's all too easy to focus on a rapidly evolving future while ignoring the concrete needs of the present. The intention to build a better future is noble but often leads to overarchitecting and overengineering. Tooling chosen for the future may be stale and out-of-date when this future arrives; the future frequently looks little like what we envisioned years before. As many life coaches would tell you, focus on the present. You should choose the best technology for the moment and near future, but in a way that supports future unknowns and evolution. Ask yourself: where are you today, and what are your goals for the future? Your answers to these questions should inform your decisions about your architecture and thus the technologies used within that architecture. This is done by understanding what is likely to change and what tends to stay the same. We have two classes of tools to consider: immutable and transitory. Immutable technologies might be components that underpin the cloud or languages and paradigms that have stood the test of time. In the cloud, examples of immutable technologies are **object storage**, networking, servers, and security. Object **storage** such as Amazon S3 and Azure Blob Storage will be around from today until the end of the decade, and probably much longer. Storing your data in object storage is a wise choice. Object storage continues to improve in various ways and constantly offers new options, but your data will be safe and usable in object storage regardless of the rapid evolution of technology. For languages, SQL and bash have been around for many decades, and we don't see them disappearing anytime soon. Immutable technologies benefit from the Lindy effect: the longer a technology has been established, the longer it will be used. Think of the power grid, relational databases, the C programming language, or the x86 processor architecture. We suggest applying the Lindy effect as a litmus test to determine whether a technology is potentially immutable. Transitory technologies are those that come and go. The typical trajectory begins with a lot of hype, followed by meteoric growth in popularity, then a slow descent into obscurity. The JavaScript frontend landscape is a classic example. How many JavaScript frontend frameworks have come and gone between 2010 and 2020? Backbone.js, Ember.js, and Knockout were popular in the early 2010s, AngularJS in the mid-2010s, and React and Vue.js have massive mindshare today. What's the popular frontend framework three years from now? Who knows. New well-funded entrants and open source projects arrive on the data front every day. Every vendor will say their product will change the industry and "make the world a better place". Most of these companies and projects don't get long-term traction and fade slowly into obscurity. Top VCs are making big-money bets, knowing that most of their data-tooling investments will fail. How can you possibly know which technologies to invest in for your data architecture? It's hard. Just consider the number of technologies in Matt Turck's (in)famous depictions of the ML, AI, and data (MAD) landscape that we introduced in Chapter 1 (Figure 4-1).

Trong một lĩnh vực thú vị như kỹ thuật dữ liệu, người ta rất dễ tập trung vào một tương lai biến đổi nhanh chóng mà phớt lờ các nhu cầu cụ thể của hiện tại. Ý định xây dựng một tương lai tốt đẹp hơn là cao quý nhưng thường dẫn đến việc thiết kế quá mức (overarchitecting) và làm kỹ thuật quá mức (overengineering). Công cụ được chọn cho tương lai có thể đã cũ kỹ và lỗi thời khi tương lai ấy đến; tương lai thường ít giống với những gì ta đã hình dung từ nhiều năm trước. Như nhiều huấn luyện viên cuộc sống sẽ nói với bạn, hãy tập trung vào hiện tại. Bạn nên chọn công nghệ tốt nhất cho thời điểm này và tương lai gần, nhưng theo cách hỗ trợ những điều chưa biết và sự tiến hóa trong tương lai. Hãy tự hỏi: bạn đang ở đâu hôm nay, và mục tiêu của bạn cho tương lai là gì? Câu trả lời của bạn cho những câu hỏi này nên định hình các quyết định về kiến trúc của bạn và do đó là các công nghệ được dùng trong kiến trúc ấy. Việc này được thực hiện

bằng cách hiểu cái gì có khả năng thay đổi và cái gì có xu hướng giữ nguyên. Chúng ta có hai lớp công cụ cần cân nhắc: bất biến (immutable) và tạm thời (transitory). Các công nghệ bất biến có thể là những thành phần nền tảng của cloud hoặc những ngôn ngữ và mô thức (paradigm) đã trụ vững qua thử thách của thời gian. Trong cloud, ví dụ về công nghệ bất biến là lưu trữ đối tượng (object storage), mạng, máy chủ và bảo mật. Lưu trữ đối tượng như Amazon S3 và Azure Blob Storage sẽ vẫn còn đó từ hôm nay đến cuối thập kỷ, và có lẽ còn lâu hơn nữa. Lưu trữ dữ liệu của bạn trong lưu trữ đối tượng là một lựa chọn khôn ngoan. Lưu trữ đối tượng tiếp tục cải thiện theo nhiều cách và liên tục cung cấp các lựa chọn mới, nhưng dữ liệu của bạn sẽ an toàn và dùng được trong lưu trữ đối tượng bất kể công nghệ tiến hóa nhanh đến đâu. Về ngôn ngữ, SQL và bash đã tồn tại nhiều thập kỷ, và chúng tôi không thấy chúng biến mất sớm. Các công nghệ bất biến hưởng lợi từ hiệu ứng Lindy: một công nghệ đã được thiết lập càng lâu thì nó sẽ còn được dùng càng lâu. Hãy nghĩ đến lưới điện, cơ sở dữ liệu quan hệ, ngôn ngữ lập trình C, hay kiến trúc bộ xử lý x86. Chúng tôi gợi ý áp dụng hiệu ứng Lindy như một phép thử quý hiếm để xác định liệu một công nghệ có tiềm năng bất biến hay không. Các công nghệ tạm thời là những thứ đến rồi đi. Quỹ đạo điển hình bắt đầu với rất nhiều cường điệu, theo sau là sự tăng trưởng độ phổ biến như sao băng, rồi từ từ chìm vào quên lãng. Bối cảnh frontend JavaScript là một ví dụ kinh điển. Bao nhiêu framework frontend JavaScript đã đến rồi đi giữa năm 2010 và 2020? Backbone.js, Ember.js và Knockout phổ biến vào đầu thập niên 2010, AngularJS vào giữa thập niên 2010, và React cùng Vue.js có sức ảnh hưởng khổng lồ ngày nay. Framework frontend phổ biến ba năm nữa sẽ là gì? Ai mà biết. Mỗi ngày lại có những tân binh được rót vốn dồi dào và các dự án mã nguồn mở xuất hiện trên mặt trận dữ liệu. Nhà cung cấp nào cũng sẽ nói sản phẩm của họ sẽ thay đổi ngành và “khiến thế giới tốt đẹp hơn”. Hầu hết các công ty và dự án này không giành được đà phát triển dài hạn và từ từ chìm vào quên lãng. Các quỹ đầu tư mạo hiểm hàng đầu đang đặt những ván cược lớn, dù biết rằng phần lớn các khoản đầu tư vào công cụ dữ liệu của họ sẽ thất bại. Làm sao bạn có thể biết được nên đầu tư vào công nghệ nào cho kiến trúc dữ liệu của mình? Khó lắm. Hãy thử nhìn số lượng công nghệ trong các bức tranh mô tả (tai) tiếng của Matt Turck về bối cảnh ML, AI và dữ liệu (MAD) mà chúng tôi đã giới thiệu ở Chương 1 (Hình 4-1).

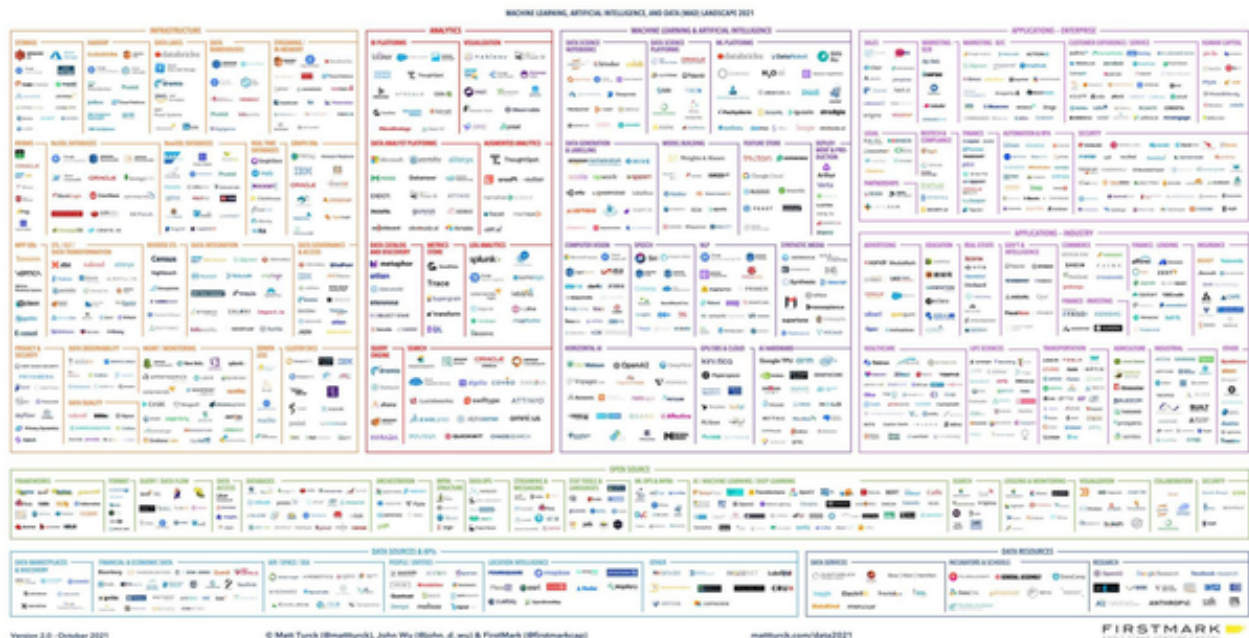


Figure 4-1. Matt Turck's 2021 MAD data landscape Even relatively successful technologies often fade into obscurity quickly, after a few years of rapid adoption, a victim of their success. For instance, in the early 2010s, Hive was met with rapid uptake because it allowed both analysts and engineers to query massive datasets without coding complex MapReduce jobs by hand. Inspired by the success of Hive but wishing to improve on its shortcomings, engineers developed Presto and other technologies. Hive now appears primarily in legacy deployments.

Hình 4-1. Bối cảnh dữ liệu MAD năm 2021 của Matt Turck. Ngay cả những công nghệ tương đối thành công cũng thường nhanh chóng chìm vào quên lãng, sau vài năm được áp dụng chóng mặt, trở thành nạn nhân của chính thành công của mình. Chẳng hạn, vào đầu thập niên 2010, Hive được đón nhận nhanh chóng vì nó cho phép cả nhà phân tích lẫn kỹ sư truy vấn những tập dữ liệu khổng lồ mà không cần viết tay các tác vụ MapReduce phức tạp. Lấy cảm hứng từ thành công của Hive nhưng mong muốn cải thiện những khiếm khuyết của nó, các kỹ sư đã phát triển Presto cùng các công nghệ khác. Hive giờ đây chủ yếu xuất hiện trong các triển khai cũ (legacy).

Our Advice — Lời khuyên của chúng tôi

Given the rapid pace of tooling and best-practice changes, we suggest evaluating tools every two years (Figure 4-2). Whenever possible, find the immutable technologies along the data engineering lifecycle, and use those as your base. Build transitory tools around the immutables.

Trước tốc độ thay đổi nhanh chóng của công cụ và thực hành tốt nhất, chúng tôi gợi ý đánh giá lại công cụ mỗi hai năm (Hình 4-2). Bất cứ khi nào có thể, hãy tìm ra các công nghệ bất biến dọc theo vòng đời kỹ thuật dữ liệu, và dùng chúng làm nền tảng. Xây các công cụ tạm thời xoay quanh những thứ bất biến.

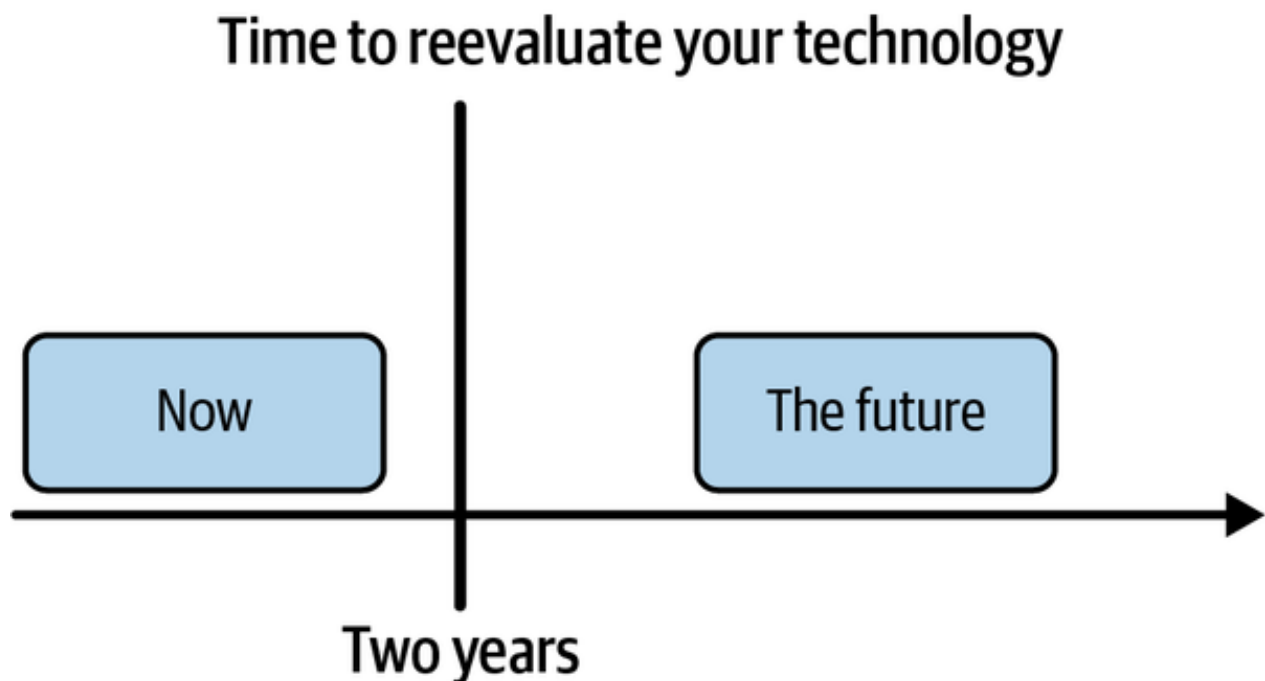


Figure 4-2. Use a two-year time horizon to reevaluate your technology choices Given the reasonable probability of failure for many data technologies, you need to consider how easy it is to transition from a chosen technology. What are the barriers to leaving? As mentioned previously in our

discussion about opportunity cost, avoid “bear traps.” Go into a new technology with eyes wide open, knowing the project might get abandoned, the company may not be viable, or the technology simply isn’t a good fit any longer.

Hình 4-2. Dùng tầm nhìn thời gian hai năm để đánh giá lại các lựa chọn công nghệ của bạn. Xét xác suất thất bại hợp lý của nhiều công nghệ dữ liệu, bạn cần cân nhắc xem chuyển khỏi một công nghệ đã chọn dễ đến đâu. Đây là các rào cản để rời đi? Như đã đề cập trước đó trong phần thảo luận về chi phí cơ hội, hãy tránh các “bẫy gấu”. Hãy bước vào một công nghệ mới với đôi mắt mở to, biết rằng dự án có thể bị bỏ rơi, công ty có thể không trụ nổi, hoặc công nghệ đơn giản là không còn phù hợp nữa.

Location — Vị trí

Companies now have numerous options when deciding where to run their technology stacks. A slow shift toward the cloud culminates in a veritable stampede of companies spinning up workloads on AWS, Azure, and Google Cloud Platform (GCP). In the last decade, many CTOs have come to view their decisions around technology hosting as having existential significance for their organizations. If they move too slowly, they risk being left behind by their more agile competition; on the other hand, a poorly planned cloud migration could lead to technological failure and catastrophic costs. Let’s look at the principal places to run your technology stack: on premises, the cloud, hybrid cloud, and multicloud.

Các công ty giờ đây có vô số lựa chọn khi quyết định nơi chạy các ngăn xếp công nghệ của mình. Một sự dịch chuyển chậm rãi hướng về cloud lên đến cao trào với cảnh tượng các công ty ồ ồ khởi chạy tải công việc trên AWS, Azure và Google Cloud Platform (GCP). Trong thập kỷ qua, nhiều CTO đã đi đến chỗ xem các quyết định của họ về nơi đặt công nghệ là mang ý nghĩa sống còn đối với tổ chức của mình. Nếu họ di chuyển quá chậm, họ có nguy cơ bị bỏ lại phía sau bởi các đối thủ nhanh nhẹn hơn; mặt khác, một cuộc di trú lên cloud được hoạch định kém có thể dẫn đến thất bại công nghệ và những chi phí thảm khốc. Hãy nhìn vào những nơi chính để chạy ngăn xếp công nghệ của bạn: on-premises, cloud, hybrid cloud và multicloud.

On Premises — On Premises

While new startups are increasingly born in the cloud, on-premises systems are still the default for established companies. Essentially, these companies own their hardware, which may live in data centers they own or in leased colocation space. In either case, companies are operationally responsible for their hardware and the software that runs on it. If hardware fails, they have to repair or replace it. They also have to manage upgrade cycles every few years as new, updated hardware is released and older hardware ages and becomes less reliable. They must ensure that they have enough hardware to handle peaks; for an online retailer, this means hosting enough capacity to handle the load spikes of Black Friday. For data engineers in charge of on-premises systems, this means buying large-enough systems to allow good performance for peak load and large jobs without overbuying and overspending. On the one hand, established companies have established operational practices that have served them well. Suppose a company that relies on information technology has been in business for some time. This means it has managed to juggle the cost and personnel requirements of running its hardware, managing software environments, deploying code from dev teams, and running databases and big data systems. On the other hand, established companies see their younger, more agile competition **scaling** rapidly and taking

advantage of cloud-managed services. They also see established competitors making forays into the cloud, allowing them to temporarily scale up enormous computing power for massive data jobs or the Black Friday shopping spike. Companies in competitive sectors generally don't have the option to stand still. Competition is fierce, and there's always the threat of being "disrupted" by more agile competition, backed by a large pile of venture capital dollars. Every company must keep its existing systems running efficiently while deciding what moves to make next. This could involve adopting newer DevOps practices, such as containers, Kubernetes, **microservices**, and continuous deployment while keeping their hardware running on premises. It could involve a complete migration to the cloud, as discussed next.

Trong khi các startup mới ngày càng được sinh ra trên cloud, các hệ thống on-premises vẫn là mặc định đối với các công ty đã thành danh. Về cơ bản, những công ty này sở hữu phần cứng của mình, thứ có thể đặt trong các trung tâm dữ liệu họ sở hữu hoặc trong không gian colocation đi thuê. Trong cả hai trường hợp, các công ty chịu trách nhiệm vận hành phần cứng của mình và phần mềm chạy trên đó. Nếu phần cứng hỏng, họ phải sửa chữa hoặc thay thế nó. Họ cũng phải quản lý các chu kỳ nâng cấp mỗi vài năm khi phần cứng mới, cập nhật được phát hành và phần cứng cũ già đi rồi trở nên kém tin cậy hơn. Họ phải đảm bảo có đủ phần cứng để xử lý các đỉnh tải; đối với một nhà bán lẻ trực tuyến, điều này nghĩa là đặt đủ năng lực để xử lý các đợt tăng vọt tải của Black Friday. Đối với kỹ sư dữ liệu phụ trách các hệ thống on-premises, điều này nghĩa là mua các hệ thống đủ lớn để cho phép hiệu năng tốt với tải đỉnh và các tác vụ lớn mà không mua thừa và chi thừa. Một mặt, các công ty đã thành danh có những thực hành vận hành đã được thiết lập và phục vụ họ tốt. Giả sử một công ty dựa vào công nghệ thông tin đã kinh doanh được một thời gian. Điều này nghĩa là nó đã xoay sở được giữa các yêu cầu về chi phí và nhân sự để vận hành phần cứng của mình, quản lý các môi trường phần mềm, triển khai mã từ các nhóm phát triển, và chạy các cơ sở dữ liệu cùng hệ thống dữ liệu lớn. Mặt khác, các công ty đã thành danh thấy các đối thủ trẻ hơn, nhanh nhẹn hơn của mình đang mở rộng quy mô nhanh chóng và tận dụng các dịch vụ cloud được quản lý. Họ cũng thấy các đối thủ đã thành danh đang lấn sang cloud, cho phép họ tạm thời mở rộng quy mô năng lực tính toán khổng lồ cho các tác vụ dữ liệu đồ sộ hoặc đợt tăng vọt mua sắm Black Friday. Các công ty trong những lĩnh vực cạnh tranh nhìn chung không có lựa chọn đứng yên. Cạnh tranh khốc liệt, và luôn có mối đe dọa bị "phá vỡ" (disrupt) bởi các đối thủ nhanh nhẹn hơn, được hậu thuẫn bởi một đồng lớn tiền đầu tư mạo hiểm. Mỗi công ty phải giữ cho các hệ thống hiện hữu của mình chạy hiệu quả trong khi quyết định những nước đi tiếp theo. Điều này có thể bao gồm việc áp dụng các thực hành DevOps mới hơn, như container, Kubernetes, microservices và triển khai liên tục trong khi vẫn giữ phần cứng chạy on-premises. Nó cũng có thể bao gồm một cuộc di trú hoàn toàn lên cloud, như được bàn tiếp theo.

Cloud — Cloud

The cloud flips the on-premises model on its head. Instead of purchasing hardware, you simply rent hardware and managed services from a cloud provider (such as AWS, Azure, or Google Cloud). These resources can often be reserved on an extremely short-term basis; VMs spin up in less than a minute, and subsequent usage is billed in per-second increments. This allows cloud users to dynamically scale resources that were inconceivable with on-premises servers. In a cloud environment, engineers can quickly launch projects and experiment without worrying about long lead time hardware planning. They can begin running servers as soon as their code is ready to deploy. This makes the cloud model extremely appealing to startups that are tight on budget and time. The early cloud era was dominated by infrastructure as a service (IaaS) offerings—products such as VMs and virtual disks that are essentially rented slices of hardware. Slowly, we've seen

a shift toward platform as a service (PaaS), while SaaS products continue to grow at a rapid clip. PaaS includes IaaS products but adds more sophisticated managed services to support applications. Examples are managed databases such as Amazon Relational Database Service (RDS) and Google Cloud SQL, managed **streaming** platforms such as Amazon Kinesis and Simple Queue Service (SQS), and managed Kubernetes such as Google Kubernetes Engine (GKE) and Azure Kubernetes Service (AKS). PaaS services allow engineers to ignore the operational details of managing individual machines and deploying frameworks across distributed systems. They provide turnkey access to complex, autoscaling systems with minimal operational overhead. SaaS offerings move one additional step up the ladder of abstraction. SaaS typically provides a fully functioning enterprise software platform with little operational management. Examples of SaaS include Salesforce, Google Workspace, Microsoft 365, Zoom, and Fivetran. Both the major public clouds and third parties offer SaaS platforms. SaaS covers a whole spectrum of enterprise domains, including video conferencing, data management, ad tech, office applications, and CRM systems. This chapter also discusses serverless, increasingly important in PaaS and SaaS offerings. Serverless products generally offer automated scaling from zero to extremely high usage rates. They are billed on a pay-as-you-go basis and allow engineers to operate without operational awareness of underlying servers. Many people quibble with the term serverless ; after all, the code must run somewhere. In practice, serverless usually means many invisible servers . Cloud services have become increasingly appealing to established businesses with existing data centers and IT infrastructure. Dynamic, seamless scaling is extremely valuable to businesses that deal with seasonality (e.g., retail businesses coping with Black Friday load) and web traffic load spikes. The advent of COVID-19 in 2020 was a major driver of cloud adoption, as companies recognized the value of rapidly scaling up data processes to gain insights in a highly uncertain business climate; businesses also had to cope with substantially increased load due to a spike in online shopping, web app usage, and remote work. Before we discuss the nuances of choosing technologies in the cloud, let's first discuss why migration to the cloud requires a dramatic shift in thinking, specifically on the pricing front; this is closely related to FinOps, introduced in “FinOps” . Enterprises that migrate to the cloud often make major deployment errors by not appropriately adapting their practices to the cloud pricing model.

Cloud lật ngược mô hình on-premises. Thay vì mua phần cứng, bạn chỉ đơn giản thuê phần cứng và các dịch vụ được quản lý từ một nhà cung cấp cloud (như AWS, Azure hay Google Cloud). Những tài nguyên này thường có thể được đặt giữ trên cơ sở cực kỳ ngắn hạn; các VM khởi chạy trong chưa đầy một phút, và mức sử dụng sau đó được tính tiền theo từng giây. Điều này cho phép người dùng cloud mở rộng quy mô tài nguyên một cách động theo những cách không thể tưởng tượng nổi với các máy chủ on-premises. Trong một môi trường cloud, kỹ sư có thể nhanh chóng khởi động các dự án và thử nghiệm mà không phải lo lắng về việc hoạch định phần cứng với thời gian chờ dài. Họ có thể bắt đầu chạy máy chủ ngay khi mã của họ sẵn sàng triển khai. Điều này khiến mô hình cloud cực kỳ hấp dẫn đối với các startup eo hẹp về ngân sách và thời gian. Kỷ nguyên đầu của cloud bị thống trị bởi các giải pháp hạ tầng như một dịch vụ (infrastructure as a service, IaaS)—những sản phẩm như VM và đĩa ảo về cơ bản là các lát phần cứng đi thuê. Chậm rãi, chúng ta đã thấy một sự dịch chuyển hướng về nền tảng như một dịch vụ (platform as a service, PaaS), trong khi các sản phẩm SaaS tiếp tục tăng trưởng với nhịp độ nhanh. PaaS bao gồm các sản phẩm IaaS nhưng thêm vào các dịch vụ được quản lý tinh vi hơn để hỗ trợ ứng dụng. Ví dụ là các cơ sở dữ liệu được quản lý như Amazon Relational Database Service (RDS) và Google Cloud SQL, các nền tảng streaming được quản lý như Amazon Kinesis và Simple Queue Service (SQS), và Kubernetes được quản lý như Google Kubernetes Engine (GKE) và Azure Kubernetes Service (AKS). Các dịch vụ PaaS cho phép kỹ sư bỏ qua các chi tiết vận hành của việc quản lý từng máy riêng lẻ và triển khai framework trên các hệ phân tán. Chúng cung cấp quyền truy cập “cắm là chạy” vào các hệ thống phức tạp, tự mở rộng quy mô với chi phí vận hành tối thiểu. Các giải pháp SaaS tiến thêm một bước nữa lên thang trù

tượng. SaaS thường cung cấp một nền tảng phần mềm doanh nghiệp hoạt động đầy đủ với rất ít quản lý vận hành. Ví dụ về SaaS bao gồm Salesforce, Google Workspace, Microsoft 365, Zoom và Fivetran. Cả các đám mây công cộng lớn lẫn các bên thứ ba đều cung cấp các nền tảng SaaS. SaaS bao trùm cả một phổ các lĩnh vực doanh nghiệp, bao gồm hội nghị truyền hình, quản lý dữ liệu, công nghệ quảng cáo, ứng dụng văn phòng và các hệ thống CRM. Chương này cũng bàn về serverless, ngày càng quan trọng trong các giải pháp PaaS và SaaS. Các sản phẩm serverless nhìn chung cung cấp khả năng tự mở rộng từ không lên đến mức sử dụng cực cao. Chúng được tính tiền trên cơ sở trả-theo-sử-dụng và cho phép kỹ sư vận hành mà không cần nhận thức về các máy chủ bên dưới. Nhiều người bắt bẻ thuật ngữ *serverless*; suy cho cùng, mã phải chạy ở đâu đó. Trên thực tế, serverless thường có nghĩa là *không có máy chủ vô hình*. Các dịch vụ cloud đã trở nên ngày càng hấp dẫn đối với các doanh nghiệp đã thành danh có sẵn các trung tâm dữ liệu và hạ tầng CNTT. Khả năng mở rộng quy mô động, liền mạch là cực kỳ giá trị đối với các doanh nghiệp phải đối phó với tính thời vụ (ví dụ, các doanh nghiệp bán lẻ chống đỡ tải Black Friday) và các đợt tăng vọt tải lưu lượng web. Sự xuất hiện của COVID-19 năm 2020 là một động lực lớn của việc áp dụng cloud, khi các công ty nhận ra giá trị của việc nhanh chóng mở rộng quy mô các quy trình dữ liệu để thu được những hiểu biết trong một bối cảnh kinh doanh hết sức bất định; các doanh nghiệp cũng phải chống đỡ tải tăng đáng kể do đợt tăng vọt của mua sắm trực tuyến, sử dụng ứng dụng web, và làm việc từ xa. Trước khi bàn về những sắc thái của việc chọn công nghệ trong cloud, hãy bàn trước về lý do tại sao việc di trú lên cloud đòi hỏi một sự thay đổi tư duy mạnh mẽ, đặc biệt trên mặt trận định giá; điều này liên quan chặt chẽ tới FinOps, đã giới thiệu trong “FinOps”. Các doanh nghiệp di trú lên cloud thường mắc các sai lầm triển khai lớn vì không điều chỉnh các thực hành của mình cho phù hợp với mô hình định giá của cloud.

A BRIEF DETOUR ON CLOUD ECONOMICS — MỘT KHÚC LẠC ĐỀ NGẮN VỀ KINH TẾ HỌC CLOUD

To understand how to use cloud services efficiently through cloud native architecture , you need to know how clouds make money. This is an extremely complex concept and one on which cloud providers offer little transparency. Consider this sidebar a starting point for your research, discovery, and process development.

Để hiểu cách dùng các dịch vụ cloud một cách hiệu quả thông qua *kiến trúc cloud-native*, bạn cần biết các đám mây kiếm tiền như thế nào. Đây là một khái niệm cực kỳ phức tạp và là một khái niệm mà các nhà cung cấp cloud cung cấp rất ít sự minh bạch. Hãy coi phần bên lề này như một điểm khởi đầu cho quá trình nghiên cứu, khám phá và phát triển quy trình của bạn.

Cloud Services and Credit Default Swaps — Dịch vụ cloud và hợp đồng hoán đổi rủi ro tín dụng

Let’s go on a little tangent about credit default swaps. Don’t worry, this will make sense in a bit. Recall that credit default swaps rose to infamy after the 2007 global financial crisis. A credit default swap was a mechanism for selling different tiers of risk attached to an asset (i.e., a mortgage.) It is not our intention to present this idea in any detail, but rather to offer an analogy wherein many cloud services are similar to financial derivatives; cloud providers not only slice hardware assets into small pieces through virtualization, but also sell these pieces with varying technical characteristics and

risks attached. While providers are extremely tight-lipped about details of their internal systems, there are massive opportunities for optimization and scaling by understanding cloud pricing and exchanging notes with other users. Look at the example of archival cloud storage. At the time of this writing, GCP openly admits that its archival class storage runs on the same clusters as standard cloud storage, yet the price per gigabyte per month of archival storage is roughly 1/17 that of standard storage. How is this possible? Here's our educated guess. When purchasing cloud storage, each disk in a storage cluster has three assets that cloud providers and consumers use. First, it has a certain storage capacity—say, 10 TB. Second, it supports a certain number of input/output operations (IOPs) per second—say, 100. Third, disks support a certain maximum bandwidth, the maximum read speed for optimally organized files. A magnetic drive might be capable of reading at 200 MB/s.

Hãy lạc đề một chút về hợp đồng hoán đổi rủi ro tín dụng (credit default swap). Đừng lo, điều này sẽ có ý nghĩa trong giây lát. Hãy nhớ lại rằng các hợp đồng hoán đổi rủi ro tín dụng trở nên tai tiếng sau cuộc khủng hoảng tài chính toàn cầu năm 2007. Một hợp đồng hoán đổi rủi ro tín dụng là một cơ chế để bán các tầng rủi ro khác nhau gắn với một tài sản (ví dụ, một khoản vay thế chấp). Chúng tôi không có ý định trình bày ý tưởng này một cách chi tiết, mà chỉ muốn đưa ra một phép so sánh trong đó nhiều dịch vụ cloud tương tự như các công cụ tài chính phái sinh; các nhà cung cấp cloud không chỉ cắt các tài sản phần cứng thành những mảnh nhỏ thông qua ảo hóa, mà còn bán những mảnh này với các đặc tính kỹ thuật và rủi ro khác nhau đi kèm. Trong khi các nhà cung cấp cực kỳ kín tiếng về chi tiết các hệ thống nội bộ của họ, có những cơ hội khổng lồ để tối ưu hóa và mở rộng quy mô bằng cách hiểu cách định giá của cloud và trao đổi ghi chú với những người dùng khác. Hãy nhìn ví dụ về lưu trữ cloud dạng lưu trữ (archival). Tại thời điểm viết cuốn sách này, GCP công khai thừa nhận rằng lớp lưu trữ archival của họ chạy trên cùng các cụm như lưu trữ cloud tiêu chuẩn, ấy thế mà giá mỗi gigabyte mỗi tháng của lưu trữ archival lại chỉ bằng khoảng 1/17 của lưu trữ tiêu chuẩn. Làm sao điều này có thể xảy ra? Đây là phỏng đoán có cơ sở của chúng tôi. Khi mua lưu trữ cloud, mỗi đĩa trong một cụm lưu trữ có ba tài sản mà nhà cung cấp và người tiêu dùng cloud sử dụng. Thứ nhất, nó có một dung lượng lưu trữ nhất định—giả sử, 10 TB. Thứ hai, nó hỗ trợ một số lượng thao tác vào/ra (IOPs) nhất định mỗi giây—giả sử, 100. Thứ ba, các đĩa hỗ trợ một băng thông tối đa nhất định, là tốc độ đọc tối đa cho các tệp được sắp xếp tối ưu. Một ổ đĩa từ có thể đọc được ở 200 MB/s.

Any of these limits (IOPs, storage capacity, bandwidth) is a potential bottleneck for a cloud provider. For instance, the cloud provider might have a disk storing 3 TB of data but hitting maximum IOPs. An alternative to leaving the remaining 7 TB empty is to sell the empty space without selling IOPs. Or, more specifically, sell cheap storage space and expensive IOPs to discourage reads. Much like traders of financial derivatives, cloud vendors also deal in risk. In the case of archival storage, vendors are selling a type of insurance, but one that pays out for the insurer rather than the policy buyer in the **event** of a catastrophe. While data storage costs per month are extremely cheap, I risk paying a high price if I ever need to retrieve data. But this is a price that I will happily pay in a true emergency. Similar considerations apply to nearly any cloud service. While on-premises servers are essentially sold as commodity hardware, the cost model in the cloud is more subtle. Rather than just charging for CPU cores, memory, and features, cloud vendors monetize characteristics such as durability, reliability, longevity, and predictability; a variety of compute platforms discount their offerings for workloads that are ephemeral or can be arbitrarily interrupted when capacity is needed elsewhere.

Bất kỳ giới hạn nào trong số này (IOPs, dung lượng lưu trữ, băng thông) đều là một nút thắt cổ chai tiềm tàng đối với một nhà cung cấp cloud. Chẳng hạn, nhà cung cấp cloud có thể có một đĩa lưu trữ 3 TB dữ liệu nhưng đã chạm mức IOPs tối đa. Một lựa chọn thay thế cho việc để 7 TB còn lại trống là bán không gian trống đó mà không bán IOPs. Hay, cụ thể hơn, bán không gian lưu trữ rẻ và IOPs đắt để không khuyến khích việc đọc. Cũng như

những người giao dịch các công cụ tài chính phái sinh, các nhà cung cấp cloud cũng buôn bán rủi ro. Trong trường hợp lưu trữ archival, các nhà cung cấp đang bán một loại bảo hiểm, nhưng là loại chi trả cho bên bảo hiểm chứ không phải cho bên mua hợp đồng trong trường hợp xảy ra thảm họa. Trong khi chi phí lưu trữ dữ liệu mỗi tháng cực kỳ rẻ, tôi có nguy cơ phải trả một cái giá cao nếu có lúc nào đó tôi cần lấy lại dữ liệu. Nhưng đây là cái giá mà tôi sẽ vui lòng trả trong một tình huống khẩn cấp thực sự. Những cân nhắc tương tự áp dụng cho gần như bất kỳ dịch vụ cloud nào. Trong khi các máy chủ on-premises về cơ bản được bán như phần cứng hàng hóa, mô hình chi phí trong cloud tinh tế hơn. Thay vì chỉ tính tiền cho nhân CPU, bộ nhớ và tính năng, các nhà cung cấp cloud kiếm tiền từ các đặc tính như độ bền (durability), độ tin cậy, tuổi thọ và khả năng dự đoán; nhiều nền tảng tính toán giảm giá các giải pháp của họ cho các tải công việc mang tính tạm thời (ephemeral) hoặc có thể bị gián đoạn tùy ý khi cần năng lực ở nơi khác.

Cloud $\neq$ On Premises — Cloud $\neq$ On Premises

This heading may seem like a silly tautology, but the belief that cloud services are just like familiar on-premises servers is a widespread cognitive error that plagues cloud migrations and leads to horrifying bills. This demonstrates a broader issue in tech that we refer to as the curse of familiarity. Many new technology products are intentionally designed to look like something familiar to facilitate ease of use and accelerate adoption. But, any new technology product has subtleties and wrinkles that users must learn to identify, accommodate, and optimize. Moving on-premises servers one by one to VMs in the cloud—known as simple lift and shift—is a perfectly reasonable strategy for the initial phase of cloud migration, especially when a company is facing some kind of financial cliff, such as the need to sign a significant new lease or hardware contract if existing hardware is not shut down. However, companies that leave their cloud assets in this initial state are in for a rude shock. On a direct comparison basis, long-running servers in the cloud are significantly more expensive than their on-premises counterparts. The key to finding value in the cloud is understanding and optimizing the cloud pricing model. Rather than deploying a set of long-running servers capable of handling full peak load, use autoscaling to allow workloads to scale down to minimal infrastructure when loads are light, and up to massive clusters during peak times. To realize discounts through more ephemeral, less durable workloads, use reserved or spot instances, or use serverless functions in place of servers. We often think of this optimization as leading to lower costs, but we should also strive to increase business value by exploiting the dynamic nature of the cloud. Data engineers can create new value in the cloud by accomplishing things that were impossible in their on-premises environment. For example, it is possible to quickly spin up massive compute clusters to run complex transformations at scales that were unaffordable for on-premises hardware.

Tiêu đề này nghe có vẻ như một sự lặp thừa ngớ ngẩn, nhưng niềm tin rằng các dịch vụ cloud cũng giống hệt các máy chủ on-premises quen thuộc là một lỗi nhận thức phổ biến gây nhức nhối cho các cuộc di trú lên cloud và dẫn đến những hóa đơn kinh hoàng. Điều này cho thấy một vấn đề rộng hơn trong công nghệ mà chúng tôi gọi là *lời nguyền của sự quen thuộc*. Nhiều sản phẩm công nghệ mới được thiết kế có chủ ý để trông giống một thứ gì đó quen thuộc nhằm tạo điều kiện cho việc dễ sử dụng và đẩy nhanh sự áp dụng. Nhưng bất kỳ sản phẩm công nghệ mới nào cũng có những điểm tinh tế và những nếp gấp mà người dùng phải học để nhận diện, thích nghi và tối ưu. Chuyển các máy chủ on-premises từng cái một sang các VM trong cloud—được gọi là *nhấc và chuyển đơn giản* (simple lift and shift)—là một chiến lược hoàn toàn hợp lý cho giai đoạn đầu của cuộc di trú lên cloud, đặc biệt khi một công ty đang đối mặt với một kiểu vách đá tài chính, chẳng hạn như nhu cầu phải ký một hợp đồng thuê hoặc hợp đồng phần cứng mới đáng kể nếu phần cứng hiện hữu không bị tắt đi. Tuy nhiên, các công ty để các tài sản cloud của mình ở trạng

thái ban đầu này sẽ phải hứng một cú sốc khó chịu. Trên cơ sở so sánh trực tiếp, các máy chủ chạy lâu dài trong cloud đắt hơn đáng kể so với các đối tác on-premises của chúng. Chìa khóa để tìm thấy giá trị trong cloud là hiểu và tối ưu mô hình định giá của cloud. Thay vì triển khai một dàn máy chủ chạy lâu dài có khả năng xử lý toàn bộ tải đỉnh, hãy dùng tự mở rộng quy mô (autoscaling) để cho phép các tải công việc thu nhỏ xuống hạ tầng tối thiểu khi tải nhẹ, và phình lên thành các cụm khổng lồ trong những thời điểm cao điểm. Để hiện thực hóa các khoản giảm giá thông qua các tải công việc mang tính tạm thời hơn, kém bền hơn, hãy dùng các instance dạng đặt giữ (reserved) hoặc dạng spot, hoặc dùng các hàm serverless thay cho máy chủ. Chúng ta thường nghĩ về việc tối ưu hóa này như dẫn đến chi phí thấp hơn, nhưng chúng ta cũng nên cố gắng tăng giá trị kinh doanh bằng cách khai thác bản chất động của cloud. Kỹ sư dữ liệu có thể tạo ra giá trị mới trong cloud 3 bằng cách hoàn thành những việc vốn bất khả thi trong môi trường on-premises của họ. Ví dụ, có thể nhanh chóng khởi chạy các cụm tính toán khổng lồ để chạy các biến đổi (transformation) phức tạp ở những quy mô vốn không thể chi trả nổi với phần cứng on-premises.

Data Gravity — Lực hấp dẫn dữ liệu

In addition to basic errors such as following on-premises operational practices in the cloud, data engineers need to watch out for other aspects of cloud pricing and incentives that frequently catch users unawares. Vendors want to lock you into their offerings. Getting data onto the platform is cheap or free on most cloud platforms, but getting data out can be extremely expensive. Be aware of data egress fees and their long-term impacts on your business before getting blindsided by a large bill. Data gravity is real: once data lands in a cloud, the cost to extract it and migrate processes can be very high.

Ngoài các sai lầm cơ bản như tuân theo các thực hành vận hành on-premises trong cloud, kỹ sư dữ liệu cần đề phòng các khía cạnh khác của định giá và ưu đãi của cloud mà thường khiến người dùng bị bất ngờ. Các nhà cung cấp muốn khóa bạn vào các giải pháp của họ. Đưa dữ liệu lên nền tảng thì rẻ hoặc miễn phí trên hầu hết các nền tảng cloud, nhưng lấy dữ liệu ra có thể cực kỳ đắt đỏ. Hãy lưu tâm đến các khoản phí xuất dữ liệu (data egress) và các tác động dài hạn của chúng đối với doanh nghiệp của bạn trước khi bị bất ngờ bởi một hóa đơn lớn. Lực hấp dẫn dữ liệu (data gravity) là có thật: một khi dữ liệu đáp xuống một cloud, chi phí để trích xuất nó và di trú các quy trình có thể rất cao.

Hybrid Cloud — Hybrid Cloud

As more established businesses migrate into the cloud, the hybrid cloud model is growing in importance. Virtually no business can migrate all of its workloads overnight. The hybrid cloud model assumes that an organization will indefinitely maintain some workloads outside the cloud. There are many reasons to consider a hybrid cloud model. Organizations may believe that they have achieved operational excellence in certain areas, such as their application stack and associated hardware. Thus, they may migrate only to specific workloads where they see immediate benefits in the cloud environment. For example, an on-premises Spark stack is migrated to ephemeral cloud clusters, reducing the operational burden of managing software and hardware for the data engineering team and allowing rapid scaling for large data jobs. This pattern of putting **analytics** in the cloud is beautiful because data flows primarily in one direction, minimizing data egress costs (Figure 4-3). That is, on-premises applications generate event data that can be pushed to the cloud essentially for free. The bulk of data remains in the cloud where it is analyzed, while smaller

amounts of data are pushed back to on premises for deploying models to applications, **reverse ETL**, etc.

Khi nhiều doanh nghiệp đã thành danh di trú vào cloud, mô hình hybrid cloud đang tăng dần tầm quan trọng. Gần như không doanh nghiệp nào có thể di trú toàn bộ tải công việc của mình chỉ sau một đêm. Mô hình hybrid cloud giả định rằng một tổ chức sẽ duy trì một số tải công việc bên ngoài cloud một cách vô thời hạn. Có nhiều lý do để cân nhắc một mô hình hybrid cloud. Các tổ chức có thể tin rằng họ đã đạt được sự xuất sắc vận hành trong một số lĩnh vực nhất định, chẳng hạn như ngăn xếp ứng dụng của họ và phần cứng liên quan. Do đó, họ có thể chỉ di trú những tải công việc cụ thể nơi họ thấy lợi ích tức thì trong môi trường cloud. Ví dụ, một ngăn xếp Spark on-premises được di trú sang các cụm cloud mang tính tạm thời, giảm gánh nặng vận hành của việc quản lý phần mềm và phần cứng cho nhóm kỹ thuật dữ liệu và cho phép mở rộng quy mô nhanh cho các tác vụ dữ liệu lớn. Mô thức đặt phân tích (analytics) trên cloud này rất đẹp vì dữ liệu chủ yếu chảy theo một hướng, giảm thiểu chi phí xuất dữ liệu (Hình 4-3). Tức là, các ứng dụng on-premises sinh ra dữ liệu sự kiện (event) có thể được đẩy lên cloud về cơ bản là miễn phí. Phần lớn dữ liệu nằm lại trên cloud nơi nó được phân tích, trong khi một lượng dữ liệu nhỏ hơn được đẩy ngược về on-premises để triển khai các mô hình vào ứng dụng, reverse ETL, v.v.

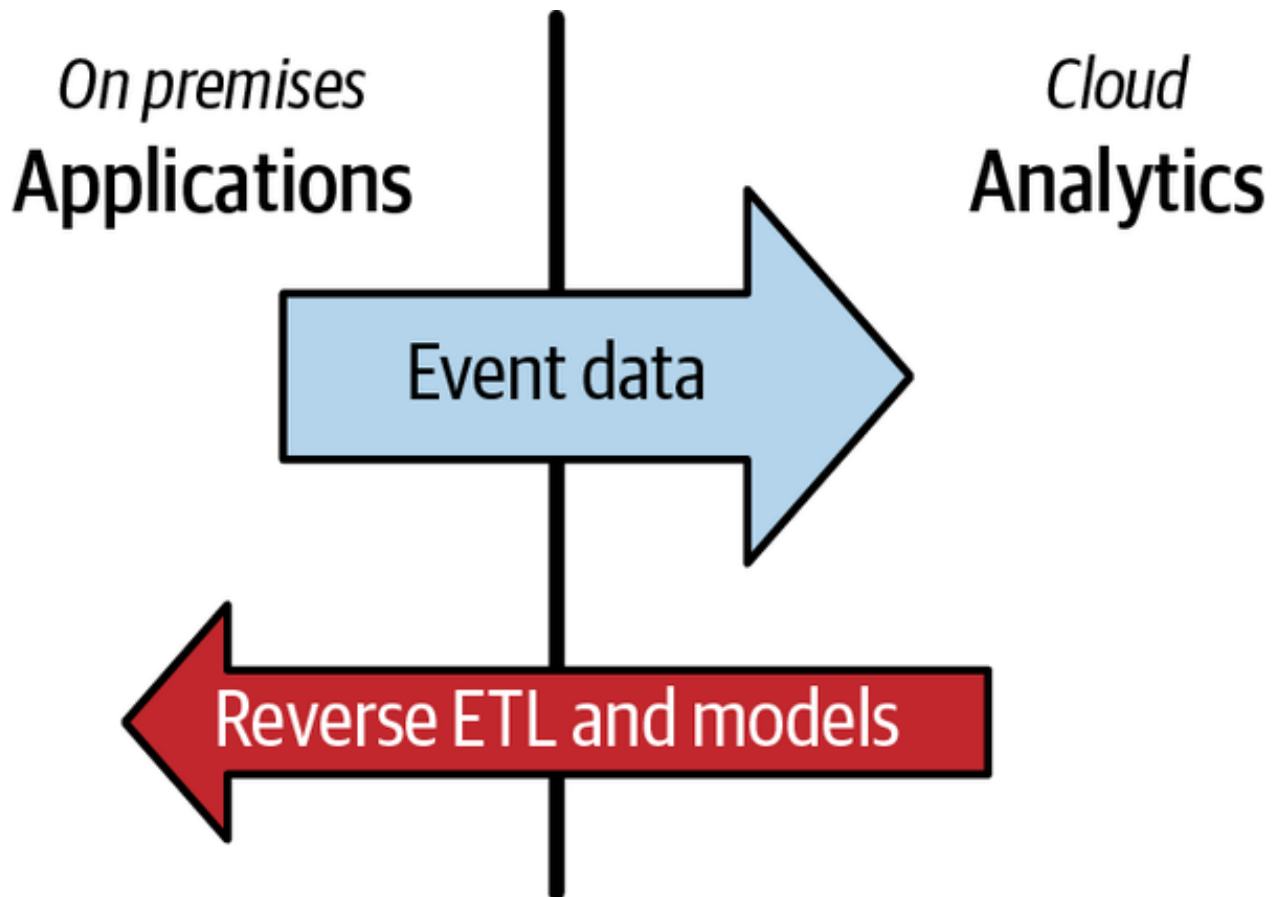


Figure 4-3. A hybrid cloud data flow model minimizing egress costs A new **generation** of managed hybrid cloud service offerings also allows customers to locate cloud-managed servers in their data centers. This gives 4 users the ability to incorporate the best features in each cloud alongside on-premises infrastructure.

Hình 4-3. Một mô hình luồng dữ liệu hybrid cloud giảm thiểu chi phí xuất dữ liệu. Một thế

hệ mới các giải pháp dịch vụ hybrid cloud được quản lý cũng cho phép khách hàng đặt các máy chủ được cloud quản lý trong các trung tâm dữ liệu của họ. Điều này 4 trao cho người dùng khả năng kết hợp những tính năng tốt nhất trong mỗi cloud bên cạnh hạ tầng on-premises.

Multicloud — Multicloud

Multicloud simply refers to deploying workloads to multiple public clouds. Companies may have several motivations for multicloud deployments. SaaS platforms often wish to offer services close to existing customer cloud workloads. Snowflake and Databricks provide their SaaS offerings across multiple clouds for this reason. This is especially critical for data-intensive applications, where network **latency** and bandwidth limitations hamper performance, and data egress costs can be prohibitive.

Multicloud chỉ đơn giản nghĩa là triển khai các tải công việc lên nhiều đám mây công cộng. Các công ty có thể có một số động cơ cho việc triển khai multicloud. Các nền tảng SaaS thường muốn cung cấp dịch vụ ở gần các tải công việc cloud hiện hữu của khách hàng. Snowflake và Databricks cung cấp các giải pháp SaaS của họ trên nhiều cloud vì lý do này. Điều này đặc biệt then chốt đối với các ứng dụng thâm dụng dữ liệu, nơi độ trễ (latency) mạng và các giới hạn băng thông cản trở hiệu năng, và chi phí xuất dữ liệu có thể cao đến mức cấm cản.

Another common motivation for employing a multicloud approach is to take advantage of the best services across several clouds. For example, a company might want to handle its Google Ads and Analytics data on Google Cloud and deploy Kubernetes through GKE. And the company might also adopt Azure specifically for Microsoft workloads. Also, the company may like AWS because it has several best-in-class services (e.g., AWS **Lambda**) and enjoys huge mindshare, making it relatively easy to hire AWS-proficient engineers. Any mix of various cloud provider services is possible. Given the intense competition among the major cloud providers, expect them to offer more best-of-breed services, making multicloud more compelling. A multicloud methodology has several disadvantages. As we just mentioned, data egress costs and networking bottlenecks are critical. Going multicloud can introduce significant complexity. Companies must now manage a dizzying array of services across several clouds; cross-cloud integration and security present a considerable challenge; multicloud networking can be diabolically complicated. A new generation of “cloud of clouds” services aims to facilitate multicloud with reduced complexity by offering services across clouds and seamlessly replicating data between clouds or managing workloads on several clouds through a single pane of glass. To cite one example, a Snowflake account runs in a single cloud region, but customers can readily spin up other accounts in GCP, AWS, or Azure. Snowflake provides simple scheduled data replication between these various cloud accounts. The Snowflake interface is essentially the same in all of these accounts, removing the training burden of switching between cloud-native data services. The “cloud of clouds” space is evolving quickly; within a few years of this book’s publication, many more of these services will be available. Data engineers and architects would do well to maintain awareness of this quickly changing landscape.

Một động cơ phổ biến khác cho việc áp dụng cách tiếp cận multicloud là tận dụng các dịch vụ tốt nhất trên nhiều cloud. Ví dụ, một công ty có thể muốn xử lý dữ liệu Google Ads và Analytics của mình trên Google Cloud và triển khai Kubernetes thông qua GKE. Và công ty cũng có thể áp dụng Azure đặc biệt cho các tải công việc Microsoft. Đồng thời, công ty có thể thích AWS vì nó có một số dịch vụ thuộc hàng tốt nhất (ví dụ, AWS Lambda) và có sức ảnh hưởng khổng lồ, khiến việc tuyển dụng các kỹ sư thành thạo AWS tương đối dễ. Bất kỳ sự pha trộn nào của các dịch vụ từ các nhà cung cấp cloud khác nhau đều khả thi.

Trước sự cạnh tranh gay gắt giữa các nhà cung cấp cloud lớn, hãy kỳ vọng họ cung cấp nhiều dịch vụ thuộc hàng tốt nhất hơn, khiến multicloud càng hấp dẫn hơn. Phương pháp multicloud có một số nhược điểm. Như chúng tôi vừa đề cập, chi phí xuất dữ liệu và các nút thắt cổ chai mạng là then chốt. Đi multicloud có thể đưa vào sự phức tạp đáng kể. Các công ty giờ đây phải quản lý một mê cung chóng mặt các dịch vụ trên nhiều cloud; tích hợp và bảo mật xuyên cloud là một thách thức đáng kể; mạng multicloud có thể phức tạp đến mức quỵ quái. Một thể hệ mới các dịch vụ “đám mây của các đám mây” (cloud of clouds) nhằm tới việc tạo điều kiện cho multicloud với độ phức tạp giảm bớt bằng cách cung cấp dịch vụ xuyên các cloud và sao chép dữ liệu một cách liền mạch giữa các cloud hoặc quản lý các tải công việc trên nhiều cloud thông qua một bảng điều khiển duy nhất. Để dẫn một ví dụ, một tài khoản Snowflake chạy trong một vùng cloud duy nhất, nhưng khách hàng có thể dễ dàng khởi chạy các tài khoản khác trong GCP, AWS hoặc Azure. Snowflake cung cấp sao chép dữ liệu theo lịch một cách đơn giản giữa các tài khoản cloud khác nhau này. Giao diện Snowflake về cơ bản giống nhau trong tất cả các tài khoản này, loại bỏ gánh nặng đào tạo khi chuyển đổi giữa các dịch vụ dữ liệu cloud-native. Không gian “đám mây của các đám mây” đang tiến hóa nhanh chóng; trong vòng vài năm sau khi cuốn sách này được xuất bản, nhiều dịch vụ kiểu này hơn sẽ có mặt. Kỹ sư dữ liệu và kiến trúc sư nên duy trì nhận thức về bối cảnh thay đổi nhanh chóng này.

Decentralized: Blockchain and the Edge — Phi tập trung: Blockchain và biên (Edge)

Though not widely used now, it's worth briefly mentioning a new trend that might become popular over the next decade: decentralized computing. Whereas today's applications mainly run on premises and in the cloud, the rise of blockchain, Web 3.0, and edge computing may invert this paradigm. For the moment, decentralized platforms have proven extremely popular but have not had a significant impact in the data space; even so, keeping an eye on these platforms is worthwhile as you assess technology decisions.

Dù hiện chưa được dùng rộng rãi, cũng đáng để đề cập ngắn gọn một xu hướng mới có thể trở nên phổ biến trong thập kỷ tới: tính toán phi tập trung (decentralized computing). Trong khi các ứng dụng ngày nay chủ yếu chạy on-premises và trong cloud, sự trỗi dậy của blockchain, Web 3.0 và tính toán biên (edge computing) có thể đảo ngược mô thức này. Hiện tại, các nền tảng phi tập trung đã chứng tỏ là cực kỳ phổ biến nhưng chưa có tác động đáng kể trong lĩnh vực dữ liệu; dù vậy, việc để mắt đến các nền tảng này là đáng giá khi bạn đánh giá các quyết định công nghệ.

Our Advice — Lời khuyên của chúng tôi

From our perspective, we are still at the beginning of the transition to the cloud. Thus the evidence and arguments around workload placement and migration are in flux. The cloud itself is changing, with a shift from the IaaS model built around Amazon EC2 that drove the early growth of AWS, toward more **managed service** offerings such as AWS Glue, Google BigQuery, and Snowflake. We've also seen the emergence of new workload placement abstractions. On-premises services are becoming more cloud-like and abstracted. Hybrid cloud services allow customers to run fully managed services within their walls while facilitating tight integration between local and remote environments. Further, the “cloud of clouds” is beginning to take shape, fueled by third-party services and public cloud vendors.

Theo quan điểm của chúng tôi, chúng ta vẫn đang ở giai đoạn đầu của quá trình chuyển dịch sang cloud. Do đó, các bằng chứng và lập luận xoay quanh việc đặt và di trú tải công việc vẫn đang biến động. Bản thân cloud cũng đang thay đổi, với một sự dịch chuyển từ mô hình IaaS được xây dựng quanh Amazon EC2 vốn thúc đẩy sự tăng trưởng ban đầu của AWS, hướng tới các giải pháp dịch vụ được quản lý hơn như AWS Glue, Google BigQuery và Snowflake. Chúng ta cũng đã thấy sự xuất hiện của các trù tượng mới về việc đặt tải công việc. Các dịch vụ on-premises đang trở nên giống cloud hơn và được trừu tượng hóa hơn. Các dịch vụ hybrid cloud cho phép khách hàng chạy các dịch vụ được quản lý đầy đủ trong phạm vi của mình trong khi tạo điều kiện cho sự tích hợp chặt chẽ giữa các môi trường cục bộ và từ xa. Hơn nữa, “đám mây của các đám mây” đang bắt đầu thành hình, được tiếp sức bởi các dịch vụ bên thứ ba và các nhà cung cấp đám mây công cộng.

Choose technologies for the present, but look toward the future — Chọn công nghệ cho hiện tại, nhưng hướng nhìn về tương lai

As we mentioned in “Today Versus the Future: Immutable Versus Transitory Technologies” , you need to keep one eye on the present while planning for unknowns. Right now is a tough time to plan workload placements and migrations. Because of the fast pace of competition and change in the cloud industry, the decision space will look very different in five to ten years. It is tempting to take into account every possible future architecture permutation. We believe that it is critical to avoid this endless trap of analysis. Instead, plan for the present. Choose the best technologies for your current needs and concrete plans for the near future. Choose your deployment platform based on real business needs while focusing on simplicity and flexibility. In particular, don’t choose a complex multicloud or hybrid-cloud strategy unless there’s a compelling reason. Do you need to serve data near customers on multiple clouds? Do industry regulations require you to house certain data in your data centers? Do you have a compelling technology need for specific services on two different clouds? Choose a single-cloud deployment strategy if these scenarios don’t apply to you. On the other hand, have an escape plan. As we’ve emphasized before, every technology—even open source software—comes with some degree of lock-in. A single-cloud strategy has significant advantages of simplicity and integration but comes with significant lock-in attached. In this instance, we’re talking about mental flexibility, the flexibility to evaluate the current state of the world and imagine alternatives. Ideally, your escape plan will remain locked behind glass, but preparing this plan will help you to make better decisions in the present and give you a way out if things go wrong in the future.

Như chúng tôi đã đề cập trong “Hôm nay so với tương lai: công nghệ bất biến so với công nghệ tạm thời”, bạn cần để một mắt vào hiện tại trong khi hoạch định cho những điều chưa biết. Ngay lúc này là một thời điểm khó để hoạch định việc đặt và di trú tải công việc. Vì nhịp độ cạnh tranh và thay đổi nhanh chóng trong ngành cloud, không gian quyết định sẽ trông rất khác sau năm đến mười năm nữa. Người ta dễ bị cám dỗ tính đến mọi hoán vị kiến trúc tương lai có thể có. Chúng tôi tin rằng điều then chốt là tránh cái bẫy phân tích bất tận này. Thay vào đó, hãy hoạch định cho hiện tại. Hãy chọn các công nghệ tốt nhất cho các nhu cầu hiện tại của bạn và các kế hoạch cụ thể cho tương lai gần. Hãy chọn nền tảng triển khai dựa trên các nhu cầu kinh doanh thực sự trong khi tập trung vào sự đơn giản và linh hoạt. Đặc biệt, đừng chọn một chiến lược multicloud hay hybrid-cloud phức tạp trừ khi có một lý do thuyết phục. Bạn có cần phục vụ dữ liệu gần khách hàng trên nhiều cloud không? Các quy định ngành có yêu cầu bạn lưu giữ một số dữ liệu nhất định trong các trung tâm dữ liệu của bạn không? Bạn có một nhu cầu công nghệ thuyết phục về các dịch vụ cụ thể trên hai cloud khác nhau không? Hãy chọn một chiến lược triển khai đơn-cloud nếu các kịch bản này không áp dụng cho bạn. Mặt khác, hãy có một kế hoạch

thoát thân. Như chúng tôi đã nhấn mạnh trước đây, mọi công nghệ—kể cả phần mềm mã nguồn mở—đều đi kèm một mức độ khóa-vào (lock-in) nào đó. Một chiến lược đơn-cloud có những lợi thế đáng kể về sự đơn giản và tích hợp nhưng đi kèm với mức khóa-vào đáng kể. Trong trường hợp này, chúng tôi đang nói về sự linh hoạt trong tư duy, sự linh hoạt để đánh giá tình trạng hiện tại của thế giới và hình dung các lựa chọn thay thế. Lý tưởng nhất, kế hoạch thoát thân của bạn sẽ vẫn nằm sau lớp kính, nhưng việc chuẩn bị kế hoạch này sẽ giúp bạn đưa ra các quyết định tốt hơn ở hiện tại và cho bạn một lối ra nếu mọi thứ trở nên tồi tệ trong tương lai.

Cloud Repatriation Arguments — Các lập luận về hồi hương khỏi cloud

As we wrote this book, Sarah Wang and Martin Casado published “The Cost of Cloud, A Trillion Dollar Paradox”, an article that generated significant sound and fury in the tech space. Readers widely interpreted the article as a call for the repatriation of cloud workloads to on-premises servers. They make a somewhat more subtle argument that companies should expend significant resources to control cloud spending and should consider repatriation as a possible option. We want to take a moment to dissect one part of their discussion. Wang and Casado cite Dropbox’s repatriation of significant workloads from AWS to Dropbox-owned servers as a case study for companies considering similar repatriation moves.

Khi chúng tôi viết cuốn sách này, Sarah Wang và Martin Casado đã xuất bản “The Cost of Cloud, A Trillion Dollar Paradox”, một bài viết tạo ra nhiều ồn ào và phẫn nộ trong giới công nghệ. Độc giả phần lớn diễn giải bài viết như một lời kêu gọi hồi hương (repatriation) các tải công việc cloud về các máy chủ on-premises. Họ đưa ra một lập luận có phần tinh tế hơn là các công ty nên bỏ ra những nguồn lực đáng kể để kiểm soát chi tiêu cloud và nên cân nhắc hồi hương như một lựa chọn khả dĩ. Chúng tôi muốn dành một chút thời gian để mổ xẻ một phần trong cuộc thảo luận của họ. Wang và Casado dẫn việc Dropbox hồi hương các tải công việc đáng kể từ AWS về các máy chủ do Dropbox sở hữu như một nghiên cứu trường hợp cho các công ty đang cân nhắc những nước đi hồi hương tương tự.

You are not Dropbox, nor are you Cloudflare — Bạn không phải Dropbox, cũng không phải Cloudflare

We believe that this case study is frequently used without appropriate context and is a compelling example of the false equivalence logical fallacy. Dropbox provides particular services where ownership of hardware and data centers can offer a competitive advantage. Companies should not rely excessively on Dropbox’s example when assessing cloud and on-premises deployment options. First, it’s important to understand that Dropbox stores enormous amounts of data. The company is tight-lipped about exactly how much data it hosts, but says it is many exabytes and continues to grow. Second, Dropbox handles a vast amount of network traffic. We know that its bandwidth consumption in 2017 was significant enough for the company to add “hundreds of gigabits of internet connectivity with transit providers (regional and global ISPs), and hundreds of new peering partners (where we exchange traffic directly rather than through an ISP).” The data egress costs would be extremely high in a public cloud environment. Third, Dropbox is essentially a cloud storage vendor, but one with a highly specialized storage product that combines object and **block storage** characteristics. Dropbox’s core competence is a differential file-update system that can efficiently synchronize actively edited files among users while minimizing network and

CPU usage. The product is not a good fit for object storage, block storage, or other standard cloud offerings. Dropbox has instead benefited from building a custom, highly integrated software and hardware stack. 6

Chúng tôi tin rằng nghiên cứu trường hợp này thường được dùng mà thiếu bối cảnh thích đáng và là một ví dụ thuyết phục về nguy hiểm logic tương đương giả (false equivalence). Dropbox cung cấp những dịch vụ đặc thù nơi việc sở hữu phần cứng và trung tâm dữ liệu có thể đem lại một lợi thế cạnh tranh. Các công ty không nên dựa dẫm quá mức vào ví dụ của Dropbox khi đánh giá các lựa chọn triển khai cloud và on-premises. Thứ nhất, điều quan trọng là phải hiểu rằng Dropbox lưu trữ những lượng dữ liệu khổng lồ. Công ty kín tiếng về việc chính xác họ lưu trữ bao nhiêu dữ liệu, nhưng nói rằng đó là nhiều exabyte và vẫn tiếp tục tăng. Thứ hai, Dropbox xử lý một lượng khổng lồ lưu lượng mạng. Chúng ta biết rằng mức tiêu thụ băng thông của họ năm 2017 đã lớn đến mức đủ để công ty bổ sung “hàng trăm gigabit kết nối internet với các nhà cung cấp truyền tải (transit providers) (các ISP khu vực và toàn cầu), và hàng trăm đối tác peering mới (nơi chúng tôi trao đổi lưu lượng trực tiếp thay vì thông qua một ISP)”. Chi phí xuất dữ liệu sẽ cực kỳ cao trong một môi trường đám mây công cộng. Thứ ba, Dropbox về cơ bản là một nhà cung cấp lưu trữ cloud, nhưng là một nhà cung cấp với một sản phẩm lưu trữ chuyên biệt cao kết hợp các đặc tính của lưu trữ đối tượng và lưu trữ khối (block storage). Năng lực cốt lõi của Dropbox là một hệ thống cập nhật-tệp theo sai biệt (differential file-update) có thể đồng bộ hóa hiệu quả các tệp đang được chỉnh sửa tích cực giữa những người dùng trong khi giảm thiểu việc sử dụng mạng và CPU. Sản phẩm này không phù hợp với lưu trữ đối tượng, lưu trữ khối hay các giải pháp cloud tiêu chuẩn khác. Thay vào đó, Dropbox đã hưởng lợi từ việc xây dựng một ngăn xếp phần mềm và phần cứng được tích hợp cao, làm riêng. 6

Fourth, while Dropbox moved its core product to its hardware, it continued building out other AWS workloads. This allows Dropbox to focus on building one highly tuned cloud service at an extraordinary scale rather than trying to replace multiple services. Dropbox can focus on its core competence in cloud storage and data synchronization while offloading software and hardware management in other areas, such as data analytics. 7

Thứ tư, trong khi Dropbox chuyển sản phẩm cốt lõi của mình sang phần cứng riêng, họ vẫn tiếp tục xây dựng các tải công việc AWS khác. Điều này cho phép Dropbox tập trung vào việc xây dựng một dịch vụ cloud được tinh chỉnh cao ở một quy mô phi thường thay vì cố gắng thay thế nhiều dịch vụ. Dropbox có thể tập trung vào năng lực cốt lõi của mình trong lưu trữ cloud và đồng bộ hóa dữ liệu trong khi chuyển gánh nặng quản lý phần mềm và phần cứng ở các lĩnh vực khác, chẳng hạn như phân tích dữ liệu. 7

Other frequently cited success stories that companies have built outside the cloud include Backblaze and Cloudflare, but these offer similar lessons. Backblaze began life as a personal cloud data backup product but has since begun to offer B2, an object storage service similar to Amazon S3. Backblaze currently stores over an exabyte of data. Cloudflare claims to provide services for over 25 million internet properties, with points of presence in over 200 cities and 51 terabits per second (Tbps) of total network capacity. Netflix offers yet another useful example. The company is famous for running its tech stack on AWS, but this is only partially true. Netflix does run video transcoding on AWS, accounting for roughly 70% of its compute needs in 2017. Netflix also runs its application backend and data analytics 8 on AWS. However, rather than using the AWS content distribution network, Netflix has built a custom CDN in collaboration with internet service providers, utilizing a highly specialized combination of software and hardware. For a company that consumes a substantial slice of all internet traffic, building out this critical infrastructure allowed it to deliver high- 9 quality video to a huge customer base cost-effectively. These case studies suggest that it makes sense for companies to manage their own hardware and network connections in particular circumstances.

The biggest modern success stories of companies building and maintaining hardware involve extraordinary scale (exabytes of data, terabits per second of bandwidth, etc.) and limited use cases where companies can realize a competitive advantage by engineering highly integrated hardware and software stacks. In addition, all of these companies consume massive network bandwidth, suggesting that data egress charges would be a major cost if they chose to operate fully from a public cloud. Consider continuing to run workloads on premises or repatriating cloud workloads if you run a truly cloud-scale service. What is cloud scale? You might be at cloud scale if you are storing an exabyte of data or handling terabits per second of traffic to and from the internet. (Achieving a terabit per second of internal network traffic is fairly easy.) In addition, consider owning your servers if data egress costs are a major factor for your business. To give a concrete example of cloud scale workloads that could benefit from repatriation, Apple might gain a significant financial and performance advantage by migrating iCloud storage to its own servers. 10

Các câu chuyện thành công khác thường được dẫn ra về việc các công ty đã xây dựng bên ngoài cloud bao gồm Backblaze và Cloudflare, nhưng những trường hợp này đưa ra các bài học tương tự. Backblaze khởi đầu như một sản phẩm sao lưu dữ liệu cloud cá nhân nhưng kể từ đó đã bắt đầu cung cấp B2, một dịch vụ lưu trữ đối tượng tương tự Amazon S3. Backblaze hiện lưu trữ hơn một exabyte dữ liệu. Cloudflare tuyên bố cung cấp dịch vụ cho hơn 25 triệu tài sản internet, với các điểm hiện diện (points of presence) tại hơn 200 thành phố và tổng năng lực mạng 51 terabit mỗi giây (Tbps). Netflix đưa ra thêm một ví dụ hữu ích nữa. Công ty nổi tiếng vì chạy ngấn xếp công nghệ của mình trên AWS, nhưng điều này chỉ đúng một phần. Netflix quả thực chạy việc chuyển mã video (video transcoding) trên AWS, chiếm khoảng 70% nhu cầu tính toán của họ vào năm 2017. Netflix cũng chạy backend ứng dụng và phân tích dữ liệu 8 trên AWS. Tuy nhiên, thay vì dùng mạng phân phối nội dung (content distribution network) của AWS, Netflix đã xây dựng một CDN riêng phối hợp với các nhà cung cấp dịch vụ internet, sử dụng một sự kết hợp chuyên biệt cao của phần mềm và phần cứng. Đối với một công ty tiêu thụ một lát đáng kể của toàn bộ lưu lượng internet, việc xây dựng nên hạ tầng trọng yếu này 9 cho phép họ phân phối video chất lượng cao đến một tệp khách hàng khổng lồ một cách hiệu quả về chi phí. Những nghiên cứu trường hợp này gợi ý rằng việc các công ty tự quản lý phần cứng và kết nối mạng của mình là hợp lý trong những hoàn cảnh đặc thù. Những câu chuyện thành công hiện đại lớn nhất về việc các công ty xây dựng và bảo trì phần cứng đều liên quan đến quy mô phi thường (exabyte dữ liệu, terabit mỗi giây bằng thông, v.v.) và các trường hợp sử dụng hạn chế nơi các công ty có thể hiện thực hóa một lợi thế cạnh tranh bằng cách thiết kế các ngấn xếp phần cứng và phần mềm tích hợp cao. Thêm vào đó, tất cả các công ty này tiêu thụ bằng thông mạng khổng lồ, gợi ý rằng các khoản phí xuất dữ liệu sẽ là một chi phí lớn nếu họ chọn vận hành hoàn toàn từ một đám mây công cộng. Hãy cân nhắc việc tiếp tục chạy các tải công việc on-premises hoặc hồi hương các tải công việc cloud nếu bạn vận hành một dịch vụ thực sự ở quy mô cloud. Quy mô cloud là gì? Bạn có thể đang ở quy mô cloud nếu bạn đang lưu trữ một exabyte dữ liệu hoặc xử lý terabit mỗi giây lưu lượng đến và đi từ *internet*. (Đạt được một terabit mỗi giây lưu lượng mạng *nội bộ* thì khá dễ.) Thêm vào đó, hãy cân nhắc sở hữu các máy chủ của bạn nếu chi phí xuất dữ liệu là một yếu tố lớn đối với doanh nghiệp của bạn. Để đưa ra một ví dụ cụ thể về các tải công việc quy mô cloud có thể hưởng lợi từ việc hồi hương, Apple có thể giành được một lợi thế tài chính và hiệu năng đáng kể bằng cách di trú lưu trữ iCloud sang các máy chủ của riêng mình. 10

Build Versus Buy — Tự xây dựng so với mua

Build versus buy is an age-old debate in technology. The argument for building is that you have end-to-end control over the solution and are not at the mercy of a vendor or open source community.

The argument supporting buying comes down to resource constraints and expertise; do you have the expertise to build a better solution than something already available? Either decision comes down to TCO, TOCO, and whether the solution provides a competitive advantage to your organization. If you've caught on to a theme in the book so far, it's that we suggest investing in building and customizing when doing so will provide a competitive advantage for your business. Otherwise, stand on the shoulders of giants and use what's already available in the market. Given the number of open source and paid services—both of which may have communities of volunteers or highly paid teams of amazing engineers—you're foolish to build everything yourself. As we often ask, "When you need new tires for your car, do you get the raw materials, build the tires from scratch, and install them yourself?" Like most people, you're probably buying tires and having someone install them. The same argument applies to build versus buy. We've seen teams that have built their databases from scratch. A simple open source RDBMS would have served their needs much better upon closer inspection. Imagine the amount of time and money invested in this homegrown database. Talk about low ROI for TCO and opportunity cost. This is where the distinction between the type A and type B data engineer comes in handy. As we pointed out earlier, type A and type B roles are often embodied in the same engineer, especially in a small organization. Whenever possible, lean toward type A behavior; avoid undifferentiated heavy lifting and embrace abstraction. Use open source frameworks, or if this is too much trouble, look at buying a suitable managed or proprietary solution. Plenty of great modular services are available to choose from in either case.

Tự xây dựng so với mua là một cuộc tranh luận lâu đời trong công nghệ. Lập luận cho việc tự xây dựng là bạn có quyền kiểm soát đầu-cuối đối với giải pháp và không phụ thuộc vào sự thương xót của một nhà cung cấp hay một cộng đồng mã nguồn mở. Lập luận ủng hộ việc mua quy về các ràng buộc nguồn lực và chuyên môn; bạn có chuyên môn để xây dựng một giải pháp tốt hơn thứ đã có sẵn không? Cả hai quyết định đều quy về TCO, TOCO, và liệu giải pháp có đem lại một lợi thế cạnh tranh cho tổ chức của bạn hay không. Nếu bạn đã nắm bắt được một chủ đề xuyên suốt cuốn sách cho đến giờ, thì đó là chúng tôi gợi ý đầu tư vào việc xây dựng và tùy biến khi làm vậy sẽ đem lại một lợi thế cạnh tranh cho doanh nghiệp của bạn. Còn không, hãy đứng trên vai những người khổng lồ và dùng cái đã có sẵn trên thị trường. Trước số lượng các dịch vụ mã nguồn mở và trả phí—cả hai đều có thể có những cộng đồng tình nguyện viên hoặc những đội ngũ kỹ sư tài ba được trả lương cao—bạn thật đại dốt nếu tự xây dựng mọi thứ. Như chúng tôi vẫn thường hỏi, "Khi bạn cần lốp mới cho xe của mình, bạn có đi mua nguyên vật liệu thô, tự chế tạo lốp từ đầu, rồi tự lắp chúng vào không?" Như hầu hết mọi người, có lẽ bạn đang mua lốp và để ai đó lắp chúng. Lập luận tương tự áp dụng cho tự xây dựng so với mua. Chúng tôi đã thấy các nhóm tự xây dựng cơ sở dữ liệu của họ từ đầu. Khi xem xét kỹ hơn, một RDBMS mã nguồn mở đơn giản hẳn đã phục vụ nhu cầu của họ tốt hơn nhiều. Hãy hình dung lượng thời gian và tiền bạc đã đầu tư vào cơ sở dữ liệu cây nhà lá vườn này. Nói chi đến ROI thấp về TCO và chi phí cơ hội. Đây là nơi sự phân biệt giữa kỹ sư dữ liệu loại A và loại B trở nên hữu ích. Như chúng tôi đã chỉ ra trước đó, các vai trò loại A và loại B thường được thể hiện trong cùng một kỹ sư, đặc biệt trong một tổ chức nhỏ. Bất cứ khi nào có thể, hãy nghiêng về hành vi loại A; tránh các công việc nặng nhọc không tạo khác biệt và đón nhận sự trừu tượng hóa. Hãy dùng các framework mã nguồn mở, hoặc nếu việc này quá rắc rối, hãy cân nhắc mua một giải pháp được quản lý hoặc độc quyền phù hợp. Có rất nhiều dịch vụ mô-đun tuyệt vời để chọn trong cả hai trường hợp.

The shifting reality of how companies adopt software is worth mentioning. Whereas in the past, IT used to make most of the software purchase and adoption decisions in a top-down manner, these days, the trend is for bottom-up software adoption in a company, driven by developers, data engineers, data scientists, and other technical roles. Technology adoption within companies is becoming an organic, continuous process. Let's look at some options for open source and proprietary

solutions.

Thực tế đang dịch chuyển về cách các công ty áp dụng phần mềm cũng đáng được đề cập. Trong khi trước đây, bộ phận CNTT thường đưa ra hầu hết các quyết định mua và áp dụng phần mềm theo kiểu từ trên xuống, ngày nay, xu hướng là áp dụng phần mềm từ dưới lên trong một công ty, được dẫn dắt bởi các nhà phát triển, kỹ sư dữ liệu, nhà khoa học dữ liệu và các vai trò kỹ thuật khác. Việc áp dụng công nghệ trong các công ty đang trở thành một quá trình hữu cơ, liên tục. Hãy nhìn vào một số lựa chọn cho các giải pháp mã nguồn mở và độc quyền.

Open Source Software — Phần mềm mã nguồn mở

Open source software (OSS) is a software distribution model in which software, and the underlying codebase, is made available for general use, typically under specific licensing terms. Often OSS is created and maintained by a distributed team of collaborators. OSS is free to use, change, and distribute most of the time, but with specific caveats. For example, many licenses require that the source code of open source–derived software be included when the software is distributed. The motivations for creating and maintaining OSS vary. Sometimes OSS is organic, springing from the mind of an individual or a small team that creates a novel solution and chooses to release it into the wild for public use. Other times, a company may make a specific tool or technology available to the public under an OSS license. OSS has two main flavors: community managed and commercial OSS.

Phần mềm mã nguồn mở (open source software, OSS) là một mô hình phân phối phần mềm trong đó phần mềm, cùng codebase nền tảng, được làm cho khả dụng để sử dụng chung, thường theo các điều khoản cấp phép cụ thể. OSS thường được tạo ra và bảo trì bởi một đội ngũ cộng tác viên phân tán. OSS thường là miễn phí để sử dụng, thay đổi và phân phối, nhưng kèm theo những điều kiện cụ thể. Ví dụ, nhiều giấy phép yêu cầu rằng mã nguồn của phần mềm phái sinh từ mã nguồn mở phải được bao gồm khi phần mềm được phân phối. Động cơ để tạo ra và bảo trì OSS rất đa dạng. Đôi khi OSS mang tính hữu cơ, nảy sinh từ tâm trí của một cá nhân hay một nhóm nhỏ tạo ra một giải pháp mới lạ và chọn phát hành nó ra ngoài cho công chúng sử dụng. Những lúc khác, một công ty có thể làm cho một công cụ hay công nghệ cụ thể trở nên khả dụng cho công chúng dưới một giấy phép OSS. OSS có hai hương vị chính: được cộng đồng quản lý (community managed) và OSS thương mại (commercial OSS).

Community-managed OSS — OSS được cộng đồng quản lý

OSS projects succeed with a strong community and vibrant user base. Community-managed OSS is a prevalent path for OSS projects. The community opens up high rates of innovations and contributions from developers worldwide with popular OSS projects. The following are factors to consider with a community-managed OSS project: Mindshare Avoid adopting OSS projects that don't have traction and popularity. Look at the number of GitHub stars, forks, and commit volume and recency. Another thing to pay attention to is community activity on related chat groups and forums. Does the project have a strong sense of community? A strong community creates a virtuous cycle of strong adoption. It also means that you'll have an easier time getting technical assistance and finding talent qualified to work with the framework.

Các dự án OSS thành công với một cộng đồng mạnh và một tệp người dùng sôi động. OSS được cộng đồng quản lý là một con đường phổ biến cho các dự án OSS. Cộng đồng mở ra những tỷ lệ đổi mới và đóng góp cao từ các nhà phát triển trên khắp thế giới với các dự án

OSS phổ biến. Sau đây là các yếu tố cần cân nhắc với một dự án OSS được cộng đồng quản lý: Sức ảnh hưởng (Mindshare) Hãy tránh áp dụng các dự án OSS không có đà phát triển và độ phổ biến. Hãy nhìn số lượng sao (star), fork trên GitHub, cùng khối lượng và độ gần đây của các commit. Một điều khác cần chú ý là hoạt động cộng đồng trên các nhóm chat và diễn đàn liên quan. Dự án có một cảm giác cộng đồng mạnh mẽ không? Một cộng đồng mạnh tạo ra một vòng tuần hoàn đức hạnh của sự áp dụng mạnh mẽ. Nó cũng có nghĩa là bạn sẽ dễ dàng hơn trong việc nhận được trợ giúp kỹ thuật và tìm được nhân tài đủ năng lực làm việc với framework đó.

Maturity

Độ trưởng thành (Maturity)

How long has the project been around, how active is it today, and how usable are people finding it in production? A project's maturity indicates that people find it useful and are willing to incorporate it into their production workflows.

Dự án đã tồn tại bao lâu, hiện nay nó hoạt động đến đâu, và mọi người thấy nó dùng được đến đâu trong sản xuất? Độ trưởng thành của một dự án cho thấy mọi người thấy nó hữu ích và sẵn lòng đưa nó vào các quy trình làm việc sản xuất của họ.

Troubleshooting

Khắc phục sự cố (Troubleshooting)

How will you have to handle problems if they arise? Are you on your own to troubleshoot issues, or can the community help you solve your problem?

Bạn sẽ phải xử lý các vấn đề như thế nào nếu chúng nảy sinh? Bạn tự thân khắc phục các vấn đề, hay cộng đồng có thể giúp bạn giải quyết vấn đề?

Project management

Quản lý dự án (Project management)

Look at Git issues and the way they're addressed. Are they addressed quickly? If so, what's the process to submit an issue and get it resolved?

Hãy nhìn các issue trên Git và cách chúng được xử lý. Chúng có được xử lý nhanh không? Nếu có, quy trình để gửi một issue và được giải quyết là gì?

Team

Đội ngũ (Team)

Is a company sponsoring the OSS project? Who are the core contributors?

Có công ty nào tài trợ cho dự án OSS không? Ai là các cộng tác viên cốt lõi?

Developer relations and community management What is the project doing to encourage uptake and adoption? Is there a vibrant Slack community that provides encouragement and support?

Quan hệ với nhà phát triển và quản lý cộng đồng (Developer relations and community management) Dự án đang làm gì để khuyến khích sự tiếp nhận và áp dụng? Có một cộng đồng Slack sôi động cung cấp sự động viên và hỗ trợ không?

Contributing

Đóng góp (Contributing)

Does the project encourage and accept pull requests?

Dự án có khuyến khích và chấp nhận các pull request không?

Roadmap

Lộ trình (Roadmap)

Is there a project roadmap? If so, is it clear and transparent?

Có một lộ trình dự án không? Nếu có, nó có rõ ràng và minh bạch không?

Self-hosting and maintenance

Tự host và bảo trì (Self-hosting and maintenance)

Do you have the resources to host and maintain the OSS solution? If so, what's the TCO and TOCO versus buying a managed service from the OSS vendor?

Bạn có các nguồn lực để host và bảo trì giải pháp OSS không? Nếu có, TCO và TOCO là bao nhiêu so với việc mua một dịch vụ được quản lý từ nhà cung cấp OSS?

Giving back to the community

Đóng góp lại cho cộng đồng (Giving back to the community)

If you like the project and are actively using it, consider investing in it. You can contribute to the codebase, help fix issues, and give advice in the community forums and chats. If the project allows donations, consider making one. Many OSS projects are essentially community-service projects, and the maintainers often have full-time jobs in addition to helping with the OSS project. Sadly, it's often a labor of love that doesn't afford the maintainer a living wage. If you can afford to donate, please do so.

Nếu bạn thích dự án và đang sử dụng nó một cách tích cực, hãy cân nhắc đầu tư vào nó. Bạn có thể đóng góp cho codebase, giúp sửa các issue, và đưa ra lời khuyên trong các diễn đàn cộng đồng và các kênh chat. Nếu dự án cho phép quyên góp, hãy cân nhắc quyên góp. Nhiều dự án OSS về cơ bản là các dự án phục vụ cộng đồng, và những người bảo trì thường có công việc toàn thời gian bên cạnh việc giúp đỡ dự án OSS. Đáng buồn thay, đây thường là một việc làm vì tình yêu mà không đem lại cho người bảo trì một mức thu nhập đủ sống. Nếu bạn có khả năng quyên góp, xin hãy làm điều đó.

Commercial OSS — OSS thương mại

Sometimes OSS has some drawbacks. Namely, you have to host and maintain the solution in your environment. This may be trivial or extremely complicated and cumbersome, depending on the OSS application. Commercial vendors try to solve this management headache by hosting and managing the OSS solution for you, typically as a cloud SaaS offering. Examples of such vendors include Databricks (Spark), Confluent (Kafka), DBT Labs (dbt), and there are many, many others. This model is called commercial OSS (COSS). Typically, a vendor will offer the “core” of the OSS for free while charging for enhancements, curated code distributions, or fully managed services. A vendor is often affiliated with the community OSS project. As an OSS project becomes more popular, the maintainers may create a separate business for a managed version of the OSS. This typically becomes a cloud SaaS platform built around a managed version of the open source code. This is a widespread trend: an OSS project becomes popular, an affiliated company raises truckloads of venture capital (VC) money to commercialize the OSS project, and the company scales as a fast-moving rocket ship. At this point, the data engineer has two options. You can continue using the community-managed OSS version, which you need to continue maintaining on your own (updates, server/container maintenance, pull requests for bug fixes, etc.). Or, you can pay the vendor and let it take care of

the administrative management of the COSS product. The following are factors to consider with a commercial OSS project: Value

Đôi khi OSS có một số nhược điểm. Cụ thể, bạn phải host và bảo trì giải pháp trong môi trường của mình. Việc này có thể là chuyện vặt hoặc cực kỳ phức tạp và cồng kềnh, tùy thuộc vào ứng dụng OSS. Các nhà cung cấp thương mại cố gắng giải quyết cơn đau đầu quản lý này bằng cách host và quản lý giải pháp OSS thay cho bạn, thường là dưới dạng một giải pháp cloud SaaS. Ví dụ về các nhà cung cấp như vậy bao gồm Databricks (Spark), Confluent (Kafka), DBT Labs (dbt), và còn rất, rất nhiều nhà cung cấp khác. Mô hình này được gọi là OSS thương mại (commercial OSS, COSS). Thông thường, một nhà cung cấp sẽ cung cấp “phần lõi” của OSS miễn phí trong khi tính tiền cho các phần nâng cao, các bản phân phối mã được tuyển chọn, hoặc các dịch vụ được quản lý đầy đủ. Một nhà cung cấp thường có liên kết với dự án OSS cộng đồng. Khi một dự án OSS trở nên phổ biến hơn, những người bảo trì có thể tạo ra một doanh nghiệp riêng cho một phiên bản được quản lý của OSS. Điều này thường trở thành một nền tảng cloud SaaS được xây dựng quanh một phiên bản được quản lý của mã nguồn mở. Đây là một xu hướng phổ biến rộng rãi: một dự án OSS trở nên phổ biến, một công ty có liên kết huy động cả đồng tiền đầu tư mạo hiểm (VC) để thương mại hóa dự án OSS, và công ty mở rộng quy mô như một con tàu tên lửa lao nhanh. Đến thời điểm này, kỹ sư dữ liệu có hai lựa chọn. Bạn có thể tiếp tục dùng phiên bản OSS được cộng đồng quản lý, thứ bạn cần tiếp tục tự bảo trì (cập nhật, bảo trì server/container, pull request để sửa lỗi, v.v.). Hoặc, bạn có thể trả tiền cho nhà cung cấp và để họ lo việc quản lý hành chính cho sản phẩm COSS. Sau đây là các yếu tố cần cân nhắc với một dự án OSS thương mại: Giá trị (Value)

Is the vendor offering a better value than if you managed the OSS technology yourself? Some vendors will add many bells and whistles to their managed offerings that aren't available in the community OSS version. Are these additions compelling to you?

Liệu nhà cung cấp có đem lại giá trị tốt hơn so với khi bạn tự quản lý công nghệ OSS không? Một số nhà cung cấp sẽ thêm nhiều tính năng phụ trợ vào các giải pháp được quản lý của họ mà không có trong phiên bản OSS cộng đồng. Những bổ sung này có thuyết phục đối với bạn không?

Delivery model

Mô hình phân phối (Delivery model)

How do you access the service? Is the product available via download, API, or web/mobile UI? Be sure you can easily access the initial version and subsequent releases.

Bạn truy cập dịch vụ như thế nào? Sản phẩm có khả dụng qua tải về, API, hay UI web/di động không? Hãy chắc chắn rằng bạn có thể dễ dàng truy cập phiên bản ban đầu và các bản phát hành sau đó.

Support

Hỗ trợ (Support)

Support cannot be understated, and it's often opaque to the buyer. What is the support model for the product, and is there an extra cost for support? Frequently, vendors will sell support for an additional fee. Be sure you clearly understand the costs of obtaining support. Also, understand what is covered in support, and what is not covered. Anything that's not covered by support will be your responsibility to own and manage.

Hỗ trợ không thể bị xem nhẹ, và nó thường mờ ám đối với người mua. Mô hình hỗ trợ cho sản phẩm là gì, và có chi phí phụ trội cho hỗ trợ không? Các nhà cung cấp thường bán hỗ

trợ với một khoản phí phụ trội. Hãy chắc chắn rằng bạn hiểu rõ các chi phí để có được hỗ trợ. Đồng thời, hãy hiểu cái gì được hỗ trợ bao phủ, và cái gì không được bao phủ. Bất cứ thứ gì không được hỗ trợ bao phủ sẽ là trách nhiệm của bạn để sở hữu và quản lý.

Releases and bug fixes

Các bản phát hành và sửa lỗi (Releases and bug fixes)

Is the vendor transparent about the release schedule, improvements, and bug fixes? Are these updates easily available to you?

Nhà cung cấp có minh bạch về lịch phát hành, các cải tiến và các bản sửa lỗi không? Những bản cập nhật này có dễ dàng khả dụng cho bạn không?

Sales cycle and pricing

Chu kỳ bán hàng và định giá (Sales cycle and pricing)

Often a vendor will offer on-demand pricing, especially for a SaaS product, and offer you a discount if you commit to an extended agreement. Be sure to understand the trade-offs of paying as you go versus paying up front. Is it worth paying a lump sum, or is your money better spent elsewhere?

Một nhà cung cấp thường sẽ cung cấp định giá theo nhu cầu (on-demand), đặc biệt cho một sản phẩm SaaS, và cho bạn một khoản giảm giá nếu bạn cam kết một thỏa thuận kéo dài. Hãy chắc chắn rằng bạn hiểu các đánh đổi giữa trả-theo-sử-dụng so với trả trước. Trả một khoản gộp có đáng không, hay tiền của bạn được chi tốt hơn ở chỗ khác?

Company finances

Tài chính của công ty (Company finances)

Is the company viable? If the company has raised VC funds, you can check their funding on sites like Crunchbase. How much runway does the company have, and will it still be in business in a couple of years?

Công ty có khả thi không? Nếu công ty đã huy động vốn VC, bạn có thể kiểm tra nguồn vốn của họ trên các trang như Crunchbase. Công ty còn bao nhiêu đường băng (runway), và liệu nó còn kinh doanh trong vài năm nữa không?

Logos versus revenue Is the company focused on growing the number of customers (logos), or is it trying to grow revenue? You may be surprised by the number of companies primarily concerned with growing their customer count, GitHub stars, or Slack channel membership without the revenue to establish sound finances.

Số logo so với doanh thu (Logos versus revenue) Công ty đang tập trung vào việc tăng số lượng khách hàng (logo), hay đang cố tăng doanh thu? Bạn có thể bất ngờ trước số lượng các công ty chủ yếu bận tâm đến việc tăng số lượng khách hàng, số sao GitHub, hay số thành viên kênh Slack mà không có doanh thu để thiết lập một nền tài chính lành mạnh.

Community support

Hỗ trợ cộng đồng (Community support)

Is the company truly supporting the community version of the OSS project? How much is the company contributing to the community OSS codebase? Controversies have arisen with certain vendors co-opting OSS projects and subsequently providing little value back to the community. How likely will the product remain viable as a community-supported open source if the company shuts down?

Liệu công ty có thực sự hỗ trợ phiên bản cộng đồng của dự án OSS không? Công ty đang đóng góp bao nhiêu cho codebase OSS cộng đồng? Đã nảy sinh những tranh cãi với một số nhà cung cấp nhất định chiếm dụng các dự án OSS rồi sau đó đem lại rất ít giá trị cho cộng đồng. Sản phẩm có khả năng vẫn còn khả thi như một mã nguồn mở được cộng đồng hỗ trợ đến đâu nếu công ty đóng cửa?

Note also that clouds offer their own managed open source products. If a cloud vendor sees traction with a particular product or project, expect that vendor to offer its version. This can range from simple examples (open source Linux offered on VMs) to extremely complex managed services (fully managed Kafka). The motivation for these offerings is simple: clouds make their money through consumption. More offerings in a cloud ecosystem mean a greater chance of “stickiness” and increased customer spending.

Cũng lưu ý rằng các đám mây cung cấp các sản phẩm mã nguồn mở được quản lý của riêng họ. Nếu một nhà cung cấp cloud thấy đà phát triển với một sản phẩm hay dự án cụ thể, hãy kỳ vọng nhà cung cấp đó sẽ cung cấp phiên bản của mình. Điều này có thể trải từ các ví dụ đơn giản (Linux mã nguồn mở được cung cấp trên các VM) đến các dịch vụ được quản lý cực kỳ phức tạp (Kafka được quản lý đầy đủ). Động cơ cho các giải pháp này rất đơn giản: các đám mây kiếm tiền thông qua mức tiêu dùng. Càng nhiều giải pháp trong một hệ sinh thái cloud thì càng có cơ hội lớn hơn cho sự “dính chặt” (stickiness) và tăng chi tiêu của khách hàng.

Proprietary Walled Gardens — Các khu vườn có tường bao độc quyền

While OSS is ubiquitous, a big market also exists for non-OSS technologies. Some of the biggest companies in the data industry sell closed source products. Let’s look at two major types of proprietary walled gardens, independent companies and cloud-platform offerings.

Trong khi OSS có mặt khắp nơi, cũng tồn tại một thị trường lớn cho các công nghệ không phải OSS. Một số công ty lớn nhất trong ngành dữ liệu bán các sản phẩm mã nguồn đóng. Hãy nhìn vào hai loại chính của các *khu vườn có tường bao độc quyền* (proprietary walled gardens), các công ty độc lập và các giải pháp nền tảng cloud.

Independent offerings — Các giải pháp độc lập

The data-tool landscape has seen exponential growth over the last several years. Every day, new independent offerings arise for data tools. With the ability to raise funds from VCs flush with capital, these data companies can scale and hire great engineering, sales, and marketing teams. This presents a situation where users have some great product choices in the marketplace while having to wade through endless sales and marketing clutter. Often a company selling a data tool will not release it as OSS, instead offering a proprietary solution. Although you won’t have the transparency of a pure OSS solution, a proprietary independent solution can work quite well, especially as a fully managed service in the cloud. The following are things to consider with an independent offering: Interoperability

Bối cảnh công cụ dữ liệu đã chứng kiến sự tăng trưởng theo cấp số nhân trong vài năm qua. Mỗi ngày, các giải pháp độc lập mới nảy sinh cho các công cụ dữ liệu. Với khả năng huy động vốn từ các quỹ VC dồi dào tiền mặt, các công ty dữ liệu này có thể mở rộng quy mô và thuê các đội ngũ kỹ thuật, bán hàng và tiếp thị tuyệt vời. Điều này tạo ra một tình

hướng nơi người dùng có một số lựa chọn sản phẩm tuyệt vời trên thị trường trong khi phải lợi qua mớ lộn xộn bất tận của bán hàng và tiếp thị. Một công ty bán một công cụ dữ liệu thường sẽ không phát hành nó dưới dạng OSS, mà thay vào đó cung cấp một giải pháp độc quyền. Mặc dù bạn sẽ không có sự minh bạch của một giải pháp OSS thuần túy, một giải pháp độc lập độc quyền có thể hoạt động khá tốt, đặc biệt khi là một dịch vụ được quản lý đầy đủ trong cloud. Sau đây là những điều cần cân nhắc với một giải pháp độc lập: Khả năng tương tác (Interoperability)

Make sure that the tool interoperates with other tools you've chosen (OSS, other independents, cloud offerings, etc.) Interoperability is key, so make sure you can try it before you buy.

Hãy chắc chắn rằng công cụ tương tác được với các công cụ khác mà bạn đã chọn (OSS, các giải pháp độc lập khác, các giải pháp cloud, v.v.). Khả năng tương tác là then chốt, vậy nên hãy chắc chắn rằng bạn có thể thử nó trước khi mua.

Mindshare and market share

Sức ảnh hưởng và thị phần (Mindshare and market share)

Is the solution popular? Does it command a presence in the marketplace? Does it enjoy positive customer reviews?

Giải pháp có phổ biến không? Nó có chiếm một vị thế trên thị trường không? Nó có nhận được các đánh giá tích cực từ khách hàng không?

Documentation and support

Tài liệu và hỗ trợ (Documentation and support)

Problems and questions will inevitably arise. Is it clear how to solve your problem, either through documentation or support?

Các vấn đề và câu hỏi chắc chắn sẽ nảy sinh. Có rõ ràng cách giải quyết vấn đề của bạn không, dù là qua tài liệu hay hỗ trợ?

Pricing

Định giá (Pricing)

Is the pricing understandable? Map out low-, medium-, and high-probability usage scenarios, with respective costs. Are you able to negotiate a contract, along with a discount? Is it worth it? How much flexibility do you lose if you sign a contract, both in negotiation and the ability to try new options? Are you able to obtain contractual commitments on future pricing?

Định giá có dễ hiểu không? Hãy phác ra các kịch bản sử dụng xác suất thấp, trung bình và cao, với các chi phí tương ứng. Bạn có thể đàm phán một hợp đồng, cùng với một khoản giảm giá không? Nó có đáng không? Bạn mất bao nhiêu linh hoạt nếu bạn ký một hợp đồng, cả trong đàm phán lẫn khả năng thử các lựa chọn mới? Bạn có thể có được các cam kết hợp đồng về định giá trong tương lai không?

Longevity

Tuổi thọ (Longevity)

Will the company survive long enough for you to get value from its product? If the company has raised money, search around for its funding situation. Look at user reviews. Ask friends and post questions on social networks about other users' experiences with the product. Make sure you know what you're getting into.

Liệu công ty có sống sót đủ lâu để bạn thu được giá trị từ sản phẩm của họ không? Nếu công ty đã huy động tiền, hãy tìm hiểu quanh về tình hình vốn của họ. Hãy nhìn các đánh giá của người dùng. Hỏi bạn bè và đăng câu hỏi trên các mạng xã hội về trải nghiệm của những người dùng khác với sản phẩm. Hãy chắc chắn rằng bạn biết mình đang dẫn thân vào điều gì.

Cloud platform proprietary service offerings — Các giải pháp dịch vụ độc quyền của nền tảng cloud

Cloud vendors develop and sell their proprietary services for storage, databases, and more. Many of these solutions are internal tools used by respective sibling companies. For example, Amazon created the database DynamoDB to overcome the limitations of traditional relational databases and handle the large amounts of user and order data as Amazon.com grew into a behemoth. Amazon later offered the DynamoDB service solely on AWS; it's now a top-rated product used by companies of all sizes and maturity levels. Cloud vendors will often bundle their products to work well together. Each cloud can create stickiness with its user base by creating a strong integrated ecosystem. The following are factors to consider with a proprietary cloud offering: Performance versus price comparisons

Các nhà cung cấp cloud phát triển và bán các dịch vụ độc quyền của họ cho lưu trữ, cơ sở dữ liệu, và nhiều hơn nữa. Nhiều trong số các giải pháp này là các công cụ nội bộ được dùng bởi các công ty anh em tương ứng. Ví dụ, Amazon đã tạo ra cơ sở dữ liệu DynamoDB để vượt qua các giới hạn của các cơ sở dữ liệu quan hệ truyền thống và xử lý những lượng lớn dữ liệu người dùng và đơn hàng khi Amazon.com lớn lên thành một gã khổng lồ. Sau đó Amazon đã cung cấp dịch vụ DynamoDB chỉ riêng trên AWS; giờ đây nó là một sản phẩm được đánh giá hàng đầu được dùng bởi các công ty thuộc mọi quy mô và mức độ trưởng thành. Các nhà cung cấp cloud thường sẽ đóng gói các sản phẩm của họ để chúng hoạt động ăn ý với nhau. Mỗi cloud có thể tạo ra sự dính chặt với tệp người dùng của mình bằng cách tạo ra một hệ sinh thái tích hợp mạnh mẽ. Sau đây là các yếu tố cần cân nhắc với một giải pháp cloud độc quyền: So sánh hiệu năng so với giá (Performance versus price comparisons)

Is the cloud offering substantially better than an independent or OSS version? What's the TCO of choosing a cloud's offering?

Liệu giải pháp cloud có tốt hơn đáng kể so với một phiên bản độc lập hay OSS không? TCO của việc chọn giải pháp của một cloud là bao nhiêu?

Purchase considerations

Các cân nhắc khi mua (Purchase considerations)

On-demand pricing can be expensive. Can you lower your cost by purchasing reserved capacity or entering into a long-term commitment agreement?

Định giá theo nhu cầu có thể đắt đỏ. Bạn có thể hạ thấp chi phí của mình bằng cách mua năng lực đặt giữ (reserved) hoặc ký một thỏa thuận cam kết dài hạn không?

Our Advice — Lời khuyên của chúng tôi

Build versus buy comes back to knowing your competitive advantage and where it makes sense to invest resources toward customization. In general, we favor OSS and COSS by default, which frees you to focus on improving those areas where these options are insufficient. Focus on a few

areas where building something will add significant value or reduce friction substantially. Don't treat internal operational overhead as a sunk cost. There's excellent value in upskilling your existing data team to build sophisticated systems on managed platforms rather than babysitting on-premises servers. In addition, think about how a company makes money, especially its sales and customer experience teams, which will generally indicate how you're treated during the sales cycle and when you're a paying customer. Finally, who is responsible for the budget at your company? How does this person decide the projects and technologies that get funded? Before making the business case for COSS or managed services, does it make sense to try to use OSS first? The last thing you want is for your technology choice to be stuck in limbo while waiting for budget approval. As the old saying goes, time kills deals. In your case, more time spent in limbo means a higher likelihood your budget approval will die. Know beforehand who controls the budget and what will successfully get approved.

Tự xây dựng so với mua quay trở về việc biết lợi thế cạnh tranh của bạn và nơi nào hợp lý để đầu tư nguồn lực vào việc tùy biến. Nhìn chung, chúng tôi ưu ái OSS và COSS theo mặc định, điều này giải phóng bạn để tập trung vào việc cải thiện những lĩnh vực nơi các lựa chọn này không đủ. Hãy tập trung vào một vài lĩnh vực nơi việc xây dựng một thứ gì đó sẽ tăng thêm giá trị đáng kể hoặc giảm ma sát một cách đáng kể. Đừng coi chi phí vận hành nội bộ như một chi phí chìm (sunk cost). Có giá trị tuyệt vời trong việc nâng cao kỹ năng cho nhóm dữ liệu hiện hữu của bạn để xây dựng các hệ thống tinh vi trên các nền tảng được quản lý thay vì trông chừng các máy chủ on-premises. Thêm vào đó, hãy nghĩ về cách một công ty kiếm tiền, đặc biệt là các nhóm bán hàng và trải nghiệm khách hàng của họ, điều này nhìn chung sẽ cho thấy cách bạn được đối xử trong chu kỳ bán hàng và khi bạn là một khách hàng trả tiền. Cuối cùng, ai chịu trách nhiệm về ngân sách ở công ty bạn? Người này quyết định các dự án và công nghệ nào được cấp vốn như thế nào? Trước khi xây dựng lý lẽ kinh doanh cho COSS hay các dịch vụ được quản lý, liệu có hợp lý không khi cố dùng OSS trước? Điều cuối cùng bạn muốn là lựa chọn công nghệ của bạn bị mắc kẹt trong tình trạng lằng lằng trong khi chờ phê duyệt ngân sách. Như câu nói cũ, *thời gian giết chết các thương vụ*. Trong trường hợp của bạn, càng nhiều thời gian trải qua trong tình trạng lằng lằng thì càng có khả năng cao hơn việc phê duyệt ngân sách của bạn sẽ chết. Hãy biết trước ai kiểm soát ngân sách và cái gì sẽ được phê duyệt thành công.

Monolith Versus Modular — Monolith so với mô-đun

Monoliths versus modular systems is another longtime debate in the software architecture space. Monolithic systems are self-contained, often performing multiple functions under a single system. The monolith camp favors the simplicity of having everything in one place. It's easier to reason about a single entity, and you can move faster because there are fewer moving parts. The modular camp leans toward decoupled, best-of-breed technologies performing tasks at which they are uniquely great. Especially given the rate of change in products in the data world, the argument is you should aim for interoperability among an ever-changing array of solutions. What approach should you take in your data engineering stack? Let's explore the trade-offs.

Monolith so với hệ thống mô-đun là một cuộc tranh luận lâu năm khác trong không gian kiến trúc phần mềm. Các hệ thống monolith là tự-chứa (self-contained), thường thực hiện nhiều chức năng trong một hệ thống duy nhất. Phe monolith ưa thích sự đơn giản của việc có mọi thứ ở một chỗ. Dễ suy luận về một thực thể duy nhất hơn, và bạn có thể di chuyển nhanh hơn vì có ít bộ phận chuyển động hơn. Phe mô-đun nghiêng về các công nghệ tách rời (decoupled), thuộc hàng tốt nhất, thực hiện những tác vụ mà chúng giỏi một cách độc nhất. Đặc biệt trước tốc độ thay đổi của các sản phẩm trong thế giới dữ liệu, lập luận là

bạn nên nhắm tới khả năng tương tác giữa một mảng các giải pháp luôn thay đổi. Bạn nên áp dụng cách tiếp cận nào trong ngăn xếp kỹ thuật dữ liệu của mình? Hãy khám phá các đánh đổi.

Monolith — Monolith

The monolith (Figure 4-4) has been a technology mainstay for decades. The old days of waterfall meant that software releases were huge, **tightly coupled**, and moved at a slow cadence. Large teams worked together to deliver a single working codebase. Monolithic data systems continue to this day, with older software vendors such as Informatica and open source frameworks such as Spark.

Monolith (Hình 4-4) đã là một trụ cột công nghệ trong nhiều thập kỷ. Những ngày xưa của mô hình thác nước (waterfall) đồng nghĩa với việc các bản phát hành phần mềm là đồ sộ, ghép chặt (tightly coupled), và di chuyển với một nhịp độ chậm chạp. Các nhóm lớn làm việc cùng nhau để giao một codebase hoạt động duy nhất. Các hệ thống dữ liệu monolith vẫn tiếp tục tồn tại đến ngày nay, với các nhà cung cấp phần mềm lâu đời hơn như Informatica và các framework mã nguồn mở như Spark.

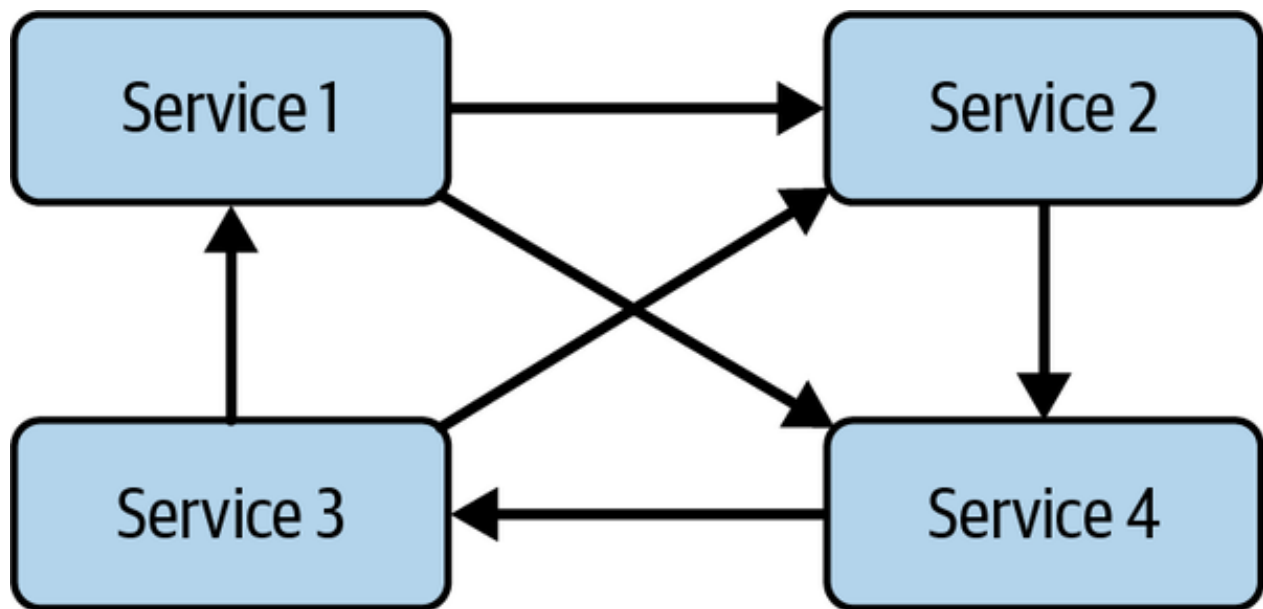


Figure 4-4. The monolith tightly couples its services The pros of the monolith are it's easy to reason about, and it requires a lower cognitive burden and context switching since everything is self-contained. Instead of dealing with dozens of technologies, you deal with “one” technology and typically one principal programming language. Monoliths are an excellent option if you want simplicity in reasoning about your architecture and processes.

Hình 4-4. Monolith ghép chặt các dịch vụ của nó. Các ưu điểm của monolith là nó dễ suy luận, và nó đòi hỏi một gánh nặng nhận thức và chuyển ngữ cảnh (context switching) thấp hơn vì mọi thứ đều tự-chứa. Thay vì phải xoay xở với hàng tá công nghệ, bạn xoay xở với “một” công nghệ và thường là một ngôn ngữ lập trình chính. Monolith là một lựa chọn tuyệt vời nếu bạn muốn sự đơn giản trong việc suy luận về kiến trúc và các quy trình của mình.

Of course, the monolith has cons. For one thing, monoliths are brittle. Because of the vast number of moving parts, updates and releases take longer and tend to bake in “the kitchen sink.” If the system

has a bug— hopefully, the software’s been thoroughly tested before release!—it can harm the entire system. User-induced problems also happen with monoliths. For example, we saw a monolithic **ETL** pipeline that took 48 hours to run. If anything broke anywhere in the pipeline, the entire process had to restart. Meanwhile, anxious business users were waiting for their reports, which were already two days late by default. Breakages were common enough that the monolithic system was eventually thrown out. Multitenancy in a monolithic system can also be a significant problem. It can be challenging to isolate the workloads of multiple users. In an on-prem **data warehouse**, one user-defined function might consume enough CPU to slow the system for other users. Conflicts between dependencies and resource contention are frequent sources of headaches. Another con of monoliths is that switching to a new system will be painful if the vendor or open source project dies. Because all of your processes are contained in the monolith, extracting yourself out of that system, and onto a new platform, will be costly in both time and money.

Tất nhiên, monolith có những nhược điểm. Một mặt, các monolith giòn (brittle). Vì số lượng khổng lồ các bộ phận chuyển động, các bản cập nhật và phát hành mất nhiều thời gian hơn và có xu hướng nhồi nhét “cả cái bồn rửa bát”. Nếu hệ thống có một lỗi—hy vọng là phần mềm đã được kiểm thử kỹ lưỡng trước khi phát hành!—nó có thể gây hại cho toàn bộ hệ thống. Các vấn đề do người dùng gây ra cũng xảy ra với monolith. Ví dụ, chúng tôi đã thấy một đường ống ETL monolith mất 48 giờ để chạy. Nếu bất cứ thứ gì hỏng ở bất cứ đâu trong đường ống, toàn bộ quá trình phải khởi động lại. Trong lúc đó, những người dùng kinh doanh lo lắng đang chờ các báo cáo của họ, vốn theo mặc định đã trễ hai ngày. Các sự cố hỏng hóc đủ thường xuyên đến mức hệ thống monolith cuối cùng bị vứt bỏ. Đa thuê bao (multitenancy) trong một hệ thống monolith cũng có thể là một vấn đề đáng kể. Có thể đầy thách thức để cô lập các tải công việc của nhiều người dùng. Trong một kho dữ liệu on-prem, một hàm do người dùng định nghĩa có thể tiêu tốn đủ CPU để làm chậm hệ thống đối với những người dùng khác. Các xung đột giữa các phụ thuộc (dependency) và sự tranh giành tài nguyên là những nguồn đau đầu thường xuyên. Một nhược điểm khác của monolith là việc chuyển sang một hệ thống mới sẽ đau đớn nếu nhà cung cấp hoặc dự án mã nguồn mở chết. Vì tất cả các quy trình của bạn được chứa trong monolith, việc lôi mình ra khỏi hệ thống đó, và lên một nền tảng mới, sẽ tốn kém cả về thời gian lẫn tiền bạc.

Modularity — Mô-đun hóa

Modularity (Figure 4-5) is an old concept in software engineering, but modular distributed systems truly came into vogue with the rise of microservices. Instead of relying on a massive monolith to handle your needs, why not break apart systems and processes into their self-contained areas of concern? Microservices can communicate via APIs, allowing developers to focus on their domains while making their applications accessible to other microservices. This is the trend in software engineering and is increasingly seen in modern data systems.

Mô-đun hóa (Hình 4-5) là một khái niệm cũ trong kỹ thuật phần mềm, nhưng các hệ phân tán mô-đun thực sự trở nên thịnh hành với sự trỗi dậy của microservices. Thay vì dựa vào một monolith đồ sộ để xử lý các nhu cầu của bạn, sao không tách rời các hệ thống và quy trình thành những khu vực quan tâm tự-chứa của chúng? Các microservice có thể giao tiếp qua các API, cho phép các nhà phát triển tập trung vào lĩnh vực của mình trong khi khiến các ứng dụng của họ có thể truy cập được đối với các microservice khác. Đây là xu hướng trong kỹ thuật phần mềm và ngày càng được thấy trong các hệ thống dữ liệu hiện đại.



Figure 4-5. With modularity, each service is decoupled from another. Major tech companies have been key drivers in the microservices movement. The famous Bezos API mandate decreases coupling between applications, allowing refactoring and decomposition. Bezos also imposed the two-pizza rule (no team should be so large that two pizzas can't feed the whole group). Effectively, this means that a team will have at most five members. This cap also limits the complexity of a team's domain of responsibility—in particular, the codebase that it can manage. Whereas an extensive monolithic application might entail a group of one hundred people, dividing developers into small groups of five requires that this application be broken into small, manageable, **loosely** coupled pieces. In a modular microservice environment, components are swappable, and it's possible to create a polyglot (multiprogramming language) application; a Java service can replace a service written in Python. Service customers need worry only about the technical specifications of the service API, not behind-the-scenes details of implementation. Data-processing technologies have shifted toward a modular model by providing strong support for interoperability. Data is stored in object storage in a standard format such as Parquet in data lakes and lakehouses. Any processing tool that supports the format can read the data and write processed results back into the lake for processing by another tool. Cloud data warehouses support interoperation with object storage through import/export using standard formats and external tables—i.e., queries run directly on data in a **data lake**. New technologies arrive on the scene at a dizzying rate in today's data ecosystem, and most get stale and outmoded quickly. Rinse and repeat. The ability to swap out tools as technology changes is invaluable. We view data modularity as a more powerful paradigm than monolithic data engineering. Modularity allows engineers to choose the best technology for each job or step along the pipeline.

Hình 4-5. Với mô-đun hóa, mỗi dịch vụ được tách rời khỏi dịch vụ khác. Các công ty công nghệ lớn đã là những động lực then chốt trong phong trào microservices. Chỉ thị API nổi tiếng của Bezos giảm sự ghép nối (coupling) giữa các ứng dụng, cho phép tái cấu trúc (refactor) và phân rã (decomposition). Bezos cũng áp đặt quy tắc hai-chiếc-pizza (không nhóm nào nên lớn đến mức hai chiếc pizza không thể nuôi cả nhóm). Về thực chất, điều này nghĩa là một nhóm sẽ có tối đa năm thành viên. Giới hạn này cũng hạn chế độ phức tạp của lĩnh vực trách nhiệm của một nhóm—cụ thể là codebase mà nó có thể quản lý. Trong khi một ứng dụng monolith đồ sộ có thể đòi hỏi một nhóm một trăm người, việc chia các nhà phát triển thành các nhóm nhỏ năm người đòi hỏi ứng dụng này phải được chia thành những mảnh nhỏ, dễ quản lý, ghép lỏng (loosely coupled). Trong một môi trường microservice mô-đun, các thành phần có thể hoán đổi, và có thể tạo ra một ứng dụng đa ngôn ngữ (polyglot, đa-ngôn-ngữ-lập-trình); một dịch vụ Java có thể thay thế một dịch vụ được viết bằng Python. Các khách hàng của dịch vụ chỉ cần lo lắng về các đặc tả kỹ thuật của API dịch vụ, không phải các chi tiết hậu trường của việc hiện thực. Các công nghệ xử lý dữ liệu đã dịch chuyển hướng về một mô hình mô-đun bằng cách cung cấp sự hỗ trợ mạnh mẽ cho khả năng tương tác. Dữ liệu được lưu trong lưu trữ đối tượng dưới một định dạng tiêu chuẩn như Parquet trong các hồ dữ liệu và lakehouse. Bất kỳ công cụ xử lý nào hỗ trợ định dạng đó đều có thể đọc dữ liệu và ghi các kết quả đã xử lý ngược lại vào hồ để một công cụ khác xử lý. Các kho dữ liệu cloud hỗ trợ tương tác với lưu trữ đối tượng thông qua nhập/xuất bằng các định dạng tiêu chuẩn và các bảng ngoài (external table)—tức là, các truy vấn chạy trực tiếp trên dữ liệu trong một hồ dữ liệu. Các công nghệ mới xuất hiện trên sân khấu với một tốc độ chóng mặt trong hệ sinh thái dữ liệu ngày nay, và hầu hết nhanh chóng trở nên cũ kỹ và lỗi thời. Lặp lại quy trình. Khả năng thay thế các công cụ khi công nghệ thay đổi là vô giá. Chúng tôi xem mô-đun hóa dữ liệu là một mô thức mạnh mẽ

hơn kỹ thuật dữ liệu monolith. Mô-đun hóa cho phép các kỹ sư chọn công nghệ tốt nhất cho mỗi công việc hoặc mỗi bước dọc theo đường ống.

The cons of modularity are that there's more to reason about. Instead of handling a single system of concern, now you potentially have countless systems to understand and operate. Interoperability is a potential headache; hopefully, these systems all play nicely together. This very problem led us to break out **orchestration** as a separate undercurrent instead of placing it under data management. Orchestration is also important for monolithic data architectures; witness the success of tools like BMC Software's Control-M in the traditional data warehousing space. But orchestrating five or ten tools is dramatically more complex than orchestrating one. Orchestration becomes the glue that binds data stack modules together.

Các nhược điểm của mô-đun hóa là có nhiều thứ hơn để suy luận. Thay vì xoay sở một hệ thống quan tâm duy nhất, giờ đây bạn có khả năng có vô số hệ thống để hiểu và vận hành. Khả năng tương tác là một cơn đau đầu tiềm tàng; hy vọng là tất cả các hệ thống này chơi đẹp với nhau. Chính vấn đề này đã dẫn chúng tôi đến việc tách điều phối (orchestration) ra thành một dòng chảy ngầm riêng biệt thay vì đặt nó dưới quản lý dữ liệu. Điều phối cũng quan trọng đối với các kiến trúc dữ liệu monolith; hãy chứng kiến thành công của các công cụ như Control-M của BMC Software trong không gian kho dữ liệu truyền thống. Nhưng điều phối năm hoặc mười công cụ thì phức tạp hơn đáng kể so với điều phối một công cụ. Điều phối trở thành chất keo gắn kết các mô-đun của ngăn xếp dữ liệu lại với nhau.

The Distributed Monolith Pattern — Mô thức Monolith phân tán

The distributed monolith pattern is a distributed architecture that still suffers from many of the limitations of monolithic architecture. The basic idea is that one runs a **distributed system** with different services to perform different tasks. Still, services and nodes share a common set of dependencies or a common codebase. One standard example is a traditional Hadoop cluster. A Hadoop cluster can simultaneously host several frameworks, such as Hive, Pig, or Spark. The cluster also has many internal dependencies. In addition, the cluster runs core Hadoop components: Hadoop common libraries, HDFS, YARN, and Java. In practice, a cluster often has one version of each component installed. A standard on-prem Hadoop system entails managing a common environment that works for all users and all jobs. Managing upgrades and installations is a significant challenge. Forcing jobs to upgrade dependencies risks breaking them; maintaining two versions of a framework entails extra complexity. Some modern Python-based orchestration technologies also suffer from this problem. While they utilize a highly decoupled and asynchronous architecture, every service runs the same codebase with the same dependencies. Any executor can execute any task, so a client library for a single task run in one **DAG** must be installed on the whole cluster. Orchestrating many tools entails installing client libraries for a host of APIs. Dependency conflicts are a constant problem. One solution to the problems of the distributed monolith is ephemeral infrastructure in a cloud setting. Each job gets its own temporary server or cluster installed with dependencies. Each cluster remains highly monolithic, but separating jobs dramatically reduces conflicts. For example, this pattern is now quite common for Spark with services like Amazon EMR and Google Cloud Dataproc. A second solution is to properly decompose the distributed monolith into multiple software environments using containers. We have more to say on containers in “Serverless Versus Servers” .

Mô thức monolith phân tán (distributed monolith) là một kiến trúc phân tán mà vẫn chịu nhiều trong số các giới hạn của kiến trúc monolith. Ý tưởng cơ bản là người ta chạy một hệ phân tán với các dịch vụ khác nhau để thực hiện các tác vụ khác nhau. Tuy vậy, các

dịch vụ và nút (node) vẫn chia sẻ một tập các phụ thuộc chung hoặc một codebase chung. Một ví dụ tiêu chuẩn là một cụm Hadoop truyền thống. Một cụm Hadoop có thể đồng thời host nhiều framework, như Hive, Pig hay Spark. Cụm này cũng có nhiều phụ thuộc nội bộ. Thêm vào đó, cụm chạy các thành phần Hadoop cốt lõi: các thư viện chung của Hadoop, HDFS, YARN và Java. Trên thực tế, một cụm thường có một phiên bản của mỗi thành phần được cài đặt. Một hệ thống Hadoop on-prem tiêu chuẩn đòi hỏi việc quản lý một môi trường chung hoạt động cho mọi người dùng và mọi tác vụ. Quản lý các bản nâng cấp và cài đặt là một thách thức đáng kể. Buộc các tác vụ phải nâng cấp các phụ thuộc có nguy cơ làm hỏng chúng; duy trì hai phiên bản của một framework đòi hỏi sự phức tạp phụ trội. Một số công nghệ điều phối dựa trên Python hiện đại cũng chịu vấn đề này. Trong khi chúng sử dụng một kiến trúc tách rời cao và bất đồng bộ (asynchronous), mọi dịch vụ đều chạy cùng một codebase với cùng các phụ thuộc. Bất kỳ executor nào cũng có thể thực thi bất kỳ tác vụ nào, vậy nên một thư viện client cho một tác vụ đơn lẻ chạy trong một DAG phải được cài đặt trên toàn bộ cụm. Điều phối nhiều công cụ đòi hỏi cài đặt các thư viện client cho cả một loạt API. Xung đột phụ thuộc là một vấn đề thường trực. Một giải pháp cho các vấn đề của monolith phân tán là hạ tầng tạm thời (ephemeral) trong một bối cảnh cloud. Mỗi tác vụ có server hoặc cụm tạm thời riêng của nó được cài đặt với các phụ thuộc. Mỗi cụm vẫn rất monolith, nhưng việc tách các tác vụ ra giảm đáng kể các xung đột. Ví dụ, mô thức này giờ đây khá phổ biến với Spark cùng các dịch vụ như Amazon EMR và Google Cloud Dataproc. Một giải pháp thứ hai là phân rã đúng cách monolith phân tán thành nhiều môi trường phần mềm bằng cách dùng container. Chúng tôi có thêm điều để nói về container trong “Serverless so với Servers”.

Our Advice — Lời khuyên của chúng tôi

While monoliths are attractive because of ease of understanding and reduced complexity, this comes at a high cost. The cost is the potential loss of flexibility, opportunity cost, and high-friction development cycles. Here are some things to consider when evaluating monoliths versus modular options: Interoperability

Trong khi các monolith hấp dẫn vì dễ hiểu và giảm độ phức tạp, điều này đi kèm một cái giá cao. Cái giá đó là khả năng mất đi sự linh hoạt, chi phí cơ hội, và các chu kỳ phát triển ma sát cao. Đây là một số điều cần cân nhắc khi đánh giá monolith so với các lựa chọn mô-đun: Khả năng tương tác (Interoperability)

Architect for sharing and interoperability.

Hãy thiết kế kiến trúc để chia sẻ và tương tác.

Avoiding the “bear trap”

Tránh “bẫy gấu” (Avoiding the “bear trap”)

Something that is easy to get into might be painful or impossible to escape.

Một thứ dễ sa vào có thể đau đớn hoặc bất khả thi để thoát ra.

Flexibility

Sự linh hoạt (Flexibility)

Things are moving so fast in the data space right now. Committing to a monolith reduces flexibility and reversible decisions.

Mọi thứ đang di chuyển quá nhanh trong không gian dữ liệu ngay lúc này. Cam kết với một monolith làm giảm sự linh hoạt và các quyết định có thể đảo ngược.

Serverless Versus Servers — Serverless so với Servers

A big trend for cloud providers is serverless , allowing developers and data engineers to run applications without managing servers behind the scenes. Serverless provides a quick time to value for the right use cases. For other cases, it might not be a good fit. Let's look at how to evaluate whether serverless is right for you.

Một xu hướng lớn đối với các nhà cung cấp cloud là serverless, cho phép các nhà phát triển và kỹ sư dữ liệu chạy các ứng dụng mà không phải quản lý các máy chủ ở hậu trường. Serverless cung cấp một thời gian-đến-giá-trị nhanh cho các trường hợp sử dụng phù hợp. Với các trường hợp khác, nó có thể không phù hợp. Hãy nhìn vào cách đánh giá liệu serverless có phù hợp với bạn hay không.

Serverless — Serverless

Though serverless has been around for quite some time, the serverless trend kicked off in full force with AWS Lambda in 2014. With the promise of executing small chunks of code on an as-needed basis without having to manage a server, serverless exploded in popularity. The main reasons for its popularity are cost and convenience. Instead of paying the cost of a server, why not just pay when your code is evoked? Serverless has many flavors. Though function as a service (FaaS) is wildly popular, serverless systems predate the advent of AWS Lambda. Google Cloud's BigQuery is serverless in that data engineers don't need to manage backend infrastructure, and the system scales to zero and scales up automatically to handle large queries. Just load data into the system and start querying. You pay for the amount of data your query consumes and a small cost to store your data. This payment model—paying for consumption and storage—is becoming more prevalent. When does serverless make sense? As with many other cloud services, it depends; and data engineers would do well to understand the details of cloud pricing to predict when serverless deployments will become expensive. Looking specifically at the case of AWS Lambda, various engineers have found hacks to run **batch** workloads at meager costs. On the other hand, serverless functions suffer from an inherent overhead inefficiency. Handling one event per function call at a high event rate can be catastrophically expensive.

Mặc dù serverless đã tồn tại khá lâu, xu hướng serverless thực sự bùng nổ mạnh mẽ với AWS Lambda vào năm 2014. Với lời hứa thực thi các đoạn mã nhỏ trên cơ sở khi-cần-thiết mà không phải quản lý một máy chủ, serverless đã bùng nổ về độ phổ biến. Các lý do chính cho độ phổ biến của nó là chi phí và sự tiện lợi. Thay vì trả chi phí cho một máy chủ, sao không chỉ trả tiền khi mã của bạn được gọi? Serverless có nhiều hương vị. Mặc dù hàm như một dịch vụ (function as a service, FaaS) cực kỳ phổ biến, các hệ thống serverless đã có trước sự ra đời của AWS Lambda. Google Cloud's BigQuery là serverless ở chỗ các kỹ sư dữ liệu không cần quản lý hạ tầng backend, và hệ thống thu nhỏ xuống không và tự mình lên để xử lý các truy vấn lớn. Chỉ cần nạp dữ liệu vào hệ thống và bắt đầu truy vấn. Bạn trả tiền cho lượng dữ liệu mà truy vấn của bạn tiêu thụ và một chi phí nhỏ để lưu trữ dữ liệu của bạn. Mô hình thanh toán này—trả tiền cho mức tiêu dùng và lưu trữ—đang trở nên phổ biến hơn. Khi nào serverless hợp lý? Cũng như nhiều dịch vụ cloud khác, còn tùy; và các kỹ sư dữ liệu sẽ làm tốt khi hiểu các chi tiết của định giá cloud để dự đoán khi nào các triển khai serverless sẽ trở nên đắt đỏ. Nhìn cụ thể vào trường hợp của AWS Lambda,

nhiều kỹ sư đã tìm ra các mảnh để chạy các tải công việc theo lô với chi phí rất thấp. Mặt 11 khác, các hàm serverless chịu một sự kém hiệu quả về chi phí phụ trội (overhead) cố hữu. Xử lý một sự kiện mỗi lần gọi hàm ở một tốc độ sự kiện cao có thể đắt đỏ một cách thảm khốc.

As with other areas of ops, it's critical to monitor and model. Monitor to determine cost per event in a real-world environment, and model using this cost per event to determine overall costs as event rates grow. Modeling should also include worst-case scenarios—what happens if my site gets hit by a bot swarm or DDoS attack?

Cũng như các lĩnh vực khác của ops, điều then chốt là giám sát và mô hình hóa. Hãy giám sát để xác định chi phí mỗi sự kiện trong một môi trường thực tế, và mô hình hóa bằng cách dùng chi phí mỗi sự kiện này để xác định tổng chi phí khi tốc độ sự kiện tăng lên. Việc mô hình hóa cũng nên bao gồm các kịch bản tệ nhất—điều gì xảy ra nếu trang của tôi bị một bầy bot hay một cuộc tấn công DDoS dội vào?

Containers — Container

In conjunction with serverless and microservices, containers are one of the most powerful trending operational technologies as of this writing. Containers play a role in both serverless and microservices. Containers are often referred to as lightweight virtual machines . Whereas a traditional VM wraps up an entire operating system, a container packages an isolated user space (such as a filesystem and a few processes); many such containers can coexist on a single host operating system. This provides some of the principal benefits of virtualization (i.e., dependency and code isolation) without the overhead of carrying around an entire operating system kernel. A single hardware node can host numerous containers with fine-grained resource allocations. At the time of this writing, containers continue to grow in popularity, along with Kubernetes, a container management system. Serverless environments typically run on containers behind the scenes. Indeed, Kubernetes is a kind of serverless environment because it allows developers and ops teams to deploy microservices without worrying about the details of the machines where they are deployed. Containers provide a partial solution to problems of the distributed monolith mentioned earlier in this chapter. For example, Hadoop now supports containers, allowing each job to have its own isolated dependencies.

Cùng với serverless và microservices, container là một trong những công nghệ vận hành đang thịnh hành mạnh mẽ nhất tại thời điểm viết cuốn sách này. Container đóng một vai trò trong cả serverless lẫn microservices. Container thường được gọi là *các máy ảo nhẹ* (lightweight virtual machines). Trong khi một VM truyền thống gói gọn cả một hệ điều hành, một container đóng gói một không gian người dùng được cô lập (chẳng hạn như một hệ thống tệp và một vài tiến trình); nhiều container như vậy có thể cùng tồn tại trên một hệ điều hành host duy nhất. Điều này cung cấp một số trong các lợi ích chính của ảo hóa (tức là, sự cô lập phụ thuộc và mã) mà không có chi phí phụ trội của việc vác theo cả một nhân (kernel) hệ điều hành. Một nút phần cứng duy nhất có thể host vô số container với sự phân bổ tài nguyên chi tiết. Tại thời điểm viết, container tiếp tục tăng về độ phổ biến, cùng với Kubernetes, một hệ thống quản lý container. Các môi trường serverless thường chạy trên container ở hậu trường. Thật vậy, Kubernetes là một kiểu môi trường serverless vì nó cho phép các nhà phát triển và các nhóm ops triển khai các microservice mà không phải lo lắng về các chi tiết của các máy nơi chúng được triển khai. Container cung cấp một giải pháp một phần cho các vấn đề của monolith phân tán đã đề cập trước đó trong chương này. Ví dụ, Hadoop giờ đây hỗ trợ container, cho phép mỗi tác vụ có các phụ thuộc cô lập của riêng nó.

WARNING — CẢNH BÁO

Container clusters do not provide the same security and isolation that full VMs offer. Container escape—broadly, a class of exploits whereby code in a container gains privileges outside the container at the OS level—is common enough to be considered a risk for multitenancy. While Amazon EC2 is a truly **multitenant** environment with VMs from many customers hosted on the same hardware, a Kubernetes cluster should host code only within an environment of mutual trust (e.g., inside the walls of a single company). In addition, code review processes and vulnerability scanning are critical to ensure that a developer doesn't introduce a security hole.

Các cụm container không cung cấp cùng mức bảo mật và cô lập mà các VM đầy đủ đem lại. *Thoát container* (container escape)—nói rộng ra, một lớp các khai thác (exploit) theo đó mã trong một container giành được các đặc quyền bên ngoài container ở mức hệ điều hành—đủ phổ biến để bị coi là một rủi ro cho đa thuê bao. Trong khi Amazon EC2 là một môi trường đa thuê bao thực sự với các VM từ nhiều khách hàng được host trên cùng phần cứng, một cụm Kubernetes chỉ nên host mã trong một môi trường tin cậy lẫn nhau (ví dụ, bên trong các bức tường của một công ty duy nhất). Thêm vào đó, các quy trình review mã và quét lỗ hổng là then chốt để đảm bảo rằng một nhà phát triển không đưa vào một lỗ hổng bảo mật.

Various flavors of container platforms add additional serverless features. Containerized function platforms run containers as ephemeral units triggered by events rather than persistent services. This gives users the simplicity of AWS Lambda with the full flexibility of a container environment instead of the highly restrictive Lambda runtime. And services such as AWS Fargate and Google App Engine run containers without managing a compute cluster required for Kubernetes. These services also fully isolate containers, preventing the security issues associated with multitenancy. Abstraction will continue working its way across the data stack. Consider the impact of Kubernetes on cluster management. While you can manage your Kubernetes cluster—and many engineering teams do so—even Kubernetes is widely available as a managed service.

Nhiều hương vị của các nền tảng container thêm vào các tính năng serverless bổ sung. Các nền tảng hàm được container hóa (containerized function platforms) chạy các container như những đơn vị tạm thời được kích hoạt bởi các sự kiện thay vì các dịch vụ thường trực. Điều này trao cho người dùng sự đơn giản của AWS Lambda với sự linh hoạt đầy đủ của một môi trường container thay vì runtime hạn chế cao của Lambda. Và các dịch vụ như AWS Fargate và Google App Engine chạy các container mà không phải quản lý một cụm tính toán vốn cần thiết cho Kubernetes. Các dịch vụ này cũng cô lập hoàn toàn các container, ngăn chặn các vấn đề bảo mật liên quan đến đa thuê bao. Sự trừu tượng hóa sẽ tiếp tục lan dần khắp ngăn xếp dữ liệu. Hãy cân nhắc tác động của Kubernetes đối với việc quản lý cụm. Trong khi bạn có thể quản lý cụm Kubernetes của mình—và nhiều nhóm kỹ thuật làm vậy—ngay cả Kubernetes cũng khả dụng rộng rãi dưới dạng một dịch vụ được quản lý.

When Infrastructure Makes Sense — Khi nào hạ tầng là hợp lý

Why would you want to run your own servers instead of using serverless? There are a few reasons. Cost is a big factor. Serverless makes less sense when the usage and cost exceed the ongoing cost of running and maintaining a server (Figure 4-6). However, at a certain scale, the economic benefits of serverless may diminish, and running servers becomes more attractive.

Tại sao bạn lại muốn chạy các máy chủ của riêng mình thay vì dùng serverless? Có một

vài lý do. Chi phí là một yếu tố lớn. Serverless trở nên kém hợp lý khi mức sử dụng và chi phí vượt quá chi phí liên tục của việc chạy và bảo trì một máy chủ (Hình 4-6). Tuy nhiên, ở một quy mô nhất định, các lợi ích kinh tế của serverless có thể giảm dần, và việc chạy các máy chủ trở nên hấp dẫn hơn.

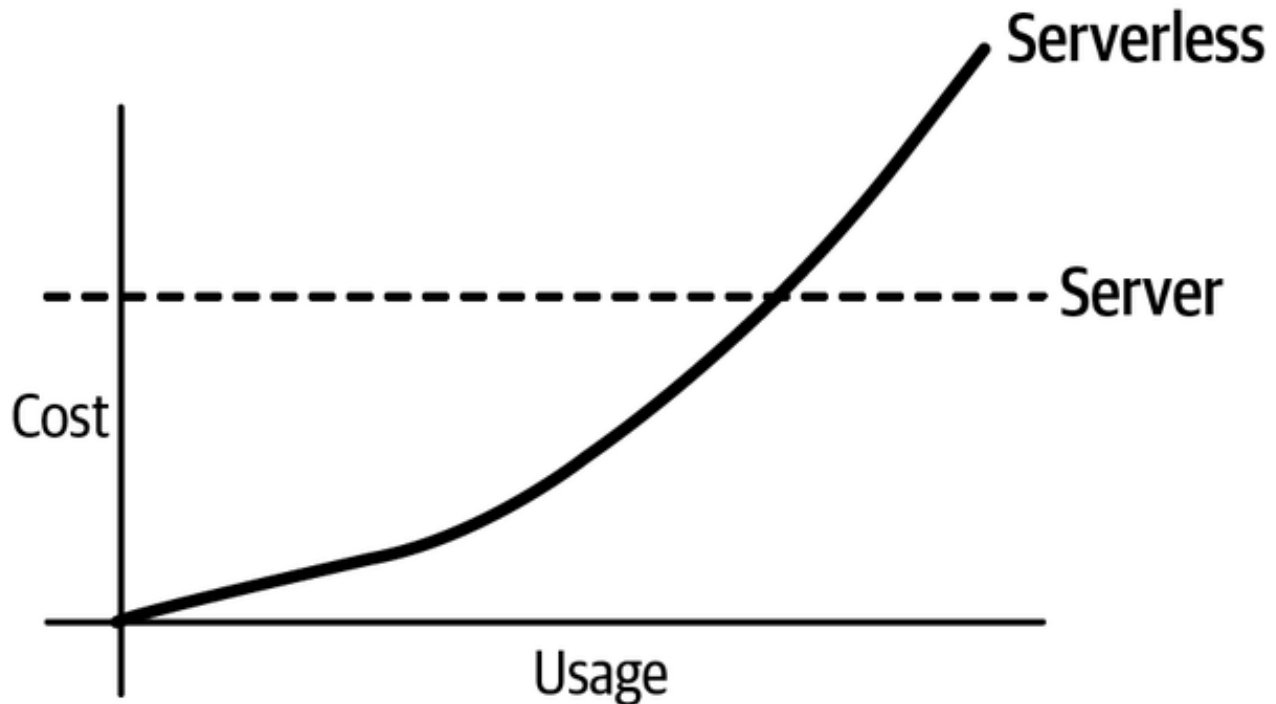


Figure 4-6. Cost of serverless versus utilizing a server Customization, power, and control are other major reasons to favor servers over serverless. Some serverless frameworks can be underpowered or limited for certain use cases. Here are some things to consider when using servers, particularly in the cloud, where server resources are ephemeral: Expect servers to fail.

Hình 4-6. Chi phí của serverless so với việc sử dụng một máy chủ. Sự tùy biến, sức mạnh và quyền kiểm soát là những lý do lớn khác để ưu ái máy chủ hơn serverless. Một số framework serverless có thể bị thiếu sức mạnh hoặc bị giới hạn đối với một số trường hợp sử dụng nhất định. Đây là một số điều cần cân nhắc khi dùng máy chủ, đặc biệt trong cloud, nơi tài nguyên máy chủ là tạm thời: Hãy kỳ vọng máy chủ sẽ hỏng.

Server failure will happen. Avoid using a “special snowflake” server that is overly customized and brittle, as this introduces a glaring vulnerability in your architecture. Instead, treat servers as ephemeral resources that you can create as needed and then delete. If your application requires specific code to be installed on the server, use a boot script or build an image. Deploy code to the server through a CI/CD pipeline.

Lỗi máy chủ sẽ xảy ra. Hãy tránh dùng một máy chủ “bông tuyết đặc biệt” được tùy biến quá mức và giòn, vì điều này đưa vào một lỗ hổng lộ liễu trong kiến trúc của bạn. Thay vào đó, hãy đối xử với máy chủ như các tài nguyên tạm thời mà bạn có thể tạo khi cần rồi sau đó xóa đi. Nếu ứng dụng của bạn đòi hỏi mã cụ thể phải được cài đặt trên máy chủ, hãy dùng một script khởi động (boot script) hoặc xây dựng một image. Hãy triển khai mã lên máy chủ thông qua một đường ống CI/CD.

Use clusters and autoscaling.

Hãy dùng cụm và tự mở rộng quy mô.

Take advantage of the cloud's ability to grow and shrink compute resources on demand. As your application increases its usage, cluster your application servers, and use autoscaling capabilities to automatically horizontally scale your application as demand grows.

Hãy tận dụng khả năng của cloud trong việc tăng và giảm các tài nguyên tính toán theo nhu cầu. Khi ứng dụng của bạn tăng mức sử dụng, hãy gom cụm các máy chủ ứng dụng của bạn, và dùng các khả năng tự mở rộng quy mô để tự động mở rộng quy mô ngang ứng dụng của bạn khi nhu cầu tăng.

Treat your infrastructure as code.

Hãy đối xử với hạ tầng của bạn như mã.

Automation doesn't apply to just servers and should extend to your infrastructure whenever possible. Deploy your infrastructure (servers or otherwise) using deployment managers such as Terraform, AWS CloudFormation, and Google Cloud Deployment Manager.

Tự động hóa không chỉ áp dụng cho máy chủ và nên mở rộng đến hạ tầng của bạn bất cứ khi nào có thể. Hãy triển khai hạ tầng của bạn (máy chủ hay thứ khác) bằng cách dùng các trình quản lý triển khai như Terraform, AWS CloudFormation và Google Cloud Deployment Manager.

Use containers.

Hãy dùng container.

For more sophisticated or heavy-duty workloads with complex installed dependencies, consider using containers on either a single server or Kubernetes.

Đối với các tải công việc tinh vi hơn hoặc nặng nề hơn với các phụ thuộc được cài đặt phức tạp, hãy cân nhắc dùng container trên hoặc một máy chủ đơn hoặc Kubernetes.

Our Advice — Lời khuyên của chúng tôi

Here are some key considerations to help you determine whether serverless is right for you: Workload size and complexity

Đây là một số cân nhắc then chốt để giúp bạn xác định liệu serverless có phù hợp với bạn hay không: Quy mô và độ phức tạp của tải công việc (Workload size and complexity)

Serverless works best for simple, discrete tasks and workloads. It's not as suitable if you have many moving parts or require a lot of compute or memory horsepower. In that case, consider using containers and a container workflow orchestration framework like Kubernetes.

Serverless hoạt động tốt nhất cho các tác vụ và tải công việc đơn giản, rời rạc. Nó không phù hợp lắm nếu bạn có nhiều bộ phận chuyển động hoặc đòi hỏi nhiều sức mạnh tính toán hoặc bộ nhớ. Trong trường hợp đó, hãy cân nhắc dùng container và một framework điều phối quy trình làm việc container như Kubernetes.

Execution frequency and duration How many requests per second will your serverless application process? How long will each request take to process? Cloud serverless platforms have limits on execution frequency, concurrency, and duration. If your application can't function neatly within these limits, it is time to consider a container-oriented approach.

Tần suất và thời lượng thực thi (Execution frequency and duration) Bao nhiêu yêu cầu mỗi giây mà ứng dụng serverless của bạn sẽ xử lý? Mỗi yêu cầu sẽ mất bao lâu để xử lý? Các nền tảng serverless cloud có các giới hạn về tần suất thực thi, mức đồng thời (concurrency)

và thời lượng. Nếu ứng dụng của bạn không thể hoạt động gọn gàng trong các giới hạn này, đã đến lúc cân nhắc một cách tiếp cận hướng container.

Requests and networking

Yêu cầu và mạng (Requests and networking)

Serverless platforms often utilize some form of simplified networking and don't support all cloud virtual networking features, such as VPCs and firewalls.

Các nền tảng serverless thường sử dụng một hình thức nào đó của mạng đơn giản hóa và không hỗ trợ tất cả các tính năng mạng ảo của cloud, chẳng hạn như VPC và tường lửa.

Language

Ngôn ngữ (Language)

What language do you typically use? If it's not one of the officially supported languages supported by the serverless platform, you should consider containers instead.

Bạn thường dùng ngôn ngữ gì? Nếu nó không phải một trong các ngôn ngữ được hỗ trợ chính thức bởi nền tảng serverless, bạn nên cân nhắc container thay thế.

Runtime limitations

Các giới hạn về runtime (Runtime limitations)

Serverless platforms don't give you complete operating system abstractions. Instead, you're limited to a specific runtime image.

Các nền tảng serverless không cho bạn các trừu tượng hệ điều hành hoàn chỉnh. Thay vào đó, bạn bị giới hạn ở một image runtime cụ thể.

Cost

Chi phí (Cost)

Serverless functions are incredibly convenient but potentially expensive. When your serverless function processes only a few events, your costs are low; costs rise rapidly as the event count increases. This scenario is a frequent source of surprise cloud bills.

Các hàm serverless cực kỳ tiện lợi nhưng có khả năng đắt đỏ. Khi hàm serverless của bạn chỉ xử lý một vài sự kiện, chi phí của bạn thấp; chi phí tăng nhanh khi số lượng sự kiện tăng. Kịch bản này là một nguồn thường xuyên của các hóa đơn cloud gây bất ngờ.

In the end, abstraction tends to win. We suggest looking at using serverless first and then servers—with containers and orchestration if possible—once you've outgrown serverless options.

Rốt cuộc, sự trừu tượng hóa có xu hướng thắng. Chúng tôi gợi ý cân nhắc dùng serverless trước rồi đến máy chủ—with container và điều phối nếu có thể—một khi bạn đã vượt qua các lựa chọn serverless.

Optimization, Performance, and the Benchmark Wars — Tối ưu hóa, hiệu năng và các cuộc chiến benchmark

Imagine that you are a billionaire shopping for new transportation. You've narrowed your choice to two options:

Hãy hình dung rằng bạn là một tỷ phú đang đi mua phương tiện di chuyển mới. Bạn đã thu hẹp lựa chọn của mình xuống còn hai phương án:

787 Business Jet

Máy bay phản lực thương gia 787

Range: 9,945 nautical miles (with 25 passengers)

Tầm bay: 9.945 hải lý (với 25 hành khách)

Maximum speed: 0.90 Mach

Tốc độ tối đa: 0,90 Mach

Cruise speed: 0.85 Mach

Tốc độ hành trình: 0,85 Mach

Fuel capacity: 101,323 kilograms

Sức chứa nhiên liệu: 101.323 kilogram

Maximum takeoff weight: 227,930 kilograms

Trọng lượng cất cánh tối đa: 227.930 kilogram

Maximum thrust: 128,000 pounds

Lực đẩy tối đa: 128.000 pound

Tesla Model S Plaid

Tesla Model S Plaid

Range: 560 kilometers

Tầm hoạt động: 560 kilomét

Maximum speed: 322 kilometers/hour

Tốc độ tối đa: 322 kilomét/giờ

0–100 kilometers/hour: 2.1 seconds

0–100 kilomét/giờ: 2,1 giây

Battery capacity: 100 kilowatt hours

Dung lượng pin: 100 kilowatt giờ

Nurburgring lap time: 7 minutes, 30.9 seconds

Thời gian một vòng Nurburgring: 7 phút, 30,9 giây

Horsepower: 1020

Mã lực: 1020

Torque: 1050 lb-ft

Mô-men xoắn: 1050 lb-ft

Which of these options offers better performance? You don't have to know much about cars or aircraft to recognize that this is an idiotic comparison.

Phương án nào trong số này cho hiệu năng tốt hơn? Bạn không cần biết nhiều về xe hơi hay máy bay để nhận ra rằng đây là một sự so sánh ngớ ngẩn.

One option is a wide-body private jet designed for intercontinental operation, while the other is an electric supercar. We see such apples-to-oranges comparisons made all the time in the database space. Benchmarks either compare databases that are optimized for completely different use cases, or use test scenarios that bear no resemblance to real-world needs. Recently, we saw a new round of benchmark wars flare up among major vendors in the data space. We applaud benchmarks and are glad to see many database vendors finally dropping DeWitt clauses from their customer contracts. Even so, let the buyer beware: the data space is full of 13 nonsensical benchmarks. Here are a few common tricks used to place a 14 thumb on the benchmark scale.

Một phương án là một chiếc máy bay riêng thân rộng được thiết kế cho vận hành xuyên lục địa, trong khi phương án kia là một chiếc siêu xe điện. Chúng ta thấy những sự so sánh kiểu táo-với-cam như vậy được thực hiện suốt trong không gian cơ sở dữ liệu. Các benchmark hoặc so sánh các cơ sở dữ liệu vốn được tối ưu cho những trường hợp sử dụng hoàn toàn khác nhau, hoặc dùng các kịch bản kiểm thử chẳng giống chút nào với các nhu cầu thực tế. Gần đây, chúng tôi đã thấy một vòng mới của các cuộc chiến benchmark bùng lên giữa các nhà cung cấp lớn trong không gian dữ liệu. Chúng tôi hoan nghênh các benchmark và vui mừng khi thấy nhiều nhà cung cấp cơ sở dữ liệu cuối cùng cũng loại bỏ các điều khoản DeWitt khỏi các hợp đồng khách hàng của họ. Dù vậy, người mua hãy coi chừng: không gian dữ liệu đầy rẫy 13 các benchmark vô nghĩa. Đây là một vài mảnh khoe 14 phổ biến được dùng để đặt một ngón tay cái lên cán cân benchmark.

Big Data...for the 1990s — Dữ liệu lớn... cho thập niên 1990

Products that claim to support “big data” at petabyte scale will often use benchmark datasets small enough to easily fit in the storage on your smartphone. For systems that rely on caching layers to deliver performance, test datasets fully reside in solid-state drive (SSD) or memory, and benchmarks can show ultra-high performance by repeatedly querying the same data. A small test dataset also minimizes RAM and SSD costs when comparing pricing. To benchmark for real-world use cases, you must simulate anticipated real-world data and query size. Evaluate query performance and resource costs based on a detailed evaluation of your needs.

Các sản phẩm tuyên bố hỗ trợ “dữ liệu lớn” (big data) ở quy mô petabyte thường sẽ dùng các tập dữ liệu benchmark đủ nhỏ để dễ dàng vừa với bộ nhớ trên điện thoại thông minh của bạn. Đối với các hệ thống dựa vào các lớp cache để đem lại hiệu năng, các tập dữ liệu kiểm thử nằm trọn vẹn trong ổ đĩa thể rắn (SSD) hoặc bộ nhớ, và các benchmark có thể cho thấy hiệu năng siêu cao bằng cách lặp đi lặp lại việc truy vấn cùng một dữ liệu. Một tập dữ liệu kiểm thử nhỏ cũng giảm thiểu chi phí RAM và SSD khi so sánh định giá. Để benchmark cho các trường hợp sử dụng thực tế, bạn phải mô phỏng dữ liệu và kích thước truy vấn thực tế dự kiến. Hãy đánh giá hiệu năng truy vấn và chi phí tài nguyên dựa trên một đánh giá chi tiết về các nhu cầu của bạn.

Nonsensical Cost Comparisons — Các so sánh chi phí vô nghĩa

Nonsensical cost comparisons are a standard trick when analyzing a price/performance or TCO. For instance, many MPP systems can’t be readily created and deleted even when they reside in a cloud environment; these systems run for years on end once they’ve been configured. Other databases support a dynamic compute model and charge either per query or per second of use. Comparing

ephemeral and non-ephemeral systems on a cost-per-second basis is nonsensical, but we see this all the time in benchmarks.

Các so sánh chi phí vô nghĩa là một mảnh khoe tiêu chuẩn khi phân tích một tỷ lệ giá/hiệu năng hoặc TCO. Chẳng hạn, nhiều hệ thống MPP không thể dễ dàng được tạo và xóa ngay cả khi chúng nằm trong một môi trường cloud; các hệ thống này chạy liên năm này qua năm khác một khi chúng đã được cấu hình. Các cơ sở dữ liệu khác hỗ trợ một mô hình tính toán động và tính tiền hoặc theo mỗi truy vấn hoặc theo mỗi giây sử dụng. So sánh các hệ thống tạm thời và không-tạm-thời trên cơ sở chi-phí-mỗi-giây là vô nghĩa, nhưng chúng ta thấy điều này suốt trong các benchmark.

Asymmetric Optimization — Tối ưu hóa bất đối xứng

The deceit of asymmetric optimization appears in many guises, but here's one example. Often a vendor will compare a row-based MPP system against a columnar database by using a benchmark that runs complex join queries on highly normalized data. The normalized data model is optimal for the row-based system, but the columnar system would realize its full potential only with some **schema** changes. To make matters worse, vendors juice their systems with an extra shot of join optimization (e.g., pre-indexing joins) without applying comparable tuning in the competing database (e.g., putting joins in a materialized view).

Sự lừa dối của tối ưu hóa bất đối xứng (asymmetric optimization) xuất hiện dưới nhiều lối khác nhau, nhưng đây là một ví dụ. Một nhà cung cấp thường sẽ so sánh một hệ thống MPP dựa trên hàng (row-based) với một cơ sở dữ liệu theo cột (columnar) bằng cách dùng một benchmark chạy các truy vấn join phức tạp trên dữ liệu được chuẩn hóa cao. Mô hình dữ liệu được chuẩn hóa là tối ưu cho hệ thống dựa trên hàng, nhưng hệ thống theo cột sẽ chỉ hiện thực hóa toàn bộ tiềm năng của nó với một số thay đổi lược đồ. Tệ hơn nữa, các nhà cung cấp tiếp thêm cho các hệ thống của họ một liều tối ưu join (ví dụ, lập chỉ mục trước cho các join) mà không áp dụng sự tinh chỉnh tương đương trong cơ sở dữ liệu đối thủ (ví dụ, đặt các join vào một view được vật chất hóa).

Caveat Emptor — Người mua hãy coi chừng

As with all things in data technology, let the buyer beware. Do your homework before blindly relying on vendor benchmarks to evaluate and choose technology.

Như với mọi thứ trong công nghệ dữ liệu, người mua hãy coi chừng. Hãy làm bài tập về nhà của bạn trước khi mù quáng dựa vào các benchmark của nhà cung cấp để đánh giá và chọn công nghệ.

Undercurrents and Their Impacts on Choosing Technologies — Các dòng chảy ngầm và tác động của chúng đối với việc chọn công nghệ

As seen in this chapter, a data engineer has a lot to consider when evaluating technologies. Whatever technology you choose, be sure to understand how it supports the undercurrents of the data engineering lifecycle. Let's briefly review them again.

Như đã thấy trong chương này, một kỹ sư dữ liệu có rất nhiều điều phải cân nhắc khi đánh giá các công nghệ. Dù bạn chọn công nghệ nào, hãy chắc chắn rằng bạn hiểu nó hỗ trợ các dòng chảy ngầm của vòng đời kỹ thuật dữ liệu như thế nào. Hãy điểm lại chúng một cách ngắn gọn lần nữa.

Data Management — Quản lý dữ liệu

Data management is a broad area, and concerning technologies, it isn't always apparent whether a technology adopts data management as a principal concern. For example, behind the scenes, a third-party vendor may use data management best practices—**regulatory compliance**, security, **privacy**, **data quality**, and governance—but hide these details behind a limited UI layer. In this case, while evaluating the product, it helps to ask the company about its data management practices. Here are some sample questions you should ask:

Quản lý dữ liệu (data management) là một lĩnh vực rộng lớn, và liên quan đến các công nghệ, không phải lúc nào cũng rõ ràng việc một công nghệ có coi quản lý dữ liệu như một mối quan tâm chính hay không. Ví dụ, ở hậu trường, một nhà cung cấp bên thứ ba có thể dùng các thực hành tốt nhất về quản lý dữ liệu—tuân thủ quy định, bảo mật, quyền riêng tư, chất lượng dữ liệu và quản trị—nhưng giấu các chi tiết này sau một lớp UI hạn chế. Trong trường hợp này, trong khi đánh giá sản phẩm, sẽ hữu ích nếu hỏi công ty về các thực hành quản lý dữ liệu của họ. Đây là một số câu hỏi mẫu bạn nên hỏi:

How are you protecting data against breaches, both from the outside and within?

Các anh đang bảo vệ dữ liệu chống lại các vụ rò rỉ như thế nào, cả từ bên ngoài lẫn bên trong?

What is your product's compliance with GDPR, CCPA, and other data privacy regulations?

Mức độ tuân thủ của sản phẩm của các anh với GDPR, CCPA và các quy định về quyền riêng tư dữ liệu khác là gì?

Do you allow me to host my data to comply with these regulations?

Các anh có cho phép tôi tự host dữ liệu của mình để tuân thủ các quy định này không?

How do you ensure data quality and that I'm viewing the correct data in your solution?

Các anh đảm bảo chất lượng dữ liệu và rằng tôi đang xem đúng dữ liệu trong giải pháp của các anh như thế nào?

There are many other questions to ask, and these are just a few of the ways to think about data management as it relates to choosing the right technologies. These same questions should also apply to the OSS solutions you're considering.

Có nhiều câu hỏi khác để hỏi, và đây chỉ là một vài trong số các cách để nghĩ về quản lý dữ liệu khi nó liên quan đến việc chọn đúng công nghệ. Những câu hỏi tương tự này cũng nên áp dụng cho các giải pháp OSS mà bạn đang cân nhắc.

DataOps — DataOps

Problems will happen. They just will. A server or database may die, a cloud's region may have an outage, you might deploy buggy code, bad data might be introduced into your data warehouse, and other unforeseen problems may occur. When evaluating a new technology, how much control do

you have over deploying new code, how will you be alerted if there's a problem, and how will you respond when there's a problem? The answer largely depends on the type of technology you're considering. If the technology is OSS, you're likely responsible for setting up monitoring, hosting, and code deployment. How will you handle issues? What's your incident response? Much of the operations are outside your control if you're using a managed offering. Consider the vendor's SLA, the way they alert you to issues, and whether they're transparent about how they're addressing the case, including providing an ETA to a fix.

Các vấn đề sẽ xảy ra. Chúng chắc chắn sẽ xảy ra. Một server hoặc cơ sở dữ liệu có thể chết, một vùng của một cloud có thể có sự cố mất kết nối, bạn có thể triển khai mã có lỗi, dữ liệu xấu có thể được đưa vào kho dữ liệu của bạn, và các vấn đề không lường trước khác có thể xảy ra. Khi đánh giá một công nghệ mới, bạn có bao nhiêu quyền kiểm soát đối với việc triển khai mã mới, bạn sẽ được cảnh báo như thế nào nếu có vấn đề, và bạn sẽ phản ứng như thế nào khi có vấn đề? Câu trả lời phần lớn phụ thuộc vào loại công nghệ bạn đang cân nhắc. Nếu công nghệ là OSS, bạn có khả năng chịu trách nhiệm thiết lập giám sát, host và triển khai mã. Bạn sẽ xử lý các vấn đề như thế nào? Quy trình ứng phó sự cố của bạn là gì? Phần lớn các vận hành nằm ngoài quyền kiểm soát của bạn nếu bạn đang dùng một giải pháp được quản lý. Hãy cân nhắc SLA của nhà cung cấp, cách họ cảnh báo bạn về các vấn đề, và liệu họ có minh bạch về cách họ đang xử lý vụ việc hay không, bao gồm cả việc cung cấp một thời gian dự kiến đến lúc sửa xong.

Data Architecture — Kiến trúc dữ liệu

As discussed in Chapter 3, good data architecture means assessing trade-offs and choosing the best tools for the job while keeping your decisions reversible. With the data landscape morphing at warp speed, the best tool for the job is a moving target. The main goals are to avoid unnecessary lock-in, ensure interoperability across the data stack, and produce high ROI. Choose your technologies accordingly.

Như đã bàn ở Chương 3, kiến trúc dữ liệu tốt nghĩa là đánh giá các đánh đổi và chọn các công cụ tốt nhất cho công việc trong khi giữ cho các quyết định của bạn có thể đảo ngược. Với bối cảnh dữ liệu biến đổi với tốc độ ánh sáng, công cụ tốt nhất cho công việc là một mục tiêu di động. Các mục tiêu chính là tránh khóa-vào không cần thiết, đảm bảo khả năng tương tác xuyên ngăn xếp dữ liệu, và tạo ra ROI cao. Hãy chọn các công nghệ của bạn cho phù hợp.

Orchestration Example: Airflow — Ví dụ về điều phối: Airflow

Throughout most of this chapter, we have actively avoided discussing any particular technology too extensively. We make an exception for orchestration because the space is currently dominated by one open source technology, Apache Airflow. Maxime Beauchemin kicked off the Airflow project at Airbnb in 2014. Airflow was developed from the beginning as a noncommercial open source project. The framework quickly grew significant mindshare outside Airbnb, becoming an Apache Incubator project in 2016 and a full Apache-sponsored project in 2019. Airflow enjoys many advantages, largely because of its dominant position in the open source marketplace. First, the Airflow open source project is extremely active, with a high rate of commits and a quick response time for bugs and security issues, and the project recently released Airflow 2, a major refactor of the codebase. Second, Airflow enjoys massive mindshare. Airflow has a vibrant, active community on many communications platforms, including Slack, Stack Overflow, and GitHub. Users can easily find

answers to questions and problems. Third, Airflow is available commercially as a managed service or software distribution through many vendors, including GCP, AWS, and Astronomer.io.

Xuyên suốt phần lớn chương này, chúng tôi đã chủ động tránh bàn về bất kỳ công nghệ cụ thể nào quá rộng rãi. Chúng tôi tạo một ngoại lệ cho điều phối vì không gian này hiện đang bị thống trị bởi một công nghệ mã nguồn mở, Apache Airflow. Maxime Beauchemin khởi động dự án Airflow tại Airbnb vào năm 2014. Airflow được phát triển ngay từ đầu như một dự án mã nguồn mở phi thương mại. Framework này nhanh chóng có được sức ảnh hưởng đáng kể bên ngoài Airbnb, trở thành một dự án Apache Incubator vào năm 2016 và một dự án được Apache bảo trợ đầy đủ vào năm 2019. Airflow hưởng nhiều lợi thế, phần lớn nhờ vị thế thống trị của nó trên thị trường mã nguồn mở. Thứ nhất, dự án mã nguồn mở Airflow cực kỳ năng động, với một tỷ lệ commit cao và thời gian phản hồi nhanh cho các lỗi và vấn đề bảo mật, và dự án gần đây đã phát hành Airflow 2, một bản tái cấu trúc lớn của codebase. Thứ hai, Airflow hưởng sức ảnh hưởng khổng lồ. Airflow có một cộng đồng sôi động, năng động trên nhiều nền tảng giao tiếp, bao gồm Slack, Stack Overflow và GitHub. Người dùng có thể dễ dàng tìm thấy các câu trả lời cho câu hỏi và vấn đề. Thứ ba, Airflow khả dụng về mặt thương mại như một dịch vụ được quản lý hoặc một bản phân phối phần mềm thông qua nhiều nhà cung cấp, bao gồm GCP, AWS và Astronomer.io.

Airflow also has some downsides. Airflow relies on a few core nonscalable components (the scheduler and backend database) that can become bottlenecks for performance, scale, and reliability; the scalable parts of Airflow still follow a distributed monolith pattern. (See “Monolith Versus Modular”.) Finally, Airflow lacks support for many data-native constructs, such as schema management, lineage, and cataloging; and it is challenging to develop and test Airflow workflows. We do not attempt an exhaustive discussion of Airflow alternatives here but just mention a couple of the key orchestration contenders at the time of writing. Prefect and Dagster aim to solve some of the problems discussed previously by rethinking components of the Airflow architecture. Will there be other orchestration frameworks and technologies not discussed here? Plan on it. We highly recommend that anyone choosing an orchestration technology study the options discussed here. They should also acquaint themselves with activity in the space, as new developments will certainly occur by the time you read this.

Airflow cũng có một số nhược điểm. Airflow dựa vào một vài thành phần cốt lõi không-mở-rộng-được (bộ lập lịch và cơ sở dữ liệu backend) mà có thể trở thành các nút thắt cổ chai cho hiệu năng, quy mô và độ tin cậy; các phần mở-rộng-được của Airflow vẫn tuân theo một mô thức monolith phân tán. (Xem “Monolith so với mô-đun”.) Cuối cùng, Airflow thiếu hỗ trợ cho nhiều cấu trúc gốc-dữ-liệu (data-native), chẳng hạn như quản lý lược đồ, truy vết nguồn gốc (lineage) và lập danh mục (cataloging); và việc phát triển cùng kiểm thử các quy trình làm việc của Airflow là đầy thách thức. Chúng tôi không cố gắng đưa ra một cuộc thảo luận đầy đủ về các lựa chọn thay thế Airflow ở đây mà chỉ đề cập một vài trong số các ứng viên điều phối then chốt tại thời điểm viết. Prefect và Dagster nhắm tới việc giải quyết một số trong các vấn đề đã bàn trước đó bằng cách suy nghĩ lại các thành phần của kiến trúc Airflow. Liệu sẽ có các framework và công nghệ điều phối khác không được bàn ở đây không? Hãy chuẩn bị tinh thần là có. Chúng tôi rất khuyên rằng bất kỳ ai chọn một công nghệ điều phối nên nghiên cứu các lựa chọn đã bàn ở đây. Họ cũng nên làm quen với hoạt động trong không gian này, vì những phát triển mới chắc chắn sẽ xảy ra vào thời điểm bạn đọc đến đây.

Software Engineering — Kỹ thuật phần mềm

As a data engineer, you should strive for simplification and abstraction across the data stack. Buy or use prebuilt open source solutions whenever possible. Eliminating undifferentiated heavy lifting should be your big goal. Focus your resources—custom coding and tooling—on areas that give you a solid competitive advantage. For example, is hand-coding a database connection between your production database and your cloud data warehouse a competitive advantage for you? Probably not. This is very much a solved problem. Pick an off-the-shelf solution (open source or managed SaaS) instead. The world doesn’t need the millionth +1 database-to-cloud data warehouse connector. On the other hand, why do customers buy from you? Your business likely has something special about the way it does things. Maybe it’s a particular algorithm that powers your fintech platform. By abstracting away a lot of the redundant workflows and processes, you can continue chipping away, refining, and customizing the things that move the needle for the business.

Là một kỹ sư dữ liệu, bạn nên phấn đấu cho sự đơn giản hóa và trừu tượng hóa xuyên ngăn xếp dữ liệu. Hãy mua hoặc dùng các giải pháp mã nguồn mở dựng sẵn bất cứ khi nào có thể. Loại bỏ các công việc nặng nhọc không tạo khác biệt nên là mục tiêu lớn của bạn. Hãy tập trung các nguồn lực của bạn—mã và công cụ tùy biến—vào những lĩnh vực đem lại cho bạn một lợi thế cạnh tranh vững chắc. Ví dụ, liệu việc viết tay một kết nối cơ sở dữ liệu giữa cơ sở dữ liệu sản xuất của bạn và kho dữ liệu cloud của bạn có là một lợi thế cạnh tranh cho bạn không? Có lẽ không. Đây là một vấn đề đã được giải quyết từ lâu. Hãy chọn một giải pháp có-sẵn (off-the-shelf) (mã nguồn mở hoặc SaaS được quản lý) thay thế. Thế giới không cần đến cái trình kết nối cơ-sở-dữ-liệu-tới-kho-dữ-liệu-cloud thứ một triệu lẻ một. Mặt khác, tại sao khách hàng mua từ bạn? Doanh nghiệp của bạn có khả năng có một điều gì đó đặc biệt về cách nó làm mọi việc. Có thể đó là một thuật toán cụ thể tiếp sức cho nền tảng fintech của bạn. Bằng cách trừu tượng hóa đi rất nhiều các quy trình làm việc và quy trình dư thừa, bạn có thể tiếp tục gọt giũa, tinh chỉnh và tùy biến những thứ tạo nên khác biệt cho doanh nghiệp.

Conclusion — Kết luận

Choosing the right technologies is no easy task, especially when new technologies and patterns emerge daily. Today is possibly the most confusing time in history for evaluating and selecting technologies. Choosing technologies is a balance of use case, cost, build versus buy, and modularization. Always approach technology the same way as architecture: assess trade-offs and aim for reversible decisions.

Chọn đúng công nghệ không phải là một nhiệm vụ dễ dàng, đặc biệt khi các công nghệ và mô thức mới xuất hiện mỗi ngày. Hôm nay có lẽ là thời điểm khó hiểu nhất trong lịch sử để đánh giá và lựa chọn công nghệ. Chọn công nghệ là một sự cân bằng giữa trường hợp sử dụng, chi phí, tự-xây-dựng-so-với-mua, và mô-đun-hóa. Hãy luôn tiếp cận công nghệ theo cùng cách như kiến trúc: đánh giá các đánh đổi và nhắm tới các quyết định có thể đảo ngược.

1 For more details, see “Total Opportunity Cost of Ownership” by Joseph Reis in 97 Things Every Data Engineer Should Know (O’Reilly). 2 J.R. Storment and Mike Fuller, Cloud FinOps (Sebastopol, CA: O’Reilly, 2019), 6, <https://oreil.ly/RvRvX>. 3 This is a major point of emphasis in Storment and Fuller, Cloud FinOps. 4 Examples include Google Cloud Anthos and AWS Outposts. 5 Raghav Bhargava, “Evolution of Dropbox’s Edge Network,” Dropbox.Tech, June 19, 2017, <https://oreil.ly/RAwPf>. 6 Akhil Gupta, “Scaling to Exabytes and Beyond,” Dropbox.Tech, March 14, 2016, <https://oreil.ly/5XPkv>. 7 “Dropbox Migrates 34 PB of Data to an Amazon S3 Data Lake for Analytics,” AWS website, 2020,

<https://oreil.ly/wpVoM> . 8 Todd Hoff, “The Eternal Cost Savings of Netflix’s Internal Spot Market,” High **Scalability**, December 4, 2017, <https://oreil.ly/LLoFt> . 9 Todd Spangler, “Netflix Bandwidth Consumption Eclipsed by Web Media Streaming Applications,” Variety , September 10, 2019, <https://oreil.ly/tTm3k> . 10 Amir Efrati and Kevin McLaughlin, “Apple’s Spending on Google Cloud Storage on Track to Soar 50% This Year,” The Information , June 29, 2021, <https://oreil.ly/OlFyR> . 11 Evan Sangaline, “Running FFmpeg on AWS Lambda for 1.9% the Cost of AWS Elastic Transcoder,” Intoli blog, May 2, 2018, <https://oreil.ly/myzOv> . 12 Examples include OpenFaaS , Knative , and Google Cloud Run .

1 Để biết thêm chi tiết, xem “Total Opportunity Cost of Ownership” của Joseph Reis trong 97 Things Every Data Engineer Should Know (O’Reilly). 2 J.R. Storment và Mike Fuller, Cloud FinOps (Sebastopol, CA: O’Reilly, 2019), 6, <https://oreil.ly/RvRvX> . 3 Đây là một điểm nhấn lớn trong Storment và Fuller, Cloud FinOps . 4 Ví dụ bao gồm Google Cloud Anthos và AWS Outposts . 5 Raghav Bhargava, “Evolution of Dropbox’s Edge Network,” Dropbox.Tech, June 19, 2017, <https://oreil.ly/RAwPf> . 6 Akhil Gupta, “Scaling to Exabytes and Beyond,” Dropbox.Tech, March 14, 2016, <https://oreil.ly/5XPKv> . 7 “Dropbox Migrates 34 PB of Data to an Amazon S3 Data Lake for Analytics,” AWS website, 2020, <https://oreil.ly/wpVoM> . 8 Todd Hoff, “The Eternal Cost Savings of Netflix’s Internal Spot Market,” High Scalability, December 4, 2017, <https://oreil.ly/LLoFt> . 9 Todd Spangler, “Netflix Bandwidth Consumption Eclipsed by Web Media Streaming Applications,” Variety , September 10, 2019, <https://oreil.ly/tTm3k> . 10 Amir Efrati và Kevin McLaughlin, “Apple’s Spending on Google Cloud Storage on Track to Soar 50% This Year,” The Information , June 29, 2021, <https://oreil.ly/OlFyR> . 11 Evan Sangaline, “Running FFmpeg on AWS Lambda for 1.9% the Cost of AWS Elastic Transcoder,” Intoli blog, May 2, 2018, <https://oreil.ly/myzOv> . 12 Ví dụ bao gồm OpenFaaS , Knative , và Google Cloud Run .

13 Justin Olsson and Reynold Xin, “Eliminating the Anti-competitive DeWitt Clause for Database Benchmarking,” Databricks, November 8, 2021, <https://oreil.ly/3iFOE> . 14 For a classic of the genre, see William McKnight and Jake Dolezal, “Data Warehouse in the Cloud Benchmark,” GigaOm, February 7, 2019, <https://oreil.ly/QjCmA> .

13 Justin Olsson và Reynold Xin, “Eliminating the Anti-competitive DeWitt Clause for Database Benchmarking,” Databricks, November 8, 2021, <https://oreil.ly/3iFOE> . 14 Về một ví dụ kinh điển của thể loại này, xem William McKnight và Jake Dolezal, “Data Warehouse in the Cloud Benchmark,” GigaOm, February 7, 2019, <https://oreil.ly/QjCmA> .

Part II. The Data Engineering Lifecycle in Depth — Phần II. Vòng đời kỹ thuật dữ liệu chuyên sâu

Chapter 5. Data Generation in Source Systems — Chương 5. Sinh dữ liệu trong các hệ thống nguồn

Welcome to the first stage of the **data engineering lifecycle**: data **generation** in source systems. As we described earlier, the job of a **data engineer** is to take data from source systems, do something with it, and make it helpful in **serving downstream** use cases. But before you get raw data, you must understand where the data exists, how it is generated, and its characteristics and quirks. This chapter covers some popular operational **source system** patterns and the significant types of source systems. Many source systems exist for data generation, and we're not exhaustively covering them all. We'll consider the data these systems generate and things you should consider when working with source systems. We also discuss how the **undercurrents** of **data engineering** apply to this first phase of the data engineering lifecycle (Figure 5-1).

Chào mừng bạn đến với giai đoạn đầu tiên của vòng đời kỹ thuật dữ liệu: sinh dữ liệu (generation) trong các hệ thống nguồn (source system). Như đã mô tả trước đó, công việc của một kỹ sư dữ liệu (data engineer) là lấy dữ liệu từ các hệ thống nguồn, làm điều gì đó với nó, và biến nó trở nên hữu ích cho việc phục vụ (serving) các tình huống sử dụng phía hạ nguồn (downstream). Nhưng trước khi bạn có được dữ liệu thô, bạn phải hiểu dữ liệu tồn tại ở đâu, nó được sinh ra như thế nào, cũng như các đặc tính và những điểm khác thường của nó. Chương này trình bày một số mẫu hình hệ thống nguồn vận hành phổ biến cùng các loại hệ thống nguồn quan trọng. Có rất nhiều hệ thống nguồn tồn tại để sinh dữ liệu, và chúng tôi không thể bao quát hết tất cả. Chúng ta sẽ xem xét dữ liệu mà các hệ thống này sinh ra cùng những điều bạn cần cân nhắc khi làm việc với hệ thống nguồn. Chúng tôi cũng thảo luận xem các dòng chảy ngầm (undercurrents) của kỹ thuật dữ liệu áp dụng như thế nào vào giai đoạn đầu tiên này của vòng đời kỹ thuật dữ liệu (Hình 5-1).

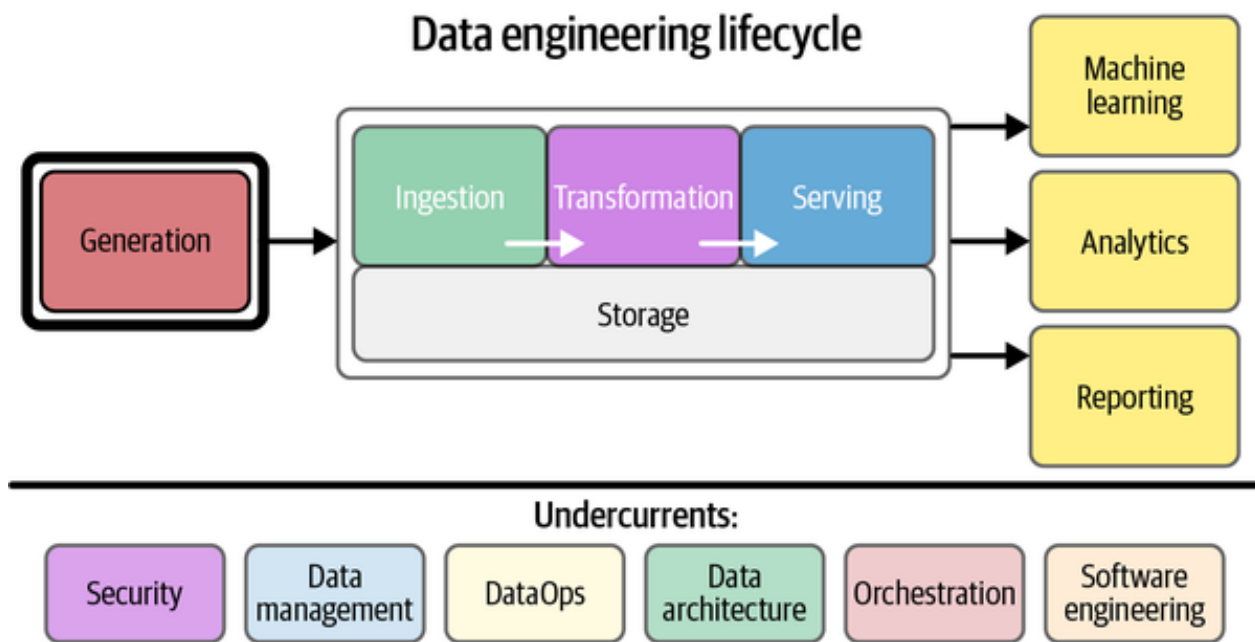


Figure 5-1. Source systems generate the data for the rest of the data engineering lifecycle. As data proliferates, especially with the rise of data sharing (discussed next), we expect that a data engineer’s role will shift heavily toward understanding the interplay between data sources and destinations. The basic plumbing tasks of data engineering—moving data from A to B—will simplify dramatically. On the other hand, it will remain critical to understand the nature of data as it’s created in source systems.

Hình 5-1. Các hệ thống nguồn sinh ra dữ liệu cho phần còn lại của vòng đời kỹ thuật dữ liệu. Khi dữ liệu sinh sôi, đặc biệt với sự nổi lên của việc chia sẻ dữ liệu (sẽ bàn tiếp theo), chúng tôi kỳ vọng vai trò của kỹ sư dữ liệu sẽ chuyển dịch mạnh sang việc hiểu sự tương tác qua lại giữa các nguồn dữ liệu và các đích đến. Những tác vụ “đường ống cơ bản” của kỹ thuật dữ liệu—di chuyển dữ liệu từ A đến B—sẽ trở nên đơn giản hơn rất nhiều. Mặt khác, việc hiểu bản chất của dữ liệu ngay khi nó được tạo ra trong các hệ thống nguồn vẫn sẽ giữ vai trò then chốt.

Sources of Data: How Is Data Created? — Các nguồn dữ liệu: Dữ liệu được tạo ra như thế nào?

As you learn about the various underlying operational patterns of the systems that generate data, it’s essential to understand how data is created. Data is an unorganized, context-less collection of facts and figures. It can be created in many ways, both analog and digital. Analog data creation occurs in the real world, such as vocal speech, sign language, writing on paper, or playing an instrument. This analog data is often transient; how often have you had a verbal conversation whose contents are lost to the ether after the conversation ends? Digital data is either created by converting analog data to digital form or is the native product of a digital system. An example of analog to digital is a mobile texting app that converts analog speech into digital text. An example of digital data creation is a credit card transaction on an ecommerce platform. A customer places an order; the transaction is charged to their credit card, and the information for the transaction is saved to various databases. We’ll utilize a few common examples in this chapter, such as data created when interacting with a website or mobile application. But in truth, data is everywhere in the world around us. We capture data from

IoT devices, credit card terminals, telescope sensors, stock trades, and more. Get familiar with your source system and how it generates data. Put in the effort to read the source system documentation and understand its patterns and quirks. If your source system is an RDBMS, learn how it operates (writes, commits, queries, etc.); learn the ins and outs of the source system that might affect your ability to ingest from it.

Khi bạn tìm hiểu về các mẫu hình vận hành nền tảng khác nhau của những hệ thống sinh dữ liệu, điều cốt yếu là phải hiểu dữ liệu được tạo ra như thế nào. Dữ liệu là một tập hợp các sự kiện và con số rời rạc, không có tổ chức và không có ngữ cảnh. Nó có thể được tạo ra theo nhiều cách, cả tương tự (analog) lẫn số (digital). Việc tạo dữ liệu tương tự diễn ra trong thế giới thực, chẳng hạn như lời nói, ngôn ngữ ký hiệu, viết trên giấy, hay chơi một loại nhạc cụ. Dữ liệu tương tự này thường có tính nhất thời; đã bao nhiêu lần bạn có một cuộc trò chuyện bằng lời mà nội dung của nó tan biến vào không trung sau khi cuộc trò chuyện kết thúc? Dữ liệu số hoặc được tạo ra bằng cách chuyển đổi dữ liệu tương tự sang dạng số, hoặc là sản phẩm bản địa của một hệ thống số. Một ví dụ về chuyển từ tương tự sang số là một ứng dụng nhắn tin di động chuyển lời nói tương tự thành văn bản số. Một ví dụ về việc tạo dữ liệu số là một giao dịch thẻ tín dụng trên một nền tảng thương mại điện tử. Khách hàng đặt một đơn hàng, giao dịch được tính vào thẻ tín dụng của họ, và thông tin của giao dịch được lưu vào nhiều database khác nhau. Chúng ta sẽ sử dụng một vài ví dụ phổ biến trong chương này, chẳng hạn như dữ liệu được tạo ra khi tương tác với một website hay ứng dụng di động. Nhưng thực ra, dữ liệu ở khắp mọi nơi trong thế giới quanh ta. Chúng ta thu thập dữ liệu từ các thiết bị IoT, các thiết bị đầu cuối thẻ tín dụng, các cảm biến kính thiên văn, các giao dịch chứng khoán, và nhiều hơn nữa. Hãy làm quen với hệ thống nguồn của bạn và cách nó sinh dữ liệu. Hãy bỏ công đọc tài liệu của hệ thống nguồn và hiểu các mẫu hình cùng những điểm khác thường của nó. Nếu hệ thống nguồn của bạn là một RDBMS, hãy tìm hiểu cách nó vận hành (ghi, commit, truy vấn, v.v.); tìm hiểu cận kề những điều của hệ thống nguồn có thể ảnh hưởng đến khả năng thu nạp (ingestion) dữ liệu từ nó của bạn.

Source Systems: Main Ideas — Các hệ thống nguồn: Những ý chính

Source systems produce data in various ways. This section discusses the main ideas you'll frequently encounter as you work with source systems.

Các hệ thống nguồn sinh dữ liệu theo nhiều cách khác nhau. Phần này thảo luận những ý chính mà bạn sẽ thường xuyên gặp khi làm việc với các hệ thống nguồn.

Files and Unstructured Data — Tập và dữ liệu phi cấu trúc

A file is a sequence of bytes, typically stored on a disk. Applications often write data to files. Files may store local parameters, events, logs, images, and audio. In addition, files are a universal medium of data exchange. As much as data engineers wish that they could get data programmatically, much of the world still sends and receives files. For example, if you're getting data from a government agency, there's an excellent chance you'll download the data as an Excel or CSV file or receive the file in an email. The main types of source file formats you'll run into as a data engineer— files that originate either manually or as an output from a source system process—are Excel, CSV, TXT, JSON, and XML. These files have their quirks and can be structured (Excel, CSV), semistructured (JSON, XML, CSV), or unstructured (TXT, CSV). Although you'll use certain formats heavily as a data engineer (such as

Parquet, ORC, and Avro), we'll cover these later and put the spotlight here on source system files. Chapter 6 covers the technical details of files.

Một tệp là một chuỗi byte, thường được lưu trên đĩa. Các ứng dụng thường ghi dữ liệu ra tệp. Tệp có thể lưu trữ các tham số cục bộ, các sự kiện, log, hình ảnh và âm thanh. Ngoài ra, tệp là một phương tiện trao đổi dữ liệu phổ quát. Dẫu các kỹ sư dữ liệu mong muốn có thể lấy dữ liệu một cách có lập trình đến đâu, phần lớn thế giới vẫn gửi và nhận tệp. Ví dụ, nếu bạn lấy dữ liệu từ một cơ quan chính phủ, rất có khả năng bạn sẽ tải dữ liệu xuống dưới dạng tệp Excel hay CSV, hoặc nhận tệp qua email. Các loại định dạng tệp nguồn chính mà bạn sẽ gặp với vai trò kỹ sư dữ liệu—những tệp bắt nguồn hoặc một cách thủ công, hoặc là đầu ra từ một tiến trình của hệ thống nguồn—là Excel, CSV, TXT, JSON và XML. Những tệp này có những điểm khác thường của riêng chúng và có thể có cấu trúc (Excel, CSV), bán cấu trúc (JSON, XML, CSV), hoặc phi cấu trúc (TXT, CSV). Mặc dù bạn sẽ sử dụng nhiều một số định dạng nhất định với vai trò kỹ sư dữ liệu (chẳng hạn như Parquet, ORC và Avro), chúng tôi sẽ đề cập đến những định dạng này sau và tập trung ánh đèn ở đây vào các tệp của hệ thống nguồn. Chương 6 trình bày các chi tiết kỹ thuật về tệp.

APIs — API

Application programming interfaces (APIs) are a standard way of exchanging data between systems. In theory, APIs simplify the data **ingestion** task for data engineers. In practice, many APIs still expose a good deal of data complexity for engineers to manage. Even with the rise of various services and frameworks, and services for automating API data ingestion, data engineers must often invest a good deal of energy into maintaining custom API connections. We discuss APIs in greater detail later in this chapter.

Giao diện lập trình ứng dụng (APIs) là một cách tiêu chuẩn để trao đổi dữ liệu giữa các hệ thống. Về lý thuyết, API đơn giản hóa tác vụ thu nạp dữ liệu cho các kỹ sư dữ liệu. Trên thực tế, nhiều API vẫn phơi bày khá nhiều phức tạp của dữ liệu để các kỹ sư phải quản lý. Ngay cả với sự nổi lên của nhiều dịch vụ và framework khác nhau, cùng các dịch vụ để tự động hóa việc thu nạp dữ liệu qua API, các kỹ sư dữ liệu vẫn thường phải đầu tư khá nhiều công sức vào việc duy trì các kết nối API tùy chỉnh. Chúng ta thảo luận về API chi tiết hơn ở phần sau của chương này.

Application Databases (OLTP systems) — Database ứng dụng (hệ thống OLTP)

An application database stores the state of an application. A standard example is a database that stores account balances for bank accounts. As customer transactions and payments happen, the application updates bank account balances. Typically, an application database is an online transaction processing (**OLTP**) system—a database that reads and writes individual data records at a high rate. OLTP systems are often referred to as transactional databases, but this does not necessarily imply that the system in question supports atomic transactions. More generally, OLTP databases support low **latency** and high concurrency. An RDBMS database can select or update a row in less than a millisecond (not accounting for network latency) and handle thousands of reads and writes per second. A document database cluster can manage even higher document commit rates at the expense of potential inconsistency. Some graph databases can also handle transactional use cases. Fundamentally, OLTP databases work well as application backends when thousands or even millions of users might be interacting with the application simultaneously, updating and

writing data concurrently. OLTP systems are less suited to use cases driven by **analytics** at scale, where a single query must scan a vast amount of data.

Một database ứng dụng lưu trữ trạng thái của một ứng dụng. Một ví dụ tiêu chuẩn là một database lưu số dư cho các tài khoản ngân hàng. Khi các giao dịch và thanh toán của khách hàng diễn ra, ứng dụng cập nhật số dư tài khoản ngân hàng. Thông thường, một database ứng dụng là một hệ thống xử lý giao dịch trực tuyến (OLTP)—một database đọc và ghi từng bản ghi dữ liệu riêng lẻ với tốc độ cao. Các hệ thống OLTP thường được gọi là *database giao dịch*, nhưng điều này không nhất thiết hàm ý rằng hệ thống đang xét hỗ trợ các giao dịch nguyên tử (atomic transactions). Tổng quát hơn, các database OLTP hỗ trợ độ trễ (latency) thấp và độ đồng thời cao. Một database RDBMS có thể chọn hay cập nhật một hàng trong chưa đầy một mili-giây (chưa tính độ trễ mạng) và xử lý hàng nghìn lượt đọc-ghi mỗi giây. Một cụm database tài liệu (document) có thể quản lý tốc độ commit tài liệu thậm chí còn cao hơn, với cái giá là khả năng thiếu nhất quán tiềm ẩn. Một số database đồ thị (graph) cũng có thể xử lý các tình huống sử dụng giao dịch. Về cơ bản, các database OLTP hoạt động tốt với vai trò backend của ứng dụng khi có hàng nghìn hay thậm chí hàng triệu người dùng có thể tương tác đồng thời với ứng dụng, cập nhật và ghi dữ liệu một cách đồng thời. Các hệ thống OLTP ít phù hợp hơn với những tình huống sử dụng được dẫn dắt bởi phân tích (analytics) ở quy mô lớn, nơi một truy vấn đơn lẻ phải quét một lượng dữ liệu khổng lồ.

ACID — ACID

Support for atomic transactions is one of a critical set of database characteristics known together as ACID (as you may recall from Chapter 3 , this stands for atomicity, consistency, isolation, durability). Consistency means that any database read will return the last written version of the retrieved item. Isolation entails that if two updates are in flight concurrently for the same thing, the end database state will be consistent with the sequential execution of these updates in the order they were submitted. Durability indicates that committed data will never be lost, even in the **event** of power loss. Note that ACID characteristics are not required to support application backends, and relaxing these constraints can be a considerable boon to performance and scale. However, ACID characteristics guarantee that the database will maintain a consistent picture of the world, dramatically simplifying the app developer’s task. All engineers (data or otherwise) must understand operating with and without ACID. For instance, to improve performance, some distributed databases use relaxed consistency constraints, such as eventual consistency , to improve performance. Understanding the consistency model you’re working with helps you prevent disasters.

Hỗ trợ cho các giao dịch nguyên tử là một trong một tập hợp các đặc tính database quan trọng được gọi chung là ACID (như bạn có thể nhớ từ Chương 3, viết tắt của *atomicity*, *consistency*, *isolation*, *durability*—tính nguyên tử, tính nhất quán, tính cô lập, tính bền vững). Tính nhất quán (consistency) nghĩa là bất kỳ lượt đọc database nào cũng sẽ trả về phiên bản được ghi gần nhất của mục được truy xuất. Tính cô lập (isolation) hàm ý rằng nếu hai lượt cập nhật đang diễn ra đồng thời cho cùng một thứ, trạng thái database cuối cùng sẽ nhất quán với việc thực thi tuần tự các lượt cập nhật này theo đúng thứ tự chúng được gửi đến. Tính bền vững (durability) chỉ ra rằng dữ liệu đã được commit sẽ không bao giờ bị mất, ngay cả trong trường hợp mất điện. Lưu ý rằng các đặc tính ACID không bắt buộc để hỗ trợ backend ứng dụng, và việc nói lỏng các ràng buộc này có thể là một lợi thế đáng kể cho hiệu năng và khả năng mở rộng. Tuy nhiên, các đặc tính ACID đảm bảo rằng database sẽ duy trì một bức tranh nhất quán về thế giới, qua đó đơn giản hóa đáng kể công

việc của lập trình viên ứng dụng. Tất cả các kỹ sư (dữ liệu hay không) đều phải hiểu cách vận hành có và không có ACID. Chẳng hạn, để cải thiện hiệu năng, một số database phân tán sử dụng các ràng buộc nhất quán được nới lỏng, như nhất quán cuối cùng (eventual consistency), để cải thiện hiệu năng. Hiểu mô hình nhất quán mà bạn đang làm việc cùng giúp bạn ngăn chặn thảm họa.

Atomic transactions — Giao dịch nguyên tử

An atomic transaction is a set of several changes that are committed as a unit. In the example in Figure 5-2, a traditional banking application running on an RDBMS executes a SQL statement that checks two account balances, one in Account A (the source) and another in Account B (the destination). Money is then moved from Account A to Account B if sufficient funds are in Account A. The entire transaction should run with updates to both account balances or fail without updating either account balance. That is, the whole operation should happen as a transaction.

Một giao dịch nguyên tử là một tập hợp gồm nhiều thay đổi được commit như một đơn vị. Trong ví dụ ở Hình 5-2, một ứng dụng ngân hàng truyền thống chạy trên một RDBMS thực thi một câu lệnh SQL kiểm tra số dư của hai tài khoản, một là Tài khoản A (nguồn) và một là Tài khoản B (đích). Tiền sau đó được chuyển từ Tài khoản A sang Tài khoản B nếu Tài khoản A có đủ tiền. Toàn bộ giao dịch nên chạy với việc cập nhật cả hai số dư tài khoản, hoặc thất bại mà không cập nhật số dư của tài khoản nào. Tức là, toàn bộ thao tác nên diễn ra như một *giao dịch*.

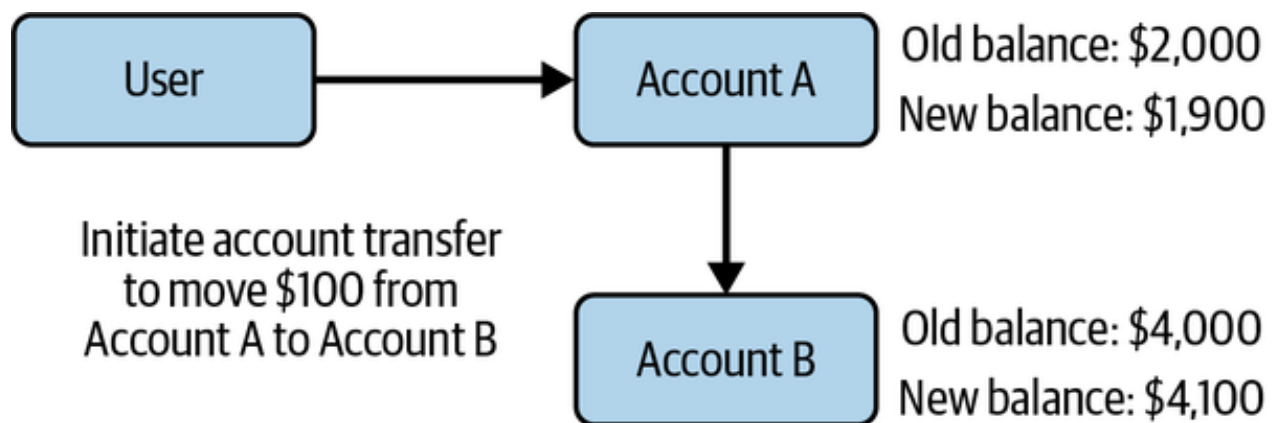


Figure 5-2. Example of an atomic transaction: a bank account transfer using OLTP

Hình 5-2. Ví dụ về một giao dịch nguyên tử: một lượt chuyển khoản ngân hàng dùng OLTP

OLTP and analytics — OLTP và phân tích

Often, small companies run analytics directly on an OLTP. This pattern works in the short term but is ultimately not scalable. At some point, running analytical queries on OLTP runs into performance issues due to structural limitations of OLTP or resource contention with competing transactional workloads. Data engineers must understand the inner workings of OLTP and application backends to set up appropriate integrations with analytics systems without degrading production application performance. As companies offer more analytics capabilities in SaaS applications, the need for hybrid capabilities—quick updates with combined analytics capabilities—has created new

challenges for data engineers. We'll use the term data application to refer to applications that hybridize transactional and analytics workloads.

Thường thì các công ty nhỏ chạy phân tích trực tiếp trên một OLTP. Mẫu hình này hiệu quả trong ngắn hạn nhưng rốt cuộc không có khả năng mở rộng. Đến một lúc nào đó, việc chạy các truy vấn phân tích trên OLTP sẽ gặp các vấn đề về hiệu năng do những giới hạn cấu trúc của OLTP hoặc do tranh chấp tài nguyên với các tải công việc giao dịch cạnh tranh. Các kỹ sư dữ liệu phải hiểu cơ chế hoạt động bên trong của OLTP và backend ứng dụng để thiết lập các tích hợp phù hợp với các hệ thống phân tích mà không làm suy giảm hiệu năng của ứng dụng production. Khi các công ty cung cấp nhiều năng lực phân tích hơn trong các ứng dụng SaaS, nhu cầu về các năng lực lai—cập nhật nhanh kết hợp với năng lực phân tích—đã tạo ra những thách thức mới cho các kỹ sư dữ liệu. Chúng ta sẽ dùng thuật ngữ *ứng dụng dữ liệu* để chỉ những ứng dụng lai giữa tải công việc giao dịch và phân tích.

Online Analytical Processing System — Hệ thống xử lý phân tích trực tuyến

In contrast to an OLTP system, an online analytical processing (**OLAP**) system is built to run large analytics queries and is typically inefficient at handling lookups of individual records. For example, modern column databases are optimized to scan large volumes of data, dispensing with indexes to improve **scalability** and scan performance. Any query typically involves scanning a minimal data block, often 100 MB or more in size. Trying to look up thousands of individual items per second in such a system will bring it to its knees unless it is combined with a caching layer designed for this use case. Note that we're using the term OLAP to refer to any database system that supports high-scale interactive analytics queries; we are not limiting ourselves to systems that support OLAP cubes (multidimensional arrays of data). The online part of OLAP implies that the system constantly listens for incoming queries, making OLAP systems suitable for interactive analytics. Although this chapter covers source systems, OLAPs are typically **storage** and query systems for analytics. Why are we talking about them in our chapter on source systems? In practical use cases, engineers often need to read data from an OLAP system. For example, a **data warehouse** might serve data used to train an ML model. Or, an OLAP system might serve a **reverse ETL** workflow, where derived data in an analytics system is sent back to a source system, such as a CRM, SaaS platform, or transactional application.

Trái với một hệ thống OLTP, một hệ thống xử lý phân tích trực tuyến (OLAP) được xây dựng để chạy các truy vấn phân tích lớn và thường không hiệu quả trong việc tra cứu từng bản ghi riêng lẻ. Ví dụ, các database cột (column) hiện đại được tối ưu để quét các khối lượng dữ liệu lớn, loại bỏ các chỉ mục để cải thiện khả năng mở rộng và hiệu năng quét. Bất kỳ truy vấn nào cũng thường liên quan đến việc quét một khối dữ liệu tối thiểu, thường có kích thước 100 MB trở lên. Cố gắng tra cứu hàng nghìn mục riêng lẻ mỗi giây trong một hệ thống như vậy sẽ khiến nó quy ngã, trừ khi nó được kết hợp với một tầng cache (caching) được thiết kế cho tình huống sử dụng này. Lưu ý rằng chúng tôi đang dùng thuật ngữ OLAP để chỉ bất kỳ hệ thống database nào hỗ trợ các truy vấn phân tích tương tác quy mô lớn; chúng tôi không tự giới hạn ở các hệ thống hỗ trợ khối OLAP (mảng dữ liệu đa chiều). Phần *trực tuyến* của OLAP hàm ý rằng hệ thống liên tục lắng nghe các truy vấn đến, khiến các hệ thống OLAP phù hợp cho phân tích tương tác. Mặc dù chương này nói về các hệ thống nguồn, OLAP thường là các hệ thống lưu trữ và truy vấn cho phân tích. Vậy tại sao chúng ta lại nói về chúng trong chương về hệ thống nguồn? Trong các tình huống sử dụng thực tế, các kỹ sư thường cần đọc dữ liệu từ một hệ thống OLAP. Ví dụ, một kho dữ liệu (data

warehouse) có thể phục vụ dữ liệu được dùng để huấn luyện một mô hình ML. Hoặc, một hệ thống OLAP có thể phục vụ một luồng công việc reverse ETL, nơi dữ liệu dẫn xuất trong một hệ thống phân tích được gửi ngược trở lại một hệ thống nguồn, chẳng hạn như một CRM, một nền tảng SaaS, hay một ứng dụng giao dịch.

Change Data Capture — Thu thập thay đổi dữ liệu

Change data capture (CDC) is a method for extracting each change event (insert, update, delete) that occurs in a database. CDC is frequently leveraged to replicate between databases in near real time or create an event stream for downstream processing. CDC is handled differently depending on the database technology. Relational databases often generate an event log stored directly on the database server that can be processed to create a stream. (See “Database Logs”.) Many cloud NoSQL databases can send a log or event stream to a target storage location.

Thu thập thay đổi dữ liệu (CDC) là một phương pháp để trích xuất từng sự kiện thay đổi (thêm, cập nhật, xóa) xảy ra trong một database. CDC thường được tận dụng để sao chép (replicate) giữa các database gần như theo thời gian thực hoặc để tạo ra một luồng sự kiện cho xử lý hạ nguồn. CDC được xử lý khác nhau tùy vào công nghệ database. Các database quan hệ thường sinh ra một log sự kiện được lưu trực tiếp trên máy chủ database, có thể được xử lý để tạo ra một luồng (stream). (Xem “Log của database”.) Nhiều database NoSQL trên cloud có thể gửi một log hay luồng sự kiện đến một vị trí lưu trữ đích.

Logs — Log

A log captures information about events that occur in systems. For example, a log may capture traffic and usage patterns on a web server. Your desktop computer’s operating system (Windows, macOS, Linux) logs events as the system boots and when applications start or crash, for example. Logs are a rich data source, potentially valuable for downstream data analysis, ML, and automation. Here are a few familiar sources of logs:

Một log ghi lại thông tin về các sự kiện xảy ra trong các hệ thống. Ví dụ, một log có thể ghi lại các mẫu hình lưu lượng và mức sử dụng trên một máy chủ web. Hệ điều hành của máy tính để bàn của bạn (Windows, macOS, Linux) ghi log các sự kiện khi hệ thống khởi động, và khi các ứng dụng khởi động hay sập, chẳng hạn vậy. Log là một nguồn dữ liệu phong phú, có khả năng có giá trị cho phân tích dữ liệu, ML và tự động hóa ở hạ nguồn. Dưới đây là một vài nguồn log quen thuộc:

Operating systems

Hệ điều hành

Applications

Ứng dụng

Servers

Máy chủ

Containers

Container

Networks

Mạng

IoT devices

Thiết bị IoT

All logs track events and event **metadata**. At a minimum, a log should capture who, what, and when:
Who

Tất cả log đều theo dõi các sự kiện và siêu dữ liệu (metadata) của sự kiện. Ở mức tối thiểu, một log nên ghi lại ai, cái gì và khi nào: Ai

The human, system, or service account associated with the event (e.g., a web browser user agent or a user ID)

Con người, hệ thống, hay tài khoản dịch vụ liên quan đến sự kiện (ví dụ, một user agent của trình duyệt web hay một ID người dùng)

What happened

Điều gì đã xảy ra

The event and related metadata

Sự kiện và siêu dữ liệu liên quan

When

Khi nào

The timestamp of the event

Dấu thời gian (timestamp) của sự kiện

Log encoding — Mã hóa log

Logs are encoded in a few ways: Binary-encoded logs These encode data in a custom compact format for space efficiency and fast I/O. Database logs, discussed in “Database Logs”, are a standard example.

Log được mã hóa theo một vài cách: Log mã hóa nhị phân Những log này mã hóa dữ liệu theo một định dạng nén tùy chỉnh nhằm tiết kiệm không gian và I/O nhanh. Log của database, được thảo luận trong “Log của database”, là một ví dụ tiêu chuẩn.

Semistructured logs

Log bán cấu trúc

These are encoded as text in an object **serialization** format (JSON, more often than not). Semistructured logs are machine-readable and portable. However, they are much less efficient than binary logs. And though they are nominally machine-readable, extracting value from them often requires significant custom code.

Những log này được mã hóa dưới dạng văn bản theo một định dạng tuần tự hóa (serialization) đối tượng (thường là JSON nhất). Log bán cấu trúc có thể đọc được bằng máy và có tính khả chuyển. Tuy nhiên, chúng kém hiệu quả hơn nhiều so với log nhị phân. Và mặc dù chúng trên danh nghĩa là đọc được bằng máy, việc trích xuất giá trị từ chúng thường đòi hỏi một lượng đáng kể mã tùy chỉnh.

Plain-text (unstructured) logs

Log văn bản thuần (phi cấu trúc)

These essentially store the console output from software. As such, no general-purpose standards exist. These logs can provide helpful information for data scientists and ML engineers, though extracting useful information from the raw text data might be complicated.

Những log này về cơ bản lưu trữ đầu ra console của phần mềm. Do đó, không có chuẩn dùng chung nào tồn tại. Những log này có thể cung cấp thông tin hữu ích cho các nhà khoa học dữ liệu và kỹ sư ML, dù việc trích xuất thông tin hữu ích từ dữ liệu văn bản thô có thể phức tạp.

Log resolution — Độ phân giải của log

Logs are created at various resolutions and log levels. The log resolution refers to the amount of event data captured in a log. For example, database logs capture enough information from database events to allow reconstructing the database state at any point in time. On the other hand, capturing all data changes in logs for a big data system often isn't practical. Instead, these logs may note only that a particular type of commit event has occurred. The log level refers to the conditions required to record a log entry, specifically concerning errors and debugging. Software is often configurable to log every event or to log only errors, for example.

Log được tạo ra ở nhiều độ phân giải và mức log khác nhau. Độ phân giải của log đề cập đến lượng dữ liệu sự kiện được ghi lại trong một log. Ví dụ, log của database ghi lại đủ thông tin từ các sự kiện database để cho phép tái dựng lại trạng thái của database tại bất kỳ thời điểm nào. Mặt khác, việc ghi lại tất cả các thay đổi dữ liệu trong log cho một hệ thống dữ liệu lớn thường không thực tế. Thay vào đó, những log này có thể chỉ ghi chú rằng một loại sự kiện commit cụ thể đã xảy ra. Mức log đề cập đến các điều kiện cần thiết để ghi lại một mục log, cụ thể liên quan đến lỗi và gỡ rối (debug). Phần mềm thường có thể được cấu hình để ghi log mọi sự kiện hoặc chỉ ghi log lỗi, chẳng hạn.

Log latency: Batch or real time — Độ trễ của log: Theo lô hay thời gian thực

Batch logs are often written continuously to a file. Individual log entries can be written to a messaging system such as Kafka or Pulsar for real-time applications.

Log theo lô (batch) thường được ghi liên tục vào một tệp. Từng mục log riêng lẻ có thể được ghi vào một hệ thống nhắn tin như Kafka hay Pulsar cho các ứng dụng thời gian thực.

Database Logs — Log của database

Database logs are essential enough that they deserve more detailed coverage. Write-ahead logs—typically, binary files stored in a specific database-native format—play a crucial role in database guarantees and recoverability. The database server receives write and update requests to a database table (see Figure 5-3), storing each operation in the log before acknowledging the request. The acknowledgment comes with a log associated guarantee: even if the server fails, it can recover its state on reboot by completing the unfinished work from the logs.

Log của database đủ thiết yếu để xứng đáng được trình bày chi tiết hơn. Write-ahead log—thường là các tệp nhị phân được lưu theo một định dạng bản địa cụ thể của database—đóng vai trò then chốt trong các đảm bảo và khả năng phục hồi của database. Máy chủ

database nhận các yêu cầu ghi và cập nhật vào một bảng database (xem Hình 5-3), lưu mỗi thao tác vào log trước khi xác nhận (acknowledge) yêu cầu. Lời xác nhận đi kèm với một đảm bảo gắn với log: ngay cả khi máy chủ thất bại, nó vẫn có thể phục hồi trạng thái của mình khi khởi động lại bằng cách hoàn thành phần công việc còn dang dở từ các log.

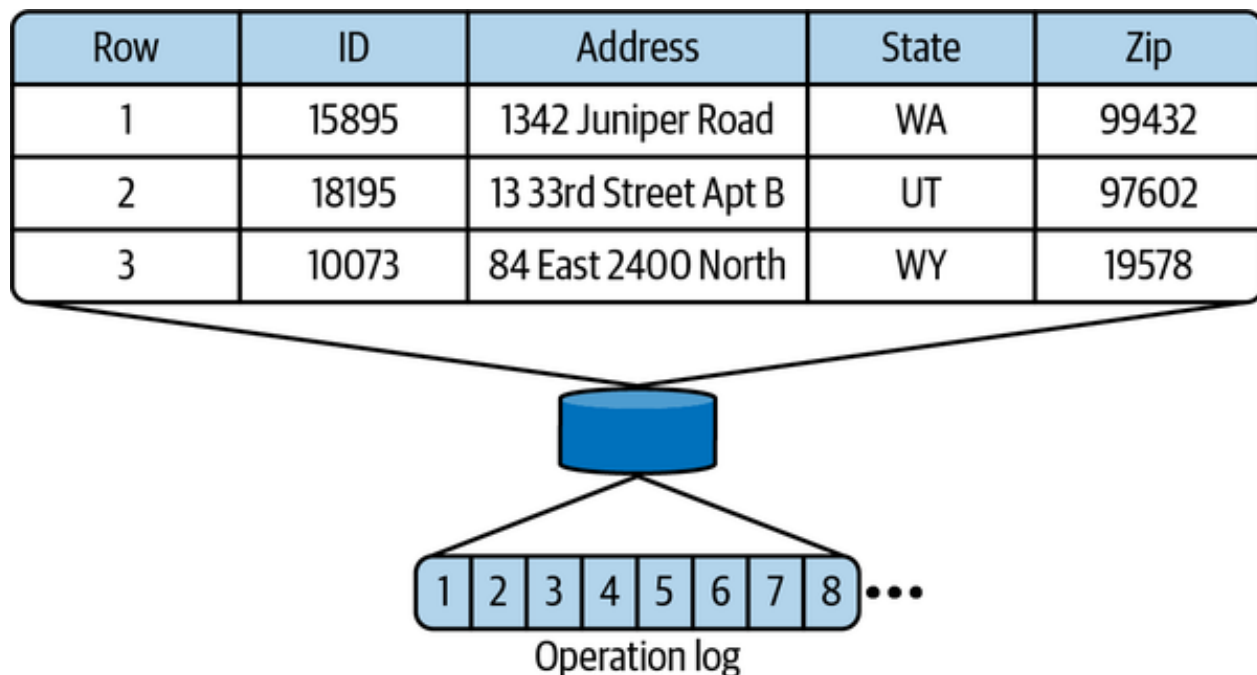


Figure 5-3. Log operations on a database table Database logs are extremely useful in data engineering, especially for CDC to generate event streams from database changes.

Hình 5-3. Các thao tác log trên một bảng database. Log của database cực kỳ hữu ích trong kỹ thuật dữ liệu, đặc biệt cho CDC để sinh ra các luồng sự kiện từ những thay đổi của database.

CRUD — CRUD

CRUD, which stands for create, read, update, and delete, is a transactional pattern commonly used in programming and represents the four basic operations of persistent storage. CRUD is the most common pattern for storing application state in a database. A basic tenet of CRUD is that data must be created before being used. After the data has been created, the data can be read and updated. Finally, the data may need to be destroyed. CRUD guarantees these four operations will occur on data, regardless of its storage. CRUD is a widely used pattern in software applications, and you'll commonly find CRUD used in APIs and databases. For example, a web application will make heavy use of CRUD for RESTful HTTP requests and storing and retrieving data from a database. As with any database, we can use snapshot-based extraction to get data from a database where our application applies CRUD operations. On the other hand, event extraction with CDC gives us a complete history of operations and potentially allows for near real-time analytics.

CRUD, viết tắt của *create* (tạo), *read* (đọc), *update* (cập nhật) và *delete* (xóa), là một mẫu hình giao dịch thường được dùng trong lập trình và đại diện cho bốn thao tác cơ bản của lưu trữ bền vững. CRUD là mẫu hình phổ biến nhất để lưu trạng thái ứng dụng trong một database. Một nguyên tắc cơ bản của CRUD là dữ liệu phải được tạo ra trước khi được sử dụng. Sau khi dữ liệu đã được tạo, nó có thể được đọc và cập nhật. Cuối cùng, dữ liệu có

thể cần được hủy. CRUD đảm bảo bốn thao tác này sẽ xảy ra trên dữ liệu, bất kể nơi lưu trữ của nó. CRUD là một mẫu hình được dùng rộng rãi trong các ứng dụng phần mềm, và bạn sẽ thường thấy CRUD được dùng trong các API và database. Ví dụ, một ứng dụng web sẽ sử dụng nhiều CRUD cho các yêu cầu HTTP RESTful cũng như cho việc lưu và truy xuất dữ liệu từ một database. Cũng như với bất kỳ database nào, chúng ta có thể dùng trích xuất dựa trên ảnh chụp (snapshot) để lấy dữ liệu từ một database mà ứng dụng của chúng ta áp dụng các thao tác CRUD. Mặt khác, trích xuất sự kiện với CDC cho chúng ta một lịch sử đầy đủ các thao tác và có khả năng cho phép phân tích gần như theo thời gian thực.

Insert-Only — Chỉ-thêm (Insert-Only)

The insert-only pattern retains history directly in a table containing data. Rather than updating records, new records get inserted with a timestamp indicating when they were created (Table 5-1). For example, suppose you have a table of customer addresses. Following a CRUD pattern, you would simply update the record if the customer changed their address. With the insert-only pattern, a new address record is inserted with the same customer ID. To read the current customer address by customer ID, you would look up the latest record under that ID.

Mẫu hình chỉ-thêm (insert-only) giữ lại lịch sử trực tiếp trong một bảng chứa dữ liệu. Thay vì cập nhật các bản ghi, các bản ghi mới được thêm vào với một dấu thời gian cho biết khi nào chúng được tạo (Bảng 5-1). Ví dụ, giả sử bạn có một bảng địa chỉ khách hàng. Theo mẫu hình CRUD, bạn sẽ chỉ đơn giản cập nhật bản ghi nếu khách hàng thay đổi địa chỉ của họ. Với mẫu hình chỉ-thêm, một bản ghi địa chỉ mới được thêm vào với cùng ID khách hàng. Để đọc địa chỉ hiện tại của khách hàng theo ID khách hàng, bạn sẽ tra cứu bản ghi mới nhất dưới ID đó.

Table 5-1. An insert-only pattern produces multiple versions of a record

Bảng 5-1. Một mẫu hình chỉ-thêm tạo ra nhiều phiên bản của một bản ghi

Record ID Value Timestamp

Record ID Value Timestamp

1 40 2021-09-19T00:10:23+00:00 1 51 2021-09-30T00:12:00+00:00 In a sense, the insert-only pattern maintains a database log directly in the table itself, making it especially useful if the application needs access to history. For example, the insert-only pattern would work well for a banking application designed to present customer address history. A separate analytics insert-only pattern is often used with regular CRUD application tables. In the insert-only **ETL** pattern, data pipelines insert a new record in the target analytics table anytime an update occurs in the CRUD table. Insert-only has a couple of disadvantages. First, tables can grow quite large, especially if data frequently changes, since each change is inserted into the table. Sometimes records are purged based on a record sunset date or a maximum number of record versions to keep table size reasonable. The second disadvantage is that record lookups incur extra overhead because looking up the current state involves running `MAX(created_timestamp)` . If hundreds or thousands of records are under a single ID, this lookup operation is expensive to run.

1 40 2021-09-19T00:10:23+00:00 1 51 2021-09-30T00:12:00+00:00 Theo một nghĩa nào đó, mẫu hình chỉ-thêm duy trì một log của database ngay trong chính bảng đó, khiến nó đặc biệt hữu ích nếu ứng dụng cần truy cập lịch sử. Ví dụ, mẫu hình chỉ-thêm sẽ hoạt động tốt cho một ứng dụng ngân hàng được thiết kế để trình bày lịch sử địa chỉ của khách hàng. Một mẫu hình chỉ-thêm dành cho phân tích riêng biệt thường được dùng cùng với các bảng ứng dụng CRUD thông thường. Trong mẫu hình ETL chỉ-thêm, các đường ống dữ liệu (data

pipeline) thêm một bản ghi mới vào bảng phân tích đích bất cứ khi nào một lượt cập nhật xảy ra trong bảng CRUD. Chỉ-thêm có một vài nhược điểm. Thứ nhất, các bảng có thể phình ra khá lớn, đặc biệt nếu dữ liệu thay đổi thường xuyên, vì mỗi thay đổi đều được thêm vào bảng. Đôi khi các bản ghi được xóa dựa trên một ngày kết thúc hiệu lực (sunset date) của bản ghi hoặc một số lượng tối đa các phiên bản bản ghi được giữ, để giữ kích thước bảng ở mức hợp lý. Nhược điểm thứ hai là việc tra cứu bản ghi phát sinh thêm chi phí phụ vì việc tra cứu trạng thái hiện tại liên quan đến việc chạy MAX(created_timestamp). Nếu có hàng trăm hay hàng nghìn bản ghi dưới một ID đơn lẻ, thao tác tra cứu này tốn kém khi chạy.

Messages and Streams — Tin nhắn và luồng

Related to **event-driven architecture**, two terms that you'll often see used interchangeably are **message queue** and **streaming platform**, but a subtle but essential difference exists between the two. Defining and contrasting these terms is worthwhile since they encompass many big ideas related to source systems and practices, and technologies spanning the entire data engineering lifecycle. A message is raw data communicated across two or more systems (Figure 5-4). For example, we have System 1 and System 2, where System 1 sends a message to System 2. These systems could be different **microservices**, a server sending a message to a **serverless** function, etc.. A message is typically sent through a message queue from a publisher to a consumer, and once the message is delivered, it is removed from the queue.

Liên quan đến kiến trúc hướng sự kiện (event-driven architecture), hai thuật ngữ mà bạn sẽ thường thấy được dùng thay thế cho nhau là *hàng đợi tin nhắn (message queue)* và *nền tảng luồng (streaming platform)*, nhưng tồn tại một khác biệt tinh tế nhưng thiết yếu giữa hai thứ này. Việc định nghĩa và đối chiếu các thuật ngữ này là xứng đáng vì chúng bao hàm nhiều ý tưởng lớn liên quan đến các hệ thống nguồn cùng các thực hành, cũng như các công nghệ trải dài toàn bộ vòng đời kỹ thuật dữ liệu. Một *tin nhắn (message)* là dữ liệu thô được truyền đạt giữa hai hay nhiều hệ thống (Hình 5-4). Ví dụ, chúng ta có Hệ thống 1 và Hệ thống 2, trong đó Hệ thống 1 gửi một tin nhắn đến Hệ thống 2. Các hệ thống này có thể là các microservices khác nhau, một máy chủ gửi một tin nhắn đến một hàm serverless, v.v. Một tin nhắn thường được gửi qua một hàng đợi tin nhắn từ một bên xuất bản (publisher) đến một bên tiêu thụ (consumer), và một khi tin nhắn đã được gửi đến, nó được loại khỏi hàng đợi.



Figure 5-4. A message passed between two systems Messages are discrete and singular signals in an event-driven system. For example, an IoT device might send a message with the latest temperature reading to a message queue. This message is then ingested by a service that determines whether the furnace should be turned on or off. This service sends a message to a furnace controller that takes the appropriate action. Once the message is received, and the action is taken, the message is removed from the message queue. By contrast, a stream is an append-only log of event records. (Streams are ingested and stored in event-streaming platforms, which we discuss at greater length in “Messages and Streams”.) As events occur, they are accumulated in an ordered sequence (Figure 5-5); a timestamp or an ID might order events. (Note that events aren’t always delivered in exact

order because of the subtleties of distributed systems.)

Hình 5-4. Một tin nhắn được truyền giữa hai hệ thống. Tin nhắn là các tín hiệu rời rạc và đơn lẻ trong một hệ thống hướng sự kiện. Ví dụ, một thiết bị IoT có thể gửi một tin nhắn với số đo nhiệt độ mới nhất đến một hàng đợi tin nhắn. Tin nhắn này sau đó được thu nạp bởi một dịch vụ xác định xem lò sưởi nên được bật hay tắt. Dịch vụ này gửi một tin nhắn đến một bộ điều khiển lò sưởi để thực hiện hành động phù hợp. Một khi tin nhắn được nhận và hành động được thực hiện, tin nhắn được loại khỏi hàng đợi tin nhắn. Ngược lại, một *luồng* (*stream*) là một log chỉ-thêm (append-only) của các bản ghi sự kiện. (Luồng được thu nạp và lưu trữ trong các *nền tảng luồng sự kiện* (*event-streaming platform*), mà chúng ta thảo luận kỹ hơn trong “Tin nhắn và luồng”.) Khi các sự kiện xảy ra, chúng được tích lũy trong một trình tự có thứ tự (Hình 5-5); một dấu thời gian hay một ID có thể sắp thứ tự các sự kiện. (Lưu ý rằng các sự kiện không phải lúc nào cũng được gửi đến theo đúng thứ tự vì những tính tế của các hệ phân tán.)

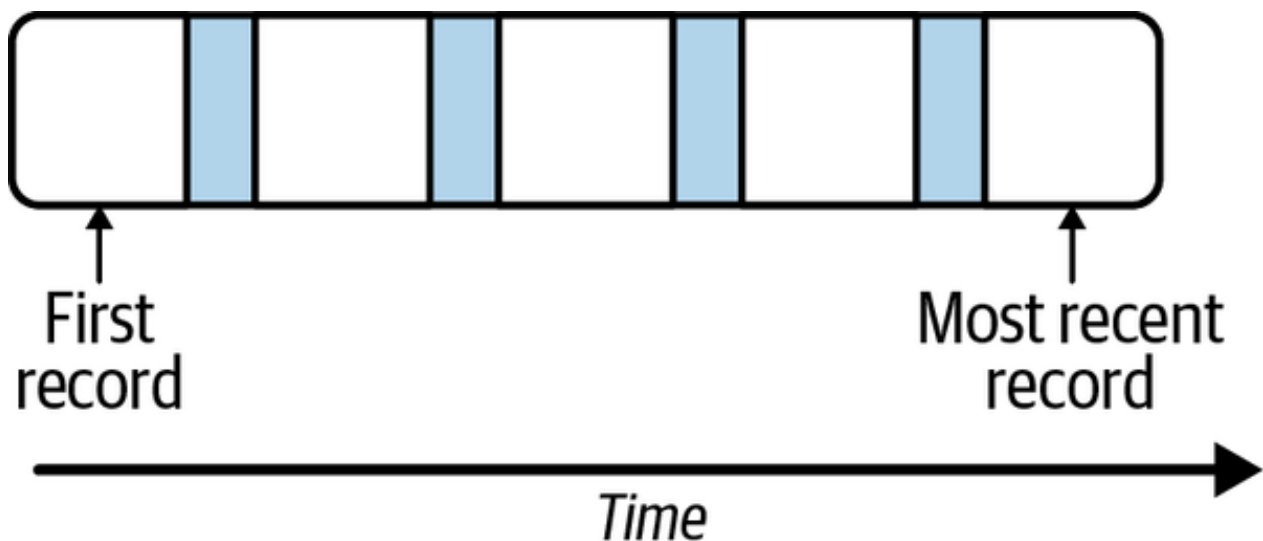


Figure 5-5. A stream, which is an ordered append-only log of records You'll use streams when you care about what happened over many events. Because of the append-only nature of streams, records in a stream are persisted over a long retention window—often weeks or months—allowing for complex operations on records such as aggregations on multiple records or the ability to rewind to a point in time within the stream. It's worth noting that systems that process streams can process messages, and streaming platforms are frequently used for message passing. We often accumulate messages in streams when we want to perform message analytics. In our IoT example, the temperature readings that trigger the furnace to turn on or off might also be later analyzed to determine temperature trends and statistics.

Hình 5-5. Một luồng, vốn là một log chỉ-thêm có thứ tự của các bản ghi. Bạn sẽ dùng luồng khi bạn quan tâm đến điều đã xảy ra qua nhiều sự kiện. Vì bản chất chỉ-thêm của luồng, các bản ghi trong một luồng được lưu giữ bền vững qua một cửa sổ lưu giữ (retention) dài—thường là hàng tuần hay hàng tháng—cho phép các thao tác phức tạp trên các bản ghi như các phép tổng hợp trên nhiều bản ghi hoặc khả năng tua ngược về một thời điểm trong luồng. Đáng lưu ý rằng các hệ thống xử lý luồng có thể xử lý tin nhắn, và các nền tảng luồng thường được dùng cho việc truyền tin nhắn. Chúng ta thường tích lũy tin nhắn trong các luồng khi muốn thực hiện phân tích tin nhắn. Trong ví dụ IoT của chúng ta, các số đo nhiệt độ kích hoạt lò sưởi bật hay tắt cũng có thể được phân tích về sau để xác định các xu hướng và thống kê nhiệt độ.

Types of Time — Các loại thời gian

While time is an essential consideration for all data ingestion, it becomes that much more critical and subtle in the context of streaming, where we view data as continuous and expect to consume it shortly after it is produced. Let's look at the key types of time you'll run into when ingesting data: the time that the event is generated, when it's ingested and processed, and how long processing took (Figure 5-6).

Mặc dù thời gian là một yếu tố cốt yếu cần cân nhắc cho mọi việc thu nạp dữ liệu, nó trở nên còn quan trọng và tinh tế hơn nhiều trong bối cảnh xử lý theo luồng (streaming), nơi chúng ta xem dữ liệu là liên tục và kỳ vọng tiêu thụ nó ngay sau khi nó được tạo ra. Hãy cùng xem các loại thời gian then chốt mà bạn sẽ gặp khi thu nạp dữ liệu: thời điểm sự kiện được sinh ra, khi nó được thu nạp và xử lý, và việc xử lý mất bao lâu (Hình 5-6).

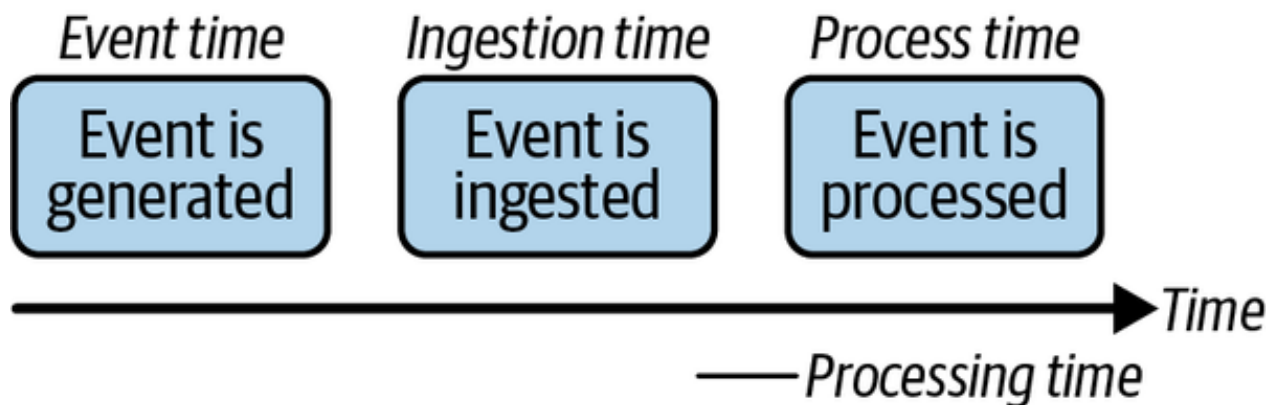


Figure 5-6. Event, ingestion, process, and processing time **Event time** indicates when an event is generated in a source system, including the timestamp of the original event itself. An undetermined time lag will occur upon event creation, before the event is ingested and processed downstream. Always include timestamps for each phase through which an event travels. Log events as they occur and at each stage of time—when they're created, ingested, and processed. Use these timestamp logs to accurately track the movement of your data through your data pipelines.

Hình 5-6. Thời điểm sự kiện, thu nạp, xử lý, và thời lượng xử lý. Thời điểm sự kiện (event time) cho biết khi nào một sự kiện được sinh ra trong một hệ thống nguồn, bao gồm dấu thời gian của chính sự kiện gốc. Một độ trễ thời gian không xác định sẽ xảy ra khi sự kiện được tạo, trước khi sự kiện được thu nạp và xử lý ở hạ nguồn. Hãy luôn bao gồm các dấu thời gian cho từng giai đoạn mà một sự kiện đi qua. Hãy ghi log các sự kiện khi chúng xảy ra và tại mỗi mốc thời gian—khi chúng được tạo, được thu nạp, và được xử lý. Hãy dùng các log dấu thời gian này để theo dõi chính xác sự di chuyển của dữ liệu của bạn qua các đường ống dữ liệu.

After data is created, it is ingested somewhere. Ingestion time indicates when an event is ingested from source systems into a message queue, cache, memory, **object storage**, a database, or any place else that data is stored (see Chapter 6). After ingestion, data may be processed immediately; or within minutes, hours, or days; or simply persist in storage indefinitely. Process time occurs after ingestion time, when the data is processed (typically, a **transformation**). Processing time is how long the data took to process, measured in seconds, minutes, hours, etc. You'll want to record these various times, preferably in an automated way. Set up monitoring along your data workflows to capture when events occur, when they're ingested and processed, and how long it took to process events.

Sau khi dữ liệu được tạo ra, nó được thu nạp đến đâu đó. Thời điểm thu nạp (ingestion

time) cho biết khi nào một sự kiện được thu nạp từ các hệ thống nguồn vào một hàng đợi tin nhắn, cache, bộ nhớ, lưu trữ đối tượng (object storage), một database, hay bất kỳ nơi nào khác mà dữ liệu được lưu trữ (xem Chương 6). Sau khi thu nạp, dữ liệu có thể được xử lý ngay lập tức; hoặc trong vòng vài phút, vài giờ, hay vài ngày; hoặc đơn giản là tồn tại bền vững trong lưu trữ vô thời hạn. Thời điểm xử lý (process time) xảy ra sau thời điểm thu nạp, khi dữ liệu được xử lý (thường là một phép biến đổi (transformation)). Thời lượng xử lý (processing time) là khoảng thời gian dữ liệu mất để được xử lý, đo bằng giây, phút, giờ, v.v. Bạn sẽ muốn ghi lại các mốc thời gian khác nhau này, tốt nhất là theo cách tự động. Hãy thiết lập giám sát dọc theo các luồng công việc dữ liệu của bạn để ghi lại khi nào các sự kiện xảy ra, khi nào chúng được thu nạp và xử lý, và việc xử lý các sự kiện mất bao lâu.

Source System Practical Details — Chi tiết thực tế của hệ thống nguồn

This section discusses the practical details of interacting with modern source systems. We'll dig into the details of commonly encountered databases, APIs, and other aspects. This information will have a shorter shelf life than the main ideas discussed previously; popular API frameworks, databases, and other details will continue to change rapidly. Nevertheless, these details are critical knowledge for working data engineers. We suggest that you study this information as baseline knowledge but read extensively to stay abreast of ongoing developments.

Phần này thảo luận các chi tiết thực tế của việc tương tác với các hệ thống nguồn hiện đại. Chúng ta sẽ đào sâu vào các chi tiết của những database, API và các khía cạnh khác thường gặp. Thông tin này sẽ có vòng đời ngắn hơn các ý chính đã thảo luận trước đó; các framework API phổ biến, database, và các chi tiết khác sẽ tiếp tục thay đổi nhanh chóng. Tuy nhiên, các chi tiết này là kiến thức then chốt cho các kỹ sư dữ liệu đang hành nghề. Chúng tôi gợi ý bạn nghiên cứu thông tin này như kiến thức nền tảng nhưng đọc rộng rãi để theo kịp các diễn biến đang diễn ra.

Databases — Database

In this section, we'll look at common source system database technologies that you'll encounter as a data engineer and high-level considerations for working with these systems. There are as many types of databases as there are use cases for data.

Trong phần này, chúng ta sẽ xem xét các công nghệ database hệ thống nguồn phổ biến mà bạn sẽ gặp với vai trò kỹ sư dữ liệu cùng các cân nhắc ở mức cao khi làm việc với những hệ thống này. Có bao nhiêu loại database thì có bấy nhiêu tình huống sử dụng cho dữ liệu.

Major considerations for understanding database technologies — Các cân nhắc chính để hiểu các công nghệ database

Here, we introduce major ideas that occur across a variety of database technologies, including those that back software applications and those that support analytics use cases: Database management system

Ở đây, chúng tôi giới thiệu các ý tưởng chính xuất hiện xuyên suốt nhiều công nghệ database khác nhau, bao gồm những công nghệ làm nền cho các ứng dụng phần mềm

và những công nghệ hỗ trợ các tình huống sử dụng phân tích: Hệ quản trị database

A database system used to store and serve data. Abbreviated as DBMS, it consists of a storage engine, **query optimizer**, disaster recovery, and other key components for managing the database system.

Một hệ thống database được dùng để lưu trữ và phục vụ dữ liệu. Viết tắt là DBMS, nó bao gồm một bộ máy lưu trữ (storage engine), một bộ tối ưu truy vấn (query optimizer), khả năng khôi phục thảm họa, cùng các thành phần then chốt khác để quản lý hệ thống database.

Lookups

Tra cứu

How does the database find and retrieve data? Indexes can help speed up lookups, but not all databases have indexes. Know whether your database uses indexes; if so, what are the best patterns for designing and maintaining them? Understand how to leverage for efficient extraction. It also helps to have a basic knowledge of the major types of indexes, including B-tree and log-structured merge-trees (LSM).

Database tìm và truy xuất dữ liệu như thế nào? Các chỉ mục có thể giúp tăng tốc tra cứu, nhưng không phải mọi database đều có chỉ mục. Hãy biết liệu database của bạn có dùng chỉ mục không; nếu có, đâu là các mẫu hình tốt nhất để thiết kế và bảo trì chúng? Hãy hiểu cách tận dụng để trích xuất hiệu quả. Việc có kiến thức cơ bản về các loại chỉ mục chính cũng hữu ích, bao gồm B-tree và log-structured merge-tree (LSM).

Query optimizer

Bộ tối ưu truy vấn

Does the database utilize an optimizer? What are its characteristics?

Database có sử dụng một bộ tối ưu không? Các đặc tính của nó là gì?

Scaling and distribution

Mở rộng và phân tán

Does the database scale with demand? What scaling strategy does it deploy? Does it scale horizontally (more database nodes) or vertically (more resources on a single machine)?

Database có mở rộng quy mô theo nhu cầu không? Nó triển khai chiến lược mở rộng nào? Nó mở rộng ngang (thêm các nút database) hay mở rộng dọc (thêm tài nguyên trên một máy đơn lẻ)?

Modeling patterns

Mẫu hình mô hình hóa

What modeling patterns work best with the database (e.g., data **normalization** or wide tables)? (See Chapter 8 for our discussion of **data modeling**.) CRUD

Các mẫu hình mô hình hóa nào hoạt động tốt nhất với database (ví dụ, chuẩn hóa dữ liệu hay các bảng rộng (wide table))? (Xem Chương 8 cho phần thảo luận của chúng tôi về mô hình hóa dữ liệu.) CRUD

How is data queried, created, updated, and deleted in the database? Every type of database handles CRUD operations differently.

Dữ liệu được truy vấn, tạo, cập nhật và xóa trong database như thế nào? Mỗi loại database xử lý các thao tác CRUD theo cách khác nhau.

Consistency

Tính nhất quán

Is the database fully consistent, or does it support a relaxed consistency model (e.g., eventual consistency)? Does the database support optional consistency modes for reads and writes (e.g., strongly consistent reads)?

Database hoàn toàn nhất quán, hay nó hỗ trợ một mô hình nhất quán được nới lỏng (ví dụ, nhất quán cuối cùng)? Database có hỗ trợ các chế độ nhất quán tùy chọn cho việc đọc và ghi không (ví dụ, đọc nhất quán mạnh)?

We divide databases into relational and nonrelational categories. In truth, the nonrelational category is far more diverse, but relational databases still occupy significant space in application backends.

Chúng ta chia các database thành các phạm trù quan hệ và phi quan hệ. Thực ra, phạm trù phi quan hệ đa dạng hơn nhiều, nhưng các database quan hệ vẫn chiếm một không gian đáng kể trong các backend ứng dụng.

Relational databases — Database quan hệ

A relational database management system (RDBMS) is one of the most common application backends. Relational databases were developed at IBM in the 1970s and popularized by Oracle in the 1980s. The growth of the internet saw the rise of the LAMP stack (Linux, Apache web server, MySQL, PHP) and an explosion of vendor and open source RDBMS options. Even with the rise of NoSQL databases (described in the following section), relational databases have remained extremely popular. Data is stored in a table of relations (rows), and each relation contains multiple fields (columns); see Figure 5-7. Note that we use the terms column and field interchangeably throughout this book. Each relation in the table has the same **schema** (a sequence of columns with assigned static types such as string, integer, or float). Rows are typically stored as a contiguous sequence of bytes on disk.

Một hệ quản trị database quan hệ (RDBMS) là một trong những backend ứng dụng phổ biến nhất. Các database quan hệ được phát triển tại IBM trong những năm 1970 và được Oracle phổ biến trong những năm 1980. Sự phát triển của internet chứng kiến sự nổi lên của ngăn xếp LAMP (Linux, máy chủ web Apache, MySQL, PHP) và một sự bùng nổ của các tùy chọn RDBMS từ các nhà cung cấp lẫn mã nguồn mở. Ngay cả với sự nổi lên của các database NoSQL (được mô tả trong phần tiếp theo), các database quan hệ vẫn cực kỳ phổ biến. Dữ liệu được lưu trong một bảng gồm các quan hệ (các hàng), và mỗi quan hệ chứa nhiều trường (các cột); xem Hình 5-7. Lưu ý rằng chúng tôi dùng các thuật ngữ *cột* và *trường* thay thế cho nhau xuyên suốt cuốn sách này. Mỗi quan hệ trong bảng có cùng một lược đồ (schema) (một trình tự các cột với các kiểu tĩnh được gán sẵn như chuỗi, số nguyên, hay số thực). Các hàng thường được lưu dưới dạng một trình tự byte liền kề trên đĩa.

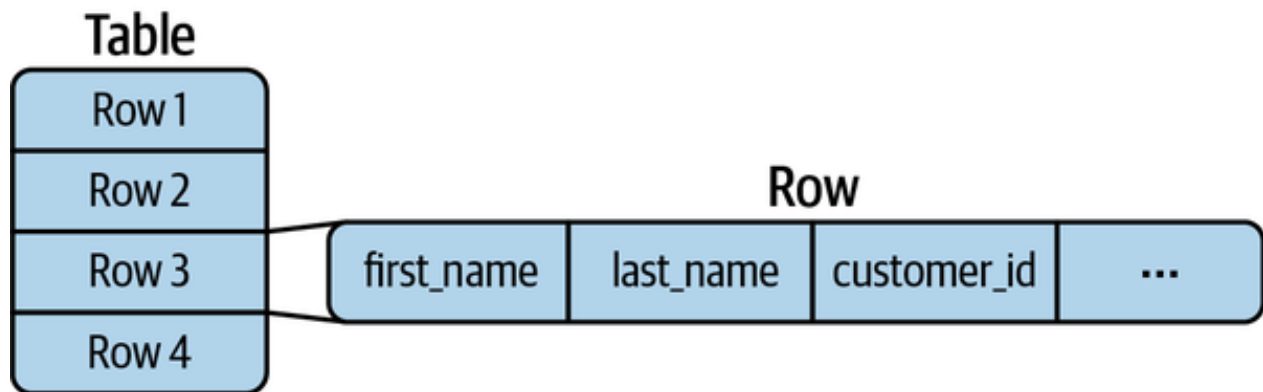


Figure 5-7. RDBMS stores and retrieves data at a row level. Tables are indexed by a primary key, a unique field for each row in the table. The indexing strategy for the primary key is closely connected with the layout of the table on disk. Tables can also have various foreign keys—fields with values connected with the values of primary keys in other tables, facilitating joins, and allowing for complex schemas that spread data across multiple tables. In particular, it is possible to design a normalized schema. Normalization is a strategy for ensuring that data in records is not duplicated in multiple places, thus avoiding the need to update states in multiple locations at once and preventing inconsistencies (see Chapter 8). RDBMS systems are typically ACID compliant. Combining a normalized schema, ACID compliance, and support for high transaction rates makes relational database systems ideal for storing rapidly changing application states. The challenge for data engineers is to determine how to capture state information over time. A full discussion of the theory, history, and technology of RDBMS is beyond the scope of this book. We encourage you to study RDBMS systems, relational algebra, and strategies for normalization because they’re widespread, and you’ll encounter them frequently. See “Additional Resources” for suggested books.

Hình 5-7. RDBMS lưu trữ và truy xuất dữ liệu ở cấp hàng. Các bảng được lập chỉ mục bởi một *khóa chính* (*primary key*), một trường duy nhất cho mỗi hàng trong bảng. Chiến lược lập chỉ mục cho khóa chính gắn chặt với cách bố trí của bảng trên đĩa. Các bảng cũng có thể có nhiều *khóa ngoại* (*foreign key*)—các trường có giá trị được kết nối với giá trị của các khóa chính trong các bảng khác, tạo thuận lợi cho các phép nối (*join*), và cho phép các lược đồ phức tạp trải dữ liệu qua nhiều bảng. Cụ thể, có thể thiết kế một *lược đồ chuẩn hóa*. Chuẩn hóa (*normalization*) là một chiến lược để đảm bảo dữ liệu trong các bản ghi không bị trùng lặp ở nhiều nơi, qua đó tránh nhu cầu cập nhật trạng thái ở nhiều vị trí cùng lúc và ngăn ngừa các bất nhất quán (xem Chương 8). Các hệ thống RDBMS thường tuân thủ ACID. Việc kết hợp một lược đồ chuẩn hóa, sự tuân thủ ACID, và hỗ trợ cho tốc độ giao dịch cao khiến các hệ thống database quan hệ trở nên lý tưởng để lưu trữ các trạng thái ứng dụng thay đổi nhanh chóng. Thách thức đối với các kỹ sư dữ liệu là xác định cách thu thập thông tin trạng thái theo thời gian. Một thảo luận đầy đủ về lý thuyết, lịch sử và công nghệ của RDBMS nằm ngoài phạm vi của cuốn sách này. Chúng tôi khuyến khích bạn nghiên cứu các hệ thống RDBMS, đại số quan hệ, và các chiến lược chuẩn hóa vì chúng phổ biến rộng rãi, và bạn sẽ gặp chúng thường xuyên. Xem “Tài nguyên bổ sung” để biết các cuốn sách được gợi ý.

Nonrelational databases: NoSQL — Database phi quan hệ: NoSQL

While relational databases are terrific for many use cases, they're not a one-size-fits-all solution. We often see that people start with a relational database under the impression it's a universal appliance and shoehorn in a ton of use cases and workloads. As data and query requirements morph, the relational database collapses under its weight. At that point, you'll want to use a database that's appropriate for the specific workload under pressure. Enter nonrelational or NoSQL databases. NoSQL, which stands for not only SQL, refers to a whole class of databases that abandon the relational paradigm. On the one hand, dropping relational constraints can improve performance, scalability, and schema flexibility. But as always in architecture, trade-offs exist. NoSQL databases also typically abandon various RDBMS characteristics, such as strong consistency, joins, or a fixed schema. A big theme of this book is that data innovation is constant. Let's take a quick look at the history of NoSQL, as it's helpful to gain a perspective on why and how data innovations impact your work as a data engineer. In the early 2000s, tech companies such as Google and Amazon began to outgrow their relational databases and pioneered new distributed, nonrelational databases to scale their web platforms. While the term NoSQL first appeared in 1998, the modern version was coined by Eric Evans in the 2000s. He tells the story in a 1 2009 blog post :

Mặc dù các database quan hệ tuyệt vời cho nhiều tình huống sử dụng, chúng không phải là một giải pháp vừa-cho-tất-cả. Chúng ta thường thấy rằng người ta bắt đầu với một database quan hệ với ấn tượng rằng nó là một thiết bị vạn năng và nhồi nhét vào đó vô số tình huống sử dụng và tải công việc. Khi các yêu cầu về dữ liệu và truy vấn biến đổi, database quan hệ sụp đổ dưới sức nặng của chính nó. Tại thời điểm đó, bạn sẽ muốn dùng một database phù hợp cho tải công việc cụ thể đang chịu áp lực. Đây là lúc các database phi quan hệ hay NoSQL xuất hiện. NoSQL, viết tắt của *not only SQL* (không chỉ SQL), đề cập đến cả một lớp các database từ bỏ mô thức quan hệ. Một mặt, việc bỏ các ràng buộc quan hệ có thể cải thiện hiệu năng, khả năng mở rộng, và sự linh hoạt của lược đồ. Nhưng như mọi khi trong kiến trúc, các đánh đổi luôn tồn tại. Các database NoSQL cũng thường từ bỏ nhiều đặc tính của RDBMS, như nhất quán mạnh, phép nối, hay một lược đồ cố định. Một chủ đề lớn của cuốn sách này là sự đổi mới về dữ liệu là không ngừng. Hãy cùng nhìn nhanh vào lịch sử của NoSQL, vì việc có một góc nhìn về tại sao và làm thế nào các đổi mới dữ liệu tác động đến công việc của bạn với vai trò kỹ sư dữ liệu là hữu ích. Vào đầu những năm 2000, các công ty công nghệ như Google và Amazon bắt đầu vượt quá khả năng của các database quan hệ của họ và tiên phong các database phân tán, phi quan hệ mới để mở rộng quy mô các nền tảng web của họ. Mặc dù thuật ngữ NoSQL lần đầu xuất hiện vào năm 1998, phiên bản hiện đại được Eric Evans đặt ra vào những năm 2000. Ông kể câu chuyện trong một bài blog năm 2009: 1

I've spent the last couple of days at nosqleast and one of the **hot** topics here is the name "nosql." Understandably, there are a lot of people who worry that the name is Bad, that it sends an inappropriate or inaccurate message. While I make no claims to the idea, I do have to accept some blame for what it is now being called. How's that? Johan Oskarsson was organizing the first meetup and asked the question "What's a good name?" on IRC; it was one of three or four suggestions that I spouted off in the span of like 45 seconds, without thinking.

Tôi đã dành mấy ngày qua tại nosqleast và một trong những chủ đề nóng ở đây là cái tên "nosql." Dễ hiểu là có nhiều người lo rằng cái tên này Tệ, rằng nó gửi đi một thông điệp không phù hợp hay không chính xác. Mặc dù tôi không nhận công về ý tưởng này, tôi phải chấp nhận một phần trách nhiệm cho cái mà nó đang được gọi hiện nay. Tại sao vậy? Johan Oskarsson đang tổ chức buổi gặp đầu

tiên và đặt câu hỏi “Tên gì thì hay?” trên IRC; đó là một trong ba bốn gợi ý mà tôi tuôn ra trong khoảng 45 giây, mà chẳng suy nghĩ gì.

My regret, however, isn’t about what the name says; it’s about what it doesn’t. When Johan originally had the idea for the first meetup, he seemed to be thinking Big Data and linearly scalable distributed systems, but the name is so vague that it opened the door to talk submissions for literally anything that stored data, and wasn’t an RDBMS.

Tuy nhiên, điều tôi tiếc không phải là về cái tên nói lên điều gì; mà là về cái nó không nói. Khi Johan ban đầu có ý tưởng cho buổi gặp đầu tiên, anh ấy dường như đang nghĩ đến Big Data và các hệ phân tán có thể mở rộng tuyến tính, nhưng cái tên mơ hồ đến mức nó mở cửa cho các bài trình bày về theo nghĩa đen là bất cứ thứ gì lưu trữ dữ liệu, và không phải là một RDBMS.

NoSQL remains vague in 2022, but it’s been widely adopted to describe a universe of “new school” databases, alternatives to relational databases. There are numerous flavors of NoSQL database designed for almost any imaginable use case. Because there are far too many NoSQL databases to cover exhaustively in this section, we consider the following database types: key-value, document, wide-column, graph, search, and time series. These databases are all wildly popular and enjoy widespread adoption. A data engineer should understand these types of databases, including usage considerations, the structure of the data they store, and how to leverage each in the data engineering lifecycle.

NoSQL vẫn mơ hồ vào năm 2022, nhưng nó đã được áp dụng rộng rãi để mô tả một vũ trụ các database “trường phái mới”, các phương án thay thế cho database quan hệ. Có vô số biến thể của database NoSQL được thiết kế cho gần như bất kỳ tình huống sử dụng nào có thể tưởng tượng được. Vì có quá nhiều database NoSQL để bao quát hết trong phần này, chúng tôi xem xét các loại database sau: key-value, document, wide-column, graph, search, và time series. Tất cả những database này đều cực kỳ phổ biến và được áp dụng rộng rãi. Một kỹ sư dữ liệu nên hiểu các loại database này, bao gồm các cân nhắc về cách sử dụng, cấu trúc của dữ liệu mà chúng lưu trữ, và cách tận dụng từng loại trong vòng đời kỹ thuật dữ liệu.

Key-value stores A key-value database is a nonrelational database that retrieves records using a key that uniquely identifies each record. This is similar to hash map or dictionary data structures presented in many programming languages but potentially more scalable. Key-value stores encompass several NoSQL database types—for example, document stores and wide column databases (discussed next). Different types of key-value databases offer a variety of performance characteristics to serve various application needs. For example, in-memory key-value databases are popular for caching session data for web and mobile applications, where ultra-fast lookup and high concurrency are required. Storage in these systems is typically temporary; if the database shuts down, the data disappears. Such caches can reduce pressure on the main application database and serve speedy responses. Of course, key-value stores can also serve applications requiring high-durability persistence. An ecommerce application may need to save and update massive amounts of event state changes for a user and their orders. A user logs into the ecommerce application, clicks around various screens, adds items to a shopping cart, and then checks out. Each event must be durably stored for retrieval. Key-value stores often persist data to disk and across multiple nodes to support such use cases.

Kho key-value Một database key-value là một database phi quan hệ truy xuất các bản ghi bằng một khóa định danh duy nhất mỗi bản ghi. Điều này tương tự như các cấu trúc dữ liệu bản đồ băm (hash map) hay từ điển (dictionary) có trong nhiều ngôn ngữ lập trình nhưng có khả năng mở rộng tốt hơn. Kho key-value bao hàm nhiều loại database NoSQL—chẳng

hạn, kho document và database wide-column (sẽ bàn tiếp). Các loại database key-value khác nhau cung cấp nhiều đặc tính hiệu năng khác nhau để phục vụ các nhu cầu ứng dụng khác nhau. Ví dụ, các database key-value trong-bộ-nhớ phổ biến cho việc cache dữ liệu phiên (session) cho các ứng dụng web và di động, nơi cần tra cứu siêu nhanh và độ đồng thời cao. Lưu trữ trong các hệ thống này thường là tạm thời; nếu database tắt, dữ liệu biến mất. Những cache như vậy có thể giảm áp lực lên database chính của ứng dụng và phục vụ các phản hồi nhanh chóng. Tất nhiên, kho key-value cũng có thể phục vụ các ứng dụng đòi hỏi sự lưu trữ bền vững với độ bền cao. Một ứng dụng thương mại điện tử có thể cần lưu và cập nhật những lượng khổng lồ các thay đổi trạng thái sự kiện cho một người dùng và các đơn hàng của họ. Một người dùng đăng nhập vào ứng dụng thương mại điện tử, nhấp quanh các màn hình khác nhau, thêm các món hàng vào giỏ mua sắm, rồi thanh toán. Mỗi sự kiện phải được lưu trữ bền vững để truy xuất. Kho key-value thường lưu trữ dữ liệu bền vững xuống đĩa và qua nhiều nút để hỗ trợ những tình huống sử dụng như vậy.

Document stores As mentioned previously, a document store is a specialized key-value store. In this context, a document is a nested object; we can usually think of each document as a JSON object for practical purposes. Documents are stored in collections and retrieved by key. A collection is roughly equivalent to a table in a relational database (see Table 5-2).

Kho document Như đã đề cập trước đó, một kho document là một kho key-value chuyên biệt. Trong bối cảnh này, một document là một đối tượng lồng nhau; chúng ta thường có thể nghĩ về mỗi document như một đối tượng JSON cho các mục đích thực tế. Document được lưu trong các tập hợp (collection) và được truy xuất bằng khóa. Một collection gần như tương đương với một bảng trong một database quan hệ (xem Bảng 5-2).

Table 5-2 . Comparison of RDBMS and document terminology

Bảng 5-2. So sánh thuật ngữ RDBMS và document

RDBMS Document database

RDBMS Document database

Table Collection Row Document, items, entity

Table Collection Row Document, items, entity

One key difference between relational databases and document stores is that the latter does not support joins. This means that data cannot be easily normalized , i.e., split across multiple tables. (Applications can still join manually. Code can look up a document, extract a property, and then retrieve another document.) Ideally, all related data can be stored in the same document. In many cases, the same data must be stored in multiple documents spread across numerous collections; software engineers must be careful to update a property everywhere it is stored. (Many document stores support a notion of transactions to facilitate this.) Document databases generally embrace all the flexibility of JSON and don't enforce schema or types; this is a blessing and a curse. On the one hand, this allows the schema to be highly flexible and expressive. The schema can also evolve as an application grows. On the flip side, we've seen document databases become absolute nightmares to manage and query. If developers are not careful in managing **schema evolution**, data may become inconsistent and bloated over time. Schema evolution can also break downstream ingestion and cause headaches for data engineers if it's not communicated in a timely fashion (before deployment). The following is an example of data that is stored in a collection called users . The collection key is the id . We also have a name (along with first and last as child elements) and an array of the person's favorite bands within each document:

Một khác biệt then chốt giữa các database quan hệ và kho document là kho document không hỗ trợ các phép nối. Điều này nghĩa là dữ liệu không thể được *chuẩn hóa* dễ dàng, tức là tách qua nhiều bảng. (Các ứng dụng vẫn có thể nối một cách thủ công. Mà có thể tra cứu một document, trích xuất một thuộc tính, rồi truy xuất một document khác.) Lý tưởng nhất là tất cả dữ liệu liên quan có thể được lưu trong cùng một document. Trong nhiều trường hợp, cùng một dữ liệu phải được lưu trong nhiều document trải ra qua vô số collection; các kỹ sư phần mềm phải cẩn thận cập nhật một thuộc tính ở mọi nơi nó được lưu. (Nhiều kho document hỗ trợ một khái niệm giao dịch để tạo thuận lợi cho việc này.) Các database document nhìn chung ôm trọn toàn bộ sự linh hoạt của JSON và không cưỡng chế lược đồ hay kiểu; đây vừa là một phúc lành vừa là một lời nguyền. Một mặt, điều này cho phép lược đồ cực kỳ linh hoạt và biểu cảm. Lược đồ cũng có thể tiến hóa khi một ứng dụng phát triển. Mặt khác, chúng tôi đã thấy các database document trở thành những cơn ác mộng tuyệt đối để quản lý và truy vấn. Nếu các nhà phát triển không cẩn thận trong việc quản lý sự tiến hóa lược đồ (schema evolution), dữ liệu có thể trở nên bất nhất quán và phình to theo thời gian. Tiến hóa lược đồ cũng có thể phá vỡ việc thu nạp ở hạ nguồn và gây đau đầu cho các kỹ sư dữ liệu nếu nó không được thông báo kịp thời (trước khi triển khai). Sau đây là một ví dụ về dữ liệu được lưu trong một collection tên là users. Khóa của collection là id. Chúng ta cũng có một name (cùng với first và last là các phần tử con) và một mảng các ban nhạc yêu thích của người đó trong mỗi document:

```
users: [
  {
    id:1234
    name:
    first:"Joe"
    last:"Reis"
    favorite_bands:["AC/DC","Slayer","WuTang Clan",
    "Action Bronson"]
  },
  {
    id:1235
    name:
    first:"Matt"
    last:"Housley"
    favorite_bands:["Dave Matthews Band","Creed",
    "Nickelback"]
  }
]
```

To query the data in this example, you can retrieve records by key. Note that most document databases also support the creation of indexes and lookup tables to allow retrieval of documents by specific properties. This is often invaluable in application development when you need to search for documents in various ways. For example, you could set an index on name .

Để truy vấn dữ liệu trong ví dụ này, bạn có thể truy xuất các bản ghi bằng khóa. Lưu ý rằng hầu hết các database document cũng hỗ trợ việc tạo các chỉ mục và bảng tra cứu để cho phép truy xuất các document theo các thuộc tính cụ thể. Điều này thường vô giá trong phát triển ứng dụng khi bạn cần tìm kiếm các document theo nhiều cách khác nhau. Ví dụ, bạn có thể đặt một chỉ mục trên name.

Another critical technical detail for data engineers is that document stores are generally not ACID compliant, unlike relational databases. Technical expertise in a particular document store is

essential to understanding performance, tuning, configuration, related effects on writes, consistency, durability, etc. For example, many document stores are eventually consistent . Allowing data distribution across a cluster is a boon for scaling and performance but can lead to catastrophes when engineers and developers don't understand the implications. 2

Một chi tiết kỹ thuật then chốt khác cho các kỹ sư dữ liệu là các kho document nhìn chung không tuân thủ ACID, không như các database quan hệ. Chuyên môn kỹ thuật về một kho document cụ thể là cốt yếu để hiểu hiệu năng, tính chỉnh, cấu hình, các ảnh hưởng liên quan đến việc ghi, tính nhất quán, độ bền, v.v. Ví dụ, nhiều kho document là *nhất quán cuối cùng*. Việc cho phép phân tán dữ liệu qua một cụm là một lợi thế cho việc mở rộng và hiệu năng nhưng có thể dẫn đến các thảm họa khi các kỹ sư và nhà phát triển không hiểu các hệ quả. 2

To run analytics on document stores, engineers generally must run a full scan to extract all data from a collection or employ a CDC strategy to send events to a target stream. The full scan approach can have both performance and cost implications. The scan often slows the database as it runs, and many serverless cloud offerings charge a significant fee for each full scan. In document databases, it's often helpful to create an index to help speed up queries. We discuss indexes and query patterns in Chapter 8 .

Để chạy phân tích trên các kho document, các kỹ sư nhìn chung phải chạy một lần quét toàn bộ để trích xuất tất cả dữ liệu từ một collection, hoặc dùng một chiến lược CDC để gửi các sự kiện đến một luồng đích. Cách tiếp cận quét toàn bộ có thể có cả hệ quả về hiệu năng lẫn chi phí. Lần quét thường làm chậm database khi nó chạy, và nhiều dịch vụ serverless trên cloud tính một khoản phí đáng kể cho mỗi lần quét toàn bộ. Trong các database document, việc tạo một chỉ mục để giúp tăng tốc các truy vấn thường hữu ích. Chúng ta thảo luận về các chỉ mục và mẫu hình truy vấn trong Chương 8.

Wide-column A wide-column database is optimized for storing massive amounts of data with high transaction rates and extremely low latency. These databases can scale to extremely high write rates and vast amounts of data. Specifically, wide-column databases can support petabytes of data, millions of requests per second, and sub-10ms latency. These characteristics have made wide-column databases popular in ecommerce, fintech, ad tech, IoT, and real-time personalization applications. Data engineers must be aware of the operational characteristics of the wide-column databases they work with to set up a suitable configuration, design the schema, and choose an appropriate row key to optimize performance and avoid common operational issues. These databases support rapid scans of massive amounts of data, but they do not support complex queries. They have only a single index (the row key) for lookups. Data engineers must generally extract data and send it to a secondary analytics system to run complex queries to deal with these limitations. This can be accomplished by running large scans for the extraction or employing CDC to capture an event stream.

Một database wide-column được tối ưu để lưu trữ những lượng khổng lồ dữ liệu với tốc độ giao dịch cao và độ trễ cực thấp. Những database này có thể mở rộng đến tốc độ ghi cực cao và những lượng dữ liệu khổng lồ. Cụ thể, các database wide-column có thể hỗ trợ hàng petabyte dữ liệu, hàng triệu yêu cầu mỗi giây, và độ trễ dưới 10ms. Những đặc tính này đã khiến các database wide-column phổ biến trong thương mại điện tử, fintech, ad tech, IoT, và các ứng dụng cá nhân hóa thời gian thực. Các kỹ sư dữ liệu phải nắm được các đặc tính vận hành của những database wide-column mà họ làm việc cùng để thiết lập một cấu hình phù hợp, thiết kế lược đồ, và chọn một khóa hàng (row key) phù hợp để tối ưu hiệu năng và tránh các vấn đề vận hành thường gặp. Những database này hỗ trợ quét nhanh những lượng dữ liệu khổng lồ, nhưng chúng không hỗ trợ các truy vấn phức tạp. Chúng chỉ có một chỉ mục duy nhất (khóa hàng) để tra cứu. Các kỹ sư dữ liệu nhìn chung phải trích xuất dữ liệu và gửi nó đến một hệ thống phân tích thứ cấp để chạy các truy vấn

phức tạp nhằm xử lý những giới hạn này. Điều này có thể đạt được bằng cách chạy các lần quét lớn để trích xuất hoặc dùng CDC để thu thập một luồng sự kiện.

Graph databases Graph databases explicitly store data with a mathematical graph structure (as a set of nodes and edges). Neo4j has proven extremely popular, while 3 Amazon, Oracle, and other vendors offer their graph database products. Roughly speaking, graph databases are a good fit when you want to analyze the connectivity between elements. For example, you could use a document database to store one document for each user describing their properties. You could add an array element for connections that contains directly connected users' IDs in a social media context. It's pretty easy to determine the number of direct connections a user has, but suppose you want to know how many users can be reached by traversing two direct connections. You could answer this question by writing complex code, but each query would run slowly and consume significant resources. The document store is simply not optimized for this use case. Graph databases are designed for precisely this type of query. Their data structures allow for queries based on the connectivity between elements; graph databases are indicated when we care about understanding complex traversals between elements. In the parlance of graphs, we store nodes (users in the preceding example) and edges (connections between users). Graph databases support rich data models for both nodes and edges. Depending on the underlying graph database engine, graph databases utilize specialized query languages such as SPARQL, Resource Description Framework (RDF), Graph Query Language (GQL), and Cypher. As an example of a graph, consider a network of four users. User 1 follows User 2, who follows User 3 and User 4; User 3 also follows User 4 (Figure 5-8).

Database đồ thị Các database đồ thị lưu trữ dữ liệu một cách tường minh với một cấu trúc đồ thị toán học (như một tập các nút và cạnh). Neo4j đã chứng tỏ cực kỳ phổ biến, trong khi 3 Amazon, Oracle, và các nhà cung cấp khác cung cấp các sản phẩm database đồ thị của họ. Nói nôm na, các database đồ thị phù hợp khi bạn muốn phân tích tính liên kết giữa các phần tử. Ví dụ, bạn có thể dùng một database document để lưu một document cho mỗi người dùng mô tả các thuộc tính của họ. Bạn có thể thêm một phần tử mảng cho các kết nối chứa ID của những người dùng được kết nối trực tiếp trong bối cảnh mạng xã hội. Khá dễ để xác định số lượng kết nối trực tiếp mà một người dùng có, nhưng giả sử bạn muốn biết có bao nhiêu người dùng có thể với tới bằng cách đi qua hai kết nối trực tiếp. Bạn có thể trả lời câu hỏi này bằng cách viết mã phức tạp, nhưng mỗi truy vấn sẽ chạy chậm và tiêu tốn tài nguyên đáng kể. Kho document đơn giản là không được tối ưu cho tình huống sử dụng này. Các database đồ thị được thiết kế đúng cho loại truy vấn này. Cấu trúc dữ liệu của chúng cho phép các truy vấn dựa trên tính liên kết giữa các phần tử; các database đồ thị được chỉ định khi chúng ta quan tâm đến việc hiểu các phép đi xuyên phức tạp giữa các phần tử. Trong ngôn ngữ của đồ thị, chúng ta lưu các nút (người dùng trong ví dụ trên) và các cạnh (các kết nối giữa người dùng). Các database đồ thị hỗ trợ các mô hình dữ liệu phong phú cho cả nút lẫn cạnh. Tùy vào bộ máy database đồ thị nền tảng, các database đồ thị sử dụng các ngôn ngữ truy vấn chuyên biệt như SPARQL, Resource Description Framework (RDF), Graph Query Language (GQL), và Cypher. Như một ví dụ về đồ thị, hãy xét một mạng gồm bốn người dùng. Người dùng 1 theo dõi Người dùng 2, người này theo dõi Người dùng 3 và Người dùng 4; Người dùng 3 cũng theo dõi Người dùng 4 (Hình 5-8).

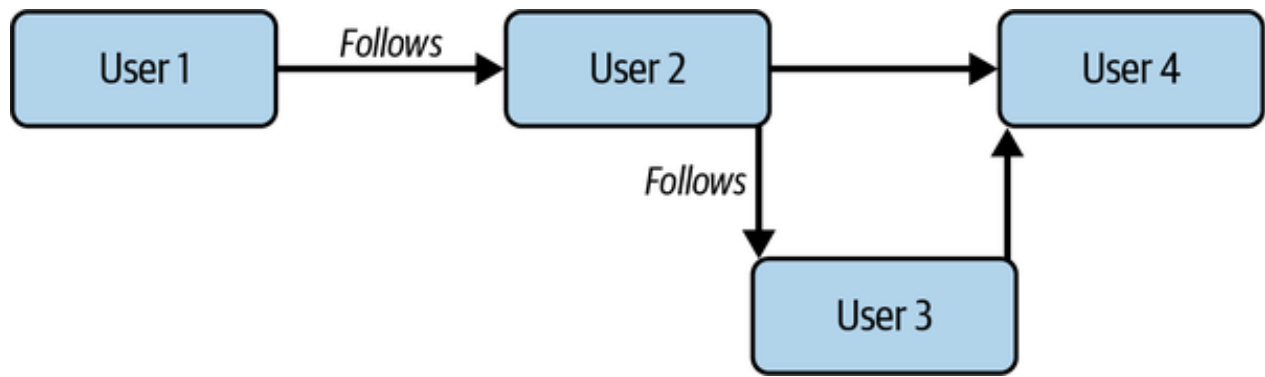


Figure 5-8. A social network graph We anticipate that graph database applications will grow dramatically outside of tech companies; market analyses also predict rapid growth. Of 4 course, graph databases are beneficial from an operational perspective and support the kinds of complex social relationships critical to modern applications. Graph structures are also fascinating from the perspective of data science and ML, potentially revealing deep insights into human interactions and behavior. This introduces unique challenges for data engineers who may be more accustomed to dealing with structured relations, documents, or unstructured data. Engineers must choose whether to do the following:

Hình 5-8. Một đồ thị mạng xã hội. Chúng tôi dự đoán rằng các ứng dụng database đồ thị sẽ phát triển mạnh mẽ bên ngoài các công ty công nghệ; các phân tích thị trường cũng dự báo sự tăng trưởng nhanh chóng. 4 Tất nhiên, các database đồ thị có lợi từ góc độ vận hành và hỗ trợ các kiểu quan hệ xã hội phức tạp vốn then chốt với các ứng dụng hiện đại. Các cấu trúc đồ thị cũng hấp dẫn từ góc độ khoa học dữ liệu và ML, có khả năng hé lộ những hiểu biết sâu sắc về tương tác và hành vi của con người. Điều này đặt ra những thách thức độc đáo cho các kỹ sư dữ liệu vốn có thể quen hơn với việc xử lý các quan hệ có cấu trúc, các document, hay dữ liệu phi cấu trúc. Các kỹ sư phải chọn liệu có nên làm những điều sau hay không:

Map source system graph data into one of their existing preferred paradigms

Ánh xạ dữ liệu đồ thị của hệ thống nguồn vào một trong các mô thức ưa thích hiện có của họ

Analyze graph data within the source system itself

Phân tích dữ liệu đồ thị ngay trong chính hệ thống nguồn

Adopt graph-specific analytics tools

Áp dụng các công cụ phân tích chuyên biệt cho đồ thị

Graph data can be reencoded into rows in a relational database, which may be a suitable solution depending on the analytics use case. Transactional graph databases are also designed for analytics, although large queries may overload production systems. Contemporary cloud-based graph databases support read-heavy graph analytics on massive quantities of data.

Dữ liệu đồ thị có thể được mã hóa lại thành các hàng trong một database quan hệ, điều có thể là một giải pháp phù hợp tùy vào tình huống sử dụng phân tích. Các database đồ thị giao dịch cũng được thiết kế cho phân tích, mặc dù các truy vấn lớn có thể làm quá tải các hệ thống production. Các database đồ thị dựa trên cloud đương đại hỗ trợ phân tích đồ thị nặng về đọc trên những lượng dữ liệu khổng lồ.

Search A search database is a nonrelational database used to search your data's complex and

straightforward **semantic** and structural characteristics. Two prominent use cases exist for a search database: text search and log analysis. Let's cover each of these separately. Text search involves searching a body of text for keywords or phrases, matching on exact, fuzzy, or semantically similar matches. Log analysis is typically used for anomaly detection, real-time monitoring, security analytics, and operational analytics. Queries can be optimized and sped up with the use of indexes. Depending on the type of company you work at, you may use search databases either regularly or not at all. Regardless, it's good to be aware they exist in case you come across them in the wild. Search databases are popular for fast search and retrieval and can be found in various applications; an ecommerce site may power its product search using a search database. As a data engineer, you might be expected to bring data from a search database (such as Elasticsearch, Apache Solr or Lucene, or Algolia) into downstream KPI reports or something similar.

Search Một database search là một database phi quan hệ được dùng để tìm kiếm các đặc tính ngữ nghĩa và cấu trúc phức tạp lẫn đơn giản của dữ liệu của bạn. Hai tình huống sử dụng nổi bật tồn tại cho một database search: tìm kiếm văn bản và phân tích log. Hãy bàn từng cái riêng. Tìm kiếm văn bản liên quan đến việc tìm trong một khối văn bản các từ khóa hay cụm từ, khớp theo các kết quả khớp chính xác, mờ (fuzzy), hay tương đồng về ngữ nghĩa. Phân tích log thường được dùng cho phát hiện bất thường, giám sát thời gian thực, phân tích an ninh, và phân tích vận hành. Các truy vấn có thể được tối ưu và tăng tốc bằng việc dùng các chỉ mục. Tùy vào loại công ty bạn làm việc, bạn có thể dùng các database search hoặc thường xuyên hoặc hoàn toàn không. Dù sao, biết rằng chúng tồn tại là điều tốt phòng khi bạn bắt gặp chúng ngoài thực tế. Các database search phổ biến cho việc tìm kiếm và truy xuất nhanh và có thể được tìm thấy trong nhiều ứng dụng khác nhau; một trang thương mại điện tử có thể vận hành tìm kiếm sản phẩm của mình bằng một database search. Với vai trò kỹ sư dữ liệu, bạn có thể được kỳ vọng đưa dữ liệu từ một database search (chẳng hạn như Elasticsearch, Apache Solr hay Lucene, hoặc Algolia) vào các báo cáo KPI ở hạ nguồn hay những thứ tương tự.

Time series A time series is a series of values organized by time. For example, stock prices might move as trades are executed throughout the day, or a weather sensor will take atmospheric temperatures every minute. Any events that are recorded over time—either regularly or sporadically—are time-series data. A time-series database is optimized for retrieving and statistical processing of time-series data. While time-series data such as orders, shipments, logs, and so forth have been stored in relational databases for ages, these data sizes and volumes were often tiny. As data grew faster and bigger, new special-purpose databases were needed. Time-series databases address the needs of growing, high-velocity data volumes from IoT, event and application logs, ad tech, and fintech, among many other use cases. Often these workloads are write-heavy. As a result, time-series databases often utilize memory buffering to support fast writes and reads. We should distinguish between measurement and event-based data, common in time-series databases. Measurement data is generated regularly, such as temperature or air-quality sensors. Event-based data is irregular and created every time an event occurs—for instance, when a motion sensor detects movement. The schema for a time series typically contains a timestamp and a small set of fields. Because the data is time-dependent, the data is ordered by the timestamp. This makes time-series databases suitable for operational analytics but not great for BI use cases. Joins are not common, though some quasi time-series databases such as Apache Druid support joins. Many time-series databases are available, both as open source and paid options.

Time series Một time series (chuỗi thời gian) là một chuỗi các giá trị được tổ chức theo thời gian. Ví dụ, giá cổ phiếu có thể biến động khi các giao dịch được thực hiện suốt cả ngày, hoặc một cảm biến thời tiết sẽ đo nhiệt độ khí quyển mỗi phút. Bất kỳ sự kiện nào được ghi lại theo thời gian—dù đều đặn hay rời rạc—đều là dữ liệu chuỗi thời gian. Một database chuỗi thời gian được tối ưu cho việc truy xuất và xử lý thống kê dữ liệu chuỗi thời gian.

Mặc dù dữ liệu chuỗi thời gian như các đơn hàng, các lô hàng, log, v.v. đã được lưu trong các database quan hệ từ lâu, các kích thước và khối lượng dữ liệu này thường nhỏ xíu. Khi dữ liệu phát triển nhanh hơn và lớn hơn, cần đến các database chuyên dụng mới. Các database chuỗi thời gian giải quyết nhu cầu của các khối lượng dữ liệu đang phát triển, vận tốc cao từ IoT, log sự kiện và ứng dụng, ad tech, và fintech, cùng nhiều tình huống sử dụng khác. Các tải công việc này thường nặng về ghi. Kết quả là, các database chuỗi thời gian thường sử dụng đệm bộ nhớ (memory buffering) để hỗ trợ việc ghi và đọc nhanh. Chúng ta nên phân biệt giữa dữ liệu dựa-trên-đo-lường và dữ liệu dựa-trên-sự-kiện, vốn phổ biến trong các database chuỗi thời gian. Dữ liệu đo lường được sinh ra đều đặn, chẳng hạn như các cảm biến nhiệt độ hay chất lượng không khí. Dữ liệu dựa-trên-sự-kiện thì bất quy tắc và được tạo ra mỗi khi một sự kiện xảy ra—chẳng hạn, khi một cảm biến chuyển động phát hiện chuyển động. Lược đồ cho một chuỗi thời gian thường chứa một dấu thời gian và một tập nhỏ các trường. Vì dữ liệu phụ thuộc vào thời gian, dữ liệu được sắp thứ tự theo dấu thời gian. Điều này khiến các database chuỗi thời gian phù hợp cho phân tích vận hành nhưng không tuyệt vời cho các tình huống sử dụng BI. Các phép nối không phổ biến, mặc dù một số database tựa-chuỗi-thời-gian như Apache Druid có hỗ trợ phép nối. Có nhiều database chuỗi thời gian sẵn có, ở cả dạng mã nguồn mở lẫn các tùy chọn trả phí.

APIs — API

APIs are now a standard and pervasive way of exchanging data in the cloud, for SaaS platforms, and between internal company systems. Many types of API interfaces exist across the web, but we are principally interested in those built around HTTP, the most popular type on the web and in the cloud.

API giờ đây là một cách tiêu chuẩn và phổ biến để trao đổi dữ liệu trên cloud, cho các nền tảng SaaS, và giữa các hệ thống nội bộ của công ty. Nhiều loại giao diện API tồn tại trên khắp web, nhưng chúng ta chủ yếu quan tâm đến những loại được xây dựng quanh HTTP, loại phổ biến nhất trên web và trên cloud.

REST — REST

We'll first talk about REST, currently the dominant API paradigm. As noted in Chapter 4 , REST stands for representational state transfer . This set of practices and philosophies for building HTTP web APIs was laid out by Roy Fielding in 2000 in a PhD dissertation. REST is built around HTTP verbs, such as GET and PUT; in practice, modern REST uses only a handful of the verb mappings outlined in the original dissertation. One of the principal ideas of REST is that interactions are stateless. Unlike in a Linux terminal session, there is no notion of a session with associated state variables such as a working directory; each REST call is independent. REST calls can change the system's state, but these changes are global, applying to the full system rather than a current session. Critics point out that REST is in no way a full specification. REST 5 stipulates basic properties of interactions, but developers utilizing an API must gain a significant amount of domain knowledge to build applications or pull data effectively. We see great variation in levels of API abstraction. In some cases, APIs are merely a thin wrapper over internals that provides the minimum functionality required to protect the system from user requests. In other examples, a REST data API is a masterpiece of engineering that prepares data for analytics applications and supports advanced reporting. A couple of developments have simplified setting up data-ingestion pipelines from REST APIs. First, data providers frequently supply client libraries in various languages, especially in Python. Client libraries remove much of the boilerplate labor of building API interaction code. Client libraries handle critical details such as authentication and map fundamental methods into

accessible classes. Second, various services and open source libraries have emerged to interact with APIs and manage data synchronization. Many SaaS and open source vendors provide off-the-shelf connectors for common APIs. Platforms also simplify the process of building custom connectors as required. There are numerous data APIs without client libraries or out-of-the-box connector support. As we emphasize throughout the book, engineers would do well to reduce undifferentiated heavy lifting by using off-the-shelf tools. However, low-level plumbing tasks still consume many resources. At virtually any large company, data engineers will need to deal with the problem of writing and maintaining custom code to pull data from APIs, which requires understanding the structure of the data as provided, developing appropriate data-extraction code, and determining a suitable data synchronization strategy.

Trước tiên chúng ta sẽ nói về REST, mô thức API thống trị hiện nay. Như đã lưu ý trong Chương 4, REST viết tắt của *representational state transfer* (chuyển giao trạng thái biểu diễn). Tập hợp các thực hành và triết lý này để xây dựng các web API HTTP đã được Roy Fielding trình bày năm 2000 trong một luận án tiến sĩ. REST được xây dựng quanh các động từ HTTP, như GET và PUT; trên thực tế, REST hiện đại chỉ dùng một số ít các ánh xạ động từ được phác thảo trong luận án gốc. Một trong những ý tưởng chính của REST là các tương tác là phi trạng thái (stateless). Không như trong một phiên terminal Linux, không có khái niệm về một phiên với các biến trạng thái liên quan như một thư mục làm việc; mỗi lệnh gọi REST là độc lập. Các lệnh gọi REST có thể thay đổi trạng thái của hệ thống, nhưng những thay đổi này là toàn cục, áp dụng cho toàn bộ hệ thống chứ không phải một phiên hiện tại. Những người chỉ trích chỉ ra rằng REST hoàn toàn không phải là một đặc tả đầy đủ. 5 REST quy định các thuộc tính cơ bản của các tương tác, nhưng các nhà phát triển sử dụng một API phải có được một lượng đáng kể kiến thức miền để xây dựng các ứng dụng hay lấy dữ liệu một cách hiệu quả. Chúng ta thấy sự biến thiên lớn ở các mức trừu tượng của API. Trong một số trường hợp, API chỉ đơn thuần là một lớp bọc mỏng trên các nội bộ, cung cấp chức năng tối thiểu cần thiết để bảo vệ hệ thống khỏi các yêu cầu của người dùng. Trong các ví dụ khác, một API dữ liệu REST là một kiệt tác kỹ thuật chuẩn bị dữ liệu cho các ứng dụng phân tích và hỗ trợ báo cáo nâng cao. Một vài diễn biến đã đơn giản hóa việc thiết lập các đường ống thu nạp dữ liệu từ các API REST. Thứ nhất, các nhà cung cấp dữ liệu thường cung cấp các thư viện client bằng nhiều ngôn ngữ khác nhau, đặc biệt là Python. Các thư viện client loại bỏ phần lớn công việc lặp đi lặp lại của việc xây dựng mã tương tác API. Các thư viện client xử lý các chi tiết then chốt như xác thực và ánh xạ các phương thức nền tảng thành các lớp để tiếp cận. Thứ hai, nhiều dịch vụ và thư viện mã nguồn mở đã xuất hiện để tương tác với các API và quản lý đồng bộ hóa dữ liệu. Nhiều nhà cung cấp SaaS và mã nguồn mở cung cấp các connector có sẵn cho các API phổ biến. Các nền tảng cũng đơn giản hóa quá trình xây dựng các connector tùy chỉnh khi cần. Có vô số API dữ liệu không có thư viện client hay hỗ trợ connector có sẵn. Như chúng tôi nhấn mạnh xuyên suốt cuốn sách, các kỹ sư nên giảm bớt phần việc nặng nhọc không tạo khác biệt bằng cách dùng các công cụ có sẵn. Tuy nhiên, các tác vụ “đường ống cấp thấp” vẫn tiêu tốn nhiều tài nguyên. Tại hầu như bất kỳ công ty lớn nào, các kỹ sư dữ liệu sẽ cần đối mặt với vấn đề viết và bảo trì mã tùy chỉnh để lấy dữ liệu từ các API, điều đòi hỏi hiểu cấu trúc của dữ liệu được cung cấp, phát triển mã trích xuất dữ liệu phù hợp, và xác định một chiến lược đồng bộ hóa dữ liệu thích hợp.

GraphQL — GraphQL

GraphQL was created at Facebook as a query language for application data and an alternative to generic REST APIs. Whereas REST APIs generally restrict your queries to a specific data model, GraphQL opens up the possibility of retrieving multiple data models in a single request. This allows

for more flexible and expressive queries than with REST. GraphQL is built around JSON and returns data in a shape resembling the JSON query. There's something of a holy war between REST and GraphQL, with some engineering teams partisans of one or the other and some using both. In reality, engineers will encounter both as they interact with source systems.

GraphQL được tạo ra tại Facebook như một ngôn ngữ truy vấn cho dữ liệu ứng dụng và một phương án thay thế cho các API REST chung chung. Trong khi các API REST nhìn chung giới hạn các truy vấn của bạn vào một mô hình dữ liệu cụ thể, GraphQL mở ra khả năng truy xuất nhiều mô hình dữ liệu trong một yêu cầu duy nhất. Điều này cho phép các truy vấn linh hoạt và biểu cảm hơn so với REST. GraphQL được xây dựng quanh JSON và trả về dữ liệu theo một hình dạng giống với truy vấn JSON. Có một kiểu “thánh chiến” giữa REST và GraphQL, với một số đội kỹ thuật là tín đồ của cái này hay cái kia, và một số dùng cả hai. Trên thực tế, các kỹ sư sẽ gặp cả hai khi họ tương tác với các hệ thống nguồn.

Webhooks — Webhook

Webhooks are a simple event-based data-transmission pattern. The data source can be an application backend, a web page, or a mobile app. When specified events happen in the source system, this triggers a call to an HTTP endpoint hosted by the **data consumer**. Notice that the connection goes from the source system to the data sink, the opposite of typical APIs. For this reason, webhooks are often called reverse APIs. The endpoint can do various things with the POST event data, potentially triggering a downstream process or storing the data for future use. For analytics purposes, we're interested in collecting these events. Engineers commonly use message queues to ingest data at high velocity and volume. We will talk about message queues and event streams later in this chapter.

Webhook là một mẫu hình truyền dữ liệu dựa-trên-sự-kiện đơn giản. Nguồn dữ liệu có thể là một backend ứng dụng, một trang web, hay một ứng dụng di động. Khi các sự kiện được chỉ định xảy ra trong hệ thống nguồn, điều này kích hoạt một lệnh gọi đến một điểm cuối (endpoint) HTTP được lưu trữ bởi bên tiêu thụ dữ liệu. Lưu ý rằng kết nối đi từ hệ thống nguồn đến đích chứa dữ liệu (data sink), ngược lại với các API thông thường. Vì lý do này, webhook thường được gọi là *API ngược (reverse API)*. Điểm cuối có thể làm nhiều việc khác nhau với dữ liệu sự kiện POST, có khả năng kích hoạt một tiến trình ở hạ nguồn hoặc lưu trữ dữ liệu để dùng trong tương lai. Cho các mục đích phân tích, chúng ta quan tâm đến việc thu thập các sự kiện này. Các kỹ sư thường dùng các hàng đợi tin nhắn để thu nạp dữ liệu với vận tốc và khối lượng cao. Chúng ta sẽ nói về các hàng đợi tin nhắn và luồng sự kiện ở phần sau của chương này.

RPC and gRPC — RPC và gRPC

A remote procedure call (RPC) is commonly used in distributed computing. It allows you to run a procedure on a remote system. gRPC is a remote procedure call library developed internally at Google in 2015 and later released as an open standard. Its use at Google alone would be enough to merit inclusion in our discussion. Many Google services, such as Google Ads and GCP, offer gRPC APIs. gRPC is built around the Protocol Buffers open data serialization standard, also developed by Google. gRPC emphasizes the efficient bidirectional exchange of data over HTTP/2. Efficiency refers to aspects such as CPU utilization, power consumption, battery life, and bandwidth. Like GraphQL, gRPC imposes much more specific technical standards than REST, thus allowing the use of common client libraries and allowing engineers to develop a skill set that will apply to any gRPC interaction code.

Một lệnh gọi thủ tục từ xa (RPC) thường được dùng trong tính toán phân tán. Nó cho phép

bạn chạy một thủ tục trên một hệ thống từ xa. gRPC là một thư viện gọi thủ tục từ xa được phát triển nội bộ tại Google năm 2015 và sau đó được phát hành như một chuẩn mở. Chỉ riêng việc nó được dùng tại Google cũng đủ để xứng đáng được đưa vào phần thảo luận của chúng ta. Nhiều dịch vụ của Google, như Google Ads và GCP, cung cấp các API gRPC. gRPC được xây dựng quanh chuẩn tuần tự hóa dữ liệu mở Protocol Buffers, cũng do Google phát triển. gRPC nhấn mạnh việc trao đổi dữ liệu hai chiều hiệu quả qua HTTP/2. Hiệu quả ở đây đề cập đến các khía cạnh như mức sử dụng CPU, mức tiêu thụ điện năng, thời lượng pin, và băng thông. Giống GraphQL, gRPC áp đặt các chuẩn kỹ thuật cụ thể hơn nhiều so với REST, qua đó cho phép việc dùng các thư viện client chung và cho phép các kỹ sư phát triển một bộ kỹ năng sẽ áp dụng được cho bất kỳ mã tương tác gRPC nào.

Data Sharing — Chia sẻ dữ liệu

The core concept of cloud data sharing is that a **multitenant** system supports security policies for sharing data among tenants. Concretely, any public cloud object storage system with a fine-grained permission system can be a platform for data sharing. Popular cloud data-warehouse platforms also support data-sharing capabilities. Of course, data can also be shared through download or exchange over email, but a multitenant system makes the process much easier. Many modern sharing platforms (especially cloud data warehouses) support row, column, and sensitive data filtering. Data sharing also streamlines the notion of the data marketplace, available on several popular clouds and data platforms. Data marketplaces provide a centralized location for data commerce, where data providers can advertise their offerings and sell them without worrying about the details of managing network access to data systems. Data sharing can also streamline data pipelines within an organization. Data sharing allows units of an organization to manage their data and selectively share it with other units while still allowing individual units to manage their compute and query costs separately, facilitating data decentralization. This facilitates decentralized data management patterns such as **data mesh**. 6

Khái niệm cốt lõi của việc chia sẻ dữ liệu trên cloud là một hệ thống đa thuê bao (multitenant) hỗ trợ các chính sách bảo mật để chia sẻ dữ liệu giữa các thuê bao. Cụ thể, bất kỳ hệ thống lưu trữ đối tượng public cloud nào có một hệ thống phân quyền chi tiết đều có thể là một nền tảng cho việc chia sẻ dữ liệu. Các nền tảng kho dữ liệu trên cloud phổ biến cũng hỗ trợ các năng lực chia sẻ dữ liệu. Tất nhiên, dữ liệu cũng có thể được chia sẻ qua tải xuống hoặc trao đổi qua email, nhưng một hệ thống đa thuê bao khiến quá trình này dễ dàng hơn nhiều. Nhiều nền tảng chia sẻ hiện đại (đặc biệt là các kho dữ liệu trên cloud) hỗ trợ lọc hàng, cột, và dữ liệu nhạy cảm. Việc chia sẻ dữ liệu cũng làm tinh gọn khái niệm *thị trường dữ liệu (data marketplace)*, sẵn có trên một số cloud và nền tảng dữ liệu phổ biến. Các thị trường dữ liệu cung cấp một vị trí tập trung cho thương mại dữ liệu, nơi các nhà cung cấp dữ liệu có thể quảng bá các sản phẩm chào bán của họ và bán chúng mà không phải lo lắng về các chi tiết của việc quản lý quyền truy cập mạng đến các hệ thống dữ liệu. Việc chia sẻ dữ liệu cũng có thể làm tinh gọn các đường ống dữ liệu trong một tổ chức. Việc chia sẻ dữ liệu cho phép các đơn vị của một tổ chức quản lý dữ liệu của họ và chia sẻ nó một cách có chọn lọc với các đơn vị khác, trong khi vẫn cho phép từng đơn vị quản lý riêng các chi phí tính toán và truy vấn của mình, tạo thuận lợi cho việc phi tập trung hóa dữ liệu. Điều này tạo thuận lợi cho các mẫu hình quản lý dữ liệu phi tập trung như data mesh. 6

Data sharing and data mesh align closely with our philosophy of common architecture components. Choose common components (Chapter 3) that allow the simple and efficient interchange of data and expertise rather than embracing the most exciting and sophisticated technology.

Chia sẻ dữ liệu và data mesh gắn chặt với triết lý của chúng tôi về các thành phần kiến trúc chung. Hãy chọn các thành phần chung (Chương 3) cho phép trao đổi dữ liệu và chuyên môn một cách đơn giản và hiệu quả, thay vì ôm trọn công nghệ thú vị và tinh vi nhất.

Third-Party Data Sources — Các nguồn dữ liệu bên thứ ba

The consumerization of technology means every company is essentially now a technology company. The consequence is that these companies—and increasingly government agencies—want to make their data available to their customers and users, either as part of their service or as a separate subscription. For example, the United States Bureau of Labor Statistics publishes various statistics about the US labor market. The National Aeronautics and Space Administration (NASA) publishes various data from its research initiatives. Facebook shares data with businesses that advertise on its platform. Why would companies want to make their data available? Data is sticky, and a flywheel is created by allowing users to integrate and extend their application into a user's application. Greater user adoption and usage means more data, which means users can integrate more data into their applications and data systems. The side effect is there are now almost infinite sources of third-party data. Direct third-party data access is commonly done via APIs, through data sharing on a cloud platform, or through data download. APIs often provide deep integration capabilities, allowing customers to pull and push data. For example, many CRMs offer APIs that their users can integrate into their systems and applications. We see a common workflow to get data from a CRM, blend the CRM data through the customer scoring model, and then use reverse ETL to send that data back into CRM for salespeople to contact better-qualified leads.

Sự “tiêu dùng hóa” của công nghệ nghĩa là về cơ bản mọi công ty giờ đây đều là một công ty công nghệ. Hệ quả là các công ty này—và ngày càng nhiều các cơ quan chính phủ—muốn làm cho dữ liệu của họ sẵn có cho khách hàng và người dùng của họ, hoặc như một phần của dịch vụ của họ, hoặc như một đăng ký riêng. Ví dụ, Cục Thống kê Lao động Hoa Kỳ công bố nhiều thống kê khác nhau về thị trường lao động Mỹ. Cơ quan Hàng không và Vũ trụ Quốc gia (NASA) công bố nhiều dữ liệu khác nhau từ các sáng kiến nghiên cứu của mình. Facebook chia sẻ dữ liệu với các doanh nghiệp quảng cáo trên nền tảng của mình. Tại sao các công ty lại muốn làm cho dữ liệu của họ sẵn có? Dữ liệu có tính “dính”, và một bánh đà (flywheel) được tạo ra bằng cách cho phép người dùng tích hợp và mở rộng ứng dụng của họ vào ứng dụng của một người dùng. Mức độ chấp nhận và sử dụng của người dùng lớn hơn nghĩa là nhiều dữ liệu hơn, nghĩa là người dùng có thể tích hợp nhiều dữ liệu hơn vào các ứng dụng và hệ thống dữ liệu của họ. Tác dụng phụ là giờ đây có gần như vô hạn các nguồn dữ liệu bên thứ ba. Việc truy cập dữ liệu bên thứ ba trực tiếp thường được thực hiện qua các API, thông qua chia sẻ dữ liệu trên một nền tảng cloud, hoặc thông qua tải dữ liệu xuống. Các API thường cung cấp các năng lực tích hợp sâu, cho phép khách hàng lấy và đẩy dữ liệu. Ví dụ, nhiều CRM cung cấp các API mà người dùng của họ có thể tích hợp vào các hệ thống và ứng dụng của họ. Chúng ta thấy một luồng công việc phổ biến là lấy dữ liệu từ một CRM, trộn dữ liệu CRM qua mô hình chấm điểm khách hàng, rồi dùng reverse ETL để gửi dữ liệu đó trở lại vào CRM để nhân viên kinh doanh liên hệ với các khách hàng tiềm năng đủ điều kiện hơn.

Message Queues and Event-Streaming Platforms — Các hàng đợi tin nhắn và nền tảng luồng sự kiện

Event-driven architectures are pervasive in software applications and are poised to grow their popularity even further. First, message queues and event-streaming platforms—critical layers in event-driven architectures— are easier to set up and manage in a cloud environment. Second, the rise of data apps—applications that directly integrate real-time analytics—are growing from strength to strength. Event-driven architectures are ideal in this setting because events can both trigger work in the application and feed near real-time analytics.

Các kiến trúc hướng sự kiện phổ biến trong các ứng dụng phần mềm và sẵn sàng để phát triển độ phổ biến của chúng còn xa hơn nữa. Thứ nhất, các hàng đợi tin nhắn và nền tảng luồng sự kiện—các tầng then chốt trong các kiến trúc hướng sự kiện—để thiết lập và quản lý hơn trong một môi trường cloud. Thứ hai, sự nổi lên của các ứng dụng dữ liệu—các ứng dụng tích hợp trực tiếp phân tích thời gian thực—đang phát triển ngày càng mạnh mẽ. Các kiến trúc hướng sự kiện lý tưởng trong bối cảnh này vì các sự kiện vừa có thể kích hoạt công việc trong ứng dụng vừa cung cấp dữ liệu cho phân tích gần như thời gian thực.

Please note that streaming data (in this case, messages and streams) cuts across many data engineering lifecycle stages. Unlike an RDBMS, which is often directly attached to an application, the lines of streaming data are sometimes less clear-cut. These systems are used as source systems, but they will often cut across the data engineering lifecycle because of their transient nature. For example, you can use an event-streaming platform for message passing in an event-driven application, a source system. The same event-streaming platform can be used in the ingestion and transformation stage to process data for real-time analytics. As source systems, message queues and event-streaming platforms are used in numerous ways, from routing messages between microservices ingesting millions of events per second of event data from web, mobile, and IoT applications. Let's look at message queues and event-streaming platforms a bit more closely.

Xin lưu ý rằng dữ liệu luồng (trong trường hợp này là tin nhắn và luồng) cắt ngang nhiều giai đoạn của vòng đời kỹ thuật dữ liệu. Không như một RDBMS, vốn thường được gắn trực tiếp vào một ứng dụng, ranh giới của dữ liệu luồng đôi khi kém rạch ròi hơn. Các hệ thống này được dùng như các hệ thống nguồn, nhưng chúng sẽ thường cắt ngang vòng đời kỹ thuật dữ liệu vì bản chất nhất thời của chúng. Ví dụ, bạn có thể dùng một nền tảng luồng sự kiện cho việc truyền tin nhắn trong một ứng dụng hướng sự kiện, một hệ thống nguồn. Cùng nền tảng luồng sự kiện đó có thể được dùng trong giai đoạn thu nạp và biến đổi để xử lý dữ liệu cho phân tích thời gian thực. Với vai trò là các hệ thống nguồn, các hàng đợi tin nhắn và nền tảng luồng sự kiện được dùng theo vô số cách, từ định tuyến tin nhắn giữa các microservices đến thu nạp hàng triệu sự kiện mỗi giây dữ liệu sự kiện từ các ứng dụng web, di động và IoT. Hãy cùng xem xét các hàng đợi tin nhắn và nền tảng luồng sự kiện kỹ hơn một chút.

Message queues — Hàng đợi tin nhắn

A message queue is a mechanism to asynchronously send data (usually as small individual messages, in the kilobytes) between discrete systems using a **publish** and **subscribe** model. Data is published to a message queue and is delivered to one or more subscribers (Figure 5-9). The subscriber acknowledges receipt of the message, removing it from the queue.

Một hàng đợi tin nhắn là một cơ chế để gửi dữ liệu một cách bất đồng bộ (thường là các tin nhắn nhỏ, riêng lẻ, ở mức kilobyte) giữa các hệ thống rời rạc bằng một mô hình xuất bản

và đăng ký (publish and subscribe). Dữ liệu được xuất bản đến một hàng đợi tin nhắn và được gửi đến một hay nhiều bên đăng ký (subscriber) (Hình 5-9). Bên đăng ký xác nhận việc nhận tin nhắn, qua đó loại nó khỏi hàng đợi.



Figure 5-9. A simple message queue Message queues allow applications and systems to be decoupled from each other and are widely used in microservices architectures. The message queue buffers messages to handle transient load spikes and makes messages durable through a distributed architecture with replication.

Hình 5-9. Một hàng đợi tin nhắn đơn giản. Các hàng đợi tin nhắn cho phép các ứng dụng và hệ thống được tách rời (decoupled) khỏi nhau và được dùng rộng rãi trong các kiến trúc microservices. Hàng đợi tin nhắn đệm các tin nhắn để xử lý các đỉnh tải nhất thời và làm cho các tin nhắn bền vững thông qua một kiến trúc phân tán với sao chép (replication).

Message queues are a critical ingredient for decoupled microservices and event-driven architectures. Some things to keep in mind with message queues are frequency of delivery, message ordering, and scalability.

Các hàng đợi tin nhắn là một thành phần then chốt cho các microservices được tách rời và các kiến trúc hướng sự kiện. Một số điều cần ghi nhớ với các hàng đợi tin nhắn là tần suất gửi, thứ tự tin nhắn, và khả năng mở rộng.

Message ordering and delivery The order in which messages are created, sent, and received can significantly impact downstream subscribers. In general, order in distributed messaging queues is a tricky problem. Message queues often apply a fuzzy notion of order and first in, first out (FIFO). Strict FIFO means that if message A is ingested before message B, message A will always be delivered before message B. In practice, messages might be published and received out of order, especially in highly distributed message systems. For example, Amazon SQS standard queues make the best effort to preserve message order. SQS also offers FIFO queues, which offer much stronger guarantees at the cost of extra overhead. In general, don't assume that your messages will be delivered in order unless your message queue technology guarantees it. You typically need to design for out-of-order message delivery.

Thứ tự và việc gửi tin nhắn Thứ tự mà các tin nhắn được tạo, gửi và nhận có thể tác động đáng kể đến các bên đăng ký ở hạ nguồn. Nhìn chung, thứ tự trong các hàng đợi nhắn tin phân tán là một vấn đề hóc búa. Các hàng đợi tin nhắn thường áp dụng một khái niệm mơ hồ về thứ tự và “vào trước, ra trước” (FIFO). FIFO nghiêm ngặt nghĩa là nếu tin nhắn A được thu nạp trước tin nhắn B, tin nhắn A sẽ luôn được gửi trước tin nhắn B. Trên thực tế, các tin nhắn có thể được xuất bản và nhận không theo thứ tự, đặc biệt trong các hệ thống nhắn tin phân tán cao. Ví dụ, các hàng đợi tiêu chuẩn của Amazon SQS nỗ lực hết sức để bảo toàn thứ tự tin nhắn. SQS cũng cung cấp các hàng đợi FIFO, vốn cung cấp các đảm bảo mạnh hơn nhiều với cái giá là thêm chi phí phụ. Nhìn chung, đừng giả định rằng các tin nhắn của bạn sẽ được gửi theo thứ tự trừ khi công nghệ hàng đợi tin nhắn của bạn đảm bảo điều đó. Bạn thường cần thiết kế cho việc gửi tin nhắn không theo thứ tự.

Delivery frequency Messages can be sent exactly once or at least once. If a message is sent exactly

once, then after the subscriber acknowledges the message, the message disappears and won't be delivered again. Messages sent at least once can be consumed by multiple subscribers or by the same subscriber more than once. This is great when duplications or redundancy don't matter. Ideally, systems should be **idempotent**. In an idempotent system, the outcome of processing a message once is identical to the outcome of processing it multiple times. This helps to account for a variety of subtle scenarios. For example, even if our system can guarantee exactly-once delivery, a consumer might fully process a message but fail right before acknowledging processing. The message will effectively be processed twice, but an idempotent system handles this scenario gracefully.

Tần suất gửi Các tin nhắn có thể được gửi đúng một lần (exactly once) hoặc ít nhất một lần (at least once). Nếu một tin nhắn được gửi *đúng một lần*, thì sau khi bên đăng ký xác nhận tin nhắn, tin nhắn biến mất và sẽ không được gửi lại. Các tin nhắn được gửi *ít nhất một lần* có thể được tiêu thụ bởi nhiều bên đăng ký hoặc bởi cùng một bên đăng ký nhiều lần. Điều này tuyệt vời khi sự trùng lặp hay dư thừa không quan trọng. Lý tưởng nhất là các hệ thống nên *bất biến lặp* (idempotent). Trong một hệ thống bất biến lặp, kết quả của việc xử lý một tin nhắn một lần là giống hệt với kết quả của việc xử lý nó nhiều lần. Điều này giúp tính đến nhiều tình huống tình tế khác nhau. Ví dụ, ngay cả khi hệ thống của chúng ta có thể đảm bảo việc gửi đúng một lần, một bên tiêu thụ có thể xử lý hoàn toàn một tin nhắn nhưng thất bại ngay trước khi xác nhận việc xử lý. Tin nhắn trên thực tế sẽ được xử lý hai lần, nhưng một hệ thống bất biến lặp xử lý tình huống này một cách uyển chuyển.

Scalability The most popular message queues utilized in event-driven applications are horizontally scalable, running across multiple servers. This allows these queues to scale up and down dynamically, buffer messages when systems fall behind, and durably store messages for resilience against failure. However, this can create a variety of complications, as mentioned previously (multiple deliveries and fuzzy ordering).

Khả năng mở rộng Các hàng đợi tin nhắn phổ biến nhất được sử dụng trong các ứng dụng hướng sự kiện có khả năng mở rộng ngang, chạy trên nhiều máy chủ. Điều này cho phép các hàng đợi này mở rộng lên và xuống một cách động, đệm các tin nhắn khi các hệ thống tụt lại phía sau, và lưu trữ các tin nhắn một cách bền vững để có khả năng chống chịu trước thất bại. Tuy nhiên, điều này có thể tạo ra nhiều phức tạp khác nhau, như đã đề cập trước đó (gửi nhiều lần và thứ tự mơ hồ).

Event-streaming platforms — Nền tảng luồng sự kiện

In some ways, an event-streaming platform is a continuation of a message queue in that messages are passed from producers to consumers. As discussed previously in this chapter, the big difference between messages and streams is that a message queue is primarily used to route messages with certain delivery guarantees. In contrast, an event-streaming platform is used to ingest and process data in an ordered log of records. In an event-streaming platform, data is retained for a while, and it is possible to replay messages from a past point in time. Let's describe an event related to an event-streaming platform. As mentioned in Chapter 3, an event is "something that happened, typically a change in the state of something." An event has the following features: a key, a value, and a timestamp. Multiple key-value timestamps might be contained in a single event. For example, an event for an ecommerce order might look like this:

Theo một số cách, một nền tảng luồng sự kiện là một sự nối tiếp của một hàng đợi tin nhắn ở chỗ các tin nhắn được truyền từ các bên sản xuất (producer) đến các bên tiêu thụ. Như đã thảo luận trước đó trong chương này, khác biệt lớn giữa tin nhắn và luồng là một hàng đợi tin nhắn chủ yếu được dùng để định tuyến các tin nhắn với những đảm bảo gửi nhất

định. Ngược lại, một nền tảng luồng sự kiện được dùng để thu nạp và xử lý dữ liệu trong một log có thứ tự gồm các bản ghi. Trong một nền tảng luồng sự kiện, dữ liệu được giữ lại trong một khoảng thời gian, và có thể phát lại các tin nhắn từ một thời điểm trong quá khứ. Hãy cùng mô tả một sự kiện liên quan đến một nền tảng luồng sự kiện. Như đã đề cập trong Chương 3, một sự kiện là “một điều gì đó đã xảy ra, thường là một sự thay đổi trong trạng thái của một thứ gì đó.” Một sự kiện có các đặc điểm sau: một khóa, một giá trị, và một dấu thời gian. Nhiều bộ ba khóa-giá-trị-dấu-thời-gian có thể được chứa trong một sự kiện đơn lẻ. Ví dụ, một sự kiện cho một đơn hàng thương mại điện tử có thể trông như thế này:

```
Key: "Order # 12345"  
Value: "SKU 123, purchase price of $100"  
Timestamp: "2023-01-02 06:01:00"
```

Let's look at some of the critical characteristics of an event-streaming platform that you should be aware of as a data engineer.

Hãy cùng xem một số đặc tính then chốt của một nền tảng luồng sự kiện mà bạn nên nắm được với vai trò kỹ sư dữ liệu.

Topics In an event-streaming platform, a producer streams events to a topic, a collection of related events. A topic might contain fraud alerts, customer orders, or temperature readings from IoT devices, for example. A topic can have zero, one, or multiple producers and customers on most event-streaming platforms. Using the preceding event example, a topic might be web orders . Also, let's send this topic to a couple of consumers, such as fulfillment and marketing . This is an excellent example of blurred lines between analytics and an event-driven system. The fulfillment subscriber will use events to trigger a fulfillment process, while marketing runs real-time analytics to tune marketing campaigns (Figure 5-10).

Topic Trong một nền tảng luồng sự kiện, một bên sản xuất phát luồng các sự kiện đến một *topic*, một tập hợp các sự kiện liên quan. Một topic có thể chứa các cảnh báo gian lận, các đơn hàng của khách hàng, hay các số đo nhiệt độ từ các thiết bị IoT, chẳng hạn vậy. Một topic có thể có không, một, hay nhiều bên sản xuất và khách hàng trên hầu hết các nền tảng luồng sự kiện. Dùng ví dụ sự kiện trước đó, một topic có thể là *web orders*. Ngoài ra, hãy gửi topic này đến một vài bên tiêu thụ, chẳng hạn như *fulfillment* và *marketing*. Đây là một ví dụ tuyệt vời về ranh giới bị làm mờ giữa phân tích và một hệ thống hướng sự kiện. Bên đăng ký fulfillment sẽ dùng các sự kiện để kích hoạt một tiến trình hoàn tất đơn hàng, trong khi marketing chạy phân tích thời gian thực để tinh chỉnh các chiến dịch tiếp thị (Hình 5-10).

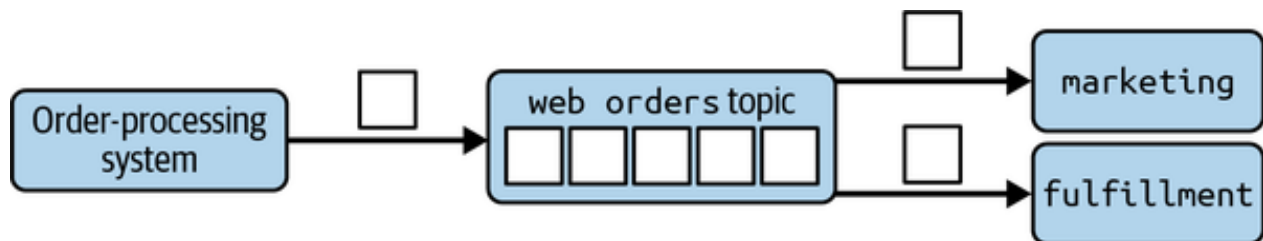


Figure 5-10. An order-processing system generates events (small rectangles) and publishes them to the web orders topic. Two subscribers— marketing and fulfillment —pull events from the topic. Stream partitions Stream partitions are subdivisions of a stream into multiple streams. A good analogy is a multilane freeway. Having multiple lanes allows for parallelism and higher **throughput**. Messages are distributed across partitions by **partition key** . Messages with the same partition key will always end up in the same partition. In Figure 5-11 , for example, each message

has a numeric ID— shown inside the circle representing the message—that we use as a partition key. To determine the partition, we divide by 3 and take the remainder. Going from bottom to top, the partitions have remainder 0, 1, and 2, respectively.

Hình 5-10. Một hệ thống xử lý đơn hàng sinh ra các sự kiện (các hình chữ nhật nhỏ) và xuất bản chúng đến topic *web orders*. Hai bên đăng ký—marketing và fulfillment—lấy các sự kiện từ topic. Phân vùng luồng Các phân vùng luồng (stream partition) là các phần chia nhỏ của một luồng thành nhiều luồng. Một ví von hay là một xa lộ nhiều làn. Việc có nhiều làn cho phép song song hóa và thông lượng (throughput) cao hơn. Các tin nhắn được phân bổ qua các phân vùng theo *khóa phân vùng* (partition key). Các tin nhắn có cùng khóa phân vùng sẽ luôn rơi vào cùng một phân vùng. Trong Hình 5-11, chẳng hạn, mỗi tin nhắn có một ID số—được hiển thị bên trong vòng tròn đại diện cho tin nhắn—mà chúng ta dùng làm khóa phân vùng. Để xác định phân vùng, chúng ta chia cho 3 và lấy phần dư. Đi từ dưới lên trên, các phân vùng có phần dư lần lượt là 0, 1, và 2.

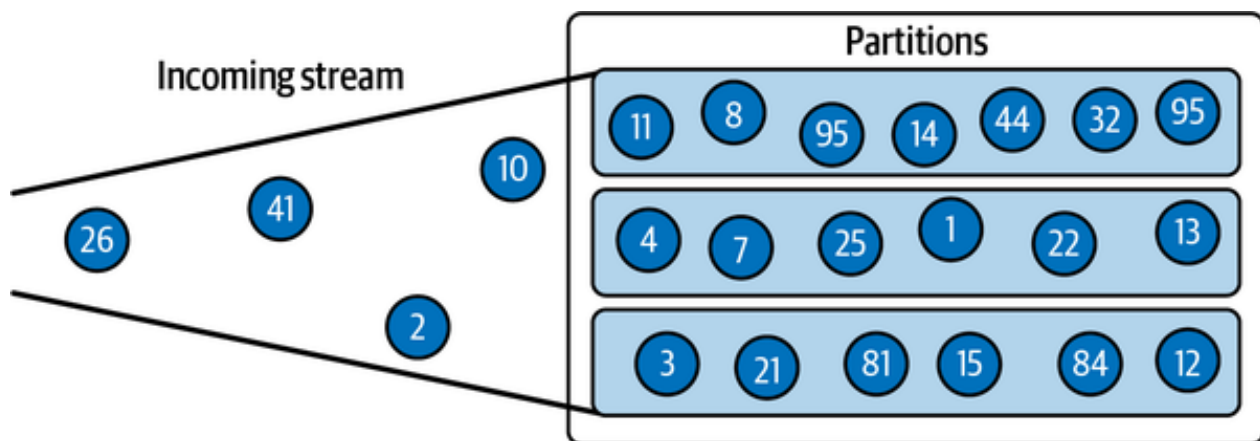


Figure 5-11. An incoming message stream broken into three partitions Set a partition key so that messages that should be processed together have the same partition key. For example, it is common in IoT settings to want to send all messages from a particular device to the same processing server. We can achieve this by using a device ID as the partition key, and then setting up one server to consume from each partition. A key concern with stream **partitioning** is ensuring that your partition key does not generate hotspotting—a disproportionate number of messages delivered to one partition. For example, if each IoT device were known to be located in a particular US state, we might use the state as the partition key. Given a device distribution proportional to state population, the partitions containing California, Texas, Florida, and New York might be overwhelmed, with other partitions relatively underutilized. Ensure that your partition key will distribute messages evenly across partitions.

Hình 5-11. Một luồng tin nhắn đến được chia thành ba phân vùng. Hãy đặt một khóa phân vùng sao cho các tin nhắn cần được xử lý cùng nhau có cùng khóa phân vùng. Ví dụ, trong các bối cảnh IoT, người ta thường muốn gửi tất cả các tin nhắn từ một thiết bị cụ thể đến cùng một máy chủ xử lý. Chúng ta có thể đạt được điều này bằng cách dùng một ID thiết bị làm khóa phân vùng, rồi thiết lập một máy chủ để tiêu thụ từ mỗi phân vùng. Một mối lo then chốt với việc phân vùng luồng là đảm bảo rằng khóa phân vùng của bạn không sinh ra *điểm nóng* (hotspotting)—một số lượng tin nhắn mất cân xứng được gửi đến một phân vùng. Ví dụ, nếu mỗi thiết bị IoT được biết là nằm tại một bang cụ thể của Mỹ, chúng ta có thể dùng bang làm khóa phân vùng. Với một phân bố thiết bị tỉ lệ với dân số bang, các phân vùng chứa California, Texas, Florida, và New York có thể bị quá tải, trong khi các phân vùng khác tương đối ít được sử dụng. Hãy đảm bảo rằng khóa phân vùng của bạn sẽ

phân bố các tin nhắn đều khắp các phân vùng.

Fault tolerance and resilience Event-streaming platforms are typically distributed systems, with streams stored on various nodes. If a node goes down, another node replaces it, and the stream is still accessible. This means records aren't lost; you may choose to delete records, but that's another story. This fault tolerance and resilience make streaming platforms a good choice when you need a system that can reliably produce, store, and ingest event data.

Khả năng chịu lỗi và khả năng phục hồi Các nền tảng luồng sự kiện thường là các hệ phân tán, với các luồng được lưu trên nhiều nút khác nhau. Nếu một nút sập, một nút khác thay thế nó, và luồng vẫn truy cập được. Điều này nghĩa là các bản ghi không bị mất; bạn có thể chọn xóa các bản ghi, nhưng đó là một câu chuyện khác. Khả năng chịu lỗi và khả năng phục hồi này khiến các nền tảng luồng trở thành một lựa chọn tốt khi bạn cần một hệ thống có thể sản xuất, lưu trữ, và thu nạp dữ liệu sự kiện một cách đáng tin cậy.

Whom You'll Work With — Bạn sẽ làm việc với ai

When accessing source systems, it's essential to understand the people with whom you'll work. In our experience, good diplomacy and relationships with the stakeholders of source systems are an underrated and crucial part of successful data engineering. Who are these stakeholders? Typically, you'll deal with two categories of stakeholders: systems and data stakeholders (Figure 5-12). A systems stakeholder builds and maintains the source systems; these might be software engineers, application developers, and third parties. Data stakeholders own and control access to the data you want, generally handled by IT, a **data governance** group, or third parties. The systems and data stakeholders are often different people or teams; sometimes, they are the same.

Khi truy cập các hệ thống nguồn, điều cốt yếu là hiểu những người mà bạn sẽ làm việc cùng. Theo kinh nghiệm của chúng tôi, ngoại giao tốt và các mối quan hệ tốt với các bên liên quan (stakeholder) của các hệ thống nguồn là một phần bị đánh giá thấp và mang tính quyết định của kỹ thuật dữ liệu thành công. Những bên liên quan này là ai? Thông thường, bạn sẽ đối mặt với hai phạm trù bên liên quan: bên liên quan về hệ thống và bên liên quan về dữ liệu (Hình 5-12). Một bên liên quan về hệ thống xây dựng và bảo trì các hệ thống nguồn; những người này có thể là các kỹ sư phần mềm, các nhà phát triển ứng dụng, và các bên thứ ba. Các bên liên quan về dữ liệu sở hữu và kiểm soát quyền truy cập đến dữ liệu mà bạn muốn, nhìn chung được xử lý bởi bộ phận IT, một nhóm quản trị dữ liệu (data governance), hay các bên thứ ba. Các bên liên quan về hệ thống và về dữ liệu thường là những người hay đội khác nhau; đôi khi, họ là một.

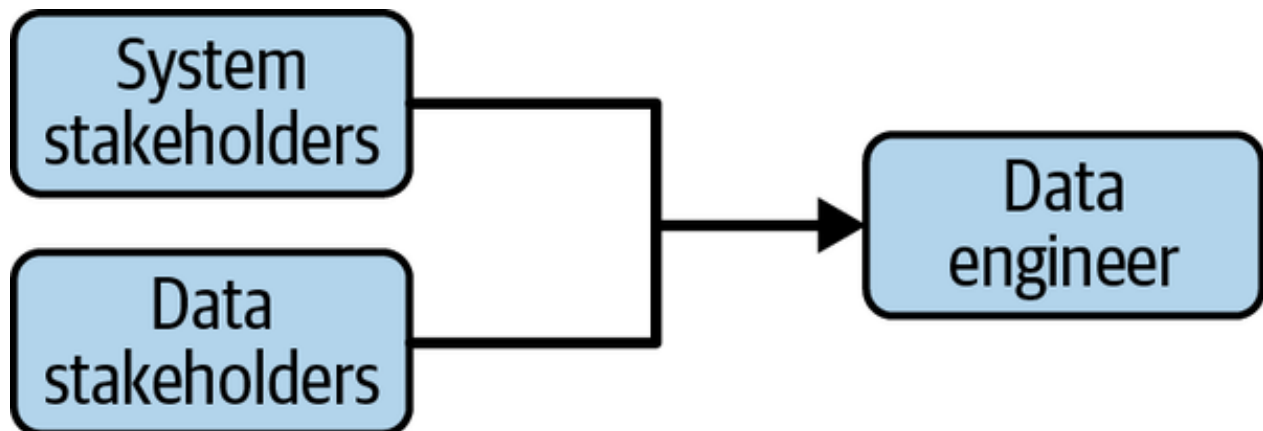


Figure 5-12. The data engineer’s **upstream** stakeholders You’re often at the mercy of the stakeholder’s ability to follow correct software engineering, database management, and development practices. Ideally, the stakeholders are doing DevOps and working in an agile manner. We suggest creating a feedback loop between data engineers and stakeholders of the source systems to create awareness of how data is consumed and used. This is among the single most overlooked areas where data engineers can get a lot of value. When something happens to the upstream source data—and something will happen, whether it’s a schema or data change, a failed server or database, or other important events—you want to make sure that you’re made aware of the impact these issues will have on your data engineering systems.

Hình 5-12. Các bên liên quan thượng nguồn của kỹ sư dữ liệu. Bạn thường phụ thuộc vào khả năng của bên liên quan trong việc tuân theo các thực hành đúng đắn về kỹ thuật phần mềm, quản lý database, và phát triển. Lý tưởng nhất là các bên liên quan đang làm DevOps và làm việc theo cách linh hoạt (agile). Chúng tôi gợi ý tạo một vòng phản hồi giữa các kỹ sư dữ liệu và các bên liên quan của các hệ thống nguồn để tạo nhận thức về cách dữ liệu được tiêu thụ và sử dụng. Đây là một trong những lĩnh vực bị bỏ qua nhiều nhất, nơi các kỹ sư dữ liệu có thể nhận được rất nhiều giá trị. Khi có điều gì đó xảy ra với dữ liệu nguồn thượng nguồn—và điều gì đó sẽ xảy ra, dù là một thay đổi về lược đồ hay dữ liệu, một máy chủ hay database thất bại, hay các sự kiện quan trọng khác—bạn muốn chắc chắn rằng mình được thông báo về tác động mà các vấn đề này sẽ gây ra cho các hệ thống kỹ thuật dữ liệu của bạn.

It might help to have a data contract in place with your upstream source system owners. What is a data contract? James Denmore offers this definition:

Có thể sẽ hữu ích nếu có một *hợp đồng dữ liệu (data contract)* với các chủ sở hữu hệ thống nguồn thượng nguồn của bạn. Hợp đồng dữ liệu là gì? James Denmore đưa ra định nghĩa này:

A data contract is a written agreement between the owner of a source system and the team ingesting data from that system for use in a **data pipeline**. The contract should state what data is being extracted, via what method (full, incremental), how often, as well as who (person, team) are the contacts for both the source system and the ingestion. Data contracts should be stored in a well-known and easy-to-find location such as a GitHub repo or internal documentation site. If possible, format data contracts in a standardized form so they can be integrated into the development process or queried programmatically. 7

Một hợp đồng dữ liệu là một thỏa thuận bằng văn bản giữa chủ sở hữu của một hệ thống nguồn và đội thu nạp dữ liệu từ hệ thống đó để dùng trong một đường ống dữ liệu. Hợp đồng nên nêu rõ dữ liệu nào đang được trích xuất, qua phương pháp nào (đầy đủ, tăng dần), tần suất ra sao, cũng như ai (người, đội) là các đầu mối liên hệ cho cả hệ thống nguồn lẫn việc thu nạp. Các hợp đồng dữ liệu nên được lưu ở một vị trí nổi tiếng và dễ tìm như một repo GitHub hay một trang tài liệu nội bộ. Nếu có thể, hãy định dạng các hợp đồng dữ liệu theo một dạng chuẩn hóa để chúng có thể được tích hợp vào quá trình phát triển hoặc được truy vấn một cách có lập trình. 7

In addition, consider establishing an SLA with upstream providers. An SLA provides expectations of what you can expect from the source systems you rely upon. An example of an SLA might be “data from source systems will be reliably available and of high quality.” A service-level objective (SLO) measures performance against what you’ve agreed to in the SLA. For example, given your example SLA, an SLO might be “source systems will have 99% uptime.” If a data contract or SLA/SLO seems too formal, at least verbally set expectations for source system guarantees for uptime, **data quality**,

and anything else of importance to you. Upstream owners of source systems need to understand your requirements so they can provide you with the data you need.

Ngoài ra, hãy cân nhắc thiết lập một SLA với các nhà cung cấp thượng nguồn. Một SLA cung cấp các kỳ vọng về những gì bạn có thể trông đợi từ các hệ thống nguồn mà bạn dựa vào. Một ví dụ về một SLA có thể là “dữ liệu từ các hệ thống nguồn sẽ sẵn có một cách đáng tin cậy và có chất lượng cao.” Một mục tiêu cấp độ dịch vụ (SLO) đo lường hiệu suất so với những gì bạn đã đồng ý trong SLA. Ví dụ, với SLA ví dụ của bạn, một SLO có thể là “các hệ thống nguồn sẽ có thời gian hoạt động 99%.” Nếu một hợp đồng dữ liệu hay SLA/SLO có vẻ quá trang trọng, ít nhất hãy thiết lập kỳ vọng bằng lời nói cho các đảm bảo của hệ thống nguồn về thời gian hoạt động, chất lượng dữ liệu, và bất cứ điều gì khác quan trọng với bạn. Các chủ sở hữu thượng nguồn của các hệ thống nguồn cần hiểu các yêu cầu của bạn để họ có thể cung cấp cho bạn dữ liệu bạn cần.

Undercurrents and Their Impact on Source Systems — Các dòng chảy ngầm và tác động của chúng lên các hệ thống nguồn

Unlike other parts of the data engineering lifecycle, source systems are generally outside the control of the data engineer. There’s an implicit assumption (some might call it hope) that the stakeholders and owners of the source systems—and the data they produce—are following best practices concerning data management, **DataOps** (and DevOps), DODD (mentioned in Chapter 2) **data architecture, orchestration**, and software engineering. The data engineer should get as much upstream support as possible to ensure that the undercurrents are applied when data is generated in source systems. Doing so will make the rest of the steps in the data engineering lifecycle proceed a lot more smoothly. How do the undercurrents impact source systems? Let’s have a look.

Không như các phần khác của vòng đời kỹ thuật dữ liệu, các hệ thống nguồn nhìn chung nằm ngoài tầm kiểm soát của kỹ sư dữ liệu. Có một giả định ngầm (một số người có thể gọi là *hy vọng*) rằng các bên liên quan và chủ sở hữu của các hệ thống nguồn—và dữ liệu mà chúng sản xuất—đang tuân theo các thực hành tốt nhất liên quan đến quản lý dữ liệu (data management), DataOps (và DevOps), DODD (được đề cập trong Chương 2), kiến trúc dữ liệu, điều phối (orchestration), và kỹ thuật phần mềm. Kỹ sư dữ liệu nên nhận được càng nhiều hỗ trợ từ thượng nguồn càng tốt để đảm bảo rằng các dòng chảy ngầm được áp dụng khi dữ liệu được sinh ra trong các hệ thống nguồn. Làm như vậy sẽ khiến các bước còn lại trong vòng đời kỹ thuật dữ liệu diễn ra trơn tru hơn rất nhiều. Các dòng chảy ngầm tác động đến các hệ thống nguồn như thế nào? Hãy cùng xem.

Security — An ninh

Security is critical, and the last thing you want is to accidentally create a point of vulnerability in a source system. Here are some areas to consider:

An ninh là then chốt, và điều cuối cùng bạn muốn là vô tình tạo ra một điểm dễ tổn thương trong một hệ thống nguồn. Dưới đây là một số lĩnh vực cần cân nhắc:

Is the source system architected so data is secure and encrypted, both with data at rest and while data is transmitted?

Hệ thống nguồn có được kiến trúc sao cho dữ liệu được an toàn và mã hóa (encryption), cả khi dữ liệu ở trạng thái nghỉ lẫn khi dữ liệu đang được truyền không?

Do you have to access the source system over the public internet, or are you using a virtual private network (VPN)?

Bạn có phải truy cập hệ thống nguồn qua internet công cộng không, hay bạn đang dùng một mạng riêng ảo (VPN)?

Keep passwords, tokens, and credentials to the source system securely locked away. For example, if you're using Secure Shell (SSH) keys, use a key manager to protect your keys; the same rule applies to passwords—use a password manager or a single sign-on (SSO) provider.

Hãy giữ mật khẩu, token, và thông tin xác thực đến hệ thống nguồn được khóa an toàn. Ví dụ, nếu bạn đang dùng các khóa Secure Shell (SSH), hãy dùng một trình quản lý khóa để bảo vệ các khóa của bạn; quy tắc tương tự áp dụng cho mật khẩu—hãy dùng một trình quản lý mật khẩu hoặc một nhà cung cấp đăng nhập một lần (SSO).

Do you trust the source system? Always be sure to trust but verify that the source system is legitimate. You don't want to be on the receiving end of data from a malicious actor.

Bạn có tin tưởng hệ thống nguồn không? Hãy luôn chắc chắn tin nhưng vẫn kiểm chứng rằng hệ thống nguồn là hợp pháp. Bạn không muốn ở đầu nhận dữ liệu từ một tác nhân độc hại.

Data Management — Quản lý dữ liệu

Data management of source systems is challenging for data engineers. In most cases, you will have only peripheral control—if any control at all—over source systems and the data they produce. To the extent possible, you should understand the way data is managed in source systems since this will directly influence how you ingest, store, and transform the data. Here are some areas to consider:

Quản lý dữ liệu của các hệ thống nguồn là một thách thức đối với các kỹ sư dữ liệu. Trong hầu hết các trường hợp, bạn sẽ chỉ có quyền kiểm soát ngoại vi—if có chút kiểm soát nào—đối với các hệ thống nguồn và dữ liệu mà chúng sản xuất. Trong phạm vi có thể, bạn nên hiểu cách dữ liệu được quản lý trong các hệ thống nguồn, vì điều này sẽ ảnh hưởng trực tiếp đến cách bạn thu nạp, lưu trữ, và biến đổi dữ liệu. Dưới đây là một số lĩnh vực cần cân nhắc:

Data governance

Quản trị dữ liệu

Are upstream data and systems governed in a reliable, easy-to-understand fashion? Who manages the data?

Dữ liệu và hệ thống thượng nguồn có được quản trị theo một cách đáng tin cậy, dễ hiểu không? Ai quản lý dữ liệu?

Data quality

Chất lượng dữ liệu

How do you ensure data quality and integrity in upstream systems? Work with source system teams to set expectations on data and communication.

Bạn đảm bảo chất lượng và toàn vẹn dữ liệu trong các hệ thống thượng nguồn như thế nào? Hãy làm việc với các đội hệ thống nguồn để thiết lập các kỳ vọng về dữ liệu và giao tiếp.

Schema

Lược đồ

Expect that upstream schemas will change. Where possible, collaborate with source system teams to be notified of looming schema changes.

Hãy dự liệu rằng các lược đồ thượng nguồn sẽ thay đổi. Trong khả năng có thể, hãy phối hợp với các đội hệ thống nguồn để được thông báo về các thay đổi lược đồ sắp tới.

Master data management

Quản lý dữ liệu chủ

Is the creation of upstream records controlled by a master data management practice or system?

Việc tạo các bản ghi thượng nguồn có được kiểm soát bởi một thực hành hay hệ thống quản lý dữ liệu chủ (MDM) không?

Privacy and ethics

Quyền riêng tư và đạo đức

Do you have access to raw data, or will the data be obfuscated? What are the implications of the source data? How long is it retained? Does it shift locations based on retention policies?

Bạn có quyền truy cập dữ liệu thô không, hay dữ liệu sẽ được làm mờ (obfuscate)? Các hệ quả của dữ liệu nguồn là gì? Nó được lưu giữ bao lâu? Nó có chuyển vị trí dựa trên các chính sách lưu giữ (retention) không?

Regulatory

Quy định

Based upon regulations, are you supposed to access the data?

Dựa trên các quy định, bạn có được phép truy cập dữ liệu không?

DataOps — DataOps

Operational excellence—DevOps, DataOps, **MLOps**, X Ops—should extend up and down the entire stack and support the data engineering and lifecycle.

Sự xuất sắc trong vận hành—DevOps, DataOps, MLOps, X Ops—nên trải dài lên xuống toàn bộ ngăn xếp và hỗ trợ kỹ thuật dữ liệu và vòng đời.

While this is ideal, it's often not fully realized. Because you're working with stakeholders who control both the source systems and the data they produce, you need to ensure that you can observe and monitor the uptime and usage of the source systems and respond when incidents occur. For example, when the application database you depend on for CDC exceeds its I/O capacity and needs to be rescaled, how will that affect your ability to receive data from this system? Will you be able to access the data, or will it be unavailable until the database is rescaled? How will this affect reports? In another example, if the software engineering team is continuously deploying, a code change may cause unanticipated failures in the application itself. How will the failure impact your ability to access the databases powering the application? Will the data be up-to-date? Set up a clear communication chain between data engineering and the teams supporting the source systems. Ideally, these stakeholder teams have incorporated DevOps into their workflow and culture. This will go a long way to accomplishing the goals of DataOps (a sibling of DevOps), to address and

reduce errors quickly. As we mentioned earlier, data engineers need to weave themselves into the DevOps practices of stakeholders, and vice versa. Successful DataOps works when all people are on board and focus on making systems holistically work. A few DataOps considerations are as follows:

Automation

Mặc dù điều này là lý tưởng, nó thường không được hiện thực hóa hoàn toàn. Vì bạn đang làm việc với các bên liên quan kiểm soát cả các hệ thống nguồn lẫn dữ liệu mà chúng sản xuất, bạn cần đảm bảo rằng mình có thể quan sát và giám sát thời gian hoạt động cũng như mức sử dụng của các hệ thống nguồn và phản ứng khi các sự cố xảy ra. Ví dụ, khi database ứng dụng mà bạn phụ thuộc vào cho CDC vượt quá năng lực I/O của nó và cần được mở rộng lại quy mô, điều đó sẽ ảnh hưởng thế nào đến khả năng nhận dữ liệu từ hệ thống này của bạn? Bạn có thể truy cập dữ liệu không, hay nó sẽ không khả dụng cho đến khi database được mở rộng lại quy mô? Điều này sẽ ảnh hưởng thế nào đến các báo cáo? Trong một ví dụ khác, nếu đội kỹ thuật phần mềm đang triển khai liên tục, một thay đổi mã có thể gây ra các thất bại ngoài dự kiến trong chính ứng dụng. Thất bại đó sẽ tác động thế nào đến khả năng truy cập các database vận hành ứng dụng của bạn? Dữ liệu có được cập nhật không? Hãy thiết lập một chuỗi giao tiếp rõ ràng giữa kỹ thuật dữ liệu và các đội hỗ trợ các hệ thống nguồn. Lý tưởng nhất là các đội bên liên quan này đã tích hợp DevOps vào luồng công việc và văn hóa của họ. Điều này sẽ đi một quãng đường dài đến việc hoàn thành các mục tiêu của DataOps (một người anh em của DevOps), nhằm xử lý và giảm thiểu lỗi một cách nhanh chóng. Như chúng tôi đã đề cập trước đó, các kỹ sư dữ liệu cần đan mình vào các thực hành DevOps của các bên liên quan, và ngược lại. DataOps thành công khi tất cả mọi người đều đồng lòng và tập trung vào việc làm cho các hệ thống hoạt động một cách tổng thể. Một vài cân nhắc về DataOps như sau: Tự động hóa

There's the automation impacting the source system, such as code updates and new features. Then there's the DataOps automation that you've set up for your data workflows. Does an issue in the source system's automation impact your data workflow automation? If so, consider decoupling these systems so they can perform automation independently.

Có sự tự động hóa tác động đến hệ thống nguồn, như các bản cập nhật mã và các tính năng mới. Rồi có sự tự động hóa DataOps mà bạn đã thiết lập cho các luồng công việc dữ liệu của mình. Một vấn đề trong sự tự động hóa của hệ thống nguồn có tác động đến sự tự động hóa luồng công việc dữ liệu của bạn không? Nếu có, hãy cân nhắc tách rời các hệ thống này để chúng có thể thực hiện tự động hóa một cách độc lập.

Observability How will you know when there's an issue with a source system, such as an outage or a data-quality issue? Set up monitoring for source system uptime (or use the monitoring created by the team that owns the source system). Set up checks to ensure that data from the source system conforms with expectations for downstream usage. For example, is the data of good quality? Is the schema conformant? Are customer records consistent? Is data hashed as stipulated by the internal policy?

Khả năng quan sát Làm sao bạn biết khi có một vấn đề với một hệ thống nguồn, như một sự cố ngừng hoạt động hay một vấn đề về chất lượng dữ liệu? Hãy thiết lập giám sát cho thời gian hoạt động của hệ thống nguồn (hoặc dùng sự giám sát do đội sở hữu hệ thống nguồn tạo ra). Hãy thiết lập các kiểm tra để đảm bảo rằng dữ liệu từ hệ thống nguồn phù hợp với các kỳ vọng cho việc sử dụng ở hạ nguồn. Ví dụ, dữ liệu có chất lượng tốt không? Lược đồ có tuân thủ không? Các bản ghi khách hàng có nhất quán không? Dữ liệu có được băm như chính sách nội bộ quy định không?

Incident response

Ứng phó sự cố

What's your plan if something bad happens? For example, how will your data pipeline behave if a source system goes offline? What's your plan to backfill the “lost” data once the source system is back online?

Kế hoạch của bạn là gì nếu điều gì đó tồi tệ xảy ra? Ví dụ, đường ống dữ liệu của bạn sẽ hành xử thế nào nếu một hệ thống nguồn ngoại tuyến? Kế hoạch của bạn để nạp bù (backfill) dữ liệu “bị mất” một khi hệ thống nguồn trở lại trực tuyến là gì?

Data Architecture — Kiến trúc dữ liệu

Similar to data management, unless you're involved in the design and maintenance of the source system architecture, you'll have little impact on the upstream source system architecture. You should also understand how the upstream architecture is designed, and its strengths and weaknesses. Talk often with the teams responsible for the source systems to understand the factors discussed in this section and ensure that their systems can meet your expectations. Knowing where the architecture performs well and where it doesn't will impact how you design your data pipeline. Here are some things to consider regarding source system architectures: Reliability

Tương tự như quản lý dữ liệu, trừ khi bạn tham gia vào việc thiết kế và bảo trì kiến trúc hệ thống nguồn, bạn sẽ có ít tác động đến kiến trúc hệ thống nguồn thượng nguồn. Bạn cũng nên hiểu cách kiến trúc thượng nguồn được thiết kế, cùng các điểm mạnh và điểm yếu của nó. Hãy nói chuyện thường xuyên với các đội chịu trách nhiệm cho các hệ thống nguồn để hiểu các yếu tố được thảo luận trong phần này và đảm bảo rằng các hệ thống của họ có thể đáp ứng các kỳ vọng của bạn. Biết nơi kiến trúc hoạt động tốt và nơi nó không sẽ tác động đến cách bạn thiết kế đường ống dữ liệu của mình. Dưới đây là một số điều cần cân nhắc liên quan đến các kiến trúc hệ thống nguồn: Độ tin cậy

All systems suffer from entropy at some point, and outputs will drift from what's expected. Bugs are introduced, and random glitches happen. Does the system produce predictable outputs? How often can we expect the system to fail? What's the mean time to repair to get the system back to sufficient reliability?

Tất cả các hệ thống đều chịu entropy tại một thời điểm nào đó, và các đầu ra sẽ trôi dạt khỏi những gì được kỳ vọng. Các lỗi được đưa vào, và các trục trặc ngẫu nhiên xảy ra. Hệ thống có sản xuất các đầu ra có thể dự đoán không? Chúng ta có thể kỳ vọng hệ thống thất bại bao lâu một lần? Thời gian sửa chữa trung bình để đưa hệ thống trở lại độ tin cậy đủ dùng là bao nhiêu?

Durability

Độ bền

Everything fails. A server might die, a cloud's zone or region could go offline, or other issues may arise. You need to account for how an inevitable failure or outage will affect your managed data systems. How does the source system handle data loss from hardware failures or network outages? What's the plan for handling outages for an extended period and limiting the blast radius of an outage?

Mọi thứ đều thất bại. Một máy chủ có thể chết, một vùng (zone) hay khu vực (region) của cloud có thể ngoại tuyến, hoặc các vấn đề khác có thể nảy sinh. Bạn cần tính đến việc một thất bại hay sự cố ngừng hoạt động không thể tránh khỏi sẽ ảnh hưởng thế nào đến các hệ thống dữ liệu được quản lý của bạn. Hệ thống nguồn xử lý việc mất dữ liệu do thất bại phần cứng hay sự cố mạng như thế nào? Kế hoạch để xử lý các sự cố ngừng hoạt động trong một khoảng thời gian kéo dài và giới hạn bán kính ảnh hưởng (blast radius) của một sự cố là gì?

Availability

Tính khả dụng

What guarantees that the source system is up, running, and available when it's supposed to be?

Điều gì đảm bảo rằng hệ thống nguồn đang chạy, hoạt động, và khả dụng khi nó được kỳ vọng phải vậy?

People

Con người

Who's in charge of the source system's design, and how will you know if breaking changes are made in the architecture? A data engineer needs to work with the teams who maintain the source systems and ensure that these systems are architected reliably. Create an SLA with the source system team to set expectations about potential system failure.

Ai chịu trách nhiệm cho thiết kế của hệ thống nguồn, và làm sao bạn biết nếu các thay đổi gây đứt gãy (breaking changes) được thực hiện trong kiến trúc? Một kỹ sư dữ liệu cần làm việc với các đội bảo trì các hệ thống nguồn và đảm bảo rằng các hệ thống này được kiến trúc một cách đáng tin cậy. Hãy tạo một SLA với đội hệ thống nguồn để thiết lập các kỳ vọng về khả năng hệ thống thất bại.

Orchestration — Điều phối

When orchestrating within your data engineering workflow, you'll primarily be concerned with making sure your orchestration can access the source system, which requires the correct network access, authentication, and authorization.

Khi điều phối trong luồng công việc kỹ thuật dữ liệu của bạn, bạn sẽ chủ yếu quan tâm đến việc đảm bảo rằng sự điều phối của bạn có thể truy cập hệ thống nguồn, điều đòi hỏi quyền truy cập mạng, xác thực, và ủy quyền đúng đắn.

Here are some things to think about concerning orchestration for source systems: Cadence and frequency

Dưới đây là một số điều cần suy nghĩ liên quan đến điều phối cho các hệ thống nguồn: Nhịp độ và tần suất

Is the data available on a fixed schedule, or can you access new data whenever you want?

Dữ liệu có sẵn theo một lịch cố định, hay bạn có thể truy cập dữ liệu mới bất cứ khi nào bạn muốn?

Common frameworks

Các framework chung

Do the software and data engineers use the same container manager, such as Kubernetes? Would it make sense to integrate application and data workloads into the same Kubernetes cluster? If you're using an orchestration framework like Airflow, does it make sense to integrate it with the upstream application team? There's no correct answer here, but you need to balance the benefits of integration with the risks of tight coupling.

Các kỹ sư phần mềm và dữ liệu có dùng cùng một trình quản lý container, chẳng hạn như Kubernetes không? Liệu có hợp lý không khi tích hợp các tải công việc ứng dụng và dữ liệu vào cùng một cụm Kubernetes? Nếu bạn đang dùng một framework điều phối như

Airflow, liệu có hợp lý không khi tích hợp nó với đội ứng dụng thượng nguồn? Không có câu trả lời đúng ở đây, nhưng bạn cần cân bằng các lợi ích của việc tích hợp với các rủi ro của việc ghép chặt.

Software Engineering — Kỹ thuật phần mềm

As the data landscape shifts to tools that simplify and automate access to source systems, you'll likely need to write code. Here are a few considerations when writing code to access a source system: Networking

Khi bối cảnh dữ liệu chuyển dịch sang các công cụ đơn giản hóa và tự động hóa việc truy cập các hệ thống nguồn, bạn nhiều khả năng sẽ cần viết mã. Dưới đây là một vài cân nhắc khi viết mã để truy cập một hệ thống nguồn: Mạng

Make sure your code will be able to access the network where the source system resides. Also, always think about secure networking. Are you accessing an HTTPS URL over the public internet, SSH, or a VPN?

Hãy đảm bảo rằng mã của bạn sẽ có thể truy cập mạng nơi hệ thống nguồn trú ngụ. Ngoài ra, hãy luôn nghĩ về mạng an toàn. Bạn đang truy cập một URL HTTPS qua internet công cộng, SSH, hay một VPN?

Authentication and authorization

Xác thực và ủy quyền

Do you have the proper credentials (tokens, username/passwords) to access the source system? Where will you store these credentials so they don't appear in your code or version control? Do you have the correct IAM roles to perform the coded tasks?

Bạn có các thông tin xác thực đúng đắn (token, tên người dùng/mật khẩu) để truy cập hệ thống nguồn không? Bạn sẽ lưu các thông tin xác thực này ở đâu để chúng không xuất hiện trong mã hay version control của bạn? Bạn có các vai trò IAM đúng đắn để thực hiện các tác vụ đã được lập mã không?

Access patterns

Mẫu hình truy cập

How are you accessing the data? Are you using an API, and how are you handling REST/GraphQL requests, response data volumes, and pagination? If you're accessing data via a database driver, is the driver compatible with the database you're accessing? For either access pattern, how are things like retries and timeouts handled?

Bạn đang truy cập dữ liệu như thế nào? Bạn có đang dùng một API không, và bạn xử lý các yêu cầu REST/GraphQL, khối lượng dữ liệu phản hồi, và phân trang (pagination) như thế nào? Nếu bạn đang truy cập dữ liệu qua một driver database, driver có tương thích với database mà bạn đang truy cập không? Với cả hai mẫu hình truy cập, những thứ như thử lại (retry) và timeout được xử lý như thế nào?

Orchestration

Điều phối

Does your code integrate with an orchestration framework, and can it be executed as an orchestrated workflow?

Mã của bạn có tích hợp với một framework điều phối không, và nó có thể được thực thi như một luồng công việc được điều phối không?

Parallelization

Song song hóa

How are you managing and scaling parallel access to source systems?

Bạn quản lý và mở rộng việc truy cập song song đến các hệ thống nguồn như thế nào?

Deployment

Triển khai

How are you handling the deployment of source code changes?

Bạn xử lý việc triển khai các thay đổi mã nguồn như thế nào?

Conclusion — Kết luận

Source systems and their data are vital in the data engineering lifecycle. Data engineers tend to treat source systems as “someone else’s problem”—do this at your peril! Data engineers who abuse source systems may need to look for another job when production goes down. If there’s a stick, there’s also a carrot. Better collaboration with source system teams can lead to higher-quality data, more successful outcomes, and better data products. Create a bidirectional flow of communications with your counterparts on these teams; set up processes to notify of schema and application changes that affect analytics and ML. Communicate your data needs proactively to assist application teams in the data engineering process. Be aware that the integration between data engineers and source system teams is growing. One example is reverse ETL, which has long lived in the shadows but has recently risen into prominence. We also discussed that the event-streaming platform could serve a role in event-driven architectures and analytics; a source system can also be a data engineering system. Build shared systems where it makes sense to do so. Look for opportunities to build user-facing data products. Talk to application teams about analytics they would like to present to their users or places where ML could improve the user experience. Make application teams stakeholders in data engineering, and find ways to share your successes. Now that you understand the types of source systems and the data they generate, we’ll next look at ways to store this data.

Các hệ thống nguồn và dữ liệu của chúng là sống còn trong vòng đời kỹ thuật dữ liệu. Các kỹ sư dữ liệu có xu hướng coi các hệ thống nguồn là “vấn đề của người khác”—làm vậy là tự rước nguy hiểm! Các kỹ sư dữ liệu lạm dụng các hệ thống nguồn có thể cần tìm một công việc khác khi production sập. Nếu có cây gậy, thì cũng có củ cà rốt. Sự cộng tác tốt hơn với các đội hệ thống nguồn có thể dẫn đến dữ liệu chất lượng cao hơn, các kết quả thành công hơn, và các sản phẩm dữ liệu tốt hơn. Hãy tạo một dòng giao tiếp hai chiều với các đối tác của bạn trong các đội này; thiết lập các tiến trình để thông báo về các thay đổi lược đồ và ứng dụng ảnh hưởng đến phân tích và ML. Hãy truyền đạt các nhu cầu dữ liệu của bạn một cách chủ động để hỗ trợ các đội ứng dụng trong quá trình kỹ thuật dữ liệu. Hãy lưu ý rằng sự tích hợp giữa các kỹ sư dữ liệu và các đội hệ thống nguồn đang phát triển. Một ví dụ là reverse ETL, vốn đã sống lâu trong bóng tối nhưng gần đây đã nổi lên thành tâm điểm. Chúng ta cũng đã thảo luận rằng nền tảng luồng sự kiện có thể đóng một vai trò trong các kiến trúc hướng sự kiện và phân tích; một hệ thống nguồn cũng có thể là một hệ thống kỹ thuật dữ liệu. Hãy xây dựng các hệ thống dùng chung nơi việc làm vậy là hợp lý. Hãy tìm kiếm các cơ hội để xây dựng các sản phẩm dữ liệu hướng-đến-người-dùng. Hãy nói chuyện với các đội ứng dụng về phân tích mà họ muốn trình bày cho người dùng

của họ hoặc những nơi mà ML có thể cải thiện trải nghiệm người dùng. Hãy biến các đội ứng dụng thành các bên liên quan trong kỹ thuật dữ liệu, và tìm cách chia sẻ các thành công của bạn. Giờ đây khi bạn đã hiểu các loại hệ thống nguồn và dữ liệu mà chúng sinh ra, tiếp theo chúng ta sẽ xem xét các cách để lưu trữ dữ liệu này.

Additional Resources

“The What, Why, and When of Single-Table Design with DynamoDB” by Alex DeBrie

“Test Data Quality at Scale with Deequ” by Dustin Lange et al.

“Modernizing Business Data Indexing” by Benjamin Douglas and Mohammad Mohtasham

Confluent’s “Schema Evolution and Compatibility” documentation

“NoSQL: What’s in a Name” by Eric Evans

“The Log: What Every Software Engineer Should Know About Real-Time Data’s Unifying Abstraction” by ay Kreps Database System Concepts by Abraham (Avi) Silberschatz et al. (McGraw Hill)

Database Internals by Alex Petrov (O’Reilly)

1 Keith D. Foote, “A Brief History of Non-Relational Databases,” Dataversity, June 19, 2018, <https://oreil.ly/5Ukg2> . 2 Nemil Dalal’s excellent series on the history of MongoDB recounts some harrowing tales of database abuse and its consequences for fledgling startups. 3 Martin Kleppmann, Designing Data-Intensive Applications (Sebastopol, CA: O’Reilly, 2017), 49, <https://oreil.ly/v1NhG> . 4 Aashish Mehra, “Graph Database Market Worth \$5.1 Billion by 2026: Exclusive Report by MarketsandMarkets,” Cision PR Newswire, July 30, 2021, <https://oreil.ly/mGVkY> . 5 For one example, see Michael S. Mikowski, “RESTful APIs: The Big Lie,” August 10, 2015, <https://oreil.ly/rqja3> . 6 Martin Fowler, “How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh,” MartinFowler.com, May 20, 2019, <https://oreil.ly/TEdJF> . 7 Read Data Pipelines Pocket Reference (O’Reilly) for more information on how a data contract should be written.

Chapter 6. Storage — Chương 6. Lưu trữ

Storage is the cornerstone of the **data engineering lifecycle** (Figure 6-1) and underlies its major stages—**ingestion**, **transformation**, and **serving**. Data gets stored many times as it moves through the lifecycle. To paraphrase an old saying, it's storage all the way down. Whether data is needed seconds, minutes, days, months, or years later, it must persist in storage until systems are ready to consume it for further processing and transmission. Knowing the use case of the data and the way you will retrieve it in the future is the first step to choosing the proper storage solutions for your **data architecture**.

Lưu trữ là nền tảng của vòng đời kỹ thuật dữ liệu (Hình 6-1) và làm cơ sở cho các giai đoạn chính của nó—thu nạp (ingestion), biến đổi (transformation) và phục vụ (serving). Dữ liệu được lưu trữ nhiều lần khi nó di chuyển qua vòng đời. Diễn giải lại một câu nói cũ, đó là lưu trữ ở mọi tầng. Bất kể dữ liệu được cần đến sau vài giây, vài phút, vài ngày, vài tháng hay vài năm, nó phải tồn tại bền vững trong lưu trữ cho đến khi các hệ thống sẵn sàng tiêu thụ nó để xử lý và truyền tải tiếp. Hiểu được tình huống sử dụng (use case) của dữ liệu và cách bạn sẽ truy xuất nó trong tương lai là bước đầu tiên để chọn các giải pháp lưu trữ phù hợp cho kiến trúc dữ liệu của bạn.

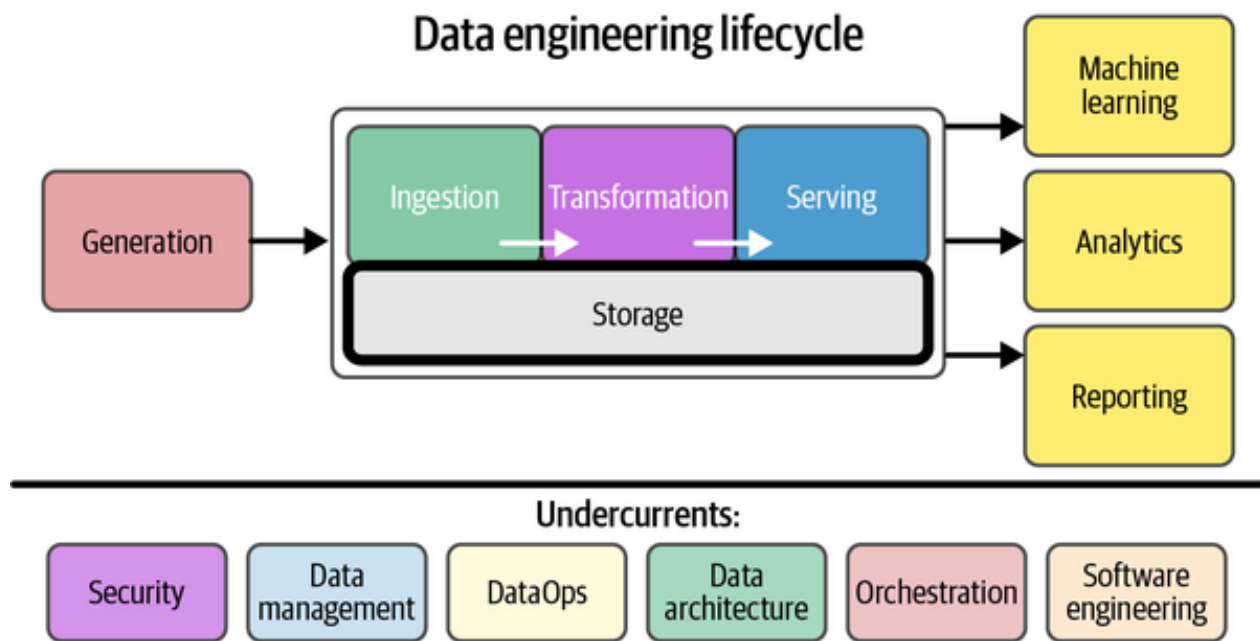


Figure 6-1. Storage plays a central role in the **data engineering** lifecycle We also discussed storage

in Chapter 5 , but with a difference in focus and domain of control. Source systems are generally not maintained or controlled by data engineers. The storage that data engineers handle directly, which we'll focus on in this chapter, encompasses the data engineering lifecycle stages of ingesting data from source systems to serving data to deliver value with **analytics**, data science, etc. Many forms of storage undercut the entire data engineering lifecycle in some fashion.

Hình 6-1. Lưu trữ đóng vai trò trung tâm trong vòng đời kỹ thuật dữ liệu. Chúng ta cũng đã bàn về lưu trữ ở Chương 5, nhưng với trọng tâm và phạm vi kiểm soát khác. Các hệ thống nguồn (source system) nhìn chung không được kỹ sư dữ liệu duy trì hay kiểm soát. Phần lưu trữ mà kỹ sư dữ liệu xử lý trực tiếp, mà chúng ta sẽ tập trung vào trong chương này, bao trùm các giai đoạn của vòng đời kỹ thuật dữ liệu từ thu nạp dữ liệu từ các hệ thống nguồn đến phục vụ dữ liệu nhằm tạo ra giá trị bằng phân tích (analytics), khoa học dữ liệu, v.v. Nhiều dạng lưu trữ làm nền cho toàn bộ vòng đời kỹ thuật dữ liệu theo cách này hay cách khác.

To understand storage, we're going to start by studying the raw ingredients that compose storage systems, including hard drives, solid state drives, and system memory (see Figure 6-2). It's essential to understand the basic characteristics of physical storage technologies to assess the trade-offs inherent in any storage architecture. This section also discusses **serialization** and **compression**, key software elements of practical storage. (We defer a deeper technical discussion of serialization and compression to Appendix A .) We also discuss caching , which is critical in assembling storage systems.

Để hiểu về lưu trữ, chúng ta sẽ bắt đầu bằng việc nghiên cứu các nguyên liệu thô cấu thành nên các hệ thống lưu trữ, bao gồm ổ đĩa cứng, ổ đĩa thể rắn (solid-state drive) và bộ nhớ hệ thống (xem Hình 6-2). Hiểu các đặc tính cơ bản của các công nghệ lưu trữ vật lý là điều thiết yếu để đánh giá những đánh đổi vốn có trong bất kỳ kiến trúc lưu trữ nào. Mục này cũng bàn về tuần tự hóa (serialization) và nén (compression), các thành phần phần mềm then chốt của lưu trữ thực tế. (Chúng ta để dành phần thảo luận kỹ thuật sâu hơn về tuần tự hóa và nén cho Phụ lục A.) Chúng ta cũng bàn về caching (lưu đệm), vốn cực kỳ quan trọng trong việc lắp ráp các hệ thống lưu trữ.

Storage abstractions

Data lake

Data
lakehouse

Data platform

Cloud data
warehouse

Storage systems

HDFS

Object storage

RDBMS

Redis

Kafka

Raw ingredients

HDD

SSD

RAM

Networking

Serialization

Compression

CPU

Figure 6-2. Raw ingredients, storage systems, and storage abstractions Next, we'll look at storage systems . In practice, we don't directly access system memory or hard disks. These physical storage components exist inside servers and clusters that can ingest and retrieve data using various access paradigms. Finally, we'll look at storage abstractions . Storage systems are assembled into a cloud **data warehouse**, a **data lake**, etc. When building data pipelines, engineers choose the appropriate abstractions for storing their data as it moves through the ingestion, transformation, and serving stages.

Hình 6-2. Nguyên liệu thô, hệ thống lưu trữ và trừu tượng lưu trữ. Tiếp theo, chúng ta sẽ xem xét các hệ thống lưu trữ. Trong thực tế, chúng ta không truy cập trực tiếp bộ nhớ hệ thống hay ổ đĩa cứng. Các thành phần lưu trữ vật lý này tồn tại bên trong các máy chủ và cụm (cluster) có thể thu nạp và truy xuất dữ liệu bằng nhiều mô hình truy cập khác nhau. Cuối cùng, chúng ta sẽ xem xét các trừu tượng lưu trữ. Các hệ thống lưu trữ được lắp ráp thành một kho dữ liệu (data warehouse) đám mây, một hồ dữ liệu (data lake), v.v. Khi xây dựng đường ống dữ liệu (data pipeline), kỹ sư chọn các trừu tượng phù hợp để lưu trữ dữ liệu của họ khi nó di chuyển qua các giai đoạn thu nạp, biến đổi và phục vụ.

Raw Ingredients of Data Storage — Nguyên liệu thô của lưu trữ dữ liệu

Storage is so common that it's easy to take it for granted. We're often surprised by the number of software and data engineers who use storage every day but have little idea how it works behind the scenes or the trade-offs inherent in various storage media. As a result, we see storage used in some pretty...interesting ways. Though current managed services potentially free data engineers from the complexities of managing servers, data engineers still need to be aware of underlying components' essential characteristics, performance considerations, durability, and costs. In most data architectures, data frequently passes through magnetic storage, SSDs, and memory as it works its way through the various processing phases of a **data pipeline**. Data storage and query systems generally follow complex recipes involving distributed systems, numerous services, and multiple hardware storage layers. These systems require the right raw ingredients to function correctly. Let's look at some of the raw ingredients of data storage: disk drives, memory, networking and CPU, serialization, compression, and caching.

Lưu trữ phổ biến đến mức rất dễ bị xem nhẹ. Chúng tôi thường ngạc nhiên về số lượng kỹ sư phần mềm và kỹ sư dữ liệu sử dụng lưu trữ hằng ngày nhưng lại không mấy hiểu nó hoạt động ra sao ở phía sau hậu trường, hoặc những đánh đổi vốn có trong các phương tiện lưu trữ khác nhau. Hệ quả là chúng ta thấy lưu trữ được dùng theo những cách khá... thú vị. Mặc dù các dịch vụ được quản lý (managed service) hiện nay có tiềm năng giải phóng kỹ sư dữ liệu khỏi sự phức tạp của việc quản lý máy chủ, kỹ sư dữ liệu vẫn cần nắm rõ các đặc tính thiết yếu, các yếu tố cần cân nhắc về hiệu năng, độ bền và chi phí của các thành phần nền tảng. Trong hầu hết các kiến trúc dữ liệu, dữ liệu thường xuyên đi qua lưu trữ từ tính, SSD và bộ nhớ khi nó di chuyển qua các pha xử lý khác nhau của một đường ống dữ liệu. Các hệ thống lưu trữ và truy vấn dữ liệu nhìn chung tuân theo những công thức phức tạp gồm các hệ phân tán, vô số dịch vụ và nhiều tầng lưu trữ phân cứng. Các hệ thống này đòi hỏi đúng nguyên liệu thô để vận hành chính xác. Hãy cùng xem một số nguyên liệu thô của lưu trữ dữ liệu: ổ đĩa, bộ nhớ, mạng và CPU, tuần tự hóa, nén và caching.

Magnetic Disk Drive — Ổ đĩa từ

Magnetic disks utilize spinning platters coated with a ferromagnetic film (Figure 6-3). This film is magnetized by a read/write head during write operations to physically encode binary data. The read/write head detects the magnetic field and outputs a bitstream during read operations. Magnetic disk drives have been around for ages. Still, they form the backbone of bulk data storage systems because they are significantly cheaper than SSDs per gigabyte of stored data.

Ổ đĩa từ sử dụng các đĩa tròn (platter) quay phủ một lớp màng sắt từ (Hình 6-3). Lớp màng này được nhiễm từ bởi một đầu đọc/ghi trong các thao tác ghi để mã hóa vật lý dữ liệu nhị phân. Đầu đọc/ghi phát hiện trường từ và xuất ra một luồng bit trong các thao tác đọc. Ổ đĩa từ đã tồn tại từ lâu đời. Tuy vậy, chúng vẫn tạo thành xương sống của các hệ thống lưu trữ dữ liệu khối lượng lớn vì chúng rẻ hơn đáng kể so với SSD trên mỗi gigabyte dữ liệu được lưu.

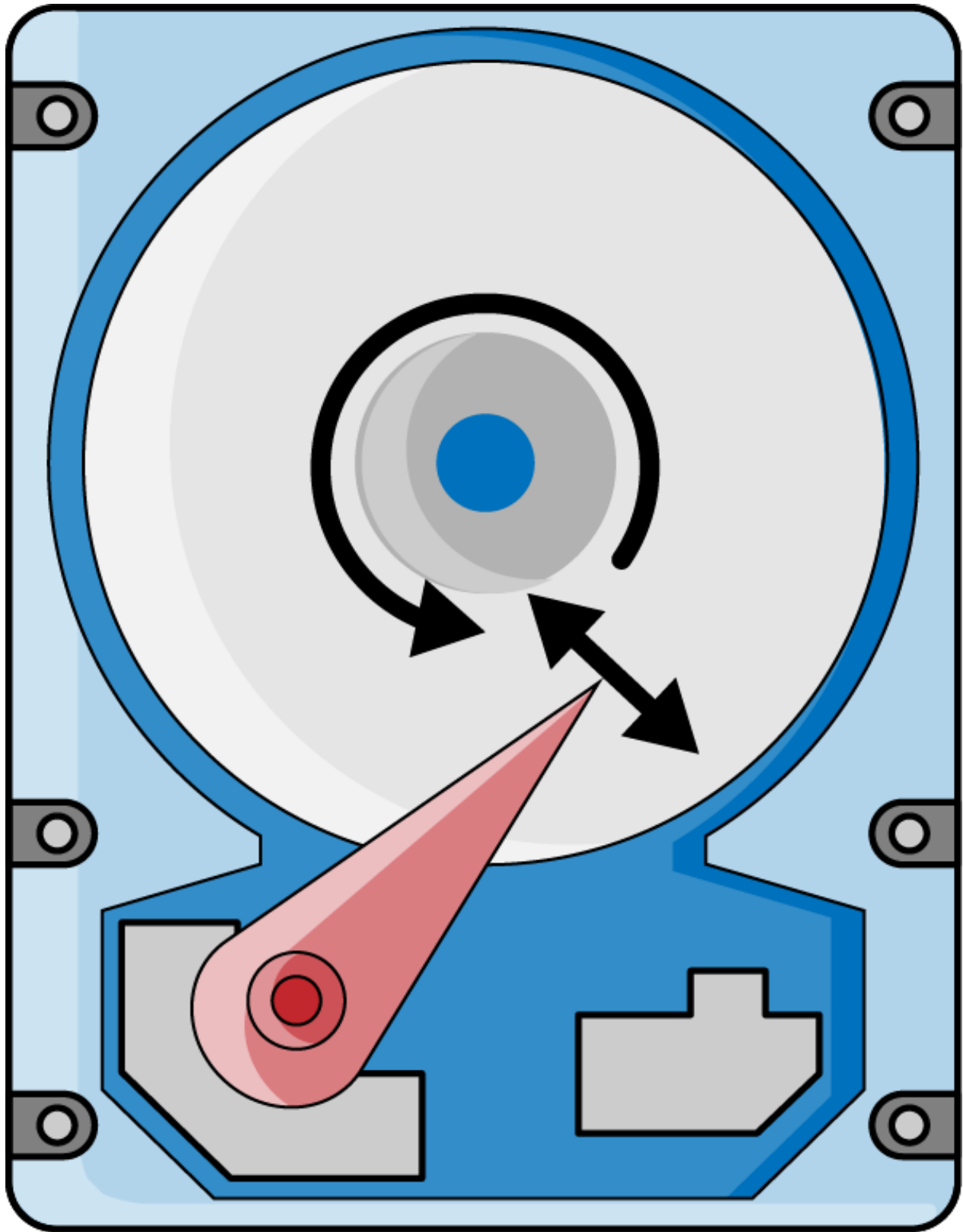


Figure 6-3. Magnetic disk head movement and rotation are essential in random access **latency** On the one hand, these disks have seen extraordinary improvements in performance, storage density, and cost. On the other hand, SSDs 1 dramatically outperform magnetic disks on various metrics.

Currently, commercial magnetic disk drives cost roughly 3 cents per gigabyte of capacity. (Note that we'll frequently use the abbreviations HDD and SSD to denote rotating magnetic disk and solid-state drives, respectively.) IBM developed magnetic disk drive technology in the 1950s. Since then, magnetic disk capacities have grown steadily. The first commercial magnetic disk drive, the IBM 350, had a capacity of 3.75 megabytes. As of this writing, magnetic drives storing 20 TB are commercially available. In fact, magnetic disks continue to see rapid innovation, with methods such as heat-assisted magnetic recording (HAMR), shingled magnetic recording (SMR), and helium-filled disk enclosures being used to realize ever greater storage densities. In spite of the continuing improvements in drive capacity, other aspects of HDD performance are hampered by physics. First, disk transfer speed, the rate at which data can be read and written, does not scale in proportion with disk capacity. Disk capacity scales with areal density (gigabits stored per square inch), whereas transfer speed scales with linear density (bits per inch). This means that if disk capacity grows by a factor of 4, transfer speed increases by only a factor of 2. Consequently, current data center drives support maximum data transfer speeds of 200–300 MB/s. To frame this another way, it takes more than 20 hours to read the entire contents of a 30 TB magnetic drive, assuming a transfer speed of 300 MB/s. A second major limitation is seek time. To access data, the drive must physically relocate the read/write heads to the appropriate track on the disk. Third, in order to find a particular piece of data on the disk, the disk controller must wait for that data to rotate under the read/write heads. This leads to rotational latency. Typical commercial drives spinning at 7,200 revolutions per minute (RPM) seek time, and rotational latency, leads to over four milliseconds of overall average latency (time to access a selected piece of data). A fourth limitation is input/output operations per second (IOPS), critical for transactional databases. A magnetic drive ranges from 50 to 500 IOPS.

Hình 6-3. Chuyển động của đầu đọc đĩa từ và sự quay là yếu tố thiết yếu trong độ trễ (latency) truy cập ngẫu nhiên. Một mặt, các đĩa này đã chứng kiến những cải tiến phi thường về hiệu năng, mật độ lưu trữ và chi phí. Mặt khác, SSD 1 vượt trội đĩa từ một cách đáng kể ở nhiều chỉ số. Hiện nay, ổ đĩa từ thương mại có giá khoảng 3 xu trên mỗi gigabyte dung lượng. (Lưu ý rằng chúng tôi sẽ thường dùng các viết tắt HDD và SSD để chỉ ổ đĩa từ quay và ổ đĩa thể rắn, tương ứng.) IBM đã phát triển công nghệ ổ đĩa từ vào những năm 1950. Kể từ đó, dung lượng đĩa từ đã tăng đều đặn. Ổ đĩa từ thương mại đầu tiên, IBM 350, có dung lượng 3,75 megabyte. Tại thời điểm viết cuốn sách này, các ổ đĩa từ lưu trữ 20 TB đã có sẵn trên thị trường. Trên thực tế, đĩa từ tiếp tục chứng kiến đổi mới nhanh chóng, với các phương pháp như ghi từ hỗ trợ bằng nhiệt (heat-assisted magnetic recording, HAMR), ghi từ chồng ngói (shingled magnetic recording, SMR) và vỏ đĩa đổ đầy heli được sử dụng để đạt được mật độ lưu trữ ngày càng lớn hơn. Bất chấp những cải tiến liên tục về dung lượng ổ đĩa, các khía cạnh khác của hiệu năng HDD lại bị vật lý cản trở. Thứ nhất, tốc độ truyền của đĩa, tức tốc độ mà dữ liệu có thể được đọc và ghi, không tăng tỷ lệ với dung lượng đĩa. Dung lượng đĩa tỷ lệ với mật độ diện tích (gigabit lưu trên mỗi inch vuông), trong khi tốc độ truyền tỷ lệ với mật độ tuyến tính (bit trên mỗi inch). Điều này có nghĩa là nếu dung lượng đĩa tăng gấp 4 lần, tốc độ truyền chỉ tăng gấp 2 lần. Hệ quả là các ổ đĩa trung tâm dữ liệu hiện nay hỗ trợ tốc độ truyền dữ liệu tối đa 200–300 MB/s. Diễn đạt theo cách khác, sẽ mất hơn 20 giờ để đọc toàn bộ nội dung của một ổ đĩa từ 30 TB, giả sử tốc độ truyền là 300 MB/s. Một hạn chế lớn thứ hai là thời gian tìm kiếm (seek time). Để truy cập dữ liệu, ổ đĩa phải di chuyển vật lý đầu đọc/ghi đến đúng rãnh (track) trên đĩa. Thứ ba, để tìm một mẫu dữ liệu cụ thể trên đĩa, bộ điều khiển đĩa phải chờ dữ liệu đó quay đến dưới đầu đọc/ghi. Điều này dẫn đến độ trễ quay (rotational latency). Các ổ đĩa thương mại điển hình quay ở tốc độ 7.200 vòng mỗi phút (RPM) có thời gian tìm kiếm và độ trễ quay dẫn đến độ trễ trung bình tổng thể hơn bốn mili-giây (thời gian để truy cập một mẫu dữ liệu được chọn). Hạn chế thứ tư là số thao tác vào/ra mỗi giây (input/output operations per second, IOPS), yếu tố then chốt đối với các cơ sở dữ liệu giao dịch (transactional database). Một ổ đĩa từ có IOPS dao động từ 50 đến 500.

Various tricks can improve latency and transfer speed. Using a higher rotational speed can increase transfer rate and decrease rotational latency. Limiting the radius of the disk platter or writing data into only a narrow band on the disk reduces seek time. However, none of these techniques makes magnetic drives remotely competitive with SSDs for random access lookups. SSDs can deliver data with significantly lower latency, higher IOPS, and higher transfer speeds, partially because there is no physically rotating disk or magnetic head to wait for. As mentioned earlier, magnetic disks are still prized in data centers for their low data-storage costs. In addition, magnetic drives can sustain extraordinarily high transfer rates through parallelism. This is the critical idea behind cloud **object storage**: data can be distributed across thousands of disks in clusters. Data-transfer rates go up dramatically by reading from numerous disks simultaneously, limited primarily by network performance rather than disk transfer rate. Thus, network components and CPUs are also key raw ingredients in storage systems, and we will return to these topics shortly.

Nhiều mẹo có thể cải thiện độ trễ và tốc độ truyền. Sử dụng tốc độ quay cao hơn có thể tăng tốc độ truyền và giảm độ trễ quay. Giới hạn bán kính của đĩa platter hoặc chỉ ghi dữ liệu vào một dải hẹp trên đĩa làm giảm thời gian tìm kiếm. Tuy nhiên, không kỹ thuật nào trong số này khiến ổ đĩa từ cạnh tranh nổi với SSD ở các tác vụ tra cứu truy cập ngẫu nhiên. SSD có thể cung cấp dữ liệu với độ trễ thấp hơn đáng kể, IOPS cao hơn và tốc độ truyền cao hơn, một phần vì không có đĩa quay vật lý hay đầu từ nào phải chờ. Như đã đề cập trước đó, đĩa từ vẫn được ưa chuộng trong các trung tâm dữ liệu nhờ chi phí lưu trữ dữ liệu thấp. Ngoài ra, các ổ đĩa từ có thể duy trì tốc độ truyền cực cao thông qua song song hóa. Đây là ý tưởng then chốt đằng sau lưu trữ đối tượng (object storage) đám mây: dữ liệu có thể được phân tán trên hàng nghìn đĩa trong các cụm. Tốc độ truyền dữ liệu tăng mạnh nhờ đọc đồng thời từ vô số đĩa, bị giới hạn chủ yếu bởi hiệu năng mạng thay vì tốc độ truyền của đĩa. Do đó, các thành phần mạng và CPU cũng là các nguyên liệu thô then chốt trong các hệ thống lưu trữ, và chúng ta sẽ quay lại các chủ đề này trong giây lát.

Solid-State Drive — Ổ đĩa thể rắn

Solid-state drives (SSDs) store data as charges in flash memory cells. SSDs eliminate the mechanical components of magnetic drives; the data is read by purely electronic means. SSDs can look up random data in less than 0.1 ms (100 microseconds). In addition, SSDs can scale both data-transfer speeds and IOPS by slicing storage into partitions with numerous storage controllers running in parallel. Commercial SSDs can support transfer speeds of many gigabytes per second and tens of thousands of IOPS. Because of these exceptional performance characteristics, SSDs have revolutionized transactional databases and are the accepted standard for commercial deployments of **OLTP** systems. SSDs allow relational databases such as PostgreSQL, MySQL, and SQL Server to handle thousands of transactions per second.

Ổ đĩa thể rắn (solid-state drive, SSD) lưu trữ dữ liệu dưới dạng điện tích trong các ô nhớ flash. SSD loại bỏ các thành phần cơ học của ổ đĩa từ; dữ liệu được đọc bằng các phương tiện hoàn toàn điện tử. SSD có thể tra cứu dữ liệu ngẫu nhiên trong chưa đến 0,1 ms (100 micro-giây). Ngoài ra, SSD có thể mở rộng cả tốc độ truyền dữ liệu lẫn IOPS bằng cách chia lưu trữ thành các phân vùng (partition) với vô số bộ điều khiển lưu trữ chạy song song. SSD thương mại có thể hỗ trợ tốc độ truyền nhiều gigabyte mỗi giây và hàng chục nghìn IOPS. Nhờ các đặc tính hiệu năng vượt trội này, SSD đã cách mạng hóa các cơ sở dữ liệu giao dịch và là tiêu chuẩn được chấp nhận cho các triển khai thương mại của hệ thống OLTP. SSD cho phép các cơ sở dữ liệu quan hệ như PostgreSQL, MySQL và SQL Server xử lý hàng nghìn giao dịch mỗi giây.

However, SSDs are not currently the default option for high-scale analytics data storage. Again, this

comes down to cost. Commercial SSDs typically cost 20–30 cents (USD) per gigabyte of capacity, nearly 10 times the cost per capacity of a magnetic drive. Thus, object storage on magnetic disks has emerged as the leading option for large-scale data storage in data lakes and cloud data warehouses. SSDs still play a significant role in **OLAP** systems. Some OLAP databases leverage SSD caching to support high-performance queries on frequently accessed data. As low-latency OLAP becomes more popular, we expect SSD usage in these systems to follow suit.

Tuy nhiên, SSD hiện chưa phải là lựa chọn mặc định cho lưu trữ dữ liệu phân tích quy mô lớn. Một lần nữa, điều này quy về chi phí. SSD thương mại thường có giá 20–30 xu (USD) trên mỗi gigabyte dung lượng, gần gấp 10 lần chi phí trên mỗi dung lượng của một ổ đĩa từ. Do đó, lưu trữ đối tượng trên đĩa từ đã nổi lên như lựa chọn hàng đầu cho lưu trữ dữ liệu quy mô lớn trong các hồ dữ liệu và kho dữ liệu đám mây. SSD vẫn đóng một vai trò quan trọng trong các hệ thống OLAP. Một số cơ sở dữ liệu OLAP tận dụng caching bằng SSD để hỗ trợ các truy vấn hiệu năng cao trên dữ liệu được truy cập thường xuyên. Khi OLAP độ trễ thấp trở nên phổ biến hơn, chúng tôi kỳ vọng việc sử dụng SSD trong các hệ thống này cũng sẽ tăng theo.

Random Access Memory — Bộ nhớ truy cập ngẫu nhiên

We commonly use the terms random access memory (RAM) and memory interchangeably. Strictly speaking, magnetic drives and SSDs also serve as memory that stores data for later random access retrieval, but RAM has several specific characteristics:

Chúng ta thường dùng các thuật ngữ bộ nhớ truy cập ngẫu nhiên (random access memory, RAM) và bộ nhớ thay thế cho nhau. Nói chặt chẽ, đĩa từ và SSD cũng đóng vai trò là bộ nhớ lưu trữ dữ liệu để truy xuất truy cập ngẫu nhiên về sau, nhưng RAM có một số đặc tính cụ thể:

Is attached to a CPU and mapped into CPU address space.

Được gắn vào một CPU và được ánh xạ vào không gian địa chỉ của CPU.

Stores the code that CPUs execute and the data that this code directly processes.

Lưu trữ mã lệnh mà các CPU thực thi và dữ liệu mà mã lệnh này xử lý trực tiếp.

Is volatile, while magnetic drives and SSDs are nonvolatile. Though they may occasionally fail and corrupt or lose data, drives generally retain data when powered off. RAM loses data in less than a second when it is unpowered.

Là khả biến (volatile), trong khi đĩa từ và SSD là bất biến mất điện (nonvolatile). Mặc dù đôi khi chúng có thể hỏng và làm hư hỏng hoặc mất dữ liệu, các ổ đĩa nhìn chung vẫn giữ được dữ liệu khi tắt nguồn. RAM mất dữ liệu trong chưa đến một giây khi mất nguồn.

Offers significantly higher transfer speeds and faster retrieval times than SSD storage. DDR5 memory—the latest widely used standard for RAM—offers data retrieval latency on the order of 100 ns, roughly 1,000 times faster than SSD. A typical CPU can support 100 GB/s bandwidth to attached memory and millions of IOPS. (Statistics vary dramatically depending on the number of memory channels and other configuration details.) Is significantly more expensive than SSD storage, at roughly \$10/GB (at the time of this writing).

Cung cấp tốc độ truyền cao hơn đáng kể và thời gian truy xuất nhanh hơn so với lưu trữ SSD. Bộ nhớ DDR5—tiêu chuẩn mới nhất được sử dụng rộng rãi cho RAM—cung cấp độ trễ truy xuất dữ liệu cỡ 100 ns, nhanh hơn SSD khoảng 1.000 lần. Một CPU điển hình có thể hỗ trợ băng thông 100 GB/s tới bộ nhớ được gắn và hàng triệu IOPS. (Các con số thống kê

biến thiên rất nhiều tùy thuộc vào số kênh bộ nhớ và các chi tiết cấu hình khác.) Đắt hơn đáng kể so với lưu trữ SSD, ở mức khoảng 10 USD/GB (tại thời điểm viết cuốn sách này).

Is limited in the amount of RAM attached to an individual CPU and memory controller. This adds further to complexity and cost. High-memory servers typically utilize many interconnected CPUs on one board, each with a block of attached RAM.

Bị giới hạn về lượng RAM gắn vào một CPU và bộ điều khiển bộ nhớ riêng lẻ. Điều này càng làm tăng thêm độ phức tạp và chi phí. Các máy chủ bộ nhớ lớn thường sử dụng nhiều CPU liên kết với nhau trên một bo mạch, mỗi CPU có một khối RAM gắn kèm.

Is still significantly slower than CPU cache, a type of memory located directly on the CPU die or in the same package. Cache stores frequently and recently accessed data for ultrafast retrieval during processing. CPU designs incorporate several layers of cache of varying size and performance characteristics.

Vẫn chậm hơn đáng kể so với bộ nhớ đệm (cache) của CPU, một loại bộ nhớ nằm ngay trên đế (die) của CPU hoặc trong cùng một gói (package). Cache lưu trữ dữ liệu được truy cập thường xuyên và gần đây để truy xuất siêu nhanh trong quá trình xử lý. Các thiết kế CPU tích hợp nhiều tầng cache với kích thước và đặc tính hiệu năng khác nhau.

When we talk about system memory, we almost always mean dynamic RAM , a high-density, low-cost form of memory. Dynamic RAM stores data as charges in capacitors. These capacitors leak over time, so the data must be frequently refreshed (read and rewritten) to prevent data loss. The hardware memory controller handles these technical details; data engineers simply need to worry about bandwidth and retrieval latency characteristics. Other forms of memory, such as static RAM , are used in specialized applications such as CPU caches. Current CPUs virtually always employ the von Neumann architecture , with code and data stored together in the same memory space. However, CPUs typically also support the option to disable code execution in specific pages of memory for enhanced security. This feature is reminiscent of the Harvard architecture , which separates code and data. RAM is used in various storage and processing systems and can be used for caching, data processing, or indexes. Several databases treat RAM as a primary storage layer, allowing ultra-fast read and write performance. In these applications, data engineers must always keep in mind the volatility of RAM. Even if data stored in memory is replicated across a cluster, a power outage that brings down several nodes could cause data loss. Architectures intended to durably store data may use battery backups and automatically dump all data to disk in the **event** of power loss.

Khi nói về bộ nhớ hệ thống, hầu như lúc nào chúng ta cũng muốn nói đến RAM động (dynamic RAM), một dạng bộ nhớ mật độ cao, chi phí thấp. RAM động lưu trữ dữ liệu dưới dạng điện tích trong các tụ điện. Các tụ điện này rò rỉ theo thời gian, nên dữ liệu phải được làm tươi (đọc lại và ghi lại) thường xuyên để ngăn mất dữ liệu. Bộ điều khiển bộ nhớ phần cứng xử lý các chi tiết kỹ thuật này; kỹ sư dữ liệu chỉ cần bận tâm đến các đặc tính băng thông và độ trễ truy xuất. Các dạng bộ nhớ khác, chẳng hạn như RAM tĩnh (static RAM), được dùng trong các ứng dụng chuyên biệt như cache của CPU. Các CPU hiện nay hầu như luôn sử dụng kiến trúc von Neumann, với mã lệnh và dữ liệu được lưu cùng nhau trong cùng một không gian bộ nhớ. Tuy nhiên, các CPU thường cũng hỗ trợ tùy chọn vô hiệu hóa việc thực thi mã trong các trang bộ nhớ cụ thể để tăng cường an ninh. Tính năng này gợi nhớ đến kiến trúc Harvard, vốn tách biệt mã và dữ liệu. RAM được dùng trong nhiều hệ thống lưu trữ và xử lý khác nhau và có thể được dùng cho caching, xử lý dữ liệu hoặc chỉ mục (index). Một số cơ sở dữ liệu coi RAM là tầng lưu trữ chính, cho phép hiệu năng đọc và ghi siêu nhanh. Trong các ứng dụng này, kỹ sư dữ liệu phải luôn ghi nhớ tính khả biến của RAM. Ngay cả khi dữ liệu lưu trong bộ nhớ được sao chép (replicate) trên một cụm, một sự cố mất điện làm sập nhiều nút (node) có thể gây mất dữ liệu. Các kiến trúc nhằm

lưu trữ dữ liệu một cách bền vững có thể dùng pin dự phòng và tự động đẩy toàn bộ dữ liệu xuống đĩa trong trường hợp mất nguồn.

Networking and CPU — Mạng và CPU

Why are we mentioning networking and CPU as raw ingredients for storing data? Increasingly, storage systems are distributed to enhance performance, durability, and availability. We mentioned specifically that individual magnetic disks offer relatively low-transfer performance, but a cluster of disks parallelizes reads for significant performance **scaling**. While storage standards such as redundant arrays of independent disks (RAID) parallelize on a single server, cloud object storage clusters operate at a much larger scale, with disks distributed across a network and even multiple data centers and availability zones. Availability zones are a standard cloud construct consisting of compute environments with independent power, water, and other resources. Multizonal storage enhances both the availability and durability of data. CPUs handle the details of servicing requests, aggregating reads, and distributing writes. Storage becomes a web application with an API, backend service components, and load balancing. Network device performance and network topology are key factors in realizing high performance. Data engineers need to understand how networking will affect the systems they build and use. Engineers constantly balance the durability and availability achieved by spreading out data geographically versus the performance and cost benefits of keeping storage in a small geographic area and close to data consumers or writers. Appendix B covers cloud networking and major relevant ideas.

Tại sao chúng ta lại đề cập đến mạng và CPU như những nguyên liệu thô để lưu trữ dữ liệu? Ngày càng nhiều, các hệ thống lưu trữ được phân tán để nâng cao hiệu năng, độ bền và tính sẵn sàng. Chúng ta đã đề cập cụ thể rằng từng đĩa từ riêng lẻ cung cấp hiệu năng truyền tương đối thấp, nhưng một cụm đĩa song song hóa các thao tác đọc để mở rộng hiệu năng đáng kể. Trong khi các chuẩn lưu trữ như mảng đĩa độc lập dự phòng (redundant arrays of independent disks, RAID) song song hóa trên một máy chủ đơn lẻ, các cụm lưu trữ đối tượng đám mây hoạt động ở quy mô lớn hơn nhiều, với các đĩa được phân tán trên một mạng và thậm chí trên nhiều trung tâm dữ liệu và vùng sẵn sàng (availability zone). Vùng sẵn sàng là một cấu trúc đám mây tiêu chuẩn bao gồm các môi trường tính toán với nguồn điện, nước và các tài nguyên khác độc lập. Lưu trữ đa vùng nâng cao cả tính sẵn sàng lẫn độ bền của dữ liệu. CPU xử lý các chi tiết về phục vụ yêu cầu, tổng hợp các thao tác đọc và phân phối các thao tác ghi. Lưu trữ trở thành một ứng dụng web với một API, các thành phần dịch vụ backend và cân bằng tải. Hiệu năng của thiết bị mạng và cấu trúc mạng (network topology) là các yếu tố then chốt để đạt hiệu năng cao. Kỹ sư dữ liệu cần hiểu mạng sẽ ảnh hưởng ra sao đến các hệ thống mà họ xây dựng và sử dụng. Kỹ sư liên tục cân bằng giữa độ bền và tính sẵn sàng đạt được bằng cách trải dữ liệu theo địa lý với những lợi ích về hiệu năng và chi phí của việc giữ lưu trữ trong một khu vực địa lý nhỏ và gần với bên tiêu thụ hoặc bên ghi dữ liệu. Phụ lục B trình bày về mạng đám mây và các ý tưởng liên quan chính.

Serialization — Tuần tự hóa

Serialization is another raw storage ingredient and a critical element of database design. The decisions around serialization will inform how well queries perform across a network, CPU overhead, query latency, and more. Designing a data lake, for example, involves choosing a base storage system (e.g., Amazon S3) and standards for serialization that balance interoperability with performance considerations.

Tuần tự hóa (serialization) là một nguyên liệu lưu trữ thô khác và là một yếu tố then chốt của thiết kế cơ sở dữ liệu. Các quyết định xoay quanh tuần tự hóa sẽ quyết định mức độ hiệu quả khi truy vấn qua mạng, chi phí CPU, độ trễ truy vấn và hơn thế nữa. Ví dụ, thiết kế một hồ dữ liệu liên quan đến việc chọn một hệ thống lưu trữ nền (ví dụ, Amazon S3) và các chuẩn tuần tự hóa cân bằng giữa khả năng tương tác (interoperability) với các yếu tố cần cân nhắc về hiệu năng.

What is serialization, exactly? Data stored in system memory by software is generally not in a format suitable for storage on disk or transmission over a network. Serialization is the process of flattening and packing data into a standard format that a reader will be able to decode. Serialization formats provide a standard of data exchange. We might encode data in a row-based manner as an XML, JSON, or CSV file and pass it to another user who can then decode it using a standard library. A serialization algorithm has logic for handling types, imposes rules on data structure, and allows exchange between programming languages and CPUs. The serialization algorithm also has rules for handling exceptions. For instance, Python objects can contain cyclic references; the serialization algorithm might throw an error or limit nesting depth on encountering a cycle. Low-level database storage is also a form of serialization. Row-oriented relational databases organize data as rows on disk to support speedy lookups and in-place updates. Columnar databases organize data into column files to optimize for highly efficient compression and support fast scans of large data volumes. Each serialization choice comes with a set of trade-offs, and data engineers tune these choices to optimize performance to requirements. We provide a more detailed catalog of common data serialization techniques and formats in Appendix A . We suggest that data engineers become familiar with common serialization practices and formats, especially the most popular current formats (e.g., Apache Parquet), hybrid serialization (e.g., Apache Hudi), and in-memory serialization (e.g., Apache Arrow).

Tuần tự hóa chính xác là gì? Dữ liệu được phần mềm lưu trong bộ nhớ hệ thống nhìn chung không ở định dạng phù hợp để lưu trữ trên đĩa hay truyền qua mạng. Tuần tự hóa là quá trình làm phẳng và đóng gói dữ liệu vào một định dạng chuẩn mà một bên đọc sẽ có thể giải mã. Các định dạng tuần tự hóa cung cấp một chuẩn để trao đổi dữ liệu. Chúng ta có thể mã hóa dữ liệu theo cách dựa trên hàng (row-based) dưới dạng một tệp XML, JSON hoặc CSV và chuyển nó cho một người dùng khác, người này sau đó có thể giải mã nó bằng một thư viện chuẩn. Một thuật toán tuần tự hóa có logic để xử lý các kiểu dữ liệu, áp đặt các quy tắc lên cấu trúc dữ liệu, và cho phép trao đổi giữa các ngôn ngữ lập trình và CPU. Thuật toán tuần tự hóa cũng có các quy tắc để xử lý ngoại lệ. Chẳng hạn, các đối tượng Python có thể chứa các tham chiếu vòng (cyclic reference); thuật toán tuần tự hóa có thể ném ra một lỗi hoặc giới hạn độ sâu lồng nhau khi gặp một vòng lặp. Lưu trữ cơ sở dữ liệu ở mức thấp cũng là một dạng tuần tự hóa. Các cơ sở dữ liệu quan hệ định hướng hàng (row-oriented) tổ chức dữ liệu thành các hàng trên đĩa để hỗ trợ tra cứu nhanh và cập nhật tại chỗ. Các cơ sở dữ liệu cột (columnar) tổ chức dữ liệu thành các tệp cột để tối ưu cho việc nén hiệu quả cao và hỗ trợ quét nhanh các khối lượng dữ liệu lớn. Mỗi lựa chọn tuần tự hóa đi kèm một tập hợp các đánh đổi, và kỹ sư dữ liệu tinh chỉnh các lựa chọn này để tối ưu hiệu năng theo yêu cầu. Chúng tôi cung cấp một danh mục chi tiết hơn về các kỹ thuật và định dạng tuần tự hóa dữ liệu phổ biến trong Phụ lục A. Chúng tôi đề xuất rằng kỹ sư dữ liệu nên làm quen với các thực hành và định dạng tuần tự hóa phổ biến, đặc biệt là các định dạng phổ biến nhất hiện nay (ví dụ, Apache Parquet), tuần tự hóa lai (ví dụ, Apache Hudi) và tuần tự hóa trong bộ nhớ (ví dụ, Apache Arrow).

Compression — Nén

Compression is another critical component of storage engineering. On a basic level, compression makes data smaller, but compression algorithms interact with other details of storage systems in complex ways. Highly efficient compression has three main advantages in storage systems. First, the data is smaller and thus takes up less space on the disk. Second, compression increases the practical scan speed per disk. With a 10:1 compression ratio, we go from scanning 200 MB/s per magnetic disk to an effective rate of 2 GB/s per disk. The third advantage is in network performance. Given that a network connection between an Amazon EC2 instance and S3 provides 10 gigabits per second (Gbps) of bandwidth, a 10:1 compression ratio increases effective network bandwidth to 100 Gbps. Compression also comes with disadvantages. Compressing and decompressing data entails extra time and resource consumption to read or write data. We undertake a more detailed discussion of compression algorithms and trade-offs in Appendix A .

Nén (compression) là một thành phần then chốt khác của kỹ thuật lưu trữ. Ở mức cơ bản, nén làm dữ liệu nhỏ hơn, nhưng các thuật toán nén tương tác với các chi tiết khác của hệ thống lưu trữ theo những cách phức tạp. Nén hiệu quả cao có ba lợi thế chính trong các hệ thống lưu trữ. Thứ nhất, dữ liệu nhỏ hơn và do đó chiếm ít không gian hơn trên đĩa. Thứ hai, nén làm tăng tốc độ quét thực tế trên mỗi đĩa. Với tỷ lệ nén 10:1, chúng ta đi từ quét 200 MB/s trên mỗi đĩa từ lên tốc độ hiệu dụng 2 GB/s trên mỗi đĩa. Lợi thế thứ ba là về hiệu năng mạng. Cho rằng một kết nối mạng giữa một phiên bản (instance) Amazon EC2 và S3 cung cấp băng thông 10 gigabit mỗi giây (Gbps), tỷ lệ nén 10:1 làm tăng băng thông mạng hiệu dụng lên 100 Gbps. Nén cũng đi kèm những bất lợi. Việc nén và giải nén dữ liệu kéo theo thời gian và tiêu thụ tài nguyên bổ sung để đọc hoặc ghi dữ liệu. Chúng tôi thực hiện một thảo luận chi tiết hơn về các thuật toán nén và những đánh đổi trong Phụ lục A.

Caching — Caching

We've already mentioned caching in our discussion of RAM. The core idea of caching is to store frequently or recently accessed data in a fast access layer. The faster the cache, the higher the cost and the less storage space available. Less frequently accessed data is stored in cheaper, slower storage. Caches are critical for data serving, processing, and transformation. As we analyze storage systems, it is helpful to put every type of storage we utilize inside a cache hierarchy (Table 6-1). Most practical data systems rely on many cache layers assembled from storage with varying performance characteristics. This starts inside CPUs; processors may deploy up to four cache tiers. We move down the hierarchy to RAM and SSDs. Cloud object storage is a lower tier that supports long-term **data retention** and durability while allowing for data serving and dynamic data movement in pipelines.

Chúng ta đã đề cập đến caching trong phần thảo luận về RAM. Ý tưởng cốt lõi của caching là lưu trữ dữ liệu được truy cập thường xuyên hoặc gần đây trong một tầng truy cập nhanh. Cache càng nhanh thì chi phí càng cao và không gian lưu trữ sẵn có càng ít. Dữ liệu ít được truy cập hơn được lưu trong lưu trữ rẻ hơn, chậm hơn. Cache là yếu tố then chốt cho việc phục vụ, xử lý và biến đổi dữ liệu. Khi phân tích các hệ thống lưu trữ, sẽ hữu ích nếu đặt mọi loại lưu trữ chúng ta sử dụng vào một phân cấp cache (Bảng 6-1). Hầu hết các hệ thống dữ liệu thực tế dựa vào nhiều tầng cache được lắp ráp từ lưu trữ với các đặc tính hiệu năng khác nhau. Điều này bắt đầu ngay bên trong các CPU; các bộ xử lý có thể triển khai tới bốn tầng cache. Chúng ta đi xuống phân cấp đến RAM và SSD. Lưu trữ đối tượng đám mây là một tầng thấp hơn, hỗ trợ lưu giữ và độ bền dữ liệu dài hạn đồng thời cho phép phục vụ dữ liệu và di chuyển dữ liệu động trong các đường ống.

Table 6-1 . A

Bảng 6-1. Một

heuristic cache hierarchy displaying storage types with approximate pricing and performance characteristics

Storage type	Storage type	Data fetch latency	Bandwidth	Data fetch latency	Bandwidth	aa	Price	Price
--------------	--------------	--------------------	-----------	--------------------	-----------	----	-------	-------

phân cấp cache mang tính kinh nghiệm (heuristic) hiển thị các loại lưu trữ cùng định giá xấp xỉ và các đặc tính hiệu năng. Loại lưu trữ Loại lưu trữ Độ trễ lấy dữ liệu Băng thông Độ trễ lấy dữ liệu Băng thông aa Giá Giá

CPU cache 1 nanosecond 1 TB/s RAM 0.1 microseconds 100 GB/s \$10/GB SSD 0.1 milliseconds 4 GB/s \$0.20/GB HDD 4 milliseconds 300 MB/s \$0.03/GB Object storage 100 milliseconds 3 GB/s per instance \$0.02/GB per month Archival storage 12 hours Same as object storage once data is \$0.004/GB per available month

CPU cache 1 nano-giây 1 TB/s RAM 0,1 micro-giây 100 GB/s 10 USD/GB SSD 0,1 mili-giây 4 GB/s 0,20 USD/GB HDD 4 mili-giây 300 MB/s 0,03 USD/GB Lưu trữ đối tượng 100 mili-giây 3 GB/s mỗi instance 0,02 USD/GB mỗi tháng Lưu trữ archival 12 giờ Giống như lưu trữ đối tượng một khi dữ liệu 0,004 USD/GB mỗi đã sẵn có tháng

a A microsecond is 1,000 nanoseconds, and a millisecond is 1,000 microseconds.

a Một micro-giây là 1.000 nano-giây, và một mili-giây là 1.000 micro-giây.

We can think of archival storage as a reverse cache . Archival storage provides inferior access characteristics for low costs. Archival storage is generally used for data backups and to meet data-retention compliance requirements. In typical scenarios, this data will be accessed only in an emergency (e.g., data in a database might be lost and need to be recovered, or a company might need to look back at historical data for legal discovery).

Chúng ta có thể coi lưu trữ archival (archival storage) như một cache ngược. Lưu trữ archival cung cấp các đặc tính truy cập kém hơn để đổi lấy chi phí thấp. Lưu trữ archival nhìn chung được dùng cho sao lưu dữ liệu và để đáp ứng các yêu cầu tuân thủ về lưu giữ dữ liệu (data retention). Trong các kịch bản điển hình, dữ liệu này sẽ chỉ được truy cập trong trường hợp khẩn cấp (ví dụ, dữ liệu trong một cơ sở dữ liệu có thể bị mất và cần được khôi phục, hoặc một công ty có thể cần xem lại dữ liệu lịch sử cho việc khám phá chứng cứ pháp lý).

Data Storage Systems — Các hệ thống lưu trữ dữ liệu

This section covers the major data storage systems you’ll encounter as a **data engineer**. Storage systems exist at a level of abstraction above raw ingredients. For example, magnetic disks are a raw storage ingredient, while major cloud object storage platforms and HDFS are storage systems that utilize magnetic disks. Still higher levels of **storage abstraction** exist, such as data lakes and lakehouses (which we cover in “Data Engineering Storage Abstractions”).

Mục này trình bày các hệ thống lưu trữ dữ liệu chính mà bạn sẽ gặp với tư cách là một kỹ sư dữ liệu. Các hệ thống lưu trữ tồn tại ở một mức trừu tượng cao hơn các nguyên liệu thô. Chẳng hạn, đĩa từ là một nguyên liệu lưu trữ thô, trong khi các nền tảng lưu trữ đối tượng đám mây chính và HDFS là các hệ thống lưu trữ sử dụng đĩa từ. Vẫn còn tồn tại các mức trừu tượng lưu trữ cao hơn nữa, chẳng hạn như hồ dữ liệu và data lakehouse (mà chúng ta trình bày trong “Các trừu tượng lưu trữ cho kỹ thuật dữ liệu”).

Single Machine Versus Distributed Storage — Lưu trữ trên một máy đơn so với lưu trữ phân tán

As data storage and access patterns become more complex and outgrow the usefulness of a single server, distributing data to more than one server becomes necessary. Data can be stored on multiple servers, known as distributed storage . This is a **distributed system** whose purpose is to store data in a distributed fashion (Figure 6-4).

Khi các mẫu hình lưu trữ và truy cập dữ liệu trở nên phức tạp hơn và vượt quá khả năng hữu dụng của một máy chủ đơn, việc phân tán dữ liệu ra nhiều hơn một máy chủ trở nên cần thiết. Dữ liệu có thể được lưu trên nhiều máy chủ, gọi là lưu trữ phân tán (distributed storage). Đây là một hệ phân tán có mục đích là lưu trữ dữ liệu theo cách phân tán (Hình 6-4).

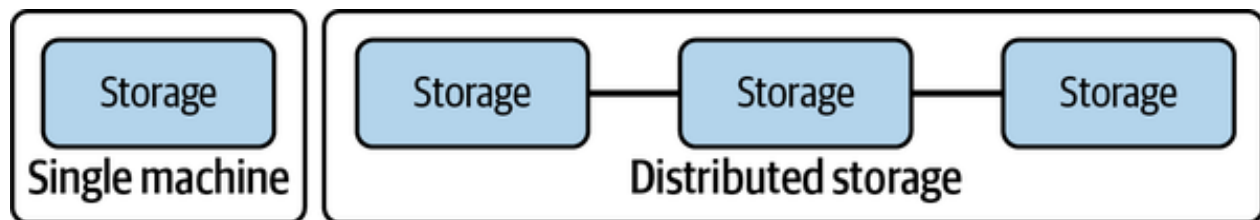


Figure 6-4. Single machine versus distributed storage on multiple servers Distributed storage coordinates the activities of multiple servers to store, retrieve, and process data faster and at a larger scale, all while providing redundancy in case a server becomes unavailable. Distributed storage is common in architectures where you want built-in redundancy and **scalability** for large amounts of data. For example, object storage, Apache Spark, and cloud data warehouses rely on distributed storage architectures. Data engineers must always be aware of the consistency paradigms of the distributed systems, which we’ll explore next.

Hình 6-4. Lưu trữ trên một máy đơn so với lưu trữ phân tán trên nhiều máy chủ. Lưu trữ phân tán điều phối hoạt động của nhiều máy chủ để lưu trữ, truy xuất và xử lý dữ liệu nhanh hơn và ở quy mô lớn hơn, đồng thời cung cấp khả năng dự phòng trong trường hợp một máy chủ trở nên không sẵn sàng. Lưu trữ phân tán phổ biến trong các kiến trúc nơi bạn muốn có khả năng dự phòng tích hợp sẵn và khả năng mở rộng cho khối lượng dữ liệu lớn. Chẳng hạn, lưu trữ đối tượng, Apache Spark và các kho dữ liệu đám mây dựa vào các kiến trúc lưu trữ phân tán. Kỹ sư dữ liệu phải luôn nhận thức được các mô hình nhất quán (consistency) của các hệ phân tán, mà chúng ta sẽ khám phá tiếp theo.

Eventual Versus Strong Consistency — Nhất quán cuối cùng so với nhất quán mạnh

A challenge with distributed systems is that your data is spread across multiple servers. How does this system keep the data consistent? Unfortunately, distributed systems pose a dilemma for storage and query accuracy. It takes time to replicate changes across the nodes of a system; often a balance exists between getting current data and getting “sorta” current data in a distributed database. Let’s look at two common consistency patterns in distributed systems: eventual and strong. We’ve covered ACID compliance throughout this book, starting in Chapter 5 . Another acronym is BASE , which stands for basically available, soft-state, eventual consistency . Think of it as the opposite of ACID. BASE is the basis of eventual consistency. Let’s briefly explore its components:

Một thách thức với các hệ phân tán là dữ liệu của bạn được trải ra trên nhiều máy chủ. Hệ thống này giữ cho dữ liệu nhất quán bằng cách nào? Tiếc thay, các hệ phân tán đặt ra một thể tiến thoái lưỡng nan cho độ chính xác của lưu trữ và truy vấn. Cần có thời gian để sao chép các thay đổi qua các nút của một hệ thống; thường tồn tại một sự cân bằng giữa việc lấy dữ liệu hiện thời và lấy dữ liệu “kiểu như” hiện thời trong một cơ sở dữ liệu phân tán. Hãy cùng xem hai mẫu hình nhất quán phổ biến trong các hệ phân tán: nhất quán cuối cùng (eventual) và nhất quán mạnh (strong). Chúng ta đã đề cập đến việc tuân thủ ACID xuyên suốt cuốn sách này, bắt đầu từ Chương 5. Một từ viết tắt khác là BASE, viết tắt của basically available, soft-state, eventual consistency (về cơ bản là sẵn sàng, trạng thái mềm, nhất quán cuối cùng). Hãy coi nó như đối lập với ACID. BASE là nền tảng của nhất quán cuối cùng. Hãy cùng khám phá ngắn gọn các thành phần của nó:

Basically available

Basically available (Về cơ bản là sẵn sàng)

Consistency is not guaranteed, but efforts at database reads and writes are made on a best-effort basis, meaning consistent data is available most of the time.

Nhất quán không được đảm bảo, nhưng các nỗ lực đọc và ghi cơ sở dữ liệu được thực hiện trên cơ sở nỗ lực tốt nhất (best-effort), nghĩa là dữ liệu nhất quán có sẵn trong phần lớn thời gian.

Soft-state

Soft-state (Trạng thái mềm)

The state of the transaction is fuzzy, and it's uncertain whether the transaction is committed or uncommitted.

Trạng thái của giao dịch là mơ hồ, và không chắc chắn liệu giao dịch đã được cam kết (commit) hay chưa cam kết.

Eventual consistency

Eventual consistency (Nhất quán cuối cùng)

At some point, reading data will return consistent values.

Tại một thời điểm nào đó, việc đọc dữ liệu sẽ trả về các giá trị nhất quán.

If reading data in an eventually consistent system is unreliable, why use it? Eventual consistency is a common trade-off in large-scale, distributed systems. If you want to scale horizontally (across multiple nodes) to process data in high volumes, then eventually, consistency is often the price you'll pay. Eventual consistency allows you to retrieve data quickly without verifying that you have the latest version across all nodes. The opposite of eventual consistency is strong consistency. With strong consistency, the distributed database ensures that writes to any node are first distributed with a consensus and that any reads against the database return consistent values. You'll use strong consistency when you can tolerate higher query latency and require correct data every time you read from the database. Generally, data engineers make decisions about consistency in three places. First, the database technology itself sets the stage for a certain level of consistency. Second, configuration parameters for the database will have an impact on consistency. Third, databases often support some consistency configuration at an individual query level. For example, DynamoDB supports eventually consistent reads and strongly consistent reads. Strongly consistent reads are slower and consume more resources, so it is best to use them sparingly, but they are available when consistency is required. You should understand how your database handles consistency. Again, data engineers are tasked with understanding technology deeply and using it to solve problems

appropriately. A data engineer might need to negotiate consistency requirements with other technical and business stakeholders. Note that this is both a technology and organizational problem; ensure that you have gathered requirements from your stakeholders and choose your technologies appropriately.

Nếu việc đọc dữ liệu trong một hệ thống nhất quán cuối cùng là không đáng tin cậy, vậy tại sao lại dùng nó? Nhất quán cuối cùng là một đánh đổi phổ biến trong các hệ phân tán quy mô lớn. Nếu bạn muốn mở rộng theo chiều ngang (qua nhiều nút) để xử lý dữ liệu với khối lượng lớn, thì cuối cùng nhất quán thường là cái giá bạn phải trả. Nhất quán cuối cùng cho phép bạn truy xuất dữ liệu nhanh chóng mà không cần xác minh rằng bạn có phiên bản mới nhất trên tất cả các nút. Đối lập với nhất quán cuối cùng là nhất quán mạnh. Với nhất quán mạnh, cơ sở dữ liệu phân tán đảm bảo rằng các thao tác ghi vào bất kỳ nút nào đều được phân phối trước tiên với một sự đồng thuận (consensus) và rằng bất kỳ thao tác đọc nào đối với cơ sở dữ liệu đều trả về các giá trị nhất quán. Bạn sẽ dùng nhất quán mạnh khi bạn có thể chịu được độ trễ truy vấn cao hơn và đòi hỏi dữ liệu chính xác mỗi lần bạn đọc từ cơ sở dữ liệu. Nhìn chung, kỹ sư dữ liệu đưa ra các quyết định về nhất quán ở ba nơi. Thứ nhất, bản thân công nghệ cơ sở dữ liệu định ra mức độ nhất quán nhất định. Thứ hai, các tham số cấu hình cho cơ sở dữ liệu sẽ có tác động đến nhất quán. Thứ ba, các cơ sở dữ liệu thường hỗ trợ một số cấu hình nhất quán ở mức truy vấn riêng lẻ. Chẳng hạn, DynamoDB hỗ trợ các thao tác đọc nhất quán cuối cùng và các thao tác đọc nhất quán mạnh. Các thao tác đọc nhất quán mạnh chậm hơn và tiêu tốn nhiều tài nguyên hơn, nên tốt nhất là dùng chúng một cách tiết kiệm, nhưng chúng có sẵn khi nhất quán là yêu cầu bắt buộc. Bạn nên hiểu cơ sở dữ liệu của mình xử lý nhất quán như thế nào. Một lần nữa, kỹ sư dữ liệu có nhiệm vụ hiểu sâu công nghệ và sử dụng nó để giải quyết vấn đề một cách thích hợp. Một kỹ sư dữ liệu có thể cần thương lượng các yêu cầu về nhất quán với các bên liên quan kỹ thuật và nghiệp vụ khác. Lưu ý rằng đây vừa là một vấn đề công nghệ vừa là một vấn đề tổ chức; hãy đảm bảo rằng bạn đã thu thập các yêu cầu từ các bên liên quan của mình và chọn công nghệ một cách thích hợp.

File Storage — Lưu trữ tệp

We deal with files every day, but the notion of a file is somewhat subtle. A file is a data entity with specific read, write, and reference characteristics used by software and operating systems. We define a file to have the following characteristics: Finite length

Chúng ta xử lý các tệp mỗi ngày, nhưng khái niệm về một tệp có phần tinh tế. Một tệp là một thực thể dữ liệu với các đặc tính đọc, ghi và tham chiếu cụ thể được phần mềm và hệ điều hành sử dụng. Chúng tôi định nghĩa một tệp có các đặc tính sau: Độ dài hữu hạn

A file is a finite-length stream of bytes.

Một tệp là một luồng byte có độ dài hữu hạn.

Append operations

Thao tác nối thêm (append)

We can append bytes to the file up to the limits of the host storage system.

Chúng ta có thể nối thêm các byte vào tệp cho đến giới hạn của hệ thống lưu trữ máy chủ.

Random access

Truy cập ngẫu nhiên

We can read from any location in the file or write updates to any location.

Chúng ta có thể đọc từ bất kỳ vị trí nào trong tệp hoặc ghi cập nhật vào bất kỳ vị trí nào.

Object storage behaves much like **file storage** but with key differences. While we set the stage for object storage by discussing file storage first, object storage is arguably much more important for the type of data engineering you'll do today. We will forward-reference the object storage discussion extensively over the next few pages. File storage systems organize files into a directory tree. The directory reference for a file might look like this:

Lưu trữ đối tượng hành xử khá giống lưu trữ tệp nhưng với những khác biệt then chốt. Mặc dù chúng ta đặt nền cho lưu trữ đối tượng bằng cách bàn về lưu trữ tệp trước, lưu trữ đối tượng được cho là quan trọng hơn nhiều đối với kiểu kỹ thuật dữ liệu mà bạn sẽ làm ngày nay. Chúng ta sẽ tham chiếu tới phần thảo luận về lưu trữ đối tượng một cách rộng rãi trong vài trang tới. Các hệ thống lưu trữ tệp tổ chức các tệp thành một cây thư mục. Tham chiếu thư mục cho một tệp có thể trông như thế này:

When I provide this to the operating system, it starts at the root directory / , finds Users , matthewhousley , and finally output.txt . Working from the left, each directory is contained inside a parent directory, until we finally reach the file output.txt . This example uses Unix semantics, but Windows file reference semantics are similar. The filesystem stores each directory as **metadata** about the files and directories that it contains. This metadata consists of the name of each entity, relevant permission details, and a pointer to the actual entity. To find a file on disk, the operating system looks at the metadata at each hierarchy level and follows the pointer to the next subdirectory entity until finally reaching the file itself. Note that other file-like data entities generally don't necessarily have all these properties. For example, objects in object storage support only the first characteristic, finite length, but are still extremely useful. We discuss this in “Object Storage” . In cases where file storage paradigms are necessary for a pipeline, be careful with state and try to use ephemeral environments as much as possible. Even if you must process files on a server with an attached disk, use object storage for intermediate storage between processing steps. Try to reserve manual, low-level file processing for one-time ingestion steps or the exploratory stages of pipeline development.

Khi tôi cung cấp tham chiếu này cho hệ điều hành, nó bắt đầu từ thư mục gốc / , tìm thấy Users, matthewhousley, và cuối cùng là output.txt. Làm việc từ bên trái, mỗi thư mục được chứa bên trong một thư mục cha, cho đến khi cuối cùng chúng ta đến tệp output.txt. Ví dụ này sử dụng ngữ nghĩa Unix, nhưng ngữ nghĩa tham chiếu tệp của Windows cũng tương tự. Hệ thống tệp lưu trữ mỗi thư mục như siêu dữ liệu (metadata) về các tệp và thư mục mà nó chứa. Siêu dữ liệu này bao gồm tên của mỗi thực thể, các chi tiết về quyền liên quan và một con trỏ đến thực thể thực tế. Để tìm một tệp trên đĩa, hệ điều hành xem siêu dữ liệu ở mỗi mức phân cấp và đi theo con trỏ đến thực thể thư mục con tiếp theo cho đến khi cuối cùng đến chính tệp đó. Lưu ý rằng các thực thể dữ liệu giống tệp khác nhìn chung không nhất thiết có tất cả các thuộc tính này. Chẳng hạn, các đối tượng trong lưu trữ đối tượng chỉ hỗ trợ đặc tính đầu tiên, độ dài hữu hạn, nhưng vẫn cực kỳ hữu ích. Chúng ta bàn về điều này trong “Lưu trữ đối tượng”. Trong các trường hợp mà các mô hình lưu trữ tệp là cần thiết cho một đường ống, hãy cẩn thận với trạng thái và cố gắng dùng các môi trường tạm thời (ephemeral) càng nhiều càng tốt. Ngay cả khi bạn phải xử lý các tệp trên một máy chủ có đĩa gắn kèm, hãy dùng lưu trữ đối tượng cho lưu trữ trung gian giữa các bước xử lý. Hãy cố gắng dành các bước xử lý tệp thủ công ở mức thấp cho các bước thu nạp một lần hoặc các giai đoạn khám phá của việc phát triển đường ống.

Local disk storage — Lưu trữ đĩa cục bộ

The most familiar type of file storage is an operating system–managed filesystem on a local disk **partition** of SSD or magnetic disk. New Technology File System (NTFS) and ext4 are popular filesystems on Windows and Linux, respectively. The operating system handles the details of storing directory entities, files, and metadata. Filesystems are designed to write data to allow for easy recovery in the event of power loss during a write, though any unwritten data will still be lost. Local filesystems generally support full read after write consistency; reading immediately after a write will return the written data. Operating systems also employ various locking strategies to manage concurrent writing attempts to a file. Local disk filesystems may also support advanced features such as journaling, snapshots, redundancy, the extension of the filesystem across multiple disks, full disk **encryption**, and compression. In “**Block Storage**”, we also discuss RAID.

Loại lưu trữ tệp quen thuộc nhất là một hệ thống tệp do hệ điều hành quản lý trên một phân vùng đĩa cục bộ của SSD hoặc đĩa từ. New Technology File System (NTFS) và ext4 là các hệ thống tệp phổ biến trên Windows và Linux, tương ứng. Hệ điều hành xử lý các chi tiết về lưu trữ các thực thể thư mục, tệp và siêu dữ liệu. Các hệ thống tệp được thiết kế để ghi dữ liệu nhằm cho phép khôi phục dễ dàng trong trường hợp mất nguồn trong khi đang ghi, mặc dù bất kỳ dữ liệu nào chưa được ghi vẫn sẽ bị mất. Các hệ thống tệp cục bộ nhìn chung hỗ trợ nhất quán đọc-sau-ghi đầy đủ (full read after write consistency); đọc ngay sau khi ghi sẽ trả về dữ liệu đã được ghi. Các hệ điều hành cũng sử dụng nhiều chiến lược khóa (locking) khác nhau để quản lý các nỗ lực ghi đồng thời vào một tệp. Các hệ thống tệp đĩa cục bộ cũng có thể hỗ trợ các tính năng nâng cao như nhật ký hóa (journaling), ảnh chụp nhanh (snapshot), dự phòng, mở rộng hệ thống tệp qua nhiều đĩa, mã hóa toàn đĩa và nén. Trong “Lưu trữ khối”, chúng ta cũng bàn về RAID.

Network-attached storage — Lưu trữ gắn mạng

Network-attached storage (NAS) systems provide a file storage system to clients over a network. NAS is a prevalent solution for servers; they quite often ship with built-in dedicated NAS interface hardware. While there are performance penalties to accessing the filesystem over a network, significant advantages to storage virtualization also exist, including redundancy and reliability, fine-grained control of resources, storage pooling across multiple disks for large virtual volumes, and file sharing across multiple machines. Engineers should be aware of the consistency model provided by their NAS solution, especially when multiple clients will potentially access the same data. A popular alternative to NAS is a storage area network (SAN), but SAN systems provide block-level access without the filesystem abstraction. We cover SAN systems in “Block Storage”.

Các hệ thống lưu trữ gắn mạng (network-attached storage, NAS) cung cấp một hệ thống lưu trữ tệp cho các máy khách (client) qua một mạng. NAS là một giải pháp phổ biến cho các máy chủ; chúng khá thường được xuất xưởng với phần cứng giao tiếp NAS chuyên dụng tích hợp sẵn. Mặc dù có những hình phạt về hiệu năng khi truy cập hệ thống tệp qua một mạng, cũng tồn tại những lợi thế đáng kể của ảo hóa lưu trữ, bao gồm dự phòng và độ tin cậy, kiểm soát tài nguyên ở mức chi tiết, gộp lưu trữ (storage pooling) qua nhiều đĩa cho các khối lượng ảo lớn, và chia sẻ tệp qua nhiều máy. Kỹ sư nên nhận thức được mô hình nhất quán mà giải pháp NAS của họ cung cấp, đặc biệt khi nhiều máy khách có khả năng truy cập cùng một dữ liệu. Một lựa chọn thay thế phổ biến cho NAS là mạng lưu trữ (storage area network, SAN), nhưng các hệ thống SAN cung cấp truy cập ở mức khối (block-level) mà không có trừu tượng hệ thống tệp. Chúng ta trình bày về các hệ thống SAN trong “Lưu trữ khối”.

Cloud file system services — Các dịch vụ hệ thống tệp đám mây

Cloud filesystem services provide a fully managed filesystem for use with multiple cloud VMs and applications, potentially including clients outside the cloud environment. Cloud filesystems should not be confused with standard storage attached to VMs—generally, block storage with a filesystem managed by the VM operating system. Cloud filesystems behave much like NAS solutions, but the details of networking, managing disk clusters, failures, and configuration are fully handled by the cloud vendor. For example, Amazon Elastic File System (EFS) is an extremely popular example of a cloud filesystem service. Storage is exposed through the NFS 4 protocol, which is also used by NAS systems. EFS provides automatic scaling and pay-per-storage pricing with no advanced storage reservation required. The service also provides local read-after-write consistency (when reading from the machine that performed the write). It also offers open-after-close consistency across the full filesystem. In other words, once an application closes a file, subsequent readers will see changes saved to the closed file.

Các dịch vụ hệ thống tệp đám mây cung cấp một hệ thống tệp được quản lý hoàn toàn để dùng với nhiều máy ảo (VM) và ứng dụng đám mây, có thể bao gồm cả các máy khách bên ngoài môi trường đám mây. Không nên nhầm lẫn các hệ thống tệp đám mây với lưu trữ tiêu chuẩn gắn vào các VM—nhìn chung là lưu trữ khối với một hệ thống tệp được quản lý bởi hệ điều hành của VM. Các hệ thống tệp đám mây hành xử khá giống các giải pháp NAS, nhưng các chi tiết về mạng, quản lý các cụm đĩa, các sự cố và cấu hình được nhà cung cấp đám mây xử lý hoàn toàn. Chẳng hạn, Amazon Elastic File System (EFS) là một ví dụ cực kỳ phổ biến về một dịch vụ hệ thống tệp đám mây. Lưu trữ được phơi bày thông qua giao thức NFS 4, vốn cũng được các hệ thống NAS sử dụng. EFS cung cấp khả năng tự động mở rộng và định giá trả theo lượng lưu trữ mà không cần đặt trước lưu trữ. Dịch vụ này cũng cung cấp nhất quán đọc-sau-ghi cục bộ (khi đọc từ máy đã thực hiện thao tác ghi). Nó cũng cung cấp nhất quán mở-sau-đóng (open-after-close consistency) trên toàn bộ hệ thống tệp. Nói cách khác, một khi một ứng dụng đóng một tệp, các bên đọc về sau sẽ thấy các thay đổi đã được lưu vào tệp đã đóng.

Block Storage — Lưu trữ khối

Fundamentally, block storage is the type of raw storage provided by SSDs and magnetic disks. In the cloud, virtualized block storage is the standard for VMs. These block storage abstractions allow fine control of storage size, scalability, and data durability beyond that offered by raw disks. In our earlier discussion of SSDs and magnetic disks, we mentioned that with these random-access devices, the operating system can seek, read, and write any data on the disk. A block is the smallest addressable unit of data supported by a disk. This was often 512 bytes of usable data on older disks, but it has now grown to 4,096 bytes for most current disks, making writes less fine-grained but dramatically reducing the overhead of managing blocks. Blocks typically contain extra bits for error detection/correction and other metadata. Blocks on magnetic disks are geometrically arranged on a physical platter. Two blocks on the same track can be read without moving the head, while reading two blocks on separate tracks requires a seek. Seek time can occur between blocks on an SSD, but this is infinitesimal compared to the seek time for magnetic disk tracks.

Về cơ bản, lưu trữ khối (block storage) là loại lưu trữ thô được cung cấp bởi SSD và đĩa từ. Trên đám mây, lưu trữ khối ảo hóa là tiêu chuẩn cho các VM. Các trừu tượng lưu trữ khối này cho phép kiểm soát ở mức chi tiết về kích thước lưu trữ, khả năng mở rộng và độ bền dữ liệu vượt ngoài những gì các đĩa thô cung cấp. Trong phần thảo luận trước về SSD và đĩa từ, chúng ta đã đề cập rằng với các thiết bị truy cập ngẫu nhiên này, hệ điều hành có

thể tìm kiếm (seek), đọc và ghi bất kỳ dữ liệu nào trên đĩa. Một khối (block) là đơn vị dữ liệu nhỏ nhất có thể định địa chỉ được mà một đĩa hỗ trợ. Đây thường là 512 byte dữ liệu khả dụng trên các đĩa cũ, nhưng giờ đã tăng lên 4.096 byte đối với hầu hết các đĩa hiện nay, khiến các thao tác ghi kém chi tiết hơn nhưng giảm đáng kể chi phí quản lý các khối. Các khối thường chứa các bit bổ sung để phát hiện/sửa lỗi và các siêu dữ liệu khác. Các khối trên đĩa từ được sắp xếp hình học trên một đĩa platter vật lý. Hai khối trên cùng một rãnh có thể được đọc mà không cần di chuyển đầu đọc, trong khi đọc hai khối trên các rãnh riêng biệt đòi hỏi một thao tác tìm kiếm. Thời gian tìm kiếm có thể xảy ra giữa các khối trên một SSD, nhưng nó nhỏ vô cùng so với thời gian tìm kiếm giữa các rãnh đĩa từ.

Block storage applications — Các ứng dụng của lưu trữ khối

Transactional database systems generally access disks at a block level to lay out data for optimal performance. For row-oriented databases, this originally meant that rows of data were written as continuous streams; the situation has grown more complicated with the arrival of SSDs and their associated seek-time performance improvements, but transactional databases still rely on the high random access performance offered by direct access to a block storage device. Block storage also remains the default option for operating system boot disks on cloud VMs. The block device is formatted much as it would be directly on a physical disk, but the storage is usually virtualized. (see “Cloud virtualized block storage”.)

Các hệ thống cơ sở dữ liệu giao dịch nhìn chung truy cập các đĩa ở mức khối để bố trí dữ liệu nhằm đạt hiệu năng tối ưu. Đối với các cơ sở dữ liệu định hướng hàng, ban đầu điều này có nghĩa là các hàng dữ liệu được ghi dưới dạng các luồng liên tục; tình hình đã trở nên phức tạp hơn với sự xuất hiện của SSD và những cải tiến hiệu năng về thời gian tìm kiếm liên quan của chúng, nhưng các cơ sở dữ liệu giao dịch vẫn dựa vào hiệu năng truy cập ngẫu nhiên cao mà việc truy cập trực tiếp vào một thiết bị lưu trữ khối mang lại. Lưu trữ khối cũng vẫn là lựa chọn mặc định cho các đĩa khởi động (boot disk) của hệ điều hành trên các VM đám mây. Thiết bị khối được định dạng giống hệt như khi nó nằm trực tiếp trên một đĩa vật lý, nhưng lưu trữ thường được ảo hóa. (xem “Lưu trữ khối ảo hóa đám mây”).

RAID — RAID

RAID stands for redundant array of independent disks , as noted previously. RAID simultaneously controls multiple disks to improve data durability, enhance performance, and combine capacity from multiple drives. An array can appear to the operating system as a single block device. Many encoding and parity schemes are available, depending on the desired balance between enhanced effective bandwidth and higher fault tolerance (tolerance for many disk failures).

RAID là viết tắt của redundant array of independent disks (mảng đĩa độc lập dự phòng), như đã lưu ý trước đó. RAID đồng thời điều khiển nhiều đĩa để cải thiện độ bền dữ liệu, nâng cao hiệu năng và kết hợp dung lượng từ nhiều ổ đĩa. Một mảng có thể xuất hiện với hệ điều hành như một thiết bị khối đơn lẻ. Có sẵn nhiều lược đồ mã hóa và chẵn lẻ (parity), tùy thuộc vào sự cân bằng mong muốn giữa băng thông hiệu dụng được nâng cao và khả năng chịu lỗi cao hơn (khả năng chịu được nhiều đĩa hỏng).

Storage area network — Mạng lưu trữ

Storage area network (SAN) systems provide virtualized block storage devices over a network, typically from a storage pool. SAN abstraction can allow fine-grained storage scaling and enhance performance, availability, and durability. You might encounter SAN systems if you're working with on-premises storage systems; you might also encounter a cloud version of SAN, as in the next subsection.

Các hệ thống mạng lưu trữ (storage area network, SAN) cung cấp các thiết bị lưu trữ khối ảo hóa qua một mạng, thường là từ một nhóm lưu trữ (storage pool). Trừu tượng SAN có thể cho phép mở rộng lưu trữ ở mức chi tiết và nâng cao hiệu năng, tính sẵn sàng và độ bền. Bạn có thể gặp các hệ thống SAN nếu bạn làm việc với các hệ thống lưu trữ tại chỗ (on-premises); bạn cũng có thể gặp một phiên bản đám mây của SAN, như trong tiểu mục tiếp theo.

Cloud virtualized block storage — Lưu trữ khối ảo hóa đám mây

Cloud virtualized block storage solutions are similar to SAN but free engineers from dealing with SAN clusters and networking details. We'll look at Amazon Elastic Block Store (EBS) as a standard example; other public clouds have similar offerings. EBS is the default storage for Amazon EC2 virtual machines; other cloud providers also treat virtualized object storage as a key component of their VM offerings. EBS offers several tiers of service with different performance characteristics. Generally, EBS performance metrics are given in IOPS and **throughput** (transfer speed). The higher performance tiers of EBS storage are backed by SSD disks, while magnetic disk-backed storage offers lower IOPS but costs less per gigabyte. EBS volumes store data separate from the instance host server but in the same zone to support high performance and low latency (Figure 6-5). This allows EBS volumes to persist when an EC2 instance shuts down, when a host server fails, or even when the instance is deleted. EBS storage is suitable for applications such as databases, where data durability is a high priority. In addition, EBS replicates all data to at least two separate host machines, protecting data if a disk fails.

Các giải pháp lưu trữ khối ảo hóa đám mây tương tự SAN nhưng giải phóng kỹ sư khỏi việc phải xử lý các cụm SAN và các chi tiết về mạng. Chúng ta sẽ xem xét Amazon Elastic Block Store (EBS) như một ví dụ tiêu chuẩn; các đám mây công cộng khác có những sản phẩm tương tự. EBS là lưu trữ mặc định cho các máy ảo Amazon EC2; các nhà cung cấp đám mây khác cũng coi lưu trữ đối tượng ảo hóa là một thành phần then chốt trong các sản phẩm VM của họ. EBS cung cấp một số tầng dịch vụ với các đặc tính hiệu năng khác nhau. Nhìn chung, các chỉ số hiệu năng của EBS được đưa ra theo IOPS và thông lượng (throughput, tốc độ truyền). Các tầng hiệu năng cao hơn của lưu trữ EBS được hậu thuẫn bởi các đĩa SSD, trong khi lưu trữ được hậu thuẫn bởi đĩa từ cung cấp IOPS thấp hơn nhưng chi phí trên mỗi gigabyte ít hơn. Các khối (volume) EBS lưu trữ dữ liệu tách biệt khỏi máy chủ host của instance nhưng trong cùng một vùng để hỗ trợ hiệu năng cao và độ trễ thấp (Hình 6-5). Điều này cho phép các khối EBS tồn tại bền vững khi một instance EC2 tắt, khi một máy chủ host hỏng, hoặc thậm chí khi instance bị xóa. Lưu trữ EBS phù hợp cho các ứng dụng như cơ sở dữ liệu, nơi độ bền dữ liệu là ưu tiên cao. Ngoài ra, EBS sao chép toàn bộ dữ liệu tới ít nhất hai máy host riêng biệt, bảo vệ dữ liệu nếu một đĩa hỏng.

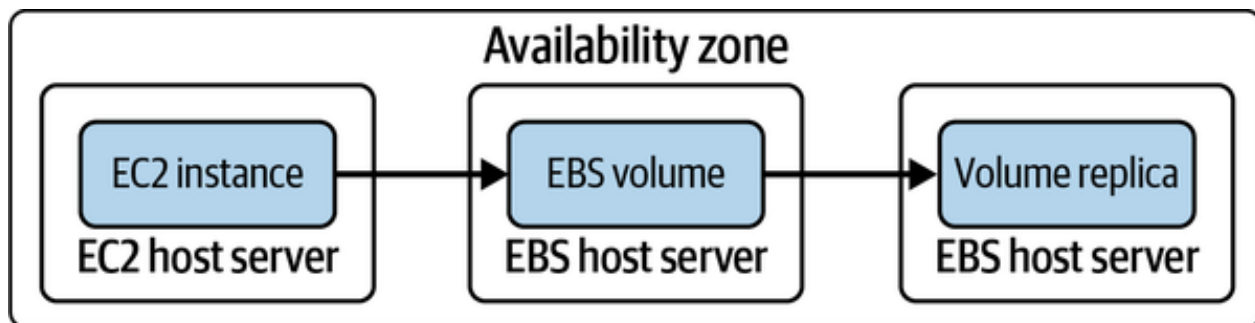


Figure 6-5. EBS volumes replicate data to multiple hosts and disks for high durability and availability, but are not resilient to the failure of an availability zone. EBS storage virtualization also supports several advanced features. For example, EBS volumes allow instantaneous point-in-time snapshots while the drive is used. Although it still takes some time for the snapshot to be replicated to S3, EBS can effectively freeze the state of data blocks when the snapshot is taken, while allowing the client machine to continue using the disk. In addition, snapshots after the initial full backup are differential; only changed blocks are written to S3 to minimize storage costs and backup time.

Hình 6-5. Các khối EBS sao chép dữ liệu tới nhiều host và đĩa để đạt độ bền và tính sẵn sàng cao, nhưng không chịu được lỗi của một vùng sẵn sàng. Ảo hóa lưu trữ EBS cũng hỗ trợ một số tính năng nâng cao. Chẳng hạn, các khối EBS cho phép chụp ảnh nhanh tức thời tại một thời điểm (point-in-time snapshot) trong khi ổ đĩa vẫn đang được sử dụng. Mặc dù vẫn mất một chút thời gian để snapshot được sao chép sang S3, EBS có thể đóng băng hiệu quả trạng thái của các khối dữ liệu tại thời điểm chụp snapshot, đồng thời cho phép máy khách tiếp tục sử dụng đĩa. Ngoài ra, các snapshot sau lần sao lưu đầy đủ ban đầu là vi sai (differential); chỉ các khối đã thay đổi mới được ghi sang S3 để giảm thiểu chi phí lưu trữ và thời gian sao lưu.

EBS volumes are also highly scalable. At the time of this writing, some EBS volume classes can scale up to 64 TiB, 256,000 IOPS, and 4,000 MiB/s.

Các khối EBS cũng có khả năng mở rộng cao. Tại thời điểm viết cuốn sách này, một số lớp khối EBS có thể mở rộng tới 64 TiB, 256.000 IOPS và 4.000 MiB/s.

Local instance volumes — Các khối instance cục bộ

Cloud providers also offer block storage volumes that are physically attached to the host server running a virtual machine. These storage volumes are generally very low cost (included with the price of the VM in the case of Amazon's EC2 instance store) and provide low latency and high IOPS. Instance store volumes (Figure 6-6) behave essentially like a disk physically attached to a server in a data center. One key difference is that when a VM shuts down or is deleted, the contents of the locally attached disk are lost, whether or not this event was caused by intentional user action. This ensures that a new virtual machine cannot read disk contents belonging to a different customer.

Các nhà cung cấp đám mây cũng cung cấp các khối lưu trữ khối được gắn vật lý vào máy chủ host đang chạy một máy ảo. Các khối lưu trữ này nhìn chung có chi phí rất thấp (được tính kèm vào giá của VM trong trường hợp instance store của Amazon EC2) và cung cấp độ trễ thấp cùng IOPS cao. Các khối instance store (Hình 6-6) hành xử về cơ bản giống một đĩa được gắn vật lý vào một máy chủ trong một trung tâm dữ liệu. Một khác biệt then chốt là khi một VM tắt hoặc bị xóa, nội dung của đĩa gắn cục bộ sẽ bị mất, bất kể sự kiện này có do hành động chủ ý của người dùng hay không. Điều này đảm bảo rằng một máy ảo mới

không thể đọc nội dung đĩa thuộc về một khách hàng khác.

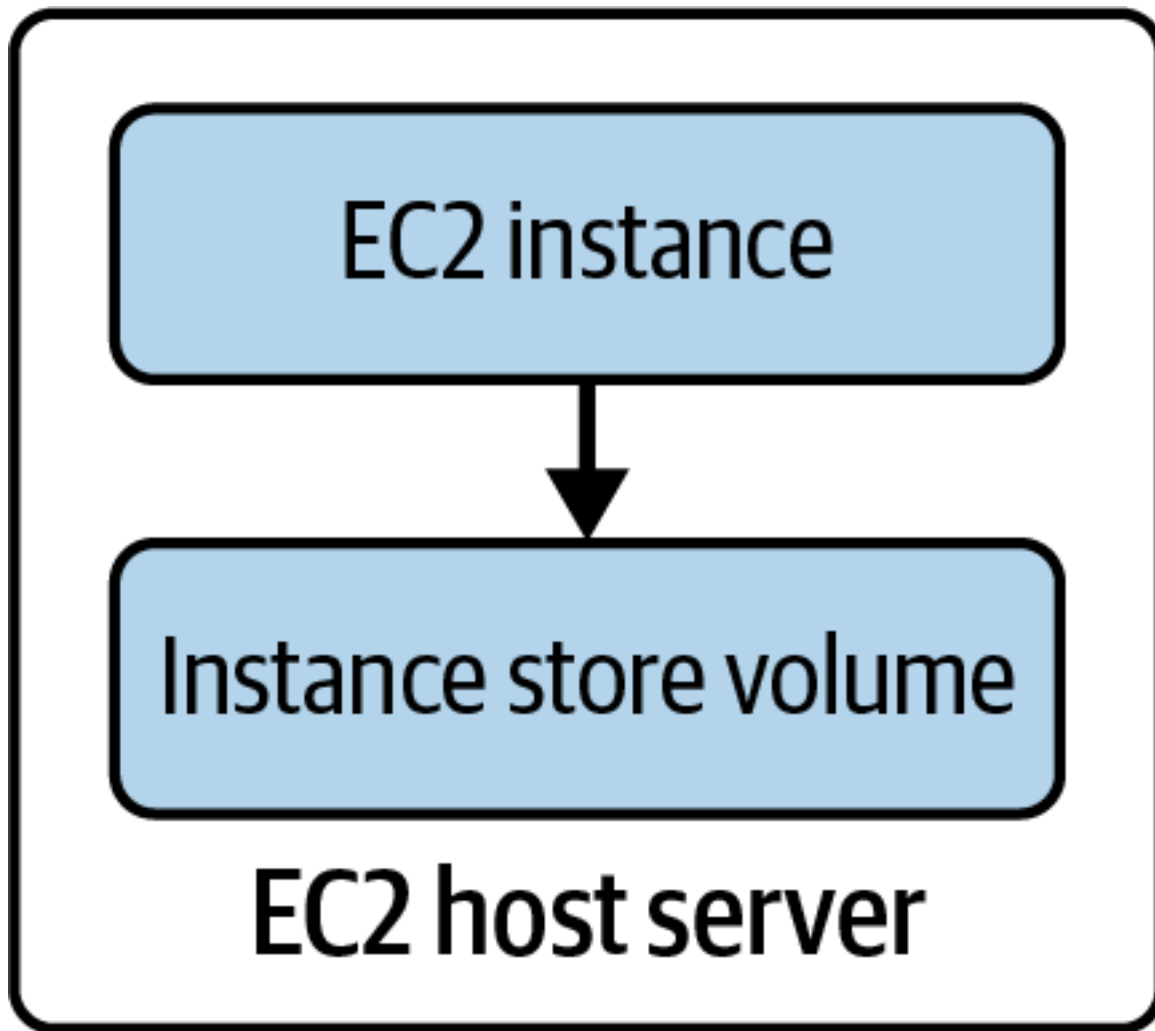


Figure 6-6. Instance store volumes offer high performance and low cost but do not protect data in the event of disk failure or VM shutdown. Locally attached disks support none of the advanced virtualization features offered by virtualized storage services like EBS. The locally attached disk is not replicated, so a physical disk failure can lose or corrupt data even if the host VM continues running. Furthermore, locally attached volumes do not support snapshots or other backup features. Despite these limitations, locally attached disks are extremely useful. In many cases, we use disks as a local cache and hence don't need all the advanced virtualization features of a service like EBS. For example, suppose we're running AWS EMR on EC2 instances. We may be running an ephemeral job that consumes data from S3, stores it temporarily in the distributed filesystem running across the instances, processes the data, and writes the results back to S3. The EMR filesystem builds in replication and redundancy and is serving as a cache rather than permanent storage. The EC2 instance store is a perfectly suitable solution in this case and can enhance performance since data can be read and processed locally without flowing over a network (see Figure 6-7).

Hình 6-6. Các khối instance store cung cấp hiệu năng cao và chi phí thấp nhưng không bảo vệ dữ liệu trong trường hợp đĩa hỏng hoặc VM tắt. Các đĩa gắn cục bộ không hỗ trợ bất kỳ tính năng ảo hóa nâng cao nào mà các dịch vụ lưu trữ ảo hóa như EBS cung cấp. Đĩa gắn cục bộ không được sao chép, nên một lỗi đĩa vật lý có thể làm mất hoặc hư hỏng dữ liệu ngay cả khi VM host vẫn tiếp tục chạy. Hơn nữa, các khối gắn cục bộ không hỗ trợ

snapshot hay các tính năng sao lưu khác. Bất chấp những hạn chế này, các đĩa gắn cục bộ cực kỳ hữu ích. Trong nhiều trường hợp, chúng ta dùng các đĩa như một cache cục bộ và do đó không cần tất cả các tính năng ảo hóa nâng cao của một dịch vụ như EBS. Chẳng hạn, giả sử chúng ta đang chạy AWS EMR trên các instance EC2. Chúng ta có thể đang chạy một công việc tạm thời (ephemeral job) tiêu thụ dữ liệu từ S3, lưu nó tạm thời trong hệ thống tệp phân tán chạy trên các instance, xử lý dữ liệu, và ghi kết quả trở lại S3. Hệ thống tệp EMR tích hợp sẵn sao chép và dự phòng và đang đóng vai trò là một cache thay vì lưu trữ vĩnh viễn. Instance store của EC2 là một giải pháp hoàn toàn phù hợp trong trường hợp này và có thể nâng cao hiệu năng vì dữ liệu có thể được đọc và xử lý cục bộ mà không cần đi qua mạng (xem Hình 6-7).

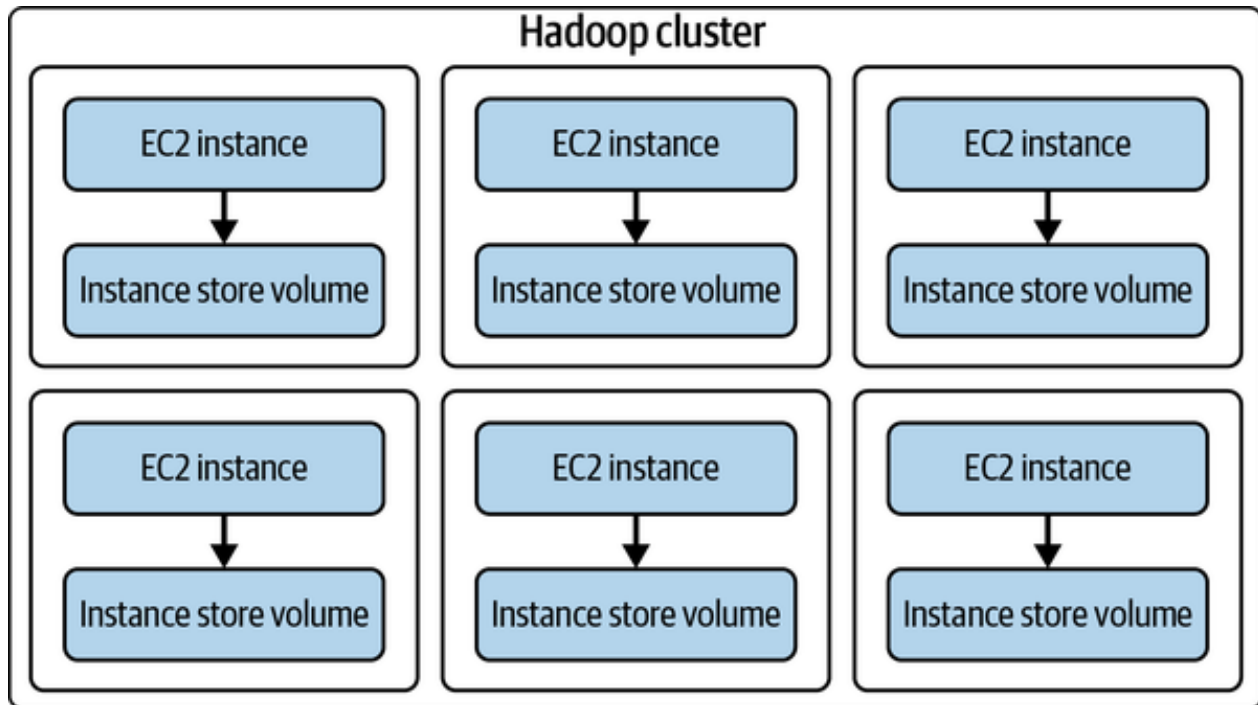


Figure 6-7. Instance store volumes can be used as a processing cache in an ephemeral Hadoop cluster. We recommend that engineers think about locally attached storage in worst-case scenarios. What are the consequences of a local disk failure? Of an accidental VM or cluster shutdown? Of a zonal or regional cloud outage? If none of these scenarios will have catastrophic consequences when data on locally attached volumes is lost, local storage may be a cost-effective and performant option. In addition, simple mitigation strategies (periodic checkpoint backups to S3) can prevent data loss.

Hình 6-7. Các khối instance store có thể được dùng như một cache xử lý trong một cụm Hadoop tạm thời. Chúng tôi đề xuất rằng kỹ sư nên suy nghĩ về lưu trữ gắn cục bộ trong các kịch bản xấu nhất. Hậu quả của một đĩa cục bộ hỏng là gì? Của việc tắt VM hay cụm ngoài ý muốn? Của một sự cố ngừng hoạt động cấp vùng hay cấp khu vực của đám mây? Nếu không kịch bản nào trong số này gây hậu quả thảm khốc khi dữ liệu trên các khối gắn cục bộ bị mất, lưu trữ cục bộ có thể là một lựa chọn tiết kiệm chi phí và hiệu năng cao. Ngoài ra, các chiến lược giảm thiểu đơn giản (sao lưu điểm kiểm tra định kỳ sang S3) có thể ngăn mất dữ liệu.

Object Storage — Lưu trữ đối tượng

Object storage contains objects of all shapes and sizes (Figure 6-8). The term object storage is somewhat confusing because object has several meanings in computer science. In this context, we're talking about a specialized file-like construct. It could be any type of file—TXT, CSV, JSON, images, videos, or audio.

Lưu trữ đối tượng (object storage) chứa các đối tượng đủ mọi hình dạng và kích thước (Hình 6-8). Thuật ngữ lưu trữ đối tượng có phần gây nhầm lẫn vì object (đối tượng) có nhiều nghĩa trong khoa học máy tính. Trong ngữ cảnh này, chúng ta đang nói về một cấu trúc chuyên biệt giống tệp. Nó có thể là bất kỳ loại tệp nào—TXT, CSV, JSON, hình ảnh, video hoặc âm thanh.

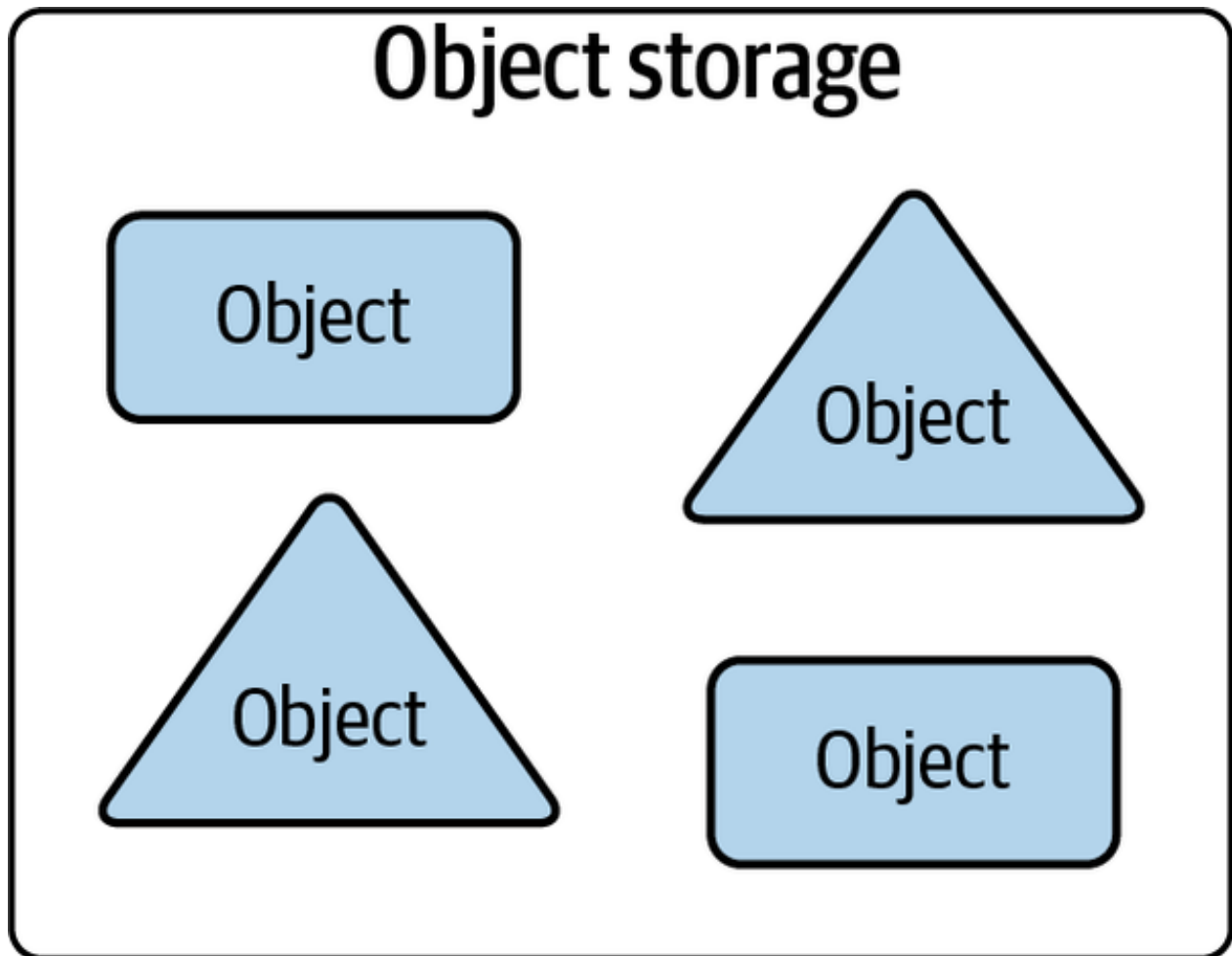


Figure 6-8. Object storage contains **immutable** objects of all shapes and sizes. Unlike files on a local disk, objects cannot be modified in place. Object stores have grown in importance and popularity with the rise of big data and the cloud. Amazon S3, Azure Blob Storage, and Google Cloud Storage (GCS) are widely used object stores. In addition, many cloud data warehouses (and a growing number of databases) utilize object storage as their storage layer, and cloud data lakes generally sit on object stores. Although many on-premises object storage systems can be installed on server clusters, we'll focus mostly on fully managed cloud object stores. From an operational perspective, one of the most attractive characteristics of cloud object storage is that it is straightforward to manage and use. Object storage was arguably one of the first “**serverless**” services; engineers don't need to consider the

characteristics of underlying server clusters or disks.

Hình 6-8. Lưu trữ đối tượng chứa các đối tượng bất biến đủ mọi hình dạng và kích thước. Không giống các tệp trên một đĩa cục bộ, các đối tượng không thể được sửa đổi tại chỗ. Các kho đối tượng (object store) đã tăng tầm quan trọng và độ phổ biến cùng với sự trỗi dậy của dữ liệu lớn (big data) và đám mây. Amazon S3, Azure Blob Storage và Google Cloud Storage (GCS) là các kho đối tượng được sử dụng rộng rãi. Ngoài ra, nhiều kho dữ liệu đám mây (và ngày càng nhiều cơ sở dữ liệu) sử dụng lưu trữ đối tượng làm tầng lưu trữ của chúng, và các hồ dữ liệu đám mây nhìn chung nằm trên các kho đối tượng. Mặc dù nhiều hệ thống lưu trữ đối tượng tại chỗ có thể được cài đặt trên các cụm máy chủ, chúng tôi sẽ tập trung chủ yếu vào các kho đối tượng đám mây được quản lý hoàn toàn. Từ góc độ vận hành, một trong những đặc tính hấp dẫn nhất của lưu trữ đối tượng đám mây là nó đơn giản để quản lý và sử dụng. Lưu trữ đối tượng được cho là một trong những dịch vụ “serverless” đầu tiên; kỹ sư không cần xem xét các đặc tính của các cụm máy chủ hay đĩa nền tảng.

An object store is a key-value store for immutable data objects. We lose much of the writing flexibility we expect with file storage on a local disk in an object store. Objects don't support random writes or append operations; instead, they are written once as a stream of bytes. After this initial write, objects become immutable. To change data in an object or append data to it, we must rewrite the full object. Object stores generally support random reads through range requests, but these lookups may perform much worse than random reads from data stored on an SSD. For a software developer used to leveraging local random access file storage, the characteristics of objects might seem like constraints, but less is more; object stores don't need to support locks or change synchronization, allowing data storage across massive disk clusters. Object stores support extremely performant parallel stream writes and reads across many disks, and this parallelism is hidden from engineers, who can simply deal with the stream rather than communicating with individual disks. In a cloud environment, write speed scales with the number of streams being written up to quota limits set by the vendor. Read bandwidth can scale with the number of parallel requests, the number of virtual machines employed to read data, and the number of CPU cores. These characteristics make object storage ideal for serving high-volume web traffic or delivering data to highly parallel distributed query engines. Typical cloud object stores save data in several availability zones, dramatically reducing the odds that storage will go fully offline or be lost in an unrecoverable way. This durability and availability are built into the cost; cloud storage vendors offer other storage classes at discounted prices in exchange for reduced durability or availability. We'll discuss this in “Storage classes and tiers”. Cloud object storage is a key ingredient in separating compute and storage, allowing engineers to process data with ephemeral clusters and scale these clusters up and down on demand. This is a key factor in making big data available to smaller organizations that can't afford to own hardware for data jobs that they'll run only occasionally. Some major tech companies will continue to run permanent Hadoop clusters on their hardware. Still, the general trend is that most organizations will move data processing to the cloud, using an object store as essential storage and serving layer while processing data on ephemeral clusters. In object storage, available storage space is also highly scalable, an ideal characteristic for big data systems. Storage space is constrained by the number of disks the storage provider owns, but these providers handle exabytes of data. In a cloud environment, available storage space is virtually limitless; in practice, the primary limit on storage space for public cloud customers is budget. From a practical standpoint, engineers can quickly store massive quantities of data for projects without planning months in advance for necessary servers and disks.

Một kho đối tượng là một kho khóa-giá trị (key-value store) cho các đối tượng dữ liệu bất biến. Chúng ta mất đi nhiều linh hoạt ghi mà chúng ta kỳ vọng với lưu trữ tệp trên một đĩa cục bộ khi dùng kho đối tượng. Các đối tượng không hỗ trợ ghi ngẫu nhiên hay thao

tác nối thêm; thay vào đó, chúng được ghi một lần dưới dạng một luồng byte. Sau lần ghi ban đầu này, các đối tượng trở nên bất biến. Để thay đổi dữ liệu trong một đối tượng hoặc nối thêm dữ liệu vào nó, chúng ta phải ghi lại toàn bộ đối tượng. Các kho đối tượng nhìn chung hỗ trợ đọc ngẫu nhiên thông qua các yêu cầu theo khoảng (range request), nhưng các tra cứu này có thể hoạt động kém hơn nhiều so với đọc ngẫu nhiên từ dữ liệu lưu trên một SSD. Đối với một nhà phát triển phần mềm quen tặn dụng lưu trữ tệp truy cập ngẫu nhiên cục bộ, các đặc tính của các đối tượng có thể có vẻ như những ràng buộc, nhưng ít hơn lại là nhiều hơn; các kho đối tượng không cần hỗ trợ khóa hay đồng bộ hóa thay đổi, cho phép lưu trữ dữ liệu trên các cụm đĩa khổng lồ. Các kho đối tượng hỗ trợ ghi và đọc luồng song song cực kỳ hiệu năng trên nhiều đĩa, và tính song song này được ẩn đi với kỹ sư, những người chỉ cần xử lý luồng thay vì giao tiếp với từng đĩa riêng lẻ. Trong một môi trường đám mây, tốc độ ghi mở rộng theo số luồng đang được ghi cho đến các giới hạn hạn ngạch (quota) do nhà cung cấp đặt ra. Băng thông đọc có thể mở rộng theo số yêu cầu song song, số máy ảo được dùng để đọc dữ liệu, và số nhân CPU. Các đặc tính này khiến lưu trữ đối tượng lý tưởng để phục vụ lưu lượng web khối lượng lớn hoặc cung cấp dữ liệu cho các công cụ truy vấn phân tán song song cao. Các kho đối tượng đám mây điển hình lưu trữ dữ liệu ở nhiều vùng sẵn sàng, giảm mạnh xác suất rằng lưu trữ sẽ hoàn toàn ngừng hoạt động hoặc bị mất theo cách không thể khôi phục. Độ bền và tính sẵn sàng này được tích hợp vào chi phí; các nhà cung cấp lưu trữ đám mây cung cấp các lớp lưu trữ khác với giá chiết khấu để đổi lấy độ bền hoặc tính sẵn sàng giảm. Chúng ta sẽ bàn về điều này trong “Các lớp và tầng lưu trữ”. Lưu trữ đối tượng đám mây là một thành phần then chốt trong việc tách biệt tính toán và lưu trữ, cho phép kỹ sư xử lý dữ liệu với các cụm tạm thời và mở rộng các cụm này lên xuống theo nhu cầu. Đây là một yếu tố then chốt giúp dữ liệu lớn trở nên khả dụng với các tổ chức nhỏ hơn vốn không đủ khả năng sở hữu phần cứng cho các công việc dữ liệu mà họ chỉ thỉnh thoảng chạy. Một số công ty công nghệ lớn sẽ tiếp tục chạy các cụm Hadoop cố định trên phần cứng của họ. Tuy vậy, xu hướng chung là hầu hết các tổ chức sẽ chuyển việc xử lý dữ liệu lên đám mây, dùng một kho đối tượng làm tầng lưu trữ và phục vụ thiết yếu trong khi xử lý dữ liệu trên các cụm tạm thời. Trong lưu trữ đối tượng, không gian lưu trữ khả dụng cũng có khả năng mở rộng cao, một đặc tính lý tưởng cho các hệ thống dữ liệu lớn. Không gian lưu trữ bị ràng buộc bởi số đĩa mà nhà cung cấp lưu trữ sở hữu, nhưng các nhà cung cấp này xử lý hàng exabyte dữ liệu. Trong một môi trường đám mây, không gian lưu trữ khả dụng gần như vô hạn; trên thực tế, giới hạn chính đối với không gian lưu trữ cho các khách hàng đám mây công cộng là ngân sách. Từ góc độ thực tiễn, kỹ sư có thể nhanh chóng lưu trữ những khối lượng dữ liệu khổng lồ cho các dự án mà không cần lên kế hoạch trước nhiều tháng cho các máy chủ và đĩa cần thiết.

Object stores for data engineering applications — Kho đối tượng cho các ứng dụng kỹ thuật dữ liệu

From the standpoint of data engineering, object stores provide excellent performance for large **batch** reads and batch writes. This corresponds well to the use case for massive OLAP systems. A bit of data engineering folklore says that object stores are not good for updates, but this is only partially true. Object stores are an inferior fit for transactional workloads with many small updates every second; these use cases are much better served by transactional databases or block storage systems. Object stores work well for a low rate of update operations, where each operation updates a large volume of data. Object stores are now the gold standard of storage for data lakes. In the early days of data lakes, write once, read many (WORM) was the operational standard, but this had more to do with the complexities of managing data versions and files than the limitations of

HDFS and object stores. Since then, systems such as Apache Hudi and Delta Lake have emerged to manage this complexity, and **privacy** regulations such as GDPR and CCPA have made deletion and update capabilities imperative. Update management for object storage is the central idea behind the **data lakehouse** concept, which we introduced in Chapter 3. Object storage is an ideal repository for unstructured data in any format beyond these structured data applications. Object storage can house any binary data with no constraints on type or structure and frequently plays a role in ML pipelines for raw text, images, video, and audio.

Từ góc độ kỹ thuật dữ liệu, các kho đối tượng cung cấp hiệu năng tuyệt vời cho các thao tác đọc theo lô lớn và ghi theo lô. Điều này tương ứng tốt với tình huống sử dụng của các hệ thống OLAP khổng lồ. Một mẫu truyền khẩu trong kỹ thuật dữ liệu nói rằng các kho đối tượng không tốt cho việc cập nhật, nhưng điều này chỉ đúng một phần. Các kho đối tượng phù hợp kém với các tải công việc giao dịch có nhiều cập nhật nhỏ mỗi giây; các tình huống sử dụng này được phục vụ tốt hơn nhiều bởi các cơ sở dữ liệu giao dịch hoặc các hệ thống lưu trữ khối. Các kho đối tượng hoạt động tốt với tần suất thao tác cập nhật thấp, nơi mỗi thao tác cập nhật một khối lượng dữ liệu lớn. Các kho đối tượng giờ là tiêu chuẩn vàng của lưu trữ cho các hồ dữ liệu. Trong những ngày đầu của hồ dữ liệu, write once, read many (ghi một lần, đọc nhiều lần, WORM) là chuẩn vận hành, nhưng điều này liên quan đến sự phức tạp của việc quản lý các phiên bản dữ liệu và tệp nhiều hơn là các hạn chế của HDFS và các kho đối tượng. Kể từ đó, các hệ thống như Apache Hudi và Delta Lake đã xuất hiện để quản lý sự phức tạp này, và các quy định về quyền riêng tư như GDPR và CCPA đã khiến các khả năng xóa và cập nhật trở nên bắt buộc. Quản lý cập nhật cho lưu trữ đối tượng là ý tưởng trung tâm đằng sau khái niệm data lakehouse, mà chúng tôi đã giới thiệu ở Chương 3. Lưu trữ đối tượng là một kho lý tưởng cho dữ liệu phi cấu trúc ở bất kỳ định dạng nào ngoài các ứng dụng dữ liệu có cấu trúc này. Lưu trữ đối tượng có thể chứa bất kỳ dữ liệu nhị phân nào mà không có ràng buộc về kiểu hay cấu trúc và thường đóng một vai trò trong các đường ống học máy (ML) cho văn bản thô, hình ảnh, video và âm thanh.

Object lookup — Tra cứu đối tượng

As we mentioned, object stores are key-value stores. What does this mean for engineers? It's critical to understand that, unlike file stores, object stores do not utilize a directory tree to find objects. The object store uses a top-level logical container (a bucket in S3 and GCS) and references objects by key. A simple example in S3 might look like this:

Như chúng ta đã đề cập, các kho đối tượng là các kho khóa-giá trị. Điều này có nghĩa gì với kỹ sư? Điều quan trọng cần hiểu là, không giống các kho tệp, các kho đối tượng không sử dụng một cây thư mục để tìm các đối tượng. Kho đối tượng sử dụng một vùng chứa logic ở cấp cao nhất (một bucket trong S3 và GCS) và tham chiếu các đối tượng bằng khóa (key). Một ví dụ đơn giản trong S3 có thể trông như thế này:

```
S3://oreilly-data-engineering-book/data-example.json
```

In this case, `S3://oreilly-data-engineering-book/` is the

bucket name, and `data-example.json` is the key pointing to a particular object. S3 bucket names must be unique across all of AWS. Keys are unique within a bucket. Although cloud object stores may appear to support directory tree semantics, no true directory hierarchy exists. We might store an object with the following full path:

là tên bucket, và `data-example.json` là khóa trỏ tới một đối tượng cụ thể. Tên bucket S3 phải duy nhất trên toàn bộ AWS. Các khóa là duy nhất trong phạm vi một bucket. Mặc dù

các kho đối tượng đám mây có thể có vẻ hỗ trợ ngữ nghĩa cây thư mục, không có phân cấp thư mục thực sự nào tồn tại. Chúng ta có thể lưu một đối tượng với đường dẫn đầy đủ sau:

```
S3://oreilly-data-engineering-book/project-  
data/11/23/2021/data.txt
```

On the surface, this looks like subdirectories you might find in a regular file folder system: project-data , 11 , 23 , and 2021 . Many cloud console interfaces allow users to view the objects inside a “directory,” and cloud command-line tools often support Unix-style commands such as ls inside an object store directory. However, behind the scenes, the object system does not traverse a directory tree to reach the object. Instead, it simply sees a key (project-data/11/23/2021/data.txt) that happens to match directory semantics. This might seem like a minor technical detail, but engineers need to understand that certain “directory”-level operations are costly in an object store. To run aws ls S3://oreilly-data-

Nhìn bề ngoài, điều này trông giống các thư mục con bạn có thể tìm thấy trong một hệ thống thư mục tệp thông thường: project-data, 11, 23 và 2021. Nhiều giao diện console đám mây cho phép người dùng xem các đối tượng bên trong một “thư mục”, và các công cụ dòng lệnh đám mây thường hỗ trợ các lệnh kiểu Unix như ls bên trong một thư mục kho đối tượng. Tuy nhiên, ở phía sau hậu trường, hệ thống đối tượng không duyệt một cây thư mục để đến đối tượng. Thay vào đó, nó chỉ đơn giản thấy một khóa (project-data/11/23/2021/data.txt) tình cờ khớp với ngữ nghĩa thư mục. Điều này có vẻ là một chi tiết kỹ thuật nhỏ, nhưng kỹ sư cần hiểu rằng một số thao tác ở “cấp thư mục” là tốn kém trong một kho đối tượng. Để chạy aws ls S3://oreilly-data-

engineering-book/project-data/11/ the object store must

filter keys on the key prefix project-data/11/. If the bucket contains millions of objects, this operation might take some time, even if the “subdirectory” houses only a few objects.

kho đối tượng phải lọc các khóa theo tiền tố khóa project-data/11/. Nếu bucket chứa hàng triệu đối tượng, thao tác này có thể mất một khoảng thời gian, ngay cả khi “thư mục con” chỉ chứa một vài đối tượng.

Object consistency and versioning — Nhất quán và phiên bản hóa đối tượng

As mentioned, object stores don’t support in-place updates or appends as a general rule. We write a new object under the same key to update an object. When data engineers utilize updates in data processes, they must be aware of the consistency model for the object store they’re using. Object stores may be eventually consistent or strongly consistent. For example, until recently, S3 was eventually consistent ; after a new version of an object was written under the same key, the object store might sometimes return the old version of the object. The eventual part of eventual consistency means that after enough time has passed, the storage cluster reaches a state such that only the latest version of the object will be returned. This contrasts with the strong consistency model we expect of local disks attached to a server: reading after a write will return the most recently written data. It might be desirable to impose strong consistency on an object store for various reasons, and standard methods are used to achieve this. One approach is to add a strongly consistent database (e.g., PostgreSQL) to the mix. Writing an object is now a two-step process:

Như đã đề cập, theo quy tắc chung, các kho đối tượng không hỗ trợ cập nhật tại chỗ hay nối thêm. Chúng ta ghi một đối tượng mới dưới cùng một khóa để cập nhật một đối tượng. Khi kỹ sư dữ liệu sử dụng cập nhật trong các quy trình dữ liệu, họ phải nhận thức được

mô hình nhất quán của kho đối tượng họ đang dùng. Các kho đối tượng có thể là nhất quán cuối cùng hoặc nhất quán mạnh. Chẳng hạn, cho đến gần đây, S3 là nhất quán cuối cùng; sau khi một phiên bản mới của một đối tượng được ghi dưới cùng một khóa, kho đối tượng đôi khi có thể trả về phiên bản cũ của đối tượng. Phần “cuối cùng” của nhất quán cuối cùng nghĩa là sau khi đã trôi qua đủ thời gian, cụm lưu trữ đạt đến một trạng thái sao cho chỉ phiên bản mới nhất của đối tượng mới được trả về. Điều này tương phản với mô hình nhất quán mạnh mà chúng ta kỳ vọng ở các đĩa cục bộ gắn vào một máy chủ: đọc sau khi ghi sẽ trả về dữ liệu được ghi gần đây nhất. Có thể là điều đáng mong muốn khi áp đặt nhất quán mạnh lên một kho đối tượng vì nhiều lý do, và các phương pháp tiêu chuẩn được dùng để đạt được điều này. Một cách tiếp cận là thêm một cơ sở dữ liệu nhất quán mạnh (ví dụ, PostgreSQL) vào hỗn hợp. Việc ghi một đối tượng giờ là một quá trình hai bước:

1. Write the object.
2. Write the returned metadata for the object version to the strongly consistent database.
 1. Ghi đối tượng.
 2. Ghi siêu dữ liệu được trả về cho phiên bản của đối tượng vào cơ sở dữ liệu nhất quán mạnh.

The version metadata (an object hash or an object timestamp) can uniquely identify an object version in conjunction with the object key. To read an object, a reader undertakes the following steps:

Siêu dữ liệu phiên bản (một hash của đối tượng hoặc một dấu thời gian của đối tượng) có thể xác định duy nhất một phiên bản đối tượng kết hợp với khóa đối tượng. Để đọc một đối tượng, bên đọc thực hiện các bước sau:

1. Fetch the latest object metadata from the strongly consistent database.
2. Query object metadata using the object key. Read the object data if it matches the metadata fetched from the consistent database.
 1. Lấy siêu dữ liệu đối tượng mới nhất từ cơ sở dữ liệu nhất quán mạnh.
 2. Truy vấn siêu dữ liệu đối tượng bằng khóa đối tượng. Đọc dữ liệu đối tượng nếu nó khớp với siêu dữ liệu được lấy từ cơ sở dữ liệu nhất quán.
3. If the object metadata does not match, repeat step 2 until the latest version of the object is returned.
 3. Nếu siêu dữ liệu đối tượng không khớp, lặp lại bước 2 cho đến khi phiên bản mới nhất của đối tượng được trả về.

A practical implementation has exceptions and edge cases to consider, such as when the object gets rewritten during this querying process. These steps can be managed behind an API so that an object reader sees a strongly consistent object store at the cost of higher latency for object access. Object versioning is closely related to object consistency. When we rewrite an object under an existing key in an object store, we’re essentially writing a brand-new object, setting references from the existing key to the object, and deleting the old object references. Updating all references across the cluster takes time, hence the potential for stale reads. Eventually, the storage cluster garbage collector deallocates the space dedicated to the dereferenced data, recycling disk capacity for use by new objects. With object versioning turned on, we add additional metadata to the object that stipulates a version. While the default key reference gets updated to point to the new object, we retain other pointers to previous versions. We also maintain a version list so that clients can get a list of all object versions, and then pull a specific version. Because old versions of the object are still referenced, they aren’t cleaned up by the garbage collector. If we reference an object with a version, the consistency issue with some object storage systems disappears: the key and version metadata

together form a unique reference to a particular, immutable data object. We will always get the same object back when we use this pair, provided that we haven't deleted it. The consistency issue still exists when a client requests the "default" or "latest" version of an object. The principal overhead that engineers need to consider with object versioning is the cost of storage. Historical versions of objects generally have the same associated storage costs as current versions. Object version costs may be nearly insignificant or catastrophically expensive, depending on various factors. The data size is an issue, as is update frequency; more object versions can lead to significantly larger data size. Keep in mind that we're talking about brute-force object versioning. Object storage systems generally store full object data for each version, not differential snapshots. Engineers also have the option of deploying storage lifecycle policies. Lifecycle policies allow automatic deletion of old object versions when certain conditions are met (e.g., when an object version reaches a certain age or many newer versions exist). Cloud vendors also offer various archival data tiers at heavily discounted prices, and the archival process can be managed using lifecycle policies.

Một triển khai thực tế có các ngoại lệ và trường hợp biên cần xem xét, chẳng hạn như khi đối tượng bị ghi lại trong quá trình truy vấn này. Các bước này có thể được quản lý đằng sau một API để bên đọc đối tượng thấy một kho đối tượng nhất quán mạnh với cái giá là độ trễ cao hơn cho việc truy cập đối tượng. Phiên bản hóa đối tượng liên quan chặt chẽ đến nhất quán đối tượng. Khi chúng ta ghi lại một đối tượng dưới một khóa hiện có trong một kho đối tượng, về cơ bản chúng ta đang ghi một đối tượng hoàn toàn mới, đặt các tham chiếu từ khóa hiện có đến đối tượng, và xóa các tham chiếu đối tượng cũ. Việc cập nhật tất cả các tham chiếu trên toàn cụm mất thời gian, do đó tiềm ẩn khả năng đọc dữ liệu cũ (stale read). Cuối cùng, bộ thu gom rác (garbage collector) của cụm lưu trữ giải phóng không gian dành cho dữ liệu đã bị bỏ tham chiếu, tái chế dung lượng đĩa để các đối tượng mới sử dụng. Khi bật phiên bản hóa đối tượng, chúng ta thêm siêu dữ liệu bổ sung vào đối tượng để quy định một phiên bản. Trong khi tham chiếu khóa mặc định được cập nhật để trỏ đến đối tượng mới, chúng ta vẫn giữ các con trỏ khác đến các phiên bản trước đó. Chúng ta cũng duy trì một danh sách phiên bản để các máy khách có thể lấy một danh sách tất cả các phiên bản đối tượng, rồi kéo về một phiên bản cụ thể. Vì các phiên bản cũ của đối tượng vẫn được tham chiếu, chúng không bị bộ thu gom rác dọn dẹp. Nếu chúng ta tham chiếu một đối tượng cùng với một phiên bản, vấn đề nhất quán với một số hệ thống lưu trữ đối tượng biến mất: khóa và siêu dữ liệu phiên bản cùng nhau tạo thành một tham chiếu duy nhất đến một đối tượng dữ liệu bất biến cụ thể. Chúng ta sẽ luôn nhận lại cùng một đối tượng khi dùng cập này, miễn là chúng ta chưa xóa nó. Vấn đề nhất quán vẫn tồn tại khi một máy khách yêu cầu phiên bản "mặc định" hoặc "mới nhất" của một đối tượng. Chi phí chính mà kỹ sư cần xem xét với phiên bản hóa đối tượng là chi phí lưu trữ. Các phiên bản lịch sử của đối tượng nhìn chung có chi phí lưu trữ liên quan giống như các phiên bản hiện tại. Chi phí phiên bản đối tượng có thể gần như không đáng kể hoặc đắt đỏ thảm khốc, tùy thuộc vào nhiều yếu tố khác nhau. Kích thước dữ liệu là một vấn đề, cũng như tần suất cập nhật; càng nhiều phiên bản đối tượng có thể dẫn đến kích thước dữ liệu lớn hơn đáng kể. Hãy nhớ rằng chúng ta đang nói về phiên bản hóa đối tượng theo kiểu vét cạn (brute-force). Các hệ thống lưu trữ đối tượng nhìn chung lưu toàn bộ dữ liệu đối tượng cho mỗi phiên bản, chứ không phải các snapshot vi sai. Kỹ sư cũng có tùy chọn triển khai các chính sách vòng đời lưu trữ. Các chính sách vòng đời cho phép tự động xóa các phiên bản đối tượng cũ khi đáp ứng một số điều kiện nhất định (ví dụ, khi một phiên bản đối tượng đạt đến một độ tuổi nhất định hoặc có nhiều phiên bản mới hơn tồn tại). Các nhà cung cấp đám mây cũng cung cấp nhiều tầng dữ liệu archival với giá chiết khấu mạnh, và quá trình archival có thể được quản lý bằng các chính sách vòng đời.

Storage classes and tiers — Các lớp và tầng lưu trữ

Cloud vendors now offer storage classes that discount data storage pricing in exchange for reduced access or reduced durability. We use the term reduced access here because many of these storage tiers still make data highly available, but with high retrieval costs in exchange for reduced storage costs. Let's look at a couple of examples in S3 since Amazon is a benchmark for cloud service standards. The S3 Standard-Infrequent Access storage class discounts monthly storage costs for increased data retrieval costs. (See “A Brief Detour on Cloud Economics” for a theoretical discussion of the economics of cloud storage tiers.) Amazon also offers the Amazon S3 One Zone-Infrequent Access tier, replicating only to a single zone. Projected availability drops from 99.9% to 99.5% to account for the possibility of a zonal outage. Amazon still claims extremely high data durability, with the caveat that data will be lost if an availability zone is destroyed. Further down the tiers of reduced access are the archival tiers in S3 Glacier. S3 Glacier promises a dramatic reduction in long-term storage costs for much higher access costs. Users have various retrieval speed options, from minutes to hours, with higher retrieval costs for faster access. For example, at the time of this writing, S3 Glacier Deep Archive discounts storage costs even further; Amazon advertises that storage costs start at \$1 per terabyte per month. In exchange, data restoration takes 12 hours. In addition, this storage class is designed for data that will be stored from 7–10 years and be accessed only one to two times per year.

Các nhà cung cấp đám mây giờ cung cấp các lớp lưu trữ chiết khấu giá lưu trữ dữ liệu để đổi lấy truy cập giảm hoặc độ bền giảm. Chúng tôi dùng thuật ngữ truy cập giảm ở đây vì nhiều tầng lưu trữ này vẫn làm cho dữ liệu sẵn sàng cao, nhưng với chi phí truy xuất cao để đổi lấy chi phí lưu trữ giảm. Hãy cùng xem một vài ví dụ trong S3 vì Amazon là một chuẩn mực cho các tiêu chuẩn dịch vụ đám mây. Lớp lưu trữ S3 Standard-Infrequent Access chiết khấu chi phí lưu trữ hằng tháng để đổi lấy chi phí truy xuất dữ liệu tăng. (Xem “A Brief Detour on Cloud Economics” để có một thảo luận lý thuyết về kinh tế học của các tầng lưu trữ đám mây.) Amazon cũng cung cấp tầng Amazon S3 One Zone-Infrequent Access, chỉ sao chép tới một vùng duy nhất. Tính sẵn sàng dự kiến giảm từ 99,9% xuống 99,5% để tính đến khả năng một sự cố ngừng hoạt động cấp vùng. Amazon vẫn tuyên bố độ bền dữ liệu cực kỳ cao, với lưu ý rằng dữ liệu sẽ bị mất nếu một vùng sẵn sàng bị phá hủy. Xuống sâu hơn nữa trong các tầng truy cập giảm là các tầng archival trong S3 Glacier. S3 Glacier hứa hẹn giảm mạnh chi phí lưu trữ dài hạn để đổi lấy chi phí truy cập cao hơn nhiều. Người dùng có nhiều tùy chọn tốc độ truy xuất, từ vài phút đến vài giờ, với chi phí truy xuất cao hơn cho truy cập nhanh hơn. Chẳng hạn, tại thời điểm viết cuốn sách này, S3 Glacier Deep Archive chiết khấu chi phí lưu trữ thậm chí còn nhiều hơn nữa; Amazon quảng cáo rằng chi phí lưu trữ bắt đầu từ 1 USD mỗi terabyte mỗi tháng. Để đổi lại, việc khôi phục dữ liệu mất 12 giờ. Ngoài ra, lớp lưu trữ này được thiết kế cho dữ liệu sẽ được lưu trong 7–10 năm và chỉ được truy cập một đến hai lần mỗi năm.

Be aware of how you plan to utilize archival storage, as it's easy to get into and often costly to access data, especially if you need it more often than expected. See Chapter 4 for a more extensive discussion of archival storage economics.

Hãy lưu ý đến cách bạn dự định sử dụng lưu trữ archival, vì rất dễ đưa dữ liệu vào nhưng thường tốn kém để truy cập, đặc biệt nếu bạn cần nó thường xuyên hơn dự kiến. Xem Chương 4 để có một thảo luận sâu rộng hơn về kinh tế học của lưu trữ archival.

Object store-backed filesystems — Hệ thống tệp dựa trên kho đối tượng

Object store synchronization solutions have become increasingly popular. Tools like s3fs and Amazon S3 File Gateway allow users to mount an S3 bucket as local storage. Users of these tools should be aware of the characteristics of writes to the filesystem and how these will interact with the characteristics and pricing of object storage. File Gateway, for example, handles changes to files fairly efficiently by combining portions of objects into a new object using the advanced capabilities of S3. However, high-speed transactional writing will overwhelm the update capabilities of an object store. Mounting object storage as a local filesystem works well for files that are updated infrequently.

Các giải pháp đồng bộ hóa kho đối tượng đã ngày càng phổ biến. Các công cụ như s3fs và Amazon S3 File Gateway cho phép người dùng gắn (mount) một bucket S3 như lưu trữ cục bộ. Người dùng các công cụ này nên nhận thức được các đặc tính của các thao tác ghi vào hệ thống tệp và cách chúng sẽ tương tác với các đặc tính và định giá của lưu trữ đối tượng. Chẳng hạn, File Gateway xử lý các thay đổi đối với tệp khá hiệu quả bằng cách kết hợp các phần của các đối tượng thành một đối tượng mới sử dụng các khả năng nâng cao của S3. Tuy nhiên, việc ghi giao dịch tốc độ cao sẽ làm quá tải các khả năng cập nhật của một kho đối tượng. Gắn lưu trữ đối tượng như một hệ thống tệp cục bộ hoạt động tốt cho các tệp được cập nhật không thường xuyên.

Cache and Memory-Based Storage Systems — Các hệ thống lưu trữ dựa trên cache và bộ nhớ

As discussed in “Raw Ingredients of Data Storage”, RAM offers excellent latency and transfer speeds. However, traditional RAM is extremely vulnerable to data loss because a power outage lasting even a second can erase data. RAM-based storage systems are generally focused on caching applications, presenting data for quick access and high bandwidth. Data should generally be written to a more durable medium for retention purposes. These ultra-fast cache systems are useful when data engineers need to serve data with ultra-fast retrieval latency.

Như đã bàn trong “Nguyên liệu thô của lưu trữ dữ liệu”, RAM cung cấp độ trễ và tốc độ truyền tuyệt vời. Tuy nhiên, RAM truyền thống cực kỳ dễ bị tổn thương trước việc mất dữ liệu vì một sự cố mất điện kéo dài thậm chí chỉ một giây có thể xóa sạch dữ liệu. Các hệ thống lưu trữ dựa trên RAM nhìn chung tập trung vào các ứng dụng caching, trình bày dữ liệu để truy cập nhanh và băng thông cao. Dữ liệu nhìn chung nên được ghi vào một phương tiện bền vững hơn cho mục đích lưu giữ. Các hệ thống cache siêu nhanh này hữu ích khi kỹ sư dữ liệu cần phục vụ dữ liệu với độ trễ truy xuất siêu nhanh.

Example: Memcached and lightweight object caching — Ví dụ: Memcached và caching đối tượng nhẹ

Memcached is a key-value store designed for caching database query results, API call responses, and more. Memcached uses simple data structures, supporting either string or integer types. Memcached can deliver results with very low latency while also taking the load off backend systems.

Memcached là một kho khóa-giá trị được thiết kế để caching các kết quả truy vấn cơ sở dữ

liệu, các phản hồi của lệnh gọi API và hơn thế nữa. Memcached sử dụng các cấu trúc dữ liệu đơn giản, hỗ trợ kiểu chuỗi (string) hoặc số nguyên (integer). Memcached có thể cung cấp kết quả với độ trễ rất thấp đồng thời giảm tải cho các hệ thống backend.

Example: Redis, memory caching with optional persistence — Ví dụ: Redis, caching bộ nhớ với khả năng bền vững tùy chọn

Like Memcached, Redis is a key-value store, but it supports somewhat more complex data types (such as lists or sets) Redis also builds in multiple persistence mechanisms, including snapshotting and journaling. With a typical configuration, Redis writes data roughly every two seconds. Redis is thus suitable for extremely high-performance applications but can tolerate a small amount of data loss.

Giống Memcached, Redis là một kho khóa-giá trị, nhưng nó hỗ trợ các kiểu dữ liệu phức tạp hơn đôi chút (chẳng hạn như danh sách hoặc tập hợp). Redis cũng tích hợp sẵn nhiều cơ chế bền vững, bao gồm chụp snapshot và nhật ký hóa. Với một cấu hình điển hình, Redis ghi dữ liệu khoảng mỗi hai giây. Do đó Redis phù hợp cho các ứng dụng hiệu năng cực cao nhưng có thể chịu được một lượng nhỏ dữ liệu bị mất.

The Hadoop Distributed File System — Hệ thống tệp phân tán Hadoop

The Hadoop Distributed File System is based on Google File System (GFS) and was initially engineered to process data with the MapReduce programming model . Hadoop is similar to object storage, but with a key difference: Hadoop combines compute and storage on the same nodes, where object stores typically have limited support for internal processing. Hadoop breaks large files into blocks , chunks of data less than a few hundred megabytes in size. The filesystem is managed by the NameNode , which maintains directories, file metadata, and a detailed catalog describing the location of file blocks in the cluster. In a typical configuration, each block of data is replicated to three nodes. This increases both the durability and availability of data. If a disk or node fails, the replication factor for some file blocks will fall below 3. The NameNode will instruct other nodes to replicate these file blocks so that they again reach the correct replication factor. Thus, the probability of losing data is very low, barring a correlated failure (e.g., an asteroid hitting the data center). Hadoop is not simply a storage system. Hadoop combines compute resources with storage nodes to allow in-place data processing. This was originally achieved using the MapReduce programming model, which we discuss in Chapter 8 .

Hệ thống tệp phân tán Hadoop (Hadoop Distributed File System) dựa trên Google File System (GFS) và ban đầu được kỹ thuật hóa để xử lý dữ liệu với mô hình lập trình MapReduce. Hadoop tương tự lưu trữ đối tượng, nhưng có một khác biệt then chốt: Hadoop kết hợp tính toán và lưu trữ trên cùng các nút, trong khi các kho đối tượng thường có hỗ trợ hạn chế cho xử lý nội bộ. Hadoop chia các tệp lớn thành các khối (block), các đoạn dữ liệu có kích thước dưới vài trăm megabyte. Hệ thống tệp được quản lý bởi NameNode, vốn duy trì các thư mục, siêu dữ liệu tệp, và một danh mục chi tiết mô tả vị trí của các khối tệp trong cụm. Trong một cấu hình điển hình, mỗi khối dữ liệu được sao chép tới ba nút. Điều này tăng cả độ bền lẫn tính sẵn sàng của dữ liệu. Nếu một đĩa hoặc nút hỏng, hệ số sao chép cho một số khối tệp sẽ tụt xuống dưới 3. NameNode sẽ chỉ thị các nút khác sao chép các khối tệp này để chúng lại đạt đúng hệ số sao chép. Do đó, xác suất mất dữ liệu là rất thấp, trừ khi có một sự cố tương quan (ví dụ, một tiểu hành tinh

đâm vào trung tâm dữ liệu). Hadoop không đơn thuần là một hệ thống lưu trữ. Hadoop kết hợp các tài nguyên tính toán với các nút lưu trữ để cho phép xử lý dữ liệu tại chỗ. Điều này ban đầu được đạt được bằng mô hình lập trình MapReduce, mà chúng ta bàn ở Chương 8.

Hadoop is dead. Long live Hadoop! — Hadoop đã chết. Hadoop vạn tuế!

We often see claims that Hadoop is dead. This is only partially true. Hadoop is no longer a **hot**, bleeding-edge technology. Many Hadoop ecosystem tools such as Apache Pig are now on life support and primarily used to run legacy jobs. The pure MapReduce programming model has fallen by the wayside. HDFS remains widely used in various applications and organizations. Hadoop still appears in many legacy installations. Many organizations that adopted Hadoop during the peak of the big data craze have no immediate plans to migrate to newer technologies. This is a good choice for companies that run massive (thousand-node) Hadoop clusters and have the resources to maintain on-premises systems effectively. Smaller companies may want to reconsider the cost overhead and scale limitations of running a small Hadoop cluster against migrating to cloud solutions. In addition, HDFS is a key ingredient of many current big data engines, such as Amazon EMR. In fact, Apache Spark is still commonly run on HDFS clusters. We discuss this in more detail in “Separation of Compute from Storage”.

Chúng ta thường thấy các tuyên bố rằng Hadoop đã chết. Điều này chỉ đúng một phần. Hadoop không còn là một công nghệ nóng, tiên phong. Nhiều công cụ trong hệ sinh thái Hadoop như Apache Pig giờ đang sống lay lắt và chủ yếu được dùng để chạy các công việc kế thừa (legacy). Mô hình lập trình MapReduce thuần túy đã bị bỏ rơi bên lề. HDFS vẫn được sử dụng rộng rãi trong nhiều ứng dụng và tổ chức. Hadoop vẫn xuất hiện trong nhiều cài đặt kế thừa. Nhiều tổ chức đã áp dụng Hadoop trong thời kỳ đỉnh cao của cơn sốt dữ liệu lớn không có kế hoạch tức thời để di chuyển sang các công nghệ mới hơn. Đây là một lựa chọn tốt cho các công ty chạy các cụm Hadoop khổng lồ (hàng nghìn nút) và có nguồn lực để duy trì các hệ thống tại chỗ một cách hiệu quả. Các công ty nhỏ hơn có thể muốn cân nhắc lại chi phí phụ trội và các hạn chế về quy mô của việc chạy một cụm Hadoop nhỏ so với việc di chuyển sang các giải pháp đám mây. Ngoài ra, HDFS là một thành phần then chốt của nhiều công cụ dữ liệu lớn hiện nay, chẳng hạn như Amazon EMR. Trên thực tế, Apache Spark vẫn thường được chạy trên các cụm HDFS. Chúng ta bàn chi tiết hơn về điều này trong “Tách biệt tính toán khỏi lưu trữ”.

Streaming Storage — Lưu trữ luồng

Streaming data has different storage requirements than nonstreaming data. In the case of message queues, stored data is temporal and expected to disappear after a certain duration. However, distributed, scalable streaming frameworks like Apache Kafka now allow extremely long-duration streaming data retention. Kafka supports indefinite data retention by pushing old, infrequently accessed messages down to object storage. Kafka competitors (including Amazon Kinesis, Apache Pulsar, and Google Cloud Pub/Sub) also support long data retention. Closely related to data retention in these systems is the notion of replay. Replay allows a streaming system to return a range of historical stored data. Replay is the standard data-retrieval mechanism for streaming storage systems. Replay can be used to run batch queries over a time range or to reprocess data in a streaming pipeline. Chapter 7 covers replay in more depth. Other storage engines have emerged

for real-time analytics applications. In some sense, transactional databases emerged as the first real-time query engines; data becomes visible to queries as soon as it is written. However, these databases have well-known scaling and locking limitations, especially for analytics queries that run across large volumes of data. While scalable versions of row-oriented transactional databases have overcome some of these limitations, they are still not truly optimized for analytics at scale.

Dữ liệu luồng (streaming) có các yêu cầu lưu trữ khác với dữ liệu không phải luồng. Trong trường hợp các hàng đợi tin nhắn (message queue), dữ liệu được lưu mang tính tạm thời và được kỳ vọng biến mất sau một khoảng thời gian nhất định. Tuy nhiên, các khung (framework) luồng phân tán, có khả năng mở rộng như Apache Kafka giờ cho phép lưu giữ dữ liệu luồng trong thời gian cực kỳ dài. Kafka hỗ trợ lưu giữ dữ liệu vô thời hạn bằng cách đẩy các tin nhắn cũ, ít được truy cập xuống lưu trữ đối tượng. Các đối thủ cạnh tranh của Kafka (bao gồm Amazon Kinesis, Apache Pulsar và Google Cloud Pub/Sub) cũng hỗ trợ lưu giữ dữ liệu lâu dài. Liên quan chặt chẽ đến lưu giữ dữ liệu trong các hệ thống này là khái niệm phát lại (replay). Phát lại cho phép một hệ thống luồng trả về một dải dữ liệu lịch sử đã lưu. Phát lại là cơ chế truy xuất dữ liệu tiêu chuẩn cho các hệ thống lưu trữ luồng. Phát lại có thể được dùng để chạy các truy vấn theo lô trên một dải thời gian hoặc để xử lý lại dữ liệu trong một đường ống luồng. Chương 7 trình bày về phát lại một cách sâu hơn. Các công cụ lưu trữ khác đã xuất hiện cho các ứng dụng phân tích thời gian thực. Theo một nghĩa nào đó, các cơ sở dữ liệu giao dịch đã nổi lên như những công cụ truy vấn thời gian thực đầu tiên; dữ liệu trở nên hiển thị với các truy vấn ngay khi nó được ghi. Tuy nhiên, các cơ sở dữ liệu này có những hạn chế nổi tiếng về mở rộng và khóa, đặc biệt đối với các truy vấn phân tích chạy trên các khối lượng dữ liệu lớn. Mặc dù các phiên bản có khả năng mở rộng của các cơ sở dữ liệu giao dịch định hướng hàng đã khắc phục được một số hạn chế này, chúng vẫn chưa thực sự được tối ưu cho phân tích ở quy mô lớn.

Indexes, Partitioning, and Clustering — Chỉ mục, phân vùng và gom cụm

Indexes provide a map of the table for particular fields and allow extremely fast lookup of individual records. Without indexes, a database would need to scan an entire table to find the records satisfying a WHERE condition.

Các chỉ mục (index) cung cấp một bản đồ của bảng cho các trường cụ thể và cho phép tra cứu cực nhanh các bản ghi riêng lẻ. Không có chỉ mục, một cơ sở dữ liệu sẽ cần quét toàn bộ một bảng để tìm các bản ghi thỏa mãn một điều kiện WHERE.

In most RDBMSs, indexes are used for primary table keys (allowing unique identification of rows) and foreign keys (allowing joins with other tables). Indexes can also be applied to other columns to serve the needs of specific applications. Using indexes, an RDBMS can look up and update thousands of rows per second. We do not cover transactional database records in depth in this book; numerous technical resources are available on this topic. Rather, we are interested in the evolution away from indexes in analytics-oriented storage systems and some new developments in indexes for analytics use cases.

Trong hầu hết các RDBMS, các chỉ mục được dùng cho các khóa chính của bảng (cho phép định danh duy nhất các hàng) và các khóa ngoại (cho phép join với các bảng khác). Các chỉ mục cũng có thể được áp dụng cho các cột khác để phục vụ nhu cầu của các ứng dụng cụ thể. Sử dụng các chỉ mục, một RDBMS có thể tra cứu và cập nhật hàng nghìn hàng mỗi giây. Chúng tôi không trình bày sâu về các bản ghi cơ sở dữ liệu giao dịch trong cuốn sách này; có sẵn vô số tài nguyên kỹ thuật về chủ đề này. Thay vào đó, chúng tôi quan tâm đến

sự tiến hóa rời xa khỏi các chỉ mục trong các hệ thống lưu trữ định hướng phân tích và một số phát triển mới trong các chỉ mục cho các tình huống sử dụng phân tích.

The evolution from rows to columns — Sự tiến hóa từ hàng sang cột

An early data warehouse was typically built on the same type of RDBMS used for transactional applications. The growing popularity of MPP systems meant a shift toward parallel processing for significant improvements in scan performance across large quantities of data for analytics purposes. However, these row-oriented MPPs still used indexes to support joins and condition checking.

Một kho dữ liệu thời kỳ đầu thường được xây dựng trên cùng loại RDBMS được dùng cho các ứng dụng giao dịch. Sự phổ biến ngày càng tăng của các hệ thống MPP có nghĩa là một sự chuyển dịch sang xử lý song song để cải thiện đáng kể hiệu năng quét trên các khối lượng dữ liệu lớn cho mục đích phân tích. Tuy nhiên, các MPP định hướng hàng này vẫn dùng các chỉ mục để hỗ trợ các thao tác join và kiểm tra điều kiện.

In “Raw Ingredients of Data Storage” , we discuss columnar serialization. Columnar serialization allows a database to scan only the columns required for a particular query, sometimes dramatically reducing the amount of data read from the disk. In addition, arranging data by column packs similar values next to each other, yielding high-compression ratios with minimal compression overhead. This allows data to be scanned more quickly from disk and over a network. Columnar databases perform poorly for transactional use cases—i.e., when we try to look up large numbers of individual rows asynchronously. However, they perform extremely well when large quantities of data must be scanned—e.g., for complex data transformations, aggregations, statistical calculations, or evaluation of complex conditions on large datasets. In the past, columnar databases performed poorly on joins, so the advice for data engineers was to denormalize data, using wide schemas, arrays, and nested data wherever possible. Join performance for columnar databases has improved dramatically in recent years, so while there can still be performance advantages in **denormalization**, this is no longer a necessity. You’ll learn more about **normalization** and denormalization in Chapter 8 .

Trong “Nguyên liệu thô của lưu trữ dữ liệu”, chúng ta bàn về tuần tự hóa theo cột. Tuần tự hóa theo cột cho phép một cơ sở dữ liệu chỉ quét các cột cần thiết cho một truy vấn cụ thể, đôi khi giảm mạnh lượng dữ liệu được đọc từ đĩa. Ngoài ra, việc sắp xếp dữ liệu theo cột đóng gói các giá trị tương tự cạnh nhau, mang lại tỷ lệ nén cao với chi phí nén tối thiểu. Điều này cho phép dữ liệu được quét nhanh hơn từ đĩa và qua một mạng. Các cơ sở dữ liệu cột hoạt động kém cho các tình huống sử dụng giao dịch—tức là khi chúng ta cố gắng tra cứu một lượng lớn các hàng riêng lẻ một cách bất đồng bộ. Tuy nhiên, chúng hoạt động cực kỳ tốt khi phải quét những khối lượng dữ liệu lớn—ví dụ, cho các biến đổi dữ liệu phức tạp, các phép tổng hợp, các tính toán thống kê, hoặc đánh giá các điều kiện phức tạp trên các tập dữ liệu lớn. Trong quá khứ, các cơ sở dữ liệu cột hoạt động kém với các thao tác join, nên lời khuyên cho kỹ sư dữ liệu là phi chuẩn hóa dữ liệu, dùng các lược đồ rộng (wide schema), mảng và dữ liệu lồng nhau bất cứ nơi nào có thể. Hiệu năng join cho các cơ sở dữ liệu cột đã cải thiện đáng kể trong những năm gần đây, nên mặc dù vẫn có thể có những lợi thế hiệu năng trong phi chuẩn hóa, đây không còn là điều bắt buộc. Bạn sẽ tìm hiểu thêm về chuẩn hóa và phi chuẩn hóa ở Chương 8.

From indexes to partitions and clustering — Từ chỉ mục đến phân vùng và gom cụm

While columnar databases allow for fast scan speeds, it's still helpful to reduce the amount of data scanned as much as possible. In addition to scanning only data in columns relevant to a query, we can partition a table into multiple subtables by splitting it on a field. It is quite common in analytics and data science use cases to scan over a time range, so date- and time-based **partitioning** is extremely common. Columnar databases generally support a variety of other partition schemes as well. Clusters allow finer-grained organization of data within partitions. A **clustering** scheme applied within a columnar database sorts data by one or a few fields, colocating similar values. This improves performance for filtering, sorting, and joining these values.

Mặc dù các cơ sở dữ liệu cột cho phép tốc độ quét nhanh, vẫn hữu ích nếu giảm lượng dữ liệu được quét càng nhiều càng tốt. Ngoài việc chỉ quét dữ liệu trong các cột liên quan đến một truy vấn, chúng ta có thể phân vùng một bảng thành nhiều bảng con bằng cách chia nó theo một trường. Việc quét trên một dải thời gian là khá phổ biến trong các tình huống sử dụng phân tích và khoa học dữ liệu, nên phân vùng dựa trên ngày và thời gian là cực kỳ phổ biến. Các cơ sở dữ liệu cột nhìn chung cũng hỗ trợ nhiều lược đồ phân vùng khác. Các cụm (cluster) cho phép tổ chức dữ liệu ở mức chi tiết hơn trong phạm vi các phân vùng. Một lược đồ gom cụm được áp dụng trong một cơ sở dữ liệu cột sắp xếp dữ liệu theo một hoặc một vài trường, đặt cùng chỗ các giá trị tương tự. Điều này cải thiện hiệu năng cho việc lọc, sắp xếp và join các giá trị này.

Example: Snowflake micro-partitioning — Ví dụ: Vi phân vùng của Snowflake

We mention Snowflake micro-partitioning because it's a good example of recent developments and evolution in approaches to **columnar storage**. Micro partitions are sets of rows between 50 and 500 megabytes in uncompressed size. Snowflake uses an algorithmic approach that attempts to cluster together similar rows. This contrasts the traditional naive approach to partitioning on a single designated field, such as a date. Snowflake specifically looks for values that are repeated in a field across many rows. This allows aggressive pruning of queries based on predicates. For example, a WHERE clause might stipulate the following:

Chúng tôi đề cập đến vi phân vùng (micro-partitioning) của Snowflake vì nó là một ví dụ tốt về các phát triển và tiến hóa gần đây trong các cách tiếp cận lưu trữ cột. Các vi phân vùng (micro partition) là các tập hợp hàng có kích thước từ 50 đến 500 megabyte ở dạng chưa nén. Snowflake sử dụng một cách tiếp cận theo thuật toán cố gắng gom cụm các hàng tương tự lại với nhau. Điều này tương phản với cách tiếp cận ngây thơ truyền thống là phân vùng theo một trường được chỉ định duy nhất, chẳng hạn như một ngày. Snowflake đặc biệt tìm kiếm các giá trị được lặp lại trong một trường trên nhiều hàng. Điều này cho phép cắt tỉa (prune) các truy vấn một cách quyết liệt dựa trên các vị từ (predicate). Chẳng hạn, một mệnh đề WHERE có thể quy định như sau:

```
WHERE created_date = '2017-01-02'
```

In such a query, Snowflake excludes any micro-partitions that don't include this date, effectively pruning this data. Snowflake also allows overlapping micro-partitions, potentially partitioning on multiple fields showing significant repeats. Efficient pruning is facilitated by Snowflake's metadata database, which stores a description of each micro-partition, including the number of rows and value

ranges for fields. At each query stage, Snowflake analyzes micro-partitions to determine which ones need to be scanned. Snowflake uses the term hybrid columnar storage , partially referring to the fact that its tables are broken into small groups of rows, even though storage is fundamentally columnar. The metadata database plays a role similar to an index in a traditional relational database.

Trong một truy vấn như vậy, Snowflake loại trừ bất kỳ vi phân vùng nào không bao gồm ngày này, cắt tỉa hiệu quả dữ liệu này. Snowflake cũng cho phép các vi phân vùng chồng lấn, có khả năng phân vùng trên nhiều trường thể hiện sự lặp lại đáng kể. Việc cắt tỉa hiệu quả được tạo điều kiện bởi cơ sở dữ liệu siêu dữ liệu của Snowflake, vốn lưu trữ một mô tả của mỗi vi phân vùng, bao gồm số hàng và các dải giá trị cho các trường. Tại mỗi giai đoạn truy vấn, Snowflake phân tích các vi phân vùng để xác định những cái nào cần được quét. Snowflake sử dụng thuật ngữ lưu trữ cột lai (hybrid columnar storage), một phần ám chỉ thực tế rằng các bảng của nó được chia thành các nhóm hàng nhỏ, mặc dù lưu trữ về cơ bản là theo cột. Cơ sở dữ liệu siêu dữ liệu đóng một vai trò tương tự như một chỉ mục trong một cơ sở dữ liệu quan hệ truyền thống.

Data Engineering Storage Abstractions — Các trừu tượng lưu trữ cho kỹ thuật dữ liệu

Data engineering storage abstractions are data organization and query patterns that sit at the heart of the data engineering lifecycle and are built atop the data storage systems discussed previously (see Figure 6-3). We introduced many of these abstractions in Chapter 3 , and we will revisit them here.

Các trừu tượng lưu trữ cho kỹ thuật dữ liệu là các mẫu hình tổ chức và truy vấn dữ liệu nằm ở trung tâm của vòng đời kỹ thuật dữ liệu và được xây dựng trên các hệ thống lưu trữ dữ liệu đã bàn trước đó (xem Hình 6-3). Chúng tôi đã giới thiệu nhiều trong số các trừu tượng này ở Chương 3, và chúng ta sẽ xem lại chúng ở đây.

The main types of abstractions we'll concern ourselves with are those that support data science, analytics, and reporting use cases. These include data warehouse, data lake, data **lakehouse**, data platforms, and data catalogs. We won't cover source systems, as they are discussed in Chapter 5 . The storage abstraction you require as a data engineer boils down to a few key considerations: Purpose and use case

Các loại trừu tượng chính mà chúng ta sẽ quan tâm là những loại hỗ trợ các tình huống sử dụng khoa học dữ liệu, phân tích và báo cáo. Chúng bao gồm kho dữ liệu, hồ dữ liệu, data lakehouse, các nền tảng dữ liệu và các danh mục dữ liệu (data catalog). Chúng tôi sẽ không trình bày về các hệ thống nguồn, vì chúng được bàn ở Chương 5. Trừu tượng lưu trữ mà bạn cần với tư cách là một kỹ sư dữ liệu quy về một vài cân nhắc then chốt: Mục đích và tình huống sử dụng

You must first identify the purpose of storing the data. What is it used for?

Trước tiên bạn phải xác định mục đích của việc lưu trữ dữ liệu. Nó được dùng để làm gì?

Update patterns

Các mẫu hình cập nhật

Is the abstraction optimized for bulk updates, streaming inserts, or upserts?

Trừu tượng được tối ưu cho các cập nhật theo lô, các thao tác chèn theo luồng, hay các thao tác upsert?

Cost

Chi phí

What are the direct and indirect financial costs? The time to value? The opportunity costs?

Các chi phí tài chính trực tiếp và gián tiếp là gì? Thời gian để tạo ra giá trị? Các chi phí cơ hội?

Separate storage and compute

Tách biệt lưu trữ và tính toán

The trend is toward separating storage and compute, but most systems hybridize separation and colocation. We cover this in “Separation of Compute from Storage” since it affects purpose, speed, and cost.

Xu hướng là hướng tới việc tách biệt lưu trữ và tính toán, nhưng hầu hết các hệ thống lai giữa tách biệt và đặt cùng chỗ (colocation). Chúng ta trình bày về điều này trong “Tách biệt tính toán khỏi lưu trữ” vì nó ảnh hưởng đến mục đích, tốc độ và chi phí.

You should know that the popularity of separating storage from compute means the lines between OLAP databases and data lakes are increasingly blurring. Major cloud data warehouses and data lakes are on a collision course. In the future, the differences between these two may be in name only since they might functionally and technically be very similar under the hood.

Bạn nên biết rằng sự phổ biến của việc tách biệt lưu trữ khỏi tính toán có nghĩa là ranh giới giữa các cơ sở dữ liệu OLAP và các hồ dữ liệu đang ngày càng mờ nhạt. Các kho dữ liệu đám mây lớn và các hồ dữ liệu đang trên một đường va chạm. Trong tương lai, sự khác biệt giữa hai loại này có thể chỉ là về tên gọi vì chúng có thể tương tự nhau rất nhiều về mặt chức năng và kỹ thuật ở phần lõi bên trong.

The Data Warehouse — Kho dữ liệu

Data warehouses are a standard OLAP data architecture. As discussed in Chapter 3, the term data warehouse refers to technology platforms (e.g., Google BigQuery and Teradata), an architecture for data centralization, and an organizational pattern within a company. In terms of storage trends, we’ve evolved from building data warehouses atop conventional transactional databases, row-based MPP systems (e.g., Teradata and IBM Netezza), and columnar MPP systems (e.g., Vertica and Teradata Columnar) to cloud data warehouses and data platforms. (See our data warehousing discussion in Chapter 3 for more details on MPP systems.) In practice, cloud data warehouses are often used to organize data into a data lake, a storage area for massive amounts of unprocessed raw data, as originally conceived by James Dixon. Cloud data warehouses can handle 3 massive amounts of raw text and complex JSON documents. The limitation is that cloud data warehouses cannot handle truly unstructured data, such as images, video, or audio, unlike a true data lake. Cloud data warehouses can be coupled with object storage to provide a complete data-lake solution.

Các kho dữ liệu là một kiến trúc dữ liệu OLAP tiêu chuẩn. Như đã bàn ở Chương 3, thuật ngữ kho dữ liệu (data warehouse) chỉ các nền tảng công nghệ (ví dụ, Google BigQuery và Teradata), một kiến trúc để tập trung hóa dữ liệu, và một mẫu hình tổ chức trong một công ty. Về các xu hướng lưu trữ, chúng ta đã tiến hóa từ việc xây dựng các kho dữ liệu trên các cơ sở dữ liệu giao dịch thông thường, các hệ thống MPP dựa trên hàng (ví dụ, Teradata và IBM Netezza), và các hệ thống MPP theo cột (ví dụ, Vertica và Teradata Columnar) đến các kho dữ liệu đám mây và các nền tảng dữ liệu. (Xem phần thảo luận về kho dữ liệu của chúng tôi ở Chương 3 để biết thêm chi tiết về các hệ thống MPP.) Trên thực tế, các kho dữ

liệu đám mây thường được dùng để tổ chức dữ liệu thành một hồ dữ liệu, một khu vực lưu trữ cho những khối lượng khổng lồ dữ liệu thô chưa qua xử lý, như được hình dung ban đầu bởi James Dixon. Các kho dữ liệu đám mây có thể xử lý 3 những khối lượng khổng lồ văn bản thô và các tài liệu JSON phức tạp. Hạn chế là các kho dữ liệu đám mây không thể xử lý dữ liệu thực sự phi cấu trúc, chẳng hạn như hình ảnh, video hoặc âm thanh, không giống một hồ dữ liệu thực thụ. Các kho dữ liệu đám mây có thể được ghép với lưu trữ đối tượng để cung cấp một giải pháp hồ dữ liệu hoàn chỉnh.

The Data Lake — Hồ dữ liệu

The data lake was originally conceived as a massive store where data was retained in raw, unprocessed form. Initially, data lakes were built primarily on Hadoop systems, where cheap storage allowed for retention of massive amounts of data without the cost overhead of a proprietary MPP system. The last five years have seen two major developments in the evolution of data lake storage. First, a major migration toward separation of compute and storage has occurred. In practice, this means a move away from Hadoop toward cloud object storage for long-term retention of data. Second, data engineers discovered that much of the functionality offered by MPP systems (**schema** management; update, merge and delete capabilities) and initially dismissed in the rush to data lakes was, in fact, extremely useful. This led to the notion of the data lakehouse.

Hồ dữ liệu ban đầu được hình dung như một kho khổng lồ nơi dữ liệu được lưu giữ ở dạng thô, chưa qua xử lý. Ban đầu, các hồ dữ liệu được xây dựng chủ yếu trên các hệ thống Hadoop, nơi lưu trữ giá rẻ cho phép lưu giữ những khối lượng khổng lồ dữ liệu mà không có chi phí phụ trội của một hệ thống MPP độc quyền. Năm năm qua đã chứng kiến hai phát triển lớn trong sự tiến hóa của lưu trữ hồ dữ liệu. Thứ nhất, một cuộc di cư lớn hướng tới việc tách biệt tính toán và lưu trữ đã diễn ra. Trên thực tế, điều này có nghĩa là một sự chuyển dịch rời xa Hadoop hướng tới lưu trữ đối tượng đám mây để lưu giữ dữ liệu dài hạn. Thứ hai, kỹ sư dữ liệu phát hiện ra rằng phần lớn chức năng do các hệ thống MPP cung cấp (quản lý lược đồ; các khả năng cập nhật, hợp nhất và xóa) và ban đầu bị gạt bỏ trong cơn vội vã đến với các hồ dữ liệu, thực ra là cực kỳ hữu ích. Điều này dẫn đến khái niệm data lakehouse.

The Data Lakehouse — Data Lakehouse

The data lakehouse is an architecture that combines aspects of the data warehouse and the data lake. As it is generally conceived, the lakehouse stores data in object storage just like a lake. However, the lakehouse adds to this arrangement features designed to streamline data management and create an engineering experience similar to a data warehouse. This means robust table and schema support and features for managing incremental updates and deletes. Lakehouses typically also support table history and rollback; this is accomplished by retaining old versions of files and metadata. A lakehouse system is a metadata and file-management layer deployed with data management and transformation tools. Databricks has heavily promoted the lakehouse concept with Delta Lake, an open source storage management system. We would be remiss not to point out that the architecture of the data lakehouse is similar to the architecture used by various commercial data platforms, including BigQuery and Snowflake. These systems store data in object storage and provide automated metadata management, table history, and update/delete capabilities. The complexities of managing underlying files and storage are fully hidden from the user. The key advantage of the data lakehouse over proprietary tools is interoperability. It's much easier to exchange data between tools when stored in an open file format. Reserializing data from a proprietary database

format incurs overhead in processing, time, and cost. In a data lakehouse architecture, various tools can connect to the metadata layer and read data directly from object storage. It is important to emphasize that much of the data in a data lakehouse may not have a table structure imposed. We can impose data warehouse features where we need them in a lakehouse, leaving other data in a raw or even unstructured format. The data lakehouse technology is evolving rapidly. A variety of new competitors to Delta Lake have emerged, including Apache Hudi and Apache Iceberg. See Appendix A for more details.

Data lakehouse là một kiến trúc kết hợp các khía cạnh của kho dữ liệu và hồ dữ liệu. Như nó thường được hình dung, lakehouse lưu trữ dữ liệu trong lưu trữ đối tượng giống như một hồ. Tuy nhiên, lakehouse bổ sung vào sự sắp xếp này các tính năng được thiết kế để tối ưu hóa việc quản lý dữ liệu và tạo ra một trải nghiệm kỹ thuật tương tự một kho dữ liệu. Điều này có nghĩa là hỗ trợ bảng và lược đồ mạnh mẽ cùng các tính năng để quản lý các cập nhật và xóa tăng tiến (incremental). Các lakehouse thường cũng hỗ trợ lịch sử bảng và quay lui (rollback); điều này đạt được bằng cách lưu giữ các phiên bản cũ của các tệp và siêu dữ liệu. Một hệ thống lakehouse là một tầng quản lý siêu dữ liệu và tệp được triển khai cùng các công cụ quản lý và biến đổi dữ liệu. Databricks đã quảng bá mạnh mẽ khái niệm lakehouse với Delta Lake, một hệ thống quản lý lưu trữ mã nguồn mở. Sẽ là thiếu sót nếu chúng tôi không chỉ ra rằng kiến trúc của data lakehouse tương tự kiến trúc được sử dụng bởi nhiều nền tảng dữ liệu thương mại, bao gồm BigQuery và Snowflake. Các hệ thống này lưu trữ dữ liệu trong lưu trữ đối tượng và cung cấp quản lý siêu dữ liệu tự động, lịch sử bảng, và các khả năng cập nhật/xóa. Sự phức tạp của việc quản lý các tệp và lưu trữ nền tảng được ẩn hoàn toàn khỏi người dùng. Lợi thế then chốt của data lakehouse so với các công cụ độc quyền là khả năng tương tác. Việc trao đổi dữ liệu giữa các công cụ dễ dàng hơn nhiều khi nó được lưu trong một định dạng tệp mở. Việc tuần tự hóa lại dữ liệu từ một định dạng cơ sở dữ liệu độc quyền kéo theo chi phí về xử lý, thời gian và tiền bạc. Trong một kiến trúc data lakehouse, nhiều công cụ khác nhau có thể kết nối với tầng siêu dữ liệu và đọc dữ liệu trực tiếp từ lưu trữ đối tượng. Điều quan trọng cần nhấn mạnh là phần lớn dữ liệu trong một data lakehouse có thể không có một cấu trúc bảng được áp đặt. Chúng ta có thể áp đặt các tính năng kho dữ liệu ở những nơi chúng ta cần chúng trong một lakehouse, để lại dữ liệu khác ở dạng thô hoặc thậm chí phi cấu trúc. Công nghệ data lakehouse đang tiến hóa nhanh chóng. Nhiều đối thủ cạnh tranh mới với Delta Lake đã xuất hiện, bao gồm Apache Hudi và Apache Iceberg. Xem Phụ lục A để biết thêm chi tiết.

Data Platforms — Các nền tảng dữ liệu

Increasingly, vendors are styling their products as data platforms. These vendors have created their ecosystems of interoperable tools with tight integration into the core data storage layer. In evaluating platforms, engineers must ensure that the tools offered meet their needs. Tools not directly provided in the platform can still interoperate, with extra data overhead for data interchange. Platforms also emphasize close integration with object storage for unstructured use cases, as mentioned in our discussion of cloud data warehouses. At this point, the notion of the **data platform** frankly has yet to be fully fleshed out. However, the race is on to create a walled garden of data tools, both simplifying the work of data engineering and generating significant vendor lock-in.

Ngày càng nhiều, các nhà cung cấp đang định vị các sản phẩm của họ là các nền tảng dữ liệu (data platform). Các nhà cung cấp này đã tạo ra các hệ sinh thái công cụ tương tác được của họ với sự tích hợp chặt chẽ vào tầng lưu trữ dữ liệu cốt lõi. Khi đánh giá các nền tảng, kỹ sư phải đảm bảo rằng các công cụ được cung cấp đáp ứng nhu cầu của họ. Các công cụ không được cung cấp trực tiếp trong nền tảng vẫn có thể tương tác được, với chi

phí dữ liệu phụ trội cho việc trao đổi dữ liệu. Các nền tảng cũng nhấn mạnh sự tích hợp chặt chẽ với lưu trữ đối tượng cho các tình huống sử dụng phi cấu trúc, như đã đề cập trong phần thảo luận của chúng tôi về các kho dữ liệu đám mây. Tại thời điểm này, thẳng thắn mà nói khái niệm nền tảng dữ liệu vẫn chưa được định hình đầy đủ. Tuy nhiên, cuộc đua đang diễn ra để tạo ra một khu vườn có tường bao của các công cụ dữ liệu, vừa đơn giản hóa công việc kỹ thuật dữ liệu vừa tạo ra sự khóa chân nhà cung cấp (vendor lock-in) đáng kể.

Stream-to-Batch Storage Architecture — Kiến trúc lưu trữ luồng-sang-lô

The stream-to-batch storage architecture has many similarities to the **Lambda** architecture, though some might quibble over the technical details. Essentially, data flowing through a topic in the streaming storage system is written out to multiple consumers. Some of these consumers might be real-time processing systems that generate statistics on the stream. In addition, a batch storage consumer writes data for long-term retention and batch queries. The batch consumer could be AWS Kinesis Firehose, which can generate S3 objects based on configurable triggers (e.g., time and batch size). Systems such as BigQuery ingest streaming data into a streaming buffer. This streaming buffer is automatically reserialized into columnar object storage. The query engine supports seamless querying of both the streaming buffer and the object data to provide users a current, nearly real-time view of the table.

Kiến trúc lưu trữ luồng-sang-lô (stream-to-batch storage architecture) có nhiều điểm tương đồng với kiến trúc Lambda, mặc dù một số người có thể tranh cãi về các chi tiết kỹ thuật. Về cơ bản, dữ liệu chảy qua một topic trong hệ thống lưu trữ luồng được ghi ra cho nhiều bên tiêu thụ. Một số trong các bên tiêu thụ này có thể là các hệ thống xử lý thời gian thực tạo ra số liệu thống kê trên luồng. Ngoài ra, một bên tiêu thụ lưu trữ theo lô ghi dữ liệu cho mục đích lưu giữ dài hạn và các truy vấn theo lô. Bên tiêu thụ theo lô có thể là AWS Kinesis Firehose, vốn có thể tạo ra các đối tượng S3 dựa trên các kích hoạt (trigger) có thể cấu hình (ví dụ, thời gian và kích thước lô). Các hệ thống như BigQuery thu nạp dữ liệu luồng vào một bộ đệm luồng (streaming buffer). Bộ đệm luồng này được tự động tuần tự hóa lại thành lưu trữ đối tượng theo cột. Công cụ truy vấn hỗ trợ truy vấn liên mạch cả bộ đệm luồng lẫn dữ liệu đối tượng để cung cấp cho người dùng một góc nhìn hiện thời, gần thời gian thực về bảng.

Big Ideas and Trends in Storage — Các ý tưởng lớn và xu hướng trong lưu trữ

In this section, we'll discuss some big ideas in storage—key considerations that you need to keep in mind as you build out your storage architecture. Many of these considerations are part of larger trends. For example, data catalogs fit under the trend toward “enterprisey” data engineering and data management. Separation of compute from storage is now largely an accomplished fact in cloud data systems. And data sharing is an increasingly important consideration as businesses adopt data technology.

Trong mục này, chúng ta sẽ bàn về một số ý tưởng lớn trong lưu trữ—các cân nhắc then chốt mà bạn cần ghi nhớ khi xây dựng kiến trúc lưu trữ của mình. Nhiều trong số các cân nhắc này là một phần của các xu hướng lớn hơn. Chẳng hạn, các danh mục dữ liệu nằm

trong xu hướng hướng tới kỹ thuật dữ liệu và quản lý dữ liệu “kiểu doanh nghiệp”. Việc tách biệt tính toán khỏi lưu trữ giờ phần lớn đã là một thực tế đã hoàn thành trong các hệ thống dữ liệu đám mây. Và việc chia sẻ dữ liệu là một cân nhắc ngày càng quan trọng khi các doanh nghiệp áp dụng công nghệ dữ liệu.

Data Catalog — Danh mục dữ liệu

A **data catalog** is a centralized metadata store for all data across an organization. Strictly speaking, a data catalog is not a top-level data storage abstraction, but it integrates with various systems and abstractions. Data catalogs typically work across operational and analytics data sources, integrate **data lineage** and presentation of data relationships, and allow user editing of data descriptions. Data catalogs are often used to provide a central place where people can view their data, queries, and data storage. As a data engineer, you’ll likely be responsible for setting up and maintaining the various data integrations of data pipeline and storage systems that will integrate with the data catalog and the integrity of the data catalog itself.

Một danh mục dữ liệu (data catalog) là một kho siêu dữ liệu tập trung cho tất cả dữ liệu trên toàn một tổ chức. Nói chặt chẽ, một danh mục dữ liệu không phải là một trừu tượng lưu trữ dữ liệu ở cấp cao nhất, nhưng nó tích hợp với nhiều hệ thống và trừu tượng khác nhau. Các danh mục dữ liệu thường hoạt động trên cả các nguồn dữ liệu vận hành và phân tích, tích hợp truy vết nguồn gốc dữ liệu (data lineage) và trình bày các mối quan hệ dữ liệu, và cho phép người dùng chỉnh sửa các mô tả dữ liệu. Các danh mục dữ liệu thường được dùng để cung cấp một nơi tập trung nơi mọi người có thể xem dữ liệu, các truy vấn và lưu trữ dữ liệu của họ. Với tư cách là một kỹ sư dữ liệu, bạn có thể sẽ chịu trách nhiệm thiết lập và duy trì các tích hợp dữ liệu khác nhau của đường ống dữ liệu và các hệ thống lưu trữ sẽ tích hợp với danh mục dữ liệu, cũng như tính toàn vẹn của chính danh mục dữ liệu đó.

Catalog application integration — Tích hợp ứng dụng với danh mục

Ideally, data applications are designed to integrate with catalog APIs to handle their metadata and updates directly. As catalogs are more widely used in an organization, it becomes easier to approach this ideal.

Lý tưởng nhất, các ứng dụng dữ liệu được thiết kế để tích hợp với các API của danh mục nhằm xử lý siêu dữ liệu và các cập nhật của chúng một cách trực tiếp. Khi các danh mục được sử dụng rộng rãi hơn trong một tổ chức, việc tiến gần đến lý tưởng này trở nên dễ dàng hơn.

Automated scanning — Quét tự động

In practice, cataloging systems typically need to rely on an automated scanning layer that collects metadata from various systems such as data lakes, data warehouses, and operational databases. Data catalogs can collect existing metadata and may also use scanning tools to infer metadata such as key relationships or the presence of sensitive data.

Trên thực tế, các hệ thống lập danh mục thường cần dựa vào một tầng quét tự động thu thập siêu dữ liệu từ nhiều hệ thống khác nhau như các hồ dữ liệu, các kho dữ liệu và các

cơ sở dữ liệu vận hành. Các danh mục dữ liệu có thể thu thập siêu dữ liệu hiện có và cũng có thể dùng các công cụ quét để suy luận ra siêu dữ liệu chẳng hạn như các mối quan hệ khóa hoặc sự hiện diện của dữ liệu nhạy cảm.

Data portal and social layer — Cổng dữ liệu và tầng xã hội

Data catalogs also typically provide a human access layer through a web interface, where users can search for data and view data relationships. Data catalogs can be enhanced with a social layer offering Wiki functionality. This allows users to provide information on their datasets, request information from other users, and post updates as they become available.

Các danh mục dữ liệu cũng thường cung cấp một tầng truy cập cho con người thông qua một giao diện web, nơi người dùng có thể tìm kiếm dữ liệu và xem các mối quan hệ dữ liệu. Các danh mục dữ liệu có thể được tăng cường bằng một tầng xã hội cung cấp chức năng kiểu Wiki. Điều này cho phép người dùng cung cấp thông tin về các tập dữ liệu của họ, yêu cầu thông tin từ những người dùng khác, và đăng các cập nhật khi chúng trở nên sẵn có.

Data catalog use cases — Các tình huống sử dụng của danh mục dữ liệu

Data catalogs have both organizational and technical use cases. Data catalogs make metadata easily available to systems. For instance, a data catalog is a key ingredient of the data lakehouse, allowing table discoverability for queries. Organizationally, data catalogs allow business users, analysts, data scientists, and engineers to search for data to answer questions. Data catalogs streamline cross-organizational communications and collaboration.

Các danh mục dữ liệu có cả các tình huống sử dụng về tổ chức lẫn kỹ thuật. Các danh mục dữ liệu làm cho siêu dữ liệu dễ dàng sẵn có cho các hệ thống. Chẳng hạn, một danh mục dữ liệu là một thành phần then chốt của data lakehouse, cho phép khả năng khám phá bảng cho các truy vấn. Về mặt tổ chức, các danh mục dữ liệu cho phép người dùng nghiệp vụ, nhà phân tích, nhà khoa học dữ liệu và kỹ sư tìm kiếm dữ liệu để trả lời các câu hỏi. Các danh mục dữ liệu tối ưu hóa giao tiếp và cộng tác xuyên tổ chức.

Data Sharing — Chia sẻ dữ liệu

Data sharing allows organizations and individuals to share specific data and carefully defined permissions with specific entities. Data sharing allows data scientists to share data from a sandbox with their collaborators within an organization. Across organizations, data sharing facilitates collaboration between partner businesses. For example, an ad tech company can share advertising data with its customers. A cloud **multitenant** environment makes interorganizational collaboration much easier. However, it also presents new security challenges. Organizations must carefully control policies that govern who can share data with whom to prevent accidental exposure or deliberate exfiltration. Data sharing is a core feature of many cloud data platforms. See Chapter 5 for a more extensive discussion of data sharing.

Chia sẻ dữ liệu (data sharing) cho phép các tổ chức và cá nhân chia sẻ dữ liệu cụ thể cùng các quyền được xác định cẩn thận với các thực thể cụ thể. Chia sẻ dữ liệu cho phép các nhà khoa học dữ liệu chia sẻ dữ liệu từ một sandbox với các cộng sự của họ trong phạm vi một

tổ chức. Xuyên các tổ chức, chia sẻ dữ liệu tạo điều kiện cho sự cộng tác giữa các doanh nghiệp đối tác. Chẳng hạn, một công ty công nghệ quảng cáo (ad tech) có thể chia sẻ dữ liệu quảng cáo với các khách hàng của mình. Một môi trường đa thuê bao (multitenant) trên đám mây làm cho sự cộng tác liên tổ chức dễ dàng hơn nhiều. Tuy nhiên, nó cũng đặt ra những thách thức an ninh mới. Các tổ chức phải kiểm soát cẩn thận các chính sách chi phối ai có thể chia sẻ dữ liệu với ai để ngăn việc lộ dữ liệu ngoài ý muốn hoặc rò rỉ dữ liệu (exfiltration) có chủ đích. Chia sẻ dữ liệu là một tính năng cốt lõi của nhiều nền tảng dữ liệu đám mây. Xem Chương 5 để có một thảo luận sâu rộng hơn về chia sẻ dữ liệu.

Schema — Lược đồ

What is the expected form of the data? What is the file format? Is it structured, semistructured, or unstructured? What data types are expected? How does the data fit into a larger hierarchy? Is it connected to other data through shared keys or other relationships? Note that schema need not be relational. Rather, data becomes more useful when we have as much information about its structure and organization. For images stored in a data lake, this schema information might explain the image format, resolution, and the way the images fit into a larger hierarchy. Schema can function as a sort of Rosetta stone, instructions that tell us how to read the data. Two major schema patterns exist: schema **on write** and **schema on read**. Schema on write is essentially the traditional data warehouse pattern: a table has an integrated schema; any writes to the table must conform. To support schema on write, a data lake must integrate a schema metastore. With schema on read, the schema is dynamically created when data is written, and a reader must determine the schema when reading the data. Ideally, schema on read is implemented using file formats that implement built-in schema information, such as Parquet or JSON. CSV files are notorious for schema inconsistency and are not recommended in this setting. The principal advantage of schema on write is that it enforces data standards, making data easier to consume and utilize in the future. Schema on read emphasizes flexibility, allowing virtually any data to be written. This comes at the cost of greater difficulty consuming data in the future.

Dạng kỳ vọng của dữ liệu là gì? Định dạng tệp là gì? Nó có cấu trúc, bán cấu trúc, hay phi cấu trúc? Các kiểu dữ liệu kỳ vọng là gì? Dữ liệu khớp vào một phân cấp lớn hơn như thế nào? Nó có được kết nối với dữ liệu khác thông qua các khóa chung hoặc các mối quan hệ khác không? Lưu ý rằng lược đồ (schema) không nhất thiết phải là quan hệ. Đúng hơn, dữ liệu trở nên hữu ích hơn khi chúng ta có càng nhiều thông tin về cấu trúc và tổ chức của nó. Đối với các hình ảnh lưu trong một hồ dữ liệu, thông tin lược đồ này có thể giải thích định dạng hình ảnh, độ phân giải, và cách các hình ảnh khớp vào một phân cấp lớn hơn. Lược đồ có thể đóng vai trò như một loại bia đá Rosetta, các chỉ dẫn cho chúng ta biết cách đọc dữ liệu. Hai mẫu hình lược đồ chính tồn tại: schema-on-write (lược đồ khi ghi) và schema-on-read (lược đồ khi đọc). Schema-on-write về cơ bản là mẫu hình kho dữ liệu truyền thống: một bảng có một lược đồ được tích hợp; bất kỳ thao tác ghi nào vào bảng đều phải tuân theo. Để hỗ trợ schema-on-write, một hồ dữ liệu phải tích hợp một kho lưu siêu dữ liệu lược đồ (schema metastore). Với schema-on-read, lược đồ được tạo động khi dữ liệu được ghi, và một bên đọc phải xác định lược đồ khi đọc dữ liệu. Lý tưởng nhất, schema-on-read được triển khai bằng các định dạng tệp triển khai thông tin lược đồ tích hợp sẵn, chẳng hạn như Parquet hoặc JSON. Các tệp CSV nổi tiếng về sự thiếu nhất quán lược đồ và không được khuyến nghị trong bối cảnh này. Lợi thế chính của schema-on-write là nó thực thi các tiêu chuẩn dữ liệu, làm cho dữ liệu dễ tiêu thụ và sử dụng hơn trong tương lai. Schema-on-read nhấn mạnh sự linh hoạt, cho phép gần như bất kỳ dữ liệu nào cũng được ghi. Điều này đi kèm cái giá là khó khăn lớn hơn trong việc tiêu thụ dữ liệu trong tương lai.

Separation of Compute from Storage — Tách biệt tính toán khỏi lưu trữ

A key idea we revisit throughout this book is the separation of compute from storage. This has emerged as a standard data access and query pattern in today's cloud era. Data lakes, as we discussed, store data in object stores and spin up temporary compute capacity to read and process it. Most fully managed OLAP products now rely on object storage behind the scenes. To understand the motivations for separating compute and storage, we should first look at the colocation of compute and storage.

Một ý tưởng then chốt mà chúng tôi xem lại xuyên suốt cuốn sách này là việc tách biệt tính toán khỏi lưu trữ. Điều này đã nổi lên như một mẫu hình truy cập và truy vấn dữ liệu tiêu chuẩn trong kỷ nguyên đám mây ngày nay. Các hồ dữ liệu, như chúng ta đã bàn, lưu trữ dữ liệu trong các kho đối tượng và khởi tạo năng lực tính toán tạm thời để đọc và xử lý nó. Hầu hết các sản phẩm OLAP được quản lý hoàn toàn giờ đều dựa vào lưu trữ đối tượng ở phía sau hậu trường. Để hiểu các động cơ cho việc tách biệt tính toán và lưu trữ, trước tiên chúng ta nên xem xét việc đặt cùng chỗ (colocation) tính toán và lưu trữ.

Colocation of compute and storage — Đặt cùng chỗ tính toán và lưu trữ

Colocation of compute and storage has long been a standard method to improve database performance. For transactional databases, data colocation allows fast, low-latency disk reads and high bandwidth. Even when we virtualize storage (e.g., using Amazon EBS), data is located relatively close to the host machine. The same basic idea applies for analytics query systems running across a cluster of machines. For example, with HDFS and MapReduce, the standard approach is to locate data blocks that need to be scanned in the cluster, and then push individual map jobs out to these blocks. The data scan and processing for the map step are strictly local. The reduce step involves shuffling data across the cluster, but keeping map steps local effectively preserves more bandwidth for shuffling, delivering better overall performance; map steps that filter heavily also dramatically reduce the amount of data to be shuffled.

Đặt cùng chỗ tính toán và lưu trữ từ lâu đã là một phương pháp tiêu chuẩn để cải thiện hiệu năng cơ sở dữ liệu. Đối với các cơ sở dữ liệu giao dịch, đặt cùng chỗ dữ liệu cho phép các thao tác đọc đĩa nhanh, độ trễ thấp và băng thông cao. Ngay cả khi chúng ta ảo hóa lưu trữ (ví dụ, dùng Amazon EBS), dữ liệu nằm tương đối gần máy host. Cùng ý tưởng cơ bản đó áp dụng cho các hệ thống truy vấn phân tích chạy trên một cụm máy. Chẳng hạn, với HDFS và MapReduce, cách tiếp cận tiêu chuẩn là định vị các khối dữ liệu cần được quét trong cụm, rồi đẩy từng công việc map ra các khối này. Việc quét và xử lý dữ liệu cho bước map là nghiêm ngặt cục bộ. Bước reduce liên quan đến việc xáo trộn (shuffle) dữ liệu trên toàn cụm, nhưng giữ các bước map cục bộ hiệu quả bảo toàn nhiều băng thông hơn cho việc xáo trộn, mang lại hiệu năng tổng thể tốt hơn; các bước map lọc mạnh cũng giảm đáng kể lượng dữ liệu cần được xáo trộn.

Separation of compute and storage — Tách biệt tính toán và lưu trữ

If colocation of compute and storage delivers high performance, why the shift toward separation of compute and storage? Several motivations exist.

Nếu việc đặt cùng chỗ tính toán và lưu trữ mang lại hiệu năng cao, vậy tại sao lại có sự chuyển dịch hướng tới việc tách biệt tính toán và lưu trữ? Có một số động cơ tồn tại.

Ephemerality and scalability In the cloud, we've seen a dramatic shift toward ephemerality. In general, it's cheaper to buy and host a server than to rent it from a cloud provider, provided that you're running it 24 hours a day nonstop for years on end. In practice, workloads vary dramatically, and significant efficiencies are realized with a pay-as-you-go model if servers can scale up and down. This is true for web servers in online retail, and it is also true for big data batch jobs that may run only periodically.

Tính tạm thời và khả năng mở rộng Trên đám mây, chúng ta đã chứng kiến một sự chuyển dịch mạnh mẽ hướng tới tính tạm thời (ephemerality). Nhìn chung, mua và lưu trữ (host) một máy chủ rẻ hơn so với thuê nó từ một nhà cung cấp đám mây, với điều kiện là bạn chạy nó 24 giờ một ngày không ngừng nghỉ trong nhiều năm liền. Trên thực tế, các tải công việc biến thiên rất nhiều, và những hiệu quả đáng kể được hiện thực hóa với mô hình trả-theo-mức-dùng (pay-as-you-go) nếu các máy chủ có thể mở rộng lên xuống. Điều này đúng với các máy chủ web trong bán lẻ trực tuyến, và cũng đúng với các công việc theo lô dữ liệu lớn vốn có thể chỉ chạy định kỳ.

Ephemeral compute resources allow engineers to spin up massive clusters to complete jobs on time, and then delete clusters when these jobs are done. The performance benefits of temporarily operating at ultra-high scale can outweigh the bandwidth limitations of object storage.

Các tài nguyên tính toán tạm thời cho phép kỹ sư khởi tạo các cụm khổng lồ để hoàn thành các công việc đúng hạn, rồi xóa các cụm này khi các công việc này hoàn thành. Những lợi ích về hiệu năng của việc tạm thời vận hành ở quy mô siêu cao có thể vượt trội những hạn chế về băng thông của lưu trữ đối tượng.

Data durability and availability Cloud object stores significantly mitigate the risk of data loss and generally provide extremely high uptime (availability). For example, S3 stores data across multiple zones; if a natural disaster destroys a zone, data is still available from the remaining zones. Having multiple zones available also reduces the odds of a data outage. If resources in one zone go down, engineers can spin up the same resources in a different zone. The potential for a misconfiguration that destroys data in object storage is still somewhat scary, but simple-to-deploy mitigations are available. Copying data to multiple cloud regions reduces this risk since configuration changes are generally deployed to only one region at a time. Replicating data to multiple storage providers can further reduce the risk.

Độ bền và tính sẵn sàng của dữ liệu Các kho đối tượng đám mây giảm đáng kể rủi ro mất dữ liệu và nhìn chung cung cấp thời gian hoạt động (tính sẵn sàng) cực kỳ cao. Chẳng hạn, S3 lưu trữ dữ liệu trên nhiều vùng; nếu một thảm họa thiên nhiên phá hủy một vùng, dữ liệu vẫn sẵn có từ các vùng còn lại. Việc có nhiều vùng sẵn sàng cũng giảm xác suất một sự cố ngừng hoạt động dữ liệu. Nếu các tài nguyên ở một vùng ngừng hoạt động, kỹ sư có thể khởi tạo cùng các tài nguyên đó ở một vùng khác. Khả năng một cấu hình sai phá hủy dữ liệu trong lưu trữ đối tượng vẫn còn hơi đáng sợ, nhưng có sẵn các biện pháp giảm thiểu đơn giản để triển khai. Sao chép dữ liệu sang nhiều khu vực (region) đám mây giảm rủi ro này vì các thay đổi cấu hình nhìn chung được triển khai chỉ tới một khu vực tại một thời

điểm. Sao chép dữ liệu sang nhiều nhà cung cấp lưu trữ có thể giảm rủi ro thêm nữa.

Hybrid separation and colocation — Tách biệt và đặt cùng chỗ theo kiểu lai

The practical realities of separating compute from storage are more complicated than we've implied. In reality, we constantly hybridize colocation and separation to realize the benefits of both approaches. This hybridization is typically done in two ways: multitier caching and hybrid object storage. With multitier caching, we utilize object storage for long-term data retention and access but spin up local storage to be used during queries and various stages of data pipelines. Both Google and Amazon offer versions of hybrid object storage (object storage that is tightly integrated with compute). Let's look at examples of how some popular processing engines hybridize separation and colocation of storage and compute.

Các thực tế thực tiễn của việc tách biệt tính toán khỏi lưu trữ phức tạp hơn những gì chúng tôi đã hàm ý. Trên thực tế, chúng ta liên tục lai (hybridize) việc đặt cùng chỗ và tách biệt để hiện thực hóa lợi ích của cả hai cách tiếp cận. Việc lai này thường được thực hiện theo hai cách: caching đa tầng và lưu trữ đối tượng lai. Với caching đa tầng (multitier caching), chúng ta sử dụng lưu trữ đối tượng cho việc lưu giữ và truy cập dữ liệu dài hạn nhưng khởi tạo lưu trữ cục bộ để dùng trong các truy vấn và các giai đoạn khác nhau của đường ống dữ liệu. Cả Google lẫn Amazon đều cung cấp các phiên bản của lưu trữ đối tượng lai (lưu trữ đối tượng được tích hợp chặt chẽ với tính toán). Hãy cùng xem các ví dụ về cách một số công cụ xử lý phổ biến lai việc tách biệt và đặt cùng chỗ của lưu trữ và tính toán.

Example: AWS EMR with S3 and HDFS Big data services like Amazon EMR spin up temporary HDFS clusters to process data. Engineers have the option of referencing both S3 and HDFS as a filesystem. A common pattern is to stand up HDFS on SSD drives, pull from S3, and save data from intermediate processing steps on local HDFS. Doing so can realize significant performance gains over processing directly from S3. Full results are written back to S3 once the cluster completes its steps, and the cluster and HDFS are deleted. Other consumers read the output data directly from S3.

Ví dụ: AWS EMR với S3 và HDFS Các dịch vụ dữ liệu lớn như Amazon EMR khởi tạo các cụm HDFS tạm thời để xử lý dữ liệu. Kỹ sư có tùy chọn tham chiếu cả S3 lẫn HDFS như một hệ thống tệp. Một mẫu hình phổ biến là dựng HDFS trên các ổ đĩa SSD, kéo dữ liệu từ S3, và lưu dữ liệu từ các bước xử lý trung gian trên HDFS cục bộ. Làm như vậy có thể hiện thực hóa những lợi ích hiệu năng đáng kể so với việc xử lý trực tiếp từ S3. Các kết quả đầy đủ được ghi trở lại S3 một khi cụm hoàn thành các bước của nó, và cụm cùng HDFS bị xóa. Các bên tiêu thụ khác đọc dữ liệu đầu ra trực tiếp từ S3.

Example: Apache Spark In practice, Spark generally runs jobs on HDFS or some other ephemeral distributed filesystem to support performant storage of data between processing steps. In addition, Spark relies heavily on in-memory storage of data to improve processing. The problem with owning the infrastructure for running Spark is that dynamic RAM (DRAM) is extremely expensive; by separating compute and storage in the cloud, we can rent large quantities of memory and then release that memory when the job completes.

Ví dụ: Apache Spark Trên thực tế, Spark nhìn chung chạy các công việc trên HDFS hoặc một hệ thống tệp phân tán tạm thời nào đó khác để hỗ trợ lưu trữ dữ liệu hiệu năng giữa các bước xử lý. Ngoài ra, Spark dựa nhiều vào lưu trữ dữ liệu trong bộ nhớ để cải thiện việc xử lý. Vấn đề của việc sở hữu hạ tầng để chạy Spark là RAM động (DRAM) cực kỳ đắt; bằng cách tách biệt tính toán và lưu trữ trên đám mây, chúng ta có thể thuê những lượng

lớn bộ nhớ rồi giải phóng bộ nhớ đó khi công việc hoàn thành.

Example: Apache Druid Apache Druid relies heavily on SSDs to realize high performance. Since SSDs are significantly more expensive than magnetic disks, Druid keeps only one copy of data in its cluster, reducing “live” storage costs by a factor of three. Of course, maintaining data durability is still critical, so Druid uses an object store as its durability layer. When data is ingested, it’s processed, serialized into compressed columns, and written to cluster SSDs and object storage. In the event of node failure or cluster data corruption, data can be automatically recovered to new nodes. In addition, the cluster can be shut down and then fully recovered from SSD storage.

Ví dụ: Apache Druid Apache Druid dựa nhiều vào SSD để hiện thực hóa hiệu năng cao. Vì SSD đắt hơn đáng kể so với đĩa từ, Druid chỉ giữ một bản sao dữ liệu trong cụm của nó, giảm chi phí lưu trữ “trực tiếp” (live) đi ba lần. Tất nhiên, việc duy trì độ bền dữ liệu vẫn là then chốt, nên Druid dùng một kho đối tượng làm tầng độ bền của nó. Khi dữ liệu được thu nạp, nó được xử lý, tuần tự hóa thành các cột được nén, và được ghi vào các SSD của cụm và lưu trữ đối tượng. Trong trường hợp một nút hỏng hoặc dữ liệu cụm bị hư hỏng, dữ liệu có thể được tự động khôi phục về các nút mới. Ngoài ra, cụm có thể bị tắt rồi được khôi phục đầy đủ từ lưu trữ SSD.

Example: Hybrid Object Storage Google’s Colossus file storage system supports fine-grained control of data block location, although this functionality is not exposed directly to the public. BigQuery uses this feature to colocate customer tables in a single location, allowing ultra-high bandwidth for queries in that location. We refer to this as hybrid object storage because it combines the clean abstractions of object storage with some advantages of colocating compute and storage. Amazon also offers some notion of hybrid object storage through S3 Select, a feature that allows users to filter S3 data directly in S3 clusters before data is returned across the network. We speculate that public clouds will adopt hybrid object storage more widely to improve the performance of their offerings and make more efficient use of available network resources. Some may be already doing so without disclosing this publicly. The concept of hybrid object storage underscores that there can still be advantages to having low-level access to hardware rather than relying on someone else’s public cloud. Public cloud services do not expose low-level details of hardware and systems (e.g., data block locations for Colossus), but these details can be extremely useful in performance optimization and enhancement. See our discussion of cloud economics in Chapter 4 . While we’re now seeing a mass migration of data to public clouds, we believe that many hyper-scale data service vendors that currently run on public clouds provided by other vendors may build their data centers in the future, albeit with deep network integration into public clouds.

Ví dụ: Lưu trữ đối tượng lai Hệ thống lưu trữ tệp Colossus của Google hỗ trợ kiểm soát ở mức chi tiết về vị trí khối dữ liệu, mặc dù chức năng này không được phơi bày trực tiếp ra công chúng. BigQuery dùng tính năng này để đặt cùng chỗ các bảng của khách hàng tại một vị trí duy nhất, cho phép băng thông siêu cao cho các truy vấn tại vị trí đó. Chúng tôi gọi đây là lưu trữ đối tượng lai vì nó kết hợp các trù tượng gọn gàng của lưu trữ đối tượng với một số lợi thế của việc đặt cùng chỗ tính toán và lưu trữ. Amazon cũng cung cấp một dạng nào đó của lưu trữ đối tượng lai thông qua S3 Select, một tính năng cho phép người dùng lọc dữ liệu S3 trực tiếp trong các cụm S3 trước khi dữ liệu được trả về qua mạng. Chúng tôi suy đoán rằng các đám mây công cộng sẽ áp dụng lưu trữ đối tượng lai rộng rãi hơn để cải thiện hiệu năng của các sản phẩm của họ và sử dụng các tài nguyên mạng sẵn có một cách hiệu quả hơn. Một số có thể đã làm điều này rồi mà không công bố công khai. Khái niệm lưu trữ đối tượng lai nhấn mạnh rằng vẫn có thể có những lợi thế khi có quyền truy cập ở mức thấp vào phần cứng thay vì dựa vào đám mây công cộng của người khác. Các dịch vụ đám mây công cộng không phơi bày các chi tiết ở mức thấp của phần cứng và hệ thống (ví dụ, vị trí khối dữ liệu cho Colossus), nhưng các chi tiết này có thể cực kỳ

hữu ích trong việc tối ưu hóa và nâng cao hiệu năng. Xem phần thảo luận của chúng tôi về kinh tế học đám mây ở Chương 4. Trong khi chúng ta giờ đang chứng kiến một cuộc di cư hàng loạt của dữ liệu sang các đám mây công cộng, chúng tôi tin rằng nhiều nhà cung cấp dịch vụ dữ liệu quy mô siêu lớn hiện đang chạy trên các đám mây công cộng do các nhà cung cấp khác cung cấp có thể sẽ xây dựng các trung tâm dữ liệu của riêng họ trong tương lai, mặc dù với sự tích hợp mạng sâu vào các đám mây công cộng.

Zero copy cloning — Sao chép không tốn bộ nhớ

Cloud-based systems based around object storage support zero copy cloning . This typically means that a new virtual copy of an object is created (e.g., a new table) without necessarily physically copying the underlying data. Typically, new pointers are created to the raw data files, and future changes to these tables will not be recorded in the old table. For those familiar with the inner workings of object-oriented languages such as Python, this type of “shallow” copying is familiar from other contexts. Zero copy cloning is a compelling feature, but engineers must understand its strengths and limitations. For example, cloning an object in a data lake environment and then deleting the files in the original object might also wipe out the new object. For fully managed object-store-based systems (e.g., Snowflake and BigQuery), engineers need to be extremely familiar with the exact limits of shallow copying. Engineers have more access to underlying object storage in data lake systems such as Databricks—a blessing and a curse. Data engineers should exercise great caution before deleting any raw files in the underlying object store. Databricks and other data lake management technologies sometimes also support a notion of deep copying , whereby all underlying data objects are copied. This is a more expensive process, but also more robust in the event that files are unintentionally lost or deleted.

Các hệ thống dựa trên đám mây xoay quanh lưu trữ đối tượng hỗ trợ sao chép không tốn bộ nhớ (zero copy cloning). Điều này thường có nghĩa là một bản sao ảo mới của một đối tượng được tạo ra (ví dụ, một bảng mới) mà không nhất thiết sao chép vật lý dữ liệu nền tảng. Thông thường, các con trỏ mới được tạo tới các tệp dữ liệu thô, và các thay đổi trong tương lai đối với các bảng này sẽ không được ghi vào bảng cũ. Đối với những ai quen thuộc với cơ chế hoạt động bên trong của các ngôn ngữ hướng đối tượng như Python, kiểu sao chép “ nông ” (shallow) này là quen thuộc từ các bối cảnh khác. Sao chép không tốn bộ nhớ là một tính năng hấp dẫn, nhưng kỹ sư phải hiểu các điểm mạnh và hạn chế của nó. Chẳng hạn, sao chép một đối tượng trong một môi trường hồ dữ liệu rồi xóa các tệp trong đối tượng gốc cũng có thể xóa sạch đối tượng mới. Đối với các hệ thống dựa trên kho đối tượng được quản lý hoàn toàn (ví dụ, Snowflake và BigQuery), kỹ sư cần cực kỳ quen thuộc với các giới hạn chính xác của sao chép nông. Kỹ sư có nhiều quyền truy cập hơn vào lưu trữ đối tượng nền tảng trong các hệ thống hồ dữ liệu như Databricks—vừa là một phước lành vừa là một lời nguyền. Kỹ sư dữ liệu nên hết sức thận trọng trước khi xóa bất kỳ tệp thô nào trong kho đối tượng nền tảng. Databricks và các công nghệ quản lý hồ dữ liệu khác đôi khi cũng hỗ trợ một khái niệm về sao chép sâu (deep copying), theo đó tất cả các đối tượng dữ liệu nền tảng đều được sao chép. Đây là một quá trình tốn kém hơn, nhưng cũng mạnh mẽ hơn trong trường hợp các tệp bị mất hoặc bị xóa ngoài ý muốn.

Data Storage Lifecycle and Data Retention — Vòng đời lưu trữ dữ liệu và lưu giữ dữ liệu

Storing data isn't as simple as just saving it to object storage or disk and forgetting about it. You need to think about the data storage lifecycle and data retention. When you think about access frequency and use cases, ask, “How important is the data to **downstream** users, and how often do they need to access it?” This is the data storage lifecycle. Another question you should ask is, “How long should I keep this data?” Do you need to retain data indefinitely, or are you fine discarding it past a certain time frame? This is data retention. Let's dive into each of these.

Lưu trữ dữ liệu không đơn giản chỉ là lưu nó vào lưu trữ đối tượng hoặc đĩa rồi quên nó đi. Bạn cần suy nghĩ về vòng đời lưu trữ dữ liệu và lưu giữ dữ liệu. Khi bạn nghĩ về tần suất truy cập và các tình huống sử dụng, hãy tự hỏi, “Dữ liệu quan trọng đến mức nào đối với những người dùng hạ nguồn, và họ cần truy cập nó thường xuyên ra sao?” Đây là vòng đời lưu trữ dữ liệu. Một câu hỏi khác bạn nên hỏi là, “Tôi nên giữ dữ liệu này trong bao lâu?” Bạn có cần lưu giữ dữ liệu vô thời hạn không, hay bạn ổn với việc loại bỏ nó sau một khung thời gian nhất định? Đây là lưu giữ dữ liệu (data retention). Hãy cùng đi sâu vào từng điều này.

Hot, warm, and cold data — Dữ liệu nóng, ấm và lạnh

Did you know that data has a temperature? Depending on how frequently data is accessed, we can roughly bucket the way it is stored into three categories of persistence: hot, **warm**, and cold. Query access patterns differ for each dataset (Figure 6-9). Typically, newer data is queried more often than older data. Let's look at hot, cold, and warm data in that order.

Bạn có biết rằng dữ liệu có nhiệt độ không? Tùy thuộc vào tần suất dữ liệu được truy cập, chúng ta có thể phân loại sơ bộ cách nó được lưu trữ thành ba loại độ bền vững: nóng, ấm và lạnh. Các mẫu hình truy cập truy vấn khác nhau đối với mỗi tập dữ liệu (Hình 6-9). Thông thường, dữ liệu mới hơn được truy vấn thường xuyên hơn dữ liệu cũ hơn. Hãy cùng xem dữ liệu nóng, lạnh và ấm theo thứ tự đó.

	Hot storage	Warm storage	Cold storage
Access	Very frequent	Infrequent	Infrequent
Storage cost	High	Medium	Cheap
Retrieval cost	Cheap	Medium	High

Figure 6-9. Hot, warm, and cold data costs associated with access frequency Hot data Hot data has instant or frequent access requirements. The underlying storage for hot data is suited for fast access and reads, such as SSD or memory. Because of the type of hardware involved with hot data, storing hot data is often the most expensive form of storage. Example use cases for hot data include retrieving product recommendations and product page results. The cost of storing hot data is the highest of these three storage tiers, but retrieval is often inexpensive. Query results cache is another example

of hot data. When a query is run, some query engines will persist the query results in the cache. For a limited time, when the same query is run, instead of rerunning the same query against storage, the query results cache serves the cached results. This allows for much faster query response times versus redundantly issuing the same query repeatedly. In upcoming chapters, we cover query results caches in more detail.

Hình 6-9. Chi phí của dữ liệu nóng, ấm và lạnh gắn với tần suất truy cập. Dữ liệu nóng Dữ liệu nóng có các yêu cầu truy cập tức thời hoặc thường xuyên. Lưu trữ nền tảng cho dữ liệu nóng phù hợp với truy cập và đọc nhanh, chẳng hạn như SSD hoặc bộ nhớ. Vì loại phần cứng liên quan đến dữ liệu nóng, lưu trữ dữ liệu nóng thường là dạng lưu trữ đắt nhất. Các tình huống sử dụng ví dụ cho dữ liệu nóng bao gồm truy xuất các gợi ý sản phẩm và các kết quả trang sản phẩm. Chi phí lưu trữ dữ liệu nóng là cao nhất trong ba tầng lưu trữ này, nhưng việc truy xuất thường rẻ. Cache kết quả truy vấn là một ví dụ khác về dữ liệu nóng. Khi một truy vấn được chạy, một số công cụ truy vấn sẽ lưu giữ các kết quả truy vấn trong cache. Trong một khoảng thời gian giới hạn, khi cùng một truy vấn được chạy, thay vì chạy lại cùng truy vấn đó đối với lưu trữ, cache kết quả truy vấn phục vụ các kết quả đã được cache. Điều này cho phép thời gian phản hồi truy vấn nhanh hơn nhiều so với việc phát đi cùng một truy vấn lặp đi lặp lại một cách dư thừa. Trong các chương sắp tới, chúng ta trình bày về các cache kết quả truy vấn chi tiết hơn.

Warm data Warm data is accessed semi-regularly, say, once per month. No hard and fast rules indicate how often warm data is accessed, but it's less than hot data and more than cold data. The major cloud providers offer object storage tiers that accommodate warm data. For example, S3 offers an Infrequently Accessed Tier, and Google Cloud has a similar storage tier called Nearline. Vendors give their models of recommended access frequency, and engineers can also do their cost modeling and monitoring.

Dữ liệu ấm Dữ liệu ấm được truy cập bán đều đặn, chẳng hạn, mỗi tháng một lần. Không có quy tắc cứng nhắc nào chỉ ra dữ liệu ấm được truy cập thường xuyên đến mức nào, nhưng nó ít hơn dữ liệu nóng và nhiều hơn dữ liệu lạnh. Các nhà cung cấp đám mây lớn cung cấp các tầng lưu trữ đối tượng đáp ứng dữ liệu ấm. Chẳng hạn, S3 cung cấp một tầng Infrequently Accessed (Truy cập không thường xuyên), và Google Cloud có một tầng lưu trữ tương tự gọi là Nearline. Các nhà cung cấp đưa ra các mô hình của họ về tần suất truy cập được khuyến nghị, và kỹ sư cũng có thể tự mô hình hóa và giám sát chi phí của mình.

Storage of warm data is cheaper than hot data, with slightly more expensive retrieval costs.

Lưu trữ dữ liệu ấm rẻ hơn dữ liệu nóng, với chi phí truy xuất đắt hơn đôi chút.

Cold data On the other extreme, cold data is infrequently accessed data. The hardware used to archive cold data is typically cheap and durable, such as HDD, tape storage, and cloud-based archival systems. Cold data is mainly meant for long-term archival, when there's little to no intention to access the data. Though storing cold data is cheap, retrieving cold data is often expensive.

Dữ liệu lạnh Ở thái cực kia, dữ liệu lạnh là dữ liệu được truy cập không thường xuyên. Phần cứng được dùng để archival dữ liệu lạnh thường rẻ và bền, chẳng hạn như HDD, lưu trữ băng từ (tape storage), và các hệ thống archival dựa trên đám mây. Dữ liệu lạnh chủ yếu nhằm cho archival dài hạn, khi có rất ít hoặc không có ý định truy cập dữ liệu. Mặc dù lưu trữ dữ liệu lạnh rẻ, việc truy xuất dữ liệu lạnh thường tốn kém.

Storage tier considerations When considering the storage tier for your data, consider the costs of each tier. If you store all of your data in hot storage, all of the data can be accessed quickly. But this comes at a tremendous price! Conversely, if you store all data in **cold storage** to save on costs, you'll certainly lower your storage costs, but at the expense of prolonged retrieval times and high retrieval

costs if you need to access data. The storage price goes down from faster/higher performing storage to lower storage. Cold storage is popular for archiving data. Historically, cold storage involved physical backups and often mailing this data to a third party that would archive it in a literal vault. Cold storage is increasingly popular in the cloud. Every cloud vendor offers a cold data solution, and you should weigh the cost of pushing data into cold storage versus the cost and time to retrieve the data. Data engineers need to account for spillover from hot to warm/cold storage. Memory is expensive and finite. For example, if hot data is stored in memory, it can be spilled to disk when there's too much new data to store and not enough memory. Some databases may move infrequently accessed data to warm or cold tiers, offloading the data to either HDD or object storage. The latter is increasingly more common because of the cost-effectiveness of object storage. If you're in the cloud and using managed services, disk spillover will happen automatically.

Các cân nhắc về tầng lưu trữ Khi xem xét tầng lưu trữ cho dữ liệu của bạn, hãy cân nhắc chi phí của mỗi tầng. Nếu bạn lưu trữ toàn bộ dữ liệu của mình trong lưu trữ nóng, toàn bộ dữ liệu có thể được truy cập nhanh chóng. Nhưng điều này đi kèm một cái giá khổng lồ! Ngược lại, nếu bạn lưu trữ toàn bộ dữ liệu trong lưu trữ lạnh để tiết kiệm chi phí, bạn chắc chắn sẽ hạ thấp chi phí lưu trữ của mình, nhưng phải đánh đổi bằng thời gian truy xuất kéo dài và chi phí truy xuất cao nếu bạn cần truy cập dữ liệu. Giá lưu trữ giảm dần từ lưu trữ nhanh hơn/hiệu năng cao hơn xuống lưu trữ thấp hơn. Lưu trữ lạnh phổ biến cho việc archival dữ liệu. Trong lịch sử, lưu trữ lạnh liên quan đến các bản sao lưu vật lý và thường là gửi dữ liệu này qua đường bưu điện đến một bên thứ ba để archival nó trong một hầm chứa theo đúng nghĩa đen. Lưu trữ lạnh ngày càng phổ biến trên đám mây. Mọi nhà cung cấp đám mây đều cung cấp một giải pháp dữ liệu lạnh, và bạn nên cân nhắc chi phí của việc đẩy dữ liệu vào lưu trữ lạnh so với chi phí và thời gian để truy xuất dữ liệu. Kỹ sư dữ liệu cần tính đến việc tràn (spillover) từ lưu trữ nóng sang lưu trữ ấm/lạnh. Bộ nhớ đắt và hữu hạn. Chẳng hạn, nếu dữ liệu nóng được lưu trong bộ nhớ, nó có thể bị tràn xuống đĩa khi có quá nhiều dữ liệu mới để lưu và không đủ bộ nhớ. Một số cơ sở dữ liệu có thể di chuyển dữ liệu ít được truy cập sang các tầng ấm hoặc lạnh, đẩy tải dữ liệu sang hoặc HDD hoặc lưu trữ đối tượng. Cách thứ hai ngày càng phổ biến hơn vì tính hiệu quả chi phí của lưu trữ đối tượng. Nếu bạn ở trên đám mây và dùng các dịch vụ được quản lý, việc tràn đĩa sẽ diễn ra tự động.

If you're using cloud-based object storage, create automated lifecycle policies for your data. This will drastically reduce your storage costs. For example, if your data needs to be accessed only once a month, move the data to an infrequent access storage tier. If your data is 180 days old and not accessed for current queries, move it to an archival storage tier. In both cases, you can automate the migration of data away from regular object storage, and you'll save money. That said, consider the retrieval costs—both in time and money—using infrequent or archival style storage tiers. Access and retrieval times and costs may vary depending on the cloud provider. Some cloud providers make it simple and cheap to migrate data into archive storage, but it is costly and slow to retrieve your data.

Nếu bạn đang dùng lưu trữ đối tượng dựa trên đám mây, hãy tạo các chính sách vòng đời tự động cho dữ liệu của bạn. Điều này sẽ giảm mạnh chi phí lưu trữ của bạn. Chẳng hạn, nếu dữ liệu của bạn chỉ cần được truy cập một lần mỗi tháng, hãy di chuyển dữ liệu sang một tầng lưu trữ truy cập không thường xuyên. Nếu dữ liệu của bạn đã 180 ngày tuổi và không được truy cập cho các truy vấn hiện thời, hãy di chuyển nó sang một tầng lưu trữ archival. Trong cả hai trường hợp, bạn có thể tự động hóa việc di chuyển dữ liệu ra khỏi lưu trữ đối tượng thông thường, và bạn sẽ tiết kiệm tiền. Dù vậy, hãy cân nhắc các chi phí truy xuất—cả về thời gian lẫn tiền bạc—khi dùng các tầng lưu trữ kiểu truy cập không thường xuyên hoặc archival. Thời gian và chi phí truy cập và truy xuất có thể biến thiên tùy thuộc vào nhà cung cấp đám mây. Một số nhà cung cấp đám mây làm cho việc di chuyển dữ liệu vào lưu trữ archive đơn giản và rẻ, nhưng lại tốn kém và chậm để truy

xuất dữ liệu của bạn.

Data retention — Lưu giữ dữ liệu

Back in the early days of “big data,” there was a tendency to err on the side of accumulating every piece of data possible, regardless of its usefulness. The expectation was, “we might need this data in the future.” This data hoarding inevitably became unwieldy and dirty, giving rise to data swamps, and regulatory crackdowns on data retention, among other consequences and nightmares. Nowadays, data engineers need to consider data retention: what data do you need to keep, and how long should you keep it? Here are some things to think about with data retention.

Trong những ngày đầu của “dữ liệu lớn”, có một xu hướng nghiêng về phía tích lũy mọi mẩu dữ liệu có thể, bất kể tính hữu ích của nó. Kỳ vọng là, “chúng ta có thể cần dữ liệu này trong tương lai”. Việc tích trữ dữ liệu này không thể tránh khỏi trở nên cồng kềnh và bẩn, làm phát sinh các đầm lầy dữ liệu (data swamp), và các cuộc trấn áp pháp quy về lưu giữ dữ liệu, cùng nhiều hệ quả và cơn ác mộng khác. Ngày nay, kỹ sư dữ liệu cần xem xét việc lưu giữ dữ liệu: bạn cần giữ dữ liệu nào, và bạn nên giữ nó trong bao lâu? Dưới đây là một số điều cần suy nghĩ về lưu giữ dữ liệu.

Value Data is an asset, so you should know the value of the data you’re storing. Of course, value is subjective and depends on what it’s worth to your immediate use case and your broader organization. Is this data impossible to re-create, or can it easily be re-created by querying **upstream** systems? What’s the impact to downstream users if this data is available versus if it is not?

Giá trị Dữ liệu là một tài sản, nên bạn nên biết giá trị của dữ liệu mà bạn đang lưu trữ. Tất nhiên, giá trị mang tính chủ quan và phụ thuộc vào việc nó đáng giá bao nhiêu đối với tình huống sử dụng trước mắt của bạn và tổ chức rộng lớn hơn của bạn. Dữ liệu này có bất khả tái tạo không, hay nó có thể dễ dàng được tái tạo bằng cách truy vấn các hệ thống thượng nguồn? Tác động đối với những người dùng hạ nguồn là gì nếu dữ liệu này sẵn có so với nếu nó không sẵn có?

Time The value to downstream users also depends upon the age of the data. New data is typically more valuable and frequently accessed than older data. Technical limitations may determine how long you can store data in certain storage tiers. For example, if you store hot data in cache or memory, you’ll likely need to set a time to live (TTL), so you can expire data after a certain point or persist it to warm or cold storage. Otherwise, your hot storage will become full, and queries against the hot data will suffer from performance lags.

Thời gian Giá trị đối với những người dùng hạ nguồn cũng phụ thuộc vào độ tuổi của dữ liệu. Dữ liệu mới thường có giá trị hơn và được truy cập thường xuyên hơn dữ liệu cũ. Các hạn chế kỹ thuật có thể quyết định bạn có thể lưu trữ dữ liệu trong các tầng lưu trữ nhất định trong bao lâu. Chẳng hạn, nếu bạn lưu dữ liệu nóng trong cache hoặc bộ nhớ, bạn có thể sẽ cần đặt một thời gian sống (time to live, TTL), để bạn có thể hết hạn dữ liệu sau một thời điểm nhất định hoặc lưu giữ bền vững nó vào lưu trữ ấm hoặc lạnh. Nếu không, lưu trữ nóng của bạn sẽ đầy, và các truy vấn đối với dữ liệu nóng sẽ chịu trễ về hiệu năng.

Compliance Certain regulations (e.g., HIPAA and Payment Card Industry, or PCI) might require you to keep data for a certain time. In these situations, the data simply needs to be accessible upon request, even if the likelihood of an access request is low. Other regulations might require you to hold data for only a limited period of time, and you’ll need to have the ability to delete specific information on time and within compliance guidelines. You’ll need a storage and archival data process—along with the ability to search the data—that fits the retention requirements of the particular regulation with

which you need to comply. Of course, you'll want to balance compliance against cost.

Tuân thủ Một số quy định (ví dụ, HIPAA và Payment Card Industry, hay PCI) có thể yêu cầu bạn giữ dữ liệu trong một khoảng thời gian nhất định. Trong các tình huống này, dữ liệu chỉ đơn giản cần có thể truy cập được khi có yêu cầu, ngay cả khi khả năng một yêu cầu truy cập là thấp. Các quy định khác có thể yêu cầu bạn chỉ giữ dữ liệu trong một khoảng thời gian giới hạn, và bạn sẽ cần có khả năng xóa thông tin cụ thể đúng hạn và trong phạm vi các hướng dẫn tuân thủ. Bạn sẽ cần một quy trình lưu trữ và archival dữ liệu—cùng với khả năng tìm kiếm dữ liệu—phù hợp với các yêu cầu lưu giữ của quy định cụ thể mà bạn cần tuân thủ. Tất nhiên, bạn sẽ muốn cân bằng việc tuân thủ với chi phí.

Cost Data is an asset that (hopefully) has an ROI. On the cost side of ROI, an obvious storage expense is associated with data. Consider the timeline in which you need to retain data. Given our discussion about hot, warm, and cold data, implement automatic **data lifecycle management** practices and move the data to cold storage if you don't need the data past the required retention date. Or delete data if it's truly not needed.

Chi phí Dữ liệu là một tài sản mà (hy vọng là) có một ROI. Về mặt chi phí của ROI, một khoản chi phí lưu trữ hiển nhiên gắn liền với dữ liệu. Hãy cân nhắc dòng thời gian mà bạn cần lưu giữ dữ liệu. Cho phần thảo luận của chúng ta về dữ liệu nóng, ấm và lạnh, hãy triển khai các thực hành quản lý vòng đời dữ liệu tự động và di chuyển dữ liệu sang lưu trữ lạnh nếu bạn không cần dữ liệu sau ngày lưu giữ bắt buộc. Hoặc xóa dữ liệu nếu nó thực sự không cần thiết.

Single-Tenant Versus Multitenant Storage — Lưu trữ đơn thuê bao so với đa thuê bao

In Chapter 3 , we covered the trade-offs between single-tenant and multitenant architecture. To recap, with single-tenant architecture, each group of tenants (e.g., individual users, groups of users, accounts, or customers) gets its own dedicated set of resources such as networking, compute, and storage. A multitenant architecture inverts this and shares these resources among groups of users. Both architectures are widely used.

Ở Chương 3, chúng tôi đã trình bày các đánh đổi giữa kiến trúc đơn thuê bao (single-tenant) và đa thuê bao (multitenant). Để tóm lược, với kiến trúc đơn thuê bao, mỗi nhóm thuê bao (ví dụ, các người dùng riêng lẻ, các nhóm người dùng, các tài khoản, hoặc các khách hàng) có được tập tài nguyên chuyên dụng riêng của mình như mạng, tính toán và lưu trữ. Một kiến trúc đa thuê bao đảo ngược điều này và chia sẻ các tài nguyên này giữa các nhóm người dùng. Cả hai kiến trúc đều được sử dụng rộng rãi.

This section looks at the implications of single-tenant and multitenant storage. Adopting single-tenant storage means that every tenant gets their dedicated storage. In the example in Figure 6-10 , each tenant gets a database. No data is shared among these databases, and storage is totally isolated. An example of using single-tenant storage is that each customer's data must be stored in isolation and cannot be blended with any other customer's data. In this case, each customer gets their own database.

Mục này xem xét các hàm ý của lưu trữ đơn thuê bao và đa thuê bao. Áp dụng lưu trữ đơn thuê bao có nghĩa là mỗi thuê bao có được lưu trữ chuyên dụng của mình. Trong ví dụ ở Hình 6-10, mỗi thuê bao có được một cơ sở dữ liệu. Không dữ liệu nào được chia sẻ giữa các cơ sở dữ liệu này, và lưu trữ hoàn toàn được cô lập. Một ví dụ về việc dùng lưu trữ đơn thuê bao là dữ liệu của mỗi khách hàng phải được lưu trữ một cách cô lập và không thể

bị trộn lẫn với dữ liệu của bất kỳ khách hàng nào khác. Trong trường hợp này, mỗi khách hàng có được cơ sở dữ liệu riêng của mình.

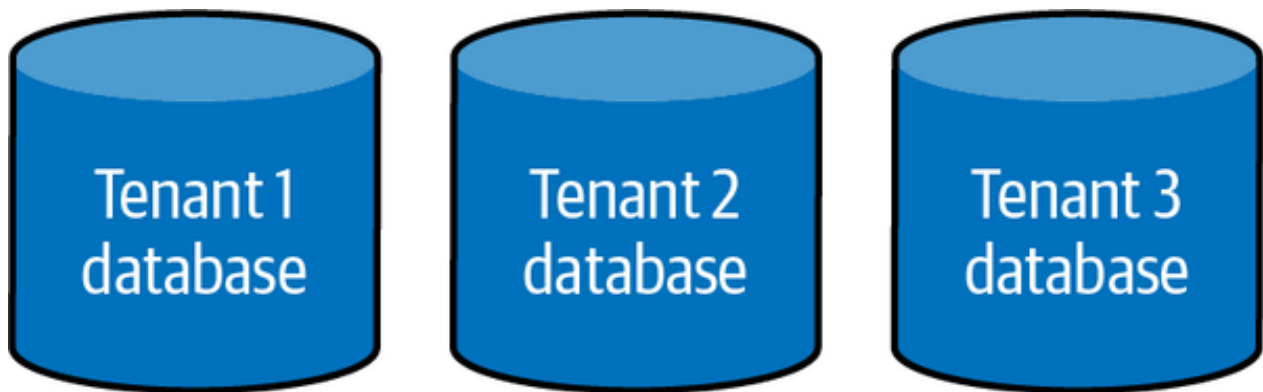


Figure 6-10. In single-tenant storage, each tenant gets their own database. Separate data storage implies separate and independent schemas, bucket structures, and everything related to storage. This means you have the liberty of designing each tenant's storage environment to be uniform or let them evolve however they may. Schema variation across customers can be an advantage and a complication; as always, consider the trade-offs. If each tenant's schema isn't uniform across all tenants, this has major consequences if you need to query multiple tenants' tables to create a unified view of all tenant data. Multitenant storage allows for the storage of multiple tenants within a single database. For example, instead of the single-tenant scenario where customers get their own database, multiple customers may reside in the same database schemas or tables in a multitenant database. Storing multitenant data means each tenant's data is stored in the same place (Figure 6-11).

Hình 6-10. Trong lưu trữ đơn thuê bao, mỗi thuê bao có được cơ sở dữ liệu riêng của mình. Lưu trữ dữ liệu riêng biệt ngụ ý các lược đồ, các cấu trúc bucket, và mọi thứ liên quan đến lưu trữ là riêng biệt và độc lập. Điều này có nghĩa là bạn có quyền tự do thiết kế môi trường lưu trữ của mỗi thuê bao đồng nhất hoặc để chúng tiến hóa theo bất cứ cách nào chúng có thể. Sự biến thiên lược đồ giữa các khách hàng có thể là một lợi thế và một sự phức tạp; như mọi khi, hãy cân nhắc các đánh đổi. Nếu lược đồ của mỗi thuê bao không đồng nhất trên tất cả các thuê bao, điều này có những hệ quả lớn nếu bạn cần truy vấn các bảng của nhiều thuê bao để tạo ra một góc nhìn thống nhất về toàn bộ dữ liệu thuê bao. Lưu trữ đa thuê bao cho phép lưu trữ nhiều thuê bao trong phạm vi một cơ sở dữ liệu duy nhất. Chẳng hạn, thay vì kịch bản đơn thuê bao nơi các khách hàng có được cơ sở dữ liệu riêng của mình, nhiều khách hàng có thể cùng cư trú trong cùng các lược đồ hoặc bảng cơ sở dữ liệu trong một cơ sở dữ liệu đa thuê bao. Lưu trữ dữ liệu đa thuê bao có nghĩa là dữ liệu của mỗi thuê bao được lưu trữ ở cùng một nơi (Hình 6-11).

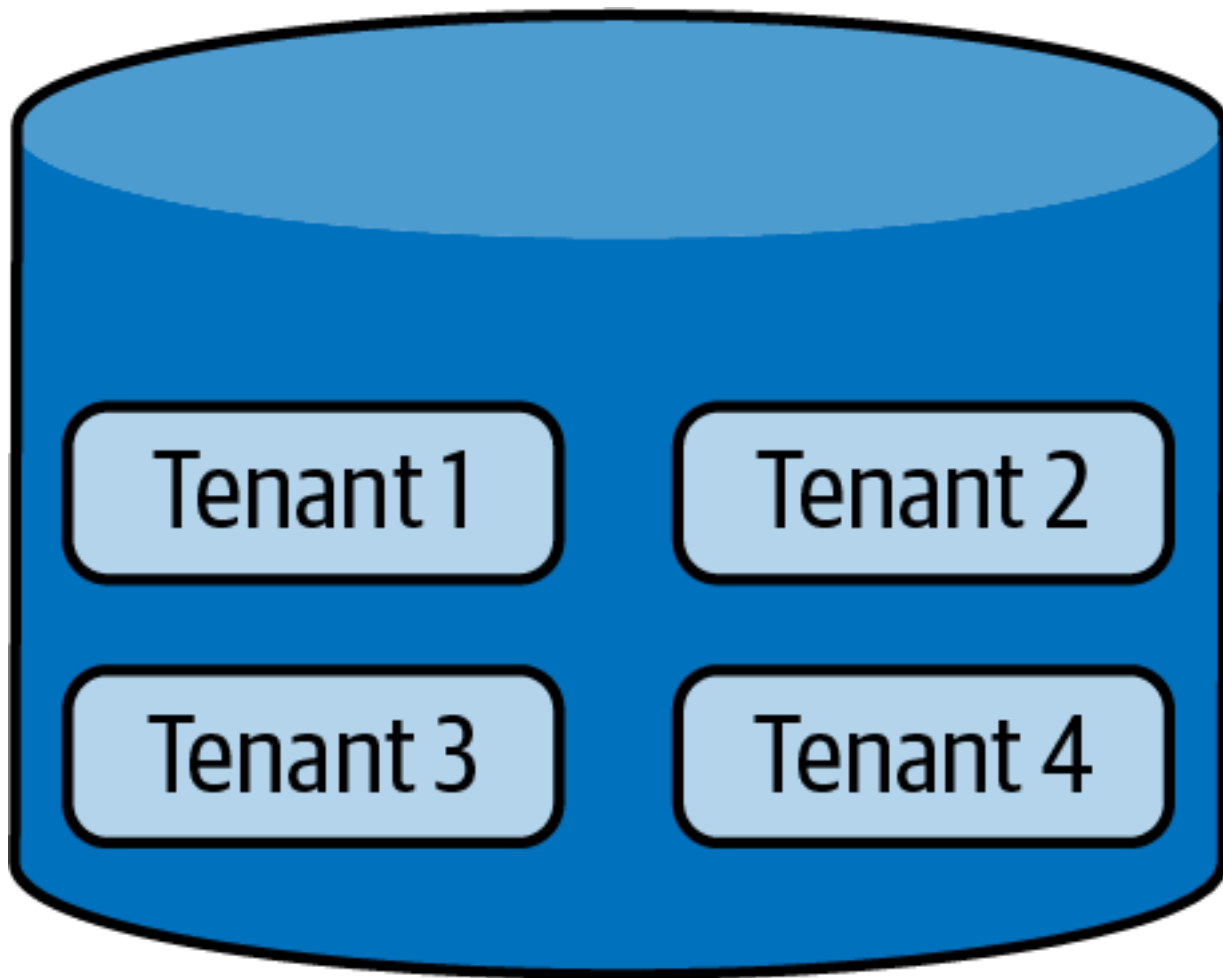


Figure 6-11. In this multitenant storage, four tenants occupy the same database. You need to be aware of querying both single and multitenant storage, which we cover in more detail in Chapter 8.

Hình 6-11. Trong lưu trữ đa thuê bao này, bốn thuê bao chiếm cùng một cơ sở dữ liệu. Bạn cần nhận thức được việc truy vấn cả lưu trữ đơn thuê bao lẫn đa thuê bao, mà chúng ta trình bày chi tiết hơn ở Chương 8.

Whom You'll Work With — Những người bạn sẽ làm việc cùng

Storage is at the heart of data engineering infrastructure. You'll interact with the people who own your IT infrastructure—typically, DevOps, security, and cloud architects. Defining domains of responsibility between data engineering and other teams is critical. Do data engineers have the authority to deploy their infrastructure in an AWS account, or must another team handle these changes? Work with other teams to define streamlined processes so that teams can work together efficiently and quickly. The division of responsibilities for data storage will depend significantly on the maturity of the organization involved. The data engineer will likely manage the storage systems and workflow if the company is early in its data maturity. If the company is later in its data maturity, the data engineer will probably manage a section of the storage system. This data engineer will also likely interact with engineers on either side of storage—ingestion and transformation. The data engineer needs to ensure that the storage systems used by downstream users are securely available, contain high-quality data, have ample storage capacity, and perform when queries and

transformations are run.

Lưu trữ nằm ở trung tâm của hạ tầng kỹ thuật dữ liệu. Bạn sẽ tương tác với những người sở hữu hạ tầng CNTT của bạn—thường là DevOps, an ninh, và các kiến trúc sư đám mây. Việc xác định các phạm vi trách nhiệm giữa kỹ thuật dữ liệu và các đội khác là then chốt. Kỹ sư dữ liệu có thẩm quyền triển khai hạ tầng của họ trong một tài khoản AWS không, hay một đội khác phải xử lý các thay đổi này? Hãy làm việc với các đội khác để xác định các quy trình được tinh giản để các đội có thể làm việc cùng nhau một cách hiệu quả và nhanh chóng. Sự phân chia trách nhiệm cho lưu trữ dữ liệu sẽ phụ thuộc đáng kể vào độ trưởng thành của tổ chức liên quan. Kỹ sư dữ liệu có thể sẽ quản lý các hệ thống lưu trữ và luồng công việc nếu công ty còn sớm trong độ trưởng thành dữ liệu của nó. Nếu công ty muộn hơn trong độ trưởng thành dữ liệu của nó, kỹ sư dữ liệu có thể sẽ quản lý một phần của hệ thống lưu trữ. Kỹ sư dữ liệu này cũng có thể sẽ tương tác với các kỹ sư ở hai phía của lưu trữ—thu nạp và biến đổi. Kỹ sư dữ liệu cần đảm bảo rằng các hệ thống lưu trữ được những người dùng hạ nguồn sử dụng là sẵn sàng một cách an toàn, chứa dữ liệu chất lượng cao, có dung lượng lưu trữ dồi dào, và hoạt động hiệu quả khi các truy vấn và biến đổi được chạy.

Undercurrents — Các dòng chảy ngầm

The **undercurrents** for storage are significant because storage is a critical hub for all stages of the data engineering lifecycle. Unlike other undercurrents for which data might be in motion (ingestion) or queried and transformed, the undercurrents for storage differ because storage is so ubiquitous.

Các dòng chảy ngầm (undercurrents) cho lưu trữ là đáng kể vì lưu trữ là một đầu mối then chốt cho tất cả các giai đoạn của vòng đời kỹ thuật dữ liệu. Không giống các dòng chảy ngầm khác nơi dữ liệu có thể đang chuyển động (thu nạp) hoặc được truy vấn và biến đổi, các dòng chảy ngầm cho lưu trữ khác biệt vì lưu trữ hiện diện ở mọi nơi.

Security — An ninh

While engineers often view security as an impediment to their work, they should embrace the idea that security is a key enabler. Robust security with fine-grained data **access control** allows data to be shared and consumed more widely within a business. The value of data goes up significantly when this is possible. As always, exercise the **principle of least privilege**. Don't give full database access to anyone unless required. This means most data engineers don't need full database access in practice. Also, pay attention to the column, row, and cell-level access controls in your database. Give users only the information they need and no more.

Trong khi kỹ sư thường coi an ninh là một cản trở đối với công việc của họ, họ nên đón nhận ý tưởng rằng an ninh là một yếu tố kích hoạt then chốt. An ninh mạnh mẽ với kiểm soát truy cập dữ liệu ở mức chi tiết cho phép dữ liệu được chia sẻ và tiêu thụ rộng rãi hơn trong một doanh nghiệp. Giá trị của dữ liệu tăng đáng kể khi điều này khả thi. Như mọi khi, hãy thực hành nguyên tắc đặc quyền tối thiểu. Đừng trao quyền truy cập cơ sở dữ liệu đầy đủ cho bất kỳ ai trừ khi được yêu cầu. Điều này có nghĩa là trên thực tế hầu hết kỹ sư dữ liệu không cần quyền truy cập cơ sở dữ liệu đầy đủ. Ngoài ra, hãy chú ý đến các kiểm soát truy cập ở mức cột, hàng và ô (cell) trong cơ sở dữ liệu của bạn. Hãy chỉ trao cho người dùng thông tin họ cần và không hơn.

Data Management — Quản lý dữ liệu

Data management is critical as we read and write data with storage systems.

Quản lý dữ liệu là then chốt khi chúng ta đọc và ghi dữ liệu với các hệ thống lưu trữ.

Data catalogs and metadata management — Danh mục dữ liệu và quản lý siêu dữ liệu

Data is enhanced by robust metadata. Cataloging enables data scientists, analysts, and ML engineers by enabling **data discovery**. Data lineage accelerates the time to track down data problems and allows consumers to locate upstream raw sources. As you build out your storage systems, invest in your metadata. Integration of a data dictionary with these other tools allows users to share and record institutional knowledge robustly. Metadata management also significantly enhances **data governance**. Beyond simply enabling passive data cataloging and lineage, consider implementing analytics over these systems to get a clear, active picture of what's happening with your data.

Dữ liệu được nâng cao bởi siêu dữ liệu mạnh mẽ. Việc lập danh mục hỗ trợ các nhà khoa học dữ liệu, nhà phân tích và kỹ sư ML bằng cách cho phép khám phá dữ liệu. Truy vết nguồn gốc dữ liệu đẩy nhanh thời gian truy ngược các vấn đề dữ liệu và cho phép các bên tiêu thụ định vị các nguồn thô thượng nguồn. Khi bạn xây dựng các hệ thống lưu trữ của mình, hãy đầu tư vào siêu dữ liệu của bạn. Việc tích hợp một từ điển dữ liệu (data dictionary) với các công cụ khác này cho phép người dùng chia sẻ và ghi lại kiến thức của tổ chức một cách mạnh mẽ. Quản lý siêu dữ liệu cũng nâng cao đáng kể quản trị dữ liệu. Ngoài việc chỉ đơn thuần cho phép lập danh mục và truy vết nguồn gốc dữ liệu một cách thụ động, hãy cân nhắc triển khai phân tích trên các hệ thống này để có một bức tranh rõ ràng, chủ động về những gì đang xảy ra với dữ liệu của bạn.

Data versioning in object storage — Phiên bản hóa dữ liệu trong lưu trữ đối tượng

Major cloud object storage systems enable data versioning. Data versioning can help with error recovery when processes fail, and data becomes corrupted. Versioning is also beneficial for tracking the history of datasets used to build models. Just as code version control allows developers to track down commits that cause bugs, data version control can aid ML engineers in tracking changes that lead to model performance degradation.

Các hệ thống lưu trữ đối tượng đám mây lớn cho phép phiên bản hóa dữ liệu. Phiên bản hóa dữ liệu có thể giúp khôi phục lỗi khi các quy trình thất bại, và dữ liệu trở nên hư hỏng. Phiên bản hóa cũng có lợi cho việc theo dõi lịch sử của các tập dữ liệu được dùng để xây dựng các mô hình. Cũng như kiểm soát phiên bản mã (code version control) cho phép các nhà phát triển truy ngược các commit gây ra lỗi, kiểm soát phiên bản dữ liệu có thể hỗ trợ các kỹ sư ML trong việc theo dõi các thay đổi dẫn đến suy giảm hiệu năng mô hình.

Privacy — Quyền riêng tư

GDPR and other privacy regulations have significantly impacted storage system design. Any data with privacy implications has a lifecycle that data engineers must manage. Data engineers must be

prepared to respond to data deletion requests and selectively remove data as required. In addition, engineers can accommodate privacy and security through anonymization and masking.

GDPR và các quy định khác về quyền riêng tư đã tác động đáng kể đến thiết kế hệ thống lưu trữ. Bất kỳ dữ liệu nào có hàm ý về quyền riêng tư đều có một vòng đời mà kỹ sư dữ liệu phải quản lý. Kỹ sư dữ liệu phải sẵn sàng đáp ứng các yêu cầu xóa dữ liệu và loại bỏ dữ liệu một cách chọn lọc theo yêu cầu. Ngoài ra, kỹ sư có thể đáp ứng quyền riêng tư và an ninh thông qua ẩn danh hóa (anonymization) và che giấu (masking).

DataOps — DataOps

DataOps is not orthogonal to data management, and a significant area of overlap exists. DataOps concerns itself with traditional operational monitoring of storage systems and monitoring the data itself, inseparable from metadata and quality.

DataOps không trực giao với quản lý dữ liệu, và tồn tại một khu vực chồng lấn đáng kể. DataOps quan tâm đến việc giám sát vận hành truyền thống của các hệ thống lưu trữ và giám sát bản thân dữ liệu, không thể tách rời khỏi siêu dữ liệu và chất lượng.

Systems monitoring — Giám sát hệ thống

Data engineers must monitor storage in a variety of ways. This includes monitoring infrastructure storage components, where they exist, but also monitoring object storage and other “serverless” systems. Data engineers should take the lead on **FinOps** (cost management), security monitoring, and access monitoring.

Kỹ sư dữ liệu phải giám sát lưu trữ theo nhiều cách khác nhau. Điều này bao gồm giám sát các thành phần lưu trữ hạ tầng, nơi chúng tồn tại, nhưng cũng giám sát lưu trữ đối tượng và các hệ thống “serverless” khác. Kỹ sư dữ liệu nên dẫn dắt về FinOps (quản lý chi phí), giám sát an ninh, và giám sát truy cập.

Observing and monitoring data — Quan sát và giám sát dữ liệu

While metadata systems as we’ve described are critical, good engineering must consider the entropic nature of data by actively seeking to understand its characteristics and watching for major changes. Engineers can monitor data statistics, apply anomaly detection methods or simple rules, and actively test and validate for logical inconsistencies.

Trong khi các hệ thống siêu dữ liệu như chúng tôi đã mô tả là then chốt, kỹ thuật tốt phải xem xét bản chất entropy của dữ liệu bằng cách chủ động tìm cách hiểu các đặc tính của nó và theo dõi các thay đổi lớn. Kỹ sư có thể giám sát các số liệu thống kê dữ liệu, áp dụng các phương pháp phát hiện bất thường hoặc các quy tắc đơn giản, và chủ động kiểm thử và xác thực các bất nhất quán về logic.

Data Architecture — Kiến trúc dữ liệu

Chapter 3 covers the basics of data architecture, as storage is the critical underbelly of the data engineering lifecycle. Consider the following data architecture tips. Design for required reliability and durability. Understand the upstream source systems and how that data, once ingested, will be

stored and accessed. Understand the types of data models and queries that will occur downstream. If data is expected to grow, can you negotiate storage with your cloud provider? Take an active approach to FinOps, and treat it as a central part of architecture conversations. Don't prematurely optimize, but prepare for scale if business opportunities exist in operating on large data volumes. Lean toward fully managed systems, and understand provider SLAs. Fully managed systems are generally far more robust and scalable than systems you have to babysit.

Chương 3 trình bày những điều cơ bản của kiến trúc dữ liệu, vì lưu trữ là phần bụng then chốt của vòng đời kỹ thuật dữ liệu. Hãy cân nhắc các mẹo kiến trúc dữ liệu sau. Hãy thiết kế cho độ tin cậy và độ bền bắt buộc. Hiểu các hệ thống nguồn thượng nguồn và cách dữ liệu đó, một khi được thu nạp, sẽ được lưu trữ và truy cập. Hiểu các loại mô hình dữ liệu và truy vấn sẽ xảy ra ở hạ nguồn. Nếu dữ liệu được kỳ vọng tăng trưởng, bạn có thể thương lượng lưu trữ với nhà cung cấp đám mây của mình không? Hãy có cách tiếp cận chủ động với FinOps, và coi nó như một phần trung tâm của các cuộc trò chuyện về kiến trúc. Đừng tối ưu hóa sớm, nhưng hãy chuẩn bị cho quy mô nếu có các cơ hội nghiệp vụ trong việc vận hành trên các khối lượng dữ liệu lớn. Hãy nghiêng về phía các hệ thống được quản lý hoàn toàn, và hiểu các SLA của nhà cung cấp. Các hệ thống được quản lý hoàn toàn nhìn chung mạnh mẽ hơn và có khả năng mở rộng tốt hơn nhiều so với các hệ thống mà bạn phải trông nom.

Orchestration — Điều phối

Orchestration is highly entangled with storage. Storage allows data to flow through pipelines, and orchestration is the pump. Orchestration also helps engineers cope with the complexity of data systems, potentially combining many storage systems and query engines.

Điều phối (orchestration) đan xen cao độ với lưu trữ. Lưu trữ cho phép dữ liệu chảy qua các đường ống, và điều phối là cái bơm. Điều phối cũng giúp kỹ sư đối phó với sự phức tạp của các hệ thống dữ liệu, có khả năng kết hợp nhiều hệ thống lưu trữ và công cụ truy vấn.

Software Engineering — Kỹ thuật phần mềm

We can think about software engineering in the context of storage in two ways. First, the code you write should perform well with your storage system. Make sure the code you write stores the data correctly and doesn't accidentally cause data, memory leaks, or performance issues. Second, define your storage infrastructure as code and use ephemeral compute resources when it's time to process your data. Because storage is increasingly distinct from compute, you can automatically spin resources up and down while keeping your data in object storage. This keeps your infrastructure clean and avoids coupling your storage and query layers.

Chúng ta có thể nghĩ về kỹ thuật phần mềm trong bối cảnh lưu trữ theo hai cách. Thứ nhất, mã bạn viết nên hoạt động tốt với hệ thống lưu trữ của bạn. Hãy đảm bảo mã bạn viết lưu trữ dữ liệu một cách chính xác và không vô tình gây ra các vấn đề về dữ liệu, rò rỉ bộ nhớ, hoặc hiệu năng. Thứ hai, hãy định nghĩa hạ tầng lưu trữ của bạn dưới dạng mã (infrastructure as code) và dùng các tài nguyên tính toán tạm thời khi đến lúc xử lý dữ liệu của bạn. Vì lưu trữ ngày càng tách biệt với tính toán, bạn có thể tự động khởi tạo và giải phóng các tài nguyên lên xuống trong khi giữ dữ liệu của bạn trong lưu trữ đối tượng. Điều này giữ cho hạ tầng của bạn gọn gàng và tránh ghép chặt tầng lưu trữ và tầng truy vấn của bạn.

Conclusion — Kết luận

Storage is everywhere and underlays many stages of the data engineering lifecycle. In this chapter, you learned about the raw ingredients, types, abstractions, and big ideas around storage systems. Gain deep knowledge of the inner workings and limitations of the storage systems you'll use. Know the types of data, activities, and workloads appropriate for your storage.

Lưu trữ hiện diện ở mọi nơi và làm nền cho nhiều giai đoạn của vòng đời kỹ thuật dữ liệu. Trong chương này, bạn đã tìm hiểu về các nguyên liệu thô, các loại, các trừu tượng, và các ý tưởng lớn xoay quanh các hệ thống lưu trữ. Hãy thu nạp kiến thức sâu về cơ chế hoạt động bên trong và các hạn chế của các hệ thống lưu trữ mà bạn sẽ dùng. Hãy biết các loại dữ liệu, hoạt động, và tải công việc phù hợp với lưu trữ của bạn.

Additional Resources

“Column-oriented DBMS” Wikipedia page

“Rowise vs. Columnar Database? Theory and in Practice” by Mangat Rai Modi

“The Design and Implementation of Modern Column-Oriented Database Systems” by Daniel Abadi et al.

“Diving Into Delta Lake: Schema Enforcement and Evolution” by Burak Yavuz et al.

“Snowflake Solution Anti-Patterns: The Probable Data Scientist” by John Aven

“The What, When, Why, and How of Incremental Loads” by Tim Mitchell

IDC’s “Data Creation and Replication Will Grow at a Faster Rate than Installed Storage Capacity, According to the IDC Global DataSphere and StorageSphere Forecasts” press release

“What Is a Vector Database?” by Bryan Turriff

“Hot Data vs. Cold Data: Why It Matters” by Afzaal Ahmad Zeeshan

1 Andy Klein, “Hard Disk Drive (HDD) vs. Solid-State Drive (SSD): What’s the Diff?,” Backblaze blog, October 5, 2021, <https://oreil.ly/XBps8> . 2 Benoit Dageville, “The Snowflake Elastic Data Warehouse,” SIGMOD ’16: Proceedings of the 2016 International Conference on Management of Data (June 2016): 215–226, <https://oreil.ly/Tc1su> . 3 James Dixon, “Data Lakes Revisited,” James Dixon’s Blog , September 25, 2014, <https://oreil.ly/FH25v> . 4 Valliappa Lakshmanan and Jordan Tigani, GoogleBig Query: The Definitive Guide (Seastopol, CA: O’Reilly, 2019), page 15, <https://oreil.ly/5aXXu>

Chapter 7. Ingestion — Chương 7.

Thu nạp (Ingestion)

You've learned about the various source systems you'll likely encounter as a **data engineer** and about ways to store data. Let's now turn our attention to the patterns and choices that apply to ingesting data from various source systems. In this chapter, we discuss data **ingestion** (see Figure 7-1), the key engineering considerations for the ingestion phase, the major patterns for **batch** and **streaming** ingestion, technologies you'll encounter, whom you'll work with as you develop your data ingestion pipeline, and how the **undercurrents** feature in the ingestion phase.

Bạn đã tìm hiểu về các hệ thống nguồn (source system) khác nhau mà bạn nhiều khả năng sẽ gặp với tư cách kỹ sư dữ liệu, và về các cách lưu trữ dữ liệu. Giờ hãy hướng sự chú ý sang các mẫu hình và lựa chọn áp dụng cho việc thu nạp (ingestion) dữ liệu từ nhiều hệ thống nguồn khác nhau. Trong chương này, chúng ta bàn về thu nạp dữ liệu (xem Hình 7-1), các cân nhắc kỹ thuật chính cho giai đoạn thu nạp, các mẫu hình chính cho thu nạp theo lô (batch) và theo luồng (streaming), các công nghệ bạn sẽ gặp, những người bạn sẽ làm việc cùng khi phát triển đường ống thu nạp dữ liệu của mình, và cách các dòng chảy ngầm (undercurrents) thể hiện trong giai đoạn thu nạp.

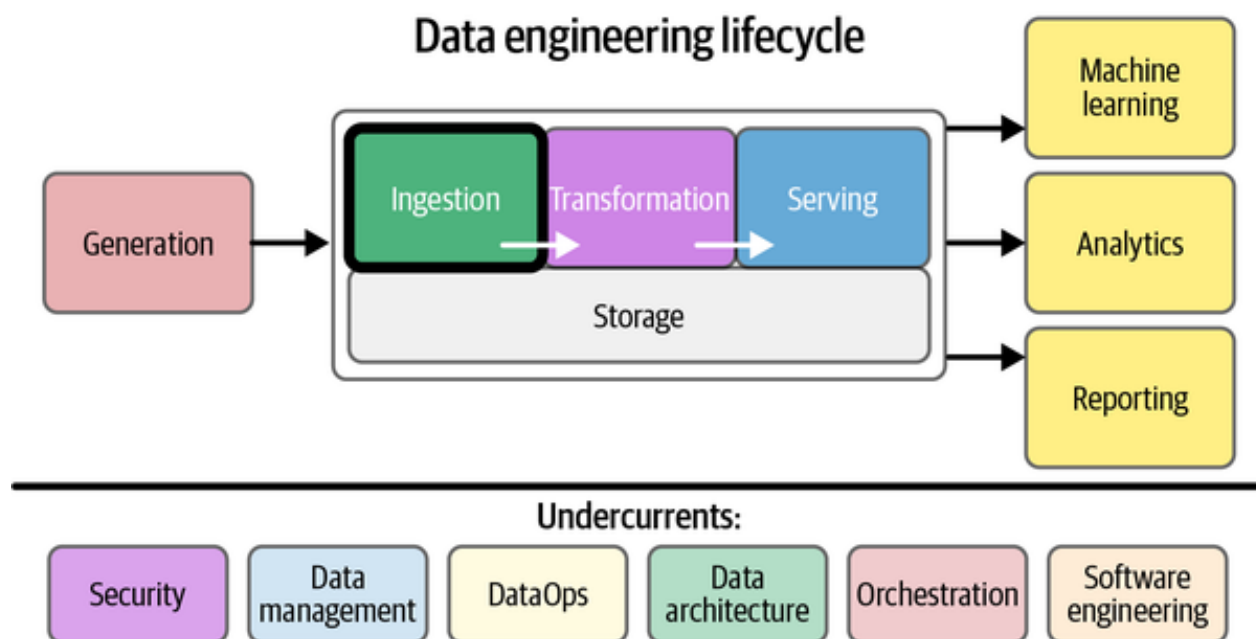


Figure 7-1. To begin processing data, we must ingest it

Hình 7-1. Để bắt đầu xử lý dữ liệu, trước tiên chúng ta phải thu nạp nó

What Is Data Ingestion? — Thu nạp dữ liệu là gì?

Data ingestion is the process of moving data from one place to another. Data ingestion implies data movement from source systems into **storage** in the **data engineering lifecycle**, with ingestion as an intermediate step (Figure 7-2).

Thu nạp dữ liệu là quá trình di chuyển dữ liệu từ nơi này sang nơi khác. Thu nạp dữ liệu ngụ ý sự di chuyển dữ liệu từ các hệ thống nguồn vào kho lưu trữ trong vòng đời kỹ thuật dữ liệu, với thu nạp là một bước trung gian (Hình 7-2).



Figure 7-2. Data from system 1 is ingested into system 2 It's worth quickly contrasting data ingestion with data integration. Whereas data ingestion is data movement from point A to B, data integration combines data from disparate sources into a new dataset. For example, you can use data integration to combine data from a CRM system, advertising **analytics** data, and web analytics to create a user profile, which is saved to your **data warehouse**. Furthermore, using **reverse ETL**, you can send this newly created user profile back to your CRM so salespeople can use the data for prioritizing leads. We describe data integration more fully in Chapter 8 , where we discuss data transformations; reverse **ETL** is covered in Chapter 9 . We also point out that data ingestion is different from internal ingestion within a system. Data stored in a database is copied from one table to another, or data in a stream is temporarily cached. We consider this another part of the general data **transformation** process covered in Chapter 8 .

Hình 7-2. Dữ liệu từ hệ thống 1 được thu nạp vào hệ thống 2. Cần phân biệt nhanh giữa thu nạp dữ liệu với tích hợp dữ liệu (data integration). Trong khi thu nạp dữ liệu là sự di chuyển dữ liệu từ điểm A sang điểm B, thì tích hợp dữ liệu kết hợp dữ liệu từ các nguồn khác nhau thành một tập dữ liệu mới. Ví dụ, bạn có thể dùng tích hợp dữ liệu để kết hợp dữ liệu từ một hệ thống CRM, dữ liệu phân tích quảng cáo, và phân tích web để tạo một hồ sơ người dùng, sau đó lưu vào kho dữ liệu (data warehouse) của bạn. Hơn nữa, bằng cách dùng reverse ETL, bạn có thể gửi hồ sơ người dùng vừa tạo này trở lại CRM để nhân viên bán hàng dùng dữ liệu đó nhằm ưu tiên các khách hàng tiềm năng (lead). Chúng tôi mô tả tích hợp dữ liệu đầy đủ hơn trong Chương 8, nơi chúng ta bàn về biến đổi (transformation) dữ liệu; reverse ETL được đề cập trong Chương 9. Chúng tôi cũng lưu ý rằng thu nạp dữ liệu khác với thu nạp nội bộ bên trong một hệ thống. Dữ liệu lưu trong một database được sao chép từ bảng này sang bảng khác, hoặc dữ liệu trong một stream được cache tạm thời. Chúng tôi xem đây là một phần khác của quá trình biến đổi dữ liệu chung được đề cập trong Chương 8.

DATA PIPELINES DEFINED — ĐỊNH NGHĨA ĐƯỜNG ỐNG DỮ LIỆU

Data pipelines begin in source systems, but ingestion is the stage where data engineers begin actively designing **data pipeline** activities. In the **data engineering** space, a good deal of ceremony occurs around data movement and processing patterns, with established patterns such as ETL, newer patterns such as **ELT**, and new names for long-established practices (reverse ETL) and data sharing. All of these concepts are encompassed in the idea of a data pipeline . It is essential to understand the details of these various patterns and know that a modern data pipeline includes all

of them. As the world moves away from a traditional monolithic approach with rigid constraints on data movement, and toward an open ecosystem of cloud services that are assembled like LEGO bricks to realize products, data engineers prioritize using the right tools to accomplish the desired outcome over adhering to a narrow philosophy of data movement. In general, here's our definition of a data pipeline:

Các đường ống dữ liệu (data pipeline) bắt đầu trong các hệ thống nguồn, nhưng thu nạp là giai đoạn mà các kỹ sư dữ liệu bắt đầu chủ động thiết kế các hoạt động của đường ống dữ liệu. Trong lĩnh vực kỹ thuật dữ liệu, có khá nhiều nghi thức xoay quanh các mẫu hình di chuyển và xử lý dữ liệu, với các mẫu đã được thiết lập như ETL, các mẫu mới hơn như ELT, và những cái tên mới cho các thực hành đã tồn tại từ lâu (reverse ETL) cùng chia sẻ dữ liệu (data sharing). Tất cả các khái niệm này đều được bao hàm trong ý tưởng về một đường ống dữ liệu. Việc hiểu chi tiết của các mẫu hình khác nhau này là thiết yếu, và cần biết rằng một đường ống dữ liệu hiện đại bao gồm tất cả chúng. Khi thế giới rời xa cách tiếp cận monolith truyền thống với các ràng buộc cứng nhắc về di chuyển dữ liệu, và tiến tới một hệ sinh thái mở gồm các dịch vụ đám mây được lắp ghép như những viên gạch LEGO để tạo ra sản phẩm, các kỹ sư dữ liệu ưu tiên dùng đúng công cụ để đạt được kết quả mong muốn hơn là bám vào một triết lý hẹp về di chuyển dữ liệu. Nhìn chung, đây là định nghĩa của chúng tôi về một đường ống dữ liệu:

A data pipeline is the combination of architecture, systems, and processes that move data through the stages of the data engineering lifecycle. Our definition is deliberately fluid—and intentionally vague—to allow data engineers to plug in whatever they need to accomplish the task at hand. A data pipeline could be a traditional ETL system, where data is ingested from an on-premises transactional system, passed through a monolithic processor, and written into a data warehouse. Or it could be a cloud-based data pipeline that pulls data from 100 sources, combines it into 20 wide tables, trains five other ML models, deploys them into production, and monitors ongoing performance. A data pipeline should be flexible enough to fit any needs along the data engineering lifecycle. Let's keep this notion of data pipelines in mind as we proceed through this chapter.

Một đường ống dữ liệu là sự kết hợp của kiến trúc, hệ thống và quy trình giúp di chuyển dữ liệu qua các giai đoạn của vòng đời kỹ thuật dữ liệu. Định nghĩa của chúng tôi cố ý linh hoạt—và cố ý mơ hồ—để cho phép các kỹ sư dữ liệu gắn vào bất cứ thứ gì họ cần nhằm hoàn thành nhiệm vụ trước mắt. Một đường ống dữ liệu có thể là một hệ thống ETL truyền thống, nơi dữ liệu được thu nạp từ một hệ thống giao dịch tại chỗ (on-premises), đi qua một bộ xử lý monolith, rồi được ghi vào một kho dữ liệu. Hoặc nó có thể là một đường ống dữ liệu dựa trên đám mây kéo dữ liệu từ 100 nguồn, kết hợp vào 20 bảng rộng (wide table), huấn luyện năm mô hình học máy (machine learning) khác, triển khai chúng vào sản xuất (production), và giám sát hiệu năng liên tục. Một đường ống dữ liệu cần đủ linh hoạt để phù hợp với mọi nhu cầu dọc theo vòng đời kỹ thuật dữ liệu. Hãy giữ khái niệm về đường ống dữ liệu này trong đầu khi chúng ta đi tiếp qua chương này.

Key Engineering Considerations for the Ingestion Phase — Các cân nhắc kỹ thuật chính cho giai đoạn thu nạp

When preparing to architect or build an ingestion system, here are some primary considerations and questions to ask yourself related to data ingestion:

Khi chuẩn bị kiến trúc hoặc xây dựng một hệ thống thu nạp, đây là một số cân nhắc và câu hỏi chính bạn nên tự đặt ra liên quan đến thu nạp dữ liệu:

What's the use case for the data I'm ingesting?

Trường hợp sử dụng (use case) cho dữ liệu tôi đang thu nạp là gì?

Can I reuse this data and avoid ingesting multiple versions of the same dataset?

Tôi có thể tái sử dụng dữ liệu này và tránh thu nạp nhiều phiên bản của cùng một tập dữ liệu không?

Where is the data going? What's the destination?

Dữ liệu sẽ đi về đâu? Đích đến là gì?

How often should the data be updated from the source?

Dữ liệu nên được cập nhật từ nguồn bao lâu một lần?

What is the expected data volume?

Khối lượng dữ liệu kỳ vọng là bao nhiêu?

What format is the data in? Can **downstream** storage and transformation accept this format?

Dữ liệu ở định dạng nào? Kho lưu trữ và biến đổi ở hạ nguồn (downstream) có chấp nhận định dạng này không?

Is the source data in good shape for immediate downstream use? That is, is the data of good quality? What post-processing is required to serve it? What are data-quality risks (e.g., could bot traffic to a website contaminate the data)?

Dữ liệu nguồn có ở trạng thái tốt để dùng ngay ở hạ nguồn không? Tức là, dữ liệu có chất lượng tốt không? Cần xử lý hậu kỳ nào để phục vụ nó? Các rủi ro về chất lượng dữ liệu là gì (ví dụ, liệu lưu lượng bot vào một website có thể làm nhiễm bẩn dữ liệu không)?

Does the data require in-flight processing for downstream ingestion if the data is from a streaming source?

Dữ liệu có cần xử lý trong khi truyền (in-flight) cho việc thu nạp hạ nguồn nếu dữ liệu đến từ một nguồn streaming không?

These questions undercut batch and streaming ingestion and apply to the underlying architecture you'll create, build, and maintain. Regardless of how often the data is ingested, you'll want to consider these factors when designing your ingestion architecture:

Những câu hỏi này nằm dưới cả thu nạp theo lô và theo luồng, và áp dụng cho kiến trúc nền tảng mà bạn sẽ tạo ra, xây dựng và bảo trì. Bất kể dữ liệu được thu nạp bao lâu một lần, bạn sẽ muốn cân nhắc những yếu tố này khi thiết kế kiến trúc thu nạp của mình:

Bounded versus unbounded

Có biên so với không biên (bounded versus unbounded)

Frequency

Tần suất

Synchronous versus asynchronous **Serialization** and deserialization

Thu nạp đồng bộ so với bất đồng bộ. Tuần tự hóa (serialization) và giải tuần tự hóa (deserialization)

Throughput and elastic **scalability**

Thông lượng (throughput) và khả năng mở rộng co giãn (elastic scalability)

Reliability and durability

Độ tin cậy và độ bền (reliability and durability)

Payload

Tải trọng (payload)

Push versus pull versus poll patterns

Các mẫu hình đẩy so với kéo so với thăm dò (push versus pull versus poll)

Let's look at each of these.

Hãy xem xét từng yếu tố này.

Bounded Versus Unbounded — Có biên so với Không biên

As you might recall from Chapter 3 , data comes in two forms: bounded and unbounded (Figure 7-3).

Unbounded data is data as it exists in reality, as events happen, either sporadically or continuously, ongoing and flowing. Bounded data is a convenient way of bucketing data across some sort of boundary, such as time.

Như bạn có thể nhớ từ Chương 3, dữ liệu đến ở hai dạng: có biên và không biên (Hình 7-3). Dữ liệu không biên là dữ liệu như nó tồn tại trong thực tế, khi các sự kiện xảy ra, hoặc rải rác hoặc liên tục, không ngừng và tuôn chảy. Dữ liệu có biên là một cách tiện lợi để gom dữ liệu vào các nhóm theo một loại ranh giới nào đó, chẳng hạn như thời gian.

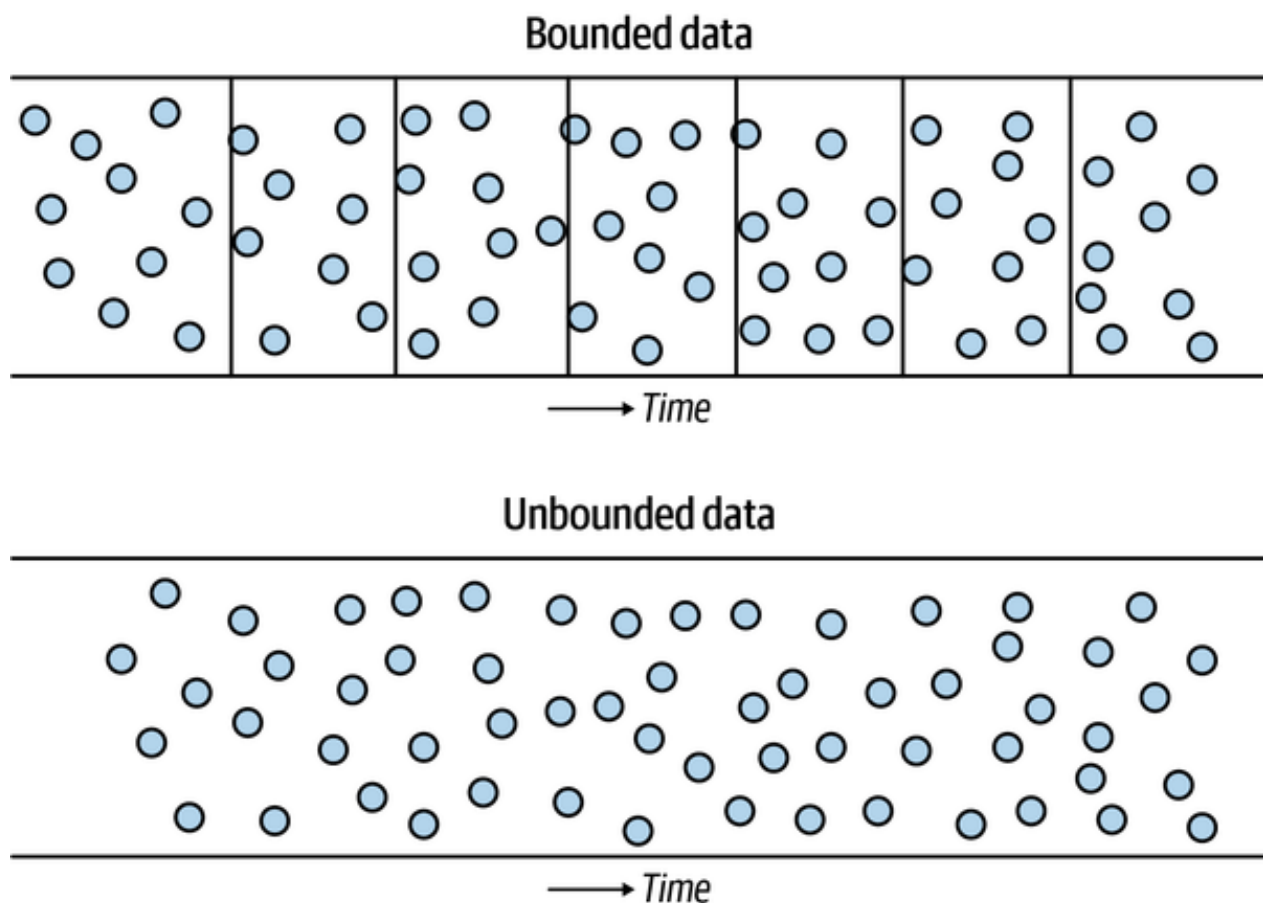


Figure 7-3. Bounded versus unbounded data Let us adopt this mantra: All data is unbounded until it's bounded. Like many mantras, this one is not precisely accurate 100% of the time. The grocery list that I scribbled this afternoon is bounded data. I wrote it as a stream of consciousness (unbounded data) onto a piece of scrap paper, where the thoughts now exist as a list of things (bounded data) I need to buy at the grocery store. However, the idea is correct for practical purposes for the vast majority of data you'll handle in a business context. For example, an online retailer will process customer transactions 24 hours a day until the business fails, the economy grinds to a halt, or the sun explodes. Business processes have long imposed artificial bounds on data by cutting discrete batches. Keep in mind the true unboundedness of your data; streaming ingestion systems are simply a tool for preserving the unbounded nature of data so that subsequent steps in the lifecycle can also process it continuously.

Hình 7-3. Dữ liệu có biên so với không biên. Hãy áp dụng câu thần chú này: Mọi dữ liệu đều không biên cho đến khi nó được đặt biên. Như nhiều câu thần chú, câu này không chính xác tuyệt đối 100% mọi lúc. Danh sách đi chợ mà tôi nguệch ngoạc viết chiều nay là dữ liệu có biên. Tôi viết nó như một dòng ý thức (dữ liệu không biên) lên một mảnh giấy nháp, nơi các suy nghĩ giờ tồn tại như một danh sách những thứ (dữ liệu có biên) tôi cần mua ở cửa hàng tạp hóa. Tuy nhiên, ý tưởng này đúng cho mục đích thực tiễn đối với đa số dữ liệu bạn sẽ xử lý trong bối cảnh kinh doanh. Ví dụ, một nhà bán lẻ trực tuyến sẽ xử lý các giao dịch của khách hàng 24 giờ mỗi ngày cho đến khi doanh nghiệp phá sản, nền kinh tế đình trệ, hoặc mặt trời nổ tung. Các quy trình kinh doanh từ lâu đã áp đặt các biên nhân tạo lên dữ liệu bằng cách cắt thành các lô rời rạc. Hãy ghi nhớ tính không biên thực sự của dữ liệu của bạn; các hệ thống thu nạp streaming chỉ đơn giản là một công cụ để bảo toàn bản chất không biên của dữ liệu sao cho các bước tiếp theo trong vòng đời cũng có thể xử lý nó liên tục.

Frequency — Tần suất

One of the critical decisions that data engineers must make in designing data-ingestion processes is the data-ingestion frequency. Ingestion processes can be batch, micro-batch, or real-time. Ingestion frequencies vary dramatically from slow to fast (Figure 7-4). On the slow end, a business might ship its tax data to an accounting firm once a year. On the faster side, a CDC system could retrieve new log updates from a source database once a minute. Even faster, a system might continuously ingest events from IoT sensors and process these within seconds. Data-ingestion frequencies are often mixed in a company, depending on the use case and technologies.

Một trong những quyết định then chốt mà các kỹ sư dữ liệu phải đưa ra khi thiết kế các quy trình thu nạp dữ liệu là tần suất thu nạp dữ liệu. Các quy trình thu nạp có thể là theo lô (batch), vi lô (micro-batch), hoặc thời gian thực (real-time). Tần suất thu nạp biến đổi rất lớn từ chậm đến nhanh (Hình 7-4). Ở đầu chậm, một doanh nghiệp có thể gửi dữ liệu thuế của mình cho một công ty kế toán mỗi năm một lần. Ở phía nhanh hơn, một hệ thống CDC có thể lấy các bản cập nhật log mới từ một database nguồn mỗi phút một lần. Thậm chí nhanh hơn, một hệ thống có thể liên tục thu nạp các sự kiện từ các cảm biến IoT và xử lý chúng trong vòng vài giây. Tần suất thu nạp dữ liệu thường được pha trộn trong một công ty, tùy thuộc vào trường hợp sử dụng và công nghệ.

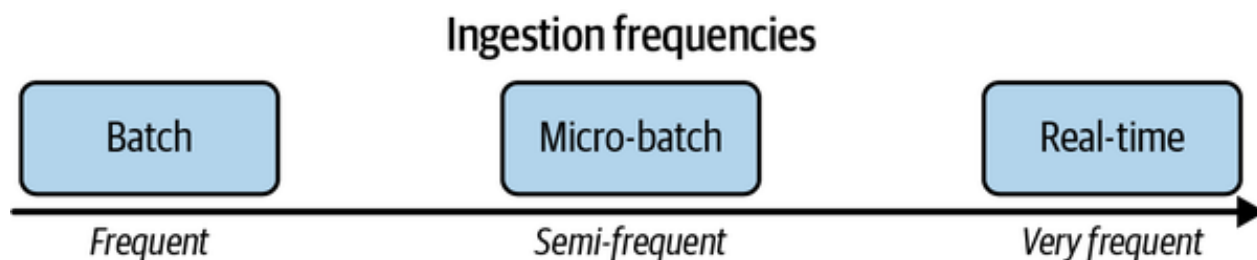


Figure 7-4. The spectrum batch to real-time ingestion frequencies We note that “real-time” ingestion patterns are becoming increasingly common. We put “real-time” in quotation marks because no ingestion system is genuinely real-time. Any database, queue or pipeline has inherent **latency** in delivering data to a target system. It is more accurate to speak of near real-time, but we often use real-time for brevity. The near real-time pattern generally does away with an explicit update frequency; events are processed in the pipeline either one by one as they arrive or in micro-batches (i.e., batches over concise time intervals). For this book, we will use real-time and streaming interchangeably. Even with a streaming data-ingestion process, batch processing downstream is relatively standard. At the time of this writing, ML models are typically trained on a batch basis, although continuous online training is becoming more prevalent. Rarely do data engineers have the option to build a purely near real-time pipeline with no batch components. Instead, they choose where batch boundaries will occur—i.e., the data engineering lifecycle data will be broken into batches. Once data reaches a batch process, the batch frequency becomes a bottleneck for all downstream processing. In addition, streaming systems are the best fit for many data source types. In IoT applications, the typical pattern is for each sensor to write events or measurements to streaming systems as they happen. While this data can be written directly into a database, a streaming ingestion platform such as Amazon Kinesis or Apache Kafka is a better fit for the application. Software applications can adopt similar patterns by writing events to a **message queue** as they happen rather than waiting for an extraction process to pull events and state information from a backend database. This pattern works exceptionally well for **event-driven** architectures already exchanging messages through queues. And again, streaming architectures generally coexist with batch processing.

Hình 7-4. Phổ tần suất thu nạp từ theo lô đến thời gian thực. Chúng tôi lưu ý rằng các mẫu hình thu nạp “thời gian thực” đang ngày càng phổ biến. Chúng tôi đặt “thời gian thực” trong ngoặc kép vì không có hệ thống thu nạp nào thực sự là thời gian thực thuần túy. Bất kỳ database, hàng đợi (queue) hay đường ống nào đều có độ trễ (latency) cố hữu trong việc đưa dữ liệu đến hệ thống đích. Sẽ chính xác hơn khi nói gần thời gian thực (near real-time), nhưng chúng tôi thường dùng thời gian thực cho ngắn gọn. Mẫu hình gần thời gian thực nhìn chung loại bỏ một tần suất cập nhật rõ ràng; các sự kiện được xử lý trong đường ống hoặc từng cái một khi chúng đến hoặc theo vi lô (tức là các lô trong những khoảng thời gian ngắn gọn). Trong cuốn sách này, chúng tôi sẽ dùng thời gian thực và streaming thay thế cho nhau. Ngay cả với một quy trình thu nạp dữ liệu streaming, xử lý theo lô ở hạ nguồn vẫn tương đối tiêu chuẩn. Tại thời điểm viết cuốn sách này, các mô hình học máy thường được huấn luyện trên cơ sở theo lô, mặc dù huấn luyện trực tuyến liên tục đang trở nên phổ biến hơn. Hiếm khi các kỹ sư dữ liệu có lựa chọn xây dựng một đường ống thuần gần thời gian thực mà không có thành phần theo lô nào. Thay vào đó, họ chọn nơi các biên lô sẽ xuất hiện—tức là dữ liệu vòng đời kỹ thuật dữ liệu sẽ được chia thành các lô. Một khi dữ liệu chạm tới một quy trình theo lô, tần suất lô trở thành nút thắt cổ chai cho mọi xử lý ở hạ nguồn. Ngoài ra, các hệ thống streaming phù hợp nhất với nhiều loại nguồn dữ liệu. Trong các ứng dụng IoT, mẫu hình điển hình là mỗi cảm biến ghi các sự kiện hoặc số đo vào các hệ thống streaming khi chúng xảy ra. Mặc dù dữ liệu này có thể được ghi trực tiếp vào một database, một nền tảng thu nạp streaming như Amazon Kinesis hoặc Apache Kafka phù hợp hơn với ứng dụng. Các ứng dụng phần mềm có thể áp dụng các mẫu hình

tương tự bằng cách ghi các sự kiện vào một hàng đợi tin nhắn (message queue) khi chúng xảy ra thay vì chờ một quy trình trích xuất kéo các sự kiện và thông tin trạng thái từ một database backend. Mẫu hình này hoạt động đặc biệt tốt cho các kiến trúc hướng sự kiện vốn đã trao đổi tin nhắn qua các hàng đợi. Và một lần nữa, các kiến trúc streaming nhìn chung cùng tồn tại với xử lý theo lô.

Synchronous Versus Asynchronous Ingestion — Thu nạp Đồng bộ so với Bất đồng bộ

With synchronous ingestion, the source, ingestion, and destination have complex dependencies and are **tightly coupled**. As you can see in Figure 7-5, each stage of the data engineering lifecycle has processes A, B, and C directly dependent upon one another. If process A fails, processes B and C cannot start; if process B fails, process C doesn't start. This type of synchronous workflow is common in older ETL systems, where data extracted from a **source system** must then be transformed before being loaded into a data warehouse. Processes downstream of ingestion can't start until all data in the batch has been ingested. If the ingestion or transformation process fails, the entire process must be rerun.

Với thu nạp đồng bộ (synchronous ingestion), nguồn, thu nạp và đích có các phụ thuộc phức tạp và được ghép chặt (tightly coupled). Như bạn có thể thấy trong Hình 7-5, mỗi giai đoạn của vòng đời kỹ thuật dữ liệu có các quy trình A, B và C phụ thuộc trực tiếp vào nhau. Nếu quy trình A thất bại, các quy trình B và C không thể bắt đầu; nếu quy trình B thất bại, quy trình C không khởi động. Loại luồng công việc đồng bộ này phổ biến trong các hệ thống ETL cũ, nơi dữ liệu được trích xuất từ một hệ thống nguồn phải được biến đổi trước khi được nạp (load) vào một kho dữ liệu. Các quy trình ở hạ nguồn của thu nạp không thể bắt đầu cho đến khi tất cả dữ liệu trong lô đã được thu nạp. Nếu quy trình thu nạp hoặc biến đổi thất bại, toàn bộ quy trình phải được chạy lại.

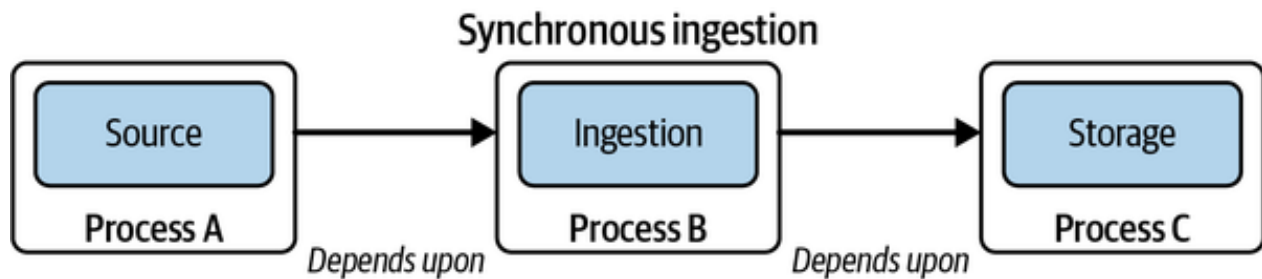


Figure 7-5. A synchronous ingestion process runs as discrete batch steps Here's a mini case study of how not to design your data pipelines. At one company, the transformation process itself was a series of dozens of tightly coupled synchronous workflows, with the entire process taking over 24 hours to finish. If any step of that transformation pipeline failed, the whole transformation process had to be restarted from the beginning! In this instance, we saw process after process fail, and because of nonexistent or cryptic error messages, fixing the pipeline was a game of whack-a-mole that took over a week to diagnose and cure. Meanwhile, the business didn't have updated reports during that time. People weren't happy. With asynchronous ingestion, dependencies can now operate at the level of individual events, much as they would in a software backend built from **microservices** (Figure 7-6). Individual events become available in storage as soon as they are ingested individually. Take the example of a web application on AWS that emits events into Amazon Kinesis Data Streams (here acting as a buffer). The stream is read by Apache Beam, which parses and enriches events, and then forwards them to a second Kinesis stream; Kinesis Data Firehose rolls up events and writes objects

to Amazon S3.

Hình 7-5. Một quy trình thu nạp đồng bộ chạy như các bước lô rời rạc. Đây là một nghiên cứu tình huống nhỏ về cách KHÔNG nên thiết kế các đường ống dữ liệu của bạn. Tại một công ty, bản thân quy trình biến đổi là một chuỗi hàng chục luồng công việc đồng bộ ghép chặt, với toàn bộ quy trình mất hơn 24 giờ để hoàn thành. Nếu bất kỳ bước nào của đường ống biến đổi đó thất bại, toàn bộ quy trình biến đổi phải được khởi động lại từ đầu! Trong trường hợp này, chúng tôi thấy hết quy trình này đến quy trình khác thất bại, và vì các thông báo lỗi không tồn tại hoặc khó hiểu, việc sửa đường ống là một trò đập chuột chũi (whack-a-mole) mất hơn một tuần để chẩn đoán và khắc phục. Trong khi đó, doanh nghiệp không có các báo cáo được cập nhật trong khoảng thời gian đó. Mọi người không vui. Với thu nạp bất đồng bộ (asynchronous ingestion), các phụ thuộc giờ có thể hoạt động ở cấp độ các sự kiện riêng lẻ, giống như chúng sẽ hoạt động trong một backend phần mềm được xây dựng từ microservices (Hình 7-6). Các sự kiện riêng lẻ trở nên sẵn có trong kho lưu trữ ngay khi chúng được thu nạp riêng lẻ. Lấy ví dụ một ứng dụng web trên AWS phát ra các sự kiện vào Amazon Kinesis Data Streams (ở đây đóng vai trò một bộ đệm). Stream được đọc bởi Apache Beam, phân tích cú pháp và làm giàu các sự kiện, rồi chuyển tiếp chúng đến một Kinesis stream thứ hai; Kinesis Data Firehose gộp các sự kiện và ghi các đối tượng vào Amazon S3.



Figure 7-6. Asynchronous processing of an event stream in AWS The big idea is that rather than relying on asynchronous processing, where a batch process runs for each stage as the input batch closes and certain time conditions are met, each stage of the asynchronous pipeline can process data items as they become available in parallel across the Beam cluster. The processing rate depends on available resources. The Kinesis Data Stream acts as the shock absorber, moderating the load so that event rate spikes will not overwhelm downstream processing. Events will move through the pipeline quickly when the event rate is low, and any backlog has cleared. Note that we could modify the scenario and use a Kinesis Data Stream for storage, eventually extracting events to S3 before they expire out of the stream.

Hình 7-6. Xử lý bất đồng bộ một stream sự kiện trong AWS. Ý tưởng lớn là thay vì dựa vào xử lý bất đồng bộ, nơi một quy trình theo lô chạy cho mỗi giai đoạn khi lô đầu vào đóng lại và các điều kiện thời gian nhất định được đáp ứng, mỗi giai đoạn của đường ống bất đồng bộ có thể xử lý các mục dữ liệu khi chúng trở nên sẵn có một cách song song trên cụm Beam. Tốc độ xử lý phụ thuộc vào tài nguyên sẵn có. Kinesis Data Stream đóng vai trò một bộ giảm xóc, điều tiết tải sao cho các đỉnh tốc độ sự kiện sẽ không làm quá tải xử lý ở hạ nguồn. Các sự kiện sẽ di chuyển qua đường ống nhanh chóng khi tốc độ sự kiện thấp, và bất kỳ tồn đọng nào đã được giải tỏa. Lưu ý rằng chúng ta có thể sửa đổi kịch bản và dùng một Kinesis Data Stream để lưu trữ, cuối cùng trích xuất các sự kiện ra S3 trước khi chúng hết hạn khỏi stream.

Serialization and Deserialization — Tuần tự hóa và Giải tuần tự hóa

Moving data from source to destination involves serialization and deserialization. As a reminder, serialization means encoding the data from a source and preparing data structures for transmission and intermediate storage stages. When ingesting data, ensure that your destination can deserialize

the data it receives. We've seen data ingested from a source but then sitting inert and unusable in the destination because the data cannot be properly deserialized. See the more extensive discussion of serialization in Appendix A .

Di chuyển dữ liệu từ nguồn đến đích liên quan đến tuần tự hóa (serialization) và giải tuần tự hóa (deserialization). Như một lời nhắc, tuần tự hóa nghĩa là mã hóa dữ liệu từ một nguồn và chuẩn bị các cấu trúc dữ liệu cho việc truyền và các giai đoạn lưu trữ trung gian. Khi thu nạp dữ liệu, hãy đảm bảo rằng đích của bạn có thể giải tuần tự hóa dữ liệu mà nó nhận được. Chúng tôi đã thấy dữ liệu được thu nạp từ một nguồn nhưng sau đó nằm im và không thể sử dụng tại đích vì dữ liệu không thể được giải tuần tự hóa đúng cách. Xem thảo luận rộng hơn về tuần tự hóa trong Phụ lục A.

Throughput and Scalability — Thông lượng và Khả năng mở rộng

In theory, your ingestion should never be a bottleneck. In practice, ingestion bottlenecks are pretty standard. Data throughput and system scalability become critical as your data volumes grow and requirements change. Design your systems to scale and shrink to flexibly match the desired data throughput. Where you're ingesting data from matters a lot. If you're receiving data as it's generated, will the **upstream** system have any issues that might impact your downstream ingestion pipelines? For example, suppose a source database goes down. When it comes back online and attempts to backfill the lapsed data loads, will your ingestion be able to keep up with this sudden influx of backlogged data? Another thing to consider is your ability to handle bursty data ingestion. Data **generation** rarely happens at a constant rate and often ebbs and flows. Built-in buffering is required to collect events during rate spikes to prevent data from getting lost. Buffering bridges the time while the system scales and allows storage systems to accommodate bursts even in a dynamically scalable system. Whenever possible, use managed services that handle the throughput **scaling** for you. While you can manually accomplish these tasks by adding more servers, shards, or workers, often this isn't value-added work, and there's a good chance you'll miss something. Much of this heavy lifting is now automated. Don't reinvent the data ingestion wheel if you don't have to.

Về lý thuyết, việc thu nạp của bạn không bao giờ nên là một nút thắt cổ chai. Trên thực tế, các nút thắt cổ chai thu nạp khá phổ biến. Thông lượng dữ liệu (data throughput) và khả năng mở rộng của hệ thống trở nên then chốt khi khối lượng dữ liệu của bạn tăng lên và các yêu cầu thay đổi. Hãy thiết kế các hệ thống của bạn để mở rộng và thu nhỏ nhằm khớp một cách linh hoạt với thông lượng dữ liệu mong muốn. Nơi bạn thu nạp dữ liệu từ đó rất quan trọng. Nếu bạn nhận dữ liệu khi nó được tạo ra, liệu hệ thống thượng nguồn (upstream) có vấn đề nào có thể ảnh hưởng đến các đường ống thu nạp ở hạ nguồn của bạn không? Ví dụ, giả sử một database nguồn ngừng hoạt động. Khi nó trở lại trực tuyến và cố gắng lấp đầy (backfill) các đợt nạp dữ liệu bị bỏ lỡ, liệu việc thu nạp của bạn có thể theo kịp dòng dữ liệu tồn đọng đột ngột này không? Một điều khác cần cân nhắc là khả năng xử lý thu nạp dữ liệu theo cụm bùng nổ (bursty). Việc sinh dữ liệu hiếm khi xảy ra ở tốc độ không đổi và thường lên xuống. Cần có bộ đệm tích hợp sẵn để thu thập các sự kiện trong các đỉnh tốc độ nhằm ngăn dữ liệu bị mất. Bộ đệm bắc cầu qua khoảng thời gian khi hệ thống mở rộng và cho phép các hệ thống lưu trữ thích ứng với các đợt bùng nổ ngay cả trong một hệ thống có thể mở rộng động. Bất cứ khi nào có thể, hãy dùng các dịch vụ được quản lý (managed service) lo việc mở rộng thông lượng cho bạn. Mặc dù bạn có thể tự hoàn thành các tác vụ này bằng cách thêm nhiều server, shard hoặc worker, thường thì đây không phải công việc tạo giá trị gia tăng, và có khả năng cao bạn sẽ bỏ sót điều gì đó. Phần lớn công việc nặng nhọc này giờ đã được tự động hóa. Đừng phát minh lại bánh xe thu nạp dữ liệu nếu bạn không cần phải làm vậy.

Reliability and Durability — Độ tin cậy và Độ bền

Reliability and durability are vital in the ingestion stages of data pipelines. Reliability entails high uptime and proper failover for ingestion systems. Durability entails making sure that data isn't lost or corrupted. Some data sources (e.g., IoT devices and caches) may not retain data if it is not correctly ingested. Once lost, it is gone for good. In this sense, the reliability of ingestion systems leads directly to the durability of generated data. If data is ingested, downstream processes can theoretically run late if they break temporarily. Our advice is to evaluate the risks and build an appropriate level of redundancy and self-healing based on the impact and cost of losing data. Reliability and durability have both direct and indirect costs. For example, will your ingestion process continue if an AWS zone goes down? How about a whole region? How about the power grid or the internet? Of course, nothing is free. How much will this cost you? You might be able to build a highly redundant system and have a team on call 24 hours a day to handle outages. This also means your cloud and labor costs become prohibitive (direct costs), and the ongoing work takes a significant toll on your team (indirect costs). There's no correct answer, and you need to evaluate the costs and benefits of your reliability and durability decisions. Don't assume that you can build a system that will reliably and durably ingest data in every possible scenario. Even the nearly infinite budget of the US federal government can't guarantee this. In many extreme scenarios, ingesting data actually won't matter. There will be little to ingest if the internet goes down, even if you build multiple air-gapped data centers in underground bunkers with independent power. Continually evaluate the trade-offs and costs of reliability and durability.

Độ tin cậy và độ bền là sống còn trong các giai đoạn thu nạp của các đường ống dữ liệu. Độ tin cậy đòi hỏi thời gian hoạt động (uptime) cao và chuyển đổi dự phòng (failover) đúng cách cho các hệ thống thu nạp. Độ bền đòi hỏi đảm bảo rằng dữ liệu không bị mất hoặc hỏng. Một số nguồn dữ liệu (ví dụ, các thiết bị IoT và cache) có thể không lưu giữ dữ liệu nếu nó không được thu nạp đúng cách. Một khi đã mất, nó mất vĩnh viễn. Theo nghĩa này, độ tin cậy của các hệ thống thu nạp dẫn trực tiếp đến độ bền của dữ liệu được sinh ra. Nếu dữ liệu được thu nạp, các quy trình hạ nguồn về lý thuyết có thể chạy muộn nếu chúng tạm thời gặp trục trặc. Lời khuyên của chúng tôi là đánh giá các rủi ro và xây dựng một mức độ dư thừa (redundancy) và tự phục hồi (self-healing) phù hợp dựa trên tác động và chi phí của việc mất dữ liệu. Độ tin cậy và độ bền có cả chi phí trực tiếp lẫn gián tiếp. Ví dụ, liệu quy trình thu nạp của bạn có tiếp tục nếu một vùng (zone) AWS ngừng hoạt động không? Còn cả một region thì sao? Còn lưới điện hoặc internet thì sao? Tất nhiên, không có gì là miễn phí. Điều này sẽ tốn của bạn bao nhiêu? Bạn có thể xây được một hệ thống dư thừa cao và có một đội trực 24 giờ mỗi ngày để xử lý sự cố. Điều này cũng có nghĩa là chi phí đám mây và nhân công của bạn trở nên cấm cản (chi phí trực tiếp), và công việc liên tục đó gây tổn hại đáng kể cho đội ngũ của bạn (chi phí gián tiếp). Không có câu trả lời đúng, và bạn cần đánh giá chi phí và lợi ích của các quyết định về độ tin cậy và độ bền của mình. Đừng cho rằng bạn có thể xây một hệ thống sẽ thu nạp dữ liệu một cách tin cậy và bền vững trong mọi kịch bản có thể. Ngay cả ngân sách gần như vô hạn của chính phủ liên bang Hoa Kỳ cũng không thể đảm bảo điều này. Trong nhiều kịch bản cực đoan, việc thu nạp dữ liệu thực ra sẽ không quan trọng. Sẽ chẳng có gì để thu nạp nếu internet sập, ngay cả khi bạn xây nhiều trung tâm dữ liệu cách ly hoàn toàn (air-gapped) trong các hầm ngầm với nguồn điện độc lập. Hãy liên tục đánh giá các đánh đổi và chi phí của độ tin cậy và độ bền.

Payload — Tải trọng

A payload is the dataset you're ingesting and has characteristics such as kind, shape, size, **schema** and data types, and **metadata**. Let's look at some of these characteristics to understand why this matters.

Một tải trọng (payload) là tập dữ liệu bạn đang thu nạp và có các đặc tính như loại (kind), hình dạng (shape), kích thước (size), lược đồ (schema) và kiểu dữ liệu, cùng siêu dữ liệu (metadata). Hãy xem một số đặc tính này để hiểu tại sao điều này quan trọng.

Kind — Loại

The kind of data you handle directly impacts how it's dealt with downstream in the data engineering lifecycle. Kind consists of type and format. Data has a type—tabular, image, video, text, etc. The type directly influences the data format or the way it is expressed in bytes, names, and file extensions. For example, a tabular kind of data may be in formats such as CSV or Parquet, with each of these formats having different byte patterns for serialization and deserialization. Another kind of data is an image, which has a format of JPG or PNG and is inherently unstructured.

Loại dữ liệu bạn xử lý ảnh hưởng trực tiếp đến cách nó được xử lý ở hạ nguồn trong vòng đời kỹ thuật dữ liệu. Loại bao gồm kiểu (type) và định dạng (format). Dữ liệu có một kiểu—dạng bảng, hình ảnh, video, văn bản, v.v. Kiểu ảnh hưởng trực tiếp đến định dạng dữ liệu hay cách nó được biểu diễn dưới dạng byte, tên và phần mở rộng tệp. Ví dụ, một loại dữ liệu dạng bảng có thể ở các định dạng như CSV hoặc Parquet, mỗi định dạng này có các mẫu byte khác nhau cho tuần tự hóa và giải tuần tự hóa. Một loại dữ liệu khác là hình ảnh, có định dạng JPG hoặc PNG và về bản chất là phi cấu trúc.

Shape — Hình dạng

Every payload has a shape that describes its dimensions. Data shape is critical across the data engineering lifecycle. For instance, an image's pixel and red, green, blue (RGB) dimensions are necessary for training deep learning models. As another example, if you're trying to import a CSV file into a database table, and your CSV has more columns than the database table, you'll likely get an error during the import process. Here are some examples of the shapes of various kinds of data: Tabular

Mỗi tải trọng có một hình dạng mô tả các chiều của nó. Hình dạng dữ liệu là then chốt xuyên suốt vòng đời kỹ thuật dữ liệu. Chẳng hạn, các chiều pixel và đỏ, xanh lá, xanh dương (RGB) của một hình ảnh là cần thiết để huấn luyện các mô hình học sâu (deep learning). Một ví dụ khác, nếu bạn cố nhập một tệp CSV vào một bảng database, và CSV của bạn có nhiều cột hơn bảng database, bạn nhiều khả năng sẽ gặp lỗi trong quá trình nhập. Đây là một số ví dụ về hình dạng của các loại dữ liệu khác nhau: Dạng bảng

The number of rows and columns in the dataset, commonly expressed as M rows and N columns

Số hàng và cột trong tập dữ liệu, thường được biểu diễn là M hàng và N cột

Semistructured JSON

JSON bán cấu trúc

The key-value pairs and nesting depth occur with subelements

Các cặp khóa-giá trị (key-value) và độ sâu lồng nhau (nesting) xảy ra với các phần tử con

Unstructured text Number of words, characters, or bytes in the text body

Văn bản phi cấu trúc. Số từ, ký tự hoặc byte trong phần thân văn bản

Images

Hình ảnh

The width, height, and RGB color depth (e.g., 8 bits per pixel)

Chiều rộng, chiều cao, và độ sâu màu RGB (ví dụ, 8 bit mỗi pixel)

Uncompressed audio

Âm thanh không nén

Number of channels (e.g., two for stereo), sample depth (e.g., 16 bits per sample), sample rate (e.g., 48 kHz), and length (e.g., 10,003 seconds)

Số kênh (ví dụ, hai cho stereo), độ sâu mẫu (ví dụ, 16 bit mỗi mẫu), tốc độ lấy mẫu (ví dụ, 48 kHz), và độ dài (ví dụ, 10.003 giây)

Size — Kích thước

The size of the data describes the number of bytes of a payload. A payload may range in size from single bytes to terabytes and larger. To reduce the size of a payload, it may be compressed into various formats such as ZIP and TAR (see the discussion of **compression** in Appendix A). A massive payload can also be split into chunks, which effectively reduces the size of the payload into smaller subsections. When loading a huge file into a cloud **object storage** or data warehouse, this is a common practice as the small individual files are easier to transmit over a network (especially if they're compressed). The smaller chunked files are sent to their destination and then reassembled after all data has arrived.

Kích thước của dữ liệu mô tả số byte của một tải trọng. Một tải trọng có thể có kích thước từ vài byte đơn lẻ đến terabyte và lớn hơn. Để giảm kích thước của một tải trọng, nó có thể được nén (compression) thành các định dạng khác nhau như ZIP và TAR (xem thảo luận về nén trong Phụ lục A). Một tải trọng khổng lồ cũng có thể được chia thành các khối (chunk), điều này thực tế giảm kích thước của tải trọng thành các phần nhỏ hơn. Khi nạp một tệp khổng lồ vào một kho lưu trữ đối tượng (object storage) đám mây hoặc kho dữ liệu, đây là một thực hành phổ biến vì các tệp nhỏ riêng lẻ dễ truyền qua mạng hơn (đặc biệt nếu chúng được nén). Các tệp nhỏ đã chia khối được gửi đến đích của chúng và sau đó được ghép lại sau khi tất cả dữ liệu đã đến.

Schema and data types — Lược đồ và kiểu dữ liệu

Many data payloads have a schema, such as tabular and semistructured data. As mentioned earlier in this book, a schema describes the fields and types of data within those fields. Other data, such as unstructured text, images, and audio, will not have an explicit schema or data types. However, they might come with technical file descriptions on shape, data and file format, encoding, size, etc. Although you can connect to databases in various ways (such as file export, CDC, JDBC/ODBC), the connection is easy. The great engineering challenge is understanding the underlying schema. Applications organize data in various ways, and engineers need to be intimately familiar with the

organization of the data and relevant update patterns to make sense of it. The problem has been somewhat exacerbated by the popularity of object-relational mapping (ORM), which automatically generates schemas based on object structure in languages such as Java or Python. Natural structures in an object-oriented language often map to something messy in an operational database. Data engineers may need to familiarize themselves with the class structure of application code. Schema is not only for databases. As we've discussed, APIs present their schema complications. Many vendor APIs have friendly reporting methods that prepare data for analytics. In other cases, engineers are not so lucky. The API is a thin wrapper around underlying systems, requiring engineers to understand application internals to use the data. Much of the work associated with ingesting from source schemas happens in the data engineering lifecycle transformation stage, which we discuss in Chapter 8. We've placed this discussion here because data engineers need to begin studying source schemas as soon they plan to ingest data from a new source. Communication is critical for understanding source data, and engineers also have the opportunity to reverse the flow of communication and help software engineers improve data where it is produced. Later in this chapter, we'll return to this topic in "Whom You'll Work With".

Nhiều tải trọng dữ liệu có một lược đồ (schema), chẳng hạn như dữ liệu dạng bảng và bán cấu trúc. Như đã đề cập trước đó trong cuốn sách này, một lược đồ mô tả các trường và kiểu dữ liệu trong các trường đó. Dữ liệu khác, chẳng hạn như văn bản phi cấu trúc, hình ảnh và âm thanh, sẽ không có lược đồ hoặc kiểu dữ liệu rõ ràng. Tuy nhiên, chúng có thể đi kèm các mô tả tệp kỹ thuật về hình dạng, định dạng dữ liệu và tệp, mã hóa, kích thước, v.v. Mặc dù bạn có thể kết nối tới các database theo nhiều cách (chẳng hạn như xuất tệp, CDC, JDBC/ODBC), kết nối thì dễ. Thách thức kỹ thuật lớn là hiểu lược đồ nền tảng. Các ứng dụng tổ chức dữ liệu theo nhiều cách khác nhau, và các kỹ sư cần làm quen sâu sắc với cách tổ chức của dữ liệu và các mẫu cập nhật liên quan để hiểu được nó. Vấn đề đã phần nào trầm trọng hơn bởi sự phổ biến của ánh xạ đối tượng-quan hệ (object-relational mapping, ORM), thử tự động sinh các lược đồ dựa trên cấu trúc đối tượng trong các ngôn ngữ như Java hoặc Python. Các cấu trúc tự nhiên trong một ngôn ngữ hướng đối tượng thường ánh xạ thành thứ gì đó lộn xộn trong một database vận hành. Các kỹ sư dữ liệu có thể cần làm quen với cấu trúc lớp (class) của mã ứng dụng. Lược đồ không chỉ dành cho database. Như chúng ta đã bàn, các API có những phức tạp về lược đồ riêng. Nhiều API của nhà cung cấp có các phương thức báo cáo thân thiện chuẩn bị dữ liệu cho phân tích. Trong các trường hợp khác, các kỹ sư không may mắn như vậy. API là một lớp bọc mỏng quanh các hệ thống nền tảng, đòi hỏi các kỹ sư hiểu nội bộ ứng dụng để dùng dữ liệu. Phần lớn công việc liên quan đến thu nạp từ các lược đồ nguồn xảy ra trong giai đoạn biến đổi của vòng đời kỹ thuật dữ liệu, mà chúng ta bàn trong Chương 8. Chúng tôi đặt thảo luận này ở đây vì các kỹ sư dữ liệu cần bắt đầu nghiên cứu các lược đồ nguồn ngay khi họ dự định thu nạp dữ liệu từ một nguồn mới. Giao tiếp là then chốt để hiểu dữ liệu nguồn, và các kỹ sư cũng có cơ hội đảo ngược dòng giao tiếp và giúp các kỹ sư phần mềm cải thiện dữ liệu ngay tại nơi nó được sinh ra. Sau này trong chương, chúng ta sẽ trở lại chủ đề này trong "Những người bạn sẽ làm việc cùng".

Detecting and handling schema changes in upstream and downstream systems Schema changes frequently occur in source systems and are often well out of data engineers' control. Examples of schema changes include the following:

Phát hiện và xử lý các thay đổi lược đồ trong các hệ thống thượng nguồn và hạ nguồn. Các thay đổi lược đồ thường xuyên xảy ra trong các hệ thống nguồn và thường nằm ngoài tầm kiểm soát của các kỹ sư dữ liệu. Các ví dụ về thay đổi lược đồ bao gồm:

Adding a new column

Thêm một cột mới

Changing a column type Creating a new table

Thay đổi kiểu cột. Tạo một bảng mới

Renaming a column

Đổi tên một cột

It's becoming increasingly common for ingestion tools to automate the detection of schema changes and even auto-update target tables. Ultimately, this is something of a mixed blessing. Schema changes can still break pipelines downstream of staging and ingestion. Engineers must still implement strategies to respond to changes automatically and alert on changes that cannot be accommodated automatically. Automation is excellent, but the analysts and data scientists who rely on this data should be informed of the schema changes that violate existing assumptions. Even if automation can accommodate a change, the new schema may adversely affect the performance of reports and models. Communication between those making schema changes and those impacted by these changes is as important as reliable automation that checks for schema changes.

Ngày càng phổ biến việc các công cụ thu nạp tự động hóa việc phát hiện các thay đổi lược đồ và thậm chí tự động cập nhật các bảng đích. Suy cho cùng, đây có phần là một phước lành lẫn lộn. Các thay đổi lược đồ vẫn có thể phá vỡ các đường ống ở hạ nguồn của giai đoạn staging và thu nạp. Các kỹ sư vẫn phải triển khai các chiến lược để phản hồi tự động với các thay đổi và cảnh báo về những thay đổi không thể thích ứng tự động. Tự động hóa là tuyệt vời, nhưng các nhà phân tích và nhà khoa học dữ liệu dựa vào dữ liệu này nên được thông báo về các thay đổi lược đồ vi phạm các giả định hiện có. Ngay cả khi tự động hóa có thể thích ứng với một thay đổi, lược đồ mới có thể ảnh hưởng bất lợi đến hiệu năng của các báo cáo và mô hình. Giao tiếp giữa những người thực hiện các thay đổi lược đồ và những người bị ảnh hưởng bởi các thay đổi này quan trọng không kém việc tự động hóa đáng tin cậy kiểm tra các thay đổi lược đồ.

Schema registries In streaming data, every message has a schema, and these schemas may evolve between producers and consumers. A schema registry is a metadata repository used to maintain schema and data type integrity in the face of constantly changing schemas. Schema registries can also track schema versions and history. It describes the data model for messages, allowing consistent serialization and deserialization between producers and consumers. Schema registries are used in most major data tools and clouds.

Schema registry. Trong dữ liệu streaming, mỗi tin nhắn có một lược đồ, và các lược đồ này có thể tiến hóa giữa các bên sản xuất (producer) và bên tiêu thụ (consumer). Một schema registry là một kho siêu dữ liệu được dùng để duy trì tính toàn vẹn của lược đồ và kiểu dữ liệu trước các lược đồ liên tục thay đổi. Schema registry cũng có thể theo dõi các phiên bản và lịch sử lược đồ. Nó mô tả mô hình dữ liệu cho các tin nhắn, cho phép tuần tự hóa và giải tuần tự hóa nhất quán giữa các bên sản xuất và bên tiêu thụ. Schema registry được dùng trong hầu hết các công cụ dữ liệu và đám mây lớn.

Metadata — Siêu dữ liệu

In addition to the apparent characteristics we've just covered, a payload often contains metadata, which we first discussed in Chapter 2 . Metadata is data about data. Metadata can be as critical as the data itself. One of the significant limitations of the early approach to the **data lake**—or data swamp, which could become a data superfund site—was a complete lack of attention to metadata. Without a detailed description of the data, it may be of little value. We've already discussed some types of metadata (e.g., schema) and will address them many times throughout this chapter.

Ngoài các đặc tính rõ ràng mà chúng ta vừa đề cập, một tải trọng thường chứa siêu dữ liệu (metadata), thứ chúng ta lần đầu bàn trong Chương 2. Siêu dữ liệu là dữ liệu về dữ liệu. Siêu dữ liệu có thể quan trọng như chính dữ liệu. Một trong những hạn chế đáng kể của cách tiếp cận sơ khai với hồ dữ liệu (data lake)—hay đầm lầy dữ liệu (data swamp), thứ có thể trở thành một bãi rác độc hại dữ liệu (data superfund site)—là sự thiếu quan tâm hoàn toàn đến siêu dữ liệu. Nếu không có mô tả chi tiết về dữ liệu, nó có thể có ít giá trị. Chúng ta đã bàn một số loại siêu dữ liệu (ví dụ, lược đồ) và sẽ đề cập đến chúng nhiều lần xuyên suốt chương này.

Push Versus Pull Versus Poll Patterns — Các mẫu hình Đẩy so với Kéo so với Thăm dò

We mentioned push versus pull when we introduced the data engineering lifecycle in Chapter 2 . A push strategy (Figure 7-7) involves a source system sending data to a target, while a pull strategy (Figure 7-8) entails a target reading data directly from a source. As we mentioned in that discussion, the lines between these strategies are blurry.

Chúng tôi đã đề cập đẩy so với kéo (push versus pull) khi giới thiệu vòng đời kỹ thuật dữ liệu trong Chương 2. Một chiến lược đẩy (push) (Hình 7-7) liên quan đến việc một hệ thống nguồn gửi dữ liệu đến một đích, trong khi một chiến lược kéo (pull) (Hình 7-8) đòi hỏi một đích đọc dữ liệu trực tiếp từ một nguồn. Như chúng tôi đã đề cập trong thảo luận đó, ranh giới giữa các chiến lược này khá mờ nhạt.

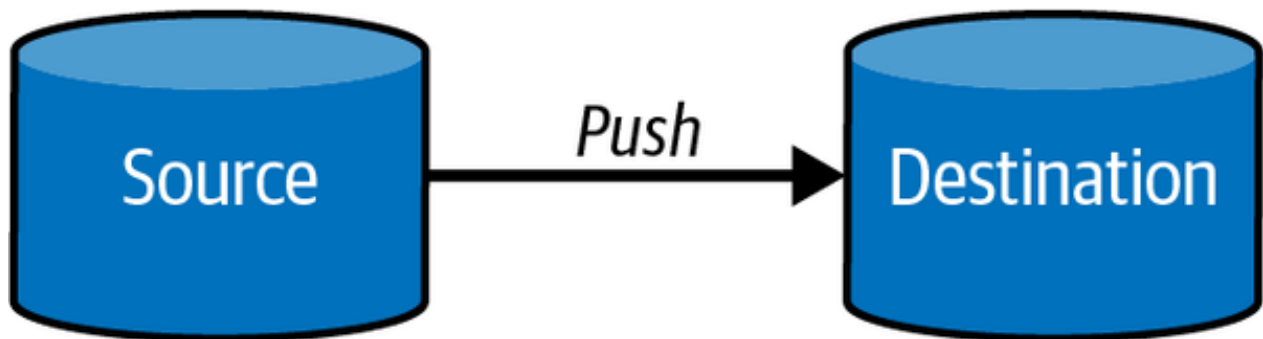


Figure 7-7. Pushing data from source to destination

Hình 7-7. Đẩy dữ liệu từ nguồn đến đích

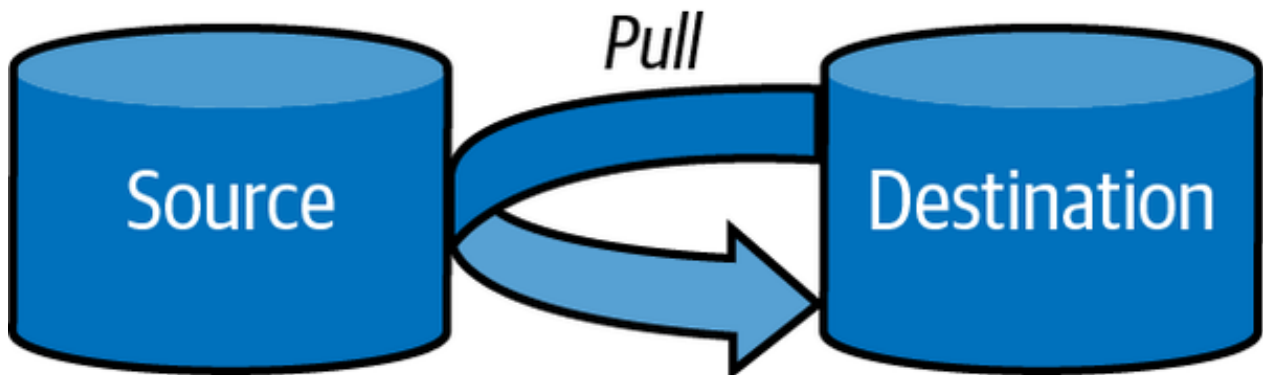


Figure 7-8. A destination pulling data from a source Another pattern related to pulling is polling for data (Figure 7-9). Polling involves periodically checking a data source for any changes. When

changes are detected, the destination pulls the data as it would in a regular pull situation.

Hình 7-8. Một đích kéo dữ liệu từ một nguồn. Một mẫu hình khác liên quan đến kéo là thăm dò (polling) dữ liệu (Hình 7-9). Thăm dò liên quan đến việc định kỳ kiểm tra một nguồn dữ liệu xem có bất kỳ thay đổi nào không. Khi phát hiện các thay đổi, đích kéo dữ liệu như nó sẽ làm trong một tình huống kéo thông thường.

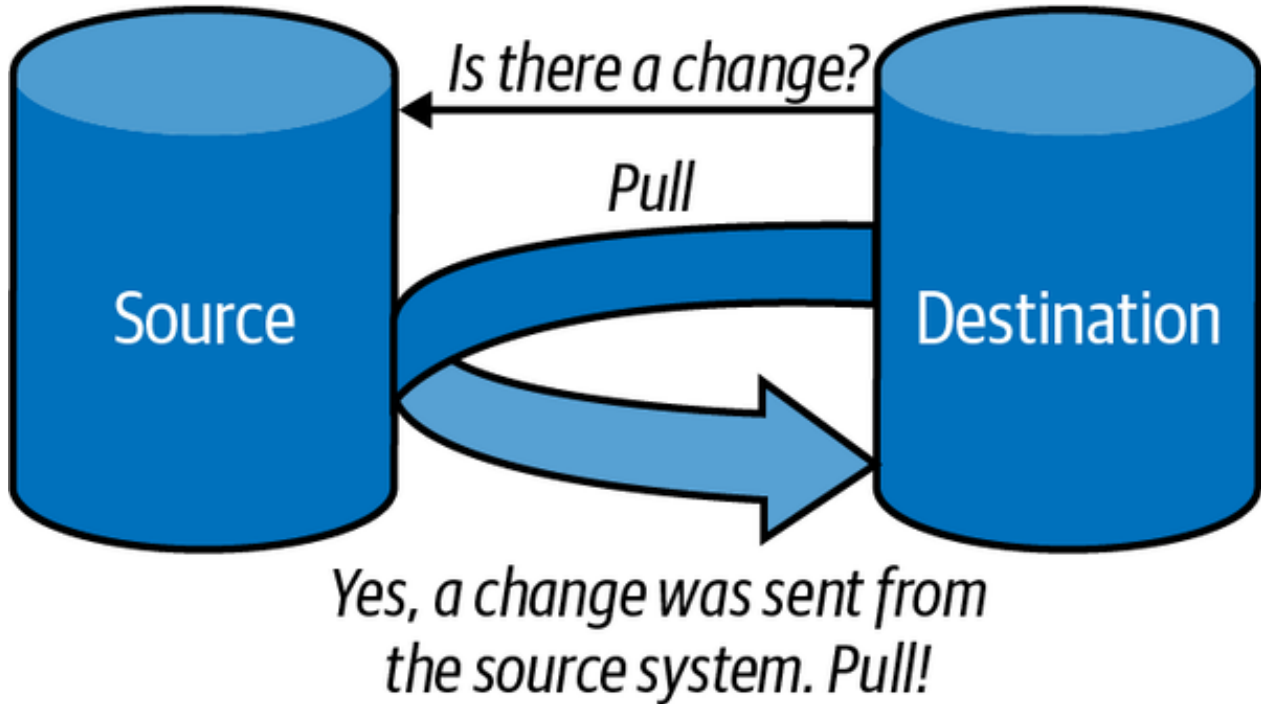


Figure 7-9. Polling for changes in a source system

Hình 7-9. Thăm dò các thay đổi trong một hệ thống nguồn

Batch Ingestion Considerations — Các cân nhắc khi Thu nạp theo Lô

Batch ingestion, which involves processing data in bulk, is often a convenient way to ingest data. This means that data is ingested by taking a subset of data from a source system, based either on a time interval or the size of accumulated data (Figure 7-10).

Thu nạp theo lô, liên quan đến việc xử lý dữ liệu hàng loạt, thường là một cách tiện lợi để thu nạp dữ liệu. Điều này nghĩa là dữ liệu được thu nạp bằng cách lấy một tập con dữ liệu từ một hệ thống nguồn, dựa trên hoặc một khoảng thời gian hoặc kích thước dữ liệu tích lũy (Hình 7-10).

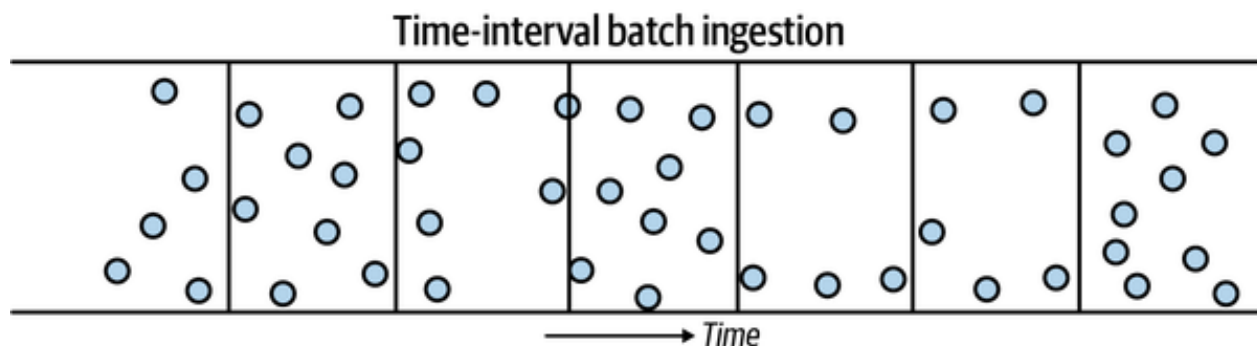


Figure 7-10. Time-interval batch ingestion Time-interval batch ingestion is widespread in traditional business ETL for data warehousing. This pattern is often used to process data once a day, overnight during off-hours, to provide daily reporting, but other frequencies can also be used. Size-based batch ingestion (Figure 7-11) is quite common when data is moved from a streaming-based system into object storage; ultimately, you must cut the data into discrete blocks for future processing in a data lake. Some size-based ingestion systems can break data into objects based on various criteria, such as the size in bytes of the total number of events.

Hình 7-10. Thu nạp theo lô theo khoảng thời gian. Thu nạp theo lô theo khoảng thời gian phổ biến rộng rãi trong ETL kinh doanh truyền thống cho kho dữ liệu. Mẫu hình này thường được dùng để xử lý dữ liệu mỗi ngày một lần, vào ban đêm trong những giờ ngoài giờ làm việc, để cung cấp báo cáo hằng ngày, nhưng các tần suất khác cũng có thể được dùng. Thu nạp theo lô theo kích thước (Hình 7-11) khá phổ biến khi dữ liệu được di chuyển từ một hệ thống dựa trên streaming vào kho lưu trữ đối tượng; cuối cùng, bạn phải cắt dữ liệu thành các khối rời rạc để xử lý trong tương lai trong một hồ dữ liệu. Một số hệ thống thu nạp dựa trên kích thước có thể chia dữ liệu thành các đối tượng dựa trên nhiều tiêu chí khác nhau, chẳng hạn như kích thước tính bằng byte của tổng số sự kiện.

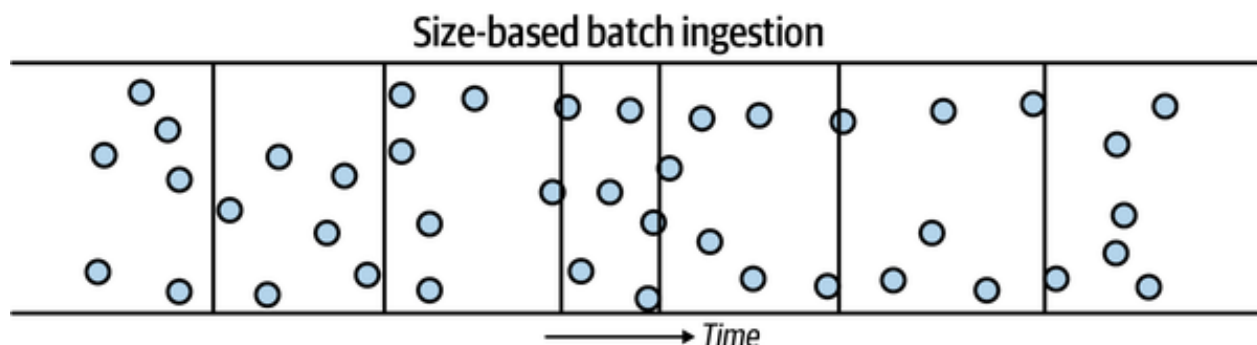


Figure 7-11. Size-based batch ingestion Some commonly used batch ingestion patterns, which we discuss in this section, include the following:

Hình 7-11. Thu nạp theo lô theo kích thước. Một số mẫu hình thu nạp theo lô thường được dùng, mà chúng ta bàn trong phần này, bao gồm:

Snapshot or differential extraction

Trích xuất ảnh chụp (snapshot) hoặc trích xuất chênh lệch (differential)

File-based export and ingestion

Xuất và thu nạp dựa trên tệp

ETL versus ELT

ETL so với ELT

Inserts, updates, and batch size

Chèn (insert), cập nhật (update) và kích thước lô

Data migration

Di trú dữ liệu (data migration)

Snapshot or Differential Extraction — Trích xuất Ảnh chụp hoặc Chênh lệch

Data engineers must choose whether to capture full snapshots of a source system or differential (sometimes called incremental) updates. With full snapshots , engineers grab the entire current state of the source system on each update read. With the differential update pattern, engineers can pull only the updates and changes since the last read from the source system.

Các kỹ sư dữ liệu phải chọn giữa việc thu các ảnh chụp (snapshot) đầy đủ của một hệ thống nguồn hay các bản cập nhật chênh lệch (differential, đôi khi gọi là tăng dần - incremental). Với ảnh chụp đầy đủ (full snapshot), các kỹ sư lấy toàn bộ trạng thái hiện tại của hệ thống nguồn ở mỗi lần đọc cập nhật. Với mẫu hình cập nhật chênh lệch, các kỹ sư có thể chỉ kéo các bản cập nhật và thay đổi kể từ lần đọc cuối cùng từ hệ thống nguồn.

While differential updates are ideal for minimizing network traffic and target storage usage, full snapshot reads remain extremely common because of their simplicity.

Mặc dù các bản cập nhật chênh lệch là lý tưởng để giảm thiểu lưu lượng mạng và mức sử dụng kho lưu trữ đích, các lần đọc ảnh chụp đầy đủ vẫn cực kỳ phổ biến vì tính đơn giản của chúng.

File-Based Export and Ingestion — Xuất và Thu nạp dựa trên Tập

Data is quite often moved between databases and systems using files. Data is serialized into files in an exchangeable format, and these files are provided to an ingestion system. We consider file-based export to be a push-based ingestion pattern. This is because data export and preparation work is done on the source system side. File-based ingestion has several potential advantages over a direct database connection approach. It is often undesirable to allow direct access to backend systems for security reasons. With file-based ingestion, export processes are run on the data-source side, giving source system engineers complete control over what data gets exported and how the data is preprocessed. Once files are done, they can be provided to the target system in various ways. Common file-exchange methods are object storage, secure file transfer protocol (SFTP), electronic data interchange (EDI), or secure copy (SCP).

Dữ liệu khá thường được di chuyển giữa các database và hệ thống bằng các tệp. Dữ liệu được tuần tự hóa thành các tệp ở một định dạng có thể trao đổi, và các tệp này được cung cấp cho một hệ thống thu nạp. Chúng tôi xem xuất dựa trên tệp là một mẫu hình thu nạp dựa trên đẩy. Điều này là vì công việc xuất và chuẩn bị dữ liệu được thực hiện ở phía hệ thống nguồn. Thu nạp dựa trên tệp có một số lợi thế tiềm năng so với cách tiếp cận kết nối database trực tiếp. Thường thì không mong muốn cho phép truy cập trực tiếp tới các hệ thống backend vì lý do bảo mật. Với thu nạp dựa trên tệp, các quy trình xuất được chạy ở phía nguồn dữ liệu, trao cho các kỹ sư hệ thống nguồn toàn quyền kiểm soát dữ liệu nào

được xuất và cách dữ liệu được tiền xử lý. Một khi các tệp hoàn thành, chúng có thể được cung cấp cho hệ thống đích theo nhiều cách khác nhau. Các phương thức trao đổi tệp phổ biến là kho lưu trữ đối tượng, giao thức truyền tệp an toàn (secure file transfer protocol, SFTP), trao đổi dữ liệu điện tử (electronic data interchange, EDI), hoặc sao chép an toàn (secure copy, SCP).

ETL Versus ELT — ETL so với ELT

Chapter 3 introduced ETL and ELT, both extremely common ingestion, storage, and transformation patterns you'll encounter in batch workloads. This section covers the extract (E) and the load (L) parts of ETL and ELT.

Chương 3 đã giới thiệu ETL và ELT, cả hai đều là các mẫu hình thu nạp, lưu trữ và biến đổi cực kỳ phổ biến mà bạn sẽ gặp trong các khối lượng công việc theo lô. Phần này đề cập đến phần trích xuất (E - extract) và phần nạp (L - load) của ETL và ELT.

Extract — Trích xuất

Extract means getting data from a source system. While extract seems to imply pulling data, it can also be push based. Extraction may also require reading metadata and schema changes.

Trích xuất (extract) nghĩa là lấy dữ liệu từ một hệ thống nguồn. Mặc dù trích xuất dường như ngụ ý kéo dữ liệu, nó cũng có thể dựa trên đẩy. Trích xuất cũng có thể đòi hỏi đọc các thay đổi siêu dữ liệu và lược đồ.

Load — Nạp

Once data is extracted, it can either be transformed (ETL) before loading it into a storage destination or simply loaded into storage for future transformation. When loading data, you should be mindful of the type of system you're loading, the schema of the data, and the performance impact of loading. We cover ETL and ELT in Chapter 8 .

Một khi dữ liệu được trích xuất, nó có thể hoặc được biến đổi (ETL) trước khi nạp vào một đích lưu trữ hoặc đơn giản được nạp vào kho lưu trữ để biến đổi trong tương lai. Khi nạp dữ liệu, bạn nên lưu ý đến loại hệ thống mà bạn đang nạp vào, lược đồ của dữ liệu, và tác động hiệu năng của việc nạp. Chúng tôi đề cập ETL và ELT trong Chương 8.

Inserts, Updates, and Batch Size — Chèn, Cập nhật và Kích thước Lô

Batch-oriented systems often perform poorly when users attempt to perform many small-batch operations rather than a smaller number of large operations. For example, while it is common to insert one row at a time in a transactional database, this is a bad pattern for many columnar databases as it forces the creation of many small, suboptimal files, and forces the system to run a high number of create object operations. Running many small in-place update operations is an even bigger problem because it causes the database to scan each existing column file to run the update. Understand the appropriate update patterns for the database or data store you're working with. Also, understand that certain technologies are purpose-built for high insert rates. For example,

Apache Druid and Apache Pinot can handle high insert rates. SingleStore can manage hybrid workloads that combine **OLAP** and **OLTP** characteristics. BigQuery performs poorly on a high rate of vanilla SQL single-row inserts but extremely well if data is fed in through its stream buffer. Know the limits and characteristics of your tools.

Các hệ thống định hướng theo lô thường hoạt động kém khi người dùng cố thực hiện nhiều thao tác lô nhỏ thay vì một số lượng nhỏ hơn các thao tác lớn. Ví dụ, mặc dù việc chèn một hàng tại một thời điểm là phổ biến trong một database giao dịch, đây là một mẫu hình tồi cho nhiều database cột (columnar) vì nó buộc phải tạo ra nhiều tệp nhỏ, không tối ưu, và buộc hệ thống chạy một số lượng lớn các thao tác tạo đối tượng. Chạy nhiều thao tác cập nhật tại chỗ (in-place) nhỏ thậm chí là một vấn đề lớn hơn vì nó khiến database quét từng tệp cột hiện có để chạy bản cập nhật. Hãy hiểu các mẫu cập nhật phù hợp cho database hoặc kho dữ liệu bạn đang làm việc. Đồng thời, hãy hiểu rằng một số công nghệ được xây dựng có chủ đích cho tốc độ chèn cao. Ví dụ, Apache Druid và Apache Pinot có thể xử lý tốc độ chèn cao. SingleStore có thể quản lý các khối lượng công việc lai (hybrid) kết hợp các đặc tính OLAP và OLTP. BigQuery hoạt động kém với tốc độ cao các lần chèn một hàng SQL thuần túy nhưng cực kỳ tốt nếu dữ liệu được đưa vào qua bộ đệm stream của nó. Hãy biết các giới hạn và đặc tính của các công cụ của bạn.

Data Migration — Di trú Dữ liệu

Migrating data to a new database or environment is not usually trivial, and data needs to be moved in bulk. Sometimes this means moving data sizes that are hundreds of terabytes or much larger, often involving the migration of specific tables and moving entire databases and systems. Data migrations probably aren't a regular occurrence as a data engineer, but you should be familiar with them. As is often the case for data ingestion, schema management is a crucial consideration. Suppose you're migrating data from one database system to a different one (say, SQL Server to Snowflake). No matter how closely the two databases resemble each other, subtle differences almost always exist in the way they handle schema. Fortunately, it is generally easy to test ingestion of a sample of data and find schema issues before undertaking a complete table migration. Most data systems perform best when data is moved in bulk rather than as individual rows or events. File or object storage is often an excellent intermediate stage for transferring data. Also, one of the biggest challenges of database migration is not the movement of the data itself but the movement of data pipeline connections from the old system to the new one. Be aware that many tools are available to automate various types of data migrations. Especially for large and complex migrations, we suggest looking at these options before doing this manually or writing your own migration solution.

Di trú (migrating) dữ liệu sang một database hoặc môi trường mới thường không phải chuyện vặt, và dữ liệu cần được di chuyển hàng loạt. Đôi khi điều này nghĩa là di chuyển các kích thước dữ liệu hàng trăm terabyte hoặc lớn hơn nhiều, thường liên quan đến việc di trú các bảng cụ thể và di chuyển toàn bộ database và hệ thống. Di trú dữ liệu có lẽ không phải là chuyện thường xuyên xảy ra với một kỹ sư dữ liệu, nhưng bạn nên quen thuộc với chúng. Như thường thấy với thu nạp dữ liệu, quản lý lược đồ là một cân nhắc thiết yếu. Giả sử bạn đang di trú dữ liệu từ một hệ thống database này sang một hệ thống khác (chẳng hạn, SQL Server sang Snowflake). Bất kể hai database giống nhau đến đâu, các khác biệt tinh tế hầu như luôn tồn tại trong cách chúng xử lý lược đồ. May mắn thay, nhìn chung dễ kiểm thử việc thu nạp một mẫu dữ liệu và tìm các vấn đề lược đồ trước khi tiến hành di trú toàn bộ bảng. Hầu hết các hệ thống dữ liệu hoạt động tốt nhất khi dữ liệu được di chuyển hàng loạt thay vì từng hàng hoặc từng sự kiện riêng lẻ. Lưu trữ tệp hoặc đối tượng thường là một giai đoạn trung gian tuyệt vời để chuyển dữ liệu. Ngoài ra, một trong những

thách thức lớn nhất của di trú database không phải là việc di chuyển bản thân dữ liệu mà là việc di chuyển các kết nối đường ống dữ liệu từ hệ thống cũ sang hệ thống mới. Hãy lưu ý rằng có nhiều công cụ sẵn có để tự động hóa nhiều loại di trú dữ liệu khác nhau. Đặc biệt với các cuộc di trú lớn và phức tạp, chúng tôi đề xuất xem xét các tùy chọn này trước khi làm thủ công hoặc viết giải pháp di trú của riêng bạn.

Message and Stream Ingestion Considerations — Các cân nhắc khi Thu nạp Tin nhắn và Stream

Ingesting event data is common. This section covers issues you should consider when ingesting events, drawing on topics covered in Chapters 5 and 6 .

Thu nạp dữ liệu sự kiện là phổ biến. Phần này đề cập các vấn đề bạn nên cân nhắc khi thu nạp các sự kiện, dựa trên các chủ đề được đề cập trong Chương 5 và 6.

Schema Evolution — Tiến hóa Lược đồ

Schema evolution is common when handling event data; fields may be added or removed, or value types might change (say, a string to an integer). Schema evolution can have unintended impacts on your data pipelines and destinations. For example, an IoT device gets a firmware update that adds a new field to the event it transmits, or a third-party API introduces changes to its event payload or countless other scenarios. All of these potentially impact your downstream capabilities. To alleviate issues related to schema evolution, here are a few suggestions. First, if your event-processing framework has a schema registry (discussed earlier in this chapter), use it to version your schema changes. Next, a dead-letter queue (described in “Error Handling and Dead-Letter Queues”) can help you investigate issues with events that are not properly handled. Finally, the low-fidelity route (and the most effective) is regularly communicating with upstream stakeholders about potential schema changes and proactively addressing schema changes with the teams introducing these changes instead of reacting to the receiving end of breaking changes.

Tiến hóa lược đồ (schema evolution) là phổ biến khi xử lý dữ liệu sự kiện; các trường có thể được thêm hoặc xóa, hoặc các kiểu giá trị có thể thay đổi (chẳng hạn, từ một chuỗi sang một số nguyên). Tiến hóa lược đồ có thể có các tác động ngoài ý muốn lên các đường ống dữ liệu và đích của bạn. Ví dụ, một thiết bị IoT nhận một bản cập nhật firmware thêm một trường mới vào sự kiện nó truyền đi, hoặc một API bên thứ ba đưa ra các thay đổi đối với tải trọng sự kiện của nó, hay vô số kịch bản khác. Tất cả những điều này đều có khả năng ảnh hưởng đến các năng lực hạ nguồn của bạn. Để giảm bớt các vấn đề liên quan đến tiến hóa lược đồ, đây là một vài gợi ý. Thứ nhất, nếu framework xử lý sự kiện của bạn có một schema registry (đã bàn trước đó trong chương này), hãy dùng nó để gắn phiên bản cho các thay đổi lược đồ của bạn. Tiếp theo, một hàng đợi thư chết (dead-letter queue, được mô tả trong “Xử lý Lỗi và Hàng đợi Thư chết”) có thể giúp bạn điều tra các vấn đề với các sự kiện không được xử lý đúng cách. Cuối cùng, con đường ít kỹ thuật nhất (và hiệu quả nhất) là thường xuyên giao tiếp với các bên liên quan thượng nguồn về các thay đổi lược đồ tiềm năng và chủ động giải quyết các thay đổi lược đồ với các đội đưa ra những thay đổi này thay vì phản ứng ở đầu nhận của các thay đổi gây vỡ.

Late-Arriving Data — Dữ liệu Đến muộn

Though you probably prefer all event data to arrive on time, event data might arrive late. A group of events might occur around the same time frame (similar event times), but some might arrive later than others (late ingestion times) because of various circumstances. For example, an IoT device might be late sending a message because of internet latency issues. This is common when ingesting data. You should be aware of late-arriving data and the impact on downstream systems and uses. Suppose you assume that ingestion or process time is the same as the **event time**. You may get some strange results if your reports or analysis depend on an accurate portrayal of when events occur. To handle late-arriving data, you need to set a cutoff time for when late-arriving data will no longer be processed.

Mặc dù bạn có lẽ thích tất cả dữ liệu sự kiện đến đúng giờ, dữ liệu sự kiện có thể đến muộn. Một nhóm các sự kiện có thể xảy ra trong khoảng cùng khung thời gian (các thời điểm sự kiện - event time tương tự nhau), nhưng một số có thể đến muộn hơn những cái khác (các thời điểm thu nạp muộn) vì nhiều hoàn cảnh khác nhau. Ví dụ, một thiết bị IoT có thể gửi một tin nhắn muộn vì các vấn đề độ trễ internet. Điều này phổ biến khi thu nạp dữ liệu. Bạn nên ý thức về dữ liệu đến muộn và tác động lên các hệ thống và mục đích sử dụng hạ nguồn. Giả sử bạn cho rằng thời điểm thu nạp hoặc xử lý giống với thời điểm sự kiện. Bạn có thể nhận được một số kết quả kỳ lạ nếu các báo cáo hoặc phân tích của bạn phụ thuộc vào việc khắc họa chính xác thời điểm các sự kiện xảy ra. Để xử lý dữ liệu đến muộn, bạn cần đặt một thời điểm cắt (cutoff time) cho khi nào dữ liệu đến muộn sẽ không còn được xử lý nữa.

Ordering and Multiple Delivery — Thứ tự và Giao nhiều lần

Streaming platforms are generally built out of distributed systems, which can cause some complications. Specifically, messages may be delivered out of order and more than once (at-least-once delivery). See the event-streaming platforms discussion in Chapter 5 for more details.

Các nền tảng streaming nhìn chung được xây dựng từ các hệ phân tán, điều có thể gây ra một số phức tạp. Cụ thể, các tin nhắn có thể được giao không đúng thứ tự và nhiều hơn một lần (giao ít-nhất-một-lần - at-least-once delivery). Xem thảo luận về các nền tảng event-streaming trong Chương 5 để biết thêm chi tiết.

Replay — Phát lại

Replay allows readers to request a range of messages from the history, allowing you to rewind your event history to a particular point in time. Replay is a key capability in many streaming ingestion platforms and is particularly useful when you need to re-ingest and reprocess data for a specific time range. For example, RabbitMQ typically deletes messages after all subscribers consume them. Kafka, Kinesis, and Pub/Sub all support event retention and replay.

Phát lại (replay) cho phép các bên đọc yêu cầu một khoảng các tin nhắn từ lịch sử, cho phép bạn tua lại lịch sử sự kiện của mình đến một thời điểm cụ thể. Phát lại là một năng lực then chốt trong nhiều nền tảng thu nạp streaming và đặc biệt hữu ích khi bạn cần thu nạp lại và xử lý lại dữ liệu cho một khoảng thời gian cụ thể. Ví dụ, RabbitMQ thường xóa các tin nhắn sau khi tất cả các bên đăng ký (subscriber) tiêu thụ chúng. Kafka, Kinesis và Pub/Sub đều hỗ trợ lưu giữ (retention) và phát lại sự kiện.

Time to Live — Thời gian tồn tại

How long will you preserve your event record? A key parameter is maximum message retention time, also known as the time to live (TTL). TTL is usually a configuration you'll set for how long you want events to live before they are acknowledged and ingested. Any unacknowledged event that's not ingested after its TTL expires automatically disappears. This is helpful to reduce backpressure and unnecessary event volume in your event-ingestion pipeline. Find the right balance of TTL impact on our data pipeline. An extremely short TTL (milliseconds or seconds) might cause most messages to disappear before processing. A very long TTL (several weeks or months) will create a backlog of many unprocessed messages, resulting in long wait times. Let's look at how some popular platforms handle TTL at the time of this writing. Google Cloud Pub/Sub supports retention periods of up to 7 days. Amazon Kinesis Data Streams retention can be turned up to 365 days. Kafka can be configured for indefinite retention, limited by available disk space. (Kafka also supports the option to write older messages to cloud object storage, unlocking virtually unlimited storage space and retention.)

Bạn sẽ bảo toàn bản ghi sự kiện của mình trong bao lâu? Một tham số then chốt là thời gian lưu giữ tin nhắn tối đa, còn được gọi là thời gian tồn tại (time to live, TTL). TTL thường là một cấu hình bạn sẽ đặt cho việc bạn muốn các sự kiện tồn tại trong bao lâu trước khi chúng được xác nhận (acknowledge) và thu nạp. Bất kỳ sự kiện chưa được xác nhận nào không được thu nạp sau khi TTL của nó hết hạn sẽ tự động biến mất. Điều này hữu ích để giảm áp lực ngược (backpressure) và khối lượng sự kiện không cần thiết trong đường ống thu nạp sự kiện của bạn. Hãy tìm sự cân bằng đúng đắn về tác động của TTL lên đường ống dữ liệu của chúng ta. Một TTL cực ngắn (mili giây hoặc giây) có thể khiến hầu hết các tin nhắn biến mất trước khi được xử lý. Một TTL rất dài (vài tuần hoặc vài tháng) sẽ tạo ra một tồn đọng nhiều tin nhắn chưa được xử lý, dẫn đến thời gian chờ dài. Hãy xem một số nền tảng phổ biến xử lý TTL ra sao tại thời điểm viết bài này. Google Cloud Pub/Sub hỗ trợ các khoảng thời gian lưu giữ lên đến 7 ngày. Lưu giữ của Amazon Kinesis Data Streams có thể được tăng lên đến 365 ngày. Kafka có thể được cấu hình để lưu giữ vô thời hạn, bị giới hạn bởi dung lượng đĩa sẵn có. (Kafka cũng hỗ trợ tùy chọn ghi các tin nhắn cũ hơn vào kho lưu trữ đối tượng đám mây, mở khóa không gian lưu trữ và lưu giữ gần như vô hạn.)

Message Size — Kích thước Tin nhắn

Message size is an easily overlooked issue: you must ensure that the streaming framework in question can handle the maximum expected message size. Amazon Kinesis supports a maximum message size of 1 MB. Kafka defaults to this maximum size but can be configured for a maximum of 20 MB or more. (Configurability may vary on **managed service** platforms.)

Kích thước tin nhắn là một vấn đề dễ bị bỏ qua: bạn phải đảm bảo rằng framework streaming đang xét có thể xử lý kích thước tin nhắn tối đa kỳ vọng. Amazon Kinesis hỗ trợ kích thước tin nhắn tối đa 1 MB. Kafka mặc định ở kích thước tối đa này nhưng có thể được cấu hình tối đa 20 MB hoặc hơn. (Khả năng cấu hình có thể khác nhau trên các nền tảng dịch vụ được quản lý.)

Error Handling and Dead-Letter Queues — Xử lý Lỗi và Hàng đợi Thư chết

Sometimes events aren't successfully ingested. Perhaps an event is sent to a nonexistent topic or message queue, the message size may be too large, or the event has expired past its TTL. Events

that cannot be ingested need to be rerouted and stored in a separate location called a dead-letter queue . A dead-letter queue segregates problematic events from events that can be accepted by the consumer (Figure 7-12). If events are not rerouted to a dead-letter queue, these erroneous events risk blocking other messages from being ingested. Data engineers can use a dead-letter queue to diagnose why event ingestions errors occur and solve data pipeline problems, and might be able to reprocess some messages in the queue after fixing the underlying cause of errors.

Đôi khi các sự kiện không được thu nạp thành công. Có lẽ một sự kiện được gửi đến một topic hoặc message queue không tồn tại, kích thước tin nhắn có thể quá lớn, hoặc sự kiện đã hết hạn quá TTL của nó. Các sự kiện không thể được thu nạp cần được định tuyến lại và lưu trong một vị trí riêng gọi là hàng đợi thư chết (dead-letter queue). Một hàng đợi thư chết tách biệt các sự kiện có vấn đề khỏi các sự kiện có thể được bên tiêu thụ chấp nhận (Hình 7-12). Nếu các sự kiện không được định tuyến lại đến một hàng đợi thư chết, các sự kiện lỗi này có nguy cơ chặn các tin nhắn khác khỏi việc được thu nạp. Các kỹ sư dữ liệu có thể dùng một hàng đợi thư chết để chẩn đoán tại sao các lỗi thu nạp sự kiện xảy ra và giải quyết các vấn đề đường ống dữ liệu, và có thể có khả năng xử lý lại một số tin nhắn trong hàng đợi sau khi sửa nguyên nhân gốc của các lỗi.

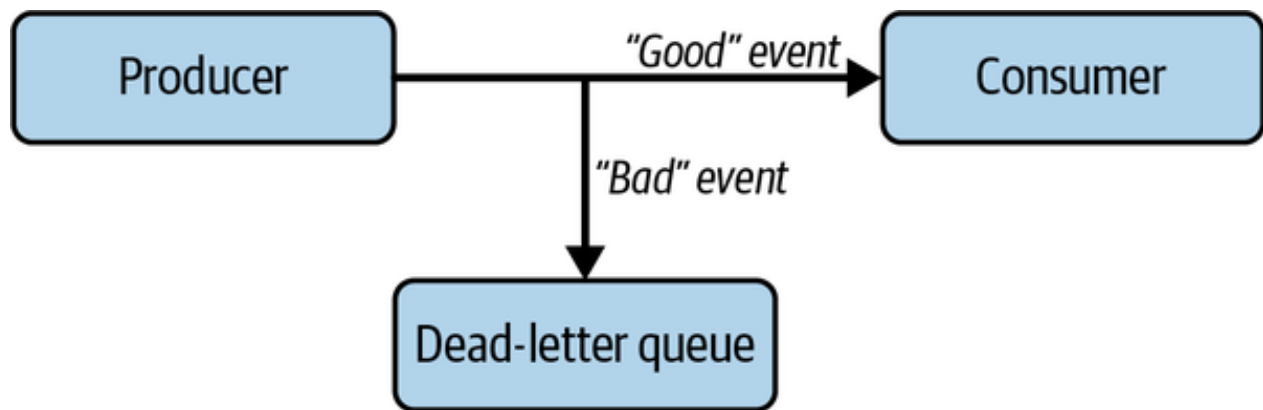


Figure 7-12. “Good” events are passed to the consumer, whereas “bad” events are stored in a dead-letter queue

Hình 7-12. Các sự kiện “tốt” được chuyển đến bên tiêu thụ, trong khi các sự kiện “xấu” được lưu trong một hàng đợi thư chết

Consumer Pull and Push — Kéo và Đẩy ở Bên tiêu thụ

A consumer subscribing to a topic can get events in two ways: push and pull. Let’s look at the ways some streaming technologies pull and push data. Kafka and Kinesis support only pull subscriptions. Subscribers read messages from a topic and confirm when they have been processed. In addition, to pull subscriptions, Pub/Sub and RabbitMQ support push subscriptions, allowing these services to write messages to a listener. Pull subscriptions are the default choice for most data engineering applications, but you may want to consider push capabilities for specialized applications. Note that pull-only message ingestion systems can still push if you add an extra layer to handle this.

Một bên tiêu thụ đăng ký một topic có thể nhận các sự kiện theo hai cách: đẩy (push) và kéo (pull). Hãy xem các cách mà một số công nghệ streaming kéo và đẩy dữ liệu. Kafka và Kinesis chỉ hỗ trợ các đăng ký kéo. Các bên đăng ký đọc các tin nhắn từ một topic và xác nhận khi chúng đã được xử lý. Ngoài các đăng ký kéo, Pub/Sub và RabbitMQ hỗ trợ các đăng ký đẩy, cho phép các dịch vụ này ghi các tin nhắn đến một bên lắng nghe (listener).

Các đăng ký kéo là lựa chọn mặc định cho hầu hết các ứng dụng kỹ thuật dữ liệu, nhưng bạn có thể muốn cân nhắc các năng lực đẩy cho các ứng dụng chuyên biệt. Lưu ý rằng các hệ thống thu nạp tin nhắn chỉ-kéo vẫn có thể đẩy nếu bạn thêm một lớp bổ sung để xử lý việc này.

Location — Vị trí

It is often desirable to integrate streaming across several locations for enhanced redundancy and to consume data close to where it is generated. As a general rule, the closer your ingestion is to where data originates, the better your bandwidth and latency. However, you need to balance this against the costs of moving data between regions to run analytics on a combined dataset. As always, data egress costs can spiral quickly. Do a careful evaluation of the trade-offs as you build out your architecture.

Thường mong muốn tích hợp streaming trên nhiều vị trí để tăng cường độ dư thừa và để tiêu thụ dữ liệu gần nơi nó được sinh ra. Theo quy tắc chung, việc thu nạp của bạn càng gần nơi dữ liệu bắt nguồn, băng thông và độ trễ của bạn càng tốt. Tuy nhiên, bạn cần cân bằng điều này với chi phí di chuyển dữ liệu giữa các region để chạy phân tích trên một tập dữ liệu kết hợp. Như mọi khi, chi phí dữ liệu đi ra (egress) có thể tăng vọt nhanh chóng. Hãy đánh giá cẩn thận các đánh đổi khi bạn xây dựng kiến trúc của mình.

Ways to Ingest Data — Các cách Thu nạp Dữ liệu

Now that we've described some of the significant patterns underlying batch and streaming ingestion, let's focus on ways you can ingest data. Although we will cite some common ways, keep in mind that the universe of data ingestion practices and technologies is vast and growing daily.

Giờ chúng ta đã mô tả một số mẫu hình quan trọng nền tảng cho thu nạp theo lô và theo luồng, hãy tập trung vào các cách bạn có thể thu nạp dữ liệu. Mặc dù chúng tôi sẽ nêu một số cách phổ biến, hãy ghi nhớ rằng vũ trụ các thực hành và công nghệ thu nạp dữ liệu là rất rộng lớn và đang tăng trưởng từng ngày.

Direct Database Connection — Kết nối Database Trực tiếp

Data can be pulled from databases for ingestion by querying and reading over a network connection. Most commonly, this connection is made using ODBC or JDBC. ODBC uses a driver hosted by a client accessing the database to translate commands issued to the standard ODBC API into commands issued to the database. The database returns query results over the wire, where the driver receives them and translates them back into a standard form and read by the client. For ingestion, the application utilizing the ODBC driver is an ingestion tool. The ingestion tool may pull data through many small queries or a single large query. JDBC is conceptually remarkably similar to ODBC. A Java driver connects to a remote database and serves as a translation layer between the standard JDBC API and the native network interface of the target database. It might seem strange to have a database API dedicated to a single programming language, but there are strong motivations for this. The Java Virtual Machine (JVM) is standard, portable across hardware architectures and operating systems, and provides the performance of compiled code through a just-in-time (JIT) compiler. The JVM is an extremely popular compiling VM for running code in a portable manner. JDBC provides extraordinary database driver portability. ODBC drivers are shipped as OS and architecture native binaries; database vendors must maintain versions for each architecture/OS version that they wish to support. On the other hand, vendors can ship a single JDBC driver that is compatible with any

JVM language (e.g., Java, Scala, Clojure, or Kotlin) and JVM data framework (i.e., Spark.) JDBC has become so popular that it is also used as an interface for non-JVM languages such as Python; the Python ecosystem provides translation tools that allow Python code to talk to a JDBC driver running on a local JVM. JDBC and ODBC are used extensively for data ingestion from relational databases, returning to the general concept of direct database connections. Various enhancements are used to accelerate data ingestion. Many data frameworks can parallelize several simultaneous connections and **partition** queries to pull data in parallel. On the other hand, nothing is free; using parallel connections also increases the load on the source database. JDBC and ODBC were long the gold standards for data ingestion from databases, but these connection standards are beginning to show their age for many data engineering applications. These connection standards struggle with nested data, and they send data as rows. This means that native nested data must be re-encoded as string data to be sent over the wire, and columns from columnar databases must be re-serialized as rows. As discussed in “File-Based Export and Ingestion”, many databases now support native file export that bypasses JDBC/ODBC and exports data directly in formats such as Parquet, ORC, and Avro. Alternatively, many cloud data warehouses provide direct REST APIs.

Dữ liệu có thể được kéo từ các database để thu nạp bằng cách truy vấn và đọc qua một kết nối mạng. Phổ biến nhất, kết nối này được thực hiện bằng ODBC hoặc JDBC. ODBC dùng một driver được lưu trữ bởi một client truy cập database để dịch các lệnh phát ra cho API ODBC tiêu chuẩn thành các lệnh phát ra cho database. Database trả về các kết quả truy vấn qua đường truyền, nơi driver nhận chúng và dịch chúng trở lại thành một dạng tiêu chuẩn được client đọc. Đối với thu nạp, ứng dụng sử dụng driver ODBC là một công cụ thu nạp. Công cụ thu nạp có thể kéo dữ liệu qua nhiều truy vấn nhỏ hoặc một truy vấn lớn duy nhất. JDBC về mặt khái niệm cực kỳ giống ODBC. Một driver Java kết nối tới một database từ xa và đóng vai trò một lớp dịch giữa API JDBC tiêu chuẩn và giao diện mạng gốc của database đích. Có vẻ kỳ lạ khi có một API database dành riêng cho một ngôn ngữ lập trình duy nhất, nhưng có những động cơ mạnh mẽ cho điều này. Máy ảo Java (Java Virtual Machine, JVM) là tiêu chuẩn, có thể chuyển dời (portable) qua các kiến trúc phần cứng và hệ điều hành, và cung cấp hiệu năng của mã được biên dịch thông qua một trình biên dịch tức thời (just-in-time, JIT). JVM là một máy ảo biên dịch cực kỳ phổ biến để chạy mã theo cách có thể chuyển dời. JDBC cung cấp khả năng chuyển dời driver database phi thường. Các driver ODBC được phát hành dưới dạng các tệp nhị phân gốc của OS và kiến trúc; các nhà cung cấp database phải duy trì các phiên bản cho mỗi phiên bản kiến trúc/OS mà họ muốn hỗ trợ. Mặt khác, các nhà cung cấp có thể phát hành một driver JDBC duy nhất tương thích với bất kỳ ngôn ngữ JVM nào (ví dụ, Java, Scala, Clojure, hoặc Kotlin) và bất kỳ framework dữ liệu JVM nào (tức là, Spark). JDBC đã trở nên phổ biến đến mức nó cũng được dùng làm một giao diện cho các ngôn ngữ phi JVM như Python; hệ sinh thái Python cung cấp các công cụ dịch cho phép mã Python giao tiếp với một driver JDBC chạy trên một JVM cục bộ. JDBC và ODBC được dùng rộng rãi cho thu nạp dữ liệu từ các database quan hệ, trở lại khái niệm chung về các kết nối database trực tiếp. Nhiều cải tiến khác nhau được dùng để tăng tốc thu nạp dữ liệu. Nhiều framework dữ liệu có thể song song hóa một số kết nối đồng thời và phân vùng các truy vấn để kéo dữ liệu song song. Mặt khác, không có gì là miễn phí; dùng các kết nối song song cũng làm tăng tải lên database nguồn. JDBC và ODBC từ lâu là các tiêu chuẩn vàng cho thu nạp dữ liệu từ các database, nhưng các tiêu chuẩn kết nối này đang bắt đầu lộ ra tuổi tác của chúng đối với nhiều ứng dụng kỹ thuật dữ liệu. Các tiêu chuẩn kết nối này gặp khó khăn với dữ liệu lồng nhau (nested), và chúng gửi dữ liệu dưới dạng các hàng. Điều này nghĩa là dữ liệu lồng nhau gốc phải được mã hóa lại thành dữ liệu chuỗi để được gửi qua đường truyền, và các cột từ các database cột phải được tuần tự hóa lại thành các hàng. Như đã bàn trong “Xuất và Thu nạp dựa trên Tệp”, nhiều database giờ hỗ trợ xuất tệp gốc bỏ qua JDBC/ODBC và xuất dữ liệu trực tiếp ở các định dạng như Parquet, ORC và Avro. Một cách khác, nhiều kho dữ liệu đám mây cung

cấp các REST API trực tiếp.

JDBC connections should generally be integrated with other ingestion technologies. For example, we commonly use a reader process to connect to a database with JDBC, write the extracted data into multiple objects, and then orchestrate ingestion into a downstream system (see Figure 7-13). The reader process can run in a wholly ephemeral cloud instance or in an **orchestration** system.

Các kết nối JDBC nhìn chung nên được tích hợp với các công nghệ thu nạp khác. Ví dụ, chúng tôi thường dùng một quy trình đọc (reader) để kết nối tới một database bằng JDBC, ghi dữ liệu được trích xuất vào nhiều đối tượng, rồi điều phối việc thu nạp vào một hệ thống hạ nguồn (xem Hình 7-13). Quy trình đọc có thể chạy trong một thực thể (instance) đám mây hoàn toàn phù du (ephemeral) hoặc trong một hệ thống điều phối.

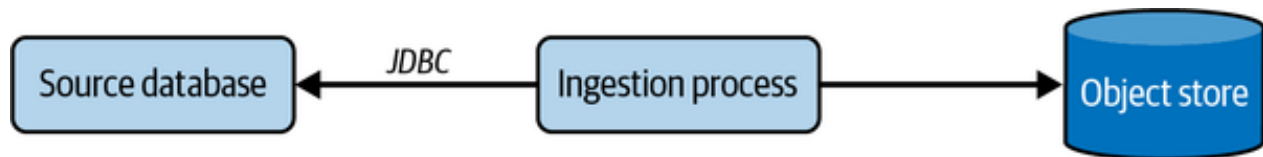


Figure 7-13. An ingestion process reads from a source database using JDBC, and then writes objects into object storage. A target database (not shown) can be triggered to ingest the data with an API call from an orchestration system.

Hình 7-13. Một quy trình thu nạp đọc từ một database nguồn bằng JDBC, rồi ghi các đối tượng vào kho lưu trữ đối tượng. Một database đích (không hiển thị) có thể được kích hoạt để thu nạp dữ liệu bằng một lời gọi API từ một hệ thống điều phối.

Change Data Capture — Thu thập Thay đổi Dữ liệu

Change data capture (CDC), introduced in Chapter 2 , is the process of ingesting changes from a source database system. For example, we might have a source PostgreSQL system that supports an application and periodically or continuously ingests table changes for analytics. Note that our discussion here is by no means exhaustive. We introduce you to common patterns but suggest that you read the documentation on a particular database to handle the details of CDC strategies.

Thu thập thay đổi dữ liệu (change data capture, CDC), được giới thiệu trong Chương 2, là quá trình thu nạp các thay đổi từ một hệ thống database nguồn. Ví dụ, chúng ta có thể có một hệ thống PostgreSQL nguồn hỗ trợ một ứng dụng và định kỳ hoặc liên tục thu nạp các thay đổi bảng để phân tích. Lưu ý rằng thảo luận của chúng tôi ở đây hoàn toàn không đầy đủ. Chúng tôi giới thiệu với bạn các mẫu hình phổ biến nhưng đề xuất rằng bạn đọc tài liệu về một database cụ thể để xử lý các chi tiết của các chiến lược CDC.

Batch-oriented CDC — CDC định hướng theo lô

If the database table in question has an `updated_at` field containing the last time a record was written or updated, we can query the table to find all updated rows since a specified time. We set the filter timestamp based on when we last captured changed rows from the tables. This process allows us to pull changes and differentially update a target table. This form of batch-oriented CDC has a key limitation: while we can easily determine which rows have changed since a point in time, we don't necessarily obtain all changes that were applied to these rows. Consider the example of running batch CDC on a bank account table every 24 hours.

Nếu bảng database đang xét có một trường `updated_at` chứa thời điểm cuối cùng một bản ghi được ghi hoặc cập nhật, chúng ta có thể truy vấn bảng để tìm tất cả các hàng được cập nhật kể từ một thời điểm chỉ định. Chúng ta đặt dấu thời gian bộ lọc dựa trên thời điểm chúng ta thu các hàng thay đổi từ các bảng lần cuối. Quá trình này cho phép chúng ta kéo các thay đổi và cập nhật chênh lệch một bảng đích. Hình thức CDC định hướng theo lô này có một hạn chế then chốt: trong khi chúng ta có thể dễ dàng xác định những hàng nào đã thay đổi kể từ một thời điểm, chúng ta không nhất thiết thu được tất cả các thay đổi đã được áp dụng cho các hàng này. Xét ví dụ chạy CDC theo lô trên một bảng tài khoản ngân hàng mỗi 24 giờ.

This operational table shows the current account balance for each account. When money is moved in and out of accounts, the banking application runs a transaction to update the balance. When we run a query to return all rows in the account table that changed in the last 24 hours, we'll see records for each account that recorded a transaction. Suppose that a certain customer withdrew money five times using a debit card in the last 24 hours. Our query will return only the last account balance recorded in the 24 hour period; other records over the period won't appear. This issue can be mitigated by utilizing an insert-only schema, where each account transaction is recorded as a new record in the table (see "Insert-Only").

Bảng vận hành này hiển thị số dư tài khoản hiện tại cho mỗi tài khoản. Khi tiền được chuyển vào và ra khỏi các tài khoản, ứng dụng ngân hàng chạy một giao dịch để cập nhật số dư. Khi chúng ta chạy một truy vấn để trả về tất cả các hàng trong bảng tài khoản đã thay đổi trong 24 giờ qua, chúng ta sẽ thấy các bản ghi cho mỗi tài khoản đã ghi nhận một giao dịch. Giả sử một khách hàng nhất định đã rút tiền năm lần bằng thẻ ghi nợ trong 24 giờ qua. Truy vấn của chúng ta sẽ chỉ trả về số dư tài khoản cuối cùng được ghi nhận trong khoảng thời gian 24 giờ; các bản ghi khác trong khoảng thời gian đó sẽ không xuất hiện. Vấn đề này có thể được giảm thiểu bằng cách sử dụng một lược đồ chỉ-chèn (insert-only), nơi mỗi giao dịch tài khoản được ghi nhận như một bản ghi mới trong bảng (xem "Chỉ-Chèn").

Continuous CDC — CDC liên tục

Continuous CDC captures all table history and can support near real-time data ingestion, either for real-time database replication or to feed real-time streaming analytics. Rather than running periodic queries to get a batch of table changes, continuous CDC treats each write to the database as an event. We can capture an event stream for continuous CDC in a couple of ways. One of the most common approaches with a transactional database such as PostgreSQL is log-based CDC . The database binary log records every change to the database sequentially (see "Database Logs") A CDC tool can read this log and send the events to a target, such as the Apache Kafka Debezium streaming platform. Some databases support a simplified, managed CDC paradigm. For instance, many cloud-hosted databases can be configured to directly trigger a **serverless** function or write to an event stream every time a change happens in the database. This completely frees engineers from worrying about the details of how events are captured in the database and forwarded.

CDC liên tục thu thập toàn bộ lịch sử bảng và có thể hỗ trợ thu nạp dữ liệu gần thời gian thực, hoặc cho sao chép (replication) database thời gian thực hoặc để cấp cho phân tích streaming thời gian thực. Thay vì chạy các truy vấn định kỳ để lấy một lô các thay đổi bảng, CDC liên tục xử lý mỗi lần ghi vào database như một sự kiện. Chúng ta có thể thu một stream sự kiện cho CDC liên tục theo một vài cách. Một trong những cách tiếp cận phổ biến nhất với một database giao dịch như PostgreSQL là CDC dựa trên log. Log nhị phân của database ghi lại mọi thay đổi đối với database một cách tuần tự (xem "Database Logs").

Một công cụ CDC có thể đọc log này và gửi các sự kiện đến một đích, chẳng hạn như nền tảng streaming Apache Kafka Debezium. Một số database hỗ trợ một mô hình CDC được quản lý, đơn giản hóa. Chẳng hạn, nhiều database lưu trữ trên đám mây có thể được cấu hình để kích hoạt trực tiếp một hàm serverless hoặc ghi vào một stream sự kiện mỗi khi một thay đổi xảy ra trong database. Điều này hoàn toàn giải phóng các kỹ sư khỏi việc lo lắng về các chi tiết của cách các sự kiện được thu trong database và chuyển tiếp.

CDC and database replication — CDC và sao chép database

CDC can be used to replicate between databases: events are buffered into a stream and asynchronously written into a second database. However, many databases natively support a tightly coupled version of replication (synchronous replication) that keeps the replica fully in sync with the primary database. Synchronous replication typically requires that the primary database and the replica are of the same type (e.g., PostgreSQL to PostgreSQL). The advantage of synchronous replication is that the secondary database can offload work from the primary database by acting as a read replica; read queries can be redirected to the replica. The query will return the same results that would be returned from the primary database. Read replicas are often used in batch data ingestion patterns to allow large scans to run without overloading the primary production database. In addition, an application can be configured to fail over to the replica if the primary database becomes unavailable. No data will be lost in the failover because the replica is entirely in sync with the primary database. The advantage of asynchronous CDC replication is a **loosely** coupled architecture pattern. While the replica might be slightly delayed from the primary database, this is often not a problem for analytics applications, and events can now be directed to a variety of targets; we might run CDC replication while simultaneously directing events to object storage and a streaming analytics processor.

CDC có thể được dùng để sao chép giữa các database: các sự kiện được đệm vào một stream và được ghi bất đồng bộ vào một database thứ hai. Tuy nhiên, nhiều database hỗ trợ một cách gốc một phiên bản sao chép ghép chặt (sao chép đồng bộ - synchronous replication) giữ cho bản sao (replica) hoàn toàn đồng bộ với database chính (primary). Sao chép đồng bộ thường yêu cầu rằng database chính và bản sao cùng loại (ví dụ, PostgreSQL sang PostgreSQL). Lợi thế của sao chép đồng bộ là database phụ có thể giảm tải công việc từ database chính bằng cách đóng vai trò một bản sao đọc (read replica); các truy vấn đọc có thể được chuyển hướng đến bản sao. Truy vấn sẽ trả về cùng các kết quả như sẽ được trả về từ database chính. Các bản sao đọc thường được dùng trong các mẫu hình thu nạp dữ liệu theo lô để cho phép các lần quét lớn chạy mà không làm quá tải database sản xuất chính. Ngoài ra, một ứng dụng có thể được cấu hình để chuyển đổi dự phòng sang bản sao nếu database chính trở nên không khả dụng. Không dữ liệu nào sẽ bị mất trong quá trình chuyển đổi dự phòng vì bản sao hoàn toàn đồng bộ với database chính. Lợi thế của sao chép CDC bất đồng bộ là một mẫu hình kiến trúc ghép lỏng. Mặc dù bản sao có thể chậm hơn một chút so với database chính, đây thường không phải là một vấn đề đối với các ứng dụng phân tích, và các sự kiện giờ có thể được hướng đến nhiều đích khác nhau; chúng ta có thể chạy sao chép CDC trong khi đồng thời hướng các sự kiện đến kho lưu trữ đối tượng và một bộ xử lý phân tích streaming.

CDC considerations — Các cân nhắc về CDC

Like anything in technology, CDC is not free. CDC consumes various database resources, such as memory, disk bandwidth, storage, CPU time, and network bandwidth. Engineers should work with

production teams and run tests before turning on CDC on production systems to avoid operational problems. Similar considerations apply to synchronous replication. For batch CDC, be aware that running any large batch query against a transactional production system can cause excessive load. Either run such queries only at off-hours or use a read replica to avoid burdening the primary database.

Như bất cứ thứ gì trong công nghệ, CDC không miễn phí. CDC tiêu thụ nhiều tài nguyên database khác nhau, chẳng hạn như bộ nhớ, băng thông đĩa, lưu trữ, thời gian CPU và băng thông mạng. Các kỹ sư nên làm việc với các đội sản xuất và chạy các bài kiểm thử trước khi bật CDC trên các hệ thống sản xuất để tránh các vấn đề vận hành. Các cân nhắc tương tự áp dụng cho sao chép đồng bộ. Đối với CDC theo lô, hãy lưu ý rằng chạy bất kỳ truy vấn lô lớn nào trên một hệ thống sản xuất giao dịch có thể gây tải quá mức. Hoặc chỉ chạy các truy vấn như vậy trong những giờ ngoài giờ làm việc hoặc dùng một bản sao đọc để tránh gây gánh nặng cho database chính.

APIs — APIs

The bulk of software engineering is just plumbing. — Karl Hughes As we mentioned in Chapter 5 , APIs are a data source that continues to grow in importance and popularity. A typical organization may have hundreds of external data sources such as SaaS platforms or partner companies. The hard reality is that no proper standard exists for data exchange over APIs. Data engineers can spend a significant amount of time reading documentation, communicating with external data owners, and writing and maintaining API connection code. Three trends are slowly changing this situation. First, many vendors provide API client libraries for various programming languages that remove much of the complexity of API access. Second, numerous data connector platforms are available now as SaaS, open source, or managed open source. These platforms provide turnkey data connectivity to many data sources; they offer frameworks for writing custom connectors for unsupported data sources. See “Managed Data Connectors” . The third trend is the emergence of data sharing (discussed in Chapter 5)— i.e., the ability to exchange data through a standard platform such as BigQuery, Snowflake, Redshift, or S3. Once data lands on one of these platforms, it is straightforward to store it, process it, or move it somewhere else. Data sharing has had a large and rapid impact in the data engineering space. Don’t reinvent the wheel when data sharing is not an option and direct API access is necessary. While a managed service might look like an expensive option, consider the value of your time and the **opportunity cost** of building API connectors when you could be spending your time on higher-value work. In addition, many managed services now support building custom API connectors. This may provide API technical specifications in a standard format or writing connector code that runs in a serverless function framework (e.g., AWS **Lambda**) while letting the managed service handle the details of scheduling and synchronization. Again, these services can be a huge time-saver for engineers, both for development and ongoing maintenance. Reserve your custom connection work for APIs that aren’t well supported by existing frameworks; you will find that there are still plenty of these to work on. Handling custom API connections has two main aspects: software development and ops. Follow software development best practices; you should use version control, continuous delivery, and automated testing. In addition to following DevOps best practices, consider an orchestration framework, which can dramatically streamline the operational burden of data ingestion.

Phần lớn kỹ thuật phần mềm chỉ là lắp ống nước. — Karl Hughes. Như chúng tôi đã đề cập trong Chương 5, các API là một nguồn dữ liệu tiếp tục tăng về tầm quan trọng và độ phổ biến. Một tổ chức điển hình có thể có hàng trăm nguồn dữ liệu bên ngoài như các nền tảng SaaS hoặc các công ty đối tác. Thực tế khắc nghiệt là không có tiêu chuẩn phù hợp

nào tồn tại cho việc trao đổi dữ liệu qua các API. Các kỹ sư dữ liệu có thể dành một lượng thời gian đáng kể để đọc tài liệu, giao tiếp với các chủ sở hữu dữ liệu bên ngoài, và viết cùng bảo trì mã kết nối API. Ba xu hướng đang dần thay đổi tình hình này. Thứ nhất, nhiều nhà cung cấp cung cấp các thư viện client API cho nhiều ngôn ngữ lập trình khác nhau loại bỏ phần lớn sự phức tạp của việc truy cập API. Thứ hai, nhiều nền tảng kết nối dữ liệu (data connector) giờ có sẵn dưới dạng SaaS, mã nguồn mở, hoặc mã nguồn mở được quản lý. Các nền tảng này cung cấp khả năng kết nối dữ liệu sẵn dùng (turnkey) tới nhiều nguồn dữ liệu; chúng cung cấp các framework để viết các connector tùy chỉnh cho các nguồn dữ liệu không được hỗ trợ. Xem “Các Connector Dữ liệu Được quản lý”. Xu hướng thứ ba là sự xuất hiện của chia sẻ dữ liệu (data sharing, được bàn trong Chương 5)—tức là, khả năng trao đổi dữ liệu thông qua một nền tảng tiêu chuẩn như BigQuery, Snowflake, Redshift, hoặc S3. Một khi dữ liệu đáp xuống một trong các nền tảng này, việc lưu trữ nó, xử lý nó, hoặc di chuyển nó đến nơi khác là đơn giản. Chia sẻ dữ liệu đã có tác động lớn và nhanh trong lĩnh vực kỹ thuật dữ liệu. Đừng phát minh lại bánh xe khi chia sẻ dữ liệu không phải là một lựa chọn và truy cập API trực tiếp là cần thiết. Mặc dù một dịch vụ được quản lý có thể trông như một lựa chọn đắt đỏ, hãy cân nhắc giá trị thời gian của bạn và chi phí cơ hội của việc xây các connector API trong khi bạn có thể đang dành thời gian cho công việc có giá trị cao hơn. Ngoài ra, nhiều dịch vụ được quản lý giờ hỗ trợ xây các connector API tùy chỉnh. Điều này có thể cung cấp các đặc tả kỹ thuật API ở một định dạng tiêu chuẩn hoặc viết mã connector chạy trong một framework hàm serverless (ví dụ, AWS Lambda) trong khi để dịch vụ được quản lý xử lý các chi tiết về lập lịch và đồng bộ hóa. Một lần nữa, các dịch vụ này có thể tiết kiệm thời gian rất lớn cho các kỹ sư, cả cho phát triển lẫn bảo trì liên tục. Hãy dành công việc kết nối tùy chỉnh của bạn cho các API không được hỗ trợ tốt bởi các framework hiện có; bạn sẽ thấy rằng vẫn còn rất nhiều cái như vậy để làm. Xử lý các kết nối API tùy chỉnh có hai khía cạnh chính: phát triển phần mềm và vận hành (ops). Hãy tuân theo các thực hành tốt nhất về phát triển phần mềm; bạn nên dùng kiểm soát phiên bản (version control), phân phối liên tục (continuous delivery), và kiểm thử tự động. Ngoài việc tuân theo các thực hành tốt nhất về DevOps, hãy cân nhắc một framework điều phối, thứ có thể giảm thiểu đáng kể gánh nặng vận hành của thu nạp dữ liệu.

Message Queues and Event-Streaming Platforms — Các Hàng đợi Tin nhắn và Nền tảng Event-Streaming

Message queues and event-streaming platforms are widespread ways to ingest real-time data from web and mobile applications, IoT sensors, and smart devices. As real-time data becomes more ubiquitous, you'll often find yourself either introducing or retrofitting ways to handle real-time data in your ingestion workflows. As such, it's essential to know how to ingest real-time data. Popular real-time data ingestion includes message queues or event-streaming platforms, which we covered in Chapter 5 . Though these are both source systems, they also act as ways to ingest data. In both cases, you consume events from the publisher you **subscribe** to. Recall the differences between messages and streams. A message is handled at the individual event level and is meant to be transient. Once a message is consumed, it is acknowledged and removed from the queue. On the other hand, a stream ingests events into an ordered log. The log persists for as long as you wish, allowing events to be queried over various ranges, aggregated, and combined with other streams to create new transformations published to downstream consumers. In Figure 7-14 , we have two producers (producers 1 and 2) sending events to two consumers (consumers 1 and 2). These events are combined into a new dataset and sent to a producer for downstream consumption.

Các hàng đợi tin nhắn và nền tảng event-streaming là những cách phổ biến rộng rãi để

thu nạp dữ liệu thời gian thực từ các ứng dụng web và di động, các cảm biến IoT, và các thiết bị thông minh. Khi dữ liệu thời gian thực trở nên phổ biến hơn, bạn sẽ thường thấy mình hoặc đang giới thiệu hoặc đang trang bị thêm các cách xử lý dữ liệu thời gian thực trong các luồng công việc thu nạp của mình. Như vậy, điều thiết yếu là biết cách thu nạp dữ liệu thời gian thực. Thu nạp dữ liệu thời gian thực phổ biến bao gồm các hàng đợi tin nhắn hoặc nền tảng event-streaming, mà chúng ta đã đề cập trong Chương 5. Mặc dù đây đều là các hệ thống nguồn, chúng cũng đóng vai trò là các cách thu nạp dữ liệu. Trong cả hai trường hợp, bạn tiêu thụ các sự kiện từ bên xuất bản (publisher) mà bạn đăng ký. Hãy nhớ lại các khác biệt giữa tin nhắn và stream. Một tin nhắn được xử lý ở cấp độ sự kiện riêng lẻ và có ý nghĩa là tạm thời. Một khi một tin nhắn được tiêu thụ, nó được xác nhận và bị xóa khỏi hàng đợi. Mặt khác, một stream thu nạp các sự kiện vào một log có thứ tự. Log tồn tại lâu tùy ý bạn muốn, cho phép các sự kiện được truy vấn qua nhiều khoảng khác nhau, được tổng hợp, và được kết hợp với các stream khác để tạo các biến đổi mới được xuất bản đến các bên tiêu thụ hạ nguồn. Trong Hình 7-14, chúng ta có hai bên sản xuất (producer 1 và 2) gửi các sự kiện đến hai bên tiêu thụ (consumer 1 và 2). Các sự kiện này được kết hợp thành một tập dữ liệu mới và gửi đến một bên sản xuất để tiêu thụ hạ nguồn.

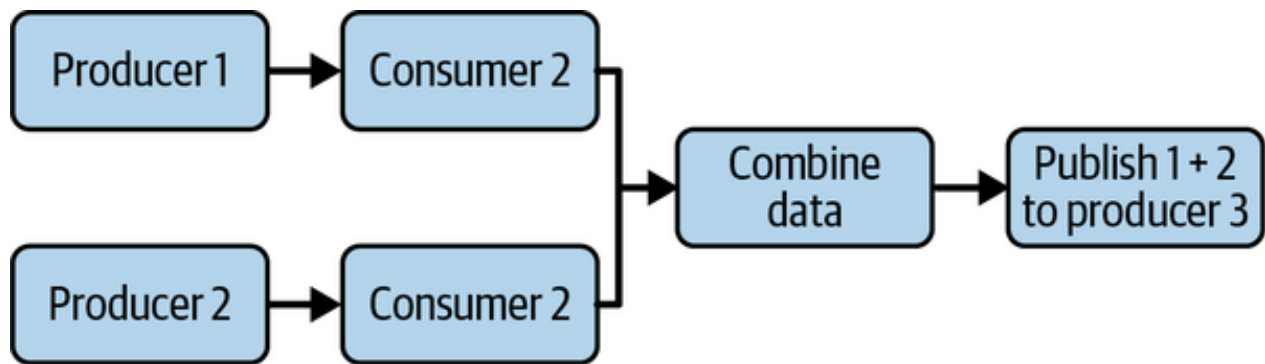


Figure 7-14. Two datasets are produced and consumed (producers 1 and 2), and then combined, with the combined data published to a new producer (producer 3). The last point is an essential difference between batch and streaming ingestion. Whereas batch usually involves static workflows (ingest data, store it, transform it, and serve it), messages and streams are fluid. Ingestion can be nonlinear, with data being published, consumed, republished, and re-consumed. When designing your real-time ingestion workflows, keep in mind how data will flow. Another consideration is the throughput of your real-time data pipelines. Messages and events should flow with as little latency as possible, meaning you should provision adequate partition (or shard) bandwidth and throughput. Provide sufficient memory, disk, and CPU resources for event processing, and if you're managing your real-time pipelines, incorporate autoscaling to handle spikes and save money as load decreases. For these reasons, managing your streaming platform can entail significant overhead. Consider managed services for your real-time ingestion pipelines, and focus your attention on ways to get value from your real-time data.

Hình 7-14. Hai tập dữ liệu được sản xuất và tiêu thụ (producer 1 và 2), rồi được kết hợp, với dữ liệu kết hợp được xuất bản đến một bên sản xuất mới (producer 3). Điểm cuối cùng là một khác biệt thiết yếu giữa thu nạp theo lô và theo luồng. Trong khi theo lô thường liên quan đến các luồng công việc tĩnh (thu nạp dữ liệu, lưu trữ nó, biến đổi nó, và phục vụ nó), tin nhắn và stream thì linh hoạt. Thu nạp có thể phi tuyến, với dữ liệu được xuất bản, tiêu thụ, tái xuất bản, và tái tiêu thụ. Khi thiết kế các luồng công việc thu nạp thời gian thực của bạn, hãy ghi nhớ cách dữ liệu sẽ chảy. Một cân nhắc khác là thông lượng của các đường ống dữ liệu thời gian thực của bạn. Tin nhắn và sự kiện nên chảy với độ trễ ít nhất

có thể, nghĩa là bạn nên cấp phát đủ băng thông và thông lượng phân vùng (hoặc shard). Hãy cung cấp đủ tài nguyên bộ nhớ, đĩa và CPU cho xử lý sự kiện, và nếu bạn đang quản lý các đường ống thời gian thực của mình, hãy tích hợp tự động mở rộng (autoscaling) để xử lý các đỉnh và tiết kiệm tiền khi tải giảm. Vì những lý do này, quản lý nền tảng streaming của bạn có thể kéo theo chi phí phụ trợ đáng kể. Hãy cân nhắc các dịch vụ được quản lý cho các đường ống thu nạp thời gian thực của bạn, và tập trung sự chú ý của bạn vào các cách thu được giá trị từ dữ liệu thời gian thực của mình.

Managed Data Connectors — Các Connector Dữ liệu Được quản lý

These days, if you're considering writing a data ingestion connector to a database or API, ask yourself: has this already been created? Furthermore, is there a service that will manage the nitty-gritty details of this connection for me? "APIs" mentions the popularity of managed data connector platforms and frameworks. These tools aim to provide a standard set of connectors available out of the box to spare data engineers building complicated plumbing to connect to a particular source. Instead of creating and managing a data connector, you outsource this service to a third party. Generally, options in the space allow users to set a target and source, ingest in various ways (e.g., CDC, replication, truncate and reload), set permissions and credentials, configure an update frequency, and begin syncing data. The vendor or cloud behind the scenes fully manages and monitors data syncs. If data synchronization fails, you'll receive an alert with logged information on the cause of the error. We suggest using managed connector platforms instead of creating and managing your connectors. Vendors and OSS projects each typically have hundreds of prebuilt connector options and can easily create custom connectors. The creation and management of data connectors is largely undifferentiated heavy lifting these days and should be outsourced whenever possible.

Ngày nay, nếu bạn đang cân nhắc viết một connector thu nạp dữ liệu tới một database hoặc API, hãy tự hỏi: cái này đã được tạo ra chưa? Hơn nữa, có một dịch vụ nào sẽ quản lý các chi tiết tỉ mỉ của kết nối này cho tôi không? Phần "APIs" đề cập đến sự phổ biến của các nền tảng và framework connector dữ liệu được quản lý. Các công cụ này nhằm cung cấp một bộ connector tiêu chuẩn sẵn có ngay khi dùng (out of the box) để giúp các kỹ sư dữ liệu khỏi phải xây dựng các đường ống phức tạp nhằm kết nối tới một nguồn cụ thể. Thay vì tạo và quản lý một connector dữ liệu, bạn thuê ngoài (outsource) dịch vụ này cho một bên thứ ba. Nhìn chung, các tùy chọn trong lĩnh vực này cho phép người dùng đặt một đích và nguồn, thu nạp theo nhiều cách khác nhau (ví dụ, CDC, sao chép, cắt và nạp lại - truncate and reload), đặt quyền và thông tin xác thực, cấu hình một tần suất cập nhật, và bắt đầu đồng bộ dữ liệu. Nhà cung cấp hoặc đám mây đứng sau hậu trường quản lý và giám sát hoàn toàn các lần đồng bộ dữ liệu. Nếu đồng bộ dữ liệu thất bại, bạn sẽ nhận được một cảnh báo với thông tin được ghi log về nguyên nhân của lỗi. Chúng tôi đề xuất dùng các nền tảng connector được quản lý thay vì tạo và quản lý các connector của bạn. Các nhà cung cấp và các dự án OSS mỗi bên thường có hàng trăm tùy chọn connector dựng sẵn và có thể dễ dàng tạo các connector tùy chỉnh. Việc tạo và quản lý các connector dữ liệu phần lớn là công việc nặng nhọc không tạo khác biệt ngày nay và nên được thuê ngoài bất cứ khi nào có thể.

Moving Data with Object Storage — Di chuyển Dữ liệu với Kho lưu trữ Đối tượng

Object storage is a **multitenant** system in public clouds, and it supports storing massive amounts of data. This makes object storage ideal for moving data in and out of data lakes, between teams, and transferring data between organizations. You can even provide short-term access to an object with a signed URL, giving a user temporary permission. In our view, object storage is the most optimal and secure way to handle file exchange. Public cloud storage implements the latest security standards, has a robust track record of scalability and reliability, accepts files of arbitrary types and sizes, and provides high-performance data movement. We discussed object storage much more extensively in Chapter 6.

Kho lưu trữ đối tượng (object storage) là một hệ thống đa thuê bao (multitenant) trong các đám mây công cộng, và nó hỗ trợ lưu trữ lượng dữ liệu khổng lồ. Điều này khiến kho lưu trữ đối tượng lý tưởng để di chuyển dữ liệu vào và ra khỏi các hồ dữ liệu, giữa các đội, và chuyển dữ liệu giữa các tổ chức. Bạn thậm chí có thể cung cấp quyền truy cập ngắn hạn tới một đối tượng bằng một URL có chữ ký (signed URL), trao cho một người dùng quyền tạm thời. Theo quan điểm của chúng tôi, kho lưu trữ đối tượng là cách tối ưu và an toàn nhất để xử lý trao đổi tệp. Lưu trữ đám mây công cộng triển khai các tiêu chuẩn bảo mật mới nhất, có một thành tích vững chắc về khả năng mở rộng và độ tin cậy, chấp nhận các tệp với kiểu và kích thước tùy ý, và cung cấp khả năng di chuyển dữ liệu hiệu năng cao. Chúng tôi đã bàn về kho lưu trữ đối tượng rộng hơn nhiều trong Chương 6.

EDI — EDI

Another practical reality for data engineers is electronic data interchange (EDI). The term is vague enough to refer to any data movement method. It usually refers to somewhat archaic means of file exchange, such as by email or flash drive. Data engineers will find that some data sources do not support more modern means of data transport, often because of archaic IT systems or human process limitations. Engineers can at least enhance EDI through automation. For example, they can set up a cloud-based email server that saves files onto company object storage as soon as they are received. This can trigger orchestration processes to ingest and process data. This is much more robust than an employee downloading the attached file and manually uploading it to an internal system, which we still frequently see.

Một thực tế thực tiễn khác đối với các kỹ sư dữ liệu là trao đổi dữ liệu điện tử (electronic data interchange, EDI). Thuật ngữ này đủ mơ hồ để chỉ bất kỳ phương pháp di chuyển dữ liệu nào. Nó thường chỉ các phương tiện trao đổi tệp có phần cổ lỗ, chẳng hạn như qua email hoặc ổ flash. Các kỹ sư dữ liệu sẽ thấy rằng một số nguồn dữ liệu không hỗ trợ các phương tiện vận chuyển dữ liệu hiện đại hơn, thường vì các hệ thống IT cổ lỗ hoặc các hạn chế của quy trình con người. Các kỹ sư ít nhất có thể tăng cường EDI thông qua tự động hóa. Ví dụ, họ có thể thiết lập một máy chủ email dựa trên đám mây lưu các tệp vào kho lưu trữ đối tượng của công ty ngay khi chúng được nhận. Điều này có thể kích hoạt các quy trình điều phối để thu nạp và xử lý dữ liệu. Cách này mạnh mẽ hơn nhiều so với việc một nhân viên tải xuống tệp đính kèm và tải nó lên một hệ thống nội bộ một cách thủ công, điều mà chúng tôi vẫn thường xuyên thấy.

Databases and File Export — Database và Xuất Tập

Engineers should be aware of how the source database systems handle file export. Export involves large data scans that significantly load the database for many transactional systems. Source system engineers must assess when these scans can be run without affecting application performance and might opt for a strategy to mitigate the load. Export queries can be broken into smaller exports by querying over key ranges or one partition at a time. Alternatively, a read replica can reduce load. Read replicas are especially appropriate if exports happen many times a day and coincide with a high source system load. Major cloud data warehouses are highly optimized for direct file export. For example, Snowflake, BigQuery, Redshift, and others support direct export to object storage in various formats.

Các kỹ sư nên ý thức về cách các hệ thống database nguồn xử lý xuất tập. Xuất liên quan đến các lần quét dữ liệu lớn gây tải đáng kể cho database đối với nhiều hệ thống giao dịch. Các kỹ sư hệ thống nguồn phải đánh giá khi nào các lần quét này có thể chạy mà không ảnh hưởng đến hiệu năng ứng dụng và có thể chọn một chiến lược để giảm thiểu tải. Các truy vấn xuất có thể được chia thành các lần xuất nhỏ hơn bằng cách truy vấn qua các khoảng khóa (key range) hoặc một phân vùng tại một thời điểm. Một cách khác, một bản sao đọc có thể giảm tải. Các bản sao đọc đặc biệt phù hợp nếu các lần xuất xảy ra nhiều lần một ngày và trùng với tải hệ thống nguồn cao. Các kho dữ liệu đám mây lớn được tối ưu hóa cao cho xuất tập trực tiếp. Ví dụ, Snowflake, BigQuery, Redshift, và các nền tảng khác hỗ trợ xuất trực tiếp tới kho lưu trữ đối tượng ở nhiều định dạng khác nhau.

Practical Issues with Common File Formats — Các Vấn đề Thực tiễn với Các Định dạng Tập Phổ biến

Engineers should also be aware of the file formats to export. CSV is still ubiquitous and highly error prone at the time of this writing. Namely, CSV's default delimiter is also one of the most familiar characters in the English language—the comma! But it gets worse. CSV is by no means a uniform format. Engineers must stipulate the delimiter, quote characters, and escaping to appropriately handle the export of string data. CSV also doesn't natively encode schema information or directly support nested structures. CSV file encoding and schema information must be configured in the target system to ensure appropriate ingestion. Autodetection is a convenience feature provided in many cloud environments but is inappropriate for production ingestion. As a best practice, engineers should record CSV encoding and schema details in file metadata. More robust and expressive export formats include Parquet, Avro, Arrow, and ORC or JSON. These formats natively encode schema information and handle arbitrary string data with no particular intervention. Many of them also handle nested data structures natively so that JSON fields are stored using internal nested structures rather than simple strings. For columnar databases, columnar formats (Parquet, Arrow, ORC) allow more efficient data export because columns can be directly transcoded between formats. These formats are also generally more optimized for query engines. The Arrow file format is designed to map data directly into processing engine memory, providing high performance in data lake environments. The disadvantage of these newer formats is that many of them are not natively supported by source systems. Data engineers are often forced to work with CSV data and then build robust exception handling and error detection to ensure **data quality** on ingestion. See Appendix A for a more extensive discussion of file formats.

Các kỹ sư cũng nên ý thức về các định dạng tập để xuất. CSV vẫn phổ biến khắp nơi và rất dễ gây lỗi tại thời điểm viết bài này. Cụ thể, dấu phân tách (delimiter) mặc định của CSV cũng là một trong những ký tự quen thuộc nhất trong tiếng Anh—dấu phẩy! Nhưng

nó còn tệ hơn. CSV hoàn toàn không phải là một định dạng đồng nhất. Các kỹ sư phải quy định dấu phân tách, các ký tự trích dẫn (quote), và việc thoát (escaping) để xử lý phù hợp việc xuất dữ liệu chuỗi. CSV cũng không mã hóa gốc thông tin lược đồ hoặc hỗ trợ trực tiếp các cấu trúc lồng nhau. Thông tin mã hóa và lược đồ của tệp CSV phải được cấu hình trong hệ thống đích để đảm bảo thu nạp phù hợp. Tự động phát hiện (autodetection) là một tính năng tiện lợi được cung cấp trong nhiều môi trường đám mây nhưng không phù hợp cho thu nạp sản xuất. Như một thực hành tốt nhất, các kỹ sư nên ghi lại các chi tiết mã hóa và lược đồ của CSV trong siêu dữ liệu của tệp. Các định dạng xuất mạnh mẽ và biểu cảm hơn bao gồm Parquet, Avro, Arrow, và ORC hoặc JSON. Các định dạng này mã hóa gốc thông tin lược đồ và xử lý dữ liệu chuỗi tùy ý mà không cần can thiệp đặc biệt. Nhiều cái trong số chúng cũng xử lý gốc các cấu trúc dữ liệu lồng nhau sao cho các trường JSON được lưu bằng các cấu trúc lồng nhau nội bộ thay vì các chuỗi đơn giản. Đối với các database cột, các định dạng cột (Parquet, Arrow, ORC) cho phép xuất dữ liệu hiệu quả hơn vì các cột có thể được chuyển mã (transcode) trực tiếp giữa các định dạng. Các định dạng này nhìn chung cũng được tối ưu hóa hơn cho các engine truy vấn. Định dạng tệp Arrow được thiết kế để ánh xạ dữ liệu trực tiếp vào bộ nhớ của engine xử lý, cung cấp hiệu năng cao trong các môi trường hồ dữ liệu. Bất lợi của các định dạng mới hơn này là nhiều cái trong số chúng không được hỗ trợ gốc bởi các hệ thống nguồn. Các kỹ sư dữ liệu thường buộc phải làm việc với dữ liệu CSV và sau đó xây dựng việc xử lý ngoại lệ mạnh mẽ và phát hiện lỗi để đảm bảo chất lượng dữ liệu khi thu nạp. Xem Phụ lục A để có thảo luận rộng hơn về các định dạng tệp.

Shell — Shell

The shell is an interface by which you may execute commands to ingest data. The shell can be used to script workflows for virtually any software tool, and shell scripting is still used extensively in ingestion processes. A shell script might read data from a database, reserialize it into a different file format, upload it to object storage, and trigger an ingestion process in a target database. While storing data on a single instance or server is not highly scalable, many of our data sources are not particularly large, and such approaches work just fine.

Shell là một giao diện mà qua đó bạn có thể thực thi các lệnh để thu nạp dữ liệu. Shell có thể được dùng để viết kịch bản (script) cho các luồng công việc của hầu như bất kỳ công cụ phần mềm nào, và viết kịch bản shell vẫn được dùng rộng rãi trong các quy trình thu nạp. Một kịch bản shell có thể đọc dữ liệu từ một database, tuần tự hóa lại nó thành một định dạng tệp khác, tải nó lên kho lưu trữ đối tượng, và kích hoạt một quy trình thu nạp trong một database đích. Mặc dù lưu trữ dữ liệu trên một thực thể hoặc server duy nhất không có khả năng mở rộng cao, nhiều nguồn dữ liệu của chúng ta không đặc biệt lớn, và các cách tiếp cận như vậy hoạt động vừa đủ tốt.

In addition, cloud vendors generally provide robust CLI-based tools. It is possible to run complex ingestion processes simply by issuing commands to the AWS CLI. As ingestion processes grow more complicated and the SLA grows more stringent, engineers should consider moving to a proper orchestration system.

Ngoài ra, các nhà cung cấp đám mây nhìn chung cung cấp các công cụ dựa trên CLI mạnh mẽ. Có thể chạy các quy trình thu nạp phức tạp đơn giản bằng cách phát các lệnh cho AWS CLI. Khi các quy trình thu nạp trở nên phức tạp hơn và SLA trở nên nghiêm ngặt hơn, các kỹ sư nên cân nhắc chuyển sang một hệ thống điều phối thực thụ.

SSH — SSH

SSH is not an ingestion strategy but a protocol used with other ingestion strategies. We use SSH in a few ways. First, SSH can be used for file transfer with SCP, as mentioned earlier. Second, SSH tunnels are used to allow secure, isolated connections to databases. Application databases should never be directly exposed on the internet. Instead, engineers can set up a bastion host—i.e., an intermediate host instance that can connect to the database in question. This host machine is exposed on the internet, although locked down for minimal access from only specified IP addresses to specified ports. To connect to the database, a remote machine first opens an SSH tunnel connection to the bastion host, and then connects from the host machine to the database.

SSH không phải là một chiến lược thu nạp mà là một giao thức được dùng với các chiến lược thu nạp khác. Chúng ta dùng SSH theo một vài cách. Thứ nhất, SSH có thể được dùng để truyền tệp với SCP, như đã đề cập trước đó. Thứ hai, các đường hầm (tunnel) SSH được dùng để cho phép các kết nối an toàn, cô lập tới các database. Các database ứng dụng không bao giờ nên được phơi bày trực tiếp trên internet. Thay vào đó, các kỹ sư có thể thiết lập một máy chủ pháo đài (bastion host)—tức là, một thực thể máy chủ trung gian có thể kết nối tới database đang xét. Máy chủ này được phơi bày trên internet, mặc dù được khóa chặt để có quyền truy cập tối thiểu chỉ từ các địa chỉ IP chỉ định đến các cổng chỉ định. Để kết nối tới database, một máy từ xa trước tiên mở một kết nối đường hầm SSH tới bastion host, rồi kết nối từ máy chủ tới database.

SFTP and SCP — SFTP và SCP

Accessing and sending data both from secure FTP (SFTP) and secure copy (SCP) are techniques you should be familiar with, even if data engineers do not typically use these regularly (IT or security/secOps will handle this). Engineers rightfully cringe at the mention of SFTP (occasionally, we even hear instances of FTP being used in production). Regardless, SFTP is still a practical reality for many businesses. They work with partner businesses that consume or provide data using SFTP, and are unwilling to rely on other standards. To avoid data leaks, security analysis is critical in these situations. SCP is a file-exchange protocol that runs over an SSH connection related to SSH. SCP can be a secure file-transfer option if it is configured correctly.

Truy cập và gửi dữ liệu cả từ FTP an toàn (SFTP) và sao chép an toàn (SCP) là các kỹ thuật bạn nên quen thuộc, ngay cả khi các kỹ sư dữ liệu thường không dùng những cái này thường xuyên (IT hoặc bảo mật/secOps sẽ xử lý việc này). Các kỹ sư hoàn toàn có lý khi rùng mình khi nghe nhắc đến SFTP (đôi khi, chúng tôi thậm chí còn nghe các trường hợp FTP được dùng trong sản xuất). Bất kể thế nào, SFTP vẫn là một thực tế thực tiễn đối với nhiều doanh nghiệp. Họ làm việc với các doanh nghiệp đối tác tiêu thụ hoặc cung cấp dữ liệu bằng SFTP, và không sẵn lòng dựa vào các tiêu chuẩn khác. Để tránh rò rỉ dữ liệu, phân tích bảo mật là then chốt trong những tình huống này. SCP là một giao thức trao đổi tệp chạy trên một kết nối SSH liên quan đến SSH. SCP có thể là một tùy chọn truyền tệp an toàn nếu nó được cấu hình đúng cách.

Again, adding additional network **access control** (defense in depth) to enhance SCP security is highly recommended.

Một lần nữa, việc thêm kiểm soát truy cập mạng bổ sung (phòng thủ theo chiều sâu - defense in depth) để tăng cường bảo mật SCP là rất được khuyến nghị.

Webhooks — Webhook

Webhooks, as we discussed in Chapter 5, are often referred to as reverse APIs. For a typical REST data API, the data provider gives engineers API specifications that they use to write their data ingestion code. The code makes requests and receives data in responses. With a webhook (Figure 7-15), the data provider defines an API request specification, but the data provider makes API calls rather than receiving them; it's the **data consumer's** responsibility to provide an API endpoint for the provider to call. The consumer is responsible for ingesting each request and handling data aggregation, storage, and processing.

Webhook, như chúng ta đã bàn trong Chương 5, thường được gọi là các API đảo ngược (reverse API). Đối với một API dữ liệu REST điển hình, nhà cung cấp dữ liệu đưa cho các kỹ sư các đặc tả API mà họ dùng để viết mã thu nạp dữ liệu của mình. Mã thực hiện các yêu cầu và nhận dữ liệu trong các phản hồi. Với một webhook (Hình 7-15), nhà cung cấp dữ liệu định nghĩa một đặc tả yêu cầu API, nhưng nhà cung cấp dữ liệu thực hiện các lời gọi API thay vì nhận chúng; trách nhiệm của bên tiêu thụ dữ liệu là cung cấp một điểm cuối (endpoint) API để nhà cung cấp gọi. Bên tiêu thụ chịu trách nhiệm thu nạp mỗi yêu cầu và xử lý việc tổng hợp, lưu trữ và xử lý dữ liệu.

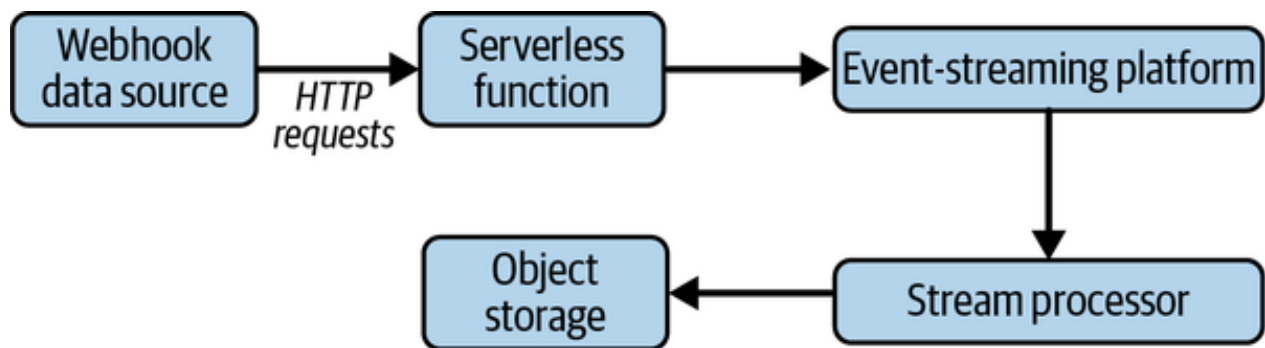


Figure 7-15. A basic webhook ingestion architecture built from cloud services Webhook-based data ingestion architectures can be brittle, difficult to maintain, and inefficient. Using appropriate off-the-shelf tools, data engineers can build more robust webhook architectures with lower maintenance and infrastructure costs. For example, a webhook pattern in AWS might use a serverless function framework (Lambda) to receive incoming events, a managed event-streaming platform to store and buffer messages (Kinesis), a stream-processing framework to handle real-time analytics (Flink), and an object store for long-term storage (S3). You'll notice that this architecture does much more than simply ingest the data. This underscores ingestion's entanglement with the other stages of the data engineering lifecycle; it is often impossible to define your ingestion architecture without making decisions about storage and processing.

Hình 7-15. Một kiến trúc thu nạp webhook cơ bản được xây dựng từ các dịch vụ đám mây. Các kiến trúc thu nạp dữ liệu dựa trên webhook có thể giòn (brittle), khó bảo trì, và không hiệu quả. Dùng các công cụ có sẵn (off-the-shelf) phù hợp, các kỹ sư dữ liệu có thể xây các kiến trúc webhook mạnh mẽ hơn với chi phí bảo trì và hạ tầng thấp hơn. Ví dụ, một mẫu hình webhook trong AWS có thể dùng một framework hàm serverless (Lambda) để nhận các sự kiện đến, một nền tảng event-streaming được quản lý để lưu trữ và đệm các tin nhắn (Kinesis), một framework xử lý stream để xử lý phân tích thời gian thực (Flink), và một kho đối tượng để lưu trữ dài hạn (S3). Bạn sẽ nhận thấy rằng kiến trúc này làm nhiều hơn nhiều so với chỉ đơn giản thu nạp dữ liệu. Điều này nhấn mạnh sự đan xen của thu nạp với các giai đoạn khác của vòng đời kỹ thuật dữ liệu; thường không thể định nghĩa kiến trúc thu nạp của bạn mà không đưa ra các quyết định về lưu trữ và xử lý.

Web Interface — Giao diện Web

Web interfaces for data access remain a practical reality for data engineers. We frequently run into situations where not all data and functionality in a SaaS platform is exposed through automated interfaces such as APIs and file drops. Instead, someone must manually access a web interface, generate a report, and download a file to a local machine. This has obvious drawbacks, such as people forgetting to run the report or having their laptop die. Where possible, choose tools and workflows that allow for automated access to data.

Các giao diện web để truy cập dữ liệu vẫn là một thực tế thực tiễn đối với các kỹ sư dữ liệu. Chúng tôi thường xuyên gặp các tình huống nơi không phải tất cả dữ liệu và chức năng trong một nền tảng SaaS được phơi bày qua các giao diện tự động như các API và các lần thả tệp (file drop). Thay vào đó, ai đó phải truy cập một giao diện web một cách thủ công, tạo một báo cáo, và tải một tệp xuống một máy cục bộ. Điều này có những bất lợi rõ ràng, chẳng hạn như người ta quên chạy báo cáo hoặc laptop của họ hết pin. Bất cứ khi nào có thể, hãy chọn các công cụ và luồng công việc cho phép truy cập tự động tới dữ liệu.

Web Scraping — Web Scraping

Web scraping automatically extracts data from web pages, often by combing the web page's various HTML elements. You might scrape ecommerce sites to extract product pricing information or scrape multiple news sites for your news aggregator. Web scraping is widespread, and you may encounter it as a data engineer. It's also a murky area where ethical and legal lines are blurry. Here is some top-level advice to be aware of before undertaking any web-scraping project. First, ask yourself if you should be web scraping or if data is available from a third party. If your decision is to web scrape, be a good citizen. Don't inadvertently create a denial-of-service (DoS) attack, and don't get your IP address blocked. Understand how much traffic you generate and pace your web-crawling activities appropriately. Just because you can spin up thousands of simultaneous Lambda functions to scrape doesn't mean you should; excessive web scraping could lead to the disabling of your AWS account. Second, be aware of the legal implications of your activities. Again, generating DoS attacks can entail legal consequences. Actions that violate terms of service may cause headaches for your employer or you personally.

Web scraping tự động trích xuất dữ liệu từ các trang web, thường bằng cách rà soát các phần tử HTML khác nhau của trang web. Bạn có thể scrape các trang thương mại điện tử để trích xuất thông tin giá sản phẩm hoặc scrape nhiều trang tin tức cho bộ tổng hợp tin tức của bạn. Web scraping phổ biến rộng rãi, và bạn có thể gặp nó với tư cách một kỹ sư dữ liệu. Nó cũng là một lĩnh vực mờ ám nơi các ranh giới đạo đức và pháp lý nhập nhằng. Đây là một số lời khuyên ở cấp cao để ý thức trước khi tiến hành bất kỳ dự án web-scraping nào. Thứ nhất, hãy tự hỏi liệu bạn có nên web scraping hay dữ liệu có sẵn từ một bên thứ ba. Nếu quyết định của bạn là web scrape, hãy là một công dân tốt. Đừng vô tình tạo ra một cuộc tấn công từ chối dịch vụ (denial-of-service, DoS), và đừng để địa chỉ IP của bạn bị chặn. Hãy hiểu bạn tạo ra bao nhiêu lưu lượng và điều tiết các hoạt động thu thập web (web-crawling) của bạn một cách phù hợp. Chỉ vì bạn có thể khởi chạy hàng nghìn hàm Lambda đồng thời để scrape không có nghĩa là bạn nên làm vậy; web scraping quá mức có thể dẫn đến việc tài khoản AWS của bạn bị vô hiệu hóa. Thứ hai, hãy ý thức về các hệ quả pháp lý của các hoạt động của bạn. Một lần nữa, tạo ra các cuộc tấn công DoS có thể kéo theo các hậu quả pháp lý. Các hành động vi phạm điều khoản dịch vụ có thể gây đau đầu cho chủ lao động của bạn hoặc cho cá nhân bạn.

Third, web pages constantly change their HTML element structure, making it tricky to keep your web

scraper updated. Ask yourself, is the headache of maintaining these systems worth the effort? Web scraping has interesting implications for the data engineering lifecycle processing stage; engineers should think about various factors at the beginning of a web-scraping project. What do you intend to do with the data? Are you just pulling required fields from the scraped HTML by using Python code and then writing these values to a database? Do you intend to maintain the complete HTML code of the scraped websites and process this data using a framework like Spark? These decisions may lead to very different architectures downstream of ingestion.

Thứ ba, các trang web liên tục thay đổi cấu trúc phần tử HTML của chúng, khiến việc giữ cho web scraper của bạn được cập nhật trở nên khó khăn. Hãy tự hỏi, cơn đau đầu của việc bảo trì các hệ thống này có đáng với công sức không? Web scraping có những hệ quả thú vị đối với giai đoạn xử lý của vòng đời kỹ thuật dữ liệu; các kỹ sư nên suy nghĩ về nhiều yếu tố khác nhau ngay từ đầu của một dự án web-scraping. Bạn định làm gì với dữ liệu? Bạn chỉ đang kéo các trường cần thiết từ HTML đã scrape bằng cách dùng mã Python rồi ghi các giá trị này vào một database? Bạn có định duy trì toàn bộ mã HTML của các website đã scrape và xử lý dữ liệu này bằng một framework như Spark không? Các quyết định này có thể dẫn đến các kiến trúc rất khác nhau ở hạ nguồn của thu nạp.

Transfer Appliances for Data Migration — Các Thiết bị Truyền tải cho Di trú Dữ liệu

For massive data (100 TB or more), transferring data directly over the internet may be a slow and costly process. At this scale, the fastest, most efficient way to move data is not over the wire but by truck. Cloud vendors offer the ability to send your data via a physical “box of hard drives.” Simply order a storage device, called a transfer appliance, load your data from your servers, and then send it back to the cloud vendor, which will upload your data. The suggestion is to consider using a transfer appliance if your data size hovers around 100 TB. On the extreme end, AWS even offers Snowmobile, a transfer appliance sent to you in a semitrailer! Snowmobile is intended to lift and shift an entire data center, in which data sizes are in the petabytes or greater. Transfer appliances are handy for creating hybrid-cloud or multicloud setups. For example, Amazon’s data transfer appliance (AWS Snowball) supports import and export. To migrate into a second cloud, users can export their data into a Snowball device, and then import it into a second transfer appliance to move data into GCP or Azure. This might sound awkward, but even when it’s feasible to push data over the internet between clouds, data egress fees make this a costly proposition. Physical transfer appliances are a cheaper alternative when the data volumes are significant. Remember that transfer appliances and data migration services are one-time data ingestion events and are not suggested for ongoing workloads. Suppose you have workloads requiring constant data movement in either a hybrid or multicloud scenario. In that case, your data sizes are presumably batching or streaming much smaller data sizes on an ongoing basis.

Đối với dữ liệu khổng lồ (100 TB hoặc hơn), việc truyền dữ liệu trực tiếp qua internet có thể là một quá trình chậm và tốn kém. Ở quy mô này, cách nhanh nhất, hiệu quả nhất để di chuyển dữ liệu không phải qua đường truyền mà bằng xe tải. Các nhà cung cấp đám mây cung cấp khả năng gửi dữ liệu của bạn qua một “hộp ổ cứng” vật lý. Đơn giản đặt mua một thiết bị lưu trữ, gọi là một thiết bị truyền tải (transfer appliance), nạp dữ liệu của bạn từ các server của bạn, rồi gửi nó trở lại cho nhà cung cấp đám mây, nơi sẽ tải dữ liệu của bạn lên. Gợi ý là cân nhắc dùng một thiết bị truyền tải nếu kích thước dữ liệu của bạn dao động quanh 100 TB. Ở đầu cực đoan, AWS thậm chí cung cấp Snowmobile, một thiết bị truyền tải được gửi đến bạn trong một xe đầu kéo bán moóc! Snowmobile được dùng để nâng và dịch chuyển (lift and shift) toàn bộ một trung tâm dữ liệu, trong đó các kích

thước dữ liệu ở mức petabyte hoặc lớn hơn. Các thiết bị truyền tải tiện dụng để tạo các thiết lập đám mây lai (hybrid-cloud) hoặc đa đám mây (multicloud). Ví dụ, thiết bị truyền tải dữ liệu của Amazon (AWS Snowball) hỗ trợ nhập và xuất. Để di trú vào một đám mây thứ hai, người dùng có thể xuất dữ liệu của họ vào một thiết bị Snowball, rồi nhập nó vào một thiết bị truyền tải thứ hai để di chuyển dữ liệu vào GCP hoặc Azure. Điều này có thể nghe vụng về, nhưng ngay cả khi khả thi đẩy dữ liệu qua internet giữa các đám mây, các khoản phí dữ liệu đi ra khiến đây trở thành một đề xuất tốn kém. Các thiết bị truyền tải vật lý là một giải pháp thay thế rẻ hơn khi các khối lượng dữ liệu đáng kể. Hãy nhớ rằng các thiết bị truyền tải và dịch vụ di trú dữ liệu là các sự kiện thu nạp dữ liệu một lần và không được đề xuất cho các khối lượng công việc liên tục. Giả sử bạn có các khối lượng công việc đòi hỏi di chuyển dữ liệu liên tục trong một kịch bản lai hoặc đa đám mây. Trong trường hợp đó, các kích thước dữ liệu của bạn có lẽ đang theo lô hoặc streaming các kích thước dữ liệu nhỏ hơn nhiều trên cơ sở liên tục.

Data Sharing — Chia sẻ Dữ liệu

Data sharing is growing as a popular option for consuming data (see Chapters 5 and 6 .) Data providers will offer datasets to third-party subscribers, either for free or at a cost. These datasets are often shared in a read-only fashion, meaning you can integrate these datasets with your own data (and other third-party datasets), but you do not own the shared dataset. In the strict sense, this isn't ingestion, where you get physical possession of the dataset. If the data provider decides to remove your access to a dataset, you'll no longer have access to it. Many cloud platforms offer data sharing, allowing you to share your data and consume data from various providers. Some of these platforms also provide data marketplaces where companies and organizations can offer their data for sale.

Chia sẻ dữ liệu (data sharing) đang tăng trưởng như một lựa chọn phổ biến để tiêu thụ dữ liệu (xem Chương 5 và 6). Các nhà cung cấp dữ liệu sẽ cung cấp các tập dữ liệu cho các bên đăng ký thứ ba, hoặc miễn phí hoặc có chi phí. Các tập dữ liệu này thường được chia sẻ theo kiểu chỉ-đọc (read-only), nghĩa là bạn có thể tích hợp các tập dữ liệu này với dữ liệu của riêng bạn (và các tập dữ liệu bên thứ ba khác), nhưng bạn không sở hữu tập dữ liệu được chia sẻ. Theo nghĩa nghiêm ngặt, đây không phải là thu nạp, nơi bạn có quyền sở hữu vật lý tập dữ liệu. Nếu nhà cung cấp dữ liệu quyết định gỡ bỏ quyền truy cập của bạn tới một tập dữ liệu, bạn sẽ không còn quyền truy cập nó nữa. Nhiều nền tảng đám mây cung cấp chia sẻ dữ liệu, cho phép bạn chia sẻ dữ liệu của mình và tiêu thụ dữ liệu từ nhiều nhà cung cấp khác nhau. Một số nền tảng này cũng cung cấp các chợ dữ liệu (data marketplace) nơi các công ty và tổ chức có thể chào bán dữ liệu của họ.

Whom You'll Work With — Những người bạn sẽ làm việc cùng

Data ingestion sits at several organizational boundaries. In developing and managing data ingestion pipelines, data engineers will work with both people and systems sitting upstream (data producers) and downstream (data consumers).

Thu nạp dữ liệu nằm tại một số ranh giới của tổ chức. Trong việc phát triển và quản lý các đường ống thu nạp dữ liệu, các kỹ sư dữ liệu sẽ làm việc với cả con người và hệ thống nằm ở thượng nguồn (các bên sản xuất dữ liệu) và hạ nguồn (các bên tiêu thụ dữ liệu).

Upstream Stakeholders — Các Bên liên quan Thượng nguồn

A significant disconnect often exists between those responsible for generating data —typically, software engineers—and the data engineers who will prepare this data for analytics and data science. Software engineers and data engineers usually sit in separate organizational silos; if they think about data engineers, they typically see them simply as downstream consumers of the data exhaust from their application, not as stakeholders. We see this current state of affairs as a problem and a significant opportunity. Data engineers can improve the quality of their data by inviting software engineers to be stakeholders in data engineering outcomes. The vast majority of software engineers are well aware of the value of analytics and data science but don't necessarily have aligned incentives to contribute to data engineering efforts directly. Simply improving communication is a significant first step. Often software engineers have already identified potentially valuable data for downstream consumption. Opening a communication channel encourages software engineers to get data into shape for consumers and communicate about data changes to prevent pipeline regressions. Beyond communication, data engineers can highlight the contributions of software engineers to team members, executives, and especially product managers. Involving product managers in the outcome and treating downstream data processed as part of a product encourages them to allocate scarce software development to collaboration with data engineers. Ideally, software engineers can work partially as extensions of the data engineering team; this allows them to collaborate on various projects, such as creating an **event-driven architecture** to enable real-time analytics.

Một sự ngắt kết nối đáng kể thường tồn tại giữa những người chịu trách nhiệm sinh dữ liệu—thường là các kỹ sư phần mềm—và các kỹ sư dữ liệu sẽ chuẩn bị dữ liệu này cho phân tích và khoa học dữ liệu. Các kỹ sư phần mềm và các kỹ sư dữ liệu thường ngồi trong các silo tổ chức riêng biệt; nếu họ nghĩ về các kỹ sư dữ liệu, họ thường chỉ xem họ đơn giản là các bên tiêu thụ hạ nguồn của khí thải dữ liệu (data exhaust) từ ứng dụng của họ, chứ không phải là các bên liên quan. Chúng tôi xem tình trạng hiện tại này như một vấn đề và một cơ hội đáng kể. Các kỹ sư dữ liệu có thể cải thiện chất lượng dữ liệu của họ bằng cách mời các kỹ sư phần mềm trở thành các bên liên quan trong các kết quả của kỹ thuật dữ liệu. Đại đa số các kỹ sư phần mềm nhận thức rõ giá trị của phân tích và khoa học dữ liệu nhưng không nhất thiết có các động lực được căn chỉnh để đóng góp trực tiếp vào các nỗ lực kỹ thuật dữ liệu. Đơn giản cải thiện giao tiếp là một bước đầu tiên đáng kể. Thường thì các kỹ sư phần mềm đã xác định được dữ liệu có giá trị tiềm năng cho tiêu thụ hạ nguồn. Mở một kênh giao tiếp khuyến khích các kỹ sư phần mềm đưa dữ liệu vào trạng thái phù hợp cho các bên tiêu thụ và giao tiếp về các thay đổi dữ liệu để ngăn các thoái lui (regression) của đường ống. Vượt ra ngoài giao tiếp, các kỹ sư dữ liệu có thể làm nổi bật các đóng góp của các kỹ sư phần mềm cho các thành viên đội, các nhà điều hành, và đặc biệt là các quản lý sản phẩm (product manager). Việc lôi kéo các quản lý sản phẩm vào kết quả và xem dữ liệu hạ nguồn được xử lý như một phần của một sản phẩm khuyến khích họ phân bổ nguồn lực phát triển phần mềm khan hiếm cho việc cộng tác với các kỹ sư dữ liệu. Lý tưởng nhất, các kỹ sư phần mềm có thể làm việc một phần như các phần mở rộng của đội kỹ thuật dữ liệu; điều này cho phép họ cộng tác trên nhiều dự án khác nhau, chẳng hạn như tạo một kiến trúc hướng sự kiện để cho phép phân tích thời gian thực.

Downstream Stakeholders — Các Bên liên quan Hạ nguồn

Who is the ultimate customer for data ingestion? Data engineers focus on data practitioners and technology leaders such as data scientists, analysts, and chief technical officers. They would do well also to remember their broader circle of business stakeholders such as marketing directors, vice presidents over the supply chain, and CEOs.

Ai là khách hàng cuối cùng cho thu nạp dữ liệu? Các kỹ sư dữ liệu tập trung vào các nhà thực hành dữ liệu và các lãnh đạo công nghệ như các nhà khoa học dữ liệu, các nhà phân tích, và các giám đốc kỹ thuật (chief technical officer). Họ cũng sẽ làm tốt khi nhớ đến vòng tròn rộng hơn các bên liên quan kinh doanh của họ như các giám đốc tiếp thị, các phó chủ tịch phụ trách chuỗi cung ứng, và các CEO.

Too often, we see data engineers pursuing sophisticated projects (e.g., real-time streaming buses or complex data systems) while digital marketing managers next door are left downloading Google Ads reports manually. View data engineering as a business, and recognize who your customers are. Often basic automation of ingestion processes has significant value, especially for departments like marketing that control massive budgets and sit at the heart of revenue for the business. Basic ingestion work may seem tedious, but delivering value to these core parts of the company will open up more budget and more exciting long-term data engineering opportunities. Data engineers can also invite more executive participation in this collaborative process. For a good reason, data-driven culture is quite fashionable in business leadership circles. Still, it is up to data engineers and other data practitioners to provide executives with guidance on the best structure for a data-driven business. This means communicating the value of lowering barriers between data producers and data engineers while supporting executives in breaking down silos and setting up incentives to lead to a more unified data-driven culture. Once again, communication is the watchword. Honest communication early and often with stakeholders will go a long way to ensure that your data ingestion adds value.

Quá thường xuyên, chúng tôi thấy các kỹ sư dữ liệu theo đuổi các dự án tinh vi (ví dụ, các bus streaming thời gian thực hoặc các hệ thống dữ liệu phức tạp) trong khi các quản lý tiếp thị số ngay bên cạnh thì phải tải xuống các báo cáo Google Ads một cách thủ công. Hãy xem kỹ thuật dữ liệu như một doanh nghiệp, và nhận ra ai là khách hàng của bạn. Thường thì việc tự động hóa cơ bản các quy trình thu nạp có giá trị đáng kể, đặc biệt cho các bộ phận như tiếp thị vốn kiểm soát các ngân sách khổng lồ và nằm ở trung tâm của doanh thu cho doanh nghiệp. Công việc thu nạp cơ bản có thể trông tẻ nhạt, nhưng việc mang lại giá trị cho các phần cốt lõi này của công ty sẽ mở ra nhiều ngân sách hơn và nhiều cơ hội kỹ thuật dữ liệu dài hạn thú vị hơn. Các kỹ sư dữ liệu cũng có thể mời sự tham gia nhiều hơn của các nhà điều hành vào quá trình cộng tác này. Vì một lý do chính đáng, văn hóa hướng dữ liệu (data-driven) khá thời thượng trong các giới lãnh đạo doanh nghiệp. Tuy nhiên, các kỹ sư dữ liệu và các nhà thực hành dữ liệu khác có trách nhiệm cung cấp cho các nhà điều hành hướng dẫn về cấu trúc tốt nhất cho một doanh nghiệp hướng dữ liệu. Điều này nghĩa là giao tiếp giá trị của việc hạ thấp các rào cản giữa các bên sản xuất dữ liệu và các kỹ sư dữ liệu trong khi hỗ trợ các nhà điều hành phá bỏ các silo và thiết lập các động lực để dẫn đến một văn hóa hướng dữ liệu thống nhất hơn. Một lần nữa, giao tiếp là khẩu hiệu. Giao tiếp trung thực sớm và thường xuyên với các bên liên quan sẽ đi một chặng đường dài để đảm bảo rằng việc thu nạp dữ liệu của bạn tạo thêm giá trị.

Undercurrents — Các dòng chảy ngầm

Virtually all the undercurrents touch the ingestion phase, but we'll emphasize the most salient ones here.

Hầu như tất cả các dòng chảy ngầm (undercurrents) đều chạm đến giai đoạn thu nạp, nhưng chúng tôi sẽ nhấn mạnh những cái nổi bật nhất ở đây.

Security — An ninh

Moving data introduces security vulnerabilities because you have to transfer data between locations. The last thing you want is to capture or compromise the data while moving.

Di chuyển dữ liệu đưa vào các lỗ hổng bảo mật vì bạn phải chuyển dữ liệu giữa các vị trí. Điều cuối cùng bạn muốn là dữ liệu bị thu giữ hoặc xâm phạm trong khi di chuyển.

Consider where the data lives and where it is going. Data that needs to move within your VPC should use secure endpoints and never leave the confines of the VPC. Use a VPN or a dedicated private connection if you need to send data between the cloud and an on-premises network. This might cost money, but the security is a good investment. If your data traverses the public internet, ensure that the transmission is encrypted. It is always a good practice to encrypt data over the wire.

Hãy cân nhắc nơi dữ liệu cư trú và nơi nó đang đi đến. Dữ liệu cần di chuyển bên trong VPC của bạn nên dùng các điểm cuối an toàn và không bao giờ rời khỏi phạm vi của VPC. Dùng một VPN hoặc một kết nối riêng tư chuyên dụng nếu bạn cần gửi dữ liệu giữa đám mây và một mạng tại chỗ. Điều này có thể tốn tiền, nhưng bảo mật là một khoản đầu tư tốt. Nếu dữ liệu của bạn đi qua internet công cộng, hãy đảm bảo rằng việc truyền được mã hóa. Luôn là một thực hành tốt khi mã hóa dữ liệu trên đường truyền.

Data Management — Quản lý Dữ liệu

Naturally, data management begins at data ingestion. This is the starting point for lineage and data cataloging; from this point on, data engineers need to think about **master data management**, ethics, **privacy**, and compliance.

Đương nhiên, quản lý dữ liệu (data management) bắt đầu tại thu nạp dữ liệu. Đây là điểm khởi đầu cho truy vết nguồn gốc (lineage) và lập danh mục dữ liệu (data cataloging); từ điểm này trở đi, các kỹ sư dữ liệu cần nghĩ về quản lý dữ liệu chủ (master data management), đạo đức, quyền riêng tư, và tuân thủ (compliance).

Schema changes — Các thay đổi lược đồ

Schema changes (such as adding, changing, or removing columns in a database table) remain, from our perspective, an unsettled issue in data management. The traditional approach is a careful command-and-control review process. Working with clients at large enterprises, we have been quoted lead times of six months for the addition of a single field. This is an unacceptable impediment to agility. On the opposite end of the spectrum, any schema change in the source triggers target tables to be re-created with the new schema. This solves schema problems at the ingestion stage but can still break downstream pipelines and destination storage systems. One possible solution, which we, the authors, have meditated on for a while, is an approach pioneered by Git version control. When Linus Torvalds was developing Git, many of his choices were inspired by the limitations of Concurrent Versions System (CVS). CVS is completely centralized; it supports only one current official version of the code, stored on a central project server. To make Git a truly **distributed system**, Torvalds used the notion of a tree; each developer could maintain their processed branch of the code and then merge to or from other branches. A few years ago, such an approach to data was unthinkable. On-premises MPP systems are typically operated at close to maximum storage capacity. However, storage is cheap in big data and cloud data warehouse environments. One may quite easily maintain multiple versions of a table with different schemas and even different

upstream transformations. Teams can support various “development” versions of a table by using orchestration tools such as Airflow; schema changes, upstream transformation, and code changes can appear in development tables before official changes to the main table.

Các thay đổi lược đồ (chẳng hạn như thêm, thay đổi, hoặc gỡ bỏ các cột trong một bảng database) vẫn, theo quan điểm của chúng tôi, là một vấn đề chưa được giải quyết trong quản lý dữ liệu. Cách tiếp cận truyền thống là một quy trình rà soát chỉ-huy-và-kiểm-soát (command-and-control) cẩn thận. Khi làm việc với các khách hàng tại các doanh nghiệp lớn, chúng tôi đã được báo các thời gian chờ (lead time) sáu tháng cho việc thêm một trường duy nhất. Đây là một cản trở không thể chấp nhận được đối với sự nhanh nhẹn (agility). Ở đầu đời của phổ, bất kỳ thay đổi lược đồ nào trong nguồn đều kích hoạt các bảng đích được tạo lại với lược đồ mới. Điều này giải quyết các vấn đề lược đồ tại giai đoạn thu nạp nhưng vẫn có thể phá vỡ các đường ống hạ nguồn và các hệ thống lưu trữ đích. Một giải pháp khả thi, mà chúng tôi, các tác giả, đã suy ngẫm một thời gian, là một cách tiếp cận được tiên phong bởi kiểm soát phiên bản Git. Khi Linus Torvalds đang phát triển Git, nhiều lựa chọn của ông được truyền cảm hứng từ các hạn chế của Concurrent Versions System (CVS). CVS hoàn toàn tập trung; nó chỉ hỗ trợ một phiên bản chính thức hiện tại duy nhất của mã, được lưu trên một server dự án trung tâm. Để biến Git thành một hệ thống thực sự phân tán, Torvalds đã dùng khái niệm về một cây (tree); mỗi nhà phát triển có thể duy trì nhánh (branch) mã đã xử lý của riêng họ rồi merge vào hoặc từ các nhánh khác. Vài năm trước, một cách tiếp cận như vậy đối với dữ liệu là không thể tưởng tượng được. Các hệ thống MPP tại chỗ thường được vận hành ở gần dung lượng lưu trữ tối đa. Tuy nhiên, lưu trữ thì rẻ trong các môi trường dữ liệu lớn (big data) và kho dữ liệu đám mây. Người ta khá dễ dàng duy trì nhiều phiên bản của một bảng với các lược đồ khác nhau và thậm chí các biến đổi thượng nguồn khác nhau. Các đội có thể hỗ trợ nhiều phiên bản “phát triển” (development) khác nhau của một bảng bằng cách dùng các công cụ điều phối như Airflow; các thay đổi lược đồ, biến đổi thượng nguồn, và các thay đổi mã có thể xuất hiện trong các bảng phát triển trước các thay đổi chính thức đối với bảng chính.

Data ethics, privacy, and compliance — Đạo đức dữ liệu, quyền riêng tư, và tuân thủ

Clients often ask for our advice on encrypting sensitive data in databases, which generally leads us to ask a fundamental question: do you need the sensitive data you’re trying to encrypt? As it turns out, this question often gets overlooked when creating requirements and solving problems. Data engineers should always train themselves to ask this question when setting up ingestion pipelines. They will inevitably encounter sensitive data; the natural tendency is to ingest it and forward it to the next step in the pipeline. But if this data is not needed, why collect it at all? Why not simply drop sensitive fields before data is stored? Data cannot leak if it is never collected. Where it is truly necessary to keep track of sensitive identities, it is common practice to apply tokenization to anonymize identities in model training and analytics. But engineers should look at where this tokenization is used. If possible, hash data at ingestion time. Data engineers cannot avoid working with highly sensitive data in some cases. Some analytics systems must present identifiable, sensitive information. Engineers must act under the highest ethical standards whenever they handle sensitive data. In addition, they can put in place a variety of practices to reduce the direct handling of sensitive data. Aim as much as possible for touchless production where sensitive data is involved. This means that engineers develop and test code on simulated or cleansed data in development and staging environments but automated code deployments to production. Touchless

production is an ideal that engineers should strive for, but situations inevitably arise that cannot be fully solved in development and staging environments. Some bugs may not be reproducible without looking at the live data that is triggering a regression. For these cases, put a broken-glass process in place: require at least two people to approve access to sensitive data in the production environment. This access should be tightly scoped to a particular issue and come with an expiration date. Our last bit of advice on sensitive data: be wary of naive technological solutions to human problems. Both **encryption** and tokenization are often treated like privacy magic bullets. Most cloud-based storage systems and nearly all databases encrypt data at rest and in motion by default. Generally, we don't see encryption problems but data access problems. Is the solution to apply an extra layer of encryption to a single field or to control access to that field? After all, one must still tightly manage access to the encryption key. Legitimate use cases exist for single-field encryption, but watch out for ritualistic encryption. On the tokenization front, use common sense and assess data access scenarios. If someone had the email of one of your customers, could they easily hash the email and find the customer in your data? Thoughtlessly hashing data without salting and other strategies may not protect privacy as well as you think.

Các khách hàng thường hỏi xin lời khuyên của chúng tôi về việc mã hóa dữ liệu nhạy cảm trong các database, điều thường dẫn chúng tôi đến việc hỏi một câu hỏi căn bản: bạn có cần dữ liệu nhạy cảm mà bạn đang cố mã hóa không? Hóa ra, câu hỏi này thường bị bỏ qua khi tạo các yêu cầu và giải quyết các vấn đề. Các kỹ sư dữ liệu nên luôn rèn luyện bản thân hỏi câu hỏi này khi thiết lập các đường ống thu nạp. Họ chắc chắn sẽ gặp dữ liệu nhạy cảm; khuynh hướng tự nhiên là thu nạp nó và chuyển tiếp nó đến bước tiếp theo trong đường ống. Nhưng nếu dữ liệu này không cần thiết, tại sao lại thu thập nó? Tại sao không đơn giản loại bỏ các trường nhạy cảm trước khi dữ liệu được lưu trữ? Dữ liệu không thể rò rỉ nếu nó không bao giờ được thu thập. Nơi thực sự cần thiết phải theo dõi các danh tính nhạy cảm, thực hành phổ biến là áp dụng tokenization để ẩn danh các danh tính trong huấn luyện mô hình và phân tích. Nhưng các kỹ sư nên xem xét nơi tokenization này được dùng. Nếu có thể, hãy băm (hash) dữ liệu tại thời điểm thu nạp. Các kỹ sư dữ liệu không thể tránh làm việc với dữ liệu cực kỳ nhạy cảm trong một số trường hợp. Một số hệ thống phân tích phải trình bày thông tin nhận dạng được, nhạy cảm. Các kỹ sư phải hành động theo các tiêu chuẩn đạo đức cao nhất bất cứ khi nào họ xử lý dữ liệu nhạy cảm. Ngoài ra, họ có thể đặt vào chỗ nhiều thực hành khác nhau để giảm việc xử lý trực tiếp dữ liệu nhạy cảm. Hãy nhắm đến càng nhiều càng tốt việc sản xuất không chạm (touchless production) nơi dữ liệu nhạy cảm liên quan. Điều này nghĩa là các kỹ sư phát triển và kiểm thử mã trên dữ liệu mô phỏng hoặc đã được làm sạch trong các môi trường phát triển và staging nhưng triển khai mã tự động tới sản xuất. Sản xuất không chạm là một lý tưởng mà các kỹ sư nên phấn đấu hướng tới, nhưng các tình huống chắc chắn sẽ nảy sinh mà không thể được giải quyết hoàn toàn trong các môi trường phát triển và staging. Một số lỗi có thể không tái tạo được nếu không nhìn vào dữ liệu trực tiếp đang kích hoạt một thoát lui. Đối với các trường hợp này, hãy đặt vào chỗ một quy trình phá-kính (broken-glass): yêu cầu ít nhất hai người phê duyệt quyền truy cập tới dữ liệu nhạy cảm trong môi trường sản xuất. Quyền truy cập này nên được giới hạn phạm vi chặt chẽ vào một vấn đề cụ thể và đi kèm một ngày hết hạn. Lời khuyên cuối cùng của chúng tôi về dữ liệu nhạy cảm: hãy cảnh giác với các giải pháp công nghệ gây thơ cho các vấn đề con người. Cả mã hóa lẫn tokenization thường được đối xử như những viên đạn bạc về quyền riêng tư. Hầu hết các hệ thống lưu trữ dựa trên đám mây và gần như tất cả các database mã hóa dữ liệu ở trạng thái nghỉ (at rest) và đang di chuyển (in motion) theo mặc định. Nhìn chung, chúng tôi không thấy các vấn đề về mã hóa mà là các vấn đề về truy cập dữ liệu. Giải pháp là áp dụng một lớp mã hóa bổ sung cho một trường duy nhất hay là kiểm soát quyền truy cập tới trường đó? Suy cho cùng, người ta vẫn phải quản lý chặt chẽ quyền truy cập tới khóa mã hóa. Các trường hợp sử dụng hợp pháp tồn tại cho việc mã hóa một-trường, nhưng hãy coi chừng

việc mã hóa mang tính nghi thức (ritualistic). Về mặt tokenization, hãy dùng lẽ thường và đánh giá các kịch bản truy cập dữ liệu. Nếu ai đó có email của một trong các khách hàng của bạn, liệu họ có thể dễ dàng bấm email và tìm thấy khách hàng trong dữ liệu của bạn không? Bấm dữ liệu một cách thiếu suy nghĩ mà không thêm muối (salting) và các chiến lược khác có thể không bảo vệ quyền riêng tư tốt như bạn nghĩ.

DataOps — DataOps

Reliable data pipelines are the cornerstone of the data engineering lifecycle. When they fail, all downstream dependencies come to a screeching halt. Data warehouses and data lakes aren't replenished with fresh data, and data scientists and analysts can't effectively do their jobs; the business is forced to fly blind.

Các đường ống dữ liệu đáng tin cậy là nền tảng của vòng đời kỹ thuật dữ liệu. Khi chúng thất bại, tất cả các phụ thuộc hạ nguồn dừng lại đột ngột. Các kho dữ liệu và hồ dữ liệu không được bổ sung dữ liệu mới, và các nhà khoa học dữ liệu cùng các nhà phân tích không thể làm công việc của họ một cách hiệu quả; doanh nghiệp buộc phải bay mù.

Ensuring that your data pipelines are properly monitored is a crucial step toward reliability and effective incident response. If there's one stage in the data engineering lifecycle where monitoring is critical, it's in the ingestion stage. Weak or nonexistent monitoring means the pipelines may or may not be working. Referring back to our earlier discussion on time, be sure to track the various aspects of time—event creation, ingestion, process, and processing times. Your data pipelines should predictably process data in batches or streams. We've seen countless examples of reports and ML models generated from stale data. In one extreme case, an ingestion pipeline failure wasn't detected for six months. (One might question the concrete utility of the data in this instance, but that's another matter.) This was very much avoidable through proper monitoring. What should you monitor? Uptime, latency, and data volumes processed are good places to start. If an ingestion job fails, how will you respond? In general, build monitoring into your pipelines from the beginning rather than waiting for deployment. Monitoring is key, and knowledge of the behavior of the upstream systems you depend on and how they generate data. You should be aware of the number of events generated per time interval you're concerned with (events/minute, events/second, and so on) and the average size of each event. Your data pipeline should handle both the frequency and size of the events you're ingesting. This also applies to third-party services. In the case of these services, what you've gained in terms of lean operational efficiencies (reduced headcount) is replaced by systems you depend on being outside of your control. If you're using a third-party service (cloud, data integration service, etc.), how will you be alerted if there's an outage? What's your response plan if a service you depend on suddenly goes offline? Sadly, no universal response plan exists for third-party failures. If you can fail over to other servers, preferably in another zone or region, definitely set this up.

Đảm bảo rằng các đường ống dữ liệu của bạn được giám sát đúng cách là một bước then chốt hướng tới độ tin cậy và phản ứng sự cố hiệu quả. Nếu có một giai đoạn trong vòng đời kỹ thuật dữ liệu nơi giám sát là then chốt, thì đó là ở giai đoạn thu nạp. Giám sát yếu hoặc không tồn tại nghĩa là các đường ống có thể đang hoạt động hoặc không. Trở lại thảo luận trước đó của chúng ta về thời gian, hãy chắc chắn theo dõi nhiều khía cạnh khác nhau của thời gian—thời điểm tạo sự kiện, thời điểm thu nạp, thời điểm xử lý, và thời gian xử lý. Các đường ống dữ liệu của bạn nên xử lý dữ liệu một cách dự đoán được theo các lô hoặc stream. Chúng tôi đã thấy vô số ví dụ về các báo cáo và mô hình học máy được sinh ra từ dữ liệu cũ kỹ. Trong một trường hợp cực đoan, một sự thất bại của đường ống thu nạp không được phát hiện trong sáu tháng. (Người ta có thể đặt câu hỏi về tính hữu dụng cụ thể của dữ liệu trong trường hợp này, nhưng đó là chuyện khác.) Điều này hoàn toàn có

thể tránh được thông qua giám sát phù hợp. Bạn nên giám sát cái gì? Thời gian hoạt động, độ trễ, và khối lượng dữ liệu được xử lý là những nơi tốt để bắt đầu. Nếu một công việc thu nạp thất bại, bạn sẽ phản ứng ra sao? Nhìn chung, hãy xây giám sát vào các đường ống của bạn ngay từ đầu thay vì chờ đến lúc triển khai. Giám sát là then chốt, cùng với kiến thức về hành vi của các hệ thống thượng nguồn mà bạn phụ thuộc vào và cách chúng sinh dữ liệu. Bạn nên ý thức về số sự kiện được sinh ra mỗi khoảng thời gian mà bạn quan tâm (sự kiện/phút, sự kiện/giây, v.v.) và kích thước trung bình của mỗi sự kiện. Đường ống dữ liệu của bạn nên xử lý cả tần suất lẫn kích thước của các sự kiện bạn đang thu nạp. Điều này cũng áp dụng cho các dịch vụ bên thứ ba. Trong trường hợp các dịch vụ này, những gì bạn đạt được về mặt hiệu quả vận hành tinh gọn (giảm số nhân sự) được thay thế bằng các hệ thống bạn phụ thuộc vào nằm ngoài tầm kiểm soát của bạn. Nếu bạn đang dùng một dịch vụ bên thứ ba (đám mây, dịch vụ tích hợp dữ liệu, v.v.), bạn sẽ được cảnh báo ra sao nếu có một sự cố ngừng hoạt động? Kế hoạch phản ứng của bạn là gì nếu một dịch vụ bạn phụ thuộc vào đột ngột ngoại tuyến? Đáng buồn thay, không có kế hoạch phản ứng phổ quát nào tồn tại cho các thất bại của bên thứ ba. Nếu bạn có thể chuyển đổi dự phòng sang các server khác, tốt nhất là ở một vùng hoặc region khác, chắc chắn hãy thiết lập điều này.

If your data ingestion processes are built internally, do you have the proper testing and deployment automation to ensure that the code functions in production? And if the code is buggy or fails, can you roll it back to a working version?

Nếu các quy trình thu nạp dữ liệu của bạn được xây dựng nội bộ, bạn có tự động hóa kiểm thử và triển khai phù hợp để đảm bảo rằng mã hoạt động trong sản xuất không? Và nếu mã có lỗi hoặc thất bại, bạn có thể cuộn (roll back) nó về một phiên bản hoạt động không?

Data-quality tests — Các bài kiểm thử chất lượng dữ liệu

We often refer to data as a silent killer. If quality, valid data is the foundation of success in today's businesses, using bad data to make decisions is much worse than having no data. Bad data has caused untold damage to businesses; these data disasters are sometimes called datastrophes. 1

Chúng tôi thường nhắc đến dữ liệu như một kẻ giết người thầm lặng. Nếu dữ liệu chất lượng, hợp lệ là nền tảng của thành công trong các doanh nghiệp ngày nay, thì dùng dữ liệu xấu để ra quyết định còn tồi tệ hơn nhiều so với không có dữ liệu. Dữ liệu xấu đã gây ra thiệt hại khôn lường cho các doanh nghiệp; các thảm họa dữ liệu này đôi khi được gọi là datastrophe. 1

Data is entropic; it often changes in unexpected ways without warning. One of the inherent differences between DevOps and **DataOps** is that we expect software regressions only when we deploy changes, while data often presents regressions independently because of events outside our control. DevOps engineers are typically able to detect problems by using binary conditions. Has the request failure rate breached a certain threshold? How about response latency? In the data space, regressions often manifest as subtle statistical distortions. Is a change in search-term statistics a result of customer behavior? Of a spike in bot traffic that has escaped the net? Of a site test tool deployed in some other part of the company? Like system failures in DevOps, some data regressions are immediately visible. For example, in the early 2000s, Google provided search terms to websites when users arrived from search. In 2011, Google began withholding this information in some cases to protect user privacy better. Analysts quickly saw “not provided” bubbling to the tops of their reports. 2

Dữ liệu có tính entropic; nó thường thay đổi theo những cách không ngờ tới mà không có cảnh báo. Một trong những khác biệt cố hữu giữa DevOps và DataOps là chúng ta chỉ kỳ

vọng các thoái lui phần mềm khi chúng ta triển khai các thay đổi, trong khi dữ liệu thường trình ra các thoái lui một cách độc lập vì các sự kiện ngoài tầm kiểm soát của chúng ta. Các kỹ sư DevOps thường có thể phát hiện các vấn đề bằng cách dùng các điều kiện nhị phân. Tỷ lệ thất bại của yêu cầu đã vượt một ngưỡng nhất định chưa? Còn độ trễ phản hồi thì sao? Trong lĩnh vực dữ liệu, các thoái lui thường biểu hiện như những méo mó thống kê tinh tế. Một thay đổi trong thống kê từ khóa tìm kiếm là kết quả của hành vi khách hàng? Cửa một đỉnh lưu lượng bot đã thoát lưới? Cửa một công cụ kiểm thử trang được triển khai ở một phần nào đó khác của công ty? Như các thất bại hệ thống trong DevOps, một số thoái lui dữ liệu lập tức hiển hiện. Ví dụ, vào đầu những năm 2000, Google cung cấp các từ khóa tìm kiếm cho các website khi người dùng đến từ tìm kiếm. Năm 2011, Google bắt đầu giữ lại thông tin này trong một số trường hợp để bảo vệ quyền riêng tư của người dùng tốt hơn. Các nhà phân tích nhanh chóng thấy “not provided” trỗi lên đầu các báo cáo của họ.

2

The truly dangerous data regressions are silent and can come from inside or outside a business. Application developers may change the meaning of database fields without adequately communicating with data teams. Changes to data from third-party sources may go unnoticed. In the best-case scenario, reports break in obvious ways. Often business metrics are distorted unbeknownst to decision makers.

Các thoái lui dữ liệu thực sự nguy hiểm là âm thầm và có thể đến từ bên trong hoặc bên ngoài một doanh nghiệp. Các nhà phát triển ứng dụng có thể thay đổi ý nghĩa của các trường database mà không giao tiếp đầy đủ với các đội dữ liệu. Các thay đổi đối với dữ liệu từ các nguồn bên thứ ba có thể không được chú ý. Trong kịch bản tốt nhất, các báo cáo vỡ theo những cách rõ ràng. Thường thì các chỉ số kinh doanh bị méo mó mà những người ra quyết định không hề hay biết.

Whenever possible, work with software engineers to fix data-quality issues at the source. It’s surprising how many data-quality issues can be handled by respecting basic best practices in software engineering, such as logs to capture the history of data changes, checks (nulls, etc.), and exception handling (try, catch, etc.). Traditional data testing tools are generally built on simple binary logic. Are nulls appearing in a non-nullable field? Are new, unexpected items showing up in a categorical column? Statistical data testing is a new realm, but one that is likely to grow dramatically in the next five years.

Bất cứ khi nào có thể, hãy làm việc với các kỹ sư phần mềm để sửa các vấn đề chất lượng dữ liệu tại nguồn. Thật đáng ngạc nhiên là có bao nhiêu vấn đề chất lượng dữ liệu có thể được xử lý bằng cách tôn trọng các thực hành tốt nhất cơ bản trong kỹ thuật phần mềm, chẳng hạn như các log để thu lịch sử các thay đổi dữ liệu, các kiểm tra (null, v.v.), và xử lý ngoại lệ (try, catch, v.v.). Các công cụ kiểm thử dữ liệu truyền thống nhìn chung được xây dựng trên logic nhị phân đơn giản. Có các giá trị null xuất hiện trong một trường không-cho-phép-null không? Có các mục mới, không mong đợi xuất hiện trong một cột phân loại (categorical) không? Kiểm thử dữ liệu thống kê là một lĩnh vực mới, nhưng là một lĩnh vực có khả năng tăng trưởng mạnh mẽ trong năm năm tới.

Orchestration — Điều phối

Ingestion generally sits at the beginning of a large and complex data graph; since ingestion is the first stage of the data engineering lifecycle, ingested data will flow into many more data processing steps, and data from many sources will commingle in complex ways. As we’ve emphasized throughout this book, orchestration is a crucial process for coordinating these steps. Organizations in an early

stage of data maturity may choose to deploy ingestion processes as simple scheduled cron jobs. However, it is crucial to recognize that this approach is brittle and can slow the velocity of data engineering deployment and development. As data pipeline complexity grows, true orchestration is necessary. By true orchestration, we mean a system capable of scheduling complete task graphs rather than individual tasks. An orchestration can start each ingestion task at the appropriate scheduled time. Downstream processing and transform steps begin as ingestion tasks are completed. Further downstream, processing steps lead to additional processing steps.

Thu nạp nhìn chung nằm ở đầu của một đồ thị dữ liệu lớn và phức tạp; vì thu nạp là giai đoạn đầu tiên của vòng đời kỹ thuật dữ liệu, dữ liệu được thu nạp sẽ chảy vào nhiều bước xử lý dữ liệu hơn, và dữ liệu từ nhiều nguồn sẽ hòa trộn theo những cách phức tạp. Như chúng tôi đã nhấn mạnh xuyên suốt cuốn sách này, điều phối (orchestration) là một quy trình then chốt để phối hợp các bước này. Các tổ chức ở giai đoạn sớm của độ trưởng thành dữ liệu có thể chọn triển khai các quy trình thu nạp như các công việc cron được lập lịch đơn giản. Tuy nhiên, điều then chốt là nhận ra rằng cách tiếp cận này giòn và có thể làm chậm vận tốc triển khai và phát triển kỹ thuật dữ liệu. Khi độ phức tạp của đường ống dữ liệu tăng lên, điều phối thực thụ là cần thiết. Bằng điều phối thực thụ, chúng tôi muốn nói đến một hệ thống có khả năng lập lịch các đồ thị tác vụ hoàn chỉnh thay vì các tác vụ riêng lẻ. Một điều phối có thể bắt đầu mỗi tác vụ thu nạp tại thời điểm được lập lịch phù hợp. Các bước xử lý và biến đổi hạ nguồn bắt đầu khi các tác vụ thu nạp được hoàn thành. Xa hơn ở hạ nguồn, các bước xử lý dẫn đến các bước xử lý bổ sung.

Software Engineering — Kỹ thuật Phần mềm

The ingestion stage of the data engineering lifecycle is engineering intensive. This stage sits at the edge of the data engineering domain and often interfaces with external systems, where software and data engineers have to build a variety of custom plumbing.

Giai đoạn thu nạp của vòng đời kỹ thuật dữ liệu là giai đoạn thâm dụng kỹ thuật. Giai đoạn này nằm ở rìa của lĩnh vực kỹ thuật dữ liệu và thường giao tiếp với các hệ thống bên ngoài, nơi các kỹ sư phần mềm và dữ liệu phải xây dựng nhiều đường ống tùy chỉnh khác nhau.

Behind the scenes, ingestion is incredibly complicated, often with teams operating open source frameworks like Kafka or Pulsar, or some of the biggest tech companies running their own forked or homegrown ingestion solutions. As discussed in this chapter, managed data connectors have simplified the ingestion process, such as Fivetran, Matillion, and Airbyte. Data engineers should take advantage of the best available tools—primarily, managed tools and services that do a lot of the heavy lifting for you—and develop high software development competency in areas where it matters. It pays to use proper version control and code review processes and implement appropriate tests even for any ingestion-related code. When writing software, your code needs to be decoupled. Avoid writing monolithic systems with tight dependencies on the source or destination systems.

Đằng sau hậu trường, thu nạp cực kỳ phức tạp, thường với các đội vận hành các framework mã nguồn mở như Kafka hoặc Pulsar, hoặc một số công ty công nghệ lớn nhất chạy các giải pháp thu nạp đã rẽ nhánh (fork) hoặc tự xây dựng của riêng họ. Như đã bàn trong chương này, các connector dữ liệu được quản lý đã đơn giản hóa quá trình thu nạp, chẳng hạn như Fivetran, Matillion, và Airbyte. Các kỹ sư dữ liệu nên tận dụng các công cụ tốt nhất sẵn có—chủ yếu là các công cụ và dịch vụ được quản lý làm nhiều công việc nặng nhọc cho bạn—và phát triển năng lực phát triển phần mềm cao trong các lĩnh vực quan trọng. Việc dùng kiểm soát phiên bản và các quy trình rà soát mã (code review) phù hợp và triển khai các bài kiểm thử phù hợp ngay cả cho bất kỳ mã liên quan đến thu nạp nào

là đáng giá. Khi viết phần mềm, mã của bạn cần được tách rời (decoupled). Tránh viết các hệ thống monolith với các phụ thuộc chặt vào các hệ thống nguồn hoặc đích.

Conclusion — Kết luận

In your work as a data engineer, ingestion will likely consume a significant part of your energy and effort. At the heart, ingestion is plumbing, connecting pipes to other pipes, ensuring that data flows consistently and securely to its destination. At times, the minutiae of ingestion may feel tedious, but the exciting data applications (e.g., analytics and ML) cannot happen without it. As we’ve emphasized, we’re also in the midst of a sea change, moving from batch toward streaming data pipelines. This is an opportunity for data engineers to discover interesting applications for streaming data, communicate these to the business, and deploy exciting new technologies.

Trong công việc của bạn với tư cách một kỹ sư dữ liệu, thu nạp có khả năng sẽ tiêu tốn một phần đáng kể năng lượng và công sức của bạn. Về cốt lõi, thu nạp là việc lắp ống nước, nối các ống với các ống khác, đảm bảo rằng dữ liệu chảy một cách nhất quán và an toàn đến đích của nó. Đôi khi, các chi tiết vụn vặt của thu nạp có thể cảm thấy tẻ nhạt, nhưng các ứng dụng dữ liệu thú vị (ví dụ, phân tích và học máy) không thể xảy ra nếu không có nó. Như chúng tôi đã nhấn mạnh, chúng ta cũng đang ở giữa một sự thay đổi sâu sắc, chuyển từ theo lô sang các đường ống dữ liệu streaming. Đây là một cơ hội cho các kỹ sư dữ liệu khám phá các ứng dụng thú vị cho dữ liệu streaming, giao tiếp những cái này với doanh nghiệp, và triển khai các công nghệ mới thú vị.

Additional Resources

Google Cloud’s “Streaming Pipelines” web page

Microsoft’s “Snapshot Window (Azure Stream Analytics)” documentation Airbyte’s “Connections and Sync Modes” web page

1 Andy Petrella, “Datastrophes,” Medium , March 1, 2021, <https://oreil.ly/h6FRW> . 2 Danny Sullivan, “Dark Google: One Year Since Search Terms Went ‘Not Provided,’” MarTech , October 19, 2012, <https://oreil.ly/Fp8ta>

Chapter 8. Queries, Modeling, and Transformation — Chương 8. Truy vấn, Mô hình hóa và Biến đổi

Up to this point, the stages of the **data engineering lifecycle** have primarily been about passing data from one place to another or storing it. In this chapter, you'll learn how to make data useful. By understanding queries, modeling, and transformations (see Figure 8-1), you'll have the tools to turn raw data ingredients into something consumable by **downstream** stakeholders.

Cho đến thời điểm này, các giai đoạn của vòng đời kỹ thuật dữ liệu chủ yếu xoay quanh việc chuyển dữ liệu từ nơi này sang nơi khác hoặc lưu trữ nó. Trong chương này, bạn sẽ học cách làm cho dữ liệu trở nên hữu ích. Bằng cách hiểu về truy vấn, mô hình hóa và biến đổi (transformation) (xem Hình 8-1), bạn sẽ có công cụ để biến nguyên liệu dữ liệu thô thành thứ mà các bên liên quan ở hạ nguồn (downstream) có thể tiêu thụ được.

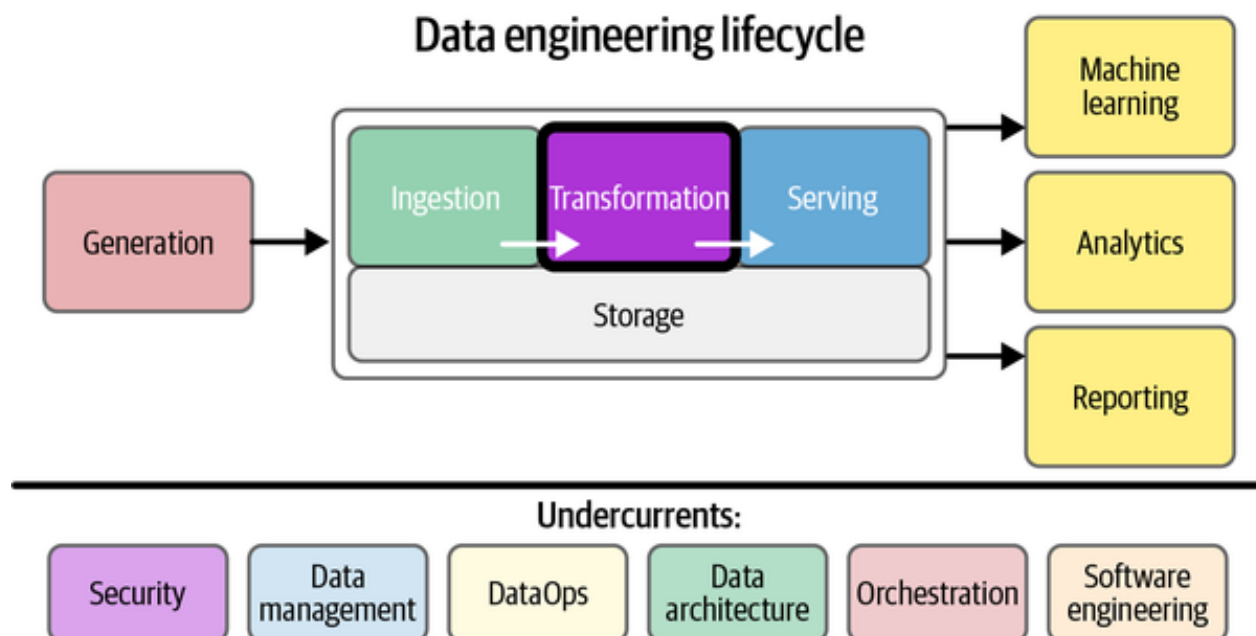


Figure 8-1. Transformations allow us to create value from data We'll first discuss queries and the significant patterns underlying them. Second, we will look at the major **data modeling** patterns you can use to introduce business logic into your data. Then, we'll cover transformations, which take the logic of your data models and the results of queries and make them useful for more straightforward downstream consumption. Finally, we'll cover whom you'll work with and the **undercurrents** as

they relate to this chapter. A variety of techniques can be used to query, model, and transform data in SQL and NoSQL databases. This section focuses on queries made to an **OLAP** system, such as a **data warehouse** or **data lake**. Although many languages exist for querying, for the sake of convenience and familiarity, throughout most of this chapter, we'll focus heavily on SQL, the most popular and universal query language. Most of the concepts for OLAP databases and SQL will translate to other types of databases and query languages. This chapter assumes you have an understanding of the SQL language, and related concepts like primary and foreign keys. If these ideas are unfamiliar to you, countless resources are available to help you get started. A note on the terms used in this chapter: For convenience, we'll use the term database as a shorthand for a query engine and the **storage** it's querying; this could be a cloud data warehouse or Apache Spark querying data stored in S3. We assume the database has a storage engine that organizes the data under the hood. This extends to file-based queries (loading a CSV file into a Python notebook) and queries against file formats such as Parquet. Also, note that this chapter focuses mainly on the query, modeling patterns, and transformations related to structured and semistructured data, which data engineers use often. Many of the practices discussed can also be applied to working with unstructured data such as images, video, and raw text. Before we get into modeling and transforming data, let's look at queries—what they are, how they work, considerations for improving query performance, and queries on **streaming** data.

Hình 8-1. Biến đổi cho phép chúng ta tạo ra giá trị từ dữ liệu. Trước hết, chúng ta sẽ thảo luận về truy vấn và những khuôn mẫu quan trọng nằm bên dưới chúng. Thứ hai, chúng ta sẽ xem xét các khuôn mẫu mô hình hóa dữ liệu chính mà bạn có thể dùng để đưa logic nghiệp vụ vào dữ liệu của mình. Sau đó, chúng ta sẽ bàn về các phép biến đổi, vốn lấy logic của các mô hình dữ liệu cùng kết quả của các truy vấn và biến chúng thành thứ hữu ích cho việc tiêu thụ ở hạ nguồn một cách dễ dàng hơn. Cuối cùng, chúng ta sẽ nói về những người bạn sẽ làm việc cùng và các dòng chảy ngầm (undercurrents) liên quan đến chương này. Có nhiều kỹ thuật khác nhau có thể dùng để truy vấn, mô hình hóa và biến đổi dữ liệu trong các database SQL và NoSQL. Phần này tập trung vào các truy vấn thực hiện trên một hệ thống OLAP, chẳng hạn như data warehouse hoặc data lake. Mặc dù tồn tại nhiều ngôn ngữ để truy vấn, nhưng vì tiện lợi và quen thuộc, xuyên suốt phần lớn chương này chúng ta sẽ tập trung mạnh vào SQL, ngôn ngữ truy vấn phổ biến và phổ quát nhất. Hầu hết các khái niệm về database OLAP và SQL sẽ chuyển đổi tương ứng được sang các loại database và ngôn ngữ truy vấn khác. Chương này giả định bạn đã hiểu ngôn ngữ SQL cùng các khái niệm liên quan như khóa chính (primary key) và khóa ngoại (foreign key). Nếu những ý tưởng này còn xa lạ với bạn, có vô số tài nguyên sẵn có để giúp bạn bắt đầu. Một lưu ý về các thuật ngữ dùng trong chương này. Để tiện lợi, chúng ta sẽ dùng thuật ngữ database như cách viết tắt cho một query engine cùng kho lưu trữ mà nó đang truy vấn; đó có thể là một cloud data warehouse hoặc Apache Spark truy vấn dữ liệu lưu trên S3. Chúng ta giả định database có một storage engine tổ chức dữ liệu bên dưới. Điều này cũng mở rộng sang các truy vấn dựa trên tệp (nạp một tệp CSV vào một notebook Python) và các truy vấn lên những định dạng tệp như Parquet. Ngoài ra, lưu ý rằng chương này chủ yếu tập trung vào truy vấn, các khuôn mẫu mô hình hóa, và các phép biến đổi liên quan đến dữ liệu có cấu trúc và bán cấu trúc, vốn là thứ mà kỹ sư dữ liệu sử dụng thường xuyên. Nhiều thực hành được bàn ở đây cũng có thể áp dụng khi làm việc với dữ liệu phi cấu trúc như hình ảnh, video và văn bản thô. Trước khi đi vào mô hình hóa và biến đổi dữ liệu, hãy cùng xem xét các truy vấn—chúng là gì, hoạt động ra sao, các cân nhắc để cải thiện hiệu năng truy vấn, và truy vấn trên dữ liệu streaming.

Queries — Truy vấn

Queries are a fundamental part of **data engineering**, data science, and analysis. Before you learn about the underlying patterns and technologies for transformations, you need to understand what queries are, how they work on various data, and techniques for improving query performance. This section primarily concerns itself with queries on tabular and semistructured data. As a **data engineer**, you'll most frequently query and transform these data types. Before we get into more complicated topics about queries and transformations, let's start by answering a pretty simple question: what is a query?

Truy vấn là một phần nền tảng của kỹ thuật dữ liệu, khoa học dữ liệu và phân tích. Trước khi bạn tìm hiểu về các khuôn mẫu và công nghệ nền tảng cho biến đổi, bạn cần hiểu truy vấn là gì, chúng hoạt động thế nào trên các loại dữ liệu khác nhau, và các kỹ thuật để cải thiện hiệu năng truy vấn. Phần này chủ yếu liên quan đến truy vấn trên dữ liệu dạng bảng và bán cấu trúc. Với tư cách kỹ sư dữ liệu, bạn sẽ truy vấn và biến đổi các kiểu dữ liệu này thường xuyên nhất. Trước khi đi vào những chủ đề phức tạp hơn về truy vấn và biến đổi, hãy bắt đầu bằng việc trả lời một câu hỏi khá đơn giản: truy vấn là gì?

What Is a Query? We often run into people who know how to write SQL but are unfamiliar with how a query works under the hood. Some of this introductory material on queries will be familiar to experienced data engineers; feel free to skip ahead if this applies to you. A query allows you to retrieve and act on data. Recall our conversation in Chapter 5 about CRUD. When a query retrieves data, it is issuing a request to read a pattern of records. This is the R (read) in CRUD. You might issue a query that gets all records from a table `foo`, such as `SELECT * FROM foo`. Or, you might apply a predicate (logical condition) to filter your data by retrieving only records where the `id` is 1, using the SQL query `SELECT`

Truy vấn là gì? Chúng ta thường gặp những người biết viết SQL nhưng không quen với cách một truy vấn hoạt động bên dưới. Một số tài liệu nhập môn về truy vấn này sẽ quen thuộc với các kỹ sư dữ liệu giàu kinh nghiệm; bạn cứ tự nhiên bỏ qua phần này nếu nó đúng với bạn. Một truy vấn cho phép bạn truy xuất và tác động lên dữ liệu. Hãy nhớ lại cuộc thảo luận của chúng ta ở Chương 5 về CRUD. Khi một truy vấn truy xuất dữ liệu, nó đang phát một yêu cầu để đọc một khuôn mẫu các bản ghi. Đây chính là R (read) trong CRUD. Bạn có thể phát một truy vấn lấy tất cả bản ghi từ một bảng `foo`, chẳng hạn `SELECT * FROM foo`. Hoặc, bạn có thể áp dụng một vị từ (điều kiện logic) để lọc dữ liệu bằng cách chỉ lấy những bản ghi có `id` bằng 1, dùng truy vấn SQL `SELECT`

`* FROM foo WHERE id=1`.

Many databases allow you to create, update, and delete data. These are the CUD in CRUD; your query will either create, mutate, or destroy existing records. Let's review some other common acronyms you'll run into when working with query languages. Data definition language At a high level, you first need to create the database objects before adding data. You'll use data definition language (DDL) commands to perform operations on database objects, such as the database itself, schemas, tables, or users; DDL defines the state of objects in your database. Data engineers use common SQL DDL expressions: `CREATE`, `DROP`, and `UPDATE`. For example, you can create a database by using the DDL expression `CREATE DATABASE bar`. After that, you can also create new tables

Nhiều database cho phép bạn tạo, cập nhật và xóa dữ liệu. Đó là CUD trong CRUD; truy vấn của bạn sẽ tạo, thay đổi, hoặc hủy các bản ghi hiện có. Hãy điểm qua một số từ viết tắt phổ biến khác mà bạn sẽ gặp khi làm việc với các ngôn ngữ truy vấn. Data definition language Ở mức cao, trước tiên bạn cần tạo các đối tượng database trước khi thêm dữ liệu. Bạn sẽ dùng các câu lệnh ngôn ngữ định nghĩa dữ liệu (data definition language, DDL)

để thực hiện các thao tác trên các đối tượng database, chẳng hạn như bản thân database, các schema, bảng, hoặc người dùng; DDL định nghĩa trạng thái của các đối tượng trong database. Kỹ sư dữ liệu sử dụng các biểu thức DDL SQL phổ biến: CREATE, DROP, và UPDATE. Ví dụ, bạn có thể tạo một database bằng biểu thức DDL CREATE DATABASE bar. Sau đó, bạn cũng có thể tạo các bảng mới

(CREATE table bar_table) or delete a table (DROP table bar_table).

Data manipulation language After using DDL to define database objects, you need to add and alter data within these objects, which is the primary purpose of data manipulation language (DML). Some common DML commands you'll use as a data engineer are as follows:

Data manipulation language Sau khi dùng DDL để định nghĩa các đối tượng database, bạn cần thêm và thay đổi dữ liệu bên trong các đối tượng này, và đó là mục đích chính của ngôn ngữ thao tác dữ liệu (data manipulation language, DML). Một số câu lệnh DML phổ biến mà bạn sẽ dùng với tư cách kỹ sư dữ liệu như sau:

SELECT
INSERT
UPDATE
DELETE
COPY
MERGE

For example, you can INSERT new records into a database table, UPDATE existing ones, and SELECT specific records. Data control language You most likely want to limit access to database objects and finely control who has access to what . Data control language (DCL) allows you to control access to the database objects or the data by using SQL commands such as

Ví dụ, bạn có thể INSERT các bản ghi mới vào một bảng database, UPDATE các bản ghi hiện có, và SELECT các bản ghi cụ thể. Data control language Bạn rất có thể muốn giới hạn quyền truy cập vào các đối tượng database và kiểm soát tinh vi xem ai có quyền truy cập vào cái gì. Ngôn ngữ kiểm soát dữ liệu (data control language, DCL) cho phép bạn kiểm soát quyền truy cập vào các đối tượng database hoặc dữ liệu bằng các câu lệnh SQL như

GRANT ,DENY , andREVOKE .

Let's walk through a brief example using DCL commands. A new data scientist named Sarah joins your company, and she needs read-only access to a database called data_science_db . You give Sarah access to this database by using the following DCL command:

Hãy cùng đi qua một ví dụ ngắn sử dụng các câu lệnh DCL. Một nhà khoa học dữ liệu mới tên Sarah gia nhập công ty bạn, và cô ấy cần quyền truy cập chỉ-đọc vào một database tên data_science_db. Bạn cấp cho Sarah quyền truy cập database này bằng câu lệnh DCL sau:

GRANTSELECTONdata_science_dbTOuser_nameSarah;

It's a **hot** job market, and Sarah has worked at the company for only a few months before getting poached by a big tech company. So long, Sarah! Being a security-minded data engineer, you remove Sarah's ability to read from the database:

Đây là thị trường việc làm nóng bỏng, và Sarah mới làm ở công ty được vài tháng trước khi bị một công ty công nghệ lớn chiêu mộ. Tạm biệt Sarah! Là một kỹ sư dữ liệu có tư duy bảo mật, bạn thu hồi khả năng đọc database của Sarah:

REVOKESELECTONdata_science_dbTOuser_nameSarah;

Access-control requests and issues are common, and understanding DCL will help you resolve problems if you or a team member can't access the data they need, as well as prevent access to data they don't need. TCL stands for transaction control language . As the name suggests, TCL supports commands that control the details of transactions. With TCL, we can define commit checkpoints, conditions when actions will be rolled back, and more. Two common TCL commands include COMMIT and ROLLBACK .

Các yêu cầu và sự cố kiểm soát truy cập là chuyện thường gặp, và hiểu DCL sẽ giúp bạn giải quyết vấn đề nếu bạn hoặc một thành viên trong nhóm không thể truy cập dữ liệu họ cần, cũng như ngăn quyền truy cập vào dữ liệu họ không cần. TCL là viết tắt của transaction control language (ngôn ngữ kiểm soát giao dịch). Như tên gọi, TCL hỗ trợ các câu lệnh kiểm soát chi tiết của các giao dịch (transaction). Với TCL, chúng ta có thể định nghĩa các điểm kiểm soát commit, các điều kiện khi nào hành động sẽ bị quay lui (rollback), và nhiều hơn nữa. Hai câu lệnh TCL phổ biến gồm COMMIT và ROLLBACK.

The Life of a Query How does a query work, and what happens when a query is executed? Let's cover the high-level basics of query execution (Figure 8-2), using an example of a typical SQL query executing in a database.

Vòng đời của một truy vấn Một truy vấn hoạt động thế nào, và điều gì xảy ra khi một truy vấn được thực thi? Hãy điểm qua những kiến thức cơ bản ở mức cao về việc thực thi truy vấn (Hình 8-2), dùng ví dụ về một truy vấn SQL điển hình thực thi trong một database.

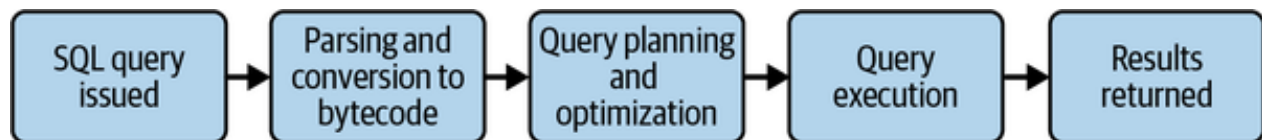


Figure 8-2. The life of a SQL query in a database While running a query might seem simple—write code, run it, and get results—a lot is going on under the hood. When you execute a SQL query, here's a summary of what happens:

Hình 8-2. Vòng đời của một truy vấn SQL trong một database. Dù chạy một truy vấn nghe có vẻ đơn giản—viết mã, chạy nó, và lấy kết quả—nhưng có rất nhiều thứ diễn ra bên dưới. Khi bạn thực thi một truy vấn SQL, đây là tóm tắt những gì xảy ra:

1. The database engine compiles the SQL, parsing the code to check for proper semantics and ensuring that the database objects referenced exist and that the current user has the appropriate access to these objects.
 2. The SQL code is converted into bytecode. This bytecode expresses the steps that must be executed on the database engine in an efficient, machine-readable format.
 3. The database's **query optimizer** analyzes the bytecode to determine how to execute the query, reordering and refactoring steps to use available resources as efficiently as possible.
 4. The query is executed, and results are produced.
1. Database engine biên dịch SQL, phân tích cú pháp mã để kiểm tra ngữ nghĩa đúng đắn và đảm bảo rằng các đối tượng database được tham chiếu là tồn tại và người dùng hiện tại có quyền truy cập phù hợp vào các đối tượng này.
 2. Mã SQL được chuyển thành bytecode. Bytecode này diễn đạt các bước phải được thực thi trên database engine theo định dạng hiệu quả, máy đọc được.
 3. Bộ tối ưu truy vấn (query optimizer) của database phân tích bytecode để xác định cách thực thi truy vấn, sắp xếp lại và tái cấu trúc các bước nhằm sử dụng các tài nguyên sẵn có hiệu quả nhất có thể.
 4. Truy vấn được thực thi, và kết quả được tạo ra.

The Query Optimizer Queries can have wildly different execution times, depending on how they're executed. A query optimizer's job is to optimize query performance and minimize costs by breaking the query into appropriate steps in an efficient order. The optimizer will assess joins, indexes, data scan size, and other factors. The query optimizer attempts to execute the query in the least expensive manner. Query optimizers are fundamental to how your query will perform. Every database is different and executes queries in ways that are obviously and subtly different from each other. You won't directly work with a query optimizer, but understanding some of its functionality will help you write more performant queries. You'll need to know how to analyze a query's performance, using things like an explain plan or query analysis, described in the following section. Improving Query Performance In data engineering, you'll inevitably encounter poorly performing queries. Knowing how to identify and fix these queries is invaluable. Don't fight your database. Learn to work with its strengths and augment its weaknesses. This section shows various ways to improve your query performance. Optimize your JOIN strategy and **schema** A single dataset (such as a table or file) is rarely useful on its own; we create value by combining it with other datasets. Joins are one of the most common means of combining datasets and creating new ones. We assume that you're familiar with the significant types of joins (e.g., inner, outer, left, cross) and the types of join relationships (e.g., one to one, one to many, many to one, and many to many). Joins are critical in data engineering and are well supported and performant in many databases. Even columnar databases, which in the past had a reputation for slow join performance, now generally offer excellent performance. A common technique for improving query performance is to pre-join data. If you find that **analytics** queries are joining the same data repeatedly, it often makes sense to join the data in advance and have queries read from the pre-joined version of the data so that you're not repeating computationally intensive work. This may mean changing the schema and relaxing **normalization** conditions to widen tables and utilize newer data structures (such as arrays or structs) for replacing frequently joined entity relationships. Another strategy is maintaining a more normalized schema but pre-joining tables for the most common analytics and data science use cases. We can simply create pre-joined tables and train users to utilize these or join inside materialized views (see "Materialized Views, Federation, and Query Virtualization"). Next, consider the details and complexity of your join conditions. Complex join logic may consume significant computational resources. We can improve performance for complex joins in a few ways. Many row-oriented databases allow you to index a result computed from a row. For instance, PostgreSQL allows you to create an index on a string field converted to lowercase; when the optimizer encounters a query where the lower() function appears inside a predicate, it can apply the index. You can also create a new derived column for joining, though you will need to train users to join on this column.

Bộ tối ưu truy vấn Các truy vấn có thể có thời gian thực thi khác nhau cực lớn, tùy thuộc vào cách chúng được thực thi. Công việc của bộ tối ưu truy vấn là tối ưu hiệu năng truy vấn và giảm thiểu chi phí bằng cách chia truy vấn thành các bước phù hợp theo một thứ tự hiệu quả. Bộ tối ưu sẽ đánh giá các phép join, các index, kích thước quét dữ liệu, và các yếu tố khác. Bộ tối ưu truy vấn cố gắng thực thi truy vấn theo cách ít tốn kém nhất. Các bộ tối ưu truy vấn có vai trò nền tảng đối với cách truy vấn của bạn vận hành. Mỗi database mỗi khác và thực thi các truy vấn theo những cách khác nhau, có khi rõ ràng và có khi tinh tế so với nhau. Bạn sẽ không làm việc trực tiếp với bộ tối ưu truy vấn, nhưng hiểu một phần chức năng của nó sẽ giúp bạn viết các truy vấn hiệu năng hơn. Bạn sẽ cần biết cách phân tích hiệu năng của một truy vấn, dùng những thứ như kế hoạch thực thi (explain plan) hoặc phân tích truy vấn, được mô tả ở phần tiếp theo. Cải thiện hiệu năng truy vấn Trong kỹ thuật dữ liệu, bạn chắc chắn sẽ gặp những truy vấn vận hành kém. Biết cách nhận diện và sửa những truy vấn này là vô giá. Đừng chống lại database của bạn. Hãy học cách làm việc với điểm mạnh của nó và bù đắp cho điểm yếu của nó. Phần này trình bày nhiều cách khác nhau để cải thiện hiệu năng truy vấn của bạn. Tối ưu chiến lược JOIN và schema của bạn Một tập dữ liệu đơn lẻ (chẳng hạn một bảng hoặc một tệp) hiếm

khi tự nó hữu ích; chúng ta tạo giá trị bằng cách kết hợp nó với các tập dữ liệu khác. Các phép join là một trong những phương tiện phổ biến nhất để kết hợp các tập dữ liệu và tạo ra các tập mới. Chúng tôi giả định rằng bạn quen thuộc với các loại join quan trọng (ví dụ inner, outer, left, cross) và các loại quan hệ join (ví dụ một-một, một-nhiều, nhiều-một, và nhiều-nhiều). Các phép join là tối quan trọng trong kỹ thuật dữ liệu và được hỗ trợ tốt cũng như hiệu năng cao trong nhiều database. Ngay cả các database dạng cột (columnar), vốn trong quá khứ mang tiếng có hiệu năng join chậm, nay nhìn chung cũng cung cấp hiệu năng tuyệt vời. Một kỹ thuật phổ biến để cải thiện hiệu năng truy vấn là join trước (pre-join) dữ liệu. Nếu bạn thấy rằng các truy vấn phân tích lặp đi lặp lại việc join cùng một dữ liệu, thường sẽ hợp lý khi join dữ liệu trước và để các truy vấn đọc từ phiên bản đã được join sẵn, để bạn không lặp lại công việc tính toán nặng. Điều này có thể đồng nghĩa với việc thay đổi schema và nói lỏng các điều kiện chuẩn hóa để mở rộng các bảng và tận dụng các cấu trúc dữ liệu mới hơn (như mảng hoặc struct) nhằm thay thế các quan hệ thực thể thường xuyên được join. Một chiến lược khác là duy trì một schema chuẩn hóa hơn nhưng join trước các bảng cho những trường hợp sử dụng phân tích và khoa học dữ liệu phổ biến nhất. Chúng ta có thể đơn giản tạo các bảng đã join sẵn và huấn luyện người dùng tận dụng chúng, hoặc join bên trong các materialized view (xem “Materialized View, Liên hợp, và Ảo hóa Truy vấn”). Tiếp theo, hãy cân nhắc các chi tiết và độ phức tạp của các điều kiện join của bạn. Logic join phức tạp có thể tiêu thụ tài nguyên tính toán đáng kể. Chúng ta có thể cải thiện hiệu năng cho các phép join phức tạp theo một vài cách. Nhiều database định hướng theo hàng cho phép bạn index một kết quả được tính từ một hàng. Chẳng hạn, PostgreSQL cho phép bạn tạo một index trên một trường chuỗi được chuyển thành chữ thường; khi bộ tối ưu gặp một truy vấn mà hàm lower () xuất hiện bên trong một vị từ, nó có thể áp dụng index đó. Bạn cũng có thể tạo một cột dẫn xuất mới để join, mặc dù bạn sẽ cần huấn luyện người dùng join trên cột này.

ROW EXPLOSION An obscure but frustrating problem is row explosion. This occurs when we have a large number of many-to-1 many matches, either because of repetition in join keys or as a consequence of join logic. Suppose the join key in table A has the value this repeated five times, and the join key in table B contains this same value repeated 10 times. This leads to a cross-join of these rows: every this row from table A paired with every this row from table B. This creates $5 \times 10 = 50$ rows in the output. Now suppose that many other repeats are in the join key. Row explosion often generates enough rows to consume a massive quantity of database resources or even cause a query to fail. It is also essential to know how your query optimizer handles joins. Some databases can reorder joins and predicates, while others cannot. A row explosion in an early query stage may cause the query to fail, even though a later predicate should correctly remove many of the repeats in the output. Predicate reordering can significantly reduce the computational resources required by a query. Finally, use common table expressions (CTEs) instead of nested subqueries or temporary tables. CTEs allow users to compose complex queries together in a readable fashion, helping you understand the flow of your query. The importance of readability for complex queries cannot be understated. In many cases, CTEs will also deliver better performance than a script that creates intermediate tables; if you have to create intermediate tables, consider creating temporary tables. If you'd like to learn more about CTEs, a quick web search will yield plenty of helpful information. Use the explain plan and understand your query's performance As you learned in the preceding section, the database's query optimizer influences the execution of a query. The query optimizer's explain plan will show you how the query optimizer determined its optimum lowest-cost query, the database objects used (tables, indexes, cache, etc.), and various resource consumption and performance statistics in each query stage. Some databases provide a visual representation of query stages. In contrast, others make the explain plan available via SQL with the EXPLAIN command, which displays the sequence of steps the database will take to execute the query. In addition to using EXPLAIN to understand how your query will run, you should monitor your query's performance,

viewing metrics on database resource consumption. The following are some areas to monitor: Usage of key resources such as disk, memory, and network. Data loading time versus processing time. Query execution time, number of records, the size of the data scanned, and the quantity of data shuffled. Competing queries that might cause resource contention in your database. Number of concurrent connections used versus connections available. Oversubscribed concurrent connections can have negative effects on your users who may not be able to connect to the database. Avoid full table scans All queries scan data, but not all scans are created equal. As a rule of thumb, you should query only the data you need. When you run `SELECT *` with no predicates, you're scanning the entire table and retrieving every row and column. This is very inefficient performance-wise and expensive, especially if you're using a pay-as-you-go database that charges you either for bytes scanned or compute resources utilized while a query is running. Whenever possible, use pruning to reduce the quantity of data scanned in a query. Columnar and row-oriented databases require different pruning strategies. In a column-oriented database, you should select only the columns you need. Most column-oriented OLAP databases also provide additional tools for optimizing your tables for better query performance. For instance, if you have a very large table (several terabytes in size or greater) Snowflake and BigQuery give you the option to define a cluster key on a table, which orders the table's data in a way that allows queries to more efficiently access portions of very large datasets. BigQuery also allows you to **partition** a table into smaller segments, allowing you to query only specific partitions instead of the entire table. (Be aware that inappropriate **clustering** and key distribution strategies can degrade performance.) In row-oriented databases, pruning usually centers around table indexes, which you learned in Chapter 6 . The general strategy is to create table indexes that will improve performance for your most performance-sensitive queries while not overloading the table with so many indexes such that you degrade performance. Know how your database handles commits A database commit is a change within a database, such as creating, updating, or deleting a record, table, or other database objects. Many databases support transactions —i.e., a notion of committing several operations simultaneously in a way that maintains a consistent state. Please note that the term transaction is somewhat overloaded; see Chapter 5 . The purpose of a transaction is to keep a consistent state of a database both while it's active and in the **event** of a failure. Transactions also handle isolation when multiple concurrent events might be reading, writing, and deleting from the same database objects. Without transactions, users would get potentially conflicting information when querying a database. You should be intimately familiar with how your database handles commits and transactions, and determine the expected consistency of query results. Does your database handle writes and updates in an ACID-compliant manner? Without ACID compliance, your query might return unexpected results. This could result from a dirty read, which happens when a row is read and an uncommitted transaction has altered the row. Are dirty reads an expected behavior of your database? If so, how do you handle this? Also, be aware that during update and delete transactions, some databases create new files to represent the new state of the database and retain the old files for failure checkpoint references. In these databases, running a large number of small commits can lead to clutter and consume significant storage space that might need to be vacuumed periodically. Let's briefly consider three databases to understand the impact of commits (note these examples are current as of the time of this writing). First, suppose we're looking at a PostgreSQL RDBMS and applying ACID transactions. Each transaction consists of a package of operations that will either fail or succeed as a group. We can also run analytics queries across many rows; these queries will present a consistent picture of the database at a point in time. The disadvantage of the PostgreSQL approach is that it requires row locking (blocking reads and writes to certain rows), which can degrade performance in various ways. PostgreSQL is not optimized for large scans or the massive amounts of data appropriate for large-scale analytics applications. Next, consider Google BigQuery. It utilizes a point-in-time full table commit model. When a read query is issued, BigQuery will read from the latest committed snapshot of the table. Whether the query runs for one second or two hours, it will read only from that snapshot and will

not see any subsequent changes. BigQuery does not lock the table while I read from it. Instead, subsequent write operations will create new commits and new snapshots while the query continues to run on the snapshot where it started. To prevent the inconsistent state, BigQuery allows only one write operation at a time. In this sense, BigQuery provides no write concurrency whatsoever. (In the sense that it can write massive amounts of data in parallel inside a single write query, it is highly concurrent.) If more than one client attempts to write simultaneously, write queries are queued in order of arrival. BigQuery's commit model is similar to the commit models used by Snowflake, Spark, and others. Last, let's consider MongoDB. We refer to MongoDB as a variable consistency database. Engineers have various configurable consistency options, both for the database and at the level of individual queries. MongoDB is celebrated for its extraordinary **scalability** and write concurrency but is somewhat notorious for issues that arise when engineers abuse it. 2 For instance, in certain modes, MongoDB supports ultra-high write performance. However, this comes at a cost: the database will unceremoniously and silently discard writes if it gets overwhelmed with traffic. This is perfectly suitable for applications that can stand to lose some data—for example, IoT applications where we simply want many measurements but don't care about capturing all measurements. It is not a great fit for applications that need to capture exact data and statistics. None of this is to say these are bad databases. They're all fantastic databases when they are chosen for appropriate applications and configured correctly. The same goes for virtually any database technology. Companies don't hire engineers simply to hack on code in isolation. To be worthy of their title, engineers should develop a deep understanding of the problems they're tasked with solving and the technology tools. This applies to commit and consistency models and every other aspect of technology performance. Appropriate technology choices and configuration can ultimately differentiate extraordinary success and massive failure. Refer to Chapter 6 for a deeper discussion of consistency.

Vacuum dead records As we just discussed, transactions incur the overhead of creating new records during certain operations, such as updates, deletes, and index operations, while retaining the old records as pointers to the last state of the database. As these old records accumulate in the database filesystem, they eventually no longer need to be referenced. You should remove these dead records in a process called vacuuming. You can vacuum a single table, multiple tables, or all tables in a database. No matter how you choose to vacuum, deleting dead database records is important for a few reasons. First, it frees up space for new records, leading to less table bloat and faster queries. Second, new and relevant records mean query plans are more accurate; outdated records can lead the query optimizer to generate suboptimal and inaccurate plans. Finally, vacuuming cleans up poor indexes, allowing for better index performance.

BÙNG NỔ HÀNG (ROW EXPLOSION) Một vấn đề khó hiểu nhưng gây bức bối là bùng nổ hàng (row explosion). Điều này xảy ra khi chúng ta có một lượng lớn các cặp khớp nhiều-một, hoặc vì sự lặp lại trong các khóa join hoặc do hệ quả của logic join. Giả sử khóa join trong bảng A có giá trị this lặp lại năm lần, và khóa join trong bảng B chứa cùng giá trị này lặp lại 10 lần. Điều này dẫn đến một phép cross-join của các hàng đó: mỗi hàng this từ bảng A ghép với mỗi hàng this từ bảng B. Điều này tạo ra $5 \times 10 = 50$ hàng trong kết quả đầu ra. Bây giờ giả sử có nhiều lần lặp khác trong khóa join. Bùng nổ hàng thường tạo ra đủ số hàng để tiêu thụ một lượng khổng lồ tài nguyên database hoặc thậm chí khiến một truy vấn thất bại. Việc biết bộ tối ưu truy vấn của bạn xử lý các phép join như thế nào cũng rất thiết yếu. Một số database có thể sắp xếp lại các phép join và vị từ, trong khi số khác thì không. Một vụ bùng nổ hàng ở giai đoạn truy vấn sớm có thể khiến truy vấn thất bại, dù một vị từ ở giai đoạn sau lẽ ra sẽ loại bỏ đúng nhiều bản lặp trong kết quả. Việc sắp xếp lại vị từ có thể giảm đáng kể tài nguyên tính toán mà một truy vấn yêu cầu. Cuối cùng, hãy dùng biểu thức bảng chung (common table expression, CTE) thay vì các truy vấn con lồng nhau hoặc bảng tạm. CTE cho phép người dùng kết hợp các truy vấn phức tạp lại với nhau theo cách dễ đọc, giúp bạn hiểu được luồng của truy vấn. Không thể đánh giá thấp tầm quan trọng của tính dễ đọc đối với các truy vấn phức tạp. Trong nhiều trường hợp,

CTE cũng sẽ cho hiệu năng tốt hơn một script tạo ra các bảng trung gian; nếu bạn buộc phải tạo các bảng trung gian, hãy cân nhắc tạo các bảng tạm. Nếu bạn muốn tìm hiểu thêm về CTE, một cú tìm kiếm web nhanh sẽ cho ra nhiều thông tin hữu ích. Dùng explain plan và hiểu hiệu năng truy vấn của bạn Như bạn đã học ở phần trước, bộ tối ưu truy vấn của database ảnh hưởng đến việc thực thi một truy vấn. Explain plan của bộ tối ưu truy vấn sẽ cho bạn thấy bộ tối ưu đã xác định truy vấn tối ưu, chi phí thấp nhất của nó như thế nào, các đối tượng database được dùng (bảng, index, cache, v.v.), và nhiều thống kê tiêu thụ tài nguyên cùng hiệu năng ở từng giai đoạn truy vấn. Một số database cung cấp biểu diễn trực quan của các giai đoạn truy vấn. Ngược lại, số khác cung cấp explain plan qua SQL bằng câu lệnh EXPLAIN, vốn hiển thị chuỗi các bước mà database sẽ thực hiện để thực thi truy vấn. Bên cạnh việc dùng EXPLAIN để hiểu truy vấn của bạn sẽ chạy ra sao, bạn nên giám sát hiệu năng của truy vấn, xem các chỉ số về tiêu thụ tài nguyên database. Dưới đây là một số khía cạnh cần giám sát: Mức sử dụng các tài nguyên then chốt như đĩa, bộ nhớ và mạng. Thời gian nạp dữ liệu so với thời gian xử lý. Thời gian thực thi truy vấn, số lượng bản ghi, kích thước dữ liệu được quét, và lượng dữ liệu được xáo trộn (shuffle). Các truy vấn cạnh tranh có thể gây tranh chấp tài nguyên trong database của bạn. Số kết nối đồng thời được dùng so với số kết nối sẵn có. Việc đăng ký vượt mức kết nối đồng thời có thể gây ảnh hưởng tiêu cực đến người dùng của bạn, những người có thể không kết nối được vào database. Tránh quét toàn bộ bảng Mọi truy vấn đều quét dữ liệu, nhưng không phải mọi lần quét đều như nhau. Theo nguyên tắc kinh nghiệm, bạn chỉ nên truy vấn dữ liệu mình cần. Khi bạn chạy `SELECT *` không có vị từ, bạn đang quét toàn bộ bảng và lấy mọi hàng và mọi cột. Điều này rất kém hiệu quả về mặt hiệu năng và tốn kém, đặc biệt nếu bạn dùng một database trả-tiền-theo-mức-dùng tính phí cho bạn dựa trên số byte được quét hoặc tài nguyên tính toán được dùng khi một truy vấn đang chạy. Bất cứ khi nào có thể, hãy dùng việc cắt tỉa (pruning) để giảm lượng dữ liệu được quét trong một truy vấn. Các database dạng cột và định hướng theo hàng đòi hỏi các chiến lược cắt tỉa khác nhau. Trong database hướng cột, bạn chỉ nên chọn các cột mình cần. Hầu hết các database OLAP hướng cột cũng cung cấp các công cụ bổ sung để tối ưu các truy vấn của bạn nhằm có hiệu năng truy vấn tốt hơn. Chẳng hạn, nếu bạn có một bảng rất lớn (cỡ vài terabyte trở lên), Snowflake và BigQuery cho bạn lựa chọn định nghĩa một cluster key trên một bảng, vốn sắp xếp dữ liệu của bảng theo cách cho phép các truy vấn truy cập hiệu quả hơn vào các phần của những tập dữ liệu rất lớn. BigQuery cũng cho phép bạn phân vùng (partition) một bảng thành các phân đoạn nhỏ hơn, cho phép bạn chỉ truy vấn các phân vùng cụ thể thay vì toàn bộ bảng. (Hãy lưu ý rằng các chiến lược gom cụm và phân bố khóa không phù hợp có thể làm giảm hiệu năng.) Trong các database định hướng theo hàng, việc cắt tỉa thường xoay quanh các index của bảng, thứ bạn đã học ở Chương 6. Chiến lược chung là tạo các index bằng sẽ cải thiện hiệu năng cho các truy vấn nhạy cảm với hiệu năng nhất, đồng thời không nạp quá nhiều index vào bảng đến mức làm giảm hiệu năng. Hiểu cách database của bạn xử lý commit Một commit database là một thay đổi bên trong một database, chẳng hạn như tạo, cập nhật, hoặc xóa một bản ghi, bảng, hoặc các đối tượng database khác. Nhiều database hỗ trợ các giao dịch (transaction)—tức là khái niệm commit nhiều thao tác cùng một lúc theo cách duy trì một trạng thái nhất quán. Xin lưu ý rằng thuật ngữ giao dịch có phần bị dùng quá tải; xem Chương 5. Mục đích của một giao dịch là giữ một trạng thái nhất quán của database cả khi nó đang hoạt động lẫn trong trường hợp xảy ra sự cố. Các giao dịch cũng xử lý sự cô lập (isolation) khi nhiều sự kiện đồng thời có thể đang đọc, ghi, và xóa từ cùng các đối tượng database. Không có giao dịch, người dùng có thể nhận được thông tin có khả năng mâu thuẫn khi truy vấn một database. Bạn nên hiểu thấu đáo cách database của mình xử lý các commit và giao dịch, và xác định mức nhất quán dự kiến của kết quả truy vấn. Liệu database của bạn có xử lý các thao tác ghi và cập nhật theo cách tuân thủ ACID? Không tuân thủ ACID, truy vấn của bạn có thể trả về kết quả ngoài dự kiến. Điều

này có thể xảy ra do một lần đọc bẩn (dirty read), tức khi một hàng được đọc mà một giao dịch chưa commit đã thay đổi hàng đó. Liệu đọc bẩn có phải là hành vi được mong đợi của database của bạn? Nếu có, bạn xử lý điều này thế nào? Ngoài ra, hãy lưu ý rằng trong các giao dịch cập nhật và xóa, một số database tạo các tệp mới để biểu diễn trạng thái mới của database và giữ lại các tệp cũ làm tham chiếu điểm kiểm soát sự cố. Trong các database này, chạy một số lượng lớn các commit nhỏ có thể dẫn đến lộn xộn và tiêu thụ không gian lưu trữ đáng kể, thứ có thể cần được dọn (vacuum) định kỳ. Hãy cùng xem xét ngắn gọn ba database để hiểu tác động của các commit (lưu ý các ví dụ này đúng tại thời điểm viết). Đầu tiên, giả sử chúng ta đang xem một RDBMS PostgreSQL và áp dụng các giao dịch ACID. Mỗi giao dịch gồm một gói các thao tác sẽ hoặc thất bại hoặc thành công như một nhóm. Chúng ta cũng có thể chạy các truy vấn phân tích trên nhiều hàng; những truy vấn này sẽ trình bày một bức tranh nhất quán của database tại một thời điểm. Nhược điểm của cách tiếp cận PostgreSQL là nó đòi hỏi khóa hàng (row locking) (chặn đọc và ghi vào những hàng nhất định), điều có thể làm giảm hiệu năng theo nhiều cách. PostgreSQL không được tối ưu cho các lần quét lớn hoặc lượng dữ liệu khổng lồ phù hợp với các ứng dụng phân tích quy mô lớn. Tiếp theo, hãy xem xét Google BigQuery. Nó sử dụng một mô hình commit toàn bảng tại-một-thời-điểm (point-in-time). Khi một truy vấn đọc được phát ra, BigQuery sẽ đọc từ ảnh chụp (snapshot) đã commit gần nhất của bảng. Dù truy vấn chạy một giây hay hai giờ, nó sẽ chỉ đọc từ snapshot đó và sẽ không thấy bất kỳ thay đổi sau đó nào. BigQuery không khóa bảng khi tôi đọc từ nó. Thay vào đó, các thao tác ghi sau đó sẽ tạo các commit mới và snapshot mới trong khi truy vấn tiếp tục chạy trên snapshot nơi nó bắt đầu. Để ngăn trạng thái không nhất quán, BigQuery chỉ cho phép một thao tác ghi tại một thời điểm. Theo nghĩa này, BigQuery hoàn toàn không cung cấp tính đồng thời ghi. (Theo nghĩa nó có thể ghi lượng dữ liệu khổng lồ song song bên trong một truy vấn ghi duy nhất, thì nó có tính đồng thời cao.) Nếu nhiều hơn một client cố ghi đồng thời, các truy vấn ghi được xếp hàng theo thứ tự đến. Mô hình commit của BigQuery tương tự các mô hình commit được dùng bởi Snowflake, Spark, và những thứ khác. Cuối cùng, hãy xem xét MongoDB. Chúng ta gọi MongoDB là một database nhất quán biến thiên (variable consistency). Các kỹ sư có nhiều tùy chọn nhất quán có thể cấu hình, cả cho database lẫn ở cấp độ từng truy vấn riêng lẻ. MongoDB được ca ngợi vì khả năng mở rộng và đồng thời ghi phi thường nhưng cũng có phần khét tiếng về các vấn đề nảy sinh khi kỹ sư lạm dụng nó. Chẳng hạn, ở một số chế độ, MongoDB hỗ trợ hiệu năng ghi siêu cao. Tuy nhiên, điều này đi kèm cái giá: database sẽ lộn xộn và không khách sáo loại bỏ các lần ghi nếu nó bị quá tải lưu lượng. Điều này hoàn toàn phù hợp cho các ứng dụng có thể chịu được mất một phần dữ liệu—chẳng hạn, các ứng dụng IoT nơi chúng ta chỉ muốn có nhiều phép đo nhưng không quan tâm đến việc thu thập tất cả các phép đo. Nó không phải lựa chọn tốt cho các ứng dụng cần thu thập dữ liệu và thống kê chính xác. Không điều nào trong số này nói rằng đây là các database tệ. Tất cả chúng đều là những database tuyệt vời khi được chọn cho các ứng dụng phù hợp và cấu hình đúng. Điều tương tự cũng đúng với gần như mọi công nghệ database. Các công ty không thuê kỹ sư đơn thuần để hire viết mã trong cô lập. Để xứng đáng với chức danh, kỹ sư nên phát triển một hiểu biết sâu sắc về các vấn đề họ được giao giải quyết cùng các công cụ công nghệ. Điều này áp dụng cho các mô hình commit và nhất quán cũng như mọi khía cạnh khác của hiệu năng công nghệ. Lựa chọn và cấu hình công nghệ phù hợp rất cuộc có thể là yếu tố phân định giữa thành công phi thường và thất bại thảm hại. Tham khảo Chương 6 để có thảo luận sâu hơn về nhất quán. Dọn các bản ghi chết Như vừa thảo luận, các giao dịch gánh chịu phụ phí của việc tạo các bản ghi mới trong một số thao tác nhất định, như cập nhật, xóa, và các thao tác index, trong khi giữ lại các bản ghi cũ làm con trỏ tới trạng thái cuối của database. Khi các bản ghi cũ này tích tụ trong hệ thống tệp của database, cuối cùng chúng không còn cần được tham chiếu nữa. Bạn nên loại bỏ các bản ghi chết này trong một quá trình gọi

là vacuuming (dọn dẹp). Bạn có thể vacuum một bảng đơn lẻ, nhiều bảng, hoặc tất cả các bảng trong một database. Dù bạn chọn vacuum thể nào, việc xóa các bản ghi database chết là quan trọng vì một vài lý do. Thứ nhất, nó giải phóng không gian cho các bản ghi mới, dẫn đến ít phình bảng hơn và truy vấn nhanh hơn. Thứ hai, các bản ghi mới và phù hợp đồng nghĩa với việc các kế hoạch truy vấn chính xác hơn; các bản ghi lỗi thời có thể khiến bộ tối ưu truy vấn sinh ra các kế hoạch dưới tối ưu và thiếu chính xác. Cuối cùng, vacuuming dọn dẹp các index kém, cho phép hiệu năng index tốt hơn.

Vacuum operations are handled differently depending on the type of database. For example, in databases backed by **object storage** (BigQuery, Snowflake, Databricks), the only downside of old **data retention** is that it uses storage space, potentially costing money depending on the storage pricing model for the database. In Snowflake, users cannot directly vacuum. Instead, they control a “time-travel” interval that determines how long table snapshots are retained before they are auto vacuumed. BigQuery utilizes a fixed seven-day history window. Databricks generally retains data indefinitely until it is manually vacuumed; vacuuming is important to control direct S3 storage costs. Amazon Redshift handles its cluster disks in many configurations, and vacuuming can impact performance and 3 available storage. VACUUM runs automatically behind the scenes, but users may sometimes want to run it manually for tuning purposes. Vacuuming becomes even more critical for relational databases such as PostgreSQL and MySQL. Large numbers of transactional operations can cause a rapid accumulation of dead records, and engineers working in these systems need to familiarize themselves with the details and impact of vacuuming. Leverage cached query results Let’s say you have an intensive query that you often run on a database that charges you for the amount of data you query. Each time a query is run, this costs you money. Instead of rerunning the same query on the database repeatedly and incurring massive charges, wouldn’t it be nice if the results of the query were stored and available for instant retrieval? Thankfully, many cloud OLAP databases cache query results. When a query is initially run, it will retrieve data from various sources, filter and join it, and output a result. This initial query—a cold query—is similar to the notion of cold data we explored in Chapter 6 . For argument’s sake, let’s say this query took 40 seconds to run. Assuming your database caches query results, rerunning the same query might return results in 1 second or less. The results were cached, and the query didn’t need to run cold. Whenever possible, leverage query cache results to help reduce pressure on your database while providing a better user experience for frequently run queries. Note also that materialized views provide another form of query caching (see “Materialized Views, Federation, and Query Virtualization”).

Các thao tác vacuum được xử lý khác nhau tùy theo loại database. Ví dụ, trong các database hậu thuẫn bởi object storage (BigQuery, Snowflake, Databricks), nhược điểm duy nhất của việc giữ lại dữ liệu cũ là nó dùng không gian lưu trữ, có khả năng tốn tiền tùy theo mô hình giá lưu trữ của database. Trong Snowflake, người dùng không thể trực tiếp vacuum. Thay vào đó, họ kiểm soát một khoảng thời gian “du hành thời gian” (time-travel) xác định các snapshot bảng được giữ lại bao lâu trước khi chúng tự động được vacuum. BigQuery sử dụng một cửa sổ lịch sử cố định bảy ngày. Databricks nhìn chung giữ dữ liệu vô thời hạn cho đến khi nó được vacuum thủ công; việc vacuum quan trọng để kiểm soát chi phí lưu trữ S3 trực tiếp. Amazon Redshift xử lý các đĩa cluster của nó theo nhiều cấu hình, và việc vacuum có thể ảnh hưởng đến hiệu năng và dung lượng lưu trữ khả dụng. VACUUM chạy tự động phía sau hậu trường, nhưng người dùng đôi khi có thể muốn chạy nó thủ công cho mục đích tinh chỉnh. Việc vacuum trở nên còn quan trọng hơn đối với các database quan hệ như PostgreSQL và MySQL. Số lượng lớn các thao tác giao dịch có thể gây tích tụ nhanh các bản ghi chết, và kỹ sư làm việc trong các hệ thống này cần làm quen với các chi tiết và tác động của việc vacuum. Tận dụng kết quả truy vấn được cache Giả sử bạn có một truy vấn nặng mà bạn thường chạy trên một database tính phí bạn theo lượng dữ liệu bạn truy vấn. Mỗi lần một truy vấn được chạy, nó tốn tiền của bạn. Thay vì chạy lại cùng một truy

vấn trên database liên tục và gánh các khoản phí khổng lồ, chẳng phải sẽ thật tuyệt nếu kết quả của truy vấn được lưu lại và sẵn sàng để truy xuất tức thì? May mắn thay, nhiều database OLAP đám mây cache kết quả truy vấn. Khi một truy vấn được chạy lần đầu, nó sẽ truy xuất dữ liệu từ nhiều nguồn, lọc và join nó, và xuất ra một kết quả. Truy vấn ban đầu này—một truy vấn lạnh (cold query)—tương tự khái niệm dữ liệu lạnh (cold data) mà chúng ta đã khám phá ở Chương 6. Để cho dễ tranh luận, giả sử truy vấn này mất 40 giây để chạy. Giả định database của bạn cache kết quả truy vấn, chạy lại cùng một truy vấn có thể trả về kết quả trong 1 giây hoặc ít hơn. Kết quả đã được cache, và truy vấn không cần chạy lạnh. Bất cứ khi nào có thể, hãy tận dụng các kết quả cache truy vấn để giúp giảm áp lực lên database đồng thời mang lại trải nghiệm người dùng tốt hơn cho các truy vấn chạy thường xuyên. Cũng lưu ý rằng các materialized view cung cấp một dạng cache truy vấn khác (xem “Materialized View, Liên hợp, và Ảo hóa Truy vấn”).

Queries on Streaming Data Streaming data is constantly in flight. As you might imagine, querying streaming data is different from **batch** data. To fully take advantage of a data stream, we must adapt query patterns that reflect its real-time nature. For example, systems such as Kafka and Pulsar make it easier to query streaming data sources. Let’s look at some common ways to do this. Basic query patterns on streams Recall continuous CDC, discussed in Chapter 7 . CDC, in this form, essentially sets up an analytics database as a fast follower to a production database. One of the longest-standing streaming query patterns simply entails querying the analytics database, retrieving statistical results and aggregations with a slight lag behind the production database. The fast-follower approach How is this a streaming query pattern? Couldn’t we accomplish the same thing simply by running our queries on the production database? In principle, yes; in practice, no. Production databases generally aren’t equipped to handle production workloads and simultaneously run large analytics scans across significant quantities of data. Running such queries can slow the production application or even cause it to crash. The basic CDC query pattern 4 allows us to serve real-time analytics with a minimal impact on the production system. The fast-follower pattern can utilize a conventional transactional database as the follower, but there are significant advantages to using a proper OLAP-oriented system (Figure 8-3). Both Druid and BigQuery combine a streaming buffer with long-term **columnar storage** in a setup somewhat similar to the **Lambda** architecture (see Chapter 3). This works extremely well for computing trailing statistics on vast historical data with near real-time updates.

Truy vấn trên Dữ liệu Streaming Dữ liệu streaming luôn không ngừng vận động. Như bạn có thể hình dung, truy vấn dữ liệu streaming khác với dữ liệu batch. Để tận dụng triệt để một luồng dữ liệu, chúng ta phải thích ứng các khuôn mẫu truy vấn phản ánh bản chất thời gian thực của nó. Ví dụ, các hệ thống như Kafka và Pulsar giúp việc truy vấn các nguồn dữ liệu streaming dễ dàng hơn. Hãy cùng xem một số cách phổ biến để làm điều này. Các khuôn mẫu truy vấn cơ bản trên luồng Hãy nhớ lại CDC liên tục, đã thảo luận ở Chương 7. CDC, ở dạng này, về cơ bản thiết lập một database phân tích như một bộ theo dõi nhanh (fast follower) của một database sản xuất. Một trong những khuôn mẫu truy vấn streaming lâu đời nhất đơn giản là truy vấn database phân tích, lấy các kết quả thống kê và các phép tổng hợp với độ trễ nhỏ phía sau database sản xuất. Cách tiếp cận fast-follower Tại sao đây lại là một khuôn mẫu truy vấn streaming? Chẳng phải chúng ta có thể đạt được điều tương tự đơn giản bằng cách chạy các truy vấn trên database sản xuất? Về nguyên tắc thì có; trên thực tế thì không. Các database sản xuất nhìn chung không được trang bị để xử lý các khối lượng công việc sản xuất đồng thời chạy các lần quét phân tích lớn trên lượng dữ liệu đáng kể. Chạy các truy vấn như vậy có thể làm chậm ứng dụng sản xuất hoặc thậm chí khiến nó sập. Khuôn mẫu truy vấn CDC cơ bản cho phép chúng ta phục vụ phân tích thời gian thực với tác động tối thiểu lên hệ thống sản xuất. Khuôn mẫu fast-follower có thể dùng một database giao dịch thông thường làm follower, nhưng có những lợi thế đáng

kể khi dùng một hệ thống định hướng OLAP đúng nghĩa (Hình 8-3). Cả Druid và BigQuery đều kết hợp một bộ đệm streaming với lưu trữ dạng cột dài hạn trong một thiết lập phần nào tương tự kiến trúc Lambda (xem Chương 3). Điều này hoạt động cực kỳ tốt để tính toán các thống kê lũy kế trên lượng dữ liệu lịch sử khổng lồ với các cập nhật gần thời gian thực.

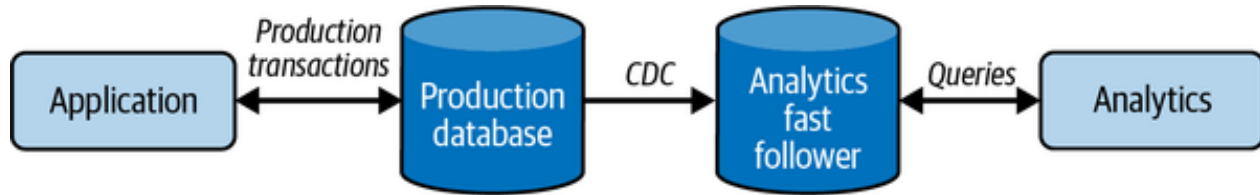


Figure 8-3. CDC with a fast-follower analytics database The fast-follower CDC approach has critical limitations. It doesn't fundamentally rethink batch query patterns. You're still running SELECT queries against the current table state, and missing the opportunity to dynamically trigger events off changes in the stream. The **Kappa architecture** Next, recall the Kappa architecture we discussed in Chapter 3 . The principal idea of this architecture is to handle all data like events and store these events as a stream rather than a table (Figure 8-4). When production application databases are the source, Kappa architecture stores events from CDC. Event streams can also flow directly from an application backend, from a swarm of IoT devices, or any system that generates events and can push them over a network. Instead of simply treating a streaming storage system as a buffer, Kappa architecture retains events in storage during a more extended retention period, and data can be directly queried from this storage. The retention period can be pretty long (months or years). Note that this is much longer than the retention period used in purely real-time oriented systems, usually a week at most.

Hình 8-3. CDC với một database phân tích fast-follower. Cách tiếp cận CDC fast-follower có những hạn chế then chốt. Nó không thực sự tư duy lại các khuôn mẫu truy vấn batch một cách căn bản. Bạn vẫn đang chạy các truy vấn SELECT trên trạng thái bảng hiện tại, và bỏ lỡ cơ hội kích hoạt động (trigger) các sự kiện một cách động dựa trên các thay đổi trong luồng. Kiến trúc Kappa Tiếp theo, hãy nhớ lại kiến trúc Kappa mà chúng ta đã thảo luận ở Chương 3. Ý tưởng chính của kiến trúc này là xử lý mọi dữ liệu như các sự kiện và lưu trữ các sự kiện này như một luồng thay vì một bảng (Hình 8-4). Khi các database ứng dụng sản xuất là nguồn, kiến trúc Kappa lưu trữ các sự kiện từ CDC. Các luồng sự kiện cũng có thể chảy trực tiếp từ một backend ứng dụng, từ một bầy thiết bị IoT, hoặc bất kỳ hệ thống nào sinh ra các sự kiện và có thể đẩy chúng qua một mạng. Thay vì đơn thuần coi một hệ thống lưu trữ streaming như một bộ đệm, kiến trúc Kappa giữ lại các sự kiện trong lưu trữ trong một khoảng thời gian lưu giữ dài hơn, và dữ liệu có thể được truy vấn trực tiếp từ kho lưu trữ này. Khoảng thời gian lưu giữ có thể khá dài (nhiều tháng hoặc năm). Lưu ý rằng điều này dài hơn nhiều so với khoảng lưu giữ dùng trong các hệ thống định hướng thuần thời gian thực, thường nhiều nhất là một tuần.

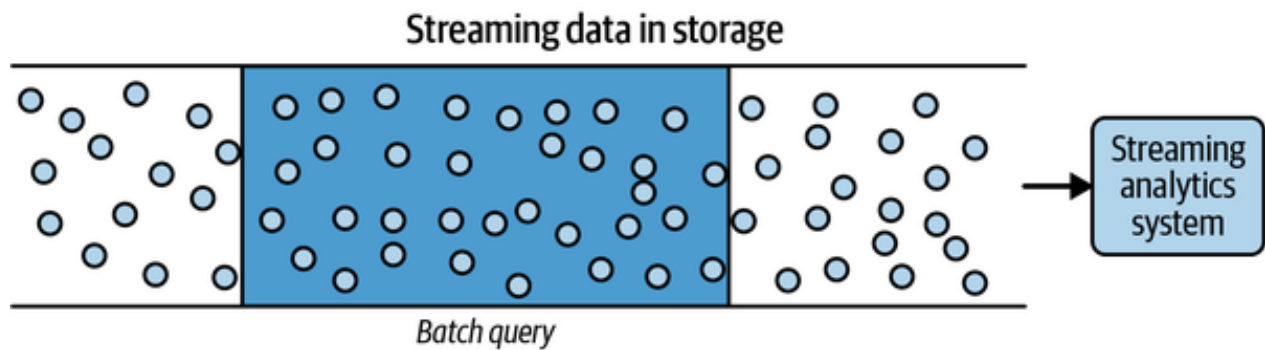


Figure 8-4. The Kappa architecture is built around streaming storage and ingest systems. The “big idea” in Kappa architecture is to treat streaming storage as a real-time transport layer and a database for retrieving and querying historical data. This happens either through the direct query capabilities of the streaming storage system or with the help of external tools. For example, Kafka KSQL supports aggregation, statistical calculations, and even sessionization. If query requirements are more complex or data needs to be combined with other data sources, an external tool such as Spark reads a time range of data from Kafka and computes the query results. The streaming storage system can also feed other applications or a stream processor such as Flink or Beam. Windows, triggers, emitted statistics, and late-arriving data. One fundamental limitation of traditional batch queries is that this paradigm generally treats the query engine as an external observer. An actor external to the data causes the query to run—perhaps an hourly cron job or a product manager opening a dashboard. Most widely used streaming systems, on the other hand, support the notion of computations triggered directly from the data itself. They might emit mean and median statistics every time a certain number of records are collected in the buffer or output a summary when a user session closes. Windows are an essential feature in streaming queries and processing. Windows are small batches that are processed based on dynamic triggers. Windows are generated dynamically over time in some ways. Let’s look at some common types of windows: session, fixed-time, and sliding. We’ll also look at watermarks. Session window A session window groups events that occur close together, and filters out periods of inactivity when no events occur. We might say that a user session is any time interval with no inactivity gap of five minutes or more. Our batch system collects data by a user ID key, orders events, determines the gaps and session boundaries, and calculates statistics for each session. Data engineers often sessionize data retrospectively by applying time conditions to user activity on web and desktop apps. In a streaming session, this process can happen dynamically. Note that session windows are per key; in the preceding example, each user gets their own set of windows. The system accumulates data per user. If a five-minute gap with no activity occurs, the system closes the window, sends its calculations, and flushes the data. If new events arrive for the use, the system starts a new session window. Session windows may also make a provision for late-arriving data. Allowing data to arrive up to five minutes late to account for network conditions and system **latency**, the system will open the window if a late-arriving event indicates activity less than five minutes after the last event. We will have more to say about late-arriving data throughout this chapter. Figure 8-5 shows three session windows, each separated by five minutes of inactivity.

Hình 8-4. Kiến trúc Kappa được xây dựng xoay quanh các hệ thống lưu trữ và thu nạp streaming. “Ý tưởng lớn” trong kiến trúc Kappa là coi lưu trữ streaming như một tầng vận chuyển thời gian thực và một database để truy xuất và truy vấn dữ liệu lịch sử. Điều này diễn ra hoặc thông qua các khả năng truy vấn trực tiếp của hệ thống lưu trữ streaming hoặc với sự trợ giúp của các công cụ bên ngoài. Ví dụ, Kafka KSQL hỗ trợ tổng hợp, tính toán thống kê, và thậm chí cả phân phiên (sessionization). Nếu yêu cầu truy vấn phức tạp hơn hoặc dữ liệu cần được kết hợp với các nguồn dữ liệu khác, một công cụ bên ngoài như

Spark đọc một khoảng thời gian của dữ liệu từ Kafka và tính toán các kết quả truy vấn. Hệ thống lưu trữ streaming cũng có thể cấp dữ liệu cho các ứng dụng khác hoặc một bộ xử lý luồng (stream processor) như Flink hoặc Beam. Cửa sổ, trigger, thống kê được phát ra, và dữ liệu đến muộn Một hạn chế nền tảng của các truy vấn batch truyền thống là khuôn mẫu này nhìn chung coi query engine như một bên quan sát từ bên ngoài. Một tác nhân bên ngoài dữ liệu khiến truy vấn chạy—có thể là một cron job mỗi giờ hoặc một product manager mở một dashboard. Mặt khác, hầu hết các hệ thống streaming được dùng rộng rãi đều hỗ trợ khái niệm các phép tính được kích hoạt trực tiếp từ chính dữ liệu. Chúng có thể phát ra thống kê trung bình và trung vị mỗi khi một số lượng bản ghi nhất định được thu thập trong bộ đệm, hoặc xuất ra một bản tóm tắt khi một phiên người dùng đóng lại. Cửa sổ (window) là một tính năng thiết yếu trong truy vấn và xử lý streaming. Cửa sổ là các lô nhỏ được xử lý dựa trên các trigger động. Cửa sổ được sinh ra một cách động theo thời gian theo một số cách nào đó. Hãy cùng xem một số loại cửa sổ phổ biến: phiên (session), thời-gian-cố-định (fixed-time), và trượt (sliding). Chúng ta cũng sẽ xem các watermark. Cửa sổ phiên Một cửa sổ phiên (session window) nhóm các sự kiện xảy ra gần nhau, và lọc bỏ các khoảng không hoạt động khi không có sự kiện nào xảy ra. Chúng ta có thể nói rằng một phiên người dùng là bất kỳ khoảng thời gian nào không có khoảng trống không hoạt động kéo dài năm phút trở lên. Hệ thống batch của chúng ta thu thập dữ liệu theo khóa ID người dùng, sắp xếp các sự kiện, xác định các khoảng trống và ranh giới phiên, và tính toán thống kê cho mỗi phiên. Kỹ sư dữ liệu thường phân phiên dữ liệu theo cách hồi cứu bằng cách áp dụng các điều kiện thời gian lên hoạt động người dùng trên các ứng dụng web và desktop. Trong một phiên streaming, quá trình này có thể diễn ra một cách động. Lưu ý rằng các cửa sổ phiên là theo từng khóa; trong ví dụ trên, mỗi người dùng có tập cửa sổ riêng. Hệ thống tích lũy dữ liệu theo từng người dùng. Nếu một khoảng trống năm phút không có hoạt động xảy ra, hệ thống đóng cửa sổ, gửi các phép tính của nó, và xả dữ liệu. Nếu các sự kiện mới đến cho người dùng, hệ thống bắt đầu một cửa sổ phiên mới. Các cửa sổ phiên cũng có thể có dự phòng cho dữ liệu đến muộn. Cho phép dữ liệu đến muộn tới năm phút để tính đến điều kiện mạng và độ trễ hệ thống, hệ thống sẽ mở cửa sổ nếu một sự kiện đến muộn cho thấy hoạt động dưới năm phút sau sự kiện cuối. Chúng ta sẽ còn nói thêm về dữ liệu đến muộn xuyên suốt chương này. Hình 8-5 cho thấy ba cửa sổ phiên, mỗi cái cách nhau bởi năm phút không hoạt động.

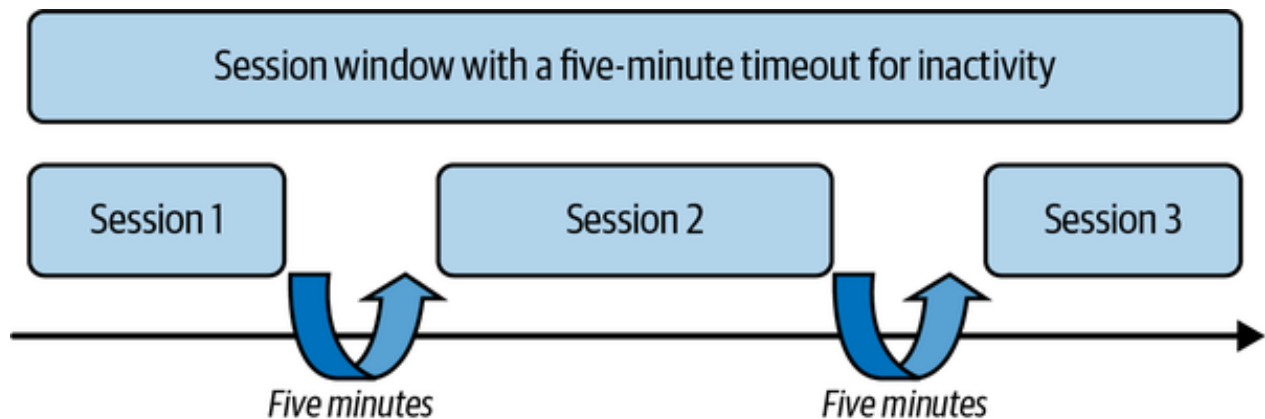


Figure 8-5. Session window with a five-minute timeout for inactivity Making sessionization dynamic and near real-time fundamentally changes its utility. With retrospective sessionization, we could automate specific actions a day or an hour after a user session closed (e.g., a follow-up email with a coupon for a product viewed by the user). With dynamic sessionization, the user could get an alert in a mobile app that is immediately useful based on their activity in the last 15 minutes. Fixed-time windows A fixed-time (aka tumbling) window features fixed time periods that run on a fixed

schedule and processes all data since the previous window is closed. For example, we might close a window every 20 seconds and process all data arriving from the previous window to give a mean and median statistic (Figure 8-6). Statistics would be emitted as soon as they could be calculated after the window closed.

Hình 8-5. Cửa sổ phiên với thời gian chờ năm phút cho việc không hoạt động. Làm cho việc phân phiên trở nên động và gần thời gian thực thay đổi tiện ích của nó một cách căn bản. Với phân phiên hồi cứu, chúng ta có thể tự động hóa các hành động cụ thể một ngày hoặc một giờ sau khi một phiên người dùng đóng lại (ví dụ một email theo dõi kèm phiếu giảm giá cho một sản phẩm người dùng đã xem). Với phân phiên động, người dùng có thể nhận một thông báo trong ứng dụng di động hữu ích ngay lập tức dựa trên hoạt động của họ trong 15 phút qua. Cửa sổ thời-gian-cố-định Một cửa sổ thời-gian-cố-định (fixed-time, còn gọi là tumbling) có các khoảng thời gian cố định chạy theo một lịch cố định và xử lý toàn bộ dữ liệu kể từ khi cửa sổ trước đóng lại. Ví dụ, chúng ta có thể đóng một cửa sổ mỗi 20 giây và xử lý toàn bộ dữ liệu đến từ cửa sổ trước để đưa ra một thống kê trung bình và trung vị (Hình 8-6). Các thống kê sẽ được phát ra ngay khi chúng có thể được tính sau khi cửa sổ đóng lại.

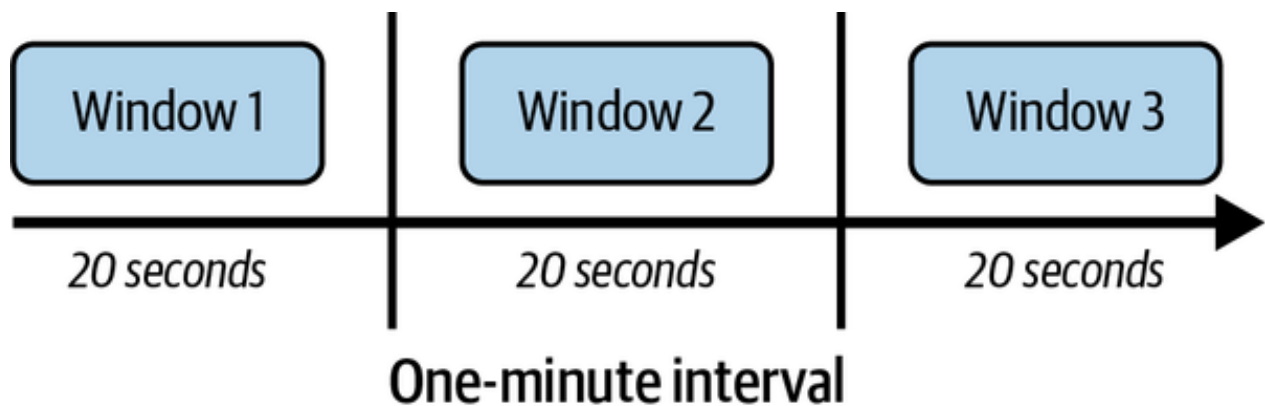


Figure 8-6. Tumbling/fixed window This is similar to traditional batch **ETL** processing, where we might run a data update job every day or every hour. The streaming system allows us to generate windows more frequently and deliver results with lower latency. As we'll repeatedly emphasize, batch is a special case of streaming. Sliding windows Events in a sliding window are bucketed into windows of fixed time length. For example, we could generate a new 60-second window every 30 seconds (Figure 8-7). Just as we did before, we can emit mean and median statistics. Sliding windows are also known as hopping windows .

Hình 8-6. Cửa sổ tumbling/cố định. Điều này tương tự xử lý ETL batch truyền thống, nơi chúng ta có thể chạy một công việc cập nhật dữ liệu mỗi ngày hoặc mỗi giờ. Hệ thống streaming cho phép chúng ta sinh ra các cửa sổ thường xuyên hơn và trả về kết quả với độ trễ thấp hơn. Như chúng ta sẽ liên tục nhấn mạnh, batch là một trường hợp đặc biệt của streaming. Cửa sổ trượt Các sự kiện trong một cửa sổ trượt (sliding window) được phân vào các cửa sổ có độ dài thời gian cố định. Ví dụ, chúng ta có thể sinh ra một cửa sổ 60 giây mới mỗi 30 giây (Hình 8-7). Cũng như trước, chúng ta có thể phát ra thống kê trung bình và trung vị. Cửa sổ trượt cũng được gọi là cửa sổ nhảy (hopping window).

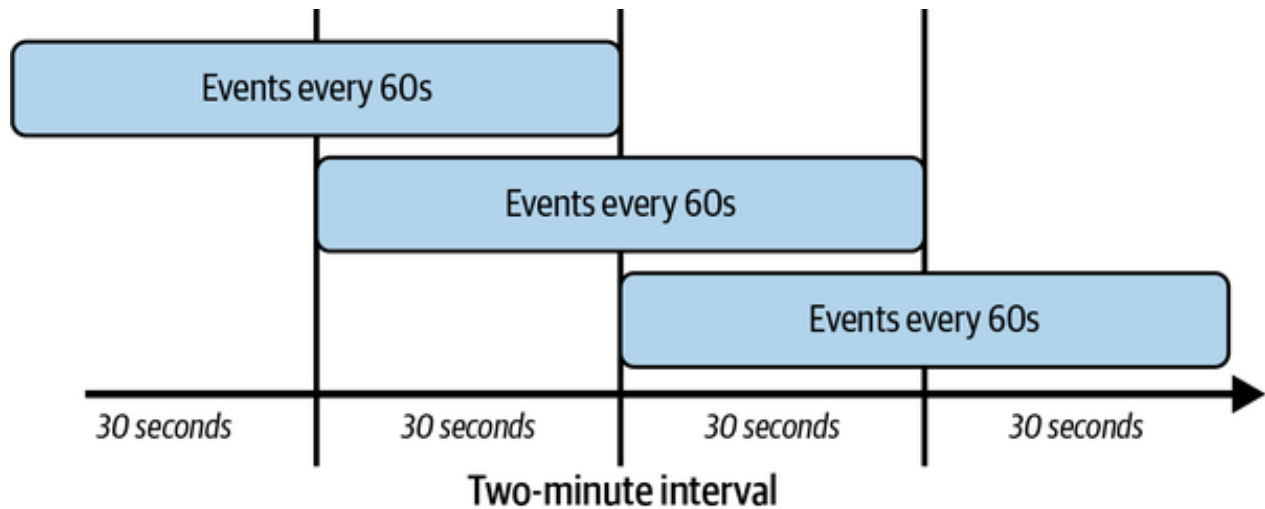


Figure 8-7. Sliding windows The sliding can vary. For example, we might think of the window as truly sliding continuously but emitting statistics only when certain conditions (triggers) are met. Suppose we used a 30-second continuously sliding window but calculated a statistic only when a user clicked a particular banner. This would lead to an extremely high rate of output when many users click the banner, and no calculations during a lull. Watermarks We've covered various types of windows and their uses. As discussed in Chapter 7, data is sometimes ingested out of the order from which it originated. A watermark (Figure 8-8) is a threshold used by a window to determine whether data in a window is within the established time interval or whether it's considered late. If data arrives that is new to the window but older than the timestamp of the watermark, it is considered to be late-arriving data.

Hình 8-7. Cửa sổ trượt. Cách trượt có thể thay đổi. Ví dụ, chúng ta có thể coi cửa sổ là thực sự trượt liên tục nhưng chỉ phát ra thống kê khi một số điều kiện nhất định (trigger) được thỏa. Giả sử chúng ta dùng một cửa sổ trượt liên tục 30 giây nhưng chỉ tính một thống kê khi một người dùng nhấp vào một banner cụ thể. Điều này sẽ dẫn đến một tốc độ đầu ra cực cao khi nhiều người dùng nhấp banner, và không có phép tính nào trong lúc vắng vẻ. Watermark Chúng ta đã đề cập nhiều loại cửa sổ khác nhau cùng cách dùng chúng. Như đã thảo luận ở Chương 7, dữ liệu đôi khi được thu nạp không theo thứ tự mà nó phát sinh. Một watermark (Hình 8-8) là một ngưỡng được một cửa sổ dùng để xác định liệu dữ liệu trong một cửa sổ có nằm trong khoảng thời gian đã thiết lập hay liệu nó bị coi là muộn. Nếu dữ liệu đến mà mới đối với cửa sổ nhưng cũ hơn nhãn thời gian (timestamp) của watermark, nó bị coi là dữ liệu đến muộn.

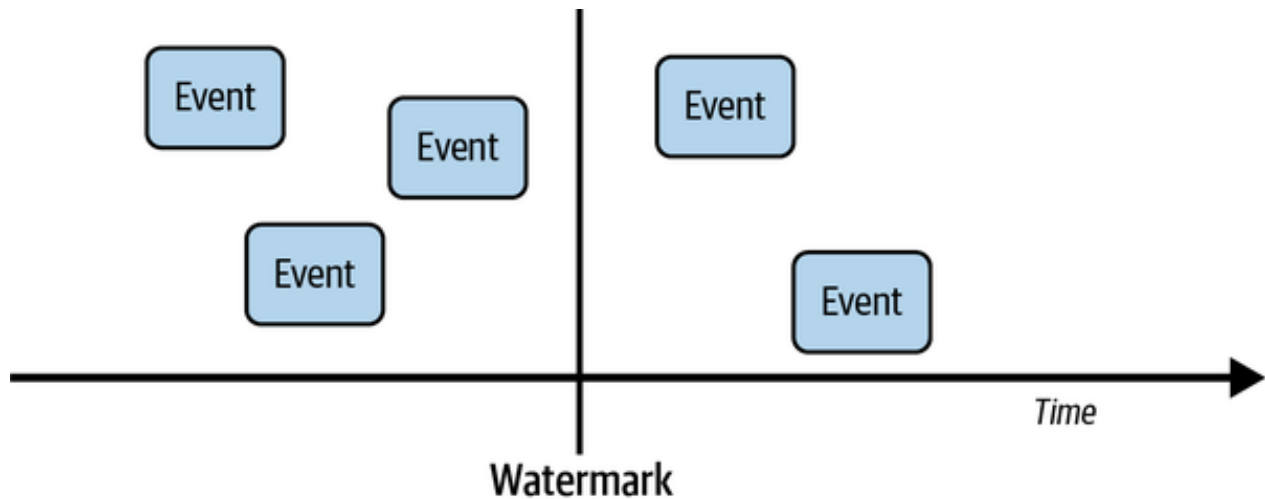


Figure 8-8. Watermark acting as a threshold for late-arriving data Combining streams with other data As we've mentioned before, we often derive value from data by combining it with other data. Streaming data is no different. For instance, multiple streams can be combined, or a stream can be combined with batch historical data. Conventional table joins Some tables may be fed by streams (Figure 8-9). The most basic approach to this problem is simply joining these two tables in a database. A stream can feed one or both of these tables.

Hình 8-8. Watermark đóng vai trò một ngưỡng cho dữ liệu đến muộn. Kết hợp các luồng với dữ liệu khác Như chúng ta đã đề cập trước đây, chúng ta thường rút ra giá trị từ dữ liệu bằng cách kết hợp nó với dữ liệu khác. Dữ liệu streaming cũng không ngoại lệ. Chẳng hạn, nhiều luồng có thể được kết hợp, hoặc một luồng có thể được kết hợp với dữ liệu lịch sử batch. Join bảng thông thường Một số bảng có thể được cấp dữ liệu bởi các luồng (Hình 8-9). Cách tiếp cận cơ bản nhất với vấn đề này đơn giản là join hai bảng đó trong một database. Một luồng có thể cấp dữ liệu cho một hoặc cả hai bảng này.

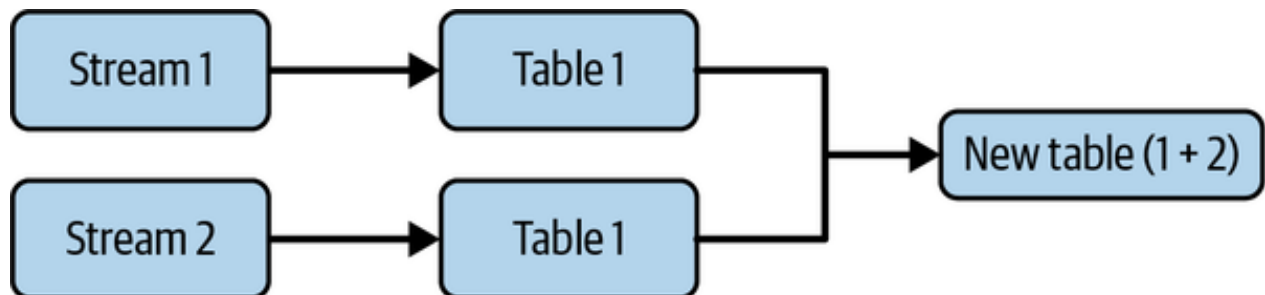


Figure 8-9. Joining two tables fed by streams Enrichment Enrichment means that we join a stream to other data (Figure 8-10). Typically, this is done to provide enhanced data into another stream. For example, suppose that an online retailer receives an event stream from a partner business containing product and user IDs. The retailer wishes to enhance these events with product details and demographic information on the users. The retailer feeds these events to a **serverless** function that looks up the product and user in an in-memory database (say, a cache), adds the required information to the event, and outputs the enhanced events to another stream.

Hình 8-9. Join hai bảng được cấp dữ liệu bởi các luồng. Làm giàu Làm giàu (enrichment) nghĩa là chúng ta join một luồng với dữ liệu khác (Hình 8-10). Thường thì điều này được làm để cung cấp dữ liệu đã được nâng cao vào một luồng khác. Ví dụ, giả sử một nhà bán lẻ trực tuyến nhận một luồng sự kiện từ một doanh nghiệp đối tác chứa các ID sản phẩm

và người dùng. Nhà bán lẻ muốn nâng cao các sự kiện này với chi tiết sản phẩm và thông tin nhân khẩu học về người dùng. Nhà bán lẻ cấp các sự kiện này cho một hàm serverless, hàm này tra cứu sản phẩm và người dùng trong một database trong-bộ-nhớ (chẳng hạn một cache), thêm thông tin cần thiết vào sự kiện, và xuất ra các sự kiện đã được nâng cao tới một luồng khác.

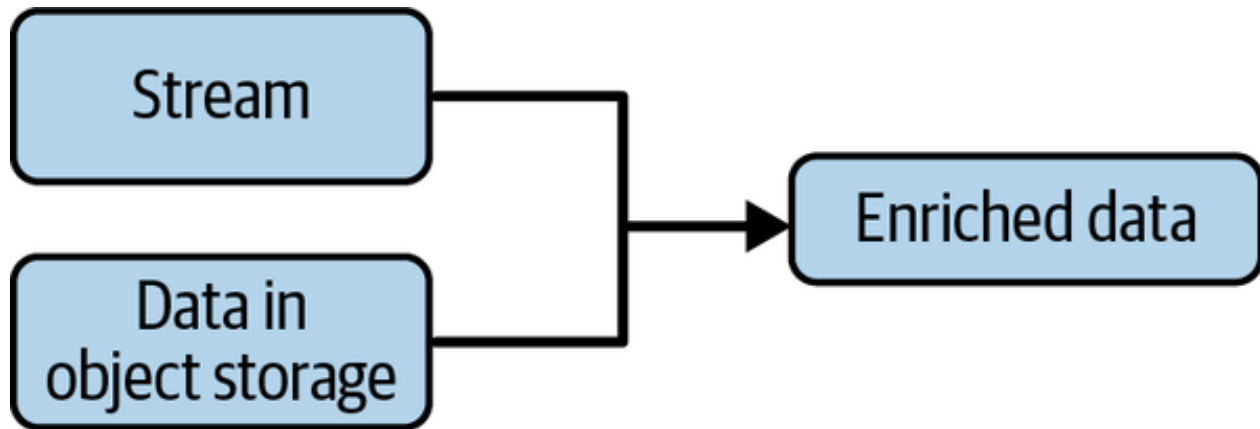


Figure 8-10. In this example, a stream is enriched with data residing in object storage, resulting in a new enriched dataset. In practice, the enrichment source could originate almost anywhere—a table in a cloud data warehouse or RDBMS, or a file in object storage. It's simply a question of reading from the source and storing the requisite enrichment data in an appropriate place for retrieval by the stream. Stream-to-stream joining Increasingly, streaming systems support direct stream-to-stream joining. Suppose that an online retailer wishes to join its web event data with streaming data from an ad platform. The company can feed both streams into Spark, but a variety of complications arise. For instance, the streams may have significantly different latencies for arrival at the point where the join is handled in the streaming system. The ad platform may provide its data with a five-minute delay. In addition, certain events may be significantly delayed—for example, a session close event for a user, or an event that happens on the phone offline and shows up in the stream only after the user is back in mobile network range. As such, typical streaming join architectures rely on streaming buffers. The buffer retention interval is configurable; a longer retention interval requires more storage and other resources. Events get joined with data in the buffer and are eventually evicted after the retention interval has passed (Figure 8-11). 5

Hình 8-10. Trong ví dụ này, một luồng được làm giàu với dữ liệu nằm trong object storage, tạo ra một tập dữ liệu mới đã được làm giàu. Trên thực tế, nguồn làm giàu có thể bắt nguồn từ gần như bất cứ đâu—một bảng trong một cloud data warehouse hoặc RDBMS, hoặc một tệp trong object storage. Đơn giản chỉ là vấn đề đọc từ nguồn và lưu trữ dữ liệu làm giàu cần thiết ở một nơi phù hợp để luồng truy xuất. Join luồng-với-luồng Ngày càng nhiều, các hệ thống streaming hỗ trợ join luồng-với-luồng trực tiếp. Giả sử một nhà bán lẻ trực tuyến muốn join dữ liệu sự kiện web của mình với dữ liệu streaming từ một nền tảng quảng cáo. Công ty có thể cấp cả hai luồng vào Spark, nhưng nhiều phức tạp nảy sinh. Chẳng hạn, các luồng có thể có độ trễ đến rất khác nhau tại điểm join được xử lý trong hệ thống streaming. Nền tảng quảng cáo có thể cung cấp dữ liệu của nó với độ trễ năm phút. Ngoài ra, một số sự kiện nhất định có thể bị trễ đáng kể—ví dụ, một sự kiện đóng phiên cho một người dùng, hoặc một sự kiện xảy ra trên điện thoại khi ngoại tuyến và chỉ xuất hiện trong luồng sau khi người dùng quay lại vùng phủ sóng mạng di động. Như vậy, các kiến trúc join streaming điển hình dựa vào các bộ đệm streaming. Khoảng lưu giữ của bộ đệm có thể cấu hình; một khoảng lưu giữ dài hơn đòi hỏi nhiều lưu trữ và các tài nguyên khác hơn. Các sự kiện được join với dữ liệu trong bộ đệm và cuối cùng bị loại bỏ sau khi

khoảng lưu giữ trôi qua (Hình 8-11).

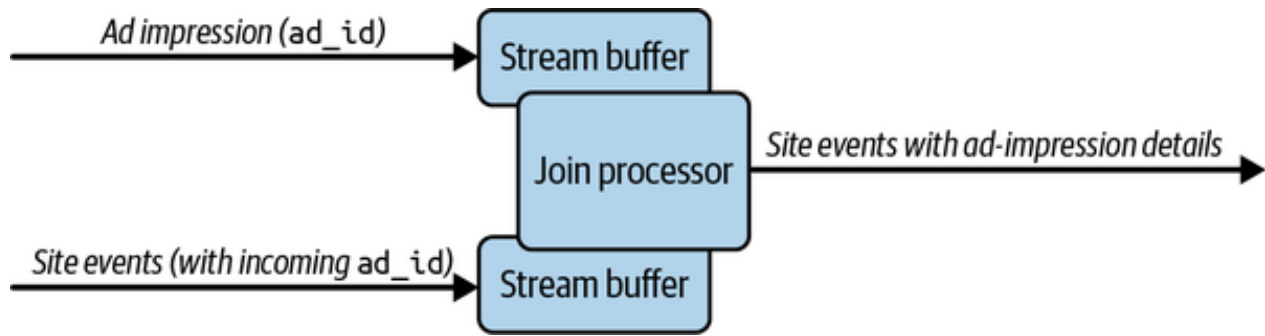


Figure 8-11. An architecture to join streams buffers each stream and joins events if related events are found during the buffer retention interval. Now that we’ve covered how queries work for batch and streaming data, let’s discuss making your data useful by modeling it.

Hình 8-11. Một kiến trúc để join các luồng đệm mỗi luồng và join các sự kiện nếu tìm thấy các sự kiện liên quan trong khoảng lưu giữ của bộ đệm. Giờ đây chúng ta đã trình bày cách các truy vấn hoạt động cho dữ liệu batch và streaming, hãy thảo luận về việc làm cho dữ liệu của bạn hữu ích bằng cách mô hình hóa nó.

Data Modeling — Mô hình hóa Dữ liệu

Data modeling is something that we see overlooked disturbingly often. We often see data teams jump into building data systems without a game plan to organize their data in a way that’s useful for the business. This is a mistake. Well-constructed data architectures must reflect the goals and business logic of the organization that relies on this data. Data modeling involves deliberately choosing a coherent structure for data, and is a critical step to make data useful for the business. Data modeling has been a practice for decades in one form or another. For example, various types of normalization techniques (discussed in “Normalization”) have been used to model data since the early days of RDBMSs; data warehousing modeling techniques have been around since at least the early 1990s, and arguably longer. As pendulums in technology often go, data modeling became somewhat unfashionable in the early to mid-2010s. The rise of data lake 1.0, NoSQL, and big data systems allowed engineers to bypass traditional data modeling, sometimes for legitimate performance gains. Other times, the lack of rigorous data modeling created data swamps, along with lots of redundant, mismatched, or simply wrong data. Nowadays, the pendulum seems to be swinging back toward data modeling. The growing popularity of data management (in particular, **data governance** and **data quality**) is pushing the need for coherent business logic. The meteoric rise of data’s prominence in companies creates a growing recognition that modeling is critical for realizing value at the higher levels of the Data Science Hierarchy of Needs pyramid. On the other hand, we believe that new paradigms are required to truly embrace the needs of streaming data and ML. In this section, we survey current data modeling techniques and briefly muse on the future of data modeling.

Mô hình hóa dữ liệu là thứ mà chúng tôi thấy bị bỏ qua đáng lo ngại một cách thường xuyên. Chúng tôi thường thấy các nhóm dữ liệu lao vào xây dựng các hệ thống dữ liệu mà không có kế hoạch tổ chức dữ liệu của họ theo cách hữu ích cho doanh nghiệp. Đây là một sai lầm. Các kiến trúc dữ liệu được xây dựng tốt phải phản ánh các mục tiêu và logic nghiệp vụ của tổ chức dựa vào dữ liệu này. Mô hình hóa dữ liệu liên quan đến việc lựa chọn có chủ ý một cấu trúc mạch lạc cho dữ liệu, và là một bước tối quan trọng để làm cho dữ liệu hữu ích cho doanh nghiệp. Mô hình hóa dữ liệu đã là một thực hành trong

nhiều thập kỷ ở dạng này hay dạng khác. Ví dụ, nhiều loại kỹ thuật chuẩn hóa khác nhau (thảo luận ở “Chuẩn hóa”) đã được dùng để mô hình hóa dữ liệu từ những ngày đầu của RDBMS; các kỹ thuật mô hình hóa data warehouse đã tồn tại ít nhất từ đầu những năm 1990, và có lẽ còn lâu hơn. Như con lắc trong công nghệ thường vận động, mô hình hóa dữ liệu trở nên phần nào lỗi mốt vào đầu đến giữa thập niên 2010. Sự trỗi dậy của data lake 1.0, NoSQL, và các hệ thống big data cho phép kỹ sư bỏ qua mô hình hóa dữ liệu truyền thống, đôi khi vì những lợi ích hiệu năng chính đáng. Những lúc khác, sự thiếu vắng mô hình hóa dữ liệu nghiêm ngặt tạo ra các đầm lầy dữ liệu (data swamp), cùng với rất nhiều dữ liệu dư thừa, lộn xộn, hoặc đơn giản là sai. Ngày nay, con lắc dường như đang đu trở lại về phía mô hình hóa dữ liệu. Sự phổ biến ngày càng tăng của quản lý dữ liệu (cụ thể là quản trị dữ liệu và chất lượng dữ liệu) đang thúc đẩy nhu cầu về logic nghiệp vụ mạch lạc. Sự trỗi dậy chóng mặt của vị thế nổi bật của dữ liệu trong các công ty tạo ra sự thừa nhận ngày càng tăng rằng mô hình hóa là tối quan trọng để hiện thực hóa giá trị ở các tầng cao hơn của kim tự tháp Hệ thống Phân cấp Nhu cầu Khoa học Dữ liệu (Data Science Hierarchy of Needs). Mặt khác, chúng tôi tin rằng cần có những khuôn mẫu mới để thực sự đáp ứng nhu cầu của dữ liệu streaming và ML. Trong phần này, chúng tôi khảo sát các kỹ thuật mô hình hóa dữ liệu hiện hành và suy ngẫm ngắn gọn về tương lai của mô hình hóa dữ liệu.

What Is a Data Model? A data model represents the way data relates to the real world. It reflects how the data must be structured and standardized to best reflect your organization’s processes, definitions, workflows, and logic. A good data model captures how communication and work naturally flow within your organization. In contrast, a poor data model (or nonexistent one) is haphazard, confusing, and incoherent. Some data professionals view data modeling as tedious and reserved for “big enterprises.” Like most good hygiene practices—such as flossing your teeth and getting a good night’s sleep—data modeling is acknowledged as a good thing to do but is often ignored in practice. Ideally, every organization should model its data if only to ensure that business logic and rules are translated at the data layer. When modeling data, it’s critical to focus on translating the model to business outcomes. A good data model should correlate with impactful business decisions. For example, a customer might mean different things to different departments in a company. Is someone who’s bought from you over the last 30 days a customer? What if they haven’t bought from you in the previous six months or a year? Carefully defining and modeling this customer data can have a massive impact on downstream reports on customer behavior or the creation of customer churn models whereby the time since the last purchase is a critical variable.

Mô hình dữ liệu là gì? Một mô hình dữ liệu biểu diễn cách dữ liệu liên hệ với thế giới thực. Nó phản ánh cách dữ liệu phải được cấu trúc và chuẩn hóa để phản ánh tốt nhất các quy trình, định nghĩa, luồng công việc, và logic của tổ chức bạn. Một mô hình dữ liệu tốt nắm bắt cách giao tiếp và công việc tự nhiên chảy trong tổ chức của bạn. Ngược lại, một mô hình dữ liệu tồi (hoặc không tồn tại) thì lộn xộn, gây bối rối, và thiếu mạch lạc. Một số chuyên gia dữ liệu xem mô hình hóa dữ liệu là việc nhàm chán và chỉ dành cho “các doanh nghiệp lớn”. Như hầu hết các thực hành vệ sinh tốt—chẳng hạn rửa răng và ngủ ngon mỗi đêm—mô hình hóa dữ liệu được thừa nhận là việc nên làm nhưng thường bị phớt lờ trong thực tế. Lý tưởng nhất, mọi tổ chức nên mô hình hóa dữ liệu của mình, dù chỉ để đảm bảo rằng logic và các quy tắc nghiệp vụ được chuyển dịch ở tầng dữ liệu. Khi mô hình hóa dữ liệu, điều tối quan trọng là tập trung vào việc chuyển dịch mô hình thành các kết quả nghiệp vụ. Một mô hình dữ liệu tốt nên tương quan với các quyết định nghiệp vụ có sức ảnh hưởng. Ví dụ, một khách hàng (customer) có thể mang ý nghĩa khác nhau đối với các phòng ban khác nhau trong một công ty. Liệu một người đã mua hàng của bạn trong 30 ngày qua có phải là khách hàng? Sẽ thế nào nếu họ chưa mua gì của bạn trong sáu tháng hoặc một năm trước? Định nghĩa và mô hình hóa cẩn thận dữ liệu khách hàng

này có thể có tác động khổng lồ lên các báo cáo hạ nguồn về hành vi khách hàng hoặc việc tạo các mô hình rời bỏ khách hàng (churn) trong đó thời gian kể từ lần mua cuối là một biến tối quan trọng.

TIP Can you think of concepts or terms in your company that might mean different things to different people?

MỆO Bạn có thể nghĩ ra các khái niệm hay thuật ngữ nào trong công ty mình mà có thể mang ý nghĩa khác nhau đối với những người khác nhau không?

Our discussion focuses mainly on batch data modeling since that's where most data modeling techniques arose. We will also look at some approaches to modeling streaming data and general considerations for modeling.

Thảo luận của chúng ta chủ yếu tập trung vào mô hình hóa dữ liệu batch vì đó là nơi hầu hết các kỹ thuật mô hình hóa dữ liệu phát sinh. Chúng ta cũng sẽ xem xét một số cách tiếp cận để mô hình hóa dữ liệu streaming và các cân nhắc chung cho việc mô hình hóa.

Conceptual, Logical, and Physical Data Models When modeling data, the idea is to move from abstract modeling concepts to concrete implementation. Along this continuum (Figure 8-12), three main data models are conceptual, logical, and physical. These models form the basis for the various modeling techniques we describe in this chapter: Conceptual Contains business logic and rules and describes the system's data, such as schemas, tables, and fields (names and types). When creating a conceptual model, it's often helpful to visualize it in an entity-relationship (ER) diagram, which is a standard tool for visualizing the relationships among various entities in your data (orders, customers, products, etc.). For example, an ER diagram might encode the connections among customer ID, customer name, customer address, and customer orders. Visualizing entity relationships is highly recommended for designing a coherent conceptual data model. Logical Details how the conceptual model will be implemented in practice by adding significantly more detail. For example, we would add information on the types of customer ID, customer names, and customer addresses. In addition, we would map out primary and foreign keys. Physical Defines how the logical model will be implemented in a database system. We would add specific databases, schemas, and tables to our logical model, including configuration details.

Mô hình Dữ liệu Khái niệm, Logic, và Vật lý Khi mô hình hóa dữ liệu, ý tưởng là di chuyển từ các khái niệm mô hình hóa trừu tượng đến hiện thực hóa cụ thể. Dọc theo trục liên tục này (Hình 8-12), ba mô hình dữ liệu chính là khái niệm (conceptual), logic (logical), và vật lý (physical). Các mô hình này tạo nền tảng cho các kỹ thuật mô hình hóa khác nhau mà chúng tôi mô tả trong chương này: Khái niệm Chứa logic và quy tắc nghiệp vụ và mô tả dữ liệu của hệ thống, như các schema, bảng, và trường (tên và kiểu). Khi tạo một mô hình khái niệm, thường hữu ích khi trực quan hóa nó trong một sơ đồ thực thể-quan hệ (entity-relationship, ER), vốn là một công cụ chuẩn để trực quan hóa các quan hệ giữa các thực thể khác nhau trong dữ liệu của bạn (đơn hàng, khách hàng, sản phẩm, v.v.). Ví dụ, một sơ đồ ER có thể mã hóa các kết nối giữa ID khách hàng, tên khách hàng, địa chỉ khách hàng, và các đơn hàng của khách hàng. Trực quan hóa các quan hệ thực thể được khuyến nghị mạnh mẽ để thiết kế một mô hình dữ liệu khái niệm mạch lạc. Logic Chi tiết hóa cách mô hình khái niệm sẽ được hiện thực hóa trong thực tế bằng cách thêm nhiều chi tiết hơn đáng kể. Ví dụ, chúng ta sẽ thêm thông tin về các kiểu của ID khách hàng, tên khách hàng, và địa chỉ khách hàng. Ngoài ra, chúng ta sẽ phác họa các khóa chính và khóa ngoại. Vật lý Định nghĩa cách mô hình logic sẽ được hiện thực hóa trong một hệ thống database. Chúng ta sẽ thêm các database, schema, và bảng cụ thể vào mô hình logic của mình, bao gồm cả các chi tiết cấu hình.

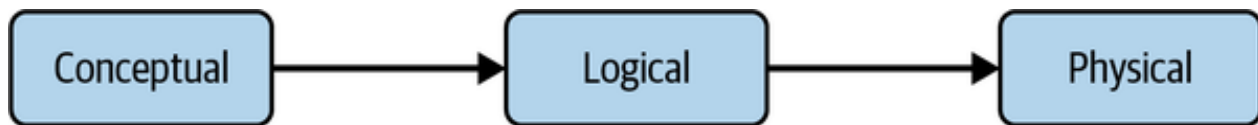


Figure 8-12. The continuum of data models: conceptual, logical, and physical Successful data modeling involves business stakeholders at the inception of the process. Engineers need to obtain definitions and business goals for the data. Modeling data should be a full-contact sport whose goal is to provide the business with quality data for actionable insights and automation. This is a practice that everyone must continuously participate in. Another important consideration for data modeling is the **grain** of the data, which is the resolution at which data is stored and queried. The grain is typically at the level of a primary key in a table, such as customer ID, order ID, and product ID; it's often accompanied by a date or timestamp for increased fidelity. For example, suppose that a company has just begun to deploy BI reporting. The company is small enough that the same person is filling the role of data engineer and analyst. A request comes in for a report that summarizes daily customer orders. Specifically, the report should list all customers who ordered, the number of orders they placed that day, and the total amount they spent. This report is inherently coarse-grained. It contains no details on spending per order or the items in each order. It is tempting for the data engineer/analyst to ingest data from the production orders database and boil it down to a reporting table with only the basic aggregated data required for the report. However, this would entail starting over when a request comes in for a report with finer-grained data aggregation. Since the data engineer is actually quite experienced, they elect to create tables with detailed data on customer orders, including each order, item, item cost, item IDs, etc. Essentially, their tables contain all details on customer orders. The data's grain is at the customer-order level. This customer-order data can be analyzed as is, or aggregated for summary statistics on customer order activity. In general, you should strive to model your data at the lowest level of grain possible. From here, it's easy to aggregate this highly granular dataset. The reverse isn't true, and it's generally impossible to restore details that have been aggregated away.

Hình 8-12. Trục liên tục của các mô hình dữ liệu: khái niệm, logic, và vật lý. Mô hình hóa dữ liệu thành công đòi hỏi sự tham gia của các bên liên quan nghiệp vụ ngay từ khi khởi đầu quá trình. Kỹ sư cần thu được các định nghĩa và mục tiêu nghiệp vụ cho dữ liệu. Mô hình hóa dữ liệu nên là một môn thể thao đối kháng toàn diện mà mục tiêu là cung cấp cho doanh nghiệp dữ liệu chất lượng cho các thông tin chuyên sâu khả thi và tự động hóa. Đây là một thực hành mà mọi người phải liên tục tham gia. Một cân nhắc quan trọng khác cho mô hình hóa dữ liệu là độ hạt (grain) của dữ liệu, tức độ phân giải tại đó dữ liệu được lưu trữ và truy vấn. Độ hạt thường ở mức một khóa chính trong một bảng, như ID khách hàng, ID đơn hàng, và ID sản phẩm; nó thường được đi kèm một ngày hoặc nhãn thời gian để tăng độ trung thực. Ví dụ, giả sử một công ty vừa bắt đầu triển khai báo cáo BI. Công ty đủ nhỏ để cùng một người đảm nhận vai trò kỹ sư dữ liệu và nhà phân tích. Một yêu cầu đến cho một báo cáo tóm tắt các đơn hàng khách hàng hằng ngày. Cụ thể, báo cáo nên liệt kê tất cả các khách hàng đã đặt hàng, số đơn họ đặt trong ngày đó, và tổng số tiền họ chi. Báo cáo này về bản chất là độ-hạt-thô. Nó không chứa chi tiết về chi tiêu trên mỗi đơn hàng hoặc các món hàng trong mỗi đơn. Sẽ rất cảm ơn với kỹ sư dữ liệu/nhà phân tích khi thu nạp dữ liệu từ database đơn hàng sản xuất và cô đọng nó thành một bảng báo cáo chỉ với dữ liệu tổng hợp cơ bản cần cho báo cáo. Tuy nhiên, điều này sẽ kéo theo việc phải bắt đầu lại từ đầu khi một yêu cầu đến cho một báo cáo với tổng hợp dữ liệu độ-hạt-mịn hơn. Vì kỹ sư dữ liệu thực ra khá giàu kinh nghiệm, họ chọn tạo các bảng với dữ liệu chi tiết về các đơn hàng khách hàng, bao gồm mỗi đơn hàng, mỗi món hàng, chi phí món hàng, ID món hàng, v.v. Về cơ bản, các bảng của họ chứa mọi chi tiết về các đơn hàng khách hàng. Độ hạt của dữ liệu ở mức khách-hàng-đơn-hàng. Dữ liệu khách-hàng-đơn-hàng này có thể được phân tích nguyên trạng, hoặc được tổng hợp cho các thống kê tóm tắt về hoạt động

đặt hàng của khách hàng. Nhìn chung, bạn nên cố gắng mô hình hóa dữ liệu của mình ở mức độ hạt thấp nhất có thể. Từ đó, việc tổng hợp tập dữ liệu độ-hạt-cao này lên là dễ dàng. Điều ngược lại không đúng, và nói chung là không thể khôi phục các chi tiết đã bị tổng hợp đi mất.

Normalization Normalization is a database data modeling practice that enforces strict control over the relationships of tables and columns within a database. The goal of normalization is to remove the redundancy of data within a database and ensure referential integrity. Basically, it's don't repeat yourself (DRY) applied to data in a database. 6 Normalization is typically applied to relational databases containing tables with rows and columns (we use the terms column and field interchangeably in this section). It was first introduced by relational database pioneer Edgar Codd in the early 1970s.

Chuẩn hóa Chuẩn hóa (normalization) là một thực hành mô hình hóa dữ liệu database thực thi kiểm soát chặt chẽ các quan hệ giữa các bảng và cột trong một database. Mục tiêu của chuẩn hóa là loại bỏ sự dư thừa của dữ liệu trong một database và đảm bảo toàn vẹn tham chiếu. Về cơ bản, đó là nguyên tắc đừng-lặp-lại-chính-mình (don't repeat yourself, DRY) áp dụng vào dữ liệu trong một database. Chuẩn hóa thường được áp dụng cho các database quan hệ chứa các bảng với các hàng và cột (chúng ta dùng các thuật ngữ cột và trường thay thế cho nhau trong phần này). Nó lần đầu được giới thiệu bởi nhà tiên phong về database quan hệ Edgar Codd vào đầu thập niên 1970.

Codd outlined four main objectives of normalization: 7 To free the collection of relations from undesirable insertion, update, and deletion dependencies To reduce the need for restructuring the collection of relations, as new types of data are introduced, and thus increase the lifespan of application programs To make the relational model more informative to users To make the collection of relations neutral to the query statistics, where these statistics are liable to change as time goes by Codd introduced the idea of normal forms . The normal forms are sequential, with each form incorporating the conditions of prior forms. We describe Codd's first three normal forms here: Denormalized No normalization. Nested and redundant data is allowed. First normal form (1NF) Each column is unique and has a single value. The table has a unique primary key. Second normal form (2NF) The requirements of 1NF, plus partial dependencies are removed. Third normal form (3NF) The requirements of 2NF, plus each table contains only relevant fields related to its primary key and has no transitive dependencies.

Codd phác họa bốn mục tiêu chính của chuẩn hóa: Giải phóng tập các quan hệ khỏi các phụ thuộc chèn, cập nhật, và xóa không mong muốn. Giảm nhu cầu tái cấu trúc tập các quan hệ khi các loại dữ liệu mới được đưa vào, từ đó tăng tuổi thọ của các chương trình ứng dụng. Làm cho mô hình quan hệ giàu thông tin hơn cho người dùng. Làm cho tập các quan hệ trung lập với các thống kê truy vấn, vốn là những thống kê dễ thay đổi theo thời gian. Codd giới thiệu ý tưởng về các dạng chuẩn (normal form). Các dạng chuẩn là tuần tự, mỗi dạng bao hàm các điều kiện của các dạng trước. Chúng tôi mô tả ba dạng chuẩn đầu của Codd ở đây: Phi chuẩn hóa Không chuẩn hóa. Dữ liệu lồng nhau và dư thừa được cho phép. Dạng chuẩn thứ nhất (1NF) Mỗi cột là duy nhất và có một giá trị đơn. Bảng có một khóa chính duy nhất. Dạng chuẩn thứ hai (2NF) Các yêu cầu của 1NF, cộng thêm việc loại bỏ các phụ thuộc bộ phận. Dạng chuẩn thứ ba (3NF) Các yêu cầu của 2NF, cộng thêm việc mỗi bảng chỉ chứa các trường liên quan đến khóa chính của nó và không có phụ thuộc bắc cầu.

It's worth spending a moment to unpack a couple of terms we just threw at you. A unique primary key is a single field or set of multiple fields that uniquely determines rows in the table. Each key value occurs at most once; otherwise, a value would map to multiple rows in the table. Thus, every other value in a row is dependent on (can be determined from) the key. A partial dependency occurs

when a subset of fields in a composite key can be used to determine a nonkey column of the table. A transitive dependency occurs when a nonkey field depends on another nonkey field. Let's look at stages of normalization—from denormalized to 3NF—using an ecommerce example of customer orders (Table 8-1). We'll provide concrete explanations of each of the concepts introduced in the previous paragraph.

Đáng để dành một chút thời gian giải nén một vài thuật ngữ mà chúng tôi vừa ném ra cho bạn. Một khóa chính duy nhất là một trường đơn hoặc một tập nhiều trường xác định duy nhất các hàng trong bảng. Mỗi giá trị khóa xuất hiện nhiều nhất một lần; nếu không, một giá trị sẽ ánh xạ tới nhiều hàng trong bảng. Như vậy, mọi giá trị khác trong một hàng đều phụ thuộc vào (có thể được xác định từ) khóa. Một phụ thuộc bộ phận (partial dependency) xảy ra khi một tập con các trường trong một khóa hỗn hợp có thể được dùng để xác định một cột không-khóa của bảng. Một phụ thuộc bắc cầu (transitive dependency) xảy ra khi một trường không-khóa phụ thuộc vào một trường không-khóa khác. Hãy cùng xem các giai đoạn chuẩn hóa—từ phi chuẩn hóa đến 3NF—dùng một ví dụ thương mại điện tử về các đơn hàng khách hàng (Bảng 8-1). Chúng tôi sẽ cung cấp các giải thích cụ thể cho mỗi khái niệm được giới thiệu ở đoạn trước.

Table 8-1. OrderDetail

Bảng 8-1. Chi tiết Đơn hàng

OrderID	OrderItems	CustomerID	CustomerName	OrderDate
100	5	Joe Reis	2022-03-01	

```
[{
  "sku":1,
  "price":50,
  "quantity":1,
  "name":"Thingamajig"
},{
  "sku":2,
  "price":25,
  "quantity":2,
  "name":"Whatchamacallit"
}]
```

First, this denormalized OrderDetail table contains five fields. The primary key is OrderID . Notice that the OrderItems field contains a nested object with two SKUs along with their price, quantity, and name. To convert this data to 1NF, let's move OrderItems into four fields (Table 8-2). Now we have an OrderDetail table in which fields do not contain repeats or nested data.

Đầu tiên, bảng OrderDetail phi chuẩn hóa này chứa năm trường. Khóa chính là OrderID. Lưu ý rằng trường OrderItems chứa một đối tượng lồng nhau với hai SKU cùng giá, số lượng, và tên của chúng. Để chuyển dữ liệu này sang 1NF, hãy chuyển OrderItems thành bốn trường (Bảng 8-2). Bây giờ chúng ta có một bảng OrderDetail trong đó các trường không chứa dữ liệu lặp hoặc lồng nhau.

Table 8-2. OrderDetail without repeats or nested data

Bảng 8-2. OrderDetail không có dữ liệu lặp hoặc lồng nhau

OrderID	Sku	Price	Quantity	ProductName	CustomerID	CustomerName	Ord
100	1	50	1	Thingamajig	5	Joe Reis	202
100	2	25	2	Whatchamacallit	5	Joe Reis	202

The problem is that now we don't have a unique primary key. That is, 100 occurs in the OrderID column in two different rows. To get a better grasp of the situation, let's look at a larger sample from our table (Table 8-3).

Vấn đề là giờ đây chúng ta không có một khóa chính duy nhất. Nghĩa là, 100 xuất hiện trong cột OrderID ở hai hàng khác nhau. Để nắm rõ hơn tình huống, hãy xem một mẫu lớn hơn từ bảng của chúng ta (Bảng 8-3).

Table 8-3. OrderDetail with a larger sample

Bảng 8-3. OrderDetail với một mẫu lớn hơn

OrderID	Sku	Price	Quantity	ProductName	CustomerID	CustomerName
100	1	50	1	Thingamajig	5	Joe Reis
202	100	2	25	2 Whatchamacallit	5	Joe Reis
202	101	3	75	1 Whozeewhatzit	7	Matt Housley
202	102	1	50	1 Thingamajig	7	Matt Housley

To create a unique primary (composite) key, let's number the lines in each order by adding a column called LineItemNumber (Table 8-4).

Để tạo một khóa chính (hỗn hợp) duy nhất, hãy đánh số các dòng trong mỗi đơn hàng bằng cách thêm một cột gọi là LineItemNumber (Bảng 8-4).

Table 8-4. OrderDetail with LineItemNumber column

Bảng 8-4. OrderDetail với cột LineItemNumber

OrderID	LineItemNumber	Sku	Price	Quantity	ProductName	CustomerID	Cus
100	1	1	50	1	Thingamajig	5	Joe
100	2	2	25	2	Whatchamacallit	5	Joe
101	1	3	75	1	Whozeewhatzit	7	Mat
102	1	1	50	1	Thingamajig	7	Mat

The composite key (OrderID , LineItemNumber) is now a unique primary key. To reach 2NF, we need to ensure that no partial dependencies exist. A partial dependency is a nonkey column that is fully determined by a subset of the columns in the unique primary (composite) key; partial dependencies can occur only when the primary key is composite. In our case, the last three columns are determined by order number. To fix this problem, let's split OrderDetail into two tables: Orders and OrderLineItem (Tables 8-5 and 8-6).

Khóa hỗn hợp (OrderID, LineItemNumber) giờ đây là một khóa chính duy nhất. Để đạt 2NF, chúng ta cần đảm bảo không tồn tại phụ thuộc bộ phận nào. Một phụ thuộc bộ phận là một cột không-khóa được xác định hoàn toàn bởi một tập con các cột trong khóa chính (hỗn hợp) duy nhất; các phụ thuộc bộ phận chỉ có thể xảy ra khi khóa chính là hỗn hợp. Trong trường hợp của chúng ta, ba cột cuối được xác định bởi số đơn hàng. Để khắc phục vấn đề này, hãy chia OrderDetail thành hai bảng: Orders và OrderLineItem (Bảng 8-5 và 8-6).

Table 8-5. Orders

Bảng 8-5. Orders

OrderID	CustomerID	CustomerName	OrderDate
100	5	Joe Reis	2022-03-01
101	7	Matt Housley	2022-03-01
102	7	Matt Housley	2022-03-01

Bảng 8-6. OrderLineItem

OrderID	LineItemNumber	Sku	Price	Quantity	ProductName
100	1	1	50	1	Thingamajig
100	2	2	25	2	Whatchamacallit
101	1	3	75	1	Whozeewhatzit
102	1	1	50	1	Thingamajig

The composite key (OrderID , LineItemNumber) is a unique primary key for OrderLineItem , while OrderID is a primary key for Orders . Notice that Sku determines ProductName in OrderLineItem . That is, Sku depends on the composite key, and ProductName depends on Sku . This is a transitive dependency. Let's break OrderLineItem into OrderLineItem and Skus (Tables 8-7 and 8-8).

Khóa hỗn hợp (OrderID, LineItemNumber) là một khóa chính duy nhất cho OrderLineItem, trong khi OrderID là một khóa chính cho Orders. Lưu ý rằng Sku xác định ProductName trong OrderLineItem. Nghĩa là, Sku phụ thuộc vào khóa hỗn hợp, và ProductName phụ thuộc vào Sku. Đây là một phụ thuộc bắc cầu. Hãy chia OrderLineItem thành OrderLineItem và Skus (Bảng 8-7 và 8-8).

Table 8-7. OrderLineItem

Bảng 8-7. OrderLineItem

OrderID LineItemNumber Sku Price Quantity 100 1 1 50 1 100 2 2 25 2 101 1 3 75 1 102 1 1 50 1 Table 8-8. Skus

Bảng 8-8. Skus

Sku ProductName 1 Thingamajig 2 Whatchamacallit 3 Whozeewhatzit

Now, both OrderLineItem and Skus are in 3NF. Notice that Orders does not satisfy 3NF. What transitive dependencies are present? How would you fix this? Additional normal forms exist (up to 6NF in the Boyce-Codd system), but these are much less common than the first three. A database is usually considered normalized if it's in third normal form, and that's the convention we use in this book. The degree of normalization that you should apply to your data depends on your use case. No one-size-fits-all solution exists, especially in databases where some **denormalization** presents performance advantages. Although denormalization may seem like an antipattern, it's common in many OLAP systems that store semistructured data. Study normalization conventions and database best practices to choose an appropriate strategy.

Bây giờ, cả OrderLineItem và Skus đều ở 3NF. Lưu ý rằng Orders không thỏa mãn 3NF. Có những phụ thuộc bắc cầu nào hiện diện? Bạn sẽ khắc phục điều này thế nào? Tồn tại các dạng chuẩn bổ sung (lên đến 6NF trong hệ thống Boyce-Codd), nhưng chúng ít phổ biến hơn nhiều so với ba dạng đầu. Một database thường được coi là đã chuẩn hóa nếu nó ở dạng chuẩn thứ ba, và đó là quy ước chúng tôi dùng trong cuốn sách này. Mức độ chuẩn hóa mà bạn nên áp dụng cho dữ liệu của mình phụ thuộc vào trường hợp sử dụng của bạn. Không có giải pháp một-cỡ-vừa-tất-cả nào tồn tại, đặc biệt trong các database nơi một mức phi chuẩn hóa nào đó mang lại lợi thế hiệu năng. Mặc dù phi chuẩn hóa có vẻ như một phản-khuôn-mẫu, nó phổ biến trong nhiều hệ thống OLAP lưu trữ dữ liệu bán cấu trúc. Hãy nghiên cứu các quy ước chuẩn hóa và các thực hành tốt nhất về database để chọn một chiến lược phù hợp.

Techniques for Modeling Batch Analytical Data When describing data modeling for data lakes or data warehouses, you should assume that the raw data takes many forms (e.g., structured and semistructured), but the output is a structured data model of rows and columns. However, several approaches to data modeling can be used in these environments. The big approaches you'll likely encounter are **Kimball**, **Inmon**, and **data vault**. In practice, some of these techniques can be combined. For example, we see some data teams start with data vault and then add a Kimball **star schema** alongside it. We'll also look at wide and denormalized data models and other batch

data-modeling techniques you should have in your arsenal. As we discuss each of these techniques, we will use the example of modeling transactions occurring in an ecommerce order system.

Các Kỹ thuật để Mô hình hóa Dữ liệu Phân tích Batch Khi mô tả mô hình hóa dữ liệu cho data lake hoặc data warehouse, bạn nên giả định rằng dữ liệu thô mang nhiều dạng (ví dụ có cấu trúc và bán cấu trúc), nhưng đầu ra là một mô hình dữ liệu có cấu trúc gồm các hàng và cột. Tuy nhiên, có một vài cách tiếp cận để mô hình hóa dữ liệu có thể dùng trong các môi trường này. Các cách tiếp cận lớn mà bạn nhiều khả năng sẽ gặp là Kimball, Inmon, và Data Vault. Trên thực tế, một số kỹ thuật này có thể được kết hợp. Ví dụ, chúng tôi thấy một số nhóm dữ liệu bắt đầu với Data Vault rồi sau đó thêm một lược đồ sao (star schema) Kimball bên cạnh nó. Chúng ta cũng sẽ xem các mô hình dữ liệu bảng rộng (wide table) và phi chuẩn hóa cùng các kỹ thuật mô hình hóa dữ liệu batch khác mà bạn nên có trong kho vũ khí của mình. Khi thảo luận từng kỹ thuật này, chúng ta sẽ dùng ví dụ về mô hình hóa các giao dịch xảy ra trong một hệ thống đơn hàng thương mại điện tử.

NOTE Our coverage of the first three approaches—Inmon, Kimball, and data vault—is cursory and hardly does justice to their respective complexity and nuance. At the end of each section, we list the canonical books from their creators. For a data engineer, these books are must-reads, and we highly encourage you to read them, if only to understand how and why data modeling is central to batch analytical data.

LƯU Ý Phần trình bày của chúng tôi về ba cách tiếp cận đầu—Inmon, Kimball, và Data Vault—là sơ lược và khó mà công bằng với độ phức tạp và sắc thái tương ứng của chúng. Ở cuối mỗi mục, chúng tôi liệt kê các cuốn sách kinh điển từ những người tạo ra chúng. Đối với một kỹ sư dữ liệu, các cuốn sách này là phải-đọc, và chúng tôi rất khuyến khích bạn đọc chúng, dù chỉ để hiểu mô hình hóa dữ liệu mang tính trung tâm như thế nào và vì sao đối với dữ liệu phân tích batch.

Inmon The father of the data warehouse, Bill Inmon, created his approach to data modeling in 1990. Before the data warehouse, the analysis would often occur directly on the **source system** itself, with the obvious consequence of bogging down production transactional databases with long-running queries. The goal of the data warehouse was to separate the source system from the analytical system. Inmon defines a data warehouse the following way: A data warehouse is a subject-oriented, integrated, nonvolatile, and time-variant collection of data in support of management's decisions. The data warehouse contains granular corporate data. Data in the data warehouse is able to be used for many different purposes, including sitting and waiting for future requirements which are unknown today. 8 The four critical parts of a data warehouse mean the following: Subject-oriented The data warehouse focuses on a specific subject area, such as sales or marketing. Nonvolatile Data remains unchanged after data is stored in a data warehouse. Integrated Data from disparate sources is consolidated and normalized. Time-variant Varying time ranges can be queried.

Inmon Cha đẻ của data warehouse, Bill Inmon, đã tạo ra cách tiếp cận của ông với mô hình hóa dữ liệu vào năm 1990. Trước khi có data warehouse, việc phân tích thường diễn ra trực tiếp trên chính hệ thống nguồn, với hệ quả hiển nhiên là làm sa lầy các database giao dịch sản xuất với các truy vấn chạy lâu. Mục tiêu của data warehouse là tách hệ thống nguồn khỏi hệ thống phân tích. Inmon định nghĩa một data warehouse theo cách sau: Một data warehouse là một tập hợp dữ liệu định hướng-chủ-đề, tích hợp, bất biến, và biến-thiên-theo-thời-gian nhằm hỗ trợ các quyết định của ban quản lý. Data warehouse chứa dữ liệu doanh nghiệp ở mức độ hạt mịn. Dữ liệu trong data warehouse có thể được dùng cho nhiều mục đích khác nhau, bao gồm cả việc nằm chờ các yêu cầu tương lai chưa biết đến hôm nay. Bốn phần tối quan trọng của một data warehouse mang ý nghĩa sau: Định hướng-chủ-đề Data warehouse tập trung vào một lĩnh vực chủ đề cụ thể, như bán hàng hoặc tiếp thị. Bất biến Dữ liệu giữ nguyên không đổi sau khi dữ liệu được lưu trong

một data warehouse. Tích hợp Dữ liệu từ các nguồn rời rạc được hợp nhất và chuẩn hóa. Biến-thiên-theo-thời-gian Có thể truy vấn các khoảng thời gian khác nhau.

Let's look at each of these parts to understand its influence on an Inmon data model. First, the logical model must focus on a specific area. For instance, if the subject orientation is “sales,” then the logical model contains all details related to sales—business keys, relationships, attributes, etc. Next, these details are integrated into a consolidated and highly normalized data model. Finally, the data is stored unchanged in a nonvolatile and time-variant way, meaning you can (theoretically) query the original data for as long as storage history allows. The Inmon data warehouse must strictly adhere to all four of these critical parts in support of management's decisions. This is a subtle point, but it positions the data warehouse for analytics, not **OLTP**. Here is another key characteristic of Inmon's data warehouse: The second salient characteristic of the data warehouse is that it is integrated. Of all the aspects of a data warehouse, integration is the most important. Data is fed from multiple, disparate sources into the data warehouse. As the data is fed, it is converted, reformatted, resequenced, summarized, etc. The result is that data—once it resides in the data warehouse—has a single physical corporate image. 9 With Inmon's data warehouse, data is integrated from across the organization in a granular, highly normalized ER model, with a relentless emphasis on ETL. Because of the subject-oriented nature of the data warehouse, the Inmon data warehouse consists of key source databases and information systems used in an organization. Data from key business source systems is ingested and integrated into a highly normalized (3NF) data warehouse that often closely resembles the normalization structure of the source system itself; data is brought in incrementally, starting with the highest-priority business areas. The strict normalization requirement ensures as little data duplication as possible, which leads to fewer downstream analytical errors because data won't diverge or suffer from redundancies. The data warehouse represents a “single source of truth,” which supports the overall business's information requirements. The data is presented for downstream reports and analysis via business and department-specific data marts, which may also be denormalized.

Hãy xem từng phần này để hiểu ảnh hưởng của nó lên một mô hình dữ liệu Inmon. Đầu tiên, mô hình logic phải tập trung vào một lĩnh vực cụ thể. Chẳng hạn, nếu định hướng chủ đề là “bán hàng”, thì mô hình logic chứa tất cả chi tiết liên quan đến bán hàng—các khóa nghiệp vụ, quan hệ, thuộc tính, v.v. Tiếp theo, các chi tiết này được tích hợp vào một mô hình dữ liệu được hợp nhất và chuẩn hóa cao. Cuối cùng, dữ liệu được lưu trữ không đổi theo cách bất biến và biến-thiên-theo-thời-gian, nghĩa là bạn có thể (về mặt lý thuyết) truy vấn dữ liệu gốc trong chừng nào lịch sử lưu trữ còn cho phép. Data warehouse của Inmon phải tuân thủ nghiêm ngặt cả bốn phần tối quan trọng này nhằm hỗ trợ các quyết định của ban quản lý. Đây là một điểm tinh tế, nhưng nó định vị data warehouse cho phân tích, không phải OLTP. Đây là một đặc điểm then chốt khác của data warehouse của Inmon: Đặc điểm nổi bật thứ hai của data warehouse là nó được tích hợp. Trong tất cả các khía cạnh của một data warehouse, tích hợp là quan trọng nhất. Dữ liệu được cấp từ nhiều nguồn rời rạc vào data warehouse. Khi dữ liệu được cấp, nó được chuyển đổi, định dạng lại, sắp xếp lại trình tự, tóm tắt, v.v. Kết quả là dữ liệu—một khi nó nằm trong data warehouse—có một hình ảnh doanh nghiệp vật lý duy nhất. Với data warehouse của Inmon, dữ liệu được tích hợp từ khắp tổ chức trong một mô hình ER độ-hạt-mịn, chuẩn hóa cao, với sự nhấn mạnh không khoan nhượng vào ETL. Vì bản chất định hướng-chủ-đề của data warehouse, data warehouse của Inmon bao gồm các database nguồn then chốt và các hệ thống thông tin được dùng trong một tổ chức. Dữ liệu từ các hệ thống nguồn nghiệp vụ then chốt được thu nạp và tích hợp vào một data warehouse được chuẩn hóa cao (3NF) thường khá giống với cấu trúc chuẩn hóa của chính hệ thống nguồn; dữ liệu được đưa vào tăng dần, bắt đầu từ các lĩnh vực nghiệp vụ ưu tiên cao nhất. Yêu cầu chuẩn hóa nghiêm ngặt đảm bảo càng ít trùng lặp dữ liệu càng tốt, dẫn đến ít lỗi phân tích hạ nguồn hơn vì dữ liệu sẽ không

phân kỳ hoặc bị dư thừa. Data warehouse biểu diễn một “nguồn sự thật duy nhất” (single source of truth), hỗ trợ các yêu cầu thông tin tổng thể của doanh nghiệp. Dữ liệu được trình bày cho các báo cáo và phân tích hạ nguồn thông qua các data mart đặc thù theo nghiệp vụ và phòng ban, vốn cũng có thể được phi chuẩn hóa.

Let’s look at how an Inmon data warehouse is used for ecommerce (Figure 8-13). The business source systems are orders, inventory, and marketing. The data from these source systems are ETLed to the data warehouse and stored in 3NF. Ideally, the data warehouse holistically encompasses the business’s information. To serve data for department-specific information requests, ETL processes take data from the data warehouse, transform the data, and place it in downstream data marts to be viewed in reports.

Hãy xem cách một data warehouse Inmon được dùng cho thương mại điện tử (Hình 8-13). Các hệ thống nguồn nghiệp vụ là đơn hàng, hàng tồn kho, và tiếp thị. Dữ liệu từ các hệ thống nguồn này được ETL tới data warehouse và lưu ở 3NF. Lý tưởng nhất, data warehouse bao quát toàn diện thông tin của doanh nghiệp. Để phục vụ dữ liệu cho các yêu cầu thông tin đặc thù theo phòng ban, các quy trình ETL lấy dữ liệu từ data warehouse, biến đổi dữ liệu, và đặt nó vào các data mart hạ nguồn để được xem trong các báo cáo.

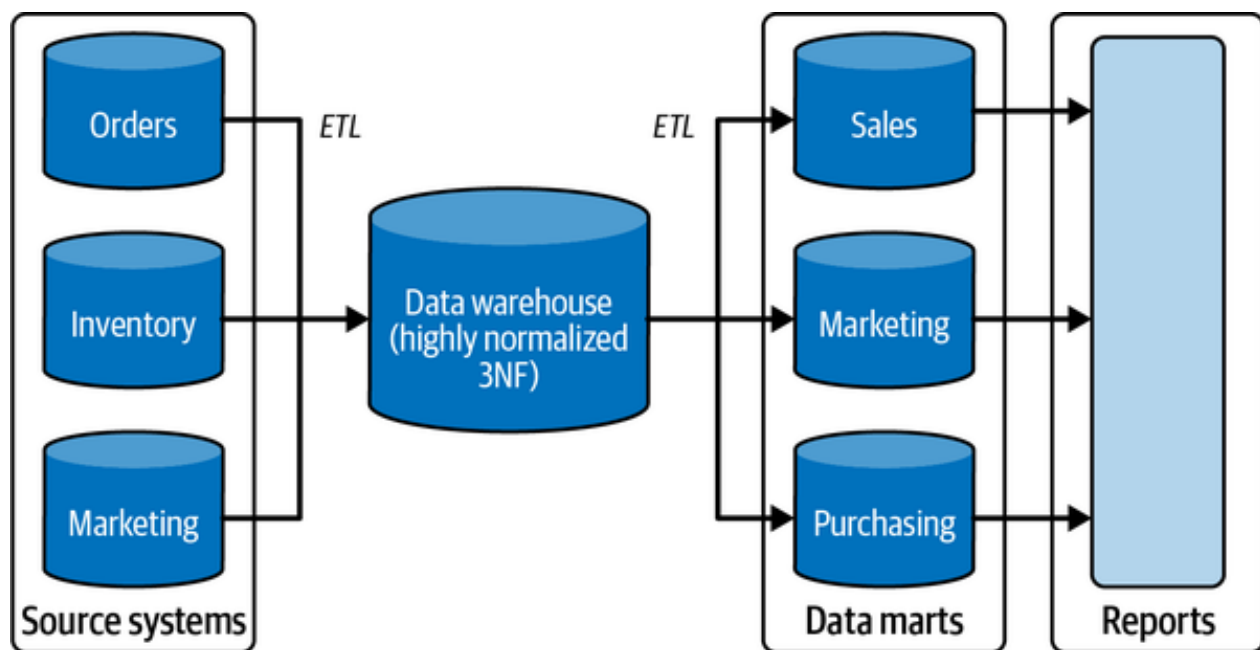


Figure 8-13. An ecommerce data warehouse A popular option for modeling data in a **data mart** is a star schema (discussed in the following section on Kimball), though any data model that provides easily accessible information is also suitable. In the preceding example, sales, marketing, and purchasing have their own star schema, fed **upstream** from the granular data in the data warehouse. This allows each department to have its own data structure that’s unique and optimized to its specific needs. Inmon continues to innovate in the data warehouse space, currently focusing on textual ETL in the data warehouse. He’s also a prolific writer and thinker, writing over 60 books and countless articles. For further reading about Inmon’s data warehouse, please refer to his books listed in “Additional Resources” . Kimball If there are spectrums to data modeling, Kimball is very much on the opposite end of Inmon. Created by Ralph Kimball in the early 1990s, this approach to data modeling focuses less on normalization, and in some cases accepting denormalization. As Inmon says about the difference between the data warehouse and data mart, “A data mart is never a substitute for a data warehouse.” 10 Whereas Inmon integrates data from across the business in

the data warehouse, and serves department-specific analytics via data marts, the Kimball model is bottom-up, encouraging you to model and serve department or business analytics in the data warehouse itself (Inmon argues this approach skews the definition of a data warehouse). This may enable faster iteration and modeling than Inmon, with the trade-off of potential looser data integration, data redundancy, and duplication. In Kimball's approach, data is modeled with two general types of tables: facts and dimensions. You can think of a **fact table** as a table of numbers, and dimension tables as qualitative data referencing a fact. Dimension tables surround a single fact table in a relationship called a star schema (Figure 8-14). Let's look at facts, dimensions, 11 and star schemas.

Hình 8-13. Một data warehouse thương mại điện tử. Một lựa chọn phổ biến để mô hình hóa dữ liệu trong một data mart là lược đồ sao (thảo luận ở mục Kimball tiếp theo), mặc dù bất kỳ mô hình dữ liệu nào cung cấp thông tin dễ truy cập cũng phù hợp. Trong ví dụ trên, bán hàng, tiếp thị, và mua hàng có lược đồ sao riêng, được cấp dữ liệu từ thượng nguồn từ dữ liệu độ-hạt-mịn trong data warehouse. Điều này cho phép mỗi phòng ban có cấu trúc dữ liệu riêng độc đáo và được tối ưu cho các nhu cầu cụ thể của nó. Inmon tiếp tục đổi mới trong không gian data warehouse, hiện đang tập trung vào ETL văn bản (textual ETL) trong data warehouse. Ông cũng là một tác giả và nhà tư tưởng năng suất, viết hơn 60 cuốn sách và vô số bài báo. Để đọc thêm về data warehouse của Inmon, vui lòng tham khảo các cuốn sách của ông được liệt kê trong “Additional Resources”. Kimball Nếu có các phổ trong mô hình hóa dữ liệu, thì Kimball nằm rất nhiều ở đầu đối diện với Inmon. Được tạo ra bởi Ralph Kimball vào đầu thập niên 1990, cách tiếp cận mô hình hóa dữ liệu này tập trung ít hơn vào chuẩn hóa, và trong một số trường hợp chấp nhận phi chuẩn hóa. Như Inmon nói về sự khác biệt giữa data warehouse và data mart, “Một data mart không bao giờ là sự thay thế cho một data warehouse.” Trong khi Inmon tích hợp dữ liệu từ khắp doanh nghiệp trong data warehouse, và phục vụ phân tích đặc thù theo phòng ban thông qua các data mart, thì mô hình Kimball là từ-dưới-lên (bottom-up), khuyến khích bạn mô hình hóa và phục vụ phân tích phòng ban hoặc nghiệp vụ ngay trong chính data warehouse (Inmon lập luận rằng cách tiếp cận này làm méo mó định nghĩa của một data warehouse). Điều này có thể cho phép lập và mô hình hóa nhanh hơn Inmon, đánh đổi với khả năng tích hợp dữ liệu lỏng lẻo hơn, dư thừa và trùng lặp dữ liệu. Trong cách tiếp cận của Kimball, dữ liệu được mô hình hóa với hai loại bảng chung: fact (sự kiện) và dimension (chiều). Bạn có thể nghĩ về một bảng fact như một bảng các con số, và các bảng dimension như dữ liệu định tính tham chiếu một fact. Các bảng dimension bao quanh một bảng fact đơn lẻ trong một quan hệ gọi là lược đồ sao (Hình 8-14). Hãy cùng xem các fact, dimension, và lược đồ sao.

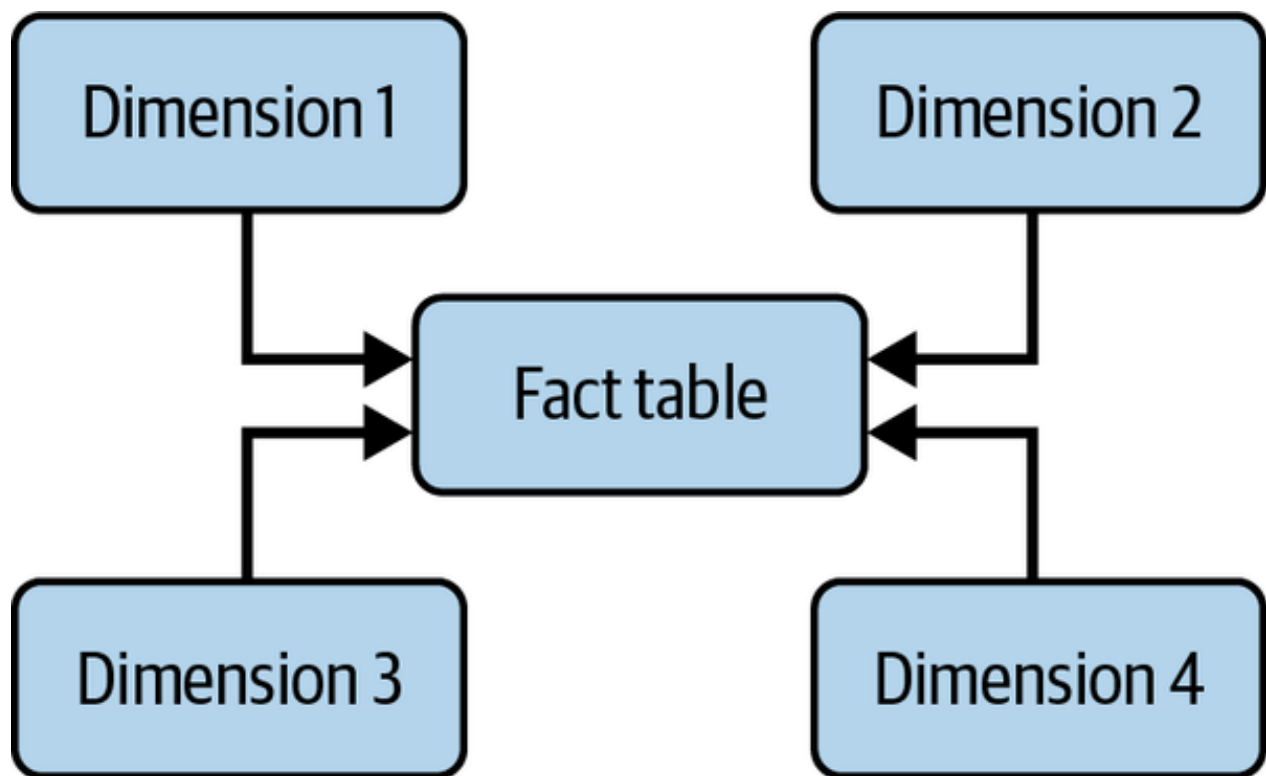


Figure 8-14. A Kimball star schema, with facts and dimensions

Fact tables The first type of table in a star schema is the fact table, which contains factual, quantitative, and event-related data. The data in a fact table is **immutable** because facts relate to events. Therefore, fact tables don't change and are append-only. Fact tables are typically narrow and long, meaning they have not a lot of columns but a lot of rows that represent events. Fact tables should be at the lowest grain possible. Queries against a star schema start with the fact table. Each row of a fact table should represent the grain of the data. Avoid aggregating or deriving data within a fact table. If you need to perform aggregations or derivations, do so in a downstream query, data mart table, or view. Finally, fact tables don't reference other fact tables; they reference only dimensions. Let's look at an example of an elementary fact table (Table 8-9). A common question in your company might be, "Show me gross sales, by each customer order, by date." Again, facts should be at the lowest grain possible—in this case, the orderID of the sale, customer, date, and gross sale amount. Notice that the data types in the fact table are all numbers (integers and floats); there are no strings. Also, in this example, CustomerKey 7 has two orders on the same day, reflecting the grain of the table. Instead, the fact table has keys that reference dimension tables containing their respective attributes, such as the customer and date information. The gross sales amount represents the total sale for the sales event.

Hình 8-14. Một lược đồ sao Kimball, với các fact và dimension. Bảng fact Loại bảng đầu tiên trong một lược đồ sao là bảng fact, vốn chứa dữ liệu thực tế, định lượng, và liên quan-đến-sự-khiến. Dữ liệu trong một bảng fact là bất biến vì các fact liên quan đến các sự kiện. Do đó, các bảng fact không thay đổi và chỉ-thêm (append-only). Các bảng fact thường hẹp và dài, nghĩa là chúng có không nhiều cột nhưng nhiều hàng biểu diễn các sự kiện. Các bảng fact nên ở độ hạt thấp nhất có thể. Các truy vấn trên một lược đồ sao bắt đầu từ bảng fact. Mỗi hàng của một bảng fact nên biểu diễn độ hạt của dữ liệu. Tránh tổng hợp hoặc dẫn xuất dữ liệu bên trong một bảng fact. Nếu bạn cần thực hiện các phép tổng hợp hoặc dẫn xuất, hãy làm trong một truy vấn hạ nguồn, một bảng data mart, hoặc một view. Cuối cùng, các bảng fact không tham chiếu các bảng fact khác; chúng chỉ tham chiếu các dimension. Hãy xem một ví dụ về một bảng fact sơ đẳng (Bảng 8-9). Một câu hỏi phổ biến

trong công ty bạn có thể là, “Cho tôi xem tổng doanh thu, theo từng đơn hàng khách hàng, theo ngày.” Một lần nữa, các fact nên ở độ hạt thấp nhất có thể—trong trường hợp này là orderID của giao dịch bán, khách hàng, ngày, và tổng số tiền bán. Lưu ý rằng các kiểu dữ liệu trong bảng fact đều là số (integer và float); không có chuỗi nào. Ngoài ra, trong ví dụ này, CustomerKey 7 có hai đơn hàng trong cùng một ngày, phản ánh độ hạt của bảng. Thay vào đó, bảng fact có các khóa tham chiếu các bảng dimension chứa các thuộc tính tương ứng của chúng, như thông tin khách hàng và ngày. Tổng số tiền bán biểu diễn tổng giao dịch bán cho sự kiện bán.

Table 8-9. A fact table

Bảng 8-9. Một bảng fact

OrderID	CustomerKey	DateKey	GrossSalesAmt	100	5	20220301	100.00	101	7	20220301	75.00	102	7	20220301	50.00
---------	-------------	---------	---------------	-----	---	----------	--------	-----	---	----------	-------	-----	---	----------	-------

Dimension tables The second primary type of table in a Kimball data model is called a dimension . Dimension tables provide the reference data, attributes, and relational context for the events stored in fact tables. Dimension tables are smaller than fact tables and take an opposite shape, typically wide and short. When joined to a fact table, dimensions can describe the events’ what, where, and when. Dimensions are denormalized, with the possibility of duplicate data. This is OK in the Kimball data model. Let’s look at the two dimensions referenced in the earlier fact table example. In a Kimball data model, dates are typically stored in a date dimension, allowing you to reference the date key (DateKey) between the fact and date **dimension table**. With the date dimension table, you can easily answer questions like, “What are my total sales in the first quarter of 2022?” or “How many more customers shop on Tuesday than Wednesday?” Notice we have five fields in addition to the date key (Table 8-10). The beauty of a date dimension is that you can add as many new fields as makes sense to analyze your data.

Bảng dimension Loại bảng chính thứ hai trong một mô hình dữ liệu Kimball được gọi là dimension. Các bảng dimension cung cấp dữ liệu tham chiếu, các thuộc tính, và ngữ cảnh quan hệ cho các sự kiện được lưu trong các bảng fact. Các bảng dimension nhỏ hơn các bảng fact và có hình dạng ngược lại, thường rộng và ngắn. Khi được join với một bảng fact, các dimension có thể mô tả cái gì, ở đâu, và khi nào của các sự kiện. Các dimension được phi chuẩn hóa, với khả năng có dữ liệu trùng lặp. Điều này là ổn trong mô hình dữ liệu Kimball. Hãy xem hai dimension được tham chiếu trong ví dụ bảng fact trước đó. Trong một mô hình dữ liệu Kimball, các ngày thường được lưu trong một dimension ngày (date dimension), cho phép bạn tham chiếu khóa ngày (DateKey) giữa bảng fact và bảng dimension ngày. Với bảng dimension ngày, bạn có thể dễ dàng trả lời các câu hỏi như, “Tổng doanh số của tôi trong quý đầu năm 2022 là bao nhiêu?” hoặc “Có nhiều hơn bao nhiêu khách hàng mua sắm vào thứ Ba so với thứ Tư?” Lưu ý chúng ta có năm trường ngoài khóa ngày (Bảng 8-10). Về đẹp của một dimension ngày là bạn có thể thêm bao nhiêu trường mới tùy ý mà hợp lý để phân tích dữ liệu của bạn.

Table 8-10. A date dimension table

Bảng 8-10. Một bảng dimension ngày

DateKey	Date-ISO	Year	Quarter	Month	Day-of-week	20220301	2022-03-01	2022	1	3	Tuesday	20220302	2022-03-02	2022	1	3	Wednesday	20220303	2022-03-03	2022	1	3	Thursday
---------	----------	------	---------	-------	-------------	----------	------------	------	---	---	---------	----------	------------	------	---	---	-----------	----------	------------	------	---	---	----------

Table 8-11 also references another dimension—the customer dimension—by the CustomerKey field.

The customer dimension contains several fields that describe the customer: first and last name, zip code, and a couple of peculiar-looking date fields. Let's look at these date fields, as they illustrate another concept in the Kimball data model: a Type 2 **slowly changing dimension**, which we'll describe in greater detail next.

Bảng 8-11 cũng tham chiếu một dimension khác—dimension khách hàng—bởi trường CustomerKey. Dimension khách hàng chứa một số trường mô tả khách hàng: tên và họ, mã bưu chính, và một vài trường ngày trông kỳ lạ. Hãy xem các trường ngày này, vì chúng minh họa một khái niệm khác trong mô hình dữ liệu Kimball: một chiều biến đổi chậm (slowly changing dimension, SCD) Loại 2, mà chúng tôi sẽ mô tả chi tiết hơn tiếp theo.

Table 8-11. A Type 2 customer dimension table

Bảng 8-11. Một bảng dimension khách hàng Loại 2

CustomerKey	FirstName	LastName	ZipCode	EFF_StartDate	EFF_EndDate
5	Joe	Reis	84108	2019-01-04	9999-01-01
7	Matt	Housley	84101	2020-05-04	2021-09-19
7	Matt	Housley	84123	2021-09-19	9999-01-01
11	Lana	Belle	90210	2022-02-04	9999-01-01

For example, take a look at CustomerKey 5, with the EFF_StartDate (EFF_StartDate means effective start date) of 2019-01-04 and an EFF_EndDate of 9999-01-01. This means Joe Reis's customer record was created in the customer dimension table on 2019-01-04 and has an end date of 9999-01-01. Interesting. What does this end date mean? It means the customer record is active and isn't changed. Now let's look at Matt Housley's customer record (CustomerKey = 7). Notice the two entries for Housley's start date: 2020-05-04 and 2021-09-19. It looks like Housley changed his zip code on 2021-09-19, resulting in a change to his customer record. When the data is queried for the most recent customer records, you will query where the end date is equal to 9999-01-01. A slowly changing dimension (SCD) is necessary to track changes in dimensions. The preceding example is a Type 2 SCD: a new record is inserted when an existing record changes. Though SCDs can go up to seven levels, let's look at the three most common ones: Type 1 Overwrite existing dimension records. This is super simple and means you have no access to the deleted historical dimension records. Type 2 Keep a full history of dimension records. When a record changes, that specific record is flagged as changed, and a new dimension record is created that reflects the current status of the attributes. In our example, Housley moved to a new zip code, which triggered his initial record to reflect an effective end date, and a new record was created to show his new zip code. Type 3 A Type 3 SCD is similar to a Type 2 SCD, but instead of creating a new row, a change in a Type 3 SCD creates a new field. Using the preceding example, let's see what this looks like as a Type 3 SCD in the following tables.

Ví dụ, hãy nhìn CustomerKey 5, với EFF_StartDate (EFF_StartDate nghĩa là ngày bắt đầu hiệu lực) là 2019-01-04 và một EFF_EndDate là 9999-01-01. Điều này nghĩa là bản ghi khách hàng của Joe Reis được tạo trong bảng dimension khách hàng vào 2019-01-04 và có một ngày kết thúc là 9999-01-01. Thú vị. Ngày kết thúc này nghĩa là gì? Nó nghĩa là bản ghi khách hàng đang hoạt động và không bị thay đổi. Bây giờ hãy nhìn bản ghi khách hàng của Matt Housley (CustomerKey = 7). Lưu ý hai mục cho ngày bắt đầu của Housley: 2020-05-04 và 2021-09-19. Có vẻ Housley đã thay đổi mã bưu chính của mình vào 2021-09-19, dẫn đến một thay đổi trong bản ghi khách hàng của anh ấy. Khi dữ liệu được truy vấn cho các bản ghi khách hàng gần nhất, bạn sẽ truy vấn nơi ngày kết thúc bằng 9999-01-01. Một chiều biến đổi chậm (SCD) là cần thiết để theo dõi các thay đổi trong các dimension. Ví dụ trên là một SCD Loại 2: một bản ghi mới được chèn vào khi một bản ghi hiện có thay đổi. Mặc dù các SCD có thể lên đến bảy cấp, hãy xem ba cấp phổ biến nhất: Loại 1 Ghi đè các bản ghi dimension hiện có. Cách này cực kỳ đơn giản và nghĩa là bạn không có quyền truy

cập vào các bản ghi dimension lịch sử đã bị xóa. Loại 2 Giữ một lịch sử đầy đủ các bản ghi dimension. Khi một bản ghi thay đổi, bản ghi cụ thể đó được đánh dấu là đã thay đổi, và một bản ghi dimension mới được tạo phản ánh trạng thái hiện tại của các thuộc tính. Trong ví dụ của chúng ta, Housley chuyển đến một mã bưu chính mới, kích hoạt bản ghi ban đầu của anh ấy để phản ánh một ngày kết thúc hiệu lực, và một bản ghi mới được tạo để cho thấy mã bưu chính mới của anh ấy. Loại 3 Một SCD Loại 3 tương tự một SCD Loại 2, nhưng thay vì tạo một hàng mới, một thay đổi trong một SCD Loại 3 tạo một trường mới. Dùng ví dụ trên, hãy xem điều này trông như thế nào ở dạng một SCD Loại 3 trong các bảng sau.

In Table 8-12, Housley lives in the 84101 zip code. When Housley moves to a new zip code, the Type 3 SCD creates two new fields, one for his new zip code and the date of the change (Table 8-13). The original zip code field is also renamed to reflect that this is the older record.

Trong Bảng 8-12, Housley sống ở mã bưu chính 84101. Khi Housley chuyển đến một mã bưu chính mới, SCD Loại 3 tạo hai trường mới, một cho mã bưu chính mới của anh ấy và ngày của thay đổi (Bảng 8-13). Trường mã bưu chính gốc cũng được đổi tên để phản ánh rằng đây là bản ghi cũ hơn.

Table 8-12. Type 3 slowly changing dimension

Bảng 8-12. Chiều biến đổi chậm Loại 3

CustomerKey	FirstName	LastName	ZipCode
7	Matt	Housley	84101

Table 8-13. Type 3 customer dimension table

Bảng 8-13. Một bảng dimension khách hàng Loại 3

CustomerKey	FirstName	LastName	Original ZipCode	Current ZipCode	CurrentDate
7	Matt	Housley	84101	84123	2021-09-19

Of the types of SCDs described, Type 1 is the default behavior of most data warehouses, and Type 2 is the one we most commonly see used in practice. There's a lot to know about dimensions, and we suggest using this section as a starting point to get familiar with how dimensions work and how they're used. Star schema Now that you have a basic understanding of facts and dimensions, it's time to integrate them into a star schema. The star schema represents the data model of the business. Unlike highly normalized approaches to data modeling, the star schema is a fact table surrounded by the necessary dimensions. This results in fewer joins than other data models, which speeds up query performance. Another advantage of a star schema is it's arguably easier for business users to understand and use. Note that the star schema shouldn't reflect a particular report, though you can model a report in a downstream data mart or directly in your BI tool. The star schema should capture the facts and attributes of your business logic and be flexible enough to answer the respective critical questions. Because a star schema has one fact table, sometimes you'll have multiple star schemas that address different facts of the business. You should strive to reduce the number of dimensions whenever possible since this reference data can potentially be reused among different fact tables. A dimension that is reused across multiple star schemas, thus sharing the same fields, is called a conformed dimension . A conformed dimension allows you to combine multiple fact tables across multiple star schemas. Remember, redundant data is OK with the Kimball method, but avoid replicating the same dimension tables to avoid drifting business definitions and **data integrity**. The Kimball data model and star schema have a lot of nuance. You should be aware that this mode is appropriate only for batch data and not for streaming data. Because the Kimball data model is popular, there's a good chance you'll run into it Data vault Whereas Kimball and Inmon focus on the structure of business logic in the data warehouse, the data vault offers a different approach to

data modeling. Created in the 1990s by Dan Linstedt, the data vault model separates the 12 structural aspects of a source system's data from its attributes. Instead of representing business logic in facts, dimensions, or highly normalized tables, a data vault simply loads data from source systems directly into a handful of purpose-built tables in an insert-only manner. Unlike the other data modeling approaches you've learned about, there's no notion of good, bad, or conformed data in a data vault. Data moves fast these days, and data models need to be agile, flexible, and scalable; the data vault model aims to meet this need. The goal of this model is to keep the data as closely aligned to the business as possible, even while the business's data evolves. A data vault consists of three main types of tables: hubs, links, and satellites (Figure 8-15). In short, a hub stores business keys, a link maintains relationships among business keys, and a satellite represents a business key's attributes and context. A user will query a hub, which will link to a satellite table containing the query's relevant attributes. Let's explore hubs, links, and satellites in more detail.

Trong các loại SCD được mô tả, Loại 1 là hành vi mặc định của hầu hết các data warehouse, và Loại 2 là loại chúng tôi thường thấy được dùng nhất trong thực tế. Có rất nhiều điều cần biết về các dimension, và chúng tôi gợi ý dùng phần này như một điểm khởi đầu để làm quen với cách các dimension hoạt động và cách chúng được dùng. Lược đồ sao Bây giờ bạn đã có một hiểu biết cơ bản về các fact và dimension, đã đến lúc tích hợp chúng vào một lược đồ sao. Lược đồ sao biểu diễn mô hình dữ liệu của doanh nghiệp. Không giống các cách tiếp cận chuẩn hóa cao với mô hình hóa dữ liệu, lược đồ sao là một bảng fact được bao quanh bởi các dimension cần thiết. Điều này dẫn đến ít phép join hơn các mô hình dữ liệu khác, làm tăng tốc hiệu năng truy vấn. Một lợi thế khác của lược đồ sao là nó được cho là dễ hơn cho người dùng nghiệp vụ hiểu và sử dụng. Lưu ý rằng lược đồ sao không nên phản ánh một báo cáo cụ thể, mặc dù bạn có thể mô hình hóa một báo cáo trong một data mart hạ nguồn hoặc trực tiếp trong công cụ BI của bạn. Lược đồ sao nên nắm bắt các fact và thuộc tính của logic nghiệp vụ của bạn và đủ linh hoạt để trả lời các câu hỏi tối quan trọng tương ứng. Vì một lược đồ sao có một bảng fact, đôi khi bạn sẽ có nhiều lược đồ sao xử lý các fact khác nhau của doanh nghiệp. Bạn nên cố gắng giảm số dimension bất cứ khi nào có thể vì dữ liệu tham chiếu này có khả năng được tái sử dụng giữa các bảng fact khác nhau. Một dimension được tái sử dụng trên nhiều lược đồ sao, từ đó chia sẻ cùng các trường, được gọi là một dimension tuân thủ (conformed dimension). Một dimension tuân thủ cho phép bạn kết hợp nhiều bảng fact trên nhiều lược đồ sao. Hãy nhớ, dữ liệu dư thừa là ổn với phương pháp Kimball, nhưng tránh sao chép cùng các bảng dimension để tránh các định nghĩa nghiệp vụ trôi dạt và toàn vẹn dữ liệu. Mô hình dữ liệu Kimball và lược đồ sao có rất nhiều sắc thái. Bạn nên lưu ý rằng chế độ này chỉ phù hợp cho dữ liệu batch chứ không phải dữ liệu streaming. Vì mô hình dữ liệu Kimball phổ biến, có nhiều khả năng bạn sẽ gặp nó. Data Vault Trong khi Kimball và Inmon tập trung vào cấu trúc của logic nghiệp vụ trong data warehouse, thì Data Vault cung cấp một cách tiếp cận khác cho mô hình hóa dữ liệu. Được tạo ra vào thập niên 1990 bởi Dan Linstedt, mô hình Data Vault tách các khía cạnh cấu trúc của dữ liệu của một hệ thống nguồn khỏi các thuộc tính của nó. Thay vì biểu diễn logic nghiệp vụ trong các fact, dimension, hoặc các bảng chuẩn hóa cao, một Data Vault đơn giản nạp dữ liệu từ các hệ thống nguồn trực tiếp vào một nhóm các bảng được xây dựng cho mục đích riêng theo cách chỉ-chèn. Không giống các cách tiếp cận mô hình hóa dữ liệu khác mà bạn đã học, không có khái niệm về dữ liệu tốt, xấu, hay tuân thủ trong một Data Vault. Dữ liệu di chuyển nhanh ngày nay, và các mô hình dữ liệu cần phải linh hoạt, mềm dẻo, và có khả năng mở rộng; mô hình Data Vault nhắm tới đáp ứng nhu cầu này. Mục tiêu của mô hình này là giữ dữ liệu càng gần với doanh nghiệp càng tốt, ngay cả khi dữ liệu của doanh nghiệp tiến hóa. Một Data Vault gồm ba loại bảng chính: hub, link, và satellite (Hình 8-15). Tóm lại, một hub lưu trữ các khóa nghiệp vụ, một link duy trì các quan hệ giữa các khóa nghiệp vụ, và một satellite biểu diễn các thuộc tính và ngữ cảnh của một khóa nghiệp vụ. Một người dùng sẽ truy vấn

một hub, vốn sẽ liên kết tới một bảng satellite chứa các thuộc tính liên quan của truy vấn. Hãy khám phá các hub, link, và satellite chi tiết hơn.

Figure 8-15. Data vault tables: hubs, links, and satellites connected together Hubs Queries often involve searching by a business key, such as a customer ID or an order ID from our ecommerce example. A hub is the central entity of a data vault that retains a record of all business keys loaded into the data vault. The hub should reflect the state of the source system from which it loads data. A hub always contains the following standard fields: Hash key The primary key used to join data between systems. This is a calculated hash field (MD5 or similar) Load date The date the data was loaded into the hub. Record source The source system.

Hình 8-15. Các bảng Data Vault: hub, link, và satellite được kết nối với nhau. Hub Các truy vấn thường liên quan đến tìm kiếm theo một khóa nghiệp vụ, như một ID khách hàng hoặc một ID đơn hàng từ ví dụ thương mại điện tử của chúng ta. Một hub là thực thể trung tâm của một Data Vault giữ lại một bản ghi của tất cả các khóa nghiệp vụ được nạp vào Data Vault. Hub nên phản ánh trạng thái của hệ thống nguồn mà nó nạp dữ liệu từ đó. Một hub luôn chứa các trường chuẩn sau: Hash key Khóa chính được dùng để join dữ liệu giữa các hệ thống. Đây là một trường hash được tính toán (MD5 hoặc tương tự) Load date Ngày dữ liệu được nạp vào hub. Record source Hệ thống nguồn.

Business key(s) The business key, or record ID, in the source system.

Khóa nghiệp vụ Khóa nghiệp vụ, hoặc ID bản ghi, trong hệ thống nguồn.

It's important to note that a hub is insert-only, and data from source systems is not altered in a hub. Once data is loaded into a hub, it's permanent. When designing a hub, identifying the business key is critical. Ask yourself: What is the identifiable business element in a source system? Put another way, how do users of a source system commonly look for data? Ideally, this is discovered as you build the conceptual data model of your organization and before you start building your data vault. Using our ecommerce scenario, let's look at an example of a hub for products. First, let's look at the physical design of a product hub (Table 8-14).

Điều quan trọng cần lưu ý là một hub là chỉ-chèn, và dữ liệu từ các hệ thống nguồn không bị thay đổi trong một hub. Một khi dữ liệu được nạp vào một hub, nó là vĩnh viễn. Khi thiết kế một hub, việc nhận diện khóa nghiệp vụ là tối quan trọng. Hãy tự hỏi: Phần tử nghiệp vụ có thể nhận diện trong một hệ thống nguồn là gì? Nói cách khác, người dùng của một hệ thống nguồn thường tìm dữ liệu như thế nào? Lý tưởng nhất, điều này được khám phá khi bạn xây dựng mô hình dữ liệu khái niệm của tổ chức bạn và trước khi bạn bắt đầu xây dựng Data Vault của mình. Dùng kịch bản thương mại điện tử của chúng ta, hãy xem một ví dụ về một hub cho các sản phẩm. Đầu tiên, hãy xem thiết kế vật lý của một hub sản phẩm (Bảng 8-14).

Table 8-14. A physical design for a product hub

Bảng 8-14. Một thiết kế vật lý cho một hub sản phẩm

HubProduct	Product	HashKey	LoadDate	RecordSource	ProductID
------------	---------	---------	----------	--------------	-----------

In practice, the product hub looks like this when populated with data (Table 8-15). In this example, three different products are loaded into a hub from an ERP system on two separate dates.

Trong thực tế, hub sản phẩm trông như thế này khi được điền đầy dữ liệu (Bảng 8-15). Trong ví dụ này, ba sản phẩm khác nhau được nạp vào một hub từ một hệ thống ERP vào hai ngày riêng biệt.

Table 8-15. A product hub populated with data

Bảng 8-15. Một hub sản phẩm được điền đầy dữ liệu

ProductHashKey	LoadDate	RecordSource	ProductID
4041fd80ab68e267490522b4a99048f5	2020-01-02	ERP 1	de8435530d6ca93caabc00cf88982dc1
2021-03-09	ERP 2	cf27369bd80f53d0e60d394817d77bab	
2021-03-09	ERP 3		

While we're at it, let's create another hub for orders (Table 8-16) using the same schema as HubProduct , and populate it with some sample order data.

Tiện thể, hãy tạo một hub khác cho các đơn hàng (Bảng 8-16) dùng cùng schema như HubProduct, và điền nó với một số dữ liệu đơn hàng mẫu.

Table 8-16. An order hub populated with data

Bảng 8-16. Một hub đơn hàng được điền đầy dữ liệu

OrderHashKey	LoadDate	RecordSource	OrderID
f899139df5e1059396431415e770c6dd	2022-03-01	Website 100	38b3eff8baf56627478ec76a704e9b52
2022-03-01	Website 101	ec8956637a99787bd197eacd77acce5e	
2022-03-01	Website 102		

Links A link table tracks the relationships of business keys between hubs. Link tables must connect at least two hubs, ideally at the lowest possible grain. Because link tables connect data from various hubs, they are many to many. The data vault model's relationships are straightforward and handled through changes to the links. This provides excellent flexibility in the inevitable event that the underlying source data changes. You add a new hub, and this hub needs to connect to existing hubs. Simply add a new relationship to that hub in your link table. That's it! Now let's look at ways to view data contextually using satellites. Back to our ecommerce example, we'd like to associate orders with products. Let's see what a link table might look like for orders and products (Table 8-17).

Link Một bảng link theo dõi các quan hệ của các khóa nghiệp vụ giữa các hub. Các bảng link phải kết nối ít nhất hai hub, lý tưởng nhất là ở độ hạt thấp nhất có thể. Vì các bảng link kết nối dữ liệu từ nhiều hub khác nhau, chúng là nhiều-nhiều. Các quan hệ của mô hình Data Vault là đơn giản và được xử lý thông qua các thay đổi đối với các link. Điều này cung cấp sự linh hoạt tuyệt vời trong trường hợp không thể tránh khỏi rằng dữ liệu nguồn cơ sở thay đổi. Bạn thêm một hub mới, và hub này cần kết nối với các hub hiện có. Đơn giản thêm một quan hệ mới tới hub đó trong bảng link của bạn. Vậy thôi! Giờ hãy xem các cách xem dữ liệu theo ngữ cảnh dùng các satellite. Quay lại ví dụ thương mại điện tử của chúng ta, chúng ta muốn liên kết các đơn hàng với các sản phẩm. Hãy xem một bảng link có thể trông như thế nào cho các đơn hàng và sản phẩm (Bảng 8-17).

Table 8-17. A link table for products and orders

Bảng 8-17. Một bảng link cho các sản phẩm và đơn hàng

LinkOrderProduct	OrderProductHashKey	LoadDate	RecordSource	ProductHashKey	OrderHashKey
------------------	---------------------	----------	--------------	----------------	--------------

When the LinkOrderProduct table is populated, here's what it looks like (Table 8-18). Note that we're using the order's record source in this example.

Table 8-18. A link table connecting orders and products

Bảng 8-18. Một bảng link kết nối các đơn hàng và sản phẩm

```

OrderProductHashKey LoadDate RecordSource ProductHashKey OrderHashKey ff64ec193d7bacf05e8b97ea04b3906
2022-03-01 Website 4041fd80ab68e267490522b4a990 f899139df5e1059396431415e770 6 48f5
c6dd ff64ec193d7bacf05e8b97ea04b3906 2022-03-01 Website de8435530d6ca93caabc00cf88982
f899139df5e1059396431415e770 6 dc1 c6dd e232628c251e3cad0f53f7e6e13cde 2022-03-01 Website
cf27369bd80f53d0e60d394817d738b3eff8baf56627478ec76a704e9 a7 7bab b52 26166a5871b6d21ae12e9c464262b
2022-03-01 Website 4041fd80ab68e267490522b4a990 ec8956637a99787bd197eacd77ac e67 48f5 ce5e

```

Satellites We’ve described relationships between hubs and links that involve keys, load dates, and record sources. How do you get a sense of what these relationships mean? Satellites are descriptive attributes that give meaning and context to hubs. Satellites can connect to either hubs or links. The only required fields in a satellite are a primary key consisting of the business key of the parent hub and a load date. Beyond that, a satellite can contain however many attributes that make sense. Let’s look at an example of a satellite for the Product hub (Table 8-19). In this example, the SatelliteProduct table contains additional information about the product, such as product name and price.

Satellite Chúng ta đã mô tả các quan hệ giữa các hub và link liên quan đến các khóa, ngày nạp, và nguồn bản ghi. Làm thế nào để bạn cảm nhận được các quan hệ này mang ý nghĩa gì? Các satellite là các thuộc tính mô tả mang lại ý nghĩa và ngữ cảnh cho các hub. Các satellite có thể kết nối với hoặc hub hoặc link. Các trường bắt buộc duy nhất trong một satellite là một khóa chính gồm khóa nghiệp vụ của hub cha và một ngày nạp. Ngoài đó, một satellite có thể chứa bao nhiêu thuộc tính tùy ý mà hợp lý. Hãy xem một ví dụ về một satellite cho hub Product (Bảng 8-19). Trong ví dụ này, bảng SatelliteProduct chứa thông tin bổ sung về sản phẩm, như tên sản phẩm và giá.

Table 8-19. SatelliteProduct

Bảng 8-19. SatelliteProduct

```

SatelliteProduct ProductHashKey LoadDate RecordSource ProductName Price

```

And here’s the SatelliteProduct table with some sample data (Table 8-20).

Và đây là bảng SatelliteProduct với một số dữ liệu mẫu (Bảng 8-20).

Table 8-20. A productsatellite table with sample data

Bảng 8-20. Một bảng satellite sản phẩm với dữ liệu mẫu

```

ProductHashKey LoadDate RecordSource ProductName Price 4041fd80ab68e267490522b4a99048f5
2020-01-02 ERP Thingamajig 50 de8435530d6ca93caabc00cf88982dc1 2021-03-09 ERP Whatchamacallit
25 cf27369bd80f53d0e60d394817d77bab 2021-03-09 ERP Whozeewhatzit 75 Let’s tie this all together
and join the hub, product, and link tables into a data vault ( Figure 8-16 ).

```

Hãy gắn tất cả lại với nhau và join các bảng hub, product, và link vào một Data Vault (Hình 8-16).

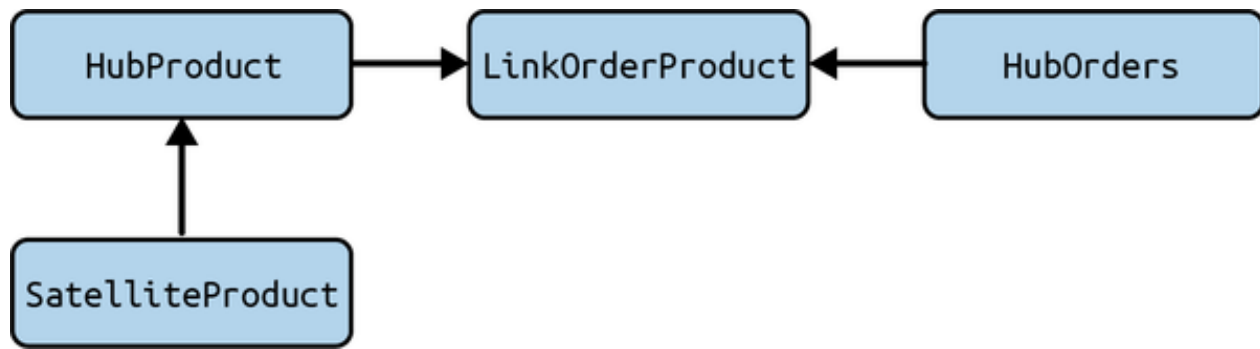


Figure 8-16. The data vault for orders and products Other types of data vault tables exist, including point in time (PIT) and bridge tables. We don't cover these here, but mention them because the data vault is quite comprehensive. Our goal is to simply give you an overview of the data vault's power. Unlike other data modeling techniques we've discussed, in a data vault, the business logic is created and interpreted when the data from these tables is queried. Please be aware that the data vault model can be used with other data modeling techniques. It's not unusual for a data vault to be the landing zone for analytical data, after which it's separately modeled in a data warehouse, commonly using a star schema. The data vault model also can be adapted for NoSQL and streaming data sources.

The data vault is a huge topic, and this section is simply meant to make you aware of its existence.

Wide denormalized tables The strict modeling approaches we've described, especially Kimball and Inmon, were developed when data warehouses were expensive, on premises, and heavily resource-constrained with **tightly coupled** compute and storage. While batch data modeling has traditionally been associated with these strict approaches, more relaxed approaches are becoming more common. There are reasons for this. First, the popularity of the cloud means that storage is dirt cheap. It's cheaper to store data than agonize over the optimum way to represent the data in storage. Second, the popularity of nested data (JSON and similar) means schemas are flexible in source and analytical systems. You have the option to rigidly model your data as we've described, or you can choose to throw all of your data into a single **wide table**. A wide table is just what it sounds like: a highly denormalized and very wide collection of many fields, typically created in a columnar database. A field may be a single value or contain nested data. The data is organized along with one or multiple keys; these keys are closely tied to the grain of the data. A wide table can potentially have thousands of columns, whereas fewer than 100 are typical in relational databases. Wide tables are usually sparse; the vast majority of entries in a given field may be null. This is extremely expensive in a traditional relational database because the database allocates a fixed amount of space for each field entry; nulls take up virtually no space in a columnar database. A wide schema in a relational database dramatically slows reading because each row must allocate all the space specified by the wide schema, and the database must read the contents of each row in its entirety. On the other hand, a columnar database reads only columns selected in a query, and reading nulls is essentially free. Wide tables generally arise through **schema evolution**; engineers gradually add fields over time. Schema evolution in a relational database is a slow and resource-heavy process. In a columnar database, adding a field is initially just a change to **metadata**. As data is written into the new field, new files are added to the column. Analytics queries on wide tables often run faster than equivalent queries on highly normalized data requiring many joins. Removing joins can have a huge impact on scan performance. The wide table simply contains all of the data you would have joined in a more rigorous modeling approach. Facts and dimensions are represented in the same table. The lack of data model rigor also means not a lot of thought is involved. Load your data into a wide table and start querying it. Especially with schemas in source systems becoming more adaptive and flexible, this data usually results from high-volume transactions, meaning there's a lot of data. Storing this as nested data in your analytical storage has a lot of benefits.

Hình 8-16. Data Vault cho các đơn hàng và sản phẩm. Tồn tại các loại bảng Data Vault khác, bao gồm các bảng point in time (PIT) và bridge. Chúng tôi không trình bày chúng ở đây, nhưng đề cập đến chúng vì Data Vault khá toàn diện. Mục tiêu của chúng tôi đơn giản là cho bạn một cái nhìn tổng quan về sức mạnh của Data Vault. Không giống các kỹ thuật mô hình hóa dữ liệu khác mà chúng ta đã thảo luận, trong một Data Vault, logic nghiệp vụ được tạo ra và diễn giải khi dữ liệu từ các bảng này được truy vấn. Hãy lưu ý rằng mô hình Data Vault có thể được dùng cùng các kỹ thuật mô hình hóa dữ liệu khác. Không hiếm khi một Data Vault là vùng đáp (landing zone) cho dữ liệu phân tích, sau đó nó được mô hình hóa riêng trong một data warehouse, thường dùng một lược đồ sao. Mô hình Data Vault cũng có thể được điều chỉnh cho NoSQL và các nguồn dữ liệu streaming. Data Vault là một chủ đề khổng lồ, và phần này chỉ nhằm khiến bạn nhận biết về sự tồn tại của nó. Bảng rộng phi chuẩn hóa Các cách tiếp cận mô hình hóa nghiêm ngặt mà chúng tôi đã mô tả, đặc biệt là Kimball và Inmon, được phát triển khi các data warehouse còn đắt đỏ, đặt tại chỗ (on premises), và bị ràng buộc tài nguyên nặng nề với tính toán và lưu trữ ghép chặt. Trong khi mô hình hóa dữ liệu batch theo truyền thống gắn với các cách tiếp cận nghiêm ngặt này, các cách tiếp cận thoải mái hơn đang trở nên phổ biến hơn. Có những lý do cho điều này. Thứ nhất, sự phổ biến của đám mây nghĩa là lưu trữ rẻ như bèo. Lưu trữ dữ liệu rẻ hơn là day dứt về cách tối ưu để biểu diễn dữ liệu trong lưu trữ. Thứ hai, sự phổ biến của dữ liệu lồng nhau (JSON và tương tự) nghĩa là các schema linh hoạt trong các hệ thống nguồn và phân tích. Bạn có lựa chọn mô hình hóa dữ liệu một cách cứng nhắc như chúng tôi đã mô tả, hoặc bạn có thể chọn ném tất cả dữ liệu của mình vào một bảng rộng duy nhất. Một bảng rộng đúng như tên gọi của nó: một tập hợp phi chuẩn hóa cao và rất rộng gồm nhiều trường, thường được tạo trong một database dạng cột. Một trường có thể là một giá trị đơn hoặc chứa dữ liệu lồng nhau. Dữ liệu được tổ chức theo một hoặc nhiều khóa; các khóa này gắn chặt với độ hạt của dữ liệu. Một bảng rộng có khả năng có hàng nghìn cột, trong khi dưới 100 là điển hình trong các database quan hệ. Các bảng rộng thường thưa (sparse); đại đa số các mục trong một trường nhất định có thể là null. Điều này cực kỳ tốn kém trong một database quan hệ truyền thống vì database cấp một lượng không gian cố định cho mỗi mục trường; các giá trị null gần như không chiếm không gian trong một database dạng cột. Một schema rộng trong một database quan hệ làm chậm việc đọc đáng kể vì mỗi hàng phải cấp toàn bộ không gian được chỉ định bởi schema rộng, và database phải đọc nội dung của mỗi hàng trong toàn bộ. Mặt khác, một database dạng cột chỉ đọc các cột được chọn trong một truy vấn, và việc đọc các null về cơ bản là miễn phí. Các bảng rộng nhìn chung nảy sinh thông qua tiến hóa schema; các kỹ sư dần dần thêm các trường theo thời gian. Tiến hóa schema trong một database quan hệ là một quá trình chậm và nặng tài nguyên. Trong một database dạng cột, thêm một trường ban đầu chỉ là một thay đổi đối với metadata. Khi dữ liệu được ghi vào trường mới, các tệp mới được thêm vào cột. Các truy vấn phân tích trên các bảng rộng thường chạy nhanh hơn các truy vấn tương đương trên dữ liệu chuẩn hóa cao đòi hỏi nhiều phép join. Loại bỏ các phép join có thể có tác động khổng lồ lên hiệu năng quét. Bảng rộng đơn giản chứa tất cả dữ liệu mà bạn lẽ ra đã join trong một cách tiếp cận mô hình hóa nghiêm ngặt hơn. Các fact và dimension được biểu diễn trong cùng một bảng. Sự thiếu nghiêm ngặt của mô hình dữ liệu cũng nghĩa là không cần nhiều suy nghĩ. Nạp dữ liệu của bạn vào một bảng rộng và bắt đầu truy vấn nó. Đặc biệt với các schema trong các hệ thống nguồn trở nên thích nghi và linh hoạt hơn, dữ liệu này thường là kết quả từ các giao dịch khối-lượng-lớn, nghĩa là có rất nhiều dữ liệu. Lưu trữ điều này dưới dạng dữ liệu lồng nhau trong kho phân tích của bạn có nhiều lợi ích.

Throwing all of your data into a single table might seem like heresy for a hardcore data modeler, and we've seen plenty of criticism. What are some of these criticisms? The biggest criticism is as you blend your data, you lose the business logic in your analytics. Another downside is the performance

of updates to things like an element in an array, which can be very painful. Let's look at an example of a wide table (Table 8-21), using the original denormalized table from our earlier normalization example. This table can have many more columns—hundreds or more!—and we include only a handful of columns for brevity and ease of understanding. As you can see, this table combines various data types, represented along a grain of orders for a customer on a date.

Ném tất cả dữ liệu của bạn vào một bảng duy nhất có thể có vẻ như là dị giáo đối với một nhà mô hình hóa dữ liệu cứng cựa, và chúng tôi đã thấy nhiều lời chỉ trích. Một số chỉ trích này là gì? Chỉ trích lớn nhất là khi bạn trộn dữ liệu của mình, bạn mất logic nghiệp vụ trong phân tích của mình. Một nhược điểm khác là hiệu năng của các cập nhật cho những thứ như một phần tử trong một mảng, có thể rất đau đớn. Hãy xem một ví dụ về một bảng rộng (Bảng 8-21), dùng bảng phi chuẩn hóa gốc từ ví dụ chuẩn hóa trước đó của chúng ta. Bảng này có thể có nhiều cột hơn nữa—hàng trăm hoặc hơn!—và chúng tôi chỉ bao gồm một nhóm cột cho ngắn gọn và dễ hiểu. Như bạn có thể thấy, bảng này kết hợp nhiều kiểu dữ liệu khác nhau, được biểu diễn dọc theo một độ hạt của các đơn hàng cho một khách hàng vào một ngày.

Table 8-21. An example of denormalized data

Bảng 8-21. Một ví dụ về dữ liệu phi chuẩn hóa

OrderID	OrderItems	CustomerID	CustomerName	OrderDate	Site	SiteRegion
100	5	Joe Reis	2022-03-01	abc.com	US	

```
[{
  "sku":1,
  "price":50,
  "quantity":1,
  "name":"Thingamajig"
},{
  "sku":2,
  "price":25,
  "quantity":2,
  "name":"Whatchamacallit"
}]
```

We suggest using a wide table when you don't care about data modeling, or when you have a lot of data that needs more flexibility than traditional data-modeling rigor provides. Wide tables also lend themselves to streaming data, which we'll discuss next. As data moves toward fast-moving schemas and streaming-first, we expect to see a new wave of data modeling, perhaps something along the lines of “relaxed normalization.”

Chúng tôi gợi ý dùng một bảng rộng khi bạn không quan tâm đến mô hình hóa dữ liệu, hoặc khi bạn có rất nhiều dữ liệu cần nhiều linh hoạt hơn so với những gì sự nghiêm

ngặt mô hình hóa dữ liệu truyền thống cung cấp. Các bảng rộng cũng phù hợp với dữ liệu streaming, mà chúng ta sẽ thảo luận tiếp theo. Khi dữ liệu di chuyển về hướng các schema chuyển động nhanh và streaming-trước-tiên, chúng tôi kỳ vọng sẽ thấy một làn sóng mới của mô hình hóa dữ liệu, có lẽ theo hướng “chuẩn hóa thoải mái” (relaxed normalization).

WHAT IF YOU DON'T MODEL YOUR DATA? You also have the option of not modeling your data. In this case, just query data sources directly. This pattern is often used, especially when companies are just getting started and want to get quick insights or share analytics with their users. While it allows you to get answers to various questions, you should consider the following: If I don't model my data, how do I know the results of my queries are consistent? Do I have proper definitions of business logic in the source system, and will my query produce truthful answers? What query load am I putting on my source systems, and how does this impact users of these systems? At some point, you'll probably gravitate toward a stricter batch data model paradigm and a dedicated **data architecture** that doesn't rely on the source systems for the heavy lifting.

NẾU BẠN KHÔNG MÔ HÌNH HÓA DỮ LIỆU CỦA MÌNH THÌ SAO? Bạn cũng có lựa chọn không mô hình hóa dữ liệu của mình. Trong trường hợp này, chỉ cần truy vấn các nguồn dữ liệu trực tiếp. Khuôn mẫu này thường được dùng, đặc biệt khi các công ty mới khởi đầu và muốn nhanh chóng có được các thông tin chuyên sâu hoặc chia sẻ phân tích với người dùng của họ. Mặc dù nó cho phép bạn có được câu trả lời cho nhiều câu hỏi khác nhau, bạn nên cân nhắc những điều sau: Nếu tôi không mô hình hóa dữ liệu của mình, làm sao tôi biết kết quả các truy vấn của mình là nhất quán? Tôi có các định nghĩa đúng đắn về logic nghiệp vụ trong hệ thống nguồn không, và truy vấn của tôi có sinh ra các câu trả lời trung thực không? Tôi đang đặt tải truy vấn nào lên các hệ thống nguồn của mình, và điều này tác động thế nào đến người dùng của các hệ thống này? Đến một thời điểm nào đó, bạn có lẽ sẽ hướng về một khuôn mẫu mô hình dữ liệu batch nghiêm ngặt hơn và một kiến trúc dữ liệu chuyên dụng không dựa vào các hệ thống nguồn để gánh nặng.

Modeling Streaming Data Whereas many data-modeling techniques are well established for batch, this is not the case for streaming data. Because of the unbounded and continuous nature of streaming data, translating batch techniques like Kimball to a streaming paradigm is tricky, if not impossible. For example, given a stream of data, how would you continuously update a Type-2 slowly changing dimension without bringing your data warehouse to its knees? The world is evolving from batch to streaming and from on premises to the cloud. The constraints of the older batch methods no longer apply. That said, big questions remain about how to model data to balance the need for business logic against fluid schema changes, fast-moving data, and self-service. What is the streaming equivalent of the preceding batch data model approaches? There isn't (yet) a consensus approach on streaming data modeling. We spoke with many experts in streaming data systems, many of whom told us that traditional batch-oriented data modeling doesn't apply to streaming. A few suggested the data vault as an option for streaming data modeling. As you may recall, two main types of streams exist: event streams and CDC. Most of the time, the shape of the data in these streams is semistructured, such as JSON. The challenge with modeling streaming data is that the payload's schema might change on a whim. For example, suppose you have an IoT device that recently upgraded its firmware and introduced a new field. In that case, it's possible that your downstream destination data warehouse or processing pipeline isn't aware of this change and breaks. That's not great. As another example, a CDC system might recast a field as a different type—say, a string instead of an International Organization for Standardization (ISO) datetime format. Again, how does the destination handle this seemingly random change? The streaming data experts we've talked with overwhelmingly suggest you anticipate changes in the source data and keep a flexible schema. This means there's no rigid data model in the analytical database. Instead, assume the source systems are providing the correct data with the right business definition and logic, as it exists today. And because storage is cheap, store the recent streaming and saved historical data in a way they can be queried together.

Mô hình hóa Dữ liệu Streaming Trong khi nhiều kỹ thuật mô hình hóa dữ liệu đã được thiết lập vững chắc cho batch, thì điều này không đúng với dữ liệu streaming. Vì bản chất không-biên và liên tục của dữ liệu streaming, việc chuyển dịch các kỹ thuật batch như Kimball sang một khuôn mẫu streaming là khó khăn, nếu không muốn nói là bất khả thi. Ví dụ, cho một luồng dữ liệu, làm thế nào bạn liên tục cập nhật một chiều biến đổi chậm Loại 2 mà không khiến data warehouse của bạn quy ngã? Thế giới đang tiến hóa từ batch sang streaming và từ on premises sang đám mây. Các ràng buộc của các phương pháp batch cũ hơn không còn áp dụng nữa. Dù vậy, vẫn còn những câu hỏi lớn về cách mô hình hóa dữ liệu để cân bằng nhu cầu về logic nghiệp vụ với các thay đổi schema linh động, dữ liệu chuyển động nhanh, và tự-phục-vụ. Tương đương streaming của các cách tiếp cận mô hình dữ liệu batch trên là gì? Chưa có (vẫn chưa) một cách tiếp cận đồng thuận về mô hình hóa dữ liệu streaming. Chúng tôi đã trò chuyện với nhiều chuyên gia trong các hệ thống dữ liệu streaming, nhiều người trong số họ nói với chúng tôi rằng mô hình hóa dữ liệu định hướng-batch truyền thống không áp dụng cho streaming. Một vài người gợi ý Data Vault như một lựa chọn cho mô hình hóa dữ liệu streaming. Như bạn có thể nhớ, hai loại luồng chính tồn tại: luồng sự kiện (event stream) và CDC. Hầu hết thời gian, hình dạng của dữ liệu trong các luồng này là bán cấu trúc, như JSON. Thách thức với mô hình hóa dữ liệu streaming là schema của payload có thể thay đổi bất chợt. Ví dụ, giả sử bạn có một thiết bị IoT vừa nâng cấp firmware của nó và giới thiệu một trường mới. Trong trường hợp đó, có khả năng data warehouse hoặc pipeline xử lý đích hạ nguồn của bạn không nhận biết được thay đổi này và bị vỡ. Điều đó không hay. Một ví dụ khác, một hệ thống CDC có thể đúc lại một trường thành một kiểu khác—chẳng hạn, một chuỗi thay vì định dạng datetime của Tổ chức Tiêu chuẩn hóa Quốc tế (ISO). Một lần nữa, đích xử lý thay đổi có vẻ ngẫu nhiên này như thế nào? Các chuyên gia dữ liệu streaming mà chúng tôi đã trò chuyện cùng đa số gợi ý bạn nên dự liệu trước các thay đổi trong dữ liệu nguồn và giữ một schema linh hoạt. Điều này nghĩa là không có mô hình dữ liệu cứng nhắc trong database phân tích. Thay vào đó, giả định các hệ thống nguồn đang cung cấp dữ liệu đúng với định nghĩa và logic nghiệp vụ đúng đắn, như nó tồn tại hôm nay. Và vì lưu trữ rẻ, hãy lưu trữ dữ liệu streaming gần đây và dữ liệu lịch sử đã lưu theo cách chúng có thể được truy vấn cùng nhau.

Optimize for comprehensive analytics against a dataset with a flexible schema. Furthermore, instead of reacting to reports, why not create automation that responds to anomalies and changes in the streaming data instead? The world of data modeling is changing, and we believe a sea change will soon occur in data model paradigms. These new approaches will likely incorporate metrics and **semantic** layers, data pipelines, and traditional analytics workflows in a streaming layer that sits directly on top of the source system. Since data is being generated in real time, the notion of artificially separating source and analytics systems into two distinct buckets may not make as much sense as when data moved more slowly and predictably. Time will tell... We have more to say on the future of streaming data in Chapter 11 .

Tối ưu cho phân tích toàn diện trên một tập dữ liệu với một schema linh hoạt. Hơn nữa, thay vì phản ứng với các báo cáo, tại sao không tạo tự động hóa phản hồi với các bất thường và thay đổi trong dữ liệu streaming? Thế giới mô hình hóa dữ liệu đang thay đổi, và chúng tôi tin rằng một sự thay đổi sâu sắc sẽ sớm xảy ra trong các khuôn mẫu mô hình dữ liệu. Các cách tiếp cận mới này có khả năng tích hợp các tầng chỉ số và ngữ nghĩa, các đường ống dữ liệu, và các luồng công việc phân tích truyền thống trong một tầng streaming nằm trực tiếp trên đỉnh hệ thống nguồn. Vì dữ liệu đang được sinh ra trong thời gian thực, khái niệm tách biệt một cách giả tạo các hệ thống nguồn và phân tích thành hai cái xô riêng biệt có thể không còn ý nghĩa nhiều như khi dữ liệu di chuyển chậm hơn và dễ dự đoán hơn. Thời gian sẽ trả lời... Chúng tôi có nhiều điều để nói hơn về tương lai của dữ liệu

Transformations — Biến đổi

The net result of transforming data is the ability to unify and integrate data. Once data is transformed, the data can be viewed as a single entity. But without transforming data, you cannot have a unified view of data across the organization. — Bill Inmon Now that we've covered queries and data modeling, you might be wondering, if I can model data, query it, and get results, why do I need to think about transformations? Transformations manipulate, enhance, and save data for downstream use, increasing its value in a scalable, reliable, and cost-effective manner. Imagine running a query every time you want to view results from a particular dataset. You'd run the same query dozens or hundreds of times a day. Imagine that this query involves parsing, cleansing, joining, unioning, and aggregating across 20 datasets. To further exacerbate the pain, the query takes 30 minutes to run, consumes significant resources, and incurs substantial cloud charges over several repetitions. You'd probably go insane. Thankfully, you can save the results of your query instead, or at least run the most compute-intensive portions only once, so subsequent queries are simplified. A **transformation** differs from a query. A query retrieves the data from various sources based on filtering and join logic. A transformation persists the results for consumption by additional transformations or queries. These results may be stored ephemerally or permanently. Besides persistence, a second aspect that differentiates transformations from queries is complexity. You'll likely build complex pipelines that combine data from multiple sources and reuse intermediate results for multiple final outputs. These complex pipelines might normalize, model, aggregate, or featurize data. While you can build complex dataflows in single queries using common table expressions, scripts, or DAGs, this quickly becomes unwieldy, inconsistent, and intractable. Enter transformations. Transformations critically rely on one of the major undercurrents in this book: **orchestration**. Orchestration combines many discrete operations, such as intermediate transformations, that store data temporarily or permanently for consumption by downstream transformations or **serving**. Increasingly, transformation pipelines span not only multiple tables and datasets, but also multiple systems.

Kết quả ròng của việc biến đổi dữ liệu là khả năng hợp nhất và tích hợp dữ liệu. Một khi dữ liệu được biến đổi, dữ liệu có thể được xem như một thực thể duy nhất. Nhưng nếu không biến đổi dữ liệu, bạn không thể có một cái nhìn hợp nhất về dữ liệu trên toàn tổ chức. — Bill Inmon. Bây giờ chúng ta đã trình bày truy vấn và mô hình hóa dữ liệu, bạn có thể đang tự hỏi, nếu tôi có thể mô hình hóa dữ liệu, truy vấn nó, và lấy kết quả, thì tại sao tôi cần nghĩ về các biến đổi? Các biến đổi thao tác, nâng cao, và lưu dữ liệu cho việc dùng ở hạ nguồn, tăng giá trị của nó theo cách có khả năng mở rộng, đáng tin cậy, và tiết kiệm chi phí. Hãy tưởng tượng chạy một truy vấn mỗi khi bạn muốn xem kết quả từ một tập dữ liệu cụ thể. Bạn sẽ chạy cùng một truy vấn hàng chục hoặc hàng trăm lần một ngày. Hãy tưởng tượng rằng truy vấn này liên quan đến phân tích cú pháp, làm sạch, join, union, và tổng hợp trên 20 tập dữ liệu. Để làm nổi đau thêm trầm trọng, truy vấn mất 30 phút để chạy, tiêu thụ tài nguyên đáng kể, và phát sinh các khoản phí đám mây lớn qua nhiều lần lặp. Bạn có lẽ sẽ phát điên. May mắn thay, bạn có thể lưu các kết quả của truy vấn của mình thay vào đó, hoặc ít nhất chạy các phần tốn-tính-toán nhất chỉ một lần, để các truy vấn sau được đơn giản hóa. Một biến đổi khác với một truy vấn. Một truy vấn truy xuất dữ liệu từ nhiều nguồn dựa trên logic lọc và join. Một biến đổi lưu giữ bền vững các kết quả để được tiêu thụ bởi các biến đổi hoặc truy vấn bổ sung. Các kết quả này có thể được lưu trữ tạm thời hoặc vĩnh viễn. Bên cạnh tính bền vững, một khía cạnh thứ hai phân biệt các biến đổi với các truy vấn là độ phức tạp. Bạn có khả năng sẽ xây dựng các pipeline phức tạp kết hợp dữ liệu từ nhiều nguồn và tái sử dụng các kết quả trung gian cho nhiều đầu ra cuối

cùng. Các pipeline phức tạp này có thể chuẩn hóa, mô hình hóa, tổng hợp, hoặc featurize (tạo đặc trưng) dữ liệu. Mặc dù bạn có thể xây dựng các luồng dữ liệu phức tạp trong các truy vấn đơn dùng các biểu thức bảng chung, script, hoặc DAG, điều này nhanh chóng trở nên khó kiểm soát, thiếu nhất quán, và nan giải. Đây là lúc các biến đổi vào cuộc. Các biến đổi phụ thuộc tối quan trọng vào một trong các dòng chảy ngầm chính trong cuốn sách này: điều phối (orchestration). Điều phối kết hợp nhiều thao tác rời rạc, như các biến đổi trung gian, vốn lưu trữ dữ liệu tạm thời hoặc vĩnh viễn để được tiêu thụ bởi các biến đổi hạ nguồn hoặc phục vụ. Ngày càng nhiều, các pipeline biến đổi trải dài không chỉ trên nhiều bảng và tập dữ liệu, mà còn trên nhiều hệ thống.

Batch Transformations Batch transformations run on discrete chunks of data, in contrast to streaming transformations, where data is processed continuously as it arrives. Batch transformations can run on a fixed schedule (e.g., daily, hourly, or every 15 minutes) to support ongoing reporting, analytics, and ML models. In this section, you'll learn various batch transformation patterns and technologies. **Distributed joins** The basic idea behind distributed joins is that we need to break a logical join (the join defined by the query logic) into much smaller node joins that run on individual servers in the cluster. The basic distributed join patterns apply whether one is in MapReduce (discussed in “MapReduce”), BigQuery, Snowflake, or Spark, though the details of intermediate storage between processing steps vary (on disk or in memory). In the best case scenario, the data on one side of the join is small enough to fit on a single node (broadcast join). Often, a more resource-intensive shuffle hash join is required. Broadcast join A broadcast join is generally asymmetric, with one large table distributed across nodes and one small table that can easily fit on a single node (Figure 8-17). The query engine “broadcasts” the small table (table A) out to all nodes, where it gets joined to the parts of the large table (table B). Broadcast joins are far less compute intensive than shuffle hash joins.

Biến đổi Batch Các biến đổi batch chạy trên các khối dữ liệu rời rạc, trái ngược với các biến đổi streaming, nơi dữ liệu được xử lý liên tục khi nó đến. Các biến đổi batch có thể chạy theo một lịch cố định (ví dụ hằng ngày, hằng giờ, hoặc mỗi 15 phút) để hỗ trợ báo cáo, phân tích, và các mô hình ML đang diễn ra. Trong phần này, bạn sẽ học nhiều khuôn mẫu và công nghệ biến đổi batch khác nhau. **Join phân tán** Ý tưởng cơ bản đằng sau các join phân tán là chúng ta cần phá vỡ một join logic (join được định nghĩa bởi logic truy vấn) thành các join node nhỏ hơn nhiều chạy trên các máy chủ riêng lẻ trong cluster. Các khuôn mẫu join phân tán cơ bản áp dụng dù người ta đang ở trong MapReduce (thảo luận ở “MapReduce”), BigQuery, Snowflake, hay Spark, mặc dù các chi tiết về lưu trữ trung gian giữa các bước xử lý khác nhau (trên đĩa hoặc trong bộ nhớ). Trong kịch bản tốt nhất, dữ liệu ở một phía của join đủ nhỏ để vừa trên một node duy nhất (broadcast join). Thường thì một shuffle hash join tốn-tài-nguyên-hơn là cần thiết. Broadcast join Một broadcast join nói chung là bất đối xứng, với một bảng lớn được phân tán trên các node và một bảng nhỏ có thể dễ dàng vừa trên một node duy nhất (Hình 8-17). Query engine “phát sóng” (broadcast) bảng nhỏ (bảng A) ra tất cả các node, nơi nó được join với các phần của bảng lớn (bảng B). Các broadcast join tốn ít tính toán hơn nhiều so với các shuffle hash join.

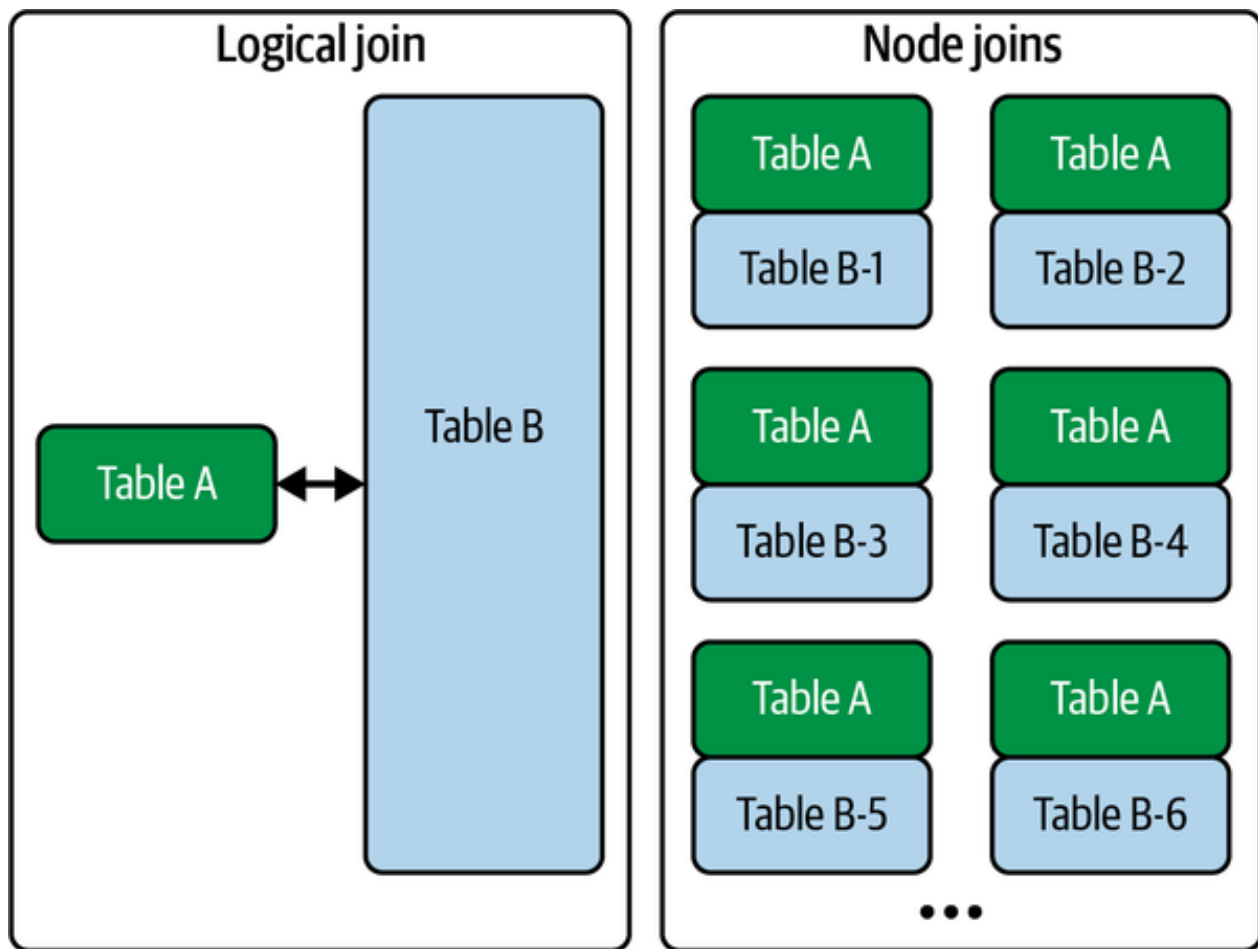


Figure 8-17. In a broadcast join, the query engine sends table A out to all nodes in the cluster to be joined with the various parts of table B. In practice, table A is often a down-filtered larger table that the query engine collects and broadcasts. One of the top priorities in query optimizers is join reordering. With the early application of filters, and movement of small tables to the left (for left joins) it is often possible to dramatically reduce the amount of data that is processed in each join. Pre-filtering data to create broadcast joins where possible can dramatically improve performance and reduce resource consumption. Shuffle hash join If neither table is small enough to fit on a single node, the query engine will use a shuffle hash join. In Figure 8-18, the same nodes are represented above and below the dotted line. The area above the dotted line represents the initial **partitioning** of tables A and B across the nodes. In general, this partitioning will have no relation to the join key. A hashing scheme is used to repartition data by join key.

Hình 8-17. Trong một broadcast join, query engine gửi bảng A ra tất cả các node trong cluster để được join với các phần khác nhau của bảng B. Trong thực tế, bảng A thường là một bảng lớn hơn đã được lọc xuống mà query engine thu thập và phát sóng. Một trong những ưu tiên hàng đầu trong các bộ tối ưu truy vấn là sắp xếp lại join. Với việc áp dụng sớm các bộ lọc, và di chuyển các bảng nhỏ sang trái (cho các left join) thường có thể giảm đáng kể lượng dữ liệu được xử lý trong mỗi join. Lọc trước dữ liệu để tạo các broadcast join ở những nơi có thể có thể cải thiện hiệu năng đáng kể và giảm tiêu thụ tài nguyên. Shuffle hash join Nếu không bảng nào đủ nhỏ để vừa trên một node duy nhất, query engine sẽ dùng một shuffle hash join. Trong Hình 8-18, cùng các node được biểu diễn ở trên và dưới đường nét đứt. Vùng phía trên đường nét đứt biểu diễn việc phân hoạch ban đầu của các bảng A và B trên các node. Nhìn chung, việc phân hoạch này sẽ không có quan hệ với khóa

join. Một sơ đồ hash được dùng để phân hoạch lại dữ liệu theo khóa join.

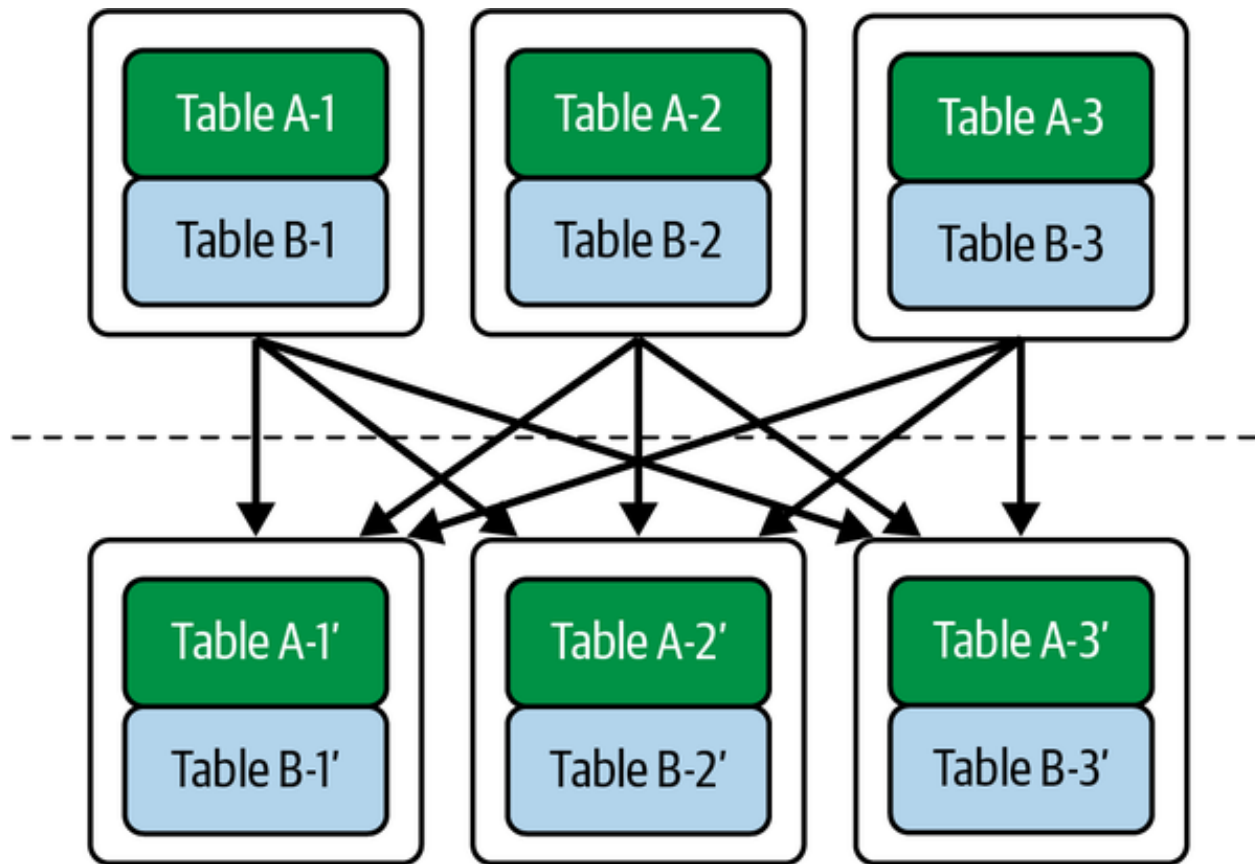


Figure 8-18. Shuffle hash join In this example, the hashing scheme will partition the join key into three parts, with each part assigned to a node. The data is then reshuffled to the appropriate node, and the new partitions for tables A and B on each node are joined. Shuffle hash joins are generally more resource intensive than broadcast joins. ETL, **ELT** and data pipelines As we discussed in Chapter 3 , a widespread transformation pattern dating to the early days of relational databases is a batch ETL. Traditional ETL relies on an external transformation system to pull, transform, and clean data while preparing it for a target schema, such as a data mart or a Kimball star schema. The transformed data would then be loaded into a target system, such as a data warehouse, where business analytics could be performed. The ETL pattern itself was driven by the limitations of both source and target systems. The extract phase tended to be a major bottleneck, with the constraints of the source RDBMS limiting the rate at which data could be pulled. And, the transformation was handled in a dedicated system because the target system was extremely resource-constrained in both storage and CPU capacity. A now-popular evolution of ETL is ELT. As data warehouse systems have grown in performance and storage capacity, it has become common to simply extract raw data from a source system, import it into a data warehouse with minimal transformation, and then clean and transform it directly in the warehouse system. (See our discussion of data warehouses in Chapter 3 for a more detailed discussion of the difference between ETL and ELT.) A second, slightly different notion of ELT was popularized with the emergence of data lakes. In this version, the data is not transformed at the time it's loaded. Indeed, massive quantities of data may be loaded with no preparation and no plan whatsoever. The assumption is that the transformation step will happen at some undetermined future time. Ingesting data without a plan is a great recipe for a data swamp. As Inmon says: I've always been a fan of ETL because of the fact that ETL forces you to transform data before you put it into a form where you can work with it. But some organizations

want to simply take the data, put it into a database, then do the transformation... I've seen too many cases where the organization says, oh we'll just put the data in and transform it later. And guess what? Six months later, that data [has] never been touched. 14 We have also seen that the line between ETL and ELT can become somewhat blurry in a **data lakehouse** environment. With object storage as a base layer, it's no longer clear what's in the database and out of the database. The ambiguity is further exacerbated with the emergence of data federation, virtualization, and live tables. (We discuss these topics later in this section.) Increasingly, we feel that the terms ETL and ELT should be applied only at the micro level (within individual transformation pipelines) rather than at the macro level (to describe a transformation pattern for a whole organization). Organizations no longer need to standardize on ETL or ELT but can instead focus on applying the proper technique on a case-by-case basis as they build data pipelines. SQL and general-purpose code-transformation tools At this juncture, the distinction between SQL-based and non-SQL-based transformation systems feels somewhat synthetic. Since the introduction of Hive on the Hadoop platform, SQL has become a first-class citizen in the big data ecosystem. For example, Spark SQL was an early feature of Apache Spark. Streaming-first frameworks such as Kafka, Flink, and Beam also support SQL, with varying features and functionality. It is more appropriate to think about SQL-only tools versus those that support more powerful, general-purpose programming paradigms. SQL-only transformation tools span a wide variety of proprietary and open source options. SQL is declarative...but it can still build complex data workflows We often hear SQL dismissed because it is "not procedural." This is technically correct. SQL is a declarative language: instead of coding a data processing procedure, SQL writers stipulate the characteristics of their final data in set-theoretic language; the SQL compiler and optimizer determine the steps required to put data in this state. People sometimes imply that because SQL is not procedural, it cannot build out complex pipelines. This is false. SQL can effectively be used to build complex DAGs using common table expressions, SQL scripts, or an orchestration tool. To be clear, SQL has limits, but we often see engineers doing things in Python and Spark that could be more easily and efficiently done in SQL. For a better idea of the trade-offs we're talking about, let's look at a couple of examples of Spark and SQL. Example: When to avoid SQL for batch transformations in Spark When you're determining whether to use native Spark or PySpark code instead of Spark SQL or another SQL engine, ask yourself the following questions:

Hình 8-18. Shuffle hash join. Trong ví dụ này, sơ đồ hash sẽ phân hoạch khóa join thành ba phần, mỗi phần được gán cho một node. Dữ liệu sau đó được xáo trộn lại tới node phù hợp, và các phân hoạch mới cho các bảng A và B trên mỗi node được join. Các shuffle hash join nhìn chung tốn tài nguyên hơn các broadcast join. ETL, ELT và các pipeline dữ liệu Như chúng ta đã thảo luận ở Chương 3, một khuôn mẫu biến đổi phổ biến có từ những ngày đầu của các database quan hệ là ETL batch. ETL truyền thống dựa vào một hệ thống biến đổi bên ngoài để kéo, biến đổi, và làm sạch dữ liệu trong khi chuẩn bị nó cho một schema đích, như một data mart hoặc một lược đồ sao Kimball. Dữ liệu đã được biến đổi sau đó sẽ được nạp vào một hệ thống đích, như một data warehouse, nơi phân tích nghiệp vụ có thể được thực hiện. Bản thân khuôn mẫu ETL được thúc đẩy bởi các giới hạn của cả hệ thống nguồn và đích. Pha trích xuất (extract) có xu hướng là một nút thắt cổ chai lớn, với các ràng buộc của RDBMS nguồn giới hạn tốc độ mà dữ liệu có thể được kéo. Và, biến đổi được xử lý trong một hệ thống chuyên dụng vì hệ thống đích bị ràng buộc tài nguyên cực kỳ nặng nề cả về lưu trữ lẫn dung lượng CPU. Một sự tiến hóa giờ-đã-phổ-biến của ETL là ELT. Khi các hệ thống data warehouse đã tăng trưởng về hiệu năng và dung lượng lưu trữ, việc đơn giản trích xuất dữ liệu thô từ một hệ thống nguồn, nhập nó vào một data warehouse với biến đổi tối thiểu, rồi sau đó làm sạch và biến đổi nó trực tiếp trong hệ thống warehouse đã trở nên phổ biến. (Xem thảo luận của chúng tôi về data warehouse ở Chương 3 để có thảo luận chi tiết hơn về sự khác biệt giữa ETL và ELT.) Một khái niệm ELT thứ hai, hơi khác, đã được phổ biến cùng với sự xuất hiện của các data lake. Trong phiên

bản này, dữ liệu không được biến đổi tại thời điểm nó được nạp. Thực vậy, lượng dữ liệu khổng lồ có thể được nạp mà không có sự chuẩn bị và không có kế hoạch nào cả. Giả định là bước biến đổi sẽ xảy ra vào một thời điểm tương lai chưa xác định. Thu nạp dữ liệu mà không có kế hoạch là một công thức tuyệt vời cho một data swamp. Như Inmon nói: Tôi luôn là một người hâm mộ ETL vì thực tế là ETL buộc bạn biến đổi dữ liệu trước khi bạn đưa nó vào một dạng mà bạn có thể làm việc với nó. Nhưng một số tổ chức muốn đơn giản lấy dữ liệu, đưa nó vào một database, rồi làm biến đổi sau... Tôi đã thấy quá nhiều trường hợp tổ chức nói, ồ chúng ta cứ đưa dữ liệu vào và biến đổi nó sau. Và đoán xem? Sáu tháng sau, dữ liệu đó [đã] không bao giờ được động đến. Chúng tôi cũng đã thấy rằng ranh giới giữa ETL và ELT có thể trở nên phần nào mờ nhạt trong một môi trường data lakehouse. Với object storage làm tầng cơ sở, không còn rõ cái gì là bên trong database và bên ngoài database. Sự mơ hồ càng bị làm trầm trọng thêm với sự xuất hiện của liên hợp dữ liệu (data federation), ảo hóa, và các bảng trực tiếp (live table). (Chúng ta thảo luận các chủ đề này sau trong phần này.) Ngày càng nhiều, chúng tôi cảm thấy rằng các thuật ngữ ETL và ELT nên chỉ được áp dụng ở cấp độ vi mô (trong các pipeline biến đổi riêng lẻ) thay vì ở cấp độ vĩ mô (để mô tả một khuôn mẫu biến đổi cho cả một tổ chức). Các tổ chức không còn cần chuẩn hóa trên ETL hoặc ELT mà thay vào đó có thể tập trung vào việc áp dụng kỹ thuật phù hợp theo từng-trường-hợp khi họ xây dựng các pipeline dữ liệu. Các công cụ biến đổi SQL và mã mục đích chung Tại thời điểm này, sự phân biệt giữa các hệ thống biến đổi dựa-trên-SQL và không-dựa-trên-SQL cảm thấy phần nào nhân tạo. Kể từ khi giới thiệu Hive trên nền tảng Hadoop, SQL đã trở thành một công dân hạng nhất trong hệ sinh thái big data. Ví dụ, Spark SQL là một tính năng sớm của Apache Spark. Các framework streaming-trước-tiên như Kafka, Flink, và Beam cũng hỗ trợ SQL, với các tính năng và chức năng khác nhau. Sẽ phù hợp hơn khi nghĩ về các công cụ chỉ-SQL so với những công cụ hỗ trợ các khuôn mẫu lập trình mục-đích-chung mạnh mẽ hơn. Các công cụ biến đổi chỉ-SQL trải dài nhiều lựa chọn độc quyền và mã nguồn mở khác nhau. SQL là khai báo... nhưng nó vẫn có thể xây dựng các luồng công việc dữ liệu phức tạp Chúng ta thường nghe SQL bị gạt bỏ vì nó “không thủ tục”. Điều này về mặt kỹ thuật là đúng. SQL là một ngôn ngữ khai báo (declarative): thay vì lập trình một thủ tục xử lý dữ liệu, những người viết SQL quy định các đặc tính của dữ liệu cuối của họ bằng ngôn ngữ lý-thuyết-tập-hợp; trình biên dịch và bộ tối ưu SQL xác định các bước cần thiết để đưa dữ liệu về trạng thái này. Người ta đôi khi ngụ ý rằng vì SQL không thủ tục, nó không thể xây dựng ra các pipeline phức tạp. Điều này là sai. SQL có thể được dùng hiệu quả để xây dựng ra các DAG phức tạp dùng các biểu thức bảng chung, các script SQL, hoặc một công cụ điều phối. Để rõ ràng, SQL có các giới hạn, nhưng chúng tôi thường thấy các kỹ sư làm những việc trong Python và Spark mà có thể được làm dễ dàng và hiệu quả hơn trong SQL. Để có một ý tưởng tốt hơn về các đánh đổi mà chúng tôi đang nói đến, hãy xem một vài ví dụ về Spark và SQL. Ví dụ: Khi nào nên tránh SQL cho các biến đổi batch trong Spark Khi bạn xác định liệu nên dùng mã Spark gốc hoặc PySpark thay vì Spark SQL hoặc một engine SQL khác, hãy tự hỏi các câu hỏi sau:

1. How difficult is it to code the transformation in SQL?
2. How readable and maintainable will the resulting SQL code be?
3. Should some of the transformation code be pushed into a custom library for future reuse across the organization? Regarding question 1, many transformations coded in Spark could be realized in fairly simple SQL statements. On the other hand, if the transformation is not realizable in SQL, or if it would be extremely awkward to implement, native Spark is a better option. For example, we might be able to implement word stemming in SQL by placing word suffixes in a table, joining with that table, using a parsing function to find suffixes in words, and then reducing the word to its stem by using a substring function. However, this sounds like an extremely complex process with numerous edge cases to consider. A more powerful procedural programming

language is a better fit here. Question 2 is closely related. The word-stemming query will be neither readable nor maintainable. Regarding question 3, one of the major limitations of SQL is that it doesn't include a natural notion of libraries or reusable code. One exception is that some SQL engines allow you to maintain user-defined functions (UDFs) as objects inside a database. However, these aren't committed to a Git repository without an external CI/CD system to manage deployment. Furthermore, SQL doesn't have a good notion of reusability for more complex query components. Of course, reusable libraries are easy to create in Spark and PySpark. We will add that it is possible to recycle SQL in two ways. First, we can easily reuse the results of a SQL query by committing to a table or creating a view. This process is often best handled in an orchestration tool such as Airflow so that downstream queries can start once the source query has finished. Second, Data Build Tool (dbt) facilitates the reuse of SQL statements and offers a templating language that makes customization easier.

1. Việc lập trình biến đổi trong SQL khó đến mức nào?
2. Mã SQL kết quả sẽ dễ đọc và dễ bảo trì đến mức nào?
3. Liệu một số mã biến đổi có nên được đẩy vào một thư viện tùy chỉnh để tái sử dụng trong tương lai trên toàn tổ chức? Về câu hỏi 1, nhiều biến đổi được lập trình trong Spark có thể được hiện thực hóa trong các câu lệnh SQL khá đơn giản. Mặt khác, nếu biến đổi không thể hiện thực hóa trong SQL, hoặc nếu sẽ cực kỳ vụng về để hiện thực, thì Spark gốc là một lựa chọn tốt hơn. Ví dụ, chúng ta có thể có khả năng hiện thực hóa việc rút gốc từ (word stemming) trong SQL bằng cách đặt các hậu tố từ trong một bảng, join với bảng đó, dùng một hàm phân tích cú pháp để tìm các hậu tố trong các từ, và sau đó rút từ về gốc của nó bằng một hàm substring. Tuy nhiên, điều này nghe như một quá trình cực kỳ phức tạp với vô số trường hợp biên cần cân nhắc. Một ngôn ngữ lập trình thủ tục mạnh mẽ hơn là một sự phù hợp tốt hơn ở đây. Câu hỏi 2 liên quan chặt chẽ. Truy vấn rút-gốc-từ sẽ không dễ đọc cũng không dễ bảo trì. Về câu hỏi 3, một trong những giới hạn lớn của SQL là nó không bao gồm một khái niệm tự nhiên về các thư viện hoặc mã tái sử dụng. Một ngoại lệ là một số engine SQL cho phép bạn duy trì các hàm do-người-dùng-định-nghĩa (user-defined function, UDF) như các đối tượng bên trong một database. Tuy nhiên, những cái này không được commit vào một kho Git mà không có một hệ thống CI/CD bên ngoài để quản lý việc triển khai. Hơn nữa, SQL không có một khái niệm tốt về khả năng tái sử dụng cho các thành phần truy vấn phức tạp hơn. Tất nhiên, các thư viện tái sử dụng dễ tạo trong Spark và PySpark. Chúng tôi xin bổ sung rằng có thể tái chế SQL theo hai cách. Thứ nhất, chúng ta có thể dễ dàng tái sử dụng các kết quả của một truy vấn SQL bằng cách commit vào một bảng hoặc tạo một view. Quá trình này thường được xử lý tốt nhất trong một công cụ điều phối như Airflow để các truy vấn hạ nguồn có thể bắt đầu một khi truy vấn nguồn đã hoàn thành. Thứ hai, Data Build Tool (dbt) tạo điều kiện cho việc tái sử dụng các câu lệnh SQL và cung cấp một ngôn ngữ tạo khuôn mẫu (templating) làm cho việc tùy chỉnh dễ dàng hơn.

Example: Optimizing Spark and other processing frameworks Spark acolytes often complain that SQL doesn't give them control over data processing. The SQL engine takes your statements, optimizes them, and compiles them into its processing steps. (In practice, optimization may happen before or after compilation, or both.) This is a fair complaint, but a corollary exists. With Spark and other code-heavy processing frameworks, the code writer becomes responsible for much of the optimization that is handled automatically in a SQL-based engine. The Spark API is powerful and complex, meaning it is not so easy to identify candidates for reordering, combination, or decomposition. When embracing Spark, data engineering teams need to actively engage with the problems of Spark optimization, especially for expensive, long-running jobs. This means building optimization expertise on the team and teaching individual engineers how to optimize. A few

top-level things to keep in mind when coding in native Spark:

Ví dụ: Tối ưu Spark và các framework xử lý khác. Những người sùng bái Spark thường than phiền rằng SQL không cho họ quyền kiểm soát đối với việc xử lý dữ liệu. Engine SQL lấy các câu lệnh của bạn, tối ưu chúng, và biên dịch chúng thành các bước xử lý của nó. (Trong thực tế, tối ưu có thể xảy ra trước hoặc sau biên dịch, hoặc cả hai.) Đây là một lời than phiền công bằng, nhưng tồn tại một hệ quả. Với Spark và các framework xử lý nặng-mã khác, người viết mã trở nên chịu trách nhiệm cho phần lớn việc tối ưu mà được xử lý tự động trong một engine dựa-trên-SQL. API của Spark mạnh mẽ và phức tạp, nghĩa là không dễ để nhận diện các ứng viên cho việc sắp xếp lại, kết hợp, hoặc phân rã. Khi đón nhận Spark, các nhóm kỹ thuật dữ liệu cần chủ động tham gia vào các vấn đề tối ưu Spark, đặc biệt cho các công việc tốn kém, chạy lâu. Điều này nghĩa là xây dựng chuyên môn tối ưu trong nhóm và dạy các kỹ sư riêng lẻ cách tối ưu. Một vài điều ở cấp cao cần ghi nhớ khi lập trình trong Spark gốc:

1. Filter early and often.
2. Rely heavily on the core Spark API, and learn to understand the Spark native way of doing things. Try to rely on well-maintained public libraries if the native Spark API doesn't support your use case. Good Spark code is substantially declarative.
3. Be careful with UDFs.
4. Consider intermixing SQL. Recommendation 1 applies to SQL optimization as well, the difference being that Spark may not be able to reorder something that SQL would handle for you automatically. Spark is a big data processing framework, but the less data you have to process, the less resource-heavy and more performant your code will be. If you find yourself writing extremely complex custom code, pause and determine whether there's a more native way of doing whatever you're trying to accomplish. Learn to understand idiomatic Spark by reading public examples and working through tutorials. Is there something in the Spark API that can accomplish what you're trying to do? Is there a well-maintained and optimized public library that can help? The third recommendation is crucial for PySpark. In general, PySpark is an API wrapper for Scala Spark. Your code pushes work into native Scala code running in the JVM by calling the API. Running Python UDFs forces data to be passed to Python, where processing is less efficient. If you find yourself using Python UDFs, look for a more Spark-native way to accomplish what you're doing. Go back to the recommendation: is there a way to accomplish your task by using the core API or a well-maintained library? If you must use UDFs, consider rewriting them in Scala or Java to improve performance. As for recommendation 4, using SQL allows us to take advantage of the Spark Catalyst optimizer, which may be able to squeeze out more performance than we can with native Spark code. SQL is often easier to write and maintain for simple operations. Combining native Spark and SQL lets us realize the best of both worlds— powerful, general-purpose functionality combined with simplicity where applicable. Much of the optimization advice in this section is fairly generic and would apply just as well to Apache Beam, for example. The main point is that programmable data processing APIs require a bit more optimization finesse than SQL, which is perhaps less powerful and easier to use. Update patterns Since transformations persist data, we will often update persisted data in place. Updating data is a major pain point for data engineering teams, especially as they transition between data engineering technologies. We're discussing DML in SQL, which we introduced earlier in the chapter. We've mentioned several times throughout the book that the original data lake concept didn't really account for updating data. This now seems nonsensical for several reasons. Updating data has long been a key part of handling data transformation results, even though the big data community dismissed it. It is silly to rerun significant amounts of work because we have no update capabilities. Thus, the data **lakehouse** concept now builds in updates. Also, GDPR and other data deletion standards

now require organizations to delete data in a targeted fashion, even in raw datasets. Let's consider several basic update patterns. Truncate and reload Truncate is an update pattern that doesn't update anything. It simply wipes the old data. In a truncate-and-reload update pattern, a table is cleared of data, and transformations are rerun and loaded into this table, effectively generating a new table version. Insert only Insert only inserts new records without changing or deleting old records. Insert-only patterns can be used to maintain a current view of data—for example, if new versions of records are inserted without deleting old records. A query or view can present the current data state by finding the newest record by primary key. Note that columnar databases don't typically enforce primary keys. The primary key would be a construct used by engineers to maintain a notion of the current state of the table. The downside to this approach is that it can be extremely computationally expensive to find the latest record at query time. Alternatively, we can use a materialized view (covered later in the chapter), an insert-only table that maintains all records, and a truncate-and-reload target table that holds the current state for serving data.

1. Lọc sớm và thường xuyên.
2. Dựa nhiều vào API cốt lõi của Spark, và học cách hiểu cách-làm-việc gốc của Spark. Cố gắng dựa vào các thư viện công khai được bảo trì tốt nếu API Spark gốc không hỗ trợ trường hợp sử dụng của bạn. Mã Spark tốt về cơ bản là khai báo.
3. Cẩn thận với các UDF.
4. Cân nhắc xen lẫn SQL. Khuyến nghị 1 cũng áp dụng cho tối ưu SQL, sự khác biệt là Spark có thể không thể sắp xếp lại một thứ gì đó mà SQL sẽ xử lý cho bạn một cách tự động. Spark là một framework xử lý big data, nhưng càng ít dữ liệu bạn phải xử lý, mã của bạn sẽ càng ít nặng-tài-nguyên và hiệu năng hơn. Nếu bạn thấy mình đang viết mã tùy chỉnh cực kỳ phức tạp, hãy dừng lại và xác định liệu có một cách gốc hơn để làm bất cứ điều gì bạn đang cố hoàn thành hay không. Học cách hiểu Spark đặc trưng bằng cách đọc các ví dụ công khai và làm qua các hướng dẫn. Có thứ gì đó trong API Spark có thể hoàn thành điều bạn đang cố làm không? Có một thư viện công khai được bảo trì tốt và được tối ưu nào có thể giúp không? Khuyến nghị thứ ba là tối quan trọng cho PySpark. Nhìn chung, PySpark là một lớp bọc API cho Scala Spark. Mã của bạn đẩy công việc vào mã Scala gốc chạy trong JVM bằng cách gọi API. Chạy các UDF Python buộc dữ liệu phải được chuyển sang Python, nơi việc xử lý kém hiệu quả hơn. Nếu bạn thấy mình đang dùng các UDF Python, hãy tìm một cách Spark-gốc hơn để hoàn thành điều bạn đang làm. Quay lại khuyến nghị: có một cách để hoàn thành nhiệm vụ của bạn bằng cách dùng API cốt lõi hoặc một thư viện được bảo trì tốt không? Nếu bạn buộc phải dùng các UDF, hãy cân nhắc viết lại chúng trong Scala hoặc Java để cải thiện hiệu năng. Còn về khuyến nghị 4, dùng SQL cho phép chúng ta tận dụng bộ tối ưu Spark Catalyst, vốn có thể vắt ra nhiều hiệu năng hơn so với những gì chúng ta có thể với mã Spark gốc. SQL thường dễ viết và dễ bảo trì hơn cho các thao tác đơn giản. Kết hợp Spark gốc và SQL cho phép chúng ta hiện thực hóa điều tốt nhất của cả hai thế giới—chức năng mạnh mẽ, mục-đích-chung kết hợp với sự đơn giản ở những nơi áp dụng được. Phần lớn lời khuyên tối ưu trong phần này khá chung chung và sẽ áp dụng tốt y như vậy cho Apache Beam, ví dụ thế. Điểm chính là các API xử lý dữ liệu có-thể-lập-trình đòi hỏi nhiều sự khéo léo tối ưu hơn một chút so với SQL, vốn có lẽ kém mạnh mẽ hơn và dễ dùng hơn. Các khuôn mẫu cập nhật Vì các biến đổi lưu giữ bền vững dữ liệu, chúng ta sẽ thường cập nhật dữ liệu được lưu giữ tại chỗ. Cập nhật dữ liệu là một điểm đau lớn cho các nhóm kỹ thuật dữ liệu, đặc biệt khi họ chuyển đổi giữa các công nghệ kỹ thuật dữ liệu. Chúng ta đang thảo luận về DML trong SQL, mà chúng ta đã giới thiệu trước đó trong chương. Chúng tôi đã đề cập nhiều lần xuyên suốt cuốn sách rằng khái niệm data lake ban đầu không thực sự tính đến việc cập nhật dữ liệu. Điều này giờ có vẻ vô lý vì nhiều lý do. Cập nhật dữ

liệu từ lâu đã là một phần then chốt của việc xử lý các kết quả biến đổi dữ liệu, mặc dù cộng đồng big data đã gạt bỏ nó. Thật ngớ ngẩn khi chạy lại lượng công việc đáng kể vì chúng ta không có khả năng cập nhật. Như vậy, khái niệm data lakehouse giờ tích hợp sẵn các cập nhật. Ngoài ra, GDPR và các tiêu chuẩn xóa dữ liệu khác giờ đòi hỏi các tổ chức xóa dữ liệu theo cách có-mục-tiêu, ngay cả trong các tập dữ liệu thô. Hãy cân nhắc một số khuôn mẫu cập nhật cơ bản. Truncate và nạp lại Truncate là một khuôn mẫu cập nhật không cập nhật bất cứ thứ gì. Nó đơn giản xóa sạch dữ liệu cũ. Trong một khuôn mẫu cập nhật truncate-và-nạp-lại, một bảng được xóa sạch dữ liệu, và các biến đổi được chạy lại và nạp vào bảng này, một cách hiệu quả sinh ra một phiên bản bảng mới. Chỉ-chèn Chỉ-chèn (insert only) chỉ chèn các bản ghi mới mà không thay đổi hoặc xóa các bản ghi cũ. Các khuôn mẫu chỉ-chèn có thể được dùng để duy trì một cái nhìn hiện tại của dữ liệu—ví dụ, nếu các phiên bản mới của các bản ghi được chèn vào mà không xóa các bản ghi cũ. Một truy vấn hoặc view có thể trình bày trạng thái dữ liệu hiện tại bằng cách tìm bản ghi mới nhất theo khóa chính. Lưu ý rằng các database dạng cột thường không thực thi các khóa chính. Khóa chính sẽ là một cấu trúc được dùng bởi các kỹ sư để duy trì một khái niệm về trạng thái hiện tại của bảng. Nhược điểm của cách tiếp cận này là có thể cực kỳ tốn-tính-toán để tìm bản ghi mới nhất tại thời điểm truy vấn. Cách khác, chúng ta có thể dùng một materialized view (trình bày sau trong chương), một bảng chỉ-chèn duy trì tất cả các bản ghi, và một bảng đích truncate-và-nạp-lại giữ trạng thái hiện tại để phục vụ dữ liệu.

WARNING When inserting data into a column-oriented OLAP database, the common problem is that engineers transitioning from row-oriented systems attempt to use single-row inserts. This antipattern puts a massive load on the system. It also causes data to be written in many separate files; this is extremely inefficient for subsequent reads, and the data must be reclustered later. Instead, we recommend loading data in a periodic micro-batch or batch fashion.

CẢNH BÁO Khi chèn dữ liệu vào một database OLAP hướng-cột, vấn đề phổ biến là các kỹ sư chuyển đổi từ các hệ thống hướng-hàng cố dùng các chèn một-hàng. Phản-khuôn-mẫu này đặt một tải khổng lồ lên hệ thống. Nó cũng khiến dữ liệu được ghi vào nhiều tệp riêng biệt; điều này cực kỳ kém hiệu quả cho các lần đọc sau đó, và dữ liệu phải được gom cụm lại (recluster) sau đó. Thay vào đó, chúng tôi khuyến nghị nạp dữ liệu theo cách vi-batch hoặc batch định kỳ.

We'll mention an exception to the advice not to insert frequently: the enhanced Lambda architecture used by BigQuery and Apache Druid, which hybridizes a streaming buffer with columnar storage. Deletes and in-place updates can still be expensive, as we'll discuss next. Delete Deletion is critical when a source system deletes data and satisfies recent regulatory changes. In columnar systems and data lakes, deletes are more expensive than inserts. When deleting data, consider whether you need to do a hard or soft delete. A hard delete permanently removes a record from a database, while a soft delete marks the record as "deleted." Hard deletes are useful when you need to remove data for performance reasons (say, a table is too big), or if there's a legal or compliance reason to do so. Soft deletes might be used when you don't want to delete a record permanently but also want to filter it out of query results. A third approach to deletes is closely related to soft deletes: insert deletion inserts a new record with a deleted flag without modifying the previous version of the record. This allows us to follow an insert-only pattern but still account for deletions. Just note that our query to get the latest table state gets a little more complicated. We must now deduplicate, find the latest version of each record by key, and not show any record whose latest version shows

Chúng tôi sẽ đề cập một ngoại lệ cho lời khuyên không chèn thường xuyên: kiến trúc Lambda nâng cao được dùng bởi BigQuery và Apache Druid, vốn lai hóa một bộ đệm

streaming với lưu trữ dạng cột. Các xóa và cập nhật tại-chỗ vẫn có thể tốn kém, như chúng ta sẽ thảo luận tiếp theo. Xóa Xóa là tối quan trọng khi một hệ thống nguồn xóa dữ liệu và thỏa mãn các thay đổi quy định gần đây. Trong các hệ thống dạng cột và data lake, các xóa tốn kém hơn các chèn. Khi xóa dữ liệu, hãy cân nhắc liệu bạn cần làm một xóa cứng (hard delete) hay mềm (soft delete). Một xóa cứng loại bỏ vĩnh viễn một bản ghi khỏi một database, trong khi một xóa mềm đánh dấu bản ghi là “đã xóa”. Các xóa cứng hữu ích khi bạn cần loại bỏ dữ liệu vì lý do hiệu năng (chẳng hạn, một bảng quá lớn), hoặc nếu có một lý do pháp lý hoặc tuân thủ để làm vậy. Các xóa mềm có thể được dùng khi bạn không muốn xóa một bản ghi vĩnh viễn nhưng cũng muốn lọc nó ra khỏi các kết quả truy vấn. Một cách tiếp cận thứ ba với các xóa liên quan chặt chẽ đến các xóa mềm: xóa-chèn (insert deletion) chèn một bản ghi mới với một cờ đã-xóa mà không sửa đổi phiên bản trước của bản ghi. Điều này cho phép chúng ta theo một khuôn mẫu chỉ-chèn nhưng vẫn tính đến các xóa. Chỉ lưu ý rằng truy vấn của chúng ta để lấy trạng thái bảng mới nhất trở nên phức tạp hơn một chút. Chúng ta giờ phải khử trùng lặp, tìm phiên bản mới nhất của mỗi bản ghi theo khóa, và không hiển thị bất kỳ bản ghi nào mà phiên bản mới nhất của nó cho thấy đã bị xóa.

Upsert/merge Of these update patterns, the upsert and merge patterns are the ones that consistently cause the most trouble for data engineering teams, especially for people transitioning from row-based data warehouses to column-based cloud systems.

Upsert/merge Trong các khuôn mẫu cập nhật này, các khuôn mẫu upsert và merge là những cái liên tục gây nhiều rắc rối nhất cho các nhóm kỹ thuật dữ liệu, đặc biệt cho những người chuyển đổi từ các data warehouse dựa-trên-hàng sang các hệ thống đám mây dựa-trên-cột.

Upserting takes a set of source records and looks for matches against a target table by using a primary key or another logical condition. (Again, it’s the responsibility of the data engineering team to manage this primary key by running appropriate queries. Most columnar systems will not enforce uniqueness.) When a key match occurs, the target record gets updated (replaced by the new record). When no match exists, the database inserts the new record. The merge pattern adds to this the ability to delete records. So, what’s the problem? The upsert/merge pattern was originally designed for row-based databases. In row-based databases, updates are a natural process: the database looks up the record in question and changes it in place. On the other hand, file-based systems don’t actually support in-place file updates. All of these systems utilize copy **on write** (COW). If one record in a file is changed or deleted, the whole file must be rewritten with the new changes. This is part of the reason that early adopters of big data and data lakes rejected updates: managing files and updates seemed too complicated. So they simply used an insert-only pattern and assumed that data consumers would determine the current state of the data at query time or in downstream transformations. In reality, columnar databases such as Vertica have long supported in-place updates by hiding the complexity of COW from users. They scan files, change the relevant records, write new files, and change file pointers for the table. The major columnar cloud data warehouses support updates and merges, although engineers should investigate update support if they consider adopting an exotic technology. There are a few key things to understand here. Even though distributed columnar data systems support native update commands, merges come at a cost: the performance impact of updating or deleting a single record can be quite high. On the other hand, merges can be extremely performant for large update sets and may even outperform transactional databases. In addition, it is important to understand that COW seldom entails rewriting the whole table. Depending on the database system in question, COW can operate at various resolutions (partition, cluster, block). To realize performant updates, focus on developing an appropriate partitioning and clustering strategy based on your needs and the innards of the database in question. As with inserts, be careful with your update or merge frequency. We’ve seen

many engineering teams transition between database systems and try to run near real-time merges from CDC just as they did on their old system. It simply doesn't work. No matter how good your CDC system is, this approach will bring most columnar data warehouses to their knees. We've seen systems fall weeks behind on updates, where an approach that simply merged every hour would make much more sense. We can use various approaches to bring columnar databases closer to real time. For example, BigQuery allows us to stream insert new records into a table, and then supports specialized materialized views that present an efficient, near real-time deduplicated table view. Druid uses two-tier storage and SSDs to support ultrafast real-time queries. Schema updates Data has entropy, and may change without your control or consent. External data sources may change their schema, or application development teams may add new fields to the schema. One advantage of columnar systems over row-based systems is that while updating the data is more difficult, updating the schema is easier. Columns can typically be added, deleted, and renamed. In spite of these technological improvements, practical organizational schema management is more challenging. Will some schema updates be automated? (This is the approach that Fivetran uses when replicating from sources.) As convenient as this sounds, there's a risk that downstream transformations will break. Is there a straightforward schema update request process? Suppose a data science team wants to add a column from a source that wasn't previously ingested. What will the review process look like? Will downstream processes break? (Are there queries that run `SELECT *` rather than using explicit column selection? This is generally bad practice in columnar databases.) How long will it take to implement the change? Is it possible to create a table fork —i.e., a new table version specific to this project?

Upsert lấy một tập các bản ghi nguồn và tìm các cặp khớp đối với một bảng đích bằng cách dùng một khóa chính hoặc một điều kiện logic khác. (Một lần nữa, đó là trách nhiệm của nhóm kỹ thuật dữ liệu để quản lý khóa chính này bằng cách chạy các truy vấn phù hợp. Hầu hết các hệ thống dạng cột sẽ không thực thi tính duy nhất.) Khi một cặp khớp khóa xảy ra, bản ghi đích được cập nhật (thay thế bởi bản ghi mới). Khi không có cặp khớp nào tồn tại, database chèn bản ghi mới. Khuôn mẫu merge thêm vào điều này khả năng xóa các bản ghi. Vậy, vấn đề là gì? Khuôn mẫu upsert/merge ban đầu được thiết kế cho các database dựa-trên-hàng. Trong các database dựa-trên-hàng, các cập nhật là một quá trình tự nhiên: database tra cứu bản ghi đang xét và thay đổi nó tại chỗ. Mặt khác, các hệ thống dựa-trên-tập không thực sự hỗ trợ các cập nhật tập tại-chỗ. Tất cả các hệ thống này sử dụng copy on write (COW). Nếu một bản ghi trong một tập được thay đổi hoặc xóa, toàn bộ tập phải được ghi lại với các thay đổi mới. Đây là một phần lý do mà những người chấp nhận sớm của big data và data lake đã từ chối các cập nhật: quản lý các tập và cập nhật có vẻ quá phức tạp. Vì vậy họ đơn giản dùng một khuôn mẫu chỉ-chèn và giả định rằng những người tiêu thụ dữ liệu sẽ xác định trạng thái hiện tại của dữ liệu tại thời điểm truy vấn hoặc trong các biến đổi hạ nguồn. Trên thực tế, các database dạng cột như Vertica đã từ lâu hỗ trợ các cập nhật tại-chỗ bằng cách ẩn đi độ phức tạp của COW khỏi người dùng. Chúng quét các tập, thay đổi các bản ghi liên quan, ghi các tập mới, và thay đổi các con trỏ tập cho bảng. Các cloud data warehouse dạng cột lớn hỗ trợ các cập nhật và merge, mặc dù các kỹ sư nên điều tra hỗ trợ cập nhật nếu họ cân nhắc áp dụng một công nghệ kỳ lạ. Có một vài điều then chốt cần hiểu ở đây. Mặc dù các hệ thống dữ liệu dạng cột phân tán hỗ trợ các lệnh cập nhật gốc, các merge đi kèm một cái giá: tác động hiệu năng của việc cập nhật hoặc xóa một bản ghi đơn lẻ có thể khá cao. Mặt khác, các merge có thể cực kỳ hiệu năng cho các tập cập nhật lớn và thậm chí có thể vượt trội hơn các database giao dịch. Ngoài ra, điều quan trọng cần hiểu là COW hiếm khi kéo theo việc ghi lại toàn bộ bảng. Tùy thuộc vào hệ thống database đang xét, COW có thể vận hành ở nhiều độ phân giải khác nhau (phân vùng, cụm, khối). Để hiện thực hóa các cập nhật hiệu năng, hãy tập trung vào phát triển một chiến lược phân vùng và gom cụm phù hợp dựa trên nhu cầu của bạn và phần ruột của database đang xét. Cũng như với các chèn, hãy cẩn thận với tần suất

cập nhật hoặc merge của bạn. Chúng tôi đã thấy nhiều nhóm kỹ thuật chuyển đổi giữa các hệ thống database và cố chạy các merge gần thời gian thực từ CDC y như họ đã làm trên hệ thống cũ của họ. Nó đơn giản là không hoạt động. Bất kể hệ thống CDC của bạn tốt đến mức nào, cách tiếp cận này sẽ khiến hầu hết các cloud data warehouse dạng cột quy ngã. Chúng tôi đã thấy các hệ thống tụt lại hàng tuần về cập nhật, nơi một cách tiếp cận đơn giản merge mỗi giờ sẽ hợp lý hơn nhiều. Chúng ta có thể dùng nhiều cách tiếp cận khác nhau để đưa các database dạng cột gần với thời gian thực hơn. Ví dụ, BigQuery cho phép chúng ta stream chèn các bản ghi mới vào một bảng, rồi hỗ trợ các materialized view chuyên biệt trình bày một cái nhìn bảng đã-khử-trùng-lập gần thời gian thực và hiệu quả. Druid dùng lưu trữ hai-tầng và các SSD để hỗ trợ các truy vấn thời gian thực siêu nhanh. Cập nhật schema Dữ liệu có entropy, và có thể thay đổi mà không có sự kiểm soát hoặc đồng thuận của bạn. Các nguồn dữ liệu bên ngoài có thể thay đổi schema của họ, hoặc các nhóm phát triển ứng dụng có thể thêm các trường mới vào schema. Một lợi thế của các hệ thống dạng cột so với các hệ thống dựa-trên-hàng là trong khi cập nhật dữ liệu khó hơn, thì cập nhật schema dễ hơn. Các cột thường có thể được thêm, xóa, và đổi tên. Bất chấp các cải tiến công nghệ này, việc quản lý schema thực tế ở cấp tổ chức thì thách thức hơn. Liệu một số cập nhật schema có được tự động hóa? (Đây là cách tiếp cận mà Fivetran dùng khi sao chép từ các nguồn.) Nghe có vẻ tiện lợi, nhưng có một rủi ro là các biến đổi hạ nguồn sẽ bị vỡ. Có một quy trình yêu cầu cập nhật schema đơn giản không? Giả sử một nhóm khoa học dữ liệu muốn thêm một cột từ một nguồn mà trước đó chưa được thu nạp. Quy trình rà soát sẽ trông như thế nào? Liệu các quá trình hạ nguồn có bị vỡ? (Có các truy vấn nào chạy `SELECT *` thay vì dùng lựa chọn cột tường minh không? Đây nói chung là một thực hành xấu trong các database dạng cột.) Việc triển khai thay đổi sẽ mất bao lâu? Có khả năng tạo một nhánh bảng (table fork)—tức một phiên bản bảng mới đặc thù cho dự án này—không?

A new interesting option has emerged for semistructured data. Borrowing an idea from document stores, many cloud data warehouses now support data types that encode arbitrary JSON data. One approach stores raw JSON in a field while storing frequently accessed data in adjacent flattened fields. This takes up additional storage space but allows for the convenience of flattened data, with the flexibility of semistructured data for advanced users. Frequently accessed data in the JSON field can be added directly into the schema over time. This approach works extremely well when data engineers must ingest data from an application document store with a frequently changing schema. Semistructured data available as a first-class citizen in data warehouses is extremely flexible and opens new opportunities for data analysts and data scientists since data is no longer constrained to rows and columns. Data wrangling Data wrangling takes messy, malformed data and turns it into useful, clean data. This is generally a batch transformation process. Data wrangling has long been a major source of pain and job security for ETL developers. For example, suppose that developers receive EDI data (see Chapter 7) from a partner business regarding transactions and invoices, potentially a mix of structured data and text. The typical process of wrangling this data involves first trying to ingest it. Often, the data is so malformed that a good deal of text preprocessing is involved. Developers may choose to ingest the data as a single text field table—an entire row ingested as a single field. Developers then begin writing queries to parse and break apart the data. Over time, they discover data anomalies and edge cases. Eventually, they will get the data into rough shape. Only then can the process of downstream transformation begin. Data wrangling tools aim to simplify significant parts of this process. These tools often put off data engineers because they claim to be no code, which sounds unsophisticated. We prefer to think of data wrangling tools as integrated development environments (IDEs) for malformed data. In practice, data engineers spend way too much time parsing nasty data; automation tools allow data engineers to spend time on more interesting tasks. Wrangling tools may also allow engineers to hand some parsing and **ingestion** work off to analysts. Graphical data-wrangling tools typically present a sample of data in a visual

interface, with inferred types, statistics including distributions, anomalous data, outliers, and nulls. Users can then add processing steps to fix data issues. A step might provide instructions for dealing with mistyped data, splitting a text field into multiple parts, or joining with a lookup table. Users can run the steps on a full dataset when the full job is ready. The job typically gets pushed to a scalable data processing system such as Spark for large datasets. After the job runs, it will return errors and unhandled exceptions. The user can further refine the recipe to deal with these outliers. We highly recommend that both aspiring and seasoned engineers experiment with wrangling tools; major cloud providers sell their version of data-wrangling tools, and many third-party options are available. Data engineers may find that these tools significantly streamline certain parts of their jobs. Organizationally, data engineering teams may want to consider training specialists in data wrangling if they frequently ingest from new, messy data sources. Example: Data transformation in Spark

Let's look at a practical, concrete example of data transformation. Suppose we build a pipeline that ingests data from three API sources in JSON format. This initial ingestion step is handled in Airflow. Each data source gets its prefix (filepath) in an S3 bucket. Airflow then triggers a Spark job by calling an API. This Spark job ingests each of the three sources into a dataframe, converting the data into a relational format, with nesting in certain columns. The Spark job combines the three sources into a single table, and then filters the results with a SQL statement. The results are finally written out to a Parquet-formatted Delta Lake table stored in S3. In practice, Spark creates a **DAG** of steps based on the code that we write for ingesting, joining, and writing out the data. The basic ingestion of data happens in cluster memory, although one of the data sources is large enough that it must spill to disk during the ingestion process. (This data gets written to cluster storage; it will be reloaded into memory for subsequent processing steps.) The join requires a shuffle operation. A key is used to redistribute data across the cluster; once again, a spill to disk occurs as the data is written to each node. The SQL transformation filters through the rows in memory and discards the unused rows. Finally, Spark converts the data into Parquet format, compresses it, and writes it back to S3. Airflow periodically calls back to Spark to see if the job is completed. Once it confirms that the job has finished, it marks the full Airflow DAG as completed. (Note that we have two DAG constructs here, an Airflow DAG and a DAG specific to the Spark job.)

Business logic and derived data

One of the most common use cases for transformation is to render business logic. We've placed this discussion under batch transformations because this is where this type of transformation happens most frequently, but note that it could also happen in a streaming pipeline. Suppose that a company uses multiple specialized internal profit calculations. One version might look at profits before marketing costs, and another might look at a profit after subtracting marketing costs. Even though this appears to be a straightforward accounting exercise, each of these metrics is highly complex to render. Profit before marketing costs might need to account for fraudulent orders; determining a reasonable profit estimate for the previous business day entails estimating what percentage of revenue and profit will ultimately be lost to orders canceled in the coming days as the fraud team investigates suspicious orders. Is there a special flag in the database that indicates an order with a high probability of fraud, or one that has been automatically canceled? Does the business assume that a certain percentage of orders will be canceled because of fraud even before the fraud-risk evaluation process has been completed for specific orders? For profits after marketing costs, we must account for all the complexities of the previous metric, plus the marketing costs attributed to the specific order. Does the company have a naive attribution model—e.g., marketing costs attributed to items weighted by price? Marketing costs might also be attributed per department, or item category, or—in the most sophisticated organizations—per individual item based on user ad clicks. The business logic transformation that generates this nuanced version of profit must integrate all the subtleties of attribution—i.e., a model that links orders to specific ads and advertising costs. Is attribution data stored in the guts of ETL scripts, or is it pulled from a table that is automatically generated from ad platform data? This type of reporting data is a quintessential example of derived data—data computed from other data stored in a data system. Derived data critics will point out

that it is challenging for the ETL to maintain consistency in the derived metrics. For example, if the company updates its attribution model, this change may need to be merged into many ETL scripts for reporting. (ETL scripts are notorious for breaking the DRY principle.) Updating these ETL scripts is a manual and labor-intensive process, involving domain expertise in processing logic and previous changes. Updated scripts must also be validated for consistency and accuracy. From our perspective, these are legitimate criticisms but not necessarily very constructive because the alternative to derived data in this instance is equally distasteful. Analysts will need to run their reporting queries if profit data is not stored in the data warehouse, including profit logic. Updating complex ETL scripts to represent changes to business logic accurately is an overwhelming, labor-intensive task, but getting analysts to update their reporting queries consistently is well-nigh impossible. One interesting alternative is to push business logic into a **metrics layer**, but still leverage the data warehouse or other tool to do the computational heavy lifting. A metrics layer encodes business logic and allows analysts and dashboard users to build complex analytics from a library of defined metrics. The metrics layer generates queries from the metrics and sends these to the database. We discuss semantic and metrics layers in more detail in Chapter 9. MapReduce No discussion of batch transformation can be complete without touching on MapReduce. This isn't because MapReduce is widely used by data engineers these days. MapReduce was the defining batch data transformation pattern of the big data era, it still influences many distributed systems data engineers use today, and it's useful for data engineers to understand at a basic level. MapReduce was introduced by Google in a follow-up to its paper on GFS. It was initially the de facto processing pattern of Hadoop, the open source analogue technology of GFS that we introduced in Chapter 6. A simple MapReduce job consists of a collection of map tasks that read individual data blocks scattered across the nodes, followed by a shuffle that redistributes result data across the cluster and a reduce step that aggregates data on each node. For example, suppose that we wanted to run the following SQL query:

Một lựa chọn thú vị mới đã nổi lên cho dữ liệu bán cấu trúc. Mượn một ý tưởng từ các document store, nhiều cloud data warehouse giờ hỗ trợ các kiểu dữ liệu mã hóa dữ liệu JSON tùy ý. Một cách tiếp cận lưu trữ JSON thô trong một trường trong khi lưu trữ dữ liệu thường-xuyên-truy-cập trong các trường được làm phẳng (flattened) liền kề. Điều này chiếm thêm không gian lưu trữ nhưng cho phép sự tiện lợi của dữ liệu được làm phẳng, với sự linh hoạt của dữ liệu bán cấu trúc cho các người dùng nâng cao. Dữ liệu thường-xuyên-truy-cập trong trường JSON có thể được thêm trực tiếp vào schema theo thời gian. Cách tiếp cận này hoạt động cực kỳ tốt khi các kỹ sư dữ liệu phải thu nạp dữ liệu từ một document store ứng dụng với một schema thay đổi thường xuyên. Dữ liệu bán cấu trúc sẵn có như một công dân hạng nhất trong các data warehouse là cực kỳ linh hoạt và mở ra các cơ hội mới cho các nhà phân tích dữ liệu và nhà khoa học dữ liệu vì dữ liệu không còn bị ràng buộc vào các hàng và cột. Vật lộn dữ liệu (data wrangling) Vật lộn dữ liệu lấy dữ liệu lộn xộn, dị dạng và biến nó thành dữ liệu hữu ích, sạch sẽ. Đây nói chung là một quá trình biến đổi batch. Vật lộn dữ liệu từ lâu đã là một nguồn đau lớn và là sự đảm bảo việc làm cho các nhà phát triển ETL. Ví dụ, giả sử các nhà phát triển nhận dữ liệu EDI (xem Chương 7) từ một doanh nghiệp đối tác liên quan đến các giao dịch và hóa đơn, có khả năng là một sự pha trộn giữa dữ liệu có cấu trúc và văn bản. Quá trình điển hình của việc vật lộn dữ liệu này liên quan đến việc đầu tiên cố thu nạp nó. Thường thì dữ liệu dị dạng đến mức cần khá nhiều việc tiền xử lý văn bản. Các nhà phát triển có thể chọn thu nạp dữ liệu như một bảng trường-văn-bản-đơn—toàn bộ một hàng được thu nạp như một trường đơn. Các nhà phát triển sau đó bắt đầu viết các truy vấn để phân tích cú pháp và tách rời dữ liệu. Theo thời gian, họ khám phá ra các bất thường dữ liệu và các trường hợp biên. Cuối cùng, họ sẽ đưa được dữ liệu vào hình dạng tạm ổn. Chỉ sau đó quá trình biến đổi hạ nguồn mới có thể bắt đầu. Các công cụ vật lộn dữ liệu nhắm tới việc đơn giản hóa các phần đáng kể của quá trình này. Các công cụ này thường khiến các kỹ sư dữ liệu mất hứng vì chúng tuyên

bố là không-mã (no code), nghe có vẻ thiếu tinh vi. Chúng tôi thích nghĩ về các công cụ vật lộn dữ liệu như các môi trường phát triển tích hợp (integrated development environment, IDE) cho dữ liệu dị dạng. Trong thực tế, các kỹ sư dữ liệu dành quá nhiều thời gian phân tích cú pháp dữ liệu khó chịu; các công cụ tự động hóa cho phép các kỹ sư dữ liệu dành thời gian cho các nhiệm vụ thú vị hơn. Các công cụ vật lộn cũng có thể cho phép các kỹ sư giao một phần công việc phân tích cú pháp và thu nạp cho các nhà phân tích. Các công cụ vật-lộn-dữ-liệu đồ họa thường trình bày một mẫu dữ liệu trong một giao diện trực quan, với các kiểu được suy ra, các thống kê bao gồm các phân phối, dữ liệu bất thường, các giá trị ngoại lai, và các null. Người dùng sau đó có thể thêm các bước xử lý để khắc phục các vấn đề dữ liệu. Một bước có thể cung cấp các hướng dẫn để xử lý dữ liệu sai kiểu, tách một trường văn bản thành nhiều phần, hoặc join với một bảng tra cứu. Người dùng có thể chạy các bước trên một tập dữ liệu đầy đủ khi toàn bộ công việc đã sẵn sàng. Công việc thường được đẩy tới một hệ thống xử lý dữ liệu có khả năng mở rộng như Spark cho các tập dữ liệu lớn. Sau khi công việc chạy, nó sẽ trả về các lỗi và các ngoại lệ chưa-được-xử-lý. Người dùng có thể tinh chỉnh thêm công thức (recipe) để xử lý các ngoại lai này. Chúng tôi rất khuyến nghị rằng cả các kỹ sư đầy tham vọng lẫn các kỹ sư dày dạn nên thử nghiệm với các công cụ vật lộn; các nhà cung cấp đám mây lớn bán phiên bản công cụ vật-lộn-dữ-liệu của họ, và nhiều lựa chọn bên-thứ-ba sẵn có. Các kỹ sư dữ liệu có thể thấy rằng các công cụ này tinh giản đáng kể một số phần công việc của họ. Về mặt tổ chức, các nhóm kỹ thuật dữ liệu có thể muốn cân nhắc đào tạo các chuyên gia về vật lộn dữ liệu nếu họ thường xuyên thu nạp từ các nguồn dữ liệu mới, lộn xộn. Ví dụ: Biến đổi dữ liệu trong Spark Hãy xem một ví dụ thực tế, cụ thể về biến đổi dữ liệu. Giả sử chúng ta xây dựng một pipeline thu nạp dữ liệu từ ba nguồn API ở định dạng JSON. Bước thu nạp ban đầu này được xử lý trong Airflow. Mỗi nguồn dữ liệu có tiền tố (đường dẫn tệp) riêng của nó trong một bucket S3. Airflow sau đó kích hoạt một công việc Spark bằng cách gọi một API. Công việc Spark này thu nạp mỗi nguồn trong ba nguồn vào một dataframe, chuyển đổi dữ liệu thành một định dạng quan hệ, với sự lồng nhau trong một số cột nhất định. Công việc Spark kết hợp ba nguồn thành một bảng duy nhất, và sau đó lọc các kết quả với một câu lệnh SQL. Các kết quả cuối cùng được ghi ra một bảng Delta Lake định-dạng-Parquet lưu trong S3. Trong thực tế, Spark tạo một DAG của các bước dựa trên mã mà chúng ta viết để thu nạp, join, và ghi ra dữ liệu. Việc thu nạp dữ liệu cơ bản xảy ra trong bộ nhớ cluster, mặc dù một trong các nguồn dữ liệu đủ lớn đến mức nó phải tràn ra đĩa (spill to disk) trong quá trình thu nạp. (Dữ liệu này được ghi vào lưu trữ cluster; nó sẽ được nạp lại vào bộ nhớ cho các bước xử lý sau đó.) Phép join đòi hỏi một thao tác shuffle. Một khóa được dùng để phân phối lại dữ liệu trên cluster; một lần nữa, một sự tràn ra đĩa xảy ra khi dữ liệu được ghi vào mỗi node. Biến đổi SQL lọc qua các hàng trong bộ nhớ và loại bỏ các hàng không dùng đến. Cuối cùng, Spark chuyển đổi dữ liệu thành định dạng Parquet, nén nó, và ghi nó trở lại S3. Airflow định kỳ gọi lại Spark để xem liệu công việc đã hoàn thành chưa. Một khi nó xác nhận rằng công việc đã kết thúc, nó đánh dấu toàn bộ DAG Airflow là đã hoàn thành. (Lưu ý rằng chúng ta có hai cấu trúc DAG ở đây, một DAG Airflow và một DAG đặc thù cho công việc Spark.) Logic nghiệp vụ và dữ liệu dẫn xuất Một trong những trường hợp sử dụng phổ biến nhất cho biến đổi là để kết xuất logic nghiệp vụ. Chúng tôi đã đặt thảo luận này dưới các biến đổi batch vì đây là nơi loại biến đổi này xảy ra thường xuyên nhất, nhưng lưu ý rằng nó cũng có thể xảy ra trong một pipeline streaming. Giả sử một công ty dùng nhiều phép tính lợi nhuận nội bộ chuyên biệt. Một phiên bản có thể nhìn vào lợi nhuận trước các chi phí tiếp thị, và một phiên bản khác có thể nhìn vào một lợi nhuận sau khi trừ các chi phí tiếp thị. Mặc dù điều này có vẻ như một bài tập kế toán đơn giản, mỗi chỉ số này đều cực kỳ phức tạp để kết xuất. Lợi nhuận trước các chi phí tiếp thị có thể cần tính đến các đơn hàng gian lận; việc xác định một ước lượng lợi nhuận hợp lý cho ngày làm việc trước đó kéo theo việc ước tính bao nhiêu phần trăm doanh thu và lợi nhuận tốt cuộc sẽ bị mất

cho các đơn hàng bị hủy trong những ngày tới khi nhóm chống gian lận điều tra các đơn hàng đáng ngờ. Có một cờ đặc biệt trong database chỉ ra một đơn hàng với xác suất gian lận cao, hoặc một đơn hàng đã bị tự động hủy không? Liệu doanh nghiệp có giả định rằng một tỷ lệ phần trăm nhất định của các đơn hàng sẽ bị hủy vì gian lận ngay cả trước khi quá trình đánh giá rủi-ro-gian-lận đã hoàn thành cho các đơn hàng cụ thể? Đối với các lợi nhuận sau các chi phí tiếp thị, chúng ta phải tính đến tất cả các phức tạp của chỉ số trước, cộng với các chi phí tiếp thị được quy cho đơn hàng cụ thể. Liệu công ty có một mô hình quy gán (attribution) ngẫu thơ—ví dụ, các chi phí tiếp thị được quy cho các món hàng có trọng số theo giá? Các chi phí tiếp thị cũng có thể được quy theo từng phòng ban, hoặc loại món hàng, hoặc—trong các tổ chức tinh vi nhất—theo từng món hàng riêng lẻ dựa trên các lần nhấp quảng cáo của người dùng. Biến đổi logic nghiệp vụ sinh ra phiên bản tinh tế này của lợi nhuận phải tích hợp tất cả các sự tinh vi của quy gán—tức một mô hình liên kết các đơn hàng với các quảng cáo và chi phí quảng cáo cụ thể. Liệu dữ liệu quy gán có được lưu trong ruột của các script ETL, hay nó được kéo từ một bảng được tự động sinh ra từ dữ liệu nền tảng quảng cáo? Loại dữ liệu báo cáo này là một ví dụ điển hình của dữ liệu dẫn xuất (derived data)—dữ liệu được tính từ dữ liệu khác được lưu trong một hệ thống dữ liệu. Những người chỉ trích dữ liệu dẫn xuất sẽ chỉ ra rằng việc ETL duy trì tính nhất quán trong các chỉ số dẫn xuất là thách thức. Ví dụ, nếu công ty cập nhật mô hình quy gán của mình, thay đổi này có thể cần được merge vào nhiều script ETL cho báo cáo. (Các script ETL khét tiếng vì phá vỡ nguyên tắc DRY.) Cập nhật các script ETL này là một quá trình thủ công và tốn-nhiều-công-sức, liên quan đến chuyên môn lĩnh vực trong logic xử lý và các thay đổi trước đó. Các script đã cập nhật cũng phải được kiểm chứng về tính nhất quán và chính xác. Theo quan điểm của chúng tôi, đây là những lời chỉ trích chính đáng nhưng không nhất thiết rất xây dựng vì cái thay thế cho dữ liệu dẫn xuất trong trường hợp này cũng đáng ghét không kém. Các nhà phân tích sẽ cần chạy các truy vấn báo cáo của họ nếu dữ liệu lợi nhuận không được lưu trong data warehouse, bao gồm cả logic lợi nhuận. Cập nhật các script ETL phức tạp để biểu diễn chính xác các thay đổi đối với logic nghiệp vụ là một nhiệm vụ áp đảo, tốn-nhiều-công-sức, nhưng việc khiến các nhà phân tích cập nhật các truy vấn báo cáo của họ một cách nhất quán thì gần như bất khả thi. Một lựa chọn thay thế thú vị là đẩy logic nghiệp vụ vào một tầng chỉ số (metrics layer), nhưng vẫn tận dụng data warehouse hoặc công cụ khác để làm việc nâng-vác-tính-toán nặng. Một tầng chỉ số mã hóa logic nghiệp vụ và cho phép các nhà phân tích và người dùng dashboard xây dựng các phân tích phức tạp từ một thư viện các chỉ số đã định nghĩa. Tầng chỉ số sinh các truy vấn từ các chỉ số và gửi chúng tới database. Chúng ta thảo luận các tầng ngữ nghĩa và chỉ số chi tiết hơn ở Chương 9. MapReduce Không thảo luận nào về biến đổi batch có thể hoàn chỉnh mà không đề cập đến MapReduce. Điều này không phải vì MapReduce được dùng rộng rãi bởi các kỹ sư dữ liệu ngày nay. MapReduce là khuôn mẫu biến đổi dữ liệu batch định nghĩa của kỷ nguyên big data, nó vẫn ảnh hưởng đến nhiều hệ thống phân tán mà các kỹ sư dữ liệu dùng ngày nay, và nó hữu ích cho các kỹ sư dữ liệu để hiểu ở một mức cơ bản. MapReduce được giới thiệu bởi Google trong một bài báo tiếp nối bài báo của họ về GFS. Ban đầu nó là khuôn mẫu xử lý trên thực tế của Hadoop, công nghệ tương tự mã nguồn mở của GFS mà chúng ta đã giới thiệu ở Chương 6. Một công việc MapReduce đơn giản gồm một tập các tác vụ map đọc các khối dữ liệu riêng lẻ rải rác trên các node, tiếp theo là một shuffle phân phối lại dữ liệu kết quả trên cluster và một bước reduce tổng hợp dữ liệu trên mỗi node. Ví dụ, giả sử chúng ta muốn chạy truy vấn SQL sau:

```
SELECT COUNT(*) , user_id
FROM user_events
GROUP BY user_id;
```

The table data is spread across nodes in data blocks; the MapReduce job generates one map task

per block. Each map task essentially runs the query on a single block—i.e., it generates a count for each user ID that appears in the block. While a block might contain hundreds of megabytes, the full table could be petabytes in size. However, the map portion of the job is a nearly perfect example of embarrassing parallelism; the data scan rate across the full cluster essentially scales linearly with the number of nodes. We then need to aggregate (reduce) to gather results from the full cluster. We're not gathering results to a single node; rather, we redistribute results by key so that each key ends up on one and only one node. This is the shuffle step, which is often executed using a hashing algorithm on keys. Once the map results have been shuffled, we sum the results for each key. The key/count pairs can be written to the local disk on the node where they are computed. We collect the results stored across nodes to view the full query results. Real-world MapReduce jobs can be far more complex than what we describe here. A complex query that filters with a WHERE clause joins three tables and applies a window function that would consist of many map and reduce stages. After MapReduce Google's original MapReduce model is extremely powerful but is now viewed as excessively rigid. It utilizes numerous short-lived ephemeral tasks that read from and write to disk. In particular, no intermediate state is preserved in memory; all data is transferred between tasks by storing it to disk or pushing it over the network. This simplifies state and workflow management, and minimizes memory consumption, but it can also drive high-disk bandwidth utilization and increase processing time. The MapReduce paradigm was constructed around the idea that magnetic disk capacity and bandwidth were so cheap that it made sense to simply throw a massive amount of disk at data to realize ultra-fast queries. This worked to an extent; MapReduce repeatedly set data processing records during the early days of Hadoop. However, we have lived in a post-MapReduce world for quite some time. Post-MapReduce processing does not truly discard MapReduce; it still includes the elements of map, shuffle, and reduce, but it relaxes the constraints of MapReduce to allow for in-memory caching. Recall that RAM is much faster than SSD and HDDs in transfer speed and seek time. Persisting even a tiny amount of judiciously chosen data in memory can dramatically speed up specific data processing tasks and utterly crush the performance of MapReduce. For example, Spark, BigQuery, and various other data processing frameworks were designed around in-memory processing. These frameworks treat data as a distributed set that resides in memory. If data overflows available memory, this causes a spill to disk. The disk is treated as a second-class data-storage layer for processing, though it is still highly valuable. The cloud is one of the drivers for the broader adoption of memory caching; it is much more effective to lease memory during a specific processing job than to own it 24 hours a day. Advancements in leveraging memory for transformations will continue to yield gains for the foreseeable future.

Dữ liệu bảng được trải trên các node trong các khối dữ liệu; công việc MapReduce sinh một tác vụ map cho mỗi khối. Mỗi tác vụ map về cơ bản chạy truy vấn trên một khối duy nhất—tức nó sinh một bộ đếm cho mỗi ID người dùng xuất hiện trong khối. Trong khi một khối có thể chứa hàng trăm megabyte, toàn bộ bảng có thể có kích thước petabyte. Tuy nhiên, phần map của công việc là một ví dụ gần-như-hoàn-hảo về song song dễ-đạt (embarrassing parallelism); tốc độ quét dữ liệu trên toàn cluster về cơ bản mở rộng tuyến tính với số node. Sau đó chúng ta cần tổng hợp (reduce) để gom các kết quả từ toàn cluster. Chúng ta không gom các kết quả về một node duy nhất; thay vào đó, chúng ta phân phối lại các kết quả theo khóa để mỗi khóa kết thúc trên một và chỉ một node. Đây là bước shuffle, vốn thường được thực thi dùng một thuật toán hashing trên các khóa. Một khi các kết quả map đã được shuffle, chúng ta cộng các kết quả cho mỗi khóa. Các cặp khóa/đếm có thể được ghi vào đĩa cục bộ trên node nơi chúng được tính. Chúng ta gom các kết quả được lưu trên các node để xem toàn bộ kết quả truy vấn. Các công việc MapReduce thực-tế có thể phức tạp hơn nhiều so với những gì chúng tôi mô tả ở đây. Một truy vấn phức tạp lọc với một mệnh đề WHERE, join ba bảng, và áp dụng một hàm cửa sổ (window function) sẽ gồm nhiều giai đoạn map và reduce. Sau MapReduce Mô hình MapReduce gốc của Google cực kỳ mạnh mẽ nhưng giờ được xem là cứng nhắc quá mức. Nó sử dụng vô số tác vụ

phù-du ngắn-hạn đọc từ và ghi vào đĩa. Cụ thể, không có trạng thái trung gian nào được giữ trong bộ nhớ; tất cả dữ liệu được chuyển giữa các tác vụ bằng cách lưu nó vào đĩa hoặc đẩy nó qua mạng. Điều này đơn giản hóa việc quản lý trạng thái và luồng công việc, và giảm thiểu tiêu thụ bộ nhớ, nhưng nó cũng có thể đẩy mức sử dụng băng thông đĩa cao và tăng thời gian xử lý. Khuôn mẫu MapReduce được xây dựng quanh ý tưởng rằng dung lượng và băng thông đĩa từ rẻ đến mức hợp lý khi đơn giản ném một lượng đĩa khổng lồ vào dữ liệu để hiện thực hóa các truy vấn siêu nhanh. Điều này hoạt động ở một mức độ nào đó; MapReduce liên tục lập các kỷ lục xử lý dữ liệu trong những ngày đầu của Hadoop. Tuy nhiên, chúng ta đã sống trong một thế giới hậu-MapReduce khá lâu rồi. Xử lý hậu-MapReduce không thực sự loại bỏ MapReduce; nó vẫn bao gồm các phần tử map, shuffle, và reduce, nhưng nó nói lỏng các ràng buộc của MapReduce để cho phép cache trong-bộ-nhớ. Hãy nhớ rằng RAM nhanh hơn nhiều so với SSD và HDD về tốc độ truyền và thời gian tìm kiếm. Lưu giữ bền vững dù chỉ một lượng nhỏ dữ liệu được chọn lựa khôn ngoan trong bộ nhớ có thể làm tăng tốc đáng kể các nhiệm vụ xử lý dữ liệu cụ thể và đè bẹp hoàn toàn hiệu năng của MapReduce. Ví dụ, Spark, BigQuery, và nhiều framework xử lý dữ liệu khác được thiết kế quanh xử lý trong-bộ-nhớ. Các framework này coi dữ liệu như một tập phân tán nằm trong bộ nhớ. Nếu dữ liệu tràn quá bộ nhớ khả dụng, điều này gây ra một sự tràn ra đĩa. Đĩa được coi như một tầng lưu-trữ-dữ-liệu hạng-hai cho việc xử lý, mặc dù nó vẫn rất giá trị. Đám mây là một trong những động lực cho việc áp dụng rộng rãi hơn việc cache bộ nhớ; sẽ hiệu quả hơn nhiều khi thuê bộ nhớ trong một công việc xử lý cụ thể hơn là sở hữu nó 24 giờ một ngày. Những tiến bộ trong việc tận dụng bộ nhớ cho các biến đổi sẽ tiếp tục mang lại lợi ích trong tương lai gần.

Materialized Views, Federation, and Query Virtualization In this section, we look at several techniques that virtualize query results by presenting them as table-like objects. These techniques can become part of a transformation pipeline or sit right before end-user data consumption. Views First, let's review views to set the stage for materialized views. A view is a database object that we can select from just like any other table. In practice, a view is just a query that references other tables. When we select from a view, that database creates a new query that combines the view subquery with our query. The query optimizer then optimizes and runs the full query. Views play a variety of roles in a database. First, views can serve a security role. For example, views can select only specific columns and filter rows, thus providing restricted data access. Various views can be created for job roles depending on user data access. Second, a view might be used to provide a current deduplicated picture of data. If we're using an insert-only pattern, a view may be used to return a deduplicated version of a table showing only the latest version of each record. Third, views can be used to present common data access patterns. Suppose that marketing analysts must frequently run a query that joins five tables. We could create a view that joins together these five tables into a wide table. Analysts can then write queries that filter and aggregate on top of this view. **Materialized views** We mentioned materialized views in our earlier discussion of query caching. A potential disadvantage of (nonmaterialized) views is that they don't do any precomputation. In the example of a view that joins five tables, this join must run every time a marketing analyst runs a query on this view, and the join could be extremely expensive. A materialized view does some or all of the view computation in advance. In our example, a materialized view might save the five table join results every time a change occurs in the source tables. Then, when a user references the view, they're querying from the prejoined data. A materialized view is a de facto transformation step, but the database manages execution for convenience. Materialized views may also serve a significant query optimization role depending on the database, even for queries that don't directly reference them. Many query optimizers can identify queries that "look like" a materialized view. An analyst may run a query that uses a filter that appears in a materialized view. The optimizer will rewrite the query to select from the precomputed results. **Composable materialized views** In general, materialized views do not allow for composition—that is, a materialized view cannot

select from another materialized view. However, we've recently seen the emergence of tools that support this capability. For example, Databricks has introduced the notion of live tables. Each table is updated as data arrives from sources. Data flows down to subsequent tables asynchronously. Federated queries Federated queries are a database feature that allows an OLAP database to select from an external data source, such as object storage or RDBMS. For example, let's say you need to combine data across object storage and various tables in MySQL and PostgreSQL databases. Your data warehouse can issue a federated query to these sources and return the combined results (Figure 8-19).

Materialized View, Liên hợp, và Ảo hóa Truy vấn Trong phần này, chúng ta xem xét một số kỹ thuật ảo hóa các kết quả truy vấn bằng cách trình bày chúng như các đối tượng giống-bảng. Các kỹ thuật này có thể trở thành một phần của một pipeline biến đổi hoặc nằm ngay trước việc tiêu thụ dữ liệu của người-dùng-cuối. View Đầu tiên, hãy điểm lại các view để dọn sân cho các materialized view. Một view là một đối tượng database mà chúng ta có thể select từ đó y như bất kỳ bảng nào khác. Trong thực tế, một view chỉ là một truy vấn tham chiếu các bảng khác. Khi chúng ta select từ một view, database đó tạo một truy vấn mới kết hợp truy vấn con của view với truy vấn của chúng ta. Bộ tối ưu truy vấn sau đó tối ưu và chạy toàn bộ truy vấn. Các view đóng nhiều vai trò khác nhau trong một database. Thứ nhất, các view có thể phục vụ một vai trò bảo mật. Ví dụ, các view có thể chỉ select các cột cụ thể và lọc các hàng, từ đó cung cấp truy cập dữ liệu bị hạn chế. Nhiều view khác nhau có thể được tạo cho các vai trò công việc tùy thuộc vào quyền truy cập dữ liệu của người dùng. Thứ hai, một view có thể được dùng để cung cấp một bức tranh dữ liệu đã-khử-trùng-lặp hiện tại. Nếu chúng ta đang dùng một khuôn mẫu chỉ-chèn, một view có thể được dùng để trả về một phiên bản đã-khử-trùng-lặp của một bảng chỉ hiển thị phiên bản mới nhất của mỗi bản ghi. Thứ ba, các view có thể được dùng để trình bày các khuôn mẫu truy cập dữ liệu phổ biến. Giả sử các nhà phân tích tiếp thị phải thường xuyên chạy một truy vấn join năm bảng. Chúng ta có thể tạo một view join năm bảng này thành một bảng rộng. Các nhà phân tích sau đó có thể viết các truy vấn lọc và tổng hợp trên đỉnh view này. Materialized view Chúng tôi đã đề cập đến các materialized view trong thảo luận trước đó về cache truy vấn. Một nhược điểm tiềm tàng của các view (không-được-vật-chất-hóa) là chúng không làm bất kỳ tiền-tính-toán nào. Trong ví dụ về một view join năm bảng, join này phải chạy mỗi khi một nhà phân tích tiếp thị chạy một truy vấn trên view này, và join có thể cực kỳ tốn kém. Một materialized view làm một phần hoặc tất cả việc tính toán của view trước. Trong ví dụ của chúng ta, một materialized view có thể lưu các kết quả join năm bảng mỗi khi một thay đổi xảy ra trong các bảng nguồn. Sau đó, khi một người dùng tham chiếu view, họ đang truy vấn từ dữ liệu đã-join-trước. Một materialized view là một bước biến đổi trên thực tế, nhưng database quản lý việc thực thi cho tiện lợi. Các materialized view cũng có thể phục vụ một vai trò tối ưu truy vấn đáng kể tùy theo database, ngay cả cho các truy vấn không trực tiếp tham chiếu chúng. Nhiều bộ tối ưu truy vấn có thể nhận diện các truy vấn “trông giống” một materialized view. Một nhà phân tích có thể chạy một truy vấn dùng một bộ lọc xuất hiện trong một materialized view. Bộ tối ưu sẽ viết lại truy vấn để select từ các kết quả đã-tiền-tính-toán. Materialized view có thể kết hợp được (composable) Nhìn chung, các materialized view không cho phép kết hợp—tức là, một materialized view không thể select từ một materialized view khác. Tuy nhiên, gần đây chúng tôi đã thấy sự xuất hiện của các công cụ hỗ trợ khả năng này. Ví dụ, Databricks đã giới thiệu khái niệm các bảng trực tiếp (live table). Mỗi bảng được cập nhật khi dữ liệu đến từ các nguồn. Dữ liệu chảy xuống các bảng tiếp theo một cách bất đồng bộ. Truy vấn liên hợp Các truy vấn liên hợp (federated query) là một tính năng database cho phép một database OLAP select từ một nguồn dữ liệu bên ngoài, như object storage hoặc RDBMS. Ví dụ, giả sử bạn cần kết hợp dữ liệu giữa object storage và nhiều bảng khác nhau trong các database MySQL và PostgreSQL. Data warehouse của bạn có thể phát một truy vấn liên

hợp tới các nguồn này và trả về các kết quả đã kết hợp (Hình 8-19).

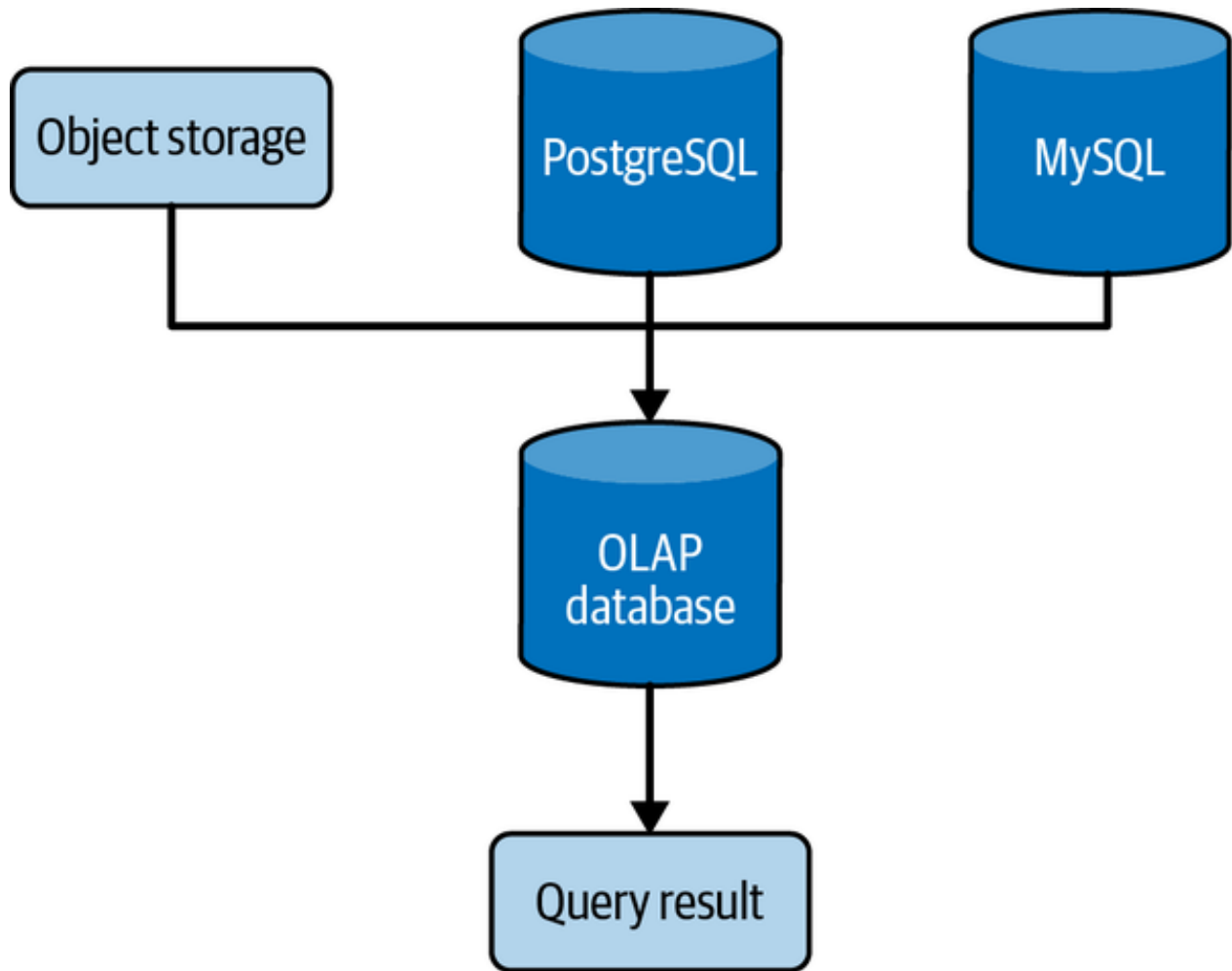


Figure 8-19. An OLAP database issues a federated query that gets data from object storage, MySQL, and PostgreSQL and returns a query result with the combined data. As another example, Snowflake supports the notion of external tables defined on S3 buckets. An external data location and a file format are defined when creating the table, but data is not yet ingested into the table. When the external table is queried, Snowflake reads from S3 and processes the data based on the parameters set at the time of the table's creation. We can even join S3 data to internal database tables. This makes Snowflake and similar databases more compatible with a data lake environment. Some OLAP systems can convert federated queries into materialized views. This gives us much of the performance of a native table without the need to manually ingest data every time the external source changes. The materialized view gets updated whenever the external data changes. Data virtualization Data virtualization is closely related to federated queries, but this typically entails a data processing and query system that doesn't store data internally. Right now, Trino (e.g., Starburst) and Presto are examples par excellence. Any query/processing engine that supports external tables can serve as a data virtualization engine. The most significant considerations with data virtualization are supported external sources and performance. A closely related concept is the notion of **query pushdown**. Suppose I wanted to query data from Snowflake, join data from a MySQL database, and filter the results. Query pushdown aims to move as much work as possible to the source databases. The engine might look for ways to push filtering predicates into the queries on the source systems. This serves two purposes: first, it offloads computation

from the virtualization layer, taking advantage of the query performance of the source. Second, it potentially reduces the quantity of data that must push across the network, a critical bottleneck for virtualization performance. Data virtualization is a good solution for organizations with data stored across various data sources. However, data virtualization should not be used haphazardly. For example, virtualizing a production MySQL database doesn't solve the core problem of analytics queries adversely impacting the production system—because Trino does not store data internally, it will pull from MySQL every time it runs a query. Alternatively, data virtualization can be used as a component of data ingestion and processing pipelines. For instance, Trino might be used to select from MySQL once a day at midnight when the load on the production system is low. Results could be saved into S3 for consumption by downstream transformations and daily queries, protecting MySQL from direct analytics queries.

Hình 8-19. Một database OLAP phát một truy vấn liên hợp lấy dữ liệu từ object storage, MySQL, và PostgreSQL và trả về một kết quả truy vấn với dữ liệu đã kết hợp. Một ví dụ khác, Snowflake hỗ trợ khái niệm các bảng ngoài (external table) được định nghĩa trên các bucket S3. Một vị trí dữ liệu ngoài và một định dạng tệp được định nghĩa khi tạo bảng, nhưng dữ liệu chưa được thu nạp vào bảng. Khi bảng ngoài được truy vấn, Snowflake đọc từ S3 và xử lý dữ liệu dựa trên các tham số được đặt tại thời điểm tạo bảng. Chúng ta thậm chí có thể join dữ liệu S3 với các bảng database nội bộ. Điều này khiến Snowflake và các database tương tự tương thích hơn với một môi trường data lake. Một số hệ thống OLAP có thể chuyển đổi các truy vấn liên hợp thành các materialized view. Điều này cho chúng ta phần lớn hiệu năng của một bảng gốc mà không cần thu nạp dữ liệu thủ công mỗi khi nguồn ngoài thay đổi. Materialized view được cập nhật bất cứ khi nào dữ liệu ngoài thay đổi. Ảo hóa dữ liệu Ảo hóa dữ liệu (data virtualization) liên quan chặt chẽ đến các truy vấn liên hợp, nhưng điều này thường kéo theo một hệ thống xử lý dữ liệu và truy vấn không lưu trữ dữ liệu nội bộ. Ngay lúc này, Trino (ví dụ Starburst) và Presto là các ví dụ tuyệt hảo. Bất kỳ engine truy vấn/xử lý nào hỗ trợ các bảng ngoài đều có thể phục vụ như một engine ảo hóa dữ liệu. Các cân nhắc đáng kể nhất với ảo hóa dữ liệu là các nguồn ngoài được hỗ trợ và hiệu năng. Một khái niệm liên quan chặt chẽ là khái niệm đẩy truy vấn xuống (query pushdown). Giả sử tôi muốn truy vấn dữ liệu từ Snowflake, join dữ liệu từ một database MySQL, và lọc các kết quả. Đẩy truy vấn xuống nhằm tới việc di chuyển càng nhiều công việc càng tốt sang các database nguồn. Engine có thể tìm các cách để đẩy các vị từ lọc vào các truy vấn trên các hệ thống nguồn. Điều này phục vụ hai mục đích: thứ nhất, nó gỡ tải tính toán khỏi tầng ảo hóa, tận dụng hiệu năng truy vấn của nguồn. Thứ hai, nó có khả năng giảm lượng dữ liệu phải đẩy qua mạng, một nút thắt cổ chai tối quan trọng cho hiệu năng ảo hóa. Ảo hóa dữ liệu là một giải pháp tốt cho các tổ chức có dữ liệu lưu trữ trên nhiều nguồn dữ liệu khác nhau. Tuy nhiên, ảo hóa dữ liệu không nên được dùng một cách bừa bãi. Ví dụ, ảo hóa một database MySQL sản xuất không giải quyết vấn đề cốt lõi của các truy vấn phân tích tác động xấu đến hệ thống sản xuất—vì Trino không lưu trữ dữ liệu nội bộ, nó sẽ kéo từ MySQL mỗi khi nó chạy một truy vấn. Cách khác, ảo hóa dữ liệu có thể được dùng như một thành phần của các pipeline thu nạp và xử lý dữ liệu. Chẳng hạn, Trino có thể được dùng để select từ MySQL một lần một ngày lúc nửa đêm khi tải trên hệ thống sản xuất thấp. Các kết quả có thể được lưu vào S3 để được tiêu thụ bởi các biến đổi hạ nguồn và các truy vấn hằng ngày, bảo vệ MySQL khỏi các truy vấn phân tích trực tiếp.

Data virtualization can be viewed as a tool that expands the data lake to many more sources by abstracting away barriers used to silo data between organizational units. An organization can store frequently accessed, transformed data in S3 and virtualize access between various parts of the company. This fits closely with the notion of a **data mesh** (discussed in Chapter 3), wherein small teams are responsible for preparing their data for analytics and sharing it with the rest of the

company; virtualization can serve as a critical access layer for practical sharing.

Ảo hóa dữ liệu có thể được xem như một công cụ mở rộng data lake tới nhiều nguồn hơn nữa bằng cách trừu tượng hóa đi các rào cản được dùng để cô lập (silo) dữ liệu giữa các đơn vị tổ chức. Một tổ chức có thể lưu trữ dữ liệu được biến đổi, thường-xuyên-truy-cập trong S3 và ảo hóa truy cập giữa các bộ phận khác nhau của công ty. Điều này khớp chặt với khái niệm data mesh (thảo luận ở Chương 3), trong đó các nhóm nhỏ chịu trách nhiệm chuẩn bị dữ liệu của họ cho phân tích và chia sẻ nó với phần còn lại của công ty; ảo hóa có thể phục vụ như một tầng truy cập tối quan trọng cho việc chia sẻ thực tế.

Streaming Transformations and Processing We've already discussed stream processing in the context of queries. The difference between streaming transformations and streaming queries is subtle and warrants more explanation. Basics Streaming queries run dynamically to present a current view of data, as discussed previously. Streaming transformations aim to prepare data for downstream consumption. For instance, a data engineering team may have an incoming stream carrying events from an IoT source. These IoT events carry a device ID and event data. We wish to dynamically enrich these events with other device metadata, which is stored in a separate database. The stream-processing engine queries a separate database containing this metadata by device ID, generates new events with the added data, and passes it on to another stream. Live queries and triggered metrics run on this enriched stream (see Figure 8-20).

Biến đổi và Xử lý Streaming Chúng ta đã thảo luận xử lý luồng trong ngữ cảnh các truy vấn. Sự khác biệt giữa các biến đổi streaming và các truy vấn streaming là tinh tế và đáng được giải thích thêm. Cơ bản Các truy vấn streaming chạy động để trình bày một cái nhìn hiện tại của dữ liệu, như đã thảo luận trước đó. Các biến đổi streaming nhằm tới việc chuẩn bị dữ liệu cho việc tiêu thụ ở hạ nguồn. Chẳng hạn, một nhóm kỹ thuật dữ liệu có thể có một luồng đến mang các sự kiện từ một nguồn IoT. Các sự kiện IoT này mang một ID thiết bị và dữ liệu sự kiện. Chúng ta muốn làm giàu một cách động các sự kiện này với metadata thiết bị khác, vốn được lưu trong một database riêng. Engine xử-lý-luồng truy vấn một database riêng chứa metadata này theo ID thiết bị, sinh các sự kiện mới với dữ liệu đã thêm, và chuyển nó tiếp tới một luồng khác. Các truy vấn trực tiếp và các chỉ số được kích hoạt chạy trên luồng đã làm giàu này (xem Hình 8-20).

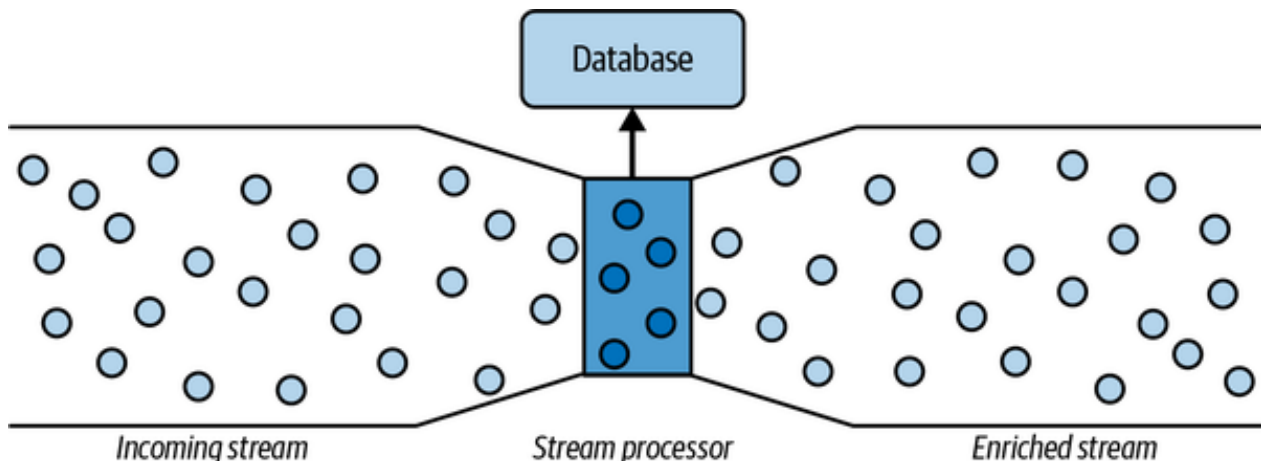


Figure 8-20. An incoming stream is carried by a streaming event platform and passed into a stream processor Transformations and queries are a continuum The line between transformations and queries is also blurry in batch processing, but the differences become even more subtle in the domain of streaming. For example, if we dynamically compute roll-up statistics on windows, and then send the output to a target stream, is this a transformation or a query? Maybe we will eventually adopt

new terminology for stream processing that better represents real-world use cases. For now, we will do our best with the terminology we have. Streaming DAGs One interesting notion closely related to stream enrichment and joins is the streaming DAG . We first talked 19 about this idea in our discussion of orchestration in Chapter 2 . Orchestration is inherently a batch concept, but what if we wanted to enrich, merge, and split multiple streams in real time? Let's take a simple example where streaming DAG would be useful. Suppose that we want to combine website clickstream data with IoT data. This will allow us to get a unified view of user activity by combining IoT events with clicks. Furthermore, each data stream needs to be preprocessed into a standard format (see Figure 8-21).

Hình 8-20. Một luồng đến được mang bởi một nền tảng sự kiện streaming và chuyển vào một bộ xử lý luồng. Biến đổi và truy vấn là một trực liên tục Ranh giới giữa các biến đổi và các truy vấn cũng mờ nhạt trong xử lý batch, nhưng các khác biệt trở nên còn tinh tế hơn trong lĩnh vực streaming. Ví dụ, nếu chúng ta tính động các thống kê tổng hợp (roll-up) trên các cửa sổ, và sau đó gửi đầu ra tới một luồng đích, thì đây là một biến đổi hay một truy vấn? Có lẽ cuối cùng chúng ta sẽ áp dụng thuật ngữ mới cho xử lý luồng phản ánh tốt hơn các trường hợp sử dụng thực-tế. Hiện tại, chúng ta sẽ làm hết sức với thuật ngữ chúng ta có. DAG streaming Một khái niệm thú vị liên quan chặt chẽ đến làm giàu luồng và các join là DAG streaming. Chúng ta đã nói lần đầu về ý tưởng này trong thảo luận của chúng ta về điều phối ở Chương 2. Điều phối về bản chất là một khái niệm batch, nhưng nếu chúng ta muốn làm giàu, merge, và tách nhiều luồng trong thời gian thực thì sao? Hãy lấy một ví dụ đơn giản nơi DAG streaming sẽ hữu ích. Giả sử chúng ta muốn kết hợp dữ liệu clickstream của website với dữ liệu IoT. Điều này sẽ cho phép chúng ta có một cái nhìn hợp nhất về hoạt động người dùng bằng cách kết hợp các sự kiện IoT với các lần nhấp. Hơn nữa, mỗi luồng dữ liệu cần được tiền-xử-lý thành một định dạng chuẩn (xem Hình 8-21).

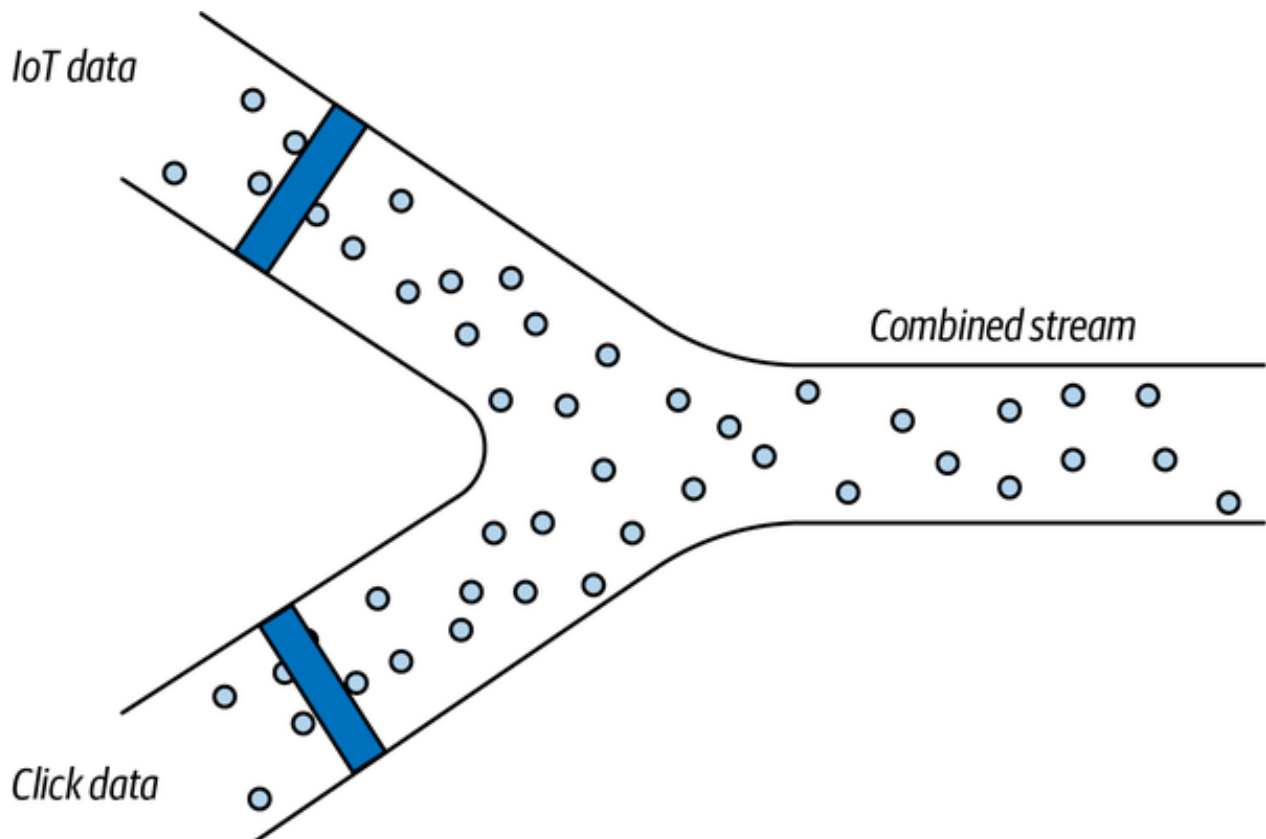


Figure 8-21. A simple streaming DAG This has long been possible by combining a streaming store (e.g., Kafka) with a stream processor (e.g., Flink). Creating the DAG amounted to building a complex Rube Goldberg machine, with numerous topics and processing jobs connected. Pulsar dramatically simplifies this process by treating DAGs as a core streaming abstraction. Rather than managing flows across several systems, engineers can define their streaming DAGs as code inside a single system. Micro-batch versus true streaming A long-running battle has been ongoing between micro-batch and true streaming approaches. Fundamentally, it's important to understand your use case, the performance requirements, and the performance capabilities of the framework in question. Micro-batching is a way to take a batch-oriented framework and apply it in a streaming situation. A micro-batch might run anywhere from every two minutes to every second. Some micro-batch frameworks (e.g., Apache Spark Streaming) are designed for this use case and will perform well with appropriately allocated resources at a high batch frequency. (In truth, DBAs and engineers have long used micro-batching with more traditional databases; this often led to horrific performance and resource consumption.) True streaming systems (e.g., Beam and Flink) are designed to process one event at a time. However, this comes with significant overhead. Also, it's important to note that even in these true streaming systems, many processes will still occur in batches. A basic enrichment process that adds data to individual events can deliver one event at a time with low latency. However, a triggered metric on windows may run every few seconds, every few minutes, etc. When you're using windows and triggers (hence, batch processing), what's the window frequency? What's the acceptable latency? If you are collecting Black Friday sales metrics published every few minutes, micro-batches are probably just fine as long as you set an appropriate micro-batch frequency. On the other hand, if your ops team is computing metrics every second to detect DDoS attacks, true streaming may be in order. When should you use one over the other? Frankly, there is no universal answer. The term micro-batch has often been used to dismiss competing technologies, but it may work just fine for your use case, and can be superior in many respects depending on your needs. If your team already has expertise in Spark, you will be able to spin up a Spark (micro-batch) streaming solution extremely fast. There's no substitute for domain expertise and real-world testing. Talk to experts who can present an even-handed opinion. You can also easily test the alternatives by spinning up tests on cloud infrastructure. Also, watch out for spurious benchmarks provided by vendors. Vendors are notorious for cherry-picking benchmarks and setting up artificial examples that don't match reality (recall our conversation on benchmarks in Chapter 4). Frequently, vendors will show massive advantages in their benchmark results but fail to deliver in the real world for your use case.

Hình 8-21. Một DAG streaming đơn giản. Điều này từ lâu đã có thể bằng cách kết hợp một kho streaming (ví dụ Kafka) với một bộ xử lý luồng (ví dụ Flink). Tạo DAG tương đương với việc xây dựng một cỗ máy Rube Goldberg phức tạp, với vô số topic và công việc xử lý được kết nối. Pulsar đơn giản hóa đáng kể quá trình này bằng cách coi các DAG như một sự trừu tượng streaming cốt lõi. Thay vì quản lý các luồng trên nhiều hệ thống, các kỹ sư có thể định nghĩa các DAG streaming của họ dưới dạng mã bên trong một hệ thống duy nhất. Vi-batch so với streaming thực Một trận chiến chạy-dài đã diễn ra liên tục giữa các cách tiếp cận vi-batch (micro-batch) và streaming thực. Về cơ bản, điều quan trọng là hiểu trường hợp sử dụng của bạn, các yêu cầu hiệu năng, và các khả năng hiệu năng của framework đang xét. Vi-batch là một cách để lấy một framework định hướng-batch và áp dụng nó trong một tình huống streaming. Một vi-batch có thể chạy bất cứ đâu từ mỗi hai phút đến mỗi giây. Một số framework vi-batch (ví dụ Apache Spark Streaming) được thiết kế cho trường hợp sử dụng này và sẽ vận hành tốt với các tài nguyên được cấp phù hợp ở một tần suất batch cao. (Thực ra, các DBA và kỹ sư từ lâu đã dùng vi-batch với các database truyền thống hơn; điều này thường dẫn đến hiệu năng và tiêu thụ tài nguyên kinh khủng.) Các hệ thống streaming thực (ví dụ Beam và Flink) được thiết kế để xử lý một sự kiện tại một thời điểm. Tuy nhiên, điều này đi kèm phụ phí đáng kể. Ngoài ra, điều quan trọng

cần lưu ý là ngay cả trong các hệ thống streaming thực này, nhiều quá trình vẫn sẽ xảy ra theo các lô. Một quá trình làm giàu cơ bản thêm dữ liệu vào các sự kiện riêng lẻ có thể giao một sự kiện tại một thời điểm với độ trễ thấp. Tuy nhiên, một chỉ số được kích hoạt trên các cửa sổ có thể chạy mỗi vài giây, mỗi vài phút, v.v. Khi bạn đang dùng các cửa sổ và trigger (do đó, xử lý batch), tần suất cửa sổ là bao nhiêu? Độ trễ chấp nhận được là bao nhiêu? Nếu bạn đang thu thập các chỉ số bán hàng Black Friday được xuất bản mỗi vài phút, các vi-batch có lẽ hoàn toàn ổn miễn là bạn đặt một tần suất vi-batch phù hợp. Mặt khác, nếu nhóm vận hành của bạn đang tính các chỉ số mỗi giây để phát hiện các cuộc tấn công DDoS, streaming thực có thể là điều cần thiết. Khi nào bạn nên dùng cái này thay vì cái kia? Thành thật mà nói, không có câu trả lời phổ quát. Thuật ngữ vi-batch thường được dùng để gạt bỏ các công nghệ cạnh tranh, nhưng nó có thể hoạt động hoàn toàn ổn cho trường hợp sử dụng của bạn, và có thể vượt trội ở nhiều khía cạnh tùy theo nhu cầu của bạn. Nếu nhóm của bạn đã có chuyên môn về Spark, bạn sẽ có thể dựng lên một giải pháp streaming Spark (vi-batch) cực kỳ nhanh. Không có gì thay thế được cho chuyên môn lĩnh vực và kiểm thử thực-tế. Hãy trao đổi với các chuyên gia có thể trình bày một ý kiến công bằng. Bạn cũng có thể dễ dàng kiểm thử các phương án bằng cách dựng lên các bài kiểm thử trên hạ tầng đám mây. Ngoài ra, hãy coi chừng các benchmark giả mạo được cung cấp bởi các nhà cung cấp. Các nhà cung cấp khét tiếng vì chọn-lọc-thiên-vị (cherry-pick) các benchmark và dựng các ví dụ nhân tạo không khớp với thực tế (hãy nhớ lại cuộc trò chuyện của chúng ta về các benchmark ở Chương 4). Thường thì các nhà cung cấp sẽ cho thấy các lợi thế khổng lồ trong các kết quả benchmark của họ nhưng không giao hàng được trong thế giới thực cho trường hợp sử dụng của bạn.

Whom You'll Work With — Những Người Bạn Sẽ Làm Việc Cùng

Queries, transformations, and modeling impact all stakeholders up and down the data engineering lifecycle. The data engineer is responsible for several things at this stage in the lifecycle. From a technical angle, the data engineer designs, builds, and maintains the integrity of the systems that query and transform data. The data engineer also implements data models within this system. This is the most “full-contact” stage where your focus is to add as much value as possible, both in terms of functioning systems and reliable and trustworthy data.

Truy vấn, biến đổi, và mô hình hóa tác động đến tất cả các bên liên quan dọc theo vòng đời kỹ thuật dữ liệu. Kỹ sư dữ liệu chịu trách nhiệm cho một số thứ ở giai đoạn này trong vòng đời. Từ góc độ kỹ thuật, kỹ sư dữ liệu thiết kế, xây dựng, và duy trì sự toàn vẹn của các hệ thống truy vấn và biến đổi dữ liệu. Kỹ sư dữ liệu cũng triển khai các mô hình dữ liệu trong hệ thống này. Đây là giai đoạn “đối kháng toàn diện” nhất nơi trọng tâm của bạn là thêm càng nhiều giá trị càng tốt, cả về mặt các hệ thống vận hành lẫn dữ liệu đáng tin cậy và đáng tin tưởng.

Upstream Stakeholders When it comes to transformations, upstream stakeholders can be broken into two broad categories: those who control the business definitions and those who control the systems generating data. When interfacing with upstream stakeholders about business definitions and logic, you'll need to know the data sources—what they are, how they're used, and the business logic and definitions involved. You'll work with the engineers in charge of these source systems and the business stakeholders who oversee the complementary products and apps. A data engineer might work alongside “the business” and technical stakeholders on a data model. The data engineer needs to be involved in designing the data model and later updates because of changes in business logic or new processes. Transformations are easy enough to do; just write a query and plop the results into a table or view. Creating them so they're both performant and valuable to the business

is another matter. Always keep the requirements and expectations of the business top of mind when transforming data. The stakeholders of the upstream systems want to make sure your queries and transformations minimally impact their systems. Ensure bidirectional communication about changes to the data models (column and index changes, for example) in source systems, as these can directly impact queries, transformations, and analytical data models. Data engineers should know about schema changes, including the addition or deletion of fields, data type changes, and anything else that might materially impact the ability to query and transform data.

Các Bên Liên quan Thượng nguồn Khi nói đến các biến đổi, các bên liên quan thượng nguồn có thể được chia thành hai loại rộng: những người kiểm soát các định nghĩa nghiệp vụ và những người kiểm soát các hệ thống sinh dữ liệu. Khi giao tiếp với các bên liên quan thượng nguồn về các định nghĩa và logic nghiệp vụ, bạn sẽ cần biết các nguồn dữ liệu—chúng là gì, chúng được dùng thế nào, và logic và các định nghĩa nghiệp vụ liên quan. Bạn sẽ làm việc với các kỹ sư phụ trách các hệ thống nguồn này và các bên liên quan nghiệp vụ giám sát các sản phẩm và ứng dụng hỗ trợ. Một kỹ sư dữ liệu có thể làm việc cùng “doanh nghiệp” và các bên liên quan kỹ thuật trên một mô hình dữ liệu. Kỹ sư dữ liệu cần tham gia vào việc thiết kế mô hình dữ liệu và các cập nhật sau này vì các thay đổi trong logic nghiệp vụ hoặc các quy trình mới. Các biến đổi đủ dễ để làm; chỉ cần viết một truy vấn và bỏ các kết quả vào một bảng hoặc view. Tạo chúng sao cho chúng vừa hiệu năng vừa giá trị cho doanh nghiệp lại là một vấn đề khác. Luôn giữ các yêu cầu và kỳ vọng của doanh nghiệp ở vị trí ưu tiên hàng đầu khi biến đổi dữ liệu. Các bên liên quan của các hệ thống thượng nguồn muốn đảm bảo các truy vấn và biến đổi của bạn tác động tối thiểu đến các hệ thống của họ. Hãy đảm bảo giao tiếp hai-chiều về các thay đổi đối với các mô hình dữ liệu (các thay đổi cột và index, chẳng hạn) trong các hệ thống nguồn, vì các thay đổi này có thể trực tiếp tác động đến các truy vấn, biến đổi, và các mô hình dữ liệu phân tích. Các kỹ sư dữ liệu nên biết về các thay đổi schema, bao gồm việc thêm hoặc xóa các trường, các thay đổi kiểu dữ liệu, và bất cứ điều gì khác có thể tác động vật chất đến khả năng truy vấn và biến đổi dữ liệu.

Downstream Stakeholders Transformations are where data starts providing utility to downstream stakeholders. Your downstream stakeholders include many people, including data analysts, data scientists, ML engineers, and “the business.” Collaborate with them to ensure the data model and transformations you provide are performant and useful. In terms of performance, queries should execute as quickly as possible in the most cost-effective way. What do we mean by useful ? Analysts, data scientists, and ML engineers should be able to query a data source with the confidence the data is of the highest quality and completeness, and can be integrated into their workflows and data products. The business should be able to trust that transformed data is accurate and actionable.

Các Bên Liên quan Hạ nguồn Các biến đổi là nơi dữ liệu bắt đầu cung cấp tiện ích cho các bên liên quan hạ nguồn. Các bên liên quan hạ nguồn của bạn bao gồm nhiều người, bao gồm các nhà phân tích dữ liệu, nhà khoa học dữ liệu, kỹ sư ML, và “doanh nghiệp”. Hãy cộng tác với họ để đảm bảo mô hình dữ liệu và các biến đổi bạn cung cấp là hiệu năng và hữu ích. Về mặt hiệu năng, các truy vấn nên thực thi càng nhanh càng tốt theo cách tiết kiệm chi phí nhất. Chúng tôi muốn nói gì khi nói hữu ích? Các nhà phân tích, nhà khoa học dữ liệu, và kỹ sư ML nên có thể truy vấn một nguồn dữ liệu với sự tự tin rằng dữ liệu có chất lượng và đầy đủ cao nhất, và có thể được tích hợp vào các luồng công việc và sản phẩm dữ liệu của họ. Doanh nghiệp nên có thể tin tưởng rằng dữ liệu đã biến đổi là chính xác và khả thi.

Undercurrents — Các Dòng chảy Ngầm

The transformation stage is where your data mutates and morphs into something useful for the business. Because there are many moving parts, the undercurrents are especially critical at this stage.

Giai đoạn biến đổi là nơi dữ liệu của bạn biến hóa và biến đổi thành thứ hữu ích cho doanh nghiệp. Vì có nhiều bộ phận chuyển động, các dòng chảy ngầm đặc biệt tối quan trọng ở giai đoạn này.

Security Queries and transformations combine disparate datasets into new datasets. Who has access to this new dataset? If someone does have access to a dataset, continue to control who has access to a dataset's column, row, and cell-level access. Be aware of attack vectors against your database at query time. Read/write privileges to the database must be tightly monitored and controlled. Query access to the database must be controlled in the same way as you normally control access to your organization's systems and environments. Keep credentials hidden; avoid copying and pasting passwords, access tokens, or other credentials into code or unencrypted files. It's shockingly common to see code in GitHub repositories with database usernames and passwords pasted directly in the codebase! It goes without saying, don't share passwords with other users. Finally, never allow unsecured or unencrypted data to traverse the public internet.

An ninh Các truy vấn và biến đổi kết hợp các tập dữ liệu rời rạc thành các tập dữ liệu mới. Ai có quyền truy cập vào tập dữ liệu mới này? Nếu ai đó có quyền truy cập vào một tập dữ liệu, hãy tiếp tục kiểm soát ai có quyền truy cập vào cột, hàng, và cấp-ô (cell-level) của một tập dữ liệu. Hãy nhận biết về các vector tấn công nhắm vào database của bạn tại thời điểm truy vấn. Các đặc quyền đọc/ghi vào database phải được giám sát và kiểm soát chặt chẽ. Quyền truy cập truy vấn vào database phải được kiểm soát theo cùng cách bạn thường kiểm soát quyền truy cập vào các hệ thống và môi trường của tổ chức bạn. Hãy giữ các thông tin xác thực được ẩn; tránh sao chép và dán mật khẩu, access token, hoặc các thông tin xác thực khác vào mã hoặc các tệp không-mã-hóa. Thật sốc là chuyện phổ biến khi thấy mã trong các kho GitHub với các tên người dùng và mật khẩu database được dán trực tiếp vào trong codebase! Khỏi cần phải nói, đừng chia sẻ mật khẩu với các người dùng khác. Cuối cùng, đừng bao giờ cho phép dữ liệu không-được-bảo-mật hoặc không-mã-hóa đi qua internet công cộng.

Data Management Though data management is essential at the source system stage (and every other stage of the data engineering lifecycle), it's especially critical at the transformation stage. Transformation inherently creates new datasets that need to be managed. As with other stages of the data engineering lifecycle, it's critical to involve all stakeholders in data models and transformations and manage their expectations. Also, make sure everyone agrees on naming conventions that align with the respective business definitions of the data. Proper naming conventions should be reflected in easy-to-understand field names. Users can also check in a **data catalog** for more clarity on what the field means when it was created, who maintains the dataset, and other relevant information. Accounting for definitional accuracy is key at the transformation stage. Does the transformation adhere to the expected business logic? Increasingly, the notion of a semantic or metrics layer that sits independent of transformations is becoming popular. Instead of enforcing business logic within the transformation at runtime, why not keep these definitions as a standalone stage before your transformation layer? While it's still early days, expect to see semantic and metrics layers becoming more popular and commonplace in data engineering and data management. Because transformations involve mutating data, it's critical to ensure that the data you're using is free of defects and represents ground truth. If MDM is an option at your company, pursue its implementation. Conformed dimensions and other transformations rely on MDM to preserve data's original integrity and ground truth. If MDM isn't possible, work with upstream stakeholders

who control the data to ensure that any data you're transforming is correct and complies with the agreed-upon business logic. Data transformations make it potentially difficult to know how a dataset was derived along the same lines. In Chapter 6, we discussed data catalogs. As we transform data, **data lineage** tools become invaluable. Data lineage tools help both data engineers, who must understand previous transformation steps as they create new transformations, and analysts, who need to understand where data came from as they run queries and build reports. Finally, what impact does **regulatory compliance** have on your data model and transformations? Are sensitive fields data masked or obfuscated if necessary? Do you have the ability to delete data in response to deletion requests? Does your data lineage tracking allow you to see data derived from deleted data, and rerun transformations to remove data downstream of raw sources?

Quản lý Dữ liệu Mặc dù quản lý dữ liệu là thiết yếu ở giai đoạn hệ thống nguồn (và mọi giai đoạn khác của vòng đời kỹ thuật dữ liệu), nó đặc biệt tối quan trọng ở giai đoạn biến đổi. Biến đổi vốn dĩ tạo ra các tập dữ liệu mới cần được quản lý. Cũng như với các giai đoạn khác của vòng đời kỹ thuật dữ liệu, điều tối quan trọng là lôi kéo tất cả các bên liên quan vào các mô hình dữ liệu và biến đổi và quản lý các kỳ vọng của họ. Ngoài ra, hãy đảm bảo mọi người đồng ý về các quy ước đặt tên căn chỉnh với các định nghĩa nghiệp vụ tương ứng của dữ liệu. Các quy ước đặt tên đúng đắn nên được phản ánh trong các tên trường dễ-hiểu. Người dùng cũng có thể tra cứu trong một data catalog để có rõ ràng hơn về ý nghĩa của trường khi nó được tạo, ai duy trì tập dữ liệu, và thông tin liên quan khác. Việc tính đến độ chính xác định nghĩa là then chốt ở giai đoạn biến đổi. Liệu biến đổi có tuân thủ logic nghiệp vụ được mong đợi? Ngày càng nhiều, khái niệm về một tầng ngữ nghĩa hoặc chỉ số nằm độc lập với các biến đổi đang trở nên phổ biến. Thay vì thực thi logic nghiệp vụ bên trong biến đổi tại thời điểm chạy, tại sao không giữ các định nghĩa này như một giai đoạn độc lập trước tầng biến đổi của bạn? Mặc dù vẫn còn những ngày đầu, hãy kỳ vọng thấy các tầng ngữ nghĩa và chỉ số trở nên phổ biến và thông dụng hơn trong kỹ thuật dữ liệu và quản lý dữ liệu. Vì các biến đổi liên quan đến việc thay đổi dữ liệu, điều tối quan trọng là đảm bảo rằng dữ liệu bạn đang dùng không có lỗi và biểu diễn sự thật nền tảng (ground truth). Nếu MDM là một lựa chọn tại công ty bạn, hãy theo đuổi việc triển khai nó. Các dimension tuân thủ và các biến đổi khác dựa vào MDM để bảo tồn sự toàn vẹn gốc và sự thật nền tảng của dữ liệu. Nếu MDM không khả thi, hãy làm việc với các bên liên quan thượng nguồn kiểm soát dữ liệu để đảm bảo rằng bất kỳ dữ liệu nào bạn đang biến đổi là đúng và tuân thủ logic nghiệp vụ đã-thống-nhất. Các biến đổi dữ liệu khiến việc biết một tập dữ liệu được dẫn xuất như thế nào có khả năng trở nên khó khăn theo cùng nghĩa đó. Ở Chương 6, chúng ta đã thảo luận các data catalog. Khi chúng ta biến đổi dữ liệu, các công cụ truy vết nguồn gốc dữ liệu (data lineage) trở nên vô giá. Các công cụ data lineage giúp cả các kỹ sư dữ liệu, những người phải hiểu các bước biến đổi trước đó khi họ tạo các biến đổi mới, lẫn các nhà phân tích, những người cần hiểu dữ liệu đến từ đâu khi họ chạy các truy vấn và xây dựng các báo cáo. Cuối cùng, tuân thủ quy định có tác động gì lên mô hình dữ liệu và các biến đổi của bạn? Các trường nhạy cảm có được che (mask) dữ liệu hoặc làm mờ nếu cần không? Bạn có khả năng xóa dữ liệu để đáp ứng các yêu cầu xóa không? Việc theo dõi data lineage của bạn có cho phép bạn thấy dữ liệu được dẫn xuất từ dữ liệu đã xóa, và chạy lại các biến đổi để loại bỏ dữ liệu ở hạ nguồn của các nguồn thô không?

DataOps With queries and transformations, DataOps has two areas of concern: data and systems. You need to monitor and be alerted for changes or anomalies in these areas. The field of **data observability** is exploding right now, with a big focus on data reliability. There's even a recent job title called data reliability engineer. This section emphasizes data observability and data health, which focuses on the query and transformation stage. Let's start with the data side of DataOps. When you query data, are the inputs and outputs correct? How do you know? If this query is saved to a table,

is the schema correct? How about the shape of the data and related statistics such as min/max values, null counts, and more? You should run data-quality tests on the input datasets and the transformed dataset, which will ensure that the data meets the expectations of upstream and downstream users. If there's a data-quality issue in the transformation, you should have the ability to flag this issue, roll back the changes, and investigate the root cause. Now let's look at the Ops part of DataOps. How are the systems performing? Monitor metrics such as query queue length, query concurrency, memory usage, storage utilization, network latency, and disk I/O. Use metric data to spot bottlenecks and poor-performing queries that might be candidates for refactoring and tuning. If the query is perfectly fine, you'll have a good idea of where to tune the database itself (for instance, by clustering a table for faster lookup performance). Or, you may need to upgrade the database's compute resources. Today's cloud and SaaS databases give you a ton of flexibility for quickly upgrading (and downgrading) your system. Take a data-driven approach and use your observability metrics to pinpoint whether you have a query or a systems-related issue. The shift toward SaaS-based analytical databases changes the cost profile of data consumption. In the days of on-premises data warehouses, the system and licenses were purchased up front, with no additional usage cost. Whereas traditional data engineers would focus on performance optimization to squeeze the maximum utility out of their expensive purchases, data engineers working with cloud data warehouses that charge on a consumption basis need to focus on cost management and cost optimization. This is the practice of **FinOps** (see Chapter 4).

DataOps Với các truy vấn và biến đổi, DataOps có hai lĩnh vực quan tâm: dữ liệu và các hệ thống. Bạn cần giám sát và được cảnh báo về các thay đổi hoặc bất thường trong các lĩnh vực này. Lĩnh vực khả năng quan sát dữ liệu (data observability) đang bùng nổ ngay lúc này, với một trọng tâm lớn vào độ tin cậy dữ liệu. Thậm chí có một chức danh công việc gần đây gọi là kỹ sư độ-tin-cậy-dữ-liệu (data reliability engineer). Phần này nhấn mạnh khả năng quan sát dữ liệu và sức khỏe dữ liệu, vốn tập trung vào giai đoạn truy vấn và biến đổi. Hãy bắt đầu với mặt dữ liệu của DataOps. Khi bạn truy vấn dữ liệu, các đầu vào và đầu ra có đúng không? Làm sao bạn biết? Nếu truy vấn này được lưu vào một bảng, schema có đúng không? Còn hình dạng của dữ liệu và các thống kê liên quan như các giá trị min/max, các đếm null, và hơn nữa thì sao? Bạn nên chạy các bài kiểm thử chất-lượng-dữ-liệu trên các tập dữ liệu đầu vào và tập dữ liệu đã biến đổi, vốn sẽ đảm bảo rằng dữ liệu đáp ứng các kỳ vọng của các người dùng thượng nguồn và hạ nguồn. Nếu có một vấn đề chất-lượng-dữ-liệu trong biến đổi, bạn nên có khả năng đánh dấu vấn đề này, quay lui các thay đổi, và điều tra nguyên nhân gốc rễ. Bây giờ hãy nhìn vào phần Ops của DataOps. Các hệ thống đang vận hành thế nào? Hãy giám sát các chỉ số như độ dài hàng đợi truy vấn, tính đồng thời truy vấn, mức dùng bộ nhớ, mức dùng lưu trữ, độ trễ mạng, và I/O đĩa. Hãy dùng dữ liệu chỉ số để phát hiện các nút thắt cổ chai và các truy vấn vận hành kém có thể là ứng viên cho việc tái cấu trúc và tinh chỉnh. Nếu truy vấn hoàn toàn ổn, bạn sẽ có một ý tưởng tốt về nơi để tinh chỉnh bản thân database (chẳng hạn, bằng cách gom cụm một bảng để có hiệu năng tra cứu nhanh hơn). Hoặc, bạn có thể cần nâng cấp các tài nguyên tính toán của database. Các database đám mây và SaaS ngày nay cho bạn vô vàn sự linh hoạt để nhanh chóng nâng cấp (và hạ cấp) hệ thống của bạn. Hãy áp dụng một cách tiếp cận dựa-trên-dữ-liệu và dùng các chỉ số quan sát của bạn để xác định chính xác liệu bạn có một vấn đề liên-quan-đến-truy-vấn hay liên-quan-đến-hệ-thống. Sự chuyển dịch về phía các database phân tích dựa-trên-SaaS thay đổi hồ sơ chi phí của việc tiêu thụ dữ liệu. Trong những ngày của các data warehouse on-premises, hệ thống và các giấy phép được mua trả-trước, không có chi phí dùng bổ sung. Trong khi các kỹ sư dữ liệu truyền thống sẽ tập trung vào tối ưu hiệu năng để vắt ra tiện ích tối đa từ các khoản mua đắt đỏ của họ, các kỹ sư dữ liệu làm việc với các cloud data warehouse tính phí theo cơ sở tiêu thụ cần tập trung vào quản lý chi phí và tối ưu chi phí. Đây là thực hành FinOps (xem Chương 4).

Data Architecture The general rules of good data architecture in Chapter 3 apply to the transformation stage. Build robust systems that can process and transform data without imploding. Your choices for ingestion and storage will directly impact your general architecture’s ability to perform reliable queries and transformations. If the ingestion and storage are appropriate to your query and transformation patterns, you should be in a great place. On the other hand, if your queries and transformations don’t work well with your upstream systems, you’re in for a world of pain. For example, we often see data teams using the wrong data pipelines and databases for the job. A data team might connect a real-time **data pipeline** to an RDBMS or Elasticsearch and use this as their data warehouse. These systems are not optimized for high-volume aggregated OLAP queries and will implode under this workload. This data team clearly didn’t understand how their architectural choices would impact query performance. Take the time to understand the trade-offs inherent in your architecture choices; be clear about how your data model will work with ingestion and storage systems, and how queries will perform.

Kiến trúc Dữ liệu Các quy tắc chung về kiến trúc dữ liệu tốt ở Chương 3 áp dụng cho giai đoạn biến đổi. Hãy xây dựng các hệ thống vững chắc có thể xử lý và biến đổi dữ liệu mà không tự nổ tung. Các lựa chọn của bạn cho thu nạp và lưu trữ sẽ trực tiếp tác động đến khả năng của kiến trúc chung của bạn trong việc thực hiện các truy vấn và biến đổi đáng tin cậy. Nếu thu nạp và lưu trữ phù hợp với các khuôn mẫu truy vấn và biến đổi của bạn, bạn nên ở một vị trí tuyệt vời. Mặt khác, nếu các truy vấn và biến đổi của bạn không hoạt động tốt với các hệ thống thượng nguồn của bạn, bạn sẽ gặp một thế giới đau khổ. Ví dụ, chúng tôi thường thấy các nhóm dữ liệu dùng sai các pipeline dữ liệu và database cho công việc. Một nhóm dữ liệu có thể kết nối một pipeline dữ liệu thời-gian-thực tới một RDBMS hoặc Elasticsearch và dùng cái này như data warehouse của họ. Các hệ thống này không được tối ưu cho các truy vấn OLAP tổng-hợp khối-lượng-cao và sẽ nổ tung dưới khối lượng công việc này. Nhóm dữ liệu này rõ ràng không hiểu các lựa chọn kiến trúc của họ sẽ tác động thế nào đến hiệu năng truy vấn. Hãy dành thời gian để hiểu các đánh đổi vốn có trong các lựa chọn kiến trúc của bạn; hãy rõ ràng về cách mô hình dữ liệu của bạn sẽ hoạt động với các hệ thống thu nạp và lưu trữ, và cách các truy vấn sẽ vận hành.

Orchestration Data teams often manage their transformation pipelines using simple time-based schedules—e.g., cron jobs. This works reasonably well at first but turns into a nightmare as workflows grow more complicated. Use orchestration to manage complex pipelines using a dependency-based approach. Orchestration is also the glue that allows us to assemble pipelines that span multiple systems. **Software Engineering** When writing transformation code, you can use many languages such as SQL, Python, and JVM-based languages, and platforms ranging from data warehouses to distributed computing clusters, and everything in between. Each language and platform has its strengths and quirks, so you should know the best practices of your tools. For example, you might write data transformations in Python, powered by a **distributed system** such as Spark or Dask. When running a data transformation, are you using a UDF when a native function might work much better? We’ve seen cases where poorly written, sluggish UDFs were replaced by a built-in SQL command, with instant and dramatic improvement in performance. The rise of analytics engineering brings software engineering practices to end users, with the notion of analytics as code . Analytics engineering transformation tools like dbt have exploded in popularity, giving analysts and data scientists the ability to write in-database transformations using SQL, without the direct intervention of a DBA or a data engineer. In this case, the data engineer is responsible for setting up the code repository and CI/CD pipeline used by the analysts and data scientists. This is a big change in the role of a data engineer, who would historically build and manage the underlying infrastructure and create the data transformations. As data tools lower the barriers to entry and become more democratized across data teams, it will be interesting to see how the workflows of data teams change. Using a GUI-based low-code tool, you’ll get useful

visualizations of the transformation workflow. You still need to understand what’s going on under the hood. These GUI-based transformation tools will often generate SQL or some other language behind the scenes. While the point of a low-code tool is to alleviate the need to be involved in low-level details, understanding the code behind the scenes will help with debugging and performance optimization. Blindly assuming that the tool is generating performant code is a mistake. We suggest that data engineers pay particular attention to software engineering best practices at the query and transformation stage. While it’s tempting to simply throw more processing resources at a dataset, knowing how to write clean, performant code is a much better approach.

Điều phối Các nhóm dữ liệu thường quản lý các pipeline biến đổi của họ dùng các lịch đơn giản dựa-trên-thời-gian—ví dụ, các cron job. Điều này hoạt động khá tốt lúc đầu nhưng biến thành một cơn ác mộng khi các luồng công việc trở nên phức tạp hơn. Hãy dùng điều phối để quản lý các pipeline phức tạp dùng một cách tiếp cận dựa-trên-phụ-thuộc. Điều phối cũng là chất keo cho phép chúng ta lắp ráp các pipeline trải dài nhiều hệ thống. Kỹ thuật Phần mềm Khi viết mã biến đổi, bạn có thể dùng nhiều ngôn ngữ như SQL, Python, và các ngôn ngữ dựa-trên-JVM, và các nền tảng trải dài từ các data warehouse đến các cluster tính toán phân tán, và mọi thứ ở giữa. Mỗi ngôn ngữ và nền tảng có các điểm mạnh và đặc trưng riêng, vì vậy bạn nên biết các thực hành tốt nhất của các công cụ của mình. Ví dụ, bạn có thể viết các biến đổi dữ liệu trong Python, được vận hành bởi một hệ thống phân tán như Spark hoặc Dask. Khi chạy một biến đổi dữ liệu, bạn có đang dùng một UDF khi một hàm gốc có thể hoạt động tốt hơn nhiều không? Chúng tôi đã thấy các trường hợp các UDF viết-kém, i-ạch được thay thế bởi một lệnh SQL tích-hợp-sẵn, với sự cải thiện hiệu năng tức thì và đáng kể. Sự trỗi dậy của kỹ thuật phân tích (analytics engineering) mang các thực hành kỹ thuật phần mềm đến các người-dùng-cuối, với khái niệm phân-tích-dưới-dạng-mã (analytics as code). Các công cụ biến đổi kỹ thuật phân tích như dbt đã bùng nổ về độ phổ biến, cho các nhà phân tích và nhà khoa học dữ liệu khả năng viết các biến đổi trong-database dùng SQL, mà không cần sự can thiệp trực tiếp của một DBA hoặc một kỹ sư dữ liệu. Trong trường hợp này, kỹ sư dữ liệu chịu trách nhiệm thiết lập kho mã và pipeline CI/CD được dùng bởi các nhà phân tích và nhà khoa học dữ liệu. Đây là một thay đổi lớn trong vai trò của một kỹ sư dữ liệu, người theo lịch sử sẽ xây dựng và quản lý hạ tầng cơ sở và tạo các biến đổi dữ liệu. Khi các công cụ dữ liệu hạ thấp các rào cản gia nhập và trở nên dân chủ hóa hơn trên các nhóm dữ liệu, sẽ thú vị để xem các luồng công việc của các nhóm dữ liệu thay đổi như thế nào. Dùng một công cụ low-code dựa-trên-GUI, bạn sẽ có các trực quan hóa hữu ích của luồng công việc biến đổi. Bạn vẫn cần hiểu những gì đang diễn ra bên dưới. Các công cụ biến đổi dựa-trên-GUI này thường sẽ sinh SQL hoặc một ngôn ngữ nào đó khác phía sau hậu trường. Mặc dù mục đích của một công cụ low-code là giảm bớt nhu cầu tham gia vào các chi tiết cấp-thấp, hiểu mã phía sau hậu trường sẽ giúp với việc gỡ lỗi và tối ưu hiệu năng. Mù quáng giả định rằng công cụ đang sinh mã hiệu năng là một sai lầm. Chúng tôi gợi ý rằng các kỹ sư dữ liệu chú ý đặc biệt đến các thực hành tốt nhất về kỹ thuật phần mềm ở giai đoạn truy vấn và biến đổi. Mặc dù sẽ cám dỗ khi đơn giản ném thêm tài nguyên xử lý vào một tập dữ liệu, biết cách viết mã sạch, hiệu năng là một cách tiếp cận tốt hơn nhiều.

Conclusion — Kết luận

Transformations sit at the heart of data pipelines. Frankly, transformations are the “sexy” part of data engineering. It’s critical to keep in mind the purpose of transformations. Ultimately, engineers are not hired to play with the latest technological toys but to serve their customers. Our opinion is that it is possible to adopt exciting transformation technologies and serve stakeholders. Chapter 11 talks about the live data stack , essentially reconfiguring the data stack around streaming

data ingestion and bringing transformation workflows closer to the source system applications themselves. Engineering teams that think about real-time data as the technology for the sake of technology will repeat the mistakes of the big data era. But in reality, the majority of organizations that we work with have a business use case that would benefit from streaming data. Identifying these use cases and focusing on the value before choosing technologies and complex systems is key. As we head into the serving stage of the data engineering lifecycle in Chapter 9, reflect on technology as a tool for realizing organizational goals. If you're a working data engineer, think about how improvements in transformation systems could help you to serve your end customers better. If you're just embarking on a path toward data engineering, think about the kinds of business problems you're interested in solving with technology.

Các biến đổi nằm ở trung tâm của các pipeline dữ liệu. Thành thật mà nói, các biến đổi là phần “quyến rũ” của kỹ thuật dữ liệu. Điều tối quan trọng là ghi nhớ mục đích của các biến đổi. Rất cuộc, các kỹ sư không được thuê để chơi với các món đồ chơi công nghệ mới nhất mà để phục vụ khách hàng của họ. Quan điểm của chúng tôi là có thể vừa đón nhận các công nghệ biến đổi thú vị vừa phục vụ các bên liên quan. Chương 11 nói về live data stack (ngăn xếp dữ liệu trực tiếp), về cơ bản tái cấu trúc hình ngăn xếp dữ liệu quanh thu nạp dữ liệu streaming và đưa các luồng công việc biến đổi gần hơn với chính các ứng dụng hệ thống nguồn. Các nhóm kỹ thuật nghĩ về dữ liệu thời-gian-thực như công nghệ vi-công-nghệ sẽ lặp lại các sai lầm của kỷ nguyên big data. Nhưng trong thực tế, đa số các tổ chức mà chúng tôi làm việc cùng có một trường hợp sử dụng nghiệp vụ sẽ được hưởng lợi từ dữ liệu streaming. Nhận diện các trường hợp sử dụng này và tập trung vào giá trị trước khi chọn các công nghệ và các hệ thống phức tạp là then chốt. Khi chúng ta tiến vào giai đoạn phục vụ của vòng đời kỹ thuật dữ liệu ở Chương 9, hãy ngừng nghĩ về công nghệ như một công cụ để hiện thực hóa các mục tiêu của tổ chức. Nếu bạn là một kỹ sư dữ liệu đang làm việc, hãy nghĩ về cách các cải tiến trong các hệ thống biến đổi có thể giúp bạn phục vụ tốt hơn các khách hàng cuối của mình. Nếu bạn chỉ vừa bắt đầu một con đường hướng tới kỹ thuật dữ liệu, hãy nghĩ về các loại vấn đề nghiệp vụ mà bạn quan tâm giải quyết bằng công nghệ.

Additional Resources

“How a SQL Database Engine Works,” by Dennis Pham “A Detailed Guide on SQL Query Optimization” tutorial by Megha “Modeling of Real-Time Streaming Data?” Stack Exchange thread “Eventual vs. Strong Consistency in Distributed Databases” by Saurabh.v “Caching in Snowflake Data Warehouse” Snowflake Community page Google Cloud’s “Using Cached Query Results” documentation Holistics’ “Cannot Combine Fields Due to Fan-Out Issues?” FAQ page “A Simple Explanation of Symmetric Aggregates or ‘Why on Earth Does My SQL Look Like That?’” by Lloyd Tabb “Building a Real-Time Data Vault in Snowflake” by Dmytro Yaroshenko and Kent Graziano “Streaming Event Modeling” by Paul Stanton Oracle’s “Slowly Changing Dimensions” tutorial “The New ‘Unified Star Schema’ Paradigm in Analytics Data Modeling Review” by Andriy Zabavskyy “Data Warehouse: The Choice of Inmon vs. Kimball” by Ian Abramson “Inmon or Kimball: Which Approach Is Suitable for Your Data Warehouse?” by Sansu George ScienceDirect’s “Corporate Information Factory” web page Zentut’s “Bill Inmon Data Warehouse” web page “The Evolution of the Corporate Information Factory” by Bill Inmon “Types of Data Warehousing Architecture” by Amritha Fernando “Difference Between Kimball and Inmon” by manmeetjuneja5 “Introduction to Data Warehousing” , “Introduction to Dimensional Modelling for Data Warehousing” , and “Introduction to Data Vault for Data Warehousing” by Simon Kitching Gavroshe USA’s “DW 2.0” web page US patent for “Method and Apparatus for Functional Integration of Metadata” Kimball Group’s “Four-Step Dimensional Design Process” , “Conformed Dimensions” , and “Dimensional Modeling

Techniques” web pages “Kimball vs. Inmon vs. Vault” Reddit thread “How Should Organizations Structure Their Data?” by Michael Berk “Data Vault—An Overview” by John Ryan “Introduction to Data Vault Modeling” document compiled by Kent Graziano and Dan Linstedt “Data Vault 2.0 Modeling Basics” by Kent Graziano Building the Data Warehouse (Wiley), Corporate Information Factory , and The Unified Star Schema (Technics Publications) by W.H. (Bill) Inmon The Data Warehouse Toolkit by Ralph Kimball and Margy Ross (Wiley) Building a Scalable Data Warehouse with Data Vault 2.0 (Morgan Kaufmann) by Daniel Linstedt and Michael Olschimke

1 “Rewriting Slow-Running Queries Which Have Explosive Equi-Join Conditions and Additional Join Conditions Based on Like Operator,” Snowflake Community website, May 1, 2020, <https://oreil.ly/kUsO9> . 2 See, for example, Emin Gün Sirer, “NoSQL Meets Bitcoin and Brings Down Two Exchanges: The Story of Flexcoin and Poloniex,” Hacking, Distributed, April 6, 2014, <https://oreil.ly/RM3QX> . 3 Some Redshift configurations rely on object storage instead. 4 The authors are aware of an incident involving a new analyst at a large grocery store chain running SELECT * on a production database and bringing down a critical inventory database for three days. 5 Figure 8-11 and the example it depicts are significantly based on “Introducing Stream—Stream Joins in Apache 2.3” by Tathagata Das and Joseph Torres, March 13, 2018. 6 For more details on the DRY principle, see The Pragmatic Programmer by David Thomas and Andrew Hunt (Addison-Wesley Professional, 2019). 7 E.F. Codd, “Further Normalization of the Data Base Relational Model,” IBM Research Laboratory (1971), <https://oreil.ly/Muajm> . 8 H.W. Inmon, Building the Data Warehouse (Hoboken: Wiley, 2005). 9 Inmon, Building the Data Warehouse . 10 Inmon, Building the Data Warehouse .

11 Although dimensions and facts are often associated with Kimball, they were first used at General Mills and Dartmouth University in the 1960s, and had early adoption at Nielsen and IRI, among other companies. 12 The data vault has two versions, 1.0 and 2.0. This section focuses on data vault 2.0, but we’ll call it data vault for the sake of brevity. 13 Kent Graziano, “Data Vault 2.0 Modeling Basics,” Vertabelo, October 20, 2015, <https://oreil.ly/iuW1U> . 14 Alex Woodie, “Lakehouses Prevent Data Swamps, Bill Inmon Says,” Datanami, June 1, 2021, <https://oreil.ly/XMwWc> . 15 We remind you to use UDFs responsibly. SQL UDFs often perform reasonably well. We’ve seen JavaScript UDFs increase query time from a few minutes to several hours. 16 Michael Blaha, “Be Careful with Derived Data,” Dataversity, December 5, 2016, <https://oreil.ly/garoL> . 17 Benn Stancil, “The Missing Piece of the Modern Data Stack,” benn.substack , April 22, 2021, <https://oreil.ly/GYf3Z> . 18 “What Is the Difference Between Apache Spark and Hadoop MapReduce?,” Knowledge Powerhouse YouTube video, May 20, 2017, <https://oreil.ly/WN0eX> . 19 For a detailed application of the concept of a streaming DAG, see “Why We Moved from Apache Kafka to Apache Pulsar” by Simba Khadder, StreamNative blog, April 21, 2020, <https://oreil.ly/Rxfko> .

Chapter 9. Serving Data for Analytics, Machine Learning, and Reverse ETL — Chương 9. Phục vụ dữ liệu cho phân tích, học máy và reverse ETL

Congratulations! You’ve reached the final stage of the **data engineering lifecycle**—**serving** data for **downstream** use cases (see Figure 9-1). In this chapter, you’ll learn about various ways to serve data for three major use cases you’ll encounter as a **data engineer**. First, you’ll serve data for **analytics** and BI. You’ll prepare data for use in statistical analysis, reporting, and dashboards. This is the most traditional area of data serving. Arguably, it predates IT and databases, but it is as important as ever for stakeholders to have visibility into the business, organizational, and financial processes.

Xin chúc mừng! Bạn đã chạm tới giai đoạn cuối cùng của vòng đời kỹ thuật dữ liệu — phục vụ (serving) dữ liệu cho các tình huống sử dụng phía hạ nguồn (downstream) (xem Hình 9-1). Trong chương này, bạn sẽ tìm hiểu nhiều cách phục vụ dữ liệu cho ba tình huống sử dụng lớn mà bạn sẽ gặp với vai trò một kỹ sư dữ liệu. Trước hết, bạn sẽ phục vụ dữ liệu cho phân tích (analytics) và BI. Bạn sẽ chuẩn bị dữ liệu để dùng trong phân tích thống kê, báo cáo và bảng điều khiển (dashboard). Đây là lĩnh vực phục vụ dữ liệu truyền thống nhất. Có thể nói nó có trước cả công nghệ thông tin và cơ sở dữ liệu, nhưng nó vẫn quan trọng như mọi khi để các bên liên quan có thể quan sát được các quy trình kinh doanh, tổ chức và tài chính.

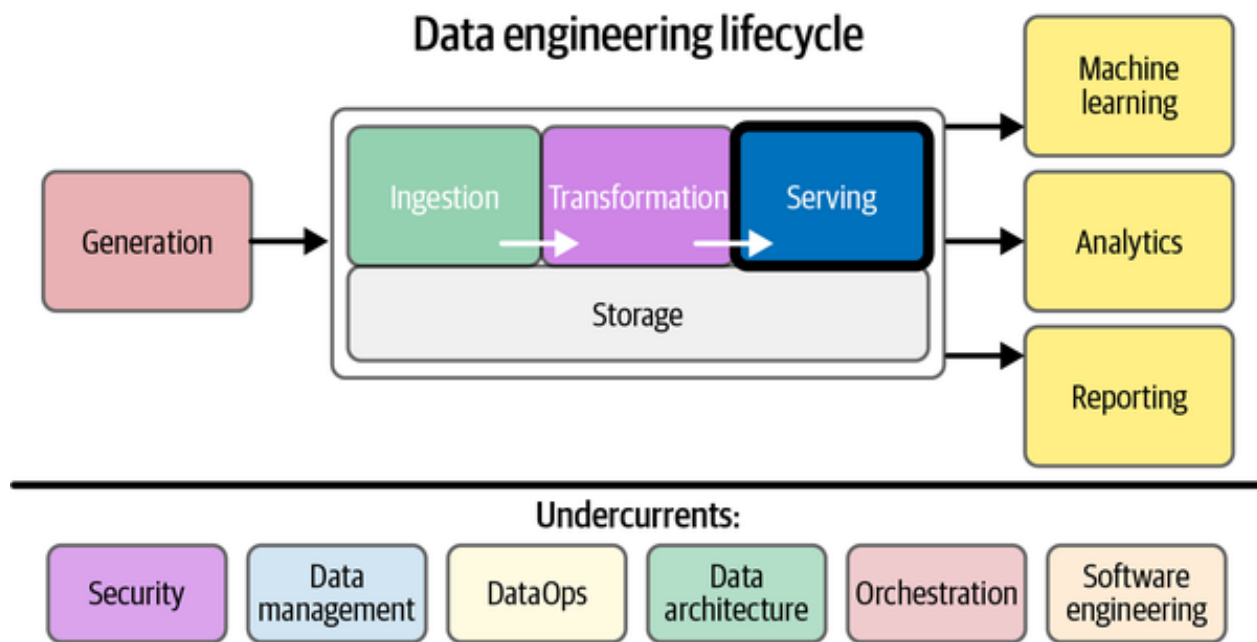


Figure 9-1. Serving delivers data for use cases Second, you'll serve data for ML applications. ML is not possible without high-quality data, appropriately prepared. Data engineers work with data scientists and ML engineers to acquire, transform, and deliver the data necessary for model training. Third, you'll serve data through **reverse ETL**. Reverse **ETL** is the process of sending data back to data sources. For example, we might acquire data from an ad tech platform, run a statistical process on this data to determine cost-per-click bids, and then feed this data back into the ad tech platform. Reverse ETL is highly entangled with BI and ML. Before we get into these three major ways of serving data, let's look at some general considerations.

Hình 9-1. Giai đoạn phục vụ chuyển giao dữ liệu cho các tình huống sử dụng. Thứ hai, bạn sẽ phục vụ dữ liệu cho các ứng dụng học máy (machine learning - ML). ML là bất khả thi nếu thiếu dữ liệu chất lượng cao, được chuẩn bị thích đáng. Các kỹ sư dữ liệu làm việc với nhà khoa học dữ liệu và kỹ sư ML để thu thập, biến đổi (transformation) và chuyển giao dữ liệu cần thiết cho việc huấn luyện mô hình. Thứ ba, bạn sẽ phục vụ dữ liệu thông qua reverse ETL. Reverse ETL là quá trình gửi dữ liệu trở lại các nguồn dữ liệu. Ví dụ, ta có thể thu thập dữ liệu từ một nền tảng công nghệ quảng cáo (ad tech), chạy một quy trình thống kê trên dữ liệu này để xác định mức giá thầu theo lượt nhấp (cost-per-click), rồi đưa dữ liệu này trở lại nền tảng ad tech. Reverse ETL đan xen chặt chẽ với BI và ML. Trước khi đi vào ba cách phục vụ dữ liệu chính này, hãy xem xét một vài điều cần cân nhắc chung.

General Considerations for Serving Data — Những điều cần cân nhắc chung khi phục vụ dữ liệu

Before we get further into serving data, we have a few big considerations. First and foremost is trust. People need to trust the data you're providing. Additionally, you need to understand your use cases and users, the data products that will be produced, how you'll be serving data (self-service or not), data definitions and logic, and **data mesh**. The considerations we'll discuss here are general and apply to any of the three ways of serving data. Understanding these considerations will help you be much more effective in serving your data customers.

Trước khi đi sâu hơn vào việc phục vụ dữ liệu, chúng ta có vài điều cần cân nhắc lớn.

Trước hết và quan trọng nhất là niềm tin (trust). Mọi người cần tin tưởng vào dữ liệu mà bạn cung cấp. Ngoài ra, bạn cần hiểu rõ các tình huống sử dụng và người dùng của mình, các sản phẩm dữ liệu (data product) sẽ được tạo ra, cách bạn sẽ phục vụ dữ liệu (tự phục vụ hay không), các định nghĩa và logic dữ liệu, cùng với data mesh. Những điều cần cân nhắc mà chúng ta thảo luận ở đây mang tính tổng quát và áp dụng cho bất kỳ cách nào trong ba cách phục vụ dữ liệu. Hiểu được những điều này sẽ giúp bạn phục vụ các khách hàng dữ liệu của mình hiệu quả hơn rất nhiều.

Trust — Niềm tin

It takes 20 years to build a reputation and five minutes to ruin it. If you think about that, you'll do things differently. —Warren Buffett Above all else, trust is the root consideration in serving data; end users need to trust the data they're receiving. The fanciest, most sophisticated **data architecture** and serving layer are irrelevant if end users don't believe the data is a reliable representation of their business. A loss of trust is often a silent death knell for a data project, even if the project isn't officially canceled until months or years later. The job of a data engineer is to serve the best data possible, so you'll want to make sure your data products always contain high-quality and trustworthy data. As you learn to serve data throughout this chapter, we'll reinforce the idea of baking trust into your data and discuss pragmatic ways to accomplish this. We see too many cases in which data teams are fixated on pushing out data without asking whether stakeholders trust it in the first place. Often, stakeholders lose trust in the data. Once trust is gone, earning it back is insanely difficult. This inevitably leads to the business not performing to its fullest potential with data, and data teams losing credibility (and possibly being dissolved). To realize **data quality** and build stakeholder trust, utilize data validation and **data observability** processes, in conjunction with visually inspecting and confirming validity with stakeholders. Data validation is analyzing data to ensure that it accurately represents financial information, customer interactions, and sales. Data observability provides an ongoing view of data and data processes. These processes must be applied throughout the **data engineering** lifecycle to realize a good result as we reach the end. We'll discuss these further in "**Undercurrents**". In addition to building trust in data quality, it is incumbent on engineers to build trust in their SLAs and SLOs with their end users and **upstream** stakeholders. Once users come to depend on data to accomplish business processes, they will require that data is consistently available and up-to-date per the commitments made by data engineers. High-quality data is of little value if it's not available as expected when it's time to make a critical business decision. Note, the SLAs and SLOs may also take the form of data contracts (see Chapter 5), formally or informally. We talked about SLAs in Chapter 5 , but discussing them again here is worthwhile. SLAs come in a variety of forms. Regardless of its form, an SLA tells users what to expect from your data product; it is a contract between you and your stakeholders. An example of an SLA might be, "Data will be reliably available and of high quality." An SLO is a key part of an SLA, and describes the ways you'll measure performance against what you've agreed to. For example, given the preceding example SLA, an SLO might be, "Our data pipelines to your dashboard or ML workflow will have 99% uptime, with 95% of data free of defects." Be sure expectations are clear and you have the ability to verify you're operating within your agreed SLA and SLO parameters. It's not enough to simply agree on an SLA. Ongoing communication is a central feature of a good SLA. Have you communicated possible issues that might affect your SLA or SLO expectations? What's your process for remediation and improvement? Trust is everything. It takes a long time to earn, and it's easy to lose.

Phải mất 20 năm để xây dựng danh tiếng và chỉ năm phút để hủy hoại nó. Nếu bạn ngẫm về điều đó, bạn sẽ làm mọi việc khác đi. —Warren Buffett. Trên hết mọi thứ, niềm tin là điều cần cân nhắc cốt lõi khi phục vụ dữ liệu; người dùng cuối cần tin tưởng vào dữ liệu họ nhận được. Kiến trúc dữ liệu và tăng phục vụ hào nhoáng, tinh vi đến mấy cũng vô

nghĩa nếu người dùng cuối không tin rằng dữ liệu là một phản ánh đáng tin cậy về hoạt động kinh doanh của họ. Việc mất niềm tin thường là hồi chuông báo tử thầm lặng cho một dự án dữ liệu, ngay cả khi dự án chỉ chính thức bị hủy bỏ vài tháng hay vài năm sau đó. Công việc của một kỹ sư dữ liệu là phục vụ dữ liệu tốt nhất có thể, nên bạn sẽ muốn chắc chắn rằng các sản phẩm dữ liệu của mình luôn chứa dữ liệu chất lượng cao và đáng tin cậy. Khi bạn học cách phục vụ dữ liệu xuyên suốt chương này, chúng tôi sẽ củng cố ý tưởng về việc bồi đắp niềm tin vào dữ liệu của bạn và thảo luận những cách thực dụng để đạt được điều đó. Chúng tôi thấy quá nhiều trường hợp các đội dữ liệu chỉ chăm chăm đẩy dữ liệu ra mà không hỏi liệu các bên liên quan có tin tưởng nó hay không ngay từ đầu. Thông thường, các bên liên quan mất niềm tin vào dữ liệu. Một khi niềm tin đã mất, lấy lại nó là việc khó đến mức điên rồ. Điều này tất yếu dẫn đến việc doanh nghiệp không phát huy hết tiềm năng với dữ liệu, và các đội dữ liệu mất uy tín (và có thể bị giải thể). Để hiện thực hóa chất lượng dữ liệu (data quality) và xây dựng niềm tin của các bên liên quan, hãy tận dụng các quy trình kiểm định dữ liệu (data validation) và khả năng quan sát dữ liệu (data observability), kết hợp với việc kiểm tra trực quan và xác nhận tính hợp lệ cùng các bên liên quan. Kiểm định dữ liệu là việc phân tích dữ liệu để đảm bảo nó phản ánh chính xác thông tin tài chính, các tương tác với khách hàng và doanh số. Khả năng quan sát dữ liệu cung cấp một góc nhìn liên tục về dữ liệu và các quy trình dữ liệu. Những quy trình này phải được áp dụng xuyên suốt vòng đời kỹ thuật dữ liệu để có được kết quả tốt khi ta đến giai đoạn cuối. Chúng ta sẽ thảo luận thêm trong phần “Các dòng chảy ngầm”. Ngoài việc xây dựng niềm tin vào chất lượng dữ liệu, các kỹ sư còn có trách nhiệm xây dựng niềm tin vào các SLA và SLO của mình với người dùng cuối và các bên liên quan ở thượng nguồn (upstream). Một khi người dùng đã đến mức phụ thuộc vào dữ liệu để hoàn thành các quy trình kinh doanh, họ sẽ yêu cầu dữ liệu luôn sẵn có một cách nhất quán và được cập nhật theo đúng các cam kết mà kỹ sư dữ liệu đưa ra. Dữ liệu chất lượng cao chẳng có mấy giá trị nếu nó không sẵn có như mong đợi vào thời điểm cần đưa ra một quyết định kinh doanh trọng yếu. Lưu ý rằng các SLA và SLO cũng có thể mang hình thức của các hợp đồng dữ liệu (data contract) (xem Chương 5), một cách chính thức hoặc không chính thức. Chúng ta đã nói về SLA trong Chương 5, nhưng bàn lại ở đây vẫn rất đáng. SLA có nhiều dạng khác nhau. Bất kể ở dạng nào, một SLA cho người dùng biết họ nên mong đợi gì từ sản phẩm dữ liệu của bạn; nó là một hợp đồng giữa bạn và các bên liên quan. Một ví dụ về SLA có thể là, “Dữ liệu sẽ luôn sẵn có một cách đáng tin cậy và có chất lượng cao.” Một SLO là một phần then chốt của SLA, và mô tả các cách bạn sẽ đo lường hiệu năng so với những gì bạn đã đồng thuận. Ví dụ, với SLA mẫu nêu trên, một SLO có thể là, “Các đường ống dữ liệu (data pipeline) của chúng tôi đến dashboard hoặc quy trình ML của bạn sẽ có thời gian hoạt động 99%, với 95% dữ liệu không có lỗi.” Hãy chắc chắn rằng các kỳ vọng là rõ ràng và bạn có khả năng kiểm chứng rằng mình đang vận hành trong các tham số SLA và SLO đã thống nhất. Chỉ đơn thuần đồng ý về một SLA là chưa đủ. Giao tiếp liên tục là một đặc tính trung tâm của một SLA tốt. Bạn đã truyền đạt những vấn đề có thể ảnh hưởng đến các kỳ vọng SLA hay SLO của mình chưa? Quy trình khắc phục và cải thiện của bạn là gì? Niềm tin là tất cả. Phải mất rất lâu để giành được, và lại rất dễ để đánh mất.

What’s the Use Case, and Who’s the User? — Tình huống sử dụng là gì, và ai là người dùng?

The serving stage is about data in action. But what is a productive use of data? You need to consider two things to answer this question: what’s the use case, and who’s the user? The use case for data goes well beyond viewing reports and dashboards. Data is at its best when it leads to action. Will an executive make a strategic decision from a report? Will a user of a mobile food delivery app

receive a coupon that entices them to purchase in the next two minutes? The data is often used in more than one use case—e.g., to train an ML model that does lead scoring and populates a CRM (reverse ETL). High-quality, high-impact data will inherently attract many interesting use cases. But in seeking use cases, always ask, “What action will this data trigger, and who will be performing this action?” with the appropriate follow-up question, “Can this action be automated?” Whenever possible, prioritize use cases with the highest possible ROI. Data engineers love to obsess over the technical implementation details of the systems they build while ignoring the basic question of purpose. Engineers want to do what they do best: engineer things. When engineers recognize the need to focus on value and use cases, they become much more valuable and effective in their roles.

Giai đoạn phục vụ là về dữ liệu trong hành động. Nhưng đâu là cách sử dụng dữ liệu mang lại hiệu quả? Bạn cần cân nhắc hai điều để trả lời câu hỏi này: tình huống sử dụng là gì, và ai là người dùng? Tình huống sử dụng của dữ liệu vượt xa việc chỉ xem báo cáo và dashboard. Dữ liệu phát huy giá trị tốt nhất khi nó dẫn đến hành động. Liệu một lãnh đạo có đưa ra quyết định chiến lược từ một báo cáo không? Liệu người dùng một ứng dụng giao đồ ăn trên di động có nhận được một phiếu giảm giá đủ hấp dẫn để khiến họ mua hàng trong vòng hai phút tới không? Dữ liệu thường được dùng cho nhiều hơn một tình huống sử dụng — ví dụ, để huấn luyện một mô hình ML thực hiện chấm điểm khách hàng tiềm năng (lead scoring) và điền dữ liệu vào một CRM (reverse ETL). Dữ liệu chất lượng cao, tác động lớn vốn dĩ sẽ thu hút nhiều tình huống sử dụng thú vị. Nhưng khi đi tìm các tình huống sử dụng, hãy luôn hỏi, “Dữ liệu này sẽ kích hoạt hành động nào, và ai sẽ thực hiện hành động đó?”, cùng câu hỏi tiếp theo thích đáng, “Hành động này có thể tự động hóa được không?” Bất cứ khi nào có thể, hãy ưu tiên các tình huống sử dụng có ROI cao nhất. Các kỹ sư dữ liệu thích sa đà vào các chi tiết kỹ thuật trong việc triển khai các hệ thống họ xây dựng trong khi bỏ qua câu hỏi căn bản về mục đích. Các kỹ sư muốn làm điều họ giỏi nhất: làm kỹ thuật. Khi các kỹ sư nhận ra nhu cầu tập trung vào giá trị và các tình huống sử dụng, họ trở nên có giá trị và hiệu quả hơn nhiều trong vai trò của mình.

When starting a new data project, working backward is helpful. While it’s tempting to focus on tools, we encourage you to start with the use case and the users. Here are some questions to ask yourself as you get started:

Khi bắt đầu một dự án dữ liệu mới, làm việc theo lối ngược (working backward) là hữu ích. Dù việc tập trung vào công cụ rất hấp dẫn, chúng tôi khuyến khích bạn bắt đầu từ tình huống sử dụng và người dùng. Sau đây là một số câu hỏi bạn nên tự đặt ra khi khởi đầu:

Who will use the data, and how will they use it?

Ai sẽ dùng dữ liệu, và họ sẽ dùng nó như thế nào?

What do stakeholders expect?

Các bên liên quan mong đợi điều gì?

How can I collaborate with data stakeholders (e.g., data scientists, analysts, business users) to understand how the data I’m working with will be used?

Tôi có thể cộng tác với các bên liên quan dữ liệu (ví dụ: nhà khoa học dữ liệu, nhà phân tích, người dùng nghiệp vụ) như thế nào để hiểu dữ liệu tôi đang xử lý sẽ được dùng ra sao?

Again, always approach data engineering from the perspective of the user and their use case. By understanding their expectations and goals, you can work backward to create amazing data products more easily. Let’s take a moment to expand on our discussion of a data product.

Một lần nữa, hãy luôn tiếp cận kỹ thuật dữ liệu từ góc nhìn của người dùng và tình huống

sử dụng của họ. Bằng cách hiểu các kỳ vọng và mục tiêu của họ, bạn có thể làm việc theo lối ngược để tạo ra những sản phẩm dữ liệu tuyệt vời một cách dễ dàng hơn. Hãy dành một chút thời gian để mở rộng phần thảo luận của chúng ta về sản phẩm dữ liệu.

Data Products — Sản phẩm dữ liệu

A good definition of a data product is a product that facilitates an end goal through the use of data. — DJ Patil Data products aren't created in a vacuum. Like so many other organizational processes that we've discussed, making data products is a full-contact sport, involving a mix of product and business alongside technology. It's important to involve key stakeholders in developing a data product. In most companies, a data engineer is a couple of steps removed from the end users of a data product; a good data engineer will seek to fully understand outcomes for direct users such as data analysts and data scientists or customers external to the company. When creating a data product, it's useful to think of the “jobs to be done.”¹ A user “hires” a product for a “job to be done.” This means you need to know what the user wants—i.e., their motivation for “hiring” your product. A classic engineering mistake is simply building without understanding the requirements, needs of the end user, or product/market fit. This disaster happens when you build data products nobody wants to use. A good data product has positive feedback loops. More usage of a data product generates more useful data, which is used to improve the data product. Rinse and repeat. When building a data product, keep these considerations in mind:

Một định nghĩa hay về sản phẩm dữ liệu là một sản phẩm tạo điều kiện đạt được một mục tiêu cuối cùng thông qua việc sử dụng dữ liệu. — DJ Patil. Sản phẩm dữ liệu không được tạo ra trong chân không. Như rất nhiều quy trình tổ chức khác mà chúng ta đã thảo luận, làm ra sản phẩm dữ liệu là một môn thể thao đối kháng toàn diện, có sự pha trộn giữa sản phẩm và kinh doanh bên cạnh công nghệ. Việc lôi cuốn các bên liên quan chủ chốt vào quá trình phát triển một sản phẩm dữ liệu là quan trọng. Ở hầu hết các công ty, một kỹ sư dữ liệu cách người dùng cuối của một sản phẩm dữ liệu vài bước; một kỹ sư dữ liệu giỏi sẽ tìm cách hiểu thấu đáo các kết quả đầu ra cho những người dùng trực tiếp như nhà phân tích dữ liệu và nhà khoa học dữ liệu, hoặc các khách hàng bên ngoài công ty. Khi tạo một sản phẩm dữ liệu, sẽ hữu ích nếu nghĩ về “các công việc cần hoàn thành” (jobs to be done). Người dùng “thuê” một sản phẩm cho một “công việc cần hoàn thành”. Điều này có nghĩa bạn cần biết người dùng muốn gì — tức là động cơ của họ khi “thuê” sản phẩm của bạn. Một sai lầm kỹ thuật kinh điển là cứ thế xây dựng mà không hiểu các yêu cầu, nhu cầu của người dùng cuối, hay sự phù hợp sản phẩm/thị trường (product/market fit). Thảm họa này xảy ra khi bạn xây dựng những sản phẩm dữ liệu không ai muốn dùng. Một sản phẩm dữ liệu tốt có các vòng phản hồi tích cực. Càng sử dụng nhiều một sản phẩm dữ liệu thì càng sinh ra nhiều dữ liệu hữu ích, vốn được dùng để cải thiện sản phẩm dữ liệu. Cứ thế lặp đi lặp lại. Khi xây dựng một sản phẩm dữ liệu, hãy ghi nhớ những điều cần cân nhắc sau:

When someone uses the data product, what do they hope to accomplish? All too often, data products are made without a clear understanding of the outcome expected by the user.

Khi ai đó dùng sản phẩm dữ liệu, họ hy vọng hoàn thành được điều gì? Quá thường xuyên, các sản phẩm dữ liệu được làm ra mà không có sự hiểu biết rõ ràng về kết quả mà người dùng mong đợi.

Will the data product serve internal or external users? In Chapter 2, we discussed internal- and external-facing data engineering. When creating a data product, knowing whether your customer is internal or external facing will impact the way data is served.

Sản phẩm dữ liệu sẽ phục vụ người dùng nội bộ hay bên ngoài? Trong Chương 2, chúng

ta đã thảo luận về kỹ thuật dữ liệu hướng nội bộ và hướng ra bên ngoài. Khi tạo một sản phẩm dữ liệu, việc biết khách hàng của bạn là nội bộ hay bên ngoài sẽ ảnh hưởng đến cách dữ liệu được phục vụ.

What are the outcomes and ROI of the data product you're building?

Các kết quả đầu ra và ROI của sản phẩm dữ liệu bạn đang xây dựng là gì?

Building data products that people will use and love is critical. Nothing will ruin the adoption of a data product more than unwanted utility and loss of trust in the data outputs. Pay attention to the adoption and usage of data products, and be willing to adjust to make users happy.

Xây dựng những sản phẩm dữ liệu mà mọi người sẽ dùng và yêu thích là điều then chốt. Không gì hủy hoại việc đón nhận một sản phẩm dữ liệu hơn là tính hữu dụng không như mong muốn và việc mất niềm tin vào các đầu ra dữ liệu. Hãy chú ý đến mức độ đón nhận và sử dụng các sản phẩm dữ liệu, và sẵn lòng điều chỉnh để làm người dùng hài lòng.

Self-Service or Not? — Tự phục vụ hay không?

How will users interface with your data product? Will a business director request a report from the data team, or can this director simply build the report? Self-service data products—giving the user the ability to build data products on their own—have been a common aspiration of data users for many years. What's better than just giving the end user the ability to directly create reports, analyses, and ML models? Today, self-service BI and data science is still mostly aspirational. While we occasionally see companies successfully doing self-service with data, this is rare. Most of the time, attempts at self-service data begin with great intentions but ultimately fail; self-service data is tough to implement in practice. Thus, the analyst or data scientist is left to perform the heavy lifting of providing ad hoc reports and maintaining dashboards. Why is self-service data so hard? The answer is nuanced, but it generally involves understanding the end user. If the user is an executive who needs to understand how the business is doing, that person probably just wants a predefined dashboard of clear and actionable metrics. The executive will likely ignore any self-serve tools for creating custom data views. If reports provoke further questions, they might have analysts at their disposal to pursue a deeper investigation. On the other hand, a user who is an analyst might already be pursuing self-service analytics via more powerful tools such as SQL. Self-service analytics through a BI layer is not useful. The same considerations apply to data science. Although granting self-service ML to “citizen data scientists” has been a goal of many automated ML vendors, adoption is still nascent for the same reasons as self-service analytics. In these two extreme cases, a self-service data product is a wrong tool for the job. Successful self-service data projects boil down to having the right audience. Identify the self-service users and the “job” they want to do. What are they trying to accomplish by using a self-service data product versus partnering with a data analyst to get the job done? A group of executives with a background in data forms an ideal audience for self-service; they likely want to slice and dice data themselves without needing to dust off their languishing SQL skills. Business leaders willing to invest the time to learn data skills through a company initiative and training program could also realize significant value from self-service. Determine how you will provide data to this group. What are their time requirements for new data? What happens if they inevitably want more data or change the scope of what's required from self-service? More data often means more questions, which requires more data. You'll need to anticipate the growing needs of your self-service users. You also need to understand the fine balance between flexibility and guardrails that will help your audience find value and insights without incorrect results and confusion.

Người dùng sẽ tương tác với sản phẩm dữ liệu của bạn như thế nào? Liệu một giám đốc nghiệp vụ sẽ yêu cầu một báo cáo từ đội dữ liệu, hay vị giám đốc này có thể tự xây dựng

báo cáo đó? Các sản phẩm dữ liệu tự phục vụ (self-service) — trao cho người dùng khả năng tự xây dựng các sản phẩm dữ liệu — đã là một khát vọng phổ biến của những người dùng dữ liệu suốt nhiều năm. Còn gì tốt hơn việc chỉ cần trao cho người dùng cuối khả năng trực tiếp tạo báo cáo, phân tích và mô hình ML? Ngày nay, BI và khoa học dữ liệu tự phục vụ phần lớn vẫn còn là khát vọng. Dù thỉnh thoảng chúng ta thấy các công ty thành công trong việc tự phục vụ với dữ liệu, điều này hiếm gặp. Hầu hết các trường hợp, những nỗ lực tự phục vụ dữ liệu khởi đầu với những ý định tốt đẹp nhưng cuối cùng thất bại; dữ liệu tự phục vụ khó triển khai trong thực tế. Do vậy, nhà phân tích hay nhà khoa học dữ liệu vẫn phải gánh phần việc nặng nhọc là cung cấp các báo cáo đột xuất (ad hoc) và bảo trì các dashboard. Tại sao tự phục vụ dữ liệu lại khó đến vậy? Câu trả lời có nhiều sắc thái, nhưng nhìn chung liên quan đến việc hiểu người dùng cuối. Nếu người dùng là một lãnh đạo cần hiểu hoạt động kinh doanh đang ra sao, người đó có lẽ chỉ muốn một dashboard định sẵn với các chỉ số rõ ràng và có thể hành động được. Vị lãnh đạo này nhiều khả năng sẽ phớt lờ mọi công cụ tự phục vụ để tạo các khung nhìn dữ liệu tùy biến. Nếu báo cáo gợi lên thêm câu hỏi, họ có thể có sẵn các nhà phân tích để theo đuổi một cuộc điều tra sâu hơn. Mặt khác, một người dùng là nhà phân tích có thể đã đang theo đuổi phân tích tự phục vụ qua các công cụ mạnh hơn như SQL. Phân tích tự phục vụ thông qua một tầng BI là không hữu ích. Những điều cần nhắc tương tự cũng áp dụng cho khoa học dữ liệu. Mặc dù việc trao ML tự phục vụ cho các “nhà khoa học dữ liệu công dân” (citizen data scientist) đã là mục tiêu của nhiều nhà cung cấp ML tự động, mức độ đón nhận vẫn còn non trẻ vì những lý do giống như phân tích tự phục vụ. Trong hai trường hợp cực đoan này, một sản phẩm dữ liệu tự phục vụ là công cụ sai cho công việc. Các dự án dữ liệu tự phục vụ thành công cuối cùng quy về việc có đúng đối tượng khán giả. Hãy xác định những người dùng tự phục vụ và “công việc” họ muốn làm. Họ đang cố hoàn thành điều gì bằng cách dùng một sản phẩm dữ liệu tự phục vụ so với việc hợp tác với một nhà phân tích dữ liệu để hoàn thành công việc? Một nhóm lãnh đạo có nền tảng về dữ liệu là đối tượng lý tưởng cho việc tự phục vụ; họ nhiều khả năng muốn tự mình mổ xẻ, phân tích dữ liệu mà không cần phải lòi ra dùng lại kỹ năng SQL đã mai một. Các lãnh đạo nghiệp vụ sẵn lòng đầu tư thời gian để học các kỹ năng dữ liệu thông qua một sáng kiến và chương trình đào tạo của công ty cũng có thể thu được giá trị đáng kể từ việc tự phục vụ. Hãy xác định cách bạn sẽ cung cấp dữ liệu cho nhóm này. Các yêu cầu về thời gian của họ đối với dữ liệu mới là gì? Điều gì sẽ xảy ra nếu họ chắc chắn sẽ muốn nhiều dữ liệu hơn hoặc thay đổi phạm vi của những gì cần có từ việc tự phục vụ? Nhiều dữ liệu hơn thường có nghĩa là nhiều câu hỏi hơn, mà điều đó lại đòi hỏi nhiều dữ liệu hơn. Bạn sẽ cần lường trước các nhu cầu ngày càng tăng của những người dùng tự phục vụ. Bạn cũng cần hiểu sự cân bằng tinh tế giữa tính linh hoạt và các rào chắn (guardrail) sẽ giúp đối tượng khán giả của bạn tìm thấy giá trị và thông tin chuyên sâu (insight) mà không gặp phải kết quả sai và sự bối rối.

Data Definitions and Logic — Định nghĩa và logic dữ liệu

As we’ve emphatically discussed, the utility of data in an organization is ultimately derived from its correctness and trustworthiness. Critically, the correctness of data goes beyond faithful reproduction of **event** values from source systems. Data correctness also encompasses proper data definitions and logic; these must be baked into data through all lifecycle stages, from source systems to data pipelines to BI tools and much more. Data definition refers to the meaning of data as it is understood throughout the organization. For example, customer has a precise meaning within a company and across departments. When the definition of a customer varies, these must be documented and made available to everyone who uses the data. Data logic stipulates formulas for deriving metrics from data—say, gross sales or customer lifetime value. Proper data logic must

encode data definitions and details of statistical calculations. To compute customer churn metrics, we would need a definition: who is a customer? To calculate net profits, we would need a set of logical rules to determine which expenses to deduct from gross revenue. Frequently, we see data definitions and logic taken for granted, often passed around the organization in the form of tribal knowledge. Tribal knowledge takes on a life of its own, often at the expense of anecdotes replacing data-driven insights, decisions, and actions. Instead, formally declaring data definitions and logic both in a **data catalog** and within the systems of the data engineering lifecycle goes a long way to ensuring data correctness, consistency, and trustworthiness. Data definitions can be served in many ways, sometimes explicitly, but mostly implicitly. By implicit, we mean that anytime you serve data for a query, a dashboard, or an ML model, the data and derived metrics are presented consistently and correctly. When you write a SQL query, you're implicitly assuming that the inputs to this query are correct, including upstream pipeline logic and definitions. This is where **data modeling** (described in Chapter 8) is incredibly useful to capture data definitions and logic in a way that's understandable and usable by multiple end users. Using a **semantic** layer, you consolidate business definitions and logic in a reusable fashion. Write once, use anywhere. This paradigm is an object-oriented approach to metrics, calculations, and logic. We'll have more to say in "Semantic and Metrics Layers".

Như chúng ta đã nhấn mạnh khi thảo luận, tính hữu dụng của dữ liệu trong một tổ chức rất cuộc bắt nguồn từ tính đúng đắn và đáng tin cậy của nó. Một cách trọng yếu, tính đúng đắn của dữ liệu vượt ra ngoài việc tái tạo trung thực các giá trị sự kiện từ các hệ thống nguồn (source system). Tính đúng đắn của dữ liệu còn bao hàm các định nghĩa và logic dữ liệu đúng đắn; những điều này phải được bồi đắp vào dữ liệu qua tất cả các giai đoạn của vòng đời, từ hệ thống nguồn đến đường ống dữ liệu rồi đến các công cụ BI và nhiều hơn nữa. Định nghĩa dữ liệu (data definition) chỉ ý nghĩa của dữ liệu như cách nó được hiểu xuyên suốt tổ chức. Ví dụ, "khách hàng" có một ý nghĩa chính xác trong một công ty và xuyên suốt các phòng ban. Khi định nghĩa về một khách hàng có sự khác biệt, những khác biệt này phải được tài liệu hóa và đưa đến cho mọi người dùng dữ liệu. Logic dữ liệu (data logic) quy định các công thức để suy ra các chỉ số từ dữ liệu — chẳng hạn doanh số gộp (gross sales) hay giá trị vòng đời khách hàng (customer lifetime value). Logic dữ liệu đúng đắn phải mã hóa các định nghĩa dữ liệu và các chi tiết của các phép tính thống kê. Để tính các chỉ số về tỷ lệ rời bỏ của khách hàng (churn), chúng ta sẽ cần một định nghĩa: ai là khách hàng? Để tính lợi nhuận ròng, chúng ta sẽ cần một bộ quy tắc logic để xác định những khoản chi phí nào cần trừ khỏi doanh thu gộp. Thường thì chúng ta thấy các định nghĩa và logic dữ liệu bị xem là hiển nhiên, thường được truyền tay nhau trong tổ chức dưới dạng kiến thức truyền miệng (tribal knowledge). Kiến thức truyền miệng tự sinh sôi theo cách riêng của nó, thường phải trả giá bằng việc các giai thoại thay thế cho các thông tin chuyên sâu, quyết định và hành động dựa trên dữ liệu. Thay vào đó, việc khai báo chính thức các định nghĩa và logic dữ liệu cả trong một danh mục dữ liệu (data catalog) lẫn bên trong các hệ thống của vòng đời kỹ thuật dữ liệu đóng góp rất nhiều vào việc đảm bảo tính đúng đắn, nhất quán và đáng tin cậy của dữ liệu. Định nghĩa dữ liệu có thể được phục vụ theo nhiều cách, đôi khi tường minh, nhưng phần lớn là ngầm định. Ngầm định, ý chúng tôi là bất cứ khi nào bạn phục vụ dữ liệu cho một truy vấn, một dashboard, hay một mô hình ML, dữ liệu và các chỉ số được suy ra đều được trình bày một cách nhất quán và đúng đắn. Khi bạn viết một truy vấn SQL, bạn ngầm giả định rằng các đầu vào của truy vấn này là đúng, bao gồm cả logic và định nghĩa của đường ống ở thượng nguồn. Đây là nơi mô hình hóa dữ liệu (mô tả trong Chương 8) cực kỳ hữu ích trong việc nắm bắt các định nghĩa và logic dữ liệu theo cách mà nhiều người dùng cuối có thể hiểu và sử dụng. Dùng một tầng ngữ nghĩa (semantic layer), bạn hợp nhất các định nghĩa và logic nghiệp vụ theo một cách có thể tái sử dụng. Viết một lần, dùng ở mọi nơi. Mô hình này là một cách tiếp cận hướng đối tượng đối với các chỉ số, phép tính và logic. Chúng ta sẽ nói thêm trong

phần “Tăng ngữ nghĩa và tăng chỉ số”.

Data Mesh — Data mesh

Data mesh will increasingly be a consideration when serving data. Data mesh fundamentally changes the way data is served within an organization. Instead of siloed data teams serving their internal constituents, every domain team takes on two aspects of decentralized, peer-to-peer data serving. First, teams are responsible for serving data to other teams by preparing it for consumption. Data must be good for use in data apps, dashboards, analytics, and BI tools across the organization. Second, each team potentially runs its dashboards and analytics for self-service. Teams consume data from across the organization based on the particular needs in their domain. Data consumed from other teams may also make its way into the software designed by a domain team through user-facing analytics or an ML feature. This dramatically changes the details and structure of serving. We introduced the concept of a data mesh in Chapter 3. Now that we’ve covered some general considerations for serving data, let’s look at the first major area: analytics.

Data mesh sẽ ngày càng là một điều cần cân nhắc khi phục vụ dữ liệu. Data mesh thay đổi về căn bản cách dữ liệu được phục vụ trong một tổ chức. Thay vì các đội dữ liệu bị cô lập phục vụ những đối tượng nội bộ của họ, mỗi đội theo lĩnh vực (domain) đảm nhận hai khía cạnh của việc phục vụ dữ liệu phi tập trung, ngang hàng (peer-to-peer). Thứ nhất, các đội chịu trách nhiệm phục vụ dữ liệu cho các đội khác bằng cách chuẩn bị dữ liệu để tiêu thụ. Dữ liệu phải đủ tốt để dùng trong các ứng dụng dữ liệu, dashboard, phân tích và các công cụ BI trên khắp tổ chức. Thứ hai, mỗi đội có khả năng tự chạy các dashboard và phân tích của riêng mình để tự phục vụ. Các đội tiêu thụ dữ liệu từ khắp tổ chức dựa trên các nhu cầu cụ thể trong lĩnh vực của họ. Dữ liệu được tiêu thụ từ các đội khác cũng có thể đi vào phần mềm do một đội lĩnh vực thiết kế thông qua phân tích hướng người dùng hoặc một tính năng ML. Điều này thay đổi đáng kể các chi tiết và cấu trúc của việc phục vụ. Chúng tôi đã giới thiệu khái niệm data mesh trong Chương 3. Giờ đây khi đã đề cập một số điều cần cân nhắc chung về việc phục vụ dữ liệu, hãy xem xét lĩnh vực lớn đầu tiên: phân tích.

Analytics — Phân tích

The first data serving use case you’ll likely encounter is analytics, which is discovering, exploring, identifying, and making visible key insights and patterns within data. Analytics has many aspects. As a practice, analytics is carried out using statistical methods, reporting, BI tools, and more. As a data engineer, knowing the various types and techniques of analytics is key to accomplishing your work. This section aims to show how you’ll serve data for analytics and presents some points to think about to help your analysts succeed. Before you even serve data for analytics, the first thing you need to do (which should sound familiar after reading the preceding section) is identify the end use case. Is the user looking at historical trends? Should users be immediately and automatically notified of an anomaly, such as a fraud alert? Is someone consuming a real-time dashboard on a mobile application? These examples highlight the differences between business analytics (usually BI), operational analytics, and user-facing analytics. Each of these analytics categories has different goals and unique serving requirements. Let’s look at how you’ll serve data for these types of analytics.

Tình huống sử dụng phục vụ dữ liệu đầu tiên mà bạn nhiều khả năng sẽ gặp là phân tích (analytics), đó là việc khám phá, thăm dò, nhận diện và làm hiển hiện các thông tin chuyên sâu và các mẫu hình then chốt bên trong dữ liệu. Phân tích có nhiều khía cạnh. Như một

bộ môn thực hành, phân tích được thực hiện bằng các phương pháp thống kê, báo cáo, các công cụ BI và nhiều thứ khác. Với vai trò một kỹ sư dữ liệu, biết các loại và kỹ thuật phân tích khác nhau là chìa khóa để hoàn thành công việc của bạn. Phần này nhằm chỉ ra cách bạn sẽ phục vụ dữ liệu cho phân tích và trình bày một số điểm cần suy ngẫm để giúp các nhà phân tích của bạn thành công. Trước khi bạn thậm chí phục vụ dữ liệu cho phân tích, điều đầu tiên bạn cần làm (điều này hẳn nghe quen thuộc sau khi đọc phần trước) là xác định tình huống sử dụng cuối cùng. Người dùng đang xem các xu hướng lịch sử ư? Liệu người dùng có nên được thông báo ngay lập tức và tự động về một bất thường, chẳng hạn một cảnh báo gian lận không? Có ai đó đang tiêu thụ một dashboard thời gian thực trên một ứng dụng di động không? Những ví dụ này làm nổi bật những khác biệt giữa phân tích nghiệp vụ (business analytics, thường là BI), phân tích vận hành (operational analytics) và phân tích hướng người dùng (user-facing analytics). Mỗi loại phân tích này có những mục tiêu khác nhau và các yêu cầu phục vụ riêng. Hãy xem cách bạn sẽ phục vụ dữ liệu cho những loại phân tích này.

Business Analytics — Phân tích nghiệp vụ

Business analytics uses historical and current data to make strategic and actionable decisions. The types of decisions tend to factor in longer-term trends, and often involve a mix of statistical and trend analysis, alongside domain expertise and human judgment. Business analysis is as much an art as it is a science. Business analytics typically falls into a few big areas—dashboards, reports, and ad hoc analysis. A business analyst might focus on one or all of these categories. Let's quickly look at the differences between these practices and related tools. Understanding an analyst's workflow will help you, the data engineer, understand how to serve data. A dashboard concisely shows decision makers how an organization is performing against a handful of core metrics, such as sales and customer retention. These core metrics are presented as visualizations (e.g., charts or heatmaps), summary statistics, or even a single number. This is similar to a car dashboard, which gives you a single readout of the critical things you need to know while driving a vehicle. An organization may have more than one dashboard, with C-level executives using an overarching dashboard, and their direct reports using dashboards with their particular metrics, KPIs, or objectives and key results (OKRs). Analysts help create and maintain these dashboards. Once business stakeholders embrace and rely on a dashboard, the analyst usually responds to requests to look into a potential issue with a metric or add a new metric to the dashboard. Currently, you might use BI platforms to create dashboards, such as Tableau, Looker, Sisense, Power BI, or Apache Superset/Preset. Analysts are often tasked by business stakeholders with creating a report. The goal of a report is to use data to drive insights and action. An analyst working at an online retail company is asked to investigate which factors are driving a higher-than-expected rate of returns for women's running shorts. The analyst runs some SQL queries in the **data warehouse**, aggregates the return codes that customers provide as the reason for their return, and discovers that the fabric in the running shorts is of inferior quality, often wearing out within a few uses. Stakeholders such as manufacturing and quality control are notified of these findings. Furthermore, the findings are summarized in a report and distributed in the same BI tool where the dashboard resides. The analyst was asked to dig into a potential issue and come back with insights. This represents an example of ad hoc analysis. Reports typically start as ad hoc requests. If the results of the ad hoc analysis are impactful, they often end up in a report or dashboard. The technologies used for reports and ad hoc analysis are similar to dashboards but may include Excel, Python, R-based notebooks, SQL queries, and much more. Good analysts constantly engage with the business and dive into the data to answer questions and uncover hidden and counterintuitive trends and insights. They also work with data engineers to provide feedback on data quality, reliability issues, and requests for new datasets. The data engineer is responsible for addressing this feedback and

providing new datasets for the analyst to use.

Phân tích nghiệp vụ (business analytics) dùng dữ liệu lịch sử và hiện tại để đưa ra các quyết định chiến lược và có thể hành động được. Các loại quyết định thường tính đến các xu hướng dài hạn hơn, và thường liên quan đến sự pha trộn giữa phân tích thống kê và phân tích xu hướng, bên cạnh chuyên môn về lĩnh vực và sự phán đoán của con người. Phân tích nghiệp vụ vừa là nghệ thuật vừa là khoa học. Phân tích nghiệp vụ thường rơi vào một vài mảng lớn — dashboard, báo cáo, và phân tích đột xuất (ad hoc). Một nhà phân tích nghiệp vụ có thể tập trung vào một hoặc tất cả các mảng này. Hãy nhanh chóng xem những khác biệt giữa các bộ môn thực hành này và các công cụ liên quan. Hiểu được quy trình làm việc của một nhà phân tích sẽ giúp bạn, kỹ sư dữ liệu, hiểu cách phục vụ dữ liệu. Một dashboard cho người ra quyết định thấy một cách súc tích hiệu quả hoạt động của tổ chức so với một số ít chỉ số cốt lõi, chẳng hạn doanh số và tỷ lệ giữ chân khách hàng. Các chỉ số cốt lõi này được trình bày dưới dạng các trực quan hóa (ví dụ: biểu đồ hay bản đồ nhiệt), thống kê tóm tắt, hoặc thậm chí một con số đơn lẻ. Điều này tương tự bảng đồng hồ trên xe hơi, nơi cho bạn một bản hiển thị duy nhất về những điều trọng yếu bạn cần biết khi đang lái xe. Một tổ chức có thể có nhiều hơn một dashboard, với các lãnh đạo cấp C dùng một dashboard bao quát, còn cấp dưới trực tiếp của họ dùng các dashboard với các chỉ số, KPI, hay các mục tiêu và kết quả then chốt (OKR) riêng của mình. Các nhà phân tích giúp tạo và bảo trì những dashboard này. Một khi các bên liên quan nghiệp vụ đón nhận và dựa vào một dashboard, nhà phân tích thường đáp ứng các yêu cầu xem xét một vấn đề tiềm ẩn với một chỉ số hoặc thêm một chỉ số mới vào dashboard. Hiện nay, bạn có thể dùng các nền tảng BI để tạo dashboard, chẳng hạn Tableau, Looker, Sisense, Power BI, hay Apache Superset/Preset. Các nhà phân tích thường được các bên liên quan nghiệp vụ giao nhiệm vụ tạo một báo cáo (report). Mục tiêu của một báo cáo là dùng dữ liệu để thúc đẩy thông tin chuyên sâu và hành động. Một nhà phân tích làm việc tại một công ty bán lẻ trực tuyến được yêu cầu điều tra những yếu tố nào đang khiến tỷ lệ trả hàng của quần short chạy bộ nữ cao hơn dự kiến. Nhà phân tích chạy một số truy vấn SQL trong kho dữ liệu, tổng hợp các mã trả hàng mà khách hàng cung cấp làm lý do trả hàng, và phát hiện rằng vải của quần short chạy bộ có chất lượng kém, thường mòn hỏng chỉ sau vài lần dùng. Các bên liên quan như bộ phận sản xuất và kiểm soát chất lượng được thông báo về những phát hiện này. Hơn nữa, các phát hiện được tóm tắt trong một báo cáo và phân phối trong cùng công cụ BI nơi đặt dashboard. Nhà phân tích được yêu cầu đào sâu vào một vấn đề tiềm ẩn và quay lại với các thông tin chuyên sâu. Đây là một ví dụ về phân tích đột xuất (ad hoc analysis). Các báo cáo thường khởi đầu như những yêu cầu đột xuất. Nếu kết quả của phân tích đột xuất có tác động, chúng thường trở thành một báo cáo hoặc dashboard. Các công nghệ dùng cho báo cáo và phân tích đột xuất tương tự như dashboard nhưng có thể bao gồm Excel, Python, các notebook dựa trên R, các truy vấn SQL, và nhiều thứ khác. Các nhà phân tích giỏi không ngừng gắn bó với hoạt động kinh doanh và lặn sâu vào dữ liệu để trả lời các câu hỏi và phát hiện những xu hướng cùng thông tin chuyên sâu ẩn giấu và phản trực giác. Họ cũng làm việc với các kỹ sư dữ liệu để cung cấp phản hồi về chất lượng dữ liệu, các vấn đề về độ tin cậy, và các yêu cầu về tập dữ liệu mới. Kỹ sư dữ liệu chịu trách nhiệm xử lý phản hồi này và cung cấp các tập dữ liệu mới để nhà phân tích sử dụng.

Returning to the running shorts example, suppose that after communicating their findings, analysts learn that manufacturing can provide them with various supply-chain details regarding the materials used in the running shorts. Data engineers undertake a project to ingest this data into the data warehouse. Once the supply-chain data is present, analysts can correlate specific garment serial numbers with the supplier of the fabric used in the item. They discover that most failures are tied to one of their three suppliers, and the factory stops using fabric from this supplier. The

data for business analytics is frequently served in **batch** mode from a data warehouse or a **data lake**. This varies wildly across companies, departments, and even data teams within companies. New data might be available every second, minute, every 30 minutes, every day, or once a week. The frequency of the batches can vary for several reasons. One key thing to note is that engineers working on analytics problems should consider various potential applications of data, current, and future. It is common to have mixed data update frequencies to serve use cases appropriately but remember that the frequency of **ingestion** sets a ceiling on downstream frequency. If **streaming** applications exist for the data, it should be ingested as a stream even if some downstream processing and serving steps are handled in batches. Of course, data engineers must address various backend technical considerations in serving business analytics. Some BI tools store data in an internal **storage** layer. Other tools run queries on your data lake or data warehouse. This is advantageous because you can take full advantage of your **OLAP** database's power. As we've discussed in earlier chapters, the downside is cost, **access control**, and **latency**.

Quay lại ví dụ quần short chạy bộ, giả sử rằng sau khi truyền đạt các phát hiện của mình, các nhà phân tích biết được rằng bộ phận sản xuất có thể cung cấp cho họ nhiều chi tiết chuỗi cung ứng về các vật liệu dùng trong quần short chạy bộ. Các kỹ sư dữ liệu thực hiện một dự án để thu nạp (ingestion) dữ liệu này vào kho dữ liệu. Một khi dữ liệu chuỗi cung ứng đã có mặt, các nhà phân tích có thể tương quan các số sê-ri cụ thể của sản phẩm may mặc với nhà cung cấp vải dùng trong món hàng đó. Họ phát hiện rằng hầu hết các lỗi gắn với một trong ba nhà cung cấp của họ, và nhà máy ngừng dùng vải từ nhà cung cấp này. Dữ liệu cho phân tích nghiệp vụ thường được phục vụ theo lô (batch) từ một kho dữ liệu hoặc một hồ dữ liệu. Điều này biến thiên rất nhiều giữa các công ty, phòng ban, và thậm chí giữa các đội dữ liệu trong cùng công ty. Dữ liệu mới có thể có sẵn mỗi giây, mỗi phút, mỗi 30 phút, mỗi ngày, hoặc mỗi tuần một lần. Tần suất của các lô có thể biến thiên vì nhiều lý do. Một điều then chốt cần lưu ý là các kỹ sư làm việc với các bài toán phân tích nên cân nhắc nhiều ứng dụng tiềm năng khác nhau của dữ liệu, cả hiện tại và tương lai. Việc có các tần suất cập nhật dữ liệu hỗn hợp để phục vụ các tình huống sử dụng một cách thích đáng là phổ biến, nhưng hãy nhớ rằng tần suất thu nạp đặt ra một trần cho tần suất ở hạ nguồn. Nếu có các ứng dụng theo luồng (streaming) cho dữ liệu, nó nên được thu nạp như một luồng ngay cả khi một số bước xử lý và phục vụ ở hạ nguồn được xử lý theo lô. Tất nhiên, các kỹ sư dữ liệu phải xử lý nhiều điều cần cân nhắc kỹ thuật ở phía backend trong việc phục vụ phân tích nghiệp vụ. Một số công cụ BI lưu trữ dữ liệu trong một tầng lưu trữ nội bộ. Các công cụ khác chạy truy vấn trên hồ dữ liệu hoặc kho dữ liệu của bạn. Điều này có lợi vì bạn có thể tận dụng triệt để sức mạnh của cơ sở dữ liệu OLAP. Như chúng ta đã thảo luận trong các chương trước, nhược điểm là chi phí, kiểm soát truy cập, và độ trễ (latency).

Operational Analytics — Phân tích vận hành

If business analytics is about using data to discover actionable insights, then operational analytics uses data to take immediate action :

Nếu phân tích nghiệp vụ là về việc dùng dữ liệu để khám phá các thông tin chuyên sâu có thể hành động được, thì phân tích vận hành (operational analytics) dùng dữ liệu để hành động ngay lập tức:

Operational analytics versus business analytics = immediate action versus actionable insights The big difference between operational and business analytics is time . Data used in business analytics takes a longer view of the question under consideration. Up-to-the-second updates are nice to know but won't materially impact the quality or outcome. Operational analytics is quite the opposite,

as real-time updates can be impactful in addressing a problem when it occurs. An example of operational analytics is real-time application monitoring. Many software engineering teams want to know how their application is performing; if issues arise, they want to be notified immediately. The engineering team might have a dashboard (see, e.g., Figure 9-2) that shows the key metrics such as requests per second, database I/O, or whatever metrics are important. Certain conditions can trigger **scaling** events, adding more capacity if servers are overloaded. If certain thresholds are breached, the monitoring system might also send alerts via text message, group chat, and email.

Phân tích vận hành so với phân tích nghiệp vụ = hành động ngay lập tức so với các thông tin chuyên sâu có thể hành động được. Khác biệt lớn giữa phân tích vận hành và phân tích nghiệp vụ là thời gian. Dữ liệu dùng trong phân tích nghiệp vụ có một góc nhìn dài hơi hơn về câu hỏi đang được xem xét. Các cập nhật theo từng giây là điều hay để biết nhưng sẽ không ảnh hưởng đáng kể đến chất lượng hay kết quả. Phân tích vận hành thì hoàn toàn ngược lại, vì các cập nhật thời gian thực có thể có tác động trong việc xử lý một vấn đề khi nó xảy ra. Một ví dụ về phân tích vận hành là giám sát ứng dụng thời gian thực. Nhiều đội kỹ thuật phần mềm muốn biết ứng dụng của họ đang hoạt động ra sao; nếu phát sinh vấn đề, họ muốn được thông báo ngay lập tức. Đội kỹ thuật có thể có một dashboard (xem ví dụ ở Hình 9-2) hiển thị các chỉ số then chốt như số yêu cầu mỗi giây, I/O của cơ sở dữ liệu, hay bất kỳ chỉ số nào quan trọng. Một số điều kiện nhất định có thể kích hoạt các sự kiện mở rộng quy mô, thêm năng lực nếu các máy chủ bị quá tải. Nếu một số ngưỡng nhất định bị vượt qua, hệ thống giám sát cũng có thể gửi cảnh báo qua tin nhắn văn bản, trò chuyện nhóm, và email.

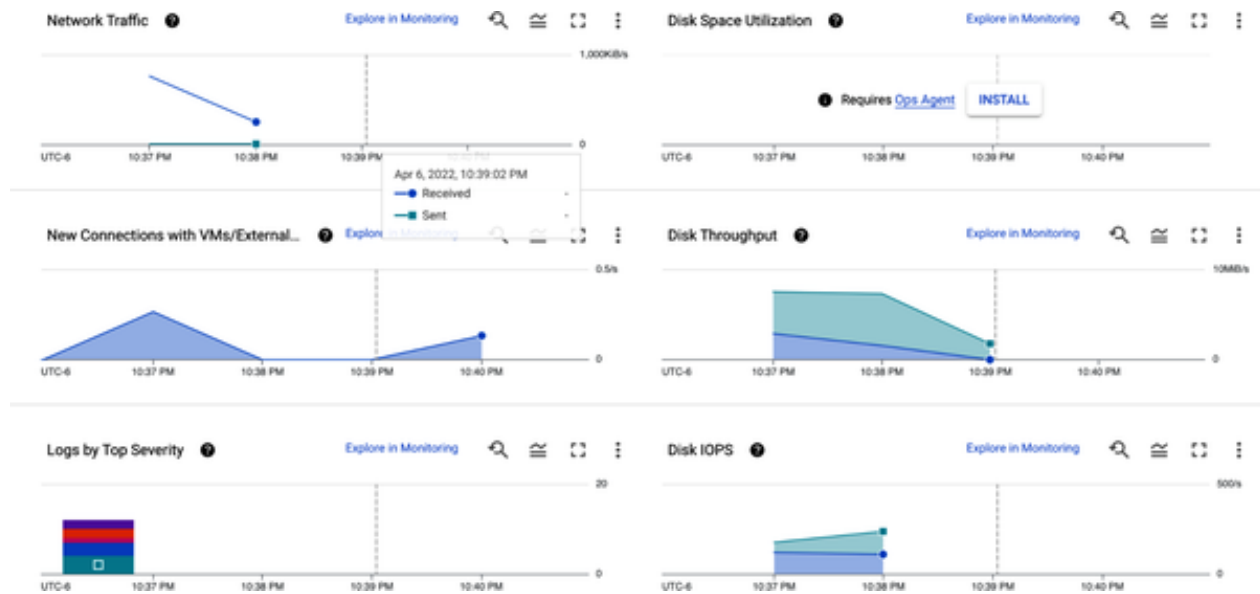


Figure 9-2. An operational analytics dashboard showing some key metrics from Google Compute Engine

Hình 9-2. Một dashboard phân tích vận hành hiển thị một số chỉ số then chốt từ Google Compute Engine.

BUSINESS AND OPERATIONAL ANALYTICS — PHÂN TÍCH NGHIỆP VỤ VÀ PHÂN TÍCH VẬN HÀNH

The line between business and operational analytics has begun to blur. As streaming and low-latency data become more pervasive, it is only natural to apply operational approaches to business analytics problems; in addition to monitoring website performance on Black Friday, an online retailer could also analyze and present sales, revenue and the impact of advertising campaigns in real time. The data architectures will change to fit into a world where you can have both your red **hot** and **warm** data in one place. The central question you should always ask yourself, and your stakeholders, is this: if you have streaming data, what are you going to do with it? What action should you take? Correct action creates impact and value. Real-time data without action is an unrelenting distraction. In the long term, we predict that streaming will supplant batch. Data products over the next 10 years will likely be streaming-first, with the ability to seamlessly blend historical data. After real-time collection, data can still be consumed and processed in batches as required.

Ranh giới giữa phân tích nghiệp vụ và phân tích vận hành đã bắt đầu mờ nhòe. Khi dữ liệu theo luồng và độ trễ thấp trở nên phổ biến hơn, việc áp dụng các cách tiếp cận vận hành vào các bài toán phân tích nghiệp vụ là điều rất tự nhiên; ngoài việc giám sát hiệu năng website trong ngày Black Friday, một nhà bán lẻ trực tuyến cũng có thể phân tích và trình bày doanh số, doanh thu cùng tác động của các chiến dịch quảng cáo theo thời gian thực. Các kiến trúc dữ liệu sẽ thay đổi để phù hợp với một thế giới nơi bạn có thể có cả dữ liệu nóng rực và dữ liệu ấm ở cùng một chỗ. Câu hỏi trung tâm mà bạn luôn nên tự hỏi mình, và hỏi các bên liên quan, là: nếu bạn có dữ liệu theo luồng, bạn sẽ làm gì với nó? Bạn nên thực hiện hành động nào? Hành động đúng tạo ra tác động và giá trị. Dữ liệu thời gian thực mà không có hành động là một sự xao nhãng không ngừng nghỉ. Về lâu dài, chúng tôi dự đoán rằng theo luồng sẽ thay thế theo lô. Các sản phẩm dữ liệu trong 10 năm tới nhiều khả năng sẽ ưu tiên theo luồng (streaming-first), với khả năng hòa trộn liền mạch dữ liệu lịch sử. Sau khi thu thập thời gian thực, dữ liệu vẫn có thể được tiêu thụ và xử lý theo lô khi cần.

Let's return once again to our running shorts example. Using analytics to discover bad fabric in the supply chain was a huge success; business leaders and data engineers want to find more opportunities to utilize data to improve product quality. The data engineers suggest deploying real-time analytics at the factory. The plant already uses a variety of machines capable of streaming real-time data. In addition, the plant has cameras recording video on the manufacturing line. Right now, technicians watch the footage in real time, look for defective items, and alert those running the line when they see a high rate of snags appearing in items. Data engineers realize that they can use an off-the-shelf cloud machine vision tool to identify defects in real time automatically. Defect data is tied to specific item serial numbers and streamed. From here, a real-time analytics process can tie defective items to streaming events from machines further up the assembly line.

Hãy quay lại một lần nữa với ví dụ quần short chạy bộ. Việc dùng phân tích để phát hiện vải kém trong chuỗi cung ứng đã là một thành công lớn; các lãnh đạo nghiệp vụ và kỹ sư dữ liệu muốn tìm thêm cơ hội tận dụng dữ liệu để cải thiện chất lượng sản phẩm. Các kỹ sư dữ liệu đề xuất triển khai phân tích thời gian thực tại nhà máy. Nhà máy vốn đã dùng nhiều loại máy móc có khả năng truyền dữ liệu thời gian thực. Ngoài ra, nhà máy có các camera ghi hình trên dây chuyền sản xuất. Hiện tại, các kỹ thuật viên xem cảnh quay theo thời gian thực, tìm các món hàng lỗi, và cảnh báo những người vận hành dây chuyền khi họ thấy một tỷ lệ cao các vết rách xuất hiện trong các món hàng. Các kỹ sư dữ liệu nhận ra rằng họ có thể dùng một công cụ thị giác máy (machine vision) trên đám mây có sẵn để tự động nhận diện lỗi theo thời gian thực. Dữ liệu lỗi được gắn với các số sê-ri sản phẩm cụ

thể và được truyền theo luồng. Từ đây, một quy trình phân tích thời gian thực có thể gắn các món hàng lỗi với các sự kiện theo luồng từ các máy ở phía trên dây chuyền lắp ráp.

Using this approach, factory floor analysts discover that the quality of raw fabric stock varies significantly from box to box. When the monitoring system shows a high rate of snag defects, line workers can remove the defective box and charge it back to the supplier. Seeing the success of this quality improvement project, the supplier decides to adopt similar quality-control processes. Data engineers from the retailer work with the supplier to deploy their real-time data analytics, dramatically improving the quality of their fabric stock.

Dùng cách tiếp cận này, các nhà phân tích tại xưởng sản xuất phát hiện rằng chất lượng của cuộn vải thô biến thiên đáng kể từ hộp này sang hộp khác. Khi hệ thống giám sát cho thấy một tỷ lệ cao lỗi vết rách, công nhân dây chuyền có thể loại bỏ hộp vải lỗi và quy trách nhiệm tính phí ngược lại cho nhà cung cấp. Thấy được sự thành công của dự án cải thiện chất lượng này, nhà cung cấp quyết định áp dụng các quy trình kiểm soát chất lượng tương tự. Các kỹ sư dữ liệu của nhà bán lẻ làm việc với nhà cung cấp để triển khai hệ thống phân tích dữ liệu thời gian thực của họ, cải thiện đáng kể chất lượng của cuộn vải thô.

Embedded Analytics — Phân tích nhúng

Whereas business and operational analytics are internally focused, a recent trend is external- or user-facing analytics. With so much data powering applications, companies increasingly provide analytics to end users. These are typically referred to as data applications , often with analytics dashboards embedded within the application itself. Also known as embedded analytics , these end-user-facing dashboards give users key metrics about their relationship with the application. A smart thermostat has a mobile application that shows the temperature in real time and up-to-date power consumption metrics, allowing the user to create a better energy-efficient heating or cooling schedule. In another example, a third-party ecommerce platform provides its sellers a real-time dashboard on sales, inventory, and returns. The seller has the option to use this information to offer deals to customers in near real time. In both cases, an application allows users to make real-time decisions (manually or automatically) based on data. The landscape of embedded analytics is snowballing, and we expect that such data applications will become increasingly pervasive within the next few years. As a data engineer, you're probably not creating the embedded analytics frontend, as the application developers handle that. Since you're responsible for the databases serving the embedded analytics, you'll need to understand the speed and latency requirements for user-facing analytics.

Trong khi phân tích nghiệp vụ và phân tích vận hành hướng về nội bộ, một xu hướng gần đây là phân tích hướng ra bên ngoài hoặc hướng người dùng. Với quá nhiều dữ liệu cung cấp sức mạnh cho các ứng dụng, các công ty ngày càng cung cấp phân tích cho người dùng cuối. Những thứ này thường được gọi là các ứng dụng dữ liệu (data application), thường đi kèm các dashboard phân tích nhúng ngay bên trong chính ứng dụng. Còn được gọi là phân tích nhúng (embedded analytics), những dashboard hướng người dùng cuối này cho người dùng các chỉ số then chốt về mối quan hệ của họ với ứng dụng. Một bộ điều nhiệt thông minh có một ứng dụng di động hiển thị nhiệt độ theo thời gian thực và các chỉ số tiêu thụ điện năng cập nhật, cho phép người dùng tạo một lịch sưởi ấm hoặc làm mát tiết kiệm năng lượng hơn. Trong một ví dụ khác, một nền tảng thương mại điện tử của bên thứ ba cung cấp cho các người bán của mình một dashboard thời gian thực về doanh số, tồn kho, và trả hàng. Người bán có tùy chọn dùng thông tin này để đưa ra các ưu đãi cho khách hàng gần như theo thời gian thực. Trong cả hai trường hợp, một ứng dụng cho phép người dùng ra các quyết định thời gian thực (thủ công hoặc tự động) dựa trên dữ liệu. Bởi

cảnh của phân tích nhúng đang phình to như quả cầu tuyết, và chúng tôi kỳ vọng những ứng dụng dữ liệu như vậy sẽ trở nên ngày càng phổ biến trong vài năm tới. Với vai trò một kỹ sư dữ liệu, bạn có lẽ không tạo phần giao diện người dùng (frontend) của phân tích nhúng, vì các nhà phát triển ứng dụng xử lý phần đó. Vì bạn chịu trách nhiệm về các cơ sở dữ liệu phục vụ phân tích nhúng, bạn sẽ cần hiểu các yêu cầu về tốc độ và độ trễ cho phân tích hướng người dùng.

Performance for embedded analytics encompasses three problems. First, app users are not as tolerant of infrequent batch processing as internal company analysts; users of a recruiting SaaS platform may expect to see a change in their statistics as soon as they upload a new resume. Users want low data latency. Second, users of data apps expect fast query performance. When they adjust parameters in an analytics dashboard, they want to see refreshed results appear in seconds. Third, data apps must often support extremely high query rates across many dashboards and numerous customers. High concurrency is critical. Google and other early major players in the data apps space developed exotic technologies to cope with these challenges. For new startups, the default is to use conventional transactional databases for data applications. As their customer bases expand, they outgrow their initial architecture. They have access to a new **generation** of databases that combine high performance—fast queries, high concurrency, and near real-time updates—with relative ease of use (e.g., SQL-based analytics).

Hiệu năng cho phân tích nhúng bao hàm ba vấn đề. Thứ nhất, người dùng ứng dụng không khoan dung với việc xử lý theo lô không thường xuyên như các nhà phân tích nội bộ của công ty; người dùng một nền tảng SaaS tuyển dụng có thể mong đợi thấy một thay đổi trong các thống kê của họ ngay khi họ tải lên một bản lý lịch mới. Người dùng muốn độ trễ dữ liệu thấp. Thứ hai, người dùng các ứng dụng dữ liệu mong đợi hiệu năng truy vấn nhanh. Khi họ điều chỉnh các tham số trong một dashboard phân tích, họ muốn thấy các kết quả được làm mới xuất hiện trong vài giây. Thứ ba, các ứng dụng dữ liệu thường phải hỗ trợ tốc độ truy vấn cực kỳ cao trên nhiều dashboard và vô số khách hàng. Mức đồng thời (concurrency) cao là điều then chốt. Google và các tay chơi lớn buổi đầu khác trong lĩnh vực ứng dụng dữ liệu đã phát triển các công nghệ kỳ lạ để đương đầu với những thách thức này. Với các startup mới, lựa chọn mặc định là dùng các cơ sở dữ liệu giao dịch (transactional) thông thường cho các ứng dụng dữ liệu. Khi nền tảng khách hàng của họ mở rộng, họ vượt qua khỏi kiến trúc ban đầu. Họ có quyền truy cập tới một thể hệ cơ sở dữ liệu mới kết hợp hiệu năng cao — truy vấn nhanh, mức đồng thời cao, và cập nhật gần như thời gian thực — với mức độ dễ dùng tương đối (ví dụ: phân tích dựa trên SQL).

Machine Learning — Học máy

The second major area for serving data is **machine learning**. ML is increasingly common, so we'll assume you're at least familiar with the concept. With the rise of ML engineering (itself almost a parallel universe to data engineering), you might ask yourself where a data engineer fits into the picture. Admittedly, the boundary between ML, data science, data engineering, and ML engineering is increasingly fuzzy, and this boundary varies dramatically between organizations. In some organizations, ML engineers take over data processing for ML applications right after data collection or may even form an entirely separate and parallel data organization that handles the entire lifecycle for all ML applications. Data engineers handle all data processing in other settings and then hand off data to ML engineers for model training. Data engineers may even handle some extremely ML-specific tasks, such as featurization of data.

Lĩnh vực lớn thứ hai để phục vụ dữ liệu là học máy (machine learning). ML ngày càng phổ biến, nên chúng tôi sẽ giả định rằng bạn ít nhất đã quen thuộc với khái niệm này. Với sự

trỗi dậy của kỹ thuật ML (bản thân nó gần như là một vũ trụ song song với kỹ thuật dữ liệu), bạn có thể tự hỏi một kỹ sư dữ liệu khớp vào đâu trong bức tranh này. Phải thừa nhận rằng ranh giới giữa ML, khoa học dữ liệu, kỹ thuật dữ liệu, và kỹ thuật ML ngày càng mờ nhòe, và ranh giới này biến thiên đáng kể giữa các tổ chức. Ở một số tổ chức, các kỹ sư ML tiếp quản việc xử lý dữ liệu cho các ứng dụng ML ngay sau khi thu thập dữ liệu hoặc thậm chí có thể hình thành một tổ chức dữ liệu hoàn toàn riêng biệt và song song, đảm nhận toàn bộ vòng đời cho tất cả các ứng dụng ML. Trong các môi trường khác, các kỹ sư dữ liệu xử lý toàn bộ việc xử lý dữ liệu rồi bàn giao dữ liệu cho các kỹ sư ML để huấn luyện mô hình. Các kỹ sư dữ liệu thậm chí có thể xử lý một số tác vụ cực kỳ đặc thù với ML, chẳng hạn việc tạo đặc trưng (featurization) cho dữ liệu.

Let's return to our example of quality for control of running shorts produced by an online retailer. Suppose that streaming data has been implemented in the factory that makes the raw fabric stock for the shorts. Data scientists discovered that the quality of the manufactured fabric is susceptible to characteristics of the input raw polyester, temperature, humidity, and various tunable parameters of the loom that weaves the fabric. Data scientists develop a basic model to optimize loom parameters. ML engineers automate model training and set up a process to automatically tune the loom based on input parameters. Data and ML engineers work together to design a featurization pipeline, and data engineers implement and maintain the pipeline.

Hãy quay lại ví dụ về kiểm soát chất lượng của quần short chạy bộ do một nhà bán lẻ trực tuyến sản xuất. Giả sử rằng dữ liệu theo luồng đã được triển khai tại nhà máy làm ra cuộn vải thô cho quần short. Các nhà khoa học dữ liệu phát hiện rằng chất lượng của vải được sản xuất bị ảnh hưởng bởi các đặc tính của polyester thô đầu vào, nhiệt độ, độ ẩm, và nhiều tham số có thể tinh chỉnh của khung dệt vải. Các nhà khoa học dữ liệu phát triển một mô hình cơ bản để tối ưu các tham số khung dệt. Các kỹ sư ML tự động hóa việc huấn luyện mô hình và thiết lập một quy trình để tự động tinh chỉnh khung dệt dựa trên các tham số đầu vào. Các kỹ sư dữ liệu và kỹ sư ML làm việc cùng nhau để thiết kế một đường ống tạo đặc trưng (featurization pipeline), và các kỹ sư dữ liệu triển khai và bảo trì đường ống đó.

What a Data Engineer Should Know About ML — Những điều kỹ sư dữ liệu nên biết về ML

Before we discuss serving data for ML, you may ask yourself how much ML you need to know as a data engineer. ML is an incredibly vast topic, and we won't attempt to teach you the field; countless books and courses are available to learn ML. While a data engineer doesn't need to have a deep understanding of ML, it helps tremendously to know the basics of how classical ML works and the fundamentals of deep learning. Knowing the basics of ML will go a long way in helping you work alongside data scientists in building data products. Here are some areas of ML that we think a data engineer should be familiar with:

Trước khi thảo luận về việc phục vụ dữ liệu cho ML, bạn có thể tự hỏi mình cần biết bao nhiêu về ML với vai trò một kỹ sư dữ liệu. ML là một chủ đề rộng lớn đến mức khó tin, và chúng tôi sẽ không cố dạy bạn cả lĩnh vực này; vô số sách và khóa học có sẵn để học ML. Mặc dù một kỹ sư dữ liệu không cần có hiểu biết sâu sắc về ML, việc biết những điều cơ bản về cách ML cổ điển hoạt động và các nền tảng của học sâu (deep learning) giúp ích rất nhiều. Biết những điều cơ bản của ML sẽ đóng góp lớn vào việc giúp bạn làm việc bên cạnh các nhà khoa học dữ liệu trong việc xây dựng các sản phẩm dữ liệu. Sau đây là một số mảng của ML mà chúng tôi nghĩ một kỹ sư dữ liệu nên quen thuộc:

The difference between supervised, unsupervised, and semisupervised learning.

Sự khác biệt giữa học có giám sát, học không giám sát, và học bán giám sát (supervised, unsupervised, semisupervised learning).

The difference between classification and regression techniques.

Sự khác biệt giữa các kỹ thuật phân loại (classification) và hồi quy (regression).

The various techniques for handling time-series data. This includes time-series analysis, as well as time-series forecasting.

Các kỹ thuật khác nhau để xử lý dữ liệu chuỗi thời gian (time-series). Điều này bao gồm phân tích chuỗi thời gian, cũng như dự báo chuỗi thời gian.

When to use the “classical” techniques (logistic regression, tree-based learning, support vector machines) versus deep learning. We constantly see data scientists immediately jump to deep learning when it’s overkill. As a data engineer, your basic knowledge of ML can help you spot whether an ML technique is appropriate and scales the data you’ll need to provide.

Khi nào dùng các kỹ thuật “cổ điển” (hồi quy logistic, học dựa trên cây, máy vector hỗ trợ) so với học sâu. Chúng tôi liên tục thấy các nhà khoa học dữ liệu lập tức nhảy ngay sang học sâu khi nó là quá mức cần thiết. Với vai trò một kỹ sư dữ liệu, kiến thức cơ bản của bạn về ML có thể giúp bạn nhận ra liệu một kỹ thuật ML có thích hợp hay không và định cỡ lượng dữ liệu bạn sẽ cần cung cấp.

When would you use automated machine learning (AutoML) versus handcrafting an ML model? What are the trade-offs with each approach regarding the data being used?

Khi nào bạn sẽ dùng học máy tự động (AutoML) so với việc tự tay chế tác một mô hình ML? Những đánh đổi với mỗi cách tiếp cận liên quan đến dữ liệu được dùng là gì?

What are data-wrangling techniques used for structured and unstructured data?

Các kỹ thuật nhào nặn dữ liệu (data-wrangling) được dùng cho dữ liệu có cấu trúc và phi cấu trúc là gì?

All data that is used for ML is converted to numbers. If you’re serving structured or semistructured data, ensure that the data can be properly converted during the feature-engineering process.

Tất cả dữ liệu dùng cho ML đều được chuyển thành số. Nếu bạn đang phục vụ dữ liệu có cấu trúc hoặc bán cấu trúc, hãy đảm bảo dữ liệu có thể được chuyển đổi đúng cách trong quá trình thiết kế đặc trưng (feature engineering).

How to encode categorical data and the embeddings for various types of data.

Cách mã hóa dữ liệu phân loại (categorical) và các phép nhúng (embedding) cho các loại dữ liệu khác nhau.

The difference between batch and online learning. Which approach is appropriate for your use case?

Sự khác biệt giữa học theo lô (batch) và học trực tuyến (online). Cách tiếp cận nào thích hợp cho tình huống sử dụng của bạn?

How does the data engineering lifecycle intersect with the ML lifecycle at your company? Will you be responsible for interfacing with or supporting ML technologies such as feature stores or ML observability?

Vòng đời kỹ thuật dữ liệu giao cắt với vòng đời ML tại công ty bạn như thế nào? Liệu bạn có chịu trách nhiệm giao tiếp với hoặc hỗ trợ các công nghệ ML như feature store hay khả năng quan sát ML (ML observability) không?

Know when it's appropriate to train locally, on a cluster, or at the edge. When would you use a GPU over a CPU? The type of hardware you use largely depends on the type of ML problem you're solving, the technique you're using, and the size of your dataset.

Hãy biết khi nào thì thích hợp để huấn luyện cục bộ, trên một cụm (cluster), hay tại biên (edge). Khi nào bạn sẽ dùng GPU thay vì CPU? Loại phần cứng bạn dùng phần lớn phụ thuộc vào loại bài toán ML bạn đang giải quyết, kỹ thuật bạn đang dùng, và kích cỡ tập dữ liệu của bạn.

Know the difference between the applications of batch and streaming data in training ML models. For example, batch data often fits well with offline model training, while streaming data works with online training.

Hãy biết sự khác biệt giữa các ứng dụng của dữ liệu theo lô và dữ liệu theo luồng trong việc huấn luyện các mô hình ML. Ví dụ, dữ liệu theo lô thường khớp tốt với việc huấn luyện mô hình ngoại tuyến (offline), trong khi dữ liệu theo luồng hoạt động với việc huấn luyện trực tuyến (online).

What are data cascades, and how might they impact ML models?

Thác dữ liệu (data cascade) là gì, và chúng có thể tác động đến các mô hình ML như thế nào?

Are results returned in real time or in batch? For example, a batch speech transcription model might process speech samples and return text in batch after an API call. A product recommendation model might need to operate in real time as the customer interacts with an online retail site.

Kết quả được trả về theo thời gian thực hay theo lô? Ví dụ, một mô hình ghi âm thành văn bản theo lô có thể xử lý các mẫu giọng nói và trả về văn bản theo lô sau một lệnh gọi API. Một mô hình gợi ý sản phẩm có thể cần hoạt động theo thời gian thực khi khách hàng tương tác với một trang bán lẻ trực tuyến.

The use of structured versus unstructured data. We might cluster tabular (structured) customer data or recognize images (unstructured) by using a neural net.

Việc dùng dữ liệu có cấu trúc so với dữ liệu phi cấu trúc. Chúng ta có thể gom cụm (clustering) dữ liệu khách hàng dạng bảng (có cấu trúc) hoặc nhận diện hình ảnh (phi cấu trúc) bằng cách dùng một mạng nơ-ron (neural net).

ML is a vast subject area, and this book won't teach you these topics, or even ML generalities. If you'd like to learn more about ML, we suggest reading *Hands on Machine Learning with Scikit-Learn, Keras, and TensorFlow* by Aurélien Géron (O'Reilly); countless other ML courses and books are available online. Because the books and online courses evolve so rapidly, do your research on what seems like a good fit for you.

ML là một mảng chủ đề rộng lớn, và cuốn sách này sẽ không dạy bạn những chủ đề này, hay thậm chí những điều tổng quát về ML. Nếu bạn muốn tìm hiểu thêm về ML, chúng tôi gợi ý đọc cuốn *Hands on Machine Learning with Scikit-Learn, Keras, and TensorFlow* của Aurélien Géron (O'Reilly); vô số khóa học và sách ML khác có sẵn trực tuyến. Vì sách và các khóa học trực tuyến tiến hóa rất nhanh, hãy tự nghiên cứu xem cái nào có vẻ phù hợp với bạn.

Ways to Serve Data for Analytics and ML — Các cách phục vụ dữ liệu cho phân tích và ML

As with analytics, data engineers provide data scientists and ML engineers with the data they need to do their jobs. We have placed serving for ML alongside analytics because the pipelines and processes are extremely similar. There are many ways to serve data for analytics and ML. Some common ways to serve this data include files, databases, query engines, and data sharing. Let's briefly look at each.

Cũng như với phân tích, các kỹ sư dữ liệu cung cấp cho các nhà khoa học dữ liệu và kỹ sư ML dữ liệu họ cần để làm công việc của mình. Chúng tôi đặt việc phục vụ cho ML bên cạnh phân tích vì các đường ống và quy trình cực kỳ giống nhau. Có nhiều cách phục vụ dữ liệu cho phân tích và ML. Một số cách phổ biến để phục vụ dữ liệu này bao gồm các tệp, cơ sở dữ liệu, công cụ truy vấn (query engine), và chia sẻ dữ liệu (data sharing). Hãy xem qua từng cách một cách ngắn gọn.

File Exchange — Trao đổi tệp

File exchange is ubiquitous in data serving. We process data and generate files to pass to data consumers. Keep in mind that a file might be used for many purposes. A data scientist might load a text file (unstructured data) of customer messages to analyze the sentiments of customer complaints. A business unit might receive invoice data from a partner company as a collection of CSVs (structured data), and an analyst must perform some statistical analysis on these files.

Trao đổi tệp (file exchange) có mặt khắp nơi trong việc phục vụ dữ liệu. Chúng ta xử lý dữ liệu và tạo ra các tệp để chuyển cho các bên tiêu thụ dữ liệu. Hãy ghi nhớ rằng một tệp có thể được dùng cho nhiều mục đích. Một nhà khoa học dữ liệu có thể nạp một tệp văn bản (dữ liệu phi cấu trúc) gồm các tin nhắn của khách hàng để phân tích cảm xúc của các khiếu nại từ khách hàng. Một đơn vị nghiệp vụ có thể nhận dữ liệu hóa đơn từ một công ty đối tác dưới dạng một bộ sưu tập các tệp CSV (dữ liệu có cấu trúc), và một nhà phân tích phải thực hiện một số phân tích thống kê trên các tệp này.

Or, a data vendor might provide an online retailer with images of products on a competitor's website (unstructured data) for automated classification using computer vision. The way you serve files depends on several factors, such as these:

Hoặc, một nhà cung cấp dữ liệu có thể cung cấp cho một nhà bán lẻ trực tuyến các hình ảnh sản phẩm trên website của đối thủ (dữ liệu phi cấu trúc) để phân loại tự động bằng thị giác máy tính (computer vision). Cách bạn phục vụ các tệp phụ thuộc vào một số yếu tố, chẳng hạn:

Use case—business analytics, operational analytics, user-facing analytics

Tình huống sử dụng — phân tích nghiệp vụ, phân tích vận hành, phân tích hướng người dùng

The **data consumer's** data-handling processes

Các quy trình xử lý dữ liệu của bên tiêu thụ dữ liệu

The size and number of individual files in storage

Kích cỡ và số lượng các tệp riêng lẻ trong lưu trữ

Who is accessing this file

Ai đang truy cập tệp này

Data type—structured, semistructured, or unstructured

Loại dữ liệu — có cấu trúc, bán cấu trúc, hoặc phi cấu trúc

The second bullet point is one of the main considerations. It is often necessary to serve data through files rather than data sharing because the data consumer cannot use a sharing platform. The simplest file to serve is something along the lines of emailing a single Excel file. This is still a common workflow even in an era when files can be collaboratively shared. The problem with emailing files is each recipient gets their version of the file. If a recipient edits the file, these edits are specific to that user's file. Deviations among files inevitably result. If you need a coherent, consistent version of a file, we suggest using a collaboration platform such as Microsoft 365 or Google Docs. Of course, serving single files is hard to scale, and your needs will eventually outgrow simple cloud **file storage**. You'll likely grow into an **object storage** bucket if you have a handful of large files, or a data lake if you have a steady supply of files. Object storage can store any type of blob file, and is especially useful for semistructured or unstructured files. We'll note that we generally consider file exchange through object storage (data lake) to land under "data sharing" rather than file exchange since the process can be significantly more scalable and streamlined than ad hoc file exchange.

Gạch đầu dòng thứ hai là một trong những điều cần cân nhắc chính. Thường thì cần phục vụ dữ liệu qua các tệp thay vì chia sẻ dữ liệu vì bên tiêu thụ dữ liệu không thể dùng một nền tảng chia sẻ. Tệp đơn giản nhất để phục vụ là một thứ đại loại như gửi email một tệp Excel đơn lẻ. Đây vẫn là một quy trình làm việc phổ biến ngay cả trong một thời đại mà các tệp có thể được chia sẻ theo kiểu cộng tác. Vấn đề với việc gửi email các tệp là mỗi người nhận có được phiên bản tệp của riêng họ. Nếu một người nhận chỉnh sửa tệp, những chỉnh sửa này chỉ riêng cho tệp của người dùng đó. Sự sai lệch giữa các tệp tất yếu xảy ra. Nếu bạn cần một phiên bản tệp mạch lạc, nhất quán, chúng tôi gợi ý dùng một nền tảng cộng tác như Microsoft 365 hay Google Docs. Tất nhiên, việc phục vụ các tệp đơn lẻ khó mở rộng quy mô, và nhu cầu của bạn cuối cùng sẽ vượt qua khỏi lưu trữ tệp đám mây đơn giản. Bạn nhiều khả năng sẽ phát triển lên một bucket lưu trữ đối tượng (object storage) nếu bạn có một số ít tệp lớn, hoặc một hồ dữ liệu nếu bạn có một nguồn cung cấp tệp đều đặn. Lưu trữ đối tượng có thể lưu bất kỳ loại tệp blob nào, và đặc biệt hữu ích cho các tệp bán cấu trúc hoặc phi cấu trúc. Chúng tôi sẽ lưu ý rằng chúng tôi thường coi việc trao đổi tệp thông qua lưu trữ đối tượng (hồ dữ liệu) thuộc về "chia sẻ dữ liệu" hơn là trao đổi tệp, vì quy trình này có thể mở rộng quy mô và được tinh gọn hơn đáng kể so với trao đổi tệp đột xuất.

Databases — Cơ sở dữ liệu

Databases are a critical layer in serving data for analytics and ML. For this discussion, we'll implicitly keep our focus on serving data from OLAP databases (e.g., data warehouses and data lakes). In the previous chapter, you learned about querying databases. Serving data involves querying a database, and then consuming those results for a use case. An analyst or data scientist might query a database by using a SQL editor and export those results to a CSV file for consumption by a downstream application, or analyze the results in a notebook (described in "Serving Data in Notebooks"). Serving data from a database carries a variety of benefits. A database imposes order and structure on the data through **schema**; databases can offer fine-grained permission controls at the table, column, and row level, allowing database administrators to craft complex access policies for various roles; and, databases can offer high serving performance for large, computationally intensive queries, high query concurrency, or both. BI systems usually share the data processing workload with a source

database, but the boundary between processing in the two systems varies. For example, a Tableau server runs an initial query to pull data from a database and stores it locally. Basic OLAP/BI slicing and dicing (interactive filtering and aggregation) runs directly on the server from the local data copy. On the other hand, Looker relies on a computational model called **query pushdown**; Looker encodes data processing logic in a specialized language (LookML), combines this with dynamic user input to generate SQL queries, runs these against the source database, and presents the output. (See “Semantic and Metrics Layers” .) Both Tableau and Looker have various configuration options for caching results to reduce the processing burden for frequently run queries. A data scientist might connect to a database, extract data, and perform feature engineering and selection. This converted dataset is then fed into an ML model; the offline model is trained and produces predictive results.

Cơ sở dữ liệu là một tầng trọng yếu trong việc phục vụ dữ liệu cho phân tích và ML. Trong phần thảo luận này, chúng tôi sẽ ngẫm giữ trọng tâm vào việc phục vụ dữ liệu từ các cơ sở dữ liệu OLAP (ví dụ: kho dữ liệu và hồ dữ liệu). Trong chương trước, bạn đã tìm hiểu về việc truy vấn các cơ sở dữ liệu. Phục vụ dữ liệu bao gồm việc truy vấn một cơ sở dữ liệu, rồi tiêu thụ những kết quả đó cho một tình huống sử dụng. Một nhà phân tích hay nhà khoa học dữ liệu có thể truy vấn một cơ sở dữ liệu bằng cách dùng một trình soạn thảo SQL và xuất những kết quả đó ra một tệp CSV để một ứng dụng hạ nguồn tiêu thụ, hoặc phân tích các kết quả trong một notebook (mô tả trong phần “Phục vụ dữ liệu trong notebook”). Phục vụ dữ liệu từ một cơ sở dữ liệu mang lại nhiều lợi ích. Một cơ sở dữ liệu áp đặt trật tự và cấu trúc lên dữ liệu thông qua lược đồ (schema); các cơ sở dữ liệu có thể cung cấp các kiểm soát quyền ở mức chi tiết tại cấp bảng, cột, và hàng, cho phép các quản trị viên cơ sở dữ liệu chế tác các chính sách truy cập phức tạp cho nhiều vai trò khác nhau; và, các cơ sở dữ liệu có thể cung cấp hiệu năng phục vụ cao cho các truy vấn lớn, đòi hỏi nhiều tính toán, mức đồng thời truy vấn cao, hoặc cả hai. Các hệ thống BI thường chia sẻ khối lượng công việc xử lý dữ liệu với một cơ sở dữ liệu nguồn, nhưng ranh giới giữa việc xử lý ở hai hệ thống biến thiên. Ví dụ, một máy chủ Tableau chạy một truy vấn ban đầu để kéo dữ liệu từ một cơ sở dữ liệu và lưu nó cục bộ. Việc mổ xẻ phân tích cơ bản kiểu OLAP/BI (lọc và tổng hợp tương tác) chạy trực tiếp trên máy chủ từ bản sao dữ liệu cục bộ. Mặt khác, Looker dựa vào một mô hình tính toán gọi là đẩy truy vấn xuống (query pushdown); Looker mã hóa logic xử lý dữ liệu trong một ngôn ngữ chuyên biệt (LookML), kết hợp nó với đầu vào động của người dùng để sinh ra các truy vấn SQL, chạy chúng trên cơ sở dữ liệu nguồn, và trình bày đầu ra. (Xem phần “Tầng ngữ nghĩa và tầng chỉ số”.) Cả Tableau và Looker đều có nhiều tùy chọn cấu hình để lưu đệm (cache) kết quả nhằm giảm gánh nặng xử lý cho các truy vấn được chạy thường xuyên. Một nhà khoa học dữ liệu có thể kết nối tới một cơ sở dữ liệu, trích xuất dữ liệu, và thực hiện thiết kế và chọn lọc đặc trưng. Tập dữ liệu đã được chuyển đổi này sau đó được đưa vào một mô hình ML; mô hình ngoại tuyến được huấn luyện và tạo ra các kết quả dự đoán.

Data engineers are quite often tasked with managing the database serving layer. This includes management of performance and costs. In databases that separate compute and storage, this is a somewhat more subtle optimization problem than in the days of fixed on-premises infrastructure. For example, it is now possible to spin up a new Spark cluster or Snowflake warehouse for each analytical or ML workload. It is generally recommended to at least split out clusters by major use cases, such as ETL and serving for analytics and data science. Often data teams choose to slice more finely, assigning one warehouse per major area. This makes it possible for different teams to budget for their query costs under the supervision of a data engineering team. Also, recall the three performance considerations that we discussed in “Embedded Analytics” . These are data latency, query performance, and concurrency. A system that can ingest directly from a stream can lower data latency. And many database architectures rely on SSD or memory caching to enhance query performance and concurrency to serve the challenging use cases inherent in user-facing analytics.

Increasingly, data platforms like Snowflake and Databricks allow analysts and data scientists to operate under a single environment, providing SQL editors and data science notebooks under one roof. Because compute and storage are separated, the analysts and data scientists can consume the underlying data in various ways without interfering with each other. This will allow high **throughput** and faster delivery of data products to stakeholders.

Các kỹ sư dữ liệu khá thường xuyên được giao nhiệm vụ quản lý tăng phục vụ của cơ sở dữ liệu. Điều này bao gồm việc quản lý hiệu năng và chi phí. Trong các cơ sở dữ liệu tách rời tính toán và lưu trữ, đây là một bài toán tối ưu hơi tình tế hơn so với thời của hạ tầng tại chỗ (on-premises) cố định. Ví dụ, giờ đây có thể khởi tạo một cụm Spark mới hoặc một kho Snowflake cho mỗi khối lượng công việc phân tích hay ML. Nhìn chung nên ít nhất tách các cụm theo các tình huống sử dụng chính, chẳng hạn ETL và phục vụ cho phân tích và khoa học dữ liệu. Các đội dữ liệu thường chọn cách chia nhỏ hơn nữa, gán một kho cho mỗi mảng chính. Điều này giúp các đội khác nhau có thể lập ngân sách cho chi phí truy vấn của họ dưới sự giám sát của một đội kỹ thuật dữ liệu. Ngoài ra, hãy nhớ lại ba điều cần cân nhắc về hiệu năng mà chúng ta đã thảo luận trong phần “Phân tích nhúng”. Đó là độ trễ dữ liệu, hiệu năng truy vấn, và mức đồng thời. Một hệ thống có thể thu nạp trực tiếp từ một luồng có thể giảm độ trễ dữ liệu. Và nhiều kiến trúc cơ sở dữ liệu dựa vào việc lưu đệm bằng SSD hoặc bộ nhớ để nâng cao hiệu năng truy vấn và mức đồng thời nhằm phục vụ các tình huống sử dụng đầy thách thức vốn có trong phân tích hướng người dùng. Ngày càng nhiều, các nền tảng dữ liệu như Snowflake và Databricks cho phép các nhà phân tích và nhà khoa học dữ liệu hoạt động trong một môi trường duy nhất, cung cấp các trình soạn thảo SQL và các notebook khoa học dữ liệu dưới cùng một mái nhà. Vì tính toán và lưu trữ được tách rời, các nhà phân tích và nhà khoa học dữ liệu có thể tiêu thụ dữ liệu nền tảng theo nhiều cách khác nhau mà không cản trở lẫn nhau. Điều này sẽ cho phép thông lượng (throughput) cao và chuyển giao các sản phẩm dữ liệu nhanh hơn tới các bên liên quan.

Streaming Systems — Hệ thống theo luồng

Streaming analytics are increasingly important in the realm of serving. At a high level, understand that this type of serving may involve emitted metrics , which are different from traditional queries. Also, we see operational analytics databases playing a growing role in this area (see “Operational Analytics”). These databases allow queries to run across a large range of historical data, encompassing up-to-the-second current data. Essentially, they combine aspects of OLAP databases with stream-processing systems. Increasingly, you’ll work with streaming systems to serve data for analytics and ML, so get familiar with this paradigm. You’ve learned about streaming systems throughout the book. For an idea of where it’s going, read about the live data stack in Chapter 11 .

Phân tích theo luồng (streaming analytics) ngày càng quan trọng trong địa hạt phục vụ. Ở mức cao, hãy hiểu rằng loại phục vụ này có thể liên quan đến các chỉ số được phát ra (emitted metrics), vốn khác với các truy vấn truyền thống. Ngoài ra, chúng ta thấy các cơ sở dữ liệu phân tích vận hành đóng một vai trò ngày càng lớn trong mảng này (xem phần “Phân tích vận hành”). Những cơ sở dữ liệu này cho phép các truy vấn chạy trên một dải rộng dữ liệu lịch sử, bao hàm cả dữ liệu hiện tại theo từng giây. Về cơ bản, chúng kết hợp các khía cạnh của cơ sở dữ liệu OLAP với các hệ thống xử lý luồng. Ngày càng nhiều, bạn sẽ làm việc với các hệ thống theo luồng để phục vụ dữ liệu cho phân tích và ML, nên hãy làm quen với mô hình này. Bạn đã tìm hiểu về các hệ thống theo luồng xuyên suốt cuốn sách. Để có ý niệm về hướng đi của nó, hãy đọc về ngăn xếp dữ liệu trực tiếp (live data

stack) trong Chương 11.

Query Federation — Liên kết truy vấn

As you learned in Chapter 8 , query federation pulls data from multiple sources, such as data lakes, RDBMSs, and data warehouses. Federation is becoming more popular as distributed query virtualization engines gain recognition as ways to serve queries without going through the trouble of centralizing data in an OLAP system. Today, you can find OSS options like Trino and Presto and managed services such as Starburst. Some of these offerings describe themselves as ways to enable the data mesh; time will tell how that unfolds. When serving data for federated queries, you should be aware that the end user might be querying several systems—**OLTP**, OLAP, APIs, filesystems, etc. (Figure 9-3). Instead of serving data from a single system, you’re now serving data from multiple systems, each with its usage patterns, quirks, and nuances. This poses challenges for serving data. If federated queries touch live production source systems, you must ensure that the federated query won’t consume excessive resources in the source.

Như bạn đã tìm hiểu trong Chương 8, liên kết truy vấn (query federation) kéo dữ liệu từ nhiều nguồn, chẳng hạn các hồ dữ liệu, các RDBMS, và các kho dữ liệu. Liên kết đang trở nên phổ biến hơn khi các công cụ ảo hóa truy vấn phân tán (distributed query virtualization engine) được công nhận như những cách phục vụ truy vấn mà không cần trải qua rắc rối tập trung hóa dữ liệu trong một hệ thống OLAP. Ngày nay, bạn có thể tìm thấy các lựa chọn OSS như Trino và Presto cùng các dịch vụ được quản lý như Starburst. Một số sản phẩm này tự mô tả mình như những cách để hiện thực hóa data mesh; thời gian sẽ trả lời điều đó diễn ra ra sao. Khi phục vụ dữ liệu cho các truy vấn liên kết, bạn nên ý thức rằng người dùng cuối có thể đang truy vấn nhiều hệ thống — OLTP, OLAP, các API, hệ thống tệp, v.v. (Hình 9-3). Thay vì phục vụ dữ liệu từ một hệ thống duy nhất, giờ đây bạn đang phục vụ dữ liệu từ nhiều hệ thống, mỗi hệ thống có các mẫu hình sử dụng, các điểm kỳ quặc, và các sắc thái riêng. Điều này đặt ra những thách thức cho việc phục vụ dữ liệu. Nếu các truy vấn liên kết chạm tới các hệ thống nguồn sản xuất đang hoạt động, bạn phải đảm bảo rằng truy vấn liên kết sẽ không tiêu tốn quá nhiều tài nguyên ở nguồn.

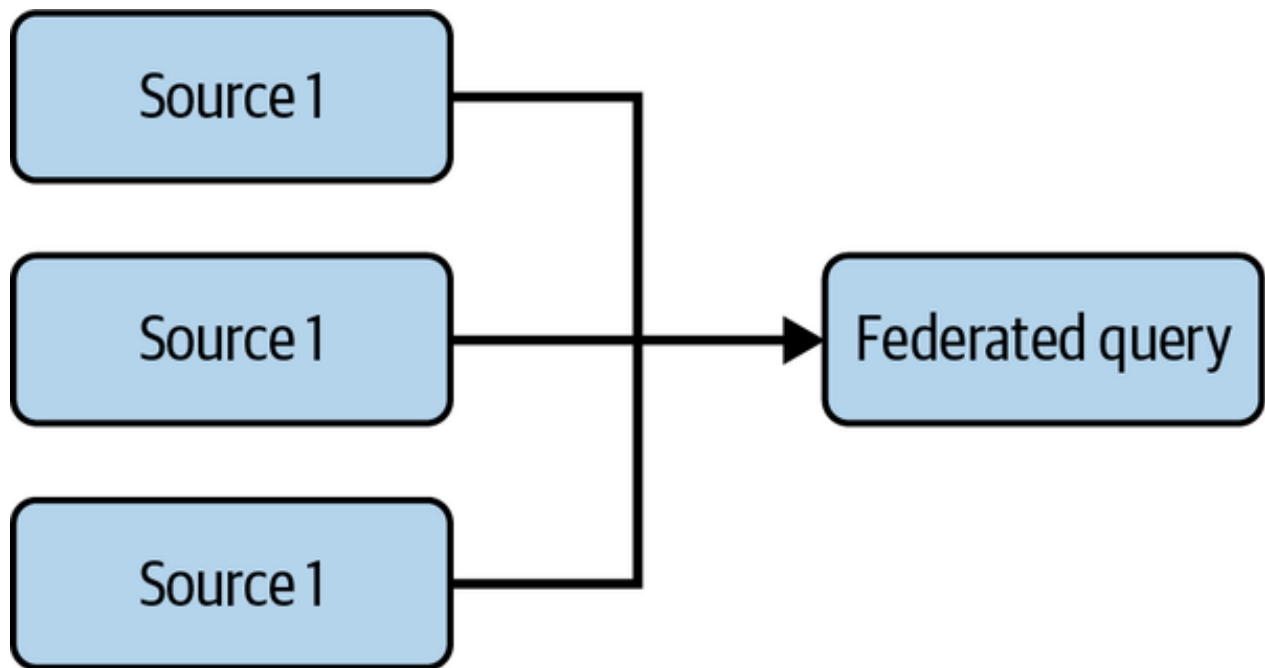


Figure 9-3. A federated query with three data sources In our experience, federated queries are ideally suited when you want flexibility in analyzing data or the source data needs to be tightly controlled. Federation allows ad hoc queries for performing exploratory analysis, blending data from various systems without the complexity of setting up data pipelines or ETL. This will allow you to determine whether the performance of a federated query is sufficient for ongoing purposes or you need to set up ingestion on some or all data sources and centralize the data in an OLAP database or data lake. Federated queries also provide read-only access to source systems, which is great when you don't want to serve files, database access, or data dumps. The end user reads only the version of the data they're supposed to access and nothing more. Query federation is a great option to explore for situations where access and compliance are critical.

Hình 9-3. Một truy vấn liên kết với ba nguồn dữ liệu. Theo kinh nghiệm của chúng tôi, các truy vấn liên kết thích hợp một cách lý tưởng khi bạn muốn sự linh hoạt trong việc phân tích dữ liệu hoặc khi dữ liệu nguồn cần được kiểm soát chặt chẽ. Liên kết cho phép các truy vấn đột xuất để thực hiện phân tích thăm dò, hòa trộn dữ liệu từ nhiều hệ thống mà không có sự phức tạp của việc thiết lập các đường ống dữ liệu hay ETL. Điều này sẽ cho phép bạn xác định hiệu năng của một truy vấn liên kết có đủ cho các mục đích liên tục hay bạn cần thiết lập thu nạp trên một số hoặc tất cả các nguồn dữ liệu và tập trung hóa dữ liệu trong một cơ sở dữ liệu OLAP hoặc hồ dữ liệu. Các truy vấn liên kết cũng cung cấp quyền truy cập chỉ đọc tới các hệ thống nguồn, điều này tuyệt vời khi bạn không muốn phục vụ các tệp, quyền truy cập cơ sở dữ liệu, hay các bản trích xuất dữ liệu (data dump). Người dùng cuối chỉ đọc đúng phiên bản dữ liệu mà họ được phép truy cập và không gì hơn. Liên kết truy vấn là một lựa chọn tuyệt vời để khám phá cho các tình huống mà truy cập và tuân thủ là điều trọng yếu.

Data Sharing — Chia sẻ dữ liệu

Chapter 5 includes an extensive discussion of data sharing. Any data exchange between organizations or units within a larger organization can be viewed as data sharing. Still, we mean specifically sharing through massively **multitenant** storage systems in a cloud environment.

Data sharing generally turns data serving into a security and access control problem. The actual queries are now handled by the data consumers (analysts and data scientists) rather than the engineers sourcing the data. Whether serving data in a data mesh within an organization, providing data to the public, or serving to partner businesses, data sharing is a compelling serving model. Data sharing is increasingly a core feature of major data platforms like Snowflake, Redshift, and BigQuery allowing companies to share data safely and securely with each other.

Chương 5 bao gồm một cuộc thảo luận sâu rộng về chia sẻ dữ liệu (data sharing). Bất kỳ sự trao đổi dữ liệu nào giữa các tổ chức hoặc các đơn vị trong một tổ chức lớn hơn đều có thể được xem là chia sẻ dữ liệu. Tuy nhiên, ý chúng tôi cụ thể là việc chia sẻ thông qua các hệ thống lưu trữ đa thuê bao quy mô lớn (massively multitenant) trong một môi trường đám mây. Chia sẻ dữ liệu nhìn chung biến việc phục vụ dữ liệu thành một bài toán về an ninh và kiểm soát truy cập. Các truy vấn thực tế giờ đây được xử lý bởi các bên tiêu thụ dữ liệu (các nhà phân tích và nhà khoa học dữ liệu) thay vì các kỹ sư cung cấp dữ liệu. Dù là phục vụ dữ liệu trong một data mesh nội bộ một tổ chức, cung cấp dữ liệu ra công chúng, hay phục vụ cho các doanh nghiệp đối tác, chia sẻ dữ liệu là một mô hình phục vụ hấp dẫn. Chia sẻ dữ liệu ngày càng là một tính năng cốt lõi của các nền tảng dữ liệu lớn như Snowflake, Redshift, và BigQuery, cho phép các công ty chia sẻ dữ liệu với nhau một cách an toàn và bảo mật.

Semantic and Metrics Layers — Tầng ngữ nghĩa và tầng chỉ số

When data engineers think about serving, they naturally tend to gravitate toward the data processing and storage technologies—i.e., will you use Spark or a cloud data warehouse? Is your data stored in object storage or cached in a fleet of SSDs? But powerful processing engines that deliver quick query results across vast datasets don't inherently make for quality business analytics. When fed poor-quality data or poor-quality queries, powerful query engines quickly return bad results. Where data quality focuses on characteristics of the data itself and various techniques to filter or improve bad data, query quality is a question of building a query with appropriate logic that returns accurate answers to business questions. Writing high-quality ETL queries and reporting is time-intensive, detailed work. Various tools can help automate this process while facilitating consistency, maintenance, and continuous improvement. Fundamentally, a **metrics layer** is a tool for maintaining and computing business logic. (A 2 semantic layer is extremely similar conceptually, and 3 headless BI is another closely related term.) This layer can live in a BI tool or in software that builds **transformation** queries. Two concrete examples are Looker and Data Build Tool (dbt). For instance, Looker's LookML allows users to define virtual, complex business logic. Reports and dashboards point to specific LookML for computing metrics. Looker allows users to define standard metrics and reference them in many downstream queries; this is meant to solve the traditional problem of repetition and inconsistency in traditional ETL scripts. Looker uses LookML to generate SQL queries, which are pushed down to the database. Results can be persisted in the Looker server or in the database itself for large result sets. dbt allows users to define complex SQL data flows encompassing many queries and standard definitions of business metrics, much like Looker. Unlike Looker, dbt runs exclusively in the transform layer, although this can include pushing queries into views that are computed at query time. Whereas Looker focuses on serving queries and reporting, dbt can serve as a robust **data pipeline orchestration** tool for analytics engineers. We believe that metrics layer tools will grow more popular with wider adoption and more entrants, as well as move upstream toward the application. Metrics layer tools help solve a central question in analytics that has plagued organizations since people have analyzed data: "Are these numbers correct?" Many new entrants are in the space beside the ones we've mentioned.

Khi các kỹ sư dữ liệu nghĩ về việc phục vụ, họ tự nhiên có xu hướng nghiêng về các công nghệ xử lý và lưu trữ dữ liệu — tức là, bạn sẽ dùng Spark hay một kho dữ liệu đám mây? Dữ liệu của bạn được lưu trong lưu trữ đối tượng hay được lưu đệm trong một dàn SSD? Nhưng các công cụ xử lý mạnh mẽ cho ra kết quả truy vấn nhanh trên những tập dữ liệu khổng lồ vốn dĩ không tạo nên phân tích nghiệp vụ chất lượng. Khi được nạp dữ liệu chất lượng kém hoặc các truy vấn chất lượng kém, các công cụ truy vấn mạnh mẽ sẽ nhanh chóng trả về các kết quả tồi. Trong khi chất lượng dữ liệu tập trung vào các đặc tính của bản thân dữ liệu và nhiều kỹ thuật khác nhau để lọc hoặc cải thiện dữ liệu tồi, thì chất lượng truy vấn là vấn đề xây dựng một truy vấn với logic thích đáng trả về các câu trả lời chính xác cho các câu hỏi nghiệp vụ. Viết các truy vấn ETL và báo cáo chất lượng cao là công việc tốn thời gian, chi tiết. Nhiều công cụ khác nhau có thể giúp tự động hóa quy trình này trong khi tạo điều kiện cho tính nhất quán, khả năng bảo trì, và cải tiến liên tục. Về căn bản, một tầng chỉ số (metrics layer) là một công cụ để duy trì và tính toán logic nghiệp vụ. (Một tầng ngữ nghĩa cực kỳ tương tự về mặt khái niệm, và BI không đầu (headless BI) là một thuật ngữ khác liên quan chặt chẽ.) Tầng này có thể tồn tại trong một công cụ BI hoặc trong phần mềm xây dựng các truy vấn biến đổi. Hai ví dụ cụ thể là Looker và Data Build Tool (dbt). Chẳng hạn, LookML của Looker cho phép người dùng định nghĩa logic nghiệp vụ ảo, phức tạp. Các báo cáo và dashboard trở tới LookML cụ thể để tính toán các chỉ số. Looker cho phép người dùng định nghĩa các chỉ số chuẩn và tham chiếu chúng trong nhiều truy vấn hạ nguồn; điều này nhằm giải quyết bài toán truyền thống về sự lặp lại và thiếu nhất quán trong các kịch bản ETL truyền thống. Looker dùng LookML để sinh ra các truy vấn SQL, vốn được đẩy xuống cơ sở dữ liệu. Kết quả có thể được lưu bền vững trong máy chủ Looker hoặc trong chính cơ sở dữ liệu đối với các tập kết quả lớn. dbt cho phép người dùng định nghĩa các luồng dữ liệu SQL phức tạp bao hàm nhiều truy vấn và các định nghĩa chuẩn về các chỉ số nghiệp vụ, rất giống Looker. Không giống Looker, dbt chạy độc quyền trong tầng biến đổi (transform), mặc dù điều này có thể bao gồm việc đẩy các truy vấn vào các view được tính toán tại thời điểm truy vấn. Trong khi Looker tập trung vào việc phục vụ các truy vấn và báo cáo, dbt có thể đóng vai trò một công cụ điều phối (orchestration) đường ống dữ liệu mạnh mẽ cho các kỹ sư phân tích (analytics engineer). Chúng tôi tin rằng các công cụ tầng chỉ số sẽ ngày càng phổ biến hơn với sự đón nhận rộng rãi hơn và nhiều tay chơi tham gia hơn, cũng như dịch chuyển lên thượng nguồn về phía ứng dụng. Các công cụ tầng chỉ số giúp giải quyết một câu hỏi trung tâm trong phân tích vốn ám ảnh các tổ chức kể từ khi con người phân tích dữ liệu: “Những con số này có đúng không?” Nhiều tay chơi mới đang ở trong lĩnh vực này bên cạnh những cái tên chúng tôi đã đề cập.

Serving Data in Notebooks — Phục vụ dữ liệu trong notebook

Data scientists often use notebooks in their day-to-day work. Whether it's exploring data, engineering features, or training a model, the data scientist will likely use a notebook. At this writing, the most popular notebook platform is Jupyter Notebook, along with its next-generation iteration, JupyterLab. Jupyter is open source and can be hosted locally on a laptop, on a server, or through various cloud-managed services. Jupyter stands for Julia, Python, and R—the latter two are popular for data science applications, especially notebooks. Regardless of the language used, the first thing you'll need to consider is how data can be accessed from a notebook. Data scientists will programmatically connect to a data source, such as an API, a database, a data warehouse, or a data lake (Figure 9-4). In a notebook, all connections are created using the appropriate built-in or imported libraries to load a file from a filepath, connect to an API endpoint, or make an ODBC connection to a database. A remote connection may require the correct credentials and privileges to establish a connection. Once

connected, a user may need the correct access to tables (and rows/columns) or files stored in object storage. The data engineer will often assist the data scientist in finding the right data, and then ensure that they have the right permissions to access the rows and columns required.

Các nhà khoa học dữ liệu thường dùng các notebook trong công việc hằng ngày của họ. Dù là thăm dò dữ liệu, thiết kế đặc trưng, hay huấn luyện một mô hình, nhà khoa học dữ liệu nhiều khả năng sẽ dùng một notebook. Tại thời điểm viết cuốn sách này, nền tảng notebook phổ biến nhất là Jupyter Notebook, cùng với phiên bản kế thừa thế hệ tiếp theo của nó, JupyterLab. Jupyter là mã nguồn mở và có thể được host cục bộ trên một laptop, trên một máy chủ, hoặc thông qua nhiều dịch vụ được quản lý trên đám mây. Jupyter là viết tắt của Julia, Python, và R — hai cái sau phổ biến cho các ứng dụng khoa học dữ liệu, đặc biệt là notebook. Bất kể ngôn ngữ được dùng, điều đầu tiên bạn sẽ cần cân nhắc là cách dữ liệu có thể được truy cập từ một notebook. Các nhà khoa học dữ liệu sẽ kết nối theo cách lập trình tới một nguồn dữ liệu, chẳng hạn một API, một cơ sở dữ liệu, một kho dữ liệu, hay một hồ dữ liệu (Hình 9-4). Trong một notebook, tất cả các kết nối được tạo bằng cách dùng các thư viện tích hợp sẵn hoặc được nhập (import) thích hợp để nạp một tệp từ một đường dẫn tệp, kết nối tới một điểm cuối API, hay thực hiện một kết nối ODBC tới một cơ sở dữ liệu. Một kết nối từ xa có thể yêu cầu đúng các thông tin xác thực (credential) và đặc quyền để thiết lập kết nối. Một khi đã kết nối, người dùng có thể cần đúng quyền truy cập tới các bảng (và hàng/cột) hoặc các tệp được lưu trong lưu trữ đối tượng. Kỹ sư dữ liệu sẽ thường hỗ trợ nhà khoa học dữ liệu tìm đúng dữ liệu, rồi đảm bảo rằng họ có đúng các quyền để truy cập các hàng và cột cần thiết.

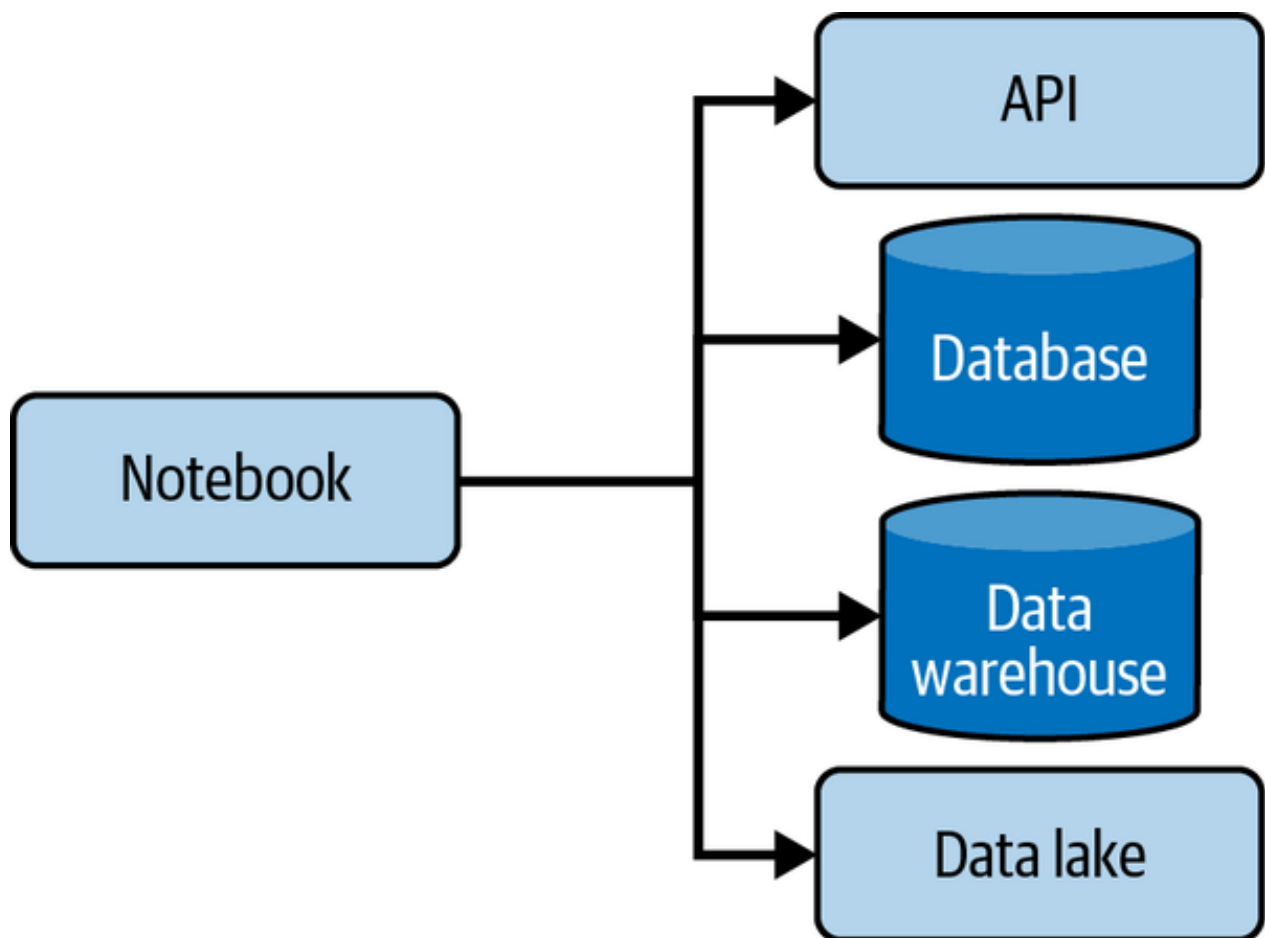


Figure 9-4. A notebook can be served data from many sources, such as object storage, a database, data warehouse, or data lake

Hình 9-4. Một notebook có thể được phục vụ dữ liệu từ nhiều nguồn, chẳng hạn lưu trữ đối tượng, một cơ sở dữ liệu, kho dữ liệu, hoặc hồ dữ liệu.

CREDENTIAL HANDLING — XỬ LÝ THÔNG TIN XÁC THỰC

Incorrectly handled credentials in notebooks and data science code are a major security risk; we constantly see credentials mishandled in this domain. It is common to embed credentials directly in code, where they often leak into version control repos. Credentials are also frequently passed around through messages and email. We encourage data engineers to audit data science security practices and work collaboratively on improvements. Data scientists are highly receptive to these conversations if they are given alternatives. Data engineers should set standards for handling credentials. Credentials should never be embedded in code; ideally, data scientists use credential managers or CLI tools to manage access.

Các thông tin xác thực được xử lý sai cách trong các notebook và mã khoa học dữ liệu là một rủi ro an ninh lớn; chúng tôi liên tục thấy các thông tin xác thực bị xử lý sai trong địa hạt này. Việc nhúng các thông tin xác thực trực tiếp vào mã, nơi chúng thường rò rỉ vào các kho quản lý phiên bản (version control), là phổ biến. Các thông tin xác thực cũng thường được truyền tay nhau qua tin nhắn và email. Chúng tôi khuyến khích các kỹ sư dữ liệu kiểm toán các thực hành an ninh trong khoa học dữ liệu và làm việc cộng tác để cải thiện. Các nhà khoa học dữ liệu rất cởi mở với những cuộc trò chuyện này nếu họ được trao các lựa chọn thay thế. Các kỹ sư dữ liệu nên đặt ra các tiêu chuẩn về việc xử lý thông tin xác thực. Thông tin xác thực không bao giờ nên được nhúng vào mã; lý tưởng nhất, các nhà khoa học dữ liệu dùng các trình quản lý thông tin xác thực (credential manager) hoặc các công cụ CLI để quản lý truy cập.

Let's look at an incredibly common workflow for data scientists: running a local notebook and loading data into a pandas dataframe. Pandas is a prevalent Python library used for data manipulation and analysis and is commonly used to load data (say, a CSV file) into a Jupyter notebook. When pandas loads a dataset, it stores this dataset in memory. What happens when the dataset size exceeds the local machine's available memory? This inevitably happens given the limited memory of laptops and workstations: it stops a data science project dead in its tracks. It's time to consider more scalable options. First, move to a cloud-based notebook where the underlying storage and memory for the notebook can be flexibly scaled. Upon outgrowing this option, look at distributed execution systems; popular Python-based options include Dask, Ray, and Spark. If a full-fledged cloud-managed offering seems appealing, consider setting up a data science workflow using Amazon SageMaker, Google Cloud Vertex AI, or Microsoft Azure Machine Learning. Finally, open source end-to-end ML workflow options such as Kubeflow and MLflow make it easy to scale ML workloads in Kubernetes and Spark, respectively. The point is to get data scientists off their laptops and take advantage of the cloud's power and **scalability**.

Hãy xem một quy trình làm việc cực kỳ phổ biến đối với các nhà khoa học dữ liệu: chạy một notebook cục bộ và nạp dữ liệu vào một dataframe của pandas. Pandas là một thư viện Python thịnh hành dùng cho việc thao tác và phân tích dữ liệu và thường được dùng để nạp dữ liệu (chẳng hạn một tệp CSV) vào một notebook Jupyter. Khi pandas nạp một tập dữ liệu, nó lưu tập dữ liệu này trong bộ nhớ. Điều gì xảy ra khi kích cỡ tập dữ liệu vượt quá bộ nhớ khả dụng của máy cục bộ? Điều này tất yếu xảy ra do bộ nhớ hạn chế của các laptop và máy trạm: nó khiến một dự án khoa học dữ liệu dừng khựng lại. Đã đến lúc cân

nhắc các lựa chọn có thể mở rộng quy mô hơn. Trước hết, hãy chuyển sang một notebook dựa trên đám mây, nơi lưu trữ và bộ nhớ nền tảng cho notebook có thể được mở rộng quy mô một cách linh hoạt. Khi vượt qua khỏi lựa chọn này, hãy xem xét các hệ thống thực thi phân tán; các lựa chọn dựa trên Python phổ biến bao gồm Dask, Ray, và Spark. Nếu một sản phẩm được quản lý trên đám mây hoàn chỉnh có vẻ hấp dẫn, hãy cân nhắc thiết lập một quy trình làm việc khoa học dữ liệu dùng Amazon SageMaker, Google Cloud Vertex AI, hay Microsoft Azure Machine Learning. Cuối cùng, các lựa chọn quy trình làm việc ML đầu cuối mã nguồn mở như Kubeflow và MLflow giúp dễ dàng mở rộng quy mô các khối lượng công việc ML lần lượt trên Kubernetes và Spark. Điểm mấu chốt là đưa các nhà khoa học dữ liệu ra khỏi laptop của họ và tận dụng sức mạnh cùng khả năng mở rộng quy mô của đám mây.

Data engineers and ML engineers play a key role in facilitating the move to scalable cloud infrastructure. The exact division of labor depends a great deal on the details of your organization. They should take the lead in setting up cloud infrastructure, overseeing the management of environments, and training data scientists on cloud-based tools. Cloud environments require significant operational work, such as managing versions and updates, controlling access, and maintaining SLAs. As with other operational work, a significant payoff can result when “data science ops” are done well. Notebooks may even become a part of production data science; notebooks are widely deployed at Netflix. This is an interesting approach with advantages and trade-offs. Productionized notebooks allow data scientists to get their work into production much faster, but they are also inherently a substandard form of production. The alternative is to have ML and data engineers convert notebooks for production use, placing a significant burden on these teams. A hybrid of these approaches may be ideal, with notebooks used for “light” production and a full productionization process for high-value projects.

Các kỹ sư dữ liệu và kỹ sư ML đóng một vai trò then chốt trong việc tạo điều kiện cho sự chuyển dịch sang hạ tầng đám mây có thể mở rộng quy mô. Sự phân chia lao động chính xác phụ thuộc rất nhiều vào các chi tiết của tổ chức bạn. Họ nên đi đầu trong việc thiết lập hạ tầng đám mây, giám sát việc quản lý các môi trường, và đào tạo các nhà khoa học dữ liệu về các công cụ dựa trên đám mây. Các môi trường đám mây đòi hỏi công việc vận hành đáng kể, chẳng hạn quản lý các phiên bản và cập nhật, kiểm soát truy cập, và duy trì các SLA. Cũng như các công việc vận hành khác, một phần thưởng đáng kể có thể đến khi “vận hành khoa học dữ liệu” (data science ops) được làm tốt. Các notebook thậm chí có thể trở thành một phần của khoa học dữ liệu trong sản xuất (production); các notebook được triển khai rộng rãi tại Netflix. Đây là một cách tiếp cận thú vị với các lợi thế và đánh đổi. Các notebook được sản phẩm hóa cho phép các nhà khoa học dữ liệu đưa công việc của họ vào sản xuất nhanh hơn nhiều, nhưng chúng cũng vốn dĩ là một hình thức sản xuất dưới chuẩn. Lựa chọn thay thế là để các kỹ sư ML và kỹ sư dữ liệu chuyển đổi các notebook để dùng trong sản xuất, đặt một gánh nặng đáng kể lên các đội này. Một cách lai giữa các cách tiếp cận này có thể là lý tưởng, với các notebook được dùng cho sản xuất “nhẹ” và một quy trình sản phẩm hóa đầy đủ cho các dự án có giá trị cao.

Reverse ETL — Reverse ETL

Today, reverse ETL is a buzzword that describes serving data by loading it from an OLAP database back into a **source system**. That said, any data engineer who’s worked in the field for more than a few years has probably done some variation of reverse ETL. Reverse ETL grew in popularity in the late 2010s/early 2020s and is increasingly recognized as a formal data engineering responsibility. A data engineer might pull customers and order data from a CRM and store it in a data warehouse. This

data is used to train a lead scoring model, whose results are returned to the data warehouse. Your company's sales team wants access to these scored leads to try to generate more sales. You have a few options to get the results of this lead scoring model into the hands of the sales team. You can put the results in a dashboard for them to view. Or you might email the results to them as an Excel file. The challenge with these approaches is that they are not connected to the CRM, where a salesperson does their work. Why not just put the scored leads back into the CRM? As we mentioned, successful data products reduce friction with the end user. In this case, the end user is the sales team. Using reverse ETL and loading the scored leads back into the CRM is the easiest and best approach for this data product. Reverse ETL takes processed data from the output side of the data engineering lifecycle and feeds it back into source systems (Figure 9-5).

Ngày nay, reverse ETL là một từ thời thượng (buzzword) mô tả việc phục vụ dữ liệu bằng cách nạp nó từ một cơ sở dữ liệu OLAP trở lại một hệ thống nguồn. Dẫu vậy, bất kỳ kỹ sư dữ liệu nào đã làm trong lĩnh vực này hơn vài năm có lẽ đã làm một biến thể nào đó của reverse ETL. Reverse ETL trở nên phổ biến vào cuối thập niên 2010/đầu thập niên 2020 và ngày càng được công nhận như một trách nhiệm chính thức của kỹ thuật dữ liệu. Một kỹ sư dữ liệu có thể kéo dữ liệu khách hàng và đơn hàng từ một CRM và lưu nó trong một kho dữ liệu. Dữ liệu này được dùng để huấn luyện một mô hình chấm điểm khách hàng tiềm năng (lead scoring), kết quả của nó được trả lại kho dữ liệu. Đội bán hàng của công ty bạn muốn có quyền truy cập tới những khách hàng tiềm năng đã được chấm điểm này để cố gắng tạo thêm doanh số. Bạn có một vài lựa chọn để đưa kết quả của mô hình chấm điểm khách hàng tiềm năng này vào tay đội bán hàng. Bạn có thể đặt kết quả vào một dashboard để họ xem. Hoặc bạn có thể gửi email kết quả cho họ dưới dạng một tệp Excel. Thách thức với những cách tiếp cận này là chúng không kết nối tới CRM, nơi một nhân viên bán hàng làm việc. Tại sao không cứ đặt những khách hàng tiềm năng đã chấm điểm trở lại CRM? Như chúng tôi đã đề cập, các sản phẩm dữ liệu thành công giảm bớt ma sát với người dùng cuối. Trong trường hợp này, người dùng cuối là đội bán hàng. Dùng reverse ETL và nạp những khách hàng tiềm năng đã chấm điểm trở lại CRM là cách tiếp cận dễ dàng nhất và tốt nhất cho sản phẩm dữ liệu này. Reverse ETL lấy dữ liệu đã được xử lý từ phía đầu ra của vòng đời kỹ thuật dữ liệu và đưa nó trở lại các hệ thống nguồn (Hình 9-5).

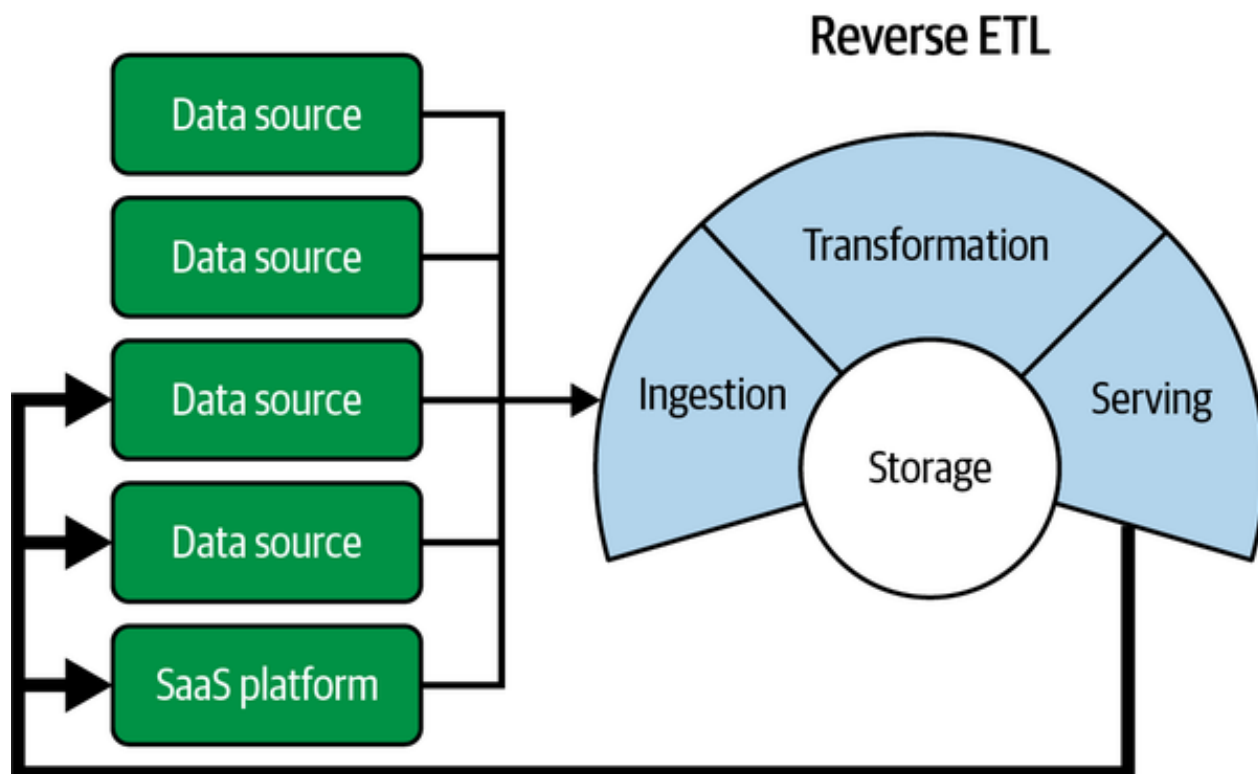


Figure 9-5. Reverse ETL

Hình 9-5. Reverse ETL.

NOTE — GHI CHÚ

Instead of reverse ETL, we, the authors, half-jokingly call it bidirectional load and transform (BLT). The term reverse ETL doesn't quite accurately describe what's happening in this process. Regardless, the term has stuck in the popular imagination and press, so we'll use it throughout the book. More broadly, whether the term reverse ETL sticks around is anyone's guess, but the practice of loading data from OLAP systems back into source systems will remain important.

Thay vì reverse ETL, chúng tôi, các tác giả, nửa đùa nửa thật gọi nó là nạp và biến đổi hai chiều (bidirectional load and transform - BLT). Thuật ngữ reverse ETL không mô tả thật chính xác điều đang diễn ra trong quy trình này. Bất kể vậy, thuật ngữ này đã ăn sâu vào trí tưởng tượng của công chúng và báo chí, nên chúng tôi sẽ dùng nó xuyên suốt cuốn sách. Một cách rộng hơn, liệu thuật ngữ reverse ETL có còn tồn tại hay không thì ai cũng chỉ đoán được, nhưng thực hành nạp dữ liệu từ các hệ thống OLAP trở lại các hệ thống nguồn sẽ vẫn quan trọng.

Ways to Serve Data with Reverse ETL — Các cách phục vụ dữ liệu với reverse ETL

How do you begin serving data with reverse ETL? While you can roll your reverse ETL solution, many off-the-shelf reverse ETL options are available. We suggest using open source, or a commercial **managed service**. That said, the reverse ETL space is changing extremely quickly. No clear winners

have emerged, and many reverse ETL products will be absorbed by major clouds or other data product vendors. Choose carefully. We do have a few words of warning regarding reverse ETL. Reverse ETL inherently creates feedback loops. For example, imagine that we download Google Ads data, use a model to compute new bids, load the bids back into Google Ads, and start the process again. Suppose that because of an error in your bid model, the bids trend ever higher, and your ads get more and more clicks. You can quickly waste massive amounts of money! Be careful, and build in monitoring and guardrails.

Bạn bắt đầu phục vụ dữ liệu với reverse ETL như thế nào? Dù bạn có thể tự cuộn (roll) giải pháp reverse ETL của mình, nhiều lựa chọn reverse ETL có sẵn được cung cấp. Chúng tôi gợi ý dùng mã nguồn mở, hoặc một dịch vụ thương mại được quản lý. Đầu vậy, lĩnh vực reverse ETL đang thay đổi cực kỳ nhanh. Chưa có người chiến thắng rõ ràng nào nổi lên, và nhiều sản phẩm reverse ETL sẽ bị thôn tóm bởi các đám mây lớn hoặc các nhà cung cấp sản phẩm dữ liệu khác. Hãy chọn lựa cẩn thận. Chúng tôi có một vài lời cảnh báo liên quan đến reverse ETL. Reverse ETL vốn dĩ tạo ra các vòng phản hồi. Ví dụ, hãy hình dung rằng chúng ta tải dữ liệu Google Ads, dùng một mô hình để tính các giá thầu mới, nạp các giá thầu trở lại Google Ads, và bắt đầu lại quy trình. Giả sử rằng vì một lỗi trong mô hình giá thầu của bạn, các giá thầu có xu hướng ngày càng cao hơn, và quảng cáo của bạn nhận được ngày càng nhiều lượt nhấp. Bạn có thể nhanh chóng lãng phí những khoản tiền khổng lồ! Hãy cẩn thận, và xây dựng cơ chế giám sát cùng các rào chắn.

Whom You'll Work With — Bạn sẽ làm việc với ai

As we've discussed, in the serving stage, a data engineer will interface with a lot of stakeholders. These include (but aren't limited to) the following:

Như chúng ta đã thảo luận, trong giai đoạn phục vụ, một kỹ sư dữ liệu sẽ giao tiếp với rất nhiều bên liên quan. Những bên này bao gồm (nhưng không giới hạn ở) những đối tượng sau:

Data analysts

Các nhà phân tích dữ liệu

Data scientists

Các nhà khoa học dữ liệu

MLOps/ML engineers The business—nondata or nontechnical stakeholders, managers, and executives

Các kỹ sư MLOps/ML. Doanh nghiệp — các bên liên quan phi dữ liệu hoặc phi kỹ thuật, các quản lý, và các lãnh đạo

As a reminder, the data engineer operates in a support role for these stakeholders and is not necessarily responsible for the end uses of data. For example, a data engineer supplies the data for a report that analysts interpret, but the data engineer isn't responsible for these interpretations. Instead, the data engineer is responsible for producing the highest-quality data products possible. A data engineer should be aware of feedback loops between the data engineering lifecycle and the broader use of data once it's in the hands of stakeholders. Data is rarely static, and the outside world will influence the data that is ingested and served and reingested and re-served. A big consideration for data engineers in the serving stage of the lifecycle is the separation of duties and concerns. If you're at an early-stage company, the data engineer may also be an ML engineer or data scientist; this is not sustainable. As the company grows, you need to establish a clear division of duties with

other data team members. Adopting a data mesh dramatically reorganizes team responsibilities, and every domain team takes on aspects of serving. For a data mesh to be successful, each team must work effectively on its data serving responsibilities, and teams must also effectively collaborate to ensure organizational success.

Như một lời nhắc, kỹ sư dữ liệu hoạt động ở một vai trò hỗ trợ cho các bên liên quan này và không nhất thiết chịu trách nhiệm về các mục đích sử dụng cuối cùng của dữ liệu. Ví dụ, một kỹ sư dữ liệu cung cấp dữ liệu cho một báo cáo mà các nhà phân tích diễn giải, nhưng kỹ sư dữ liệu không chịu trách nhiệm về những diễn giải này. Thay vào đó, kỹ sư dữ liệu chịu trách nhiệm tạo ra các sản phẩm dữ liệu chất lượng cao nhất có thể. Một kỹ sư dữ liệu nên ý thức về các vòng phản hồi giữa vòng đời kỹ thuật dữ liệu và việc sử dụng dữ liệu rộng hơn một khi nó đã ở trong tay các bên liên quan. Dữ liệu hiếm khi tĩnh, và thế giới bên ngoài sẽ ảnh hưởng đến dữ liệu được thu nạp và phục vụ rồi lại được thu nạp lại và phục vụ lại. Một điều cần cân nhắc lớn đối với các kỹ sư dữ liệu trong giai đoạn phục vụ của vòng đời là sự tách bạch các nhiệm vụ và các mối quan tâm (separation of duties and concerns). Nếu bạn ở một công ty giai đoạn đầu, kỹ sư dữ liệu cũng có thể là một kỹ sư ML hay nhà khoa học dữ liệu; điều này không bền vững. Khi công ty phát triển, bạn cần thiết lập một sự phân chia nhiệm vụ rõ ràng với các thành viên khác của đội dữ liệu. Việc áp dụng một data mesh tổ chức lại đáng kể các trách nhiệm của đội, và mỗi đội lĩnh vực đảm nhận các khía cạnh của việc phục vụ. Để một data mesh thành công, mỗi đội phải làm việc hiệu quả với các trách nhiệm phục vụ dữ liệu của mình, và các đội cũng phải cộng tác hiệu quả để đảm bảo thành công của tổ chức.

Undercurrents — Các dòng chảy ngầm

The undercurrents come to finality with serving. Remember that the data engineering lifecycle is just that—a lifecycle. What goes around comes around. We see many instances where serving data highlights something missed earlier in the lifecycle. Always be on the lookout for how the undercurrents can help you spot ways to improve data products. We're fond of saying, "Data is a silent killer," and the undercurrents come to a head in the serving stage. Serving is your final chance to make sure your data is in great shape before it gets into the hands of end users.

Các dòng chảy ngầm đi đến hồi kết với việc phục vụ. Hãy nhớ rằng vòng đời kỹ thuật dữ liệu đúng là như vậy — một vòng đời. Gieo gì gặt nấy. Chúng tôi thấy nhiều trường hợp việc phục vụ dữ liệu làm nổi bật một thứ gì đó bị bỏ sót sớm hơn trong vòng đời. Hãy luôn để mắt xem các dòng chảy ngầm có thể giúp bạn phát hiện những cách cải thiện các sản phẩm dữ liệu như thế nào. Chúng tôi thích nói rằng, "Dữ liệu là một kẻ sát nhân thầm lặng," và các dòng chảy ngầm lên đến đỉnh điểm trong giai đoạn phục vụ. Phục vụ là cơ hội cuối cùng của bạn để đảm bảo dữ liệu của mình ở trạng thái tuyệt vời trước khi nó đến tay các người dùng cuối.

Security — An ninh

The same security principles apply whether sharing data with people or systems. We often see data shared indiscriminately, with little to no access controls or thought as to what the data will be used for. This is a huge mistake that can have catastrophic results, such as a data breach and the resulting fines, bad press, and lost jobs. Take security seriously, especially in this stage of the lifecycle. Of all the lifecycle stages, serving presents the largest security surface. As always, exercise the **principle of least privilege** both for people and systems, and provide only the access required for the purpose

at hand, and the job to be done. What data does an executive need versus an analyst or data scientist? What about an ML pipeline or reverse ETL process? These users and destinations all have different data needs, and access should be provided accordingly. Avoid giving carte blanche permissions to everyone and everything. Serving data is often read-only unless a person or process needs to update data in the system from which it is queried. People should be given read-only access to specific databases and datasets unless their role requires something more advanced like write or update access. This can be accomplished by combining groups of users with certain IAM roles (i.e., analysts group, data scientist group) or custom IAM roles if this makes sense. For systems, provide service accounts and roles in a similar fashion. For both users and systems, narrow access to a dataset's fields, rows, columns, and cells if this is warranted. Access controls should be as fine-grained as possible and revoked when access is no longer required. Access controls are critical when serving data in a multitenant environment. Make sure users can access only their data and nothing more. A good approach is to mediate access through filtered views, thus alleviating the security risks inherent in sharing access to a common table. Another suggestion is to use data sharing in your workflows, which allows for read-only granular controls between you and people consuming your data. Check how often data products are used, and whether it makes sense to stop sharing certain data products. It's extremely common for an executive to urgently request an analyst to create a report, only to have this report very quickly go unused. If data products aren't used, ask the users if they're still needed. If not, kill off the data product. This means one less security vulnerability floating around. Finally, you should view access control and security not as impediments to serving but as key enablers. We're aware of many instances where complex, advanced data systems were built, potentially having a significant impact on a company. Because security was not implemented correctly, few people were allowed to access the data, so it languished. Fine-grained, robust access control means that more interesting data analytics and ML can be done while still protecting the business and its customers.

Các nguyên tắc an ninh giống nhau được áp dụng dù bạn chia sẻ dữ liệu với con người hay các hệ thống. Chúng tôi thường thấy dữ liệu được chia sẻ một cách bừa bãi, với ít hoặc không có kiểm soát truy cập hay sự suy xét về việc dữ liệu sẽ được dùng để làm gì. Đây là một sai lầm khổng lồ có thể có những hậu quả thảm khốc, chẳng hạn một vụ rò rỉ dữ liệu cùng các khoản phạt, tin xấu trên báo chí, và mất việc làm. Hãy xem trọng an ninh, đặc biệt trong giai đoạn này của vòng đời. Trong tất cả các giai đoạn của vòng đời, phục vụ trình ra bề mặt an ninh lớn nhất. Như mọi khi, hãy thực thi nguyên tắc đặc quyền tối thiểu cho cả con người lẫn các hệ thống, và chỉ cung cấp quyền truy cập cần thiết cho mục đích trước mắt, và công việc cần hoàn thành. Một lãnh đạo cần dữ liệu gì so với một nhà phân tích hay nhà khoa học dữ liệu? Còn một đường ống ML hay một quy trình reverse ETL thì sao? Những người dùng và đích đến này đều có các nhu cầu dữ liệu khác nhau, và quyền truy cập nên được cung cấp tương ứng. Hãy tránh trao toàn quyền cho mọi người và mọi thứ. Phục vụ dữ liệu thường là chỉ đọc trừ khi một người hay một quy trình cần cập nhật dữ liệu trong hệ thống mà nó được truy vấn từ đó. Mọi người nên được trao quyền truy cập chỉ đọc tới các cơ sở dữ liệu và tập dữ liệu cụ thể trừ khi vai trò của họ yêu cầu thứ gì đó nâng cao hơn như quyền ghi hoặc quyền cập nhật. Điều này có thể được thực hiện bằng cách kết hợp các nhóm người dùng với một số vai trò IAM nhất định (tức là nhóm nhà phân tích, nhóm nhà khoa học dữ liệu) hoặc các vai trò IAM tùy chỉnh nếu điều này hợp lý. Đối với các hệ thống, hãy cung cấp các tài khoản dịch vụ và các vai trò theo cách tương tự. Đối với cả người dùng và hệ thống, hãy thu hẹp quyền truy cập tới các trường, hàng, cột, và ô của một tập dữ liệu nếu điều này là chính đáng. Các kiểm soát truy cập nên ở mức chi tiết nhất có thể và bị thu hồi khi quyền truy cập không còn cần thiết. Các kiểm soát truy cập là trọng yếu khi phục vụ dữ liệu trong một môi trường đa thuê bao. Hãy chắc chắn rằng người dùng chỉ có thể truy cập dữ liệu của họ và không gì hơn. Một cách tiếp cận tốt là trung gian hóa quyền truy cập thông qua các view đã được lọc, qua đó giảm nhẹ các rủi ro an ninh vốn có trong việc chia sẻ quyền truy cập tới một bảng chung. Một gợi ý

khác là dùng chia sẻ dữ liệu trong các quy trình làm việc của bạn, cho phép các kiểm soát chỉ đọc ở mức chi tiết giữa bạn và những người tiêu thụ dữ liệu của bạn. Hãy kiểm tra xem các sản phẩm dữ liệu được dùng thường xuyên đến mức nào, và liệu có hợp lý không khi ngừng chia sẻ một số sản phẩm dữ liệu nhất định. Việc một lãnh đạo khẩn thiết yêu cầu một nhà phân tích tạo một báo cáo, để rồi báo cáo này rất nhanh chóng không được dùng đến, là cực kỳ phổ biến. Nếu các sản phẩm dữ liệu không được dùng, hãy hỏi người dùng xem chúng có còn cần thiết không. Nếu không, hãy khai tử sản phẩm dữ liệu đó. Điều này nghĩa là bớt đi một lỗ hổng an ninh trôi nổi quanh đó. Cuối cùng, bạn nên xem kiểm soát truy cập và an ninh không phải như những trở ngại cho việc phục vụ mà như những yếu tố hỗ trợ then chốt. Chúng tôi biết nhiều trường hợp các hệ thống dữ liệu phức tạp, tiên tiến đã được xây dựng, có khả năng tác động đáng kể đến một công ty. Vì an ninh không được triển khai đúng cách, ít người được phép truy cập dữ liệu, nên nó mai một dần. Kiểm soát truy cập ở mức chi tiết, vững chắc nghĩa là có thể thực hiện được nhiều phân tích dữ liệu và ML thú vị hơn trong khi vẫn bảo vệ được doanh nghiệp và các khách hàng của nó.

Data Management — Quản lý dữ liệu

You’ve been incorporating data management along the data engineering lifecycle, and the impact of your efforts will soon become apparent as people use your data products. At the serving stage, you’re mainly concerned with ensuring that people can access high-quality and trustworthy data. As we mentioned at the beginning of this chapter, trust is perhaps the most critical variable in data serving. If people trust their data, they will use it; untrusted data will go unused. Be sure to make data trust and data improvement an active process by providing feedback loops. As users interact with data, they can report problems and request improvements. Actively communicate back to your users as changes are made. What data do people need to do their jobs? Especially with regulatory and compliance concerns weighing on data teams, giving people access to the raw data—even with limited fields and rows—poses a problem of tracing data back to an entity, such as a person or a group of people. Thankfully, advancements in data obfuscation allow you to serve synthetic, scrambled, or anonymized data to end users. These “fake” datasets should sufficiently allow an analyst or data scientist to get the necessary signal from the data, but in a way that makes identifying protected information difficult. Though this isn’t a perfect process—with enough effort, many datasets can be de-anonymized or reverse-engineered—it at least reduces the risk of data leakage. Also, incorporate semantic and metrics layers into your serving layer, alongside rigorous data modeling that properly expresses business logic and definitions. This provides a single source of truth, whether for analytics, ML, reverse ETL, or other serving uses.

Bạn đã tích hợp việc quản lý dữ liệu (data management) suốt vòng đời kỹ thuật dữ liệu, và tác động của các nỗ lực của bạn sẽ sớm trở nên rõ ràng khi mọi người dùng các sản phẩm dữ liệu của bạn. Ở giai đoạn phục vụ, bạn chủ yếu quan tâm đến việc đảm bảo rằng mọi người có thể truy cập dữ liệu chất lượng cao và đáng tin cậy. Như chúng tôi đã đề cập ở đầu chương này, niềm tin có lẽ là biến số trọng yếu nhất trong việc phục vụ dữ liệu. Nếu mọi người tin tưởng dữ liệu của họ, họ sẽ dùng nó; dữ liệu không được tin tưởng sẽ không được dùng. Hãy chắc chắn biến niềm tin vào dữ liệu và việc cải thiện dữ liệu thành một quy trình chủ động bằng cách cung cấp các vòng phản hồi. Khi người dùng tương tác với dữ liệu, họ có thể báo cáo các vấn đề và yêu cầu cải thiện. Hãy chủ động giao tiếp lại với người dùng của bạn khi các thay đổi được thực hiện. Mọi người cần dữ liệu gì để làm công việc của họ? Đặc biệt với các mối lo ngại về quy định và tuân thủ đề nặng lên các đội dữ liệu, việc trao cho mọi người quyền truy cập tới dữ liệu thô — ngay cả với các trường và hàng hạn chế — đặt ra một vấn đề về việc truy ngược dữ liệu về một thực thể, chẳng hạn một người hay một nhóm người. May mắn thay, các tiến bộ trong việc làm mờ dữ liệu

(data obfuscation) cho phép bạn phục vụ dữ liệu tổng hợp (synthetic), bị xáo trộn, hoặc đã ẩn danh tới các người dùng cuối. Những tập dữ liệu “giả” này nên đủ để cho phép một nhà phân tích hay nhà khoa học dữ liệu lấy được tín hiệu cần thiết từ dữ liệu, nhưng theo một cách khiến việc nhận diện thông tin được bảo vệ trở nên khó khăn. Mặc dù đây không phải một quy trình hoàn hảo — với đủ nỗ lực, nhiều tập dữ liệu có thể bị giải ẩn danh hoặc bị dịch ngược (reverse-engineer) — nó ít nhất giảm thiểu rủi ro rò rỉ dữ liệu. Ngoài ra, hãy tích hợp các tầng ngữ nghĩa và tầng chỉ số vào tầng phục vụ của bạn, bên cạnh việc mô hình hóa dữ liệu một cách nghiêm ngặt nhằm biểu đạt đúng logic và các định nghĩa nghiệp vụ. Điều này cung cấp một nguồn sự thật duy nhất (single source of truth), dù là cho phân tích, ML, reverse ETL, hay các mục đích phục vụ khác.

DataOps — DataOps

The steps you take in data management—data quality, governance, and security—are monitored in **DataOps**. Essentially, DataOps operationalizes data management. The following are some things to monitor:

Các bước bạn thực hiện trong quản lý dữ liệu — chất lượng dữ liệu, quản trị, và an ninh — được giám sát trong DataOps. Về cơ bản, DataOps vận hành hóa việc quản lý dữ liệu. Sau đây là một số thứ cần giám sát:

Data health and data downtime

Tình trạng sức khỏe dữ liệu và thời gian ngừng hoạt động của dữ liệu (data downtime)

Latency of systems serving data—dashboards, databases, etc.

Độ trễ của các hệ thống phục vụ dữ liệu — dashboard, cơ sở dữ liệu, v.v.

Data quality

Chất lượng dữ liệu

Data and system security and access

An ninh và truy cập dữ liệu cùng hệ thống

Data and model versions being served

Các phiên bản dữ liệu và mô hình đang được phục vụ

Uptime to achieve an SLO

Thời gian hoạt động để đạt được một SLO

A variety of new tools have sprung up to address various monitoring aspects. For example, many popular data observability tools aim to minimize data downtime and maximize data quality. Observability tools may cross over from data to ML, supporting monitoring of models and model performance. More conventional DevOps monitoring is also critical to DataOps—e.g., you need to monitor whether connections are stable among storage, transformation, and serving.

Nhiều công cụ mới đã nảy nở để xử lý các khía cạnh giám sát khác nhau. Ví dụ, nhiều công cụ khả năng quan sát dữ liệu phổ biến nhằm giảm thiểu thời gian ngừng hoạt động của dữ liệu và tối đa hóa chất lượng dữ liệu. Các công cụ khả năng quan sát có thể vượt từ dữ liệu sang ML, hỗ trợ việc giám sát các mô hình và hiệu năng mô hình. Việc giám sát DevOps thông thường hơn cũng trọng yếu đối với DataOps — ví dụ, bạn cần giám sát xem các kết nối giữa lưu trữ, biến đổi, và phục vụ có ổn định không.

As in every stage of the data engineering lifecycle, version-control code and operationalize deployment. This applies to analytical code, data logic code, ML scripts, and orchestration jobs. Use multiple stages of deployment (dev, test, prod) for reports and models.

Cũng như trong mọi giai đoạn của vòng đời kỹ thuật dữ liệu, hãy quản lý phiên bản mã và vận hành hóa việc triển khai (deployment). Điều này áp dụng cho mã phân tích, mã logic dữ liệu, các kịch bản ML, và các tác vụ điều phối. Hãy dùng nhiều giai đoạn triển khai (dev, test, prod) cho các báo cáo và mô hình.

Data Architecture — Kiến trúc dữ liệu

Serving data should have the same architectural considerations as other data engineering lifecycle stages. At the serving stage, feedback loops must be fast and tight. Users should be able to access the data they need as quickly as possible when they need it. Data scientists are notorious for doing most development on their local machines. As discussed earlier, encourage them to migrate these workflows to common systems in a cloud environment, where data teams can collaborate in dev, test, and production environments and create proper production architectures. Facilitate your analysts and data scientists by supporting tools for publishing data insights with little encumbrance.

Phục vụ dữ liệu nên có những điều cần cân nhắc về kiến trúc giống như các giai đoạn khác của vòng đời kỹ thuật dữ liệu. Ở giai đoạn phục vụ, các vòng phản hồi phải nhanh và chặt chẽ. Người dùng nên có thể truy cập dữ liệu họ cần càng nhanh càng tốt khi họ cần nó. Các nhà khoa học dữ liệu nổi tiếng về việc làm hầu hết công việc phát triển trên các máy cục bộ của họ. Như đã thảo luận trước đó, hãy khuyến khích họ di chuyển các quy trình làm việc này sang các hệ thống chung trong một môi trường đám mây, nơi các đội dữ liệu có thể cộng tác trong các môi trường dev, test, và production và tạo ra các kiến trúc sản xuất đúng đắn. Hãy tạo điều kiện cho các nhà phân tích và nhà khoa học dữ liệu của bạn bằng cách hỗ trợ các công cụ để công bố các thông tin chuyên sâu về dữ liệu với ít trở ngại nhất.

Orchestration — Điều phối

Data serving is the last stage of the data engineering lifecycle. Because serving is downstream of so many processes, it's an area of extremely complex overlap. Orchestration is not simply a way of organizing and automating complex work but a means of coordinating data flow across teams so that data is made available to consumers at the promised time. Ownership of orchestration is a key organizational decision. Will orchestration be centralized or decentralized? A decentralized approach allows small teams to manage their data flows, but it can increase the burden of cross-team coordination. Instead of simply managing flows within a single system, directly triggering the completion of DAGs or tasks belonging to other teams, teams must pass messages or queries between systems. A centralized approach means that work is easier to coordinate, but significant gatekeeping must also exist to protect a single production asset.

Phục vụ dữ liệu là giai đoạn cuối cùng của vòng đời kỹ thuật dữ liệu. Vì phục vụ ở hạ nguồn của rất nhiều quy trình, nó là một mảng có sự chồng lấn cực kỳ phức tạp. Điều phối (orchestration) không chỉ đơn thuần là một cách tổ chức và tự động hóa công việc phức tạp mà là một phương tiện để phối hợp luồng dữ liệu xuyên các đội sao cho dữ liệu được làm sẵn có cho các bên tiêu thụ vào đúng thời điểm đã hứa. Quyền sở hữu việc điều phối là một quyết định tổ chức then chốt. Liệu việc điều phối sẽ được tập trung hay phi tập trung? Một cách tiếp cận phi tập trung cho phép các đội nhỏ quản lý các luồng dữ liệu của họ, nhưng nó có thể làm tăng gánh nặng phối hợp xuyên các đội. Thay vì chỉ đơn thuần

quản lý các luồng trong một hệ thống duy nhất, trực tiếp kích hoạt việc hoàn thành các DAG hoặc các tác vụ thuộc về các đội khác, các đội phải truyền các tin nhắn hoặc truy vấn giữa các hệ thống. Một cách tiếp cận tập trung nghĩa là công việc dễ phối hợp hơn, nhưng việc canh giữ cửa (gatekeeping) đáng kể cũng phải tồn tại để bảo vệ một tài sản sản xuất duy nhất.

For example, a poorly written **DAG** can bring Airflow to a halt. The centralized approach would mean bringing down data processes and serving across the whole organization. Centralized orchestration management requires high standards, automated testing of DAGs, and gatekeeping. If orchestration is centralized, who will own it? When a company has a DataOps team, orchestration usually lands here. Often, a team involved in serving is a natural fit because it has a fairly holistic view of all data engineering lifecycle stages. This could be the DBAs, analytics engineers, data engineers, or ML engineers. ML engineers coordinate complex model-training processes but may or may not want to add the operational complexity of managing orchestration to an already crowded docket of responsibilities.

Ví dụ, một DAG được viết kém có thể khiến Airflow dừng khựng. Cách tiếp cận tập trung sẽ có nghĩa là làm sập các quy trình dữ liệu và phục vụ trên khắp toàn bộ tổ chức. Việc quản lý điều phối tập trung đòi hỏi các tiêu chuẩn cao, kiểm thử tự động các DAG, và việc canh giữ cửa. Nếu việc điều phối được tập trung, ai sẽ sở hữu nó? Khi một công ty có một đội DataOps, việc điều phối thường rơi vào đây. Thông thường, một đội tham gia vào việc phục vụ là một sự phù hợp tự nhiên vì nó có một góc nhìn khá toàn diện về tất cả các giai đoạn của vòng đời kỹ thuật dữ liệu. Đây có thể là các DBA, các kỹ sư phân tích, các kỹ sư dữ liệu, hay các kỹ sư ML. Các kỹ sư ML phối hợp các quy trình huấn luyện mô hình phức tạp nhưng có thể muốn hoặc không muốn thêm sự phức tạp vận hành của việc quản lý điều phối vào một danh sách trách nhiệm vốn đã chật chội.

Software Engineering — Kỹ thuật phần mềm

Compared to a few years ago, serving data has become simpler. The need to write code has been drastically simplified. Data has also become more code-first, with the proliferation of open source frameworks focused on simplifying the serving of data. Many ways exist to serve data to end users, and a data engineer's focus should be on knowing how these systems work and how data is delivered. Despite the simplicity of serving data, if code is involved, a data engineer should still understand how the main serving interfaces work. For example, a data engineer may need to translate the code a data scientist is running locally on a notebook and convert it into a report or a basic ML model to operate. Another area where data engineers will be useful is understanding the impact of how code and queries will perform against the storage systems. Analysts can generate SQL in various programmatic ways, including LookML, Jinja via dbt, various object-relational mapping (ORM) tools, and metrics layers. When these programmatic layers compile to SQL, how will this SQL perform? A data engineer can suggest optimizations where the SQL code might not perform as well as handwritten SQL.

So với vài năm trước, việc phục vụ dữ liệu đã trở nên đơn giản hơn. Nhu cầu viết mã đã được đơn giản hóa một cách đáng kể. Dữ liệu cũng đã trở nên thiên về mã hơn (code-first), với sự sinh sôi của các framework mã nguồn mở tập trung vào việc đơn giản hóa việc phục vụ dữ liệu. Nhiều cách tồn tại để phục vụ dữ liệu tới các người dùng cuối, và trọng tâm của một kỹ sư dữ liệu nên là biết các hệ thống này hoạt động như thế nào và dữ liệu được chuyển giao ra sao. Bất chấp sự đơn giản của việc phục vụ dữ liệu, nếu có mã liên quan, một kỹ sư dữ liệu vẫn nên hiểu các giao diện phục vụ chính hoạt động như thế nào. Ví dụ, một kỹ sư dữ liệu có thể cần dịch mã mà một nhà khoa học dữ liệu đang chạy cục bộ trên

một notebook và chuyển đổi nó thành một báo cáo hoặc một mô hình ML cơ bản để vận hành. Một mảng khác nơi các kỹ sư dữ liệu sẽ hữu ích là việc hiểu tác động của cách mã và truy vấn sẽ hoạt động như thế nào trên các hệ thống lưu trữ. Các nhà phân tích có thể sinh ra SQL theo nhiều cách lập trình khác nhau, bao gồm LookML, Jinja qua dbt, nhiều công cụ ánh xạ quan hệ - đối tượng (ORM), và các tầng chỉ số. Khi các tầng lập trình này biên dịch thành SQL, SQL này sẽ hoạt động ra sao? Một kỹ sư dữ liệu có thể gợi ý các tối ưu hóa ở những chỗ mà mã SQL có thể không hoạt động tốt bằng SQL viết tay.

The rise of analytics and ML IaC means the role of writing code is moving toward building the systems that support data scientists and analysts. Data engineers might be responsible for setting up the CI/CD pipelines and building processes for their data team. They would also do well to train and support their data team in using the Data/MLOps infrastructure they've built so that these data teams can be as self-sufficient as possible. For embedded analytics, data engineers may need to work with application developers to ensure that queries are returned quickly and cost-effectively. The application developer will control the frontend code that users deal with. The data engineer is there to ensure that developers receive the correct payloads as they're requested.

Sự trỗi dậy của IaC cho phân tích và ML có nghĩa là vai trò của việc viết mã đang chuyển dịch về phía xây dựng các hệ thống hỗ trợ các nhà khoa học dữ liệu và các nhà phân tích. Các kỹ sư dữ liệu có thể chịu trách nhiệm thiết lập các đường ống CI/CD và xây dựng các quy trình cho đội dữ liệu của họ. Họ cũng nên đào tạo và hỗ trợ đội dữ liệu của mình trong việc dùng hạ tầng Data/MLOps mà họ đã xây dựng sao cho các đội dữ liệu này có thể tự chủ nhất có thể. Đối với phân tích nhúng, các kỹ sư dữ liệu có thể cần làm việc với các nhà phát triển ứng dụng để đảm bảo rằng các truy vấn được trả về nhanh chóng và tiết kiệm chi phí. Nhà phát triển ứng dụng sẽ kiểm soát mã frontend mà người dùng tương tác. Kỹ sư dữ liệu ở đó để đảm bảo rằng các nhà phát triển nhận được đúng các tải trọng (payload) khi chúng được yêu cầu.

Conclusion — Kết luận

The data engineering lifecycle has a logical ending at the serving stage. As with all lifecycles, a feedback loop occurs (Figure 9-6). You should view the serving stage as a chance to learn what's working and what can be improved. Listen to your stakeholders. If they bring up issues—and they inevitably will—try not to take offense. Instead, use this as an opportunity to improve what you've built.

Vòng đời kỹ thuật dữ liệu có một kết thúc hợp lý ở giai đoạn phục vụ. Như với tất cả các vòng đời, một vòng phản hồi xảy ra (Hình 9-6). Bạn nên xem giai đoạn phục vụ như một cơ hội để học những gì đang hiệu quả và những gì có thể được cải thiện. Hãy lắng nghe các bên liên quan của bạn. Nếu họ nêu lên các vấn đề — và họ chắc chắn sẽ làm vậy — hãy cố đừng cảm thấy bị xúc phạm. Thay vào đó, hãy dùng điều này như một cơ hội để cải thiện những gì bạn đã xây dựng.

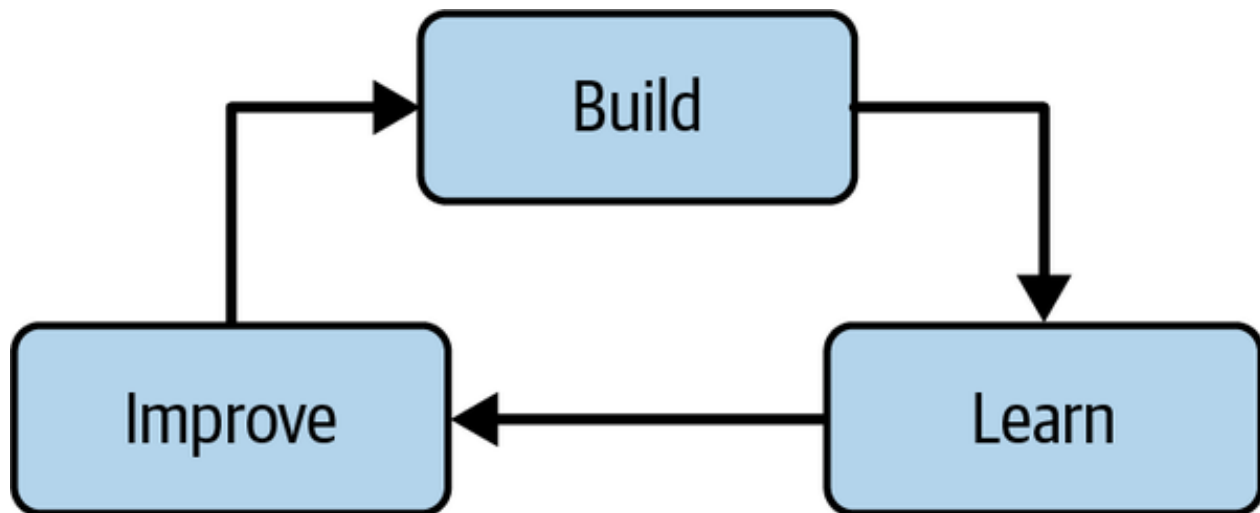


Figure 9-6. Build, learn, improve A good data engineer is always open to new feedback and constantly finds ways to improve their craft. Now that we’ve taken a journey through the data engineering lifecycle, you know how to design, architect, build, maintain, and improve your data engineering systems and products. Let’s turn our attention to Part III of the book, where we’ll cover some aspects of data engineering we’re constantly asked about and, frankly, deserve more attention.

Hình 9-6. Xây dựng, học hỏi, cải thiện. Một kỹ sư dữ liệu giỏi luôn cởi mở với phản hồi mới và không ngừng tìm cách cải thiện nghề của mình. Giờ đây khi chúng ta đã thực hiện một hành trình xuyên qua vòng đời kỹ thuật dữ liệu, bạn biết cách thiết kế, kiến trúc, xây dựng, bảo trì, và cải thiện các hệ thống và sản phẩm kỹ thuật dữ liệu của mình. Hãy hướng sự chú ý của chúng ta sang Phần III của cuốn sách, nơi chúng ta sẽ đề cập một số khía cạnh của kỹ thuật dữ liệu mà chúng tôi liên tục được hỏi và, thật lòng mà nói, xứng đáng được chú ý nhiều hơn.

Additional Resources

“Designing Data Products” by Seth O’Regan

“Data Jujitsu: The Art of Turning Data into Product” by DJ Patil

“What Is User-Facing Analytics?” by Chinmon Soman

“Data as a Product vs. Data Products: What Are the Differences?” by Xavier Gumara Rigol

“How to Build Great Data Products” by Emily Glassberg Sands

“The Evolution of Data Products” and “What Is Data Science” by Mike Loukides

“Know Your Customers’ ‘Jobs to Be Done’” by Clayton M. Christensen et al.

“Data Mesh Principles and Logical Architecture” by Martin Fowler

“Understanding the Superset Semantic Layer” by Srini Kadamati

“The Future of BI Is Headless” by ZD

“How to Structure a Data Analytics Team” by Niall Napier

“What Is Operational Analytics (and How Is It Changing How We Work with Data)?” by Sylvain Giuliani “Fundamentals of Self-Service Machine Learning” by Paramita (Guha) Ghosh

“What Do Modern Self-Service BI and Data Analytics Really Mean?” by Harry Dix

“Self-Service Analytics” in the Gartner Glossary

“The Missing Piece of the Modern Data Stack” and “Why Is Self-Serve Still a Problem?” by Benn Stancil

Forrester’s “Self-Service Business Intelligence: Dissolving the Barriers to Creative Decision-Support Solutions” blog article

Data Mesh by Zhamak Dehghani (O’Reilly)

1 “Know Your Customers’ ‘Jobs to Be Done’” by Clayton M. Christensen et al., Harvard Business Review, September 2016, <https://oreil.ly/3uU4j> . 2 Benn Stancil, “The Missing Piece of the Modern Data Stack,” benn.substack, April 22, 2021, <https://oreil.ly/wQyPb> . 3 Sriniketh Kadamati, “Understanding the Superset Semantic Layer,” Preset blog, December 21, 2021, <https://oreil.ly/6smWC> .

Part III. Security, Privacy, and the Future of Data Engineering — Phần III. An ninh, Quyền riêng tư và Tương lai của Kỹ thuật Dữ liệu

Chapter 10. Security and Privacy —

Chương 10. An ninh và Quyền riêng tư

Security is vital to the practice of **data engineering**. This should be blindingly obvious, but we're constantly amazed at how often data engineers view security as an afterthought. We believe that security is the first thing a **data engineer** needs to think about in every aspect of their job and every stage of the **data engineering lifecycle**. You deal with sensitive data, information, and access daily. Your organization, customers, and business partners expect these valuable assets to be handled with the utmost care and concern. One security breach or a data leak can leave your business dead in the water; your career and reputation are ruined if it's your fault. Security is a key ingredient for **privacy**. Privacy has long been critical to trust in the corporate information technology space; engineers directly or indirectly handle data related to people's private lives. This includes financial information, data on private communications (emails, texts, phone calls), medical history, educational records, and job history. A company that leaked this information or misused it could find itself a pariah when the breach came to light. Increasingly, privacy is a matter of significant legal importance. For example, the Family Educational Rights and Privacy Act (FERPA) went into effect in the US in the 1970s; the Health Insurance Portability and Accountability Act (HIPAA) followed in the 1990s; GDPR was passed in Europe in the mid-2010s. Several US-based privacy bills have passed or will soon. This is just a tiny sampling of privacy-related statutes (and we believe just the beginning). Still, the penalties for violation of any of these laws can be significant, even devastating, to a business. And because data systems are woven into the fabric of education, health care, and business, data engineers handle sensitive data related to each of these laws.

An ninh là yếu tố sống còn đối với thực hành kỹ thuật dữ liệu (data engineering). Điều này lẽ ra phải hiển nhiên đến chói mắt, nhưng chúng tôi liên tục ngạc nhiên trước việc các kỹ sư dữ liệu thường xem an ninh như một thứ nghĩ đến sau cùng. Chúng tôi tin rằng an ninh là điều đầu tiên một kỹ sư dữ liệu cần nghĩ đến trong mọi khía cạnh công việc của mình và mọi giai đoạn của vòng đời kỹ thuật dữ liệu. Hằng ngày bạn làm việc với dữ liệu nhạy cảm, thông tin và quyền truy cập. Tổ chức, khách hàng và đối tác kinh doanh của bạn kỳ vọng những tài sản giá trị này được xử lý với sự cẩn trọng và quan tâm tối đa. Một vụ vi phạm an ninh hay rò rỉ dữ liệu có thể khiến doanh nghiệp của bạn chết đứng giữa dòng; sự nghiệp và danh tiếng của bạn sẽ tan tành nếu đó là lỗi của bạn. An ninh là thành phần cốt yếu cho quyền riêng tư. Quyền riêng tư từ lâu đã là yếu tố then chốt đối với niềm tin trong không gian công nghệ thông tin doanh nghiệp; các kỹ sư trực tiếp hoặc gián tiếp xử lý dữ liệu liên quan đến đời tư của con người. Điều này bao gồm thông tin tài chính, dữ liệu về các liên lạc riêng tư (email, tin nhắn, cuộc gọi điện thoại), tiền sử bệnh án, hồ sơ học tập và lịch sử việc làm. Một công ty làm rò rỉ thông tin này hoặc sử dụng sai

mục đích có thể thấy mình bị xa lánh khi vụ vi phạm bị phơi bày. Ngày càng nhiều, quyền riêng tư là một vấn đề có tầm quan trọng pháp lý đáng kể. Ví dụ, Đạo luật về Quyền Giáo dục và Quyền riêng tư của Gia đình (FERPA) có hiệu lực tại Mỹ vào những năm 1970; Đạo luật về Khả năng Chuyển đổi và Trách nhiệm Bảo hiểm Y tế (HIPAA) tiếp nối vào những năm 1990; GDPR được thông qua tại châu Âu vào giữa những năm 2010. Một số dự luật về quyền riêng tư tại Mỹ đã hoặc sắp được thông qua. Đây chỉ là một mẫu rất nhỏ trong số các đạo luật liên quan đến quyền riêng tư (và chúng tôi tin rằng đây mới chỉ là khởi đầu). Tuy vậy, hình phạt cho việc vi phạm bất kỳ luật nào trong số này có thể rất nặng nề, thậm chí hủy diệt, đối với một doanh nghiệp. Và bởi vì các hệ thống dữ liệu được đan dệt vào kết cấu của giáo dục, y tế và kinh doanh, các kỹ sư dữ liệu xử lý dữ liệu nhạy cảm liên quan đến mỗi luật này.

A data engineer's exact security and privacy responsibilities will vary significantly between organizations. At a small startup, a data engineer may do double duty as a **data security** engineer. A large tech company will have armies of security engineers and security researchers. Even in this situation, data engineers will often be able to identify security practices and technology vulnerabilities within their own teams and systems that they can report and mitigate in collaboration with dedicated security personnel. Because security and privacy are critical to data engineering (security being an undercurrent), we want to spend some more time covering security and privacy. In this chapter, we lay out some things data engineers should consider around security, particularly in people, processes, and technology (in that order). This isn't a complete list, but lays out the major things we'd wish would improve based on our experience.

Trách nhiệm cụ thể về an ninh và quyền riêng tư của một kỹ sư dữ liệu sẽ khác nhau đáng kể giữa các tổ chức. Tại một startup nhỏ, một kỹ sư dữ liệu có thể kiêm luôn vai trò kỹ sư an ninh dữ liệu. Một công ty công nghệ lớn sẽ có cả đội quân kỹ sư an ninh và nhà nghiên cứu an ninh. Ngay cả trong tình huống này, các kỹ sư dữ liệu thường vẫn có thể nhận diện các thực hành an ninh và lỗ hổng công nghệ trong chính nhóm và hệ thống của mình mà họ có thể báo cáo và giảm thiểu trong sự phối hợp với nhân sự an ninh chuyên trách. Bởi an ninh và quyền riêng tư là cốt yếu đối với kỹ thuật dữ liệu (an ninh là một dòng chảy ngầm (undercurrent)), chúng tôi muốn dành thêm thời gian bàn về an ninh và quyền riêng tư. Trong chương này, chúng tôi trình bày một số điều mà các kỹ sư dữ liệu nên cân nhắc xoay quanh an ninh, đặc biệt ở con người, quy trình và công nghệ (theo thứ tự đó). Đây không phải là một danh sách đầy đủ, mà chỉ nêu ra những điều chính mà dựa trên kinh nghiệm của chúng tôi, chúng tôi mong sẽ được cải thiện.

People — Con người

The weakest link in security and privacy is you . Security is often compromised at the human level, so conduct yourself as if you're always a target. A bot or human actor is trying to infiltrate your sensitive credentials and information at any given time. This is our reality, and it's not going away. Take a defensive posture with everything you do online and offline. Exercise the power of negative thinking and always be paranoid.

Mắt xích yếu nhất trong an ninh và quyền riêng tư là chính bạn. An ninh thường bị xâm phạm ở cấp độ con người, vì vậy hãy hành xử như thể bạn luôn là mục tiêu. Một bot hoặc một tác nhân là con người đang cố gắng xâm nhập vào thông tin xác thực và thông tin nhạy cảm của bạn vào bất kỳ thời điểm nào. Đây là thực tế của chúng ta, và nó sẽ không biến mất. Hãy giữ tư thế phòng thủ với mọi việc bạn làm trực tuyến và ngoại tuyến. Hãy vận dụng sức mạnh của tư duy tiêu cực và luôn luôn hoài nghi.

The Power of Negative Thinking — Sức mạnh của Tư duy Tiêu cực

In a world obsessed with positive thinking, negative thinking is distasteful. However, American surgeon Atul Gawande wrote a 2007 op-ed in the New York Times on precisely this subject. His central thesis is that positive thinking can blind us to the possibility of terrorist attacks or medical emergencies and deter preparation. Negative thinking allows us to consider disastrous scenarios and act to prevent them. Data engineers should actively think through the scenarios for data utilization and collect sensitive data only if there is an actual need **downstream**. The best way to protect private and sensitive data is to avoid ingesting this data in the first place. Data engineers should think about the attack and leak scenarios with any **data pipeline** or **storage** system they utilize. When deciding on security strategies, ensure that your approach delivers proper security and not just the illusion of safety.

Trong một thế giới ám ảnh với tư duy tích cực, tư duy tiêu cực thật khó ưa. Tuy nhiên, bác sĩ phẫu thuật người Mỹ Atul Gawande đã viết một bài xã luận năm 2007 trên tờ New York Times về chính chủ đề này. Luận điểm trung tâm của ông là tư duy tích cực có thể làm chúng ta mù quáng trước khả năng xảy ra các cuộc tấn công khủng bố hay các tình huống cấp cứu y tế và cản trở việc chuẩn bị. Tư duy tiêu cực cho phép chúng ta cân nhắc các kịch bản thảm họa và hành động để ngăn chặn chúng. Các kỹ sư dữ liệu nên chủ động suy nghĩ thấu đáo các kịch bản sử dụng dữ liệu và chỉ thu thập dữ liệu nhạy cảm khi thực sự có nhu cầu ở hạ nguồn (downstream). Cách tốt nhất để bảo vệ dữ liệu riêng tư và nhạy cảm là tránh thu nạp (ingest) dữ liệu này ngay từ đầu. Các kỹ sư dữ liệu nên suy nghĩ về các kịch bản tấn công và rò rỉ với bất kỳ đường ống dữ liệu (data pipeline) hay hệ thống lưu trữ nào họ sử dụng. Khi quyết định các chiến lược an ninh, hãy đảm bảo cách tiếp cận của bạn mang lại an ninh đích thực chứ không chỉ là ảo giác về sự an toàn.

Always Be Paranoid — Luôn Luôn Hoài Nghi

Always exercise caution when someone asks you for your credentials. When in doubt—and you should always be in extreme doubt when asked for credentials—hold off and get second opinions from your coworkers and friends. Confirm with other people that the request is indeed legitimate. A quick chat or phone call is cheaper than a ransomware attack triggered through an email click. Trust nobody at face value when asked for credentials, sensitive data, or confidential information, including from your coworkers. You are also the first line of defense in respecting privacy and ethics. Are you uncomfortable with sensitive data you’ve been tasked to collect? Do you have ethical questions about the way data is being handled in a project? Raise your concerns with colleagues and leadership. Ensure that your work is both legally compliant and ethical.

Hãy luôn thận trọng khi ai đó hỏi bạn về thông tin xác thực của mình. Khi nghi ngờ — và bạn luôn nên hết sức nghi ngờ khi được hỏi về thông tin xác thực — hãy khoan đã và xin ý kiến thứ hai từ đồng nghiệp và bạn bè. Hãy xác nhận với người khác rằng yêu cầu đó thực sự hợp lệ. Một cuộc trò chuyện hay cuộc gọi điện thoại nhanh thì rẻ hơn một cuộc tấn công ransomware kích hoạt qua một cú nhấp vào email. Đừng tin ai chỉ qua vẻ bề ngoài khi được hỏi về thông tin xác thực, dữ liệu nhạy cảm hay thông tin mật, kể cả từ đồng nghiệp của bạn. Bạn cũng là tuyến phòng thủ đầu tiên trong việc tôn trọng quyền riêng tư và đạo đức. Bạn có thấy không thoải mái với dữ liệu nhạy cảm mà mình được giao thu thập không? Bạn có câu hỏi về mặt đạo đức đối với cách dữ liệu đang được xử lý trong một dự án không? Hãy nêu mối lo ngại của bạn với đồng nghiệp và ban lãnh đạo. Hãy đảm bảo công việc của bạn vừa tuân thủ pháp luật vừa hợp đạo đức.

Processes — Quy trình

When people follow regular security processes, security becomes part of the job. Make security a habit, regularly practice real security, exercise the **principle of least privilege**, and understand the **shared responsibility model** in the cloud.

Khi con người tuân theo các quy trình an ninh thường lệ, an ninh trở thành một phần của công việc. Hãy biến an ninh thành thói quen, thường xuyên thực hành an ninh thực sự, vận dụng nguyên tắc đặc quyền tối thiểu (principle of least privilege), và hiểu mô hình trách nhiệm chia sẻ (shared responsibility model) trên cloud.

Security Theater Versus Security Habit — An ninh Hình thức so với Thói quen An ninh

With our corporate clients, we see a pervasive focus on compliance (with internal rules, laws, recommendations from standards bodies), but not enough attention to potentially bad scenarios. Unfortunately, this creates an illusion of security but often leaves gaping holes that would be evident with a few minutes of reflection. Security needs to be simple and effective enough to become habitual throughout an organization. We're amazed at the number of companies with security policies in the hundreds of pages that nobody reads, the annual security policy review that people immediately forget, all in checking a box for a security audit. This is security theater, where security is done in the letter of compliance (SOC-2, ISO 27001, and related) without real commitment. Instead, pursue the spirit of genuine and habitual security; bake a security mindset into your culture. Security doesn't need to be complicated. For example, at our company, we run security training and policy review at least once a month to ingrain this into our team's DNA and update each other on security practices we can improve. Security must not be an afterthought for your data team. Everyone is responsible and has a role to play. It must be the priority for you and everyone else you work with.

Với các khách hàng doanh nghiệp của mình, chúng tôi thấy một sự tập trung tràn lan vào việc tuân thủ (các quy tắc nội bộ, luật pháp, khuyến nghị từ các tổ chức tiêu chuẩn), nhưng không đủ quan tâm đến các kịch bản tệ hại tiềm tàng. Đáng tiếc, điều này tạo ra ảo giác về an ninh nhưng thường để lại những lỗ hổng toang hoác mà chỉ cần vài phút suy ngẫm là thấy rõ. An ninh cần phải đủ đơn giản và hiệu quả để trở thành thói quen xuyên suốt một tổ chức. Chúng tôi ngạc nhiên trước số lượng các công ty có chính sách an ninh dày hàng trăm trang mà không ai đọc, đợt rà soát chính sách an ninh hằng năm mà người ta quên ngay lập tức, tất cả chỉ để đánh dấu vào một ô trong cuộc kiểm toán an ninh. Đây là an ninh hình thức, nơi an ninh được thực hiện theo đúng câu chữ của việc tuân thủ (SOC-2, ISO 27001 và những thứ liên quan) mà không có cam kết thực sự. Thay vào đó, hãy theo đuổi tinh thần của an ninh chân thực và thành thói quen; hãy nung tư duy an ninh vào trong văn hóa của bạn. An ninh không cần phải phức tạp. Ví dụ, tại công ty chúng tôi, chúng tôi tổ chức đào tạo an ninh và rà soát chính sách ít nhất mỗi tháng một lần để khắc sâu điều này vào DNA của đội ngũ và cập nhật cho nhau về các thực hành an ninh có thể cải thiện. An ninh không được phép là thứ nghĩ đến sau cùng đối với nhóm dữ liệu của bạn. Mọi người đều có trách nhiệm và có vai trò để đảm nhận. Nó phải là ưu tiên đối với bạn và mọi người khác mà bạn làm việc cùng.

Active Security — An ninh Chủ động

Returning to the idea of negative thinking, active security entails thinking about and researching security threats in a dynamic and changing world. Rather than simply deploying scheduled simulated phishing attacks, you can take an active security posture by researching successful phishing attacks and thinking through your organizational security vulnerabilities. Rather than simply adopting a standard compliance checklist, you can think about internal vulnerabilities specific to your organization and incentives employees might have to leak or misuse private information. We have more to say about active security in “Technology” .

Trở lại với ý tưởng tư duy tiêu cực, an ninh chủ động đòi hỏi việc suy nghĩ và nghiên cứu các mối đe dọa an ninh trong một thế giới năng động và biến đổi. Thay vì chỉ đơn thuần triển khai các cuộc tấn công phishing mô phỏng theo lịch, bạn có thể giữ tư thế an ninh chủ động bằng cách nghiên cứu các cuộc tấn công phishing thành công và suy nghĩ thấu đáo các lỗ hổng an ninh của tổ chức mình. Thay vì chỉ đơn thuần áp dụng một danh sách kiểm tra tuân thủ tiêu chuẩn, bạn có thể suy nghĩ về các lỗ hổng nội bộ đặc thù của tổ chức mình và những động cơ mà nhân viên có thể có để rò rỉ hoặc sử dụng sai thông tin riêng tư. Chúng tôi còn nhiều điều để nói về an ninh chủ động trong phần “Công nghệ”.

The Principle of Least Privilege — Nguyên tắc Đặc quyền Tối thiểu

The principle of least privilege means that a person or system should be given only the privileges and data they need to complete the task at hand and nothing more. Often, we see an antipattern in the cloud: a regular user is given administrative access to everything, when that person may need just a handful of IAM roles to do their work. Giving someone carte blanche administrative access is a huge mistake and should never happen under the principle of least privilege. Instead, provide the user (or group they belong to) the IAM roles they need when they need them. When these roles are no longer needed, take them away. The same rule applies to service accounts. Treat humans and machines the same way: give them only the privileges and data they need to do their jobs, and only for the timespan when needed. Of course, the principle of least privilege is also critical to privacy. Your users and customers expect that people will look at their sensitive data only when necessary. Make sure that this is the case. Implement column, row, and cell-level access controls around sensitive data; consider masking PII and other sensitive data and create views that contain only the information the viewer needs to access. Some data must be retained, but should be accessed only in an emergency. Put this data behind a broken glass process : users can access it only after going through an emergency approval process to fix a problem, query critical historical information, etc. Access is revoked immediately once the work is done.

Nguyên tắc đặc quyền tối thiểu nghĩa là một người hay một hệ thống chỉ nên được trao những đặc quyền và dữ liệu họ cần để hoàn thành nhiệm vụ trước mắt và không gì hơn thế. Chúng tôi thường thấy một phản mẫu (antipattern) trên cloud: một người dùng thông thường được trao quyền truy cập quản trị vào mọi thứ, trong khi người đó có thể chỉ cần một số ít vai trò IAM để làm việc. Trao cho ai đó quyền truy cập quản trị toàn quyền vô điều kiện là một sai lầm khổng lồ và không bao giờ nên xảy ra dưới nguyên tắc đặc quyền tối thiểu. Thay vào đó, hãy cung cấp cho người dùng (hoặc nhóm mà họ thuộc về) các vai trò IAM họ cần khi họ cần. Khi không còn cần đến những vai trò này nữa, hãy thu hồi chúng. Quy tắc tương tự áp dụng cho các tài khoản dịch vụ (service account). Hãy đối xử với con người và máy móc theo cùng một cách: chỉ trao cho họ những đặc quyền và dữ liệu họ cần để làm công việc của mình, và chỉ trong khoảng thời gian cần đến. Tất nhiên,

nguyên tắc đặc quyền tối thiểu cũng cốt yếu đối với quyền riêng tư. Người dùng và khách hàng của bạn kỳ vọng rằng người ta chỉ xem dữ liệu nhạy cảm của họ khi cần thiết. Hãy đảm bảo điều đó đúng là như vậy. Hãy triển khai kiểm soát truy cập ở cấp cột, hàng và ô xung quanh dữ liệu nhạy cảm; cân nhắc che giấu (mask) PII và dữ liệu nhạy cảm khác và tạo các khung nhìn (view) chỉ chứa thông tin mà người xem cần truy cập. Một số dữ liệu phải được lưu giữ, nhưng chỉ nên được truy cập trong trường hợp khẩn cấp. Hãy đặt dữ liệu này sau một quy trình “đập kính khẩn cấp” (broken glass): người dùng chỉ có thể truy cập sau khi trải qua một quy trình phê duyệt khẩn cấp để khắc phục sự cố, truy vấn thông tin lịch sử trọng yếu, v.v. Quyền truy cập bị thu hồi ngay lập tức sau khi công việc hoàn thành.

Shared Responsibility in the Cloud — Trách nhiệm Chia sẻ trên Cloud

Security is a shared responsibility in the cloud. The cloud vendor is responsible for ensuring the physical security of its data center and hardware. At the same time, you are responsible for the security of the applications and systems you build and maintain in the cloud. Most cloud security breaches continue to be caused by end users, not the cloud. Breaches occur because of unintended misconfigurations, mistakes, oversights, and sloppiness.

An ninh là một trách nhiệm chia sẻ trên cloud. Nhà cung cấp cloud chịu trách nhiệm đảm bảo an ninh vật lý cho trung tâm dữ liệu và phần cứng của họ. Đồng thời, bạn chịu trách nhiệm về an ninh của các ứng dụng và hệ thống mà bạn xây dựng và duy trì trên cloud. Hầu hết các vụ vi phạm an ninh cloud vẫn tiếp tục do người dùng cuối gây ra, chứ không phải do cloud. Các vụ vi phạm xảy ra vì cấu hình sai ngoài ý muốn, sai sót, sơ suất và cẩu thả.

Always Back Up Your Data — Luôn Sao lưu Dữ liệu của Bạn

Data disappears. Sometimes it's a dead hard drive or server; in other cases, someone might accidentally delete a database or an **object storage** bucket. A bad actor can also lock away data. Ransomware attacks are widespread these days. Some insurance companies are reducing payouts in the **event** of an attack, leaving you on the hook both to recover your data and pay the bad actor who's holding it hostage. You need to back up your data regularly, both for disaster recovery and continuity of business operations, if a version of your data is compromised in a ransomware attack. Additionally, test the restoration of your data backups on a regular basis. Data backup doesn't strictly fit under security and privacy practices; it goes under the larger heading of disaster prevention, but it's adjacent to security, especially in the era of ransomware attacks.

Dữ liệu biến mất. Đôi khi đó là một ổ cứng hay máy chủ chết; trong các trường hợp khác, ai đó có thể vô tình xóa một cơ sở dữ liệu hay một bucket lưu trữ đối tượng (object storage). Một tác nhân xấu cũng có thể khóa dữ liệu lại. Các cuộc tấn công ransomware lan tràn ngày nay. Một số công ty bảo hiểm đang giảm khoản chi trả trong trường hợp xảy ra tấn công, để bạn phải tự gánh cả việc khôi phục dữ liệu lẫn trả tiền cho kẻ xấu đang giữ dữ liệu làm con tin. Bạn cần sao lưu dữ liệu thường xuyên, vừa để phục hồi sau thảm họa vừa để duy trì tính liên tục của hoạt động kinh doanh, nếu một phiên bản dữ liệu của bạn bị xâm phạm trong một cuộc tấn công ransomware. Ngoài ra, hãy kiểm tra việc khôi phục các bản sao lưu dữ liệu của bạn một cách thường xuyên. Sao lưu dữ liệu không hoàn toàn nằm dưới các thực hành an ninh và quyền riêng tư; nó nằm dưới tiêu đề lớn hơn là phòng

chống thảm họa, nhưng nó kề cận với an ninh, đặc biệt trong kỷ nguyên của các cuộc tấn công ransomware.

An Example Security Policy — Một Ví dụ về Chính sách An ninh

This section presents a sample security policy regarding credentials, devices, and sensitive information. Notice that we don't overcomplicate things; instead, we give people a short list of practical actions they can take immediately.

Phần này trình bày một chính sách an ninh mẫu liên quan đến thông tin xác thực, thiết bị và thông tin nhạy cảm. Hãy lưu ý rằng chúng tôi không làm phức tạp hóa mọi thứ; thay vào đó, chúng tôi đưa cho mọi người một danh sách ngắn các hành động thực tế mà họ có thể thực hiện ngay.

EXAMPLE SECURITY POLICY Protect Your Credentials — CHÍNH SÁCH AN NINH MẪU Bảo vệ Thông tin Xác thực của Bạn

Protect your credentials at all costs. Here are some ground rules for credentials:

Hãy bảo vệ thông tin xác thực của bạn bằng mọi giá. Dưới đây là một số quy tắc nền tảng cho thông tin xác thực:

Use a single-sign-on (SSO) for everything. Avoid passwords whenever possible, and use SSO as the default.

Sử dụng đăng nhập một lần (SSO) cho mọi thứ. Tránh dùng mật khẩu bất cứ khi nào có thể, và dùng SSO làm mặc định.

Use multifactor authentication with SSO.

Sử dụng xác thực đa yếu tố với SSO.

Don't share passwords or credentials. This includes client passwords and credentials. If in doubt, see the person you report to. If that person is in doubt, keep digging until you find an answer.

Đừng chia sẻ mật khẩu hay thông tin xác thực. Điều này bao gồm cả mật khẩu và thông tin xác thực của khách hàng. Nếu nghi ngờ, hãy gặp người mà bạn báo cáo cho. Nếu người đó cũng nghi ngờ, hãy tiếp tục đào sâu cho đến khi tìm ra câu trả lời.

Beware of phishing and scam calls. Don't ever give your passwords out. (Again, prioritize SSO.)

Cảnh giác với các cuộc gọi phishing và lừa đảo. Đừng bao giờ tiết lộ mật khẩu của bạn. (Một lần nữa, hãy ưu tiên SSO.)

Disable or delete old credentials. Preferably the latter.

Vô hiệu hóa hoặc xóa các thông tin xác thực cũ. Tốt nhất là xóa hẳn.

Don't put your credentials in code. Handle secrets as configuration and never commit them to version control. Use a secrets manager where possible.

Đừng đặt thông tin xác thực của bạn trong mã. Hãy xử lý các bí mật (secret) như cấu hình và không bao giờ commit chúng vào version control. Hãy dùng một trình quản lý bí mật (secrets manager) khi có thể.

Always exercise the principle of least privilege. Never give more access than is required to do the job. This applies to all credentials and privileges in the cloud and on premises.

Luôn vận dụng nguyên tắc đặc quyền tối thiểu. Đừng bao giờ trao nhiều quyền truy cập hơn mức cần thiết để làm công việc. Điều này áp dụng cho mọi thông tin xác thực và đặc quyền trên cloud và tại chỗ (on premises).

Protect Your Devices — Bảo vệ Thiết bị của Bạn

Use device management for all devices used by employees. If an employee leaves the company or your device gets lost, the device can be remotely wiped.

Sử dụng quản lý thiết bị cho tất cả các thiết bị mà nhân viên sử dụng. Nếu một nhân viên rời công ty hoặc thiết bị của bạn bị mất, thiết bị đó có thể bị xóa sạch từ xa.

Use multifactor authentication for all devices.

Sử dụng xác thực đa yếu tố cho tất cả các thiết bị.

Sign in to your device using your company email credentials.

Đăng nhập vào thiết bị của bạn bằng thông tin xác thực email công ty của bạn.

All policies covering credentials and behavior apply to your device(s).

Tất cả các chính sách bao trùm thông tin xác thực và hành vi đều áp dụng cho (các) thiết bị của bạn.

Treat your device as an extension of yourself. Don't let your assigned device(s) out of your sight.

Hãy đối xử với thiết bị của bạn như một phần mở rộng của chính bạn. Đừng để (các) thiết bị được cấp cho bạn rời khỏi tầm mắt.

When screen sharing, be aware of exactly what you're sharing to protect sensitive information and communications. Share only single documents, browser tabs, or windows, and avoid sharing your full desktop. Share only what's required to convey your point.

Khi chia sẻ màn hình, hãy ý thức chính xác bạn đang chia sẻ những gì để bảo vệ thông tin và liên lạc nhạy cảm. Chỉ chia sẻ từng tài liệu, tab trình duyệt hoặc cửa sổ riêng lẻ, và tránh chia sẻ toàn bộ màn hình nền. Chỉ chia sẻ những gì cần thiết để truyền đạt ý của bạn.

Use “do not disturb” mode when on video calls; this prevents messages from appearing during calls or recordings.

Sử dụng chế độ “không làm phiền” khi đang trong cuộc gọi video; điều này ngăn các tin nhắn xuất hiện trong cuộc gọi hay bản ghi hình.

Software Update Policy — Chính sách Cập nhật Phần mềm

Restart your web browser when you see an update alert.

Khởi động lại trình duyệt web khi bạn thấy cảnh báo cập nhật.

Run minor OS updates on company and personal devices.

Chạy các bản cập nhật nhỏ của hệ điều hành trên thiết bị công ty và cá nhân.

The company will identify critical major OS updates and provide guidance.

Công ty sẽ xác định các bản cập nhật lớn quan trọng của hệ điều hành và cung cấp hướng dẫn.

Don't use the beta version of an OS.

Đừng dùng phiên bản beta của một hệ điều hành.

Wait a week or two for new major OS version releases.

Hãy đợi một hai tuần với các bản phát hành phiên bản lớn mới của hệ điều hành.

These are some basic examples of how security can be simple and effective. Based on your company's security profile, you may need to add more requirements for people to follow. And again, always remember that people are your weakest link in security.

Đây là một số ví dụ cơ bản về việc an ninh có thể đơn giản và hiệu quả như thế nào. Tùy vào hồ sơ an ninh của công ty bạn, bạn có thể cần thêm các yêu cầu để mọi người tuân theo. Và một lần nữa, hãy luôn nhớ rằng con người là mắt xích yếu nhất của bạn trong an ninh.

Technology — Công nghệ

After you've addressed security with people and processes, it's time to look at how you leverage technology to secure your systems and data assets. The following are some significant areas you should prioritize.

Sau khi bạn đã giải quyết an ninh với con người và quy trình, đã đến lúc xem xét cách bạn tận dụng công nghệ để bảo vệ các hệ thống và tài sản dữ liệu của mình. Sau đây là một số lĩnh vực quan trọng mà bạn nên ưu tiên.

Patch and Update Systems — Vá lỗi và Cập nhật Hệ thống

Software gets stale, and security vulnerabilities are constantly discovered. To avoid exposing a security flaw in an older version of the tools you're using, always patch and update operating systems and software as new updates become available. Thankfully, many SaaS and cloud-managed services automatically perform upgrades and other maintenance without your intervention. To update your own code and dependencies, either automate builds or set alerts on releases and vulnerabilities so you can be prompted to perform the updates manually.

Phần mềm trở nên cũ kỹ, và các lỗ hổng an ninh liên tục được phát hiện. Để tránh phơi bày một lỗ hổng an ninh trong một phiên bản cũ hơn của các công cụ bạn đang dùng, hãy luôn vá lỗi và cập nhật hệ điều hành cùng phần mềm khi có các bản cập nhật mới. May thay, nhiều dịch vụ SaaS và dịch vụ được quản lý trên cloud tự động thực hiện nâng cấp và bảo trì khác mà không cần bạn can thiệp. Để cập nhật mã và các phụ thuộc của riêng bạn, hãy hoặc tự động hóa quá trình build hoặc đặt cảnh báo về các bản phát hành và lỗ hổng để bạn có thể được nhắc thực hiện các bản cập nhật một cách thủ công.

Encryption — Mã hóa

Encryption is not a magic bullet. It will do little to protect you in the event of a human security breach that grants access to credentials. Encryption is a baseline requirement for any organization that respects security and privacy. It will protect you from basic attacks, such as network traffic interception. Let's look separately at encryption at rest and in transit.

Mã hóa (encryption) không phải là viên đạn thần kỳ. Nó sẽ chẳng giúp được mấy để bảo vệ bạn trong trường hợp một vụ vi phạm an ninh ở con người dẫn đến việc truy cập được thông tin xác thực. Mã hóa là một yêu cầu nền tảng cho bất kỳ tổ chức nào tôn trọng an ninh và quyền riêng tư. Nó sẽ bảo vệ bạn khỏi các cuộc tấn công cơ bản, chẳng hạn như chặn bắt lưu lượng mạng. Hãy cùng xem riêng mã hóa khi lưu trữ (at rest) và khi truyền (in transit).

Encryption at rest — Mã hóa khi lưu trữ

Be sure your data is encrypted when it is at rest (on a storage device). Your company laptops should have full-disk encryption enabled to protect data if a device is stolen. Implement server-side encryption for all data stored in servers, filesystems, databases, and object storage in the cloud. All data backups for archival purposes should also be encrypted. Finally, incorporate application-level encryption where applicable.

Hãy đảm bảo dữ liệu của bạn được mã hóa khi nó ở trạng thái lưu trữ (at rest) (trên một thiết bị lưu trữ). Các laptop công ty của bạn nên bật mã hóa toàn ổ đĩa để bảo vệ dữ liệu nếu một thiết bị bị đánh cắp. Hãy triển khai mã hóa phía máy chủ cho tất cả dữ liệu được lưu trong các máy chủ, hệ thống tệp, cơ sở dữ liệu và lưu trữ đối tượng trên cloud. Tất cả các bản sao lưu dữ liệu phục vụ mục đích lưu trữ dài hạn cũng nên được mã hóa. Cuối cùng, hãy tích hợp mã hóa ở cấp ứng dụng khi áp dụng được.

Encryption over the wire — Mã hóa trên đường truyền

Encryption over the wire is now the default for current protocols. For instance, HTTPS is generally required for modern cloud APIs. Data engineers should always be aware of how keys are handled; bad key handling is a significant source of data leaks. In addition, HTTPS does nothing to protect data if bucket permissions are left open to the public, another cause of several data scandals over the last decade.

Mã hóa trên đường truyền giờ đây là mặc định đối với các giao thức hiện hành. Chẳng hạn, HTTPS nhìn chung là bắt buộc đối với các API cloud hiện đại. Các kỹ sư dữ liệu nên luôn ý thức về cách các khóa được xử lý; xử lý khóa kém là một nguồn rò rỉ dữ liệu đáng kể. Ngoài ra, HTTPS chẳng làm gì để bảo vệ dữ liệu nếu quyền truy cập bucket bị để mở cho công chúng, một nguyên nhân khác của vài vụ bê bối dữ liệu trong thập kỷ qua.

Engineers should also be aware of the security limitations of older protocols. For example, FTP is simply not secure on a public network. While this may not appear to be a problem when data is already public, FTP is vulnerable to man-in-the-middle attacks, whereby an attacker intercepts downloaded data and changes it before it arrives at the client. It is best to simply avoid FTP. Make sure everything is encrypted over the wire, even with legacy protocols. When in doubt, use robust technology with encryption baked in.

Các kỹ sư cũng nên ý thức về những giới hạn an ninh của các giao thức cũ hơn. Ví dụ, FTP đơn giản là không an toàn trên một mạng công cộng. Mặc dù điều này có thể trông không phải là vấn đề khi dữ liệu đã công khai, FTP dễ bị tấn công xen giữa (man-in-the-middle), theo đó kẻ tấn công chặn bắt dữ liệu được tải về và thay đổi nó trước khi nó đến được client. Tốt nhất là đơn giản tránh dùng FTP. Hãy đảm bảo mọi thứ đều được mã hóa trên đường truyền, kể cả với các giao thức cũ. Khi nghỉ ngơi, hãy dùng công nghệ vững chắc đã tích hợp sẵn mã hóa.

Logging, Monitoring, and Alerting — Ghi nhật ký, Giám sát và Cảnh báo

Hackers and bad actors typically don't announce that they're infiltrating your systems. Most companies don't find out about security incidents until well after the fact. Part of **DataOps** is to observe, detect, and alert on incidents. As a data engineer, you should set up automated monitoring, logging, and alerting to be aware of peculiar events when they happen in your systems. If possible, set up automatic anomaly detection. Here are some areas you should monitor: Access

Tin tặc và các tác nhân xấu thường không tuyên bố rằng chúng đang xâm nhập hệ thống của bạn. Hầu hết các công ty không phát hiện ra các sự cố an ninh cho đến tận sau khi sự việc đã xảy ra rất lâu. Một phần của DataOps là quan sát, phát hiện và cảnh báo về các sự cố. Là một kỹ sư dữ liệu, bạn nên thiết lập giám sát, ghi nhật ký và cảnh báo tự động để nhận biết các sự kiện kỳ lạ khi chúng xảy ra trong hệ thống của bạn. Nếu có thể, hãy thiết lập phát hiện dị thường tự động. Dưới đây là một số lĩnh vực bạn nên giám sát: Quyền truy cập

Who's accessing what, when, and from where? What new accesses were granted? Are there strange patterns with your current users that might indicate their account is compromised, such as trying to access systems they don't usually access or shouldn't have access to? Do you see new unrecognized users accessing your system? Be sure to regularly comb through access logs, users, and their roles to ensure that everything looks OK.

Ai đang truy cập cái gì, khi nào và từ đâu? Những quyền truy cập mới nào đã được cấp? Có những mẫu hành vi lạ với người dùng hiện tại của bạn mà có thể cho thấy tài khoản của họ đã bị xâm phạm hay không, chẳng hạn như cố gắng truy cập các hệ thống họ thường không truy cập hoặc không nên có quyền truy cập? Bạn có thấy những người dùng mới không nhận diện được đang truy cập hệ thống của bạn không? Hãy đảm bảo thường xuyên rà soát kỹ các nhật ký truy cập, người dùng và vai trò của họ để đảm bảo mọi thứ trông vẫn ổn.

Resources

Tài nguyên

Monitor your disk, CPU, memory, and I/O for patterns that seem out of the ordinary. Did your resources suddenly change? If so, this might indicate a security breach.

Hãy giám sát ổ đĩa, CPU, bộ nhớ và I/O của bạn để phát hiện các mẫu trông bất thường. Tài nguyên của bạn có thay đổi đột ngột không? Nếu có, điều này có thể cho thấy một vụ vi phạm an ninh.

Billing

Hóa đơn

Especially with SaaS and cloud-managed services, you need to oversee costs. Set up budget alerts to make sure your spending is within expectations. If an unexpected spike occurs in your billing, this might indicate someone or something is utilizing your resources for malicious purposes.

Đặc biệt với các dịch vụ SaaS và dịch vụ được quản lý trên cloud, bạn cần giám sát chi phí. Hãy thiết lập các cảnh báo ngân sách để đảm bảo chi tiêu của bạn nằm trong kỳ vọng. Nếu một đợt tăng vọt ngoài dự kiến xảy ra trong hóa đơn của bạn, điều này có thể cho thấy ai đó hoặc thứ gì đó đang sử dụng tài nguyên của bạn cho các mục đích xấu.

Excess permissions Increasingly, vendors are providing tools that monitor for permissions that are not utilized by a user or service account over some time. These tools can often be configured to automatically alert an administrator or remove permissions after a specified elapsed time. For example, suppose that a particular analyst hasn't accessed Redshift for six months. These permissions can be removed, closing a potential security hole. If the analyst needs to access Redshift in the future, they can put in a ticket to restore permissions.

Quyền dư thừa Ngày càng nhiều, các nhà cung cấp đang cung cấp các công cụ giám sát các quyền không được người dùng hay tài khoản dịch vụ sử dụng trong một khoảng thời gian nào đó. Các công cụ này thường có thể được cấu hình để tự động cảnh báo một quản trị viên hoặc gỡ bỏ quyền sau một khoảng thời gian trôi qua nhất định. Ví dụ, giả sử một nhà phân tích cụ thể đã không truy cập Redshift trong sáu tháng. Các quyền này có thể được gỡ bỏ, đóng lại một lỗ hổng an ninh tiềm tàng. Nếu nhà phân tích đó cần truy cập Redshift trong tương lai, họ có thể gửi một phiếu yêu cầu (ticket) để khôi phục quyền.

It's best to combine these areas in your monitoring to get a cross-sectional view of your resource, access, and billing profile. We suggest setting up a dashboard for everyone on the data team to view monitoring and receive alerts when something seems out of the ordinary. Couple this with an effective incident response plan to manage security breaches when they occur, and run through the plan on a regular basis so you are prepared.

Tốt nhất là kết hợp các lĩnh vực này trong việc giám sát của bạn để có được một góc nhìn cắt ngang về hồ sơ tài nguyên, truy cập và hóa đơn của bạn. Chúng tôi gợi ý thiết lập một bảng điều khiển (dashboard) để mọi người trong nhóm dữ liệu xem việc giám sát và nhận cảnh báo khi có điều gì đó trông bất thường. Hãy kết hợp điều này với một kế hoạch ứng phó sự cố hiệu quả để xử lý các vụ vi phạm an ninh khi chúng xảy ra, và chạy thử kế hoạch một cách thường xuyên để bạn luôn sẵn sàng.

Network Access — Truy cập Mạng

We often see data engineers doing pretty wild things regarding network access. In several instances, we've seen publicly available Amazon S3 buckets housing lots of sensitive data. We've also witnessed Amazon EC2 instances with inbound SSH access open to the whole world for 0.0.0.0/0 (all IPs) or databases with open access to all inbound requests over the public internet. These are just a few examples of terrible network security practices. In principle, network security should be left to security experts at your company. (In practice, you may need to assume significant responsibility for network security in a small company.) As a data engineer, you will encounter databases, object storage, and servers so often that you should at least be aware of simple measures you can take to make sure you're in line with good network access practices. Understand what IPs and ports are open, to whom, and why. Allow the incoming IP addresses of the systems and users that will access these ports and avoid broadly opening connections for any reason. When accessing the cloud or a SaaS tool, use an encrypted connection. For example, don't use an unencrypted website from a coffee shop. Also, while this book has focused almost entirely on running workloads in the cloud, we add a brief

note here about hosting on-premises servers. Recall that in Chapter 3, we discussed the difference between a hardened perimeter and zero-trust security. The cloud is generally closer to zero-trust security—every action requires authentication. We believe that the cloud is a more secure option for most organizations because it imposes zero-trust practices and allows companies to leverage the army of security engineers employed by the public clouds. However, sometimes hardened perimeter security still makes sense; we find some solace in the knowledge that nuclear missile silos are air gapped (not connected to any networks). Air-gapped servers are the ultimate example of a hardened security perimeter. Just keep in mind that even on premises, air-gapped servers are vulnerable to human security failings.

Chúng tôi thường thấy các kỹ sư dữ liệu làm những điều khá hoang đại liên quan đến truy cập mạng. Trong vài trường hợp, chúng tôi đã thấy các bucket Amazon S3 công khai chứa rất nhiều dữ liệu nhạy cảm. Chúng tôi cũng đã chứng kiến các instance Amazon EC2 với quyền truy cập SSH vào mở toang cho cả thế giới ở mức 0.0.0.0/0 (tất cả các IP) hoặc các cơ sở dữ liệu mở quyền truy cập cho tất cả các yêu cầu đến qua internet công cộng. Đây chỉ là một vài ví dụ về các thực hành an ninh mạng tồi tệ. Về nguyên tắc, an ninh mạng nên được để cho các chuyên gia an ninh tại công ty của bạn lo. (Trên thực tế, bạn có thể phải gánh trách nhiệm đáng kể về an ninh mạng tại một công ty nhỏ.) Là một kỹ sư dữ liệu, bạn sẽ thường xuyên gặp các cơ sở dữ liệu, lưu trữ đối tượng và máy chủ đến mức ít nhất bạn nên ý thức về những biện pháp đơn giản mà bạn có thể thực hiện để đảm bảo mình tuân theo các thực hành truy cập mạng tốt. Hãy hiểu những IP và cổng nào đang mở, cho ai và vì sao. Hãy cho phép các địa chỉ IP đến của các hệ thống và người dùng sẽ truy cập những cổng này và tránh mở rộng kết nối một cách rộng rãi vì bất kỳ lý do gì. Khi truy cập cloud hay một công cụ SaaS, hãy dùng một kết nối được mã hóa. Ví dụ, đừng dùng một trang web không mã hóa từ một quán cà phê. Ngoài ra, mặc dù cuốn sách này gần như hoàn toàn tập trung vào việc chạy khối lượng công việc trên cloud, chúng tôi thêm một ghi chú ngắn ở đây về việc tự vận hành (host) máy chủ tại chỗ. Hãy nhớ lại rằng ở Chương 3, chúng ta đã bàn về sự khác biệt giữa một vành đai được củng cố (hardened perimeter) và an ninh không tin cậy (zero-trust). Cloud nhìn chung gần với an ninh không tin cậy hơn — mọi hành động đều đòi hỏi xác thực. Chúng tôi tin rằng cloud là một lựa chọn an toàn hơn cho hầu hết các tổ chức vì nó áp đặt các thực hành không tin cậy và cho phép các công ty tận dụng cả đội quân kỹ sư an ninh do các đám mây công cộng tuyển dụng. Tuy nhiên, đôi khi an ninh vành đai được củng cố vẫn hợp lý; chúng tôi tìm thấy chút an ủi trong việc biết rằng các hầm phóng tên lửa hạt nhân được cách ly (air gapped) (không kết nối với bất kỳ mạng nào). Các máy chủ cách ly là ví dụ tối thượng về một vành đai an ninh được củng cố. Chỉ cần nhớ rằng ngay cả tại chỗ, các máy chủ cách ly vẫn dễ bị tổn thương trước các thất bại an ninh do con người.

Security for Low-Level Data Engineering — An ninh cho Kỹ thuật Dữ liệu Cấp thấp

For engineers who work in the guts of data storage and processing systems, it is critical to consider the security implications of every element. Any software library, storage system, or compute node is a potential security vulnerability. A flaw in an obscure logging library might allow attackers to bypass access controls or encryption. Even CPU architectures and microcode represent potential vulnerabilities; sensitive data can be vulnerable when it's at rest in memory or a CPU cache. No link in the chain can be taken for granted. Of course, this book is principally about high-level data engineering— stitching together tools to handle the entire lifecycle. Thus, we'll leave it to you to dig into the gory technical details.

Đối với các kỹ sư làm việc trong ruột gan của các hệ thống lưu trữ và xử lý dữ liệu, việc cân nhắc các hệ quả an ninh của mọi thành phần là cốt yếu. Bất kỳ thư viện phần mềm, hệ thống lưu trữ hay nút tính toán nào cũng là một lỗ hổng an ninh tiềm tàng. Một lỗi trong một thư viện ghi nhật ký ít người biết đến có thể cho phép kẻ tấn công vượt qua các kiểm soát truy cập hoặc mã hóa. Ngay cả các kiến trúc CPU và microcode cũng đại diện cho các lỗ hổng tiềm tàng; dữ liệu nhạy cảm có thể bị tổn thương khi nó ở trạng thái lưu trữ trong bộ nhớ hoặc bộ đệm CPU (CPU cache). Không một mắt xích nào trong chuỗi có thể được coi là đương nhiên an toàn. Tất nhiên, cuốn sách này chủ yếu nói về kỹ thuật dữ liệu cấp cao — ghép nối các công cụ để xử lý toàn bộ vòng đời. Vì vậy, chúng tôi xin để bạn tự đào sâu vào các chi tiết kỹ thuật rồi nhé.

Internal security research — Nghiên cứu an ninh nội bộ

We discussed the idea of active security in “Processes” . We also highly recommend adopting an active security approach to technology. Specifically, this means that every technology employee should think about security problems. Why is this important? Every technology contributor develops a domain of technical expertise. Even if your company employs an army of security researchers, data engineers will become intimately familiar with specific data systems and cloud services in their purview. Experts in a particular technology are well positioned to identify security holes in this technology. Encourage every data engineer to be actively involved in security. When they identify potential security risks in their systems, they should think through mitigations and take an active role in deploying these.

Chúng ta đã bàn về ý tưởng an ninh chủ động trong phần “Quy trình”. Chúng tôi cũng hết sức khuyến nghị áp dụng cách tiếp cận an ninh chủ động đối với công nghệ. Cụ thể, điều này nghĩa là mọi nhân viên công nghệ đều nên suy nghĩ về các vấn đề an ninh. Vì sao điều này quan trọng? Mọi người đóng góp về công nghệ đều phát triển một lĩnh vực chuyên môn kỹ thuật. Ngay cả khi công ty của bạn tuyển cả đội quân nhà nghiên cứu an ninh, các kỹ sư dữ liệu sẽ trở nên am hiểu sâu sắc các hệ thống dữ liệu và dịch vụ cloud cụ thể trong phạm vi phụ trách của họ. Các chuyên gia về một công nghệ cụ thể có vị thế tốt để nhận diện các lỗ hổng an ninh trong công nghệ này. Hãy khuyến khích mọi kỹ sư dữ liệu chủ động tham gia vào an ninh. Khi họ nhận diện các rủi ro an ninh tiềm tàng trong hệ thống của mình, họ nên suy nghĩ thấu đáo các biện pháp giảm thiểu và đảm nhận vai trò chủ động trong việc triển khai chúng.

Conclusion — Kết luận

Security needs to be a habit of mind and action; treat data like your wallet or smartphone. Although you won’t likely be in charge of security for your company, knowing basic security practices and keeping security top of mind will help reduce the risk of data security breaches at your organization.

An ninh cần trở thành một thói quen của tư duy và hành động; hãy đối xử với dữ liệu như cái ví hay điện thoại thông minh của bạn. Mặc dù nhiều khả năng bạn sẽ không phụ trách an ninh cho công ty của mình, việc biết các thực hành an ninh cơ bản và luôn để an ninh trong tâm trí sẽ giúp giảm thiểu rủi ro về các vụ vi phạm an ninh dữ liệu tại tổ chức của bạn.

Additional Resources

Open Web Application Security Project (OWASP) publications

Building Secure and Reliable Systems by Heather Adkins et al. (O'Reilly)

Practical Cloud Security by Chris Dotson (O'Reilly)

Chapter 11. The Future of Data Engineering — Chương 11. Tương lai của Kỹ thuật Dữ liệu

This book grew out of the authors' recognition that warp speed changes in the field have created a significant knowledge gap for existing data engineers, people interested in moving into a career in **data engineering**, technology managers, and executives who want to better understand how data engineering fits into their companies. When we started thinking about how to organize this book, we got quite a bit of pushback from friends who'd ask, "How dare you write about a field that is changing so quickly?!" In many ways, they're right. It certainly feels like the field of data engineering—and, really, all things data—is changing daily. Sifting through the noise and finding the signal of what's unlikely to change was among the most challenging parts of organizing and writing this book. In this book, we focus on big ideas that we feel will be useful for the next several years—hence the continuum of the **data engineering lifecycle** and its **undercurrents**. The order of operations and names of best practices and technologies might change, but the primary stages of the lifecycle will likely remain intact for many years to come. We're keenly aware that technology continues to change at an exhausting pace; working in the technology sector in our present era can feel like a rollercoaster ride or perhaps a hall of mirrors. Several years ago, data engineering didn't even exist as a field or job title. Now you're reading a book called Fundamentals of Data Engineering ! You've learned all about the fundamentals of data engineering—its lifecycle, undercurrents, technologies, and best practices. You might be asking yourself, what's next in data engineering? While nobody can predict the future, we have a good perspective on the past, the present, and current trends. We've been fortunate to watch the genesis and evolution of data engineering from a front-row seat. This final chapter presents our thoughts on the future, including observations of ongoing developments and wild future speculation.

Cuốn sách này ra đời từ nhận thức của các tác giả rằng những thay đổi với tốc độ siêu nhanh trong lĩnh vực này đã tạo ra một khoảng trống kiến thức đáng kể cho các kỹ sư dữ liệu (data engineer) hiện tại, cho những người quan tâm đến việc chuyển hướng sang sự nghiệp kỹ thuật dữ liệu (data engineering), cho các nhà quản lý công nghệ, và cho các lãnh đạo muốn hiểu rõ hơn cách kỹ thuật dữ liệu phù hợp với công ty của họ. Khi chúng tôi bắt đầu nghĩ về cách tổ chức cuốn sách này, chúng tôi đã nhận khá nhiều ý kiến phản đối từ bạn bè, những người hỏi rằng: "Sao các anh dám viết về một lĩnh vực đang thay đổi nhanh đến vậy?!" Theo nhiều cách, họ nói đúng. Quả thật, có cảm giác như lĩnh vực kỹ thuật dữ liệu—và thực ra là tất cả mọi thứ liên quan đến dữ liệu—đang thay đổi từng ngày. Việc sàng lọc qua nhiều để tìm ra tín hiệu của những điều khó có khả năng thay đổi là một trong những phần thử thách nhất khi tổ chức và viết cuốn sách này. Trong cuốn sách này, chúng tôi tập trung vào những ý tưởng lớn mà chúng tôi cho rằng sẽ hữu ích trong

vài năm tới—do đó mới có sự liên tục của vòng đời kỹ thuật dữ liệu (data engineering lifecycle) và các dòng chảy ngầm (undercurrents) của nó. Trình tự các thao tác và tên gọi của các thông lệ tốt nhất cũng như công nghệ có thể thay đổi, nhưng các giai đoạn chính của vòng đời nhiều khả năng vẫn còn nguyên vẹn trong nhiều năm tới. Chúng tôi nhận thức rõ rằng công nghệ tiếp tục thay đổi với tốc độ một mét mỗi; làm việc trong lĩnh vực công nghệ ở thời đại của chúng ta hiện nay có thể giống như một chuyến tàu lượn siêu tốc hoặc có lẽ là một mê cung gương. Vài năm trước, kỹ thuật dữ liệu thậm chí còn chưa tồn tại như một lĩnh vực hay một chức danh công việc. Giờ đây bạn đang đọc một cuốn sách có tên Fundamentals of Data Engineering! Bạn đã học tất cả về các nền tảng của kỹ thuật dữ liệu—vòng đời, các dòng chảy ngầm, công nghệ, và các thông lệ tốt nhất của nó. Bạn có thể đang tự hỏi, điều gì tiếp theo trong kỹ thuật dữ liệu? Dù không ai có thể dự đoán tương lai, chúng tôi có một góc nhìn tốt về quá khứ, hiện tại, và các xu hướng hiện thời. Chúng tôi may mắn được chứng kiến sự khởi nguyên và tiến hóa của kỹ thuật dữ liệu từ một vị trí hàng đầu. Chương cuối cùng này trình bày những suy nghĩ của chúng tôi về tương lai, bao gồm cả những quan sát về các diễn biến đang diễn ra lẫn những suy đoán táo bạo về tương lai.

The Data Engineering Lifecycle Isn't Going Away — Vòng đời Kỹ thuật Dữ liệu sẽ không Biến mất

While data science has received the bulk of the attention in recent years, data engineering is rapidly maturing into a distinct and visible field. It's one of the fastest-growing careers in technology, with no signs of losing momentum. As companies realize they first need to build a data foundation before moving to “sexier” things like AI and ML, data engineering will continue growing in popularity and importance. This progress centers around the data engineering lifecycle. Some question whether increasingly simple tools and practices will lead to the disappearance of data engineers. This thinking is shallow, lazy, and shortsighted. As organizations leverage data in new ways, new foundations, systems, and workflows will be needed to address these needs. Data engineers sit at the center of designing, architecting, building, and maintaining these systems. If tooling becomes easier to use, data engineers will move up the value chain to focus on higher-level work. The data engineering lifecycle isn't going away anytime soon.

Trong khi khoa học dữ liệu (data science) nhận được phần lớn sự chú ý trong những năm gần đây, kỹ thuật dữ liệu đang nhanh chóng trưởng thành thành một lĩnh vực riêng biệt và dễ nhận thấy. Đây là một trong những nghề nghiệp phát triển nhanh nhất trong công nghệ, không có dấu hiệu mất đà. Khi các công ty nhận ra rằng trước tiên họ cần xây dựng một nền tảng dữ liệu trước khi chuyển sang những thứ “hấp dẫn” hơn như AI và ML, kỹ thuật dữ liệu sẽ tiếp tục gia tăng về độ phổ biến lẫn tầm quan trọng. Tiến trình này xoay quanh vòng đời kỹ thuật dữ liệu. Một số người đặt câu hỏi liệu các công cụ và thông lệ ngày càng đơn giản có dẫn đến sự biến mất của các kỹ sư dữ liệu hay không. Lối nghĩ này nông cạn, lười biếng và thiển cận. Khi các tổ chức khai thác dữ liệu theo những cách mới, sẽ cần đến những nền tảng, hệ thống và quy trình làm việc mới để giải quyết những nhu cầu này. Các kỹ sư dữ liệu nằm ở trung tâm của việc thiết kế, kiến tạo, xây dựng và bảo trì các hệ thống này. Nếu công cụ trở nên dễ sử dụng hơn, các kỹ sư dữ liệu sẽ tiến lên trong chuỗi giá trị để tập trung vào công việc ở cấp cao hơn. Vòng đời kỹ thuật dữ liệu sẽ không biến mất trong tương lai gần.

The Decline of Complexity and the Rise of Easy-to-Use Data Tools — Sự suy giảm của Độ phức tạp và Sự trỗi dậy của các Công cụ Dữ liệu Dễ sử dụng

Simplified, easy-to-use tools continue to lower the barrier to entry for data engineering. This is a great thing, especially given the shortage of data engineers we’ve discussed. The trend toward simplicity will continue. Data engineering isn’t dependent on a particular technology or data size. It’s also not just for large companies. In the 2000s, deploying “big data” technologies required a large team and deep pockets. The ascendance of SaaS-managed services has largely removed the complexity of understanding the guts of various “big data” systems. Data engineering is now something that all companies can do. Big data is a victim of its extraordinary success. For example, Google BigQuery, a descendant of GFS and MapReduce, can query petabytes of data. Once reserved for internal use at Google, this insanely powerful technology is now available to anybody with a GCP account. Users simply pay for the data they store and query rather than having to build a massive infrastructure stack. Snowflake, Amazon EMR, and many other hyper-scalable cloud data solutions compete in the space and offer similar capabilities. The cloud is responsible for a significant shift in the usage of open source tools. Even in the early 2010s, using open source typically entailed downloading the code and configuring it yourself. Nowadays, many open source data tools are available as managed cloud services that compete directly with proprietary services. Linux is available preconfigured and installed on server instances on all major clouds. **Serverless** platforms like AWS **Lambda** and Google Cloud Functions allow you to deploy **event**-driven applications in minutes, using mainstream languages such as Python, Java, and Go running atop Linux behind the scenes. Engineers wishing to use Apache Airflow can adopt Google’s Cloud Composer or AWS’s managed Airflow service. Managed Kubernetes allows us to build highly scalable microservice architectures. And so on. This fundamentally changes the conversation around open source code. In many cases, managed open source is just as easy to use as its proprietary service competitors. Companies with highly specialized needs can also deploy managed open source, then move to self-managed open source later if they need to customize the underlying code. Another significant trend is the growth in popularity of off-the-shelf data connectors (at the time of this writing, popular ones include Fivetran and Airbyte). Data engineers have traditionally spent a lot of time and resources building and maintaining plumbing to connect to external data sources. The new **generation** of managed connectors is highly compelling, even for highly technical engineers, as they begin to recognize the value of recapturing time and mental bandwidth for other projects. API connectors will be an outsourced problem so that data engineers can focus on the unique issues that drive their businesses. The intersection of red-**hot** competition in the data-tooling space with a growing number of data engineers means data tools will continue decreasing in complexity while adding even more functionality and features. This simplification will only grow the practice of data engineering, as more and more companies find opportunities to discover value in data.

Các công cụ được đơn giản hóa, dễ sử dụng tiếp tục hạ thấp rào cản gia nhập đối với kỹ thuật dữ liệu. Đây là một điều tuyệt vời, đặc biệt là khi xét đến tình trạng thiếu hụt kỹ sư dữ liệu mà chúng ta đã thảo luận. Xu hướng hướng tới sự đơn giản sẽ tiếp tục. Kỹ thuật dữ liệu không phụ thuộc vào một công nghệ cụ thể hay một kích cỡ dữ liệu cụ thể. Nó cũng không chỉ dành cho các công ty lớn. Vào những năm 2000, việc triển khai các công nghệ “big data” đòi hỏi một đội ngũ lớn và túi tiền sâu. Sự thăng tiến của các dịch vụ được quản lý theo mô hình SaaS đã gần như loại bỏ độ phức tạp của việc phải hiểu rõ ruột gan của nhiều hệ thống “big data” khác nhau. Kỹ thuật dữ liệu giờ đây là điều mà mọi công ty đều có thể làm. Big data là nạn nhân của chính thành công phi thường của nó. Ví dụ, Google BigQuery, một hậu duệ của GFS và MapReduce, có thể truy vấn hàng petabyte dữ liệu. Từng được dành riêng cho sử dụng nội bộ tại Google, công nghệ mạnh đến điên rồ

này giờ đây có sẵn cho bất kỳ ai có tài khoản GCP. Người dùng chỉ đơn giản trả tiền cho dữ liệu họ lưu trữ và truy vấn thay vì phải xây dựng một ngăn xếp hạ tầng đồ sộ. Snowflake, Amazon EMR, và nhiều giải pháp dữ liệu đám mây siêu mở rộng khác cạnh tranh trong không gian này và cung cấp những năng lực tương tự. Đám mây chịu trách nhiệm cho một sự dịch chuyển đáng kể trong cách sử dụng các công cụ mã nguồn mở. Ngay cả vào đầu những năm 2010, việc sử dụng mã nguồn mở thường đồng nghĩa với việc tải mã về và tự cấu hình. Ngày nay, nhiều công cụ dữ liệu mã nguồn mở có sẵn dưới dạng các dịch vụ đám mây được quản lý, cạnh tranh trực tiếp với các dịch vụ độc quyền. Linux có sẵn được cấu hình và cài đặt trước trên các phiên bản máy chủ ở tất cả các đám mây lớn. Các nền tảng serverless như AWS Lambda và Google Cloud Functions cho phép bạn triển khai các ứng dụng hưởng sự kiện trong vài phút, sử dụng các ngôn ngữ phổ thông như Python, Java và Go chạy bên trên Linux ở phía sau hậu trường. Các kỹ sư muốn dùng Apache Airflow có thể chọn Google's Cloud Composer hoặc dịch vụ Airflow được quản lý của AWS. Kubernetes được quản lý cho phép chúng ta xây dựng các kiến trúc microservices có khả năng mở rộng cao. Và cứ thế tiếp tục. Điều này thay đổi căn bản cuộc đối thoại xung quanh mã nguồn mở. Trong nhiều trường hợp, mã nguồn mở được quản lý cũng dễ sử dụng như các dịch vụ độc quyền đối thủ của nó. Các công ty có nhu cầu chuyên biệt cao cũng có thể triển khai mã nguồn mở được quản lý, sau đó chuyển sang mã nguồn mở tự quản lý về sau nếu họ cần tùy biến mã nền tảng bên dưới. Một xu hướng đáng kể khác là sự gia tăng độ phổ biến của các connector dữ liệu có sẵn (vào thời điểm viết cuốn sách này, những cái phổ biến gồm Fivetran và Airbyte). Các kỹ sư dữ liệu theo truyền thống đã tốn rất nhiều thời gian và nguồn lực để xây dựng và bảo trì hệ thống ống dẫn nhằm kết nối tới các nguồn dữ liệu bên ngoài. Hệ thống connector được quản lý mới đây hấp dẫn, ngay cả với những kỹ sư có trình độ kỹ thuật cao, khi họ bắt đầu nhận ra giá trị của việc giành lại thời gian và băng thông tinh thần cho các dự án khác. Các connector API sẽ trở thành một vấn đề được thuê ngoài, để các kỹ sư dữ liệu có thể tập trung vào những vấn đề độc đáo dẫn dắt doanh nghiệp của họ. Sự giao thoa giữa cuộc cạnh tranh nóng bỏng trong không gian công cụ dữ liệu với số lượng kỹ sư dữ liệu ngày càng tăng có nghĩa là các công cụ dữ liệu sẽ tiếp tục giảm độ phức tạp trong khi bổ sung thêm cả chức năng và tính năng. Sự đơn giản hóa này sẽ chỉ làm phát triển thực hành kỹ thuật dữ liệu, khi ngày càng nhiều công ty tìm thấy cơ hội để khám phá giá trị trong dữ liệu.

The Cloud-Scale Data OS and Improved Interoperability — Hệ điều hành Dữ liệu Quy mô Đám mây và Khả năng Tương tác Cải thiện

Let's briefly review some of the inner workings of (single-device) operating systems, then tie this back to data and the cloud. Whether you're utilizing a smartphone, a laptop, an application server, or a smart thermostat, these devices rely on an operating system to provide essential services and orchestrate tasks and processes. For example, I can see roughly 300 processes running on the MacBook Pro that I'm typing on. Among other things, I see services such as WindowServer (responsible for providing windows in a graphical interface) and CoreAudio (tasked with providing low-level audio capabilities). When I run an application on this machine, it doesn't directly access sound and graphics hardware. Instead, it sends commands to operating system services to draw windows and play sound. These commands are issued to standard APIs; a specification tells software developers how to communicate with operating system services. The operating system orchestrates a boot process to provide these services, starting each service in the correct order based on dependencies among them; it also maintains services by monitoring them and restarting

them in the correct order in case of a failure.

Hãy điểm nhanh qua một số cơ chế bên trong của các hệ điều hành (trên thiết bị đơn lẻ), rồi liên hệ điều này trở lại với dữ liệu và đám mây. Dù bạn đang dùng điện thoại thông minh, laptop, máy chủ ứng dụng, hay một bộ điều nhiệt thông minh, các thiết bị này đều dựa vào một hệ điều hành để cung cấp các dịch vụ thiết yếu và điều phối (orchestrate) các tác vụ cũng như tiến trình. Ví dụ, tôi có thể thấy khoảng 300 tiến trình đang chạy trên chiếc MacBook Pro mà tôi đang gõ. Trong số những thứ khác, tôi thấy các dịch vụ như WindowServer (chịu trách nhiệm cung cấp các cửa sổ trong giao diện đồ họa) và CoreAudio (đảm nhận việc cung cấp các năng lực âm thanh cấp thấp). Khi tôi chạy một ứng dụng trên máy này, nó không truy cập trực tiếp phần cứng âm thanh và đồ họa. Thay vào đó, nó gửi các lệnh đến các dịch vụ của hệ điều hành để vẽ cửa sổ và phát âm thanh. Các lệnh này được phát hành tới các API chuẩn; một đặc tả cho các nhà phát triển phần mềm biết cách giao tiếp với các dịch vụ của hệ điều hành. Hệ điều hành điều phối một quy trình khởi động để cung cấp các dịch vụ này, khởi động từng dịch vụ theo đúng thứ tự dựa trên các phụ thuộc giữa chúng; nó cũng duy trì các dịch vụ bằng cách giám sát chúng và khởi động lại chúng theo đúng thứ tự trong trường hợp có lỗi.

Now let's return to data in the cloud. The simplified data services that we've mentioned throughout this book (e.g., Google Cloud BigQuery, Azure Blob **Storage**, Snowflake, and AWS Lambda) resemble operating system services, but at a much larger scale, running across many machines rather than a single server. Now that these simplified services are available, the next frontier of evolution for this notion of a cloud data operating system will happen at a higher level of abstraction. Benn Stancil called for the emergence of standardized data APIs for building data pipelines and data applications. We predict that data engineering will gradually coalesce around a handful of data interoperability standards. **Object storage** in the cloud will grow in importance as a **batch** interface layer between various data services. New generation file formats (such as Parquet and Avro) are already taking over for the purposes of cloud data interchange, significantly improving on the dreadful interoperability of CSV, and the poor performance of raw JSON. Another critical ingredient of a data API ecosystem is a **metadata** catalog that describes schemas and data hierarchies. Currently, this role is largely filled by the legacy Hive Metastore. We expect that new entrants will emerge to take its place. Metadata will play a crucial role in data interoperability, both across applications and systems, and across clouds and networks, driving automation and simplification. We will also see significant improvements in the scaffolding that manages cloud data services. Apache Airflow has emerged as the first truly cloud-oriented data **orchestration** platform, but we are on the cusp of significant enhancement. Airflow will grow in capabilities, building on its massive mindshare. New entrants such as Dagster and Prefect will compete by rebuilding orchestration architecture from the ground up. This next generation of data orchestration platforms will feature enhanced data integration and data awareness. Orchestration platforms will integrate with data cataloging and lineage, becoming significantly more data-aware in the process. In addition, orchestration platforms will build IaC capabilities (similar to Terraform) and code deployment features (like GitHub Actions and Jenkins). This will allow engineers to code a pipeline, and then pass it to the orchestration platform to automatically build, test, deploy and monitor. Engineers will be able to write infrastructure specifications directly into their pipelines; missing infrastructure and services (e.g., Snowflake databases, Databricks clusters, and Amazon Kinesis streams) will be deployed the first time the pipeline runs. We will also see significant enhancements in the domain of live data—e.g., **streaming** pipelines and databases capable of ingesting and querying streaming data. In the past, building a streaming **DAG** was an extremely complex process with a high ongoing operational burden (see Chapter 8). Tools like Apache Pulsar point the way toward a future in which streaming DAGs can be deployed with complex transformations using relatively simple code. We have already seen the emergence of managed stream processors (such as Amazon Kinesis Data **Analytics** and

Google Cloud Dataflow), but we will see a new generation of orchestration tools for managing these services, stitching them together, and monitoring them. We discuss live data in “The Live Data Stack”. What does this enhanced abstraction mean for data engineers? As we’ve already argued in this chapter, the role of the **data engineer** won’t go away, but it will evolve significantly. By comparison, more sophisticated mobile operating systems and frameworks have not eliminated mobile app developers. Instead, mobile app developers can now focus on building better-quality, more sophisticated applications. We expect similar developments for data engineering as the cloud-scale data OS paradigm increases interoperability and simplicity across various applications and systems.

Giờ hãy quay trở lại với dữ liệu trên đám mây. Các dịch vụ dữ liệu được đơn giản hóa mà chúng tôi đã đề cập xuyên suốt cuốn sách này (ví dụ Google Cloud BigQuery, Azure Blob Storage, Snowflake, và AWS Lambda) giống như các dịch vụ của hệ điều hành, nhưng ở quy mô lớn hơn nhiều, chạy trên nhiều máy thay vì một máy chủ đơn lẻ. Giờ đây khi các dịch vụ đơn giản hóa này đã sẵn có, biên giới tiến hóa kế tiếp cho khái niệm về một hệ điều hành dữ liệu trên đám mây sẽ diễn ra ở một mức trừu tượng cao hơn. Benn Stancil đã kêu gọi sự xuất hiện của các API dữ liệu được chuẩn hóa để xây dựng các đường ống dữ liệu (data pipeline) và các ứng dụng dữ liệu.¹ Chúng tôi dự đoán rằng kỹ thuật dữ liệu sẽ dần dần kết tụ xung quanh một số ít các tiêu chuẩn về khả năng tương tác dữ liệu. Lưu trữ đối tượng (object storage) trên đám mây sẽ tăng tầm quan trọng như một lớp giao diện theo lô (batch) giữa nhiều dịch vụ dữ liệu khác nhau. Các định dạng tệp thể hệ mới (như Parquet và Avro) đã và đang thay thế cho các mục đích trao đổi dữ liệu trên đám mây, cải thiện đáng kể so với khả năng tương tác tệp của CSV, và hiệu năng kém của JSON thô. Một thành phần then chốt khác của một hệ sinh thái API dữ liệu là một danh mục siêu dữ liệu (metadata catalog) mô tả các lược đồ (schema) và các phân cấp dữ liệu. Hiện tại, vai trò này phần lớn được đảm nhận bởi Hive Metastore cũ kỹ. Chúng tôi kỳ vọng những tay chơi mới sẽ xuất hiện để thay thế nó. Siêu dữ liệu (metadata) sẽ đóng một vai trò quan trọng trong khả năng tương tác dữ liệu, cả giữa các ứng dụng và hệ thống, lẫn giữa các đám mây và mạng, thúc đẩy sự tự động hóa và đơn giản hóa. Chúng ta cũng sẽ thấy những cải thiện đáng kể trong bộ khung quản lý các dịch vụ dữ liệu đám mây. Apache Airflow đã nổi lên như nền tảng điều phối (orchestration) dữ liệu hướng đám mây thực sự đầu tiên, nhưng chúng ta đang ở ngưỡng cửa của sự nâng cấp đáng kể. Airflow sẽ tăng trưởng về năng lực, dựa trên thị phần tâm trí khổng lồ của nó. Những tay chơi mới như Dagster và Prefect sẽ cạnh tranh bằng cách xây dựng lại kiến trúc điều phối từ con số không. Thế hệ nền tảng điều phối dữ liệu kế tiếp này sẽ có khả năng tích hợp dữ liệu và nhận biết dữ liệu được nâng cao. Các nền tảng điều phối sẽ tích hợp với việc lập danh mục dữ liệu và truy vết nguồn gốc dữ liệu (data lineage), trở nên nhận biết dữ liệu hơn nhiều trong quá trình đó. Ngoài ra, các nền tảng điều phối sẽ xây dựng các năng lực IaC (tương tự Terraform) và các tính năng triển khai mã (như GitHub Actions và Jenkins). Điều này sẽ cho phép các kỹ sư viết mã cho một đường ống, rồi chuyển nó cho nền tảng điều phối để tự động xây dựng, kiểm thử, triển khai và giám sát. Các kỹ sư sẽ có thể viết các đặc tả hạ tầng trực tiếp vào các đường ống của họ; các hạ tầng và dịch vụ còn thiếu (ví dụ các database Snowflake, các cụm Databricks, và các luồng Amazon Kinesis) sẽ được triển khai trong lần chạy đầu tiên của đường ống. Chúng ta cũng sẽ thấy những nâng cấp đáng kể trong lĩnh vực dữ liệu trực tiếp (live data)—ví dụ các đường ống theo luồng (streaming) và các database có khả năng thu nạp (ingest) và truy vấn dữ liệu streaming. Trong quá khứ, việc xây dựng một DAG streaming là một quy trình cực kỳ phức tạp với gánh nặng vận hành liên tục cao (xem Chương 8). Các công cụ như Apache Pulsar chỉ ra con đường hướng tới một tương lai trong đó các DAG streaming có thể được triển khai với các phép biến đổi (transformation) phức tạp bằng mã tương đối đơn giản. Chúng ta đã chứng kiến sự xuất hiện của các bộ xử lý luồng được quản lý (như Amazon Kinesis Data Analytics và Google Cloud Dataflow),

nhưng chúng ta sẽ thấy một thể hệ công cụ điều phối mới để quản lý các dịch vụ này, ghép nối chúng lại với nhau, và giám sát chúng. Chúng tôi thảo luận về dữ liệu trực tiếp trong mục “The Live Data Stack”. Sự trù tượng được nâng cao này có ý nghĩa gì đối với các kỹ sư dữ liệu? Như chúng tôi đã lập luận trong chương này, vai trò của kỹ sư dữ liệu sẽ không biến mất, nhưng nó sẽ tiến hóa đáng kể. So sánh ra, các hệ điều hành và framework di động tinh vi hơn đã không loại bỏ các nhà phát triển ứng dụng di động. Thay vào đó, các nhà phát triển ứng dụng di động giờ đây có thể tập trung vào việc xây dựng các ứng dụng chất lượng cao hơn, tinh vi hơn. Chúng tôi kỳ vọng những diễn biến tương tự đối với kỹ thuật dữ liệu khi mô hình hệ điều hành dữ liệu quy mô đám mây gia tăng khả năng tương tác và sự đơn giản trên nhiều ứng dụng và hệ thống khác nhau.

“Enterprisey” Data Engineering — Kỹ thuật Dữ liệu kiểu “Doanh nghiệp”

The increasing simplification of data tools and the emergence and documentation of best practices means data engineering will become more “enterprisey.” This will make many readers violently cringe. The term enterprise, for some, conjures Kafkaesque nightmares of faceless committees dressed in overly starched blue shirts and khakis, endless red tape, and waterfall-managed development projects with constantly slipping schedules and ballooning budgets. In short, some of you read “enterprise” and imagine a soulless place where innovation goes to die. Fortunately, this is not what we’re talking about; we’re referring to some of the good things that larger companies do with data—management, operations, governance, and other “boring” stuff. We’re presently living through the golden age of “enterprisey” data management tools. Technologies and practices once reserved for giant organizations are trickling **downstream**. The once hard parts of big data and streaming data have now largely been abstracted away, with the focus shifting to ease of use, interoperability, and other refinements. This allows data engineers working on new tooling to find opportunities in the abstractions of data management, **DataOps**, and all the other undercurrents of data engineering. Data engineers will become “enterprisey.” Speaking of which...

Sự đơn giản hóa ngày càng tăng của các công cụ dữ liệu cùng sự xuất hiện và tài liệu hóa của các thông lệ tốt nhất có nghĩa là kỹ thuật dữ liệu sẽ trở nên “doanh nghiệp” hơn (enterprisey). Điều này sẽ khiến nhiều độc giả nhăn mặt dữ dội. Với một số người, thuật ngữ doanh nghiệp (enterprise) gợi lên những cơn ác mộng kiểu Kafka về những ủy ban vô danh mặc áo sơ mi xanh hồ cứng quá mức và quần kaki, vô số thủ tục giấy tờ rườm rà, và các dự án phát triển được quản lý theo mô hình waterfall với lịch trình liên tục trượt và ngân sách phình to. Nói tóm lại, một số bạn đọc chữ “enterprise” và tưởng tượng ra một nơi vô hồn nơi sự đổi mới đến để chết. May thay, đó không phải điều chúng tôi đang nói đến; chúng tôi đang nhắc đến một số điều tốt mà các công ty lớn làm với dữ liệu—quản lý, vận hành, quản trị, và những thứ “nhàm chán” khác. Hiện tại chúng ta đang sống qua thời kỳ hoàng kim của các công cụ quản lý dữ liệu kiểu “doanh nghiệp”. Các công nghệ và thông lệ từng được dành riêng cho các tổ chức khổng lồ giờ đây đang chảy xuống hạ nguồn (downstream). Những phần từng khó của big data và dữ liệu streaming giờ đã phần lớn được trù tượng hóa đi, với trọng tâm chuyển sang sự dễ sử dụng, khả năng tương tác, và các tính chỉnh khác. Điều này cho phép các kỹ sư dữ liệu làm việc trên công cụ mới tìm thấy cơ hội trong các trù tượng của quản lý dữ liệu, DataOps, và tất cả các dòng chảy ngầm khác của kỹ thuật dữ liệu. Các kỹ sư dữ liệu sẽ trở nên “doanh nghiệp”. Nhân tiện nói đến điều đó...

Titles and Responsibilities Will Morph... — Chức danh và Trách nhiệm sẽ Biến hóa...

While the data engineering lifecycle isn't going anywhere anytime soon, the boundaries between software engineering, data engineering, data science, and ML engineering are increasingly fuzzy. In fact, like the authors, many data scientists are transformed into data engineers through an organic process; tasked with doing “data science” but lacking the tools to do their jobs, they take on the job of designing and building systems to serve the data engineering lifecycle. As simplicity moves up the stack, data scientists will spend a smaller slice of their time gathering and munging data. But this trend will extend beyond data scientists. Simplification also means data engineers will spend less time on low-level tasks in the data engineering lifecycle (managing servers, configuration, etc.), and “enterprise” data engineering will become more prevalent.

Trong khi vòng đời kỹ thuật dữ liệu sẽ không đi đâu trong tương lai gần, các ranh giới giữa kỹ thuật phần mềm, kỹ thuật dữ liệu, khoa học dữ liệu, và kỹ thuật ML ngày càng mờ nhạt. Thực tế, giống như các tác giả, nhiều nhà khoa học dữ liệu được chuyển hóa thành kỹ sư dữ liệu qua một quá trình hữu cơ; được giao nhiệm vụ làm “khoa học dữ liệu” nhưng lại thiếu công cụ để làm công việc của mình, họ đảm nhận luôn công việc thiết kế và xây dựng các hệ thống để phục vụ vòng đời kỹ thuật dữ liệu. Khi sự đơn giản tiến lên ngăn xếp, các nhà khoa học dữ liệu sẽ dành một lát thời gian nhỏ hơn để thu thập và nhào nặn dữ liệu. Nhưng xu hướng này sẽ mở rộng vượt ra ngoài các nhà khoa học dữ liệu. Sự đơn giản hóa cũng có nghĩa là các kỹ sư dữ liệu sẽ dành ít thời gian hơn cho các tác vụ cấp thấp trong vòng đời kỹ thuật dữ liệu (quản lý máy chủ, cấu hình, v.v.), và kỹ thuật dữ liệu kiểu “doanh nghiệp” sẽ trở nên phổ biến hơn.

As data becomes more tightly embedded in every business's processes, new roles will emerge in the realm of data and algorithms. One possibility is a role that sits between ML engineering and data engineering. As ML toolsets become easier to use and managed cloud ML services grow in capabilities, ML is shifting away from ad hoc exploration and model development to become an operational discipline. This new ML-focused engineer who straddles this divide will know algorithms, ML techniques, model optimization, model monitoring, and data monitoring. However, their primary role will be to create or utilize the systems that automatically train models, monitor performance, and operationalize the full ML process for model types that are well understood. They will also monitor data pipelines and quality, overlapping into the current realm of data engineering. ML engineers will become more specialized to work on model types that are closer to research and less well understood. Another area in which titles may morph is at the intersection of software engineering and data engineering. Data applications, which blend traditional software applications with analytics, will drive this trend. Software engineers will need to have a much deeper understanding of data engineering. They will develop expertise in things like streaming, data pipelines, **data modeling**, and **data quality**. We will move beyond the “throw it over the wall” approach that is now pervasive. Data engineers will be integrated into application development teams, and software developers will acquire data engineering skills. The boundaries that exist between application backend systems and data engineering tools will be lowered as well, with deep integration through streaming and event-driven architectures.

Khi dữ liệu trở nên được nhúng chặt chẽ hơn vào mọi quy trình của doanh nghiệp, các vai trò mới sẽ xuất hiện trong lĩnh vực dữ liệu và thuật toán. Một khả năng là một vai trò nằm giữa kỹ thuật ML và kỹ thuật dữ liệu. Khi các bộ công cụ ML trở nên dễ sử dụng hơn và các dịch vụ ML đám mây được quản lý tăng trưởng về năng lực, ML đang dịch chuyển khỏi sự khám phá tùy hứng và phát triển mô hình để trở thành một bộ môn vận hành. Kỹ sư tập trung vào ML kiểu mới này, người trải mình qua ranh giới đó, sẽ am hiểu các thuật toán,

các kỹ thuật ML, tối ưu hóa mô hình, giám sát mô hình, và giám sát dữ liệu. Tuy nhiên, vai trò chính của họ sẽ là tạo ra hoặc sử dụng các hệ thống tự động huấn luyện mô hình, giám sát hiệu năng, và vận hành hóa toàn bộ quy trình ML cho các loại mô hình đã được hiểu rõ. Họ cũng sẽ giám sát các đường ống dữ liệu và chất lượng dữ liệu, chôn vùi vào lãnh địa hiện tại của kỹ thuật dữ liệu. Các kỹ sư ML sẽ trở nên chuyên môn hóa hơn để làm việc với các loại mô hình gắn với nghiên cứu hơn và ít được hiểu rõ hơn. Một lĩnh vực khác mà các chức danh có thể biến hóa là tại sự giao thoa giữa kỹ thuật phần mềm và kỹ thuật dữ liệu. Các ứng dụng dữ liệu, vốn pha trộn các ứng dụng phần mềm truyền thống với phân tích (analytics), sẽ dẫn dắt xu hướng này. Các kỹ sư phần mềm sẽ cần có một sự hiểu biết sâu sắc hơn nhiều về kỹ thuật dữ liệu. Họ sẽ phát triển chuyên môn về những thứ như streaming, các đường ống dữ liệu, mô hình hóa dữ liệu, và chất lượng dữ liệu. Chúng ta sẽ vượt qua cách tiếp cận “ném nó qua tường” hiện đang phổ biến. Các kỹ sư dữ liệu sẽ được tích hợp vào các đội phát triển ứng dụng, và các nhà phát triển phần mềm sẽ thu nạp các kỹ năng kỹ thuật dữ liệu. Các ranh giới tồn tại giữa các hệ thống backend của ứng dụng và các công cụ kỹ thuật dữ liệu cũng sẽ được hạ thấp, với sự tích hợp sâu thông qua các kiến trúc hướng sự kiện và streaming.

Moving Beyond the Modern Data Stack, Toward the Live Data Stack — Vượt ra ngoài Modern Data Stack, Hướng tới Live Data Stack

We'll be frank: the **modern data stack** (MDS) isn't so modern. We applaud the MDS for bringing a great selection of powerful data tools to the masses, lowering prices, and empowering data analysts to take control of their data stack. The rise of **ELT**, cloud data warehouses, and the abstraction of SaaS data pipelines certainly changed the game for many companies, opening up new powers for BI, analytics, and data science. Having said that, the MDS is basically a repackaging of old **data warehouse** practices using modern cloud and SaaS technologies; because the MDS is built around the cloud data warehouse paradigm, it has some serious limitations when compared to the potential of next-generation real-time data applications. From our point of view, the world is moving beyond the use of data-warehouse-based internal-facing analytics and data science, toward powering entire businesses and applications in real time with next generation real-time databases. What's driving this evolution? In many cases, analytics (BI and operational analytics) will be replaced by automation. Presently, most dashboards and reports answer questions concerning what and when. Ask yourself, “If I'm asking a what or when question, what action do I take next?” If the action is repetitive, it is a candidate for automation. Why look at a report to determine whether to take action when you can instead automate the action based on events as they occur? And it goes much further than this. Why does using a product like TikTok, Uber, Google, or DoorDash feel like magic? While it seems to you like a click of a button to watch a short video, order a ride or a meal, or find a search result, a lot is happening under the hood. These products are examples of true real-time data applications, delivering the actions you need at the click of a button while performing extremely sophisticated data processing and ML behind the scenes with miniscule **latency**. Presently, this level of sophistication is locked away behind custom-built technologies at large technology companies, but this sophistication and power are becoming democratized, similar to the way the MDS brought cloud-scale data warehouses and pipelines to the masses. The data world will soon go “live.”

Chúng tôi sẽ nói thẳng: ngăn xếp dữ liệu hiện đại (modern data stack, MDS) không hiện đại đến vậy. Chúng tôi tán dương MDS vì đã mang đến cho đại chúng một lựa chọn tuyệt vời các công cụ dữ liệu mạnh mẽ, hạ giá thành, và trao quyền cho các nhà phân tích dữ liệu nắm quyền kiểm soát ngăn xếp dữ liệu của họ. Sự trỗi dậy của ELT, các kho dữ liệu

(data warehouse) đám mây, và sự trừu tượng hóa của các đường ống dữ liệu SaaS chắc chắn đã thay đổi cuộc chơi cho nhiều công ty, mở ra những năng lực mới cho BI, phân tích, và khoa học dữ liệu. Dù vậy, MDS về cơ bản là một sự đóng gói lại các thông lệ kho dữ liệu cũ bằng các công nghệ đám mây và SaaS hiện đại; bởi vì MDS được xây dựng xoay quanh mô hình kho dữ liệu đám mây, nó có một số hạn chế nghiêm trọng khi so với tiềm năng của các ứng dụng dữ liệu thời gian thực thể hệ mới. Theo quan điểm của chúng tôi, thế giới đang vượt ra ngoài việc sử dụng phân tích và khoa học dữ liệu hướng nội dựa trên kho dữ liệu, hướng tới việc cấp năng lượng cho toàn bộ doanh nghiệp và ứng dụng trong thời gian thực bằng các database thời gian thực thể hệ mới. Điều gì đang dẫn dắt sự tiến hóa này? Trong nhiều trường hợp, phân tích (BI và phân tích vận hành) sẽ được thay thế bằng tự động hóa. Hiện tại, hầu hết các dashboard và báo cáo trả lời các câu hỏi liên quan đến cái gì và khi nào. Hãy tự hỏi: “Nếu tôi đang hỏi một câu hỏi cái gì hoặc khi nào, thì hành động tiếp theo tôi thực hiện là gì?” Nếu hành động đó lặp đi lặp lại, nó là một ứng viên cho tự động hóa. Tại sao phải nhìn vào một báo cáo để xác định có nên hành động hay không trong khi bạn có thể tự động hóa hành động đó dựa trên các sự kiện ngay khi chúng xảy ra? Và nó còn đi xa hơn thế này nhiều. Tại sao việc sử dụng một sản phẩm như TikTok, Uber, Google, hay DoorDash lại có cảm giác như phép thuật? Trong khi với bạn nó dường như chỉ là một cú nhấp nút để xem một video ngắn, đặt một chuyến xe hay một bữa ăn, hoặc tìm một kết quả tìm kiếm, thì rất nhiều thứ đang diễn ra dưới mui xe. Những sản phẩm này là các ví dụ về ứng dụng dữ liệu thời gian thực thực thụ, mang đến các hành động bạn cần chỉ bằng một cú nhấp nút trong khi thực hiện việc xử lý dữ liệu và ML cực kỳ tinh vi ở hậu trường với độ trễ (latency) cực nhỏ. Hiện tại, mức độ tinh vi này bị khóa kín đằng sau các công nghệ được xây dựng riêng tại các công ty công nghệ lớn, nhưng sự tinh vi và sức mạnh này đang được dân chủ hóa, tương tự cách MDS mang các kho dữ liệu và đường ống quy mô đám mây đến với đại chúng. Thế giới dữ liệu sẽ sớm trở nên “trực tiếp” (live).

The Live Data Stack — Live Data Stack

This democratization of real-time technologies will lead us to the successor to the MDS: the live data stack will soon be accessible and pervasive . The live data stack, depicted in Figure 11-1 , will fuse real-time analytics and ML into applications by using streaming technologies, covering the full data lifecycle from application source systems to data processing to ML, and back.

Sự dân chủ hóa các công nghệ thời gian thực này sẽ dẫn chúng ta đến kế kế nhiệm của MDS: live data stack sẽ sớm trở nên dễ tiếp cận và phổ biến rộng khắp. Live data stack, được mô tả trong Hình 11-1, sẽ hợp nhất phân tích thời gian thực và ML vào trong các ứng dụng bằng cách sử dụng các công nghệ streaming, bao trùm toàn bộ vòng đời dữ liệu từ các hệ thống nguồn (source system) của ứng dụng đến xử lý dữ liệu rồi đến ML, và quay trở lại.

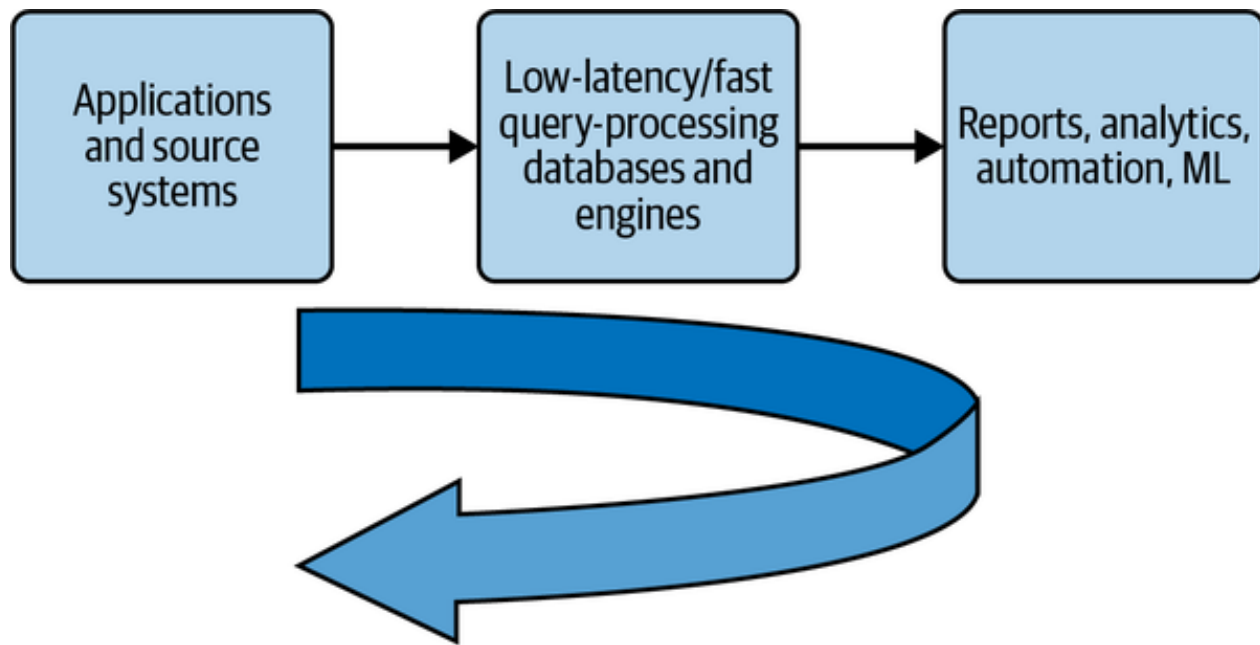


Figure 11-1. In the live data stack, data and intelligence moves in real time between the application and supporting systems. Just as the MDS took advantage of the cloud and brought on-premises data warehouse and pipeline technologies to the masses, the live data stack takes real-time data application technologies used at elite tech companies and makes them available to companies of all sizes as easy-to-use cloud-based offerings. This will open up a new world of possibilities for creating even better user experiences and business value.

Hình 11-1. Trong live data stack, dữ liệu và trí tuệ di chuyển trong thời gian thực giữa ứng dụng và các hệ thống hỗ trợ. Cũng như MDS đã tận dụng đám mây và mang các công nghệ kho dữ liệu cùng đường ống tại chỗ (on-premises) đến với đại chúng, live data stack lấy các công nghệ ứng dụng dữ liệu thời gian thực được dùng tại các công ty công nghệ ưu tú và làm cho chúng có sẵn cho các công ty thuộc mọi quy mô dưới dạng các sản phẩm trên nền đám mây để sử dụng. Điều này sẽ mở ra một thế giới mới của những khả năng để tạo ra những trải nghiệm người dùng và giá trị doanh nghiệp còn tốt hơn nữa.

Streaming Pipelines and Real-Time Analytical Databases — Đường ống Streaming và Database Phân tích Thời gian thực

The MDS limits itself to batch techniques that treat data as **bounded**. In contrast, real-time data applications treat data as an unbounded, continuous stream. Streaming pipelines and real-time analytical databases are the two core technologies that will facilitate the move from the MDS to the live data stack. While these technologies have been around for some time, rapidly maturing managed cloud services will see them be deployed much more widely. Streaming technologies will continue to see extreme growth for the foreseeable future. This will happen in conjunction with a clearer focus on the business utility of streaming data. Up to the present, streaming systems have frequently been treated like an expensive novelty or a dumb pipe for getting data from A to B. In the future, streaming will radically transform organizational technology and business processes; data architects and engineers will take the lead in these fundamental changes. Real-time analytical databases enable both fast **ingestion** and subsecond queries on this data. This data can be enriched or combined with historical datasets. When combined with a streaming pipeline and

automation, or dashboard that is capable of real-time analytics, a whole new level of possibilities opens up. No longer are you constrained by slow-running ELT processes, 15-minute updates, or other slow-moving parts. Data moves in a continuous flow. As streaming ingestion becomes more prevalent, batch ingestion will be less and less common. Why create a batch bottleneck at the head of your **data pipeline**? We'll eventually look at batch ingestion the same way we now look at dial-up modems. In conjunction with the rise of streams, we expect a back-to-the-future moment for data transformations. We'll shift away from ELT—in database transformations—to something that looks more like **ETL**. We provisionally refer to this as stream, transform, and load (STL). In a streaming context, extraction is an ongoing, continuous process. Of course, batch transformations won't entirely go away. Batch will still be very useful for model training, quarterly reporting, and more. But streaming **transformation** will become the norm. While the data warehouse and **data lake** are great for housing large amounts of data and performing ad hoc queries, they are not so well optimized for low-latency data ingestion or queries on rapidly moving data. The live data stack will be powered by **OLAP** databases that are purpose-built for streaming. Today, databases like Druid, ClickHouse, Rockset, and Firebolt are leading the way in powering the backend of the next generation of data applications. We expect that streaming technologies will continue to evolve rapidly and that new technologies will proliferate. Another area we think is ripe for disruption is data modeling, where there hasn't been serious innovation since the early 2000s. The traditional batch-oriented data modeling techniques you learned about in Chapter 8 aren't suited for streaming data. New data-modeling techniques will occur not within the data warehouse, but in the systems that generate the data. We expect data modeling will involve some notion of an **upstream** definitions layer—including semantics, metrics, lineage, and data definitions (see Chapter 9)—beginning where data is generated in the application. Modeling will also happen at every stage as data flows and evolves through the full lifecycle.

MDS tự giới hạn mình trong các kỹ thuật theo lô (batch) vốn coi dữ liệu là có biên. Ngược lại, các ứng dụng dữ liệu thời gian thực coi dữ liệu là một luồng không biên, liên tục. Các đường ống streaming và các database phân tích thời gian thực là hai công nghệ cốt lõi sẽ tạo điều kiện cho việc chuyển dịch từ MDS sang live data stack. Trong khi các công nghệ này đã tồn tại được một thời gian, các dịch vụ đám mây được quản lý đang nhanh chóng trưởng thành sẽ chứng kiến chúng được triển khai rộng rãi hơn nhiều. Các công nghệ streaming sẽ tiếp tục chứng kiến sự tăng trưởng cực mạnh trong tương lai gần. Điều này sẽ diễn ra song song với sự tập trung rõ ràng hơn vào tính hữu dụng kinh doanh của dữ liệu streaming. Cho đến hiện tại, các hệ thống streaming thường bị coi như một thứ đồ mới lạ đắt tiền hoặc một ống dẫn ngớ ngẩn chỉ để chuyển dữ liệu từ A đến B. Trong tương lai, streaming sẽ biến đổi triệt để công nghệ của tổ chức và các quy trình kinh doanh; các kiến trúc sư và kỹ sư dữ liệu sẽ dẫn dắt những thay đổi căn bản này. Các database phân tích thời gian thực cho phép cả việc thu nạp nhanh lẫn các truy vấn dưới một giây trên dữ liệu này. Dữ liệu này có thể được làm giàu hoặc kết hợp với các tập dữ liệu lịch sử. Khi kết hợp với một đường ống streaming và tự động hóa, hoặc một dashboard có khả năng phân tích thời gian thực, một cấp độ khả năng hoàn toàn mới sẽ được mở ra. Bạn không còn bị ràng buộc bởi các quy trình ELT chạy chậm, các bản cập nhật 15 phút một lần, hay các bộ phận di chuyển chậm khác. Dữ liệu di chuyển trong một dòng chảy liên tục. Khi thu nạp theo luồng (streaming ingestion) trở nên phổ biến hơn, thu nạp theo lô (batch ingestion) sẽ ngày càng ít gặp. Tại sao phải tạo ra một nút thắt cổ chai theo lô ở đầu đường ống dữ liệu của bạn? Cuối cùng chúng ta sẽ nhìn nhận thu nạp theo lô giống như cách chúng ta nhìn nhận các modem quay số bây giờ. Song song với sự trỗi dậy của các luồng, chúng tôi kỳ vọng một khoảnh khắc “về lại tương lai” đối với các phép biến đổi dữ liệu. Chúng ta sẽ dịch chuyển khỏi ELT—biến đổi bên trong database—sang một thứ trông giống ETL hơn. Chúng tôi tạm thời gọi đây là stream, transform, and load (STL). Trong ngữ cảnh streaming, trích xuất là một quá trình diễn ra liên tục, không ngừng. Tất nhiên, các phép biến đổi theo

lô sẽ không hoàn toàn biến mất. Batch vẫn sẽ rất hữu ích cho việc huấn luyện mô hình, báo cáo theo quý, và nhiều thứ khác. Nhưng biến đổi theo luồng sẽ trở thành chuẩn mực. Trong khi kho dữ liệu và hồ dữ liệu (data lake) rất tốt cho việc chứa các lượng lớn dữ liệu và thực hiện các truy vấn tùy hứng, chúng lại không được tối ưu hóa tốt lắm cho việc thu nạp dữ liệu độ trễ thấp hoặc các truy vấn trên dữ liệu di chuyển nhanh. Live data stack sẽ được cấp năng lượng bởi các database OLAP được xây dựng có chủ đích cho streaming. Ngày nay, các database như Druid, ClickHouse, Rockset, và Firebolt đang dẫn đầu trong việc cấp năng lượng cho backend của thể hệ ứng dụng dữ liệu kế tiếp. Chúng tôi kỳ vọng rằng các công nghệ streaming sẽ tiếp tục tiến hóa nhanh chóng và rằng các công nghệ mới sẽ sinh sôi. Một lĩnh vực khác mà chúng tôi cho là đã chín muồi để bị phá vỡ là mô hình hóa dữ liệu, nơi chưa có sự đổi mới nghiêm túc nào kể từ đầu những năm 2000. Các kỹ thuật mô hình hóa dữ liệu hướng lô truyền thống mà bạn đã học trong Chương 8 không phù hợp với dữ liệu streaming. Các kỹ thuật mô hình hóa dữ liệu mới sẽ diễn ra không phải bên trong kho dữ liệu, mà trong các hệ thống sinh ra dữ liệu. Chúng tôi kỳ vọng rằng mô hình hóa dữ liệu sẽ bao hàm một khái niệm nào đó về một tầng định nghĩa ở thượng nguồn (upstream)—bao gồm ngữ nghĩa, các chỉ số, truy vết nguồn gốc, và các định nghĩa dữ liệu (xem Chương 9)—bắt đầu từ nơi dữ liệu được sinh ra trong ứng dụng. Mô hình hóa cũng sẽ diễn ra ở mọi giai đoạn khi dữ liệu chảy và tiến hóa qua toàn bộ vòng đời.

The Fusion of Data with Applications — Sự hợp nhất của Dữ liệu với Ứng dụng

We expect the next revolution will be the fusion of the application and data layers. Right now, applications sit in one area, and the MDS sits in another. To make matters worse, data is created with no regard for how it will be used for analytics. Consequently, lots of duct tape is needed to make systems talk with one another. This patchwork, siloed setup is awkward and ungainly. Soon, application stacks will be data stacks, and vice versa. Applications will integrate real-time automation and decision making, powered by the streaming pipelines and ML. The data engineering lifecycle won't necessarily change, but the time between stages of the lifecycle will drastically shorten. A lot of innovation will occur in new technologies and practices that will improve the experience of engineering the live data stack. Pay attention to emerging database technologies designed to address the mix of **OLTP** and OLAP use cases; feature stores may also play a similar role for ML use cases.

Chúng tôi kỳ vọng cuộc cách mạng kế tiếp sẽ là sự hợp nhất của tầng ứng dụng và tầng dữ liệu. Ngay lúc này, các ứng dụng nằm ở một khu vực, còn MDS nằm ở một khu vực khác. Tệ hơn nữa, dữ liệu được tạo ra mà không hề quan tâm đến việc nó sẽ được dùng cho phân tích như thế nào. Hệ quả là cần rất nhiều băng dính để khiến các hệ thống nói chuyện được với nhau. Sự sắp đặt chắp vá, tách rời này thật vụng về và lóng ngóng. Chẳng bao lâu nữa, các ngăn xếp ứng dụng sẽ là các ngăn xếp dữ liệu, và ngược lại. Các ứng dụng sẽ tích hợp tự động hóa và ra quyết định trong thời gian thực, được cấp năng lượng bởi các đường ống streaming và ML. Vòng đời kỹ thuật dữ liệu không nhất thiết thay đổi, nhưng thời gian giữa các giai đoạn của vòng đời sẽ rút ngắn đi đáng kể. Rất nhiều đổi mới sẽ diễn ra trong các công nghệ và thông lệ mới giúp cải thiện trải nghiệm xây dựng live data stack. Hãy chú ý đến các công nghệ database mới nổi được thiết kế để giải quyết hỗn hợp các trường hợp sử dụng OLTP và OLAP; các feature store cũng có thể đóng một vai trò tương tự cho các trường hợp sử dụng ML.

The Tight Feedback Between Applications and ML — Vòng Phản hồi Chặt chẽ giữa Ứng dụng và ML

Another area we're excited about is the fusion of applications and ML. Today, applications and ML are disjointed systems, like applications and analytics. Software engineers do their thing over here, data scientists and ML engineers do their thing over there. ML is well-suited for scenarios where data is generated at such a high rate and volume that humans cannot feasibly process it by hand. As data sizes and velocity grow, this applies to every scenario. High volumes of fast-moving data, coupled with sophisticated workflows and actions, are candidates for ML. As data feedback loops become shorter, we expect most applications to integrate ML. As data moves more quickly, the feedback loop between applications and ML will tighten. The applications in the live data stack are intelligent and able to adapt in real time to changes in the data. This creates a cycle of ever-smarter applications and increasing business value.

Một lĩnh vực khác mà chúng tôi hào hứng là sự hợp nhất của các ứng dụng và ML. Ngày nay, các ứng dụng và ML là các hệ thống rời rạc, giống như các ứng dụng và phân tích. Các kỹ sư phần mềm làm việc của họ ở bên này, các nhà khoa học dữ liệu và kỹ sư ML làm việc của họ ở bên kia. ML rất phù hợp với các kịch bản nơi dữ liệu được sinh ra với tốc độ và khối lượng cao đến mức con người không thể xử lý thủ công một cách khả thi. Khi kích cỡ và vận tốc dữ liệu tăng lên, điều này áp dụng cho mọi kịch bản. Khối lượng lớn dữ liệu di chuyển nhanh, kết hợp với các quy trình làm việc và hành động tinh vi, là các ứng viên cho ML. Khi các vòng phản hồi dữ liệu trở nên ngắn hơn, chúng tôi kỳ vọng hầu hết các ứng dụng sẽ tích hợp ML. Khi dữ liệu di chuyển nhanh hơn, vòng phản hồi giữa các ứng dụng và ML sẽ thắt chặt lại. Các ứng dụng trong live data stack thông minh và có khả năng thích nghi trong thời gian thực với những thay đổi trong dữ liệu. Điều này tạo ra một chu trình các ứng dụng ngày càng thông minh hơn và giá trị doanh nghiệp ngày càng tăng.

Dark Matter Data and the Rise of...Spreadsheets?! — Dữ liệu Vật chất Tối và Sự trỗi dậy của... Bảng tính?!

We've talked about fast-moving data and how feedback loops will shrink as applications, data, and ML work more closely together. This section might seem odd, but we need to address something that's widely ignored in today's data world, especially by engineers. What's the most widely used **data platform**? It's the humble spreadsheet. Depending on the estimates you read, the user base of spreadsheets is between 700 million and 2 billion people. Spreadsheets are the dark matter of the data world. A good deal of data analytics runs in spreadsheets and never makes its way into the sophisticated data systems that we describe in this book. In many organizations, spreadsheets handle financial reporting, supply-chain analytics, and even CRM. At heart, what is a spreadsheet? A spreadsheet is an interactive data application that supports complex analytics. Unlike purely code-based tools such as pandas (Python Data Analysis Library), spreadsheets are accessible to a whole spectrum of users, ranging from those who just know how to open files and look at reports to power users who can script sophisticated procedural data processing. So far, BI tools have failed to bring comparable interactivity to databases. Users who interact with the UI are typically limited to slicing and dicing data within certain guardrails, not general-purpose programmable analytics. We predict that a new class of tools will emerge that combines the interactive analytics capabilities of a spreadsheet with the backend power of cloud OLAP systems. Indeed, some candidates are already in the running. The ultimate winner in this product category may continue to use spreadsheet paradigms, or may define entirely new interface idioms for interacting with data.

Chúng ta đã nói về dữ liệu di chuyển nhanh và cách các vòng phản hồi sẽ thu hẹp lại khi các ứng dụng, dữ liệu, và ML làm việc gần gũi hơn với nhau. Mục này có vẻ kỳ lạ, nhưng chúng ta cần đề cập đến một thứ bị bỏ qua rộng rãi trong thế giới dữ liệu ngày nay, đặc biệt là bởi các kỹ sư. Nền tảng dữ liệu được sử dụng rộng rãi nhất là gì? Đó là bảng tính khiêm tốn. Tùy vào ước tính bạn đọc, lượng người dùng bảng tính nằm trong khoảng từ 700 triệu đến 2 tỷ người. Bảng tính là vật chất tối của thế giới dữ liệu. Một lượng lớn hoạt động phân tích dữ liệu chạy trong các bảng tính và không bao giờ tìm được đường vào các hệ thống dữ liệu tinh vi mà chúng tôi mô tả trong cuốn sách này. Trong nhiều tổ chức, các bảng tính xử lý báo cáo tài chính, phân tích chuỗi cung ứng, và thậm chí cả CRM. Về bản chất, một bảng tính là gì? Một bảng tính là một ứng dụng dữ liệu tương tác hỗ trợ phân tích phức tạp. Không giống các công cụ thuần dựa trên mã như pandas (Python Data Analysis Library), các bảng tính có thể tiếp cận được bởi cả một dải người dùng đa dạng, từ những người chỉ biết cách mở tệp và xem báo cáo cho đến những người dùng cao cấp có thể viết kịch bản xử lý dữ liệu theo thủ tục tinh vi. Cho đến nay, các công cụ BI đã thất bại trong việc mang lại tính tương tác tương đương cho các database. Những người dùng tương tác với giao diện thường bị giới hạn ở việc cắt lát và xẻ nhỏ dữ liệu trong một số rào chắn nhất định, chứ không phải phân tích lập trình được cho mục đích tổng quát. Chúng tôi dự đoán rằng một lớp công cụ mới sẽ xuất hiện, kết hợp các năng lực phân tích tương tác của một bảng tính với sức mạnh backend của các hệ thống OLAP đám mây. Quả thực, một số ứng viên đã có mặt trong cuộc đua. Kẻ chiến thắng cuối cùng trong hạng mục sản phẩm này có thể tiếp tục sử dụng các mô hình bảng tính, hoặc có thể định nghĩa những thành ngữ giao diện hoàn toàn mới để tương tác với dữ liệu.

Conclusion — Kết luận

Thank you for joining us on this journey through data engineering! We traversed good architecture, the stages of the data engineering lifecycle, and security best practices. We've discussed strategies for choosing technologies at a time when our field continues to change at an extraordinary pace. In this chapter, we laid out our wild speculation about the near and intermediate future. Some aspects of our prognostication sit on a relatively secure footing. The simplification of managed tooling and the rise of “enterprisey” data engineering have proceeded day by day as we've written this book. Other predictions are much more speculative in nature; we see hints of an emerging live data stack, but this entails a significant paradigm shift for both individual engineers and the organizations that employ them. Perhaps the trend toward real-time data will stall once again, with most companies continuing to focus on basic batch processing. Surely, other trends exist that we have completely failed to identify. The evolution of technology involves complex interactions of technology and culture. Both are unpredictable. Data engineering is a vast topic; while we could not go into any technical depth in individual areas, we hope that we have succeeded in creating a kind of travel guide that will help current data engineers, future data engineers, and those who work adjacent to the field to find their way in a domain that is in flux. We advise you to continue exploration on your own. As you discover interesting topics and ideas in this book, continue the conversation as part of a community. Identify domain experts who can help you to uncover the strengths and pitfalls of trendy technologies and practices. Read extensively from the latest books, blog posts, and papers. Participate in meetups and listen to talks. Ask questions and share your own expertise. Keep an eye on vendor announcements to stay abreast of the latest developments, taking all claims with a healthy **grain** of salt. Through this process, you can choose technology. Next, you will need to adopt technology and develop expertise, perhaps as an individual contributor, perhaps within your team as a lead, perhaps across an entire technology organization. As you do this, don't lose sight of the larger goals of data engineering. Focus on the lifecycle, on **serving** your customers— internal and

external—on your business, on serving and on your larger goals. Regarding the future, many of you will play a role in determining what comes next. Technology trends are defined not only by those who create the underlying technology, but by those who adopt it and put it to good use. Successful tool use is as critical as tool creation . Find opportunities to apply real-time technology that will improve the user experience, create value, and define entirely new types of applications. It is this kind of practical application that will materialize the live data stack as a new industry standard; or perhaps some other new technology trend that we failed to identify will win the day. Finally, we wish you an exciting career! We chose to work in data engineering, to consult, and to write this book not simply because it was trendy, but because it was fascinating. We hope that we've managed to convey to you a bit of the joy we've found working in this field.

Cảm ơn bạn đã đồng hành cùng chúng tôi trong hành trình qua kỹ thuật dữ liệu này! Chúng ta đã đi qua kiến trúc tốt, các giai đoạn của vòng đời kỹ thuật dữ liệu, và các thông lệ tốt nhất về an ninh. Chúng ta đã thảo luận các chiến lược chọn công nghệ vào một thời điểm khi lĩnh vực của chúng ta tiếp tục thay đổi với tốc độ phi thường. Trong chương này, chúng tôi đã trình bày những suy đoán táo bạo của mình về tương lai gần và trung hạn. Một số khía cạnh trong những tiên đoán của chúng tôi đứng trên một nền móng tương đối vững chắc. Sự đơn giản hóa của công cụ được quản lý và sự trỗi dậy của kỹ thuật dữ liệu kiểu “doanh nghiệp” đã diễn ra ngày qua ngày trong khi chúng tôi viết cuốn sách này. Những dự đoán khác mang bản chất suy đoán hơn nhiều; chúng tôi thấy những gợi ý về một live data stack đang nổi lên, nhưng điều này kéo theo một sự dịch chuyển mô hình đáng kể cho cả từng kỹ sư cá nhân lẫn các tổ chức tuyển dụng họ. Có lẽ xu hướng hướng tới dữ liệu thời gian thực sẽ một lần nữa khựng lại, với hầu hết các công ty tiếp tục tập trung vào xử lý theo lô cơ bản. Chắc chắn, còn tồn tại các xu hướng khác mà chúng tôi đã hoàn toàn không nhận ra được. Sự tiến hóa của công nghệ liên quan đến những tương tác phức tạp giữa công nghệ và văn hóa. Cả hai đều khó dự đoán. Kỹ thuật dữ liệu là một chủ đề rộng lớn; trong khi chúng tôi không thể đi vào bất kỳ chiều sâu kỹ thuật nào ở từng lĩnh vực riêng lẻ, chúng tôi hy vọng rằng mình đã thành công trong việc tạo ra một dạng cẩm nang du lịch sẽ giúp các kỹ sư dữ liệu hiện tại, các kỹ sư dữ liệu tương lai, và những người làm việc cận kề lĩnh vực này tìm được đường đi của mình trong một địa hạt đang biến động. Chúng tôi khuyên bạn hãy tiếp tục khám phá theo cách riêng của mình. Khi bạn phát hiện ra những chủ đề và ý tưởng thú vị trong cuốn sách này, hãy tiếp tục cuộc đối thoại như một phần của một cộng đồng. Hãy xác định các chuyên gia lĩnh vực, những người có thể giúp bạn khám phá ra các điểm mạnh và cạm bẫy của các công nghệ và thông lệ thời thượng. Hãy đọc rộng rãi từ những cuốn sách, bài viết blog, và bài báo mới nhất. Hãy tham gia các buổi gặp mặt và lắng nghe các bài nói. Hãy đặt câu hỏi và chia sẻ chuyên môn của riêng bạn. Hãy để mắt đến các thông báo của nhà cung cấp để cập nhật những diễn biến mới nhất, đón nhận mọi tuyên bố với một thái độ hoài nghi lành mạnh. Thông qua quá trình này, bạn có thể chọn công nghệ. Tiếp theo, bạn sẽ cần áp dụng công nghệ và phát triển chuyên môn, có lẽ với tư cách một người đóng góp cá nhân, có lẽ trong nhóm của bạn với vai trò trưởng nhóm, có lẽ trên toàn bộ một tổ chức công nghệ. Khi bạn làm điều này, đừng đánh mất tầm nhìn về các mục tiêu lớn hơn của kỹ thuật dữ liệu. Hãy tập trung vào vòng đời, vào việc phục vụ khách hàng của bạn—nội bộ và bên ngoài—vào doanh nghiệp của bạn, vào việc phục vụ và vào các mục tiêu lớn hơn của bạn. Về tương lai, nhiều người trong số các bạn sẽ đóng một vai trò trong việc quyết định điều gì sẽ đến tiếp theo. Các xu hướng công nghệ được định hình không chỉ bởi những người tạo ra công nghệ nền tảng, mà còn bởi những người áp dụng nó và đưa nó vào sử dụng tốt. Việc sử dụng công cụ thành công cũng quan trọng như việc tạo ra công cụ. Hãy tìm các cơ hội để áp dụng công nghệ thời gian thực nhằm cải thiện trải nghiệm người dùng, tạo ra giá trị, và định nghĩa những loại ứng dụng hoàn toàn mới. Chính loại ứng dụng thực tế này sẽ hiện thực hóa live data stack thành một chuẩn mực mới của ngành; hoặc có lẽ một xu hướng công nghệ mới nào đó

khác mà chúng tôi đã không nhận ra được sẽ chiến thắng. Cuối cùng, chúng tôi chúc bạn một sự nghiệp đầy hứng khởi! Chúng tôi chọn làm việc trong kỹ thuật dữ liệu, làm tư vấn, và viết cuốn sách này không đơn thuần vì nó thời thượng, mà vì nó hấp dẫn. Chúng tôi hy vọng rằng mình đã truyền tải được đến bạn một chút niềm vui mà chúng tôi đã tìm thấy khi làm việc trong lĩnh vực này.

1 Benn Stancil, “The Data OS,” benn.substack , September 3, 2021, <https://oreil.ly/HetE9> .

1 Benn Stancil, “The Data OS,” benn.substack, September 3, 2021, <https://oreil.ly/HetE9>.

2 Ben Rogojan, “Three Data Engineering Experts Share Their Thoughts on Where Data Is Headed,” Better Programming , May 27, 2021, <https://oreil.ly/IsY4W> .

2 Ben Rogojan, “Three Data Engineering Experts Share Their Thoughts on Where Data Is Headed,” Better Programming, May 27, 2021, <https://oreil.ly/IsY4W>.

Appendix A. Serialization and Compression Technical Details — Phụ lục A. Chi tiết kỹ thuật về Tuần tự hóa và Nén

Data engineers working in the cloud are generally freed from the complexities of managing **object storage** systems. Still, they need to understand details of **serialization** and deserialization formats. As we mentioned in Chapter 6 about **storage** raw ingredients, serialization and **compression** algorithms go hand in hand.

Các kỹ sư dữ liệu làm việc trên đám mây nhìn chung được giải phóng khỏi những phức tạp của việc quản lý các hệ thống lưu trữ đối tượng (object storage). Tuy vậy, họ vẫn cần hiểu chi tiết về các định dạng tuần tự hóa (serialization) và giải tuần tự hóa (deserialization). Như chúng tôi đã đề cập ở Chương 6 về các nguyên liệu thô của lưu trữ, các thuật toán tuần tự hóa và nén (compression) luôn đi đôi với nhau.

Serialization Formats — Các định dạng Tuần tự hóa

Many serialization algorithms and formats are available to data engineers. While the abundance of options is a significant source of pain in **data engineering**, they are also a massive opportunity for performance improvements. We’ve sometimes seen job performance improve by a factor of 100 simply by switching from CSV to Parquet serialization. As data moves through a pipeline, engineers will also manage reserialization— conversion from one format to another. Sometimes data engineers have no choice but to accept data in an ancient, nasty form; they must design processes to deserialize this format and handle exceptions, and then clean up and convert data for consistent, fast **downstream** processing and consumption.

Có nhiều thuật toán và định dạng tuần tự hóa dành cho kỹ sư dữ liệu. Mặc dù sự dồi dào các lựa chọn là một nguồn gây đau đầu đáng kể trong kỹ thuật dữ liệu (data engineering), nó cũng là một cơ hội to lớn để cải thiện hiệu năng. Đôi khi chúng tôi đã thấy hiệu năng của một tác vụ cải thiện gấp 100 lần chỉ đơn giản bằng cách chuyển từ tuần tự hóa CSV sang Parquet. Khi dữ liệu di chuyển qua một đường ống (pipeline), kỹ sư cũng sẽ phải quản lý việc tái tuần tự hóa — chuyển đổi từ định dạng này sang định dạng khác. Đôi khi kỹ sư dữ liệu không còn lựa chọn nào khác ngoài việc chấp nhận dữ liệu ở một dạng cổ lỗ, khó chịu; họ phải thiết kế các quy trình để giải tuần tự hóa định dạng này và xử lý các ngoại lệ, rồi làm sạch và chuyển đổi dữ liệu để xử lý và tiêu thụ ở hạ nguồn (downstream) một

cách nhất quán, nhanh chóng.

Row-Based Serialization — Tuần tự hóa theo Hàng

As its name suggests, row-based serialization organizes data by row. CSV format is an archetypal row-based format. For semistructured data (data objects that support nesting and **schema** variation), row-oriented serialization entails storing each object as a unit.

Đúng như tên gọi, tuần tự hóa theo hàng tổ chức dữ liệu theo từng hàng. Định dạng CSV là một định dạng theo hàng điển hình. Với dữ liệu bán cấu trúc (các đối tượng dữ liệu hỗ trợ lồng nhau và biến đổi lược đồ), tuần tự hóa theo hàng đòi hỏi lưu trữ mỗi đối tượng như một đơn vị.

CSV: The nonstandard standard — CSV: Chuẩn không chuẩn

We discussed CSV in Chapter 7. CSV is a serialization format that data engineers love to hate. The term CSV is essentially a catchall for delimited text, but there is flexibility in conventions of escaping, quote characters, delimiter, and more. Data engineers should avoid using CSV files in pipelines because they are highly error-prone and deliver poor performance. Engineers are often required to use CSV format to exchange data with systems and business processes outside their control. CSV is a common format for data archival. If you use CSV for archival, include a complete technical description of the serialization configuration for your files so that future consumers can ingest the data.

Chúng ta đã thảo luận về CSV ở Chương 7. CSV là một định dạng tuần tự hóa mà các kỹ sư dữ liệu vừa yêu vừa ghét. Thuật ngữ CSV về cơ bản là một cái tên chung cho văn bản phân tách bằng ký tự ngăn cách, nhưng có sự linh hoạt trong các quy ước về thoát ký tự (escaping), ký tự trích dẫn, dấu phân cách và nhiều thứ khác. Kỹ sư dữ liệu nên tránh sử dụng tệp CSV trong các đường ống vì chúng rất dễ phát sinh lỗi và cho hiệu năng kém. Kỹ sư thường buộc phải dùng định dạng CSV để trao đổi dữ liệu với các hệ thống và quy trình nghiệp vụ nằm ngoài tầm kiểm soát của họ. CSV là một định dạng phổ biến để lưu trữ dữ liệu (archival). Nếu bạn dùng CSV để lưu trữ, hãy kèm theo một mô tả kỹ thuật đầy đủ về cấu hình tuần tự hóa cho các tệp của bạn để những bên tiêu thụ trong tương lai có thể thu nạp (ingest) dữ liệu.

XML — XML

Extensible Markup Language (XML) was popular when HTML and the internet were new, but is now viewed as legacy; it is generally slow to deserialize and serialize for data engineering applications. XML is another standard that data engineers are often forced to interact with as they exchange data with legacy systems and software. JSON has largely replaced XML for plain-text object serialization.

Ngôn ngữ Đánh dấu Mở rộng (Extensible Markup Language - XML) từng phổ biến khi HTML và internet còn mới mẻ, nhưng nay được xem là di sản (legacy); nó nhìn chung chậm trong việc giải tuần tự hóa và tuần tự hóa cho các ứng dụng kỹ thuật dữ liệu. XML là một chuẩn khác mà kỹ sư dữ liệu thường buộc phải tương tác khi họ trao đổi dữ liệu với các hệ thống và phần mềm di sản. JSON phần lớn đã thay thế XML cho việc tuần tự hóa đối tượng dưới dạng văn bản thuần.

JSON and JSONL — JSON và JSONL

JavaScript Object Notation (JSON) has emerged as the new standard for data exchange over APIs, and it has also become an extremely popular format for data storage. In the context of databases, the popularity of JSON has grown apace with the rise of MongoDB and other document stores. Databases such as Snowflake, BigQuery, and SQL Server also offer extensive native support, facilitating easy data exchange between applications, APIs, and database systems. JSON Lines (JSONL) is a specialized version of JSON for storing bulk semistructured data in files. JSONL stores a sequence of JSON objects, with objects delimited by line breaks. From our perspective, JSONL is an extremely useful format for storing data right after it is ingested from API or applications. However, many columnar formats offer significantly better performance. Consider moving to another format for intermediate pipeline stages and **serving**.

JavaScript Object Notation (JSON) đã nổi lên như chuẩn mới cho trao đổi dữ liệu qua các API, và nó cũng đã trở thành một định dạng cực kỳ phổ biến để lưu trữ dữ liệu. Trong bối cảnh các cơ sở dữ liệu, sự phổ biến của JSON đã tăng song hành với sự trỗi dậy của MongoDB và các kho tài liệu (document store) khác. Các cơ sở dữ liệu như Snowflake, BigQuery và SQL Server cũng cung cấp hỗ trợ gốc rộng rãi, tạo điều kiện trao đổi dữ liệu dễ dàng giữa các ứng dụng, API và hệ thống cơ sở dữ liệu. JSON Lines (JSONL) là một phiên bản chuyên biệt của JSON để lưu trữ dữ liệu bán cấu trúc số lượng lớn trong các tệp. JSONL lưu một chuỗi các đối tượng JSON, với các đối tượng được phân tách bằng ngắt dòng. Theo góc nhìn của chúng tôi, JSONL là một định dạng cực kỳ hữu ích để lưu trữ dữ liệu ngay sau khi nó được thu nạp từ API hoặc ứng dụng. Tuy nhiên, nhiều định dạng theo cột (columnar) cho hiệu năng tốt hơn đáng kể. Hãy cân nhắc chuyển sang một định dạng khác cho các giai đoạn trung gian của đường ống và cho việc phục vụ (serving).

Avro — Avro

Avro is a row-oriented data format designed for RPCs and data serialization. Avro encodes data into a binary format, with schema **metadata** specified in JSON. Avro is popular in the Hadoop ecosystem and is also supported by various cloud data tools.

Avro là một định dạng dữ liệu theo hàng được thiết kế cho các RPC và tuần tự hóa dữ liệu. Avro mã hóa dữ liệu thành định dạng nhị phân, với siêu dữ liệu (metadata) lược đồ được đặc tả bằng JSON. Avro phổ biến trong hệ sinh thái Hadoop và cũng được nhiều công cụ dữ liệu đám mây hỗ trợ.

Columnar Serialization — Tuần tự hóa theo Cột

The serialization formats we've discussed so far are row-oriented. Data is encoded as complete relations (CSV) or documents (XML and JSON), and these are written into files sequentially. With columnar serialization, data organization is essentially pivoted by storing each column into its own set of files. One obvious advantage to **columnar storage** is that it allows us to read data from only a subset of fields rather than having to read full rows at once. This is a common scenario in **analytics** applications and can dramatically reduce the amount of data that must be scanned to execute a query. Storing data as columns also puts similar values next to each other, allowing us to encode columnar data efficiently. One common technique involves looking for repeated values and tokenizing these, a simple but highly efficient compression method for columns with large numbers of repeats. Even when columns don't contain large numbers of repeated values, they may manifest

high redundancy. Suppose that we organized customer support messages into a single column of data. We likely see the same themes and verbiage again and again across these messages, allowing data compression algorithms to realize a high ratio. For this reason, columnar storage is usually combined with compression, allowing us to maximize disk and network bandwidth resources.

Các định dạng tuần tự hóa mà chúng ta đã thảo luận đến giờ đều là theo hàng. Dữ liệu được mã hóa thành các quan hệ hoàn chỉnh (CSV) hoặc tài liệu (XML và JSON), và những thứ này được ghi tuần tự vào các tệp. Với tuần tự hóa theo cột (columnar serialization), cách tổ chức dữ liệu về cơ bản được xoay ngang bằng cách lưu mỗi cột vào một tập tệp riêng của nó. Một lợi thế rõ ràng của lưu trữ theo cột (columnar) là nó cho phép chúng ta chỉ đọc dữ liệu từ một tập con các trường thay vì phải đọc toàn bộ các hàng cùng lúc. Đây là một kịch bản phổ biến trong các ứng dụng phân tích (analytics) và có thể giảm đáng kể lượng dữ liệu phải quét để thực thi một truy vấn. Lưu dữ liệu theo cột cũng đặt các giá trị tương tự cạnh nhau, cho phép chúng ta mã hóa dữ liệu cột một cách hiệu quả. Một kỹ thuật phổ biến là tìm các giá trị lặp lại và token hóa chúng, một phương pháp nén đơn giản nhưng cực kỳ hiệu quả cho các cột có số lượng lớn các giá trị lặp. Ngay cả khi các cột không chứa số lượng lớn giá trị lặp, chúng vẫn có thể biểu lộ độ dư thừa cao. Giả sử chúng ta tổ chức các tin nhắn hỗ trợ khách hàng vào một cột dữ liệu duy nhất. Chúng ta có khả năng thấy đi thấy lại cùng những chủ đề và lối diễn đạt trong các tin nhắn này, cho phép các thuật toán nén dữ liệu đạt tỷ lệ cao. Vì lý do này, lưu trữ theo cột thường được kết hợp với nén, cho phép chúng ta tối đa hóa tài nguyên băng thông đĩa và mạng.

Columnar storage and compression come with some disadvantages too. We cannot easily access individual data records; we must reconstruct records by reading data from several column files. Record updates are also challenging. To change one field in one record, we must decompress the column file, modify it, recompress it, and write it back to storage. To avoid rewriting full columns on each update, columns are broken into many files, typically using **partitioning** and **clustering** strategies that organize data according to query and update patterns for the table. Even so, the overhead for updating a single row is horrendous. Columnar databases are a terrible fit for transactional workloads, so transactional databases generally utilize some form of row- or record-oriented storage.

Lưu trữ theo cột và nén cũng đi kèm một số nhược điểm. Chúng ta không thể truy cập từng bản ghi dữ liệu riêng lẻ một cách dễ dàng; chúng ta phải tái dựng các bản ghi bằng cách đọc dữ liệu từ nhiều tệp cột. Việc cập nhật bản ghi cũng đầy thách thức. Để thay đổi một trường trong một bản ghi, chúng ta phải giải nén tệp cột, sửa đổi nó, nén lại và ghi nó trở lại bộ lưu trữ. Để tránh phải ghi lại toàn bộ cột mỗi lần cập nhật, các cột được chia thành nhiều tệp, thường dùng các chiến lược phân vùng (partition) và gom cụm (clustering) tổ chức dữ liệu theo các mẫu truy vấn và cập nhật của bảng. Dù vậy, chi phí phụ trội để cập nhật một hàng đơn lẻ là khủng khiếp. Các cơ sở dữ liệu theo cột rất không phù hợp cho khối lượng công việc giao dịch (transactional), nên các cơ sở dữ liệu giao dịch nhìn chung sử dụng một dạng lưu trữ theo hàng hoặc theo bản ghi nào đó.

Parquet — Parquet

Parquet stores data in a columnar format and is designed to realize excellent read and write performance in a **data lake** environment. Parquet solves a few problems that frequently bedevil data engineers. Parquet-encoded data builds in schema information and natively supports nested data, unlike CSV. Furthermore, Parquet is portable; while databases such as BigQuery and Snowflake serialize data in proprietary columnar formats and offer excellent query performance on data stored internally, a huge performance hit occurs when interoperating with external tools. Data must be

deserialized, reserialized into an exchangeable format, and exported to use data lake tools such as Spark and Presto. Parquet files in a data lake may be a superior option to proprietary cloud data warehouses in a polyglot tool environment. Parquet format is used with various compression algorithms; speed optimized compression algorithms such as Snappy (discussed later in this appendix) are especially popular.

Parquet lưu dữ liệu ở định dạng theo cột và được thiết kế để đạt hiệu năng đọc và ghi xuất sắc trong môi trường hồ dữ liệu (data lake). Parquet giải quyết một vài vấn đề thường xuyên gây khổ sở cho kỹ sư dữ liệu. Dữ liệu mã hóa bằng Parquet tích hợp sẵn thông tin lược đồ và hỗ trợ gộp dữ liệu lồng nhau, không như CSV. Hơn nữa, Parquet có tính khả chuyển; trong khi các cơ sở dữ liệu như BigQuery và Snowflake tuân tự hóa dữ liệu ở các định dạng cột độc quyền và cho hiệu năng truy vấn xuất sắc trên dữ liệu lưu nội bộ, một sự sụt giảm hiệu năng lớn xảy ra khi liên thông với các công cụ bên ngoài. Dữ liệu phải được giải tuân tự hóa, tái tuân tự hóa thành một định dạng có thể trao đổi, và xuất ra để dùng với các công cụ hồ dữ liệu như Spark và Presto. Các tệp Parquet trong một hồ dữ liệu có thể là lựa chọn vượt trội so với các kho dữ liệu (data warehouse) đám mây độc quyền trong một môi trường công cụ đa ngôn ngữ (polyglot). Định dạng Parquet được dùng với nhiều thuật toán nén khác nhau; các thuật toán nén tối ưu cho tốc độ như Snappy (thảo luận sau trong phụ lục này) đặc biệt phổ biến.

ORC — ORC

Optimized Row Columnar (ORC) is a columnar storage format similar to Parquet. ORC was very popular for use with Apache Hive; while still widely used, we generally see it much less than Apache Parquet, and it enjoys somewhat less support in modern cloud ecosystem tools. For example, Snowflake and BigQuery support Parquet file import and export; while they can read from ORC files, neither tool can export to ORC.

Optimized Row Columnar (ORC) là một định dạng lưu trữ theo cột tương tự Parquet. ORC từng rất phổ biến để dùng với Apache Hive; tuy vẫn được sử dụng rộng rãi, chúng tôi nhìn chung thấy nó ít hơn nhiều so với Apache Parquet, và nó được hỗ trợ phần nào ít hơn trong các công cụ của hệ sinh thái đám mây hiện đại. Ví dụ, Snowflake và BigQuery hỗ trợ nhập và xuất tệp Parquet; trong khi chúng có thể đọc từ các tệp ORC, không công cụ nào có thể xuất ra ORC.

Apache Arrow or in-memory serialization — Apache Arrow hay tuân tự hóa trong bộ nhớ

When we introduced serialization as a storage raw ingredient at the beginning of this chapter, we mentioned that software could store data in complex objects scattered in memory and connected by pointers, or more orderly, densely packed structures such as Fortran and C arrays. Generally, densely packed in-memory data structures were limited to simple types (e.g., INT64) or fixed-width data structures (e.g., fixed-width strings). More complex structures (e.g., JSON documents) could not be densely stored in memory and required serialization for storage and transfer between systems. The idea of Apache Arrow is to rethink serialization by utilizing a binary data format that is suitable for both in-memory processing and export. This 1 allows us to avoid the overhead of serialization and deserialization; we simply use the same format for in-memory processing, export over the network, and long-term storage. Arrow relies on columnar storage, where each column essentially gets its own chunks of memory. For nested data, we use a technique called shredding, which maps each location

in the schema of JSON documents into a separate column. This technique means that we can store a data file on disk, swap it directly into program address space by using virtual memory, and begin running a query against the data without deserialization overhead. In fact, we can swap chunks of the file into memory as we scan it, and then swap them back out to avoid running out of memory for large datasets. One obvious headache with this approach is that different programming languages serialize data in different ways. To address this issue, the Arrow Project has created software libraries for a variety of programming languages (including C, Go, Java, JavaScript, MATLAB, Python, R, and Rust) that allow these languages to interoperate with Arrow data in memory. In some cases, these libraries use an interface between the chosen language and low-level code in another language (e.g., C) to read and write from Arrow. This allows high interoperability between languages without extra serialization overhead. For example, a Scala program can use the Java library to write arrow data and then pass it as a message to a Python program. Arrow is seeing rapid uptake with a variety of popular frameworks such as Apache Spark. Arrow has also spawned a new **data warehouse** product; Dremio is a query engine and data warehouse built around Arrow serialization to support fast queries.

Khi chúng tôi giới thiệu tuần tự hóa như một nguyên liệu thô của lưu trữ ở đầu chương này, chúng tôi đã đề cập rằng phần mềm có thể lưu dữ liệu trong các đối tượng phức tạp rải rác trong bộ nhớ và kết nối bằng con trỏ, hoặc trong các cấu trúc ngăn nắp hơn, đóng gói dày đặc như mảng Fortran và C. Nhìn chung, các cấu trúc dữ liệu đóng gói dày đặc trong bộ nhớ bị giới hạn ở các kiểu đơn giản (ví dụ INT64) hoặc các cấu trúc dữ liệu có độ rộng cố định (ví dụ chuỗi độ rộng cố định). Các cấu trúc phức tạp hơn (ví dụ tài liệu JSON) không thể được lưu dày đặc trong bộ nhớ và đòi hỏi tuần tự hóa để lưu trữ và truyền giữa các hệ thống. Ý tưởng của Apache Arrow là suy nghĩ lại về tuần tự hóa bằng cách sử dụng một định dạng dữ liệu nhị phân phù hợp cho cả xử lý trong bộ nhớ lẫn xuất ra. Điều này 1 cho phép chúng ta tránh được chi phí phụ trội của tuần tự hóa và giải tuần tự hóa; chúng ta chỉ đơn giản dùng cùng một định dạng cho xử lý trong bộ nhớ, xuất qua mạng, và lưu trữ dài hạn. Arrow dựa vào lưu trữ theo cột, nơi mỗi cột về cơ bản nhận các khối bộ nhớ riêng của nó. Với dữ liệu lồng nhau, chúng ta dùng một kỹ thuật gọi là shredding (băm nhỏ), ánh xạ mỗi vị trí trong lược đồ của các tài liệu JSON vào một cột riêng. Kỹ thuật này nghĩa là chúng ta có thể lưu một tệp dữ liệu trên đĩa, hoán đổi nó trực tiếp vào không gian địa chỉ chương trình bằng cách dùng bộ nhớ ảo, và bắt đầu chạy một truy vấn trên dữ liệu mà không có chi phí phụ trội của giải tuần tự hóa. Thực tế, chúng ta có thể hoán đổi các khối của tệp vào bộ nhớ khi quét nó, rồi hoán đổi chúng ra để tránh hết bộ nhớ với các tệp dữ liệu lớn. Một nhược điểm rõ ràng với cách tiếp cận này là các ngôn ngữ lập trình khác nhau tuần tự hóa dữ liệu theo những cách khác nhau. Để giải quyết vấn đề này, Dự án Arrow đã tạo ra các thư viện phần mềm cho nhiều ngôn ngữ lập trình (bao gồm C, Go, Java, JavaScript, MATLAB, Python, R, và Rust) cho phép các ngôn ngữ này liên thông với dữ liệu Arrow trong bộ nhớ. Trong một số trường hợp, các thư viện này dùng một giao diện giữa ngôn ngữ được chọn và mã cấp thấp trong một ngôn ngữ khác (ví dụ C) để đọc và ghi từ Arrow. Điều này cho phép liên thông cao giữa các ngôn ngữ mà không có chi phí phụ trội tuần tự hóa bổ sung. Ví dụ, một chương trình Scala có thể dùng thư viện Java để ghi dữ liệu arrow rồi chuyển nó như một thông điệp tới một chương trình Python. Arrow đang được tiếp nhận nhanh chóng với nhiều framework phổ biến như Apache Spark. Arrow cũng đã khai sinh một sản phẩm kho dữ liệu mới; Dremio là một công cụ truy vấn và kho dữ liệu được xây dựng quanh tuần tự hóa Arrow để hỗ trợ truy vấn nhanh.

Hybrid Serialization — Tuần tự hóa Lai

We use the term hybrid serialization to refer to technologies that combine multiple serialization techniques or integrate serialization with additional abstraction layers, such as schema management.

We cite as examples Apache Hudi and Apache Iceberg.

Chúng tôi dùng thuật ngữ tuần tự hóa lai (hybrid serialization) để chỉ các công nghệ kết hợp nhiều kỹ thuật tuần tự hóa hoặc tích hợp tuần tự hóa với các tầng trừu tượng bổ sung, chẳng hạn quản lý lược đồ. Chúng tôi nêu ví dụ là Apache Hudi và Apache Iceberg.

Hudi — Hudi

Hudi stands for Hadoop Update Delete Incremental . This table management technology combines multiple serialization techniques to allow columnar database performance for analytics queries while also supporting atomic, transactional updates. A typical Hudi application is a table that is updated from a CDC stream from a transactional application database. The stream is captured into a row-oriented serialization format, while the bulk of the table is retained in a columnar format. A query runs over both columnar and row-oriented files to return results for the current state of the table. Periodically, a repacking process runs that combines the row and columnar files into updated columnar files to maximize query efficiency.

Hudi là viết tắt của Hadoop Update Delete Incremental. Công nghệ quản lý bảng này kết hợp nhiều kỹ thuật tuần tự hóa để cho phép hiệu năng cơ sở dữ liệu theo cột đối với các truy vấn phân tích trong khi vẫn hỗ trợ các cập nhật giao dịch, nguyên tử. Một ứng dụng Hudi điển hình là một bảng được cập nhật từ một luồng CDC từ cơ sở dữ liệu ứng dụng giao dịch. Luồng này được bắt vào một định dạng tuần tự hóa theo hàng, trong khi phần lớn bảng được giữ ở định dạng theo cột. Một truy vấn chạy trên cả các tệp theo cột lẫn theo hàng để trả về kết quả cho trạng thái hiện tại của bảng. Định kỳ, một tiến trình đóng gói lại chạy để kết hợp các tệp theo hàng và theo cột thành các tệp theo cột đã cập nhật nhằm tối đa hóa hiệu quả truy vấn.

Iceberg — Iceberg

Like Hudi, Iceberg is a table management technology. Iceberg can track all files that make up a table. It can also track files in each table snapshot over time, allowing table time travel in a data lake environment. Iceberg supports **schema evolution** and can readily manage tables at a petabyte scale.

Giống như Hudi, Iceberg là một công nghệ quản lý bảng. Iceberg có thể theo dõi tất cả các tệp tạo nên một bảng. Nó cũng có thể theo dõi các tệp trong mỗi ảnh chụp (snapshot) bảng theo thời gian, cho phép du hành thời gian của bảng trong môi trường hồ dữ liệu. Iceberg hỗ trợ tiến hóa lược đồ (schema evolution) và có thể dễ dàng quản lý các bảng ở quy mô petabyte.

Database Storage Engines — Các Engine Lưu trữ Cơ sở dữ liệu

To round out the discussion of serialization, we briefly discuss database storage engines. All databases have an underlying storage engine; many don't expose their storage engines as a separate abstraction (for example, BigQuery, Snowflake). Some (notably, MySQL) support fully pluggable storage engines. Others (e.g., SQL Server) offer major storage engine configuration options (columnar versus row-based storage) that dramatically affect database behavior. Typically, the storage engine is a separate software layer from the query engine. The storage engine manages all aspects of how data is stored on a disk, including serialization, the physical arrangement of data, and indexes.

Storage engines have seen significant innovation in the 2000s and 2010s. While storage engines in the past were optimized for direct access to spinning disks, modern storage engines are much better optimized to support the performance characteristics of SSDs. Storage engines also offer improved support for modern types and data structures, such as variable-length strings, arrays, and nested data. Another major change in storage engines is a shift toward columnar storage for analytics and data warehouse applications. SQL Server, PostgreSQL, and MySQL offer robust columnar storage support.

Để khép lại phần thảo luận về tuần tự hóa, chúng ta bàn ngắn gọn về các engine lưu trữ cơ sở dữ liệu. Mọi cơ sở dữ liệu đều có một engine lưu trữ nền tảng; nhiều cơ sở dữ liệu không phơi bày engine lưu trữ của chúng như một trừu tượng riêng (ví dụ BigQuery, Snowflake). Một số (đáng chú ý là MySQL) hỗ trợ các engine lưu trữ hoàn toàn cắm-được (pluggable). Số khác (ví dụ SQL Server) cung cấp các tùy chọn cấu hình engine lưu trữ lớn (lưu trữ theo cột so với theo hàng) ảnh hưởng mạnh đến hành vi của cơ sở dữ liệu. Thông thường, engine lưu trữ là một tầng phần mềm riêng biệt với engine truy vấn. Engine lưu trữ quản lý mọi khía cạnh của cách dữ liệu được lưu trên đĩa, bao gồm tuần tự hóa, sự sắp xếp vật lý của dữ liệu, và các chỉ mục. Các engine lưu trữ đã chứng kiến sự đổi mới đáng kể trong những năm 2000 và 2010. Trong khi các engine lưu trữ trong quá khứ được tối ưu cho truy cập trực tiếp tới các đĩa quay, các engine lưu trữ hiện đại được tối ưu tốt hơn nhiều để hỗ trợ các đặc tính hiệu năng của SSD. Các engine lưu trữ cũng cung cấp hỗ trợ tốt hơn cho các kiểu và cấu trúc dữ liệu hiện đại, chẳng hạn chuỗi độ dài biến thiên, mảng, và dữ liệu lồng nhau. Một thay đổi lớn khác trong các engine lưu trữ là sự dịch chuyển sang lưu trữ theo cột cho các ứng dụng phân tích và kho dữ liệu. SQL Server, PostgreSQL, và MySQL cung cấp hỗ trợ lưu trữ theo cột vững chắc.

Compression: gzip, bzip2, Snappy, etc. — Nén: gzip, bzip2, Snappy, v.v.

The math behind compression algorithms is complex, but the basic idea is easy to understand: compression algorithms look for redundancy and repetition in data, then re-encode data to reduce redundancy. When we want to read the raw data, we decompress it by reversing the algorithm and putting the redundancy back in.

Toán học đằng sau các thuật toán nén thì phức tạp, nhưng ý tưởng cơ bản thì dễ hiểu: các thuật toán nén tìm độ dư thừa và sự lặp lại trong dữ liệu, rồi mã hóa lại dữ liệu để giảm độ dư thừa. Khi chúng ta muốn đọc dữ liệu thô, chúng ta giải nén nó bằng cách đảo ngược thuật toán và đặt phần dư thừa trở lại.

For example, you've noticed that certain words appear repeatedly in reading this book. Running some quick analytics on the text, you could identify the words that occur most frequently and create shortened tokens for these words. To compress, you would replace common words with their tokens; to decompress, you would replace the tokens with their respective words. Perhaps we could use this naive technique to realize a compression ratio of 2:1 or more. Compression algorithms utilize more sophisticated mathematical techniques to identify and remove redundancy; they can often realize compression ratios of 10: 1 on text data. Note that we're talking about lossless compression algorithms. Decompressing data encoded with a lossless algorithm recovers a bit-for-bit exact copy of the original data. Lossy compression algorithms for audio, images, and video aim for sensory fidelity; decompression recovers something that sounds like or looks like the original but is not an exact copy. Data engineers might deal with lossy compression algorithms in media processing pipelines but not in serialization for analytics, where exact data fidelity is required. Traditional

compression engines such as gzip and bzip2 compress text data extremely well; they are frequently applied to JSON, JSONL, XML, CSV, and other text-based data formats. Engineers have created a new **generation** of compression algorithms that prioritize speed over compression ratio in recent years. Major examples are Snappy, Zstandard, LZFSE, and LZ4. These algorithms are frequently used to compress data in data lakes or columnar databases to optimize fast query performance.

Ví dụ, bạn đã nhận thấy rằng một số từ nhất định xuất hiện lặp đi lặp lại khi đọc cuốn sách này. Chạy vài phân tích nhanh trên văn bản, bạn có thể xác định những từ xuất hiện thường xuyên nhất và tạo các token rút gọn cho những từ này. Để nén, bạn sẽ thay các từ phổ biến bằng token của chúng; để giải nén, bạn sẽ thay các token bằng các từ tương ứng. Có lẽ chúng ta có thể dùng kỹ thuật ngây thơ này để đạt tỷ lệ nén 2:1 hoặc hơn. Các thuật toán nén sử dụng các kỹ thuật toán học tinh vi hơn để nhận diện và loại bỏ độ dư thừa; chúng thường có thể đạt tỷ lệ nén 10:1 trên dữ liệu văn bản. Lưu ý rằng chúng ta đang nói về các thuật toán nén không mất mát (lossless). Giải nén dữ liệu được mã hóa bằng một thuật toán không mất mát sẽ khôi phục một bản sao chính xác từng bit so với dữ liệu gốc. Các thuật toán nén mất mát (lossy) cho âm thanh, hình ảnh, và video nhắm tới độ trung thực cảm quan; giải nén khôi phục thứ gì đó nghe giống hoặc trông giống bản gốc nhưng không phải là một bản sao chính xác. Kỹ sư dữ liệu có thể phải xử lý các thuật toán nén mất mát trong các đường ống xử lý phương tiện (media) nhưng không phải trong tuần tự hóa cho phân tích, nơi đòi hỏi độ trung thực dữ liệu chính xác. Các engine nén truyền thống như gzip và bzip2 nén dữ liệu văn bản cực kỳ tốt; chúng thường được áp dụng cho JSON, JSONL, XML, CSV, và các định dạng dữ liệu dựa trên văn bản khác. Trong những năm gần đây, các kỹ sư đã tạo ra một thế hệ thuật toán nén mới ưu tiên tốc độ hơn tỷ lệ nén. Các ví dụ lớn là Snappy, Zstandard, LZFSE, và LZ4. Các thuật toán này thường được dùng để nén dữ liệu trong các hồ dữ liệu hoặc cơ sở dữ liệu theo cột nhằm tối ưu hiệu năng truy vấn nhanh.

1 Dejan Simic, “Apache Arrow: Read DataFrame with Zero Memory,” Towards Data Science, June 25, 2020, <https://oreil.ly/TDAyY>.

1 Dejan Simic, “Apache Arrow: Read DataFrame with Zero Memory,” Towards Data Science, ngày 25 tháng 6 năm 2020, <https://oreil.ly/TDAyY>.

Appendix B. Cloud Networking — Phụ lục B. Mạng trên đám mây (Cloud Networking)

This appendix discusses some factors data engineers should consider about networking in the cloud. Data engineers frequently encounter networking in their careers, and often ignore it despite its importance.

Phụ lục này bàn về một số yếu tố liên quan đến mạng trên đám mây mà kỹ sư dữ liệu nên cân nhắc. Kỹ sư dữ liệu thường xuyên gặp phải vấn đề mạng trong sự nghiệp của mình, và thường bỏ qua nó dù tầm quan trọng của nó là rất lớn.

Cloud Network Topology — Cấu trúc liên kết mạng trên đám mây (Cloud Network Topology)

A cloud network topology describes how various components in the cloud are arranged and connected, such as cloud services, networks, locations (zones, regions), and more. Data engineers should always know how cloud network topology will affect connectivity across the data systems they build. Microsoft Azure, Google Cloud Platform (GCP), and Amazon Web Services (AWS) all use remarkably similar resource hierarchies of availability zones and regions. At the time of this writing, GCP has added one additional layer, discussed in “GCP-Specific Networking and Multiregional Redundancy”.

Cấu trúc liên kết mạng trên đám mây (cloud network topology) mô tả cách các thành phần khác nhau trong đám mây được sắp xếp và kết nối, chẳng hạn như các dịch vụ đám mây, mạng, vị trí (zone, region), và nhiều thứ khác. Kỹ sư dữ liệu luôn cần biết cấu trúc liên kết mạng trên đám mây sẽ ảnh hưởng thế nào đến khả năng kết nối xuyên suốt các hệ thống dữ liệu mà họ xây dựng. Microsoft Azure, Google Cloud Platform (GCP) và Amazon Web Services (AWS) đều dùng các phân cấp tài nguyên gồm availability zone và region giống nhau đến mức đáng kinh ngạc. Tại thời điểm viết cuốn sách này, GCP đã bổ sung thêm một lớp nữa, được bàn trong “Mạng đặc thù của GCP và dự phòng đa region”.

Data Egress Charges — Phí xuất dữ liệu (Data Egress Charges)

Chapter 4 discusses cloud economics and how actual provider costs don’t necessarily drive cloud pricing. Regarding networking, clouds allow inbound traffic for free but charge for outbound traffic

to the internet. Outbound traffic is not inherently cheaper, but clouds use this method to create a moat around their services and increase the stickiness of stored data, a practice that has been widely criticized. Note that data egress charges can also apply to data passing between availability zones and regions within a cloud.

Chương 4 bàn về kinh tế học đám mây và cách mà chi phí thực tế của nhà cung cấp không nhất thiết chi phối giá đám mây. Về mặt mạng, các đám mây cho phép lưu lượng vào (inbound) miễn phí nhưng tính phí cho lưu lượng ra (outbound) đi tới internet. Lưu lượng outbound vốn không hề rẻ hơn, nhưng các đám mây dùng cách này để tạo một con hào bảo vệ quanh dịch vụ của họ và làm tăng tính “núi chân” của dữ liệu đã lưu trữ, một thực hành đã bị chỉ trích rộng rãi. Lưu ý rằng phí xuất dữ liệu (data egress) 1 cũng có thể áp dụng cho dữ liệu di chuyển giữa các availability zone và region trong cùng một đám mây.

Availability Zones — Availability Zone

The availability zone is the smallest unit of network topology that public clouds make visible to customers (Figure B-1). While a zone can potentially consist of multiple data centers, cloud customers cannot control resource placement at this level.

Availability zone là đơn vị nhỏ nhất của cấu trúc liên kết mạng mà các đám mây công cộng (public cloud) hiển thị cho khách hàng (Hình B-1). Mặc dù một zone về mặt lý thuyết có thể bao gồm nhiều trung tâm dữ liệu, khách hàng đám mây không thể kiểm soát việc đặt tài nguyên ở cấp độ này.

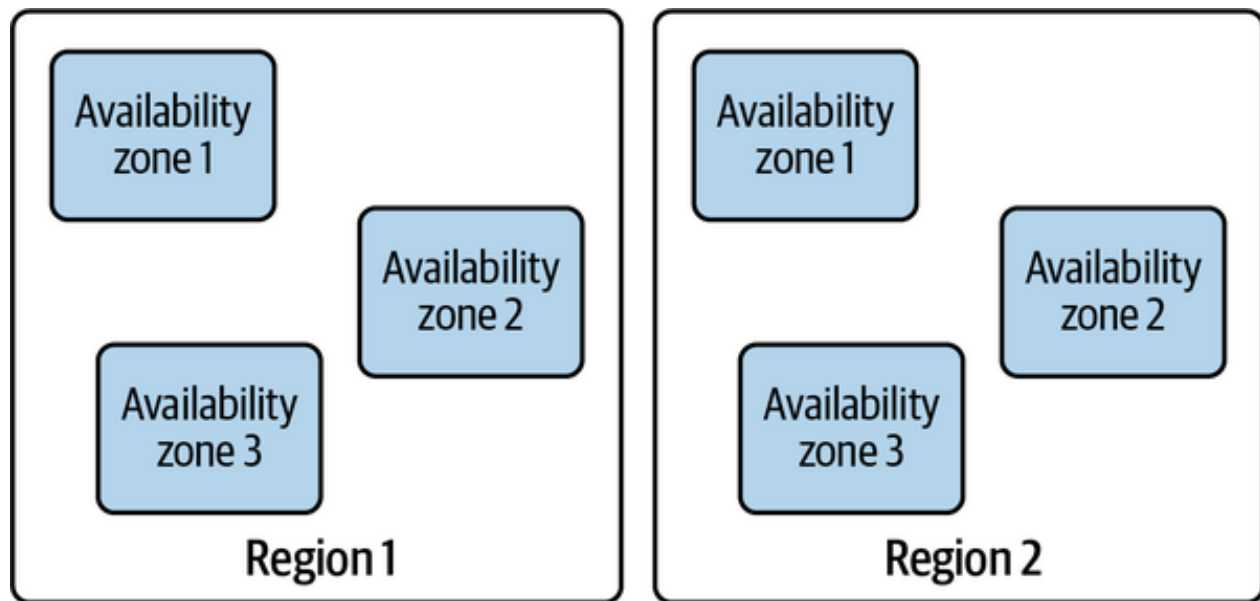


Figure B-1. Availability zones in two separate regions Generally, clouds support their highest network bandwidth and lowest **latency** between systems and services within a zone. High **throughput** data workloads should run on clusters located in a single zone for performance and cost reasons. For example, an ephemeral Amazon EMR cluster should generally sit in a single availability zone. In addition, network traffic sent to VMs within a zone is free, but with a significant caveat: traffic must be sent to private IP addresses. The major clouds utilize virtual networks known as virtual private clouds (VPCs). Virtual machines have private IP addresses within the VPC. They may also be assigned public IP addresses to communicate with the outside world and receive traffic from the internet, but communications using external IP addresses can incur data egress charges.

Hình B-1. Các availability zone trong hai region riêng biệt. Nhìn chung, các đám mây hỗ trợ băng thông mạng cao nhất và độ trễ (latency) thấp nhất giữa các hệ thống và dịch vụ nằm trong cùng một zone. Các tải công việc dữ liệu thông lượng (throughput) cao nên chạy trên các cụm (cluster) đặt trong một zone duy nhất vì lý do hiệu năng và chi phí. Ví dụ, một cụm Amazon EMR phù du (ephemeral) nói chung nên nằm trong một availability zone duy nhất. Ngoài ra, lưu lượng mạng gửi tới các VM trong cùng một zone là miễn phí, nhưng kèm theo một lưu ý quan trọng: lưu lượng phải được gửi tới các địa chỉ IP riêng tư (private IP). Các đám mây lớn sử dụng mạng ảo gọi là virtual private cloud (VPC). Các máy ảo có địa chỉ IP riêng tư bên trong VPC. Chúng cũng có thể được gán địa chỉ IP công cộng (public IP) để giao tiếp với thế giới bên ngoài và nhận lưu lượng từ internet, nhưng việc giao tiếp dùng địa chỉ IP bên ngoài có thể phát sinh phí xuất dữ liệu.

Regions — Region

A region is a collection of two or more availability zones. Data centers require many resources to run (electrical power, water, etc.). The resources of separate availability zones are independent so that a local power outage doesn't take down multiple availability zones. Engineers can build highly resilient, separate infrastructure even within a single region by running servers in multiple zones or creating automated cross-zone failover processes. Offering multiple regions allows engineers to put resources close to any of their users. Close means that users can realize good network performance in connecting to services, minimizing physical distance along the network path, and a minimal number of hops through routers. Both physical distance and network hops can increase latency and decrease performance. Major cloud providers continue to add new regions. In general, regions support fast, low-latency networking between zones; networking performance between zones will be worse than within a single zone and incur nominal data egress charges between VMs. Network data movement between regions is even slower and may incur higher egress fees. In general, **object storage** is a regional resource. Some data may pass between zones to reach a virtual machine, but this is mainly invisible to cloud customers, and there are no direct networking charges for this. (Of course, customers are still responsible for object access costs.) Despite regions' geo-redundant design, many major cloud service failures have affected entire regions, an example of correlated failure. Engineers often deploy code and configuration to entire regions; the regional failures we've observed have generally resulted from code or configuration problems occurring at the regional level.

Một region là một tập hợp gồm hai hay nhiều availability zone. Các trung tâm dữ liệu cần nhiều tài nguyên để vận hành (điện, nước, v.v.). Tài nguyên của các availability zone riêng biệt là độc lập với nhau, nhờ vậy một sự cố mất điện cục bộ sẽ không làm sập nhiều availability zone cùng lúc. Kỹ sư có thể xây dựng hạ tầng tách biệt, có khả năng phục hồi cao ngay cả trong một region duy nhất bằng cách chạy máy chủ ở nhiều zone hoặc tạo các quy trình chuyển đổi dự phòng (failover) tự động giữa các zone. Việc cung cấp nhiều region cho phép kỹ sư đặt tài nguyên gần với bất kỳ người dùng nào của họ. Gần ở đây nghĩa là người dùng có thể đạt hiệu năng mạng tốt khi kết nối tới dịch vụ, giảm thiểu khoảng cách vật lý dọc theo đường đi của mạng, và số bước nhảy (hop) qua các router là tối thiểu. Cả khoảng cách vật lý lẫn số bước nhảy mạng đều có thể làm tăng độ trễ và giảm hiệu năng. Các nhà cung cấp đám mây lớn vẫn liên tục bổ sung region mới. Nhìn chung, các region hỗ trợ mạng nhanh, độ trễ thấp giữa các zone; hiệu năng mạng giữa các zone sẽ kém hơn so với trong cùng một zone và phát sinh một khoản phí xuất dữ liệu nhỏ giữa các VM. Việc di chuyển dữ liệu mạng giữa các region thậm chí còn chậm hơn và có thể phát sinh phí xuất dữ liệu cao hơn. Nhìn chung, lưu trữ đối tượng (object storage) là một tài nguyên cấp region. Một số dữ liệu có thể đi qua giữa các zone để tới một máy ảo, nhưng điều này phần lớn là vô hình đối với khách hàng đám mây, và không có phí mạng trực tiếp

cho việc này. (Tất nhiên, khách hàng vẫn phải chịu chi phí truy cập đối tượng.) Bất chấp thiết kế dự phòng địa lý (geo-redundant) của các region, nhiều sự cố dịch vụ đám mây lớn đã ảnh hưởng tới toàn bộ region, một ví dụ về lỗi tương quan (correlated failure). Kỹ sư thường triển khai mã và cấu hình tới toàn bộ region; các sự cố cấp region mà chúng tôi quan sát được nhìn chung bắt nguồn từ các vấn đề về mã hoặc cấu hình xảy ra ở cấp độ region.

GCP-Specific Networking and Multiregional Redundancy — Mạng đặc thù của GCP và dự phòng đa region

GCP offers a handful of unique abstractions that engineers should be aware of if they work in this cloud. The first is the multiregion, a layer in the resource hierarchy; a multiregion contains multiple regions. Current multiregions are US (data centers in the United States), EU (data centers in European Union member states), and ASIA. Several GCP resources support multiregions, including Cloud **Storage** and BigQuery. Data is stored in multiple zones within the multiregion in a geo-redundant manner so that it should remain available in the **event** of a regional failure. Multiregional storage is also designed to deliver data efficiently to users within the multiregion without setting up complex replication processes between regions. In addition, there are no data egress fees for VMs in a multiregion to access Cloud Storage data in the same multiregion. Cloud customers can set up multiregional infrastructure on AWS or Azure. In the case of databases or object storage, this involves duplicating data between regions to increase redundancy and put data closer to users. Google also essentially owns significantly more global scale networking resources than other cloud providers, something which it offers to its customers as premium tier networking. Premium tier networking allows traffic between zones and regions to pass entirely over Google-owned networks without traversing the public internet.

GCP cung cấp một số trừu tượng độc đáo mà kỹ sư nên biết nếu họ làm việc trên đám mây này. Đầu tiên là multiregion, một lớp trong phân cấp tài nguyên; một multiregion chứa nhiều region. Các multiregion hiện tại là US (các trung tâm dữ liệu ở Hoa Kỳ), EU (các trung tâm dữ liệu ở các quốc gia thành viên Liên minh châu Âu), và ASIA. Một số tài nguyên của GCP hỗ trợ multiregion, bao gồm Cloud Storage và BigQuery. Dữ liệu được lưu trữ ở nhiều zone trong multiregion theo cách dự phòng địa lý sao cho nó vẫn khả dụng trong trường hợp xảy ra lỗi cấp region. Lưu trữ đa region cũng được thiết kế để phân phối dữ liệu hiệu quả tới người dùng trong multiregion mà không cần thiết lập các quy trình sao chép (replication) phức tạp giữa các region. Ngoài ra, không có phí xuất dữ liệu cho các VM trong một multiregion khi truy cập dữ liệu Cloud Storage trong cùng multiregion đó. Khách hàng đám mây có thể thiết lập hạ tầng đa region trên AWS hoặc Azure. Trong trường hợp cơ sở dữ liệu hoặc lưu trữ đối tượng, điều này liên quan đến việc nhân bản dữ liệu giữa các region để tăng tính dự phòng và đặt dữ liệu gần người dùng hơn. Google về cơ bản cũng sở hữu nhiều tài nguyên mạng quy mô toàn cầu hơn đáng kể so với các nhà cung cấp đám mây khác, một thứ mà họ cung cấp cho khách hàng dưới dạng mạng bậc cao cấp (premium tier networking). Mạng bậc cao cấp cho phép lưu lượng giữa các zone và region đi hoàn toàn trên mạng do Google sở hữu mà không phải đi qua internet công cộng.

Direct Network Connections to the Clouds — Kết nối mạng trực tiếp tới các đám mây

Each major public cloud offers enhanced connectivity options, allowing customers to integrate their networks with a cloud region or VPC directly. For example, Amazon offers AWS Direct Connect. In addition to providing higher bandwidth and lower latency, these connection options often offer dramatic discounts on data egress charges. In a typical scenario in the US, AWS egress charges drop from 9 cents per gigabyte over the public internet to 2 cents per gigabyte over direct connect.

Mỗi đám mây công cộng lớn đều cung cấp các tùy chọn kết nối nâng cao, cho phép khách hàng tích hợp mạng của họ trực tiếp với một region đám mây hoặc một VPC. Ví dụ, Amazon cung cấp AWS Direct Connect. Bên cạnh việc cung cấp băng thông cao hơn và độ trễ thấp hơn, các tùy chọn kết nối này thường còn đưa ra mức chiết khấu đáng kể cho phí xuất dữ liệu. Trong một kịch bản điển hình ở Mỹ, phí xuất dữ liệu của AWS giảm từ 9 cent mỗi gigabyte qua internet công cộng xuống còn 2 cent mỗi gigabyte qua direct connect.

CDNs — CDN

Content delivery networks (CDNs) can offer dramatic performance enhancements and discounts for delivering data assets to the public or customers. Cloud providers offer CDN options and many other providers, such as Cloudflare. CDNs work best when delivering the same data repeatedly, but make sure that you read the fine print. Remember that CDNs don't work everywhere, and certain countries may block internet traffic and CDN delivery.

Mạng phân phối nội dung (content delivery network — CDN) có thể mang lại sự cải thiện hiệu năng đáng kể và mức chiết khấu khi phân phối các tài sản dữ liệu tới công chúng hoặc khách hàng. Các nhà cung cấp đám mây có cung cấp tùy chọn CDN, cùng với nhiều nhà cung cấp khác, chẳng hạn như Cloudflare. CDN hoạt động tốt nhất khi phân phối lặp đi lặp lại cùng một dữ liệu, nhưng hãy chắc chắn rằng bạn đọc kỹ các điều khoản chi tiết. Hãy nhớ rằng CDN không hoạt động ở mọi nơi, và một số quốc gia có thể chặn lưu lượng internet và việc phân phối qua CDN.

The Future of Data Egress Fees — Tương lai của phí xuất dữ liệu

Data egress fees are a significant impediment to interoperability, data sharing, and data movement to the cloud. Right now, data egress fees are a moat designed to prevent public cloud customers from leaving or deploying across multiple clouds. But interesting signals indicate that change may be on the horizon. In particular, Zoom's announcement in 2020 near the beginning of the COVID-19 pandemic that it chose Oracle as its cloud infrastructure provider caught the attention of many cloud watchers. How did Oracle win this significant cloud contract for critical remote work infrastructure against the cloud heavyweights? AWS expert Corey Quinn offers a reasonably straightforward answer. By his back-of-the-envelope calculation, Zoom's 3 AWS monthly data egress fees would run over \$11 million dollars at list price; Oracle's would cost less than \$2 million. We suspect that one of GCP, AWS, or Azure will announce significant cuts in egress fees in the next few years, leading to a sea change in the cloud business model.

Phí xuất dữ liệu là một trở ngại đáng kể đối với khả năng tương tác (interoperability), việc chia sẻ dữ liệu, và việc di chuyển dữ liệu lên đám mây. Hiện nay, phí xuất dữ liệu là một con hào được thiết kế để ngăn khách hàng của đám mây công cộng rời đi hoặc triển khai

trên nhiều đám mây. Nhưng có những tín hiệu thú vị cho thấy sự thay đổi có thể đang đến gần. Cụ thể, thông báo của Zoom vào năm 2020, gần thời điểm bắt đầu đại dịch COVID-19, rằng họ chọn Oracle làm nhà cung cấp hạ tầng đám mây đã thu hút sự chú ý của nhiều người theo dõi giới đám mây. Làm thế nào mà Oracle giành được 2 hợp đồng đám mây quan trọng này cho hạ tầng làm việc từ xa trọng yếu, vượt qua những ông lớn trong làng đám mây? Chuyên gia về AWS Corey Quinn đưa ra một câu trả lời khá thẳng thắn. Theo phép tính nhẩm của ông, phí xuất dữ liệu hằng tháng của Zoom 3 trên AWS sẽ vượt quá 11 triệu đô la theo giá niêm yết; còn của Oracle sẽ tốn ít hơn 2 triệu đô la. Chúng tôi ngờ rằng một trong số GCP, AWS hoặc Azure sẽ công bố cắt giảm đáng kể phí xuất dữ liệu trong vài năm tới, dẫn đến một thay đổi sâu sắc trong mô hình kinh doanh đám mây.

1 Matthew Prince and Nitin Rao, “AWS’s Egregious Egress,” The Cloudflare Blog , July 23, 2021, <https://oreil.ly/NZqKa> . 2 Mark Haranas and Steven Burke, “Oracle Bests Cloud Rivals to Win Blockbuster Cloud Deal,” CRN, April 28, 2020, <https://oreil.ly/LkqOi> . 3 Corey Quinn, “Why Zoom Chose Oracle Cloud Over AWS and Maybe You Should Too,” Last Week in AWS, April 28, 2020, <https://oreil.ly/Lx5uu> .

1 Matthew Prince and Nitin Rao, “AWS’s Egregious Egress,” The Cloudflare Blog , July 23, 2021, <https://oreil.ly/NZqKa> . 2 Mark Haranas and Steven Burke, “Oracle Bests Cloud Rivals to Win Blockbuster Cloud Deal,” CRN, April 28, 2020, <https://oreil.ly/LkqOi> . 3 Corey Quinn, “Why Zoom Chose Oracle Cloud Over AWS and Maybe You Should Too,” Last Week in AWS, April 28, 2020, <https://oreil.ly/Lx5uu> .